MOST IMPORTANT FIRST | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB

Google study path (clean full code)

288 questions in 43 groups of 5 to 9. Each group shares ONE code template, shown on the page before its questions. Biggest groups first (9, then 8, 7, 6, 5).

38	197	53
EASY	MEDIUM	HARD

Sections (tap one to jump to it)

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance — 9 questions | page 12
2. Sweep over events in time order — 9 questions | page 42
3. Tree recursion, Tier 1: one function that visits a node, asks its children, a... — 9 questions | page 74
4. A heap of what is available: pop the best, put back the changed one — 9 questions | page 106
5. Recurrence on a position or a range (memo or table) — 9 questions | page 137
6. Best-first search with a heap: Dijkstra and its minimax twin — 9 questions | page 168
7. Backtracking: choose, recurse, undo — 9 questions | page 198
8. Runs and groups: extend while it fits, restart or close when it does not — 9 questions | page 229
9. Union-find: merge the pairs, then ask who is together — 8 questions | page 254
10. A dict of per-key state that you update, expire, or rank — 8 questions | page 282
11. Binary search on a yes/no condition — 8 questions | page 309
12. Greedy: scan once, carry the best choice so far, act only when forced — 8 questions | page 336
13. Sets and counters: remember the first collection, check the second aga... — 8 questions | page 361
14. bisect on a sorted list: find a position, count a range, keep it sorted — 8 questions | page 385
15. Prefix sums, Tier 1: precompute once, answer ranges and balance poin... — 8 questions | page 410
16. Sorted intervals: merge while they touch, then read the gaps or search — 8 questions | page 435
17. Trie: nested dicts, one character per level — 8 questions | page 461
18. A dict for lookup plus an ordered container for who is next (lazy deletion) — 8 questions | page 490

Sections, continued (tap one to jump to it)

19. Rolling state: the next state comes only from the previous one	7 questions	page 520
20. Two pointers: move i and j toward each other (or along two lists)	7 questions	page 543
21. Walk two strings at once with two indices	7 questions	page 566
22. Simulate a machine: a class with state and one branch per kind of event	7 questions	page 590
23. Top-K by frequency: Counter plus nsmallest	6 questions	page 615
24. Digits and characters: peel one at a time and carry	6 questions	page 640
25. Look at the input once and keep only a small summary (and count your...	6 questions	page 661
26. Grid DP: each cell from its neighbours above and to the left	5 questions	page 682
27. Nested structures: a stack of contexts, or recursion with a cache	5 questions	page 698
28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours...	5 questions	page 717
29. Topological sort (Kahn) and cycle detection	5 questions	page 736
30. Counting pairs by looking up each partner in a Counter or a bucket	5 questions	page 755
31. Tree DP: build a parent order once, then add up from the leaves	5 questions	page 769
32. Counting or reaching an amount: the knapsack loop	5 questions	page 787
33. Prefix sums, Tier 2: running total plus a dict of earlier totals	5 questions	page 806
34. Group by a canonical key	5 questions	page 824
35. K-way merge: a heap of the next item from each source	5 questions	page 842
36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up	5 questions	page 859
37. Random pick without repeats: swap the chosen item with the last one a...	5 questions	page 878
38. Build and serialize: write a structure out, read it back with the same walk	5 questions	page 894
39. Games: win or lose with a cache, and guessing with feedback	5 questions	page 916
40. Sliding window: grow on the right, shrink on the left while it is too bad	5 questions	page 935
41. Parent pointers: store who is above each node and climb	5 questions	page 953
42. Build or transform a grid with index arithmetic	5 questions	page 971
43. Split into pieces, fix each piece, join (and pack words into lines)	5 questions	page 988

- The master contents lists every main topic and question; each row is a link.
- Inside a topic, the bar at the top of each page moves Prev / Contents / Next; the link at the bottom returns here.
- Questions run Easy, then Medium, then Hard inside each main topic, newest date first.

Go to Contents >

Contents

288 questions | tap a topic, a main topic or a question to jump to it

START HERE: topics in the order of importance, the most important question first inside each

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9) Go >

E	1. Flood Fill	ColorStack 2026-01-28
M	2. Number of Islands	1Point3Acres 2026-09-30
M	3. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres 2026-09-08
M	4. Solve Grid and Counting Problems	PracHub 2025-01-25
M	5. Find shortest relationship path using BFS	PracHub 2026-02-12
M	6. Farthest City for a New Data Center	ColorStack 2025-10-10
M	7. Compute distance to nearest taxi in grid	PracHub 2026-01-10
H	8. Infection Spread on a Grid (Cellular Automaton)	PracHub 2026-06-14
H	9. Shortest Distance From All Buildings	ColorStack 2025-09-22

2. Sweep over events in time order (9) Go >

M	10. Meeting Rooms / Interval Scheduling Variants	1Point3Acres 2026-09-29
M	11. Train / Flight Schedule Reachability	1Point3Acres 2026-10-08
M	12. Solve Rooms and Top-K Streams	PracHub 2026-06-05
M	13. Compute servers needed for daily recurring jobs	PracHub 2025-11-05
M	14. Gate Entry / Exit Time Resolution with Tie-Break Rules	1Point3Acres 2026-05-04
M	15. Meeting Rooms III Variant	HackTheRounds no date
H	16. Count Overlapping Rectangle Updates on a Grid	PracHub 2026-07-10
H	17. Compute city skyline outline	PracHub 2025-12-08
H	18. Compute Turnstile Crossing Times	PracHub 2026-05-04

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upw... Go >

E	19. Given two trees, determine if the pre-order traversals of two trees are same or not.	same or not. Exponent no date
E	20. Calculate the height of a binary tree.	Exponent no date
E	21. Find right-side view of binary tree	PracHub 2025-11-01
M	22. Expression Tree Single-Leaf Mutation	1Point3Acres 2026-09-27
M	23. Longest Consecutive Path in Binary Tree	1Point3Acres 2026-07-02
M	24. Given the root of a binary tree, return the length of the diameter of the tree.	the tree. Exponent no date
M	25. Implement zigzag level-order traversal	PracHub 2025-08-01
H	26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	Tree PracHub 2026-09-16
H	27. Delete an Entire Binary Tree	ColorStack 2025-09-29

4. A heap of what is available: pop the best, put back the changed one (9) Go >

M	28. Streaming Median (LC 295) with Follow-ups	1Point3Acres 2026-08-03
M	29. Compute minimum number of rooms needed	PracHub 2026-02-12
M	30. Implement a Reusable Locker Allocation System	PracHub 2026-08-14
M	31. Simulate room assignments for scheduled meetings	PracHub 2026-01-06
M	32. Trip Expense Settlement	HackTheRounds no date
M	33. Playlist Shuffle No Adjacent Artists	HackTheRounds no date
M	34. Minimum Makespan on Identical CPUs	ColorStack 2025-09-23
M	35. Design server allocation for multi-type nodes	PracHub 2025-09-06
H	36. Huffman Tree Construction	1Point3Acres 2026-06-01

Contents

288 questions | tap a topic, a main topic or a question to jump to it

5. Recurrence on a position or a range (memo or table) (9) Go >

M 37. Target Expressions from Ordered Digits — 1Point3Acres 2026-08-13
M 38. Count Word Decompositions into Chemical Element Symbols — 1Point3Acres 2026-07-14
M 39. Find the longest palindromic subsequence using dynamic programming. — Exponent no date
M 40. Jump Game with Profit — 1Point3Acres no date
M 41. Fence Painting Minimum Operations (Gemini Team) — 1Point3Acres 2026-04-27
M 42. Maximize 1D deliveries within distance and minimize total distance — PracHub 2025-12-27
H 43. Edit distance — Exponent no date
H 44. Count Perfect Wake Numbers up to N — 1Point3Acres 2026-05-07
H 45. Word Break: All Sentences — ColorStack 2026-01-28

6. Best-first search with a heap: Dijkstra and its minimax twin (9) Go >

M 46. Path Minimizing the Maximum Cell (Swim in Rising Water) — 1Point3Acres 2026-09-29
M 47. Nearest Favorite City by Travel Time — ColorStack 2025-10-17
M 48. Given a train schedule with departure city, arrival city, start time, end time,... — time, and duration, and given... Exponent no date
M 49. Find the shortest path in a graph. — Exponent no date
M 50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat... — the rat to the bread that maxi... Exponent no date
H 51. Minimize maximum height along a grid path — PracHub 2026-01-04
H 52. Find minimum threshold enabling grid path — PracHub 2026-02-11
H 53. Minimize travel time with optional meeting point — PracHub 2026-01-06
H 54. Compute minimax grid path and network delay — PracHub 2026-01-22

7. Backtracking: choose, recurse, undo (9) Go >

M 55. Mahjong Winning Hand Detection (14 Tiles) — 1Point3Acres 2026-10-09
M 56. Count Prefix-Matching Simple Paths in a Labeled Graph — PracHub 2026-08-09
M 57. Generate Parentheses — Exponent no date
M 58. American Football Score Combinations — ColorStack 2025-08-25
M 59. Permutations — Exponent no date
M 60. Determine if a 14-tile hand is winning — PracHub 2026-03-09
M 61. Count Passcode Patterns on a Keypad — ColorStack 2025-11-07
M 62. Match people to questions using tags and priorities — PracHub 2025-12-27
H 63. Matrix Flower Placement with House Adjacency — 1Point3Acres 2026-05-31

8. Runs and groups: extend while it fits, restart or close when it does not (9) Go >

E 64. Group Timestamps Into Close Intervals — ColorStack 2025-10-03
E 65. First Out-of-Order Character — ColorStack 2026-01-30
M 66. Longest Non-Decreasing Subarray with One Replacement — 1Point3Acres 2026-10-04
M 67. Piano Hand Movement Grouping — 1Point3Acres 2026-07-14
M 68. Find longest increasing contiguous subarray — PracHub 2026-01-06
M 69. Sum of Good Subarrays (Adjacent Diff ±1) — 1Point3Acres 2026-02-08
M 70. Sustained-Error Time Window — ColorStack 2025-09-16
M 71. Longest Non-Decreasing Subarray From Two Arrays — ColorStack 2025-09-24
M 72. Count Super Streaks — ColorStack 2025-10-03

Contents

288 questions | tap a topic, a main topic or a question to jump to it

9. Union-find: merge the pairs, then ask who is together (8) Go >

M 73. Count Islands in a Grid and Track the Count as Land Cells Are Added — PracHub 2026-10-02
M 74. 2D Points Connected Components by Radius — 1Point3Acres 2026-08-04
M 75. Count Clusters of 2D Points Within a Radius — PracHub 2026-06-18
M 76. Find A Threshold-Limited Path With Minimum Required Safety — PracHub 2026-07-06
M 77. Movie Recommendations via Transitive Similarity — ColorStack 2025-10-09
M 78. Determine Reporting Relationships — PracHub 2026-02-22
M 79. Determine whether two nodes are related — PracHub 2026-01-22
H 80. Friend Graph with Timeline (Friend / Unfriend Events) — 1Point3Acres 2026-09-29

10. A dict of per-key state that you update, expire, or rank (8) Go >

E 81. Robot Status Message Deduplication Window — 1Point3Acres 2026-09-10
M 82. Active Communication Top Users — 1Point3Acres 2026-09-22
M 83. Filter Repeated Robot Status Messages — PracHub 2026-07-05
M 84. Next-Word Predictor — 1Point3Acres 2026-09-02
M 85. Batch and Streaming Log Deduplication — 1Point3Acres 2026-07-23
M 86. Design a logger that returns top-K messages — PracHub 2026-01-06
M 87. Hit Counter — HackTheRounds no date
M 88. Website Activity Analytics: Unique Users + Session Time — 1Point3Acres 2026-06-26

11. Binary search on a yes/no condition (8) Go >

E 89. First Index Reaching a Threshold (Binary Search) — ColorStack 2025-10-11
M 90. Cutting Ribbons (LC 1891) — 1Point3Acres 2026-08-16

90 QUESTIONS DONE: topics 1 to 10 are finished. Keep going down the list for the rest.

M 91. First Bad Version with Parallel Search Follow-up — 1Point3Acres 2026-06-15
M 92. Solve Banana Speed and Interval Merge — PracHub 2025-01-27
M 93. Given a list with response time for each server, calculate how long time i... — time it will take to process n req... Exponent no date
H 94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — 1Point3Acres 2026-10-09
H 95. Find the median of two sorted arrays. — Exponent no date
H 96. Find longest duplicated substring — PracHub 2025-12-08

12. Greedy: scan once, carry the best choice so far, act only when forced (8) Go >

E 97. Maximum Profit — Exponent no date
E 98. Alarm for the Next Upcoming Birthday — ColorStack 2025-10-07
E 99. Coins on the Table — ColorStack 2025-09-20
M 100. Visible People in a Line (Last Person, Then Everyone) — 1Point3Acres 2026-10-07
M 101. Bounded Knapsack Variant — 1Point3Acres 2026-10-16
M 102. Find the Largest Monotone Increasing Number — PracHub 2026-07-04
M 103. Min boards to cover roof holes — PracHub 2026-02-12
M 104. Jump Game II (Minimum Jumps) — ColorStack 2025-08-19

13. Sets and counters: remember the first collection, check the second against it (8) Go >

E 105. Exclude First-K-Unique From Another List — ColorStack 2025-10-17
E 106. Find the Duplicates — Exponent no date
E 107. Positional Matches Between Two Lists — ColorStack 2025-10-15
E 108. Contains Duplicate — ColorStack 2025-09-16

Contents

288 questions | tap a topic, a main topic or a question to jump to it

M 109. Filter Common Elements Per Window (Two Lists) **ColorStack 2025-10-08**
M 110. Reconstruct original array from doubled shuffle **PracHub 2026-01-04**
M 111. Min deletions to avoid overlap in first k **PracHub 2026-02-12**
H 112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space **Space PracHub 2026-09-09**

14. bisect on a sorted list: find a position, count a range, keep it sorted (8) **Go >**

E 113. Count People in a Postal Code Range **ColorStack 2025-10-19**
M 114. Sliding-Window Mean After Removing the Largest K Values **PracHub 2026-09-01**
M 115. Russian Doll Envelopes **1Point3Acres no date**
M 116. Count Sorted Strings With a Given Prefix **ColorStack 2025-10-08**
M 117. Streaming Insert + Find K-th Largest **1Point3Acres 2026-02-04**
M 118. Given positions of N kids and M ice cream sellers on a 1D line, find the s... **the smallest distance E such... Exponent no date**
H 119. Find Maximum Envelope Nesting **PracHub 2026-03-18**
H 120. Count of Smaller Numbers After Self **HackTheRounds no date**

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8) **Go >**

E 121. Parity Alternating Subarray Query **HackTheRounds no date**
E 122. Cake Balance Line **ColorStack 2025-09-26**
E 123. Find the Pivot Index (Balanced Split) in O(n) **ColorStack 2025-09-21**
M 124. Range Duplicate Check with Constant-Time Queries **1Point3Acres 2026-09-29**
M 125. Find a Horizontal Cut That Bisects Rectangle Area **PracHub 2026-06-04**
M 126. Check pivot after removing one element **PracHub 2026-02-12**
M 127. Gold Chain Split into Two Equal Halves **1Point3Acres 2026-04-21**
M 128. Count the number of ways to split a string. **Exponent no date**

16. Sorted intervals: merge while they touch, then read the gaps or search (8) **Go >**

E 129. Check an Interval Against Existing Intervals for Conflicts **PracHub 2026-08-08**
M 130. Find Containing Range **PracHub 2026-06-04**
M 131. Union of Sorted Interval Lists **1Point3Acres 2026-08-16**
M 132. Range Module (Insert and Point Query) **ColorStack 2026-01-20**
M 133. Meetings Minus a Do-Not-Schedule Block **ColorStack 2025-10-03**
M 134. Merge Intervals **Exponent no date**
H 135. Find safe travel intervals between planet influences **PracHub 2025-11-22**
H 136. Implement a function that processes a stream of interval data. **Exponent no date**

17. Trie: nested dicts, one character per level (8) **Go >**

M 137. URL Request Handler Router **1Point3Acres 2026-10-16**
M 138. Trie Prefix Search / Autocomplete **1Point3Acres 2026-07-02**
M 139. Build Prefix Lookup with a Trie **PracHub 2026-07-03**
M 140. Greedy Longest-Match Dictionary Tokenizer **1Point3Acres 2026-06-15**
M 141. Check if all substrings are dictionary words **PracHub 2026-03-11**
M 142. Return dictionary words matching a prefix **PracHub 2026-03-09**
M 143. Implement Trie Insert and Exact-Word Search **PracHub 2026-02-20**
H 144. Implement Longest-Match Text Replacement **PracHub 2026-01-20**

Contents

288 questions | tap a topic, a main topic or a question to jump to it

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8) Go >

E 145. Implement Batched Undo/Redo Layer — PracHub 2026-02-04
M 146. Design a restaurant waitlist system — PracHub 2026-03-01
M 147. Implement Python's itertools.tee — 1Point3Acres 2026-08-20
M 148. Store With Cheapest-Item Removal — ColorStack 2025-09-29
M 149. Round-Robin Job Scheduler — ColorStack 2025-08-19
M 150. Aggregate FizzBuzz Offer Matches — ColorStack 2025-10-13
M 151. Restaurant Waitlist: join / delete / find_first_match — 1Point3Acres 2026-02-01
H 152. Implement LRU Cache. — Exponent no date

19. Rolling state: the next state comes only from the previous one (7) Go >

E 153. Climbing Stairs — Exponent no date
E 154. Fibonacci Numbers — Exponent no date
M 155. Maximize sum without choosing adjacent elements — PracHub 2026-01-06
M 156. Given n houses in a line with money in each, find the maximum amount a rob... — a robber can steal, witho... Exponent no date
M 157. Paint House II — ColorStack 2025-10-17
M 158. Write SQL code to publish the Fibonacci series. — Exponent no date
H 159. Constrained Dice Roll Sequences — ColorStack 2025-10-06

20. Two pointers: move i and j toward each other (or along two lists) (7) Go >

E 160. Reverse a linked list. — Exponent no date
E 161. Sort a binary array in linear time. — Exponent no date
E 162. Is this a valid palindrome? — Exponent no date
E 163. Reverse a Sentence — Exponent no date
M 164. Find median of merged RLE arrays — PracHub 2026-01-06
M 165. Run-Length Vector Storage + Dot Product — 1Point3Acres 2026-01-21
M 166. Count Unique Triplets That Sum to Zero — ColorStack 2025-09-20

21. Walk two strings at once with two indices (7) Go >

E 167. Find the First Occurrence of a Substring — ColorStack 2025-09-16
E 168. Determine if two words are sorted in lexicographic order. — Exponent no date
E 169. Return all unique substrings of a given string. — Exponent no date
M 170. Can board states be transformed? — PracHub 2026-04-01
M 171. Decide string transform with directional tokens and walls — PracHub 2025-07-15
M 172. Determine if two sentences are similar. — Exponent no date
H 173. Find Dictionary Matches for a Jammed String — PracHub 2026-04-09

22. Simulate a machine: a class with state and one branch per kind of event (7) Go >

E 174. Mario-Style Score Tracker (OOP) — ColorStack 2025-10-11
M 175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win — 1Point3Acres 2026-05-18
M 176. Design Tic-Tac-Toe With K Players — PracHub 2026-02-12
M 177. Infinite Deck Card Duel — ColorStack 2025-09-26
M 178. Implement iterator merging lists with filtering — PracHub 2026-01-06
M 179. Design an elevator control system — PracHub 2025-09-06
H 180. Simulate In-Place Cellular State Updates — PracHub 2026-05-04

Contents

288 questions | tap a topic, a main topic or a question to jump to it

23. Top-K by frequency: Counter plus nsmallest (6) Go >

- M 181. Top-K Frequent in Large Logs — 1Point3Acres 2026-07-19
- M 182. Rearrange Tasks With Cooldown — PracHub 2026-06-03
- M 183. Implement frequency + distance top-K queries — PracHub 2026-02-12
- M 184. Solve chat, grid paths, and car rentals — PracHub 2025-12-02
- M 185. Top-K Chatters by Word Count — ColorStack 2025-09-21
- M 186. Find top/bottom-k words in list or stream — PracHub 2026-01-11

24. Digits and characters: peel one at a time and carry (6) Go >

- E 187. Caesar Cipher: Recover Key, Encrypt, and Decrypt — ColorStack 2026-02-08
- M 188. Add two big integers from digit lists — PracHub 2025-11-24
- M 189. Solve several coding interview problems — PracHub 2026-03-04
- M 190. Check Whether Digits Are Monotone Increasing — PracHub 2026-07-04
- H 191. Check Digits and Combine Ranges — PracHub 2026-03-29
- H 192. Permutation Sequence — ColorStack 2025-10-17

25. Look at the input once and keep only a small summary (and count your calls) (6) Go >

- M 193. Two-Day Log Intersection under Memory Limit — 1Point3Acres 2025-11-27
- M 194. Find shared objects across two log files — PracHub 2025-11-05
- M 195. Streaming Points: Emit K Within Distance — 1Point3Acres 2026-06-23
- M 196. Minimize calls to find all bad test pairs — PracHub 2025-11-05
- H 197. Find All Bad Pairs Using runTest Oracle — 1Point3Acres 2025-11-27
- H 198. Find maximum value in huge text file — PracHub 2025-12-08

26. Grid DP: each cell from its neighbours above and to the left (5) Go >

- M 199. Grid Path Count with Diagonal Moves and Checkpoints — 1Point3Acres 2026-09-29
- M 200. Longest Same-Color Grid Sequence — 1Point3Acres 2026-09-08
- M 201. Count Same-Color Squares in Matrix — 1Point3Acres no date
- H 202. Find the Longest Straight Run of One Color — PracHub 2026-08-23
- H 203. Count same-color squares in a character grid — PracHub 2025-11-22

27. Nested structures: a stack of contexts, or recursion with a cache (5) Go >

- M 204. Decode String — 1Point3Acres 2026-07-29
- M 205. Template Variable Expansion with Follow-Ups — 1Point3Acres 2026-10-17
- M 206. Expand Recursive Key-Value Placeholders in a Template — PracHub 2026-08-04
- M 207. Nested List Weight Sum (LC 339) with Follow-Ups — 1Point3Acres 2026-08-16
- M 208. Nested Function Expression Evaluator — 1Point3Acres 2026-06-10

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on th... Go >

- M 209. Shortest Path Nodes and Escaping a Pursuer — 1Point3Acres 2026-09-29
- M 210. Radio Broadcast Reachability — ColorStack 2026-01-30
- M 211. Minimum Direction Violations in a Directed Graph — 1Point3Acres 2026-07-19
- M 212. Compute shortest paths with blocked nodes — PracHub 2025-09-06
- M 213. Shortest Directed Cycle Through a Node — 1Point3Acres no date

Contents

288 questions | tap a topic, a main topic or a question to jump to it

29. Topological sort (Kahn) and cycle detection (5) Go >

M 214. Course Scheduling and Dependency Variants — 1Point3Acres 2026-10-04
M 215. Validate Race Results (Cycle Detection) — ColorStack 2025-10-01
M 216. Solve tree, graph, sliding-window problems — PracHub 2025-08-04
M 217. Validate course catalog dependencies — PracHub 2025-08-01
M 218. Course Schedule — Exponent no date

30. Counting pairs by looking up each partner in a Counter or a bucket (5) Go >

M 219. Progressive Array Partitioning — 1Point3Acres 2026-08-11
M 220. K-diff Pairs in an Array — ColorStack 2025-09-16
M 221. Count Axis-Aligned Squares Among Grid Points — ColorStack 2025-11-03
M 222. Sum of Two Squares in Two Ways — HackTheRounds no date
H 223. Maximum-Area Rectangle from Planar Points — 1Point3Acres 2026-09-29

31. Tree DP: build a parent order once, then add up from the leaves (5) Go >

M 224. Compute File System Total Size — 1Point3Acres 2026-09-02
M 225. Minimum-Cost Tree Disconnection — 1Point3Acres 2026-09-09
M 226. Given a large n-ary tree where the root node can pass information to on... — to one child per iteration, and i... Exponent no date
H 227. Compute distance-sum from every tree node — PracHub 2026-01-22
H 228. Tree Distance Sum (Re-root DP) — 1Point3Acres 2026-01-19

32. Counting or reaching an amount: the knapsack loop (5) Go >

M 229. Maximize Shelf Space (Unbounded Knapsack) — ColorStack 2025-10-17
M 230. Coin Change — Exponent no date
M 231. Partition an array into two sub-arrays with equal sum. — Exponent no date
M 232. Explain how to find a target sum in an array. — Exponent no date
M 233. Reverse Coin Change — HackTheRounds no date

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5) Go >

M 234. Maximum-Sum Subarray With Equal Endpoints — ColorStack 2025-09-29
M 235. Maximum Subarray Sum With a Minimum Length Constraint — ColorStack 2025-08-19
M 236. Solve array-sum and city-community problems — PracHub 2026-02-20
M 237. Find the maximum subarray sum. — Exponent no date
H 238. Given an integer array nums and an integer k, return true if nums has a... — has a subarray of at least two e... Exponent no date

34. Group by a canonical key (5) Go >

E 239. Given an array of strings, return the words that can be typed using letter... — letters of the alphabet on only... Exponent no date
M 240. Check if all substrings are anagrams of words — PracHub 2026-03-11
M 241. Group anagrams — Exponent no date
M 242. Given an array of strings, return groups of "buddy strings," where the st... — the strings have the same lengt... Exponent no date
H 243. Given two strings and two allowed operations, swapping positions of lett... — of letters and converting all oc... Exponent no date

35. K-way merge: a heap of the next item from each source (5) Go >

E 244. Find k pairs with smallest sums — PracHub 2025-11-01
M 245. K Largest Elements Across Sorted Lists — ColorStack 2025-10-31
M 246. Sort a nearly sorted (or K sorted) array. — Exponent no date

Contents

288 questions | tap a topic, a main topic or a question to jump to it

H 247. Merge k sorted linked lists. **Exponent no date**

H 248. Return top-k subset sums in descending order **PracHub 2026-01-07**

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5) Go >

E 249. Next Birthday (BST In-Order Successor with Wraparound) **ColorStack 2025-10-17**

M 250. Lowest Common Ancestor **HackTheRounds no date**

M 251. Determine if a given binary tree is a binary search tree (BST). **Exponent no date**

M 252. Find mode in a trinary search tree **PracHub 2025-11-16**

M 253. Next Upcoming Birthday in a Family Tree **ColorStack 2025-10-02**

37. Random pick without repeats: swap the chosen item with the last one and pop (5) Go >

M 254. Random Generator Without Repeats **ColorStack 2025-10-10**

M 255. Random Song Shuffler With a No-Repeat Window **ColorStack 2025-10-06**

M 256. Design set with O(1) random access **PracHub 2026-01-01**

M 257. Unique Random Number Generator **ColorStack 2025-10-03**

H 258. Shuffle array using random integer API **PracHub 2025-12-08**

38. Build and serialize: write a structure out, read it back with the same walk (5) Go >

M 259. Construct a binary tree given its preorder and inorder traversals. **Exponent no date**

M 260. Write functions to serialize and deserialize a list of strings. **Exponent no date**

M 261. Print a Directory Tree **ColorStack 2025-09-21**

H 262. Implement a decode and encode function of a general N-ary tree. **Exponent no date**

H 263. Serialize and Deserialize a Graph **ColorStack 2025-09-24**

39. Games: win or lose with a cache, and guessing with feedback (5) Go >

M 264. Choose optimal guesses for green-only Wordle **PracHub 2025-10-14**

H 265. Wordle-Style 5-Letter Minimum-Guess Strategy **1Point3Acres 2025-12-15**

H 266. Determine winner in stack-merging game **PracHub 2026-01-06**

H 267. Find secret word with match-count feedback **PracHub 2026-02-11**

H 268. Babylon Tile-Stack Merge Game **1Point3Acres 2025-12-29**

40. Sliding window: grow on the right, shrink on the left while it is too bad (5) Go >

M 269. Temperature Monitoring Data Structure **1Point3Acres 2026-09-10**

M 270. Shortest Subarray with K Distinct Integers **HackTheRounds no date**

M 271. Find the minimum window substring. **Exponent no date**

M 272. Footpath Beauty After Repainting **ColorStack 2025-09-22**

H 273. Necklace Cut into Two Halves with Equal D/R Counts **1Point3Acres 2026-05-03**

41. Parent pointers: store who is above each node and climb (5) Go >

E 274. Tree Node. **Exponent no date**

M 275. Employee Hierarchy Add / Delete **1Point3Acres 2026-06-10**

M 276. Validate parent array forms a tree **PracHub 2026-02-11**

M 277. Lowest Common Ancestor of a Binary Tree III **HackTheRounds no date**

M 278. Root a Tree So It Becomes a Binary Tree **ColorStack 2025-11-07**

Contents

288 questions | tap a topic, a main topic or a question to jump to it

42. Build or transform a grid with index arithmetic (5) Go >

E 279. Filled ASCII Diamond **ColorStack 2025-09-26**
M 280. Connected Crop Arrangement on a Grid **1Point3Acres 2026-10-04**
M 281. Rotate Any m×n 2D Matrix **1Point3Acres no date**
M 282. Solve matrix groups and recipe inventory **PracHub 2026-01-22**
M 283. Range Sum Query 2D (Immutable Matrix) **ColorStack 2025-08-19**

43. Split into pieces, fix each piece, join (and pack words into lines) (5) Go >

E 284. Implement string formatting and request tracking tasks **PracHub 2026-02-11**
M 285. Snake Case to Camel Case with Repository-Wide Refactor **1Point3Acres 2026-07-14**
M 286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) **1Point3Acres 2026-02-01**
H 287. Implement a function to justify a list of words within a given width. **Exponent no date**
H 288. Convert Valid Snake Case to Lower Camel Case **PracHub 2026-04-16**

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 1 OF 43 |

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	6	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

E	1. Flood Fill	ColorStack	2026-01-28
M	2. Number of Islands	1Point3Acres	2026-09-30
M	3. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres	2026-09-08
M	4. Solve Grid and Counting Problems	PracHub	2025-01-25
M	5. Find shortest relationship path using BFS	PracHub	2026-02-12
M	6. Farthest City for a New Data Center	ColorStack	2025-10-10
M	7. Compute distance to nearest taxi in grid	PracHub	2026-01-10
H	8. Infection Spread on a Grid (Cellular Automaton)	PracHub	2026-06-14
H	9. Shortest Distance From All Buildings	ColorStack	2025-09-22

CODE TEMPLATE | GROUP 1 OF 43 | 9 QUESTIONS, ONE PATTERN

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

The one idea these questions share

Every one of these keeps a frontier of cells or nodes still to visit, takes one out, looks at its neighbours, and adds the new ones. A dict of distances doubles as the seen set. Change what you start from, which neighbours are allowed, and what you do when a node comes out.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


def frontier_search(starts, neighbors, usable=lambda v: True):
    dist = {s: 0 for s in starts}          # doubles as the seen set
    frontier = deque(starts)               # popleft = BFS, pop = DFS
    while frontier:
        u = frontier.popleft()
        # CHANGE 1: act on u here (repaint it, count it, stop at the goal)
        for v in neighbors(u):             # CHANGE 2: grid steps or adj[u]
            if v not in dist and usable(v):    # CHANGE 3: colour, wall, blocked
                dist[v] = dist[u] + 1
                frontier.append(v)
    return dist
```

2. Helper functions that usually get written next to it

```
def grid_steps(rows, cols):
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc)
    return step


def rebuild_path(parent, target):         # walk parent[] back from the target
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return path[::-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
1	Flood Fill	starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour
2	Number of Islands	loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start
3	Rotten Oranges / Multi-Source BFS (taxis)	starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached
4	Solve Grid and Counting Problems	the island count again (queue version); the second part of the question is a short counting formula, no search
5	Find shortest relationship path using BFS	one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path

Contents Start the questions >

#	Question	What you change in the template
6	Farthest City for a New Data Center	same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance
7	Compute distance to nearest taxi in grid	starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1
8	Infection Spread on a Grid (Cellular Automaton)	starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)
9	Shortest Distance From All Buildings	run it once per building (one start each); add each run into a total table and count how many buildings reached each cell

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 1 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 1 of 9

Template fit: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour | asked 265 days ago | frequency Single report

EASY | IN APP LC 733 | ColorStack | technical interview | 1 of 288

Flood Fill

ASKED 2026-01-28 | 265 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 242/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: your app's interview approach (LeetCode #733)

LeetCode match: LC 733 Flood Fill

Frequency: Single report | Last asked: 2026-01-28 | Roles: SWE Intern

QUESTION

You are given an m x n grid image where image[r][c] is the integer color of the pixel at row r , column c . You are also given a starting pixel (sr, sc) and a new_color . Perform a flood fill: starting from (sr, sc) , repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to new_color . Pixels connected only diagonally are not filled. Return the modified grid.

Function signature

from typing import List
def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
...

Follow-up (as reported): Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when new_color equals the original color.

EXAMPLES

Example 1	
Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Why	`Starting color is 1. All 1-pixels connected 4-directionally to (1,1) become 2. The bottom-right 1 is only diagonally reachable, so it stays.`

Example 2	
Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5`
Output	`[[5,5,5],[5,5,5]]`
Why	`The whole grid is one connected region of color 0.`

QUESTION

Example 3	
Input	`image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1`
Output	`[[0,0,0],[0,1,0]]`
Why	`The start pixel already equals new_color, so nothing changes.`

CONSTRAINTS

- 1 <= m, n <= 50
- 0 <= image[r][c], new_color < 2^16
- 0 <= sr < m , 0 <= sc < n
- If the starting pixel already has new_color , the grid is returned unchanged (guard against infinite recursion).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_flood(image, sr, sc, new_color):
    old = image[sr][sc]
    marked = {(sr, sc)}
    changed = True
    while changed:
        changed = False
        for r in range(len(image)):
            for c in range(len(image[0])):
                if (
                    image[r][c] == old
                    and (r, c) not in marked
                ):
                    near = (
                        (r + 1, c),
                        (r - 1, c),
                        (r, c + 1),
                        (r, c - 1),
                    )
                    if any(
                        p in marked
                        for p in near
                    ):
                        marked.add((r, c))
                        changed = True
    return [
        [
            (
                new_color
                if (r, c) in marked
                else
                    v
            )
            for c, v in enumerate(row)
        ]
        for r, row in enumerate(image)
    ]
```

TIME	SPACE
O((mn)^2)	O(1)

WHY IT IS SLOW

Each sweep touches every pixel even if only one more pixel changes.

OPTIMAL SOLUTION · Iterative DFS with a guard

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it with BFS. Why is the early return needed?

A deque gives BFS with the same cost. If the new colour equals the old one, repainting never changes the test 'pixel has the old colour', so pixels would be pushed again forever; the early return (or a visited set) stops that.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Flood Fill
#
# You are given an m x n grid image where image[r][c] is
# the integer color of the pixel at row r , column c . You
# are also given a starting pixel (sr, sc) and a new_color
# . Perform a flood fill: starting from (sr, sc) , repaint
# that pixel and every pixel connected to it
# 4-directionally (up, down, left, right) that shares the
# SAME original color as the starting pixel, changing them
# all to new_color . Pixels connected only diagonally are
# not filled. Return the modified grid. Function
# signature: from typing import List def flood_fill(image:
# List[List[int]], sr: int, sc: int, new_color: int) ->
# List[List[int]]: ... Follow-up (as reported): Implement
# with DFS (recursive or explicit stack) or BFS; discuss
# the base case that prevents re-visiting when new_color
# equals the original color.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with BFS. Why is the early return needed?
#
# INTERVIEW NOTES
# Remember the starting colour, then spread from the start
# pixel to its four neighbours that have that colour,
# repainting as you go. If the new colour equals the old
# colour there is nothing to do, and returning early also
# prevents an infinite loop. An explicit stack avoids
# Python's recursion limit.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random
import copy
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Iterative DFS with a guard
# Idea: Spread over same-coloured neighbours and repaint;
# skip everything when the colour does not change.
def flood_fill(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    stack = [(sr, sc)]
    image[sr][sc] = new_color
    while stack:
        r, c = stack.pop()
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < len(image)
                and 0 <= nc < len(image[0])
                and image[nr][nc] == old
            ):
                image[nr][nc] = new_color
                stack.append((nr, nc))
    return image

# ---- main question: tests ----
def test_main_question():
    random.seed(86)
    img = [[1, 1, 1], [1, 1, 0], [1, 0, 1]]
    assert flood_fill(copy.deepcopy(img), 1, 1, 2) == [
        [2, 2, 2],
        [2, 2, 0],
        [2, 0, 1],
    ]


# ============================================================
# FOLLOW-UP 1: Do it with BFS. Why is the early return
# ============================================================

# ---- follow-up 1: function ----
# Answer: A deque gives BFS with the same cost. If the new
# colour equals the old one, repainting never changes the
```

```
# test 'pixel has the old colour', so pixels would be
# pushed again forever; the early return (or a visited
# set) stops that.
def flood_fill_bfs(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    image[sr][sc] = new_color
    queue = deque([(sr, sc)])
    while queue:
        r, c = queue.popleft()
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < len(image)
                and 0 <= nc < len(image[0])
                and image[nr][nc] == old
            ):
                image[nr][nc] = new_color
```

```
                queue.append((nr, nc))
    return image


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 2 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 2 of 9

Template fit: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start | asked 20 days ago | frequency Medium

MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 2 of 288

★ TOP 75: rank 17 in the Medium and Hard priority list

★ Number of Islands

ASKED 2026-09-30 | 20 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 64.8** | **30 DAYS 33** | **PRIORITY 23/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: your app's interview approach (LeetCode #200)

LeetCode match: LC 200 Number of Islands

Frequency: Medium | Last asked: 2026-09-30 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/number-of-islands

QUESTION

Given a grid of 0s (water) and 1s (land), count the islands. An island is a group of 1s connected up, down, left or right (diagonal cells are not connected).

Function signature

def num_islands(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]`
Output	`2`
Why	`Two 2x2 blocks of land that only touch the water between them: 2 islands.`

Example 2	
Input	`[[1, 0, 1], [0, 1, 0], [1, 0, 1]]`
Output	`5`
Why	`Land that touches only diagonally is not connected, so every 1 is its own island.`

Example 3	
Input	`[[0, 0], [0, 0]]`
Output	`0`
Why	`No land, no islands.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == 0 or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
def dfs(r, c):
    if out_of_grid(r, c) or grid[r][c] == 0:
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in DIRECTIONS:
        dfs(r + dr, c + dc)
```

TIME	SPACE
O(m * n)	**O(m * n)** recursion

WHY IT IS SLOW

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

OPTIMAL SOLUTION · Iterative DFS flood fill

TIME	SPACE
O(mn)	**O(mn)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

FOLLOW-UP 2 How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

FOLLOW-UP 3 Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

FOLLOW-UP 4 The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. **Cost:** O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Number of Islands
#
# Given a grid of 0s (water) and 1s (land), count the
# islands. An island is a group of 1s connected up, down,
# left or right (diagonal cells are not connected).
#
# Signature:
#   def num_islands(grid: list) -> int
#
# Examples:
# [[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]
#     ->  2
# [[1, 0, 1], [0, 1, 0], [1, 0, 1]]  ->  5
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if land can also be removed?
# 2. How do you handle a grid too large for memory?
# 3. Find the area of the largest island.
# 4. The grid is huge and does not fit in memory.
#
# INTERVIEW NOTES
# Each island is a connected group of land cells, so count
# how many times you start a fresh search at an unvisited
# land cell. The search itself (DFS or BFS) just marks
# everything reachable. If land is added over time, do not
# rerun the search: Union-Find keeps the count up to date.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc


def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size


def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes

# ---- main question: main function ----
# BASE QUESTION (optimal): Iterative DFS flood fill
# Idea: Scan every cell; each unvisited land cell starts a
# new island, and an explicit stack marks all connected
# land as visited. Iterative DFS avoids Python's recursion
# limit on large grids.
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))


# VARIANT: land cells arrive one at a time
```

```
class IslandCounter:
    """Follow-up: land cells arrive one at a time."""

    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        # land cell -> parent cell (union-find)
        self.parent = {}
        self.count = 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_land(self, r, c):
        # adding the same cell twice changes nothing
        if (r, c) in self.parent:
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        # join with every land neighbour
        for nb in neighbors(r, c, self.rows, self.cols):
            if nb in self.parent:
                a, b = self.find((r, c)), self.find(nb)
                if a != b:
                    self.parent[a] = b
```

```
                    # two islands merged into one
                    self.count -= 1
        return self.count

# ---- main question: tests ----
def test_main_question():
    g = [[1, 1, 0, 0], [1, 1, 0, 0],
         [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
```

```
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Find the area of the largest island.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Count the cells during each search and keep the
# maximum. Same traversal, one counter.
def largest_island(grid):
    # follow-up: the area of the biggest island
    return max(island_sizes(grid), default=0)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = [[1, 1, 0, 0], [1, 1, 0, 0],
         [0, 0, 1, 1], [0, 0, 1, 1]]
    assert largest_island(g) == 4
    assert num_islands([]) == 0
    # streaming adds: counts stay 1, 1, 1
    ic = IslandCounter(2, 2)
    adds = [(0, 0), (0, 1), (1, 1)]
    counts = [ic.add_land(r, c) for r, c in adds]
    assert counts == [1, 1, 1]
    # two separate cells make two islands
    ic = IslandCounter(3, 3)
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

STUDY PATH: question 3 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 3 of 9

Template fit: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached | asked 42 days ago | frequency Low

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 3 of 288

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 0** | **PRIORITY 77/288**

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app: LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

Each cell of a grid is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, each fresh orange that is next to (up, down, left or right) a rotten one becomes rotten. Return the number of minutes until no fresh orange is left, or -1 if some fresh orange can never rot.

Function signature

def rotting_minutes(grid: list) -> int

EXAMPLES

Example 1

Input	`[[2, 1, 1], [1, 1, 0], [0, 1, 1]]`
Output	`4`
Why	`The rot spreads outward from the corner and reaches the last orange after 4 minutes.`

Example 2

Input	`[[2, 1, 1], [0, 1, 1], [1, 0, 1]]`
Output	`-1`
Why	`The orange at the bottom left is cut off by empty cells, so it never rots.`

Example 3

Input	`[[0, 2]]`
Output	`0`
Why	`Nothing fresh to rot: 0 minutes.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Rotten Oranges / Multi-Source BFS (taxis)
#
# Each cell of a grid is 0 (empty), 1 (a fresh orange) or
# 2 (a rotten orange). Every minute, each fresh orange
# that is next to (up, down, left or right) a rotten one
# becomes rotten. Return the number of minutes until no
# fresh orange is left, or -1 if some fresh orange can
# never rot.
#
# Signature:
#   def rotting_minutes(grid: list) -> int
#
# Examples:
# [[2, 1, 1], [1, 1, 0], [0, 1, 1]]  ->  4
# [[2, 1, 1], [0, 1, 1], [1, 0, 1]]  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Sources move or new taxis appear. How do you keep the
# distances fresh?
# 2. Why is multi-source BFS the same cost as single-
# source BFS?
# 3. Signal variant: a signal spreads from sources, how
# long until every target gets it?
# 4. A signal spreads from sources to targets: how long
# until every target is reached?
#
# INTERVIEW NOTES
# 'Spread outward from several starting points at the same
# time and count the rounds' is multi-source BFS: put
# every source in the queue before you start, and process
# the queue one round at a time. Rounds are minutes.
# Always count the fresh ones first so that unreachable
# oranges are detected at the end.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist

# ---- main question: main function ----
# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
            for r in range(rows)]

# ---- main question: tests ----
def test_main_question():
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]


# ============================================================
# FOLLOW-UP 1
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: A signal spreads from sources to targets
# ============================================================

# ---- follow-up 4: function ----
# Answer: One multi-source BFS from the sources, then the
# answer is the largest distance over the targets (-1 if
# any target is unreachable, 0 if there are no targets).
# Per-source BFS would cost O(mn * sources).
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)

# ---- follow-up 4: tests ----
def test_follow_up_4():
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

STUDY PATH: question 4 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 4 of 9

Template fit: the island count again (queue version); the second part of the question is a short counting formula, no search | asked 633 days ago

MEDIUM | PracHub | Technical Screen | 4 of 288

★ TOP 75: rank 27 in the Medium and Hard priority list

★ Solve Grid and Counting Problems

POSTED 2025-01-25 | 633 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 64.8 | 30 DAYS 33 | PRIORITY 22/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #200 Number of Islands

LeetCode match: LC 200 Number of Islands (+ a counting task)

Posted on PracHub: 2025-01-25 (updated 2026-04-24)

Original: https://prachub.com/coding-questions/solve-grid-and-counting-problems

QUESTION

A software engineering intern phone screen included two coding tasks: 1.

Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.

Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.

Constraints

0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:

Explanation: Empty grid contains no land, so there are no islands.

QUESTION

```
Input: [['1']]
```

Expected Output:

Explanation: A single land cell forms one island.

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []

Expected Output:

Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']

Expected Output:

Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

BRUTE FORCE · Islands: recursive flood fill

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == "0" or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is O(m * n), but the recursion can be m * n deep on a snake-shaped island and overflow the stack. A queue or an explicit stack avoids that.

BRUTE FORCE · Counting: enumerate every 6-element selection

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    count = 0
    for idx in combinations(range(len(a)), 6):
        letters = sorted(a[i] for i in idx)
        if letters == sorted("GGOOLE"):
            count += 1
    return count
```

TIME	SPACE
O(C(n, 6) * 6)	O(1)

WHY IT IS SLOW

C(n, 6) selections. The letters needed are fixed, so the answer is C(#G, 2) * C(#O, 2) * #L * #E from one pass of counts.

OPTIMAL SOLUTION · Islands: iterative BFS

TIME	SPACE
O(m * n)	O(min(m, n)) queue

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Counting: product of binomial coefficients

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

FOLLOW-UP 2 The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

FOLLOW-UP 3 Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve Grid and Counting Problems
#
# A software engineering intern phone screen included two
# coding tasks: 1. **Count islands in a grid** Given an m
# x n grid of characters where '1' represents land and '0'
# represents water, return the number of islands. An
# island is formed by connecting adjacent land cells
# horizontally or vertically. You may assume all four
# edges of the grid are surrounded by water. 2. **Count
# ways to form a target word** Given an array of uppercase
# letters, count how many different ways you can choose
# letters from the array so that the chosen letters can be
# rearranged to spell GOOGLE. Each array element can be
# used at most once, and letters at different indices are
# considered different choices even if they contain the
# same character. Design efficient algorithms for both
# tasks, and discuss the time and space complexity of your
# approach.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Any target word, not only GOOGLE.
# 2. The letters must appear in order (a subsequence of
# the array spells the word).
# 3. Islands arrive one cell at a time.
#
# INTERVIEW NOTES
# Islands: every unvisited land cell starts a new island;
# mark everything connected to it. Counting: the target
# multiset is fixed (G twice, O twice, L once, E once), so
# the number of ways to choose the indices is a product of
# binomial coefficients, one factor per distinct letter.
# Do not enumerate arrangements.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter, deque
from math import comb



# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# ---- main question: main function ----
# BASE QUESTION (optimal): Islands: iterative BFS
# Idea: Same scan, but spread with a queue instead of
# recursion, so the depth of an island cannot overflow the
# stack. Marking visited cells in place (or with a set)
# keeps each cell processed once.
def num_islands(grid):
    # '1' is land: a BFS marks each island once
```

```
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < m and 0 <= b < n):
                        continue
                    land = grid[a][b] == "1"
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        queue.append((a, b))
    return count


# PART 2: count GOOGLE in the array
# FOLLOW-UP 1: Any target word, not only GOOGLE.
# Answer: Count the needed letters of the word; the answer
# is the product over distinct letters of C(available,
# needed).
def ways_to_spell(letters, word="GOOGLE"):
    # choose which array positions to use: for every
```

```
    # needed
    # letter choose that many of the available copies
    have, need = Counter(letters), Counter(word)
    total = 1
    for ch, k in need.items():
        total *= comb(have[ch], k)
    return total

# ---- main question: tests ----
def test_main_question():
    assert num_islands([]) == 0
    assert num_islands([["1"]]) == 1
    grid = ["11000", "11000", "00100", "00011"]
    assert num_islands(grid) == 3
    assert ways_to_spell([]) == 0
    assert ways_to_spell(list("GOOGLE")) == 1
    # three O's: choose 2 of them -> 3 ways
    assert ways_to_spell(list("GOOOGLE")) == 3
    # an extra G and an extra E: 3 * 2 = 6 ways
    assert ways_to_spell(list("GGGOOLEE")) == 3 * 2
    assert ways_to_spell(list("GOGLE")) == 0
    # any target word
    assert ways_to_spell(list("AABB"), "AB") == 4


# ============================================================
# FOLLOW-UP 1: Any target word, not only GOOGLE.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: The letters must appear in order (a
# ============================================================

# ---- follow-up 2: function ----
# Answer: Then it is the 'distinct subsequences' DP:
# ways[j] counts the ways to spell the first j letters;
# scan the array and update j from high to low. O(n *
# len(word)).
```

```
def ways_in_order(letters, word="GOOGLE"):
    # the letters must appear in order (a subsequence):
    # the
    # "distinct subsequences" DP, updated from high to low
    ways = [1] + [0] * len(word)
    for ch in letters:
```

```
        for j in range(len(word), 0, -1):
            if word[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(word)]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert ways_in_order(list("GOOGLE")) == 1
    assert ways_in_order(list("GGOOGLE")) == 2
    assert ways_in_order(list("GOGOLE")) == 0
    assert ways_in_order(list("EGOOGL")) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[str]]) -> int:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`0`
`[['1']]`	`1`

STUDY PATH: question 5 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 5 of 9

Template fit: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path | asked 250 days ago

MEDIUM | IN APP LC 127 | PracHub | Technical Screen | 5 of 288

Find shortest relationship path using BFS

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 41.8 | 30 DAYS 0 | PRIORITY 100/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #127 Word Ladder

LeetCode match: LC 127 Word Ladder (BFS shortest path) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/find-shortest-relationship-path-using-bfs

QUESTION

People are connected by relationships, given as an adjacency dict that maps each person to the people they are directly related to. shortest_path(adj, s, t) returns the shortest chain of people from s to t (both ends included), or None when t cannot be reached.

Function signature

def shortest_path(adj: dict, s, t) -> list | None

EXAMPLES

Example 1

Input	`adj = {'ann': ['bob', 'cy'], 'bob': ['dee'], 'cy': ['dee', 'eli'], 'dee': ['fay'], 'eli': ['fay'], 'fay': []}`
	`shortest_path(adj, 'ann', 'fay')`
	`shortest_path(adj, 'ann', 'ann')`
	`shortest_path(adj, 'fay', 'ann')`

Output	`shortest_path(adj, 'ann', 'fay') -> ['ann', 'bob', 'dee', 'fay']`
	`shortest_path(adj, 'ann', 'ann') -> ['ann']`
	`shortest_path(adj, 'fay', 'ann') -> null`

Why	`Several chains of 4 people exist; BFS finds the first. A person is trivially related to themselves, and fay has no outgoing relationships.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = None
def dfs(node, path):
    global best
    if node == target:
        if not best or len(path) < len(best):
            best = list(path)
        return
    for nxt in graph[node]:
        if nxt not in path:
            dfs(nxt, path + [nxt])
dfs(source, [source])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Breadth-first search reaches the target along a shortest path first, O(V + E), and bidirectional BFS explores far fewer nodes on big graphs.

OPTIMAL SOLUTION · Breadth-first search with a parent map

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

FOLLOW-UP 2 Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

FOLLOW-UP 3 Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find shortest relationship path using BFS
#
# People are connected by relationships, given as an
# adjacency dict that maps each person to the people they
# are directly related to. shortest_path(adj, s, t)
# returns the shortest chain of people from s to t (both
# ends included), or None when t cannot be reached.
#
# Signature:
#   def shortest_path(adj: dict, s, t) -> list | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is huge (a social network).
# 2. Return every shortest path, or count them.
# 3. Relationships have different strengths (weights).
#
# INTERVIEW NOTES
# Unweighted means every hop costs the same, so breadth-
# first search finds the fewest hops: it explores all
# people one hop away, then two hops, and the first time
# it reaches the target is the shortest path. Keep a
# parent map to rebuild the path and a visited set so
# nobody is queued twice.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
```

```
# parents back to the source and reverse to get the path.
def shortest_path(adj, s, t):
    # BFS: the first time we reach a node is by a shortest
    # path
    if s == t:
        return [s]
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in parent:
                parent[v] = u
                if v == t:
                    # walk the parent links back to s
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(v)
    return None

# ---- main question: tests ----
def test_main_question():
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"],
           "d": ["e"], "e": []}
    assert shortest_path(adj, "a", "e") in (
        ["a", "b", "d", "e"], ["a", "c", "d", "e"])
    assert shortest_path(adj, "a", "a") == ["a"]
    assert shortest_path(adj, "e", "a") is None
    both_ways = {"a": ["b", "c"], "b": ["a", "d"],
                 "c": ["a", "d"], "d": ["b", "c", "e"],
                 "e": ["d"]}


# ============================================================
# FOLLOW-UP 1: The graph is huge (a social network).
# ============================================================

# ---- follow-up 1: function ----
# Answer: Bidirectional BFS searches from both ends and
# stops when the frontiers meet; with branching factor b
# and distance d it explores about 2 * b^(d/2) nodes
# instead of b^d.
def bidirectional(adj, s, t):
    # a huge graph: search from both ends, expand the
```

```
    # smaller
    # side, stop when the searches meet
    if s == t:
        return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) > len(front_t):
            front_s, front_t = front_t, front_s
            dist_s, dist_t = dist_t, dist_s
        nxt = []
        for u in front_s:
            for v in adj.get(u, ()):
                if v in dist_t:
                    return dist_s[u] + 1 + dist_t[v]
                if v not in dist_s:
                    dist_s[v] = dist_s[u] + 1
                    nxt.append(v)
        front_s = nxt
    return None

# ---- follow-up 1: tests ----
def test_follow_up_1():
    both_ways = {"a": ["b", "c"], "b": ["a", "d"],
                 "c": ["a", "d"], "d": ["b", "c", "e"],
                 "e": ["d"]}
    assert bidirectional(both_ways, "a", "e") == 3
    assert bidirectional(both_ways, "a", "a") == 0


# ============================================================
# FOLLOW-UP 2: Return every shortest path, or count them.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep, for each node, its distance and a list of
# all parents at that distance; counting is a DP over the
# BFS layers, and listing the paths is a DFS over the
# parent lists.
def all_shortest_paths(adj, s, t):
    # every shortest path: remember ALL parents at the
```

```
    # best level
    dist, queue = {s: 0}, deque([s])
    parents = defaultdict(list)
    while queue:
```

```
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
            if dist[v] == dist[u] + 1:
                parents[v].append(u)
    if t not in dist:
        return []
    paths = []

    def build(node, tail):
        if node == s:
            paths.append([s] + tail)
            return
        for p in parents[node]:
            build(p, [node] + tail)

    build(t, [])
    return paths

# ---- follow-up 2: tests ----
def test_follow_up_2():
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"],
           "d": ["e"], "e": []}
    paths = all_shortest_paths(adj, "a", "e")
    assert sorted(paths) == [["a", "b", "d", "e"],
                             ["a", "c", "d", "e"]]
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}


# ======================================================
# FOLLOW-UP 3: Relationships have different strengths
# ======================================================

# ---- follow-up 3: function ----
# Answer: BFS is no longer right: use Dijkstra with a heap
# for non-negative weights. Keep the same parent map to
# rebuild the path.
def weighted_path(graph, s, t):
    # strengths differ: Dijkstra, same parent map
    dist, parent = {s: 0}, {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
```

```
        if d > dist[u]:
            continue
        for v, w in graph.get(u, ()):
            if d + w < dist.get(v, float("inf")):
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    if t not in dist:
        return None
    path = [t]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return dist[t], path[::-1]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}
    cheapest = weighted_path(graph, "a", "c")
    assert cheapest == (2, ["a", "b", "c"])
    assert weighted_path(graph, "c", "a") is None


# ======================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(relationships: list[list[int]], start: int, t
arget: int) -> int:
```

Sample call

```
solution([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')
```

Test cases

Input	Expected
([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')	3
([[1, 2], [2, 5], [1, 3], [3, 4], [4, 5]], 1, 5)	2

ALSO REPORTED · 1 more

Degrees of Friendship | Exponent

https://www.aced.io/questions/3628/degrees-of-friendship

STUDY PATH: question 6 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 6 of 9

Template fit: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance | asked 375 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 6 of 288

Farthest City for a New Data Center

ASKED 2025-10-10 | 375 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 119/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix / LC 1162 As Far from Land as Possible (related: multi-source BFS)

Frequency: Single report | Last asked: 2025-10-10 | Roles: SWE Intern

QUESTION

Cities are nodes 0..num_cities-1 of an undirected, unweighted graph; edges[i] = [u, v] is a direct connection. You are given a non-empty list dcs of cities that already host a data center. You want to build a new data center in the city that is as far as possible from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized. Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Function signature

def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:
    ...

Follow-up (as reported): Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

EXAMPLES

Example 1	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]`
Output	`4`
Why	`Distances from city 0 are 0,1,2,3,4; city 4 is farthest.`

Example 2	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]`
Output	`2`
Why	`Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest at distance 2.`

CONSTRAINTS

QUESTION

- 1 <= num_cities <= 10**5 , 0 <= len(edges) <= 2*10**5 .
- 1 <= len(dcs) <= num_cities , dcs are distinct valid city indices.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from collections import deque
def brute_farthest(n, edges, dcs):
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    best = [float("inf")] * n
    for src in dcs:
        dist = {src: 0}
        queue = deque([src])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        for city, d in dist.items():
            best[city] = min(best[city], d)
    return best.index(max(best))
```

TIME	SPACE
O(D * (V + E))	O(V)

WHY IT IS SLOW

Each data center repeats a full search over the graph.

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The roads have lengths. How does it change?

Replace the BFS queue with a min-heap (Dijkstra) seeded with every data center at distance 0. The first time a city is popped its distance is final. O((V + E) log V).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Farthest City for a New Data Center
#
# Cities are nodes 0..num_cities-1 of an undirected,
```

```
# unweighted graph; edges[i] = [u, v] is a direct
# connection. You are given a non-empty list dcs of cities
# that already host a data center. You want to build a new
# data center in the city that is as far as possible from
# the nearest existing data center — that is, the city
# whose shortest-path distance (in number of edges) to its
# closest data center is maximized. Return that city's
# index. Break ties by choosing the smallest index. It is
# guaranteed every city can reach at least one data
# center. Function signature: def
# farthest_city(num_cities: int, edges: list[list[int]],
# dcs: list[int]) -> int: ... Follow-up (as reported): Run
# a single BFS seeded from all data centers at once
# (distance 0) instead of a BFS per data center.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The roads have lengths. How does it change?
#
# INTERVIEW NOTES
# The distance of a city to its nearest data center is the
# shortest path from the closest source, so start one BFS
# with every data center in the queue at distance 0. The
# first time a city is reached is its nearest-DC distance;
# the answer is the city with the largest such distance,
# the smallest index on ties.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): One multi-source BFS
# Idea: Seed the queue with every data center at distance
# 0; the first visit is the nearest one.
def farthest_city(num_cities, edges, dcs):
    adj = [[] for _ in range(num_cities)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    dist = [-1] * num_cities
    queue = deque(dcs)
    for d in dcs:
        dist[d] = 0
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if dist[v] < 0:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist.index(max(dist))

# ---- main question: tests ----
def test_main_question():
    random.seed(58)
    path = [[0, 1], [1, 2], [2, 3], [3, 4]]
    assert farthest_city(5, path, [0]) == 4
    assert farthest_city(5, path, [0, 4]) == 2
```

```
# ============================================================
# FOLLOW-UP 1: The roads have lengths. How does it change?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Replace the BFS queue with a min-heap (Dijkstra)
# seeded with every data center at distance 0. The first
# time a city is popped its distance is final. O((V + E)
# log V).
def farthest_city_weighted(num_cities, roads, dcs):
    adj = [[] for _ in range(num_cities)]
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dist = [float("inf")] * num_cities
    heap = [(0, d) for d in dcs]
```

```
    for d in dcs:
        dist[d] = 0
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return dist.index(max(dist))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(400):
        n = random.randint(2, 8)
        edges = [
            [i, random.randrange(i)]
            for i in range(1, n)
        ]
        edges += [
            [random.randrange(n), random.randrange(n)]
            for _ in range(random.randint(0, 3))
        ]
        dcs = random.sample(
            range(n), random.randint(1, min(3, n))
        )
        unit = [[u, v, 1] for u, v in edges]
        assert farthest_city_weighted(
            n, unit, dcs
        ) == farthest_city(n, edges, dcs)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 7 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 7 of 9

Template fit: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1 | asked 283 days ago

MEDIUM | IN APP LC 542 | PracHub | Technical Screen | 7 of 288

Compute distance to nearest taxi in grid

POSTED 2026-01-10 | 283 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 117/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix (inferred from overview)

Posted on PracHub: 2026-01-10 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/compute-distance-to-nearest-taxi-in-grid

QUESTION

A grid has 'T' for a taxi, '#' for a blocked cell and '.' for an open cell. For every cell return the number of steps (up, down, left or right, never through '#') to the nearest taxi: 0 for a taxi, and -1 for a blocked cell or a cell no taxi can reach.

Function signature

def taxi_distances(grid: list) -> list

EXAMPLES

Example 1	
Input	`['T.#', '...', '#.T']`
Output	`[[0, 1, -1], [1, 2, 1], [-1, 1, 0]]`
Why	The two taxis are in opposite corners; the middle cell is 2 steps from both; the blocked cells show -1.

Example 2	
Input	`['..', '..']`
Output	`[[-1, -1], [-1, -1]]`
Why	No taxi at all: every open cell is unreachable.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def nearest_taxi(grid, start):
    queue, seen = deque([(start, 0)]), {start}
    while queue:
        cell, d = queue.popleft()
        if is_taxi(grid, cell):
            return d
        for nxt in neighbours(grid, cell):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, d + 1))
    return -1
answer = {}
for c in cells:
    answer[c] = nearest_taxi(grid, c)
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A separate BFS per cell, O((m * n)^2). One BFS that starts from all taxis at once reaches every cell at its nearest-taxi distance, O(m * n).

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

FOLLOW-UP 2 Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

FOLLOW-UP 3 Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute distance to nearest taxi in grid
#
# A grid has 'T' for a taxi, '#' for a blocked cell and
# '.' for an open cell. For every cell return the number
# of steps (up, down, left or right, never through '#') to
# the nearest taxi: 0 for a taxi, and -1 for a blocked
# cell or a cell no taxi can reach.
#
# Signature:
#   def taxi_distances(grid: list) -> list
#
# Examples:
# ['T.#', '...', '#.T']  ->  [[0, 1, -1], [1, 2, 1], [-1,
#     1, 0]]
# ['..', '..']  ->  [[-1, -1], [-1, -1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. A new taxi appears.
# 2. Taxis move or disappear.
# 3. Return which taxi is nearest, not only the distance.
#
# INTERVIEW NOTES
# Asking for the distance from every cell to its nearest
# source is one BFS started from all sources at once, not
# one BFS per cell. Put all taxis in the queue at distance
# 0; the first time the search reaches a cell is its
# shortest distance, because BFS visits cells in order of
# distance.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```
def spread(grid, dist, queue, better=False):
    # grow outward; with better=True a cell may be updated
    # when a
    # new, closer source reaches it (distances only go
    # down)
    R, C = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if not (0 <= a < R and 0 <= b < C):
                continue
            if grid[a][b] == "#":
                continue
            new = dist[r][c] + 1
            unseen = dist[a][b] == -1
            if unseen or (better and new < dist[a][b]):
                dist[a][b] = new
                queue.append((a, b))

# ---- main question: main function ----
# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    # 'T' taxi, '.' road, '#' wall. One BFS that starts
    # from
    # every taxi at once gives each cell its nearest taxi.
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    queue = deque()
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist

# ---- main question: tests ----
def test_main_question():
```

```
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    # the second taxi at the bottom right is closer to (0,
    # 3)
    assert d[0] == [0, 1, 2, 2]
    assert d[2] == [2, 2, 1, 0]
    assert d[1][1] == -1


# ============================================================
# FOLLOW-UP 1: A new taxi appears.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Start a BFS from the new taxi but only continue
# into cells where you improve the stored distance. The
# distances can only decrease, so the work is bounded by
# the cells whose answer actually changes.
def add_taxi(grid, dist, r, c):
    # a new taxi: only cells it beats are touched
    dist[r][c] = 0
    spread(grid, dist, deque([(r, c)]), better=True)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    # a new taxi in the middle of the top row
    add_taxi(grid, d, 0, 2)
    assert d[0] == [0, 1, 0, 1]
    assert d[2][0] == 2
    # unreachable cells stay -1
    walled = ["T#.", "##."]
    assert taxi_distances(walled)[0][2] == -1


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
```

```
-UP 2.

# ============================================================
# FOLLOW-UP 3: Return which taxi is nearest, not only the
# ============================================================
```

```
# ---- follow-up 3: function ----
# Answer: Store a source id with each cell in the BFS: a
# cell copies the id of the cell that first reached it.
# Ties go to whichever reached it first, so say how you
# want ties broken.
def nearest_taxi_ids(grid):
    # which taxi is nearest? each cell copies the id of
    # the
    # cell that reached it first (ties: the first to
    # arrive)
    R, C = len(grid), len(grid[0])
    owner = [[None] * C for _ in range(R)]
    queue, taxi_id = deque(), 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "T":
                owner[r][c] = taxi_id
                taxi_id += 1
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            inside = 0 <= a < R and 0 <= b < C
            if not inside or grid[a][b] == "#":
                continue
            if owner[a][b] is None:
                owner[a][b] = owner[r][c]
                queue.append((a, b))
    return owner

# ---- follow-up 3: tests ----
def test_follow_up_3():
    grid = ["T...", ".#..", "...T"]
    ids = nearest_taxi_ids(grid)
    assert ids[0][0] == 0 and ids[2][3] == 1
    assert ids[1][1] is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0,0,0],[0,1,0],[0,0,0]])
```

Test cases

Input	Expected
`([[0,0,0],[0,1,0],[0,0,0]],)`	`[[2,1,2],[1,0,1],[2,1,2]]`
`([[1,0,0,0],[0,0,0,1],[0,0,0,0]],)`	`[[0,1,2,1],[1,2,1,0],[2,3,2,1]]`

STUDY PATH: question 8 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 8 of 9

Template fit: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds) | asked 128 days ago

HARD | PracHub | Technical Screen | 8 of 288

Infection Spread on a Grid (Cellular Automaton)

POSTED 2026-06-14 | 128 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 75/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #994 Rotting Oranges, LeetCode #289 Game of Life

LeetCode match: LC 994 Rotting Oranges / LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-06-14 (updated 2026-06-24)

Original: https://prachub.com/coding-questions/infection-spread-on-a-grid-cellular-automaton

QUESTION

A grid holds '.' for a healthy cell, 'I' for an infected cell and any other character (such as '#') for a barrier that is never infected. Every step, all infected cells infect their healthy neighbours (up, down, left, right) at the same time. infection_times(grid) returns the step at which each infected cell was infected (0 for the cells infected at the start); spread_steps(grid) returns the number of steps until nothing new gets infected.

Function signature

def infection_times(grid: list) -> dict

def spread_steps(grid: list) -> int

EXAMPLES

Example 1

Input	`['I..', '...', '..#']`
Output	`3`
Why	The infection travels from the corner: (0,1) and (1,0) at step 1, then three cells at step 2, and so on. The barrier at the bottom right is never infected.

Example 2

Input	`['I...']`
Output	`{(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3): 3}`
Why	A one-row grid: one cell per step.

QUESTION

Example 3

Input	`['I#.', 'I#.']`
Output	`0`
Why	Both cells next to the barrier are already infected and the barrier blocks the right column, so nothing can spread: 0 steps.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    steps = 0
    while True:
        nxt = copy(grid)
        for r, c in all_cells(grid):
            if grid[r][c] == HEALTHY:
                if sick_neighbour(grid, r, c):
                    nxt[r][c] = INFECTED
        if nxt == grid:
            return steps
        grid = nxt
        steps += 1
```

TIME	SPACE
O(steps * m * n)	O(m * n)

WHY IT IS SLOW

Every step scans the whole grid even when only a thin edge is changing, O(steps * m * n). Keeping a frontier of newly infected cells (BFS) touches each cell once, O(m * n).

OPTIMAL SOLUTION · Frontier BFS (only recently infected cells spread)

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

FOLLOW-UP 2 Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

FOLLOW-UP 3 Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Infection Spread on a Grid (Cellular
# Automaton)
#
# A grid holds '.' for a healthy cell, 'I' for an infected
# cell and any other character (such as '#') for a barrier
# that is never infected. Every step, all infected cells
# infect their healthy neighbours (up, down, left, right)
# at the same time. infection_times(grid) returns the step
# at which each infected cell was infected (0 for the
# cells infected at the start); spread_steps(grid) returns
# the number of steps until nothing new gets infected.
#
# Signature:
#   def infection_times(grid: list) -> dict
#   def spread_steps(grid: list) -> int
#
# Examples:
# ['I..', '...', '..#']  ->  3
# ['I...']  ->  {(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3):
#     3}
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a recovery delay: an infected cell recovers after
# d steps and cannot spread afterwards.
# 2. Why can you stop early?
# 3. Make the spread probabilistic.
#
# INTERVIEW NOTES
# A synchronous cellular automaton is a BFS in disguise:
# the step number is the BFS level. Only cells next to
# cells that just changed can change next, so keep a
# frontier instead of rescanning the grid. Always compute
# the next state from the old state (not in place) so all
# cells update together.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# ---- main question: main function ----
# TIME each cell gets infected
def infection_times(grid):
    # step at which each cell gets infected (a BFS level);
    # 'I' = infected at time 0, '.' = healthy, '#' =
    # blocked
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = [(r, c) for r in range(rows)
                for c in range(cols) if grid[r][c] == "I"]
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in STEPS:
                a, b = r + dr, c + dc
                inside = 0 <= a < rows and 0 <= b < cols
                # every healthy neighbour is infected next
                # step
                if inside and grid[a][b] == "." \
                        and (a, b) not in when:
                    when[(a, b)] = step
                    nxt.append((a, b))
        frontier = nxt
    return when


# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier.
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
    return max(when.values(), default=0)
```

```


# ---- main question: tests ----
def test_main_question():
    g = ["I..", "...", "..."]
    assert spread_steps(g) == 4


# ============================================================
# FOLLOW-UP 1: Add a recovery delay: an infected cell
# ============================================================

# ---- follow-up 1: function ----
# Answer: A cell infects every healthy neighbor on the
# very next step, so recovery does not change how long
# spreading takes; it only changes what the grid looks
# like at time t.
def grid_at(grid, t, recovery=None):
    # the grid after t steps; with a recovery delay d an
    # infected cell turns into 'R' (it spread already, so
    # the
    # timing above does not change)
    when = infection_times(grid)
```

```
    out = []
    for r, row in enumerate(grid):
        line = ""
        for c, ch in enumerate(row):
            start = when.get((r, c))
            if start is None or start > t:
                line += ch
            elif recovery and t >= start + recovery:
                line += "R"
            else:
                line += "I"
        out.append(line)
    return out

# ---- follow-up 1: tests ----
def test_follow_up_1():
    g = ["I..", "...", "..."]
    assert grid_at(g, 1) == ["II.", "I..", "..."]
    # with recovery after 1 step the source recovers first
    rec = grid_at(g, 1, recovery=1)
    assert rec == ["RI.", "I..", "..."]
    # a wall splits the grid: the far cell is never
    # infected
```

```
    wall = ["I#.", "I#."]
    assert spread_steps(wall) == 0
    assert grid_at(wall, 9)[0] == "I#."
    assert spread_steps(["..."]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> list[str]:
```

Sample call

```
solution(['HHH', 'HIH', 'HHH'])
```

Test cases

Input	Expected
`(['HHH', 'HIH', 'HHH'],)`	`['HIH', 'III', 'HIH']`
`(['HHH'],)`	`['HHH']`

STUDY PATH: question 9 of 288 | topic 1 of 43: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance | 9 of 9

Template fit: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell | asked 393 days ago | frequency Single report

HARD | IN APP LC 317 | ColorStack | technical interview | 9 of 288

Shortest Distance From All Buildings

ASKED 2025-09-22 | 393 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 202/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

Optimal answer from: your app's interview approach (LeetCode #317)

LeetCode match: LC 317 Shortest Distance from All Buildings

Frequency: Single report | Last asked: 2025-09-22 | Roles: SWE Intern

QUESTION

You are given a 2D grid where each cell is:
0 = empty land (a school can be built here), 1 = a house, 2 = a tree (an obstacle that cannot be entered or crossed). You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from every house. Return the coordinates (row, col) of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return (-1, -1) .

Function signature

```
def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...
```

Follow-up (as reported): (Reported) Minimize the average distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

EXAMPLES

Example 1

Input	`grid = [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]`
Output	`(1, 2)`
Why	`Cell (1,2) reaches all three houses with total distance 3 + 3 + 1 = 7, the minimum over all reachable empty cells.`

QUESTION

Example 2

Input	`grid = [[1, 0, 1]]`
Output	`(0, 1)`
Why	`The only empty cell is distance 1 from each house; total 2.`

Example 3

Input	`grid = [[1, 2, 1]]`
Output	`(-1, -1)`
Why	`There is no empty cell, so no valid location exists.`

CONSTRAINTS

- 1 <= rows, cols <= 50
- Each cell is 0 , 1 , or 2 .
- There is at least one house.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
from collections import deque
def brute_school(grid):
    rows, cols = len(grid), len(grid[0])
    houses = sum(
        v == 1 for row in grid for v in row
    )
    best = (float("inf"), -1, -1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 0:
                continue
            dist, queue, total, found = (
                {(r, c): 0},
                deque([(r, c)]),
                0,
                0,
            )
            while queue:
                x, y = queue.popleft()
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < rows
                        and 0 <= ny < cols
                        and (nx, ny)
                        not in dist
                        and grid[nx][ny] != 2
                    ):
                        dist[(nx, ny)] = (
                            dist[(x, y)] + 1
                        )
                        if grid[nx][ny] == 1:
                            total, found = (
                                total
                                + dist[
                                    (nx, ny)
                                ],
                                found + 1,
                            )
                        else:
                            queue.append(
                                (nx, ny)
                            )
            if (
                found == houses
                and (total, r, c) < best
            ):
                best = (total, r, c)
    return (
        (best[1], best[2])
        if best[1] >= 0
        else
            (-1, -1)
    )
```

TIME	SPACE
O(cells * R * C)	O(R * C)

WHY IT IS SLOW

Each empty cell runs its own full search for the houses.

OPTIMAL SOLUTION · One BFS per house

TIME	SPACE
O(houses * R * C)	O(R * C)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Minimize the average distance instead of the total.

With a fixed number of houses the average is total / houses, so the best cell is the same; compute it once at the end to report the average.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Distance From All Buildings
#
# You are given a 2D grid where each cell is: 0 = empty
# land (a school can be built here), 1 = a house, 2 = a
# tree (an obstacle that cannot be entered or crossed).
# You want to build one school on an empty cell so that
# the total travel distance from every house to the school
# is minimized. Distance is measured by 4-directional
# steps (up/down/left/right); trees block movement. A
# valid school location must be reachable from every
# house. Return the coordinates (row, col) of the empty
# cell with the minimum total distance to all houses. If
# several cells tie on total distance, return the one that
# comes first in row-major order (smallest row, then
# smallest column). If no empty cell can reach all houses,
# return (-1, -1) . Function signature: def
# shortest_distance_from_all_buildings(grid:
# list[list[int]]) -> tuple[int, int]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Minimize the average distance instead of the total.
#
# INTERVIEW NOTES
# Search from the houses, not from every empty cell: one
# BFS per house adds that house's distance to every empty
# cell it reaches and counts how many houses reached the
# cell. A cell is valid only when all houses reached it.
# Choose the smallest total, ties in row-major order.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
def school_totals(grid):
    rows, cols = len(grid), len(grid[0])
    total = [[0] * cols for _ in range(rows)]
    reached = [[0] * cols for _ in range(rows)]
    houses = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 1:
                continue
            houses += 1
            seen, queue = {(r, c)}, deque([(r, c, 0)])
            while queue:
                x, y, d = queue.popleft()
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < rows
                        and 0 <= ny < cols
                        and (nx, ny) not in seen
                        and grid[nx][ny] == 0
                    ):
                        seen.add((nx, ny))
                        total[nx][ny] += d + 1
                        reached[nx][ny] += 1
                        queue.append((nx, ny, d + 1))
    return total, reached, houses


# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best, spot = float("inf"), (-1, -1)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if (
                grid[r][c] == 0
```

```
                and reached[r][c] == houses
                and total[r][c] < best
            ):
                best, spot = total[r][c], (r, c)
    return spot

# ---- main question: tests ----
def test_main_question():
    random.seed(21)
    assert shortest_distance_from_all_buildings(
        [
            [1, 0, 2, 0, 1],
            [0, 0, 0, 0, 0],
            [0, 0, 1, 0, 0],
        ]
    ) == (1, 2)
    assert shortest_distance_from_all_buildings(
        [[1, 0, 1]]
    ) == (0, 1)
    assert shortest_distance_from_all_buildings(
        [[1, 2, 1]]
    ) == (-1, -1)
```

```
# ============================================================
# FOLLOW-UP 1: Minimize the average distance instead of
# ============================================================

# ---- follow-up 1: function ----
# Answer: With a fixed number of houses the average is
# total / houses, so the best cell is the same; compute it
# once at the end to report the average.
def best_school_average(grid):
    spot = shortest_distance_from_all_buildings(grid)
    if spot == (-1, -1):
        return spot, None
    total, _, houses = school_totals(grid)
    return spot, total[spot[0]][spot[1]] / houses

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_school_average([[1, 0, 1]]) == (
        (0, 1),
        1.0,
    )
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 2 OF 43 |

Sweep over events in time order

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6	3
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sweep over events in time order | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

M	10. Meeting Rooms / Interval Scheduling Variants	1Point3Acres	2026-09-29
M	11. Train / Flight Schedule Reachability	1Point3Acres	2026-10-08
M	12. Solve Rooms and Top-K Streams	PracHub	2026-06-05
M	13. Compute servers needed for daily recurring jobs	PracHub	2025-11-05
M	14. Gate Entry / Exit Time Resolution with Tie-Break Rules	1Point3Acres	2026-05-04
M	15. Meeting Rooms III Variant	HackTheRounds	no date
H	16. Count Overlapping Rectangle Updates on a Grid	PracHub	2026-07-10
H	17. Compute city skyline outline	PracHub	2025-12-08
H	18. Compute Turnstile Crossing Times	PracHub	2026-05-04

CODE TEMPLATE | GROUP 2 OF 43 | 9 QUESTIONS, ONE PATTERN

Sweep over events in time order

The one idea these questions share

Turn every item into events (a start adds one, an end removes one, an arrival joins a queue), sort the events by time, and walk them keeping a small state: a counter, a queue, a heap or a running total. What you record at each event is the only thing that changes.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def max_overlap(intervals):
    events = sorted([(s, 1) for s, e in intervals] + [(e, -1) for s, e in intervals])
    cur = best = 0
    for t, delta in events:                  # CHANGE 1: what an event does to your state
        cur += delta
        best = max(best, cur)                # CHANGE 2: what you record
    return best                              # ends sort before starts at the same time (-1 < 1)
```

2. Helper functions that usually get written next to it

```
import heapq
from collections import deque


def two_queue_clock(arrivals):               # who is served when two queues share one server
    order = sorted(range(len(arrivals)), key=lambda i: arrivals[i])
    queue, t, i, served = deque(), 0, 0, []
    while i < len(order) or queue:
        if not queue:
            t = max(t, arrivals[order[i]])
        while i < len(order) and arrivals[order[i]] <= t:
            queue.append(order[i]); i += 1
        served.append((queue.popleft(), t))
        t += 1
    return served
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
10	Meeting Rooms / Interval Scheduling Variants	the template as written: the largest running count is the number of rooms
11	Train / Flight Schedule Reachability	sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place
12	Solve Rooms and Top-K Streams	sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap
13	Compute servers needed for daily recurring jobs	merge each job's own intervals first so one job counts once, then sweep all of them together
14	Gate Entry / Exit Time Resolution with Tie-Break Rules	the same two-queue simulation with the tie-break rules from the statement
15	Meeting Rooms III Variant	split meetings that cross midnight into two pieces first, then the same sweep
16	Count Overlapping Rectangle Updates on a Grid	the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes
17	Compute city skyline outline	events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

#	Question	What you change in the template
18	Compute Turnstile Crossing Times	sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 10 of 288 | topic 2 of 43: Sweep over events in time order | 1 of 9

Template fit: the template as written: the largest running count is the number of rooms | asked 21 days ago | frequency Very high

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 10 of 288

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 58** | **30 DAYS 0** | **PRIORITY 51/288**

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

QUESTION

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

QUESTION

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling). Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example:
    [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40],
    D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap:
    intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Meeting Rooms / Interval Scheduling
# Variants
#
# LC 253 (Meeting Rooms II) appears in many forms across
# phone screen and onsite coding rounds — minimum number
# of rooms, room assignment, cyclic time, daily server
# jobs, gate entry/exit. Common follow-ups push toward
# sweep line, min-heap, or distributed processing. The
# base task in every variant: given a list of intervals
# (start, end) — or pickup/return pairs, meeting time
# pairs, server job runs — answer one or more of: Min
# rooms / cars / servers needed to honor all intervals
# concurrently (the canonical min-meeting-rooms problem).
# Assignment: which interval goes to which room/car, so
# the caller can use the rental record. Single-interval
# overlap test between two intervals. Variant: parking-lot
# capacity Arrival and departure intervals represent cars
# occupying a parking lot. Compute the peak simultaneous
# occupancy using the same interval-overlap model.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Meetings wrap around midnight (cyclic time).
# 2. The input is huge and spread over many machines.
#
# INTERVIEW NOTES
# The number of rooms needed is the largest number of
# intervals overlapping at any moment. Turn each interval
# into +1 at its start and -1 at its end, sort the events
# (ends before starts at the same time) and keep a running
# count. To say which room each meeting gets, keep a min-
# heap of (end time, room) and reuse the room that frees
# up first.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
```

```python
# ---- main question: main function ----
# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best


# VARIANT: which room for each meeting
def assign_rooms(intervals):
    # which room does each meeting get?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        # reuse the room that frees first, if it is free
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# VARIANT: on-call consolidation
def oncall_segments(rotations):
```

```python
    # rotations: (name, start, end), half-open intervals
    delta = {}
    for name, s, e in rotations:
        delta.setdefault(s, []).append((name, 1))
        delta.setdefault(e, []).append((name, -1))
    active, out, times = Counter(), [], sorted(delta)
    for t, nxt in zip(times, times[1:]):
        # apply every change that happens at time t
        for name, d in delta[t]:
            active[name] += d
        who = frozenset(n for n in active if active[n])
        # skip gaps; merge with the previous equal segment
        if not who:
            continue
        if out and out[-1][1] == t and out[-1][2] == who:
            out[-1] = (out[-1][0], nxt, who)
        else:
            out.append((t, nxt, who))
    return out

# ---- main question: tests ----
def test_main_question():
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    # touching meetings share a room
    assert min_rooms([(5, 8), (8, 9)]) == 1
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
```

```python
# ============================================================
# FOLLOW-UP 1: Meetings wrap around midnight (cyclic
# ============================================================

# ---- follow-up 1: function ----
# Answer: Split every wrapped meeting [s, e) with s >= e
# into [s, 24h) and [0, e) and run the same sweep: the
# overlap at any clock time is unchanged. A meeting that
# is exactly 24 hours long covers the whole day.
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for s, e in intervals:
        if s < e:
            parts.append((s, e))
```

```python
        else:
            # wraps past midnight: split at the boundary
            parts += [(s, day), (0, e)]
    return min_rooms(parts)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3
    rot = [("A", 1, 5), ("B", 3, 8)]
    assert oncall_segments(rot) == [
        (1, 3, frozenset("A")),
        (3, 5, frozenset("AB")),
        (5, 8, frozenset("B"))]
    # a gap with nobody on call is left out
    gap = [("A", 1, 2), ("B", 4, 5)]
    assert len(oncall_segments(gap)) == 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY PATH: question 11 of 288 | topic 2 of 43: Sweep over events in time order | 2 of 9

Template fit: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place | asked 12 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 11 of 288

Train / Flight Schedule Reachability

ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 212/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late
    for the Y->B connection, but the second one makes
    it.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

TIME	SPACE
O(legs^2)	O(legs)

WHY IT IS SLOW

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

TIME	SPACE
O(L log L)	O(S)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Train / Flight Schedule Reachability
#
# Given trains (or flights) as (from, to, departure,
# arrival), decide whether you can travel from a source to
# a destination, boarding a later leg only if it departs
# after you arrive. Graph reachability with time state,
# solved with BFS / DFS. Input: a schedule of legs, each
# with a departure station, an arrival station, a
# departure time, and an arrival time; plus a source and a
# destination. Output: whether the destination is
# reachable from the source, where a leg can only be taken
# if it departs after you arrive at its departure station.
# The same prompt appears as a flight schedule between
# airports in new-grad R1. Examples: trains = [ ("A", "Y",
# dep=1, arr=20), ("A", "Y", dep=2, arr=5), ("Y", "B",
# dep=6, arr=9) ] source = "A", destination = "B" Output:
# true — the first A->Y train arrives too late for the Y->
B
# connection, but the second one makes it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the earliest arrival time at the destination.
# 2. Print the route, not just yes or no.
# 3. Many queries on the same schedule: what changes in
# the graph representation?
#
# INTERVIEW NOTES
# Plain station-to-station reachability is wrong because a
# leg is only usable if you are already at its departure
# station before it leaves. Time is part of the state, so
# ask what you need to know about each station: the
# earliest time you can be there. Process the legs in
# order of departure time; a leg is usable if the earliest
# arrival at its origin is not later than its departure,
# and then it can improve the earliest arrival at its
# destination. One sort and one pass answer reachability,
# and the same table gives the earliest arrival time.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
INF = float("inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)

# ---- main question: tests ----
def test_main_question():
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert reachable(legs, "A", "B")
```

```
    # without the early A to Y train the connection is
    # lost
    assert not reachable(legs[:1] + legs[2:], "A", "B")
    assert earliest_arrivals(legs, "A")["B"] == 9


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Print the route, not just yes or no.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remember the leg that last improved each
# station's earliest arrival, then walk back from the
# destination to the source and reverse.
def itinerary(legs, source, dest, strict=True):
    """Follow-up: the route. Remember the leg that last
    improved each station, then walk back from the end."""
    earliest, came = {source: -INF}, {}
    for leg in sorted(legs, key=lambda x: x[2]):
        frm, to, dep, arr = leg
        have = earliest.get(frm, INF)
        usable = have < dep or (not strict
                                and have == dep)
        if usable and arr < earliest.get(to, INF):
            earliest[to], came[to] = arr, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert itinerary(legs, "A", "B") == [legs[1], legs[2]]
    assert itinerary(legs, "B", "A") is None
```

```
    assert reachable(legs, "A", "A")
    touch = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touch, "A", "C")
    assert reachable(touch, "A", "C", strict=False)
    for _ in range(300):
        many = []
        for _ in range(random.randint(0, 8)):
            dep = random.randint(0, 9)
            many.append((random.choice("ABCD"),
                         random.choice("ABCD"), dep,
                         dep + random.randint(1, 4)))
        src = random.choice("ABCD")
        dst = random.choice("ABCD")


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

STUDY PATH: question 12 of 288 | topic 2 of 43: Sweep over events in time order | 3 of 9

Template fit: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap | asked 137 days ago

MEDIUM | PracHub | Technical Screen | 12 of 288

★ TOP 75: rank 69 in the Medium and Hard priority list

★ Solve Rooms and Top-K Streams

POSTED 2026-06-05 | 137 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58** | **30 DAYS 0** | **PRIORITY 47/288**

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II, LeetCode #703 Kth Largest Element in a Stream

LeetCode match: LC 253 Meeting Rooms II + LC 703 Kth Largest Element in a Stream (inferred from overview)

Posted on PracHub: 2026-06-05 (updated 2026-06-14)

Original:
https://prachub.com/coding-questions/solve-rooms-and-top-k-streams

QUESTION

Two small tasks. (1) min_rooms(intervals): the minimum number of rooms needed to hold all meetings; a meeting may start exactly when another ends. (2) TopKStream(k): numbers arrive one at a time with add(x); top() returns the k largest numbers seen so far, largest first.

Function signature

def min_rooms(intervals: list) -> int
class TopKStream:
  def __init__(self, k: int)
  def add(self, x: int) -> None
  def top(self) -> list

EXAMPLES

Example 1

Input	`[(0, 30), (5, 10), (15, 20)]`
Output	`2`
Why	`(5, 10) and (15, 20) both overlap the long meeting (0, 30): 2 rooms.`

Example 2

Input	`[(7, 10), (2, 4)]`
Output	`1`
Why	`The second meeting ends before the first starts: one room is enough.`

QUESTION

Example 3

Input	`s = TopKStream(3)`
	`s.add(5)`
	`s.add(1)`
	`s.add(9)`
	`s.add(7)`
	`s.top()`

Output	`s.top() -> [9, 7, 5]`
Why	`Of 5, 1, 9, 7 the three largest are 9, 7 and 5.`

BRUTE FORCE · Rooms: count the overlap at every time unit

BRUTE FORCE · PSEUDO-CODE

```
def rooms(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, not only on n: O(range * n). Sorted starts and ends with two pointers (or a heap of end times) are O(n log n).

BRUTE FORCE · Stream: sort everything on every query

BRUTE FORCE · PSEUDO-CODE

```
values = []
def add(x):
    values.append(x)
    values.sort(reverse=True)
    return values[K - 1]
```

TIME	SPACE
O(n log n) per add	O(n)

WHY IT IS SLOW

Every add re-sorts the whole history, O(n log n). A min-heap of size K keeps the K-th largest at its root in O(log K) per add.

OPTIMAL SOLUTION · Rooms: sorted starts and ends with two pointers

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Stream: min-heap of size K

TIME	SPACE
O(log k) per add	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

FOLLOW-UP 2 Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. **Cost:** a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

FOLLOW-UP 3 Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve Rooms and Top-K Streams
#
# Two small tasks. (1) min_rooms(intervals): the minimum
# number of rooms needed to hold all meetings; a meeting
# may start exactly when another ends. (2) TopKStream(k):
# numbers arrive one at a time with add(x); top() returns
# the k largest numbers seen so far, largest first.
#
```

```
# Signature:
#   def min_rooms(intervals: list) -> int
#   class TopKStream:
#       def __init__(self, k: int)
#       def add(self, x: int) -> None
#       def top(self) -> list
#
# Examples:
# [(0, 30), (5, 10), (15, 20)]  ->  2
# [(7, 10), (2, 4)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rooms: return which room each meeting uses.
# 2. Stream: K changes over time, or you need the median.
# 3. Rooms: meetings on several floors with capacities.
#
# INTERVIEW NOTES
# Rooms: the number of rooms needed is the largest number
# of meetings happening at the same time, so sort the
# starts and the ends separately and sweep: a start that
# comes before the earliest end needs a new room. Stream:
# only the K largest values can matter, so keep exactly
# those in a min-heap; the root is the K-th largest.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Rooms: sorted starts and ends
# with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed.
def min_rooms(intervals):
    # two sorted lists: every start with no room free
    # needs
    # a new room, otherwise it reuses one that just ended
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            # the earliest-ending meeting is over: reuse
            # it
            j += 1
        else:
            rooms += 1
    return rooms


# PART 2: top K values of a stream
class TopKStream:
    """The k largest values seen so far (fixed k)."""

    def __init__(self, k):
        self.k, self.heap = k, []
```

```
    def add(self, x):
        # a min-heap of size k: the root is the k-th
        # largest
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
        return sorted(self.heap, reverse=True)


# VARIANT: median of the stream
```

```
class MedianStream:
    """Follow-up: the median instead of a fixed top-k."""

    def __init__(self):
        # low is a max-heap (negated), high is a min-heap
        self.low, self.high = [], []

    def add(self, x):
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        if len(self.high) > len(self.low):
            back = heapq.heappop(self.high)
            heapq.heappush(self.low, -back)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2

# ---- main question: tests ----
def test_main_question():
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(meetings) == 2
    # a meeting may start the moment another ends
    assert min_rooms([(1, 3), (3, 5)]) == 1
    assert min_rooms([]) == 0


# ============================================================
# FOLLOW-UP 1: Rooms: return which room each meeting uses.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by start and keep a min-heap of (end time,
# room). If the earliest-ending room is free at the next
# start, reuse it, otherwise open a new room. The heap
# size never exceeds the answer.
def assign_rooms(intervals):
    # which room does each meeting use? a heap of (end,
    # room)
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
```

```
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms

# ---- follow-up 1: tests ----
def test_follow_up_1():
    meetings = [(0, 30), (5, 10), (15, 20)]
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = TopKStream(3)
    for x in [5, 1, 9, 7, 3, 8]:
        t.add(x)
    assert t.top() == [9, 8, 7]
    m = MedianStream()
    for x in [5, 2, 8, 1]:
        m.add(x)
    assert m.median() == 3.5


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 30], [5, 10], [15, 20]])
```

Test cases

Input	Expected
([[0, 30], [5, 10], [15, 20]],)	2
([],)	0

STUDY PATH: question 13 of 288 | topic 2 of 43: Sweep over events in time order | 4 of 9

Template fit: merge each job's own intervals first so one job counts once, then sweep all of them together | asked 349 days ago

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 13 of 288

★ TOP 75: rank 58 in the Medium and Hard priority list

★ Compute servers needed for daily recurring jobs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 48/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2025-11-05 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/compute-servers-needed-for-daily-recurring-jobs

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

QUESTION

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.

Constraints

0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240,
    300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.

Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def servers_needed(intervals):
    best = 0
    for t in range(DAY):
        jobs = set()
        for job_id, s, e in intervals:
            if s <= t < e:
                jobs.add(job_id)
        best = max(best, len(jobs))
    return best
```

TIME	SPACE
O(T * m)	O(m)

WHY IT IS SLOW

Depends on the length of the day, O(T * m). Merge each job's own intervals first, then sort start and end events and keep a running count, O(m log m).

OPTIMAL SOLUTION · Merge each job's intervals, then sweep start and end events

TIME	SPACE
O(m log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

FOLLOW-UP 2 Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

FOLLOW-UP 3 Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute servers needed for daily
# recurring jobs
#
# You operate a cluster of identical servers that run
# recurring **daily** jobs. For a single day, you are
# given a list of job execution intervals. Each interval
# has: - job_id (string or integer), - start_time
# (timestamp within that day), - end_time (timestamp
# within that day), with start_time < end_time. Interpret
# each interval as a half-open time range [start_time,
# end_time) during which that job must be running on some
# server **on that day**. Rules and constraints: - Each
# physical server can execute at most **one job at a
# time**. - However, if multiple intervals in the input
# correspond to the **same job_id**, they may overlap in
# time **on the same server** (they are considered parts
# or retries of the same job and do not consume extra
# capacity).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Some rare jobs run for several days without restart
# and get a dedicated server.
# 2. Jobs that cross midnight.
# 3. Return which server each job runs on.
#
# INTERVIEW NOTES
# The number of servers is the largest number of jobs
# running at the same time. First remove the twist:
# overlapping intervals of the same job are really one
# longer interval, so merge the intervals of each job_id.
# Then it is the plain maximum overlap: sort start and end
# events and keep a running count.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import defaultdict



# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def merge_by_job(intervals):
    # intervals of the SAME job may overlap on one server,
    # so
    # merge them first: (job, start, end) -> disjoint
    # blocks
    by_job = defaultdict(list)
    for job, start, end in intervals:
        by_job[job].append((start, end))
    blocks = []
    for job, spans in by_job.items():
```

```
        cur = None
        for s, e in sorted(spans):
            if cur and s <= cur[1]:
                cur[1] = max(cur[1], e)
            else:
                if cur:
                    blocks.append((cur[0], cur[1], job))
                cur = [s, e]
        blocks.append((cur[0], cur[1], job))
    return blocks

# ---- main question: main function ----
# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals.
# FOLLOW-UP 1: Some rare jobs run for several days without
# restart and get a dedicated server.
# Answer: Remove those jobs from the daily intervals and
# add one server per such job: servers = (number of
# dedicated jobs) + (maximum overlap of the remaining
# jobs).
def servers_needed(intervals, dedicated=()):
    # FOLLOW-UP 1: rare multi-day jobs get their own
    # server
    # dedicated jobs (they span several days) get their
    # own
    # server; the rest share servers: the answer is the
    # largest number of blocks running at the same moment
    dedicated = set(dedicated)
```

```
    shared = [i for i in intervals
              if i[0] not in dedicated]
    events = []
    for start, end, _ in merge_by_job(shared):
        events.append((start, 1))
        # [start, end): at equal times an end comes first
        events.append((end, -1))
    run = best = 0
    for _, delta in sorted(events):
        run += delta
        best = max(best, run)
    used = {i[0] for i in intervals if i[0] in dedicated}
    return best + len(used)

# ---- main question: tests ----
def test_main_question():
    jobs = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8),
            ("c", 10, 15)]
    # job a is one block [0, 12); b overlaps it; c starts
    # at 10
    assert servers_needed(jobs) == 2
    # touching half-open intervals share a server
    assert servers_needed([("a", 0, 5), ("b", 5, 9)]) == 1
    assert servers_needed([]) == 0
    # job b runs for days: it gets a dedicated server
    assert servers_needed(jobs, dedicated=["b"]) == 3
    assert servers_needed(jobs, dedicated=["a", "b"]) == 3


# ============================================================
# FOLLOW-UP 1: Some rare jobs run for several days without
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Jobs that cross midnight.
# ============================================================

# ---- follow-up 2: function ----
```

```
# Answer: Truncate at the end of the day for each day's
# schedule (as the statement says).
def split_over_midnight(start, end, day=24 * 60):
    # a daily job that wraps past midnight: two pieces
    if start < end:
        return [(start, end)]
```

```
    return [(start, day), (0, end)]

# ============================================================
# FOLLOW-UP 3: Return which server each job runs on.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Sort merged intervals by start and keep a min-
# heap of (end time, server id): reuse the server that
# frees up first if it is free, otherwise open a new one.
# The number of servers opened equals the maximum overlap.
def assign_servers(intervals):
    # which server runs each block of each job?
    heap, plan, opened = [], [], 0
    blocks = sorted(merge_by_job(intervals))
    for start, end, job in blocks:
        if heap and heap[0][0] <= start:
            _, server = heapq.heappop(heap)
        else:
            server, opened = opened, opened + 1
        plan.append((job, server))
        heapq.heappush(heap, (end, server))
    return plan, opened

# ---- follow-up 3: tests ----
def test_follow_up_3():
    jobs = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8),
            ("c", 10, 15)]
    plan, count = assign_servers(jobs)
    assert count == 2 and len(plan) == 3
    assert split_over_midnight(1380, 60) == [(1380, 1440),
                                             (0, 60)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([])
```

Test cases

Input	Expected
`([],)`	`0`
`([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)`	`1`

ALSO REPORTED · 1 more

Compute minimum servers for cyclic tasks | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-servers-for-cyclic-tasks

STUDY PATH: question 14 of 288 | topic 2 of 43: Sweep over events in time order | 5 of 9

Template fit: the same two-queue simulation with the tie-break rules from the statement | asked 169 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 14 of 288

Gate Entry / Exit Time Resolution with Tie-Break Rules

ASKED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 27.4 | 30 DAYS 0 | PRIORITY 152/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2534 Time Taken to Cross the Door

Frequency: Single report | Last asked: 2026-05-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gate-entry-exit-scheduler

QUESTION

A gate lets one person through per second. events[i] = (time, action) for person i, where action is 'enter' or 'exit'. When several people want the gate in the same second: if the gate was not used in the previous second, people who exit go first; if it was used, the direction used in the previous second goes first again. Within one direction, people go in order of arrival (person id). Return the second at which each person passes.

Function signature

def gate_pass_times(events: list) -> list

EXAMPLES

Example 1

Input	`[(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]`
Output	`[1, 0, 2, 5]`
Why	At second 0 the gate was idle, so the exit (person 1) goes first. Person 0 follows at 1, person 2 (arrived at 1) at 2, and person 3 arrives alone at 5.

Example 2

Input	`[(0, 'enter'), (1, 'enter'), (1, 'exit')]`
Output	`[0, 1, 2]`
Why	The gate was used for entering in the previous second, so the waiting enter keeps priority over the exit.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        queue = choose(prev, enter_q, exit_q)
        if queue:
            person = queue.popleft()
            passed[person] = t
            prev = name_of(queue)
        t += 1
    return passed
```

TIME	SPACE
O(T + n)	O(n)
where T is the last timestamp	

WHY IT IS SLOW

It runs through every second between the first and last timestamp, so a timestamp of 10^9 means a billion iterations. Jumping to the next arrival when the queues are empty makes it O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n)	O(n)
for the sort, O(n) simulation	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the optimization need care across an empty gap?

If the door sits idle for even one second the previous state becomes 'none', which favours exits again. Jumping time forward without resetting the state would wrongly keep 'enter' priority from before the gap. Compare t with last_used + 1 to know whether the gap really was idle.

FOLLOW-UP 2 What is the complexity, and what if events come online?

Offline it is O(n log n) for the sort, then O(n) for the simulation, independent of how large the timestamps are. Online, keep the two queues and the previous-state variable and advance the clock on every arrival or tick; the work per person stays O(1).

FOLLOW-UP 3 Timestamps go up to 10^9.

That is why the event-driven loop jumps to the next arrival: its running time is O(n log n) and does not depend on the size of the timestamps. Remember to reset the previous state to idle across a gap.

FOLLOW-UP 4 The tie-break when the gate was idle differs in this version of the question.

Keep the simulation and make the idle winner a parameter: only the line that picks the queue changes. With 'exit first' it equals the standard rule; with 'enter first' an enter and an exit arriving together swap places.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Gate Entry / Exit Time Resolution with
# Tie-Break Rules
#
# A gate lets one person through per second. events[i] =
# (time, action) for person i, where action is 'enter' or
# 'exit'. When several people want the gate in the same
# second: if the gate was not used in the previous second,
# people who exit go first; if it was used, the direction
# used in the previous second goes first again. Within one
# direction, people go in order of arrival (person id).
# Return the second at which each person passes.
#
# Signature:
#   def gate_pass_times(events: list) -> list
#
# Examples:
# [(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]
#     ->  [1, 0, 2, 5]
# [(0, 'enter'), (1, 'enter'), (1, 'exit')]  ->  [0, 1, 2]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the optimization need care across an empty
# gap?
# 2. What is the complexity, and what if events come
# online?
# 3. Timestamps go up to 10^9.
# 4. The tie-break when the gate was idle differs in this
# version of the question.
#
# INTERVIEW NOTES
# Write the rule as a small state machine first: the state
# is what the gate did in the previous second (entered,
# exited or idle), and the rule picks who goes next from
# that state. Then replace the second-by-second loop with
# a loop over people: when nobody is waiting jump the
# clock forward, and remember that an idle gap resets the
# state to idle.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort people by (arrival time, id) and keep one
# FIFO queue for waiting enters and one for exits. At each
# second pick from the queues using the previous-second
# state.
# FOLLOW-UP 4: The tie-break when the gate was idle
# differs in this version of the question.
# Answer: Keep the simulation and make the idle winner a
# parameter: only the line that picks the queue changes.
# With 'exit first' it equals the standard rule; with
# 'enter first' an enter and an exit arriving together
# swap places.
def gate_pass_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit")
    n = len(events)
    # people sorted by arrival, then by id
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    queues = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or queues["enter"] or queues["exit"]:
        if not queues["enter"] and not queues["exit"]:
            # nobody waits: jump the clock to the next
            # arrival
            t = max(t, events[order[i]][0])
            # a gap of idle time forgets the previous
            # action
            if t > last_used + 1:
                prev = None
        # everyone who arrived by now joins their line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            queues[events[person][1]].append(person)
            i += 1
        both = queues["enter"] and queues["exit"]
```

```
        if both:
            # FOLLOW-UP 4: idle tie-break set by
            # idle_first
            # tie: continue the direction used last second
            # (when idle, use idle_first)
            want = prev or idle_first
        else:
            want = "enter" if queues["enter"] else "exit"
        person = queues[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result

# ---- main question: tests ----
def test_main_question():
    # one enter and one exit arrive together: exit first
    ev = [(0, "enter"), (0, "exit")]
    assert gate_pass_times(ev) == [1, 0]
    # with "enter" winning the idle tie it swaps
    assert gate_pass_times(ev, "enter") == [0, 1]
    # the same direction continues while people keep
    # coming
```

```
    ev2 = [(0, "enter"), (0, "enter"), (1, "exit")]
    assert gate_pass_times(ev2) == [0, 1, 2]
    # a long idle gap resets the state
    ev3 = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert gate_pass_times(ev3) == [0, 11, 10]
    # huge timestamps cost nothing extra
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert gate_pass_times(far) == [10 ** 9 + 1, 10 ** 9]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: The tie-break when the gate was idle
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

STUDY PATH: question 15 of 288 | topic 2 of 43: Sweep over events in time order | 6 of 9

Template fit: split meetings that cross midnight into two pieces first, then the same sweep | no date

MEDIUM | HackTheRounds | Phone Screen | 15 of 288

★ TOP 75: rank 59 in the Medium and Hard priority list

★ Meeting Rooms III Variant

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 46/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 (cyclic variant)

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rooms_needed(meetings):
    slots = [minutes_used(m) for m in meetings]
    for k in range(1, len(meetings) + 1):
        rooms = [set() for _ in range(k)]
        if assign(0, rooms, slots):
            return k
def assign(i, rooms, slots):
    if i == len(slots):
        return True
    for room in rooms:
        busy = slots[i]
        if not room & busy:  # no shared minute
            room |= busy
            if assign(i + 1, rooms, slots):
                return True
            room -= busy
    return False
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Backtracking over room assignments is exponential. Splitting wrapped meetings and sweeping start and end events gives the maximum overlap in O(n log n); that equals the exact answer when meetings do not wrap.

OPTIMAL SOLUTION · Split wrapped meetings and sweep (maximum overlap)

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A wrapped meeting must stay in a single room.

Then it is circular-arc colouring, NP-hard in general. A common practical method: cut the circle at the minute with the fewest active meetings; the meetings crossing the cut use fixed rooms, and the rest are scheduled greedily with a heap. It can use more rooms than the maximum overlap, and the five-arc example shows why.

FOLLOW-UP 2 Return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room id); reuse the earliest-ending room if it is free, otherwise open a new one. For wrapped meetings schedule them first so their rooms are fixed.

FOLLOW-UP 3 Days can repeat (a wrapped meeting recurs every day).

A recurring meeting occupies its slots every day, so the circle model above is exactly right, and the no-wrap shortcut of ordering by start time is not.

FOLLOW-UP 4 Return which room each meeting uses.

Sort by start and keep a heap of (end time, room): reuse the earliest-ending room if it is free, otherwise open a new one. For meetings that do not wrap this uses exactly the minimum number of rooms (verified against the sweep); wrapped meetings need the cut-the-circle method.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Meeting Rooms III Variant
#
# Given an array of meeting time intervals where each
# interval is [start, end], determine the minimum number
# of conference rooms required to hold all meetings
# without overlap. Variant: times are cyclic within a
# 24-hour period (meetings can wrap around midnight).
#
# FOLLOW-UPS (answered below, in this order):
# 1. A wrapped meeting must stay in a single room.
# 2. Return which room each meeting uses.
# 3. Days can repeat (a wrapped meeting recurs every day).
# 4. Return which room each meeting uses.
#
# INTERVIEW NOTES
# Split a wrapped meeting into two pieces (from start to
# the end of the day, and from midnight to its end) and
# run the normal sweep line: the largest number of
# meetings active at once is the number of rooms needed.
# That is exact when pieces may sit in different rooms; if
# a meeting must keep one room, the arcs form a circular-
# arc graph and the true minimum can exceed the maximum
# overlap (five overlapping arcs around a circle need 3
# rooms with overlap 2). Say which reading you assume.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
# HELPERS used by the code below
DAY = 24
```

```
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past
    # midnight: split it into the part before midnight and
    # the part after it
    for s, e in meetings:
        if e < s:
            yield s, day
            yield 0, e
        else:
            yield s, e

# ---- main question: main function ----
# BASE QUESTION (optimal): Split wrapped meetings and
# sweep (maximum overlap)
# Idea: Turn every meeting into one or two ordinary
# intervals, create +1 events at starts and -1 events at
# ends, sort them (ends before starts at the same minute,
# because intervals are half-open), and take the maximum
# running count.
def min_rooms_sweep(meetings, day=DAY):
    # sweep the time line: +1 at a start, -1 at an end. At
    # equal times an end comes first (a meeting ending at
    # 5
    # and one starting at 5 can share a room).
    events = []
    for s, e in pieces(meetings, day):
        events.append((s, 1))
        events.append((e, -1))
    events.sort()
    cur = best = 0
    for _, step in events:
        cur += step
        best = max(best, cur)
    return best

# ---- main question: tests ----
def test_main_question():
    long = [[0, 30], [5, 10], [15, 20]]
    assert min_rooms_sweep(long) == 2
    assert min_rooms_sweep([[7, 10], [2, 4]]) == 1
    night = [[22, 2], [23, 1], [0, 3]]
    assert min_rooms_sweep(night) == 3
    assert min_rooms_sweep([]) == 0
    # one ends exactly when the next starts: same room
    assert min_rooms_sweep([[1, 5], [5, 9]]) == 1
```

```
    flat = [[0, 30], [5, 10], [15, 20]]


# ============================================================
# FOLLOW-UP 1: A wrapped meeting must stay in a single
# ============================================================

# ---- follow-up 1: helper functions ----
def hours(meeting, day=DAY):
    # the hours a meeting occupies on the 24 hour circle
    s, e = meeting
    length = (e - s) % day or day
    return {(s + k) % day for k in range(length)}

# ---- follow-up 1: function ----
# Answer: Then it is circular-arc colouring, NP-hard in
# general.
def assign_single_rooms(meetings, day=DAY):
    # follow-up: a meeting that wraps must stay in ONE
    # room.
    # That is circular-arc colouring (NP-hard), so use the
    # practical method: cut the circle at the hour where
    # the
```

```
    # fewest meetings are running, give those meetings
    # their
    # own rooms, and place the rest greedily.
    busy = [sum(1 for m in meetings
                if cut in hours(m, day))
            for cut in range(day)]
    cut = busy.index(min(busy))
    # rotate so that the cut is time 0: (start, end) on a
    # line
    line = []
    for s, e in meetings:
        start = (s - cut) % day
        length = (e - s) % day or day
        line.append((start, start + length))
    free_at, deadline = [], []     # per room
    room_of = {}
    for i, (s, e) in enumerate(line):
        if e > day:
            # crosses the cut: busy until e - day, and
            # again
            # from s on, so the room is blocked after s
            room_of[i] = len(free_at)
```

```
            free_at.append(e - day)
            deadline.append(s)
    rest = [i for i in range(len(line))
            if i not in room_of]
    others = sorted(rest, key=lambda i: line[i])
    for i in others:
        s, e = line[i]
        # the room that has been free the longest and fits
        fits = [r for r in range(len(free_at))
                if free_at[r] <= s and e <= deadline[r]]
        if fits:
            room = max(fits, key=lambda r: free_at[r])
        else:
            room = len(free_at)
            free_at.append(0)
            deadline.append(day)
        room_of[i] = room
        free_at[room] = e
    return [room_of[i] for i in range(len(meetings))]

# ============================================================
# FOLLOW-UP 2: Return which room each meeting uses.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sort by start and keep a min-heap of (end time,
# room id); reuse the earliest-ending room if it is free,
# otherwise open a new one. For wrapped meetings schedule
# them first so their rooms are fixed.
# FOLLOW-UP 4: Return which room each meeting uses.
# Answer: Sort by start and keep a heap of (end time,
# room): reuse the earliest-ending room if it is free,
# otherwise open a new one.
def assign_rooms(meetings):
    # follow-up: which room for each meeting (no
    # wrapping).
    # Sort by start; a heap of (end, room) gives the room
    # that
    # frees up first; reuse it if it is free, else open
    # one.
    order = sorted(range(len(meetings)),
                   key=lambda i: meetings[i][0])
    heap, room_of, rooms = [], {}, 0
    for i in order:
        s, e = meetings[i]
```

```
        if heap and heap[0][0] <= s:
            _, room = heapq.heappop(heap)
        else:
            room, rooms = rooms, rooms + 1
        room_of[i] = room
        heapq.heappush(heap, (e, room))
    return [room_of[i] for i in range(len(meetings))]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    flat = [[0, 30], [5, 10], [15, 20]]
    rooms = assign_rooms(flat)
    assert rooms == [0, 1, 1]
    assert len(set(rooms)) == min_rooms_sweep(flat)
    two = [(22, 2), (1, 5)]
    assert valid_assignment(two, assign_single_rooms(two))
    for _ in range(400):
        ms = []
        for _ in range(random.randint(0, 7)):
            s, e = random.sample(range(DAY), 2)
            ms.append((s, e))
        rooms = assign_single_rooms(ms)
        assert valid_assignment(ms, rooms)
        if all(s < e for s, e in ms):
            plain = assign_rooms(ms)
            assert valid_assignment(ms, plain)


# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Return which room each meeting uses.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def valid_assignment(meetings, rooms, day=DAY):
    # check that no room holds two meetings at the same
    # hour
```

```
    used = set()
    for m, room in zip(meetings, rooms):
        for h in hours(m, day):
            if (room, h) in used:
                return False
            used.add((room, h))
    return True


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 16 of 288 | topic 2 of 43: Sweep over events in time order | 7 of 9

Template fit: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes | asked 102 days ago

HARD | IN APP LC 2536 | PracHub | Technical Screen | 16 of 288

Count Overlapping Rectangle Updates on a Grid

POSTED 2026-07-10 | 102 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 172/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2536 Increment Submatrices by One

LeetCode match: LC 2536 Increment Submatrices by One

Posted on PracHub: 2026-07-10 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/count-overlapping-rectangle-updates-on-a-grid

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement `range_add_counts(n, rectangles)`. Start with an `n x n` matrix of zeroes. Each rectangle is represented as `[row1, col1, row2, col2]` with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For `n = 3` and `rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- `1 <= n <= 500`
- `0 <= len(rectangles) <= 100_000`
- `0 <= row1 <= row2 < n`
- `0 <= col1 <= col2 < n`

EXAMPLES

Example 1	
Input	`3, [(0, 0, 1, 1), (1, 1, 2, 2)]`
Output	`[[1, 1, 0], [1, 2, 1], [0, 1, 1]]`
Why	`The cell (1, 1) is covered by both rectangles, so it is counted twice.`

Example 2	
Input	`2, []`
Output	`[[0, 0], [0, 0]]`
Why	`No rectangles: all zeros.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def range_add_counts(n, rectangles):
    grid = [[0] * n for _ in range(n)]
    for r1, c1, r2, c2 in rectangles:
        for r in range(r1, r2 + 1):
            for c in range(c1, c2 + 1):
                grid[r][c] += 1
    return grid
```

TIME	SPACE
O(R * area)	O(n^2)

WHY IT IS SLOW

O(R * area): 100,000 rectangles of area up to 250,000 is billions of cell updates. A 2D difference array touches four corners per rectangle and one prefix-sum pass rebuilds the counts, O(R + n^2).

OPTIMAL SOLUTION · 2D difference array and prefix sums

TIME	SPACE
O(R + n^2)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many cells are covered by at least k rectangles?

Build the count matrix with the difference array and count the cells with value at least k. O(R + n^2).

FOLLOW-UP 2 The grid is huge (10^9 x 10^9) but there are few rectangles.

Coordinate-compress the corner coordinates and apply the same difference array on the compressed grid (at most (2R)^2 cells), or use a sweep line with a segment tree over columns. **Cost:** O(R log R) to sort the corner coordinates plus up to O(R^2) compressed cells, instead of 10^18 cells.

FOLLOW-UP 3 Rectangles are added and the count of a cell is queried online.

Use a 2D Fenwick tree with range update and point query (the difference-array idea turned into a BIT): O(log^2 n) per operation.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Overlapping Rectangle Updates on a
# Grid
#
# # Count Overlapping Rectangle Updates on a Grid
# Implement range_add_counts(n, rectangles). Start with an
# n x n matrix of zeroes. Each rectangle is represented as
# [row1, col1, row2, col2] with inclusive, zero-based
# coordinates. For every rectangle, add one to each matrix
# cell inside that rectangle. Return the final matrix,
# where each cell therefore contains the number of input
# rectangles covering it. ## Example For n = 3 and
# rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]], return: text
# [ [1, 1, 0], [1, 2, 1], [0, 1, 1] ] ## Constraints - 1
# <= n <= 500 - 0 <= len(rectangles) <= 100_000 - 0 <=
# row1 <= row2 < n - 0 <= col1 <= col2 < n
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many cells are covered by at least k rectangles?
# 2. The grid is huge (10^9 x 10^9) but there are few
# rectangles.
# 3. Rectangles are added and the count of a cell is
# queried online.
#
# INTERVIEW NOTES
# Adding 1 to every cell of up to 100,000 rectangles one
# cell at a time is too slow. Record only the corners of
# each rectangle (+1 at the top-left, -1 just beyond the
# right and bottom edges, +1 at the bottom-right corner
# beyond), and rebuild the counts with one pass of 2D
# prefix sums. Each rectangle then costs O(1) instead of
# its area.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): 2D difference array and prefix
# sums
# Idea: For each rectangle add +1 at (r1, c1), -1 at (r1,
# c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1)
# in an (n + 1) x (n + 1) array.
def range_add_counts(n, rects):
    # rects are [r1, c1, r2, c2], inclusive. Mark only the
    # 4
    # corners of each rectangle in a difference array ...
    d = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    # ... then two prefix sums spread them over the cells
    for r in range(n + 1):
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    return [row[:n] for row in d[:n]]

# ---- main question: tests ----
def test_main_question():
    rects = [[0, 0, 1, 1], [1, 1, 2, 2]]
    assert range_add_counts(3, rects) == [
        [1, 1, 0], [1, 2, 1], [0, 1, 1]]


# ============================================================
# FOLLOW-UP 1: How many cells are covered by at least k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the count matrix with the difference array
# and count the cells with value at least k. O(R + n^2).
def cells_at_least(n, rects, k):
    # how many cells are covered by k or more rectangles?
    counts = range_add_counts(n, rects)
    return sum(1 for row in counts for v in row if v >= k)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    rects = [[0, 0, 1, 1], [1, 1, 2, 2]]
    assert cells_at_least(3, rects, 2) == 1
    assert cells_at_least(3, rects, 1) == 7



# ============================================================
# FOLLOW-UP 2: The grid is huge (10^9 x 10^9) but there
# ============================================================

# ---- follow-up 2: function ----
# Answer: Coordinate-compress the corner coordinates and
# apply the same difference array on the compressed grid
# (at most (2R)^2 cells), or use a sweep line with a
# segment tree over columns.
def cells_at_least_compressed(rects, k):
    # a huge grid, few rectangles: compress the
    # coordinates
    rows = sorted({r for r1, _, r2, _ in rects
                   for r in (r1, r2 + 1)})
    cols = sorted({c for _, c1, _, c2 in rects
                   for c in (c1, c2 + 1)})
    ri = {r: i for i, r in enumerate(rows)}
    ci = {c: i for i, c in enumerate(cols)}
    shape = (len(rows) + 1, len(cols) + 1)
    d = [[0] * shape[1] for _ in range(shape[0])]
    for r1, c1, r2, c2 in rects:
        a, b = ri[r1], ci[c1]
        e, f = ri[r2 + 1], ci[c2 + 1]
        d[a][b] += 1
        d[a][f] -= 1
        d[e][b] -= 1
        d[e][f] += 1
    total = 0
    run_col = [0] * (len(cols) + 1)
    for i in range(len(rows) - 1):
        row, run = [0] * (len(cols) + 1), 0
        for j in range(len(cols) - 1):
            run_col[j] += d[i][j]
            run += run_col[j]
            # each compressed cell stands for a block of
            # cells
```

```
            if run >= k:
                total += (rows[i + 1] - rows[i]) * (
                    cols[j + 1] - cols[j])
    return total
```

```

# ---- follow-up 2: tests ----
def test_follow_up_2():
    rects = [[0, 0, 1, 1], [1, 1, 2, 2]]
    # the compressed version gives the same area
    assert cells_at_least_compressed(rects, 2) == 1
    assert cells_at_least_compressed(rects, 1) == 7
    big = [[0, 0, 10 ** 9, 10 ** 9], [5, 5, 10, 10]]
    assert cells_at_least_compressed(big, 2) == 36


# ============================================================
# FOLLOW-UP 3: Rectangles are added and the count of a
# ============================================================

# ---- follow-up 3: function ----
# Answer: Use a 2D Fenwick tree with range update and
# point query (the difference-array idea turned into a
# BIT): O(log^2 n) per operation.
class Fenwick2D:
    """Online updates and point reads: O(log^2 n)."""

    def __init__(self, n):
        self.n = n
        self.t = [[0] * (n + 2) for _ in range(n + 2)]

    def _add(self, r, c, v):
        i = r + 1
        while i <= self.n + 1:
            j = c + 1
            while j <= self.n + 1:
                self.t[i][j] += v
                j += j & -j
            i += i & -i

    def add_rect(self, r1, c1, r2, c2):
        corners = ((r1, c1, 1), (r1, c2 + 1, -1),
                   (r2 + 1, c1, -1), (r2 + 1, c2 + 1, 1))
        for r, c, v in corners:
            self._add(r, c, v)

    def get(self, r, c):
        total, i = 0, r + 1
        while i > 0:
            j = c + 1
```

```
            while j > 0:
                total += self.t[i][j]
                j -= j & -j
            i -= i & -i
        return total

# ---- follow-up 3: tests ----
def test_follow_up_3():
    rects = [[0, 0, 1, 1], [1, 1, 2, 2]]
    # online updates and point queries
    f = Fenwick2D(3)
    for r in rects:
        f.add_rect(*r)
    assert f.get(1, 1) == 2 and f.get(0, 2) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def range_add_counts(n: int, rectangles: list[list[int]])
-> list[list[int]]:
```

Sample call

```
range_add_counts(1, [])
```

Test cases

Input	Expected
`(1, [])`	`[[0]]`
`(1, [[0, 0, 0, 0]])`	`[[1]]`

STUDY PATH: question 17 of 288 | topic 2 of 43: Sweep over events in time order | 8 of 9

Template fit: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes | asked 316 days ago

HARD | IN APP LC 218 | PracHub | Technical Screen | 17 of 288

Compute city skyline outline

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 145/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #218 The Skyline Problem

LeetCode match: LC 218 The Skyline Problem

Posted on PracHub: 2025-12-08 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/compute-city-skyline-outline

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers `[L, R, H]`:

- `L`: the x-coordinate of the left edge of the building,
- `R`: the x-coordinate of the right edge of the building (`L < R`),
- `H`: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height `H`. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing `x`.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **n** — the number of buildings.
- A list of **n** buildings, where each building is given as a triple `[L, R, H]` of integers. You may assume:
- `1 <= n <= 10^4`
- `0 <= L < R <= 10^9`
- `1 <= H <= 10^9`

Output

Return a list of key points representing the skyline. Each key point is a pair `[x, height]` (or a 2-element array/tuple) of integers.

Example

Input:

- `n = 3` - buildings = `[[2, 9, 10], [3, 7, 15], [5, 12, 12]]` One valid output:
- `[[2, 10], [3, 15], [7, 12], [12, 0]]` Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

EXAMPLES

Example 1

Input	`[[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]`
Output	`[[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]`
Why	`Buildings are [left, right, height]; the outline lists the points where the height changes.`

Example 2

Input	`[[0, 2, 3]]`
Output	`[[0, 3], [2, 0]]`
Why	`One building: a point at its left edge and one at its right edge, where the height returns to 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(buildings):
    xs = set()
    for L, R, H in buildings:
        xs |= {L, R}
    out, prev = [], 0
    for x in sorted(xs):
        h = 0
        for L, R, H in buildings:
            if L <= x < R:
                h = max(h, H)
        if h != prev:
            out.append([x, h])
            prev = h
    return out
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each edge scans all buildings, O(n^2). A sweep with a max-heap of active buildings and lazy deletion is O(n log n).

OPTIMAL SOLUTION · Sweep line with a max-heap and lazy deletion

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Divide and conquer instead of a heap.

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

FOLLOW-UP 2 Coordinates up to 10^9 or buildings of a very large count.

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

FOLLOW-UP 3 Output the area under the skyline.

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute city skyline outline
#
# You are given a list of rectangular buildings in a 2D
# city skyline. Each building is represented by three
# integers [L, R, H]: - L: the x-coordinate of the left
# edge of the building, - R: the x-coordinate of the right
# edge of the building (L < R), - H: the height of the
# building. All buildings stand on a flat horizontal line
# (y = 0) and extend vertically up to height H. Buildings
# may overlap in the x-dimension. Your task is to compute
# the *skyline* formed by these buildings, as seen from a
# distance. The skyline should be represented as a list of
# **key points** [x, height] that describe the outer
# contour of the union of all buildings. A **key point**
# is a point where the height of the skyline changes. The
# output list must satisfy all of the following: 1. The
# list is sorted by increasing x. 2. No two consecutive
# points have the same height (i.e., the height must
# change at each key point). 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer instead of a heap.
# 2. Coordinates up to 10^9 or buildings of a very large
# count.
# 3. Output the area under the skyline.
```

```
#
# INTERVIEW NOTES
# The skyline height at x is the tallest building covering
# x, and it can only change at a building's left or right
# edge. So sweep over those x positions in order, keep the
# active buildings in a max-heap by height, and report a
# key point whenever the maximum changes. The details that
# matter are the event order at the same x and removing
# finished buildings lazily.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (x, -height) for left edges and (x, 0,
# right) for right edges, sorted so that at the same x
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge).
def skyline(buildings):
    # buildings are [left, right, height]
    # start events carry -height so taller starts sort
    # first;
    # end events carry 0
    events = sorted([(l, -h, r) for l, r, h in buildings]
                    + [(b[1], 0, 0) for b in buildings])
    out = []
    # the active buildings: (-height, right edge), ground
    # first
    heap = [(0, float("inf"))]
    for x, neg_h, right in events:
        # buildings that ended at or before x are gone
        while heap[0][1] <= x:
            heapq.heappop(heap)
        if neg_h:
            heapq.heappush(heap, (neg_h, right))
        top = -heap[0][0]
        # a key point only where the height changes
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out

# ---- main question: tests ----
def test_main_question():
    b = [[2, 9, 10], [3, 7, 15], [5, 12, 12],
         [15, 20, 10], [19, 24, 8]]
    want = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10],
            [20, 8], [24, 0]]
    assert skyline(b) == want


# ============================================================
# FOLLOW-UP 1: Divide and conquer instead of a heap.
```

```
# ============================================================

# ---- follow-up 1: helper functions ----
def merge(a, b):
```

```
    # divide and conquer: merge two skylines with the
    # current
    # height of each; the result height is their maximum
    out, i, j, ha, hb = [], 0, 0, 0, 0
    while i < len(a) or j < len(b):
        take_a = i < len(a) and (j == len(b)
                                 or a[i][0] <= b[j][0])
        if take_a:
            x, ha = a[i]
            i += 1
        else:
            x, hb = b[j]
            j += 1
        # when both change at the same x, wait for the
        # second
        if i < len(a) and a[i][0] == x:
            continue
        if j < len(b) and b[j][0] == x:
            continue
        top = max(ha, hb)
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out

# ---- follow-up 1: function ----
# Answer: Build the skyline of each half and merge the two
# by sweeping both lists with the current height of each;
# the merged height is the max of the two. T(n) = 2T(n/2)
# + O(n) = O(n log n), same as the heap.
def skyline_dc(buildings):
    if len(buildings) == 1:
        l, r, h = buildings[0]
        return [[l, h], [r, 0]]
    if not buildings:
        return []
    mid = len(buildings) // 2
    return merge(skyline_dc(buildings[:mid]),
                 skyline_dc(buildings[mid:]))

# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    b = [[2, 9, 10], [3, 7, 15], [5, 12, 12],
         [15, 20, 10], [19, 24, 8]]
    want = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10],
            [20, 8], [24, 0]]
    assert skyline_dc(b) == want
    flat = skyline([[0, 2, 3], [2, 5, 3]])
    assert flat == [[0, 3], [5, 0]]
    assert skyline([]) == []


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Output the area under the skyline.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Sum height * (next x - x) over consecutive key
# points: one extra pass over the output.
def area_under(points):
    # sum of height * width between consecutive key points
    return sum(h * (points[i + 1][0] - x)
               for i, (x, h) in enumerate(points[:-1]))

# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    want = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10],
            [20, 8], [24, 0]]
    # 10*1 + 15*4 + 12*5 + 10*5 + 8*4 = 212
    assert area_under(want) == 212
    assert skyline([[1, 2, 1], [1, 2, 2], [1, 2, 3]]) == [
        [1, 3], [2, 0]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
getSkyline(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])
```

Test cases

Input	Expected
(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])	[[2, 10], [3, 15], [7, 12], [12, 0]]
(1, [[0, 2, 3]])	[[0, 3], [2, 0]]

STUDY PATH: question 18 of 288 | topic 2 of 43: Sweep over events in time order | 9 of 9

Template fit: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second | asked 169 days ago

HARD | PracHub | Technical Screen | 18 of 288

Compute Turnstile Crossing Times

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 200/288

ABOUT THIS QUESTION

Topic: Sweep over events in time order

Optimal answer from: the interview site's own solution

LeetCode match: LeetCode 'Time Taken to Cross the Door' (exact number unverified) (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-10)

Original: https://prachub.com/coding-questions/compute-turnstile-crossing-times

QUESTION

A turnstile lets one person through per second. events[i] = (arrival time, direction) for person i, with direction 'enter' or 'exit'. When people in both directions wait in the same second: if the turnstile was not used in the previous second, people exiting go first; otherwise the direction used in the previous second goes first again. People of the same direction go in order of arrival (person id). Return the second at which each person crosses.

Function signature

def turnstile_times(events: list) -> list

EXAMPLES

Example 1

Input	`[(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]`
Output	`[0, 1, 2, 3]`
Why	The idle turnstile lets the exit (person 0) go first at second 0, then the two entering people follow at 1 and 2; person 3 arrives alone at 3.

Example 2

Input	`[(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]`
Output	`[2, 0, 1, 3]`
Why	Idle turnstile: the exits (persons 1 and 2) go first at seconds 0 and 1, then the entering people (persons 0 and 3) at seconds 2 and 3.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        who = pick_by_priority(prev, enter_q,
                               exit_q)
        if who is not None:
            passed[who] = t
            prev = direction_of(who)
        t += 1
    return passed
```

TIME	SPACE
O(T + n)	O(n)
where T is the largest timestamp	

WHY IT IS SLOW

A timestamp of 10^9 means a billion iterations even if nothing happens. Jumping to the next arrival when both queues are empty is O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

FOLLOW-UP 2 Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

FOLLOW-UP 3 Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute Turnstile Crossing Times
#
# A turnstile lets one person through per second.
# events[i] = (arrival time, direction) for person i, with
# direction 'enter' or 'exit'. When people in both
# directions wait in the same second: if the turnstile was
# not used in the previous second, people exiting go
# first; otherwise the direction used in the previous
# second goes first again. People of the same direction go
# in order of arrival (person id). Return the second at
# which each person crosses.
#
# Signature:
#   def turnstile_times(events: list) -> list
#
# Examples:
# [(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]
#     ->  [0, 1, 2, 3]
# [(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]
#     ->  [2, 0, 1, 3]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority when the turnstile was idle is different
# in this version of the question.
# 2. Timestamps are up to 10^9.
# 3. Two turnstiles or a capacity of two people per
# second.
#
# INTERVIEW NOTES
# A scheduling rule that looks at 'what happened last
# second' is a state machine: write the state down
# (previous direction: enter, exit, none) before coding.
# If time values can be huge, simulate events, not
# seconds: jump the clock to the next arrival when both
# queues are empty, and remember an idle gap resets the
# state.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'.
# FOLLOW-UP 1: The priority when the turnstile was idle is
# different in this version of the question.
# Answer: Keep the same loop and make the idle-state
# winner a parameter. Only the line that picks the queue
# changes, which is why the state machine should be
# written first.
def turnstile_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit");
    # the result is the second each person goes through
    n = len(events)
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            # nobody is waiting: jump to the next arrival
            t = max(t, events[order[i]][0])
            # an idle second forgets the last direction
            if t > last_used + 1:
                prev = None
        # everyone who has arrived by now joins a line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            lines[events[person][1]].append(person)
            i += 1
        if lines["enter"] and lines["exit"]:
            # FOLLOW-UP 1: idle tie-break set by
            # idle_first
            # both lines: keep the direction used last
            # second,
            # and use idle_first after an idle gap
            want = prev or idle_first
        else:
            want = "enter" if lines["enter"] else "exit"
        person = lines[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result

# ---- main question: tests ----
def test_main_question():
    both = [(0, "enter"), (0, "exit")]
    # idle tie: the exit goes first
    assert turnstile_times(both) == [1, 0]
    # a version where enter wins the idle tie
    assert turnstile_times(both, "enter") == [0, 1]
    # the direction in use keeps going
    keep = [(0, "enter"), (0, "enter"), (1, "exit")]
    assert turnstile_times(keep) == [0, 1, 2]
    # an idle second resets the state
    gap = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert turnstile_times(gap) == [0, 11, 10]
    # huge timestamps are no problem: one loop per person
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert turnstile_times(far) == [10 ** 9 + 1, 10 ** 9]
    assert turnstile_times([]) == []


# ============================================================
# FOLLOW-UP 1: The priority when the turnstile was idle is
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(time: list[int], direction: list[int]) -> lis
t[int]:
SAMPLE INPUT
[[0,0,1,5],[0,1,1,0]]
```

Test cases

Input	Expected
[[0, 0, 1, 5], [0, 1, 1, 0]]	[2, 0, 1, 5]
[[0, 0, 0], [0, 1, 0]]	[1, 0, 2]

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 3 OF 43 |

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	4	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

E	19. Given two trees, determine if the pre-order traversals of two trees are same or not.	Exponent	no date
E	20. Calculate the height of a binary tree.	Exponent	no date
E	21. Find right-side view of binary tree	PracHub	2025-11-01
M	22. Expression Tree Single-Leaf Mutation	1Point3Acres	2026-09-27
M	23. Longest Consecutive Path in Binary Tree	1Point3Acres	2026-07-02
M	24. Given the root of a binary tree, return the length of the diameter of the tree.	Exponent	no date
M	25. Implement zigzag level-order traversal	PracHub	2025-08-01
H	26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	PracHub	2026-09-16
H	27. Delete an Entire Binary Tree	ColorStack	2025-09-29

CODE TEMPLATE | GROUP 3 OF 43 | 9 QUESTIONS, ONE PATTERN

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

The one idea these questions share

Write dfs(node, depth). An empty node returns a base value, a real node calls dfs on both children, updates the best answer so far, and returns one number or flag to its parent. Only three lines change: the base value, the update, and what you return.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def solve(root):
    best = 0

    def dfs(node, depth=0):
        nonlocal best
        if node is None:
            return 0                          # CHANGE 1: what an empty tree reports
        left = dfs(node.left, depth + 1)
        right = dfs(node.right, depth + 1)
        best = max(best, left + right)        # CHANGE 2: update the answer at this node
        return 1 + max(left, right)           # CHANGE 3: what this node reports upward

    dfs(root)
    return best
```

2. Helper functions that usually get written next to it

```
from collections import deque


class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def levels(root):                             # BFS by level: height, right-side view, zigzag
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(c for c in (n.left, n.right) if c)
    return out


def preorder(root):                           # iterative: push right first, then left
    stack = [root] if root else []
    while stack:
        n = stack.pop()
        yield n.val
        stack.extend(c for c in (n.right, n.left) if c)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
19	Given two trees, determine if the pre-order traversals of two trees are same or not.	walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference

#	Question	What you change in the template
20	Calculate the height of a binary tree.	no update needed; return 1 + max(left, right) and read the value at the root
21	Find right-side view of binary tree	pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see
22	Expression Tree Single-Leaf Mutation	post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed
23	Longest Consecutive Path in Binary Tree	pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1
24	Given the root of a binary tree, return the length of the diameter of the tree.	best = max(best, left + right); return 1 + max(left, right) (the diameter)
25	Implement zigzag level-order traversal	collect values by depth; reverse every odd level (or fill each level from the other end)
26	Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value
27	Delete an Entire Binary Tree	post-order: free the children first, then the node itself

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 19 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 1 of 9

Template fit: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference | no date

EASY | Exponent | 19 of 288

Given two trees, determine if the pre-order traversals of two trees are same or not.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TYPE | ALL TIME 45.8 | 30 DAYS 0 | PRIORITY 91/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #100 Same Tree

LeetCode match: LC 100 Same Tree (related: traversal comparison) (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/7182/compare-preorder-traversals-two-trees

QUESTION

Given two binary trees, determine whether their pre-order traversals (node, then left subtree, then right subtree) produce the same sequence of values. The trees themselves may have different shapes.

Function signature

def same_preorder(a: TreeNode, b: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3], [1, 2, 3]`
Output	`true`
Why	`Identical trees.`

Example 2	
Input	`[1, 2, null, 3], [1, null, 2, null, 3]`
Output	`true`
Why	`Different shapes (a chain going left versus a chain going right) but both read 1, 2, 3.`

Example 3	
Input	`[1, 2, 3], [1, 3, 2]`
Output	`false`
Why	`1, 2, 3 versus 1, 3, 2.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def preorder(node):
    if node is None:
        return []
    left = preorder(node.left)
    right = preorder(node.right)
    return [node.val] + left + right
same = preorder(a) == preorder(b)
```

TIME	SPACE
O(n + m)	O(n + m)

WHY IT IS SLOW

It always visits and stores both trees completely, O(n + m) time and space. Walking both pre-orders lazily in lockstep stops at the first difference with O(h) space.

OPTIMAL SOLUTION · Lazy pre-order generators compared in lockstep

TIME	SPACE
O(min(n, m)) with early exit, O(n + m) worst case	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the interviewer meant the trees must be identical?

Then compare structure as well as values: same_tree above walks both trees together and treats two missing nodes as equal. Equivalently, serialize pre-order with null markers and compare the strings, since pre-order with nulls is unique. Ask which definition they mean before coding.

FOLLOW-UP 2 Do it with O(1) extra space.

Morris traversal visits pre-order using temporary threads on right pointers, giving O(1) space, but it mutates the tree while running (restoring it after), so it is not safe for concurrent readers. Mention that trade-off.

FOLLOW-UP 3 They want identical trees, not just sequences.

Compare structure as well: walk both trees together and treat a missing child on one side only as a mismatch, or serialise pre-order with null markers and compare the strings.

FOLLOW-UP 4 They want identical trees (structure and values), not just the same pre-order values.

Compare the two trees together: a node must match in value and both children must match; two missing children match. Equivalently compare the pre-order serialisation with null markers (which is unique). O(min(n, m)) with early exit.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given two trees, determine if the pre-
# order traversals of two trees are same or not.
#
# Given two binary trees, determine whether their pre-
# order traversals (node, then left subtree, then right
# subtree) produce the same sequence of values. The trees
# themselves may have different shapes.
#
# Signature:
#   def same_preorder(a: TreeNode, b: TreeNode) -> bool
#
# Examples:
# [1, 2, 3], [1, 2, 3]  ->  true
# [1, 2, null, 3], [1, null, 2, null, 3]  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the interviewer meant the trees must be
# identical?
# 2. Do it with O(1) extra space.
# 3. They want identical trees, not just sequences.
# 4. They want identical trees (structure and values), not
# just the same pre-order values.
#
# INTERVIEW NOTES
# Ask whether they want identical pre-order value
# sequences or identical trees; different trees can share
# a pre-order. For sequences, compare lazily so the first
# difference ends the work, instead of building two full
# lists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from itertools import zip_longest


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
MISSING = object()


def preorder(root):
    # root, then left, then right; iterative with a stack
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        yield node.val
        # right goes first so the left child comes out
        # first
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)

# ---- main question: main function ----
# BASE QUESTION (optimal): Lazy pre-order generators
# compared in lockstep
# Idea: Generate each pre-order lazily with an explicit
# stack (push right before left), and compare the two
# streams element by element with zip_longest, so the
# first mismatch stops the work and no list is built.
def same_preorder(a, b):
    # lazily compare the two sequences value by value; a
    # shorter sequence is padded with MISSING and so
    # differs
    pairs = zip_longest(preorder(a), preorder(b),
                        fillvalue=MISSING)
    return all(x == y for x, y in pairs)

# ---- main question: tests ----
def test_main_question():
    t1 = TreeNode(1, TreeNode(2), TreeNode(3))
    # a different shape with the same pre-order: 1, 2, 3
    t2 = TreeNode(1, TreeNode(2, None, TreeNode(3)))
    assert same_preorder(t1, t2)


# ============================================================
# FOLLOW-UP 1: What if the interviewer meant the trees
# ============================================================

# ---- follow-up 1: function ----
# Answer: Then compare structure as well as values:
```

```
# same_tree above walks both trees together and treats two
# missing nodes as equal.
# FOLLOW-UP 3: They want identical trees, not just
# sequences.
# Answer: Compare structure as well: walk both trees
# together and treat a missing child on one side only as a
# mismatch, or serialise pre-order with null markers and
# compare the strings.
# FOLLOW-UP 4: They want identical trees (structure and
# values), not just the same pre-order values.
# Answer: Compare the two trees together: a node must
# match in value and both children must match; two missing
# children match. Equivalently compare the pre-order
# serialisation with null markers (which is unique).
def same_tree(a, b):
    # follow-up: truly identical trees (shape and values).
    # Walk both together; one missing child is a mismatch.
    stack = [(a, b)]
    while stack:
        x, y = stack.pop()
        if x is None and y is None:
            continue
        if x is None or y is None or x.val != y.val:
            return False
        stack.append((x.left, y.left))
        stack.append((x.right, y.right))
    return True


# Answer: Then compare structure as well as values:
# same_tree above walks both trees together and treats two
```

```
# missing nodes as equal.
# FOLLOW-UP 3: They want identical trees, not just
# sequences.
# Answer: Compare structure as well: walk both trees
# together and treat a missing child on one side only as a
# mismatch, or serialise pre-order with null markers and
# compare the strings.
# FOLLOW-UP 4: They want identical trees (structure and
# values), not just the same pre-order values.
# Answer: Compare the two trees together: a node must
# match in value and both children must match; two missing
# children match. Equivalently compare the pre-order
# serialisation with null markers (which is unique).
def serialise(root):
```

```
    # follow-up: pre-order with null markers is unique, so
    # equal strings mean identical trees
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if node is None:
            out.append("#")
            continue
        out.append(str(node.val))
        stack.append(node.right)
        stack.append(node.left)
    return ",".join(out)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    t1 = TreeNode(1, TreeNode(2), TreeNode(3))
    # a different shape with the same pre-order: 1, 2, 3
    t2 = TreeNode(1, TreeNode(2, None, TreeNode(3)))
    assert not same_tree(t1, t2)
    assert serialise(t1) != serialise(t2)
    copy = TreeNode(1, TreeNode(2), TreeNode(3))
    assert same_tree(t1, copy)
    assert same_preorder(None, None)
    assert same_tree(None, None)
    assert not same_preorder(t1, None)
    assert not same_preorder(t1, TreeNode(1, TreeNode(2)))
    assert same_preorder_constant_space(t1, t2)
    # the trees are left exactly as they were
    assert t1.left.right is None and t1.right.left is None
    assert serialise(t1) == "1,2,#,#,3,#,#"


# ============================================================
# FOLLOW-UP 2: Do it with O(1) extra space.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Morris traversal visits pre-order using
# temporary threads on right pointers, giving O(1) space,
# but it mutates the tree while running (restoring it
# after), so it is not safe for concurrent readers.
# Mention that trade-off.
def morris_preorder(root):
    # follow-up: O(1) extra space. A temporary link from
    # the
```

```
    # rightmost node of the left subtree back to the node
    # lets the walk climb up again. The tree is mutated
    # while
    # running, so the generator must be run to the end (it
    # removes every link on the way).
    node = root
    while node:
        if node.left is None:
            yield node.val
            node = node.right
            continue
        pred = node.left
        while pred.right and pred.right is not node:
            pred = pred.right
        if pred.right is None:
            yield node.val
            pred.right = node
            node = node.left
        else:
            pred.right = None
            node = node.right

# ---- follow-up 2: tests ----
def test_follow_up_2():
    t1 = TreeNode(1, TreeNode(2), TreeNode(3))
    # a different shape with the same pre-order: 1, 2, 3
    t2 = TreeNode(1, TreeNode(2, None, TreeNode(3)))
    assert list(morris_preorder(t2)) == [1, 2, 3]
    assert not same_preorder_constant_space(
        t1, TreeNode(1, TreeNode(9), TreeNode(3)))


# ============================================================
# FOLLOW-UP 3: They want identical trees, not just
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: They want identical trees (structure and
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
```

```
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def same_preorder_constant_space(a, b):
    # never stop early: that would leave links in the
    # trees
    same = True
    one, two = morris_preorder(a), morris_preorder(b)
    pairs = zip_longest(one, two, fillvalue=MISSING)
    for x, y in pairs:
        if x != y:
            same = False
    return same


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 20 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 2 of 9

Template fit: no update needed; return 1 + max(left, right) and read the value at the root | no date

EASY | IN APP LC 104 | Exponent | 20 of 288

Calculate the height of a binary tree.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 63/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: your app's interview approach (LeetCode #104)

LeetCode match: LC 104 Maximum Depth of Binary Tree (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/1153/calculate-height-binary-tree

QUESTION

Given the root of a binary tree, return its height: the number of nodes on the longest path from the root down to a leaf (an empty tree has height 0).

Function signature

def height(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`3`
Why	`Levels: [3], [9, 20], [15, 7]: height 3.`

Example 2	
Input	`[1, 2]`
Output	`2`
Why	`A root with one child.`

Example 3	
Input	`[]`
Output	`0`
Why	`An empty tree.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def height(node):
    if node is None:
        return 0
    left = height(node.left)
    right = height(node.right)
    return 1 + max(left, right)
```

BRUTE FORCE

TIME	SPACE
O(n)	O(h) call stack

WHY IT IS SLOW

Linear time, which is optimal; the call stack is as deep as the tree, so a chain of 100,000 nodes can overflow. Counting levels with a queue avoids recursion.

OPTIMAL SOLUTION · Level-order traversal counting levels

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Nodes or edges?

Height in nodes is what the code returns; in edges subtract one (an empty tree is -1 and a single node is 0). Say which you use before you code.

FOLLOW-UP 2 Diameter (the longest path between any two nodes).

Return the height from the recursion and update a global best with left + right at each node: one pass, O(n).

FOLLOW-UP 3 Is the tree balanced?

Return the height, or -1 as soon as a subtree is unbalanced or the heights differ by more than one, so the whole check is one O(n) pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Calculate the height of a binary tree.
#
# Given the root of a binary tree, return its height: the
# number of nodes on the longest path from the root down
# to a leaf (an empty tree has height 0).
#
# Signature:
#   def height(root: TreeNode) -> int
#
# Examples:
# [3, 9, 20, null, null, 15, 7]  ->  3
# [1, 2]  ->  2
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Nodes or edges?
# 2. Diameter (the longest path between any two nodes).
# 3. Is the tree balanced?
#
# INTERVIEW NOTES
# Height is defined recursively: 1 plus the larger child
# height, and 0 for an empty tree. Say whether you count
# nodes or edges. A level-order traversal gives the same
# number without recursion, which is the safe choice for a
# very deep tree.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Level-order traversal counting
# levels
# Idea: Process the tree one level at a time with a queue
# and count the levels. No recursion, so depth cannot
# overflow the stack; memory is the widest level.
```

```
def height(root):
    # height in NODES: level order, count the levels. An
    # empty tree is 0 and a single node is 1. (In edges,
    # subtract one: empty is -1, single node is 0.)
    levels, queue = 0, deque([root] if root else [])
    while queue:
        levels += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return levels

# ---- main question: tests ----
def test_main_question():
    root = TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(
15), TreeNode(7)))
    assert height(root) == 3
    assert height(None) == 0


# ============================================================
# FOLLOW-UP 1: Nodes or edges?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Height in nodes is what the code returns; in
# edges subtract one (an empty tree is -1 and a single
# node is 0). Say which you use before you code.
def height_edges(root):
    return height(root) - 1

# ============================================================
# FOLLOW-UP 2: Diameter (the longest path between any two
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Return the height from the recursion and update
# a global best with left + right at each node: one pass,
# O(n).
def diameter(root):
    # follow-up: the longest path between any two nodes,
    # in
    # edges. At every node the best path through it is
```

```
    # left + right; the recursion returns the node height.
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left, right = depth(node.left), depth(node.right)
        best = max(best, left + right)
        return 1 + max(left, right)

    depth(root)
    return best

# ============================================================
# FOLLOW-UP 3: Is the tree balanced?
# ============================================================

# ---- follow-up 3: function ----
# Answer: Return the height, or -1 as soon as a subtree is
# unbalanced or the heights differ by more than one, so
# the whole check is one O(n) pass.
def is_balanced(root):
    # follow-up: heights of the two sides differ by at
    # most
    # one at EVERY node. Return -1 as soon as a subtree is
    # unbalanced, so the whole check is one pass.
    def depth(node):
        if node is None:
            return 0
        left = depth(node.left)
        if left == -1:
            return -1
        right = depth(node.right)
        if right == -1 or abs(left - right) > 1:
            return -1
        return 1 + max(left, right)

    return depth(root) != -1

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
def slow_height(node):
    # oracle: the textbook recursion
```

```
    if node is None:
        return 0
    left = slow_height(node.left)
    right = slow_height(node.right)
    return 1 + max(left, right)


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(depth_left):
    # a random small tree for the tests
    import random
    if depth_left == 0 or random.random() < 0.2:
```

```
        return None
    below = depth_left - 1
    return TreeNode(0, build(below), build(below))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY PATH: question 21 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 3 of 9

Template fit: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see | asked 353 days ago

EASY | IN APP LC 199 | PracHub | Technical Screen | 21 of 288

Find right-side view of binary tree

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK ALL TIME 34.9 30 DAYS 0 PRIORITY 128/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #199 Binary Tree Right Side View

LeetCode match: LC 199 Binary Tree Right Side View

Posted on PracHub: 2025-11-01 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-right-side-view-of-binary-tree

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

EXAMPLES

Example 1	
Input	`[1, 2, 3, null, 5, null, 4]`
Output	`[1, 3, 4]`
Why	`Looking from the right you see 1, then 3 (hiding 2), then 4 (hiding 5).`

QUESTION

Example 2	
Input	`[1, 2]`
Output	`[1, 2]`
Why	`A left child is visible when there is no right child at that depth.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree shows nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def right_side_view(root):
    levels = level_order(root)  # list of lists
    return [level[-1] for level in levels]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Stores every node's value when only one per level is needed, O(n) space. A depth-first walk visiting the right child first and recording the first value at each depth needs O(h).

OPTIMAL SOLUTION · DFS visiting right before left, first node at each depth

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

FOLLOW-UP 2 Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

FOLLOW-UP 3 Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find right-side view of binary tree
#
# You are given the root of a binary tree. From the
# **right side** of the tree, at each depth you can see
# exactly one node: the rightmost node at that depth.
# Write a function that returns a list of the values of
# the nodes visible when looking at the tree from the
# right side, ordered from **top to bottom**. --- ###
# Function Signature (one possible form) text
# rightSideView(root: TreeNode) -> List[int] Where
# TreeNode is defined as: text class TreeNode { int val;
# TreeNode left; TreeNode right; } --- ### Example
# Consider this binary tree: text 1 / \ 2 3 \ \ 5 4 The
# right-side view is: text [1, 3, 4] Explain the algorithm
# you would use, including its time and space complexity.
# You may use either breadth-first search (BFS) or depth-
# first search (DFS).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Left side view, or both.
# 2. Avoid recursion.
# 3. Top view or bottom view.
#
# INTERVIEW NOTES
# The right side view is the last node of every level, or
# equivalently the first node you reach at each depth if
# you visit right children before left. Either way you
# need the depth of each node, so carry it in the
# traversal.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
def right_side_view(root):
    # DFS, right child first: the first node seen at each
    # depth is the rightmost one
    out = []

    def dfs(node, depth):
        if node is None:
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return out

# ---- main question: tests ----
def test_main_question():
    #     1
    #    / \
    #   2   3
    #    \   \
    #     5   4
    tree = TreeNode(1, TreeNode(2, None, TreeNode(5)),
                    TreeNode(3, None, TreeNode(4)))
    assert right_side_view(tree) == [1, 3, 4]


# ============================================================
# FOLLOW-UP 1: Left side view, or both.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Swap the order of the two recursive calls. For
# both views record the first and last node per depth in
# one pass.
def left_and_right_views(root):
    # both views in one pass: first and last node per
    # level
    left, right = [], []
    queue = deque([root] if root else [])
    while queue:
        left.append(queue[0].val)
        right.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return left, right

# ============================================================
# FOLLOW-UP 2: Avoid recursion.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use the BFS version and keep only the last node
# per level (no per-level list), or an explicit stack of
# (node, depth) pushing left before right so that right is
# popped first.
def right_side_view_bfs(root):
    # no recursion: keep only the LAST node of every level
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out

# ---- follow-up 2: tests ----
def test_follow_up_2():
    #     1
    #    / \
    #   2   3
    #    \   \
    #     5   4
    tree = TreeNode(1, TreeNode(2, None, TreeNode(5)),
```

```
                    TreeNode(3, None, TreeNode(4)))
    assert right_side_view_bfs(tree) == [1, 3, 4]
    views = left_and_right_views(tree)
    assert views == ([1, 2, 5], [1, 3, 4])
```

```
    assert right_side_view(None) == []
    # a left-only tree is still seen from the right
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert right_side_view(chain) == [1, 2, 3]


# ============================================================
# FOLLOW-UP 3: Top view or bottom view.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Track a horizontal distance per node (left minus
# one, right plus one) and keep the first (top) or last
# (bottom) node seen per distance in BFS order.
def top_and_bottom_view(root):
    # horizontal distance: left child -1, right child +1
    top, bottom = {}, {}
    queue = deque([(root, 0)] if root else [])
    while queue:
        node, hd = queue.popleft()
        # the first node seen at a distance is the top
        # one;
        # the last one seen is the bottom one
        top.setdefault(hd, node.val)
        bottom[hd] = node.val
        if node.left:
            queue.append((node.left, hd - 1))
        if node.right:
            queue.append((node.right, hd + 1))
    order = sorted(top)
    return ([top[h] for h in order],
            [bottom[h] for h in order])

# ---- follow-up 3: tests ----
def test_follow_up_3():
    #     1
    #    / \
    #   2   3
    #    \   \
    #     5   4
    tree = TreeNode(1, TreeNode(2, None, TreeNode(5)),
                    TreeNode(3, None, TreeNode(4)))
    top, bottom = top_and_bottom_view(tree)
    assert top == [2, 1, 3, 4] and bottom == [2, 5, 3, 4]
```

```


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def rightSideView(level_order: list[int | None]) -> list[i
nt]:
```

Sample call

```
rightSideView([1, 2, 3, None, 5, None, 4])
```

Test cases

Input	Expected
([1, 2, 3, None, 5, None, 4],)	[1, 3, 4]
([1, 2, 3, 4],)	[1, 3, 4]

STUDY PATH: question 22 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 4 of 9

Template fit: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed | asked 23 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 22 of 288

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 263/288**

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

A boolean expression tree has leaves holding True or False and internal nodes AND, OR, XOR (two children) or NOT (one child). For each leaf in turn, imagine flipping only that leaf and report the value of the whole tree after the flip (every flip is independent of the others). Return the leaves and the list of answers, in leaf order.

Function signature

def flip_answers(root) -> (list, list)

(the leaves, the root value

after flipping each one)

EXAMPLES

Example 1

Input	`t = Node('AND', Node(val=True), Node('OR', Node(val=False), Node(val=True)))`
	`evaluate(t)`
	`[leaf.val for leaf in flip_answers(t)[0]]`
	`flip_answers(t)[1]`

Output	`evaluate(t) -> true`
	`[leaf.val for leaf in flip_answers(t)[0]] -> [true, false, true]`
	`flip_answers(t)[1] -> [false, true, false]`

Why	`The tree is True AND (False OR True) = True. Flipping the first leaf gives False, flipping the False leaf changes nothing (the OR is still True), flipping the last leaf makes the OR False.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# A boolean expression tree has leaves holding True or
# False and internal nodes AND, OR, XOR (two children) or
# NOT (one child). For each leaf in turn, imagine flipping
# only that leaf and report the value of the whole tree
# after the flip (every flip is independent of the
# others). Return the leaves and the list of answers, in
# leaf order.
#
# Signature:
#   def flip_answers(root) -> (list, list)    # (the leave
s, the root value after flipping each one)
#
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves

# ---- main question: main function ----
# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers

# ---- main question: tests ----
def test_main_question():
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # every cached answer matches a full re-evaluation
    for leaf, ans in zip(leaves, answers):
        leaf.val = not leaf.val
        assert evaluate(tree) == ans
        leaf.val = not leaf.val
    # the tree is unchanged afterwards
    assert evaluate(tree) is False
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.
```

```
# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Which leaves change the answer at all?
```

```
# ============================================================

# ---- follow-up 3: function ----
# Answer: Compute top-down 'sensitivity': a leaf matters
# if every ancestor's other input lets it through (AND
# with true, OR with false, any XOR or NOT). One pass
# gives all of them in O(n).
# FOLLOW-UP 4: Which leaves can change the answer at all?
# Answer: Propagate a 'matters' flag downward: through AND
# the other side must be true, through OR the other side
# must be false, through XOR and NOT it always passes.
def sensitive_leaves(root):
    # leaves whose flip can change the root value
    _, value, _ = build_cache(root)
    found, stack = [], [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        l, r = node.left, node.right
        # the flag passes if the other side lets it
        left_ok = right_ok = matters
        if node.op == "AND":
            left_ok = matters and value[id(r)]
            right_ok = matters and value[id(l)]
        elif node.op == "OR":
            left_ok = matters and not value[id(r)]
            right_ok = matters and not value[id(l)]
        stack.append((l, left_ok))
        if r is not None:
            stack.append((r, right_ok))
    return found

# ---- follow-up 3: tests ----
def test_follow_up_3():
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)
    for leaf in leaves:
        leaf.val = not leaf.val
```

```
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in sensitive_leaves(tree))


# ============================================================
# FOLLOW-UP 4: Which leaves can change the answer at all?
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS used by the code below
```

```
def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()


def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(
        node.op,
        _eval_full(node.left),
        _eval_full(node.right),
    )


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
```

```
    return out


# HELPERS for the checks below
class Node:
    def __init__(self, op=None, left=None,
                 right=None, val=None):
        # op is AND, OR, XOR, NOT, or None for a leaf
        self.op, self.val = op, val
        self.left, self.right = left, right


def evaluate(node):
    # plain recursive evaluation (also the checker)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = evaluate(node.right) if node.right else None
    return combine(node.op, a, b)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

STUDY PATH: question 23 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 5 of 9

Template fit: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1 | asked 110 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 23 of 288

Longest Consecutive Path in Binary Tree

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 178/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest path that goes from a parent down to a child where every value is exactly 1 more than the previous one (counted in nodes).

Function signature

def longestConsecutive(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[1, null, 3, 2, 4, null, null, null, 5]`
Output	`3`
Why	`3 -> 4 -> 5 is the longest increasing-by-one downward path.`

Example 2	
Input	`[2, null, 3, 2, null, 1]`
Output	`2`
Why	`2 -> 3 is the longest; 3 -> 2 and 2 -> 1 go the wrong way.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def walk(node):
    best = 1
    for child in (node.left, node.right):
        if child and child.val == node.val + 1:
            best = max(best, 1 + walk(child))
    return best
answer = 0
for node in all_nodes(root):
    answer = max(answer, walk(node))
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION · Top-down DFS carrying the current run

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

FOLLOW-UP 2 Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

FOLLOW-UP 3 Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

FOLLOW-UP 4 The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

FOLLOW-UP 5 The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Consecutive Path in Binary Tree
#
# Given a binary tree, return the length of the longest
# path that goes from a parent down to a child where every
# value is exactly 1 more than the previous one (counted
# in nodes).
#
# Signature:
#   def longestConsecutive(root: TreeNode) -> int
#
# Examples:
# [1, null, 3, 2, 4, null, null, null, 5]  ->  3
# [2, null, 3, 2, null, 1]  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does the any-direction version return for a
# single node, or for duplicate values?
# 2. Can you avoid recursion?
# 3. Return the path itself, not only its length.
# 4. The path may go in any direction (LeetCode 549).
# 5. The values must increase by 1 or decrease by 1 along
# the path but not mix.
#
# INTERVIEW NOTES
# Each node needs to know the longest run that ends at its
# parent, so pass the current length down and reset it to
# 1 when the value is not parent + 1. For the bending
# version each node must report two lengths upward
# (increasing and decreasing chains going down), and the
# best path through a node combines one of each.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    # FOLLOW-UP 3: want_path=True returns the path itself
    return best_path if want_path else best

# ---- main question: tests ----
def test_main_question():
    root = TreeNode(1, None, TreeNode(3, TreeNode(2), TreeNode(4, None, TreeNode(5))))
    assert longest_consecutive(root, False) == 3


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 3.

# ============================================================
# FOLLOW-UP 4: The path may go in any direction (LeetCode
# ============================================================

# ---- follow-up 4: function ----
# Answer: Return the pair (longest increasing chain down,
# longest decreasing chain down) from each node, and
# update the global best with inc + dec - 1 at the node.
# Verified against exhaustive path enumeration for small
# trees.
def longest_any_direction(root):
    # LeetCode 549: the path may bend at one node
    if root is None:
        return 0
    order, stack = [], [root]
    while stack:
```

```
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    # inc[x] / dec[x]: longest chain going down from x
    inc, dec, best = {}, {}, 1
    # children are processed before their parents
    for node in reversed(order):
        up = down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                up = max(up, inc[child] + 1)
            if child.val == node.val - 1:
                down = max(down, dec[child] + 1)
        inc[node], dec[node] = up, down
        # the bend joins both chains (counted node once)
        best = max(best, up + down - 1)
    return best

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

STUDY PATH: question 24 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 6 of 9

Template fit: best = max(best, left + right); return 1 + max(left, right) (the diameter) | no date

MEDIUM | IN APP LC 543 | Exponent | 24 of 288

Given the root of a binary tree, return the length of the diameter of the tree.

ASKED (ANY CO.) ~3 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 44.4 | 30 DAYS 0 | PRIORITY 93/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: your app's interview approach (LeetCode #543)

LeetCode match: LC 543 Diameter of Binary Tree

Last asked (any company): 3 months ago, about 2026-07 (Exponent shows only a rough age) | +18 interviews

Original: https://www.aced.io/questions/2705/binary-tree-diameter-length

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online

round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

EXAMPLES

Example 1

Input	`[1, 2, 3, 4, 5]`
Output	`4`
Why	`The longest path 4 - 2 - 1 - 3 has 4 nodes.`

QUESTION

Example 2

Input	`[1]`
Output	`1`
Why	`A single node.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def diameter(root):
    best = 0
    for node in all_nodes(root):
        left = height(node.left)  # fresh walks
        right = height(node.right)
        best = max(best, left + right + 1)
    return best
```

TIME	SPACE
O(n^2)	O(h)
worst case	

WHY IT IS SLOW

Heights of the same subtrees are recomputed for every ancestor, O(n^2) on a skewed tree. One post-order pass returns heights and updates the best on the way, O(n).

OPTIMAL SOLUTION · One post-order pass returning heights and updating the best

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 An n-ary tree.

Take the two largest child heights at each node (not just left and right): best = max(best, top1 + top2). Sorting the children's heights costs O(d log d) per node, or track the top two in O(d).

FOLLOW-UP 2 Edges have weights, or you want the path itself.

Add the edge weight to each child's height before comparing; for the path, also store which child gave each of the two best heights and rebuild from the node that produced the best sum.

FOLLOW-UP 3 Count the diameter in nodes instead of edges.

Add one to the answer (a path of k edges has k + 1 nodes). State which convention the interviewer wants before you code.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given the root of a binary tree, return
# the length of the diameter of the tree.
#
# Given the root of a binary tree, create a function
# diameterOfTree that determines and returns the length of
# the diameter of the tree, where the diameter of a tree
# is defined as the number of nodes along the longest path
# between any two nodes in the tree.
#
# FOLLOW-UPS (answered below, in this order):
# 1. An n-ary tree.
# 2. Edges have weights, or you want the path itself.
# 3. Count the diameter in nodes instead of edges.
#
# INTERVIEW NOTES
# The longest path in a tree has a highest node, and the
# length of the best path through a node is the height of
# its left subtree plus the height of its right subtree.
# So compute heights bottom-up and, at every node, check
# left height + right height against a running best. One
# traversal does both jobs.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One post-order pass returning
# heights and updating the best
# Idea: depth(node) returns the height of the node's
# subtree and, on the way, updates best = max(best, left +
# right). Every node is visited once, and the answer is
# best when the traversal ends (edges, not nodes).
def diameter_nodes(root):
    # longest path between any two nodes, counted in
    # NODES.
    # Iterative post-order: height of every node, then the
```

```
    # best path through each node = left + right + 1.
    if root is None:
        return 0
    height, best, stack = {None: 0}, 0, [(root, False)]
    while stack:
        node, done = stack.pop()
        if not done:
            stack.append((node, True))
            for child in (node.left, node.right):
                if child:
                    stack.append((child, False))
            continue
        lh, rh = height[node.left], height[node.right]
        height[node] = 1 + max(lh, rh)
        best = max(best, lh + rh + 1)
    return best

# ---- main question: tests ----
def test_main_question():
    root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5
)), TreeNode(3))
    assert diameter_nodes(root) == 4


# ============================================================
# FOLLOW-UP 1: An n-ary tree.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Take the two largest child heights at each node
# (not just left and right): best = max(best, top1 +
# top2). Sorting the children's heights costs O(d log d)
# per node, or track the top two in O(d).
def diameter_nary(root):
    # n-ary tree: add the two tallest child heights at
    # each node
    if root is None:
        return 0
    height, best, stack = {}, 0, [(root, False)]
    while stack:
        node, done = stack.pop()
        if not done:
            stack.append((node, True))
            kids = node.children
            stack.extend((c, False) for c in kids)
            continue
        tops = sorted((height[c] for c in node.children),
                      reverse=True)[:2]
        height[node] = 1 + (tops[0] if tops else 0)
        best = max(best, 1 + sum(tops))
    return best

# ============================================================
# FOLLOW-UP 2: Edges have weights, or you want the path
# ============================================================

# ---- follow-up 2: function ----
# Answer: Add the edge weight to each child's height
# before comparing; for the path, also store which child
# gave each of the two best heights and rebuild from the
# node that produced the best sum.
def diameter_weighted(root):
    # edge weights: children are (weight, node) pairs;
    # return
    # the longest weighted path
    best, stack, down = 0, [(root, False)], {}
    while stack:
        node, done = stack.pop()
        if not done:
            stack.append((node, True))
            kids = node.children
            stack.extend((c, False) for _, c in kids)
            continue
        lengths = [w + down[c] for w, c in node.children]
        arms = sorted(lengths, reverse=True)[:2]
        down[node] = arms[0] if arms else 0
        best = max(best, sum(arms))
    return best
```

```
# ============================================================
# FOLLOW-UP 3: Count the diameter in nodes instead of
# ============================================================

# ---- follow-up 3: function ----
# Answer: Add one to the answer (a path of k edges has k +
# 1 nodes). State which convention the interviewer wants
# before you code.
def diameter_edges(root):
    # the other convention: a path of k edges has k + 1
    # nodes
    return max(diameter_nodes(root) - 1, 0)
```

```
# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


class NNode:
    def __init__(self, val, children=()):
        self.val, self.children = val, list(children)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree:

```
// tree
        1
       / \
      2   3
     / \
    4   5
```

The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.

STUDY PATH: question 25 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 7 of 9

Template fit: collect values by depth; reverse every odd level (or fill each level from the other end) | asked 445 days ago

MEDIUM | IN APP LC 103 | PracHub | Technical Screen | 25 of 288

Implement zigzag level-order traversal

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 38.7 | 30 DAYS 0 | PRIORITY 110/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #103 Binary Tree Zigzag Level Order Traversal

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/implement-zigzag-level-order-traversal

QUESTION

Given the root of a binary tree, return its values level by level, reading the first level from left to right, the next from right to left, and so on, alternating. The result is a list of lists, one per level. Aim for O(n) time.

Function signature

def zigzag_level_order(root: TreeNode) -> list

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`[[3], [20, 9], [15, 7]]`
Why	`Level 0 is [3], level 1 is read right to left [20, 9], level 2 left to right [15, 7].`

Example 2	
Input	`[1]`
Output	`[[1]]`
Why	`A single node.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree has no levels.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time, plus an extra pass copying the reversed levels. Filling each level from the left or right end of a deque as it is built needs no reversal.

OPTIMAL SOLUTION · BFS filling each level from the left or the right

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

FOLLOW-UP 2 Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

FOLLOW-UP 3 N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement zigzag level-order traversal
#
# Given the root of a binary tree, return its values level
# by level, reading the first level from left to right,
# the next from right to left, and so on, alternating. The
# result is a list of lists, one per level. Aim for O(n)
# time.
#
```

```
# Signature:
#   def zigzag_level_order(root: TreeNode) -> list
#
# Examples:
# [3, 9, 20, null, null, 15, 7]  ->  [[3], [20, 9], [15,
#     7]]
# [1]  ->  [[1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bottom-up level order (leaf level first).
# 2. Do it with DFS.
# 3. N-ary tree.
#
# INTERVIEW NOTES
# It is a plain level-order traversal where every second
# level is read in the other direction. Either collect
# each level and reverse the odd ones, or fill each level
# from the correct end of a deque so no reversal is
# needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
```

```
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
def zigzag_level_order(root):
    # BFS by levels; flip the direction on every level
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            if ltr:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        ltr = not ltr
    return out

# ---- main question: tests ----
def test_main_question():
    right = TreeNode(20, TreeNode(15), TreeNode(7))
    tree = TreeNode(3, TreeNode(9), right)
    want = [[3], [20, 9], [15, 7]]
    assert zigzag_level_order(tree) == want


# ============================================================
```

```
# FOLLOW-UP 1: Bottom-up level order (leaf level first).
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the normal traversal and reverse the list of
# levels at the end.
def level_order_bottom_up(root):
    # leaf level first: the normal traversal, reversed at
    # the end
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
```

```
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out[::-1]

# ============================================================
# FOLLOW-UP 2: Do it with DFS.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Pass the depth; append the value to
# levels[depth], at the end when the depth is even and at
# the front (insert at 0 or a deque) when odd. Recursion
# depth is the tree height.
def zigzag_dfs(root):
    # DFS: pass the depth; odd levels are filled at the
    # front
    levels = []

    def go(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append(deque())
        if depth % 2 == 0:
            levels[depth].append(node.val)
        else:
            levels[depth].appendleft(node.val)
        go(node.left, depth + 1)
        go(node.right, depth + 1)

    go(root, 0)
    return [list(x) for x in levels]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    right = TreeNode(20, TreeNode(15), TreeNode(7))
    tree = TreeNode(3, TreeNode(9), right)
    want = [[3], [20, 9], [15, 7]]
    assert zigzag_dfs(tree) == want
    up = level_order_bottom_up(tree)
    assert up == [[15, 7], [9, 20], [3]]
    assert zigzag_level_order(None) == []
    assert zigzag_dfs(None) == []
```

```
    kids = [NNode(2, [NNode(5)]), NNode(3), NNode(4)]
    root = NNode(1, kids)


# ============================================================
# FOLLOW-UP 3: N-ary tree.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Replace the two children by the list of
# children; the direction flag works the same way.
def zigzag_nary(root):
    # an n-ary tree: a list of children instead of two
```

```
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        vals = [n.val for n in queue]
        out.append(vals if ltr else vals[::-1])
        for _ in range(len(queue)):
            queue.extend(queue.popleft().children)
        ltr = not ltr
    return out

# ---- follow-up 3: tests ----
def test_follow_up_3():
    kids = [NNode(2, [NNode(5)]), NNode(3), NNode(4)]
    root = NNode(1, kids)
    assert zigzag_nary(root) == [[1], [4, 3, 2], [5]]


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


class NNode:
    def __init__(self, val, children=()):
        self.val, self.children = val, list(children)


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def zigzagLevelOrder(values: list[int | None]) -> list[lis
t[int]]:
```

Sample call

```
zigzagLevelOrder([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

STUDY PATH: question 26 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 8 of 9

Template fit: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value | asked 34 days ago

HARD | IN APP LC 1325 | PracHub | Technical Screen | 26 of 288

Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

POSTED 2026-09-16 | 34 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 10.6 | 30 DAYS 0 | PRIORITY 188/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1325 Delete Leaves With a Given Value

LeetCode match: LC 1325 Delete Leaves With a Given Value

Posted on PracHub: 2026-09-16 (updated 2026-09-20)

Original: https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

EXAMPLES

Example 1

Input	`tree = [1, 2, 3, 2, null, 2, 4], target = 2 and tree = [1, 3, 3, 3, 2], target = 3`
Output	`values(remove_leaves(tree([1, 2, 3, 2, None, 2, 4]), 2)[0]) -> [1, null, 3, null, 4]` `values(remove_leaves(tree([1, 3, 3, 3, 2]), 3)[0]) -> [1, 3, null, null, 2]`
Why	`remove_leaves returns (the new root, how many leaves were removed). Deleting a leaf can turn its parent into a leaf, which is deleted too if it also equals the target: in the first tree the 2s go and the root's left branch disappears entirely.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def delete_pass(node):
    if node is None:
        return None, 0
    node.left, a = delete_pass(node.left)
    node.right, b = delete_pass(node.right)
    if is_leaf(node) and node.val == target:
        return None, a + b + 1
    return node, a + b
while True:
    root, removed = delete_pass(root)
    if removed == 0:
        break
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

BRUTE FORCE

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

OPTIMAL SOLUTION · One post-order pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

FOLLOW-UP 2 Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. **Cost:** one post-order pass, O(n) time and O(h) space.

FOLLOW-UP 3 Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. **Cost:** O(1) extra, counted inside the same O(n) pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Repeatedly Delete Leaves Matching a
# Target Value From a Binary Tree
#
# Given a binary tree and an integer target, delete every
# leaf whose value equals target. Deleting a leaf can turn
# its parent into a leaf; if that parent's value also
# equals target, it must be deleted as well, and so on,
# until no leaf with value target remains. Return the
# resulting tree, which is empty if the root itself ends
# up deleted. ### Function Signature
# remove_target_leaves(tree: list[int | None], target:
# int) -> list[int | None] ### Tree Encoding The input and
# the output use the same level-order encoding: - An empty
# tree is []. - Otherwise the first element is the root's
# value. Present nodes are then visited in breadth-first
# order, left to right, and for each one its left child
# and then its right child are appended, using None for an
# absent child. Children of absent nodes are not listed. -
# Trailing None values are omitted.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a 100,000-node chain. How do you avoid a
# stack overflow?
# 2. Delete any node whose whole subtree sums to zero, or
# whose value matches a predicate.
# 3. Return how many nodes were deleted.
#
# INTERVIEW NOTES
# 'Repeatedly' hides an order: a node becomes deletable
# only after its children are processed. That is a post-
# order dependency, so a single bottom-up pass does what
# many passes would do. Whenever a rule says 'repeat until
# nothing changes' on a tree, ask whether children-first
# order collapses it to one pass.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node

    return dfs(root), deleted

# ---- main question: tests ----
def test_main_question():
    assert remove_target_leaves([1, 2, 3, 2, None, 2, 4],
2) == [1, None, 3, None, 4]


# ============================================================
# FOLLOW-UP 1: The tree is a 100,000-node chain. How do
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect nodes in pre-order with an explicit
```

```
# stack and remember each node's parent, then walk that
# list in reverse (children before parents) and unlink
# matching leaves. This keeps the post-order guarantee
# without recursion.
def remove_leaves_iterative(root, target):
    # no recursion: safe for a 100,000-node chain
    if root is None:
        return None, 0
    parent, order, stack = {root: None}, [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    # reversed order: children are handled before parents
    for node in reversed(order):
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(values):
    # level-order list with None for missing children
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        # next two values are this node's children
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


def to_list(root):
    # the same encoding, with trailing Nones removed
    out, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        out.append(node.val if node else None)
        if node:
            queue.append(node.left)
            queue.append(node.right)
    while out and out[-1] is None:
        out.pop()
    return out


def remove_target_leaves(values, target):
    root, _ = remove_leaves(build(values), target)
    return to_list(root)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def remove_target_leaves(tree: list[int | None], target: i
nt) -> list[int | None]:
```

Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

Test cases

Input	Expected
([5, 7, 7, 7, 3, None, 7], 7)	[5, 7, None, None, 3]
([3, 3, 1], 3)	[3, None, 1]

STUDY PATH: question 27 of 288 | topic 3 of 43: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward | 9 of 9

Template fit: post-order: free the children first, then the node itself | asked 386 days ago | frequency Single report

HARD | IN APP LC 145 | ColorStack | technical interview | 27 of 288

Delete an Entire Binary Tree

ASKED 2025-09-29 | 386 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 204/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

Optimal answer from: your app's interview approach (LeetCode #145)

LeetCode match: LC 145 Binary Tree Postorder Traversal (free every node)

Frequency: Single report | Last asked: 2025-09-29 | Roles: SWE Intern

QUESTION

You are given the root of a binary tree (not a BST); each node has only left and right child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a post-order traversal. Implement delete_tree(root) that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

Follow-up (as reported): Recursive DFS uses O(h) stack space. Achieve O(1) extra space by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

EXAMPLES

Example 1

Input	`root = 1 / \ 2 3`
Output	`[2, 3, 1]`
Why	`Post-order frees the left child, then the right child, then the root.`

QUESTION

Example 2

Input	`root = None`
Output	`[]`
Why	`An empty tree frees nothing.`

Example 3

Input	`root = 1 / 2 / 3`
Output	`[3, 2, 1]`
Why	`A left-leaning chain frees deepest-first.`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 .
- Node values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_post_order(root):
    out, stack = [], [root] if root else []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    return out[::-1]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It stores every node twice (the work stack and the output).

OPTIMAL SOLUTION · Recursive post-order

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it with O(1) extra space.

Rotate instead of recursing: while the root has a left child, rotate it up (the left child becomes the root and the old root becomes its right child). When the root has no left child, free it and move to its right child. Every node is freed once and no stack is used; the order is no longer post-order.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Delete an Entire Binary Tree
#
# You are given the root of a binary tree (not a BST);
# each node has only left and right child pointers. You
# must "free" every node so that after the call the tree
# is empty. Because a node must not be freed before its
# children, the correct freeing order is a post-order
# traversal. Implement delete_tree(root) that frees all
# nodes and returns the list of node values in the exact
# order they are freed (post-order: left subtree, right
# subtree, then node). Function signature: from typing
# import Optional class TreeNode: def __init__(self, val:
# int = 0, left: "Optional[TreeNode]" = None, right:
# "Optional[TreeNode]" = None) -> None: self.val = val
# self.left = left self.right = right def
# delete_tree(root: Optional[TreeNode]) -> list[int]:
# """Free every node and return the values in the order
# freed (post-order)."""  ... Follow-up (as reported):
# Recursive DFS uses O(h) stack space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with O(1) extra space.
#
# INTERVIEW NOTES
# A node may only be freed after its children, which is
# exactly post-order: left subtree, right subtree, then
# the node. Recursion uses O(h) stack; two stacks or an
# explicit stack avoid recursion. A constant-space version
# rotates left children up until the root has no left
# child, frees the root, and moves right.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
def delete_tree(root):
    freed = []

    def free(node):
        if node is None:
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = node.right = None

    free(root)
    return freed

# ---- main question: tests ----
def test_main_question():
    random.seed(35)
    def build(shape):
        if shape is None:
            return None
        val, l, r = shape
        return TreeNode(val, build(l), build(r))
    def random_shape(depth, counter):
        if depth == 0 or random.random() < 0.25:
            return None
        counter[0] += 1
        return (
            counter[0],
            random_shape(depth - 1, counter),
            random_shape(depth - 1, counter),
        )
```

```
    assert delete_tree(
        build((1, (2, None, None), (3, None, None)))
    ) == [2, 3, 1]
    assert delete_tree(None) == []
    assert delete_tree(
        build((1, (2, (3, None, None), None), None))
    ) == [3, 2, 1]


# ============================================================
# FOLLOW-UP 1: Do it with O(1) extra space.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Rotate instead of recursing: while the root has
# a left child, rotate it up (the left child becomes the
# root and the old root becomes its right child). When the
# root has no left child, free it and move to its right
# child.
def delete_tree_constant_space(root):
    freed = []
    while root:
        if root.left:
            left = root.left
            root.left = left.right
            left.right = root
            root = left
        else:
            nxt = root.right
            freed.append(root.val)
            root.right = None
            root = nxt
    return freed


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 4 OF 43 |

A heap of what is available: pop the best, put back the changed one

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

8	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A heap of what is available: pop the best, put back the changed one | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

M	28. Streaming Median (LC 295) with Follow-ups	1Point3Acres	2026-08-03
M	29. Compute minimum number of rooms needed	PracHub	2026-02-12
M	30. Implement a Reusable Locker Allocation System	PracHub	2026-08-14
M	31. Simulate room assignments for scheduled meetings	PracHub	2026-01-06
M	32. Trip Expense Settlement	HackTheRounds	no date
M	33. Playlist Shuffle No Adjacent Artists	HackTheRounds	no date
M	34. Minimum Makespan on Identical CPUs	ColorStack	2025-09-23
M	35. Design server allocation for multi-type nodes	PracHub	2025-09-06
H	36. Huffman Tree Construction	1Point3Acres	2026-06-01

CODE TEMPLATE | GROUP 4 OF 43 | 9 QUESTIONS, ONE PATTERN

A heap of what is available: pop the best, put back the changed one

The one idea these questions share

The heap holds the things you can choose from right now: machines and when they are free, rooms, lockers, the biggest remaining count, the two smallest weights. Each step pops the best choice, uses it, and pushes it back changed (or a combined item).

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def run_jobs(jobs, workers):
    free = [0] * workers                        # CHANGE 1: what is available (times, ids, counts)
    heapq.heapify(free)
    finish = 0
    for ready, duration in sorted(jobs):
        start = max(ready, heapq.heappop(free))         # CHANGE 2: take the best available
        heapq.heappush(free, start + duration)          # CHANGE 3: put it back, changed
        finish = max(finish, start + duration)
    return finish
```

2. Helper functions that usually get written next to it

```
def pop_two_push_sum(weights):              # the Huffman step, also the generic "combine two best"
    heap = list(weights)
    heapq.heapify(heap)
    total = 0
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        heapq.heappush(heap, a + b)
        total += a + b
    return total


class MaxHeap:                              # heapq only has a min-heap: negate on the way in and out
    def __init__(self):
        self.h = []

    def push(self, x):
        heapq.heappush(self.h, -x)

    def pop(self):
        return -heapq.heappop(self.h)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
28	Streaming Median (LC 295) with Follow-ups	two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced
29	Compute minimum number of rooms needed	heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room
30	Implement a Reusable Locker Allocation System	heap of freed locker numbers; take the smallest freed one before handing out a new number
31	Simulate room assignments for scheduled meetings	a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

Contents Start the questions >

Master contents (all topics)

#	Question	What you change in the template
32	Trip Expense Settlement	a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back
33	Playlist Shuffle No Adjacent Artists	pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count
34	Minimum Makespan on Identical CPUs	the template as written: heap of the times the CPUs become free
35	Design server allocation for multi-type nodes	a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)
36	Huffman Tree Construction	pop the two smallest weights, push their sum as a new node; repeat until one node remains

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 28 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 1 of 9

Template fit: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced | asked 78 days ago | frequency Single report

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 28 of 288

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 0 | PRIORITY 79/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

Design a structure that receives numbers one at a time and can report the median of everything received so far at any moment. addNum(x) adds a number; findMedian() returns the middle value, or the mean of the two middle values when the count is even.

Function signature

class MedianFinder:
  def addNum(self, x: float) -> None
  def findMedian(self) -> float

EXAMPLES

Example 1

Input	`m = MedianFinder()`
	`m.addNum(5)`
	`m.addNum(2)`
	`m.findMedian()`
	`m.addNum(8)`
	`m.findMedian()`
	`m.addNum(1)`
	`m.findMedian()`

Output	`m.findMedian() -> 3.5`
	`m.findMedian() -> 5`
	`m.findMedian() -> 3.5`

Why	`Sorted so far: [2, 5] -> 3.5; [2, 5, 8] -> 5; [1, 2, 5, 8] -> 3.5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n)	**O(n)**
per insert (list shift), O(1) median	

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n)	**O(n)**
add, O(1) median	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Streaming Median (LC 295) with Follow-ups
#
# Design a structure that receives numbers one at a time
# and can report the median of everything received so far
# at any moment. addNum(x) adds a number; findMedian()
# returns the middle value, or the mean of the two middle
# values when the count is even.
#
# Signature:
#   class MedianFinder:
#       def addNum(self, x: float) -> None
#       def findMedian(self) -> float
#
# FOLLOW-UPS (answered below, in this order):
# 1. Almost every value is in a small range (say 0 to
# 100). Can you do better than two heaps?
# 2. Now support an arbitrary k-th smallest, not only the
# median.
# 3. What if the data stream is too large to store?
# 4. The median of only the last K values.
#
# INTERVIEW NOTES
# Keep the data split at the median: a max-heap for the
# smaller half and a min-heap for the larger half, with
# sizes equal or differing by one. Then the median is read
# from the tops. Insert by pushing through one heap into
# the other to keep every smaller value on the left, and
# rebalance the sizes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
```

```
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2

# ---- main question: tests ----
def test_main_question():
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.add(x)
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5


# ============================================================
# FOLLOW-UP 1: Almost every value is in a small range (say
# ============================================================

# ---- follow-up 1: function ----
# Answer: Yes: a counting array gives O(1) inserts, and
# the median needs one pass over at most V counts, so O(V)
# per query with V fixed. Keep a moving pointer to the
# median bucket to make queries amortized O(1).
class CountingMedian:
```

```
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value

    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2

# ---- follow-up 1: tests ----
def test_follow_up_1():
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
    assert c.median() == 3.5
```

```
# ============================================================
# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# ============================================================

# ---- follow-up 2: function ----
# Answer: Two heaps only work for one fixed rank. Use an
# order-statistics structure: a Fenwick tree over
# (compressed) values, or a balanced BST augmented with
# subtree sizes.
class Fenwick:
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
```

```
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos

# ---- follow-up 2: tests ----
def test_follow_up_2():
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]


# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: The median of only the last K values.
# ============================================================

# ---- follow-up 4: function ----
# Answer: Keep the window in a deque (to know what
# expires) and a sorted list (to read the middle); insert
# and remove with bisect. O(K) per update because of list
# shifting; two heaps with lazy deletion give O(log K).
class WindowMedian:
    """Follow-up: median of only the last k values."""
```

```
    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2

# ---- follow-up 4: tests ----
def test_follow_up_4():
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

STUDY PATH: question 29 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 2 of 9

Template fit: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room | asked 250 days ago

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 29 of 288

★ TOP 75: rank 50 in the Medium and Hard priority list

★ Compute minimum number of rooms needed

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 49/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/compute-minimum-number-of-rooms-needed

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of **n** pairs `[start, end]` where `0 <= start <= end`.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- `1 <= n <= 200000`
- Times are integers in a reasonable range (e.g., `0 . 10^9`).

Example

- Input: `[[0,30],[5,10],[15,20]]`
- Output: **2** (One room can host `[0,30]`, the other can host `[5,10]` then `[15,20]`.)

QUESTION

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.

Constraints

0 <= n <= 2000000 <= start <= end <= 10^9

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and
    [15,20], so at most 2 rooms are needed at the
    same time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). A min-heap of room end times, or sorted starts and ends with two pointers, is O(n log n).

OPTIMAL SOLUTION · Sort by start with a min-heap of end times

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

FOLLOW-UP 2 Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

FOLLOW-UP 3 Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute minimum number of rooms needed
#
# ## Problem You are given a list of meetings, each with a
# start time and end time. A single room can host only one
# meeting at a time. Two meetings **overlap** if one
# starts before the other ends (treat meetings as half-
# open intervals **[start, end)** so a meeting ending at
# time t does not conflict with another starting at time
# t). ### Task Return the **minimum number of rooms**
# required to host all meetings. ### Input - intervals: an
# array of n pairs [start, end] where 0 <= start <= end.
# ### Output - An integer: the minimum number of rooms
# needed. ### Constraints (typical interview bounds) - 1
# <= n <= 200000 - Times are integers in a reasonable
# range (e.g., 0 . 10^9).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can one person attend all meetings (LeetCode 252)?
# 2. Return which room each meeting uses.
# 3. Meetings arrive online.
#
# INTERVIEW NOTES
# The rooms needed equal the largest number of meetings
# that overlap at one moment. Two ways to find that
# without checking every moment: sort by start and keep a
# min-heap of the end times of rooms in use (reuse the
# room that frees up first), or sort starts and ends
# separately and sweep with two pointers.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
import heapq


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push).
def min_meeting_rooms(intervals):
    # half-open [start, end): a meeting ending at t does
    # not
    # clash with one starting at t. A heap holds the end
    # times
    # of the rooms in use.
    ends = []
    meetings = sorted(m for m in intervals if m[0] < m[1])
    for start, end in meetings:
        if ends and ends[0] <= start:
            # the earliest room is free: reuse it
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)

# ---- main question: tests ----
def test_main_question():
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert min_meeting_rooms(meetings) == 2
    assert min_meeting_rooms([(7, 10), (2, 4)]) == 1
    # touching meetings share a room; empty ones need none
    touching = [(1, 3), (3, 5), (4, 4)]
    assert min_meeting_rooms(touching) == 1
    assert min_meeting_rooms([]) == 0


# ============================================================
# FOLLOW-UP 1: Can one person attend all meetings
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort by start and check that each meeting ends
# no later than the next one starts. O(n log n), O(1)
# extra.
def can_attend_all(intervals):
    # LeetCode 252: one person, no overlap at all
    ordered = sorted(intervals)
    pairs = zip(ordered, ordered[1:])
    return all(a[1] <= b[0] for a, b in pairs)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert not can_attend_all(meetings)
    assert can_attend_all([(7, 10), (2, 4)])


# ============================================================
# FOLLOW-UP 2: Return which room each meeting uses.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a heap of (end time, room id): reuse the
# room that frees first, otherwise open the next id, and
# record the id per meeting.
```

```
def assign_rooms(intervals):
    # which room does each meeting use?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [None] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms

# ---- follow-up 2: tests ----
def test_follow_up_2():
    meetings = [(0, 30), (5, 10), (15, 20)]
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]


# ============================================================
# FOLLOW-UP 3: Meetings arrive online.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Keep the end-time heap between calls: pop every
# end at or before the new start (those rooms are free),
```

```
# then push the new end. The heap size is the number of
# rooms in use right now; the peak is the answer so far.
class RoomTracker:
    """Meetings arrive online (in order of start time)."""

    def __init__(self):
        self.ends, self.peak = [], 0

    def book(self, start, end):
        # rooms whose meeting already ended are free again
        while self.ends and self.ends[0] <= start:
            heapq.heappop(self.ends)
        heapq.heappush(self.ends, end)
        # the heap size is the rooms in use right now
        self.peak = max(self.peak, len(self.ends))
        return len(self.ends)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    meetings = [(0, 30), (5, 10), (15, 20)]
    t = RoomTracker()
    in_use = [t.book(s, e) for s, e in meetings]
    assert in_use == [1, 2, 2]
    assert t.peak == 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0,30],[5,10],[15,20]])
```

Test cases

Input	Expected
`([[0,30],[5,10],[15,20]],)`	`2`
`([[7,10],[2,4],[4,7],[10,12]],)`	`1`

ALSO REPORTED · 4 more

Find Minimum Rooms Needed | PracHub | Technical Screen

https://prachub.com/coding-questions/find-minimum-rooms-needed

Compute minimum rooms for time intervals | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-rooms-for-time-intervals

Given a list of car rental requests with start and end times, assign cars to rentals such | Exponent

https://www.aced.io/questions/5425/assign-cars-minimize-total-cars-rented

Meeting Rooms / Interval Scheduling Variants | 1Point3Acres | phone-screen, tech-screen, onsite-coding | asked 2026-09-20 | Very high

https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.
The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

Preparation: Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY PATH: question 30 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 3 of 9

Template fit: heap of freed locker numbers; take the smallest freed one before handing out a new number | asked 67 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 30 of 288

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 8.1 30 DAYS 0 PRIORITY 191/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

QUESTION

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	**O(lockers released and not reused)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. **Cost:** O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAN QUESTION: Implement a Reusable Locker Allocation
# System
#
# ## Problem A locker bank has lockers numbered from 1
# through n. Process operations in order. store(customer)
# assigns the smallest available locker and returns its
# number, or -1 if that customer already has a locker or
# none is available. pickup(customer) releases and returns
# that customer's locker, or -1 if no locker is assigned.
# last() returns the locker number from the most recent
# successful store, or -1 if there has not been one.
# Return one result per operation. ### Constraints &
# Assumptions - 1 <= n <= 200,000. - There are at most
# 200,000 operations. - Customer IDs are non-empty
# strings. - A released locker becomes immediately
# available for reuse. ### Clarifications - Failed store
# and pickup operations do not change last(). - The
# allocation rule is always the smallest currently free
# locker number. - A customer can store again after a
# successful pickup.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are 10^9 lockers.
# 2. Many threads call store and pickup.
# 3. Lockers of different sizes.
#
# INTERVIEW NOTES
# 'Smallest free number' with numbers coming back means a
# min-heap of returned numbers. Numbers that were never
# handed out do not need to be stored: keep a counter for
# the next unused number and use the heap first. That
# keeps memory proportional to the number of lockers in
# use, not to n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
        return locker
```

```
    def last(self):
        # failed operations never change this
        return self.last_store

# ---- main question: tests ----
def test_main_question():
    ops = [["store", "a"], ["store", "b"],
           ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, ops) == [1, 2, 1, 1, 1]
    # nothing stored yet, and a missing pickup
    empty = [["last"], ["pickup", "x"]]
    assert process(2, empty) == [-1, -1]
    # full, and a customer who already has a locker
    full = [["store", "a"], ["store", "a"],
            ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]
    # failed operations leave last() alone
    more = full + [["last"]]
    assert process(2, more)[-1] == 2
    # a huge locker bank costs nothing up front
    one = [["store", "a"], ["last"]]
    assert process(10 ** 9, one) == [1, 1]


# =============================================================
# FOLLOW-UP 1
# =============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# =============================================================
# FOLLOW-UP 2: Many threads call store and pickup.
# =============================================================

# ---- follow-up 2: function ----
# Answer: Protect the heap, the counter and the customer
# map with one lock so a locker number is never handed out
# twice (or fine-grained: a lock around the (heap,
# counter) pair).
class SafeLockers(Lockers):
    """Many threads: one lock around every operation."""

    def __init__(self, n):
        import threading
        super().__init__(n)
```

```
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)

    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    s = SafeLockers(2)
    assert s.store("a") == 1 and s.pickup("a") == 1


# =============================================================
# SMALL HELPERS THE TESTS USE (they run after everything above is defined)
# =============================================================
# HELPERS for the checks below
def process(n, operations):
    box, out = Lockers(n), []
    for op in operations:
        if op[0] == "store":
            out.append(box.store(op[1]))
```

```
        elif op[0] == "pickup":
            out.append(box.pickup(op[1]))
        else:
            out.append(box.last())
    return out


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[list[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])	[1, 2, 1, 1, 1]
(5, [])	[]

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

STUDY PATH: question 31 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 4 of 9

Template fit: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later | asked 287 days ago

MEDIUM | IN APP LC 2402 | PracHub | Technical Screen | 31 of 288

Simulate room assignments for scheduled meetings

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40 | 30 DAYS 0 | PRIORITY 109/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: your app's interview approach (LeetCode #2402)

LeetCode match: LC 2402 Meeting Rooms III (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/simulate-room-assignments-for-scheduled-meetings

QUESTION

There are n rooms numbered 0 to n - 1 and a list of meetings (start, end). Meetings are handled in order of start time. A meeting takes the lowest-numbered free room; if every room is busy it waits until the room that frees up first becomes free and then runs for its full original length, so it ends later. Return the room that hosted the most meetings (the lowest room number on a tie).

Function signature

def mostBookedRoom(n: int, meetings: list) -> int

EXAMPLES

Example 1	
Input	`2, [[0, 10], [1, 5], [2, 7], [3, 4]]`
Output	`0`
Why	`Rooms 0 and 1 each host two meetings; the lower number wins the tie.`

Example 2	
Input	`3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]`
Output	`1`
Why	`With three rooms, room 1 hosts two of the five meetings and no room hosts more.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(n, meetings):
    end = [0] * n
    count = [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n)
                if end[r] <= start]
        if free:
            r = min(free)
            end[r] = finish
        else:
            r = end.index(min(end))  # earliest
            wait = end[r] - start
            end[r] = finish + wait
        count[r] += 1
    return count.index(max(count))
```

TIME	SPACE
O(M * N)	O(N)

WHY IT IS SLOW

Every meeting scans all N rooms, O(M * N). Two heaps (free rooms and busy rooms by end time) make each meeting O(log N).

OPTIMAL SOLUTION · Two heaps: free rooms and busy rooms

TIME	SPACE
O(M log M + M log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

FOLLOW-UP 2 How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

FOLLOW-UP 3 Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Simulate room assignments for scheduled
# meetings
#
# There are n rooms numbered 0 to n - 1 and a list of
# meetings (start, end). Meetings are handled in order of
# start time. A meeting takes the lowest-numbered free
# room; if every room is busy it waits until the room that
# frees up first becomes free and then runs for its full
# original length, so it ends later. Return the room that
# hosted the most meetings (the lowest room number on a
# tie).
#
# Signature:
#   def mostBookedRoom(n: int, meetings: list) -> int
#
# Examples:
# 2, [[0, 10], [1, 5], [2, 7], [3, 4]]  ->  0
# 3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if meetings arrive online and cannot be
# reordered?
# 2. How do you report the busiest room by total busy time
# instead of meeting count?
# 3. Rank rooms by total busy time instead of the number
# of meetings.
#
# INTERVIEW NOTES
# Simulate in start order with two questions per meeting:
# which rooms are free now (lowest index first), and if
# none, which room frees up first (the meeting is delayed
# but keeps its length). Two heaps answer both quickly:
# one of free room ids, one of (end time, room id) for
# busy rooms.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
# FOLLOW-UP 3: Rank rooms by total busy time instead of
# the number of meetings.
# Answer: Add each meeting's duration (its original
# length, not the delay) to the room's tally; the choice
# of room is unchanged.
```

```
def most_booked_room(n, meetings, by="count"):
    # a meeting takes the lowest-numbered free room; if
    # none is
    # free it waits for the earliest room and keeps its
    # length
    free = list(range(n))
    heapq.heapify(free)
    # busy rooms: (time it becomes free, room)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        # rooms that finished by now are free again
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            # delayed: starts when the earliest room frees
            # up
            finish, room = heapq.heappop(busy)
            delayed_end = finish + (end - start)
            heapq.heappush(busy, (delayed_end, room))
        # FOLLOW-UP 3: by="time" ranks by total busy time
        # count meetings, or total length (not the delay)
        used[room] += 1 if by == "count" else end - start
    # ties go to the lowest room number
    return used.index(max(used))
```

```
# ---- main question: tests ----
def test_main_question():
    assert most_booked_room(2, [[0, 10], [1, 5], [2, 7],
                                [3, 4]]) == 0
    assert most_booked_room(3, [[1, 20], [2, 10], [3, 5],
                                [4, 9], [6, 8]]) == 1
    # a single room takes everything
    assert most_booked_room(1, [[0, 1], [0, 1]]) == 0
    # by busy time instead of the number of meetings
    meetings = [[0, 100], [1, 2], [2, 3], [3, 4]]
    assert most_booked_room(2, meetings, "count") == 1
    assert most_booked_room(2, meetings, "time") == 0


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Rank rooms by total busy time instead of
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are given n meeting rooms labeled 0..n-1 and a list of meetings meetings, where each meeting is [start, end] with start < end. Meetings are processed in increasing order of start time.

Assign each meeting to a room using these rules:

If one or more rooms are free at start , assign the meeting to the free room with the smallest index .
If no rooms are free at start , the meeting must wait until a room becomes free. It is then scheduled in the room that becomes available earliest (break ties by smallest room index). If it waits, its duration stays the same , so if the original duration is end - start and the room becomes free at time t , the meeting runs on [t, t + (end - start)] .

Each time a meeting is assigned to a room (even after waiting), that room’s meeting count increases by 1.

Task

Return the index of the room that hosted the most meetings. If there is a tie, return the smallest index.

Input/Output
Input: n (int), meetings (list of [start,end])
Output: an integer room index
Constraints (reasonable interview assumptions)
1 <= n <= 10^5
1 <= len(meetings) <= 10^5
Times fit in 32-bit signed integer; waiting can increase end times beyond original but still fits in 64-bit.

ALSO REPORTED · 1 more

Assign meetings to rooms with delays | PracHub | Technical Screen

https://prachub.com/interview-questions/assign-meetings-to-rooms-with-delays

STUDY PATH: question 32 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 5 of 9

Template fit: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back | no date

MEDIUM | IN APP LC 465 | HackTheRounds | Onsite | 32 of 288

Trip Expense Settlement

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 161/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #465 Optimal Account Balancing

LeetCode match: LC 465 (Optimal Account Balancing)

QUESTION

A group of friends go on a trip and each friend may pay for others. Every record gives the payer, the amount and the list of people who share it equally (the payer may be one of them). Compute a small set of payments that settles everyone's debts: each payment is {payer, amount, payee}.

Function signature

def settle_greedy(records: list) -> list

EXAMPLES

Example 1

Input	`trip = [{'payer': 'alice', 'amount': 4000, 'payees': ['bob', 'jess', 'alice', 'sam']}, {'payer': 'jess', 'amount': 2000, 'payees': ['jess', 'alice']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'sam', 'amount': 1000, 'payee': 'alice'}]`
Why	`Alice paid 4000 shared four ways and owes 1000 of Jess's 2000; Bob and Sam each owe 1000, and Jess is even. Two payments settle everything.`

Example 2

Input	`trip = [{'payer': 'alice', 'amount': 3000, 'payees': ['alice', 'bob', 'cy']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'cy', 'amount': 1000, 'payee': 'alice'}]`
Why	`Bob and Cy each owe Alice their 1000 share.`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6

QUESTION

- 1 <= payees.length <= 100

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def settle(i):
    while i < n and net[i] == 0:
        i += 1  # already settled
    if i == n:
        return 0
    best = INF
    for j in range(i + 1, n):
        if net[j] * net[i] < 0:  # opposite sign
            net[j] += net[i]  # move balance i
            best = min(best, 1 + settle(i + 1))
            net[j] -= net[i]  # undo
    return best
```

TIME	SPACE
O(n!) worst case	O(n)

WHY IT IS SLOW

It tries every pairing, up to n! in the worst case. Matching the largest creditor with the largest debtor (heaps) is O(n log n) and uses at most n - 1 payments, though not always the minimum.

OPTIMAL SOLUTION · Heaps: match the largest debtor with the largest creditor

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do you need the true minimum number of payments?

That is NP-hard in general (it is the zero-sum partition problem). The greedy answer is at most n - 1 payments and is usually accepted; for exact answers use the bitmask DP above (about 20 people) or backtracking with pruning.

FOLLOW-UP 2 Amounts that do not divide evenly (cents).

Work in integer cents and decide where the remainder goes (to the payer, or round-robin among payees). Never use floats for money, and make the rule deterministic.

FOLLOW-UP 3 Different shares per person instead of equal split.

Replace the equal share with an explicit amount per payee in each record; the net balance per person is computed the same way.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Trip Expense Settlement
#
# A group of friends go on a trip and each friend may pay
# for others. Every record gives the payer, the amount and
# the list of people who share it equally (the payer may
# be one of them). Compute a small set of payments that
# settles everyone's debts: each payment is {payer,
# amount, payee}.
#
# Signature:
#   def settle_greedy(records: list) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do you need the true minimum number of payments?
# 2. Amounts that do not divide evenly (cents).
# 3. Different shares per person instead of equal split.
#
# INTERVIEW NOTES
# First reduce the records to one number per person: net
# balance = paid - owed. Only that matters; who owed whom
# originally is irrelevant. Then settling is moving money
# between people with positive and negative balances.
# Matching largest creditor with largest debtor is quick
# and uses at most n - 1 payments, but only searching
# groups of people who sum to zero gives the true minimum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
def balances(records):
    # net balance per person: what they paid minus what
```

```
    # they
    # owe. Positive: the group owes them. All sum to zero.
    net = {}
    for rec in records:
        payer = rec["payer"]
        net[payer] = net.get(payer, 0) + rec["amount"]
        for person, share in shares_of(rec).items():
            net[person] = net.get(person, 0) - share
    return {p: v for p, v in net.items() if v}


# FOLLOW-UP 2: Amounts that do not divide evenly (cents).
# Answer: Work in integer cents and decide where the
# remainder goes (to the payer, or round-robin among
# payees). Never use floats for money, and make the rule
# deterministic.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_equally(amount, people):
    # follow-up: cents that do not divide evenly. Work in
    # whole cents; the leftover cents go to the first
    # people
    # in sorted order, so the rule is always the same.
    people = sorted(people)
    base, extra = divmod(amount, len(people))
    return {p: base + (1 if i < extra else 0)
            for i, p in enumerate(people)}


# FOLLOW-UP 3: Different shares per person instead of
# equal split.
# Answer: Replace the equal share with an explicit amount
# per payee in each record; the net balance per person is
# computed the same way.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def shares_of(record):
    # follow-up: explicit shares per person instead of
    # equal
    payees = record["payees"]
    if isinstance(payees, dict):
        if sum(payees.values()) != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Heaps: match the largest debtor
# with the largest creditor
# Idea: Push creditors (positive net) in a max-heap and
# debtors (negative net) in another. Repeatedly pay
# min(debt, credit) from the biggest debtor to the biggest
# creditor, and push back whoever still has a remainder.
def settle_greedy(records):
    # the biggest debtor pays the biggest creditor, as
    # much
    # as one side can: at most n - 1 payments
    net = balances(records)
    owed = [(-v, p) for p, v in net.items() if v > 0]
    debt = [(v, p) for p, v in net.items() if v < 0]
    heapq.heapify(owed)
    heapq.heapify(debt)
    out = []
    while owed and debt:
        o_amt, creditor = heapq.heappop(owed)
        d_amt, debtor = heapq.heappop(debt)
        pay = min(-o_amt, -d_amt)
        out.append({"payer": debtor, "amount": pay,
                    "payee": creditor})
        if -o_amt > pay:
            heapq.heappush(owed, (o_amt + pay, creditor))
        if -d_amt > pay:
            heapq.heappush(debt, (d_amt + pay, debtor))
    return out

# ---- main question: tests ----
def test_main_question():
```

```
    trip = [{"payer": "alice", "amount": 4000,
             "payees": ["bob", "jess", "alice", "sam"]},
            {"payer": "jess", "amount": 2000,
             "payees": ["jess", "alice"]}]
    got = settle_greedy(trip)
    want = [
        {"payer": "bob", "amount": 1000,
         "payee": "alice"},
        {"payer": "sam", "amount": 1000,
         "payee": "alice"}]
    assert sorted(got, key=lambda p: p["payer"]) == want
    assert balances(trip) == {"alice": 2000, "bob": -1000,
                              "sam": -1000}
```

```
    # 10.00 split three ways: 3.34 + 3.33 + 3.33
    assert split_equally(1000, ["c", "a", "b"]) == {
        "a": 334, "b": 333, "c": 333}
    custom = [{"payer": "x", "amount": 90,
               "payees": {"x": 10, "y": 30, "z": 50}}]
    owed = {"x": 80, "y": -30, "z": -50}
    assert balances(custom) == owed


# ========================================================
# FOLLOW-UP 1: Do you need the true minimum number of
# ========================================================

# ---- follow-up 1: function ----
# Answer: That is NP-hard in general (it is the zero-sum
# partition problem). The greedy answer is at most n - 1
# payments and is usually accepted; for exact answers use
# the bitmask DP above (about 20 people) or backtracking
# with pruning.
def min_payments_exact(values):
    # follow-up: the true minimum. People split into
    # groups
    # that settle among themselves (sum zero); a group of
    # g
    # people needs g - 1 payments. So find the most
    # groups:
    # a DP over subsets of people.
    n = len(values)
    total = [0] * (1 << n)
    best = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        rest = mask & (mask - 1)
        total[mask] = total[rest] + values[low]
        top = max((best[mask ^ (1 << i)]
                   for i in range(n) if mask >> i & 1))
        best[mask] = top + (1 if total[mask] == 0 else 0)
    return n - best[(1 << n) - 1]


# Answer: That is NP-hard in general (it is the zero-sum
# partition problem). The greedy answer is at most n - 1
# payments and is usually accepted; for exact answers use
# the bitmask DP above (about 20 people) or backtracking
# with pruning.
```

```
def min_payments_search(values):
    # the same answer by backtracking: settle the first
    # open
    # debt against every opposite one
    values = [v for v in values if v]

    def go(start):
        while start < len(values) and values[start] == 0:
            start += 1
        if start == len(values):
            return 0
        best = float("inf")
        seen = set()
        for j in range(start + 1, len(values)):
            if values[j] * values[start] < 0 and (
                    values[j] not in seen):
                seen.add(values[j])
                values[j] += values[start]
                best = min(best, 1 + go(start + 1))
                values[j] -= values[start]
        return best

    return go(0)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_payments_exact([5, -5]) == 1
    assert min_payments_exact([3, -3, 2, -2]) == 2
    assert min_payments_search([3, -3, 2, -2]) == 2
    for _ in range(100):
        names = "abcdef"
        records = []
        for _ in range(random.randint(1, 5)):
            count = random.randint(1, 4)
            who = random.sample(names, count)
            cents = random.randint(1, 500)
            records.append({"payer": random.choice(names),
                            "amount": cents,
                            "payees": who})
        net = balances(records)
        assert sum(net.values()) == 0
        pays = settle_greedy(records)
        assert apply(pays, net)
        assert len(pays) <= max(len(net) - 1, 0)
        values = list(net.values())
```

```
        exact = min_payments_exact(values)
        assert exact == min_payments_search(values)
        assert exact <= len(pays)


# ========================================================
# FOLLOW-UP 2: Amounts that do not divide evenly (cents).
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ========================================================
# FOLLOW-UP 3: Different shares per person instead of
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ========================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ========================================================
# HELPERS for the checks below
def apply(payments, net):
    # check: after the payments every balance is zero
    left = dict(net)
    for pay in payments:
        who, whom = pay["payer"], pay["payee"]
        left[who] = left.get(who, 0) + pay["amount"]
        left[whom] = left.get(whom, 0) - pay["amount"]
    return all(v == 0 for v in left.values())


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Compute minimal transfers to settle group expenses | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimal-transfers-to-settle-group-expenses

---

STUDY PATH: question 33 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 6 of 9

Template fit: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count | no date

MEDIUM | HackTheRounds | Phone Screen | 33 of 288

Playlist Shuffle No Adjacent Artists

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 127/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #767 Reorganize String

LeetCode match: LC 767 / LC 1054

QUESTION

Given the artist of every song in a playlist, return an order of the artists in which as few neighbours as possible are by the same artist: with no repeats next to each other whenever that is possible.

Function signature

def shuffle_no_adjacent(artists: list) -> list

EXAMPLES

Example 1

Input	`['a', 'a', 'b', 'b', 'c']`
Output	`['a', 'b', 'a', 'b', 'c']`
Why	Two a's and two b's alternate; c fits wherever it likes.

Example 2

Input	`['a', 'a', 'a', 'b']`
Output	`['a', 'b', 'a', 'a']`
Why	Three songs by 'a' cannot all be separated by a single 'b': the best is to keep two a's together at the end.

Example 3

Input	`['a', 'a']`
Output	`['a', 'a']`
Why	A single artist: nothing can be done.

CONSTRAINTS

- 1 <= songs.length <= 10^4

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(songs):
    best, best_pairs = None, INF
    for order in permutations(songs):
        pairs = 0
        for a, b in zip(order, order[1:]):
            if a.artist == b.artist:
                pairs += 1
        if pairs < best_pairs:
            best, best_pairs = order, pairs
    return best
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. A greedy that always plays the artist with the most songs left (not the one that just played), using a heap, is O(n log A).

OPTIMAL SOLUTION · Greedy with a max-heap, holding back the last artist

TIME	SPACE
O(n log A)	O(A)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The shuffle should look random, not always the same arrangement.

Break ties between equally frequent artists randomly and pick the song within the artist at random, as in the code. The result is valid but not uniform among all valid shuffles; a uniform sampler needs counting arrangements.

FOLLOW-UP 2 Songs by the same artist must be at least K apart.

Same idea with a waiting queue of length K: after playing an artist, hold them for K - 1 more songs before they are eligible again (the LeetCode 358 construction).

FOLLOW-UP 3 What is the minimum number of unavoidable repeats?

With n songs and a dominant artist with m songs, the other n - m songs separate at most n - m + 1 groups, so at least m - (n - m + 1) = 2m - n - 1 neighbouring pairs of that artist are forced (if positive).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Playlist Shuffle No Adjacent Artists
#
# Given the artist of every song in a playlist, return an
# order of the artists in which as few neighbours as
# possible are by the same artist: with no repeats next to
# each other whenever that is possible.
#
# Signature:
#   def shuffle_no_adjacent(artists: list) -> list
#
# Examples:
# ['a', 'a', 'b', 'b', 'c']  ->  ['a', 'b', 'a', 'b', 'c']
# ['a', 'a', 'a', 'b']  ->  ['a', 'b', 'a', 'a']
#
# FOLLOW-UPS (answered below, in this order):
# 1. The shuffle should look random, not always the same
# arrangement.
# 2. Songs by the same artist must be at least K apart.
# 3. What is the minimum number of unavoidable repeats?
#
# INTERVIEW NOTES
# A perfect arrangement exists exactly when the most
# frequent artist has at most ceil(n / 2) songs. Build it
# greedily: always play from the artist with the most
# songs left, except the artist who just played. A max-
# heap gives the most frequent artist; hold the previous
# artist back for one step. If it is impossible, the
# leftover songs of the dominant artist are the only
# unavoidable adjacencies.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def release(waiting, heap):
    # the oldest waiting artist may play again (if it has
    # any song left)
    entry = waiting.popleft()
    if entry[0]:
        heapq.heappush(heap, entry)


def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist
    # that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0]:
            heapq.heappush(heap, entry)
            return

# ---- main question: main function ----
# BASE QUESTION (optimal): Greedy with a max-heap, holding
# back the last artist
# Idea: Keep a heap of (remaining songs, artist). Each
# step pop the artist with the most songs left; play one
# of their songs, and push them back only after the next
# artist has played (so they cannot repeat).
# FOLLOW-UP 1: The shuffle should look random, not always
# the same arrangement.
# Answer: Break ties between equally frequent artists
# randomly and pick the song within the artist at random,
# as in the code. The result is valid but not uniform
# among all valid shuffles; a uniform sampler needs
# counting arrangements.
# FOLLOW-UP 2: Songs by the same artist must be at least K
# apart.
# Answer: Same idea with a waiting queue of length K:
# after playing an artist, hold them for K - 1 more songs
# before they are eligible again (the LeetCode 358
# construction).
def shuffle_playlist(songs, gap=2, strict=False,
                     rng=random):
    # songs are (artist, title). The artist with the most
    # songs left plays next and then waits `gap - 1` other
    # songs before it may play again (gap 2 = never twice
    # in
```

```
    # a row). Ties between artists and the song picked
    # inside
    # an artist are random, so every run looks different.
    by_artist = {}
    for artist, title in songs:
        by_artist.setdefault(artist, []).append(title)
    for titles in by_artist.values():
        # FOLLOW-UP 1: random song order inside an artist
        rng.shuffle(titles)
    # FOLLOW-UP 1: random tie-break between artists
    heap = [(-len(t), rng.random(), a)
            for a, t in by_artist.items()]
    heapq.heapify(heap)
    waiting, out = deque(), []
    while len(out) < len(songs):
        if not heap:
            # everyone left is still waiting
            if strict:
                return None
            take_oldest(waiting, heap)
        left, _, artist = heapq.heappop(heap)
        out.append((artist, by_artist[artist].pop()))
        # an artist stays in the queue even with no songs
        # left: it still marks one position of spacing
        waiting.append((left + 1, rng.random(), artist))
        # FOLLOW-UP 2: gap=K keeps an artist out for K - 1
        # songs
        if len(waiting) >= gap:
            release(waiting, heap)
    return out

# ---- main question: tests ----
def test_main_question():
    rng = random.Random(7)
    songs = [("A", "song1"), ("A", "song2"),
             ("B", "song3"), ("B", "song4"),
             ("C", "song5")]
    mixed = shuffle_playlist(songs, rng=rng)
    assert sorted(mixed) == sorted(songs)
    assert adjacent_pairs(mixed) == 0
    # 4 songs by A and 1 by B: 2 touching pairs are forced
```

```
    heavy = [("A", str(i)) for i in range(4)]
    heavy.append(("B", "x"))
    out = shuffle_playlist(heavy, rng=rng)
    assert adjacent_pairs(out) == forced_repeats([4, 1])
```

```
    assert adjacent_pairs(out) == 2
    strict = shuffle_playlist(heavy, strict=True, rng=rng)
    assert strict is None
    assert shuffle_playlist([], rng=rng) == []
    # at least 3 apart: three artists, two songs each
    trio = [(a, a + str(i))
            for a in "ABC" for i in range(2)]
    far = shuffle_playlist(trio, 3, True, rng)
    assert sorted(far) == sorted(trio)
    for i, (artist, _) in enumerate(far):
        recent = far[max(0, i - 2):i]
        assert all(other != artist for other, _ in recent)
    # runs differ from each other but are all valid
    seen = {tuple(shuffle_playlist(songs, rng=rng))
            for _ in range(30)}
    assert len(seen) > 1
    for _ in range(150):
        size = random.randint(1, 7)
        songs = [(random.choice("ABC"), str(i))
                 for i in range(size)]
        counts = [sum(1 for a, _ in songs if a == x)
                  for x in "ABC"]
        got = shuffle_playlist(songs, rng=rng)
        assert sorted(got) == sorted(songs)
        pairs = adjacent_pairs(got)
        assert pairs == forced_repeats(counts)


# ============================================================
# FOLLOW-UP 1: The shuffle should look random, not always
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Songs by the same artist must be at least K
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: What is the minimum number of unavoidable
# ============================================================

# ---- follow-up 3: function ----
# Answer: With n songs and a dominant artist with m songs,
# the other n - m songs separate at most n - m + 1 groups,
```

```
# so at least m - (n - m + 1) = 2m - n - 1 neighbouring
# pairs of that artist are forced (if positive).
def forced_repeats(counts):
    # follow-up: the fewest unavoidable neighbouring
    # pairs.
    # The other songs split the dominant artist into at
    # most
    # (n - m + 1) groups, so 2m - n - 1 pairs touch.
    n, m = sum(counts), max(counts)
    return max(0, 2 * m - n - 1)

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS used by the code below
def adjacent_pairs(playlist):
    pairs = zip(playlist, playlist[1:])
    return sum(1 for a, b in pairs if a[0] == b[0])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 34 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 7 of 9

Template fit: the template as written: heap of the times the CPUs become free | asked 392 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 34 of 288

Minimum Makespan on Identical CPUs

ASKED 2025-09-23 | 392 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 221/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1834 Single-Threaded CPU

LeetCode match: LC 1834 Single-Threaded CPU (related: identical CPUs)

Frequency: Single report | Last asked: 2025-09-23 | Roles: SWE Intern

QUESTION

You have C identical CPUs and a list release_times , where release_times[i] is the earliest time job i may start. Every job runs non-preemptively for the same fixed duration S . When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time. Scheduling optimally, compute the minimum possible makespan: the time at which the last job finishes.

Function signature

def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:

...

Follow-up (as reported): (Reported) The naive approach tracked an O(n*C) table of CPU availability; reduce it to a min-heap of the C next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

EXAMPLES

Example 1

Input	`release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3`
Output	`7`
Why	`CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6). CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7). Last finish = 7.`

Example 2

Input	`release_times = [0, 0, 0], num_cpus = 1, duration = 2`
Output	`6`
Why	`One CPU runs them back to back: [0,2), [2,4), [4,6).`

QUESTION

Example 3

Input	`release_times = [5], num_cpus = 3, duration = 4`
Output	`9`
Why	`The single job starts at its release time 5, finishes at 9.`

CONSTRAINTS

- 1 <= C <= 10^5
- 0 <= len(release_times) <= 10^5
- 0 <= release_times[i] <= 10^9
- 1 <= S <= 10^9
- If there are no jobs, the makespan is 0 .
- Target complexity: O(n log C) time, O(C) space.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import product
def brute_makespan(release, cpus, duration):
    best = float("inf")
    jobs = sorted(release)
    for assign in product(
        range(cpus), repeat=len(jobs)
    ):
        free, last = [0] * cpus, 0
        for r, cpu in zip(jobs, assign):
            free[cpu] = (
                max(free[cpu], r) + duration
            )
            last = max(last, free[cpu])
        best = min(best, last)
    return best if jobs else 0
```

TIME	SPACE
O(C^n)	O(n)

WHY IT IS SLOW

It enumerates all C^n ways to distribute the jobs; each CPU runs its jobs in release order.

OPTIMAL SOLUTION · Sorted jobs and a heap of free times

TIME	SPACE
O(n log n)	O(C)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The first idea tracked every CPU's availability in a list. Why is the heap better?

With a list you scan all C CPUs for the earliest free one on every job: O(n * C). A min-heap returns the earliest free time in O(log C) and updates in O(log C), so the total is O(n log C) after the sort.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimum Makespan on Identical CPUs
#
# You have C identical CPUs and a list release_times ,
# where release_times[i] is the earliest time job i may
# start. Every job runs non-preemptively for the same
# fixed duration S . When a job's release time arrives it
# may start on any free CPU; if all CPUs are busy it waits
# and starts on the first CPU that becomes free (never
# before its release time). Each CPU runs one job at a
# time. Scheduling optimally, compute the minimum possible
# makespan: the time at which the last job finishes.
# Function signature: def min_makespan(release_times:
# list[int], num_cpus: int, duration: int) -> int: ...
# Follow-up (as reported): (Reported) The naive approach
# tracked an O(n*C) table of CPU availability; reduce it
# to a min-heap of the C next-free times, popping the
# earliest-free CPU per job after sorting jobs by release
# time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The first idea tracked every CPU's availability in a
# list. Why is the heap better?
#
# INTERVIEW NOTES
# Jobs have equal duration, so the greedy order is release
# order: sort by release time and give each job the CPU
# that frees up first. Only the C free times matter, so
# keep them in a min-heap instead of scanning an O(n * C)
# table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Sorted jobs and a heap of free
# times
# Idea: Each job takes the CPU that frees first, but never
# starts before its release.
def min_makespan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    last = 0
    for r in sorted(release_times):
        start = max(r, heapq.heappop(free))
        heapq.heappush(free, start + duration)
        last = max(last, start + duration)
    return last

# ---- main question: tests ----
def test_main_question():
    random.seed(22)
    assert min_makespan([0, 1, 2, 3], 2, 3) == 7
    assert min_makespan([0, 0, 0], 1, 2) == 6
    assert min_makespan([5], 3, 4) == 9


# ============================================================
# FOLLOW-UP 1: The first idea tracked every CPU's
# ============================================================

# ---- follow-up 1: function ----
# Answer: With a list you scan all C CPUs for the earliest
# free one on every job: O(n * C). A min-heap returns the
# earliest free time in O(log C) and updates in O(log C),
# so the total is O(n log C) after the sort.
def min_makespan_scan(
    release_times, num_cpus, duration
):
    free = [0] * num_cpus
    for r in sorted(release_times):
        i = free.index(min(free))
        free[i] = max(r, free[i]) + duration
    return max(free) if release_times else 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 35 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 8 of 9

Template fit: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion) | asked 409 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 35 of 288

Design server allocation for multi-type nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 280/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match; related LC 1912 Design Movie Rental System

Posted on PracHub: 2025-09-06 (updated 2026-08-05)

Original: https://prachub.com/interview-questions/design-server-allocation-for-multi-type-nodes

QUESTION

Design a service that manages a pool of server nodes. Each node supports several instance types. getServer(count, types) hands out up to count servers that together cover all the requested types, preferring the least flexible servers (the ones that support the fewest types) so that versatile servers stay free, and returns None (allocating nothing) if the request cannot be met. releaseServer(ids) returns servers to the pool.

Function signature

class Servers:

def __init__(self, servers: dict)

id -> set of instance types

def getServer(self, count: int, types: list) -> list | None

def releaseServer(self, ids: list) -> None

EXAMPLES

Example 1

Input	`s = Servers({1: {'a'}, 2: {'a', 'b'}, 3: {'b'}, 4: {'a', 'b', 'c'}})`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(1, ['a'])`
	`s.releaseServer([1])`
	`s.getServer(1, ['a'])`

Output	`s.getServer(2, ['a', 'b']) -> [1, 3]`
	`s.getServer(2, ['a', 'b']) -> [2, 4]`
	`s.getServer(1, ['a']) -> null`
	`s.getServer(1, ['a']) -> [1]`

Why	The first request takes the two least flexible servers (1 for a, 3 for b); the second takes 2 and 4; nothing is left for the third; after server 1 is released it can serve type a again.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def get_server(count, types):
    chosen = []
    for t in types:
        pool = [s for s in free if t in s.types
                and s not in chosen]
        chosen.append(min(pool, key=flexibility))
    rest = [s for s in free if s not in chosen
            and s.types & set(types)]
    chosen += rest[:count - len(chosen)]
    if len(chosen) < count:
        return None  # not enough
    return chosen
```

TIME	SPACE
O(N * T) per request (N servers)	O(N)

WHY IT IS SLOW

Every request scans all N servers, O(N * types). Per-type heaps ordered by flexibility with lazy deletion make a request O((count + types) log N).

OPTIMAL SOLUTION · Per-type heaps ordered by flexibility, with lazy deletion

TIME	SPACE
O((count + types) log N) amortized per request	O(total type memberships)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

FOLLOW-UP 2 Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

FOLLOW-UP 3 Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

FOLLOW-UP 4 A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design server allocation for multi-type
# nodes
#
# Design a service that manages a pool of server nodes.
# Each node supports several instance types.
# getServer(count, types) hands out up to count servers
# that together cover all the requested types, preferring
# the least flexible servers (the ones that support the
# fewest types) so that versatile servers stay free, and
# returns None (allocating nothing) if the request cannot
# be met. releaseServer(ids) returns servers to the pool.
#
# Signature:
#   class Servers:
#       def __init__(self, servers: dict)      # id -> set
 of instance types
#       def getServer(self, count: int, types: list) -> li
st | None
#       def releaseServer(self, ids: list) -> None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many clients call at the same time.
# 2. Clients keep asking for rare types and starve.
# 3. Servers fail while allocated.
# 4. A client dies without releasing its servers.
#
# INTERVIEW NOTES
# A request mentions a few types, so index the free
# servers by type and avoid scanning all 100,000. To keep
# versatile servers available for harder requests, always
# prefer the server that supports the fewest types.
# Allocation touches every type index of a server, so use
# lazy deletion: mark it allocated and let the stale heap
# entries be skipped when they surface.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import defaultdict


# ============================================================
```

```
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it.
# FOLLOW-UP 4: A client dies without releasing its
# servers.
# Answer: Give every allocation a lease (expiry time). A
# periodic sweep returns servers whose lease has expired
# to the free pool (the lazy-deletion indexes make a
# release cheap), and clients renew the lease while they
# are alive.
class ServerPool:
    """Servers with several instance types. Each type has
    a heap of (number of types, id): the LEAST flexible
    server goes first, which keeps versatile ones free."""

    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {s: frozenset(t)
                      for s, t in servers.items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        # FOLLOW-UP 4: a dead client's lease expires
        # lease expiry time of every allocated server
        self.lease = {}
        for s, ts in self.types.items():
            for t in ts:
                heap = self.heaps[t]
                heapq.heappush(heap, (len(ts), s))

    def best_for(self, t, skip):
        # the best free server of type t that is not in
        # `skip`;
        # allocated entries are dropped lazily
        heap, held = self.heaps[t], []
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
```

```
            else:
                break
        best = heap[0] if heap else None
        # put back the entries we only looked past
        for item in held:
            heapq.heappush(heap, item)
        return best

    def get_server(self, count, types):
        # pick servers first, commit only if the whole
        # request works
        chosen = []
        for t in types:
            # a server already chosen may cover this type
            if any(t in self.types[c] for c in chosen):
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        # fill up to `count` with the least flexible free
        # servers
        while len(chosen) < count:
            skip = set(chosen)
            found = [self.best_for(t, skip)
                     for t in types]
            options = [o for o in found if o]
            if not options:
                return None
            chosen.append(min(options)[1])
```

```
        # more servers than allowed were needed to cover
        # the types
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def release(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                self.lease.pop(s, None)
                flex = len(self.types[s])
                for t in self.types[s]:
                    heap = self.heaps[t]
```

```
                    heapq.heappush(heap, (flex, s))

    def rent(self, count, types, now, ttl):
        # a client that dies must not hold servers forever
        got = self.get_server(count, types)
        for s in got or []:
            self.lease[s] = now + ttl
        return got

    def expire(self, now):
        # return servers whose lease ran out to the free
        # pool
        late = [s for s in self.lease
                if self.lease[s] <= now]
        self.release(late)
        return late

# ---- main question: tests ----
def test_main_question():
    pool = ServerPool({1: {"gpu"}, 2: {"gpu", "big"},
                       3: {"big"}, 4: {"cpu"}})
    # the least flexible server for each type is used
    # first
    assert pool.get_server(2, ["gpu", "big"]) == [1, 3]
    # the versatile server 2 is still there for a combined
    # need
    assert pool.get_server(1, ["gpu", "big"]) == [2]
    # nothing is left for a gpu request
    assert pool.get_server(1, ["gpu"]) is None
    # a request that cannot fit in `count` servers changes
    # nothing
    assert pool.get_server(1, ["cpu", "gpu"]) is None
    assert pool.get_server(1, ["cpu"]) == [4]
    pool.release([2])
    assert pool.get_server(1, ["gpu"]) == [2]
    # leases: a dead client's servers come back
    p2 = ServerPool({1: {"a"}})
    assert p2.rent(1, ["a"], now=0, ttl=10) == [1]
    assert p2.get_server(1, ["a"]) is None
    assert p2.expire(now=5) == []
    assert p2.expire(now=10) == [1]
    assert p2.get_server(1, ["a"]) == [1]


# ============================================================
```

```
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: A client dies without releasing its
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

STUDY PATH: question 36 of 288 | topic 4 of 43: A heap of what is available: pop the best, put back the changed one | 9 of 9

Template fit: pop the two smallest weights, push their sum as a new node; repeat until one node remains | asked 141 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 36 of 288

Huffman Tree Construction

ASKED 2026-06-01 | 141 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 249/288

ABOUT THIS QUESTION

Topic: A heap of what is available: pop the best, put back the changed one

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

QUESTION

Given symbol frequencies, build the Huffman code: repeatedly merge the two least frequent items into one until a single tree is left; the path from the root gives each symbol its bit string. Return the code of every symbol. To make the answer unique, the codes are assigned canonically: shorter codes first, then alphabetical order.

Function signature

def huffman_codes(freq: dict) -> dict

EXAMPLES

Example 1	
Input	{'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}
Output	{'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a': '1110', 'b': '1111'}
Why	The frequent symbol f gets a 1-bit code; the rare ones get longer codes.

Example 2	
Input	{'x': 3, 'y': 8}
Output	{'x': '0', 'y': '1'}
Why	Two symbols always get one bit each.

Example 3	
Input	{'z': 10}
Output	{'z': '0'}
Why	A single symbol still needs a code of one bit.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]  # two smallest
        items = items[2:] + [a + b]
        cost += a + b
    return cost
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

OPTIMAL SOLUTION · Min-heap merge of the two lightest nodes

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

FOLLOW-UP 2 How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

FOLLOW-UP 3 Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

FOLLOW-UP 4 Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Huffman Tree Construction
#
# Given symbol frequencies, build the Huffman code:
# repeatedly merge the two least frequent items into one
# until a single tree is left; the path from the root
# gives each symbol its bit string. Return the code of
# every symbol. To make the answer unique, the codes are
# assigned canonically: shorter codes first, then
# alphabetical order.
#
# Signature:
#   def huffman_codes(freq: dict) -> dict
#
# Examples:
# {'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}  ->
#     {'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a':
#     '1110', 'b': '1111'}
# {'x': 3, 'y': 8}  ->  {'x': '0', 'y': '1'}
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the greedy choice optimal?
# 2. How do you make the code canonical, and why would
# you?
# 3. Time complexity if symbols arrive already sorted?
# 4. Make the codes canonical so only the code lengths
# need to be stored.
#
# INTERVIEW NOTES
# The two rarest symbols belong at the deepest level as
# siblings, so merge them first, treat the merged node as
# a new symbol with the combined weight, and repeat. 'Take
# the two smallest' is the job of a min-heap; tie-breaking
# with a counter keeps the heap from comparing tree nodes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth


# FOLLOW-UP 2: How do you make the code canonical, and why
# would you?
# Answer: Keep only each symbol's code length from the
# tree, sort symbols by (length, symbol), and assign
# consecutive binary codes. A decoder then needs just the
# lengths rather than the whole tree, which is how DEFLATE
# stores its tables.
# FOLLOW-UP 4: Make the codes canonical so only the code
# lengths need to be stored.
# Answer: Sort symbols by (code length, symbol) and assign
# consecutive binary numbers, shifting left when the
# length grows. A decoder rebuilds the codes from the
# lengths alone (this is how DEFLATE stores its tables).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def canonical_codes(lengths):
```

```
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out

# ---- main question: main function ----
# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))

# ---- main question: tests ----
def test_main_question():
    freq = {"a": 5, "b": 9, "c": 12,
            "d": 13, "e": 16, "f": 45}
    lengths = huffman_lengths(freq)
    # the most frequent symbol is next to the root
    assert lengths["f"] == 1
    # total cost of the optimal tree
    assert sum(freq[s] * lengths[s] for s in freq) == 224
```

```
    codes = huffman_codes(freq)
    # prefix-free: no code starts another code
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)
    text = "abfadc"
    bits = "".join(codes[ch] for ch in text)
    assert decode(bits, codes) == text
    items = sorted(freq.items(), key=lambda x: x[1])


# ====================================================
# FOLLOW-UP 1
# ====================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ====================================================
# FOLLOW-UP 2: How do you make the code canonical, and why
# ====================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ====================================================
# FOLLOW-UP 3: Time complexity if symbols arrive already
# ====================================================

# ---- follow-up 3: function ----
# Answer: Two-queue method: one queue holds the sorted
# leaves, the other the merged nodes (which are produced
# in nondecreasing order). Always pop the smaller front of
# the two queues. That is O(n) after sorting.
def lengths_two_queues(sorted_items):
    # input already sorted by weight: O(n) with two queues
    from collections import deque
    leaves = deque((w, [s]) for s, w in sorted_items)
    merged = deque()
    depth = {s: 0 for s, _ in sorted_items}

    def pop_smaller():
        # the smaller front of the two queues
        if not merged or (leaves and
                          leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        w1, a = pop_smaller()
        w2, b = pop_smaller()
        for s in a + b:
            depth[s] += 1
        merged.append((w1 + w2, a + b))
    return depth

# ---- follow-up 3: tests ----
def test_follow_up_3():
    freq = {"a": 5, "b": 9, "c": 12,
            "d": 13, "e": 16, "f": 45}
    items = sorted(freq.items(), key=lambda x: x[1])
    two = lengths_two_queues(items)
    assert sum(freq[s] * two[s] for s in freq) == 224
```

```
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}


# ====================================================
# FOLLOW-UP 4: Make the codes canonical so only the code
# ====================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ====================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ====================================================
# HELPERS for the checks below
def decode(bits, codes):
    # read bits until they match a code (prefix-free)
    back = {v: k for k, v in codes.items()}
    out, cur = [], ""
    for b in bits:
        cur += b
        if cur in back:
            out.append(back[cur])
            cur = ""
    return "".join(out)


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 5 OF 43 |

Recurrence on a position or a range (memo or table)

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6	3
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Recurrence on a position or a range (memo or table) | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

M	37. Target Expressions from Ordered Digits	1Point3Acres	2026-08-13
M	38. Count Word Decompositions into Chemical Element Symbols	1Point3Acres	2026-07-14
M	39. Find the longest palindromic subsequence using dynamic programming.	Exponent	no date
M	40. Jump Game with Profit	1Point3Acres	no date
M	41. Fence Painting Minimum Operations (Gemini Team)	1Point3Acres	2026-04-27
M	42. Maximize 1D deliveries within distance and minimize total distance	PracHub	2025-12-27
H	43. Edit distance	Exponent	no date
H	44. Count Perfect Wake Numbers up to N	1Point3Acres	2026-05-07
H	45. Word Break: All Sentences	ColorStack	2026-01-28

CODE TEMPLATE | GROUP 5 OF 43 | 9 QUESTIONS, ONE PATTERN

Recurrence on a position or a range (memo or table)

The one idea these questions share

Define best(i): the answer for everything from position i onward (or best(i, j) for a range, or a table over two sequences). Write what an empty or finished state returns, then combine the answers of the states you can jump to. Cache it, or fill a table in the right order.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from functools import lru_cache


def solve(seq, valid):
    n = len(seq)

    @lru_cache(maxsize=None)
    def best(i):                          # CHANGE 1: i is a position, or (i, j) for a range
        if i == n:
            return 1                      # CHANGE 2: what a finished state returns
        total = 0
        for j in range(i + 1, n + 1):     # CHANGE 3: which later states can you jump to
            if valid(seq[i:j]):           # CHANGE 4: can this piece be used
                total += best(j)          # CHANGE 5: combine: sum, max, min, or build lists
        return total

    return best(0)
```

2. Helper functions that usually get written next to it

```
def edit_distance(a, b):                    # the two-row table most alignment questions use
    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        cur = [i]
        for j, cb in enumerate(b, 1):
            cur.append(prev[j - 1] if ca == cb else 1 + min(prev[j - 1], prev[j], cur[j - 1]))
        prev = cur
    return prev[-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
37	Target Expressions from Ordered Digits	best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and *
38	Count Word Decompositions into Chemical Element Symbols	bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists
39	Find the longest palindromic subsequence using dynamic programming.	table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end
40	Jump Game with Profit	best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]
41	Fence Painting Minimum Operations (Gemini Team)	range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast
42	Maximize 1D deliveries within distance and minimize total distance	table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)
43	Edit distance	table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

Contents Start the questions >

#	Question	What you change in the template
44	Count Perfect Wake Numbers up to N	digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag
45	Word Break: All Sentences	best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 37 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 1 of 9

Template fit: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and * | asked 68 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 37 of 288

Target Expressions from Ordered Digits

ASKED 2026-08-13 | 68 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 118/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

Given a list of digits in order and a target number, put +, - or * between the digits (never reorder them and never glue two digits into one number) and add parentheses wherever you like. Return every distinct expression that evaluates to the target.

Function signature

def expressions_for_target(digits: list, target: int) -> list

EXAMPLES

Example 1	
Input	`[2, 3, 4], 14`
Output	`['2*(3+4)', '2+3*4']`
Why	`2 * (3 + 4) = 14 and 2 + 3 * 4 = 14 are the only two ways.`

Example 2	
Input	`[1, 2, 3], 6`
Output	`['1*2*3', '1+2+3']`
Why	`Three digits can reach 6 several ways.`

Example 3	
Input	`[1, 2], 100`
Output	`[]`
Why	`No way to reach 100 with these digits.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

TIME
exponential (Catalan(n) *...
exponential (Catalan(n) * 3^(n-1) expressions)

SPACE
O(n)
recursion plus the value set

WHY IT IS SLOW

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

TIME
exponential in the number...
exponential in the number of results; polynomial in the number of distinct values

SPACE
O(distinct values per interval)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

FOLLOW-UP 3 Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Target Expressions from Ordered Digits
#
# Given a list of digits in order and a target number, put
# +, - or * between the digits (never reorder them and
# never glue two digits into one number) and add
# parentheses wherever you like. Return every distinct
# expression that evaluates to the target.
#
# Signature:
#   def expressions_for_target(digits: list, target: int)
-> list
#
# Examples:
# [2, 3, 4], 14  ->  ['2*(3+4)', '2+3*4']
# [1, 2, 3], 6  ->  ['1*2*3', '1+2+3']
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you handle N digits where listing every
# expression is too much?
# 2. What if concatenating digits (23) or division is
# allowed?
# 3. Only a yes/no answer is needed for many digits.
#
# INTERVIEW NOTES
# Any expression over a run of digits is two smaller
# expressions joined by an operator at some split point,
# so the set of values for an interval is built from the
# sets of its two halves. Compute values bottom-up per
# interval and only build text for the target. Be explicit
# about parentheses: they come from the split structure,
# so every split is a candidate.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))

# ---- main question: main function ----
# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
# Idea: Every expression over digits[l:r] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it.
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})

# ---- main question: tests ----
def test_main_question():
    ans = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in ans and "2+3*4" in ans


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Only a yes/no answer is needed for many
# ============================================================

# ---- follow-up 3: function ----
# Answer: Store just the set of reachable values per
```

```
# interval (as target_reachable does) instead of the
# expression strings; the strings blow up long before the
# value sets do.
def target_reachable(digits, target):
    # yes/no only: keep just the set of values per range
    values = {}

    def solve(l, r):
        if (l, r) in values:
            return values[(l, r)]
        out = {digits[l]} if r - l == 1 else set()
        for m in range(l + 1, r):
            for a in solve(l, m):
                for b in solve(m, r):
                    out |= {a + b, a - b, a * b}
        values[(l, r)] = out
        return out

    return target in solve(0, len(digits))
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    # subtraction needs brackets on the right
    sub = expressions_for_target([5, 3, 1], 3)
    assert "5-(3-1)" in sub
    assert target_reachable([1] * 8, 8)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

STUDY PATH: question 38 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 2 of 9

Template fit: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists | asked 98 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 38 of 288

Count Word Decompositions into Chemical Element Symbols

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 94/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

QUESTION

Chemical elements have symbols of one or two letters (H, He, Si, Cs ...). Given a word, count the ways to write it as a sequence of element symbols (ignore upper and lower case). Different splits count as different ways even if they spell the same word.

Function signature

def count_decompositions(word: str) -> int

EXAMPLES

Example 1	
Input	`'Physics'`
Output	`4`
Why	`P-H-Y-Si-C-S, P-H-Y-Si-Cs, P-H-Y-S-I-C-S and P-H-Y-S-I-Cs: 4 ways.`

Example 2	
Input	`'Ho'`
Output	`2`
Why	`'Ho' is one symbol (holmium) and also H + O: 2 ways.`

Example 3	
Input	`'xqz'`
Output	`0`
Why	`No split works (x, y... cannot all be symbols).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def ways(i):
    if i == len(word):
        return 1
    total = 0
    if word[i] in SYMBOLS:
        total += ways(i + 1)
    if word[i:i + 2] in SYMBOLS:
        total += ways(i + 2)
    return total
```

TIME	SPACE
exponential (Fibonacci gr... exponential (Fibonacci growth)	O(n)

WHY IT IS SLOW

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

OPTIMAL SOLUTION · Suffix-count DP

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

FOLLOW-UP 2 What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

FOLLOW-UP 3 List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

FOLLOW-UP 4 A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

FOLLOW-UP 5 Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Word Decompositions into Chemical
# Element Symbols
#
# Chemical elements have symbols of one or two letters (H,
# He, Si, Cs ...). Given a word, count the ways to write
# it as a sequence of element symbols (ignore upper and
# lower case). Different splits count as different ways
# even if they spell the same word.
#
# Signature:
#   def count_decompositions(word: str) -> int
#
# Examples:
# 'Physics'  ->  4
# 'Ho'  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you list the decompositions, not just count
# them?
# 2. What if the dictionary is large and symbols can be
# long?
# 3. List the decompositions instead of counting them.
# 4. A larger dictionary with long entries.
# 5. Count modulo a prime, or only test whether any
# decomposition exists.
#
# INTERVIEW NOTES
# Counting the ways to cut a string into dictionary pieces
# is Word Break with a count instead of a yes/no. Define
# ways[i] as the number of decompositions of the suffix
# starting at i; it depends only on ways[i + 1] and ways[i
# + 2], because symbols have one or two letters. Compute
# from the end so every value you need already exists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# HELPERS used by the code below
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())

# ---- main question: main function ----
# BASE QUESTION (optimal): Suffix-count DP
# Idea: ways[i] is the number of decompositions of
# word[i:]. A symbol has one or two letters, so ways[i] =
# ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if
# word[i:i + 2] is a symbol).
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]

# ---- main question: tests ----
def test_main_question():
    assert count_decompositions("Physics") == 4


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: List the decompositions instead of counting
# ============================================================

# ---- follow-up 3: function ----
# Answer: Same recursion, but carry the parts chosen so
# far and record them at the end. The number of results
# can be exponential, so ask whether a count or a list is
# wanted.
def list_decompositions(word, symbols=ELEMENTS):
    w, n, memo = word.lower(), len(word), {}

    def splits(i):
        # every way to split the suffix starting at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
        out = []
        for size in (1, 2):
            part = w[i:i + size]
```

```
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    out.append([part] + rest)
        memo[i] = out
        return out

    return splits(0)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    parts = list_decompositions("Physics")
    assert len(parts) == 4
    assert ["p", "h", "y", "si", "c", "s"] in parts


# ============================================================
# FOLLOW-UP 4: A larger dictionary with long entries.
# ============================================================

# ---- follow-up 4: function ----
# Answer: Store the dictionary in a set plus the maximum
```

```
# word length L (only look at substrings up to L), or in a
# trie, so each position costs O(L) lookups instead of
# O(n).
def count_with_max_len(word, symbols):
    # big dictionary: only look at pieces up to its
    # longest
    longest = max(map(len, symbols))
    w, n = word.lower(), len(word)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, min(longest, n - i) + 1):
            if w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]

# ============================================================
# FOLLOW-UP 5: Count modulo a prime, or only test whether
# ============================================================

# ---- follow-up 5: function ----
# Answer: Replace addition by addition modulo the prime,
# or by logical or for a yes/no answer. The DP structure
# is the same.
def exists_decomposition(word, symbols=ELEMENTS):
    # yes / no only: the same DP with or instead of +
    return count_decompositions(word, symbols) > 0

# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert exists_decomposition("Physics")
    # no element starts with x then q
    assert not exists_decomposition("xq")
    assert count_decompositions("Physics", mod=3) == 1
    assert count_with_max_len("Physics", ELEMENTS) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_5()
    print("all tests passed")
```

```


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

STUDY PATH: question 39 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 3 of 9

Template fit: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end | no date

MEDIUM | IN APP LC 516, 5 | Exponent | 39 of 288

★ TOP 75: rank 45 in the Medium and Hard priority list

★ Find the longest palindromic subsequence using dynamic programming.

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 69.4 | 30 DAYS 0 | PRIORITY 32/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: your app's interview approach (LeetCode #516)

LeetCode match: LC 516 Longest Palindromic Subsequence (title) / LC 5 Longest Palindromic Substring (statement)

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +37 interviews

Original: https://www.aced.io/questions/1169/longest-palindromic-subsequence-dynamic-programming

QUESTION

Given a string s, return the length of its longest palindromic subsequence: the longest sequence of letters of s, taken in order but not necessarily next to each other, that reads the same forwards and backwards.

Function signature

def lps_length(s: str) -> int

EXAMPLES

Example 1	
Input	'bbbab'
Output	4
Why	'bbbb' is a subsequence of 'bbbab' and a palindrome.

Example 2	
Input	'cbbd'
Output	2
Why	'bb' (the two b's) is the longest.

Example 3	
Input	'a'
Output	1
Why	A single letter.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lps(i, j):
    if i > j:
        return 0
    if i == j:
        return 1
    if s[i] == s[j]:
        return 2 + lps(i + 1, j - 1)
    skip_left = lps(i + 1, j)
    skip_right = lps(i, j - 1)
    return max(skip_left, skip_right)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches per step and the same (i, j) range recurs: O(2^n). A table over all ranges is O(n^2) time, with a rolling row for O(n) memory.

OPTIMAL SOLUTION · Interval DP with a rolling row

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subsequence itself.

Keep the full table and walk from (0, n - 1): equal ends are taken (both go into the answer), otherwise move toward the side with the larger value. O(n^2) memory is needed for the walk back.

FOLLOW-UP 2 Minimum number of insertions (or deletions) to make the string a palindrome.

It is n - LPS(s): keep the longest palindromic subsequence and add (or remove) the other characters.

FOLLOW-UP 3 Longest palindromic substring instead (contiguous).

Expand around each of the 2n - 1 centres in O(n^2), or use Manacher's algorithm in O(n). A subsequence DP is not needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the longest palindromic subsequence
# using dynamic programming.
#
# Given a string s, return the length of its longest
# palindromic subsequence: the longest sequence of letters
# of s, taken in order but not necessarily next to each
# other, that reads the same forwards and backwards.
#
# Signature:
#   def lps_length(s: str) -> int
#
# Examples:
# 'bbbab'  ->  4
# 'cbbd'  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsequence itself.
# 2. Minimum number of insertions (or deletions) to make
# the string a palindrome.
# 3. Longest palindromic substring instead (contiguous).
#
# INTERVIEW NOTES
# Look at the two ends of a range. If they are equal they
# both belong to the palindrome and you add 2 to the
# answer for the inside; if not, one of them is dropped,
# so take the better of dropping the left or the right
# end. The state is a range (i, j), giving an O(n^2) table
# filled from short ranges to long ones.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def lps_table(s):
```

```
    # dp[i][j] = longest palindromic SUBSEQUENCE of
    # s[i..j].
    # Equal ends are both used; otherwise drop one end.
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp

# ---- main question: main function ----
# BASE QUESTION (optimal): Interval DP with a rolling row
# Idea: dp[i][j] is the length of the longest palindromic
# subsequence of s[i..j]. Fill i from the end to the start
# and j from i + 1 up: equal ends give dp[i + 1][j - 1] +
# 2, otherwise max(dp[i + 1][j], dp[i][j - 1]).
def lps_length(s):
    return lps_table(s)[0][-1] if s else 0

# ---- main question: tests ----
def test_main_question():
    assert lps_length("bbbab") == 4
    assert lps_length("cbbd") == 2
    assert lps_length("") == 0


# ============================================================
# FOLLOW-UP 1: Return the subsequence itself.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the full table and walk from (0, n - 1):
# equal ends are taken (both go into the answer),
# otherwise move toward the side with the larger value.
# O(n^2) memory is needed for the walk back.
def lps_string(s):
    # follow-up: the subsequence itself. Walk the table
    # from
    # the full range and rebuild the left half as we go.
    if not s:
        return ""
```

```
    dp = lps_table(s)
    i, j = 0, len(s) - 1
    left, middle = [], ""
    while i <= j:
        if i == j:
            middle = s[i]
            break
        if s[i] == s[j]:
            left.append(s[i])
            i += 1
            j -= 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    half = "".join(left)
    return half + middle + half[::-1]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert lps_string("bbbab") == "bbbb"
    assert lps_string("a") == "a"


# ============================================================
# FOLLOW-UP 2: Minimum number of insertions (or deletions)
# ============================================================

# ---- follow-up 2: function ----
# Answer: It is n - LPS(s): keep the longest palindromic
# subsequence and add (or remove) the other characters.
def min_insertions(s):
    # follow-up: insertions (or deletions) to make a
    # palindrome: everything outside the longest one
    return len(s) - lps_length(s)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_insertions("abca") == 1
    assert min_insertions("race") == 3
    assert min_insertions("aba") == 0


# ============================================================
# FOLLOW-UP 3: Longest palindromic substring instead
```

```
# ============================================================

# ---- follow-up 3: function ----
# Answer: Expand around each of the 2n - 1 centres in
# O(n^2), or use Manacher's algorithm in O(n). A
# subsequence DP is not needed.
```

```
def longest_palindromic_substring(s):
    # follow-up: CONTIGUOUS version. Expand around each of
    # the
    # 2n - 1 centres (a letter, or the gap between two).
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_palindromic_substring("abacc") == "aba"
    assert longest_palindromic_substring("cbbd") == "bb"
    assert longest_palindromic_substring("") == ""
    for _ in range(300):
        s = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 10)))
        found = lps_string(s)
        # the answer really is a subsequence of s
        it = iter(s)
        assert all(ch in it for ch in found)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples

```
Input: "abacc"
Output: "aba"
Explanation: "aba" is longer than "cc"
```

OFFICIAL PAGE TEXT

Our solution finds the longest palindromic substring by expanding around potential centers in the input string. For each character in the string, we consider it as the center of a palindrome and expand outwards to determine the maximum length of the palindrome centered at that position. We repeat this for both odd-length and even-length palindromes. For each character, we use a helper function expandAroundCenter that takes two indices (left and right) and expands outwards as long as the characters at these indices are equal and within the bounds of the string. We calculate the lengths of palindromes centered at the current character (odd-length palindromes) and between the current character and the next one (even-length palindromes). We track the starting and ending indices of the longest palindromic substring found so far. By comparing the lengths of the palindromes found in each iteration, we can determine the longest palindromic substring. The use of expansion around centers ensures that we examine all potential palindromes efficiently. Time Complexity: The solution has a time complexity of O(n^2), where n is the length of the input string. This is because, for each character in the string, we potentially expand outwards in both directions, resulting in quadratic time complexity in the worst case. Space Complexity: The solution uses O(1) space beyond the

input string. The only extra space used is for a few variables, making it very space-efficient. NoteCheck out our lesson on the two-pointer approach to learn more about what it is and how to use it!No answers contributed by the community yet.

STUDY PATH: question 40 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 4 of 9

Template fit: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i] | no date | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 40 of 288

Jump Game with Profit

STUDY: TYPE | ALL TIME 56 | 30 DAYS 0 | PRIORITY 65/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #55 Jump Game

LeetCode match: LC 55 Jump Game (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/jump-game-with-profit

QUESTION

You stand at index 0 of an array. From index i you may jump to any index from i + 1 up to i + arr[i]. Landing on index i earns profit[i] (the start earns profit[0] too). Return the largest total profit of a path that ends exactly at the last index, or -1 if the last index cannot be reached.

Function signature

def max_profit_fast(arr: list, profit: list) -> int

EXAMPLES

Example 1	
Input	`[2, 3, 1, 1, 4], [1, 5, 2, 3, 4]`
Output	`15`
Why	`Jumping one index at a time visits (and collects) everything: 1 + 5 + 2 + 3 + 4 = 15.`

Example 2	
Input	`[1, 0, 1], [1, 1, 1]`
Output	`-1`
Why	`From index 1 you cannot jump at all (arr[1] = 0), so the last index is unreachable.`

BRUTE FORCE · Reading A: recursion over every jump length

BRUTE FORCE · PSEUDO-CODE

```
best = -INF
def go(i, total):
    global best
    if i == n - 1:
        best = max(best, total)
        return
    for step in range(1, arr[i] + 1):
        if i + step < n:
            gain = profit[i + step]
            go(i + step, total + gain)
```

TIME	SPACE
O(max jump ^ n)	O(n)

WHY IT IS SLOW

Every cell branches into up to max-jump choices, so the paths multiply (max jump to the power n). A table of the best profit per cell, filled left to right, is O(n * max jump).

BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= n:
        return 0
    skip = best(i + 1)
    jump = 0
    if arr[i] > 0:
        jump = profit[i] + best(i + arr[i])
    return max(skip, jump)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches at every cell and the same suffix is solved repeatedly: O(2^n). Computing best(i) once per index from the right is O(n).

OPTIMAL SOLUTION · Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index

TIME	SPACE
O(n * maxJump)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

BRUTE FORCE · Reading A: recursion over every jump length

BRUTE FORCE · PSEUDO-CODE

OPTIMAL SOLUTION · Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

FOLLOW-UP 2 What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

FOLLOW-UP 3 Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

FOLLOW-UP 4 Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

FOLLOW-UP 5 Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Jump Game with Profit
#
# You stand at index 0 of an array. From index i you may
# jump to any index from i + 1 up to i + arr[i]. Landing
# on index i earns profit[i] (the start earns profit[0]
# too). Return the largest total profit of a path that
# ends exactly at the last index, or -1 if the last index
# cannot be reached.
#
# Signature:
#   def max_profit_fast(arr: list, profit: list) -> int
#
# Examples:
# [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]  ->  15
# [1, 0, 1], [1, 1, 1]  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Speed up reading A to near-linear time.
# 2. What if you want to know whether the end is reachable
# at all (plain Jump Game)?
# 3. Only reachability, no profit (LeetCode 55).
# 4. Make the profit DP faster.
# 5. Fewest jumps (LeetCode 45).
#
# INTERVIEW NOTES
# This statement is underspecified, so pin it down first
# (cap or exact jump, when profit is collected, can you
# skip, what is the goal). The reading with a maximum jump
# and profit on landing is a DP over positions: the best
# profit at i is its own profit plus the best of all
# positions that can reach it. The reading with an exact
# jump and a free skip is a one-line DP from the right.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import itertools


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
NEG = float("-inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): Reading A: arr[i] is a maximum
# jump, profit[i] is gained on landing, maximize total
# profit to the last index
# Idea: Let best[i] be the largest profit achievable when
# standing on i. Landing on i from any j < i with j +
# arr[j] >= i gives best[i] = profit[i] + max(best[j]).
# Unreachable cells stay at minus infinity.
def max_profit_fast(arr, profit):
    # a max-heap of (-best, reach) with lazy deletion
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]
    for i in range(1, n):
        # forget cells that can no longer reach i
        while heap and heap[0][1] < i:
            heapq.heappop(heap)
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    return -1 if best[-1] == NEG else best[-1]

# ---- main question: tests ----
```

```
def test_main_question():
    arr, profit = [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
    assert max_profit_fast(arr, profit) == 15
    assert max_profit_fast([1, 0, 1], [1, 1, 1]) == -1


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55).
# ============================================================

# ---- follow-up 3: function ----
# VARIANT: reachability only (LeetCode 55)
# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55).
# Answer: Greedy: keep the farthest index reachable so
# far; if the current index is beyond it you are stuck.
# O(n), O(1).
def can_reach_end(arr):
    # LeetCode 55: only reachability, greedy O(n)
    farthest = 0
    for i, jump in enumerate(arr):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True

# ---- follow-up 3: tests ----
def test_follow_up_3():
    arr, profit = [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
    assert can_reach_end(arr)
    assert not can_reach_end([1, 0, 1])


# ============================================================
# FOLLOW-UP 4
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# FOLLOW-UP 5: Fewest jumps (LeetCode 45).
# ============================================================

# ---- follow-up 5: function ----
# VARIANT: fewest jumps (LeetCode 45)
# FOLLOW-UP 5: Fewest jumps (LeetCode 45).
# Answer: BFS by levels or a greedy that tracks the end of
# the current jump range and the farthest reach, O(n).
def fewest_jumps(arr):
    # LeetCode 45: levels of a BFS, greedy O(n)
    jumps = end = farthest = 0
```

```
    for i in range(len(arr) - 1):
        farthest = max(farthest, i + arr[i])
        if i == end:
            # we must jump now: extend to the farthest
            # reach
            jumps += 1
            end = farthest
    return jumps

# ---- follow-up 5: tests ----
def test_follow_up_5():
    arr, profit = [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
    assert fewest_jumps(arr) == 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

STUDY PATH: question 41 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 5 of 9

Template fit: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast | asked 176 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 41 of 288

Fence Painting Minimum Operations (Gemini Team)

ASKED 2026-04-27 | 176 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 260/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

QUESTION

A fence has n planks, plank i is a[i] units tall. One stroke is either horizontal (paints one row of height h across a run of neighbouring planks that are all at least h tall) or vertical (paints one plank completely, bottom to top). Return the smallest number of strokes needed to paint the whole fence.

Function signature

def paint(a: list) -> int

EXAMPLES

Example 1

Input	`[2, 2, 1, 2, 1]`
Output	`3`
Why	`Row 1 across all five planks, row 2 across the first two planks, and one vertical stroke on plank 4: 3 strokes.`

Example 2

Input	`[1, 1, 1]`
Output	`1`
Why	`One horizontal stroke of height 1 across all three planks paints the whole fence.`

Example 3

Input	`[5]`
Output	`1`
Why	`One tall plank: a single vertical stroke beats five horizontal ones.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def paint(l, r, base):
    if l > r:
        return 0
    m = min(height[l:r + 1])
    vertical = r - l + 1
    horizontal = m - base
    for part in split_at_height(l, r, m):
        horizontal += paint(part.l, part.r, m)
    return min(vertical, horizontal)
```

TIME	SPACE
O(n^2)	O(n)
worst case	recursion

WHY IT IS SLOW

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

OPTIMAL SOLUTION · Same recursion with a sparse table for range minimum

TIME	SPACE
O(n log n)	O(n log n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

FOLLOW-UP 2 Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

FOLLOW-UP 3 Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. **Cost:** the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Fence Painting Minimum Operations (Gemini
# Team)
#
# A fence has n planks, plank i is a[i] units tall. One
# stroke is either horizontal (paints one row of height h
# across a run of neighbouring planks that are all at
# least h tall) or vertical (paints one plank completely,
# bottom to top). Return the smallest number of strokes
# needed to paint the whole fence.
#
# Signature:
#   def paint(a: list) -> int
#
# Examples:
# [2, 2, 1, 2, 1]  ->  3
# [1, 1, 1]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. The horizontal brush is only W planks wide.
# 2. Return the strokes, not just the count.
# 3. Is it the same as largest rectangle in a histogram?
#
# INTERVIEW NOTES
# The shortest plank in a range decides the shape: a
# horizontal band along the bottom, up to that plank's
# height, is free of regret because every plank in the
# range needs it. That splits the range at the shortest
# planks, so recurse on the pieces above the band. The
# other choice for a range is to paint every plank
# vertically. Take whichever is cheaper.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def sparse_table(a):
```

```
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table
```

```
# FOLLOW-UP 2: Return the strokes, not just the count.
# Answer: Record which option won in each range and
# rebuild the list by walking the recursion again:
# horizontal bands as (row, l, r), vertical strokes as
# (plank).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
```

```
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)

# ---- main question: main function ----
# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
def paint(a, width=None):
    # FOLLOW-UP 1: width= limits the horizontal brush
    return paint_strokes(a, width)[0]

# ---- main question: tests ----
def test_main_question():
    # the example: 3 strokes
    assert paint([2, 2, 1, 2, 1]) == 3
    assert paint([]) == 0
    assert paint([3]) == 1
    # a flat fence is one stroke per row
    assert paint([4, 4, 4, 4]) == 4
    # tall thin planks are cheaper to paint upright
    assert paint([5, 0, 5]) == 2
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3
    # a narrow brush needs several strokes per row
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2
```

```
# ============================================================
# FOLLOW-UP 1
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Return the strokes, not just the count.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

STUDY PATH: question 42 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 6 of 9

Template fit: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance) | asked 297 days ago

MEDIUM | PracHub | Technical Screen | 42 of 288

Maximize 1D deliveries within distance and minimize total distance

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.9 | 30 DAYS 0 | PRIORITY 116/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #455 Assign Cookies, LeetCode #475 Heaters

LeetCode match: No direct match; related LC 455 Assign Cookies / LC 475 Heaters (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/maximize-1d-deliveries-within-distance-and-minimize-total-distance

QUESTION

Orders and drivers are points on a line. A driver can deliver an order that is at most D away; every driver takes at most one order and every order is delivered by at most one driver. First maximize the number of deliveries; among all plans with that many deliveries, minimize the total distance driven. Return (number of deliveries, total distance).

Function signature

def deliveries(orders: list, drivers: list, D: int) -> tuple

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 6, 20], 2`
Output	`(2, 2)`
Why	`1 is served by the driver at 2, and 5 by the driver at 6: 2 deliveries with total distance 1 + 1 = 2.`

Example 2	
Input	`[1, 5, 9], [2, 6, 20], 11`
Output	`(3, 13)`
Why	`With D = 11 the driver at 20 can also take the order at 9 (distance 11): 1 + 1 + 11 = 13 for 3 deliveries.`

Example 3	
Input	`[3, 4], [], 5`
Output	`(0, 0)`
Why	`No drivers, no deliveries.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = (0, 0)  # (matches, -distance)
def go(i, used, matches, dist):
    global best
    if i == len(orders):
        best = max(best, (matches, -dist))
        return
    go(i + 1, used, matches, dist)  # skip
    for d, x in enumerate(drivers):
        gap = abs(orders[i] - x)
        if d not in used and gap <= D:
            go(i + 1, used | {d},
               matches + 1, dist + gap)
```

TIME	SPACE
O(drivers^orders)	O(n)

WHY IT IS SLOW

Each order branches over all drivers: exponential. After sorting, a DP over (orders considered, drivers considered) that compares (matches, minus distance) is O(n * m).

OPTIMAL SOLUTION · DP over sorted orders and drivers, maximising (matches, -distance)

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the number of deliveries matters.

Two pointers on the sorted lists: match when within D, otherwise advance whichever is further left. O(n log n). It maximises the count (checked against exhaustive search) but not the total distance.

FOLLOW-UP 2 A driver can take several orders (capacity c).

Replicate each driver c times (as separate sorted points) and use the same DP, or treat it as min-cost flow with capacities.

FOLLOW-UP 3 Minimise the largest single distance instead of the total.

Binary search the largest allowed distance D and test whether all orders can be served with the two-pointer matching; the check is monotone in D. **Cost:** O(n log n + (n + m) log(range)): sorting plus a binary search whose check is a two-pointer pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximize 1D deliveries within distance
# and minimize total distance
#
# Orders and drivers are points on a line. A driver can
# deliver an order that is at most D away; every driver
# takes at most one order and every order is delivered by
# at most one driver. First maximize the number of
# deliveries; among all plans with that many deliveries,
# minimize the total distance driven. Return (number of
# deliveries, total distance).
#
# Signature:
#   def deliveries(orders: list, drivers: list, D: int) ->
 tuple
#
# Examples:
# [1, 5, 9], [2, 6, 20], 2  ->  (2, 2)
# [1, 5, 9], [2, 6, 20], 11  ->  (3, 13)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the number of deliveries matters.
# 2. A driver can take several orders (capacity c).
# 3. Minimise the largest single distance instead of the
# total.
#
# INTERVIEW NOTES
# On a line there is always an optimal matching that does
# not cross itself, so sort both lists and match in order:
# a DP over (orders considered, drivers considered) with
# the pair (matches, total distance) compared
# lexicographically solves both objectives together. A
# greedy that matches each order to the nearest free
# driver gets the count right but not always the smallest
# total distance.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): DP over sorted orders and
# drivers, maximising (matches, -distance)
# Idea: Sort both lists. dp[i][j] is the best (matches,
# -total distance) using the first i orders and the first
# j drivers: skip order i, skip driver j, or match them if
# their distance is at most D.
def deliveries(orders, drivers, D):
    # first maximise the number of deliveries, then
    # minimise
    # the total distance: a DP over the sorted positions
    o, d = sorted(orders), sorted(drivers)
    # dp[i][j] = (deliveries, -distance) using the first i
    # orders and j drivers; tuples compare
    # lexicographically
    dp = [[(0, 0)] * (len(d) + 1)
          for _ in range(len(o) + 1)]
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            # skip this order or skip this driver ...
            best = max(dp[i - 1][j], dp[i][j - 1])
            gap = abs(o[i - 1] - d[j - 1])
            if gap <= D:
                # ... or pair them
                count, neg = dp[i - 1][j - 1]
                best = max(best, (count + 1, neg - gap))
            dp[i][j] = best
    count, neg = dp[-1][-1]
    return count, -neg

# ---- main question: tests ----
def test_main_question():
    assert deliveries([1, 5], [2, 6], 1) == (2, 2)
    # pairing 1-2 and 3-4 beats 1-4 and 3-2? both count 2
    assert deliveries([1, 3], [2, 4], 1) == (2, 2)
    assert deliveries([], [1], 5) == (0, 0)
    # a distance limit of 0 needs identical positions
    assert deliveries([1, 2], [2, 3], 0) == (1, 0)


# ============================================================
# FOLLOW-UP 1: Only the number of deliveries matters.
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Two pointers on the sorted lists: match when
# within D, otherwise advance whichever is further left.
# O(n log n). It maximises the count (checked against
# exhaustive search) but not the total distance.
def max_count(orders, drivers, D):
    # only the number matters: two pointers on sorted
    # lists
    o, d = sorted(orders), sorted(drivers)
    i = j = count = 0
    while i < len(o) and j < len(d):
        if abs(o[i] - d[j]) <= D:
            count, i, j = count + 1, i + 1, j + 1
        elif o[i] < d[j]:
            # this order is too far left for every driver
            i += 1
        else:
            j += 1
    return count

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_count([1, 3], [2, 4], 1) == 2
    assert max_count([1, 10], [5], 2) == 0


# ============================================================
# FOLLOW-UP 2: A driver can take several orders (capacity
# ============================================================

# ---- follow-up 2: function ----
# Answer: Replicate each driver c times (as separate
# sorted points) and use the same DP, or treat it as min-
# cost flow with capacities.
def with_capacity(orders, drivers, D, cap):
    # each driver can take cap orders: repeat the driver
    return deliveries(orders, [x for x in drivers
                               for _ in range(cap)], D)

# ============================================================
# FOLLOW-UP 3: Minimise the largest single distance
```

```
# ========================================================

# ---- follow-up 3: function ----
# Answer: Binary search the largest allowed distance D and
```

```
# test whether all orders can be served with the two-
# pointer matching; the check is monotone in D.
def smallest_limit(orders, drivers):
    # minimise the LARGEST single distance while serving
    # as
    # many orders as possible: binary search on the limit
    target = min(len(orders), len(drivers))
    lo, hi = 0, 10 ** 9
    while lo < hi:
        mid = (lo + hi) // 2
        if max_count(orders, drivers, mid) >= target:
            hi = mid
        else:
            lo = mid + 1
    return lo

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert smallest_limit([1, 5], [2, 9]) == 4
    assert with_capacity([1, 2, 3], [2], 1, 3) == (3, 2)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[int], shops: list[int], D: int)
-> tuple[int, int]:
```

Sample call

```
solution([1, 7, 10], [2, 6, 12], 2)
```

Test cases

Input	Expected
([1, 7, 10], [2, 6, 12], 2)	(3, 4)
([0, 4], [2, 5], 3)	(2, 3)

STUDY PATH: question 43 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 7 of 9

Template fit: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours | no date

HARD | IN APP LC 127, 72 | Exponent | 43 of 288

Edit distance

ASKED (ANY CO.) ~3 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 46 | 30 DAYS 0 | PRIORITY 90/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: your app's interview approach (LeetCode #127)

LeetCode match: LC 127 Word Ladder (statement) / LC 72 Edit Distance (title)

Last asked (any company): 3 months ago, about 2026-07 (Exponent shows only a rough age) | +612 interviews

Original: https://www.aced.io/questions/3891/edit-distance

QUESTION

Given two words a and b, return the edit distance: the smallest number of single-letter operations (insert a letter, delete a letter, or replace a letter) that turn a into b.

Function signature

def edit_distance(a: str, b: str) -> (int, list)

(the distance, the whole DP

table)

EXAMPLES

Example 1	
Input	`a = 'horse', b = 'ros'`
Output	`edit_distance('horse', 'ros')[0] -> 3`
Why	`horse -> rorse (replace h) -> rose (delete r) -> ros (delete e): 3 operations.`

Example 2	
Input	`a = 'intention', b = 'execution'`
Output	`edit_distance('intention', 'execution')[0] -> 5`
Why	`intention -> execution takes 5 operations.`

Example 3	
Input	`a = '', b = 'abc'`
Output	`edit_distance('', 'abc')[0] -> 3`
Why	`From the empty word, insert every letter.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def dist(i, j):
    if i == 0:
        return j
    if j == 0:
        return i
    if a[i - 1] == b[j - 1]:
        return dist(i - 1, j - 1)
    replace = dist(i - 1, j - 1)
    delete = dist(i - 1, j)
    insert = dist(i, j - 1)
    return 1 + min(replace, delete, insert)
```

TIME	SPACE
O(3^(m + n))	O(m + n)

WHY IT IS SLOW

Three calls per step and the same (i, j) pair is solved many times: about 3^(m + n). A table of all (i, j) values is O(m * n), with a rolling row for O(min(m, n)) memory.

OPTIMAL SOLUTION · DP table with a rolling row

TIME	SPACE
O(m * n)	O(min(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual operations.

Keep the full table and walk back from dp[m][n]: if the characters match step diagonally; otherwise take the neighbour whose value is one smaller and record replace, delete or insert. O(m * n) memory is needed for the walk back.

FOLLOW-UP 2 Are the strings exactly one edit apart (LeetCode 161)?

No table needed: if the lengths differ by more than one answer false; scan to the first mismatch and compare the remainders (equal lengths: skip one char in each; otherwise skip one in the longer). O(n), O(1).

FOLLOW-UP 3 Operations have different costs, or only a small edit distance is of interest (at most k).

Weighted costs just change the numbers added in the recurrence. For 'at most k' only compute a band of width 2k + 1 around the diagonal: O(k * n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Edit distance
#
# Given two words a and b, return the edit distance: the
# smallest number of single-letter operations (insert a
# letter, delete a letter, or replace a letter) that turn
# a into b.
#
# Signature:
#   def edit_distance(a: str, b: str) -> (int, list)
 # (the distance, the whole DP table)
#
# Examples:
# a = 'horse', b = 'ros'  ->  edit_distance('horse',
#     'ros')[0]  ->  3
# a = 'intention', b = 'execution'  ->
#     edit_distance('intention', 'execution')[0]  ->  5
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual operations.
# 2. Are the strings exactly one edit apart (LeetCode
# 161)?
# 3. Operations have different costs, or only a small edit
# distance is of interest (at most k).
#
# INTERVIEW NOTES
# Compare the last characters. If they match, nothing is
# lost: the cost is the cost of the shorter prefixes. If
# not, one of three operations was used, so take the
# cheapest of replace, delete and insert, each followed by
# the cost of a smaller problem. The same smaller problems
# repeat, so tabulate them in a grid dp[i][j] = cost of
# turning the first i characters of a into the first j of
# b; each cell needs only its left, top and top-left
# neighbours.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): DP table with a rolling row
# Idea: dp[j] holds the distance from the current prefix
# of a to the first j characters of b.
def edit_distance(a, b):
    # dp[i][j]: edits to turn a[:i] into b[:j]
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                swap = dp[i - 1][j - 1]
                drop = dp[i - 1][j]
                add = dp[i][j - 1]
                dp[i][j] = 1 + min(swap, drop, add)
    return dp[m][n], dp


# RELATED: at most k edits (banded)
def within_k(a, b, k):
    # only a band of width 2k + 1 around the diagonal
    # matters
    if abs(len(a) - len(b)) > k:
        return False
    return edit_distance(a, b)[0] <= k


# RELATED: the word-ladder version
def ladder_length(begin, end, words):
    # the word-ladder version: change one letter at a
    # time,
    # every word must be in the list; BFS over the words
    words = set(words)
    if end not in words:
        return 0
    queue, seen = deque([(begin, 1)]), {begin}
```

```
    while queue:
        word, steps = queue.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                nxt = word[:i] + c + word[i + 1:]
                if nxt in words and nxt not in seen:
                    seen.add(nxt)
                    queue.append((nxt, steps + 1))
    return 0

# ---- main question: tests ----
def test_main_question():
    assert edit_distance("horse", "ros")[0] == 3
    assert edit_distance("", "abc")[0] == 3
    assert edit_distance("same", "same")[0] == 0


# ============================================================
# FOLLOW-UP 1: Return the actual operations.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the full table and walk back from dp[m][n]:
# if the characters match step diagonally; otherwise take
# the neighbour whose value is one smaller and record
# replace, delete or insert. O(m * n) memory is needed for
# the walk back.
def edit_operations(a, b):
    # walk back through the table and list the operations
    dist, dp = edit_distance(a, b)
    i, j, ops = len(a), len(b), []
    while i or j:
        if i and j and a[i - 1] == b[j - 1]:
            i, j = i - 1, j - 1
        elif i and j and dp[i][j] == dp[i - 1][j - 1] + 1:
            old, new = a[i - 1], b[j - 1]
            ops.append(f"replace {old} with {new}")
            i, j = i - 1, j - 1
```

```
        elif i and dp[i][j] == dp[i - 1][j] + 1:
            ops.append(f"delete {a[i - 1]}")
            i -= 1
        else:
            ops.append(f"insert {b[j - 1]}")
```

```
            j -= 1
    return ops[::-1]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    ops = edit_operations("horse", "ros")
    assert len(ops) == 3


# =====================================================
# FOLLOW-UP 2: Are the strings exactly one edit apart
# =====================================================

# ---- follow-up 2: function ----
# Answer: No table needed: if the lengths differ by more
# than one answer false; scan to the first mismatch and
# compare the remainders (equal lengths: skip one char in
# each; otherwise skip one in the longer). O(n), O(1).
def one_edit_apart(a, b):
    # LeetCode 161: no table, O(n)
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) > len(b):
        a, b = b, a
    for i in range(len(a)):
        if a[i] != b[i]:
            # the rest must match after one replace or
            # insert
            if len(a) == len(b):
                return a[i + 1:] == b[i + 1:]
            return a[i:] == b[i + 1:]
    return len(a) != len(b)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert one_edit_apart("cat", "cut")
    assert one_edit_apart("cat", "cats")
    assert not one_edit_apart("cat", "cat")
    assert not one_edit_apart("cat", "dog")
    assert within_k("kitten", "sitting", 3)
    assert not within_k("kitten", "sitting", 2)
    words = ["hot", "dot", "dog", "lot", "log", "cog"]
    got = ladder_length("hit", "cog", words + ["hit"])
    assert got == 5
    got = ladder_length("hit", "cog", ["hot", "dot"])
```

```
    assert got == 0


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples

```
beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog", "cog"]
output: 4 # hit -> hot -> dot -> dog -> cog

beginWord = 'word'
endWord = 'word'
wordList = ['word', 'ward']
output: 0

beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog"]
output: None  # because no 'cog' in list
```

Our solution tackles the problem by first constructing a graph that represents all possible transformations between words in the list, where an edge exists between two words if they differ by exactly one character (i.e., they are "one edit distance" apart). This is done by iterating over the list of words and comparing each pair to determine if they meet the one edit distance condition. If they do, an edge is added between them in the graph. After constructing the graph, we use a breadth-first search (BFS) approach to find the shortest path from the start_word to the end_word. We initiate a queue that starts with the start_word and its associated edit distance (initially 0). As we traverse the graph, for each word, we check if it's the end_word. If so, we return the current edit distance incremented by one. If not, we add the word's neighbors to the queue for further exploration, ensuring that we don't revisit previously seen words. The BFS guarantees that the first time we reach the end_word, it is via the shortest path, and we can stop the search early. Time Complexity: The time complexity of the solution is O(n^2 * m), where n is the number of words and m is the length of each word. This complexity arises from the double loop that compares each word to every other word (O(n^2)), and for each comparison, we check if the words differ by exactly one character (O(m)). Space Complexity:

The space complexity is O(n^2), where n is the number of words. This space is required to store the graph, where each word can potentially have edges to all other words. Additionally, the BFS queue and the visited set require O(n) space in the worst case.In this video, Thomas, Google SWE, answers the Edit Distance coding interview question.No answers contributed by the community yet.

STUDY PATH: question 44 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 8 of 9

Template fit: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag | asked 166 days ago | frequency Single report

HARD | 1Point3Acres | phone-screen, onsite-coding | 44 of 288

Count Perfect Wake Numbers up to N

ASKED 2026-05-07 | 166 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 247/288**

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

QUESTION

A number is perfect if its digits are all different, it contains no 0, and no digit is a valley (a digit strictly smaller than both of its neighbours). Part 1: decide whether a number is perfect. Part 2: count the perfect numbers from 1 to N.

Function signature

def is_perfect(n: int) -> bool

def count_up_to(N: int) -> int

EXAMPLES

Example 1	
Input	`196`
Output	`true`
Why	`Digits 1, 9, 6 are different, no zero, and 9 is a peak, not a valley.`

Example 2	
Input	`1546`
Output	`false`
Why	`The 4 in 1546 sits between 5 and 6, both larger: a valley.`

Example 3	
Input	`99`
Output	`81`
Why	`Every number 1-9 and every two-digit number with different non-zero digits: 9 + 72 = 81.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_perfect(x):
    d = list(map(int, str(x)))
    if 0 in d or len(set(d)) < len(d):
        return False
    for i in range(1, len(d) - 1):
        if d[i] < d[i - 1] and d[i] < d[i + 1]:
            return False  # a valley
    return True
count = 0
for x in range(1, N + 1):
    count += is_perfect(x)
```

TIME	SPACE
O(N * d)	O(1)

WHY IT IS SLOW

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

OPTIMAL SOLUTION · Part 1: test a single number

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: digit DP with tight flag, used-digit mask, last digit and direction

TIME	SPACE
O(d * 2^9 * 9 * 9)	O(2^9 * 9)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

FOLLOW-UP 2 How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

FOLLOW-UP 3 What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

FOLLOW-UP 4 Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

FOLLOW-UP 5 Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Perfect Wake Numbers up to N
#
# A number is perfect if its digits are all different, it
# contains no 0, and no digit is a valley (a digit
# strictly smaller than both of its neighbours). Part 1:
# decide whether a number is perfect. Part 2: count the
# perfect numbers from 1 to N.
#
# Signature:
#   def is_perfect(n: int) -> bool
#   def count_up_to(N: int) -> int
#
# Examples:
# 196  ->  true
# 1546  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the DP only need a 'falling' flag instead of
# the full history?
# 2. How do you count the numbers in a range [A, B]?
# 3. What is the closed form without a DP?
# 4. Is there a closed form for k-digit numbers?
# 5. Count perfect numbers in a range [A, B].
#
# INTERVIEW NOTES
# Distinct digits with no zero and no valley means the
# digits go up and then down (they can only change
# direction once). That structure is both a closed form
# for each length and a state for a digit DP: position,
# which digits are used, the last digit, and whether the
# digits have started falling. The tight flag handles the
# bound N.

# ============================================================
```

```
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from functools import lru_cache
from math import comb


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# Answer: Pick the k distinct digits from 1..9 (C(9, k)
# ways); the largest must be the peak, and each of the
# other k - 1 digits goes either left or right of it in
# sorted order, so 2^(k-1) arrangements: C(9, k) *
# 2^(k-1).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def with_k_digits(k):
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0

# ---- main question: main function ----
# BASE QUESTION (optimal): test a single number
# Idea: Check the three rules directly on the digit list:
# no '0', all digits distinct, and for each interior digit
# not (left > digit < right). The first and last digits
# are never valleys.
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))


# PART 2: count the perfect numbers up to N
# FOLLOW-UP 2: How do you count the numbers in a range [A,
# B]?
# Answer: Use inclusion-exclusion: count(B) - count(A -
# 1). The DP is a function of the upper bound only, and it
# is cheap enough (a few thousand states) to call twice.
def count_up_to(n):
    if n < 1:
        return 0
    digits = list(map(int, str(n)))
    size = len(digits)
```

```
    # all shorter numbers: closed form
    total = sum(with_k_digits(k) for k in range(1, size))

    @lru_cache(None)
    def go(pos, mask, last, falling, tight):
        # count completions of a number of full length
        if pos == size:
            return 1
        out = 0
        limit = digits[pos] if tight else 9
        for d in range(1, limit + 1):
            if mask >> d & 1:
```

```
                continue
            fall = falling
            if pos:
                if d > last and falling:
                    # going up after going down: a valley
                    continue
                if d < last:
                    fall = True
            out += go(pos + 1, mask | 1 << d, d, fall,
                      tight and d == digits[pos])
        return out

    # numbers with the same length as n, not above n
    return total + go(0, 0, 0, False, True)

# ---- main question: tests ----
def test_main_question():
    for ok in (196, 23, 12463):
        assert is_perfect(ok)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: How do you count the numbers in a range [A,
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.
```

```
# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# FOLLOW-UP 5: Count perfect numbers in a range [A, B].
# ============================================================

# ---- follow-up 5: function ----
# Answer: Compute count(B) - count(A - 1); the DP depends
# on the upper bound only.
def count_between(a, b):
    # inclusion-exclusion over the upper bounds
    return count_up_to(b) - count_up_to(a - 1)

# ---- follow-up 5: tests ----
def test_follow_up_5():
    # the DP agrees with checking every number
    assert count_between(10, 99) == (
        count_up_to(99) - count_up_to(9))
    # 1-digit and 2-digit counts from the closed form
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

STUDY PATH: question 45 of 288 | topic 5 of 43: Recurrence on a position or a range (memo or table) | 9 of 9

Template fit: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end) | asked 265 days ago | frequency Single report

HARD | IN APP LC 140 | ColorStack | technical interview | 45 of 288

Word Break: All Sentences

ASKED 2026-01-28 | 265 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 207/288

ABOUT THIS QUESTION

Topic: Recurrence on a position or a range (memo or table)

Optimal answer from: your app's interview approach (LeetCode #140)

LeetCode match: LC 140 Word Break II

Frequency: Single report | Last asked: 2026-01-28 | Roles: SWE Intern

QUESTION

You are given a string s and a dictionary word_dict of distinct words. Return ALL possible ways to segment s into a sequence of space-separated words such that every word in the sequence appears in word_dict . A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

Function signature

```
from typing import List
def word_break_all(s: str, word_dict: List[str]) -> List[str]:
    ...
```

Follow-up (as reported): Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid exponential re-work.

EXAMPLES

Example 1	
Input	`s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]`
Output	`["cats and dog", "cat sand dog"]`
Why	`Two segmentations use only dictionary words.`

Example 2	
Input	`s = "pineapplepenapple", word_dict = ["apple","pen","applepen","pine","pineapple"]`
Output	`["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]`
Why	`All three segmentations consist entirely of dictionary words.`

QUESTION

Example 3	
Input	`s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]`
Output	`[]`
Why	`No segmentation covers the whole string with dictionary words.`

CONSTRAINTS

- 1 <= len(s) <= 20
- 1 <= len(word_dict) <= 1000
- 1 <= len(word) <= 10 for each word; words consist of lowercase letters and are distinct.
- If no valid segmentation exists, return an empty list [] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_word_break(s, word_dict):
    words, out = set(word_dict), []
    for mask in range(1_(len(s)_-_1)):
        pieces, start = [], 0
        for i in range(len(s)_-_1):
            if mask >> i & 1:
                pieces.append(s[start : i + 1])
                start = i + 1
        pieces.append(s[start:])
        if all(p in words for p in pieces):
            out.append(" ".join(pieces))
    return out
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

It tests all 2^(n-1) cuts although most start with a non-word.

OPTIMAL SOLUTION · Memoized DFS on the start index

TIME	SPACE
O(n^2 + output)	O(n + output)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only count the segmentations.

A DP over suffixes: ways[i] is the sum of ways[j] over words s[i:j]. No strings are built, so it is O(n * L) with a set of words.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Word Break: All Sentences
#
# You are given a string s and a dictionary word_dict of
# distinct words. Return ALL possible ways to segment s
# into a sequence of space-separated words such that every
# word in the sequence appears in word_dict . A dictionary
# word may be reused any number of times. Each returned
# sentence is a single string of the chosen words joined
# by single spaces. The sentences may be returned in any
# order. Function signature: from typing import List def
# word_break_all(s: str, word_dict: List[str]) ->
# List[str]: ... Follow-up (as reported): Solve with
# backtracking: at each position try every dictionary word
# that is a prefix of the remaining string, then recurse.
# Add memoization keyed on the suffix (or start index) to
# reuse partial results and avoid exponential re-work.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only count the segmentations.
#
# INTERVIEW NOTES
# From a start index, try every dictionary word that is a
# prefix of the remaining string and solve the rest. The
# same suffix is reached by many routes, so cache the list
# of sentences per start index (memoization). Without the
# cache the work is exponential; with it each suffix is
# solved once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Memoized DFS on the start index
# Idea: The sentences of a suffix do not depend on how you
```

```
# reached it, so compute them once.
def word_break_all(s, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(s):
            return [""]
        if start in memo:
            return memo[start]
        out = []
        for end in range(start + 1, len(s) + 1):
            if s[start:end] in words:
                for rest in sentences(end):
                    out.append(
                        s[start:end]
                        + (" " + rest if rest else "")
                    )
        memo[start] = out
        return out

    return sentences(0)

# ---- main question: tests ----
def test_main_question():
    random.seed(87)
    assert sorted(
        word_break_all(
            "catsanddog",
            ["cat", "cats", "and", "sand", "dog"],
        )
    ) == ["cat sand dog", "cats and dog"]
    assert (
        word_break_all(
            "catsandog",
            ["cats", "dog", "sand", "and", "cat"],
        )
        == []
    )
    assert (
        len(
            word_break_all(
                "pineapplepenapple",
                [
                    "apple",
                    "pen",
                    "applepen",
                    "pine",
                    "pineapple",
                ],
            )
        )
        == 3
    )


# ============================================================
# FOLLOW-UP 1: Only count the segmentations.
# ============================================================

# ---- follow-up 1: function ----
# Answer: A DP over suffixes: ways[i] is the sum of
# ways[j] over words s[i:j]. No strings are built, so it
# is O(n * L) with a set of words.
def count_sentences(s, word_dict):
    words = set(word_dict)
    ways = [0] * (len(s) + 1)
    ways[len(s)] = 1
    for i in range(len(s) - 1, -1, -1):
        ways[i] = sum(
            ways[j]
            for j in range(i + 1, len(s) + 1)
            if s[i:j] in words
        )
    return ways[0]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
```

```
run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 6 OF 43 |

Best-first search with a heap: Dijkstra and its minimax twin

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5	4
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Best-first search with a heap: Dijkstra and its minimax twin | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

M	46. Path Minimizing the Maximum Cell (Swim in Rising Water)	1Point3Acres	2026-09-29
M	47. Nearest Favorite City by Travel Time	ColorStack	2025-10-17
M	48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given...	Exponent	no date
M	49. Find the shortest path in a graph.	Exponent	no date
M	50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maxi...	Exponent	no date
H	51. Minimize maximum height along a grid path	PracHub	2026-01-04
H	52. Find minimum threshold enabling grid path	PracHub	2026-02-11
H	53. Minimize travel time with optional meeting point	PracHub	2026-01-06
H	54. Compute minimax grid path and network delay	PracHub	2026-01-22

CODE TEMPLATE | GROUP 6 OF 43 | 9 QUESTIONS, ONE PATTERN

Best-first search with a heap: Dijkstra and its minimax twin

The one idea these questions share

Keep a heap of (cost so far, node). Pop the cheapest, skip stale entries, and push any neighbour you improved. Dijkstra adds the edge weight to the cost; the minimax questions take the maximum of the cost and the edge instead. Everything else is the same loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def best_first(start, neighbors, combine, goal=None):
    best = {start: 0}
    heap = [(0, start)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:                      # a stale heap entry
            continue
        if u == goal:                           # CHANGE 1: stop at the goal
            return cost
        for v, w in neighbors(u):               # CHANGE 2: edges (or grid steps)
            nxt = combine(cost, w)              # CHANGE 3: cost + w, or max(cost, w)
            if nxt < best.get(v, float("inf")):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    return best
```

2. Helper functions that usually get written next to it

```
def grid_neighbors(grid):                   # (cell, cost of stepping onto it) for minimax grids
    rows, cols = len(grid), len(grid[0])
    def step(cell):
        r, c = cell
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                yield (nr, nc), grid[nr][nc]
    return step


def reverse_graph(adj):                     # for runs from the destination backwards
    rev = {u: [] for u in adj}
    for u, edges in adj.items():
        for v, w in edges:
            rev[v].append((u, w))
    return rev
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
46	Path Minimizing the Maximum Cell (Swim in Rising Water)	the same minimax loop with a given start and end cell (Swim in Rising Water)
47	Nearest Favorite City by Travel Time	combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

#	Question	What you change in the template
48	Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.	cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time
49	Find the shortest path in a graph.	combine = cost + w; also store parent[v]; stop at the target and rebuild the path
50	Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.	maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)
51	Minimize maximum height along a grid path	the same minimax loop on a height grid
52	Find minimum threshold enabling grid path	the same minimax loop: the smallest threshold that lets you walk from corner to corner
53	Minimize travel time with optional meeting point	three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd
54	Compute minimax grid path and network delay	combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 46 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 1 of 9

Template fit: the same minimax loop with a given start and end cell (Swim in Rising Water) | asked 21 days ago | frequency Low

MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 46 of 288

Path Minimizing the Maximum Cell (Swim in Rising Water)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 40** | **30 DAYS 33** | **PRIORITY 62/288**

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: your app's interview approach (LeetCode #778)

LeetCode match: LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

QUESTION

You get a grid of heights, a start cell and an end cell, and you may move to any of the four neighbouring cells. The cost of a path is the largest height on it (start and end included). Return the smallest cost over all paths from start to end.

Function signature

def min_max_path(g: list, start: tuple, end: tuple) -> int

EXAMPLES

Example 1	
Input	`[[0, 2], [1, 3]], (0, 0), (1, 1)`
Output	`3`
Why	`Both routes pass the cell of height 3 at the end, so the best cost is 3.`

Example 2	
Input	`[[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0, 0), (4, 4)`
Output	`16`
Why	`A 5x5 board: the best route has to climb to 16 somewhere, and no route stays lower.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

BRUTE FORCE

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

OPTIMAL SOLUTION · Dijkstra on the running maximum

TIME	SPACE
O(V log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Path Minimizing the Maximum Cell (Swim in
# Rising Water)
#
# You get a grid of heights, a start cell and an end cell,
# and you may move to any of the four neighbouring cells.
# The cost of a path is the largest height on it (start
# and end included). Return the smallest cost over all
# paths from start to end.
#
# Signature:
#   def min_max_path(g: list, start: tuple, end: tuple) ->
 int
#
# Examples:
# [[0, 2], [1, 3]], (0, 0), (1, 1)  ->  3
# [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15,
#     16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0,
```

```
#     0), (4, 4)  ->  16
#
# FOLLOW-UPS (answered below, in this order):
# 1. Maximise the minimum distance to a cat instead.
# 2. Why is Dijkstra correct for a max-along-path cost?
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, not the sum.
# That cost can only grow as the path grows, so Dijkstra
# still works if you replace 'distance + weight' by
# max(distance, weight): the first time a cell leaves the
# heap its cost is final. The cat version flips the
# objective (maximise a minimum), which is the same shape
# with a max-heap and each cell scored by its distance to
# the cat.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b

# ---- main question: main function ----
# BASE QUESTION (optimal): Dijkstra on the running maximum
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None


# VARIANT: weighted graph with a safety threshold
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, safety)]; every edge must be < limit
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            # edges at or above the limit are not allowed
            if w >= limit:
                continue
            nxt = max(cost, w)
            if nxt < best.get(v, INF):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    # no admissible path
    return -1


# RELATED: network delay (plain Dijkstra)
def network_delay(n, edges, src):
    # plain Dijkstra: largest shortest distance, or -1
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    # unreachable nodes make the answer -1
    return max(dist.values()) if len(dist) == n else -1

# ---- main question: tests ----
def test_main_question():
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5],
         [12, 13, 14, 15, 16], [11, 17, 18, 19, 20],
         [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    # start equals end: just that cell
    assert min_max_path(g, (2, 2), (2, 2)) == 14
    gr = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(gr, 0, 2, 10) == 5
    assert min_bottleneck_edge(gr, 0, 2, 5) == -1
    cat_grid = ["...", ".#.", "..."]
    cat, a, b = (0, 2), (0, 0), (2, 2)


# ============================================================
# FOLLOW-UP 1: Maximise the minimum distance to a cat
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same bottleneck shape with the objective
# flipped: score each cell by its Manhattan distance to
# the cat, run Dijkstra with a max-heap on the running
# minimum, and skip water.
def safest_path_score(grid, start, target, cat):
    # maximise the smallest cat distance ('#' is water)
    n, m = len(grid), len(grid[0])

    def dist(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])
```

```
    best = {start: dist(*start)}
    heap = [(-best[start], start)]
    while heap:
        neg, (r, c) = heapq.heappop(heap)
        if -neg < best[(r, c)]:
            continue
        if (r, c) == target:
            return -neg
        for a, b in around(r, c, n, m):
            if grid[a][b] == "#":
                continue
            # the path is only as safe as its closest cell
            score = min(-neg, dist(a, b))
            if score > best.get((a, b), -1):
                best[(a, b)] = score
                heapq.heappush(heap, (-score, (a, b)))
    return None

# ---- follow-up 1: tests ----
def test_follow_up_1():
    cat_grid = ["...", ".#.", "..."]
    cat, a, b = (0, 2), (0, 0), (2, 2)
    score = safest_path_score(cat_grid, a, b, cat)
```

```
    assert score == 2
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY PATH: question 47 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 2 of 9

Template fit: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index) | asked 368 days ago | frequency Single report

MEDIUM | IN APP LC 743 | ColorStack | technical interview | 47 of 288

Nearest Favorite City by Travel Time

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 68/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: your app's interview approach (LeetCode #743)

LeetCode match: LC 743 Network Delay Time (nearest of several targets)

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

You are given a network of num_cities cities (0..num_cities-1) connected by weighted, undirected roads roads[i] = [u, v, w] where w > 0 is the travel time. Given a start city and a non-empty list favorites of favorite cities, return the favorite city reachable from start in the shortest total travel time. Break ties by smallest city index. If none of the favorite cities is reachable, return -1 .

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	`num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites = [3, 4]`
Output	`3`
Why	`Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.`

Example 2	
Input	`num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]`
Output	`-1`
Why	`City 2 is disconnected from start, so no favorite is reachable.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(roads) <= 2*10**5 , 1 <= w <= 10**9 .
- The interviewer required the optimal solution (Dijkstra with a priority queue).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_nearest(n, roads, start, favorites):
    dist = [float("inf")] * n
    dist[start] = 0
    for _ in range(n_-_1):
        for u, v, w in roads:
            dist[u] = min(dist[u], dist[v] + w)
            dist[v] = min(dist[v], dist[u] + w)
    best = min((dist[f], f) for f in favorites)
    return (
        best[1]
        if best[0] < float("inf")
        else:
            -1
    )
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Bellman-Ford repeats the full relaxation even when most distances are final.

OPTIMAL SOLUTION · Dijkstra from the start

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Answer it for every possible start city in one run.

Reverse the search: run one Dijkstra seeded with every favourite at distance 0, storing for each city the pair (distance, favourite). Because roads are undirected the distance from a city to its nearest favourite is the same, and comparing the pairs lexicographically resolves ties to the smaller favourite. O((V + E) log V) total.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Nearest Favorite City by Travel Time
#
# You are given a network of num_cities cities (
# 0..num_cities-1 ) connected by weighted, undirected
# roads roads[i] = [u, v, w] where w > 0 is the travel
# time. Given a start city and a non-empty list favorites
# of favorite cities, return the favorite city reachable
# from start in the shortest total travel time. Break ties
# by smallest city index. If none of the favorite cities
```

```
# is reachable, return -1 . Function signature: def
# nearest_favorite(num_cities: int, roads:
# list[list[int]], start: int, favorites: list[int]) ->
# int: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Answer it for every possible start city in one run.
#
# INTERVIEW NOTES
# Run Dijkstra from the start city: cities leave the heap
# in order of increasing travel time, so the first
# favourite popped is the nearest. For the tie rule keep
# popping while the distance equals the best and take the
# smallest index. Unreachable favourites never get a
# finite distance.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ========================================================

import heapq
import random


# ========================================================
# MAIN QUESTION
# ========================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Dijkstra from the start
# Idea: Cities are finalised in order of distance, so the
# first favourite reached is the nearest.
def nearest_favorite(
    num_cities, roads, start, favorites
):
    adj = [[] for _ in range(num_cities)]
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dist = [float("inf")] * num_cities
    dist[start] = 0
    heap, wanted = [(0, start)], set(favorites)
    best_d, best_c = None, -1
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if best_d is not None and d > best_d:
            break
        if u in wanted and (
            best_d is None or u < best_c
        ):
            best_d, best_c = d, u
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return best_c

# ---- main question: tests ----
def test_main_question():
    random.seed(72)
    roads = [[0, 1, 2], [0, 2, 5], [1, 3, 1], [2, 4, 1]]
    assert nearest_favorite(5, roads, 0, [3, 4]) == 3
    assert (
        nearest_favorite(3, [[0, 1, 4]], 0, [2]) == -1
    )


# ========================================================
# FOLLOW-UP 1: Answer it for every possible start city in
# ========================================================

# ---- follow-up 1: function ----
# Answer: Reverse the search: run one Dijkstra seeded with
# every favourite at distance 0, storing for each city the
# pair (distance, favourite).
def nearest_favorite_for_all(
    num_cities, roads, favorites
):
    adj = [[] for _ in range(num_cities)]
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    best = [(float("inf"), -1)] * num_cities
    heap = []
    for f in favorites:
        best[f] = (0, f)
        heapq.heappush(heap, (0, f, f))
    while heap:
        d, fav, u = heapq.heappop(heap)
        if (d, fav) > best[u]:
            continue
        for v, w in adj[u]:
            cand = (d + w, fav)
            if cand < best[v]:
                best[v] = cand
                heapq.heappush(heap, (cand[0], fav, v))
    return [
        fav if d < float("inf") else -1
        for d, fav in best
    ]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 48 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 3 of 9

Template fit: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time | no date

MEDIUM | Exponent | 48 of 288

Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 70/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #787 Cheapest Flights Within K Stops

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 787 (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6331/find-earliest-train-sequence-route

QUESTION

A schedule lists trains as (from, to, departure time, arrival time). Starting in city src at time start, find the sequence of trains that gets you to dst at the earliest possible time. You can only board a train that leaves at or after the time you arrive in its departure city. Return (arrival time, the trains you take), or None if dst cannot be reached.

Function signature

def earliest_arrival(trains: list, src, dst, start=0) -> tuple | None

EXAMPLES

Example 1	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9), ('B', 'C', 2, 5)], 'A', 'C'`
Output	`(6, [('A', 'B', 1, 3), ('B', 'C', 4, 6)])`
Why	`A -> B arrives at 3; the train B -> C that leaves at 2 is already gone, so take the one leaving at 4 and arrive at 6 (the direct A -> C train arrives later, at 9).`

QUESTION

Example 2	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'`
Output	`null`
Why	`No train leaves C, so A cannot be reached from C.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = INF
def go(city, time):
    global best
    if city == destination:
        best = min(best, time)
    for tr in trains_from(city):
        if tr.dep >= time:
            go(tr.dst, tr.arr)
```

TIME	SPACE
exponential	O(trains)

WHY IT IS SLOW

The number of train sequences can be exponential. Because waiting is free, an earlier arrival is never worse, so Dijkstra on earliest arrival times is O((T + C) log C).

OPTIMAL SOLUTION · Dijkstra on earliest arrival times, with the train sequence

TIME	SPACE
O((T + C) log C) with trains sorted by departure per city	O(T + C)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A minimum layover is required when changing trains.

Replace the boarding test dep >= t with dep >= t + layover for every city except the origin. The earliest-arrival property still holds (a later arrival never helps because the layover is the same), so Dijkstra remains valid.

FOLLOW-UP 2 Fewest transfers instead of earliest arrival.

Run BFS over (city) counting trains taken, but track the earliest arrival for each count (a state is (city, number of trains)). Or Dijkstra with the pair (transfers, arrival time) compared lexicographically.

FOLLOW-UP 3 The schedule repeats daily.

Add the day to the time (departure + 24h * k) or treat times modulo 24h and carry the day count. The same Dijkstra works on absolute times.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a train schedule with departure
# city, arrival city, start time, end time, and duration,
# and given a source and destination, find the sequence of
# trains that gets you there at the earliest possible
# time.
#
# A schedule lists trains as (from, to, departure time,
# arrival time). Starting in city src at time start, find
# the sequence of trains that gets you to dst at the
# earliest possible time. You can only board a train that
# leaves at or after the time you arrive in its departure
# city. Return (arrival time, the trains you take), or
# None if dst cannot be reached.
#
# Signature:
#   def earliest_arrival(trains: list, src, dst, start=0)
-> tuple | None
#
# Examples:
# [('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9),
#     ('B', 'C', 2, 5)], 'A', 'C'  ->  (6, [('A', 'B', 1,
#     3), ('B', 'C', 4, 6)])
# [('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'  ->  null
#
# FOLLOW-UPS (answered below, in this order):
# 1. A minimum layover is required when changing trains.
# 2. Fewest transfers instead of earliest arrival.
# 3. The schedule repeats daily.
#
# INTERVIEW NOTES
# Waiting is free, so arriving earlier at a city is never
# worse than arriving later: that is exactly the property
# Dijkstra needs. Keep the earliest known arrival time per
# city in a heap, and from a city at time t relax only the
# trains that depart at or after t. Remember which train
# gave each improvement to rebuild the sequence.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
DAY = 24 * 60   # times are minutes from midnight


class Train:
    def __init__(self, src, dst, dep, arr):
        self.src, self.dst = src, dst
        self.dep, self.arr = dep, arr

# ---- main question: main function ----
# BASE QUESTION (optimal): Dijkstra on earliest arrival
# times, with the train sequence
# Idea: best[city] is the earliest time you can be there.
# FOLLOW-UP 1: A minimum layover is required when changing
# trains.
# Answer: Replace the boarding test dep >= t with dep >= t
# + layover for every city except the origin. The
# earliest-arrival property still holds (a later arrival
# never helps because the layover is the same), so
# Dijkstra remains valid.
def earliest_route(trains, source, target, layover=0):
    # Dijkstra on arrival time. You may board a train if
    # it
    # leaves at or after your arrival plus the layover (no
    # layover is needed at the origin).
    by_city = {}
    for tr in trains:
        by_city.setdefault(tr.src, []).append(tr)
    best = {source: 0}
    took = {}
    heap = [(0, source)]
    while heap:
        time, city = heapq.heappop(heap)
        if time > best.get(city, float("inf")):
            continue
        if city == target:
            break
        # FOLLOW-UP 1: a minimum layover when changing
        # trains
```

```
        wait = 0 if city == source else layover
        for tr in by_city.get(city, []):
            old = best.get(tr.dst, float("inf"))
            if tr.dep >= time + wait and tr.arr < old:
                best[tr.dst] = tr.arr
                took[tr.dst] = tr
                heapq.heappush(heap, (tr.arr, tr.dst))
    if target not in best:
        return None
    # walk back from the target to list the trains used
    path, city = [], target
    while city != source:
        path.append(took[city])
        city = took[city].src
    return best[target], path[::-1]

# ---- main question: tests ----
def test_main_question():
    trains = [Train("A", "B", 60, 120),
              Train("B", "C", 130, 200),
              Train("A", "C", 100, 400),
              Train("B", "C", 110, 150)]
    time, path = earliest_route(trains, "A", "C")
    assert time == 200
    assert [(t.src, t.dst) for t in path] == [("A", "B"),
                                              ("B", "C")]
    # a 15 minute layover rules out the 130 departure
    # (arrive 120, 120 + 15 = 135): the direct train wins
    time, path = earliest_route(trains, "A", "C", 15)
    assert time == 400 and len(path) == 1
    assert earliest_route(trains, "C", "A") is None
    assert earliest_route(trains, "A", "A") == (0, [])
```

```
# ========================================================
# FOLLOW-UP 1: A minimum layover is required when changing
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ========================================================
# FOLLOW-UP 2: Fewest transfers instead of earliest
# ========================================================

# ---- follow-up 2: function ----
# Answer: Run BFS over (city) counting trains taken, but
```

```
# track the earliest arrival for each count (a state is
# (city, number of trains)). Or Dijkstra with the pair
# (transfers, arrival time) compared lexicographically.
def fewest_trains(trains, source, target, layover=0):
    # follow-up: fewest transfers. Round k holds the
    # earliest
    # arrival at every city using at most k trains.
    best = {source: (0, [])}
    for _ in range(len(trains)):
        step = dict(best)
        for tr in trains:
            if tr.src not in best:
                continue
            time, path = best[tr.src]
            wait = 0 if tr.src == source else layover
            if tr.dep < time + wait:
                continue
            old = step.get(tr.dst)
            if old is None or tr.arr < old[0]:
                step[tr.dst] = (tr.arr, path + [tr])
        if target in step:
            return step[target]
        best = step
    return None

# ---- follow-up 2: tests ----
def test_follow_up_2():
    trains = [Train("A", "B", 60, 120),
              Train("B", "C", 130, 200),
              Train("A", "C", 100, 400),
              Train("B", "C", 110, 150)]
    time, path = fewest_trains(trains, "A", "C")
    assert len(path) == 1 and time == 400


# ========================================================
# FOLLOW-UP 3: The schedule repeats daily.
# ========================================================

# ---- follow-up 3: function ----
# Answer: Add the day to the time (departure + 24h * k) or
# treat times modulo 24h and carry the day count. The same
# Dijkstra works on absolute times.
def daily(trains, days):
    # follow-up: the timetable repeats every day. Unroll
```

```
    # it
    # into absolute times; a train that arrives after
    # midnight has an arrival earlier than its departure.
    out = []
    for d in range(days):
        for tr in trains:
            arr = tr.arr + (DAY if tr.arr < tr.dep else 0)
            shift = d * DAY
            out.append(Train(tr.src, tr.dst,
                             tr.dep + shift, arr + shift))
    return out
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    week = daily([Train("A", "B", 1400, 60)], 2)
    # the 23:20 train arrives at 01:00 the next day
    assert week[0].arr == 1500 and week[1].dep == 2840
    # miss the last train of the day: take it tomorrow
    late = daily([Train("A", "B", 600, 700)], 2)
    got = earliest_route(late, "A", "B")
    assert got[0] == 700
    for _ in range(300):
        names = "ABCD"
        trains = []
        for _ in range(random.randint(1, 9)):
            a, b = random.sample(names, 2)
            dep = random.randint(0, 100)
            ride = random.randint(1, 50)
            trains.append(Train(a, b, dep, dep + ride))
        gap = random.choice([0, 5, 20])
        got = earliest_route(trains, "A", "D", gap)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 49 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 4 of 9

Template fit: combine = cost + w; also store parent[v]; stop at the target and rebuild the path | no date

MEDIUM | Exponent | 49 of 288

Find the shortest path in a graph.

ASKED (ANY CO.) ~11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 37.9** | **30 DAYS 33** | **PRIORITY 71/288**

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #1091 Shortest Path in Binary Matrix

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 1091 (inferred from short statement)

Last asked (any company): 11 days ago, about 2026-09 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2094/find-shortest-path-graph

QUESTION

A directed graph is given as a list of weighted edges (u, v, w) with non-negative weights. Return the total length and the nodes of a shortest path from s to t, or None if t cannot be reached.

Function signature

def dijkstra_path(edges: list, s, t) -> tuple | None

(distance, path)

EXAMPLES

Example 1	
Input	`[('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D', 1)], 'A', 'D'`
Output	`(4, ['A', 'B', 'C', 'D'])`
Why	`A -> B -> C -> D costs 1 + 2 + 1 = 4; the direct edge A -> C costs 4, which is longer overall.`

Example 2	
Input	`[('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'`
Output	`null`
Why	`There is no edge leading from D back to A.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(n, edges, source, target):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[target]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Each round relaxes every edge: O(V * E). It is the right tool for negative weights; for non-negative weights Dijkstra's heap is O((V + E) log V), and for unweighted graphs BFS is O(V + E).

OPTIMAL SOLUTION · Dijkstra with a heap (non-negative weights), returning the path

TIME	SPACE
O((V + E) log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is unweighted.

BFS from the source with a parent map gives a shortest path in O(V + E); Dijkstra would be correct but needlessly slow. Stop as soon as you pop the target.

FOLLOW-UP 2 Weights can be negative.

Dijkstra can give wrong answers. Use Bellman-Ford (O(V * E)); one more pass that still improves a distance proves a negative cycle reachable from the source, where shortest paths are undefined.

FOLLOW-UP 3 Weights are only 0 or 1, or the graph is huge and you know the target.

0-1 BFS uses a deque in O(V + E). With a good distance estimate use A* (Dijkstra ordered by distance plus a heuristic that never overestimates); bidirectional search roughly squares the work saved on large graphs.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the shortest path in a graph.
#
# A directed graph is given as a list of weighted edges
# (u, v, w) with non-negative weights. Return the total
# length and the nodes of a shortest path from s to t, or
# None if t cannot be reached.
#
# Signature:
#   def dijkstra_path(edges: list, s, t) -> tuple | None
    # (distance, path)
#
# Examples:
# [('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D',
#     1)], 'A', 'D'  ->  (4, ['A', 'B', 'C', 'D'])
# [('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'
#     ->  null
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is unweighted.
# 2. Weights can be negative.
# 3. Weights are only 0 or 1, or the graph is huge and you
# know the target.
#
# INTERVIEW NOTES
# Ask the graph questions first: directed or undirected,
# weighted or not, can weights be negative, and do you
# want the length or the path. That choice picks the
# algorithm: unweighted means breadth-first search, non-
# negative weights mean Dijkstra with a heap, negative
# weights mean Bellman-Ford. Remember parent pointers if
# the path itself is needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import defaultdict, deque



# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# HELPERS used by the code below
INF = float("inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): Dijkstra with a heap (non-
# negative weights), returning the path
# Idea: Keep the best known distance to each node and a
# min-heap of (distance, node). Pop the closest unsettled
# node, relax its edges, and remember the parent of every
# improvement.
def dijkstra_path(edges, s, t):
    # weighted, non-negative edges (u, v, w)
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    dist, parent, heap = {s: 0}, {s: None}, [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if u == t:
            # the first pop of t is final: rebuild the
            # path
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return d, path[::-1]
        for v, w in graph[u]:
            if d + w < dist.get(v, INF):
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    return None

# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert dijkstra_path(edges, 0, 3) == (4, [0, 2, 1, 3])
    assert dijkstra_path(edges, 3, 0) is None
    assert dijkstra_path(edges, 0, 0) == (0, [0])


# ============================================================
```

```
# FOLLOW-UP 1: The graph is unweighted.
# ============================================================

# ---- follow-up 1: function ----
# Answer: BFS from the source with a parent map gives a
# shortest path in O(V + E); Dijkstra would be correct but
# needlessly slow. Stop as soon as you pop the target.
def bfs_path(edges, s, t):
    # unweighted: BFS is enough, O(V + E)
    graph = defaultdict(list)
    for u, v, *_ in edges:
        graph[u].append(v)
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        if u == t:
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return len(path) - 1, path[::-1]
        for v in graph[u]:
            if v not in parent:
                parent[v] = u
                queue.append(v)
    return None

# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert bfs_path(edges, 0, 3) == (2, [0, 1, 3])
    neg = [(0, 1, 5), (1, 2, -3)]


# ============================================================
# FOLLOW-UP 2: Weights can be negative.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Dijkstra can give wrong answers. Use Bellman-
# Ford (O(V * E)); one more pass that still improves a
# distance proves a negative cycle reachable from the
# source, where shortest paths are undefined.
def bellman_ford(n, edges, s):
    # negative weights allowed: n - 1 rounds of relaxing
```

```
    # every
    # edge; one more improving round proves a negative
    # cycle
    dist = [INF] * n
    dist[s] = 0
```

```
    for _ in range(n - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            raise ValueError("negative cycle")
    return dist

# ---- follow-up 2: tests ----
def test_follow_up_2():
    neg = [(0, 1, 5), (1, 2, -3)]
    assert bellman_ford(4, neg, 0) == [0, 5, 2, INF]
    try:
        bellman_ford(2, [(0, 1, 1), (1, 0, -3)], 0)
        raise AssertionError("negative cycle not found")
    except ValueError:
        pass
    cheap = [(0, 1, 1), (1, 2, 0), (0, 2, 1)]


# ============================================================
# FOLLOW-UP 3: Weights are only 0 or 1, or the graph is
# ============================================================

# ---- follow-up 3: function ----
# Answer: 0-1 BFS uses a deque in O(V + E). With a good
# distance estimate use A* (Dijkstra ordered by distance
# plus a heuristic that never overestimates);
# bidirectional search roughly squares the work saved on
# large graphs.
def zero_one_bfs(n, edges, s):
    # weights are only 0 or 1: a deque keeps the order
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
```

```
    dist = [INF] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # a free edge goes to the front, a cost-1
                # edge back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist

# ---- follow-up 3: tests ----
def test_follow_up_3():
    cheap = [(0, 1, 1), (1, 2, 0), (0, 2, 1)]
    assert zero_one_bfs(3, cheap, 0) == [0, 1, 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 50 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 5 of 9

Template fit: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell) | no date

MEDIUM | Exponent | 50 of 288

Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 162/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1102 Path With Maximum Minimum Value, LeetCode #1730 Shortest Path to Get Food

LeetCode match: No direct match; related LC 1102 Path With Maximum Minimum Value, LC 1730 (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5244/safest-path-maximize-minimum-distance-matrix

QUESTION

A grid contains a rat R, a piece of bread B, a cat C, walls

and open cells '.'. Move

the rat from R to B through open cells (up, down, left, right). The safety of a path is the smallest distance to the cat of any cell on it (distances count steps around the walls). Return the largest safety you can achieve, or -1 if the bread cannot be reached.

Function signature

def safest_path(grid: list) -> int

EXAMPLES

Example 1	
Input	`['R..', '...', 'C.B']`
Output	`2`
Why	`The path along the top and right side keeps at least 2 steps from the cat at the bottom left.`

Example 2	
Input	`['R#B']`
Output	`-1`
Why	`The wall blocks the only route to the bread.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    dist = bfs_from_cat(grid)  # per cell
    best = -1
    for path in all_paths(grid, RAT, BREAD):
        cells = [dist[r][c] for r, c in path]
        closest = min(cells)
        best = max(best, closest)
    return best
```

TIME	SPACE
exponential	O(path length)

WHY IT IS SLOW

The number of simple paths grows exponentially with the grid. A max-heap search that extends the path with the best bottleneck so far (or binary search on the threshold) is O(m * n log(m * n)).

OPTIMAL SOLUTION · Multi-source BFS from the cat, then a max-heap bottleneck search

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use binary search instead of a heap.

Binary search the safety threshold d and test with BFS whether the rat can reach the bread using only cells at least d from the cat. The test is monotone in d. O(m * n log(m + n)).

FOLLOW-UP 2 Among the safest paths, return the shortest one.

First find the best safety value d*, then run an ordinary BFS restricted to cells at distance at least d*. The result is the shortest path with maximum safety.

FOLLOW-UP 3 The cat moves one step toward the rat each turn.

Then distances depend on time, and the problem becomes a game or a time-expanded graph search; ask for the cat's movement rules before designing anything.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a matrix with a rat, bread, cat,
# and obstacles, find a path from the rat to the bread
# that maximizes the minimum distance from cat.
#
# A grid contains a rat R, a piece of bread B, a cat C,
# walls # and open cells '.'. Move the rat from R to B
# through open cells (up, down, left, right). The safety
# of a path is the smallest distance to the cat of any
# cell on it (distances count steps around the walls).
# Return the largest safety you can achieve, or -1 if the
# bread cannot be reached.
#
# Signature:
#   def safest_path(grid: list) -> int
#
# Examples:
# ['R..', '...', 'C.B']  ->  2
# ['R#B']  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use binary search instead of a heap.
# 2. Among the safest paths, return the shortest one.
# 3. The cat moves one step toward the rat each turn.
#
# INTERVIEW NOTES
# Two separate ideas. First, how far every cell is from
# the cat: that is one BFS from the cat (all cats at
# once). Second, choose the path whose worst cell is as
# far as possible: this is a 'bottleneck' (maximin) path.
# A max-heap search that always extends the path with the
# best bottleneck so far (Dijkstra with min instead of
# sum) solves it; binary search on the threshold plus a
# reachability check also works.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import deque
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
INF = 10 ** 9


def around(r, c, grid):
    # the open neighbours of a cell (obstacles are "#")
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                yield nr, nc


def find(grid, ch):
    for r, row in enumerate(grid):
        if ch in row:
            return r, row.index(ch)
    return None
```

```
def cat_distances(grid):
    # one BFS from the cat gives every cell's distance to
    # it
    dist = [[INF] * len(grid[0]) for _ in grid]
    cat = find(grid, "C")
    if cat is None:
        return dist
    dist[cat[0]][cat[1]] = 0
    queue = deque([cat])
    while queue:
        r, c = queue.popleft()
        for nr, nc in around(r, c, grid):
            if dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist

# ---- main question: main function ----
# BASE QUESTION (optimal): Multi-source BFS from the cat,
```

```
# then a max-heap bottleneck search
# Idea: BFS from every cat gives the distance of each
# cell.
def safest_value(grid):
    # a path is as safe as its closest cell to the cat.
    # Always extend the safest frontier cell (like
    # Dijkstra
    # with max and min instead of min and sum).
    dist = cat_distances(grid)
    start, goal = find(grid, "R"), find(grid, "B")
    best = {start: dist[start[0]][start[1]]}
    heap = [(-best[start], start)]
    while heap:
        neg, cell = heapq.heappop(heap)
        if cell == goal:
            return -neg
        if -neg < best[cell]:
            continue
        for nr, nc in around(cell[0], cell[1], grid):
            safety = min(-neg, dist[nr][nc])
            if safety > best.get((nr, nc), -1):
                best[(nr, nc)] = safety
                heapq.heappush(heap, (-safety, (nr, nc)))
    return None

# ---- main question: tests ----
def test_main_question():
    grid = ["R.....",
            ".###..",
            "...C.B"]
    assert cat_distances(grid)[0][0] == 5
    # the bread itself is 2 steps from the cat
    assert safest_value(grid) == 2


# ============================================================
# FOLLOW-UP 1: Use binary search instead of a heap.
# ============================================================

# ---- follow-up 1: helper functions ----
def reach(grid, dist, d):
    # BFS using only cells at least d away from the cat;
    # returns the shortest such path (list of cells) or
    # None
    start, goal = find(grid, "R"), find(grid, "B")
```

```
    if dist[start[0]][start[1]] < d:
        return None
    parent = {start: None}
    queue = deque([start])
    while queue:
        cell = queue.popleft()
        if cell == goal:
```

```
            path = []
            while cell:
                path.append(cell)
                cell = parent[cell]
            return path[::-1]
        for nxt in around(cell[0], cell[1], grid):
            safe = dist[nxt[0]][nxt[1]] >= d
            if nxt not in parent and safe:
                parent[nxt] = cell
                queue.append(nxt)
    return None

# ---- follow-up 1: function ----
# Answer: Binary search the safety threshold d and test
# with BFS whether the rat can reach the bread using only
# cells at least d from the cat. The test is monotone in
# d. O(m * n log(m + n)).
def safest_by_search(grid):
    # follow-up: binary search the safety d. "reachable
    # using
    # only cells >= d" is true for small d and false for
    # big.
    dist = cat_distances(grid)
    lo, hi = 0, INF
    if reach(grid, dist, 0) is None:
        return None
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if reach(grid, dist, mid):
            lo = mid
        else:
            hi = mid - 1
    return lo

# ---- follow-up 1: tests ----
def test_follow_up_1():
    grid = ["R.....",
            ".###..",
            "...C.B"]
    assert safest_by_search(grid) == 2


# ============================================================
# FOLLOW-UP 2: Among the safest paths, return the shortest
# ============================================================

# ---- follow-up 2: function ----
# Answer: First find the best safety value d*, then run an
# ordinary BFS restricted to cells at distance at least
# d*. The result is the shortest path with maximum safety.
def shortest_safest_path(grid):
    # follow-up: among the safest paths, the shortest one
    best = safest_value(grid)
    if best is None:
        return None
    return reach(grid, cat_distances(grid), best)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    grid = ["R.....",
            ".###..",
            "...C.B"]
    path = shortest_safest_path(grid)
    assert path[0] == (0, 0) and path[-1] == (2, 5)
    dist = cat_distances(grid)
    assert min(dist[r][c] for r, c in path) == 2
    walled = ["R#B"]
    assert safest_value(walled) is None
    assert safest_by_search(walled) is None
    # no cat at all: every path is perfectly safe
    assert safest_value(["R.B"]) == INF


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")

if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 51 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 6 of 9

Template fit: the same minimax loop on a height grid | asked 289 days ago

HARD | IN APP LC 778, 1631 | PracHub | Technical Screen | 51 of 288

Minimize maximum height along a grid path

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 61/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #778 Swim in Rising Water, LeetCode #1631 Path with Minimum Effort

LeetCode match: LC 778 Swim in Rising Water / LC 1631

Posted on PracHub: 2026-01-04 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/minimize-maximum-height-along-a-grid-path

QUESTION

You are given a grid of heights and walk from the top-left cell to the bottom-right cell, moving up, down, left or right. The cost of a path is the largest height of any cell on it. Return the smallest possible cost (and, with with_path=True, a path that achieves it).

Function signature

def min_max_path(H: list, with_path=False)

EXAMPLES

Example 1	
Input	`[[1, 9], [2, 3]]`
Output	`3`
Why	`Down then right passes cells 1, 2, 3: the largest is 3; going right first would have to cross the 9.`

Example 2	
Input	`[[1, 2], [4, 3]], with_path = true`
Output	`(3, [(0, 0), (0, 1), (1, 1)])`
Why	`With with_path=True you also get the route.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimize maximum height along a grid path
#
# You are given a grid of heights and walk from the top-
# left cell to the bottom-right cell, moving up, down,
# left or right. The cost of a path is the largest height
# of any cell on it. Return the smallest possible cost
# (and, with with_path=True, a path that achieves it).
#
# Signature:
#   def min_max_path(H: list, with_path=False)
#
```

```
# Examples:
# [[1, 9], [2, 3]]  ->  3
# [[1, 2], [4, 3]], with_path = true  ->  (3, [(0, 0), (0,
#     1), (1, 1)])
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


def reachable(H, limit):
    # flood fill using only cells with height <= limit
    m, n = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen, stack = {(0, 0)}, [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (m - 1, n - 1):
            return True
        for a, b in around(r, c, m, n):
            if (a, b) not in seen and H[a][b] <= limit:
                seen.add((a, b))
                stack.append((a, b))
    return False

# ---- main question: main function ----
# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
```

```
def min_max_path(H, with_path=False):
    # Dijkstra where a path costs the MAX of its cells
    m, n = len(H), len(H[0])
    best, parent = {(0, 0): H[0][0]}, {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
```

```
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))


# ALTERNATIVE: binary search the answer
def min_max_path_bsearch(H):
    # binary search the answer over the sorted heights
    heights = sorted({h for row in H for h in row})
    lo, hi = 0, len(heights) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if reachable(H, heights[mid]):
            hi = mid
        else:
            lo = mid + 1
    return heights[lo]

# ---- main question: tests ----
def test_main_question():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert min_max_path(H) == min_max_path_bsearch(H)
    cost, path = min_max_path(H, True)
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[7]]) == 7
    grid = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    assert min_max_path(grid) == 6
    assert min_max_path_bsearch(grid) == 6


# ============================================================
# FOLLOW-UP 1: The cost is the largest difference between
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height DIFFERENCE
    m, n = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in around(r, c, m, n):
```

```
            nxt = max(cost, abs(H[a][b] - H[r][c]))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_effort_path([[1, 2, 2], [3, 8, 2],
                            [5, 3, 5]]) == 2


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Test cases

Input	Expected
([[1, 2, 3], [4, 5, 6], [7, 8, 9]],)	9
([[8, 4, 7], [6, 5, 9], [3, 2, 1]],)	8

ALSO REPORTED · 1 more

Path Minimizing the Maximum Cell (Swim in Rising Water) | 1Point3Acres | onsite-coding | asked 2026-07-08 | Low

https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter. Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

ALSO REPORTED · 1 more

Preparation: Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation. Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY PATH: question 52 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 7 of 9

Template fit: the same minimax loop: the smallest threshold that lets you walk from corner to corner | asked 251 days ago

HARD | IN APP LC 1631, 778 | PracHub | Technical Screen | 52 of 288

Find minimum threshold enabling grid path

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 60/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1631 Path with Minimum Effort, LeetCode #778 Swim in Rising Water

LeetCode match: LC 1631 Path With Minimum Effort / LC 778 Swim in Rising Water (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/find-minimum-threshold-enabling-grid-path

QUESTION

You are given a grid of heights. Find the smallest threshold T such that you can walk from the top-left cell to the bottom-right cell, moving up, down, left or right, using only cells of height at most T. Return T together with one path that achieves it.

Function signature

def min_max_path(H: list) -> (int, list)

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 2]]`
Output	`(2, [(0, 0), (1, 0), (1, 1)])`
Why	`Going down then right uses cells 1, 2, 2: the largest is 2, and no path stays below that.`

Example 2	
Input	`[[7]]`
Output	`(7, [(0, 0)])`
Why	`A single cell: the threshold is its own height.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find minimum threshold enabling grid path
#
# You are given a grid of heights. Find the smallest
# threshold T such that you can walk from the top-left
# cell to the bottom-right cell, moving up, down, left or
# right, using only cells of height at most T. Return T
# together with one path that achieves it.
#
# Signature:
#   def min_max_path(H: list) -> (int, list)
```

```
#
# Examples:
# [[1, 3], [2, 2]]  ->  (2, [(0, 0), (1, 0), (1, 1)])
# [[7]]  ->  (7, [(0, 0)])
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b

# ---- main question: main function ----
# ALSO: the Dijkstra version
def best_path(H, step_cost, start_cost):
    # Dijkstra where a path costs the MAX of its steps;
    # the
    # first time a cell leaves the heap its cost is final
    m, n = len(H), len(H[0])
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, step_cost(H, r, c, a, b))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
    return None, []


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
```

```
def min_max_path(H):
    # cost of a path = its largest cell value
    # (water rising over time is the same problem)
    def step(H, r, c, a, b):
        return H[a][b]
    return best_path(H, step, H[0][0])

# ---- main question: tests ----
def test_main_question():
    H = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    cost, path = min_max_path(H)
    # 0 -> 1 -> 3 -> 7 -> 6 or via 2, 5, 6: the max is 6
    assert cost == 6
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[5]]) == (5, [(0, 0)])


# ============================================================
# FOLLOW-UP 1: The cost is the largest difference between
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height difference
    # between two neighbouring cells on the path
    def step(H, r, c, a, b):
        return abs(H[a][b] - H[r][c])
    return best_path(H, step, 0)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    effort, path2 = min_effort_path([[1, 2, 2], [3, 8, 2],
                                     [5, 3, 5]])
    assert effort == 2
    assert min_effort_path([[7]]) == (0, [(0, 0)])


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 2], [1, 3]])
```

Test cases

Input	Expected
([[0, 2], [1, 3]],)	3
([[0, 100, 2], [1, 2, 2], [3, 4, 1]],)	2

STUDY PATH: question 53 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 8 of 9

Template fit: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd | asked 287 days ago

HARD | PracHub | Technical Screen | 53 of 288

Minimize travel time with optional meeting point

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 66/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #2203 Minimum Weighted Subgraph With the Required Paths

LeetCode match: LC 743 Network Delay Time (multi-source Dijkstra); related LC 2203 (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/minimize-travel-time-with-optional-meeting-point

QUESTION

A weighted graph has n nodes and edges (u, v, w) (undirected unless directed=True). Two people start at nodes a and b and both must end at dest. They may meet at some node m and walk the rest of the way together, so the total time is max(time a to m, time b to m) + time m to dest. Return [the minimum total time, the meeting node] (or [-1, -1] when dest cannot be reached). Meeting at dest itself, or at a or b, is allowed.

Function signature

def meeting_point(n: int, edges: list, a: int, b: int, dest: int, directed=False) -> list

EXAMPLES

Example 1	
Input	`5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4`
Output	`[7, 1]`
Why	`They meet at node 1: a needs 2, b needs 1, then 5 more to node 4: 2 + 5 = 7. Meeting at node 4 itself also takes 7, and the smaller node number is reported.`

Example 2	
Input	`3, [(0, 1, 1)], 0, 1, 2`
Output	`[-1, -1]`
Why	`Node 2 is not connected to anything.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, A, B, dest):
    d = floyd_warshall(n, edges)  # all pairs
    best, best_m = INF, None
    for m in range(n):
        cost = max(d[A][m], d[B][m]) + d[m][dest]
        if cost < best:
            best, best_m = cost, m
    return best
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) when only three single-source distances are needed. Dijkstra from A, from B and into the destination is O((V + E) log V).

OPTIMAL SOLUTION · Three Dijkstra runs (from A, from B, and into the destination)

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

FOLLOW-UP 2 Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

FOLLOW-UP 3 Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimize travel time with optional
# meeting point
#
# A weighted graph has n nodes and edges (u, v, w)
# (undirected unless directed=True). Two people start at
# nodes a and b and both must end at dest. They may meet
# at some node m and walk the rest of the way together, so
# the total time is max(time a to m, time b to m) + time m
# to dest. Return [the minimum total time, the meeting
# node] (or [-1, -1] when dest cannot be reached). Meeting
# at dest itself, or at a or b, is allowed.
#
# Signature:
#   def meeting_point(n: int, edges: list, a: int, b: int,
 dest: int, directed=False) -> list
#
# Examples:
# 5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4  ->  [7,
#     1]
# 3, [(0, 1, 1)], 0, 1, 2  ->  [-1, -1]
#
# FOLLOW-UPS (answered below, in this order):
# 1. More than two travellers.
# 2. Meeting is optional.
# 3. Edge weights change over time (traffic).
#
# INTERVIEW NOTES
# Break it into shortest paths from three places: from A,
# from B, and into the destination (a Dijkstra on the
# reversed graph if directed). For each node m the cost is
# max(distA[m], distB[m]) + dist(m to destination); take
# the minimum. Three single-source searches replace trying
# every meeting node with its own search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
INF = float("inf")


def dijkstra(graph, source):
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # forward graph, and the reversed graph for "to dest"
```

```
    fwd = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]
    for u, v, w in edges:
        fwd[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            fwd[v].append((u, w))
            rev[u].append((v, w))
    return fwd, rev

# ---- main question: main function ----
# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected).
def meeting_point(n, edges, a, b, dest, directed=False):
    # both start at the same time, meet at node m, wait
    # for
    # each other, then travel together to dest. Meeting at
```

```
    # dest itself covers the "no meeting" plan.
    fwd, rev = build(n, edges, directed)
    da, db = dijkstra(fwd, a), dijkstra(fwd, b)
    # distance from every node TO dest = search the
    # reverse graph
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
    for m in range(n):
        total = max(da[m], db[m]) + to_dest[m]
        if total < best[0]:
            best = (total, m)
    return [-1, -1] if best[0] == INF else list(best)

# ---- main question: tests ----
def test_main_question():
    # a star: both travellers reach the hub 2, then go to
    # 3
    edges = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9),
             (1, 3, 9)]
    # meet at 2: max(1, 2) + 4 = 6; going alone costs 9
    assert meeting_point(4, edges, 0, 1, 3) == [6, 2]
    # nobody can reach the destination
    short = [(0, 1, 1)]
    assert meeting_point(3, short, 0, 1, 2) == [-1, -1]
    # same start: they simply walk together
    assert meeting_point(4, edges, 0, 0, 3)[0] == 5


# ============================================================
# FOLLOW-UP 1: More than two travellers.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run one Dijkstra per traveller and one from the
# destination, then for each node take the maximum arrival
# time (if they wait for each other) or the sum of
# distances (if the cost is total distance) plus the
# shared leg.
def meeting_point_many(n, edges, starts, dest,
                       directed=False):
    # k travellers: one search each, plus one from dest
    fwd, rev = build(n, edges, directed)
    dists = [dijkstra(fwd, s) for s in starts]
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
```

```
    for m in range(n):
        wait = max(d[m] for d in dists)
        if wait + to_dest[m] < best[0]:
            best = (wait + to_dest[m], m)
    return [-1, -1] if best[0] == INF else list(best)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    # a star: both travellers reach the hub 2, then go to
    # 3
    edges = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9),
             (1, 3, 9)]
    # three travellers
    got = meeting_point_many(4, edges, [0, 1, 1], 3)
    assert got == [6, 2]
    # directed roads: reverse edges are not usable
    one_way = [(0, 1, 1), (1, 2, 1)]
    walk = meeting_point(3, one_way, 0, 1, 2, True)
    assert walk == [2, 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]], startA: int,
startB: int, dest: int, directed: bool = False) -> list[in
t]:
```

Sample call

```
solution(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)],
0, 2, 3, False)
```

Test cases

Input	Expected
(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False)	[4, 1]
(4, [(0, 1, 2), (2, 1, 3), (1, 3, 2), (0, 3, 10), (2, 3, 5)], 0, 2, 3, True)	[5, 1]

STUDY PATH: question 54 of 288 | topic 6 of 43: Best-first search with a heap: Dijkstra and its minimax twin | 9 of 9

Template fit: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell | asked 271 days ago

HARD | PracHub | Technical Screen | 54 of 288

Compute minimax grid path and network delay

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 113/288

ABOUT THIS QUESTION

Topic: Best-first search with a heap: Dijkstra and its minimax twin

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1631 Path with Minimum Effort, LeetCode #743 Network Delay Time

LeetCode match: LC 1631 Path With Minimum Effort + LC 743 Network Delay Time (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-minimax-grid-path-and-network-delay

QUESTION

Two tasks. (1) minimax_path(H): given a grid of heights, find a path from the top-left to the bottom-right cell (moving up, down, left or right) whose largest height is as small as possible, and return that largest height. (2) network_delay(n, edges, source, node_delay=None): a signal starts at source and travels along directed weighted edges (u, v, w); return the time until every node has received it, or -1 if some node never does. node_delay[u], when given, is a pause at node u before it forwards the signal.

Function signature

def minimax_path(H: list) -> int
def network_delay(n: int, edges: list, source: int, node_delay=None) -> int

EXAMPLES

Example 1

Input	`[[1, 5, 1], [2, 2, 5]]`
Output	`5`
Why	`Both ways around pass a cell of height 5, so the answer is 5.`

Example 2

Input	`4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0`
Output	`10`
Why	`The signal reaches node 1 at 2, node 2 at 5, node 3 at 10: everyone has it after 10.`

QUESTION

Example 3

Input	`3, [(0, 1, 1)], 0`
Output	`-1`
Why	`Node 2 has no incoming edge: it is never reached.`

Example 4

Input	`4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0, node_delay = [1, 0, 0, 0]`
Output	`11`
Why	`With a pause of 1 at node 0, every arrival is 1 later: node 3 is reached at 11.`

BRUTE FORCE · Part 1: DFS over every simple path

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Dijkstra where the cost of a path is its maximum cell (or a union-find over cells added in increasing order) is O(m * n log(m * n)).

BRUTE FORCE · Part 2: Bellman-Ford from the source

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    worst = max(dist)
    return -1 if worst == INF else worst
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

V - 1 rounds over all edges, O(V * E). Dijkstra with a heap is O((V + E) log V) for non-negative weights.

OPTIMAL SOLUTION · Part 1: Dijkstra where the cost of a path is its maximum cell

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: Dijkstra from the source, answer is the largest distance

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

FOLLOW-UP 2 Part 2: weights are huge.

Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. **Cost:** unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

FOLLOW-UP 3 Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.

A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute minimax grid path and network
# delay
#
# Two tasks. (1) minimax_path(H): given a grid of heights,
# find a path from the top-left to the bottom-right cell
# (moving up, down, left or right) whose largest height is
# as small as possible, and return that largest height.
# (2) network_delay(n, edges, source, node_delay=None): a
# signal starts at source and travels along directed
# weighted edges (u, v, w); return the time until every
# node has received it, or -1 if some node never does.
# node_delay[u], when given, is a pause at node u before
# it forwards the signal.
#
# Signature:
#   def minimax_path(H: list) -> int
#   def network_delay(n: int, edges: list, source: int, no
de_delay=None) -> int
#
# Examples:
# [[1, 5, 1], [2, 2, 5]]  ->  5
# 4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0  ->  10
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 1 with a union-find (sort cells, add them one by
# one).
# 2. Part 2: weights are huge.
# 3. Part 2: the signal can be blocked at some nodes, or
# each node adds a processing delay.
#
# INTERVIEW NOTES
# Both are 'shortest path' with a different notion of
# cost. For the grid the cost of a path is its maximum,
# which still works in Dijkstra: the best way to reach a
# cell is the one with the smallest maximum, and extending
# a path can never lower it. For the network, Dijkstra
# from the source gives every arrival time, and the answer
# is the largest of them (-1 if some node is never
# reached).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
import heapq


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")


def neighbors(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b

# ---- main question: main function ----
# BASE QUESTION (optimal): Dijkstra where the cost of a
# path is its maximum cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller.
def minimax_path(H):
    # Part 1: the path whose highest cell is as low as
    # possible
    m, n = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
```

```
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in neighbors(r, c, m, n):
            # the cost of a path is its maximum, never a
            # sum
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))
    return -1
```

```
# PART 2: network delay
def network_delay(n, edges, source, node_delay=None):
    # Part 2: when does the signal reach every node?
    # edges are (u, v, weight); -1 if some node is
    # unreachable
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        # a processing delay is paid when leaving a node
        pause = node_delay[u] if node_delay else 0
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
    worst = max(dist)
    return -1 if worst == INF else worst

# ---- main question: tests ----
def test_main_question():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == 2


# ============================================================
# FOLLOW-UP 1: Part 1 with a union-find (sort cells, add
# ============================================================

# ---- follow-up 1: function ----
# Answer: Add cells in increasing order of height, union
# neighbours already added, and stop when the start and
# the goal are connected: that height is the answer. O(mn
# log mn) for the sort.
def minimax_path_union_find(H):
    # the same answer: add cells from lowest to highest
    # and
    # stop when the start and the goal are connected
```

```
    m, n = len(H), len(H[0])
    parent = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    cells = sorted((H[r][c], r, c)
                   for r in range(m) for c in range(n))
    for h, r, c in cells:
        parent[(r, c)] = (r, c)
        for a, b in neighbors(r, c, m, n):
            if (a, b) in parent:
                parent[find((r, c))] = find((a, b))
        if (0, 0) in parent and (m - 1, n - 1) in parent:
            if find((0, 0)) == find((m - 1, n - 1)):
                return h
    return -1

# ---- follow-up 1: tests ----
def test_follow_up_1():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == minimax_path_union_find(H)
    assert minimax_path([[5]]) == 5
    edges = [(0, 1, 2), (1, 2, 3), (0, 2, 10)]
    assert network_delay(3, edges, 0) == 5
    # a node that cannot be reached
    assert network_delay(4, edges, 0) == -1
    # huge weights are fine with Python integers
    big = [(0, 1, 10 ** 9), (1, 2, 10 ** 9)]
    assert network_delay(3, big, 0) == 2 * 10 ** 9
    # each node adds a processing delay of 1
    slow = network_delay(3, big, 0, [1, 1, 1])
    assert slow == 2 * 10 ** 9 + 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(H: list[list[int]]) -> int:
SAMPLE INPUT
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]
```

Test cases

Input	Expected
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]	3
[[5, 1, 7], [4, 8, 2], [3, 6, 0]]	6

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 7 OF 43 |

Backtracking: choose, recurse, undo

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

8	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Backtracking: choose, recurse, undo | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

M	55. Mahjong Winning Hand Detection (14 Tiles)	1Point3Acres	2026-10-09
M	56. Count Prefix-Matching Simple Paths in a Labeled Graph	PracHub	2026-08-09
M	57. Generate Parentheses	Exponent	no date
M	58. American Football Score Combinations	ColorStack	2025-08-25
M	59. Permutations	Exponent	no date
M	60. Determine if a 14-tile hand is winning	PracHub	2026-03-09
M	61. Count Passcode Patterns on a Keypad	ColorStack	2025-11-07
M	62. Match people to questions using tags and priorities	PracHub	2025-12-27
H	63. Matrix Flower Placement with House Adjacency	1Point3Acres	2026-05-31

CODE TEMPLATE | GROUP 7 OF 43 | 9 QUESTIONS, ONE PATTERN

Backtracking: choose, recurse, undo

The one idea these questions share

Build the answer one choice at a time in a list. When the list is complete, record it. Otherwise loop over the allowed choices: append one, recurse, and then remove exactly what you added so the next choice starts from the same state. Pruning is just a check that skips a choice.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def permutations(nums):
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):                # CHANGE 1: when a path is complete
            out.append(cur[:])
            return
        for i, x in enumerate(nums):             # CHANGE 2: the choices at each step
            if not used[i]:                      # CHANGE 3: the rule that prunes a choice
                used[i] = True
                cur.append(x)
                go()
                cur.pop()                        # undo exactly what you did
                used[i] = False

    go()
    return out
```

2. Helper functions that usually get written next to it

```
def combinations(nums, k):                       # same shape, with a start index so order does not matter
    out, cur = [], []

    def go(start):
        if len(cur) == k:
            out.append(cur[:])
            return
        for i in range(start, len(nums)):
            cur.append(nums[i])
            go(i + 1)
            cur.pop()

    go(0)
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
55	Mahjong Winning Hand Detection (14 Tiles)	take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try
56	Count Prefix-Matching Simple Paths in a Labeled Graph	extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth
57	Generate Parentheses	choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n
58	American Football Score Combinations	choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0
59	Permutations	the template as written

Contents Start the questions >

#	Question	What you change in the template
60	Determine if a 14-tile hand is winning	the same try-triple-then-run recursion over a count array, after choosing the pair
61	Count Passcode Patterns on a Keypad	the choice is the next adjacent unused key; complete at k keys; count instead of storing
62	Match people to questions using tags and priorities	try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure
63	Matrix Flower Placement with House Adjacency	place row by row; reject a row early when the houses two rows above are still not next to a flower

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 55 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 1 of 9

Template fit: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try | asked 11 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 55 of 288

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 270/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 31]`
Output	`true`
Why	`Tiles are numbers: 1-9 one suit, 11-19 a second suit, 21-29 a third, 31 and up honours. Pair 31-31, triplets 1-1-1 and 21-21-21, and sequences 2-3-4 and 11-12-13.`

Example 2	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 32]`
Output	`false`
Why	`31 and 32 are different honours, so there is no pair to finish the hand.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Mahjong Winning Hand Detection (14 Tiles)
#
# Mountain View onsite coding round 1: given a hand of 14
# tiles, decide whether it is a winning mahjong hand. The
# candidate must clarify the rule set (typically: 4 melds
# of three identical tiles or three consecutive ranks + 1
# pair). Input: array of 14 integers representing tile
# values (clarify suit / range during the interview). A
# winning hand consists of: One pair (two identical
# tiles), and Four melds, each being either: A triplet (3
# identical tiles), or A run (3 consecutive ranks in the
# same suit). Minority variant: A newer version instead
# specifies one pair plus two melds while retaining a
# 14-tile input. Because that accounts for only eight
# tiles, clarify the intended meld count and how the
# remaining six tiles should be handled before coding.
# Output: boolean.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A variant needs only two melds plus a pair (8 tiles
# of 14).
#
# INTERVIEW NOTES
# A winning hand is one pair plus four melds (a triplet of
# identical tiles or a run of three consecutive ranks in
# one suit). Try every tile as the pair, then repeatedly
# take the smallest remaining tile: it can only be covered
# by a triplet of itself or by a run that starts at it, so
# there are at most two choices at each step and the
# search is tiny for 14 tiles.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False

# ---- main question: main function ----
# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
# Answer: Ask what happens to the other six tiles before
# coding: if they are free, check whether any 8 tiles form
# a pair and two melds with the same search over subsets
```

```
# of the 14.
def is_winning_hand(hand, melds=4):
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False

# ---- main question: tests ----
def test_main_question():
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])


# ============================================================
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY PATH: question 56 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 2 of 9

Template fit: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth | asked 72 days ago

MEDIUM | PracHub | Technical Screen | 56 of 288

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 72 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 0 | PRIORITY 96/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

A directed graph has labelled vertices (labels[i] is the label of vertex i) and edges (u, v). A simple path never visits a vertex twice. For a query word, count for every prefix length L (1 up to the length of the word) the number of simple paths with L vertices whose labels spell the first L letters of the word. Return the list of counts.

Function signature

def prefix_paths(labels: list, edges: list, query: str) -> list

EXAMPLES

Example 1	
Input	`['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'`
Output	`[2, 2, 2]`
Why	`Two paths spell 'a' (vertices 0 and 3), and both continue to b (vertex 1) and then c (vertex 2).`

Example 2	
Input	`['a', 'a'], [(0, 1), (1, 0)], 'aaa'`
Output	`[2, 2, 0]`
Why	`Two paths spell 'a' and two spell 'aa' (0 -> 1 and 1 -> 0); none spells 'aaa' because that would reuse a vertex.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Prefix-Matching Simple Paths in a
# Labeled Graph
#
# A directed graph has labelled vertices (labels[i] is the
# label of vertex i) and edges (u, v). A simple path never
# visits a vertex twice. For a query word, count for every
# prefix length L (1 up to the length of the word) the
# number of simple paths with L vertices whose labels
# spell the first L letters of the word. Return the list
# of counts.
#
# Signature:
#   def prefix_paths(labels: list, edges: list, query: str
) -> list
#
# Examples:
# ['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'
#     ->  [2, 2, 2]
# ['a', 'a'], [(0, 1), (1, 0)], 'aaa'  ->  [2, 2, 0]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges are undirected.
# 2. Count only the paths that spell the whole query.
# 3. The graph has up to 100,000 vertices.
#
# INTERVIEW NOTES
# A path is only worth extending while its labels still
# match the query, so prune on the label before going
# deeper. Distinctness is tracked with a visited set (a
# bitmask for small graphs). Count at every depth, since
# each prefix length has its own answer.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts


# VARIANT: walks may revisit vertices
def prefix_walks(labels, edges, query):
    # walks may revisit vertices: a DP over (depth,
    # vertex)
    # costs O(L * E) and works for very large graphs
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    ways = {v: 1 for v in range(len(labels))
            if query and labels[v] == query[0]}
    counts = [sum(ways.values())] if query else []
    for ch in query[1:]:
        nxt = defaultdict(int)
        for u, n in ways.items():
            for w in adj[u]:
                if labels[w] == ch:
                    nxt[w] += n
        ways = nxt
        counts.append(sum(ways.values()))
    return counts

# ---- main question: tests ----
def test_main_question():
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # "aba": directed square a>b>a>b>...
    assert prefix_paths(labels, edges, "aba") == [2, 2, 2]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Count only the paths that spell the whole
# ============================================================

# ---- follow-up 2: function ----
# Answer: Return counts[-1] instead of the list. For long
# queries on bigger graphs, memoise on (vertex, depth) if
# repeated vertices are allowed (walks, not simple paths);
# for simple paths the visited set prevents memoisation.
def whole_query_paths(labels, edges, query,
                      undirected=False):
    # only the paths that spell the entire query
```

```
    both = undirected
    counts = prefix_paths(labels, edges, query, both)
    return counts[-1] if counts else 0

# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    assert whole_query_paths(labels, edges, "aba") == 2
    # undirected: every edge works both ways
    und = prefix_paths(labels, edges, "ab", True)
    assert und == [2, 4]
    # a walk may reuse vertices, a simple path may not
    walks = prefix_walks(labels, edges, "abab")
    assert walks == [2, 2, 2, 2]
    five = prefix_paths(labels, edges, "ababa")
    assert five == [2, 2, 2, 2, 0]
    assert prefix_paths(labels, edges, "") == []


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')	[2, 2, 1, 1, 0]
('a', [], 'aa')	[1, 0]

STUDY PATH: question 57 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 3 of 9

Template fit: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n | no date

MEDIUM | IN APP LC 22 | Exponent | 57 of 288

★ TOP 75: rank 19 in the Medium and Hard priority list

★ Generate Parentheses

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 61.6 | 30 DAYS 46.9 | PRIORITY 16/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: your app's interview approach (LeetCode #22)

LeetCode match: LC 22 Generate Parentheses

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age) | +36 interviews

Original: https://www.aced.io/questions/1594/generate-parentheses

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and

closing parentheses that can still be added. It also

EXAMPLES

Example 1	
Input	3
Output	`['((()))', '(()())', '(())()', '()(())', '()()()']`
Why	Five well-formed combinations of three pairs.

QUESTION

Example 2	
Input	1
Output	`['()']`
Why	One pair.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for seq in product("()", repeat=2 * n):
        if is_balanced(seq):
            result.append("".join(seq))
    return result
```

TIME	SPACE
O(2^(2n) * n)	O(2^(2n))

WHY IT IS SLOW

Builds exponentially many invalid strings. Backtracking only adds '(' while fewer than n are used and ')' while fewer than the opens, so it builds only valid prefixes (the Catalan number of results).

OPTIMAL SOLUTION · Backtracking with open and close counters

TIME	SPACE
O(4^n / sqrt(n))	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the k-th string, or just the number of strings.

The count is the Catalan number C(n) = (2n choose n) / (n + 1). For the k-th string count the completions that start with '(' at each step and skip them if k is larger (unranking), using a small DP over (opens left, balance).

FOLLOW-UP 2 Several bracket types, such as () [] {}.

Multiply the choices: at each position you may open any of the types (until its count is used) and may close only the type on top of a stack. The pruning rule (never close more than you have opened) becomes 'close only the most recent unclosed bracket'.

FOLLOW-UP 3 Avoid recursion.

Use an explicit stack of (string so far, opens, closes) states or build the strings by length with a queue. The pruning conditions stay the same.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Generate Parentheses
#
# Generate all combinations of well-formed parentheses. A
# parenthesis combination is well-formed if each opening
# parenthesis "(" is closed by a matching closing
# parenthesis ")" in the correct order. For example, "()",
# "()()", and "((()))" are all combinations of well-formed
# parentheses, while ")(", "((" and "(()" are not.
# Examples n = 1 output: ["()"] n = 2 output: ["(())",
# "()()"] n = 3 output: ["((()))", "(()())", "(())()",
# "()(())", "()()()"] n = 4 output should contain:
# ["(((())))", "((()()))", "((())())", "((()))()",
# "(()(()))", "(()()())", "(())(())", "(())()()",
# "()((()))", "()(()())", "()(())()", "()()(())",
# "()()()()"]The problem is to generate all
# combinations of well-formed parentheses given n pairs of
# parentheses. A combination is well-formed if every
# opening parenthesis has a corresponding closing
# parenthesis and they are properly nested.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the k-th string, or just the number of
# strings.
# 2. Several bracket types, such as () [] {}.
# 3. Avoid recursion.
#
# INTERVIEW NOTES
# Build the string left to right and only make moves that
# can still lead to a valid result: you may add '(' while
# fewer than n have been opened, and ')' only while there
# are more opens than closes. Because invalid prefixes are
# never extended, every leaf is a valid answer, and the
# count is the Catalan number.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from functools import lru_cache
from math import comb
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Backtracking with open and
# close counters
```

```
# Idea: Track how many '(' and ')' have been placed. Add
# '(' if open < n; add ')' if close < open. When the
# string reaches length 2n it is valid.
def generate(n):
    # build the string one character at a time. Adding "("
    # is allowed while opens < n; adding ")" only while
    # closes < opens: that rule keeps every string valid.
    out = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return out

# ---- main question: tests ----
def test_main_question():
    assert generate(1) == ["()"]
    assert generate(2) == ["(())", "()()"]
    assert sorted(generate(3)) == [
        "((()))", "(()())", "(())()", "()(())", "()()()"]
    assert generate(0) == [""]


# ============================================================
# FOLLOW-UP 1: Return only the k-th string, or just the
# ============================================================

# ---- follow-up 1: helper functions ----
@lru_cache(maxsize=None)
def completions(opens_left, balance):
    # how many ways to finish a string that still has
```

```
    # opens_left "(" to place and balance unclosed ones
    if opens_left == 0:
        return 1
    total = completions(opens_left - 1, balance + 1)
    if balance:
        total += completions(opens_left, balance - 1)
    return total

# ---- follow-up 1: function ----
# Answer: The count is the Catalan number C(n) = (2n
# choose n) / (n + 1). For the k-th string count the
# completions that start with '(' at each step and skip
# them if k is larger (unranking), using a small DP over
# (opens left, balance).
def catalan(n):
    # follow-up: just the NUMBER of strings
    return comb(2 * n, n) // (n + 1)


# Answer: The count is the Catalan number C(n) = (2n
# choose n) / (n + 1). For the k-th string count the
# completions that start with '(' at each step and skip
# them if k is larger (unranking), using a small DP over
# (opens left, balance).
def kth(n, k):
    # follow-up: the k-th string (1-based, sorted order,
    # "("
    # before ")") without listing them: at each step count
    # the
    # strings that continue with "(" and skip them if
    # needed
    if not 1 <= k <= catalan(n):
        raise ValueError("k is out of range")
    text, opens_left, balance = "", n, 0
```

```
    while len(text) < 2 * n:
        ways = 0
        if opens_left:
            ways = completions(opens_left - 1,
                               balance + 1)
        if opens_left and k <= ways:
            text += "("
            opens_left -= 1
            balance += 1
        else:
            if opens_left:
```

```
                k -= ways
            text += ")"
            balance -= 1
    return text

# ============================================================
# FOLLOW-UP 2: Several bracket types, such as () [] {}.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Multiply the choices: at each position you may
# open any of the types (until its count is used) and may
# close only the type on top of a stack.
def generate_multi(n, kinds="()[]{}"):
    # follow-up: several bracket types. Any type can open;
    # only the type on top of the stack can close.
    pairs = [kinds[i:i + 2]
             for i in range(0, len(kinds), 2)]
    out = []

    def build(text, stack, opened):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opened < n:
            for open_ch, close_ch in pairs:
                build(text + open_ch, stack + [close_ch],
                      opened + 1)
        if stack:
            build(text + stack[-1], stack[:-1], opened)

    build("", [], 0)
    return out

# ============================================================
# FOLLOW-UP 3: Avoid recursion.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Use an explicit stack of (string so far, opens,
# closes) states or build the strings by length with a
# queue. The pruning conditions stay the same.
def generate_iter(n):
    # follow-up: no recursion. The same states on a stack;
    # ")" is pushed first so "(" is handled first and the
```

```
    # strings come out in sorted order.
    out, stack = [], [("", 0, 0)]
    while stack:
        text, opens, closes = stack.pop()
        if len(text) == 2 * n:
            out.append(text)
            continue
        if closes < opens:
            stack.append((text + ")", opens, closes + 1))
        if opens < n:
            stack.append((text + "(", opens + 1, closes))
    return out

# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    for n in range(0, 8):
        listed = generate(n)
        assert len(listed) == catalan(n)
        assert generate_iter(n) == sorted(listed)
    for n in range(1, 7):
        every = sorted(generate(n))
        for k, text in enumerate(every, 1):
            assert kth(n, k) == text
    assert catalan(3) == 5 and catalan(10) == 16796
    # 3 types: Catalan(n) shapes times 3 choices per pair
    for n in range(1, 5):
        got = generate_multi(n, "()[]{}")
        assert len(got) == catalan(n) * 3 ** n
        assert len(set(got)) == len(got)
    two = sorted(generate_multi(1, "()[]"))
    assert two == ["()", "[]"]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples

```
n = 1
output: ["()"]

n = 2
output: ["(())", "()()"]

n = 3
output: ["((()))", "(()())", "(())()", "()(())", "()()
()"]

n = 4
output should contain: ["(((())))", "((()()))", "((()))
()", "((())())", "(()(()))", "(()()())", "(())())",
"(())(())", "(())()()", "()((()))", "()(()())", "()(()
)()", "()()(())", "()()()()"]
```

OFFICIAL PAGE TEXT

The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also maintains the current combination of the string being formed. Base Case: If no more parentheses are remaining to add (both remaining_open and remaining_close are zero), the current combination is a valid sequence and is added to the result list. Recursive Case: Add an opening parenthesis (if there are any remaining and recursively call the function with one less opening parenthesis. Add a closing parenthesis) if the number of remaining closing parentheses is greater than the remaining opening

parentheses and recursively call the function with one less closing parenthesis. Pruning Invalid Paths: If at any point the number of closing parentheses left is less than the number of opening parentheses left, it means the current sequence cannot be completed to a valid sequence, so we stop exploring that path. Implementation Steps Initialize: Create a result list to store all valid sequences. Call the recursive helper function with initial values (n opening and n closing parentheses). Recursive Helper Function: Track the number of remaining opening and closing parentheses. Track the current combination being formed. Append valid combinations to the result list when the base case is reached. Backtrack: After adding an opening or closing parenthesis, remove it (backtrack) before trying the next possibility. This ensures that we explore all possible valid combinations. Example Let's consider n = 3: Start with an empty combination "", 3 opening, and 3 closing parentheses. Add an opening parenthesis: "(", 2 opening, and 3 closing. Continue adding opening parentheses until you can no longer add without violating the rules. Then add closing parentheses and ensure that at no point the closing parentheses exceed the opening ones. Once you form a valid combination like "((()))", add it to the result list. Backtrack and explore other combinations. Solution Time Complexity: The time

OFFICIAL PAGE TEXT

complexity of this solution is O(22n)O(2^{2n})O(22n). This is because each position in the string can either be an opening or closing parenthesis, and we need to generate all possible sequences of length 2n2n2n. Space Complexity: The space complexity of the solution is O(2n)O(2n)O(2n) which simplifies to O(n)O(n)O(n). This is because: The recursion depth is 2n2n2n in the worst case (each recursive call adds either an opening or a closing parenthesis). The combination string being formed is of length 2n2n2n. In this mock interview video, Aman (Google SWE) answers the interview question, "Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses."No answers contributed by the community yet.

STUDY PATH: question 58 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 4 of 9

Template fit: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0 | asked 421 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 58 of 288

American Football Score Combinations

ASKED 2025-08-25 | 421 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 115/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #518 Coin Change II

LeetCode match: LC 518 Coin Change II (related: list the combinations)

Frequency: Single report | Last asked: 2025-08-25 | Roles: SWE Intern

QUESTION

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: 2 (safety), 3 (field goal), 6 (touchdown), 7 (touchdown + extra point), or 8 (touchdown + 2-point conversion). Given a target total , return every distinct combination of scoring plays whose values sum to total . Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

Function signature

```
def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted
ascending."""
    ...
```

Follow-up (as reported): If you only need the count of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

EXAMPLES

Example 1	
Input	`total = 6`
Output	`[[2, 2, 2], [3, 3], [6]]`
Why	`2+2+2 = 3+3 = 6 = 6.`

Example 2	
Input	`total = 7`
Output	`[[2, 2, 3], [7]]`
Why	`2+2+3 = 7 and the single play 7.`

QUESTION

Example 3	
Input	`total = 0`
Output	`[[]]`
Why	`The empty combination is the only way to score 0.`

CONSTRAINTS

- 0 <= total <= 60
- Allowed play values are exactly [2, 3, 6, 7, 8] .
- If total == 0 , the only combination is the empty one, so return [[]] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
PLAYS = (2, 3, 6, 7, 8)
def brute_combinations(total):
    found = set()
    def go(left, seq):
        if left == 0:
            found.add(tuple(sorted(seq)))
            return
        for v in PLAYS:
            if v <= left:
                go(left - v, seq + [v])
    go(total, [])
    return sorted(list(c) for c in found)
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

It builds every ordering of the plays and only then collapses permutations.

OPTIMAL SOLUTION · Backtracking in increasing order

TIME	SPACE
O(number of combinations * length)	O(total) recursion

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You only need how many combinations there are.

Use the unbounded coin DP: ways[0] = 1 and for each play value v, ways[t] += ways[t - v] for t from v to total. Looping over the values on the outside counts multisets. O(total * values) time and O(total) space.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: American Football Score Combinations
#
# Bryan's DP question was: enumerate all the ways a
# football team can accumulate a given total number of
# points. In American football a single scoring play is
# worth one of: 2 (safety), 3 (field goal), 6 (touchdown),
# 7 (touchdown + extra point), or 8 (touchdown + 2-point
# conversion). Given a target total , return every
# distinct combination of scoring plays whose values sum
# to total . Two combinations are the same if they use the
# same multiset of play values (order does not matter).
# Each combination should be returned as a list of play
# values sorted in non-decreasing order. The overall list
# of combinations may be in any order. A play value may be
# used any number of times. Function signature: def
# football_combinations(total: int) -> list[list[int]]:
# """Return all distinct multisets of {2,3,6,7,8} that sum
# to total, each sorted ascending.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. You only need how many combinations there are.
#
# INTERVIEW NOTES
# This is coin change where order does not matter. To
# avoid counting 2+3 and 3+2 twice, fix an order: go
# through the play values one at a time and decide how
# many of each to use (or recurse with a minimum index).
# Listing the combinations is backtracking; counting them
# is the unbounded coin DP.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Backtracking in increasing
# order
# Idea: Choosing plays in non-decreasing order lists every
# multiset exactly once.
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    out = []

    def dfs(start, left, path):
        if left == 0:
            out.append(path[:])
            return
        for i in range(start, len(plays)):
            if plays[i] <= left:
                path.append(plays[i])
                # the same play may be used again, so i
                # stays
                dfs(i, left - plays[i], path)
                path.pop()

    dfs(0, total, [])
    return out

# ---- main question: tests ----
def test_main_question():
    random.seed(6)
    assert football_combinations(6) == [
        [2, 2, 2],
        [3, 3],
        [6],
    ]
    assert football_combinations(7) == [[2, 2, 3], [7]]
    assert football_combinations(0) == [[]]
    assert football_combinations(1) == []


# ============================================================
# FOLLOW-UP 1: You only need how many combinations there
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the unbounded coin DP: ways[0] = 1 and for
# each play value v, ways[t] += ways[t - v] for t from v
# to total. Looping over the values on the outside counts
# multisets. O(total * values) time and O(total) space.
```

```
def count_combinations(total):
    ways = [1] + [0] * total
    for v in (2, 3, 6, 7, 8):
        for t in range(v, total + 1):
            ways[t] += ways[t - v]
    return ways[total]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for t in range(0, 19):
        assert count_combinations(t) == len(
            football_combinations(t)
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 59 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 5 of 9

Template fit: the template as written | no date

MEDIUM | IN APP LC 46 | Exponent | 59 of 288

Permutations

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 49.8 | 30 DAYS 0 | PRIORITY 80/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: your app's interview approach (LeetCode #46)

LeetCode match: LC 46 Permutations

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +25 interviews

Original:
https://www.aced.io/questions/2212/leetcode-medium-permutations-dp

QUESTION

Given an array of distinct integers, return all possible permutations of the array (every rearrangement of its elements), in any order.

Function signature

def permute(nums: list) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3]`
Output	`[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]`
Why	`Six orders of three numbers.`

Example 2	
Input	`[0, 1]`
Output	`[[0, 1], [1, 0]]`
Why	`Two numbers have two orders.`

Example 3	
Input	`[1]`
Output	`[[1]]`
Why	`One number has a single permutation.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    result = []
    for seq in product(nums, repeat=len(nums)):
        if len(set(seq)) == len(nums):
            result.append(list(seq))
    return result
```

BRUTE FORCE

TIME	SPACE
O(n^n * n)	O(n!)

WHY IT IS SLOW

n^n sequences when only n! are valid, so almost all the work is wasted. Backtracking with a used marker only builds valid prefixes, O(n * n!).

OPTIMAL SOLUTION · Backtracking with a used array

TIME	SPACE
O(n * n!)	O(n) beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input has duplicates (Permutations II).

Sort first, and skip a value that equals the previous one unless the previous one is already used in the current prefix. That forces equal values to appear in a fixed order and removes duplicates without a set.

FOLLOW-UP 2 Return the next lexicographic permutation in place.

Find the last position i with a[i] < a[i + 1], swap a[i] with the smallest larger value to its right, then reverse the suffix. O(n), O(1) space. Repeat to enumerate in order.

FOLLOW-UP 3 Return only the k-th permutation of 1..n.

Use the factorial number system: the first digit is index (k - 1) // (n - 1)! of the remaining digits, then continue with the remainder. O(n^2), no enumeration.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Permutations
#
# Given an array of distinct integers, return all possible
# permutations of the array (every rearrangement of its
# elements), in any order.
#
# Signature:
#   def permute(nums: list) -> list
#
# Examples:
# [1, 2, 3]  ->  [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3,
#     1], [3, 1, 2], [3, 2, 1]]
```

```
# [0, 1]  ->  [[0, 1], [1, 0]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input has duplicates (Permutations II).
# 2. Return the next lexicographic permutation in place.
# 3. Return only the k-th permutation of 1..n.
#
# INTERVIEW NOTES
# The output has n! items, so no algorithm can beat O(n *
# n!); the question is only whether you generate them
# without waste. Build a permutation one position at a
# time, marking which elements are used, and undo the
# choice when you come back (backtracking). Every follow-
# up (duplicates, k-th, next) is a variation on 'what are
# the legal choices at this position'.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ========================================================

from math import factorial


# ========================================================
# MAIN QUESTION
# ========================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Backtracking with a used array
# Idea: Fill positions left to right. At each position try
```

```
# every unused value, mark it, recurse, and unmark. Only
# valid prefixes are ever extended, so every step leads to
# a permutation.
def permute(nums):
    # backtracking: choose an unused element, recurse,
    # undo
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True
                cur.append(x)
                go()
                cur.pop()
                used[i] = False

    go()
    return out

# ---- main question: tests ----
def test_main_question():
    assert permute([1, 2, 3]) == [[1, 2, 3], [1, 3, 2], [2
, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    assert permute([0, 1]) == [[0, 1], [1, 0]]
    assert permute([1]) == [[1]]


# ========================================================
# FOLLOW-UP 1: The input has duplicates (Permutations II).
# ========================================================

# ---- follow-up 1: function ----
```

```
# Answer: Sort first, and skip a value that equals the
# previous one unless the previous one is already used in
# the current prefix. That forces equal values to appear
# in a fixed order and removes duplicates without a set.
def permute_unique(nums):
    # duplicates: sort, and skip a value equal to the
    # previous
    # one unless that one is already in the current prefix
    nums = sorted(nums)
    out, cur, used = [], [], [False] * len(nums)
```

```
    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if used[i]:
                continue
            same = i and nums[i] == nums[i - 1]
            if same and not used[i - 1]:
                continue
            used[i] = True
            cur.append(x)
            go()
            cur.pop()
            used[i] = False

    go()
    return out

# ========================================================
# FOLLOW-UP 2: Return the next lexicographic permutation
# ========================================================

# ---- follow-up 2: function ----
# Answer: Find the last position i with a[i] < a[i + 1],
# swap a[i] with the smallest larger value to its right,
# then reverse the suffix. O(n), O(1) space. Repeat to
# enumerate in order.
def next_permutation(a):
    # in place: O(n) time, O(1) space; False when it was
    # the last
    i = len(a) - 2
    # the last position that can still increase
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1
    if i < 0:
        a.reverse()
        return False
    j = len(a) - 1
    while a[j] <= a[i]:
        j -= 1
    a[i], a[j] = a[j], a[i]
    # the suffix is decreasing: make it the smallest
    a[i + 1:] = reversed(a[i + 1:])
```

```
    return True

# ========================================================
# FOLLOW-UP 3: Return only the k-th permutation of 1..n.
# ========================================================

# ---- follow-up 3: function ----
# Answer: Use the factorial number system: the first digit
# is index (k - 1) // (n - 1)! of the remaining digits,
# then continue with the remainder. O(n^2), no
# enumeration.
def kth_permutation(n, k):
    # the k-th (1-based) permutation of 1..n without
    # listing
    digits, out, k = list(range(1, n + 1)), [], k - 1
    for i in range(n, 0, -1):
```

```
        block = factorial(i - 1)
        out.append(digits.pop(k // block))
        k %= block
    return out


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example

```
Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [
3, 1, 2], [3, 2, 1]]
```

Our solution generates all permutations of a given array by utilizing a recursive backtracking approach. The idea is to explore every possible way of arranging the elements by swapping them at each recursive call. Here's a breakdown of how the algorithm works: We define a helper function (backtrack) that recursively builds the permutations. This function takes two parameters: start (the index from where we start permuting) and end (the length of the array). The base case occurs when start == end, meaning that a complete permutation has been formed. We add a copy of the current permutation (stored in nums) to the result. In each recursive call, we iterate over the elements starting from the start index. For each element at position i, we: Swap the elements at start and i to bring a new element into the current position. Recursively call backtrack with start + 1, moving to the next index to fill. After the recursive call, we swap back the elements to restore the original order and continue exploring other possibilities. This process continues until all permutations are generated. The swapping ensures that we explore every possible combination of the elements. Time Complexity: The time complexity of this solution is O(n * n!), where n is the number of elements in the array. The reason for this is that generating each permutation takes O(n) time, and there are n! permutations. Space Complexity: The space

complexity is O(n!) due to the storage of all permutations in the result list. Additionally, the recursion stack uses O(n) space for the recursive calls, but this is dominated by the result storage.No answers contributed by the community yet.

STUDY PATH: question 60 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 6 of 9

Template fit: the same try-triple-then-run recursion over a count array, after choosing the pair | asked 225 days ago

MEDIUM | PracHub | Technical Screen | 60 of 288

Determine if a 14-tile hand is winning

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 170/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #659 Split Array into Consecutive Subsequences

LeetCode match: No direct match; related LC 659 Split Array into Consecutive Subsequences

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/determine-if-a-14-tile-hand-is-winning

QUESTION

A hand is 14 tiles with values 1 to 9 of one suit (you may reorder them). It wins if it splits into one pair (two identical tiles) and four melds of three tiles, where a meld is either a triplet (three identical tiles) or a sequence (three consecutive values). Return whether the hand is winning.

Function signature

def is_winning_hand(tiles: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]`
Output	`true`
Why	`Pair 9-9, triplets 1-1-1 and 5-5-5, sequences 2-3-4 and 6-7-8.`

Example 2	
Input	`[1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]`
Output	`false`
Why	`These tiles cannot be split into a pair and four melds.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def is_win(counts):
    for v in range(1, 10):
        if counts[v] >= 2:
            counts[v] -= 2  # the pair
            ok = remove_melds(counts)
            counts[v] += 2
            if ok:
                return True
    return False
def remove_melds(counts):
    v = first_value_with_tiles(counts)
    if v is None:
        return True
    for meld in (triple(v), run(v)):
        if can_take(counts, meld):
            take(counts, meld)
            ok = remove_melds(counts)
            give_back(counts, meld)
            if ok:
                return True
    return False
```

TIME
exponential but small for...
exponential but small for 14 tiles

SPACE
O(depth)

WHY IT IS SLOW

It tries the same groupings in many different orders (still tiny for 14 tiles). Always grouping the smallest remaining tile tries each grouping once.

OPTIMAL SOLUTION · Counts, try each pair, then always group the smallest remaining tile

TIME
O(9 * small branching)
= constant for 14 tiles

SPACE
O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

FOLLOW-UP 2 More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

FOLLOW-UP 3 Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if a 14-tile hand is winning
#
# A hand is 14 tiles with values 1 to 9 of one suit (you
# may reorder them). It wins if it splits into one pair
# (two identical tiles) and four melds of three tiles,
# where a meld is either a triplet (three identical tiles)
# or a sequence (three consecutive values). Return whether
# the hand is winning.
#
# Signature:
#   def is_winning_hand(tiles: list) -> bool
#
# Examples:
# [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]  ->  true
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which tile would complete a 13-tile hand?
# 2. More suits or special hands (seven pairs).
# 3. Why is the 'smallest remaining value' rule enough?
#
# INTERVIEW NOTES
# Work with counts per value (at most nine numbers), not
# with the tile list. Try every value as the pair, then
# handle the lowest remaining value: it must belong to a
# triplet or to a sequence starting at it (nothing smaller
# is left to pair with it). Always processing the smallest
# value makes the search tiny and avoids trying the same
# grouping in different orders.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def melds_ok(cnt, v=1):
    # can the tiles in cnt (value -> count) form only
    # melds?
    while v <= 9 and cnt[v] == 0:
        v += 1
    if v > 9:
        return True
    # the smallest tile is either in a triplet ...
    if cnt[v] >= 3:
        cnt[v] -= 3
        ok = melds_ok(cnt, v)
        cnt[v] += 3
        if ok:
            return True
    # ... or the start of a run v, v + 1, v + 2
    if v <= 7 and cnt[v + 1] and cnt[v + 2]:
        for t in (v, v + 1, v + 2):
            cnt[t] -= 1
        ok = melds_ok(cnt, v)
        for t in (v, v + 1, v + 2):
            cnt[t] += 1
        return ok
    return False

# ---- main question: main function ----
# BASE QUESTION (optimal): Counts, try each pair, then
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair.
def is_winning_hand(tiles):
    # one pair plus four melds, single suit, values 1..9
    cnt = Counter({v: 0 for v in range(1, 12)})
    for t in tiles:
        cnt[t] += 1
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False

# ---- main question: tests ----
def test_main_question():
```

```
    win = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
    assert is_winning_hand(win)
    # 3 and 6 are missing, so the runs cannot be formed
    lose = [1, 2, 4, 5, 7, 8, 1, 2, 4, 5, 7, 8, 9, 9]
    assert not is_winning_hand(lose)
    # thirteen tiles waiting for a 9 (or a 6)
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]


# ============================================================
# FOLLOW-UP 1: Which tile would complete a 13-tile hand?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try each of the nine values as the 14th tile and
# run the same check: 9 checks of a tiny search. That is
# how a player finds the tiles they are waiting for.
def waiting_tiles(tiles13):
    # follow-up: which 14th tile completes the hand?
    return [t for t in range(1, 10)
            if is_winning_hand(tiles13 + [t])]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # thirteen tiles waiting for a 9 (or a 6)
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]
    assert 9 in waiting_tiles(wait)


# ============================================================
# FOLLOW-UP 2: More suits or special hands (seven pairs).
```

```
# ====================================================

# ---- follow-up 2: helper functions ----
def count_of(values):
    cnt = Counter({v: 0 for v in range(1, 12)})
    for v in values:
        cnt[v] += 1
    return cnt


def is_winning_hand_with_len(values):
    cnt = count_of(values)
    for pair in range(1, 10):
        if cnt[pair] >= 2:
```

```
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False

# ---- follow-up 2: function ----
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
def is_seven_pairs(tiles):
    # a special hand: seven different pairs
    counts = Counter(tiles)
    pairs = all(c == 2 for c in counts.values())
    return len(tiles) == 14 and pairs


# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
def is_winning_multi(hand):
    # several suits: hand is a list of (suit, value);
    # melds
    # never cross suits, and exactly one suit holds the
    # pair
    suits = {}
    for suit, v in hand:
        suits.setdefault(suit, []).append(v)
    pair_suits = [s for s in suits
                  if len(suits[s]) % 3 == 2]
    if len(pair_suits) != 1:
        return False
    for s, vs in suits.items():
        if s == pair_suits[0]:
            if not is_winning_hand_with_len(vs):
                return False
        elif len(vs) % 3 or not melds_ok(count_of(vs)):
            return False
    return True

# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    assert is_seven_pairs([1, 1, 2, 2, 3, 3, 4, 4, 5, 5,
                           6, 6, 7, 7])
    hand = [("m", v) for v in (1, 1, 1, 2, 3)] + [
        ("p", v) for v in (4, 5, 6, 7, 7)] + [
        ("s", v) for v in (2, 2, 2)]
    assert is_winning_multi(hand)


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(tiles: list[int]) -> bool:
```

Sample call

```
solution([1,1,1,2,3,4,2,3,4,5,6,7,9,9])
```

Test cases

Input	Expected
`([1,1,1,2,3,4,2,3,4,5,6,7,9,9],)`	`True`
`([2,2,2,3,3,3,4,4,4,5,5,5,9,9],)`	`True`

ALSO REPORTED · 1 more

Mahjong Winning Hand Detection (14 Tiles) | 1Point3Acres | onsite-coding | asked 2026-07-20 | Low

https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:
Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.
For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

Preparation: Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY PATH: question 61 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 7 of 9

Template fit: the choice is the next adjacent unused key; complete at k keys; count instead of storing | asked 347 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 61 of 288

Count Passcode Patterns on a Keypad

ASKED 2025-11-07 | 347 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 229/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #980 Unique Paths III

LeetCode match: LC 980 / LC 351 Android Unlock Patterns (related: paths on a grid)

Frequency: Single report | Last asked: 2025-11-07 | Roles: SWE Intern

QUESTION

A keypad is a rows x cols grid of keys (the classic phone dial is 3x3):
1 2 3 4 5 6 7 8 9 A passcode pattern is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are different patterns. Given the grid dimensions and an integer k , return the number of distinct passcode patterns that use exactly k keys.

Function signature

```
def count_patterns(rows: int, cols: int, k: int) -> int:
    """Return the number of distinct passcode patterns using exactly k keys,
    moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""
    ...
```

Follow-up (as reported): Count patterns of length at least k instead of exactly k . Hint: sum the DFS counts over all lengths from k to rows*cols .

EXAMPLES

Example 1

Input	`rows = 3, cols = 3, k = 1`
Output	`9`
Why	`Each of the 9 keys alone is one pattern.`

Example 2

Input	`rows = 3, cols = 3, k = 2`
Output	`24`
Why	`There are 12 adjacent key pairs; each can be traversed in 2 directions, giving 24 patterns.`

QUESTION

Example 3

Input	`rows = 3, cols = 3, k = 3`
Output	`44`
Why	`Summing over each possible middle key b, deg(b)*(deg(b)-1) directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.`

CONSTRAINTS

- 1 <= rows, cols <= 4 .
- 1 <= k <= rows * cols .
- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import permutations
def brute_patterns(rows, cols, k):
    cells = [
        (r, c)
        for r in range(rows)
        for c in range(cols)
    ]
    count = 0
    for path in permutations(cells,_k):
        if all(
            abs(a[0] - b[0]) + abs(a[1] - b[1])
            == 1
            for a, b in zip(path, path[1:])
        ):
            count += 1
    return count
```

TIME	SPACE
O(P(rc, k) * k)	O(k)

WHY IT IS SLOW

It builds selections that are not even connected before testing them.

OPTIMAL SOLUTION · DFS with backtracking from every key

TIME	SPACE
O(number of simple paths)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count patterns that use at least k keys.

Use the same DFS but add one at every depth that is at least k and keep extending instead of stopping.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Passcode Patterns on a Keypad
#
# A keypad is a rows x cols grid of keys (the classic
# phone dial is 3x3): 1 2 3 4 5 6 7 8 9 A passcode pattern
# is a path across keys where each move goes only to an
# orthogonally adjacent key (horizontal or vertical — no
# diagonals), and no key is visited more than once. The
# order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are
# different patterns. Given the grid dimensions and an
# integer k , return the number of distinct passcode
# patterns that use exactly k keys. Function signature:
# def count_patterns(rows: int, cols: int, k: int) -> int:
# """Return the number of distinct passcode patterns using
# exactly k keys, moving only to orthogonally adjacent,
# unvisited keys on a rows x cols grid.""" ... Follow-up
# (as reported): Count patterns of length at least k
# instead of exactly k . Hint: sum the DFS counts over all
# lengths from k to rows*cols .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count patterns that use at least k keys.
#
# INTERVIEW NOTES
# Count simple paths of exactly k keys on a grid: start a
# DFS from every key, mark the key used, step to each
# unused orthogonal neighbour, and count one when the path
# has k keys. Unmark on the way back (backtracking). The
# grid is at most 4x4, so the search stays small.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): DFS with backtracking from
```

```
# every key
# Idea: Extend the path one adjacent unused key at a time
# and undo the choice afterwards.
def count_patterns(rows, cols, k):
    def dfs(r, c, used, seen):
        if used == k:
            return 1
        total = 0
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            nr, nc = r + dr, c + dc
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and (nr, nc) not in seen
            ):
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))
        return total

    return sum(
        dfs(r, c, 1, {(r, c)})
        for r in range(rows)
        for c in range(cols)
    )

# ---- main question: tests ----
def test_main_question():
    random.seed(80)
    assert count_patterns(3, 3, 1) == 9
    assert count_patterns(3, 3, 2) == 24
    assert count_patterns(3, 3, 3) == 44


# ============================================================
# FOLLOW-UP 1: Count patterns that use at least k keys.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the same DFS but add one at every depth that
```

```
# is at least k and keep extending instead of stopping.
def count_patterns_at_least(rows, cols, k):
    def dfs(r, c, used, seen):
        total = 1 if used >= k else 0
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            nr, nc = r + dr, c + dc
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and (nr, nc) not in seen
            ):
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))
        return total

    return sum(
        dfs(r, c, 1, {(r, c)})
        for r in range(rows)
        for c in range(cols)
    )

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for rows, cols in ((1, 4), (2, 2), (2, 3), (3, 3)):
        for k in range(1, 4):
            want = sum(
                count_patterns(rows, cols, j)
                for j in range(k, rows * cols + 1)
```

```
            )
            assert (
                count_patterns_at_least(rows, cols, k)
                == want
            )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 62 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 8 of 9

Template fit: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure | asked 297 days ago

MEDIUM | PracHub | Technical Screen | 62 of 288

Match people to questions using tags and priorities

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 196/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: the interview site's own solution

LeetCode match: LC 1820 Maximum Number of Accepted Invitations (bipartite matching) (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-06-04)

Original: https://prachub.com/coding-questions/match-people-to-questions-using-tags-and-priorities

QUESTION

A set of people and a set of interview questions: adj[p] lists the questions person p is allowed to answer (their tags match). Each person is given at most one question and each question goes to at most one person. Return the largest number of people that can be matched.

Function signature

def max_matching(adj: list) -> int

EXAMPLES

Example 1	
Input	`[[0, 1], [0], [1, 2]]`
Output	`3`
Why	`Person 0 takes question 0, person 1 has only question 0 so person 0 moves to question 1, person 2 takes question 2: all three are matched.`

Example 2	
Input	`[[0], [0], [0]]`
Output	`1`
Why	`Three people can only answer the same single question: just one match.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
best = 0
def go(person, used, matches):
    global best
    if person == len(people):
        best = max(best, matches)
        return
    go(person + 1, used, matches)  # skip
    for q in range(len(questions)):
        if q in used:
            continue
        if not shares_tag(person, q):
            continue
        go(person + 1, used | {q},
           matches + 1)
```

TIME	SPACE
O((Q + 1)^P)	O(P)

WHY IT IS SLOW

Each person branches over all questions: exponential. Augmenting paths (move an earlier person to another question when needed) solve it in O(people * edges).

OPTIMAL SOLUTION · Augmenting paths (Kuhn's algorithm)

TIME	SPACE
O(P * E)	O(P + Q)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

FOLLOW-UP 2 Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

FOLLOW-UP 3 A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Match people to questions using tags and
# priorities
#
# A set of people and a set of interview questions: adj[p]
# lists the questions person p is allowed to answer (their
# tags match). Each person is given at most one question
# and each question goes to at most one person. Return the
# largest number of people that can be matched.
#
# Signature:
#   def max_matching(adj: list) -> int
#
# Examples:
# [[0, 1], [0], [1, 2]]  ->  3
# [[0], [0], [0]]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Priorities: higher-priority people must be served
# first.
# 2. Edges have weights (how good a match is).
# 3. A question may take several people (capacity c).
#
# INTERVIEW NOTES
# 'Assign each person at most one question, each question
# at most one person, allowed pairs given by tags' is
# maximum bipartite matching. The tag index only builds
# the edges faster. For maximum count use augmenting
# paths: try to give a person a question, and if taken ask
# its current owner to move. If priorities decide who is
# served, process people in priority order with the same
# augmenting step.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BUILD the person-question graph
def build_graph(people, questions):
    # a person can take a question if they share a tag
    # people: [(name, priority, tags)], questions: [(id,
    # tags, cap)]
    adj = {}
    for name, _, tags in people:
        adj[name] = [q for q, qtags, _ in questions
                     if set(tags) & set(qtags)]
    return adj


# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt.
def max_matching(adj):
    # the plain version: adj[p] = list of questions for
    # person p
    owner = {}

    def try_assign(p, seen):
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            free = q not in owner
            if free or try_assign(owner[q], seen):
                owner[q] = p
                return True
        return False

    people = range(len(adj))
    return sum(try_assign(p, set()) for p in people)

# ---- main question: tests ----
def test_main_question():
    assert max_matching([[0, 1], [0], [1, 2]]) == 3
    assert max_matching([[0], [0], [0]]) == 1
    people = [("ann", 1, {"dp"}),
              ("bob", 5, {"dp", "graph"}),
              ("cy", 3, {"graph"})]
```

```
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]


# ============================================================
# FOLLOW-UP 1: Priorities: higher-priority people must be
# ============================================================

# ---- follow-up 1: function ----
# Answer: Process people in decreasing priority and run
# the same augmenting step. Matchings of this kind form a
# matroid, so this greedy choice gives a matched set with
# the maximum total priority (checked against exhaustive
# search).
def assign(people, questions):
    # Kuhn's augmenting paths. People are tried by
    # priority
    # (highest first): such matchings form a matroid, so
    # this
    # greedy order keeps the best possible set of people.
    adj = build_graph(people, questions)
    cap = {q: c for q, _, c in questions}
    # question -> the people currently holding its places
    held = {q: [] for q in cap}

    def try_assign(person, seen):
        for q in adj[person]:
            if q in seen:
                continue
            seen.add(q)
            # a free place, or someone who can move
            # elsewhere
            if len(held[q]) < cap[q]:
                held[q].append(person)
                return True
            for other in list(held[q]):
                if try_assign(other, seen):
                    held[q].remove(other)
                    held[q].append(person)
                    return True
        return False
```

```
    order = sorted(people, key=lambda p: -p[1])
    for name, _, _ in order:
        try_assign(name, set())
    return {p: q for q, ps in held.items() for p in ps}
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    people = [("ann", 1, {"dp"}),
              ("bob", 5, {"dp", "graph"}),
              ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    got = assign(people, questions)
    # bob (priority 5) is served first, and the matching
    # is
    # still the largest possible: ann -> q1, bob or cy ->
    # q2
    assert len(got) == 2
    assert got["bob"] in ("q1", "q2")
    # a question with two places takes two people
    q = [("q1", {"dp"}, 2)]
    both = assign(people[:2], q)
    assert set(both) == {"ann", "bob"}
    # nobody shares a tag: nobody is matched
    nobody = assign([("x", 1, {"a"})], [("q", {"b"}, 1)])
    assert nobody == {}


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[list[str]], question_tags: list[
list[str]], priorities: list[int]) -> list[int]:
```

Sample call

```
solution([['python', 'sql'], ['java'], ['ml']], [['sql'],
['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])
```

Test cases

Input	Expected
`([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])`	`(3, 16)`
`([['a'], ['a']], [['a'], ['a'], ['b']], [1, 10, 100])`	`(2, 11)`

STUDY PATH: question 63 of 288 | topic 7 of 43: Backtracking: choose, recurse, undo | 9 of 9

Template fit: place row by row; reject a row early when the houses two rows above are still not next to a flower | asked 142 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 63 of 288

Matrix Flower Placement with House Adjacency

ASKED 2026-05-31 | 142 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 250/288

ABOUT THIS QUESTION

Topic: Backtracking: choose, recurse, undo

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-05-31 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/matrix-flower-placement-with-house-adjacency

QUESTION

A grid has houses (H) and empty cells (0). Place flowers (F) on empty cells so that every row and every column holds at most one flower (exactly one of each when exactly_one is True) and every house has a flower directly beside it (up, down, left or right). Return one valid grid as a list of strings, or None if there is none.

Function signature

def place_flowers(grid: list, exactly_one=False) -> list | None

EXAMPLES

Example 1	
Input	`['H000', 'H000', '000H', '0000']`
Output	`['HF00', 'H0F0', 'F00H', '000F']`
Why	Flowers at (0,1), (2,0) and (1,3): one per row and column, and each house touches a flower.

Example 2	
Input	`['H000', 'H000', '000H', '0000']`, exactly_one = true
Output	`['HF00', 'H0F0', 'F00H', '000F']`
Why	Exactly one flower in every row and column: a fourth flower at (3,2) completes it.

Example 3	
Input	`['H', 'H']`
Output	`null`
Why	The two houses can only be served by the same column, so no valid placement exists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    empty = cells_with(grid, 0)
    count = 0
    for k in range(len(empty) + 1):
        for flowers in combinations(empty, k):
            if not one_per_row_and_col(flowers):
                continue
            if houses_covered(grid, flowers):
                count += 1
    return count
```

TIME	SPACE
O(2^cells)	O(cells)

WHY IT IS SLOW

2^cells subsets: exponential. Backtracking row by row with a used-columns set and early house checks prunes almost all of them (still exponential in the worst case).

OPTIMAL SOLUTION · Row-by-row backtracking with used-column set and delayed house checks

TIME	SPACE
exponential worst case, h... exponential worst case, heavily pruned	O(rows + cols)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

FOLLOW-UP 2 How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. **Cost:** the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

FOLLOW-UP 3 Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

FOLLOW-UP 4 Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Matrix Flower Placement with House
# Adjacency
#
# A grid has houses (H) and empty cells (0). Place flowers
# (F) on empty cells so that every row and every column
# holds at most one flower (exactly one of each when
# exactly_one is True) and every house has a flower
# directly beside it (up, down, left or right). Return one
# valid grid as a list of strings, or None if there is
# none.
#
# Signature:
#   def place_flowers(grid: list, exactly_one=False) -> li
st | None
#
# Examples:
# ['H000', 'H000', '000H', '0000']  ->  ['HF00', 'H0F0',
#     'F00H', '000F']
# ['H000', 'H000', '000H', '0000'], exactly_one = true  ->
#     ['HF00', 'H0F0', 'F00H', '000F']
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes between 'at most one' and 'exactly one'
# flower per row and column?
# 2. How would you speed up the search?
# 3. Exactly one flower per row and column.
# 4. Count the solutions instead of listing them.
#
# INTERVIEW NOTES
# At most one flower per row and column is the N-Queens
# pattern: decide row by row which column (if any) gets
# the flower and remember the used columns. The extra rule
# about houses is local, so check a house as soon as the
# last row that can affect it is decided, and prune the
# branch if it fails.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# ---- main question: main function ----
# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0').
def place_flowers(grid, exactly_one=False,
                  find_all=False):
    # 'H' house, '0' empty. At most one F per row and
    # column; every house needs an F on a side.
    R, C = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C)
              if cells[r][c] == "H"]
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if 0 <= a < R and 0 <= b < C \
                    and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        return all(house_ok(hr, hc)
                   for hr, hc in houses if hr == r)

    def solve(r, used):
        # FOLLOW-UP 4: find_all=True lists every solution
        if found and not find_all:
            return
        # rows up to r-1 are decided, so row r-2 is final
        if r >= 2 and not row_houses_ok(r - 2):
            return
        if r == R:
            # FOLLOW-UP 3: exactly_one=True: a flower in
            # every column
            full = len(used) == C
            if row_houses_ok(R - 1) and (full or
```

```
                                        not exactly_one):
                found.append(["".join(x) for x in cells])
            return
        # try a flower in every free column of this row
        for c in range(C):
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used | {c})
                cells[r][c] = "0"
        # or leave the row empty (not allowed for exactly
        # one)
        if not exactly_one:
            solve(r + 1, used)

    solve(0, frozenset())
    if find_all:
        return found
    return found[0] if found else None

# ---- main question: tests ----
def test_main_question():
    grid = ["H00", "0H0", "00H"]
    sol = place_flowers(grid)
```

```
    assert sol is not None
    # every house has a flower beside it
    for r in range(3):
        for c in range(3):
            if grid[r][c] == "H":
                near = []
                for a, b in STEPS:
                    x, y = r + a, c + b
                    if 0 <= x < 3 and 0 <= y < 3:
                        near.append(sol[x][y])
                assert "F" in near
    # one flower per row and column
    assert all(row.count("F") <= 1 for row in sol)
    for c in range(3):
        assert [row[c] for row in sol].count("F") <= 1
    # a lone house in a 1 x 1 grid cannot be served
    assert place_flowers(["H"]) is None


# ============================================================
# FOLLOW-UP 1
# ============================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Count the solutions instead of listing
# ============================================================

# ---- follow-up 4: function ----
# Answer: Keep the same recursion but add one to a counter
# at the leaf instead of copying the grid; with a bitmask
# of used columns and memoisation on (row, mask, pending-
# house state) it can become a DP.
def count_solutions(grid, exactly_one=False):
    return len(place_flowers(grid, exactly_one, True))

# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert count_solutions(["00", "00"], True) == 2
    assert place_flowers(["000", "000"], True) is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 8 OF 43 |

Runs and groups: extend while it fits, restart or close when it does not

9 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	7
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Runs and groups: extend while it fits, restart or close when it does not | 9 questions | tap a title to jump to it

READ FIRST: the code template for these 9 questions (tap here) >

E	64. Group Timestamps Into Close Intervals	ColorStack	2025-10-03
E	65. First Out-of-Order Character	ColorStack	2026-01-30
M	66. Longest Non-Decreasing Subarray with One Replacement	1Point3Acres	2026-10-04
M	67. Piano Hand Movement Grouping	1Point3Acres	2026-07-14
M	68. Find longest increasing contiguous subarray	PracHub	2026-01-06
M	69. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres	2026-02-08
M	70. Sustained-Error Time Window	ColorStack	2025-09-16
M	71. Longest Non-Decreasing Subarray From Two Arrays	ColorStack	2025-09-24
M	72. Count Super Streaks	ColorStack	2025-10-03

CODE TEMPLATE | GROUP 8 OF 43 | 9 QUESTIONS, ONE PATTERN

Runs and groups: extend while it fits, restart or close when it does not

The one idea these questions share

Walk the sequence once with a counter. If the next item fits the rule, the run grows; if not, the run restarts (or the group closes). Longest run, first break, number of groups, and sums over runs are all small changes to what you record.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def longest_run(a, fits):
    run = best = 1 if a else 0
    for i in range(1, len(a)):
        # CHANGE 1: the rule for continuing a run
        run = run + 1 if fits(a[i - 1], a[i]) else 1
        best = max(best, run)            # CHANGE 2: record longest, first break, count, sum
    return best
```

2. Helper functions that usually get written next to it

```
def groups_of(items, fits_group):          # close a group when the next item does not fit
    groups, current = [], []
    for x in items:
        if current and not fits_group(current, x):
            groups.append(current)
            current = []
        current.append(x)
    if current:
        groups.append(current)
    return groups


def breaks(a, fits):                       # every index where the rule fails
    return [i for i in range(1, len(a)) if not fits(a[i - 1], a[i])]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
64	Group Timestamps Into Close Intervals	close the group when the gap to the next timestamp is at least k; otherwise extend it
65	First Out-of-Order Character	fits = s[i] >= s[i-1]; return i at the first item that does not fit
66	Longest Non-Decreasing Subarray with One Replacement	left[i] and right[i] run lengths; the best answer joins the run before and after one changed element
67	Piano Hand Movement Grouping	start a new group when adding the note would make max - min exceed k
68	Find longest increasing contiguous subarray	fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs
69	Sum of Good Subarrays (Adjacent Diff ±1)	keep the running length and running sum of the current up-run and down-run; add both to the total at every index
70	Sustained-Error Time Window	from the given second extend left and right while errors >= baseline; compare the time span with the interval
71	Longest Non-Decreasing Subarray From Two Arrays	two run lengths per index, one for each array you may take the value from
72	Count Super Streaks	sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

Contents Start the questions >

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 64 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 1 of 9

Template fit: close the group when the gap to the next timestamp is at least k; otherwise extend it | asked 382 days ago | frequency Single report

EASY | ColorStack | technical interview | 64 of 288

Group Timestamps Into Close Intervals

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 69/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #228 Summary Ranges

LeetCode match: Array scan / grouping (related: LC 228 Summary Ranges)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

You are given an array of positive integer timestamps in ascending order and an integer k . Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than k . A run breaks wherever an adjacent difference is >= k . Return each run as an interval [start, end] (its first and last value), in order.

Function signature

def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:

    ...

Follow-up (as reported): One pass, O(n) time and O(1) extra space (besides the output).

EXAMPLES

Example 1

Input	`timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2`
Output	`[[1,4],[11,11],[14,15],[17,17],[21,23]]`
Why	`Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15->17, 17->21) starts a new run.`

Example 2

Input	`timestamps = [5], k = 100`
Output	`[[5,5]]`
Why	`A single timestamp forms a run by itself.`

QUESTION

Example 3

Input	`timestamps = [1,2,3], k = 1`
Output	`[[1,1],[2,2],[3,3]]`
Why	`A diff of 1 is not strictly less than k=1, so every element stands alone.`

CONSTRAINTS

- 1 <= len(timestamps) <= 10^5 , values strictly increasing.
- k >= 1 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_group(ts, k):
    if not ts:
        return []
    cuts = (
        [0]
        + [
            i
            for i in range(1, len(ts))
            if ts[i] - ts[i - 1] >= k
        ]
        + [len(ts)]
    )
    return [
        [ts[a], ts[b - 1]]
        for a, b in zip(cuts, cuts[1:])
    ]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It builds a cut list although a single running start suffices.

OPTIMAL SOLUTION · One pass with a run start

TIME	SPACE
O(n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A run must also never last k seconds or more from its first value (a session limit).

Compare against the run's start: close the run when cur - start >= k (the neighbour gap rule is then implied). One pass again.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Group Timestamps Into Close Intervals
#
# You are given an array of positive integer timestamps in
# ascending order and an integer k . Partition the
# timestamps into maximal runs such that within each run,
# the difference between every pair of adjacent values is
# strictly less than k . A run breaks wherever an adjacent
# difference is >= k . Return each run as an interval
# [start, end] (its first and last value), in order.
# Function signature: def group_into_intervals(timestamps:
# list[int], k: int) -> list[list[int]]: ... Follow-up (as
# reported): One pass, O(n) time and O(1) extra space
# (besides the output).
#
# FOLLOW-UPS (answered below, in this order):
# 1. A run must also never last k seconds or more from its
# first value (a session limit).
#
# INTERVIEW NOTES
# An adjacent difference of at least k cuts the sorted
# list into runs. One left-to-right pass remembers where
# the current run started and closes it whenever the next
# gap is k or more; nothing else is needed, so extra space
# is O(1) besides the output.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One pass with a run start
# Idea: Close the run whenever the gap to the previous
# value reaches k.
def group_into_intervals(timestamps, k):
    if not timestamps:
        return []
    runs, start = [], timestamps[0]
    for prev, cur in zip(timestamps, timestamps[1:]):
        if cur - prev >= k:
            runs.append([start, prev])
            start = cur
    runs.append([start, timestamps[-1]])
    return runs

# ---- main question: tests ----
def test_main_question():
    random.seed(45)
    assert group_into_intervals(
        [1, 2, 3, 4, 11, 14, 15, 17, 21, 22, 23], 2
    ) == [
        [1, 4],
        [11, 11],
        [14, 15],
        [17, 17],
        [21, 23],
    ]
    assert group_into_intervals([5], 100) == [[5, 5]]
    assert group_into_intervals([1, 2, 3], 1) == [
        [1, 1],
        [2, 2],
        [3, 3],
    ]


# ============================================================
# FOLLOW-UP 1: A run must also never last k seconds or
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compare against the run's start: close the run
# when cur - start >= k (the neighbour gap rule is then
# implied). One pass again.
def group_by_span(timestamps, k):
    if not timestamps:
        return []
    runs, start, prev = [], timestamps[0], timestamps[0]
    for cur in timestamps[1:]:
        if cur - start >= k:
            runs.append([start, prev])
            start = cur
        prev = cur
    runs.append([start, prev])
    return runs

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(400):
        ts = sorted(
            random.sample(
                range(1, 40), random.randint(0, 9)
            )
        )
        k = random.randint(1, 6)
        for a, b in group_by_span(ts, k):
            assert b - a < k


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 65 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 2 of 9

Template fit: fits = s[i] >= s[i-1]; return i at the first item that does not fit | asked 263 days ago | frequency Single report

EASY | ColorStack | technical interview | 65 of 288

First Out-of-Order Character

ASKED 2026-01-30 | 263 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 241/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1752 Check if Array Is Sorted and Rotated (related: first violation)

Frequency: Single report | Last asked: 2026-01-30 | Roles: SWE Intern

QUESTION

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed). Return the index of the FIRST character that breaks this rule — that is, the first index i >= 1 where s[i] < s[i-1] . If the entire string is non-decreasing, return -1 .

Function signature

def first_out_of_order(s: str) -> int:

...

Follow-up (as reported): Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

EXAMPLES

Example 1	
Input	`"abcda"`
Output	`4`
Why	`a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.`

Example 2	
Input	`"aabbz"`
Output	`-1`
Why	`Every character is >= the previous one; fully non-decreasing.`

Example 3	
Input	`"cba"`
Output	`1`
Why	`At index 1, 'b' < 'c', the first violation.`

QUESTION

Example 4	
Input	`""`
Output	`-1`
Why	`An empty string is trivially in order.`

CONSTRAINTS

- 0 <= len(s) <= 10^5 s consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order -> return -1 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_first_break(s):
    for i in range(1,_len(s)):
        if list(s[:
            i + 1]) != sorted(
            s[: i + 1]
        ):
            return i
    return -1
```

TIME	SPACE
O(n^2 log n)	O(n)

WHY IT IS SLOW

It sorts every prefix just to learn one comparison.

OPTIMAL SOLUTION · One pass, compare with the previous letter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report every break and the longest in-order run.

Collect each index where s[i] < s[i - 1]; the runs are the gaps between breaks, so the longest run is the largest difference between consecutive break positions (with the ends included).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: First Out-of-Order Character
#
# You are given a string of lowercase English letters.
# Scanning left to right, the string is considered "in
# order" as long as each character is greater than or
# equal to the character before it (a non-decreasing
# sequence; equal adjacent characters are allowed). Return
# the index of the FIRST character that breaks this rule —
# that is, the first index i >= 1 where s[i] < s[i-1] . If
# the entire string is non-decreasing, return -1 .
# Function signature: def first_out_of_order(s: str) ->
# int: ... Follow-up (as reported): Solve in a single O(N)
# pass with O(1) extra space; state the time/space
# complexity as the interviewer expected.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Report every break and the longest in-order run.
#
# INTERVIEW NOTES
# Compare each character with the one before it as you
# scan. The first index where s[i] < s[i - 1] is the
# answer, and no other state is needed: one pass, constant
# extra space.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One pass, compare with the
# previous letter
# Idea: Order is broken exactly where a letter is smaller
# than the one before it.
def first_out_of_order(s):
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            return i
    return -1

# ---- main question: tests ----
def test_main_question():
    random.seed(84)
    assert first_out_of_order("abcda") == 4
    assert first_out_of_order("aabbz") == -1
    assert first_out_of_order("cba") == 1
    assert first_out_of_order("") == -1


# ============================================================
# FOLLOW-UP 1: Report every break and the longest in-order
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect each index where s[i] < s[i - 1]; the
# runs are the gaps between breaks, so the longest run is
# the largest difference between consecutive break
# positions (with the ends included).
def order_breaks(s):
    return [
        i for i in range(1, len(s)) if s[i] < s[i - 1]
    ]


# Answer: Collect each index where s[i] < s[i - 1]; the
# runs are the gaps between breaks, so the longest run is
# the largest difference between consecutive break
# positions (with the ends included).
def longest_in_order_run(s):
    edges = [0] + order_breaks(s) + [len(s)]
    return max(
        (b - a for a, b in zip(edges, edges[1:])),
        default=0,
    )

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(400):
        s = "".join(
            random.choice("abcd")
            for _ in range(random.randint(0, 9))
        )
        runs = longest_in_order_run(s)
        assert runs == max(
            [
                len(x)
                for x in __import__("re").findall(
                    "a*b*c*d*", s
                )
            ]
            or [0]
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 66 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 3 of 9

Template fit: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element | asked 16 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 66 of 288

Longest Non-Decreasing Subarray with One Replacement

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 269/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-non-decreasing-subarray-one-replacement

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of
    length 5.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = longest_non_decreasing(a)
    for i in range(len(a)):
        for v in candidate_values(a):
            b = a[:i] + [v] + a[i + 1:]
            run = longest_non_decreasing(b)
            best = max(best, run)
    return best
```

BRUTE FORCE

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^3). The run lengths on both sides of each position, computed once, answer every change in O(1).

OPTIMAL SOLUTION · Left and right run lengths

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The run must be strictly increasing.

Use < in the run lengths, and when the replaced value must fit between the neighbours check a[i+1] - a[i-1] >= 2 (an integer strictly between them exists) instead of a[i-1] <= a[i+1]. Clarify strict versus non-strict before coding.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray with One
# Replacement
#
# Find the maximum length of a contiguous non-decreasing
# subarray if at most one element may be changed to any
# value. Some rounds start from the plain longest-
# increasing-subarray question and add the one-replacement
# follow-up; those versions ask for an increasing run, so
# clarify strict vs non-strict. Input: an integer array
# nums. You may change at most one element to any value.
# Return the maximum length of a contiguous subarray that
# is non-decreasing (nums[i] <= nums[i + 1] for every
# adjacent pair inside it) after the change. Variant:
# other rounds ask for a strictly increasing run and open
# with the no-replacement version (longest increasing
# subarray) before adding the one-replacement follow-up —
# clarify strict vs non-strict before coding. Expect to
# discuss and compare the complexity of several
# approaches; a brute-force first pass is usually followed
# by a request to optimize.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The run must be strictly increasing.
#
# INTERVIEW NOTES
# Precompute left[i], the length of the non-decreasing run
# ending at i, and right[i], the length of the run
# starting at i. If you change element i to any value, it
# can join the run on its left and the run on its right
# when a[i-1] <= a[i+1] (a value between them exists);
# otherwise it can join only the longer of the two. Take
```

```
# the best over all i.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right

# ---- main question: main function ----
# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[i] and right[i] are the non-decreasing run
# lengths ending and starting at i. Changing a[i] gives
# left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1],
# otherwise max(left[i-1], right[i+1]) + 1.
# FOLLOW-UP 1: The run must be strictly increasing.
# Answer: Use < in the run lengths, and when the replaced
# value must fit between the neighbours check a[i+1] -
# a[i-1] >= 2 (an integer strictly between them exists)
# instead of a[i-1] <= a[i+1]. Clarify strict versus non-
# strict before coding.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    # FOLLOW-UP 1: strict=True means strictly increasing
    # non-decreasing uses <=, strictly increasing uses <
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
```

```
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best


# WARM-UP: no replacement allowed
def longest_without_change(a):
    # the plain longest increasing subarray warm-up
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best

# ---- main question: tests ----
def test_main_question():
    # change 0 to 1: [1, 1, 3, 4, 5] has length 5
    nums = [1, 0, 3, 4, 5, 2, 3]
    assert longest_with_one_change(nums) == 5
    # tiny arrays
    assert longest_with_one_change([]) == 0
    assert longest_with_one_change([7]) == 1
    # strict version needs room between neighbours
    strict = longest_with_one_change
    assert strict([1, 2, 2, 3, 4], True) == 4
    assert strict([1, 1, 1, 1], True) == 2
    # the warm-up without any change
    assert longest_without_change([1, 3, 5, 4, 7]) == 3


# ============================================================
# FOLLOW-UP 1: The run must be strictly increasing.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

STUDY PATH: question 67 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 4 of 9

Template fit: start a new group when adding the note would make max - min exceed k | asked 98 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 67 of 288

Piano Hand Movement Grouping

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 265/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

A pianist plays a sequence of notes in order (a bigger number is a higher key). The hand can stay in one position while every note it plays lies within a span of k (highest minus lowest at most k). Starting a new hand position costs a move. Return the smallest number of hand positions needed to play all the notes in order.

Function signature

def groups(notes: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 3, 5, 7, 9, 11], 4`
Output	`2`
Why	`[1, 3, 5] fits in a span of 4, then [7, 9, 11]: 2 positions.`

Example 2	
Input	`[1, 2, 3, 4, 5], 4`
Output	`1`
Why	`All five notes fit in a span of 4: one position.`

Example 3	
Input	`[5, 9, 1], 4`
Output	`2`
Why	`5 and 9 fit together, then 1 is too far from them: 2 positions.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

FOLLOW-UP 2 Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

FOLLOW-UP 3 The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Piano Hand Movement Grouping
#
# A pianist plays a sequence of notes in order (a bigger
# number is a higher key). The hand can stay in one
# position while every note it plays lies within a span of
# k (highest minus lowest at most k). Starting a new hand
# position costs a move. Return the smallest number of
# hand positions needed to play all the notes in order.
#
# Signature:
#   def groups(notes: list, k: int) -> int
#
# Examples:
# [1, 3, 5, 7, 9, 11], 4  ->  2
# [1, 2, 3, 4, 5], 4  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where each group starts and where to place the
# hand.
# 2. Count lifts, not placements.
# 3. The hand may slide without a lift (costing
# something).
#
# INTERVIEW NOTES
# Turn the physical rule into an invariant: the current
# group is valid while (largest note - smallest note) <=
# k, because you can place the hand at the smallest. The
# first note that breaks it must start a new group, and
# starting later never helps, so a single greedy pass is
# optimal. Ask whether the first placement counts as a
# move.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups

# ---- main question: main function ----
# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
# Idea: Keep lo and hi of the current group. Extend while
# hi - lo stays within k, otherwise close the group and
# start a new one at this note.
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))
```

```


# ---- main question: tests ----
def test_main_question():
    # the stated example: 2 placements for reach 4
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2


# ============================================================
# FOLLOW-UP 1: Return where each group starts and where to
# ============================================================

# ---- follow-up 1: function ----
# Answer: Record the start index when a group opens and
# emit (start, end, smallest note) when it closes. The
# state transition is unchanged.
def plan(notes, k):
    # where each group starts and ends, and where to put
    # the hand (on the lowest note of the group)
    out, spots = [], hand_positions(notes, k)
    for n, (start, lo, hi) in enumerate(spots):
```

```
        end = spots[n + 1][0] - 1 if n + 1 < len(spots) \
            else len(notes) - 1
        out.append((start, end, lo))
    return out

# ============================================================
# FOLLOW-UP 2: Count lifts, not placements.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Lifts are placements minus one if the first
# placement is free; the examples differ on this, so ask
# and state it. In code it is a single subtraction on the
# group count.
def lifts(notes, k):
    # the first placement is free; every move is a lift
    return max(groups(notes, k) - 1, 0)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # the earlier examples, counted as lifts
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1
    assert groups([], 4) == 0
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1),
                                            (3, 5, 7)]
    # a reach of 0 means a new placement for every change
    assert groups([2, 2, 3], 0) == 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

STUDY PATH: question 68 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 5 of 9

Template fit: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs | asked 287 days ago

MEDIUM | PracHub | Technical Screen | 68 of 288

Find longest increasing contiguous subarray

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 167/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: the interview site's own solution

LeetCode match: LC 674 Longest Continuous Increasing Subsequence (variant: one modification) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-longest-increasing-contiguous-subarray

QUESTION

Given an array you may change at most one element to any value (an integer when integers=True, any real number otherwise). Return the length of the longest strictly increasing contiguous subarray you can obtain.

Function signature

def longest_one_change(a: list, integers: bool = True) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 3, 2, 5, 6]`
Output	`6`
Why	`Change the second 2 to 4 and the whole array 1, 2, 3, 4, 5, 6 increases.`

Example 2	
Input	`[1, 3, 2, 4]`
Output	`3`
Why	`Between 1 and 2 there is no integer, so the array 1, 3, 2, 4 only improves to a run of 3.`

Example 3	
Input	`[1, 3, 2, 4], integers = false`
Output	`4`
Why	`With real numbers, 3 can become 1.5 and the whole array increases.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if fixable_with_one_change(part):
                best = max(best, len(part))
    return best
def fixable_with_one_change(part):
    bad = [k for k in range(1, len(part))
           if part[k] <= part[k - 1]]
    if not bad:
        return True
    if len(bad) > 1:
        return False
    return has_room(part, bad[0])
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays, each tested in O(n): O(n^3). Run lengths ending and starting at each index, then trying each index as the changed element, is O(n).

OPTIMAL SOLUTION · Left and right run lengths, try every position as the changed element

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when a[i] <= a[i - 1]. O(n), O(1).

FOLLOW-UP 2 Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

FOLLOW-UP 3 Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find longest increasing contiguous
# subarray
#
# Given an array you may change at most one element to any
# value (an integer when integers=True, any real number
# otherwise). Return the length of the longest strictly
# increasing contiguous subarray you can obtain.
#
# Signature:
#   def longest_one_change(a: list, integers: bool = True)
 -> int
#
# Examples:
# [1, 2, 3, 2, 5, 6]  ->  6
# [1, 3, 2, 4]  ->  3
#
# FOLLOW-UPS (answered below, in this order):
# 1. No modification allowed (LeetCode 674).
# 2. Up to k modifications.
# 3. Values are real numbers, not integers.
#
# INTERVIEW NOTES
# A run that has a single 'bad' spot can be repaired by
# changing one element, so look at each position i as the
# element to change. The best run through i joins the
# increasing run that ends just before it and the one that
# starts just after it, and the join is legal only if the
# values leave enough room between the two neighbours (for
# integers, a[i + 1] - a[i - 1] >= 2).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[i] is the length of the strictly increasing
```

```
# run ending at i and right[i] the length of the one
# starting at i.
def longest_one_change(a, integers=True):
    # strictly increasing after changing AT MOST one value
    n = len(a)
    if n == 0:
        return 0
    # left[i]: run ending at i; right[i]: run starting at
    # i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    # the gap needed between two neighbours: 2 for whole
    # numbers (one number must fit between), > 0 for reals
    best = max(left)
    for i in range(n):
        # change a[i] to join the run on its left ...
        if i > 0:
            best = max(best, left[i - 1] + 1)
        # ... or the run on its right ...
        if i < n - 1:
            best = max(best, right[i + 1] + 1)
        # ... or both, when a value fits between the
        # neighbours
        if 0 < i < n - 1:
            room = a[i + 1] - a[i - 1]
            fits = room >= 2 if integers else room > 0
            if fits:
                joined = left[i - 1] + 1 + right[i + 1]
                best = max(best, joined)
    return min(best, n)

# ---- main question: tests ----
def test_main_question():
    assert longest_one_change([1, 2, 3, 2, 5, 6]) == 6
    assert longest_one_change([1, 3, 2, 4]) == 3
    assert longest_one_change([1, 3, 2, 4], integers=False
) == 4


# ============================================================
# FOLLOW-UP 1: No modification allowed (LeetCode 674).
```

```
# ============================================================

# ---- follow-up 1: function ----
# Answer: One pass keeping the length of the current
# strictly increasing run; reset to 1 when a[i] <= a[i -
# 1]. O(n), O(1).
def longest_increasing(a):
    # LeetCode 674: no change allowed, one pass
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_increasing([1, 3, 5, 4, 7]) == 3
    assert longest_increasing([]) == 0
    assert longest_one_change([1, 3, 5, 4, 7]) == 5
    # equal neighbours leave no room for a whole number
    assert longest_one_change([1, 2, 2, 3]) == 3
    nums = [1, 2, 2, 3]
    reals = longest_one_change(nums, integers=False)
    assert reals == 4
    assert longest_one_change([5, 4, 3, 2, 1]) == 2
    assert longest_one_change([7]) == 1
    assert longest_one_change([]) == 0
    # never more than the array length
    assert longest_one_change([1, 2, 3]) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def longestIncreasing(nums: list[int]) -> int:
```

Sample call

```
longestIncreasing([1, 2, 2, 3, 4])
```

Test cases

Input	Expected
([1, 2, 2, 3, 4],)	3
([1, 2, 3, 4, 5],)	5

STUDY PATH: question 69 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 6 of 9

Template fit: keep the running length and running sum of the current up-run and down-run; add both to the total at every index | asked 254 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 69 of 288

Sum of Good Subarrays (Adjacent Diff ±1)

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 268/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

Original: https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
    -1).
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

TIME	SPACE
O(n^2) or O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

OPTIMAL SOLUTION · Two running run totals

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sum of Good Subarrays (Adjacent Diff ±1)
#
# MLE L5 coding round: sum over all contiguous subarrays
# whose elements form a strictly monotonic ±1 step
# sequence (every adjacent pair differs by exactly +1, OR
# every adjacent pair differs by exactly -1). Required
# time complexity is O(N). Input: integer array arr of
# length N. A subarray is good if its consecutive
# differences are either all +1 (monotone increasing by 1)
# or all -1 (monotone decreasing by 1). Singleton
# subarrays are always good. Output: the sum of element-
# sums across all good subarrays (each good subarray
# contributes the sum of its elements once). Required:
# O(N) time. Examples: arr = [3, 5, 6, 7, 6] Good
# subarrays: singletons: [3], [5], [6], [7], [6] monotone
```

```
# +1 runs: [5, 6], [6, 7], [5, 6, 7] monotone -1 runs: [7,
# 6] Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
# -1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the singleton subtracted?
#
# INTERVIEW NOTES
# A good subarray is a stretch where every step is +1 or
# every step is -1, so every good subarray ending at index
# i lies inside the current run. Instead of listing
# subarrays, keep for each direction the total of all
# suffix sums inside the run: extending a run of length L
# by a new element x adds (L + 1) * x to that total. Both
# directions can be updated in O(1), and the singleton is
# counted once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import itertools


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * x and
# L becomes L + 1; a break restarts at S = x, L = 1.
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total

# ---- main question: tests ----
def test_main_question():
    arr = [3, 5, 6, 7, 6]
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 +
    # [7,6] 13
    assert good_subarray_sum(arr) == 82
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

STUDY PATH: question 70 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 7 of 9

Template fit: from the given second extend left and right while errors >= baseline; compare the time span with the interval | asked 399 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 70 of 288

Sustained-Error Time Window

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 217/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1004 Max Consecutive Ones III

LeetCode match: LC 1004 Max Consecutive Ones III (related: runs of readings)

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: times (timestamps in seconds, strictly increasing) and errors (the error reading recorded at each timestamp), where errors[i] was recorded at times[i] . You are also given three integers: second , interval , and baseline . Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp second (guaranteed to exist exactly once in times), 2. Has every reading in the run satisfy error >= baseline , and 3. Spans a duration of at least interval seconds — that is, times[right] - times[left] >= interval , where left and right are the endpoints of the run. Return True if such a run exists, otherwise False . Note that if the reading at second is itself below baseline , the answer is immediately False .

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes
second and spans >=
interval seconds."""
    ...
```

EXAMPLES

QUESTION

Example 1

Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 1, interval = 1, baseline = 8`
Output	`True`
Why	`Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8) but stops at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval 1.`

Example 2

Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 4, interval = 2, baseline = 8`
Output	`False`
Why	`Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3 (err 5) blocks going left. The run covers times 4..5, span 1 < interval 2.`

Example 3

Input	`times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8`
Output	`False`
Why	`The reading at second=2 is 4 < baseline 8, so no valid run includes it.`

CONSTRAINTS

- 1 <= len(times) == len(errors) <= 10^5 times is strictly increasing; 0 <= times[i] <= 10^9
- -10^9 <= errors[i] <= 10^9 , baseline in the same range second is one of the values in times ; 0 <= interval <= 10^9

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_sustained(
    times, errors, second, interval, baseline
):
    at = times.index(second)
    for left in range(at_+_1):
        for right in range(at,_len(times)):
            run = errors[left : right + 1]
            if (
                min(run) >= baseline
                and times[right] - times[left]
                >= interval
            ):
                return True
    return False
```

TIME	SPACE
O(n^3)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Every candidate run is re-checked reading by reading.

OPTIMAL SOLUTION · Expand around the reading

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You get thousands of (second, interval) queries on the same data.

Compute once, for every index, the start and end of the maximal run of readings at or above the baseline (one left-to-right pass and one right-to-left pass). Each query is then a lookup plus a binary search for the index of second, O(log n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sustained-Error Time Window
#
# Reconstructing the "two arrays - Error and Time"
# question. You are given two parallel arrays: times
# (timestamps in seconds, strictly increasing) and errors
# (the error reading recorded at each timestamp), where
# errors[i] was recorded at times[i] . You are also given
# three integers: second , interval , and baseline .
# Determine whether there exists a contiguous run of
# readings that: 1. Includes the reading taken at
# timestamp second (guaranteed to exist exactly once in
# times ), 2. Has every reading in the run satisfy error
# >= baseline , and 3. Spans a duration of at least
# interval seconds — that is, times[right] - times[left]
# >= interval , where left and right are the endpoints of
# the run. Return True if such a run exists, otherwise
# False . Note that if the reading at second is itself
# below baseline , the answer is immediately False .
#
# FOLLOW-UPS (answered below, in this order):
# 1. You get thousands of (second, interval) queries on
# the same data.
#
# INTERVIEW NOTES
# The run that contains the reading at second is the
# maximal block of consecutive readings with error >=

# =============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
```

```
# =============================================================

from bisect import bisect_left
import random


# =============================================================
# MAIN QUESTION
# =============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Expand around the reading
# Idea: The longest valid run around the index is the best
```

```
# one, so only its span matters.
def has_sustained_error_window(
    times, errors, second, interval, baseline
):
    at = times.index(second)
    if errors[at] < baseline:
        return False
    left = right = at
    while left > 0 and errors[left - 1] >= baseline:
        left -= 1
    while (
        right + 1 < len(times)
        and errors[right + 1] >= baseline
    ):
        right += 1
    return times[right] - times[left] >= interval

# ---- main question: tests ----
def test_main_question():
    random.seed(13)
    t = [1, 2, 3, 4, 5]
    e = [10, 20, 5, 30, 40]
    assert (
        has_sustained_error_window(t, e, 1, 1, 8)
        is True
    )
    assert (
        has_sustained_error_window(t, e, 4, 2, 8)
        is False
    )
    assert (
        has_sustained_error_window(t, e, 3, 0, 8)
        is False
    )


# =============================================================
# FOLLOW-UP 1: You get thousands of (second, interval)
# =============================================================

# ---- follow-up 1: function ----
# Answer: Compute once, for every index, the start and end
# of the maximal run of readings at or above the baseline
# (one left-to-right pass and one right-to-left pass).
def run_bounds(times, errors, baseline):
```

```
    n = len(times)
    left, right = [0] * n, [0] * n
    for i in range(n):
        left[i] = (
            left[i - 1]
            if i
            and errors[i - 1] >= baseline
            and errors[i] >= baseline
            else i
        )
    for i in range(n - 1, -1, -1):
        right[i] = (
            right[i + 1]
```

```
            if i + 1 < n
            and errors[i + 1] >= baseline
            and errors[i] >= baseline
            else i
        )
    return left, right


# Answer: Compute once, for every index, the start and end
# of the maximal run of readings at or above the baseline
# (one left-to-right pass and one right-to-left pass).
def sustained_query(
    times, errors, bounds, second, interval, baseline
):
    at = bisect_left(times, second)
    if errors[at] < baseline:
        return False
    left, right = bounds
    return (
        times[right[at]] - times[left[at]] >= interval
    )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

STUDY PATH: question 71 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 8 of 9

Template fit: two run lengths per index, one for each array you may take the value from | asked 391 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 71 of 288

Longest Non-Decreasing Subarray From Two Arrays

ASKED 2025-09-24 | 391 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 222/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2771 Longest Non-decreasing Subarray From Two Arrays

Frequency: Single report | Last asked: 2025-09-24 | Roles: SWE Intern

QUESTION

You are given two integer arrays nums1 and nums2 , both of length n . You must construct a third array nums3 of length n where, for each index i , nums3[i] is chosen to be either nums1[i] or nums2[i] (chosen independently per index). Return the length of the longest non-decreasing subarray (a contiguous run) achievable in nums3 over all valid choices.

Function signature

def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:

...

Follow-up (as reported): (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to write test cases.

EXAMPLES

Example 1

Input	`nums1 = [2, 3, 1], nums2 = [1, 2, 1]`
Output	`2`
Why	`Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2. No choice yields a non-decreasing run of length 3.`

Example 2

Input	`nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]`
Output	`4`
Why	`nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3]) is non-decreasing over all 4 positions.`

QUESTION

Example 3

Input	`nums1 = [1, 1], nums2 = [2, 2]`
Output	`2`
Why	`Any choice, e.g. [1, 1], is non-decreasing across both.`

CONSTRAINTS

- 1 <= n <= 10^5
- nums1 and nums2 have the same length n .
- 1 <= nums1[i], nums2[i] <= 10^9
- Target complexity: O(n) time, O(1) extra space.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import product
def brute_longest(nums1, nums2):
    best = 0
    for picks in product(
        (0, 1), repeat=len(nums1)
    ):
        arr = [
            (nums1, nums2)[p][i]
            for i, p in enumerate(picks)
        ]
        run = 1
        best = max(best, run)
        for a, b in zip(arr, arr[1:]):
            run = run + 1 if a <= b else 1
            best = max(best, run)
    return best
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

It builds all 2^n possible nums3 arrays.

OPTIMAL SOLUTION · Two-state DP

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 First solve it for a single array, then generalize to picking per index from two arrays.

One array: run = run + 1 if nums[i - 1] <= nums[i] else 1 and keep the maximum, O(n) time and O(1) space. The two-array version keeps one run length per choice instead of one.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray From Two
# Arrays
#
# You are given two integer arrays nums1 and nums2 , both
# of length n . You must construct a third array nums3 of
# length n where, for each index i , nums3[i] is chosen to
# be either nums1[i] or nums2[i] (chosen independently per
# index). Return the length of the longest non-decreasing
# subarray (a contiguous run) achievable in nums3 over all
# valid choices. Function signature: def
# longest_non_decreasing_subarray(nums1: list[int], nums2:
# list[int]) -> int: ... Follow-up (as reported):
# (Reported) The interviewer first asked for the longest
# contiguous non-decreasing subarray of a single array,
# then generalized to picking per index from two arrays;
# also asked for the time complexity and to write test
# cases.
#
# FOLLOW-UPS (answered below, in this order):
# 1. First solve it for a single array, then generalize to
# picking per index from two arrays.
#
# INTERVIEW NOTES
# Choosing nums3[i] from either array is a two-state DP:
# keep the length of the best run that ends at i if we
# picked nums1[i], and the same if we picked nums2[i].
# Each state extends the previous run when the previous
# choice is not larger than the current value.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two-state DP
```

```
# Idea: dp1 and dp2 are the best run ending at i using
# nums1[i] or nums2[i].
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = dp2 = best = 1
    for i in range(1, len(nums1)):
        n1 = n2 = 1
        for prev_val, prev_len in (
            (nums1[i - 1], dp1),
            (nums2[i - 1], dp2),
        ):
            if prev_val <= nums1[i]:
                n1 = max(n1, prev_len + 1)
            if prev_val <= nums2[i]:
                n2 = max(n2, prev_len + 1)
        dp1, dp2 = n1, n2
        best = max(best, dp1, dp2)
    return best

# ---- main question: tests ----
def test_main_question():
    random.seed(26)
    assert (
        longest_non_decreasing_subarray(
            [2, 3, 1], [1, 2, 1]
        )
        == 2
    )
    assert (
        longest_non_decreasing_subarray(
            [1, 3, 2, 1], [2, 2, 3, 4]
        )
        == 4
    )
    assert (
        longest_non_decreasing_subarray([1, 1], [2, 2])
        == 2
    )


# ============================================================
# FOLLOW-UP 1: First solve it for a single array, then
# ============================================================

# ---- follow-up 1: function ----
# Answer: One array: run = run + 1 if nums[i - 1] <=
```

```
# nums[i] else 1 and keep the maximum, O(n) time and O(1)
# space. The two-array version keeps one run length per
# choice instead of one.
def longest_non_decreasing_one(nums):
    best = run = 1 if nums else 0
    for a, b in zip(nums, nums[1:]):
        run = run + 1 if a <= b else 1
        best = max(best, run)
    return best


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 72 of 288 | topic 8 of 43: Runs and groups: extend while it fits, restart or close when it does not | 9 of 9

Template fit: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X | asked 382 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 72 of 288

Count Super Streaks

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 225/288

ABOUT THIS QUESTION

Topic: Runs and groups: extend while it fits, restart or close when it does not

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #128 Longest Consecutive Sequence

LeetCode match: Sorting + grouping (related: LC 128 / session windows)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

A player produces a list of events, each (event_id, event_type, event_time) . Sort events by event_time . A streak is a maximal run of events, consecutive in time order, in which every event has the same event_type AND the gap between each event and the previous one in the run is at most T seconds. A streak ends when the next event has a different event_type or its gap from the previous event exceeds T . A streak is a Super Streak if it contains at least N events AND spans at least X seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Function signature

```
def count_super_streaks(events: list[tuple[int, str, int]],
                        N: int, T: int, X: int) -> int:
    ...
```

Follow-up (as reported): Follow-up 1: events are tagged with user_id ; return a per-user count of Super Streaks. Follow-up 2: after processing, support count_streaks(user_id, time_A, time_B) returning how many of that user's Super Streaks completed entirely within [time_A, time_B] . Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

EXAMPLES

Example 1

Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10, X=30`
Output	`1`
Why	`One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4). Super.`

QUESTION

Example 2

Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40),(6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30`
Output	`1`
Why	`The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is Super; the second [55,56,57] has only 3 events and span 2, so not Super.`

Example 3

Input	`events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1`
Output	`0`
Why	`Type changes each step, so every streak has length 1 — none reaches N=2.`

CONSTRAINTS

- 0 <= len(events) <= 10^5 .
- N >= 1 , T >= 0 , X >= 0 ; times are non-negative integers, possibly with duplicate timestamps.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_super(events, N, T, X):
    events = sorted(events, key=lambda e: e[2])
    streaks = []
    for e in events:
        if (
            streaks
            and streaks[-1][-1][1] == e[1]
            and e[2] - streaks[-1][-1][2] <= T
        ):
            streaks[-1].append(e)
        else:
            streaks.append([e])
    return sum(
        1
        for s in streaks
        if len(s) >= N
        and s[-1][2] - s[0][2] >= X
    )
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

It stores every streak as a list although only the size and the first and last time matter.

OPTIMAL SOLUTION · One pass over the sorted events

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events carry a user_id. Return the number of super streaks for each user.

Group the events by user first and run the same pass on each group, or key the open streak state by user in one pass over the sorted events. The cost stays O(n log n).

FOLLOW-UP 2 Then answer count_streaks(user_id, time_A, time_B): how many of that user's super streaks finished entirely inside [A, B]?

Record each super streak as (start, end) when it closes, per user, sorted by start (they do not overlap). A query counts the streaks with start >= A and end <= B: bisect on the starts and scan, or keep the ends in a second sorted list. O(log s + answer) per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Super Streaks
#
# A player produces a list of events, each (event_id,
# event_type, event_time) . Sort events by event_time . A
# streak is a maximal run of events, consecutive in time
# order, in which every event has the same event_type AND
# the gap between each event and the previous one in the
# run is at most T seconds. A streak ends when the next
# event has a different event_type or its gap from the
# previous event exceeds T . A streak is a Super Streak if
# it contains at least N events AND spans at least X
# seconds (span = last event time minus first event time
# within the streak). Return the number of Super Streaks.
# Function signature: def count_super_streaks(events:
# list[tuple[int, str, int]], N: int, T: int, X: int) ->
# int: ... Follow-up (as reported): Follow-up 1: events
# are tagged with user_id ; return a per-user count of
# Super Streaks.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events carry a user_id. Return the number of super
# streaks for each user.
# 2. Then answer count_streaks(user_id, time_A, time_B):
# how many of that user's super streaks finished entirely
# inside [A, B]?
#
# INTERVIEW NOTES
# Sort by time, then one pass: extend the current streak
# while the type is the same and the gap from the previous
# event is at most T; otherwise close it. A closed streak
# counts when it has at least N events and spans at least
# X seconds. Only the streak start, the previous time and
# the size are needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import defaultdict
from bisect import bisect_left
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One pass over the sorted events
# Idea: Keep only the open streak's start, last time, size
# and type.
def count_super_streaks(events, N, T, X):
    count = 0
    start = prev = kind = None
    size = 0
    for _, etype, etime in sorted(
        events, key=lambda e: e[2]
    ):
        if kind == etype and etime - prev <= T:
            size += 1
        else:
            if size >= N and prev - start >= X:
                count += 1
            start, kind, size = etime, etype, 1
        prev = etime
    if size >= N and prev - start >= X:
        count += 1
    return count

# ---- main question: tests ----
def test_main_question():
    random.seed(42)
    ev = [
        (1, "A", 0),
        (2, "A", 10),
        (3, "A", 20),
        (4, "A", 30),
        (5, "A", 40),
    ]
    assert count_super_streaks(ev, 4, 10, 30) == 1
    ev2 = ev + [
        (6, "A", 55),
        (7, "A", 56),
        (8, "A", 57),
    ]
    assert count_super_streaks(ev2, 4, 10, 30) == 1
    assert count_super_streaks([], 1, 1, 0) == 0


# ============================================================
# FOLLOW-UP 1: Events carry a user_id. Return the number
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group the events by user first and run the same
```

```
# pass on each group, or key the open streak state by user
# in one pass over the sorted events. The cost stays O(n
# log n).
def super_streaks_per_user(events, N, T, X):
    by_user = defaultdict(list)
    for eid, etype, etime, user in events:
        by_user[user].append((eid, etype, etime))
    return {
        u: count_super_streaks(evs, N, T, X)
        for u, evs in by_user.items()
    }

# ============================================================
# FOLLOW-UP 2: Then answer count_streaks(user_id, time_A,
# ============================================================

# ---- follow-up 2: function ----
# Answer: Record each super streak as (start, end) when it
# closes, per user, sorted by start (they do not overlap).
class StreakIndex:
    def __init__(self, events, N, T, X):
        self.spans = defaultdict(list)
        by_user = defaultdict(list)
        for eid, etype, etime, user in events:
            by_user[user].append((eid, etype, etime))
        for user, evs in by_user.items():
            evs.sort(key=lambda e: e[2])
            run = [evs[0]]
            for e in evs[1:] + [None]:
                if (
                    e
                    and e[1] == run[-1][1]
                    and e[2] - run[-1][2] <= T
                ):
                    run.append(e)
                    continue
                if (
                    len(run) >= N
                    and run[-1][2] - run[0][2] >= X
                ):
                    self.spans[user].append(
                        (run[0][2], run[-1][2])
                    )
                run = [e] if e else []

    def count_streaks(self, user, a, b):
        spans = self.spans.get(user, [])
        first = bisect_left(spans, (a, float("-inf")))
        return sum(
            1
            for s, e in spans[first:]
            if s <= b and e <= b
        )

# ---- follow-up 2: tests ----
def test_follow_up_2():
    for _ in range(400):
        events = [
            (
                i,
                random.choice("AB"),
                random.randint(0, 40),
            )
            for i in range(random.randint(0, 9))
        ]
        N, T, X = (
            random.randint(1, 3),
            random.randint(1, 8),
            random.randint(0, 12),
        )
        tagged = [
            (i, t, tm, random.choice("uv"))
            for i, t, tm in events
        ]
        per = super_streaks_per_user(tagged, N, T, X)
        if tagged:
            idx = StreakIndex(tagged, N, T, X)
            user = tagged[0][3]
            assert (
                idx.count_streaks(user, 0, 100)
                == per[user]
            )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 9 OF 43 |

Union-find: merge the pairs, then ask who is together

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Union-find: merge the pairs, then ask who is together | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

M	73. Count Islands in a Grid and Track the Count as Land Cells Are Added	PracHub	2026-10-02
M	74. 2D Points Connected Components by Radius	1Point3Acres	2026-08-04
M	75. Count Clusters of 2D Points Within a Radius	PracHub	2026-06-18
M	76. Find A Threshold-Limited Path With Minimum Required Safety	PracHub	2026-07-06
M	77. Movie Recommendations via Transitive Similarity	ColorStack	2025-10-09
M	78. Determine Reporting Relationships	PracHub	2026-02-22
M	79. Determine whether two nodes are related	PracHub	2026-01-22
H	80. Friend Graph with Timeline (Friend / Unfriend Events)	1Point3Acres	2026-09-29

CODE TEMPLATE | GROUP 9 OF 43 | 8 QUESTIONS, ONE PATTERN

Union-find: merge the pairs, then ask who is together

The one idea these questions share

Keep a parent array. find climbs to the root (with path compression), union joins two roots. The only thing that changes between questions is where the pairs to merge come from and what you read at the end: same group, group size, number of groups, or the moment everything became one group.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.groups = n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.parent[ra] = rb
        self.groups -= 1
        return True


# for a, b in pairs:        # CHANGE 1: where the pairs come from
#     dsu.union(a, b)
# CHANGE 2: read dsu.find(x) == dsu.find(y), dsu.groups, or the pair that made groups == 1
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict


def cell_buckets(points, r):               # spatial hash: only nearby cells can hold neighbours
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // r, y // r)].append(i)
    return cells


def group_sizes(dsu, n):
    sizes = defaultdict(int)
    for x in range(n):
        sizes[dsu.find(x)] += 1
    return sizes
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
73	Count Islands in a Grid and Track the Count as Land Cells Are Added	each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition
74	2D Points Connected Components by Radius	pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

Contents Start the questions >

#	Question	What you change in the template
75	Count Clusters of 2D Points Within a Radius	identical to the line above: cell buckets give the candidate pairs, answer = groups
76	Find A Threshold-Limited Path With Minimum Required Safety	pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge
77	Movie Recommendations via Transitive Similarity	pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating
78	Determine Reporting Relationships	pairs = peer links; keep the boss at the root and carry it over when two groups merge
79	Determine whether two nodes are related	pairs = the given edges; the query is find(a) == find(b)
80	Friend Graph with Timeline (Friend / Unfriend Events)	pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 73 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 1 of 8

Template fit: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition | asked 18 days ago

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 73 of 288

Count Islands in a Grid and Track the Count as Land Cells Are Added

POSTED 2026-10-02 | 18 DAYS AGO | LAST 30 DAYS

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 120/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Posted on PracHub: 2026-10-02 (updated 2026-10-03)

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with **m** rows and **n** columns in which every cell is land (**1**) or water (**0**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **grid[r][c]** is **1** for land and **0** for water, where **r** is the row index and **c** is the column index.
- An addition **(r, c)** turns cell **(r, c)** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **len(additions) + 1**. Element **0** is the number of islands in the initial grid. For **i >= 1**, element **i** is the number of islands after the first **i** additions have been applied.

Constraints

- **1 <= m <= 1000**, **1 <= n <= 1000** and **m * n <= 200000**
- Every row of **grid** has exactly **n** entries, each **0** or **1**.
- **0 <= len(additions) <= 200000**
- Every addition **(r, c)** satisfies **0 <= r < m** and **0 <= c < n**. The same cell may appear more than once.

QUESTION

- Every count is at most **m * n**, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_land(r, c):
    grid[r][c] = 1
    return count_islands(grid)  # full flood fill
answers = [add_land(r, c) for r, c in additions]
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION · Union-Find updated cell by cell

TIME	SPACE
O(m * n + A * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

FOLLOW-UP 2 The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. **Cost:** O(a * alpha) for a additions and O(a) memory, independent of the grid size.

FOLLOW-UP 3 Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. **Cost:** O(alpha) per addition with size tracking; the maximum is one running variable.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# ---- main question: main function ----
# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1
        self.largest = max(self.largest, self.size[a])


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.count)
    return out

# ---- main question: tests ----
def test_main_question():
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    # 2 islands; the adds join them into one; a repeat is
    # ignored
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
```

```
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Report the size of the largest island after
# ============================================================

# ---- follow-up 3: function ----
# Answer: Store a size per root in the Union-Find (union
# by size) and keep a running maximum, which only grows
# when land is added; each step stays near-constant time.
# Checked against recomputing the largest island from
# scratch.
def largest_after_each(grid, additions):
    # the follow-up: size of the biggest island each time
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.largest]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.largest)
    return out

# ---- follow-up 3: tests ----
def test_follow_up_3():
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    sizes = largest_after_each(grid, adds)
    assert sizes == [1, 2, 3, 5, 5]
    # corners do not connect cells
    assert island_counts([[1, 0], [0, 1]], []) == [2]
    # the grid may be huge: only land cells are stored
    big = Islands()
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
def _count_islands(grid):
    m, n = len(grid), len(grid[0]) if grid else 0
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for nx, ny in (
                        (x + 1, y),
                        (x - 1, y),
                        (x, y + 1),
                        (x, y - 1),
                    ):
                        if (
                            0 <= nx < m
                            and 0 <= ny < n
                            and grid[nx][ny] == 1
                            and (nx, ny) not in seen
                        ):
                            seen.add((nx, ny))
                            stack.append((nx, ny))
    return count


def island_counts_recount(grid, additions):
    g = [row[:] for row in grid]
    out = [_count_islands(g)]
    for r, c in additions:
        g[r][c] = 1
        out.append(_count_islands(g))
    return out


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[l
ist[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
[])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

STUDY PATH: question 74 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 2 of 8

Template fit: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups | asked 77 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen | 74 of 288

2D Points Connected Components by Radius

ASKED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 40.7** | **30 DAYS 0** | **PRIORITY 104/288**

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Points within distance 2 of each other form a cluster: two near the origin, two near (10, 10), one alone.`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0), (3, 0)], 1`
Output	`1`
Why	`A chain is one cluster even though its ends are far apart.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

OPTIMAL SOLUTION · Union-find with a spatial grid

TIME	SPACE
O(n) about when points are spread out (O(n^2) if they all share a cell)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: 2D Points Connected Components by Radius
#
# Given 2D points and radius r, connect two points when
# their distance is at most r; connectivity is transitive,
# and the output is the final number of clusters. The
# interviewer accepted or preferred a brute-force neighbor
# scan after discussing optimization ideas. Input: a list
# of 2D points and a radius r. Two points are connected if
# their Euclidean distance is <= r. Connectivity is
# transitive: if A connects to B and B connects to C, all
```

```
# three belong to the same cluster. Return the number of
# final clusters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why compare squared distances?
#
# INTERVIEW NOTES
# Two points are neighbours when their squared distance is
# at most r squared (no square roots needed), and
# connectivity is transitive, so this is counting
# connected components: union-find or a DFS over the
# neighbour relation. Checking every pair is O(n^2);
# bucketing points into cells of side r means a point only
# needs to be compared with the 3 x 3 cells around it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# FOLLOW-UP 1: Why compare squared distances?
# Answer: sqrt is slow and inexact; dist <= r is the same
# as dist^2 <= r^2 for non-negative numbers, and with
# integer coordinates the comparison is exact. It also
# avoids floating-point surprises right at the boundary.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r

# ---- main question: main function ----
# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (x // r, y // r). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared.
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


# ---- main question: tests ----
def test_main_question():
    # three points joined only through the middle one
    line = [(0, 0), (3, 0), (6, 0)]
    assert count_clusters(line, 3) == 1
    # the far point stays alone
    far = [(0, 0), (3, 0), (7, 0)]
    assert count_clusters(far, 3) == 2
    # radius 0 only joins identical points
    same = [(1, 1), (1, 1), (2, 2)]
    assert count_clusters(same, 0) == 2
    assert count_clusters([], 5) == 0
    # both methods agree on a scatter of points
    pts = [(i * 7 % 13, i * 5 % 11) for i in range(30)]


# ============================================================
# FOLLOW-UP 1: Why compare squared distances?
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY PATH: question 75 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 3 of 8

Template fit: identical to the line above: cell buckets give the candidate pairs, answer = groups | asked 124 days ago

MEDIUM | PracHub | Technical Screen | 75 of 288

Count Clusters of 2D Points Within a Radius

POSTED 2026-06-18 | 124 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 40.7 | 30 DAYS 0 | PRIORITY 103/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (inferred from overview)

Posted on PracHub: 2026-06-18 (updated 2026-07-01)

Original: https://prachub.com/coding-questions/count-clusters-of-2d-points-within-a-radius

QUESTION

Given 2D points and a radius r, two points are connected when their distance is at most r. A cluster is a group of points connected directly or through chains of other points. Return the number of clusters.

Function signature

def count_clusters(points: list, r: float) -> int

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Three groups: the two points near the origin, the two near (10, 10), and the lone point at (20, 20).`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0)], 1`
Output	`1`
Why	`A chain: (0,0)-(1,0)-(2,0) are linked neighbour to neighbour, so it is one cluster.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
for a, b in combinations(range(n), 2):
    if dist2(points[a], points[b]) <= r * r:
        parent[find(a)] = find(b)
clusters = len({find(i) for i in range(n)})
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n^2 / 2 distance checks. Hashing points into grid cells of side r means each point is compared only with points in its own and neighbouring cells.

OPTIMAL SOLUTION · Grid hashing with cells of side r, then Union-Find

TIME	SPACE
O(n) expected for evenly spread points	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(i)). The unions are the same.

FOLLOW-UP 2 Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

FOLLOW-UP 3 Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. **Cost:** grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import Counter, defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 2: Points arrive one at a time.
# Answer: Insert the point into its grid cell, union it
# with every point within r in the 9 surrounding cells,
# and keep the component count: components += 1 - (number
# of distinct roots merged).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Clusters:
    """Points arrive one by one; count the clusters."""

    def __init__(self, r):
        self.r = r
        # grid cell (side r) -> indices of points inside
        self.cells = defaultdict(list)
        self.points, self.parent = [], []
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx, dy = p[0] - q[0], p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
```

```
        self.parent.append(i)
        self.components += 1
        size = self.r or 1
        cx, cy = x // size, y // size
        self.cells[(cx, cy)].append(i)
        # look only at the 9 cells around the new point
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                key = (cx + dx, cy + dy)
                cell = self.cells.get(key, ())
                for j in cell:
                    q = self.points[j]
                    if j != i and self.near((x, y), q):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        sizes = Counter(self.find(i)
                        for i in range(len(self.points)))
        return max(sizes.values(), default=0)

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (x // r, y // r). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those.
def count_clusters(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.components

# ---- main question: tests ----
def test_main_question():
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert count_clusters(line, 3) == 2


# ============================================================
# FOLLOW-UP 1: Report the size of the largest cluster.
```

```
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count points per root after the unions (a
# Counter of find(i)). The unions are the same.
def largest_cluster(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.largest()

# ---- follow-up 1: tests ----
def test_follow_up_1():
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert largest_cluster(line, 3) == 3
    assert count_clusters([], 3) == 0
    # online: the count changes as points arrive
    c = Clusters(2)
    assert [c.add(*p) for p in [(0, 0), (10, 10), (1, 1),
                                (5, 5)]] == [1, 2, 2, 3]
    # (3, 3) is alone; (4, 4) then bridges it to (5, 5)
    assert c.add(3, 3) == 4
    assert c.add(4, 4) == 3
    assert count_clusters([(1, 1), (1, 1)], 0) == 1


# ============================================================
# FOLLOW-UP 2: Points arrive one at a time.
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 2.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything above is defined)
# ============================================================
def _find_factory(n):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    return parent, find
```

```
def clusters_all_pairs(points, r):
    parent, find = _find_factory(len(points))
    for i in range(len(points)):
```

```
        for j in range(i + 1, len(points)):
            if (points[i][0] - points[j][0]) ** 2 + (
                points[i][1] - points[j][1]
            ) ** 2 <= r * r:
                parent[find(i)] = find(j)
    return len({find(i) for i in range(len(points))})


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def countClusters(points: list[list[int]], r: int) -> int:
```

Sample call

```
countClusters([[0, 0], [1, 0], [10, 10]], 2)
```

Test cases

Input	Expected
([[0, 0], [1, 0], [10, 10]], 2)	2
([[0, 0], [3, 0], [6, 0]], 3)	1

ALSO REPORTED · 2 more

Count Connected Clusters of Two-Dimensional Points | PracHub | Technical Screen

https://prachub.com/coding-questions/count-connected-clusters-of-two-dimensional-points

2D Points Connected Components by Radius | 1Point3Acres | phone-screen | asked 2026-08-04 | Low

https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

DFS, BFS, or Union-Find all fit the core connected-components model.
For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.
Clarify squared-distance comparison to avoid unnecessary square roots.

Preparation: Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY PATH: question 76 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 4 of 8

Template fit: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge | asked 106 days ago

MEDIUM | IN APP LC 1102 | PracHub | Technical Screen | 76 of 288

Find A Threshold-Limited Path With Minimum Required Safety

POSTED 2026-07-06 | 106 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 157/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1102 Path With Maximum Minimum Value

LeetCode match: LC 1102 Path With Maximum Minimum Value (inferred from overview)

Posted on PracHub: 2026-07-06 (updated 2026-07-08)

Original: https://prachub.com/coding-questions/find-a-threshold-limited-path-with-minimum-required-safety

QUESTION

A graph has n nodes and edges (u, v, safety). The safety of a path is the smallest safety among its edges. Using only edges whose safety is at least threshold (no limit when threshold is None), return the best (largest) safety you can reach going from s to t, or -1 if no such path exists. A path from a node to itself has infinite safety.

Function signature

def best_path_safety(n: int, edges: list, s: int, t: int, threshold=None) -> int

EXAMPLES

Example 1

Input	`3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2`
Output	`3`
Why	`Direct edge 0-2 has safety 1; the route 0-1-2 has safety min(5, 3) = 3, which is better.`

Example 2

Input	`3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold = 4`
Output	`-1`
Why	`Only edges with safety at least 4 may be used: 0-1 is allowed but nothing leads to node 2.`

QUESTION

Example 3

Input	`3, [(0, 1, 5)], 1, 1`
Output	`infinity`
Why	`Source and target are the same node.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, src, dst):
    best = -INF
    for path in all_paths(graph, src, dst):
        weakest = min(edge_safety(path))
        best = max(best, weakest)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential. Adding edges from strongest to weakest in a Union-Find until the source and target connect is O(E log E).

OPTIMAL SOLUTION · Union-Find adding edges from strongest to weakest

TIME	SPACE
O(E log E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

FOLLOW-UP 2 Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

FOLLOW-UP 3 Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find A Threshold-Limited Path With
# Minimum Required Safety
#
# A graph has n nodes and edges (u, v, safety). The safety
# of a path is the smallest safety among its edges. Using
# only edges whose safety is at least threshold (no limit
# when threshold is None), return the best (largest)
# safety you can reach going from s to t, or -1 if no such
# path exists. A path from a node to itself has infinite
# safety.
#
# Signature:
#   def best_path_safety(n: int, edges: list, s: int, t: i
nt, threshold=None) -> int
#
# Examples:
# 3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2  ->  3
# 3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold =
#     4  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use a max-heap search instead of Union-Find.
# 2. Answer many (source, target) queries.
# 3. Edges below a threshold are forbidden (threshold-
# limited).
#
# INTERVIEW NOTES
# When the quality of a path is decided by its weakest
# edge, the best path does not need to be short, only
# strong. Two equivalent tools: add edges from strongest
# to weakest with Union-Find until source and target
# connect (that edge's value is the answer), or run
# Dijkstra with a max-heap where extending a path takes
# the minimum so far. Disconnected graphs must return the
# 'no path' value.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)

# ---- main question: main function ----
# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
# union their endpoints one by one.
def best_path_safety(n, edges, s, t, threshold=None):
    # edges are (u, v, safety). The safety of a path is
    # its
    # weakest edge; return the best safety, or -1 if s and
    # t
    # are not connected through edges >= threshold
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    # add the safest edges first: the edge that joins s
    # and t
    # is the weakest edge of the best path
    for u, v, w in sorted(edges, key=lambda e: -e[2]):
        if threshold is not None and w < threshold:
            break
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1

# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    # 0-1-2-3 has weakest edge 3; 0-2-3 only 1
    assert best_path_safety(4, edges, 0, 3) == 3


# ============================================================
# FOLLOW-UP 1: Use a max-heap search instead of Union-
# ============================================================

# ---- follow-up 1: function ----
# Answer: Dijkstra where the value of a path is the
# minimum edge so far and you maximise it: pop the best
# bottleneck, extend with min(value, edge). O(E log V);
# Union-Find is simpler when you need only the value.
def best_path_safety_heap(n, edges, s, t, threshold=None):
    # same answer with a max-heap (Dijkstra on the
    # bottleneck)
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        if threshold is None or w >= threshold:
            graph[u].append((v, w))
            graph[v].append((u, w))
    best = [-1] * n
    best[s] = float("inf")
    heap = [(-best[s], s)]
    while heap:
        neg, u = heapq.heappop(heap)
        if -neg < best[u]:
            continue
        for v, w in graph[u]:
            # extending a path can only lower its weakest
            # edge
            cand = min(-neg, w)
            if cand > best[v]:
                best[v] = cand
                heapq.heappush(heap, (-cand, v))
    return best[t]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
```

```
    assert best_path_safety_heap(4, edges, 0, 3) == 3
    # edges below the threshold are forbidden
    assert best_path_safety(4, edges, 0, 3, 4) == -1
    assert best_path_safety_heap(4, edges, 0, 3, 4) == -1
```

```
    # disconnected, and source equals target
    assert best_path_safety(3, [(0, 1, 9)], 0, 2) == -1
    assert best_path_safety(3, [], 1, 1) == float("inf")
    one = [(0, 1, 9)]
    assert best_path_safety_heap(3, one, 0, 2) == -1


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)
```

Test cases

Input	Expected
(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)	5
(3, [(0,1,7),(1,2,3)], 0, 2, 5)	-1

STUDY PATH: question 77 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 5 of 8

Template fit: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating | asked 376 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 77 of 288

Movie Recommendations via Transitive Similarity

ASKED 2025-10-09 | 376 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.4 | 30 DAYS 33 | PRIORITY 72/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: LC 721 Accounts Merge (related: transitive groups)

Frequency: Single report | Last asked: 2025-10-09 | Roles: SWE Intern

QUESTION

You are given a target movie name, a dictionary ratings mapping each movie to its numeric rating, an integer n , and a list similarities of [movieX, movieY] pairs meaning the two movies are directly similar. Similarity is transitive: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group. Return up to n recommended movies from the target's similarity group, excluding the target itself, ordered by rating descending (break ties alphabetically by name). If n == 1 , this returns the single highest- rated similar movie. If the group has fewer than n other movies, return all of them.

Function signature

```
def recommend(target: str, ratings: dict[str, float], n: int,
              similarities: list[list[str]]) -> list[str]:
    ...
```

EXAMPLES

Example 1

Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C"]`
Why	`A, B, C form one group. Excluding A, the highest rated is C (9).`

QUESTION

Example 2

Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C", "B"]`
Why	`Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].`

Example 3

Input	`target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3, similarities = [["A", "B"], ["B", "C"]]`
Output	`[]`
Why	`D is in no similarity pair, so its group is just {D}; excluding D leaves nothing.`

CONSTRAINTS

- Every movie mentioned appears as a key in ratings .
- 1 <= n ; ratings are numbers, names are distinct strings.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_recommend(target, ratings, n, pairs):
    group, grew = {target}, True
    while grew:
        grew = False
        for x, y in pairs:
            if (x in group) != (y in group):
                group.update((x, y))
                grew = True
    group.discard(target)
    return sorted(
        group, key=lambda m: (-ratings[m], m)
    )[:n]
```

TIME	SPACE
O(P * rounds)	O(P)

WHY IT IS SLOW

Every round rescans all pairs until the group stops growing.

OPTIMAL SOLUTION · Union-find and a sort

TIME	SPACE
O(P alpha + g log g)	O(P)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Thousands of recommendation queries on the same data.

Build the groups once with union-find, sort each group by (-rating, name) once, and store the group id of every movie. A query looks up the group, takes the first n + 1 entries and removes the target: O(n) per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Movie Recommendations via Transitive
# Similarity
#
# You are given a target movie name, a dictionary ratings
# mapping each movie to its numeric rating, an integer n ,
# and a list similarities of [movieX, movieY] pairs
# meaning the two movies are directly similar. Similarity
# is transitive: if A is similar to B and B is similar to
# C, then A, B, and C all belong to the same similarity
# group. Return up to n recommended movies from the
# target's similarity group, excluding the target itself,
# ordered by rating descending (break ties alphabetically
# by name). If n == 1 , this returns the single highest-
# rated similar movie. If the group has fewer than n other
# movies, return all of them. Function signature: def
# recommend(target: str, ratings: dict[str, float], n:
# int, similarities: list[list[str]]) -> list[str]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of recommendation queries on the same data.
#
# INTERVIEW NOTES
# Transitive similarity means connected components: movies
# that can reach each other through the pairs form one
# group. Find the target's component (union-find, or a BFS
# from the target), drop the target, sort the rest by
# rating descending then name, and return the first n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Union-find and a sort
```

```
# Idea: The target's component is its group; rank it by
# rating.
def recommend(target, ratings, n, similarities):
    parent = {}
    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for x, y in similarities:
        parent[find(x)] = find(y)
    root = find(target)
    group = [
        m
        for m in list(parent)
        if m != target and find(m) == root
    ]
    group.sort(key=lambda m: (-ratings[m], m))
    return group[:n]

# ---- main question: tests ----
def test_main_question():
    random.seed(56)
    r = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}
    assert recommend(
        "A", r, 1, [["A", "B"], ["B", "C"]]
    ) == ["C"]
    assert recommend(
        "A", r, 2, [["A", "B"], ["B", "C"]]
    ) == ["C", "B"]
    assert (
        recommend("D", r, 3, [["A", "B"], ["B", "C"]])
        == []
    )
    names = list("ABCDEFG")


# ============================================================
# FOLLOW-UP 1: Thousands of recommendation queries on the
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Build the groups once with union-find, sort each
# group by (-rating, name) once, and store the group id of
# every movie. A query looks up the group, takes the first
# n + 1 entries and removes the target: O(n) per query.
class Recommender:
    def __init__(self, ratings, similarities):
        parent = {}

        def find(x):
            parent.setdefault(x, x)
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for x, y in similarities:
            parent[find(x)] = find(y)
        groups = defaultdict(list)
        for m in list(parent):
            groups[find(m)].append(m)
        self.group_of = {m: find(m) for m in parent}
        self.groups = {
            r: sorted(
                ms, key=lambda m: (-ratings[m], m)
            )
            for r, ms in groups.items()
        }

    def recommend(self, target, n):
        if target not in self.group_of:
            return []
```

```
        ranked = self.groups[self.group_of[target]]
        return [m for m in ranked if m != target][:n]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 78 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 6 of 8

Template fit: pairs = peer links; keep the boss at the root and carry it over when two groups merge | asked 240 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 78 of 288

Determine Reporting Relationships

POSTED 2026-02-22 | 240 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.4 | 30 DAYS 33 | PRIORITY 73/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: No direct match; related LC 721 Accounts Merge (union-find)

Posted on PracHub: 2026-02-22 (updated 2026-04-21)

Original: https://prachub.com/coding-questions/determine-reporting-relationships

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.

QUESTION

- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints
0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3),
    ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4),
    ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
managers, peers = [], []  # the stored facts
def is_manager(a, b):
    groups = merge_peers(peers)
    manager_of = {}
    for x, y in managers:  # x manages y
        boss = groups.find(x)
        manager_of[groups.find(y)] = boss
    node = groups.find(b)
    while node in manager_of:
        node = manager_of[node]
        if node == groups.find(a):
            return True
    return False
```

TIME	SPACE
O(F * employees) per query	O(F)

WHY IT IS SLOW

Each query redoes the whole inference over all facts. Union-Find for peer groups with a manager stored on each group keeps every update near-constant and a query O(depth).

OPTIMAL SOLUTION · Union-Find for peer groups with a manager stored on each group

TIME	SPACE
O(alpha) per update, O(depth) per query	**O(employees)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

FOLLOW-UP 2 Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

FOLLOW-UP 3 Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine Reporting Relationships
#
# Build an in-memory organization model that supports
# three operations on employee IDs: 1. add_manager(a, b):
# employee a is the direct manager of employee b. 2.
# add_peer(a, b): employees a and b share the same direct
# manager. 3. is_manager(a, b): return whether employee a
# is a manager of employee b. The answer should account
# for relationships that can be inferred from prior
# operations. For example, if add_manager(m, x) and
# add_peer(x, y) were previously recorded, then
# is_manager(m, y) should return true. You may assume the
# input is logically consistent and does not create
# invalid cycles in the org chart. Explain how you would
# design the data structures and implement these
# operations efficiently.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the closest common manager of two employees.
# 2. Facts can be wrong or contradictory.
# 3. Remove a relationship.
#
# INTERVIEW NOTES
# Peers form groups (a peer relation is transitive: same
# manager as a person who has the same manager as
# another). A group has at most one direct manager, so
# store the manager on the group. To answer is_manager(a,
# b) go from b's group to its manager, then that manager's
# group to its manager, and so on, looking for a. Union-
# find makes the groups cheap to merge.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a.
# FOLLOW-UP 1: Return the closest common manager of two
# employees.
# Answer: Collect the chain of managers above one employee
# into a set, then climb from the other until you meet it.
# O(depth).
class Org:
    def __init__(self):
        # union-find over peers (people with the same
        # manager)
        self.parent = {}
        # peer-group root -> the manager of that group
        self.boss = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # two groups with different managers contradict
        if ra in self.boss and rb in self.boss \
                and self.boss[ra] != self.boss[rb]:
            # FOLLOW-UP 2: contradictory facts are
            # rejected
            raise ValueError("conflicting managers")
        self.parent[rb] = ra
        # the merged group keeps whichever manager is
        # known
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        # a is the direct manager of b (and b's peers)
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")
        self.boss[root] = a
```

```
    def manager_of(self, x):
        return self.boss.get(self.find(x))

    def is_manager(self, a, b):
        # climb b's chain of managers; a visited set stops
        # cycles
        seen, cur = set(), b
        while cur is not None and cur not in seen:
            seen.add(cur)
            cur = self.manager_of(cur)
            if cur == a:
                return True
        return False

    # FOLLOW-UP 1: the closest common manager
    def common_manager(self, a, b):
        # the closest manager above both people
        chain, cur = set(), self.manager_of(a)
        while cur is not None and cur not in chain:
            chain.add(cur)
            cur = self.manager_of(cur)
        cur, seen = self.manager_of(b), set()
        while cur is not None and cur not in seen:
            if cur in chain:
                return cur
            seen.add(cur)
            cur = self.manager_of(cur)
        return None

# ---- main question: tests ----
def test_main_question():
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    # qa shares dev's manager, so vp manages qa too
    assert o.is_manager("vp", "qa")
    assert o.is_manager("ceo", "qa")
    assert not o.is_manager("qa", "vp")
    assert not o.is_manager("dev", "qa")
    assert o.common_manager("dev", "qa") == "vp"
```

```
    assert o.common_manager("vp", "qa") == "ceo"
    # peers added before the manager is known
    p = Org()
    p.add_peer("a", "b")
    p.add_manager("boss", "a")
    assert p.is_manager("boss", "b")
    # contradictory facts are reported
    try:
        o.add_manager("ceo", "dev")
        raise AssertionError("should conflict")
    except ValueError:
        pass


# ============================================================
# FOLLOW-UP 1: Return the closest common manager of two
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[tuple[str, int, int]]) -> li
st[bool]:
SAMPLE INPUT
([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager"
, 1, 3)])
```

Test cases

Input	Expected
`([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])`	`[True]`
`([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])`	`[False, True, True, False]`

STUDY PATH: question 79 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 7 of 8

Template fit: pairs = the given edges; the query is find(a) == find(b) | asked 271 days ago

MEDIUM | PracHub | Technical Screen | 79 of 288

Determine whether two nodes are related

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 24.5 | 30 DAYS 0 | PRIORITY 163/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: the interview site's own solution

LeetCode match: LC 1971 Find if Path Exists in Graph (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/determine-whether-two-nodes-are-related

QUESTION

Nodes 0 to n - 1 are linked by edges, and two nodes are related when a chain of edges joins them (links work in both directions; the graph may be disconnected and may contain cycles). Support union(a, b) to add a link and related(a, b) to answer the query.

Function signature

class Relations:

def __init__(self, n: int, edges=())

def union(self, a: int, b: int) -> None

def related(self, a: int, b: int) -> bool

EXAMPLES

Example 1

Input	`r = Relations(5, [(0, 1), (1, 2)])`
	`r.related(0, 2)`
	`r.related(0, 3)`
	`r.union(3, 4)`
	`r.related(3, 4)`
	`r.related(2, 4)`

Output	`r.related(0, 2) -> true`
	`r.related(0, 3) -> false`
	`r.related(3, 4) -> true`
	`r.related(2, 4) -> false`

Why	`0, 1, 2 form one group and 3, 4 another until they are joined; the two groups are never linked to each other.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def related(a, b):
    graph = build_adjacency(edges)
    seen = set()
    dfs(graph, a, seen)
    return b in seen
```

TIME	SPACE
O(V + E) per query	O(V)

WHY IT IS SLOW

Every query repeats the same traversal, O(V + E) each. A Union-Find built once answers each query in near-constant time.

OPTIMAL SOLUTION · Union-Find built once, constant-time queries

TIME	SPACE
O(E alpha(V)) to build, O(alpha(V)) per query	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 'Related' means ancestor or descendant, or sharing an ancestor.

Store the data as a parent map and walk up from one node while recording ancestors, then walk up from the other until you hit one. Keep a seen set so a corrupt cycle cannot loop forever.

FOLLOW-UP 2 Relationships arrive over time and you must answer queries between updates.

Union-Find handles incremental unions in near-constant time. If relationships can also be removed, you need offline processing or a dynamic-connectivity structure.

FOLLOW-UP 3 The graph is directed and 'related' means reachability from a to b.

Union-Find no longer works (it ignores direction). Use DFS/BFS per query, or compute strongly connected components and reachability on the condensed DAG for many queries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine whether two nodes are related
#
# Nodes 0 to n - 1 are linked by edges, and two nodes are
# related when a chain of edges joins them (links work in
# both directions; the graph may be disconnected and may
# contain cycles). Support union(a, b) to add a link and
# related(a, b) to answer the query.
#
# Signature:
#   class Relations:
#       def __init__(self, n: int, edges=())
#       def union(self, a: int, b: int) -> None
#       def related(self, a: int, b: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. 'Related' means ancestor or descendant, or sharing an
# ancestor.
# 2. Relationships arrive over time and you must answer
# queries between updates.
# 3. The graph is directed and 'related' means
# reachability from a to b.
#
# INTERVIEW NOTES
# Ask what 'related' means: connected in any direction (a
# connectivity question: union-find or one traversal), or
# ancestor/descendant, or share an ancestor. With many
# queries, preprocess once: components give O(1) answers.
# Always make the visited set part of the design, since
# relationship data can have cycles.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Union-Find built once,
# constant-time queries
# Idea: Union the endpoints of every relationship; two
# nodes are related exactly when they have the same root.
class Relations:
    """Related = joined by a chain of relations."""

    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant
        # time
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # the smaller group joins the bigger one
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)


# HELPER: ancestors of a node
def ancestors(parent, x):
    # x and everything above it; a seen set stops cycles
    seen = []
    while x is not None and x not in seen:
        seen.append(x)
        x = parent.get(x)
    return seen

# ---- main question: tests ----
```

```
def test_main_question():
    r = Relations(5, [(0, 1), (1, 2)])
    assert r.related(0, 2) and not r.related(0, 3)
    r.union(3, 4)
    assert r.related(3, 4) and not r.related(2, 4)
    r.union(2, 3)
    assert r.related(0, 4)
    assert r.related(1, 1)
    # a family tree: parent[child] = parent
    parent = {"kid": "mom", "mom": "gran",
              "uncle": "gran", "stranger": None}


# ============================================================
# FOLLOW-UP 1: 'Related' means ancestor or descendant, or
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the data as a parent map and walk up from
# one node while recording ancestors, then walk up from
# the other until you hit one. Keep a seen set so a
# corrupt cycle cannot loop forever.
def kin(parent, a, b):
    # related if one is an ancestor of the other, or they
    # share an ancestor
    up_a = ancestors(parent, a)
    up_b = ancestors(parent, b)
    return bool(set(up_a) & set(up_b))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # a family tree: parent[child] = parent
    parent = {"kid": "mom", "mom": "gran",
              "uncle": "gran", "stranger": None}
    assert kin(parent, "kid", "gran")
    assert kin(parent, "kid", "uncle")
    assert not kin(parent, "kid", "stranger")
    # a corrupt cycle cannot loop forever
    assert kin({"a": "b", "b": "a"}, "a", "b")
    # direction matters for reachability
    g = {"a": ["b"], "b": ["c"]}
```

```
# ============================================================
# FOLLOW-UP 2
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: The graph is directed and 'related' means
# ============================================================

# ---- follow-up 3: function ----
# Answer: Union-Find no longer works (it ignores
# direction). Use DFS/BFS per query, or compute strongly
# connected components and reachability on the condensed
# DAG for many queries.
def reachable(graph, a, b):
    # directed: can we get from a to b? (Union-Find cannot
    # tell, it ignores direction)
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in graph.get(u, ()):
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # direction matters for reachability
    g = {"a": ["b"], "b": ["c"]}
    assert reachable(g, "a", "c")
    assert not reachable(g, "c", "a")


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(relations: list[list[int]], a: int, b: int) -
> bool:
```

Sample call

```
solution([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)
```

Test cases

Input	Expected
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)`	`True`
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 11)`	`False`

STUDY PATH: question 80 of 288 | topic 9 of 43: Union-find: merge the pairs, then ask who is together | 8 of 8

Template fit: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1 | asked 21 days ago | frequency Medium

HARD | 1Point3Acres | onsite-coding, phone-screen | 80 of 288

Friend Graph with Timeline (Friend / Unfriend Events)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 246/288

ABOUT THIS QUESTION

Topic: Union-find: merge the pairs, then ask who is together

Optimal answer from: written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

QUESTION

There are n people numbered 0 to n - 1. Each event is (time, a, b, kind). A 'friend' event makes a and b friends from that time on, and two people are connected when a chain of friends links them. Return the earliest time at which all n people are connected, or -1 if that never happens.

Function signature

def earliest_all_connected(n: int, events: list) -> int

EXAMPLES

Example 1	
Input	`3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]`
Output	`5`
Why	`After time 1, {0, 1} are connected; at time 5 person 2 joins, so everyone is connected at 5.`

Example 2	
Input	`4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]`
Output	`-1`
Why	`Person 3 is never befriended, so the group never becomes one.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, events):
    for t in sorted({e.time for e in events}):
        graph = build_graph(events, upto=t)
        if dfs_count(graph, start=0) == n:
            return t  # everyone reached
    return -1
```

BRUTE FORCE

TIME	SPACE
O(T * (N + E)) for T distinct times	O(N + E)

WHY IT IS SLOW

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

OPTIMAL SOLUTION · Base: earliest time everyone is connected (Union-Find on sorted events)

TIME	SPACE
O(E log E)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

FOLLOW-UP 2 What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

FOLLOW-UP 3 Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

FOLLOW-UP 4 Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Friend Graph with Timeline (Friend /
# Unfriend Events)
#
# There are n people numbered 0 to n - 1. Each event is
# (time, a, b, kind). A 'friend' event makes a and b
# friends from that time on, and two people are connected
# when a chain of friends links them. Return the earliest
# time at which all n people are connected, or -1 if that
# never happens.
#
# Signature:
#   def earliest_all_connected(n: int, events: list) -> in
t
#
# Examples:
# 3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]  ->  5
# 4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does unfriending break plain Union-Find?
# 2. What is the simple answer if the interviewer wants
# something quick?
# 3. Report how many friend groups exist after each event
# (friends only).
# 4. Queries ask 'were A and B connected at time t?' with
# no unfriending.
#
# INTERVIEW NOTES
# Everyone becomes connected at the first moment the graph
# has one component. If friendships only appear, sort by
# time and use union-find (components only merge).
# Unfriending breaks that: removing an edge can split a
# component and union-find cannot undo. With all events
# known up front, handle each edge's lifetime in a segment
# tree over time and use a rollback union-find.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict, deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])


# FOLLOW-UP 4: Queries ask 'were A and B connected at time
# t?' with no unfriending.
# Answer: Union by size without path compression,
# recording the time of each union on the edge it creates;
# two people are connected at t if the largest time on the
# path between them is at most t.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t

# ---- main question: main function ----
# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted events)
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does.
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
```

```
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
```

```
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1

# ---- main question: tests ----
def test_main_question():
    # people A, B, C are 0, 1, 2
    ev = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    # everyone is connected at time 5
    assert earliest_all_connected(3, ev) == 5
    # a missing friendship means never connected
    assert earliest_all_connected(3, ev[:1]) == -1
    assert earliest_all_connected(1, []) == 0


# ============================================================
# FOLLOW-UP 1: Why does unfriending break plain Union-
# ============================================================

# ---- follow-up 1: function ----
# Answer: A union merges two sets and forgets which edge
# caused it, so removing that edge cannot tell which
# members should separate.
# FOLLOW-UP 2: What is the simple answer if the
# interviewer wants something quick?
# Answer: Rebuild the graph from the events up to time t
# and run BFS or DFS for each query: O(E + N) per query.
# State the cost and then explain how the offline method
# amortizes it.
def connected_with_unfriends(n, events, a, b, t):
    # replay every event up to time t on a set of edges
    edges = set()
    for time, x, y, kind in by_time(events):
        if time > t:
            break
        pair = (min(x, y), max(x, y))
        # friend adds the edge, unfriend removes it
        if kind == "friend":
            edges.add(pair)
        else:
```

```
            edges.discard(pair)
    # build neighbour lists from the surviving edges
    adj = defaultdict(list)
    for x, y in edges:
        adj[x].append(y)
        adj[y].append(x)
    # breadth-first search from a looking for b
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in adj[u]:
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False

# ============================================================
# FOLLOW-UP 2: What is the simple answer if the
# ============================================================
# Its code sits in the main question section above, becaus
```

```
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Report how many friend groups exist after
# ============================================================

# ---- follow-up 3: function ----
# Answer: Union-find with a component counter: decrement
# when a union merges two different roots; record the
# counter after each event. O(E alpha(N)).
def groups_after_each_event(n, events):
    uf = TimedUnionFind(n)
    counts = []
    # replay friend events in time order
    for t, a, b, kind in by_time(events):
        if kind == "friend":
            uf.union(a, b, t)
            # record the group count after this event
            counts.append(uf.groups)
    return counts

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # people A, B, C are 0, 1, 2
```

```
    ev = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    # groups: 3 -> 2 -> 1
    assert groups_after_each_event(3, ev) == [2, 1]
    # A and C are apart at time 0, together at time 5
    uf = TimedUnionFind(3)
    for t, a, b, k in ev:
        uf.union(a, b, t)
    assert not uf.connected_at(0, 2, 4)
    assert uf.connected_at(0, 2, 5)
    # an unfriend breaks the link again
    ev2 = ev + [(7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, ev2, 0, 2, 6)
    assert not connected_with_unfriends(3, ev2, 0, 2, 8)


# ============================================================
# FOLLOW-UP 4: Queries ask 'were A and B connected at time
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Have weighted/path-compressed union-find ready and warm. Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 10 OF 43

A dict of per-key state that you update, expire, or rank

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	7
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A dict of per-key state that you update, expire, or rank | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	81. Robot Status Message Deduplication Window	1Point3Acres	2026-09-10
M	82. Active Communication Top Users	1Point3Acres	2026-09-22
M	83. Filter Repeated Robot Status Messages	PracHub	2026-07-05
M	84. Next-Word Predictor	1Point3Acres	2026-09-02
M	85. Batch and Streaming Log Deduplication	1Point3Acres	2026-07-23
M	86. Design a logger that returns top-K messages	PracHub	2026-01-06
M	87. Hit Counter	HackTheRounds	no date
M	88. Website Activity Analytics: Unique Users + Session Time	1Point3Acres	2026-06-26

CODE TEMPLATE | GROUP 10 OF 43 | 8 QUESTIONS, ONE PATTERN

A dict of per-key state that you update, expire, or rank

The one idea these questions share

Keep a dict from a key (message, user, word, activity) to the little bit of state you need: the last time seen, a Counter, a set, a list of timestamps. Every event updates one key's state; the rule that decides what to accept, expire or report is the only part that changes.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class KeyedState:
    def __init__(self, window):
        self.window = window
        self.last = {}                          # CHANGE 1: what you keep per key

    def accept(self, t, key):
        if key in self.last and t - self.last[key] < self.window:   # CHANGE 2: the rule
            return False
        self.last[key] = t                      # CHANGE 3: update this key's state
        return True
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict, Counter, deque


def group_by(pairs):                        # key -> list of values, in arrival order
    out = defaultdict(list)
    for k, v in pairs:
        out[k].append(v)
    return out


def hits_in_window(times, now, span=300):
    q = deque(times)
    while q and q[0] <= now - span:
        q.popleft()
    return len(q)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
81	Robot Status Message Deduplication Window	the template as written: show a message only if it was not shown in the last 10 seconds
82	Active Communication Top Users	a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)
83	Filter Repeated Robot Status Messages	the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted
84	Next-Word Predictor	defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word
85	Batch and Streaming Log Deduplication	dict message -> first (or latest) timestamp depending on the policy; sort the result at the end
86	Design a logger that returns top-K messages	dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest
87	Hit Counter	one deque of hit times; drop the ones older than 300 seconds; the answer is the length

#	Question	What you change in the template
88	Website Activity Analytics: Unique Users + Session Time	dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 81 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 1 of 8

Template fit: the template as written: show a message only if it was not shown in the last 10 seconds | asked 40 days ago | frequency Low

EASY | 1Point3Acres | onsite-coding | 81 of 288

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 33** | **PRIORITY 41/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Status events arrive from a robot as (time, message). Show a message only if the same message was not shown during the previous 10 seconds; a hidden message does not restart the timer, and exactly 10 seconds later it is shown again. show(t, msg) returns True when the message should be displayed.

Function signature

class Dedup:
    def show(self, t: int, msg: str) -> bool

EXAMPLES

Example 1

Input	`d = Dedup()`
	`d.show(0, 'low battery')`
	`d.show(4, 'low battery')`
	`d.show(6, 'door open')`
	`d.show(9, 'low battery')`
	`d.show(10, 'low battery')`

Output	`d.show(0, 'low battery') -> true`
	`d.show(4, 'low battery') -> false`
	`d.show(6, 'door open') -> true`
	`d.show(9, 'low battery') -> false`
	`d.show(10, 'low battery') -> true`

Why	`The repeats at 4 and 9 are hidden and do not move the timer; at time 10 it is exactly 10 seconds after the last display, so it shows again.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

TIME	SPACE
O(displayed) per event	**O(displayed)**

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	**O(distinct messages)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Robot Status Message Deduplication Window
#
# Status events arrive from a robot as (time, message).
# Show a message only if the same message was not shown
# during the previous 10 seconds; a hidden message does
# not restart the timer, and exactly 10 seconds later it
# is shown again. show(t, msg) returns True when the
# message should be displayed.
#
# Signature:
#   class Dedup:
#       def show(self, t: int, msg: str) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events can arrive out of order.
#
# INTERVIEW NOTES
# The state that matters is the last time each message was
# displayed, not the last time it arrived. A message that
```

```
# is hidden must not move that time, otherwise a message
# repeated every few seconds would be hidden forever. A
# hash map from message to last displayed time answers
# each event in O(1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right, insort


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""
```

```
    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True

# ---- main question: tests ----
def test_main_question():
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")


# ============================================================
# FOLLOW-UP 1: Events can arrive out of order.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Decide the contract first: either a late event
# is judged against the stored time with abs(t - last), or
```

```
# you keep a sorted history per message and compare with
# the nearest displayed neighbours on both sides.
```

```
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
            return False
        insort(times, t)
        return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY PATH: question 82 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 2 of 8

Template fit: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1) | asked 28 days ago | frequency Low

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 82 of 288

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 288/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

A stream of messages like 'A-B' means users A and B talked. Each distinct pair counts once (A-B and B-A are the same pair) and a pair of a user with themselves is ignored. A user's count is how many different partners they have. Support add(pair), most_active() (the set of users with the highest count) and top_k(k) (k users by count, ties broken by name).

Function signature

class Active:
  def add(self, pair: str) -> None
  def most_active(self) -> set
  def top_k(self, k: int) -> list

EXAMPLES

Example 1

Input	`a = Active()`
	`a.add('A-B')`
	`a.add('B-C')`
	`a.add('B-A')`
	`a.most_active()`
	`a.top_k(2)`

Output	`a.most_active() -> {'B'}` `a.top_k(2) -> ['B', 'A']`
Why	`B talked to A and C (2 partners). The repeated pair B-A adds nothing. A and C have 1 each, so the top 2 are B, then A (ties by name).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Active Communication Top Users
#
# A stream of messages like 'A-B' means users A and B
# talked. Each distinct pair counts once (A-B and B-A are
# the same pair) and a pair of a user with themselves is
# ignored. A user's count is how many different partners
# they have. Support add(pair), most_active() (the set of
# users with the highest count) and top_k(k) (k users by
# count, ties broken by name).
#
# Signature:
#   class Active:
#       def add(self, pair: str) -> None
#       def most_active(self) -> set
#       def top_k(self, k: int) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Self communication (A-A) should now count.
# 2. Top-K with a changing K, or a fixed-size priority
# queue.
# 3. The space claim.
#
# INTERVIEW NOTES
# Normalize the pair first (sorted tuple) so A-B and B-A
# are one edge, then keep an edge set for dedup and a
# count per user. The ranking part is a separate question:
# top-K comes from a heap, and the single most active user
# can be maintained in O(1) because counts only ever go
# up. Mention that the edge set uses quadratic space in
# the worst case.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter, defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
# FOLLOW-UP 1: Self communication (A-A) should now count.
# Answer: Pass a flag to the class: a self pair is one
# edge that adds one to that user's count (and only once).
# The dedup set already handles repeated self pairs.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)
```

```
    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)

# ---- main question: tests ----
def test_main_question():
    a = Active()
    for x, y in [("A", "B"), ("B", "C"), ("B", "A")]:
        a.add(x, y)
    # B talks to A and C; the repeat B-A is ignored
    assert a.most_active() == {"B"}
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    # self pairs are ignored by default
    a.add("C", "C")
    assert a.count["C"] == 1
    # ... but can be switched on, counting once
    s = Active(count_self=True)
    s.add("C", "C")
    s.add("C", "C")
    assert s.count["C"] == 1
    assert Active().most_active() == set()


# ============================================================
# FOLLOW-UP 1: Self communication (A-A) should now count.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

STUDY PATH: question 83 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 3 of 8

Template fit: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted | asked 107 days ago

MEDIUM | PracHub | Technical Screen | 83 of 288

★ TOP 75: rank 51 in the Medium and Hard priority list

★ Filter Repeated Robot Status Messages

POSTED 2026-07-05 | 107 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 33 | PRIORITY 39/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (inferred from overview)

Posted on PracHub: 2026-07-05 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/filter-repeated-robot-status-messages

QUESTION

A robot status stream is a list of (timestamp, message). Show a message when it has never been shown, or when strictly more than gap seconds (default 10) have passed since it was last shown. Messages are compared exactly (case-sensitive), a hidden repeat does not restart the interval, and the shown messages keep their input order. Return the shown (timestamp, message) pairs.

Function signature

def filter_status(stream: list, gap: int = 10) -> list

EXAMPLES

Example 1

Input	`[(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11, 'OK')]`
Output	`[(0, 'ok'), (11, 'ok'), (11, 'OK')]`
Why	`'ok' at 5 and 10 are hidden (not more than 10 seconds after 0); at 11 it is 11 seconds later, so it shows; 'OK' is a different message.`

Example 2

Input	`[(0, 'low'), (3, 'low'), (7, 'low')], gap = 2`
Output	`[(0, 'low'), (3, 'low'), (7, 'low')]`
Why	`A smaller gap lets repeats through sooner.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message)
def on_message(t, msg):
    last = None
    for t0, m0 in shown:
        if m0 == msg:
            last = t0  # most recent display
    if last is None or t - last > 10:
        shown.append((t, msg))
        return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each message searches the whole history, O(n^2). A dictionary from message to its last displayed time makes each decision O(1).

OPTIMAL SOLUTION · Dictionary from message to the time it was last shown

TIME	SPACE
O(n)	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

FOLLOW-UP 2 Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

FOLLOW-UP 3 The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Filter Repeated Robot Status Messages
#
# A robot status stream is a list of (timestamp, message).
# Show a message when it has never been shown, or when
# strictly more than gap seconds (default 10) have passed
# since it was last shown. Messages are compared exactly
# (case-sensitive), a hidden repeat does not restart the
# interval, and the shown messages keep their input order.
# Return the shown (timestamp, message) pairs.
#
# Signature:
#   def filter_status(stream: list, gap: int = 10) -> list
#
# Examples:
# [(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11,
#     'OK')]  ->  [(0, 'ok'), (11, 'ok'), (11, 'OK')]
# [(0, 'low'), (3, 'low'), (7, 'low')], gap = 2  ->  [(0,
#     'low'), (3, 'low'), (7, 'low')]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream never ends; memory must be bounded.
# 2. Several robots send messages.
# 3. The interval depends on the message severity.
#
# INTERVIEW NOTES
# The decision for a message depends only on when that
# exact message was last displayed, so a dictionary from
# message to last display time is all the state you need.
# The detail that decides correctness: the stored time
# changes only when a message is shown (a hidden repeat
# must not reset the clock), and the comparison is strict
# (> 10).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last > 10, emit it and set last = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream, gap=10):
    # show a message only if MORE than `gap` seconds
    # passed
    # since it was last shown; hidden repeats change
    # nothing
    last, shown = {}, []
    for ts, msg in stream:
        if msg not in last or ts - last[msg] > gap:
            last[msg] = ts
            shown.append((ts, msg))
    return shown

# ---- main question: tests ----
def test_main_question():
    stream = [(0, "a"), (5, "a"), (10, "a"), (11, "a"),
              (12, "b"), (22, "a")]
    # 10 is hidden (not MORE than 10 after 0); 11 shows
    assert filter_status(stream) == [(0, "a"), (11, "a"),
                                     (12, "b"), (22, "a")]
    # messages are case sensitive
    assert filter_status([(0, "A"), (1, "a")]) == [
        (0, "A"), (1, "a")]


# ============================================================
# FOLLOW-UP 1: The stream never ends; memory must be
# ============================================================

# ---- follow-up 1: function ----
# Answer: A message older than the gap can never be
# blocked again, so evict it. Keep a min-heap of (time
# shown, message) and pop entries older than the gap
# before each decision; delete the dictionary key only if
# it still points at that time.
class BoundedFilter:
    """An endless stream: forget messages that can no
```

```
    longer be blocked, so memory stays small."""

    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}
        # (time shown, message), oldest first
        self.heap = []

    def show(self, ts, msg):
        # drop entries older than the gap
        while self.heap:
            if ts - self.heap[0][0] <= self.gap:
                break
            old_ts, old_msg = heapq.heappop(self.heap)
            # delete only if it was not shown again since
            if self.last.get(old_msg) == old_ts:
                del self.last[old_msg]
        if msg in self.last:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    stream = [(0, "a"), (5, "a"), (10, "a"), (11, "a"),
              (12, "b"), (22, "a")]
    b = BoundedFilter()
    shows = [b.show(ts, msg) for ts, msg in stream]
    assert shows == [True, False, False, True, True, True]
    assert len(b.last) <= 2


# ============================================================
# FOLLOW-UP 2: Several robots send messages.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use (robot id, message) as the key, or one
# filter per robot. The same logic applies per key.
class PerRobotFilter:
    """Several robots; the gap depends on severity."""

    def __init__(self, gaps, default=10):
        self.gaps, self.default = gaps, default
```

```
        self.last = {}

    def show(self, robot, ts, msg, severity=None):
        key = (robot, msg)
        gap = self.gaps.get(severity, self.default)
        last = self.last.get(key)
```

```
        if last is not None and ts - last <= gap:
            return False
        self.last[key] = ts
        return True

# ---- follow-up 2: tests ----
def test_follow_up_2():
    r = PerRobotFilter({"high": 2})
    assert r.show("r1", 0, "x")
    assert not r.show("r1", 9, "x")
    # another robot, and a stricter severity
    assert r.show("r2", 1, "x")
    assert r.show("r1", 1, "y", "high")
    assert r.show("r1", 4, "y", "high")


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def filter_messages(events: list[tuple[int, str]]) -> list
[tuple[int, str]]:
```

Sample call

```
filter_messages([])
```

Test cases

Input	Expected
`([],)`	`[]`
`([[10, "solar panel activated"], [13, "solar panel activated"], [20, "solar panel activated"], [21, "solar panel activated"]],)`	`[[10, "solar panel activated"], [21, "solar panel activated"]]`

ALSO REPORTED · 1 more

Robot Status Message Deduplication Window | 1Point3Acres | onsite-coding | asked 2026-09-10 | Low

https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

ALSO REPORTED · 1 more

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.
For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

Preparation: Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand. Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY PATH: question 84 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 4 of 8

Template fit: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word | asked 48 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 84 of 288

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 284/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Train on sentences, then predict the word that most often follows a given word. train(sentence) learns every pair of neighbouring words; predict(word) returns the word that followed it most often (ties go to the alphabetically smaller word) or None if nothing ever followed it. predict must run in O(1).

Function signature

class Predictor:
    def train(self, sentence: str) -> None
    def predict(self, word: str) -> str | None

EXAMPLES

Example 1

Input	`p = Predictor()`
	`p.train('I love apples')`
	`p.train('I love pears')`
	`p.train('you love pears too')`
	`p.predict('love')`
	`p.predict('I')`
	`p.predict('too')`

Output	`p.predict('love') -> 'pears'`
	`p.predict('I') -> 'love'`
	`p.predict('too') -> null`

Why	`After 'love' came 'apples' once and 'pears' twice, so 'pears' wins. Nothing ever followed 'too'.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	**O(corpus size)**

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	**O(number of distinct pairs)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Next-Word Predictor
#
# Train on sentences, then predict the word that most
# often follows a given word. train(sentence) learns every
# pair of neighbouring words; predict(word) returns the
# word that followed it most often (ties go to the
# alphabetically smaller word) or None if nothing ever
# followed it. predict must run in O(1).
#
# Signature:
#   class Predictor:
#       def train(self, sentence: str) -> None
#       def predict(self, word: str) -> str | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top K predictions.
# 2. Use two words of context (trigrams).
# 3. The corpus keeps changing and old text should fade.
#
# INTERVIEW NOTES
# Prediction in O(1) means the answer must already be
# sitting in a table when the query arrives, so do the
# work while training. Keep counts per (word, next word)
# and, for each word, the current best successor. Each
# count only increases by one, so the best can be updated
# by comparing just the one count that changed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter, defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Counts plus a maintained best
# successor

# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
# FOLLOW-UP 1: Return the top K predictions.
# Answer: Keep the Counter per word and use
# heapq.nsmallest on (-count, word): O(d log K) per query,
# or keep a sorted structure per word if queries dominate.
# FOLLOW-UP 2: Use two words of context (trigrams).
# Answer: The key becomes the pair (previous two words)
# and everything else is identical, at the price of more
# memory. Back off to the bigram table when the pair was
# never seen.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None

    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))

# ---- main question: tests ----
def test_main_question():
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
    q.train("a b")
    assert q.predict("a") == "b"


# ============================================================
# FOLLOW-UP 1: Return the top K predictions.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
```

```
-UP 1.

# ============================================================
# FOLLOW-UP 2: Use two words of context (trigrams).
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.
```

```


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

STUDY PATH: question 85 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 5 of 8

Template fit: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end | asked 89 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 85 of 288

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 89 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 286/288**

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Given log entries (timestamp, message), keep one entry per message and return the kept entries sorted by timestamp. With policy 'first' keep the entry seen first for each message; with policy 'latest' keep the one with the latest timestamp.

Function signature

def dedupe(logs: list, policy: str = 'first') -> list

EXAMPLES

Example 1	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]`
Output	`[(1, 'b'), (3, 'c'), (5, 'a')]`
Why	`Messages 'a' and 'b' each appear twice; the first one seen is kept.`

Example 2	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')], policy = 'latest'`
Output	`[(2, 'b'), (3, 'c'), (9, 'a')]`
Why	`With 'latest', the later timestamp of each repeated message wins.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Batch and Streaming Log Deduplication
#
# Given log entries (timestamp, message), keep one entry
# per message and return the kept entries sorted by
# timestamp. With policy 'first' keep the entry seen first
# for each message; with policy 'latest' keep the one with
# the latest timestamp.
#
# Signature:
#   def dedupe(logs: list, policy: str = 'first') -> list
#
# Examples:
# [(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]  ->
#     [(1, 'b'), (3, 'c'), (5, 'a')]
# [(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')],
#     policy = 'latest'  ->  [(2, 'b'), (3, 'c'), (9,
#     'a')]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Late data: how do you decide when it is safe to emit?
# 2. Memory must stay bounded.
# 3. Keep the entry with the most occurrences instead.
# 4. Only release entries once they are safe (late data
# may still arrive).
#
# INTERVIEW NOTES
# Dedup is a dictionary keyed by the message. Sorting is
# only for the output order. For a stream, a heap gives
# the smallest timestamp quickly, but when an entry is
# replaced you cannot delete from the middle of a heap, so
# leave the stale entry and skip it when it surfaces (lazy
# deletion).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
```

```
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())

# ---- main question: tests ----
def test_main_question():
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
```

```
# FOLLOW-UP 4: Only release entries once they are safe
# ============================================================

# ---- follow-up 4: function ----
# Answer: Add a watermark: release only entries whose
# timestamp is at most (the largest timestamp seen minus
# the allowed lateness), in order; anything older than the
# watermark that arrives later goes to a late-data path.
class StreamDedup:
    """add(ts, msg) and get_next() with a lazy heap."""

    def __init__(self, policy="first", lateness=0):
        self.policy, self.lateness = policy, lateness
        # message -> the timestamp currently kept
        self.kept = {}
        # (timestamp, message), may hold outdated entries
        self.heap = []
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
```

```
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        wm = self.watermark()
        # too late: older than the watermark, drop it
        if wm is not None and ts < wm:
            return False
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (
                self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        wm = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            # lazy deletion: skip entries that were
            # replaced
```

```
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            # not safe yet: later data may still arrive
            if ts > wm:
                return None
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None

# ---- follow-up 4: tests ----
def test_follow_up_4():
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    # watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (1, "a")
    assert s.get_next() == (3, "c")
    # 5 is still above the watermark
    assert s.get_next() is None
    # data older than the watermark is dropped
    assert not s.add(2, "late")
    # a "latest" policy replaces the kept entry
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")
    assert t.get_next() is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

STUDY PATH: question 86 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 6 of 8

Template fit: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest | asked 287 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 86 of 288

★ TOP 75: rank 36 in the Medium and Hard priority list

★ Design a logger that returns top-K messages

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 33 | PRIORITY 40/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter, LeetCode #692 Top K Frequent Words

LeetCode match: LC 359 Logger Rate Limiter + LC 692 Top K Frequent Words (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/design-a-logger-that-returns-top-k-messages

QUESTION

Design a logger. log(ts, msg) records a message at time ts (timestamps arrive in non-decreasing order). top_k(k, start, end) returns the k most frequent messages whose timestamps fall in [start, end]; leave start and end out to use the whole history. Ties are broken alphabetically.

Function signature

class Logger:

def log(self, ts: int, msg: str) -> None

def top_k(self, k: int, start=None, end=None) -> list

EXAMPLES

Example 1	

Input	`lg = Logger()`
	`lg.log(1, 'disk full')`
	`lg.log(2, 'timeout')`
	`lg.log(3, 'timeout')`
	`lg.log(4, 'disk full')`
	`lg.log(5, 'timeout')`
	`lg.top_k(1)`
	`lg.top_k(2, 4, 5)`
	`lg.top_k(2, 1, 1)`

Output	`lg.top_k(1) -> ['timeout']`
	`lg.top_k(2, 4, 5) -> ['disk full', 'timeout']`
	`lg.top_k(2, 1, 1) -> ['disk full']`

Why	`Overall 'timeout' occurred 3 times. In the window [4, 5] 'disk full' and 'timeout' occur once each (tie, alphabetical). The window [1, 1] only holds 'disk full'.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
logs = []  # (timestamp, message)
def top_k(k, start, end):
    counts = Counter()
    for ts, msg in logs:
        if start <= ts <= end:
            counts[msg] += 1
    def rank(m):
        return (-counts[m], m)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d) per query	O(n)

WHY IT IS SLOW

Every query reads the whole log, O(n + d log d). Per-message sorted timestamps with binary search answer a window count in O(log n), and a heap picks the top k.

OPTIMAL SOLUTION · Per-message sorted timestamps plus a heap for the top K

TIME	SPACE
O(1) per log, O(d log n + d log k) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

FOLLOW-UP 2 Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. **Cost:** O(1) amortized to drop expired timestamps; memory O(messages inside the window).

FOLLOW-UP 3 Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design a logger that returns top-K
# messages
#
# Design a logger. log(ts, msg) records a message at time
# ts (timestamps arrive in non-decreasing order). top_k(k,
# start, end) returns the k most frequent messages whose
# timestamps fall in [start, end]; leave start and end out
# to use the whole history. Ties are broken
# alphabetically.
#
# Signature:
#   class Logger:
#       def log(self, ts: int, msg: str) -> None
#       def top_k(self, k: int, start=None, end=None) -> l
ist
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print a message only if it was not printed in the
# last 10 seconds (LeetCode 359).
# 2. Memory must stay bounded.
# 3. Very high top_k frequency of calls.
#
# INTERVIEW NOTES
# Store enough to answer both questions without scanning
# everything. A dict from message to its list of
# timestamps (appended in time order) answers 'how many in
# [start, end]' with two binary searches, and a heap of
# size K over (count, message) gives the top K. Say how
# the logger handles repeated messages and what 'top'
# means with ties.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from bisect import bisect_left, bisect_right
from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start).
class Logger:
    def __init__(self):
        # message -> its timestamps, in order of arrival
        self.times = defaultdict(list)

    def log(self, ts, msg):
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        # the k most frequent messages in [start, end]
        counts = {}
        for msg, ts in self.times.items():
            lo, hi = 0, len(ts)
            if start is not None:
                lo = bisect_left(ts, start)
            if end is not None:
                hi = bisect_right(ts, end)
            if hi > lo:
                counts[msg] = hi - lo
        # ties are broken alphabetically
        def rank(m):
            return (-counts[m], m)
        return heapq.nsmallest(k, counts, key=rank)

    def expire(self, before):
        # bounded memory: forget timestamps older than
        # `before`
        for msg in list(self.times):
            ts = self.times[msg]
            self.times[msg] = ts[bisect_left(ts, before):]
            if not self.times[msg]:
                del self.times[msg]

# ---- main question: tests ----
def test_main_question():
    log = Logger()
    events = [(1, "a"), (2, "b"), (3, "a"),
              (4, "c"), (5, "a"), (6, "b")]
    for ts, msg in events:
        log.log(ts, msg)
    assert log.top_k(2) == ["a", "b"]
    # only the window [4, 6]: a once, b once, c once
    assert log.top_k(3, 4, 6) == ["a", "b", "c"]
    assert log.top_k(2, 100, 200) == []
    log.expire(4)
    assert log.top_k(5) == ["a", "b", "c"]


# ============================================================
# FOLLOW-UP 1: Print a message only if it was not printed
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a dict message to the time it was last
# printed and compare: print if the gap is at least 10
# (and update the time only when you print). O(1) per
# call.
class RateLimiter:
    """LeetCode 359: one print per message per gap."""

    def __init__(self, gap=10):
        self.gap, self.last = gap, {}

    def should_print(self, ts, msg):
        last = self.last.get(msg)
        if last is not None and ts - last < self.gap:
            return False
        # only a printed message starts a new window
        self.last[msg] = ts
        return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    r = RateLimiter()
    times = (1, 2, 11, 12, 21)
    got = [r.should_print(t, "x") for t in times]
    assert got == [True, False, True, False, True]
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(logs: list[tuple[int, str]], queries: list[li
st[int]]) -> list[list[str]]:
```

Sample call

```
solution([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3
, 2), (1, 10, 3)])
```

Test cases

Input	Expected
`([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])`	`[['A', 'B'], ['A', 'B', 'C']]`
`([(1, 'dog'), (2, 'cat'), (2, 'dog'), (4, 'cat'), (5, 'ant')], [(1, 4, 2)])`	`[['cat', 'dog']]`

STUDY PATH: question 87 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 7 of 8

Template fit: one deque of hit times; drop the ones older than 300 seconds; the answer is the length | no date

MEDIUM | IN APP LC 362 | DESIGN-STYLE (skippable) | HackTheRounds | Phone Screen | 87 of 288

Hit Counter

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 132/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #362 Design Hit Counter

LeetCode match: LC 362

QUESTION

Design a hit counter. hit(timestamp) records a hit at that second, and get_hits(timestamp) returns the number of hits in the past 300 seconds (timestamps never go backwards).

Function signature

```
class QueueCounter:
    def hit(self, ts: int) -> None
    def get_hits(self, ts: int) -> int
```

EXAMPLES

Example 1

Input	`h = QueueCounter()`
	`h.hit(1)`
	`h.hit(2)`
	`h.hit(3)`
	`h.get_hits(4)`
	`h.hit(300)`
	`h.get_hits(300)`
	`h.get_hits(301)`

Output	`h.get_hits(4) -> 3`
	`h.get_hits(300) -> 4`
	`h.get_hits(301) -> 3`

Why	`At time 301 the hit at time 1 is exactly 300 seconds old and no longer counts.`

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
hits = []
def hit(t):
    hits.append(t)
def get_hits(t):
    return sum(1 for x in hits if x > t - 300)
```

BRUTE FORCE

TIME	SPACE
O(n) per getHits	O(n)

WHY IT IS SLOW

Memory grows with every hit forever, and each query scans them all, O(n). A queue that drops timestamps older than 300 seconds (or 300 per-second buckets) keeps both bounded.

OPTIMAL SOLUTION · Queue of timestamps with eviction

TIME	SPACE
O(1) amortized	O(hits in the window)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Millions of hits per second.

Use 300 buckets (one per second) instead of one entry per hit: hit is O(1), getHits is O(300) = O(1), and memory is constant. A bucket whose stored second is older than the window is reset on reuse.

FOLLOW-UP 2 Timestamps can arrive out of order.

A deque no longer works. Keep a sorted structure (a sorted list with bisect or a balanced tree) and count with two binary searches, or use the buckets, which are order-insensitive within the window.

FOLLOW-UP 3 Thread safety.

Guard the queue with a lock (or use per-thread counters merged on read). The bucket version needs the compare-then-reset of a slot to be atomic. **Cost:** one lock per call, O(1) extra; contention rather than complexity is the concern.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Hit Counter
#
# Design a hit counter. hit(timestamp) records a hit at
# that second, and get_hits(timestamp) returns the number
# of hits in the past 300 seconds (timestamps never go
# backwards).
#
# Signature:
```

```
#   class QueueCounter:
#       def hit(self, ts: int) -> None
#       def get_hits(self, ts: int) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. Millions of hits per second.
# 2. Timestamps can arrive out of order.
# 3. Thread safety.
#
# INTERVIEW NOTES
# Only the last 300 seconds matter, so anything older is
# dead weight: evict from the front of a queue of
# timestamps. If hits arrive at very high rates, store
# counts per second in a circular array of 300 buckets so
# memory does not depend on the number of hits.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import threading
from bisect import bisect_right, insort
from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
WINDOW = 300   # seconds
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Queue of timestamps with
# eviction
# Idea: Keep a deque of timestamps in arrival order.
# getHits pops from the front while the oldest is outside
# the window, and the answer is the deque length. Each
# timestamp is pushed and popped once.
class QueueCounter:
    """One entry per hit; old entries are dropped when
    asked. Timestamps must not go backwards."""

    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        # keep only the hits in (ts - 300, ts]
        while self.hits and self.hits[0] <= ts - WINDOW:
            self.hits.popleft()
        return len(self.hits)

# ---- main question: tests ----
def test_main_question():
    counter = QueueCounter()
    counter.hit(1)
    counter.hit(2)
    counter.hit(3)
    assert counter.get_hits(4) == 3
    counter.hit(300)
    assert counter.get_hits(300) == 4
    assert counter.get_hits(301) == 3
```

```
# ============================================================
# FOLLOW-UP 1: Millions of hits per second.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use 300 buckets (one per second) instead of one
# entry per hit: hit is O(1), getHits is O(300) = O(1),
# and memory is constant. A bucket whose stored second is
# older than the window is reset on reuse.
class BucketCounter:
    """Follow-up: millions of hits per second. One bucket
    per second of the window: constant memory and time."""

    def __init__(self):
        self.stamp = [0] * WINDOW
        self.count = [0] * WINDOW

    def hit(self, ts):
        slot = ts % WINDOW
        if self.stamp[slot] != ts:
            # the slot still holds an old second: reuse it
            self.stamp[slot], self.count[slot] = ts, 0
        self.count[slot] += 1

    def get_hits(self, ts):
        slots = zip(self.stamp, self.count)
        return sum(c for t, c in slots
                   if 0 <= ts - t < WINDOW)

# ============================================================
# FOLLOW-UP 2: Timestamps can arrive out of order.
# ============================================================

# ---- follow-up 2: function ----
# Answer: A deque no longer works. Keep a sorted structure
# (a sorted list with bisect or a balanced tree) and count
# with two binary searches, or use the buckets, which are
# order-insensitive within the window.
class SortedCounter:
    """Follow-up: timestamps can arrive out of order.
    A sorted list and two binary searches count the
    window."""

    def __init__(self):
        self.hits = []

    def hit(self, ts):
        insort(self.hits, ts)

    def get_hits(self, ts):
        newest = bisect_right(self.hits, ts)
        oldest = bisect_right(self.hits, ts - WINDOW)
        return newest - oldest

# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    kinds = (QueueCounter, BucketCounter, SortedCounter)
    for make in kinds:
        c = make()
        for ts in (1, 2, 3, 300):
            c.hit(ts)
        assert c.get_hits(300) == 4
        # the hit at second 1 is 300 seconds old: it is
        # out
        assert c.get_hits(301) == 3
        assert c.get_hits(599) == 1
        assert c.get_hits(600) == 0
        assert c.get_hits(1000) == 0
    for make in kinds:
```

```
        c, log, now = make(), [], 0
        for _ in range(500):
            now += random.randint(0, 40)
            if random.random() < 0.6:
                c.hit(now)
                log.append(now)
    # out of order input only works with the sorted
    # version
    late = SortedCounter()
    for ts in (50, 10, 400, 30):
        late.hit(ts)
    assert late.get_hits(400) == 1
    assert late.get_hits(310) == 2


# ============================================================
# FOLLOW-UP 3: Thread safety.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Guard the queue with a lock (or use per-thread
# counters merged on read). The bucket version needs the
# compare-then-reset of a slot to be atomic. Cost: one
# lock per call, O(1) extra; contention rather than
# complexity is the concern.
class Locked:
    """Follow-up: thread safety. One lock around every
    call keeps check-then-update steps atomic."""

    def __init__(self, counter):
        self.counter = counter
```

```
        self.lock = threading.Lock()

    def hit(self, ts):
        with self.lock:
            self.counter.hit(ts)

    def get_hits(self, ts):
        with self.lock:
            return self.counter.get_hits(ts)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    shared = Locked(BucketCounter())
    def spam():
        for _ in range(1000):
            shared.hit(7)
    workers = [threading.Thread(target=spam)
               for _ in range(4)]
    for w in workers:
        w.start()
    for w in workers:
        w.join()
    assert shared.get_hits(7) == 4000


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Design a rolling hit counter API | PracHub | Technical Screen

https://prachub.com/coding-questions/design-a-rolling-hit-counter-api

STUDY PATH: question 88 of 288 | topic 10 of 43: A dict of per-key state that you update, expire, or rank | 8 of 8

Template fit: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time | asked 116 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 88 of 288

Website Activity Analytics: Unique Users + Session Time

ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 277/288

ABOUT THIS QUESTION

Topic: A dict of per-key state that you update, expire, or rank

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

Original: https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

QUESTION

Each record has a user_id, a timestamp ('YYYY-MM-DD HH:MM:SS') and an activity. Return (1) the number of distinct users per activity and (2) the average number of seconds a user spends on the site. A user's time is the sum of the gaps from each 'login' to the next 'logout'; a login without a logout counts until the user's last event.

Function signature

def activity_stats(records: list) -> (dict, float)

EXAMPLES

Example 1	

Input	`rec = lambda u, t, a: {'user_id': u, 'timestamp': '2026-01-01 ' + t, 'activity': a}`
	`records = [rec(1, '10:00:00', 'login'), rec(1, '10:10:00', 'click'), rec(1, '10:30:00', 'logout'), rec(2, '10:05:00', 'login'), rec(2, '10:20:00', 'logout')]`
	`activity_stats(records)`

Output	`activity_stats(records) -> ({'click': 1, 'login': 2, 'logout': 2}, 1350)`
Why	`User 1 spent 30 minutes (1800 s), user 2 spent 15 minutes (900 s): the average is 1350 s. Both users logged in and out; only user 1 clicked.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

TIME	SPACE
O(U * N + A * N)	O(U + A)

WHY IT IS SLOW

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

OPTIMAL SOLUTION · One pass to group, then sessions per user

TIME	SPACE
O(N log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Website Activity Analytics: Unique Users
# + Session Time
#
# Each record has a user_id, a timestamp ('YYYY-MM-DD
# HH:MM:SS') and an activity. Return (1) the number of
# distinct users per activity and (2) the average number
# of seconds a user spends on the site. A user's time is
# the sum of the gaps from each 'login' to the next
# 'logout'; a login without a logout counts until the
```

```
# user's last event.
#
# Signature:
#   def activity_stats(records: list) -> (dict, float)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Users with several sessions, missing logouts or out-
# of-order rows.
#
# INTERVIEW NOTES
# Unique users per activity is a map from activity to a
# set of user ids. Time on site needs a definition: pair
# each login with the next logout of the same user, and
# say what you do for a login without a logout (here: the
# session ends at that user's last event). Parse
# timestamps once, sort each user's events by time, and
# agree on out-of-order rows and repeated logins before
# coding.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict
from datetime import datetime


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
FORMAT = "%Y-%m-%d %H:%M:%S"
```

```
def when(record):
    # timestamp text -> datetime
    return datetime.strptime(record["timestamp"], FORMAT)

# ---- main question: main function ----
# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
# FOLLOW-UP 1: Users with several sessions, missing
# logouts or out-of-order rows.
# Answer: Sort each user's events by time before pairing;
# a second login before a logout is ignored (or closes the
# old session, depending on the contract you agree).
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total
```

```
# AVERAGE time on the site per user
def average_time_per_user(records):
    events = defaultdict(list)
    for r in records:
        item = (when(r), r["activity"])
        events[r["user_id"]].append(item)
    if not events:
        return 0.0
    # state the denominator: every user who appears
    per_user = events.values()
    total = sum(session_seconds(e) for e in per_user)
    return total / len(events)

# ---- main question: tests ----
def test_main_question():
    def rec(user, minute, activity):
        stamp = f"2024-07-26 10:{minute:02}:00"
        return {"user_id": user, "activity": activity,
                "timestamp": stamp}
    data = [rec(1, 0, "login"), rec(2, 5, "login"),
            rec(1, 10, "view"), rec(1, 15, "purchase"),
            rec(2, 20, "view"), rec(3, 25, "login"),
            rec(1, 30, "logout")]
    assert unique_users_by_activity(data) == {
        "login": 3, "logout": 1, "purchase": 1, "view": 2}
    # user 1: 1800 s, user 2: 900 s (no logout), user 3: 0
    assert average_time_per_user(data) == 900
    assert average_time_per_user([]) == 0.0
    # out-of-order rows are sorted first
    shuffled = [rec(1, 30, "logout"), rec(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800
    # two sessions add up
    two = [rec(1, 0, "login"), rec(1, 5, "logout"),
           rec(1, 10, "login"), rec(1, 20, "logout")]
    assert average_time_per_user(two) == 900


# ============================================================
# FOLLOW-UP 1: Users with several sessions, missing
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 11 OF 43

Binary search on a yes/no condition

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	4	3
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Binary search on a yes/no condition | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	89. First Index Reaching a Threshold (Binary Search)	ColorStack	2025-10-11
M	90. Cutting Ribbons (LC 1891)	1Point3Acres	2026-08-16
M	91. First Bad Version with Parallel Search Follow-up	1Point3Acres	2026-06-15
M	92. Solve Banana Speed and Interval Merge	PracHub	2025-01-27
M	93. Given a list with response time for each server, calculate how long time it will take to process n req...	Exponent	no date
H	94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	1Point3Acres	2026-10-09
H	95. Find the median of two sorted arrays.	Exponent	no date
H	96. Find longest duplicated substring	PracHub	2025-12-08

CODE TEMPLATE | GROUP 11 OF 43 | 8 QUESTIONS, ONE PATTERN

Binary search on a yes/no condition

The one idea these questions share

If answers go no, no, no, yes, yes, yes as the value grows, binary search finds the first yes. Write ok(x), set lo and hi, and halve. The questions only differ in what x is (a length, a speed, a time, an index, a version) and how ok(x) is checked.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_true(lo, hi, ok):                    # smallest x in [lo, hi) with ok(x); hi if none
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):                            # CHANGE: the yes/no test
            hi = mid
        else:
            lo = mid + 1
    return lo
```

2. Helper functions that usually get written next to it

```
def last_true(lo, hi, ok):                     # largest x in [lo, hi] with ok(x); lo - 1 if none
    return first_true(lo, hi + 1, lambda x: not ok(x)) - 1


def ceil_div(a, b):
    return -(-a // b)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
89	First Index Reaching a Threshold (Binary Search)	x = an index; ok(x) = arr[x] >= k on a sorted array
90	Cutting Ribbons (LC 1891)	x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1
91	First Bad Version with Parallel Search Follow-up	x = the version number; ok(x) = is_bad(x)
92	Solve Banana Speed and Interval Merge	x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile
93	Given a list with response time for each server, calculate how long time it will take to process n requests.	x = the elapsed time; ok(x) = sum(x // time_i) >= n requests
94	Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest
95	Find the median of two sorted arrays.	the same partition search on the shorter array
96	Find longest duplicated substring	x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 89 of 288 | topic 11 of 43: Binary search on a yes/no condition | 1 of 8

Template fit: x = an index; ok(x) = arr[x] >= k on a sorted array | asked 374 days ago | frequency Single report

EASY | IN APP LC 35 | ColorStack | technical interview | 89 of 288

First Index Reaching a Threshold (Binary Search)

ASKED 2025-10-11 | 374 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 237/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: your app's interview approach (LeetCode #35)

LeetCode match: LC 35 Search Insert Position / lower bound

Frequency: Single report | Last asked: 2025-10-11 | Roles: SWE Intern

QUESTION

Given a non-decreasing (sorted ascending, duplicates allowed) array arr and a target value k , return the smallest index i such that arr[i] >= k — the first position where the array "reaches" the value k . If no element is >= k , return -1 . Solve in O(log n) time and O(1) extra space.

Function signature

def first_index_at_least(arr: list[int], k: int) -> int:
    ...

Follow-up (as reported): The interviewer probed space complexity; note the solution is already O(1) space. If arr is a huge sorted stream/file, describe reading via random access rather than loading it all.

EXAMPLES

Example 1	
Input	arr = [1, 2, 2, 3, 5, 7], k = 2
Output	1
Why	arr[1] = 2 is the first element >= 2.

Example 2	
Input	arr = [1, 2, 2, 3, 5, 7], k = 4
Output	4
Why	The first element >= 4 is 5 at index 4.

Example 3	
Input	arr = [1, 2, 2, 3, 5, 7], k = 8
Output	-1
Why	No element is >= 8.

CONSTRAINTS

- 0 <= len(arr) <= 10**7
- Values and k are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_first_at_least(arr, k):
    for i, x in enumerate(arr):
        if x >= k:
            return i
    return -1
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

It ignores that the array is sorted.

OPTIMAL SOLUTION · Binary search for the lower bound

TIME	SPACE
O(log n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array is a huge sorted file with random access and an unknown length.

Probe positions 1, 2, 4, 8, ... until the value there is at least k or the position is past the end, then binary search inside the last doubled range. It needs O(log position) reads and no loading of the data.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: First Index Reaching a Threshold (Binary
# Search)
#
# Given a non-decreasing (sorted ascending, duplicates
# allowed) array arr and a target value k , return the
# smallest index i such that arr[i] >= k — the first
# position where the array "reaches" the value k . If no
# element is >= k , return -1 . Solve in O(log n) time and
# O(1) extra space. Function signature: def
# first_index_at_least(arr: list[int], k: int) -> int: ...
# Follow-up (as reported): The interviewer probed space
# complexity; note the solution is already O(1) space. If
# arr is a huge sorted stream/file, describe reading via
# random access rather than loading it all.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array is a huge sorted file with random access
# and an unknown length.
#
# INTERVIEW NOTES
# The smallest index with arr[i] >= k is a lower bound:
# binary search for the boundary between 'less than k' and
```

```
# 'at least k'. Keep lo and hi as the search range and
# move hi to mid when arr[mid] >= k, lo to mid + 1
# otherwise; the answer is lo if it is inside the array.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Binary search for the lower
# bound
# Idea: Halve the range, keeping the boundary between
# values below k and values at least k.
```

```
def first_index_at_least(arr, k):
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo if lo < len(arr) else -1

# ---- main question: tests ----
def test_main_question():
    random.seed(59)
    a = [1, 2, 2, 3, 5, 7]
    assert first_index_at_least(a, 2) == 1
    assert first_index_at_least(a, 4) == 4
    assert first_index_at_least(a, 8) == -1
    assert first_index_at_least([], 1) == -1


# ============================================================
# FOLLOW-UP 1: The array is a huge sorted file with random
# ============================================================

# ---- follow-up 1: function ----
# Answer: Probe positions 1, 2, 4, 8, ... until the value
# there is at least k or the position is past the end,
# then binary search inside the last doubled range. It
# needs O(log position) reads and no loading of the data.
def first_at_least_unbounded(get, k):
    # get(i) returns the i-th value or raises IndexError
    # past the end
    def value(i):
        try:
            return get(i)
        except IndexError:
            return float("inf")

    hi = 1
    while value(hi - 1) < k:
        hi *= 2
        if hi > 1 << 40:
            return -1
    lo = hi // 2
    while lo < hi:
        mid = (lo + hi) // 2
        if value(mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo if value(lo) != float("inf") else -1


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 90 of 288 | topic 11 of 43: Binary search on a yes/no condition | 2 of 8

Template fit: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1 | asked 65 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 90 of 288

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 168/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

You have ribbons of given lengths and an integer k. Each ribbon may be cut into pieces of one common integer length L (leftover bits are thrown away). Return the largest L that gives at least k pieces in total, or 0 if even length 1 cannot give k pieces.

Function signature

def max_length(ribbons: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[9, 7, 5], 3`
Output	`5`
Why	`Length 5 gives 1 + 1 + 1 = 3 pieces, length 6 only 2.`

Example 2	
Input	`[5, 7], 12`
Output	`1`
Why	`Length 1 already gives 12 pieces.`

Example 3	
Input	`[1, 2], 10`
Output	`0`
Why	`Total length is 3 but 10 pieces are needed: impossible, so 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Cutting Ribbons (LC 1891)
#
# You have ribbons of given lengths and an integer k. Each
# ribbon may be cut into pieces of one common integer
# length L (leftover bits are thrown away). Return the
# largest L that gives at least k pieces in total, or 0 if
# even length 1 cannot give k pieces.
#
# Signature:
#   def max_length(ribbons: list, k: int) -> int
#
# Examples:
# [9, 7, 5], 3  ->  5
# [5, 7], 12  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the running time depend on the value range
# and not just on the number of ribbons?
# 2. What if the candidate length is given to you?
# 3. What is a good upper bound for the search?
# 4. Give the search a tighter upper bound.
#
# INTERVIEW NOTES
# 'Largest length such that at least K pieces fit' is a
# maximisation with a yes/no check. Larger lengths yield
# fewer pieces, so the check is monotone and you can
# binary search the answer. The check itself is one pass:
# sum of ribbon // length.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)
```

```


# FOLLOW-UP 4: Give the search a tighter upper bound.
# Answer: No piece can be longer than the longest ribbon,
# and K pieces of length L need K * L total length, so L
# <= sum // K. Starting hi at min(max(ribbons), sum // K)
# saves a few probes and also returns 0 immediately when
# sum < K.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)
```

```


# ---- main question: main function ----
# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best


# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
```

```
    return lo

# ---- main question: tests ----
def test_main_question():
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Give the search a tighter upper bound.
# ============================================================
# Its code sits in the main question section above, becaus
```

```
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

STUDY PATH: question 91 of 288 | topic 11 of 43: Binary search on a yes/no condition | 3 of 8

Template fit: x = the version number; ok(x) = is_bad(x) | asked 127 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 91 of 288

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 41.1 | 30 DAYS 33 | PRIORITY 54/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Versions are numbered 1 to n and from some version on every version is bad. The function is_bad(v) tells whether version v is bad. Find the first bad version with as few calls to is_bad as possible.

Function signature

def first_bad_version(n: int, is_bad) -> int

EXAMPLES

Example 1

Input	`bad_from = 4 first_bad_version(10, lambda v: v >= bad_from)`
Output	`first_bad_version(10, lambda v: v >= bad_from) -> 4`
Why	`Versions 4 to 10 are bad, so 4 is the first; binary search needs about log2(10) calls.`

Example 2

Input	`bad_from = 1 first_bad_version(1, lambda v: v >= bad_from)`
Output	`first_bad_version(1, lambda v: v >= bad_from) -> 1`
Why	`With a single version that is bad, the answer is 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    for v in range(1, n + 1):
        if is_bad_version(v):
            return v
```

TIME	SPACE
O(n) calls	O(1)

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION · Base problem: binary search

TIME	SPACE
O(log n) calls	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: probe k points at once, then recurse into the bucket that flips

TIME	SPACE
O(log_{k+1} n) rounds	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

FOLLOW-UP 2 What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

FOLLOW-UP 3 Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

FOLLOW-UP 4 Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: First Bad Version with Parallel Search
# Follow-up
#
# Versions are numbered 1 to n and from some version on
# every version is bad. The function is_bad(v) tells
# whether version v is bad. Find the first bad version
# with as few calls to is_bad as possible.
#
# Signature:
#   def first_bad_version(n: int, is_bad) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does parallelism buy you, and what does it cost?
# 2. What if the oracle is rate limited or can fail?
# 3. Count how many rounds each approach needs.
# 4. Show how many rounds and calls the parallel search
# uses.
#
# INTERVIEW NOTES
# 'First bad version' is a boundary in a sorted yes/no
# sequence, which is binary search. The parallel part is a
# different trade: instead of one probe per round, send
# several probes in the same round and keep the gap that
# contains the boundary. Fewer rounds, more total calls.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [lo,
# hi]' and halve the interval. Use lo + (hi - lo) // 2 in
```

```
# languages with fixed-width integers.
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
    return lo


# PARALLEL SEARCH helper
def probe_points(lo, hi, workers):
    # evenly spaced versions inside [lo, hi - 1]
    step = max(1, (hi - lo + 1) // (workers + 1))
    pts = {min(hi - 1, lo + step * (i + 1) - 1)
           for i in range(workers)}
    return sorted(pts)

# ---- main question: tests ----
def test_main_question():
    calls, lock = [], Lock()
    def oracle(first_bad):
        def is_bad(v):
            with lock:
                calls.append(v)
            return v >= first_bad
        return is_bad
    cases = ((1, 1), (10, 1), (10, 10), (1000, 637))


# ============================================================
# FOLLOW-UP 1: What does parallelism buy you, and what
# ============================================================

# ---- follow-up 1: function ----
# Answer: It cuts the number of sequential rounds from
# log2(n) to log_{k+1}(n), so wall-clock time drops when
# each probe has high latency. It increases total probes
# (about k per round instead of 1), so it costs more calls
# and money.
# FOLLOW-UP 3: Count how many rounds each approach needs.
```

```
# Answer: Single binary search: log2(n) rounds of one
# probe each. k workers: about log base (k + 1) of n
# rounds with k probes each. For n = 10^9, one probe per
# round needs 30 rounds, while 7 probes per round need 10.
# FOLLOW-UP 4: Show how many rounds and calls the parallel
# search uses.
# Answer: With k workers each round shrinks the interval
# by about k + 1, so the number of sequential rounds is
# about log base (k + 1) of n and the total calls at most
# k times that.
def first_bad_parallel(n, is_bad, workers=4):
    # invariant: the answer is in [lo, hi]
    lo, hi, cache, rounds = 1, n, {}, 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
            points = probe_points(lo, hi, workers)
            # ask the oracle about new points at the same
            # time
```

```
            fresh = [p for p in points if p not in cache]
            answers = pool.map(is_bad, fresh)
            for p, bad in zip(fresh, answers):
                cache[p] = bad
            rounds += 1
            bad_points = [p for p in points if cache[p]]
            if not bad_points:
                # every probe was good: answer is after
                # them
                lo = points[-1] + 1
            else:
                first = bad_points[0]
                # the answer sits between the last good
                # probe before it and the first bad one
                before = [p for p in points if p < first]
                if before:
                    lo = before[-1] + 1
                hi = first
    return lo, rounds, len(cache)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    calls, lock = [], Lock()
    def oracle(first_bad):
        def is_bad(v):
            with lock:
                calls.append(v)
            return v >= first_bad
        return is_bad
    cases = ((1, 1), (10, 1), (10, 10), (1000, 637))
    for n, bad in cases:
        assert first_bad_version(n, oracle(bad)) == bad
        ans, rounds, used = first_bad_parallel(
            n, oracle(bad), workers=4)
        assert ans == bad
    # fewer rounds than the 20 of a plain binary search
    ans, rounds, used = first_bad_parallel(
        10 ** 6, oracle(123456), workers=7)
    assert ans == 123456 and rounds < 20
    assert used <= 7 * rounds


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Count how many rounds each approach needs.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Show how many rounds and calls the parallel
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")

if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

STUDY PATH: question 92 of 288 | topic 11 of 43: Binary search on a yes/no condition | 4 of 8

Template fit: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile | asked 631 days ago

MEDIUM | PracHub | Technical Screen | 92 of 288

★ TOP 75: rank 12 in the Medium and Hard priority list

★ Solve Banana Speed and Interval Merge

POSTED 2025-01-27 | 631 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 11/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas, LeetCode #56 Merge Intervals

LeetCode match: LC 875 Koko Eating Bananas + LC 56 Merge Intervals

Posted on PracHub: 2025-01-27 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/solve-banana-speed-and-interval-merge

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

QUESTION

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`**
  Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4:
    ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) =
    1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 = 10 > 8, so
    4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

QUESTION

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

BRUTE FORCE · Bananas: try every speed upward

BRUTE FORCE · PSEUDO-CODE

```
def brute(piles, h):
    for speed in range(1, max(piles) + 1):
        hours = 0
        for p in piles:
            hours += ceil(p / speed)
        if hours <= h:
            return speed
```

TIME	SPACE
O(max * n)	O(1)

WHY IT IS SLOW

Up to max(piles) speeds, each costing O(n); piles can be 10^9. Hours are non-increasing in the speed, so binary search over speeds is O(n log max).

BRUTE FORCE · Intervals: merge any overlapping pair until none is left

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE · Intervals: merge any overlapping pair until none is left

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Bananas: binary search the speed

TIME	SPACE
O(n log max)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Intervals: sort and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bananas: h varies per query, or the piles change.

The check is independent of the search; for many queries keep the piles sorted with prefix sums so the hours needed at speed s can be computed in O(log n) per distinct pile size.

FOLLOW-UP 2 Intervals: insert one new interval into an already merged list (LeetCode 57).

Copy intervals ending before the new one, merge the ones that overlap it, then copy the rest. O(n), no sort needed.

FOLLOW-UP 3 Bananas: the guards come back in fewer hours than piles.

Then no speed works (an hour can finish at most one pile): check h >= len(piles) first and return -1 otherwise.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve Banana Speed and Interval Merge
#
# During two technical phone screens for a new graduate
# software engineering role, the candidate was asked to
# solve two coding problems: minimum eating speed and
# merging overlapping intervals. ### Constraints &
# Assumptions - Provide clear algorithms, correctness
# reasoning, complexity, and edge cases. - For minimum
# eating speed, each hour can be spent on only one pile. -
```

```
# For interval merging, intervals overlap when the next
# start is less than or equal to the current merged end. -
# Use sorted or binary-search approaches where
# appropriate. ### Clarifying Questions to Ask - Are piles
# and h always positive? - Is h at least the number of
# piles? - Are interval endpoints inclusive? - Can
# intervals be empty or unsorted? ### Part 1 - Minimum
# Eating Speed Given piles and h, return the minimum
# integer speed k such that all piles can be eaten within
# h hours.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bananas: h varies per query, or the piles change.
# 2. Intervals: insert one new interval into an already
# merged list (LeetCode 57).
# 3. Bananas: the guards come back in fewer hours than
# piles.
#
# INTERVIEW NOTES
# Bananas: the time needed is non-increasing in the speed,
# so the smallest speed that finishes in h hours can be
# found by binary search over speeds, with a simple sum of
# ceil(pile / speed) as the check. Intervals: sort by
# start and sweep with one 'current' interval; the sort
# guarantees nothing earlier can still overlap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile
    # rounds up
    return sum(-(-p // speed) for p in piles)

# ---- main question: main function ----
# BASE QUESTION (optimal): Bananas: binary search the
# speed
# Idea: Hours needed are non-increasing in the speed, so
# keep lo = 1 and hi = max(piles) and halve: if speed mid
# finishes within h, try slower, otherwise faster. Use
# ceiling division (an hour finishes at most one pile).
def min_eating_speed(piles, h):
    # fewer hours than piles can never work
    if h < len(piles):
        return -1
    lo, hi = 1, max(piles)
    # more speed never takes more hours: binary search
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(piles, mid) <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo


# PART 2: merge intervals
def merge_intervals(intervals):
    out = []
    for s, e in sorted(intervals):
        # overlap (sharing a point counts): extend the
        # last one
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out

# ---- main question: tests ----
def test_main_question():
    assert min_eating_speed([3, 6, 7, 11], 8) == 4
```

```
    assert min_eating_speed([30, 11, 23, 4, 20], 5) == 30
    assert min_eating_speed([30, 11, 23, 4, 20], 6) == 23
    # fewer hours than piles: impossible
    assert min_eating_speed([1, 1, 1], 2) == -1
    assert hours_needed([3, 6, 7, 11], 4) == 8
    spans = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_intervals(spans)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Intervals: insert one new interval into an
# ============================================================

# ---- follow-up 2: function ----
# Answer: Copy intervals ending before the new one, merge
# the ones that overlap it, then copy the rest. O(n), no
# sort needed.
def insert_interval(merged, new):
    # LeetCode 57: the list is already merged and sorted:
    # O(n)
    out, i, n = [], 0, len(merged)
    # 1) intervals that end before the new one starts
    while i < n and merged[i][1] < new[0]:
        out.append(merged[i])
        i += 1
    # 2) intervals that overlap it are absorbed
    s, e = new
    while i < n and merged[i][0] <= e:
        s, e = min(s, merged[i][0]), max(e, merged[i][1])
        i += 1
    out.append([s, e])
    # 3) the rest comes after
    out += merged[i:]
    return out

# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    got = insert_interval([[1, 3], [6, 9]], [2, 5])
    assert got == [[1, 5], [6, 9]]
    assert insert_interval([], [4, 8]) == [[4, 8]]
    assert insert_interval([[1, 2], [5, 6]], [3, 4]) == [
        [1, 2], [3, 4], [5, 6]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(piles: list[int], h: int) -> int:
```

Sample call

```
solution([3, 6, 7, 11], 8)
```

Test cases

Input	Expected
([3, 6, 7, 11], 8)	4
([30, 11, 23, 4, 20], 5)	30

STUDY PATH: question 93 of 288 | topic 11 of 43: Binary search on a yes/no condition | 5 of 8

Template fit: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests | no date

MEDIUM | Exponent | 93 of 288

★ TOP 75: rank 24 in the Medium and Hard priority list

★ Given a list with response time for each server, calculate how long time it will take to process n requests.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TYPE | **ALL TIME 59.1** | **30 DAYS 46.9** | **PRIORITY 21/288**

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas

LeetCode match: No direct match; related LC 875 Koko Eating Bananas (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5485/calculate-time-to-process-given-server-response-time

QUESTION

Servers work in parallel and server i needs times[i] time units to process one request (optionally server i only starts at starts[i]). Return the earliest time at which n requests have been processed in total.

Function signature

def time_for(times: list, n: int, starts=None) -> int

EXAMPLES

Example 1	
Input	`[1, 2], 3`
Output	`2`
Why	`By time 2 the first server has done 2 requests and the second one: 3 in total.`

Example 2	
Input	`[3], 4`
Output	`12`
Why	`One server needs 3 units per request: 4 requests take 12.`

Example 3	
Input	`[2, 1], 6, starts = [0, 5]`
Output	`8`
Why	`The second server only starts at time 5, so the first one does most of the work.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(times, n):
    heap = [(r, r) for r in times]  # (finish, r)
    heapify(heap)
    last = 0
    for _ in range(n):
        finish, r = heappop(heap)
        last = finish
        heappush(heap, (finish + r, r))
    return last
```

TIME	SPACE
O(n log S)	O(S)

WHY IT IS SLOW

One heap operation per request, O(n log S); with n up to 10^9 requests that is too slow. The finish time is a monotone threshold, so binary search on T with sum(T // response time) >= n is O(S log answer).

OPTIMAL SOLUTION · Binary search on the total time

TIME	SPACE
O(S log(min(times) * n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Servers may fail, or there is a startup time per server.

Add the startup time to the check: server i completes max(0, (T - start_i) // r_i) requests. The check stays monotone in T, so binary search still applies.

FOLLOW-UP 2 The inverse question: how many servers are needed to finish within a given time?

Divide: each server finishes time // r requests, so you need ceil(n / that). No search needed.

FOLLOW-UP 3 Requests have different sizes.

Then it is a scheduling problem (minimise the makespan), NP-hard in general; the greedy 'longest request first to the least-loaded server' is within 4/3 of optimal.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a list with response time for each
# server, calculate how long time it will take to process
# n requests.
#
# Servers work in parallel and server i needs times[i]
# time units to process one request (optionally server i
# only starts at starts[i]). Return the earliest time at
# which n requests have been processed in total.
#
# Signature:
#   def time_for(times: list, n: int, starts=None) -> int
#
# Examples:
# [1, 2], 3  ->  2
# [3], 4  ->  12
#
# FOLLOW-UPS (answered below, in this order):
# 1. Servers may fail, or there is a startup time per
# server.
# 2. The inverse question: how many servers are needed to
# finish within a given time?
# 3. Requests have different sizes.
#
# INTERVIEW NOTES
# 'Smallest time T so that the servers can finish n
# requests' has a monotone yes/no check: if T is enough,
# any larger time is too. The check is a sum of T //
# response_time over the servers. Whenever the answer is a
# threshold on a monotone condition, binary search the
# threshold instead of simulating.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# FOLLOW-UP 1: Servers may fail, or there is a startup
# time per server.
# Answer: Add the startup time to the check: server i
# completes max(0, (T - start_i) // r_i) requests. The
# check stays monotone in T, so binary search still
# applies.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def done_by(times, t, starts=None):
    # how many requests all servers finish within time t.
    # starts[i] is a startup delay of server i (default
    # 0).
    total = 0
    for i, r in enumerate(times):
        begin = starts[i] if starts else 0
        total += max(0, (t - begin) // r)
    return total

# ---- main question: main function ----
# BASE QUESTION (optimal): Binary search on the total time
# Idea: In time T server i completes T // r_i requests, so
# the servers finish at least n requests exactly when
# sum(T // r_i) >= n, which is monotone in T.
def time_for(times, n, starts=None):
    # smallest t with done_by(t) >= n. done_by never goes
    # down as t grows, so binary search works.
    if n <= 0:
        return 0
    lo = 0
    first = starts[0] if starts else 0
    hi = first + times[0] * n
    for i, r in enumerate(times):
        begin = starts[i] if starts else 0
        hi = min(hi, begin + r * n)
    while lo < hi:
        mid = (lo + hi) // 2
        if done_by(times, mid, starts) >= n:
            hi = mid
        else:
            lo = mid + 1
    return lo

# ---- main question: tests ----
```

```
def test_main_question():
    assert time_for([1, 2, 3], 6) == 4
    assert time_for([4], 3) == 12
    assert time_for([5, 5], 0) == 0
    assert time_for([2, 3], 1) == 2
    # server 0 starts after 10, so server 1 does the work
    assert time_for([1, 5], 2, starts=[10, 0]) == 10


# ============================================================
# FOLLOW-UP 1: Servers may fail, or there is a startup
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: The inverse question: how many servers are
# ============================================================

# ---- follow-up 2: function ----
# Answer: Divide: each server finishes time // r requests,
# so you need ceil(n / that). No search needed.
def servers_needed(n, r, t):
    # follow-up: identical servers with response time r.
    # In
    # time t each one finishes t // r requests, so divide.
    per = t // r
    if per == 0:
        return None
    return -(-n // per)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert servers_needed(10, 4, 8) == 5
    assert servers_needed(10, 4, 3) is None
    assert servers_needed(1, 7, 7) == 1


# ============================================================
# FOLLOW-UP 3: Requests have different sizes.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Then it is a scheduling problem (minimise the
# makespan), NP-hard in general; the greedy 'longest
# request first to the least-loaded server' is within 4/3
```

```
# of optimal.
def makespan(sizes, servers):
    # follow-up: requests of different sizes. Minimising
    # the
```

```
    # finish time is NP-hard; the greedy rule "biggest
    # request
    # to the least loaded server" is within 4/3 of the
    # best.
    loads = [0] * servers
    heapq.heapify(loads)
    for size in sorted(sizes, reverse=True):
        lightest = heapq.heappop(loads)
        heapq.heappush(loads, lightest + size)
    return max(loads)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # the best split is 9, 9, 9; greedy gets 11 (11/9 <
    # 4/3)
    assert makespan([5, 5, 4, 4, 3, 3, 3], 3) == 11
    assert makespan([], 2) == 0


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY PATH: question 94 of 288 | topic 11 of 43: Binary search on a yes/no condition | 6 of 8

Template fit: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest | asked 11 days ago | frequency Single report

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 94 of 288

Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 4/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

QUESTION

Given two sorted arrays of numbers, return the median of all their elements together, without merging them into one array. The median is the middle element when the total count is odd, and the average of the two middle elements when it is even. Aim for O(log(min(m, n))) time.

Function signature

def median_two(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	`[1, 3], [2]`
Output	`2`
Why	`Combined [1, 2, 3]: the middle element is 2.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`2.5`
Why	`Combined [1, 2, 3, 4]: the two middle elements are 2 and 3, so the median is 2.5.`

Example 3	
Input	`[], [7, 9]`
Output	`8`
Why	`One array may be empty: the median is just the other array's median.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log min(m, n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Median of Two Sorted Arrays (LC 4) with a
# Follow-up Variant
#
# Given two sorted arrays of numbers, return the median of
# all their elements together, without merging them into
# one array. The median is the middle element when the
# total count is odd, and the average of the two middle
# elements when it is even. Aim for O(log(min(m, n)))
# time.
#
# Signature:
#   def median_two(a: list, b: list) -> float
#
# Examples:
# [1, 3], [2]  ->  2
# [1, 2], [3, 4]  ->  2.5
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now there are k sorted arrays instead of two. How do
# you find the median?
# 2. What is the heap version and when would you choose
# it?
#
# INTERVIEW NOTES
# Both inputs are already sorted, so the median is a
# partition problem, not a streaming one (the two-heap
# design of LeetCode 295 is the wrong family and the
# interviewer redirects it). Split the small array at i
# and the large one at j so that the left halves together
# hold half the elements; the split is right when every
# left value is at most every right value, which only
# needs two comparisons across the cut. That condition is
# monotone in i, so binary search it in O(log min(m, n)).
# For k arrays the same idea becomes: binary search on the
# value and count elements at most x with one bisect per
# array.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import bisect
```

```
import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1

# ---- main question: tests ----
```

```
def test_main_question():
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5


# ============================================================
# FOLLOW-UP 1: Now there are k sorted arrays instead of
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search on the answer value. For a
# candidate x, count the elements at most x with one
# bisect per array (O(k log n)); the median is the
# smallest x whose count reaches the middle rank.
def median_k_value(arrays):
    """k arrays: binary search the value; one bisect per
    array counts how many elements are at most x."""
    arrays = [a for a in arrays if a]
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest x with at least `rank` elements <= x
        lo = min(a[0] for a in arrays)
        hi = max(a[-1] for a in arrays)
        while lo < hi:
            mid = (lo + hi) // 2
            count = sum(bisect.bisect_right(a, mid)
                        for a in arrays)
            if count >= rank:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5
```

```
# ============================================================
# FOLLOW-UP 2: What is the heap version and when would you
# ============================================================

# ---- follow-up 2: function ----
# Answer: A min-heap holds the front of every array. Pop
```

```
# (N + 1) / 2 times, remembering the one or two middle
# values.
def median_k_heap(arrays):
    """k arrays, simple version: pop from a k-way merge
    until the middle one or two values have been seen."""
    heap = [(a[0], i, 0)
            for i, a in enumerate(arrays) if a]
    heapq.heapify(heap)
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            nxt = (arrays[i][j + 1], i, j + 1)
            heapq.heappush(heap, nxt)
    if total % 2:
        return middle[0]
    return (middle[0] + middle[1]) / 2

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    for _ in range(300):
        arrays = []
        for _ in range(random.randint(1, 5)):
            size = random.randint(0, 6)
            arrays.append(sorted(random.randint(-9, 9)
                                 for _ in range(size)))
        if len(arrays) >= 2:
            rest = sum(arrays[1:], [])
```

```
            rest.sort()


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

STUDY PATH: question 95 of 288 | topic 11 of 43: Binary search on a yes/no condition | 7 of 8

Template fit: the same partition search on the shorter array | no date

HARD | IN APP LC 4 | Exponent | 95 of 288

★ TOP 75: rank 6 in the Medium and Hard priority list

★ Find the median of two sorted arrays.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 3/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4322/median-of-two-sorted-arrays

QUESTION

Given two sorted arrays, return the median of all their elements together. The median is the middle value when the total count is odd and the average of the two middle values when it is even. The best solutions run in O(log(min(m, n))).

Function signature

def findMedianSortedArrays(nums1: list, nums2: list) -> float

EXAMPLES

Example 1	
Input	`[1, 3], [2]`
Output	`2`
Why	`Combined: 1, 2, 3.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`2.5`
Why	`Combined: 1, 2, 3, 4: the middle two are 2 and 3.`

Example 3	
Input	`[], [5]`
Output	`5`
Why	`One empty array.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(a, b):
    merged = sorted(a + b)
    n = len(merged)
    if n % 2:
        return merged[n // 2]
    mid = n // 2
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

It sorts m + n numbers from scratch, O((m + n) log(m + n)), and stores them all, even though both inputs were already sorted. The partition binary search needs only O(log min(m, n)).

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log(min(m, n)))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if there are k sorted arrays instead of two?

Binary search on the answer value instead of on a position. For a candidate value v, count how many elements are <= v in each array with bisect; the median is the smallest v whose total count reaches the middle rank. That costs O(k log n log(range)) time and O(1) extra space and needs no merging.

FOLLOW-UP 2 What if the arrays are too large for memory?

The two-array binary search only touches O(log n) positions, so with random access (an indexed file or a remote array) it reads almost nothing. If only sequential access is possible, do a streaming selection: a two-pointer merge that stops after (m + n) / 2 elements, using O(1) memory and O(m + n) time.

FOLLOW-UP 3 Find the k-th smallest element, not just the median.

The same partition search: choose i elements from the shorter array and k - i from the other, and move i until the borders are in order. The median is just k = (m + n + 1) // 2.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the median of two sorted arrays.
#
# Given two sorted arrays, return the median of all their
# elements together. The median is the middle value when
# the total count is odd and the average of the two middle
# values when it is even. The best solutions run in
# O(log(min(m, n))).
#
# Signature:
#   def findMedianSortedArrays(nums1: list, nums2: list) -
> float
#
# Examples:
# [1, 3], [2]  ->  2
# [1, 2], [3, 4]  ->  2.5
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if there are k sorted arrays instead of two?
# 2. What if the arrays are too large for memory?
# 3. Find the k-th smallest element, not just the median.
#
# INTERVIEW NOTES
# Because both arrays are sorted you never need to merge
# them. The median splits the combined data into a left
# half and a right half of equal size; you only have to
# decide how many elements the shorter array contributes
# to the left half. Whether a split is right depends on
# four border values, and moving the split one way or the
# other is a binary search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
# FOLLOW-UP 3: Find the k-th smallest element, not just
# the median.
# Answer: The same partition search: choose i elements
# from the shorter array and k - i from the other, and
# move i until the borders are in order. The median is
# just k = (m + n + 1) // 2.
def kth_of_two(a, b, k):
    # FOLLOW-UP 3: any k-th smallest, not only the median
    # k-th smallest (k starts at 1) of two sorted arrays.
    # Take i items from a and k - i from b so that the
    # borders are in order. Binary search on i: O(log n).
    if len(a) > len(b):
        a, b = b, a
    lo, hi = max(0, k - len(b)), min(k, len(a))
    while lo <= hi:
        i = (lo + hi) // 2
        j = k - i
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < len(a) else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < len(b) else INF
        if a_left > b_right:
            hi = i - 1
        elif b_left > a_right:
            lo = i + 1
        else:
            return max(a_left, b_left)
    raise ValueError("k is out of range")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
def median_of_two(a, b):
    # the median is the middle item, or the mean of the
    # two
    # middle items when the total is even
    total = len(a) + len(b)
    if total == 0:
        raise ValueError("no data")
    if total % 2:
        return kth_of_two(a, b, total // 2 + 1)
    low = kth_of_two(a, b, total // 2)
    high = kth_of_two(a, b, total // 2 + 1)
    return (low + high) / 2

# ---- main question: tests ----
def test_main_question():
    assert median_of_two([1, 3], [2]) == 2
    assert median_of_two([1, 2], [3, 4]) == 2.5
    assert median_of_two([], [5]) == 5


# ============================================================
# FOLLOW-UP 1: What if there are k sorted arrays instead
# ============================================================

# ---- follow-up 1: helper functions ----
def count_at_most(arrays, v):
    # how many items are <= v in all the arrays
    return sum(bisect_right(arr, v) for arr in arrays)

# ---- follow-up 1: function ----
# Answer: Binary search on the answer value instead of on
# a position. For a candidate value v, count how many
# elements are <= v in each array with bisect; the median
# is the smallest v whose total count reaches the middle
# rank.
def kth_of_many(arrays, k):
    # follow-up: k sorted arrays (integers). Binary search
    # the ANSWER value: the k-th smallest is the smallest
    # v
    # that has at least k items <= v.
    arrays = [arr for arr in arrays if arr]
    lo = min(arr[0] for arr in arrays)
    hi = max(arr[-1] for arr in arrays)
```

```
    while lo < hi:
        mid = (lo + hi) // 2
        if count_at_most(arrays, mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# Answer: Binary search on the answer value instead of on
# a position. For a candidate value v, count how many
# elements are <= v in each array with bisect; the median
# is the smallest v whose total count reaches the middle
# rank.
def median_of_many(arrays):
    total = sum(len(arr) for arr in arrays)
    if total % 2:
        return kth_of_many(arrays, total // 2 + 1)
    low = kth_of_many(arrays, total // 2)
    high = kth_of_many(arrays, total // 2 + 1)
    return (low + high) / 2

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Find the k-th smallest element, not just
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade now

STUDY PATH: question 96 of 288 | topic 11 of 43: Binary search on a yes/no condition | 8 of 8

Template fit: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok | asked 316 days ago

HARD | IN APP LC 1044 | PracHub | Technical Screen | 96 of 288

Find longest duplicated substring

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 142/288

ABOUT THIS QUESTION

Topic: Binary search on a yes/no condition

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1044 Longest Duplicate Substring

LeetCode match: LC 1044 Longest Duplicate Substring

Posted on PracHub: 2025-12-08 (updated 2026-07-09)

Original: https://prachub.com/coding-questions/find-longest-duplicated-substring

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., `s[i..j)` for `0 <= i < j <= len(s)`. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., `1 <= n <= 2 × 10^5`).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive `O(n^2)` substring comparison approach.

Examples

- `s = "banana"`
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include `"ana"`.
- `s = "abcd"`
- No duplicated substring, so output should be `""` (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

EXAMPLES

Example 1	
Input	`'banana'`
Output	`'ana'`
Why	`'ana' occurs twice (overlapping is allowed).`

Example 2	
Input	`'abcd'`
Output	`''`
Why	`No substring repeats.`

QUESTION

Example 3	
Input	`'aaaaa'`
Output	`'aaaa'`
Why	`'aaaa' appears at positions 0 and 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for L in range(n - 1, 0, -1):
        seen = set()
        for i in range(n - L + 1):
            sub = s[i:i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""
```

TIME	SPACE
O(n^3)	O(n^2)

WHY IT IS SLOW

Each length builds up to n substrings of up to n characters: O(n^3) time and O(n^2) space. If a duplicate of length L exists then so does one of every shorter length, so binary search on L with a rolling hash is O(n log n).

OPTIMAL SOLUTION · Binary search on the length with a rolling hash (Rabin-Karp)

TIME	SPACE
O(n log n) expected	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

FOLLOW-UP 2 The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

FOLLOW-UP 3 The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. **Cost:** O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find longest duplicated substring
#
# You are given a string s consisting of lowercase English
# letters. A **substring** of s is any contiguous sequence
# of characters, i.e., s[i..j) for 0 <= i < j <= len(s).
# Design an algorithm to find **any one** of the longest
# substrings that appears **at least twice** in s. The two
# occurrences may overlap. If no substring appears at
# least twice, return the empty string. ### Requirements -
# Input: a string s of length n (e.g., 1 <= n <= 2 × 10^5)
.
# - Output: a string representing one of the longest
# duplicated substrings. - Aim for an algorithm
# substantially better than the naive O(n^2) substring
# comparison approach. ### Examples - s = "banana" -
# Possible duplicated substrings: "a", "an", "ana" -
# Longest length is 3 ("ana"), so acceptable outputs
# include "ana". - s = "abcd" - No duplicated substring,
# so output should be "" (empty string).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Avoid hash collisions entirely.
# 2. The substring must appear at least k times, or must
# not overlap.
# 3. The string is huge and does not fit in memory.
#
# INTERVIEW NOTES
# If a substring of length L repeats, then a substring of
# every shorter length repeats too, so 'is there a
# duplicate of length L' is monotone and the answer length
# can be binary searched. Each check slides a window of
# length L over the string with a rolling hash and looks
# for a repeated hash, verifying the match to guard
# against collisions.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
MOD = (1 << 61) - 1


BASE = 131


def find_length(s, length):
    # a repeated window of this length? (rolling hash)
    codes = [ord(c) for c in s]
    power = pow(BASE, length, MOD)
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}
    for i in range(1, len(s) - length + 1):
        # slide the window: drop the left, add the right
        h = (h * BASE - codes[i - 1] * power
             + codes[i + length - 1]) % MOD
        # a hash match may be a collision: compare the
        # text
        for j in seen.get(h, ()):
            if s[j:j + length] == s[i:i + length]:
                return s[i:i + length]
        seen.setdefault(h, []).append(i)
    return None

# ---- main question: main function ----
# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
def longest_dup(s):
    # binary search the length: if length L repeats, so do
    # all shorter lengths
    lo, hi, best = 1, len(s) - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        found = find_length(s, mid)
        if found:
            best, lo = found, mid + 1
        else:
```

```
            hi = mid - 1
    return best

# ---- main question: tests ----
def test_main_question():
    assert longest_dup("banana") in ("ana",)
    assert longest_dup("abcd") == ""
    assert len(longest_dup("aaaaa")) == 4


# ============================================================
# FOLLOW-UP 1: Avoid hash collisions entirely.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use a suffix array with the LCP array: the
# longest duplicate is the largest LCP between neighbours
# in suffix order. Naive construction (sorting suffixes)
# is O(n^2 log n); real suffix array algorithms are O(n
# log n) or O(n).
def longest_dup_suffix(s):
    # no hashing: sort the suffixes, compare neighbours
    # (the
    # simple O(n^2 log n) version of a suffix array)
    suffixes = sorted(s[i:] for i in range(len(s)))
    best = ""
    for a, b in zip(suffixes, suffixes[1:]):
        k = 0
```

```
        while k < min(len(a), len(b)) and a[k] == b[k]:
            k += 1
        if k > len(best):
            best = a[:k]
    return best

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert len(longest_dup_suffix("banana")) == 3
    assert longest_dup_suffix("abcd") == ""
    for s in ("abracadabra", "mississippi", "abab"):
        fast = len(longest_dup(s))
        assert fast == len(longest_dup_suffix(s))


# ============================================================
# FOLLOW-UP 2: The substring must appear at least k times,
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: At least k times: count equal hashes in the
# check (need k of them). Non-overlapping: when two
# windows match, require their start positions to differ
# by at least L.
def longest_repeated(s, times=2, non_overlapping=False):
    # at least `times` occurrences; optionally not
    # overlapping
    best = ""
    n = len(s)
    for length in range(1, n):
        spots = {}
        for i in range(n - length + 1):
            piece = s[i:i + length]
            spots.setdefault(piece, []).append(i)
        ok = None
        for text, starts in spots.items():
            if non_overlapping:
                # greedy: count starts at least `length`
                # apart
                taken, last = 0, -length
                for p in starts:
                    if p - last >= length:
                        taken, last = taken + 1, p
            else:
                taken = len(starts)
            if taken >= times:
                ok = text
                break
        if ok is None:
            break
        best = ok
    return best

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # three occurrences needed
    assert longest_repeated("banana", 3) == "a"
    # overlapping copies are not allowed: "aa" cannot fit
    # in aaaa
    assert len(longest_repeated("aaaa", 2, True)) == 2
    assert len(longest_repeated("aaaa", 2)) == 3
```

```


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> str:
```

Sample call

```
solution('banana')
```

Test cases

Input	Expected
`('banana',)`	`'ana'`
`('abcd',)`	`''`

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 12 OF 43

Greedy: scan once, carry the best choice so far, act only when forced

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	5
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Greedy: scan once, carry the best choice so far, act only when forced | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	97. Maximum Profit	Exponent	no date
E	98. Alarm for the Next Upcoming Birthday	ColorStack	2025-10-07
E	99. Coins on the Table	ColorStack	2025-09-20
M	100. Visible People in a Line (Last Person, Then Everyone)	1Point3Acres	2026-10-07
M	101. Bounded Knapsack Variant	1Point3Acres	2026-10-16
M	102. Find the Largest Monotone Increasing Number	PracHub	2026-07-04
M	103. Min boards to cover roof holes	PracHub	2026-02-12
M	104. Jump Game II (Minimum Jumps)	ColorStack	2025-08-19

CODE TEMPLATE | GROUP 12 OF 43 | 8 QUESTIONS, ONE PATTERN

Greedy: scan once, carry the best choice so far, act only when forced

The one idea these questions share

Decide locally and never look back: sort or scan, carry one value (how far you can reach, how far you are covered, the largest seen so far), and take an action only at the moment the current choice stops being enough.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def min_cover(points, length):
    count, covered_to = 0, float("-inf")
    for p in sorted(set(points)):               # CHANGE 1: the order you scan in
        if p > covered_to:                      # CHANGE 2: not covered yet, so you must act
            count += 1
            covered_to = p + length - 1         # CHANGE 3: how far the new choice reaches
    return count
```

2. Helper functions that usually get written next to it

```
def circular_gap(day, today, cycle=365):        # how far ahead a date is, wrapping around the year
    return (day - today) % cycle


def farthest_reach(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
97	Maximum Profit	pick the cheapest and the dearest city with a fixed tie-break (min and max once)
98	Alarm for the Next Upcoming Birthday	minimum over people of the circular gap (their day - today) modulo 365, ties by name
99	Coins on the Table	scan the events with one counter: add on '+', subtract on '-' but never below 0
100	Visible People in a Line (Last Person, Then Everyone)	scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view
101	Bounded Knapsack Variant	sort items by value, highest first; take min(available count, remaining capacity) of each
102	Find the Largest Monotone Increasing Number	scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it
103	Min boards to cover roof holes	the template as written
104	Jump Game II (Minimum Jumps)	scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 97 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 1 of 8

Template fit: pick the cheapest and the dearest city with a fixed tie-break (min and max once) | no date

EASY | Exponent | 97 of 288

★ TOP 10 EASY: rank 2 among Easy problems

★ Maximum Profit

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 72 | 30 DAYS 61 | PRIORITY 6/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #121 Best Time to Buy and Sell Stock

LeetCode match: LC 121 Best Time to Buy and Sell Stock (likely) (inferred from short statement)

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | 4 interviews

Original: https://www.aced.io/questions/3630/maximum-profit

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values
Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

EXAMPLES

QUESTION

Example 1

Input	`{'A': 3, 'B': 10, 'C': 1}`
Output	`9`
Why	`Buy in the cheapest city (C, price 1) and sell in the dearest (B, price 10): profit 9.`

Example 2

Input	`{'A': 5}`
Output	`0`
Why	`With a single city there is nothing to trade.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(price, cities):
    best = (-INF, None, None)
    for buy in cities:
        for sell in cities:
            if buy == sell:
                continue
            profit = price[sell] - price[buy]
            best = max(best, (profit, buy, sell))
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

n^2 pairs. Since city order does not matter, the best pair is the cheapest city and the dearest city, found in one O(n) scan.

OPTIMAL SOLUTION · One pass for the cheapest and the dearest city

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The prices are a time series: you must buy before you sell (LeetCode 121).

Keep the minimum price seen so far and the best difference: O(n), O(1). The dictionary version ignores order, so the two extremes are enough.

FOLLOW-UP 2 Allow many trades (LeetCode 122) or a limit of k trades.

Many trades: add every positive day-to-day rise. At most k trades: DP over (trades used, holding or not), O(n * k).

FOLLOW-UP 3 Transaction fee or cooldown.

A two-state DP (holding, not holding) per day: hold = max(hold, free - price), free = max(free, hold + price - fee).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum Profit
#
# You and your friend have decided to start an
# international trading business. You want to earn a
# profit through some ol' fashioned arbitrage: You'll buy
# widgets in one country and sell them in another at a
# higher price. Your friend has already found the current
# market prices for widgets in cities around the world,
# and you'd like to use this data to pick which cities to
# buy and sell widgets in. Write a function
# max_profit(prices) that efficiently finds the two cities
# that maximize your profit (i.e. largest difference in
# prices). Input: A dictionary with cities as keys and
# prices as values Output: An array containing the names
# of the cities (min, max) If the prices dictionary is
# empty, return None/null (depending on the programming
# language used). If there is 0 profit to be earned,
# return None/null as well. If you are using C++, return
# {"", ""} in such cases.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The prices are a time series: you must buy before you
# sell (LeetCode 121).
# 2. Allow many trades (LeetCode 122) or a limit of k
# trades.
# 3. Transaction fee or cooldown.
#
# INTERVIEW NOTES
# Check whether order matters. A dictionary has none, so
# the profit is largest price minus smallest price, and
# the answer is just the two extremes. If this were a list
# of prices over time you would have to buy before you
# sell, which is a different one-pass problem with a
# running minimum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One pass for the cheapest and
# the dearest city
# Idea: Find the city with the lowest price and the city
# with the highest price in one scan (ties by name). They
# are the best pair because profit is sell minus buy and
# the order of cities does not matter.
def max_profit_prices(prices):
    # a dictionary city -> price: buy at the cheapest city
    # and
    # sell at the dearest, order does not matter
    if len(prices) < 2:
        return 0
    values = list(prices.values())
    return max(values) - min(values)

# ---- main question: tests ----
def test_main_question():
    cities = {"a": 10, "b": 4, "c": 9}
    assert max_profit_prices(cities) == 6
    assert max_profit_prices({"a": 3}) == 0


# ============================================================
# FOLLOW-UP 1: The prices are a time series: you must buy
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the minimum price seen so far and the best
# difference: O(n), O(1). The dictionary version ignores
# order, so the two extremes are enough.
def best_one_trade(series):
    # a time series: buy BEFORE you sell (LeetCode 121)
    low, best = float("inf"), 0
    for p in series:
        low = min(low, p)
        best = max(best, p - low)
    return best

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_one_trade([7, 1, 5, 3, 6, 4]) == 5
    assert best_one_trade([7, 6, 4, 3, 1]) == 0
```

```
# ============================================================
# FOLLOW-UP 2: Allow many trades (LeetCode 122) or a limit
# ============================================================

# ---- follow-up 2: function ----
# Answer: Many trades: add every positive day-to-day rise.
# At most k trades: DP over (trades used, holding or not),
# O(n * k).
def best_many_trades(series):
    # unlimited trades (LeetCode 122): take every rise
    pairs = zip(series, series[1:])
    return sum(max(0, b - a) for a, b in pairs)


# Answer: Many trades: add every positive day-to-day rise.
# At most k trades: DP over (trades used, holding or not),
# O(n * k).
def best_k_trades(series, k):
    # at most k trades: dp[t][holding]
    if not series or k == 0:
        return 0
    hold = [float("-inf")] * (k + 1)
    cash = [0] * (k + 1)
    for p in series:
        for t in range(1, k + 1):
            # buy with the cash of t - 1 finished trades,
```

```
            # or sell
            hold[t] = max(hold[t], cash[t - 1] - p)
            cash[t] = max(cash[t], hold[t] + p)
    return cash[k]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert best_many_trades([7, 1, 5, 3, 6, 4]) == 7
    assert best_k_trades([3, 2, 6, 5, 0, 3], 2) == 7
    assert best_k_trades([1, 2, 3], 1) == 2
    assert best_k_trades([], 3) == 0


# =====================================================
# FOLLOW-UP 3: Transaction fee or cooldown.
# =====================================================
```

```
# ---- follow-up 3: function ----
# Answer: A two-state DP (holding, not holding) per day:
# hold = max(hold, free - price), free = max(free, hold +
# price - fee).
def best_with_fee(series, fee):
    # a fee per trade: two states each day
    hold, cash = float("-inf"), 0
    for p in series:
        hold = max(hold, cash - p)
        cash = max(cash, hold + p - fee)
    return cash

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert best_with_fee([1, 3, 2, 8, 4, 9], 2) == 8


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values
Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example

OFFICIAL PAGE TEXT

```
Pythonprices = {'London': 72, 'New York': 70, 'Tokyo':
 67, 'Miami': 62}

max_profit(prices) # => ['Miami', 'London']
```

Hint 1You can think of this as two sub-problems: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return different cities?For our solution, we'll simply iterate through the prices list and simultaneously keep track of the minimum and maximum values we've seen so far. This only requires one iteration, so it's more efficient than the 'brute force' approach of calculating the profit for each pair of cities, which would require O(n^2) time. A more concise—but slightly less efficient solution—would be to sort the cities according to their prices and then return the first and last elements. In some languages like Python, you can also use the built-in min and max functions to compare dictionary values:

```
Pythondef max_profit(prices):

  # Check if the dictionary is empty
  if not prices:
    return None

  min_city = min(prices, key=prices.get)
  max_city = max(prices, key=prices.get)

  # Check if no profits earned
  if min_city == max_city:
    return None

  return [min_city, max_city]
```

Time Complexity: Iterating over prices.items() takes O(n) time, where n is the number of cities in the prices dictionary. Within the loop, there are constant-time operations, so the overall time complexity is O(n). Space Complexity: The space complexity is also O(1) because the size of the auxiliary array returned ([min_city, max_city]) is constant and independent of the input size.No answers contributed by the community yet.

STUDY PATH: question 98 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 2 of 8

Template fit: minimum over people of the circular gap (their day - today) modulo 365, ties by name | asked 378 days ago | frequency Single report

EASY | ColorStack | technical interview | 98 of 288

Alarm for the Next Upcoming Birthday

ASKED 2025-10-07 | 378 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 236/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

LeetCode match: Date arithmetic / cyclic minimum (related: LC 1360 Number of Days Between Two Dates)

Frequency: Single report | Last asked: 2025-10-07 | Roles: SWE Intern

QUESTION

You maintain a list of people and their birthdays. Given today's date and a list of (name, month, day) records, return the name of the person whose birthday comes soonest starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as 0 days away (soonest). Ignore leap years; treat every date simply as a (month, day) pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Function signature

```
def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:
    ...
```

EXAMPLES

Example 1

Input	`today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]`
Output	`"Alice"`
Why	`Alice's birthday is 2 days away; Carol's is later this year; Bob's is next January. Alice is soonest.`

Example 2

Input	`today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]`
Output	`"Dan"`
Why	`Dan's birthday is today (0 days away), which beats Alice's tomorrow.`

QUESTION

Example 3

Input	`today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]`
Output	`"Eve"`
Why	`Both birthdays already passed this year, so both are ~next-year and tie; alphabetical order picks "Eve".`

CONSTRAINTS

- 1 <= len(people) <= 10**4
- 1 <= month <= 12 , 1 <= day <= 31 (assume valid calendar dates). today is a valid (month, day) pair.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
MONTH_DAYS = [
    31,
    28,
    31,
    30,
    31,
    30,
    31,
    31,
    30,
    31,
    30,
    31,
]
def brute_next(today, people):
    month, day = today
    for _ in range(365):
        hits = [
            name
            for name, m, d in people
            if (m, d) == (month, day)
        ]
        if hits:
            return min(hits)
        day += 1
        if day > MONTH_DAYS[month - 1]:
            month, day = month % 12 + 1, 1
```

TIME	SPACE
O(365 * p)	O(p)

WHY IT IS SLOW

It checks every day of the year against every person.

OPTIMAL SOLUTION · Day of the year and a cyclic minimum

TIME	SPACE
O(p)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many queries with different dates. How do you make each one fast?

Sort the birthdays by (day of year, name) once. For a query binary search the first birthday at or after today's day; if there is none, wrap around to the first birthday in the list. O(log p) per query after an O(p log p) setup.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Alarm for the Next Upcoming Birthday
#
# You maintain a list of people and their birthdays. Given
# today's date and a list of (name, month, day) records,
# return the name of the person whose birthday comes
# soonest starting from today. Birthdays recur every year,
# so if no birthday remains later this calendar year, the
# soonest one is the earliest birthday next year. A
# birthday that falls exactly on today counts as 0 days
# away (soonest). Ignore leap years; treat every date
# simply as a (month, day) pair. If two people share the
# soonest birthday, return the one that is alphabetically
# first by name. Function signature: def
# next_birthday(today: tuple[int, int], people:
# list[tuple[str, int, int]]) -> str: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many queries with different dates. How do you make
# each one fast?
#
# INTERVIEW NOTES
# Turn each (month, day) into a day of the year with fixed
# month lengths (no leap years), then the number of days
# until the next occurrence is (their day - today's day)
# mod 365. The soonest birthday is the smallest of those,
# with the name as the tie-break.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
MONTH_DAYS = [
    31,
    28,
    31,
    30,
    31,
    30,
    31,
    31,
    30,
    31,
    30,
    31,
]

# ---- main question: main function ----
# BASE QUESTION (optimal): Day of the year and a cyclic
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def day_of_year(month, day):
    return sum(MONTH_DAYS[: month - 1]) + day


# BASE QUESTION (optimal): Day of the year and a cyclic
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def next_birthday(today, people):
    now = day_of_year(*today)
    return min(
        people,
        key=lambda p: (
            (day_of_year(p[1], p[2]) - now) % 365,
            p[0],
        ),
    )[0]

# ---- main question: tests ----
def test_main_question():
    random.seed(50)
    assert (
        next_birthday(
            (10, 7),
            [
                ("Alice", 10, 9),
                ("Bob", 1, 15),
                ("Carol", 12, 25),
            ],
        )
        == "Alice"
    )
    assert (
        next_birthday(
            (12, 31), [("Alice", 1, 1), ("Dan", 12, 31)]
        )
        == "Dan"
    )
    assert (
        next_birthday(
            (6, 1), [("Eve", 5, 30), ("Finn", 5, 30)]
        )
        == "Eve"
    )
    names = ["Ann", "Bo", "Cy", "Di", "Ed"]


# ============================================================
# FOLLOW-UP 1: Many queries with different dates. How do
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort the birthdays by (day of year, name) once.
# For a query binary search the first birthday at or after
# today's day; if there is none, wrap around to the first
# birthday in the list. O(log p) per query after an O(p
# log p) setup.
```

```
class BirthdayBook:
    def __init__(self, people):
        self.items = sorted(
            (day_of_year(m, d), name)
            for name, m, d in people
        )
        self.days = [d for d, _ in self.items]

    def next_birthday(self, today):
        at = bisect_left(self.days, day_of_year(*today))
        return self.items[at % len(self.items)][1]


# =============================================================
```

```
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 99 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 3 of 8

Template fit: scan the events with one counter: add on '+', subtract on '-' but never below 0 | asked 395 days ago | frequency Single report

EASY | ColorStack | technical interview | 99 of 288

Coins on the Table

ASKED 2025-09-20 | 395 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 234/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1700 / simulation (related: clamped counter)

Frequency: Single report | Last asked: 2025-09-20 | Roles: SWE Intern

QUESTION

There is a table that starts with 0 coins on it. You are given a string events , where each character is a single event applied left to right: '+' places one coin on the table. '-' removes one coin from the table. The table can never hold a negative number of coins: a '-' event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Function signature

def coins_on_table(events: str) -> int:

...

Follow-up (as reported): Generalize so each event is an integer delta (e.g. "+3" , "-2"), still clamping the running total at 0. Hint: parse tokens instead of single characters.

EXAMPLES

Example 1

Input	`events = "+++-"`
Output	`2`
Why	`+1, +1, +1, then -1 -> 2 coins remain.`

Example 2

Input	`events = "----"`
Output	`0`
Why	`Every removal happens on an empty table and is ignored.`

Example 3

Input	`events = "+-+-+"`
Output	`1`
Why	`1, 0, 1, 0, 1 -> 1 coin remains.`

CONSTRAINTS

- 0 <= len(events) <= 10^5
- Every character of events is either '+' or '-' .

QUESTION

- An empty string returns 0 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_coins(events):
    coins = []
    for e in events:
        if e == "+":
            coins.append(1)
        elif coins:
            coins.pop()
    return len(coins)
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It stores every coin although only their number matters.

OPTIMAL SOLUTION · One clamped counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each event is an integer delta like +3 or -2, still never below zero.

Split the string into tokens, convert each with int(), and apply total = max(0, total + delta). The clamp must happen after every event, not once at the end.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Coins on the Table
#
# There is a table that starts with 0 coins on it. You are
# given a string events , where each character is a single
# event applied left to right: '+' places one coin on the
# table. '-' removes one coin from the table. The table
# can never hold a negative number of coins: a '-' event
# when the table is empty has no effect (it is ignored).
# After processing every event, return the number of coins
# on the table. Function signature: def
# coins_on_table(events: str) -> int: ... Follow-up (as
# reported): Generalize so each event is an integer delta
# (e.g. "+3" , "-2" ), still clamping the running total at
# 0. Hint: parse tokens instead of single characters.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Each event is an integer delta like +3 or -2, still
# never below zero.
#
# INTERVIEW NOTES
# Each event changes a counter by one, with a floor at
# zero: total = max(0, total + delta). One pass is enough.
# For the integer-delta follow-up the only change is
# parsing tokens such as +3 and -2 instead of single
# characters.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One clamped counter
# Idea: total = max(0, total + delta) for every event.
def coins_on_table(events):
    total = 0
```

```
    for e in events:
        total = (
            total + 1 if e == "+" else max(0, total - 1)
        )
    return total

# ---- main question: tests ----
def test_main_question():
    random.seed(16)
    assert coins_on_table("+++-") == 2
    assert coins_on_table("----") == 0
    assert coins_on_table("+-+-+") == 1
    assert coins_on_table("") == 0


# ============================================================
# FOLLOW-UP 1: Each event is an integer delta like +3 or
# ============================================================

# ---- follow-up 1: function ----
# Answer: Split the string into tokens, convert each with
# int(), and apply total = max(0, total + delta). The
# clamp must happen after every event, not once at the
# end.
def coins_with_deltas(events):
    total = 0
    for token in events.split():
        total = max(0, total + int(token))
    return total

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        s = "".join(
            random.choice("+-")
            for _ in range(random.randint(0, 12))
        )
        assert coins_with_deltas(
            " ".join(
                ("+1" if c == "+" else "-1") for c in s
```

```
            )
        ) == coins_on_table(s)
    assert coins_with_deltas("+3 -2 -5 +4") == 4
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 100 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 4 of 8

Template fit: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view | asked 13 days ago | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 100 of 288

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 211/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people. Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person,
    height 5, can see 3, 4, 2, 8, 9, 10).
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3) for everyone, O(n^2) for the last person	O(1)

BRUTE FORCE

WHY IT IS SLOW

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Visible People in a Line (Last Person,
# Then Everyone)
#
# Given heights of people in a line, A can see B if nobody
# between them is taller than both. Part 1: count how many
# people the last person can see. Part 2: return that
# count for every person. Written in a plain editor with
# no compilation. Input: an array of heights representing
# a line of people. Visibility rule: person A can see
# person B if no one standing between them is taller than
# both A and B. Part 1: return how many people the last
# person in line can see. Part 2: for the same array,
# return, for every person in the line, how many people
# they can see. Examples: [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
# -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9,
# 10).
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 2: return the number of people each person can
# see.
# 2. How do ties and the direction of sight change the
# answer?
#
# INTERVIEW NOTES
# Person A sees person B when nobody strictly between them
# is taller than both, which means the tallest person
# between them is at most as tall as the taller of A and
# B. Walking away from the last person and keeping the
# running maximum of everyone passed lets you test each
# person in O(1). For everyone at once the visible people
# to the right form a chain: everything up to the first
# taller person, then only the people who are at least as
# tall as everyone before them. A monotonic stack builds
# those chains for all positions in O(n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count

# ---- main question: tests ----
def test_main_question():
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
    assert last_sees(line) == 6


# ============================================================
# FOLLOW-UP 1: Part 2: return the number of people each
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_right(a):
```

```
    """Part 2, looking right: everyone up to the first
    strictly taller person is visible, then only people at
    least as tall as everyone from that person on."""
    n = len(a)
```

```
    next_ge, next_gt = [None] * n, [None] * n
    stack = []
    for k in range(n - 1, -1, -1):
        # next person at least as tall as person k
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        next_ge[k] = stack[-1] if stack else None
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        # next person strictly taller than person k
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        next_gt[k] = stack[-1] if stack else None
        stack.append(k)
    chain = [0] * n
    for k in range(n - 1, -1, -1):
        # how many "at least as tall" steps follow k
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    out = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            out.append(n - 1 - i)
        else:
            out.append(t - i + chain[t])
    return out


# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    return [x + y for x, y in zip(left, see_right(a))]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
    assert see_all(line)[-1] == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    # equal heights never block each other
    assert last_sees([5, 5, 5]) == 2
```

```
    assert see_all([5, 5, 5]) == [2, 2, 2]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

STUDY PATH: question 101 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 5 of 8

Template fit: sort items by value, highest first; take min(available count, remaining capacity) of each | asked 4 days ago | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 101 of 288

Bounded Knapsack Variant

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 210/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1710 Maximum Units on a Truck

Frequency: Single report | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/unit-weight-bounded-knapsack

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

EXAMPLES

Example 1	
Input	`[(10, 2), (6, 3), (3, 5)], 4`
Output	`32`
Why	`Items are (value, available count), each weighs 1. Take both 10s and two 6s: 20 + 12 = 32.`

Example 2	
Input	`[(10, 2), (6, 3), (3, 5)], 7`
Output	`44`
Why	`A bigger capacity takes more items.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def knapsack(items, W):
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]
```

BRUTE FORCE

TIME	SPACE
O(total copies * W)	O(W)

WHY IT IS SLOW

Every copy is its own pass over the table, O(total copies * W). With unit weights the most valuable items are always the right ones, so sorting by value and taking greedily is O(n log n).

OPTIMAL SOLUTION · Sort by value and take greedily

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is greedy correct here but not for a general knapsack?

With unit weights each chosen item uses exactly one slot, so the only thing that matters is value per slot, which is just the value. If an optimal bag held a cheaper item while a dearer one was available, swapping them keeps the weight and raises the value, so the optimal bag can always be rearranged into the greedy one. With different weights a cheap item can leave room for several others, so value per weight and the leftover space interact and greedy fails.

FOLLOW-UP 2 What if the weights are not all 1?

Then it is a real bounded knapsack and greedy no longer works; use DP over the capacity. Splitting each count into 1, 2, 4, ... copies keeps the number of 0/1 items at about log(count) per type, so the cost is O(n * log(max count) * W) instead of O(total copies * W).

FOLLOW-UP 3 W is up to 10^9. Which version still works?

Only the greedy ones: sorting is O(n log n) and quickselect O(n), and neither depends on W, while any table over the capacity needs O(W) memory. The loop simply stops when the remaining capacity reaches zero.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Bounded Knapsack Variant
#
# Each item type has a value and an available count; every
```

```
# item weighs 1. Given capacity W, maximize total value.
# DP or DFS + memo works at O(n·W), greedy by value after
# sorting is O(n log n), and the best answer uses
# quickselect for O(n). Input: for each item type, its
# value and the number of copies available. Every item has
# weight 1. Input: a maximum total weight W. Output: the
# maximum total value achievable without exceeding W.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is greedy correct here but not for a general
# knapsack?
# 2. What if the weights are not all 1?
# 3. W is up to 10^9. Which version still works?
#
# INTERVIEW NOTES
# The word knapsack points at DP, but the statement says
# every item weighs 1. With equal weights every slot in
# the bag is interchangeable, so an item that is worth
# more is never worse than one that is worth less:
# swapping a cheaper item for a dearer one always helps.
# Take item types by value, highest first, until W slots
# are used. You do not even need the full sort:
# quickselect finds where the W-th slot falls in O(n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
```

```
# times that. Stop when the bag is full.
def max_value_sorted(items, W):
    """Unit weights: every slot is worth the same, so the
    best value always goes in first."""
    total = 0
    for value, count in sorted(items, reverse=True):
        # take as many copies as still fit in the bag
        take = min(count, W)
        total += value * take
        W -= take
        if W == 0:
            break
    return total


# SAME answer in O(n): quickselect
def max_value_select(items, W):
    """Same answer in O(n) expected: quickselect finds
    where the W-th slot falls without a full sort."""
    total, pool = 0, list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = [it for it in pool if it[0] > pivot]
        mid = [it for it in pool if it[0] == pivot]
        low = [it for it in pool if it[0] < pivot]
        high_count = sum(c for _, c in high)
        if high_count >= W:
```

```
            # the bag fills up inside the dearer types
            pool = high
            continue
        # every dearer copy fits, then fill from the pivot
        total += sum(v * c for v, c in high)
        W -= high_count
        take = min(W, sum(c for _, c in mid))
        total += pivot * take
        W -= take
        pool = low
    return total

# ---- main question: tests ----
def test_main_question():
    demo = [(3, 2), (5, 1), (4, 4)]
    assert max_value_sorted(demo, 4) == 17
    assert max_value_select(demo, 4) == 17
    assert max_value_sorted([], 5) == 0
```

```
    assert max_value_sorted([(7, 3)], 0) == 0
    assert max_value_sorted([(7, 3)], 10) == 21
    mixed = [(10, 5, 1), (6, 3, 2)]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: What if the weights are not all 1?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Then it is a real bounded knapsack and greedy no
# longer works; use DP over the capacity. Splitting each
# count into 1, 2, 4, ...
def knapsack_bounded(items, W):
    """Weights differ: greedy fails, so use DP. Splitting
    each count into 1, 2, 4, ... copies keeps it small."""
    best = [0] * (W + 1)
    for value, weight, count in items:
        chunk = 1
        while count > 0:
            take = min(chunk, count)
            count -= take
            cost, gain = take * weight, take * value
            for w in range(W, cost - 1, -1):
                best[w] = max(best[w],
                              best[w - cost] + gain)
            chunk *= 2
    return best[W]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    mixed = [(10, 5, 1), (6, 3, 2)]
    assert knapsack_bounded(mixed, 6) == 12
    # greedy is only valid when every weight is 1
    unit = [(3, 1, 2), (5, 1, 1), (4, 1, 4)]
    assert knapsack_bounded(unit, 4) == 17


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

STUDY PATH: question 102 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 6 of 8

Template fit: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it | asked 108 days ago

MEDIUM | PracHub | Technical Screen | 102 of 288

Find the Largest Monotone Increasing Number

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 195/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: the interview site's own solution

LeetCode match: LC 738 Monotone Increasing Digits (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/find-the-largest-monotone-increasing-number

QUESTION

Given an integer n, return the largest number not greater than n whose digits never decrease from left to right (for example 1223 qualifies, 1232 does not).

Function signature

def largest_monotone(n: int) -> int

EXAMPLES

Example 1	
Input	`10`
Output	`9`
Why	`10 itself has a drop (1 then 0); the largest valid number below it is 9.`

Example 2	
Input	`1234`
Output	`1234`
Why	`1234 is already monotone.`

Example 3	
Input	`332`
Output	`299`
Why	`332: lower the first 3 to 2 and fill the rest with 9s: 299.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    while not is_non_decreasing(str(n)):
        n -= 1
    return n
```

BRUTE FORCE

TIME	SPACE
O(n * d)	O(1)

WHY IT IS SLOW

n can be 10^9, so it may take a billion steps. A right-to-left scan that lowers a digit where order breaks and fills the rest with 9s is O(digits).

OPTIMAL SOLUTION · Right-to-left scan, then fill with nines

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The number is given as a string with 100,000 digits.

The scan works directly on the digit list, so keep it as a list of characters and never convert to an integer until the end (or not at all). The algorithm is O(d).

FOLLOW-UP 2 Return the smallest monotone number that is at least n instead.

Scan left to right for the first digit that is larger than its right neighbour and copy that digit over the whole suffix (332 becomes 333). The prefix before it is already monotone.

FOLLOW-UP 3 Digits must be strictly increasing.

Same idea, but the suffix to fill is the largest strictly increasing run (for example ...6789), and you must check that the prefix leaves enough room. Say the extra care and write the check as a brute-force comparison first.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Largest Monotone Increasing
# Number
#
# Given an integer n, return the largest number not
# greater than n whose digits never decrease from left to
# right (for example 1223 qualifies, 1232 does not).
#
# Signature:
#   def largest_monotone(n: int) -> int
#
# Examples:
```

```
# 10  ->  9
# 1234  ->  1234
#
# FOLLOW-UPS (answered below, in this order):
# 1. The number is given as a string with 100,000 digits.
# 2. Return the smallest monotone number that is at least
# n instead.
# 3. Digits must be strictly increasing.
#
# INTERVIEW NOTES
# If a digit is smaller than the one before it, the
# earlier digit has to drop by one and everything after
# that point can become 9 (as large as possible). Dropping
# a digit can break the pair before it, so scan from the
# right: each repair is checked against the digit to its
# left. That is why one right-to-left pass is enough.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Right-to-left scan, then fill
# with nines
# Idea: Walk i from the last digit to the first. When
# digit[i - 1] > digit[i], decrease digit[i - 1] by one
```

```
# and remember i as the start of the nines.
def largest_monotone(n):
    # largest number <= n whose digits never decrease
    d = list(str(n))
    mark = len(d)
    # scan from the right; a drop forces the left digit
    # down by
    # one and everything after it up to 9
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            mark = i
    for i in range(mark, len(d)):
        d[i] = "9"
    return int("".join(d))

# ---- main question: tests ----
def test_main_question():
    assert largest_monotone(10) == 9
    assert largest_monotone(1234) == 1234
    assert largest_monotone(332) == 299
    assert largest_monotone(0) == 0


# ============================================================
# FOLLOW-UP 1: The number is given as a string with
# ============================================================

# ---- follow-up 1: function ----
# Answer: The scan works directly on the digit list, so
# keep it as a list of characters and never convert to an
# integer until the end (or not at all). The algorithm is
# O(d).
def largest_monotone_string(digits):
    # the same on a string of any length (no big integers)
    d = list(digits)
    mark = len(d)
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = chr(ord(d[i - 1]) - 1)
            mark = i
    d[mark:] = "9" * (len(d) - mark)
    # a leading zero (from 10 -> 09) is dropped
    return "".join(d).lstrip("0") or "0"
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_monotone_string("332") == "299"
    assert largest_monotone_string("10") == "9"


# ============================================================
# FOLLOW-UP 2: Return the smallest monotone number that is
# ============================================================

# ---- follow-up 2: function ----
# Answer: Scan left to right for the first digit that is
# larger than its right neighbour and copy that digit over
# the whole suffix (332 becomes 333). The prefix before it
# is already monotone.
def smallest_monotone_at_least(n):
    # follow-up: smallest monotone number >= n
    d = list(str(n))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            # copy the bigger digit over the rest: 332 ->
            # 333
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert smallest_monotone_at_least(332) == 333
    assert smallest_monotone_at_least(120) == 122
    for n in range(0, 3000):
        every = range(n + 1)
        want = max(x for x in every if is_monotone(x))
        assert largest_monotone(n) == want
        text = largest_monotone_string(str(n))
        assert text == str(want)
        later = range(n, 99999)
        up = next(x for x in later if is_monotone(x))
        assert smallest_monotone_at_least(n) == up
    # a 100,000 digit input is fine as a string
    big = "9" * 50000 + "1" * 50000
    assert largest_monotone_string(big)[:3] == "899"


# ============================================================
```

```
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def is_monotone(n):
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
largest_monotone_increasing(0)
```

Test cases

Input	Expected
(0,)	0
(9,)	9

STUDY PATH: question 103 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 7 of 8

Template fit: the template as written | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 103 of 288

Min boards to cover roof holes

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 171/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #452 Minimum Number of Arrows to Burst Balloons

LeetCode match: No direct match; related LC 1326 Minimum Number of Taps / LC 452 (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-05)

Original: https://prachub.com/coding-questions/min-boards-to-cover-roof-holes

QUESTION

Holes in a roof are at integer positions along a line (the same position may be listed twice). A board of the given length covers that many consecutive positions. Return the fewest boards that cover every hole.

Function signature

def min_boards(holes: list, length: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 4, 7, 8], 3`
Output	`3`
Why	`Boards cover [1..3], [4..6] and [7..9].`

Example 2	
Input	`[1, 10], 5`
Output	`2`
Why	`The holes are far apart: one board each.`

Example 3	
Input	`[5, 5, 5], 1`
Output	`1`
Why	`Repeated positions need only one board.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (len(holes) + 1)
best[0] = 0
for i in range(1, len(holes) + 1):
    for j in range(i):
        span = holes[i - 1] - holes[j] + 1
        if span <= L:  # one board covers j..i-1
            best[i] = min(best[i], best[j] + 1)
answer = best[len(holes)]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every i tries every j, O(n^2). Starting a board at the leftmost uncovered hole and skipping everything it covers is optimal and O(n log n) including the sort.

OPTIMAL SOLUTION · Greedy: start a board at the leftmost uncovered hole

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

FOLLOW-UP 2 Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

FOLLOW-UP 3 Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Min boards to cover roof holes
#
# Holes in a roof are at integer positions along a line
# (the same position may be listed twice). A board of the
# given length covers that many consecutive positions.
# Return the fewest boards that cover every hole.
#
# Signature:
#   def min_boards(holes: list, length: int) -> int
#
# Examples:
# [1, 2, 4, 7, 8], 3  ->  3
# [1, 10], 5  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. You have only K boards: what is the smallest board
# length that covers all holes?
# 2. Boards of different lengths with a cost each.
# 3. Holes are in a grid and boards are 1 x L
# horizontally.
#
# INTERVIEW NOTES
# Cover the leftmost hole first: any board that covers it
# can be slid right until it starts exactly at that hole
# without uncovering anything to its right, so start a
# board at the leftmost uncovered hole and skip everything
# it covers. Repeating that is optimal because starting
# later never covers more.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Greedy: start a board at the
```

```
# leftmost uncovered hole
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + L; skip those and repeat.
def min_boards(holes, length):
    # greedy: the leftmost uncovered hole starts a board,
    # which
    # then covers `length` positions
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
        if p > covered_to:
            count += 1
            covered_to = p + length - 1
    return count
```

```
# ---- main question: tests ----
def test_main_question():
    assert min_boards([1, 2, 4, 5, 9], 3) == 3
    assert min_boards([], 3) == 0
    assert min_boards([5, 5, 5], 1) == 1


# ============================================================
# FOLLOW-UP 1: You have only K boards: what is the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Binary search the length L and test with the
# greedy (is the number of boards at most K?); feasibility
# is monotone in L. O(n log range).
def smallest_length(holes, k):
    # only k boards: the shortest board length that still
    # covers everything (more length never hurts: binary
    # search)
    pts = sorted(set(holes))
    if not pts:
        return 0
    lo, hi = 1, pts[-1] - pts[0] + 1
    while lo < hi:
        mid = (lo + hi) // 2
        if min_boards(pts, mid) <= k:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    # with 2 boards the shortest length is 5 (covers 1..5,
    # 9..13)
    assert smallest_length([1, 2, 4, 5, 9], 2) == 5
    assert smallest_length([1, 2, 4, 5, 9], 1) == 9
    assert smallest_length([], 1) == 0


# ============================================================
# FOLLOW-UP 2: Boards of different lengths with a cost
# ============================================================

# ---- follow-up 2: function ----
# Answer: The greedy exchange argument breaks. Use DP over
# the sorted holes: best[i] = min over board types of
# best[j] + cost, where j is the first hole not covered by
# that board starting at hole i.
def min_cost(holes, boards):
    # boards = [(length, cost)], any number of each: a DP
    # over
    # the sorted holes (the greedy no longer works)
    pts = sorted(set(holes))
    INF = float("inf")
    best = [0] * (len(pts) + 1)
    for i in range(len(pts) - 1, -1, -1):
        best[i] = INF
        for length, cost in boards:
            # the first hole this board does not reach
            j = bisect_left(pts, pts[i] + length)
            best[i] = min(best[i], cost + best[j])
    return best[0]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # a long board costs 5, two short ones cost 3 each
    assert min_cost([1, 2, 3, 10], [(3, 5), (1, 3)]) == 8
    assert min_cost([1, 2, 3, 4], [(2, 3), (4, 5)]) == 5
    roof = ["#.#.", "..##", "...."]
```

```
# ============================================================
# FOLLOW-UP 3: Holes are in a grid and boards are 1 x L
# ============================================================
```

```
# ---- follow-up 3: function ----
# Answer: Apply the greedy independently on every row. If
# boards may also be vertical it becomes a covering
# problem on a bipartite graph (König's theorem).
def boards_per_row(grid, length):
    # holes in a grid ('#'), horizontal boards: greedy per
    # row
    total = 0
    for row in grid:
        holes = [c for c in range(len(row))
                 if row[c] == "#"]
        total += min_boards(holes, length)
    return total

# ---- follow-up 3: tests ----
def test_follow_up_3():
    roof = ["#.#.", "..##", "...."]
    assert boards_per_row(roof, 2) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(roof: list[list[int]]) -> int:
SAMPLE INPUT
[[1,0,1],[0,1,1]]
```

Test cases

Input	Expected
[[1,0,1],[0,1,1]]	2
[[1,1],[1,1]]	0

STUDY PATH: question 104 of 288 | topic 12 of 43: Greedy: scan once, carry the best choice so far, act only when forced | 8 of 8

Template fit: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest | asked 427 days ago | frequency Single report

MEDIUM | IN APP LC 45 | ColorStack | technical interview | 104 of 288

Jump Game II (Minimum Jumps)

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 215/288

ABOUT THIS QUESTION

Topic: Greedy: scan once, carry the best choice so far, act only when forced

Optimal answer from: your app's interview approach (LeetCode #45)

LeetCode match: LC 45 Jump Game II

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

You are given a 0-indexed array of non-negative integers nums . You start at index 0. From any index i , you can jump to any index in the range [i + 1, i + nums[i]] (i.e., nums[i] is the maximum forward jump length from i). Return the minimum number of jumps required to reach the last index len(nums)

- 1 . The test cases guarantee that the last index is always reachable.

Function signature

def jump(nums: list[int]) -> int:

"""Return the minimum number of jumps to reach the last index."""

...

Follow-up (as reported): Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

EXAMPLES

Example 1	
Input	`nums = [2, 3, 1, 1, 4]`
Output	`2`
Why	`Jump 1 step from index 0 to 1, then 3 steps to the last index.`

Example 2	
Input	`nums = [2, 3, 0, 1, 4]`
Output	`2`
Why	`0 -> 1 -> 4.`

Example 3	
Input	`nums = [0]`
Output	`0`
Why	`Already at the last index.`

CONSTRAINTS

QUESTION

- 1 <= len(nums) <= 10^4
- 0 <= nums[i] <= 1000
- If the array has length 1 you are already at the last index, so the answer is 0.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_jump(nums):
    n = len(nums)
    best = [float("inf")] * n
    best[0] = 0
    for i in range(1,_n):
        for j in range(i):
            if j + nums[j] >= i:
                best[i] = min(
                    best[i], best[j] + 1
                )
    return best[-1]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every index looks back at every earlier index.

OPTIMAL SOLUTION · Greedy by ranges

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual indexes you jump through, not just the count.

Remember for each range the index that reached farthest. When the range ends, append that index to the path. The path still costs O(n) time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Jump Game II (Minimum Jumps)
#
# You are given a 0-indexed array of non-negative integers
# nums . You start at index 0. From any index i , you can
# jump to any index in the range [i + 1, i + nums[i]]
# (i.e., nums[i] is the maximum forward jump length from i
# ). Return the minimum number of jumps required to reach
# the last index len(nums) - 1 . The test cases guarantee
```

```
# that the last index is always reachable. Function
# signature: def jump(nums: list[int]) -> int: """Return
# the minimum number of jumps to reach the last index."""
# ... Follow-up (as reported): Solve it in O(n) time and
# O(1) extra space with a greedy "current reach vs.
# farthest reach" scan rather than O(n) DP.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual indexes you jump through, not just
# the count.
#
# INTERVIEW NOTES
# Each jump lands in a range of reachable indexes, so this
# is a BFS over ranges: the first jump covers [1,
# nums[0]], the second jump covers everything reachable
# from that range, and so on. Greedy tracks the end of the
# current range and the farthest index seen; reaching the
# range end forces one more jump.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Greedy by ranges
# Idea: Each jump extends the covered range to the
# farthest index seen so far.
```

```
def jump(nums):
    jumps, current_end, farthest = 0, 0, 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            # the range of the current jump is used up:
            # take another
            jumps += 1
            current_end = farthest
    return jumps

# ---- main question: tests ----
def test_main_question():
    random.seed(5)
    assert jump([2, 3, 1, 1, 4]) == 2
    assert jump([2, 3, 0, 1, 4]) == 2
    assert jump([0]) == 0


# ========================================================
# FOLLOW-UP 1: Return the actual indexes you jump through,
# ========================================================

# ---- follow-up 1: function ----
# Answer: Remember for each range the index that reached
# farthest. When the range ends, append that index to the
# path. The path still costs O(n) time.
def jump_path(nums):
    path, current_end, farthest, best_i = [0], 0, 0, 0
    for i in range(len(nums) - 1):
        if i + nums[i] > farthest:
            farthest, best_i = i + nums[i], i
```

```
        if i == current_end:
            current_end = farthest
            path.append(
                best_i if best_i != path[-1] else i
            )
    return path


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 13 OF 43

Sets and counters: remember the first collection, check the second against it

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	3	1
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sets and counters: remember the first collection, check the second against it | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	105. Exclude First-K-Unique From Another List	ColorStack	2025-10-17
E	106. Find the Duplicates	Exponent	no date
E	107. Positional Matches Between Two Lists	ColorStack	2025-10-15
E	108. Contains Duplicate	ColorStack	2025-09-16
M	109. Filter Common Elements Per Window (Two Lists)	ColorStack	2025-10-08
M	110. Reconstruct original array from doubled shuffle	PracHub	2026-01-04
M	111. Min deletions to avoid overlap in first k	PracHub	2026-02-12
H	112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space	PracHub	2026-09-09

CODE TEMPLATE | GROUP 13 OF 43 | 8 QUESTIONS, ONE PATTERN

Sets and counters: remember the first collection, check the second against it

The one idea these questions share

Build a set (or a Counter) from one collection, then walk the other and ask 'is this in it?'. Membership in a set is O(1), which is the whole trick. Duplicates, intersections, filters, and missing values are the same loop with a different question.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def keep_not_in(first, second):
    seen = set(first)                          # CHANGE 1: a set, or a Counter if counts matter
    return [x for x in second if x not in seen]     # CHANGE 2: keep, count, or stop at the first hit
```

2. Helper functions that usually get written next to it

```
def common_counts(a, b):                   # how many of each value the two lists share
    ca, cb = Counter(a), Counter(b)
    return {v: min(ca[v], cb[v]) for v in ca if v in cb}


def first_repeat(items):
    seen = set()
    for x in items:
        if x in seen:
            return x
        seen.add(x)
    return None
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
105	Exclude First-K-Unique From Another List	the set holds the first k distinct values of list 1; keep list 2's values that are not in it
106	Find the Duplicates	values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)
107	Positional Matches Between Two Lists	no set needed: zip both lists and keep a where a == b
108	Contains Duplicate	add values to a set as you go; the first value already in the set means a duplicate
109	Filter Common Elements Per Window (Two Lists)	build a set per window of the first list; keep the matching window of the second list minus that set
110	Reconstruct original array from doubled shuffle	Counter of the values; go from the smallest, pairing each x with 2x and using both counts up
111	Min deletions to avoid overlap in first k	Counter of the left part and of the right part; for each value in both, delete min(left count, right count)
112	Smallest Missing Positive Integer in Linear Time and Constant Extra Space	use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 105 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 1 of 8

Template fit: the set holds the first k distinct values of list 1; keep list 2's values that are not in it | asked 368 days ago | frequency Single report

EASY | ColorStack | technical interview | 105 of 288

Exclude First-K-Unique From Another List

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 29/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

LeetCode match: Set filtering (related: LC 349 Intersection of Two Arrays)

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

You are given two lists list1 and list2 and an integer k . Scan list1 left to right and collect the first k distinct values it contains (in order of first appearance; if list1 has fewer than k distinct values, take all of them). Then return list2 with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Function signature

def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:
    ...

EXAMPLES

Example 1	
Input	`list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2`
Output	`[2, 4, 5]`
Why	`First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from list2 leaves [2, 4, 5].`

Example 2	
Input	`list1 = [7], list2 = [7, 8, 9], k = 3`
Output	`[8, 9]`
Why	`list1 has only one distinct value (7); remove all 7s from list2.`

CONSTRAINTS

- 0 <= len(list1), len(list2) <= 10**5 , 1 <= k .
- Values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_exclude(list1, list2, k):
    picked = []
    for x in list1:
        if len(picked) == k:
            break
        if x not in picked:
            picked.append(x)
    return [
        y for y in list2 if y not in picked
    ]
```

TIME	SPACE
O(n + m * k)	O(k)

WHY IT IS SLOW

Each element of list2 searches the whole collected list linearly.

OPTIMAL SOLUTION · A set of the first k distinct values

TIME	SPACE
O(n + m)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 list2 is a huge stream. How do you avoid loading it?

Build the blocked set from list1 first (O(k) memory), then yield the stream's values lazily, skipping the blocked ones. The output is a generator.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Exclude First-K-Unique From Another List
#
# You are given two lists list1 and list2 and an integer k
# . Scan list1 left to right and collect the first k
# distinct values it contains (in order of first
# appearance; if list1 has fewer than k distinct values,
# take all of them). Then return list2 with every
# occurrence of any of those collected values removed,
# preserving the order of the remaining elements. Function
# signature: def exclude_first_k_unique(list1: list[int],
# list2: list[int], k: int) -> list[int]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. list2 is a huge stream. How do you avoid loading it?
#
# INTERVIEW NOTES
# Two steps: collect the first k distinct values of list1
```

```
# in a set (stopping as soon as the set has k values),
# then filter list2 by membership in that set. The set
# makes the filtering O(len(list2)).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A set of the first k distinct
# values
# Idea: Membership in a set makes the filter linear.
def exclude_first_k_unique(list1, list2, k):
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return [y for y in list2 if y not in blocked]

# ---- main question: tests ----
def test_main_question():
    random.seed(69)
    assert exclude_first_k_unique(
        [3, 3, 1, 4, 1, 5], [1, 2, 3, 4, 3, 5], 2
    ) == [2, 4, 5]
    assert exclude_first_k_unique(
        [7], [7, 8, 9], 3
    ) == [8, 9]


# ============================================================
# FOLLOW-UP 1: list2 is a huge stream. How do you avoid
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the blocked set from list1 first (O(k)
# memory), then yield the stream's values lazily, skipping
# the blocked ones. The output is a generator.
def exclude_stream(list1, stream, k):
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return (y for y in stream if y not in blocked)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 106 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 2 of 8

Template fit: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists) | no date

EASY | Exponent | 106 of 288

Find the Duplicates

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 28.2 | 30 DAYS 0 | PRIORITY 151/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #350 Intersection of Two Arrays II

LeetCode match: LC 350 Intersection of Two Arrays II

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +15 interviews

Original: https://www.aced.io/questions/3056/find-the-duplicates

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

EXAMPLES

QUESTION

Example 1	
Input	`[1, 3, 5, 7], [3, 4, 5, 8]`
Output	`[3, 5]`
Why	`Two sorted lists of passport numbers: 3 and 5 appear in both.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`[]`
Why	`Nothing in common.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr1, arr2):
    found = []
    for x in arr1:
        for y in arr2:
            if x == y and x not in found:
                found.append(x)
    return sorted(found)
```

TIME	SPACE
O(N * M)	O(1) extra

WHY IT IS SLOW

N * M comparisons. Two pointers on the sorted arrays are O(N + M), and binary searching the big array for each element of the small one is O(N log M).

OPTIMAL SOLUTION · Case 1 (M close to N): two pointers

TIME	SPACE
O(N + M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Case 2 (M much bigger than N): binary search the big array

TIME	SPACE
O(N log M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Three or more arrays.

Intersect pairwise, smallest arrays first so the running result shrinks quickly: O(total size). With K arrays you can also keep K pointers and advance the smallest.

FOLLOW-UP 2 The arrays are too big for memory.

Read both as streams (two pointers need only the current element of each), or partition both by value range into files that fit in memory and intersect partition by partition. **Cost:** O(N + M) time and O(1) memory with two streaming pointers; partitioning by value range multiplies the disk passes by the number of partitions.

FOLLOW-UP 3 Which one do you choose when N = 1,000 and M = 10^9?

Binary search: N log M is about 30,000 steps while two pointers need 10^9. The break-even is when N log M is around M.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Duplicates
#
# Given two sorted arrays arr1 and arr2 of passport
# numbers, implement a function findDuplicates that
# returns an array of all passport numbers that are both
# arrays. Note that the output array should be sorted in
# ascending order. Let N and M be the lengths of arr1 and
# arr2, respectively. Solve for two cases and analyze the
# time & space complexities of your solutions: M ~ N - the
# array lengths are approximately the same M >> N - arr2 i
s
# much bigger than arr1. Example input: arr1 = [1, 2, 3,
# 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7] #
# since only these three values are both in arr1 and
# arr2Hint 1Start with a brute force solution and optimize
# from there.Hint 2Note that the question asks for a
# sorted array.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Three or more arrays.
# 2. The arrays are too big for memory.
# 3. Which one do you choose when N = 1,000 and M = 10^9?
#
# INTERVIEW NOTES
# Both arrays are sorted, so the question is how the two
# sizes compare. If they are about the same size, walk
# both with two pointers and advance the smaller one. If
# one is much smaller, do not walk the big one: binary
# search each element of the small array in the big array.
# Always say the cost in terms of both sizes N and M.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left
from math import log2
import random


# ============================================================
# MAIN QUESTION
# ============================================================


# ---- main question: main function ----
# CASE 1: M close to N (two pointers)
def common_two_pointers(a, b):
    # M close to N: walk both sorted arrays together and
    # advance the smaller side. O(N + M) time, O(1) space
    # besides the output.
    out, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not out or out[-1] != a[i]:
                out.append(a[i])
            i += 1
            j += 1
    return out


# CASE 2: M much bigger than N (binary search)
def common_binary_search(small, big):
    # M much bigger than N: binary search every item of
    # the
    # short array in the long one. O(N log M); the search
    # starts where the previous one stopped.
    out, lo = [], 0
    for x in small:
        lo = bisect_left(big, x, lo)
        if lo == len(big):
            break
        if big[lo] == x and (not out or out[-1] != x):
            out.append(x)
    return out


# BASE QUESTION (optimal): Case 1 (M close to N): two
# pointers
# Idea: Advance whichever pointer points at the smaller
# value; when equal, record the value once (skip repeats)
# and advance both. Each element is passed at most once.
def common(a, b):
    # choose by cost: N log M against N + M
    small, big = (a, b) if len(a) <= len(b) else (b, a)
    if len(small) * log2(len(big) + 2) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)

# ---- main question: tests ----
def test_main_question():
    a = [1, 2, 3, 5, 6, 7]
    b = [3, 6, 7, 8, 20]
    assert common_two_pointers(a, b) == [3, 6, 7]
    assert common_binary_search(b, a) == [3, 6, 7]
    assert common(a, b) == [3, 6, 7]
    assert common([], [1]) == []
```

```
    assert common([1, 1, 2], [1, 1, 3]) == [1]


# =====================================================
# FOLLOW-UP 1: Three or more arrays.
# =====================================================

# ---- follow-up 1: function ----
# Answer: Intersect pairwise, smallest arrays first so the
# running result shrinks quickly: O(total size). With K
# arrays you can also keep K pointers and advance the
# smallest.
def common_many(arrays):
    # follow-up: three or more arrays. Intersect pairwise,
    # smallest first, so the running result shrinks fast.
    ordered = sorted(arrays, key=len)
    result = ordered[0]
    for arr in ordered[1:]:
        result = common(result, arr)
        if not result:
            break
    return result

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert common_many([[1, 2, 3], [7, 8], [3]]) == []
    assert common_many([[1, 2, 3, 4], [2, 3, 9],
                        [2, 3, 5, 6, 7]]) == [2, 3]


# =====================================================
# FOLLOW-UP 2: The arrays are too big for memory.
# =====================================================
```

```
# ---- follow-up 2: function ----
# Answer: Read both as streams (two pointers need only the
# current element of each), or partition both by value
# range into files that fit in memory and intersect
# partition by partition.
def common_stream(a_iter, b_iter):
    # follow-up: arrays too big for memory. Only the
    # current
    # item of each stream is held.
    a_iter, b_iter = iter(a_iter), iter(b_iter)
    x, y = next(a_iter, None), next(b_iter, None)
    last = None
    while x is not None and y is not None:
        if x < y:
            x = next(a_iter, None)
        elif x > y:
            y = next(b_iter, None)
        else:
            if x != last:
                yield x
                last = x
            x, y = next(a_iter, None), next(b_iter, None)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 2, 3, 5, 6, 7]
    b = [3, 6, 7, 8, 20]
    streamed = list(common_stream(iter(a), iter(b)))
    assert streamed == [3, 6, 7]
    for _ in range(300):
        nx = random.randint(0, 9)
        ny = random.randint(0, 9)
        x = sorted(random.sample(range(40), nx))
        y = sorted(random.sample(range(40), ny))
        want = sorted(set(x) & set(y))
        assert common_two_pointers(x, y) == want
        assert common_binary_search(x, y) == want
        assert common(x, y) == want
```

```
        assert list(common_stream(x, y)) == want
    # one short array against a very long one
    tiny, huge = [5, 999999], list(range(0, 1000000, 3))
    assert common(tiny, huge) == [999999]
```

```
# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example

```
input:  arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8,
 20]

output: [3, 6, 7] # since only these three values are
both in arr1 and arr2
```

Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices, let's say i without any loss of generality, points to a value that is smaller than the value pointed by other index, we increment i. If arr1[i] == arr2[j], we add the value to the output array and increment both indices. Pseudocode:

OFFICIAL PAGE TEXT

```
function findDuplicates(arr1, arr2):
   duplicates = []
   i = 0
   j = 0

   while (i < arr1.length AND j < arr2.length):
      if (arr1[i] == arr2[j]):
         duplicates.push(arr1[i])
         i = i + 1
         j = j + 1
      else if (arr1[i] < arr2[j]):
         i = i + 1
      else:
         j = j + 1

   return duplicates
```

Time Complexity: O(N+M) since in the worst case scenario we traverse both arrays once. Note that O(N+M) is a linear time complexity because the input size is O(N+M) as well. Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. Case 2 (M >> N) When one array is substantially longer than the other, we should try to avoid traversing the longer one. Instead, we can traverse the shorter array and look up its values in the longer array by using the binary search algorithm. We explain why this approach is superior in this case to the previous one in the complexity analysis section. Pseudocode:

```
function findDuplicates(arr1, arr2):
    duplicates = []

    for number in arr1:
        if binarySearch(arr2, number) != -1:
            duplicates.push(number);

    return duplicates

function binarySearch(arr, num):
    begin = 0
    end = arr.length - 1

    while (begin <= end):
        mid = begin + floor((end-begin)/2)
        if arr[mid] < num:
            begin = mid + 1
        else if arr[mid] == num:
            return mid
        else:
            end = mid - 1

    return -1
```

Time Complexity: we running a binary search on arr2 N times. Hence the time complexity is O(N*log(M)). So why is this algorithm better than the previous one when M >> N? To demonstrate that let's analyze the case that M = N^C, where C is a constant such that C >= 2. In this case, the time complexity of the second algorithm is:

OFFICIAL PAGE TEXT

```
O(N*log(M)) = O(N*log(N^C)) = O(C*N*log(N)) = O(N*log(
N))
```

And the time complexity of the first algorithm is:

```
O(N + M) = O(N + N^C) = O(N^C)
```

As we all know O(N^C) > O(N*log(N)). Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. No answers contributed by the community yet.

STUDY PATH: question 107 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 3 of 8

Template fit: no set needed: zip both lists and keep a where a == b | asked 370 days ago | frequency Single report

EASY | ColorStack | technical interview | 107 of 288

Positional Matches Between Two Lists

ASKED 2025-10-15 | 370 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 238/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

LeetCode match: Two-list comparison (related: LC 1346 / zip scan)

Frequency: Single report | Last asked: 2025-10-15 | Roles: SWE Intern

QUESTION

You are given two lists of equal length, first and second . Return, in order, the values at every position where the two lists agree — that is, all first[i] such that first[i] == second[i] .

Function signature

def positional_matches(first: list[int], second: list[int]) -> list[int]:
    ...

EXAMPLES

Example 1	
Input	first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]
Output	[5, 7]
Why	Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches" between a digits-of-pi list and an index list.)

Example 2	
Input	first = [1, 2, 3], second = [4, 5, 6]
Output	[]
Why	No position matches.

CONSTRAINTS

- len(first) == len(second) , 0 <= length <= 10**6 .
- Values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_positional(first, second):
    out = []
    for i in range(len(first)):
        if first[i] == second[i]:
            out.append(first[i])
    return out
```

BRUTE FORCE

TIME	SPACE
O(n)	O(matches)

WHY IT IS SLOW

Correct and linear; it is only longer than the zip form.

OPTIMAL SOLUTION · zip and a filter

TIME	SPACE
O(n)	O(matches)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the positions instead of the values, lazily for a huge input.

Use enumerate over zip inside a generator: nothing is stored, and the caller can stop early. Memory is O(1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Positional Matches Between Two Lists
#
# You are given two lists of equal length, first and
# second . Return, in order, the values at every position
# where the two lists agree — that is, all first[i] such
# that first[i] == second[i] . Function signature: def
# positional_matches(first: list[int], second: list[int])
# -> list[int]: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the positions instead of the values, lazily
# for a huge input.
#
# INTERVIEW NOTES
# One pass over both lists at the same time: keep first[i]
# whenever first[i] == second[i]. zip walks them together,
# so there is no indexing and no extra structure.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```python
# ---- main question: main function ----
# BASE QUESTION (optimal): zip and a filter
# Idea: Walk both lists together and keep the agreeing
# values.
def positional_matches(first, second):
    return [a for a, b in zip(first, second) if a == b]

# ---- main question: tests ----
def test_main_question():
    random.seed(64)
    assert positional_matches(
        [3, 1, 4, 1, 5, 5, 7, 9],
        [1, 2, 3, 4, 5, 6, 7, 8],
    ) == [5, 7]
    assert (
        positional_matches([1, 2, 3], [4, 5, 6]) == []
    )


# ========================================================
# FOLLOW-UP 1: Return the positions instead of the values,
# ========================================================

# ---- follow-up 1: function ----
# Answer: Use enumerate over zip inside a generator:
# nothing is stored, and the caller can stop early. Memory
# is O(1).
def matching_positions(first, second):
    for i, (a, b) in enumerate(zip(first, second)):
        if a == b:
            yield i

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        n = random.randint(0, 9)
        a = [random.randint(0, 3) for _ in range(n)]
        b = [random.randint(0, 3) for _ in range(n)]
        assert [
            a[i] for i in matching_positions(a, b)
        ] == positional_matches(a, b)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 108 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 4 of 8

Template fit: add values to a set as you go; the first value already in the set means a duplicate | asked 399 days ago | frequency Single report

EASY | IN APP LC 217 | ColorStack | technical interview | 108 of 288

Contains Duplicate

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 232/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: your app's interview approach (LeetCode #217)

LeetCode match: LC 217 Contains Duplicate

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Given an integer array nums , return True if any value appears at least twice in the array, and False if every element is distinct.

Function signature

```
def contains_duplicate(nums: list[int]) -> bool:
    """Return True if any value appears more than once."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 1]`
Output	`True`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`False`

Example 3	
Input	`nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
Output	`True`

CONSTRAINTS

- 1 <= len(nums) <= 10^5
- -10^9 <= nums[i] <= 10^9

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_duplicate(nums):
    for i in range(len(nums)):
        for j in range(i_+_1,_len(nums)):
            if nums[i] == nums[j]:
                return True
    return False
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each element is compared with all the later ones.

OPTIMAL SOLUTION · A set of seen values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if you cannot use extra memory?

Sort in place and compare neighbours: duplicates become adjacent. O(n log n) time and O(1) extra space (not counting the sort's own stack).

FOLLOW-UP 2 Only count it if the two equal values are at most k positions apart.

Keep a set of the last k values (a sliding window): add each new value, drop the one that left the window, and check membership before adding. O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Contains Duplicate
#
# Given an integer array nums , return True if any value
# appears at least twice in the array, and False if every
# element is distinct. Function signature: def
# contains_duplicate(nums: list[int]) -> bool: """Return
# True if any value appears more than once."""  ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if you cannot use extra memory?
# 2. Only count it if the two equal values are at most k
# positions apart.
#
# INTERVIEW NOTES
# A repeated value means the number of distinct values is
# smaller than the length. A set gives the answer in one
# pass, and sorting trades the memory for time.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
```

```
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A set of seen values
# Idea: Add each value to a set; seeing one again means a
# duplicate.
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False

# ---- main question: tests ----
```

```
def test_main_question():
    random.seed(7)
    assert contains_duplicate([1, 2, 3, 1]) is True
    assert contains_duplicate([1, 2, 3, 4]) is False
    assert contains_duplicate([]) is False


# ============================================================
# FOLLOW-UP 1: What if you cannot use extra memory?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort in place and compare neighbours: duplicates
# become adjacent. O(n log n) time and O(1) extra space
# (not counting the sort's own stack).
def contains_duplicate_sorted(nums):
    ordered = sorted(nums)
    return any(
        a == b for a, b in zip(ordered, ordered[1:])
    )

# ============================================================
# FOLLOW-UP 2: Only count it if the two equal values are
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep a set of the last k values (a sliding
# window): add each new value, drop the one that left the
# window, and check membership before adding. O(n).
def duplicate_within_k(nums, k):
    window = set()
    for i, x in enumerate(nums):
        if x in window:
            return True
        window.add(x)
        if len(window) > k:
            window.remove(nums[i - k])
    return False

# ---- follow-up 2: tests ----
def test_follow_up_2():
    for _ in range(300):
        nums = [
            random.randint(0, 8)
            for _ in range(random.randint(0, 9))
```

```
        ]
        k = random.randint(1, 5)
        slow = any(
            nums[i] == nums[j]
            for i in range(len(nums))
            for j in range(
                i + 1, min(len(nums), i + k + 1)
            )
        )
        assert duplicate_within_k(nums, k) == slow


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 109 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 5 of 8

Template fit: build a set per window of the first list; keep the matching window of the second list minus that set | asked 377 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 109 of 288

Filter Common Elements Per Window (Two Lists)

ASKED 2025-10-08 | 377 DAYS AGO | OLDER

STUDY: TALK | **ALL TIME 28.2** | **30 DAYS 0** | **PRIORITY 150/288**

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #350 Intersection of Two Arrays II

LeetCode match: LC 350 Intersection of Two Arrays II (related: per window)

Frequency: Single report | Last asked: 2025-10-08 | Roles: SWE Intern

QUESTION

You are given two lists a and b of equal length n and a window size k that divides n . Split both lists into consecutive, non-overlapping windows of size k (window 0 is indices 0..k-1 , window 1 is k..2k-1 , and so on). For each window, remove from b 's window every value that also appears in the corresponding window of a . Concatenate the surviving elements of b , preserving their original order, and return that list.

Function signature

def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:
    ...

Follow-up (as reported): The interviewer asked to do the removal in place / O(1) extra space. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

EXAMPLES

Example 1	
Input	`a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3`
Output	`[7, 8, 9]`
Why	`Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8]. Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].`

Example 2	
Input	`a = [1, 1], b = [1, 2], k = 2`
Output	`[2]`
Why	`Single window: a={1}, b=[1,2] -> remove 1 -> [2].`

CONSTRAINTS

QUESTION

- 0 <= n <= 10**5 , n % k == 0 , 1 <= k <= n (for n > 0).
- Values are integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_filter(a, b, k):
    out = []
    for i in range(0,_len(a),_k):
        window = a[i : i + k]
        out += [
            x
            for x in b[i:
                i + k]
            if x not in window
        ]
    return out
```

TIME	SPACE
O(n * k)	O(n)

WHY IT IS SLOW

Every element of b searches a whole window of a linearly.

OPTIMAL SOLUTION · A set per window

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it in place: no new output list.

Keep a write pointer w. For each window build the set of a's window, then copy each surviving element of b to b[w] and advance w; at the end truncate b to length w. Time O(n), extra space O(k) for one window's set.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Filter Common Elements Per Window (Two
# Lists)
#
# You are given two lists a and b of equal length n and a
# window size k that divides n . Split both lists into
# consecutive, non-overlapping windows of size k (window 0
# is indices 0..k-1 , window 1 is k..2k-1 , and so on).
# For each window, remove from b 's window every value
# that also appears in the corresponding window of a .
# Concatenate the surviving elements of b , preserving
# their original order, and return that list. Function
```

```
# signature: def filter_windows(a: list[int], b:
# list[int], k: int) -> list[int]: ... Follow-up (as
# reported): The interviewer asked to do the removal in
# place / O(1) extra space. Naive in-place removal becomes
# O(n*k); discuss compaction with a write pointer to keep
# it O(n) time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it in place: no new output list.
#
# INTERVIEW NOTES
# Each window is independent: put the window of a into a
# set and keep the elements of the matching window of b
# that are not in it. The set makes each membership test
# O(1), so the whole job is O(n) instead of O(n * k). In
# place, a write pointer compacts the survivors to the
# front.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A set per window
```

```
# Idea: Membership in the window of a is a set lookup.
def filter_windows(a, b, k):
    out = []
    for i in range(0, len(a), k):
        seen = set(a[i : i + k])
        out.extend(
            x for x in b[i : i + k] if x not in seen
        )
    return out

# ---- main question: tests ----
def test_main_question():
    random.seed(53)
    assert filter_windows(
        [1, 2, 3, 4, 5, 6], [3, 7, 8, 6, 9, 5], 3
    ) == [7, 8, 9]
    assert filter_windows([1, 1], [1, 2], 2) == [2]


# ============================================================
# FOLLOW-UP 1: Do it in place: no new output list.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a write pointer w. For each window build
# the set of a's window, then copy each surviving element
# of b to b[w] and advance w; at the end truncate b to
# length w. Time O(n), extra space O(k) for one window's
# set.
def filter_windows_in_place(a, b, k):
    w = 0
    for i in range(0, len(a), k):
        seen = set(a[i : i + k])
        for j in range(i, min(i + k, len(b))):
            if b[j] not in seen:
                b[w] = b[j]
                w += 1
    del b[w:]
    return b


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 110 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 6 of 8

Template fit: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up | asked 289 days ago

MEDIUM | PracHub | Technical Screen | 110 of 288

Reconstruct original array from doubled shuffle

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 147/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: the interview site's own solution

LeetCode match: LC 2007 Find Original Array From Doubled Array (inferred from overview)

Posted on PracHub: 2026-01-04 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/reconstruct-original-array-from-doubled-shuffle

QUESTION

An array of numbers was doubled: for every element x of the original, the value 2x was added, and then all the numbers were shuffled. Given the shuffled array changed, return an original array (in any order), or an empty list if changed cannot be a doubled array. Zero and negative numbers are possible.

Function signature

def find_original(changed: list) -> list

EXAMPLES

Example 1	
Input	`[1, 3, 4, 2, 6, 8]`
Output	`[1, 3, 4]`
Why	`Original [1, 3, 4] gives doubles [2, 6, 8].`

Example 2	
Input	`[1, 3, 2]`
Output	`[]`
Why	`An odd number of values cannot be doubled.`

Example 3	
Input	`[0, 0, 0, 0]`
Output	`[0, 0]`
Why	`Zeros pair with zeros.`

QUESTION

Example 4	
Input	`[-2, -1, -4, -2]`
Output	`[-1, -2]`
Why	`Negatives work too: -2 is the double of -1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(changed):
    items = sorted(changed, key=abs)
    originals = []
    while items:
        x = items.pop(0)
        if 2 * x not in items:
            return []
        items.remove(2 * x)
        originals.append(x)
    return originals
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

list.remove is a linear search, so each step is O(n) and the loop is O(n^2). A Counter processed in order of absolute value is O(n log n).

OPTIMAL SOLUTION · Counter processed in order of absolute value

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

FOLLOW-UP 2 Values are small non-negative integers.

Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

FOLLOW-UP 3 Why does processing by absolute value work with negatives?

For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reconstruct original array from doubled
# shuffle
#
# An array of numbers was doubled: for every element x of
# the original, the value 2x was added, and then all the
# numbers were shuffled. Given the shuffled array changed,
# return an original array (in any order), or an empty
# list if changed cannot be a doubled array. Zero and
# negative numbers are possible.
#
# Signature:
#   def find_original(changed: list) -> list
#
# Examples:
# [1, 3, 4, 2, 6, 8]  ->  [1, 3, 4]
# [1, 3, 2]  ->  []
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the originals in the order they appear in the
# input.
# 2. Values are small non-negative integers.
# 3. Why does processing by absolute value work with
# negatives?
#
# INTERVIEW NOTES
# The element with the smallest absolute value cannot be
# the double of anything, so it is an original; its double
# must be present too. Pair them off, take the next
# smallest, and so on. Process values in order of absolute
# value so negatives and zeros follow the same rule, and
# handle zero separately (it is its own double, so it
# needs an even count).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[x]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times.
def find_original(changed):
    # `changed` holds every original x and its double 2x,
    # in
    # any order. Smallest absolute value first: it can
    # only be
    # an original, never someone's double.
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            # zeros pair with each other
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        # every x needs its own 2x
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out

# ---- main question: tests ----
def test_main_question():
    got = sorted(find_original([1, 3, 4, 2, 6, 8]))
    assert got == [1, 3, 4]
    assert find_original([6, 3, 0, 1]) == []
    assert find_original([1]) == []
    assert find_original([0, 0, 0, 0]) == [0, 0]
    assert find_original([0, 0, 0]) == []
    # negative numbers: -2 is the double of -1
```

```
    neg = sorted(find_original([-1, -2, 4, 2]))
    assert neg == [-1, 2]


# ============================================================
# FOLLOW-UP 1: Return the originals in the order they
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute the multiset of originals, then scan the
# input once and keep each element while its remaining
# original count is positive.
def find_original_in_order(changed):
    # originals in the order they appear in the input
    originals = find_original(changed)
    if not originals and changed:
        return []
    left = Counter(originals)
    out = []
    for x in changed:
        if left[x] > 0:
            left[x] -= 1
            out.append(x)
    return out

# ---- follow-up 1: tests ----
def test_follow_up_1():
    ordered = find_original_in_order([4, 1, 2, 8, 2, 4])
    assert ordered == [4, 1, 2]


# ============================================================
```

```
# FOLLOW-UP 2: Values are small non-negative integers.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use a counting array instead of sorting and a
# Counter: process values from small to large, O(n +
# range).
def find_original_small(changed, limit):
    # values 0..limit: a counting array, O(n + limit)
    if len(changed) % 2:
        return []
    count = [0] * (2 * limit + 2)
    for x in changed:
```

```
        count[x] += 1
    out = []
    if count[0] % 2:
        return []
    out += [0] * (count[0] // 2)
    for x in range(1, limit + 1):
        if count[x]:
            if count[2 * x] < count[x]:
                return []
            out += [x] * count[x]
            count[2 * x] -= count[x]
    return out

# ---- follow-up 2: tests ----
def test_follow_up_2():
    small = sorted(find_original_small([1, 2, 4, 8], 8))
    assert small == [1, 4]
    assert find_original_small([1, 3], 3) == []


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(changed: list[int]) -> list[int]:
```

Sample call

```
solution([2, 1, 4, 8])
```

Test cases

Input	Expected
`([2, 1, 4, 8],)`	`[1, 4]`
`([-4, -2, -2, -1, 1, 2],)`	`[-2, -1, 1]`

STUDY PATH: question 111 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 7 of 8

Template fit: Counter of the left part and of the right part; for each value in both, delete min(left count, right count) | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 111 of 288

Min deletions to avoid overlap in first k

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 255/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (inferred) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-04-22)

Original: https://prachub.com/coding-questions/min-deletions-to-avoid-overlap-in-first-k

QUESTION

Split an array into its first k elements and the rest. Return the minimum number of elements to delete so that no value appears on both sides (for every shared value delete all copies from the side that has fewer).

Function signature

def min_deletions(a: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 2, 3, 2, 1], 3`
Output	`2`
Why	`Left [1, 2, 2], right [3, 2, 1]: value 1 appears once on each side (delete 1), value 2 appears twice on the left and once on the right (delete 1): 2 deletions.`

Example 2	
Input	`[1, 2, 3, 4], 2`
Output	`0`
Why	`No value is shared between [1, 2] and [3, 4].`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(a, k):
    for size in range(len(a) + 1):
        ids = range(len(a))
        for gone in combinations(ids, size):
            keep = [i for i in ids
                    if i not in gone]
            head = {a[i] for i in keep if i < k}
            tail = {a[i] for i in keep if i >= k}
            if not head & tail:
                return size
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n sets to try: exponential. Each shared value must disappear from one side, so the cost is the sum over shared values of the smaller of its two counts, in O(n).

OPTIMAL SOLUTION · Sum of min(count in the first k, count in the rest) over shared values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only elements of the first part may be deleted.

Then every shared value must be removed from the first part entirely, so the cost is the number of first-part elements whose value occurs in the rest (a set lookup per element).

FOLLOW-UP 2 Elements have weights (deleting some is more costly).

Replace counts by the sum of weights per side for each value; the choice per value is still independent, so take the cheaper side. **Cost:** unchanged O(n); the per-value choice sums weights instead of counts.

FOLLOW-UP 3 The statement may differ from this reading.

Ask whether deletions can come from both parts, whether duplicates count separately, and whether order matters. Everything above relies on those three answers.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Min deletions to avoid overlap in first k
#
# Split an array into its first k elements and the rest.
# Return the minimum number of elements to delete so that
# no value appears on both sides (for every shared value
# delete all copies from the side that has fewer).
#
# Signature:
#   def min_deletions(a: list, k: int) -> int
#
# Examples:
# [1, 2, 2, 3, 2, 1], 3  ->  2
# [1, 2, 3, 4], 2  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only elements of the first part may be deleted.
# 2. Elements have weights (deleting some is more costly).
# 3. The statement may differ from this reading.
#
# INTERVIEW NOTES
# Values do not interact with each other, so each value
# can be decided on its own. A value that appears on both
# sides must disappear from one side entirely, and the
# cheapest side is the one with fewer copies. The answer
# is the sum, over values that appear on both sides, of
# the smaller of the two counts.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sum of min(count in the first
# k, count in the rest) over shared values
# Idea: Count each value in the first k elements and in
```

```
# the rest. For every value with a positive count on both
# sides, one side has to lose all of its copies, so add
# the smaller count.
def min_deletions(a, k):
    # the values in a[:k] and a[k:] must not overlap; for
    # each
    # shared value delete the copies on the cheaper side
    left, right = Counter(a[:k]), Counter(a[k:])
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())


# PLAN: which indices to delete
def deletions_plan(a, k):
    # which indices to delete (both sides allowed)
    left, right = Counter(a[:k]), Counter(a[k:])
    out = []
    for v in left.keys() & right.keys():
        # delete the cheaper side's copies of the value
        side = range(k, len(a))
        if left[v] <= right[v]:
            side = range(k)
        out += [i for i in side if a[i] == v]
    return sorted(out)
```

```


# ---- main question: tests ----
def test_main_question():
    a = [1, 2, 2, 3, 2, 1]
    # first part [1, 2, 2], rest [3, 2, 1]: shared 1 and 2
    assert min_deletions(a, 3) == 1 + 1


# ============================================================
# FOLLOW-UP 1: Only elements of the first part may be
# ============================================================

# ---- follow-up 1: function ----
# Answer: Then every shared value must be removed from the
# first part entirely, so the cost is the number of first-
# part elements whose value occurs in the rest (a set
# lookup per element).
def min_deletions_first_only(a, k):
    # only the first part may lose elements: every first-
    # part
    # element whose value appears later must go
```

```
    rest = set(a[k:])
    return sum(1 for x in a[:k] if x in rest)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    a = [1, 2, 2, 3, 2, 1]
    assert min_deletions_first_only(a, 3) == 3
    assert min_deletions([1, 2, 3], 1) == 0
    assert min_deletions([], 0) == 0
    # a heavy copy on one side makes the other side
    # cheaper
    w = [1, 5, 5, 1, 1, 1]


# ============================================================
# FOLLOW-UP 2: Elements have weights (deleting some is
# ============================================================

# ---- follow-up 2: function ----
# Answer: Replace counts by the sum of weights per side
# for each value; the choice per value is still
# independent, so take the cheaper side. Cost: unchanged
# O(n); the per-value choice sums weights instead of
# counts.
def min_deletions_weighted(a, weights, k):
    # deleting element i costs weights[i]: per value take
    # the
    # cheaper side as a whole
    left, right = Counter(), Counter()
    for i, x in enumerate(a):
        (left if i < k else right)[x] += weights[i]
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())

# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 2, 2, 3, 2, 1]
    # a heavy copy on one side makes the other side
    # cheaper
    w = [1, 5, 5, 1, 1, 1]
    assert min_deletions_weighted(a, w, 3) == 1 + 1
    plan = deletions_plan(a, 3)
    kept = [x for i, x in enumerate(a) if i not in plan]
    kept_left = set(kept[:3 - sum(i < 3 for i in plan)])
    kept_right = set(kept[3 - sum(i < 3 for i in plan):])
```

```
    assert not kept_left & kept_right
    assert len(plan) == min_deletions(a, 3)


# ============================================================
# RUN EVERYTHING
```

```
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(list1: list[int], list2: list[int], k: int) -
> int:
```

Sample call

```
solution([1, 2, 3], [2, 1, 4, 5], 2)
```

Test cases

Input	Expected
([1, 2, 3], [2, 1, 4, 5], 2)	2
([1, 2, 3, 4], [2, 5, 3, 6], 2)	1

STUDY PATH: question 112 of 288 | topic 13 of 43: Sets and counters: remember the first collection, check the second against it | 8 of 8

Template fit: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer | asked 41 days ago

HARD | IN APP LC 41 | PracHub | Technical Screen | 112 of 288

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 48 30 DAYS 0 PRIORITY 84/288

ABOUT THIS QUESTION

Topic: Sets and counters: remember the first collection, check the second against it

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Smallest Missing Positive Integer in
# Linear Time and Constant Extra Space
#
# You are given an unsorted list of integers nums, which
# may contain negative numbers, zeros, duplicates, and
# values far larger than the length of the list. Return
# the smallest positive integer, that is the smallest
# integer that is at least 1, that does not appear in
# nums. In the interview, a first solution that ran in
# linear time but used extra memory proportional to the
# input was accepted as a starting point, and the
# interviewer then asked for the same result using only
# constant extra space. Aim for that tighter target. ###
# Function Signature python def
# first_missing_positive(nums: list[int]) -> int: ###
# Rules - Target complexity: O(n) time and O(1) auxiliary
# space, where n = len(nums). The list may be rearranged
# or overwritten in place; its contents after the call do
# not matter. - Return only the missing value.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without swapping (sign marking).
# 2. The array must not be modified.
# 3. Smallest missing non-negative integer, or the k-th
# missing positive.
#
# INTERVIEW NOTES
# The answer is between 1 and n + 1, because n numbers can
# cover at most n of the positive integers. So the array
# itself can be used as the 'seen' table: put each value v
# (1 <= v <= n) at index v - 1. After that, the first
# index i where nums[i] != i + 1 gives the answer i + 1.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Cyclic placement: put each
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# EASY VERSION: O(n) memory (say it first)
def first_missing_set(nums):
    # the easy O(n) memory answer (fine as a first
    # version)
    seen, v = set(nums), 1
    while v in seen:
        v += 1
    return v


# VARIANT: k-th missing positive
def kth_missing_positive(sorted_nums, k):
    # follow-up: k-th missing positive in a sorted array.
    # missing before index i is a[i] - (i + 1): binary
    # search
    lo, hi = 0, len(sorted_nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_nums[mid] - (mid + 1) < k:
            lo = mid + 1
        else:
            hi = mid
    return lo + k
```

```
# ---- main question: tests ----
def test_main_question():
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]


# ============================================================
# FOLLOW-UP 1: Do it without swapping (sign marking).
# ============================================================

# ---- follow-up 1: function ----
# Answer: Replace out-of-range values by n + 1, then for
# every value v in 1..n make the entry at index v - 1
# negative as a 'seen' flag; the first positive entry
# marks the missing number. Same O(n) time and O(1) space.
def first_missing_marking(nums):
    # no swaps: use the sign of a[v - 1] as a "seen" flag
    a, n = list(nums), len(nums)
    # values outside 1..n cannot matter: replace them
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    # the first positive entry marks the missing number
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1

# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]
    for nums, want in cases:
        assert first_missing_positive(nums) == want
        assert first_missing_marking(nums) == want
        assert first_missing_set(nums) == want
    # the input list is left untouched
    original = [3, 4, -1, 1]
    first_missing_positive(original)
```

```
    assert original == [3, 4, -1, 1]
    # 2, 3, 4, 7, 11: the 5th missing positive is 9
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
([4, -2, 1, 2],)	3
([5, 6, 100, -3],)	1

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 14 OF 43

bisect on a sorted list: find a position, count a range, keep it sorted

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	5	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

bisect on a sorted list: find a position, count a range, keep it sorted | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	113. Count People in a Postal Code Range	ColorStack	2025-10-19
M	114. Sliding-Window Mean After Removing the Largest K Values	PracHub	2026-09-01
M	115. Russian Doll Envelopes	1Point3Acres	no date
M	116. Count Sorted Strings With a Given Prefix	ColorStack	2025-10-08
M	117. Streaming Insert + Find K-th Largest	1Point3Acres	2026-02-04
M	118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such...	Exponent	no date
H	119. Find Maximum Envelope Nesting	PracHub	2026-03-18
H	120. Count of Smaller Numbers After Self	HackTheRounds	no date

CODE TEMPLATE | GROUP 14 OF 43 | 8 QUESTIONS, ONE PATTERN

bisect on a sorted list: find a position, count a range, keep it sorted

The one idea these questions share

bisect_left and bisect_right give the position of a value in a sorted list in O(log n). Subtract two positions to count a range, slice between them to list a range, insort to keep inserting in sorted order, and replace an element at a position for the 'longest increasing' trick.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from bisect import bisect_left, bisect_right, insort


def count_in_range(sorted_list, low, high):
    hi = bisect_right(sorted_list, high)       # CHANGE: which positions you read
    lo = bisect_left(sorted_list, low)
    return hi - lo


# insort(sorted_list, x)                    keeps the list sorted as items arrive
# sorted_list[i]                            rank queries: i-th smallest, sorted_list[-1 - k] = k-th largest
```

2. Helper functions that usually get written next to it

```
def lis_length(values):                        # the tails list for "longest increasing"
    tails = []
    for v in values:
        i = bisect_left(tails, v)
        if i == len(tails):
            tails.append(v)
        else:
            tails[i] = v
    return len(tails)


def nearest(sorted_list, x):
    i = bisect_left(sorted_list, x)
    near = [sorted_list[j] for j in (i - 1, i) if 0 <= j < len(sorted_list)]
    return min(near, key=lambda v: abs(v - x))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
113	Count People in a Postal Code Range	sort the codes once; the answer is the template as written, never below 0
114	Sliding-Window Mean After Removing the Largest K Values	keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region
115	Russian Doll Envelopes	sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left
116	Count Sorted Strings With a Given Prefix	lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo
117	Streaming Insert + Find K-th Largest	insort every new value; the k-th largest is sorted_list[-1 - k]
118	Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.	for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

#	Question	What you change in the template
119	Find Maximum Envelope Nesting	exactly the same sort and tails list
120	Count of Smaller Numbers After Self	go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 113 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 1 of 8

Template fit: sort the codes once; the answer is the template as written, never below 0 | asked 366 days ago | frequency Single report

EASY | ColorStack | technical interview | 113 of 288

Count People in a Postal Code Range

ASKED 2025-10-19 | 366 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 239/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #327 Count of Range Sum

LeetCode match: LC 327 Count of Range Sum (related: sorted codes and bisect)

Frequency: Single report | Last asked: 2025-10-19 | Roles: SWE Intern

QUESTION

You are given a list of people, each as a tuple (name, postal_code) . Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range [low, high] ?" The interviewer explicitly wanted the optimal query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in O(log n) . Implement a class with: a constructor that accepts the list of people, and count_in_range(low, high) returning the number of people whose postal code is in [low, high] inclusive.

Function signature

```
from typing import List, Tuple
class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...
    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Follow-up (as reported): Also return the list of matching names. Hint: sort by (postal_code, name) and slice between bisect_left(low) and bisect_right(high) .

EXAMPLES

Example 1

Input	`people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)] count_in_range(90000, 99999)`
Output	`2`
Why	`Alice (90210) and Cara (90211) fall in the range.`

QUESTION

Example 2

Input	`same people; count_in_range(10001, 30301)`
Output	`2`
Why	`Bob (10001) and Dan (30301); range is inclusive on both ends.`

Example 3

Input	`same people; count_in_range(50000, 60000)`
Output	`0`
Why	`No postal codes in that range.`

CONSTRAINTS

- 0 <= number of people <= 10^5 ; the same postal code may appear for multiple people (duplicates allowed).
- Postal codes are integers, 0 <= postal_code <= 10^9 .
- 0 <= low <= high <= 10^9 .
- Each count_in_range call must run in O(log n) after an O(n log n) preprocessing step.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class ScanRegistry:
    def __init__(self, people):
        self.people = list(people)
    def count_in_range(self, low, high):
        return sum(
            1
            for _, code in self.people
            if low <= code <= high
        )
```

TIME	SPACE
O(n) per query	O(1)

WHY IT IS SLOW

Every query touches every person.

OPTIMAL SOLUTION · Sorted codes and two bisects

TIME	SPACE
O(log n) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Also return the names of the people in the range.

Store the (code, name) pairs sorted. Find the two positions with bisect (using (low, '') and (high, a sentinel after every name)) and slice: the names come out ordered by code, then name. O(log n + answer size).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count People in a Postal Code Range
#
# You are given a list of people, each as a tuple (name,
# postal_code) . Build a structure that can answer many
# queries of the form: "how many people have a postal code
# within the inclusive range [low, high] ?" The
# interviewer explicitly wanted the optimal query
# implementation — not a linear scan of the people (or a
# scan over the range and a hash-map lookup for each
# value). Preprocess once, then answer each query in O(log
# n) . Implement a class with: a constructor that accepts
# the list of people, and count_in_range(low, high)
# returning the number of people whose postal code is in
# [low, high] inclusive. Function signature: from typing
# import List, Tuple class PostalRegistry: def
# __init__(self, people: List[Tuple[str, int]]) -> None:
# """Preprocess the (name, postal_code) pairs for fast
# range counts.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Also return the names of the people in the range.
#
# INTERVIEW NOTES
# Sort the postal codes once. The number of codes in [low,
# high] is the position just after the last code <= high
# minus the position of the first code >= low, two binary
# searches with bisect, O(log n) per query. Sorting (code,
# name) pairs lets you also return the names by slicing
# between the same two positions.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Sorted codes and two bisects
# Idea: The codes in the range form one block of the
# sorted list.
class PostalRegistry:
    def __init__(self, people):
        self.codes = sorted(code for _, code in people)

    def count_in_range(self, low, high):
        return max(
            0,
            bisect_right(self.codes, high)
            - bisect_left(self.codes, low),
        )

# ---- main question: tests ----
def test_main_question():
    random.seed(76)
    people = [
        ("Alice", 90210),
        ("Bob", 10001),
        ("Cara", 90211),
        ("Dan", 30301),
    ]
    reg = PostalRegistry(people)
    assert reg.count_in_range(90000, 99999) == 2
    assert reg.count_in_range(10001, 30301) == 2
    assert reg.count_in_range(50000, 60000) == 0


# ============================================================
# FOLLOW-UP 1: Also return the names of the people in the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the (code, name) pairs sorted. Find the
# two positions with bisect (using (low, '') and (high, a
# sentinel after every name)) and slice: the names come
# out ordered by code, then name. O(log n + answer size).
class NamedPostalRegistry:
    def __init__(self, people):
        self.items = sorted(
            (code, name) for name, code in people
        )
```

```
    def names_in_range(self, low, high):
        lo = bisect_left(self.items, (low, ""))
        hi = bisect_left(self.items, (high + 1, ""))
        return [name for _, name in self.items[lo:hi]]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    people = [
        ("Alice", 90210),
        ("Bob", 10001),
        ("Cara", 90211),
        ("Dan", 30301),
    ]
    named = NamedPostalRegistry(people)
    assert named.names_in_range(10001, 30301) == [
        "Bob",
        "Dan",
    ]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 114 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 2 of 8

Template fit: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region | asked 49 days ago

MEDIUM | PracHub | Technical Screen | 114 of 288

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 49 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 146/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every window of a fixed size over the array, remove the k largest values of the window (repeated values count separately) and compute the mean of the rest. Return the means in window order as reduced fractions written 'numerator/denominator'. Update the windows incrementally instead of sorting each one again.

Function signature

def trimmed_window_means(nums: list, window: int, k: int) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4, 5], 3, 1`
Output	`['3/2', '5/2', '7/2']`
Why	`Window [1,2,3] without its largest value 3 averages (1+2)/2 = 3/2; then 5/2 and 7/2.`

Example 2	
Input	`[2, 4, 6, 8], 2, 0`
Output	`['3/1', '5/1', '7/1']`
Why	`With k = 0 nothing is removed: plain window means.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sliding-Window Mean After Removing the
# Largest K Values
#
# For every window of a fixed size over the array, remove
# the k largest values of the window (repeated values
# count separately) and compute the mean of the rest.
# Return the means in window order as reduced fractions
# written 'numerator/denominator'. Update the windows
# incrementally instead of sorting each one again.
#
# Signature:
#   def trimmed_window_means(nums: list, window: int, k: i
nt) -> list
#
# Examples:
# [1, 2, 3, 4, 5], 3, 1  ->  ['3/2', '5/2', '7/2']
# [2, 4, 6, 8], 2, 0  ->  ['3/1', '5/1', '7/1']
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the update O(log w) in the worst case.
# 2. Remove the smallest K instead, or both ends (trimmed
# mean).
# 3. Duplicates on the boundary.
# 4. Trim the smallest K as well (a trimmed mean).
#
# INTERVIEW NOTES
# The mean needs two sums: the whole window and the sum of
# its K largest values. Keep the window in a sorted
# container; the K largest are its last K elements. When
# one value enters or leaves, only one element crosses the
# boundary between 'kept' and 'removed', so the sum of the
# K largest can be updated in O(1) after an O(log) search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right
from fractions import Fraction


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below

def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# mean).
# Answer: Maintain the sum of the K smallest alongside the
# K largest with the mirror-image boundary updates; the
# mean of the middle values is (total - top - bottom) / (w
# - 2K). Checked against sorting every window.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
```

```
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]

# ---- main question: main function ----
# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
```

```
            out.append(as_text(mean))
    return out

# ---- main question: tests ----
def test_main_question():
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    fast = trimmed_window_means
    # a reduced fraction: (1 + 4) / 2 = 5/2
    assert fast([5, 1, 4], 3, 1) == ["5/2"]
    # duplicates at the boundary: remove exactly k copies
    dup = [3, 3, 3, 3]
    assert fast(dup, 3, 1) == ["3/1", "3/1"]
    # trimmed mean: drop k smallest and k largest
    both = fast(nums, 5, 1, True)
```

```
# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k:
int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
`([1, 9, 2, 8], 3, 1)`	`['3/2', '5/1']`
`([-1, 1, 3], 2, 0)`	`['0/1', '2/1']`

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

STUDY PATH: question 115 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 3 of 8

Template fit: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left | no date | frequency Single report

MEDIUM | IN APP LC 354 | 1Point3Acres | phone-screen | 115 of 288

Russian Doll Envelopes

STUDY: TYPE | **ALL TIME 52.8** | **30 DAYS 33** | **PRIORITY 34/288**

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: your app's interview approach (LeetCode #354)

LeetCode match: LC 354 Russian Doll Envelopes

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights. LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each envelope is compared with all earlier ones, O(n^2). After sorting, a longest increasing subsequence of the heights with binary search is O(n log n).

OPTIMAL SOLUTION · Sort, then LIS with binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why sort heights descending when widths tie?

If two envelopes have the same width they cannot nest, but an ascending height order would let the LIS pick both. Descending order makes a strictly increasing height run use at most one envelope per width.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Russian Doll Envelopes
#
# LeetCode 354. Maximum chain of envelopes that strictly
# fit inside each other. Sort + LIS reduction. Input: list
# of (width, height) envelopes. Output: longest chain
# where each envelope strictly fits in the next. Trick:
# sort by width ascending, height descending (when widths
# tie); run LIS (Longest Increasing Subsequence) on
# heights. LIS with patience-sort gives O(n log n).
# Examples: [[5,4],[6,4],[6,7],[2,3]] -> 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why sort heights descending when widths tie?
#
# INTERVIEW NOTES
# Sort by width ascending so that only the heights matter,
# then find the longest strictly increasing subsequence of
# heights. Envelopes with equal width cannot nest, so sort
# those by height descending: then a strictly increasing
# run of heights can never use two envelopes of the same
# width.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left
import itertools


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort, then LIS with binary
# search
# Idea: Sort by (width ascending, height descending).
```

```
def max_envelopes(envelopes):
    # sort by width; for equal widths put TALLER first so
    # a
    # strictly increasing height run can use only one of
```

```
    # them
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    # tails[k]: smallest ending height of a chain of k + 1
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)

# ---- main question: tests ----
def test_main_question():
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    assert max_envelopes([]) == 0
    # equal widths cannot nest, however the heights go
    assert max_envelopes([[1, 1], [1, 2], [1, 3]]) == 1
    assert max_envelopes([[4, 5], [4, 6], [6, 7], [2, 3],
                          [1, 1]]) == 4
    pts = [(w, h) for w in range(1, 4)
           for h in range(1, 4)]


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY PATH: question 116 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 4 of 8

Template fit: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo | asked 377 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 116 of 288

Count Sorted Strings With a Given Prefix

ASKED 2025-10-08 | 377 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 125/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (related: counting by prefix on a sorted list)

Frequency: Single report | Last asked: 2025-10-08 | Roles: SWE Intern

QUESTION

Given a lexicographically sorted array of strings words and a query string prefix , return the number of strings in words that start with prefix . Because words is sorted, all matching strings occupy a contiguous range; find that range with binary search in O(log n * L) time (where L is the prefix length), not by scanning.

Function signature

def count_with_prefix(words: list[str], prefix: str) -> int:
    ...

Follow-up (as reported): Many repeated queries: cache counts in a hash map keyed by prefix. All queries unique: build a trie of all words storing subtree sizes, so each query is O(L) .

EXAMPLES

Example 1

Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "app"`
Output	`3`
Why	`"app", "apple", "application" all start with "app".`

Example 2

Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"`
Output	`2`
Why	`"banana" and "bat".`

QUESTION

Example 3

Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"`
Output	`0`
Why	`No string starts with "cat".`

CONSTRAINTS

- 0 <= len(words) <= 10**6 , sorted in non-decreasing lexicographic order.
- 0 <= len(prefix) <= 100 . An empty prefix matches every string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_prefix(words, prefix):
    return sum(
        1
        for w in words
        if w.startswith(prefix)
    )
```

TIME	SPACE
O(n * L)	O(1)

WHY IT IS SLOW

It ignores that the list is sorted and tests every word.

OPTIMAL SOLUTION · Two binary searches

TIME	SPACE
O(L log n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Thousands of different queries on the same list. Can each be O(L)?

Build a trie once where every node stores how many words pass through it. A query walks the prefix and returns the count in that node (0 if the path breaks). Building costs O(total characters); a query costs O(L), independent of the number of words.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Sorted Strings With a Given Prefix
#
# Given a lexicographically sorted array of strings words
# and a query string prefix , return the number of strings
# in words that start with prefix . Because words is
# sorted, all matching strings occupy a contiguous range;
# find that range with binary search in O(log n * L) time
# (where L is the prefix length), not by scanning.
# Function signature: def count_with_prefix(words:
# list[str], prefix: str) -> int: ... Follow-up (as
# reported): Many repeated queries: cache counts in a hash
# map keyed by prefix. All queries unique: build a trie of
# all words storing subtree sizes, so each query is O(L) .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of different queries on the same list. Can
# each be O(L)?
#
# INTERVIEW NOTES
# In a sorted list all words with a given prefix are
# contiguous. Binary search the first index that is >=
# prefix, and the first index that is greater than every
# word starting with the prefix (the prefix followed by
# the largest possible character); the count is the
# difference. Each comparison costs O(L).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two binary searches
# Idea: The matching words form one contiguous block of
# the sorted list; find both ends.
```

```
def count_with_prefix(words, prefix):
    lo = bisect_left(words, prefix)
    hi = bisect_left(words, prefix + "\uffff")
    return hi - lo

# ---- main question: tests ----
def test_main_question():
    random.seed(54)
    words = [
        "app",
        "apple",
        "application",
        "banana",
        "bat",
    ]
    assert count_with_prefix(words, "app") == 3
    assert count_with_prefix(words, "ba") == 2
    assert count_with_prefix(words, "cat") == 0


# ============================================================
# FOLLOW-UP 1: Thousands of different queries on the same
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Build a trie once where every node stores how
# many words pass through it. A query walks the prefix and
# returns the count in that node (0 if the path breaks).
class PrefixCounter:
    def __init__(self, words):
        self.root = {}
        for w in words:
            node = self.root
            for ch in w:
                node = node.setdefault(ch, {})
                node["#"] = node.get("#", 0) + 1

    def count(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node:
                return 0
            node = node[ch]
        return (
            node.get("#", 0)
```

```
            if prefix
            else sum(
                v.get("#", 0)
                for k, v in self.root.items()
                if k != "#"
            )
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 117 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 5 of 8

Template fit: insort every new value; the k-th largest is sorted_list[-1 - k] | asked 258 days ago | frequency Single report

MEDIUM | IN APP LC 703 | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 117 of 288

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

ASKED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 137/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

Design a structure with insert(x), which adds a number, and findLargest(k), which returns the k-th largest number inserted so far (k = 0 is the largest), counting repeated values separately, or None when fewer than k + 1 numbers exist.

Function signature

class StreamKth:
  def insert(self, x: float) -> None
  def findLargest(self, k: int) -> float | None

EXAMPLES

Example 1

Input	`s = StreamKth()`
	`s.insert(3)`
	`s.insert(3)`
	`s.insert(2)`
	`s.findLargest(0)`
	`s.findLargest(1)`
	`s.findLargest(2)`
	`s.findLargest(3)`

Output	`s.findLargest(0) -> 3`
	`s.findLargest(1) -> 3`
	`s.findLargest(2) -> 2`
	`s.findLargest(3) -> null`

Why	`Sorted descending the numbers are 3, 3, 2, so the 0th, 1st and 2nd largest are 3, 3 and 2, and there is no 3rd.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
items = []
def insert(x):
    items.append(x)
def find_largest(k):
    ordered = sorted(items, reverse=True)
    return ordered[k]
```

TIME	SPACE
O(n log n) per query	O(n)

WHY IT IS SLOW

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION · Sorted list with bisect (k chosen per query)

TIME	SPACE
O(log n) search + O(n) shift per insert, O(1) query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

FOLLOW-UP 2 What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

FOLLOW-UP 3 Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Streaming Insert + Find K-th Largest
#
# Design a structure with insert(x), which adds a number,
# and findLargest(k), which returns the k-th largest
# number inserted so far (k = 0 is the largest), counting
# repeated values separately, or None when fewer than k +
# 1 numbers exist.
#
# Signature:
#   class StreamKth:
#       def insert(self, x: float) -> None
#       def findLargest(self, k: int) -> float | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if values can also be deleted?
# 2. What is the trade-off between the two designs?
# 3. Values can also be deleted.
#
# INTERVIEW NOTES
# If k can change from query to query you need every
# value, in a structure that supports rank queries; if k
# is fixed you only need the k largest, and a min-heap of
# size k gives the answer at its root. Ask which of the
# two it is: it decides everything.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
```

```
# - k].
# FOLLOW-UP 1: What if values can also be deleted?
# Answer: The heap can no longer answer because removing
# an arbitrary element is O(k).
# FOLLOW-UP 3: Values can also be deleted.
# Answer: A heap cannot delete an arbitrary value
# efficiently.
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)

    # FOLLOW-UP 1, 3: values can also be deleted
    def delete(self, x):
        # remove one copy of x if present
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False

    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None

# ---- main question: tests ----
def test_main_question():
    s = StreamKth()
    for x in (3, 3, 2):
        s.insert(x)
    top3 = [s.find_largest(k) for k in range(3)]
    assert top3 == [3, 3, 2]
    assert s.find_largest(3) is None
    # deletes work too
    assert s.delete(3) and not s.delete(9)
    assert [s.find_largest(k) for k in range(2)] == [3, 2]
```

```
# ============================================================
# FOLLOW-UP 1: What if values can also be deleted?
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: What is the trade-off between the two
# ============================================================

# ---- follow-up 2: function ----
# Answer: The heap is the smallest and fastest when k is
# fixed and small (O(log k), O(k) memory) but cannot
# change k or delete. The sorted list answers any k and is
# simplest, at O(n) memory and a linear shift per insert.
class FixedKth:
    """Fixed k, inserts only: a min-heap of the best k."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def insert(self, x):
        heapq.heappush(self.heap, x)
        # keep only the k largest values
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

    def kth_largest(self):
        # the smallest of the k largest
        if len(self.heap) < self.k:
            return None
        return self.heap[0]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
    # the two largest are 9 and 7, so the 2nd largest is 7
```

```
    assert f.kth_largest() == 7


# ============================================================
# FOLLOW-UP 3: Values can also be deleted.
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

STUDY PATH: question 118 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 6 of 8

Template fit: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances | no date

MEDIUM | Exponent | 118 of 288

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TALK | ALL TIME 26.5 | 30 DAYS 0 | PRIORITY 154/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #475 Heaters

LeetCode match: No direct match; related LC 475 Heaters (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5138/minimum-distance-kids-ice-cream-sellers

QUESTION

Kids and ice cream sellers stand at positions on a line. Find the smallest distance E such that every kid is within E of at least one seller (the kid at position K is served by a seller at P when P - E <= K <= P + E).

Function signature

def smallest_e(kids: list, sellers: list) -> int

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 8]`
Output	`3`
Why	`The kid at 5 is 3 away from both sellers (at 2 and 8); the others are 1 away.`

Example 2	
Input	`[4], [4]`
Output	`0`
Why	`A kid standing on a seller needs distance 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(kids, sellers):
    worst = 0
    for kid in kids:
        gaps = [abs(kid - s) for s in sellers]
        nearest = min(gaps)
        worst = max(worst, nearest)
    return worst
```

TIME	SPACE
O(N * M)	O(1)

WHY IT IS SLOW

N * M distance checks. With the sellers sorted, binary search finds each kid's nearest seller among two neighbours, O((N + M) log M).

OPTIMAL SOLUTION · Sort the sellers, binary search each kid

TIME	SPACE
O((N + M) log M)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both lists are already sorted, or you want to avoid the binary search.

Walk the kids in order with one pointer on the sellers that only moves forward while the next seller is at least as close. O(N + M) after sorting.

FOLLOW-UP 2 Each seller can serve at most C kids.

Now it is an assignment problem. Binary search the radius E and test feasibility with a greedy left-to-right matching that gives each seller up to C of the leftmost uncovered kids in its range.

FOLLOW-UP 3 Sellers can be placed anywhere (minimise the radius with S sellers).

That is the 1D k-center problem: binary search the radius and greedily place a seller at the farthest point covering the leftmost uncovered kid. O(N log range).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given positions of N kids and M ice cream
# sellers on a 1D line, find the smallest distance E such
# that every kid lies within the range [P - E, P + E] of
# at least one seller.
#
# Kids and ice cream sellers stand at positions on a line.
# Find the smallest distance E such that every kid is
# within E of at least one seller (the kid at position K
# is served by a seller at P when P - E <= K <= P + E).
#
# Signature:
#   def smallest_e(kids: list, sellers: list) -> int
#
# Examples:
# [1, 5, 9], [2, 8]  ->  3
# [4], [4]  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both lists are already sorted, or you want to avoid
# the binary search.
# 2. Each seller can serve at most C kids.
# 3. Sellers can be placed anywhere (minimise the radius
# with S sellers).
#
# INTERVIEW NOTES
# Each kid only cares about its nearest seller, and the
# radius E must cover the worst kid. So E is the maximum
# over kids of the distance to the nearest seller. With
# sellers sorted, the nearest seller of a kid is one of
# the two neighbours around its insertion point, which is
# a binary search.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left
import random


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort the sellers, binary search
# each kid
# Idea: With the sellers sorted, bisect_left gives the
# first seller at or after the kid; the nearest seller is
# that one or the one before it. E is the largest of these
# nearest distances.
def smallest_e(kids, sellers):
    # each kid goes to the nearest seller; the answer is
    # the
    # worst such distance. The nearest seller is next to
    # kid's place in the sorted list of sellers.
    sellers = sorted(sellers)
    worst = 0
    for k in kids:
        i = bisect_left(sellers, k)
        near = INF = float("inf")
        if i < len(sellers):
            near = sellers[i] - k
        if i > 0:
            near = min(near, k - sellers[i - 1])
        worst = max(worst, near)
    return worst

# ---- main question: tests ----
def test_main_question():
    assert smallest_e([1, 5, 9], [2, 8]) == 3
    assert smallest_e([4], [4]) == 0


# ============================================================
# FOLLOW-UP 1: Both lists are already sorted, or you want
# ============================================================

# ---- follow-up 1: function ----
# Answer: Walk the kids in order with one pointer on the
# sellers that only moves forward while the next seller is
# at least as close. O(N + M) after sorting.
def smallest_e_sorted(kids, sellers):
    # follow-up: sorted input, no binary search. One
    # pointer
    # on the sellers only moves forward.
    kids, sellers = sorted(kids), sorted(sellers)
```

```
    worst, j = 0, 0
    for k in kids:
        # move on while the next seller is at least as
        # close
        while j + 1 < len(sellers):
            here = abs(sellers[j] - k)
            if abs(sellers[j + 1] - k) > here:
                break
            j += 1
        worst = max(worst, abs(sellers[j] - k))
    return worst

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert smallest_e_sorted([1, 5, 9], [2, 8]) == 3


# ============================================================
# FOLLOW-UP 2: Each seller can serve at most C kids.
# ============================================================

# ---- follow-up 2: helper functions ----
def feasible_capacity(kids, sellers, e, cap):
    # can every kid be served when a seller takes at most
    # `cap` kids? Sellers left to right each take the
    # leftmost kids that are still unserved and in range.
    i = 0
    for s in sellers:
        if i < len(kids) and kids[i] < s - e:
            return False
        taken = 0
        while i < len(kids) and taken < cap:
            if kids[i] > s + e:
                break
            i += 1
            taken += 1
    return i == len(kids)

# ---- follow-up 2: function ----
# Answer: Now it is an assignment problem. Binary search
# the radius E and test feasibility with a greedy left-to-
```

```
# right matching that gives each seller up to C of the
# leftmost uncovered kids in its range.
def smallest_e_capacity(kids, sellers, cap):
    # follow-up: binary search the distance
```

```
    kids, sellers = sorted(kids), sorted(sellers)
    if len(kids) > cap * len(sellers):
        return None
    lo, hi = 0, max(kids[-1], sellers[-1]) - min(
        kids[0], sellers[0])
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible_capacity(kids, sellers, mid, cap):
            hi = mid
        else:
            lo = mid + 1
    return lo

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # capacity 1: three kids need three sellers' worth
    assert smallest_e_capacity([1, 2, 3], [2], 1) is None
    spread = smallest_e_capacity([1, 2, 3], [1, 3, 9], 1)
    assert spread == 6
    assert smallest_e_capacity([1, 2], [5, 6], 1) == 4


# ========================================================
# FOLLOW-UP 3: Sellers can be placed anywhere (minimise
# ========================================================

# ---- follow-up 3: helper functions ----
def sellers_needed(kids, e):
    # greedy: the leftmost uncovered kid at x gets a
    # seller at
    # x + e, who covers every kid up to x + 2e
    count, reach = 0, None
    for k in sorted(kids):
        if reach is None or k > reach:
            count += 1
            reach = k + 2 * e
    return count

# ---- follow-up 3: function ----
# Answer: That is the 1D k-center problem: binary search
# the radius and greedily place a seller at the farthest
# point covering the leftmost uncovered kid. O(N log
# range).
def best_radius(kids, how_many):
    # follow-up: sellers can stand ANYWHERE; the smallest
```

```
    # (whole number) radius that is enough for how_many
    kids = sorted(kids)
    lo, hi = 0, kids[-1] - kids[0]
    while lo < hi:
        mid = (lo + hi) // 2
        if sellers_needed(kids, mid) <= how_many:
            hi = mid
        else:
            lo = mid + 1
    return lo

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert sellers_needed([1, 2, 3, 20], 1) == 2
    assert best_radius([1, 2, 3, 20], 2) == 1
    assert best_radius([5], 1) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY PATH: question 119 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 7 of 8

Template fit: exactly the same sort and tails list | asked 216 days ago

HARD | IN APP LC 354 | PracHub | Technical Screen | 119 of 288

★ TOP 75: rank 14 in the Medium and Hard priority list

★ Find Maximum Envelope Nesting

POSTED 2026-03-18 | 216 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 33/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #354 Russian Doll Envelopes

LeetCode match: LC 354 Russian Doll Envelopes

Posted on PracHub: 2026-03-18 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/find-maximum-envelope-nesting

QUESTION

Envelopes are given as (width, height). One envelope fits inside another only when both its width and its height are strictly smaller. Return the largest number of envelopes that can be nested one inside the next (Russian dolls).

Function signature

def max_envelopes(envelopes: list) -> int

EXAMPLES

Example 1	
Input	`[(5, 4), (6, 4), (6, 7), (2, 3)]`
Output	`3`
Why	`(2, 3) fits in (5, 4), which fits in (6, 7): 3 nested.`

Example 2	
Input	`[(1, 1), (1, 1), (1, 1)]`
Output	`1`
Why	`Equal envelopes cannot go inside each other.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every envelope compares with all earlier ones, O(n^2). Sorting by width ascending and height descending, then a longest increasing subsequence on heights with binary search, is O(n log n).

OPTIMAL SOLUTION · Sort by width ascending and height descending, then LIS by binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

FOLLOW-UP 2 Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

FOLLOW-UP 3 Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find Maximum Envelope Nesting
#
# Envelopes are given as (width, height). One envelope
# fits inside another only when both its width and its
# height are strictly smaller. Return the largest number
# of envelopes that can be nested one inside the next
# (Russian dolls).
#
# Signature:
#   def max_envelopes(envelopes: list) -> int
#
# Examples:
# [(5, 4), (6, 4), (6, 7), (2, 3)]  ->  3
```

```
# [(1, 1), (1, 1), (1, 1)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual chain of envelopes.
# 2. Three dimensions (boxes).
# 3. Non-strict nesting (equal sizes allowed).
#
# INTERVIEW NOTES
# Two dimensions are hard, one is easy. Sort by width so
# that width order is settled, then the question becomes a
# longest strictly increasing subsequence of heights. The
# trap is envelopes with equal width: they cannot nest, so
# sort equal widths by height descending, which prevents
# two of them from forming an increasing pair. The LIS is
# then done with patience sorting (binary search over
# tails).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence.
def max_envelopes(envelopes):
    # sort by width, taller first on ties, then find the
    # longest
    # strictly increasing run of heights
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)

# ---- main question: tests ----
def test_main_question():
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3


# ============================================================
# FOLLOW-UP 1: Return the actual chain of envelopes.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember, for each element, the index of the
# tail it extended (parent pointers), and rebuild from the
# last tail of the longest length. The sorted order and
# binary search are unchanged.
def envelope_chain(envelopes):
    # also return the chain: remember what each element
    # extended
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails, tail_idx = [], []
    prev = [-1] * len(ordered)
    for i, (_, h) in enumerate(ordered):
        k = bisect_left(tails, h)
        if k == len(tails):
            tails.append(h)
            tail_idx.append(i)
        else:
            tails[k] = h
            tail_idx[k] = i
        # the element that ended a chain one shorter
        prev[i] = tail_idx[k - 1] if k else -1
    chain = []
    i = tail_idx[-1] if tail_idx else -1
    while i != -1:
        chain.append(ordered[i])
        i = prev[i]
    return chain[::-1]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    chain = envelope_chain(envs)
    assert len(chain) == 3
    # every envelope fits strictly inside the next
    for a, b in zip(chain, chain[1:]):
        assert a[0] < b[0] and a[1] < b[1]
    assert envelope_chain([]) == []
    # equal sizes: only the non-strict version nests them
    twins = [[3, 3], [3, 3], [4, 4]]
    assert max_envelopes(twins) == 2


# ============================================================
# FOLLOW-UP 2: Three dimensions (boxes).
# ============================================================

# ---- follow-up 2: function ----
# Answer: Sorting plus LIS no longer reduces the problem;
# use an O(n^2) DP, or CDQ divide and conquer with a
# Fenwick tree for O(n log^2 n).
def max_boxes_3d(boxes):
    # three sizes: the reduction no longer works, use
    # O(n^2)
    ordered = sorted(boxes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            a, b = ordered[j], ordered[i]
            if all(x < y for x, y in zip(a, b)):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)

# ============================================================
# FOLLOW-UP 3: Non-strict nesting (equal sizes allowed).
# ============================================================

# ---- follow-up 3: function ----
# Answer: Sort by width ascending and height ascending,
# and use bisect_right (longest non-decreasing
# subsequence) instead of bisect_left.
def max_nested_non_strict(envelopes):
    # equal sizes may nest: sort both ways ascending and
    # use
    # bisect_right (a longest non-decreasing subsequence)
    tails = []
    for _, h in sorted(envelopes):
        i = bisect_right(tails, h)
```

```
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # equal sizes: only the non-strict version nests them
    twins = [[3, 3], [3, 3], [4, 4]]
    assert max_nested_non_strict(twins) == 3
    boxes = [(1, 1, 1), (2, 2, 2), (3, 1, 4), (3, 3, 3)]
    assert max_boxes_3d(boxes) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(envelopes: list[list[int]]) -> int:
SAMPLE INPUT
[[5, 4], [6, 4], [6, 7], [2, 3]]
```

Test cases

Input	Expected
`[[5, 4], [6, 4], [6, 7], [2, 3]]`	`3`
`[]`	`0`

ALSO REPORTED · 1 more

Russian Doll Envelopes | 1Point3Acres | phone-screen | Single report

https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

Preparation: Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY PATH: question 120 of 288 | topic 14 of 43: bisect on a sorted list: find a position, count a range, keep it sorted | 8 of 8

Template fit: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version) | no date

HARD | IN APP LC 315 | HackTheRounds | Phone Screen | 120 of 288

Count of Smaller Numbers After Self

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 131/288

ABOUT THIS QUESTION

Topic: bisect on a sorted list: find a position, count a range, keep it sorted

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #315 Count of Smaller Numbers After Self

LeetCode match: LC 315

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of elements to the right of nums[i] that are strictly smaller than nums[i].

Function signature

def count_smaller_after(nums: list) -> list

EXAMPLES

Example 1	
Input	`[5, 2, 6, 1]`
Output	`[2, 1, 1, 0]`
Why	`5 has two smaller numbers to its right (2 and 1), 2 has one (1), 6 has one (1), 1 has none.`

Example 2	
Input	`[-1]`
Output	`[0]`
Why	`A single number has nothing to its right.`

Example 3	
Input	`[-1, -1]`
Output	`[0, 0]`
Why	`Equal numbers are not smaller.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(nums):
    counts = []
    for i in range(len(nums)):
        smaller = 0
        for j in range(i + 1, len(nums)):
            if nums[j] < nums[i]:
                smaller += 1
        counts.append(smaller)
    return counts
```

TIME	SPACE
O(n^2)	O(1) extra

WHY IT IS SLOW

Two nested loops, O(n^2): too slow at n = 10^5. A Fenwick tree over ranks scanned right to left (or merge sort) is O(n log n).

OPTIMAL SOLUTION · Fenwick tree over compressed values, scanning right to left

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it with merge sort instead of a Fenwick tree.

Sort indices by value with merge sort; whenever you place an element from the left half, add the number of right-half elements already placed (they are smaller). Same O(n log n), and it avoids coordinate compression.

FOLLOW-UP 2 Count larger numbers after self, or smaller numbers before self.

Larger after: query the suffix instead of the prefix of the Fenwick tree. Smaller before: scan left to right instead of right to left. **Cost:** still O(n log n) with the Fenwick tree.

FOLLOW-UP 3 Count inversions in an array.

The sum of the answers above is the number of inversions; merge sort counts them directly in O(n log n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count of Smaller Numbers After Self
#
# Given an integer array nums, return an array counts
# where counts[i] is the number of elements to the right
# of nums[i] that are strictly smaller than nums[i].
#
# Signature:
#   def count_smaller_after(nums: list) -> list
#
# Examples:
# [5, 2, 6, 1]  ->  [2, 1, 1, 0]
# [-1]  ->  [0]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with merge sort instead of a Fenwick tree.
# 2. Count larger numbers after self, or smaller numbers
# before self.
# 3. Count inversions in an array.
#
# INTERVIEW NOTES
# For each element you want the count of smaller elements
# to its right: an order-statistics query on the elements
# seen so far if you scan right to left. A Fenwick tree
# over compressed values gives rank counts in O(log n).
# The same count falls out of merge sort: when a left
# element is placed after right elements that are smaller,
# those right elements are exactly the smaller numbers
# after it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class Fenwick:
    """Counts of seen values; prefix(i) = how many seen
    values have a rank below i."""

    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


def ranks(nums):
    # coordinate compression: value -> 0, 1, 2 ... by size
    return {v: i for i, v in enumerate(sorted(set(nums)))}


# ---- main question: main function ----
# BASE QUESTION (optimal): Fenwick tree over compressed
# values, scanning right to left
# Idea: Compress values to ranks 1..m. Scan from the
# right: the answer for x is the number of already
# inserted values with rank less than rank(x) (a prefix
# sum query), then insert rank(x). Both operations are
# O(log m).
def count_smaller_after(nums):
    # go from the right; before inserting x ask how many
    # already-seen values are strictly smaller than x
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out = []
    for x in reversed(nums):
        out.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return out[::-1]

# ---- main question: tests ----
def test_main_question():
    sample = [5, 2, 6, 1]
    assert count_smaller_after(sample) == [2, 1, 1, 0]
    assert count_smaller_after([-1]) == [0]
    assert count_smaller_after([-1, -1]) == [0, 0]
    assert count_smaller_after([]) == []


# ============================================================
# FOLLOW-UP 1: Do it with merge sort instead of a Fenwick
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort indices by value with merge sort; whenever
# you place an element from the left half, add the number
# of right-half elements already placed (they are
# smaller). Same O(n log n), and it avoids coordinate
# compression.
def count_smaller_merge(nums):
    # follow-up: merge sort on indexes. When a left item
    # is
    # placed, every right item already placed is smaller.
    counts = [0] * len(nums)
    order = list(range(len(nums)))

    def sort(lo, hi):
        if hi - lo <= 1:
            return
        mid = (lo + hi) // 2
        sort(lo, mid)
        sort(mid, hi)
        merged, i, j = [], lo, mid
        while i < mid or j < hi:
            left_first = False
            if i < mid and j < hi:
                a, b = nums[order[i]], nums[order[j]]
                left_first = a <= b
            if j >= hi or left_first:
                counts[order[i]] += j - mid
                merged.append(order[i])
                i += 1
            else:
                merged.append(order[j])
                j += 1
        order[lo:hi] = merged


    sort(0, len(nums))
    return counts

# ============================================================
# FOLLOW-UP 2: Count larger numbers after self, or smaller
```

```
# ============================================================

# ---- follow-up 2: function ----
# Answer: Larger after: query the suffix instead of the
# prefix of the Fenwick tree. Smaller before: scan left to
# right instead of right to left. Cost: still O(n log n)
# with the Fenwick tree.
def count_larger_after(nums):
    # follow-up: the suffix of the tree instead of the
    # prefix
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out, seen = [], 0
    for x in reversed(nums):
        out.append(seen - tree.prefix(rank[x] + 1))
        tree.add(rank[x])
        seen += 1
    return out[::-1]


# Answer: Larger after: query the suffix instead of the
# prefix of the Fenwick tree. Smaller before: scan left to
# right instead of right to left. Cost: still O(n log n)
# with the Fenwick tree.
def count_smaller_before(nums):
    # follow-up: scan from the left instead
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out = []
    for x in nums:
        out.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return out

# ---- follow-up 2: tests ----
def test_follow_up_2():
    sample = [5, 2, 6, 1]
    assert count_larger_after(sample) == [1, 1, 0, 0]
    assert count_smaller_before(sample) == [0, 0, 2, 0]
```

```
# ============================================================
# FOLLOW-UP 3: Count inversions in an array.
# ============================================================

# ---- follow-up 3: function ----
# Answer: The sum of the answers above is the number of
# inversions; merge sort counts them directly in O(n log
# n).
def inversions(nums):
    # follow-up: the answers added up are the inversions
    return sum(count_smaller_merge(nums))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert inversions([2, 4, 1, 3, 5]) == 3
    assert inversions([3, 2, 1]) == 3
    for _ in range(300):
        nums = [random.randint(-5, 5)
                for _ in range(random.randint(0, 12))]
        bigger = [sum(y > x for y in nums[i + 1:])
                  for i, x in enumerate(nums)]
        assert count_larger_after(nums) == bigger
        before = [sum(y < x for y in nums[:i])
                  for i, x in enumerate(nums)]
        assert count_smaller_before(nums) == before


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 15 OF 43

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	5
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	121. Parity Alternating Subarray Query	HackTheRounds	no date
E	122. Cake Balance Line	ColorStack	2025-09-26
E	123. Find the Pivot Index (Balanced Split) in O(n)	ColorStack	2025-09-21
M	124. Range Duplicate Check with Constant-Time Queries	1Point3Acres	2026-09-29
M	125. Find a Horizontal Cut That Bisects Rectangle Area	PracHub	2026-06-04
M	126. Check pivot after removing one element	PracHub	2026-02-12
M	127. Gold Chain Split into Two Equal Halves	1Point3Acres	2026-04-21
M	128. Count the number of ways to split a string.	Exponent	no date

CODE TEMPLATE | GROUP 15 OF 43 | 8 QUESTIONS, ONE PATTERN

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

The one idea these questions share

pre[i] is the total of the first i items. Any range sum is pre[r + 1] - pre[l], and a balance point is an index where the left total equals the right total. Count something other than values (equal pairs, ones, repeats) by accumulating that instead.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def build_prefix(nums):
    pre = [0]
    for x in nums:                          # CHANGE 1: what you accumulate (value, 1 per match, 1 per one)
        pre.append(pre[-1] + x)
    return pre


def range_sum(pre, l, r):                   # inclusive l..r
    return pre[r + 1] - pre[l]


def pivot_index(nums):
    pre = build_prefix(nums)
    for i in range(len(nums)):              # CHANGE 2: the test at each index
        if pre[i] == pre[-1] - pre[i + 1]:
            return i
    return -1
```

2. Helper functions that usually get written next to it

```
def first_last(values):                     # first and last position of every value
    first, last = {}, {}
    for i, v in enumerate(values):
        first.setdefault(v, i)
        last[v] = i
    return first, last
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
121	Parity Alternating Subarray Query	pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0
122	Cake Balance Line	the same balance test on the cake heights
123	Find the Pivot Index (Balanced Split) in O(n)	the template as written: left total equals total - left - x
124	Range Duplicate Check with Constant-Time Queries	next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r
125	Find a Horizontal Cut That Bisects Rectangle Area	events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip
126	Check pivot after removing one element	prefix sums plus first/last position maps: look up the value whose removal would balance both sides
127	Gold Chain Split into Two Equal Halves	prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link
128	Count the number of ways to split a string.	positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

Contents Start the questions >

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 121 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 1 of 8

Template fit: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0 | no date

EASY | HackTheRounds | Onsite | 121 of 288

Parity Alternating Subarray Query

STUDY: TALK | **ALL TIME 34.9** | **30 DAYS 0** | **PRIORITY 129/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #303 Range Sum Query - Immutable

LeetCode match: No direct match; prefix-sum range queries (idea of LC 303)

QUESTION

Given an array nums, answer many queries alternates(l, r): do the elements nums[l], ..., nums[r] (both ends included) alternate in parity (odd, even, odd, ... or even, odd, even, ...)? Preprocess the array once so that every query takes O(1).

Function signature

```
class ParityQueries:
    def __init__(self, nums: list)
    def alternates(self, l: int, r: int) -> bool
```

EXAMPLES

Example 1	

Input	`p = ParityQueries([1, 2, 3, 5, 4])`
	`p.alternates(0, 2)`
	`p.alternates(0, 3)`
	`p.alternates(3, 4)`

Output	`p.alternates(0, 2) -> true`
	`p.alternates(0, 3) -> false`
	`p.alternates(3, 4) -> true`

Why	`1, 2, 3 alternates; the neighbours 3 and 5 are both odd, so any range containing both fails; 5, 4 alternates.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def alternates(nums, l, r):
    for i in range(l, r):
        if nums[i] % 2 == nums[i + 1] % 2:
            return False
    return True
```

BRUTE FORCE

TIME	SPACE
O(length) per query	**O(1)**

WHY IT IS SLOW

Each query scans up to n elements. A prefix count of equal-parity neighbour pairs answers every query in O(1) after one O(n) pass.

OPTIMAL SOLUTION · Prefix count of equal-parity neighbour pairs

TIME	SPACE
O(n) to build, O(1) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array also changes (point updates).

Keep the bad-pair indicators in a Fenwick tree: an update changes at most two indicators, each an O(log n) update, and a query is two prefix sums.

FOLLOW-UP 2 Find the longest alternating subarray.

One pass with a running length that resets to 1 on a bad pair. O(n), O(1).

FOLLOW-UP 3 Offline queries, different property (for example all elements distinct).

The prefix-sum trick applies whenever the property is 'no violating neighbour pair'. Properties about distinctness need Mo's algorithm or an offline sweep with last-occurrence tracking. **Cost:** Mo's algorithm is O((n + q) sqrt(n)); the prefix trick for 'no violating neighbour pair' stays O(n + q).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Parity Alternating Subarray Query
#
# Given an array nums, answer many queries alternates(l,
# r): do the elements nums[l], ..., nums[r] (both ends
# included) alternate in parity (odd, even, odd, ... or
# even, odd, even, ...)? Preprocess the array once so that
# every query takes O(1).
#
# Signature:
#   class ParityQueries:
```

```
#       def __init__(self, nums: list)
#       def alternates(self, l: int, r: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array also changes (point updates).
# 2. Find the longest alternating subarray.
# 3. Offline queries, different property (for example all
# elements distinct).
#
# INTERVIEW NOTES
# A subarray alternates in parity exactly when it contains
# no pair of neighbours with the same parity. 'Count of
# bad pairs in a range' is a prefix sum question, so
# precompute once and answer every query with one
# subtraction. Mark each position i by whether (nums[i],
# nums[i + 1]) is a bad pair.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Prefix count of equal-parity
# neighbour pairs
# Idea: bad[i] is the number of positions j < i with
```

```
# nums[j] and nums[j + 1] of the same parity. The pairs
# inside nums[l..r] are j = l .. r - 1, so the subarray
# alternates when bad[r] - bad[l] == 0.
class ParityQueries:
    """Static array. bad[i] = number of neighbouring pairs
    with equal parity among the first i pairs. A range
    alternates when it contains no such pair."""

    def __init__(self, nums):
        self.bad = [0]
        for a, b in zip(nums, nums[1:]):
            same = a % 2 == b % 2
            self.bad.append(self.bad[-1] + same)

    def alternates(self, l, r):
        # pairs l .. r - 1 are inside the range [l, r]
        return self.bad[r] - self.bad[l] == 0

# ---- main question: tests ----
def test_main_question():
    q = ParityQueries([1, 2, 3, 4, 5])
    assert q.alternates(0, 4) and q.alternates(1, 3)
    q = ParityQueries([1, 3, 2, 4])
    assert not q.alternates(0, 1)
    assert not q.alternates(0, 3)
    assert q.alternates(1, 2)
    # a single element always alternates
    assert q.alternates(2, 2)


# ============================================================
# FOLLOW-UP 1: The array also changes (point updates).
# ============================================================
```

```
# ---- follow-up 1: helper functions ----
class Fenwick:
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i, delta):
        i += 1
        while i <= self.size:
            self.tree[i] += delta
            i += i & -i
```

```
    def prefix(self, i):
        # sum of the first i items
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total

# ---- follow-up 1: function ----
# Answer: Keep the bad-pair indicators in a Fenwick tree:
# an update changes at most two indicators, each an O(log
# n) update, and a query is two prefix sums.
class ParityWithUpdates:
    """Follow-up: point updates. The equal-parity flags
    live in a Fenwick tree; changing one number changes
    at most two flags, and a query is two prefix sums."""

    def __init__(self, nums):
        self.nums = list(nums)
        self.flags = [0] * max(len(nums) - 1, 0)
        self.tree = Fenwick(len(self.flags))
        for i in range(len(self.flags)):
            self.refresh(i)

    def refresh(self, i):
        # recompute the flag of the pair (i, i + 1)
        a, b = self.nums[i], self.nums[i + 1]
        now = int(a % 2 == b % 2)
        self.tree.add(i, now - self.flags[i])
        self.flags[i] = now

    def update(self, i, value):
        self.nums[i] = value
        for pair in (i - 1, i):
            if 0 <= pair < len(self.flags):
                self.refresh(pair)

    def alternates(self, l, r):
        inside = self.tree.prefix(r) - self.tree.prefix(l)
        return inside == 0

# ============================================================
# FOLLOW-UP 2: Find the longest alternating subarray.
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: One pass with a running length that resets to 1
# on a bad pair. O(n), O(1).
def longest_alternating(nums):
    # follow-up: one pass with a running length that
    # restarts at 1 after a bad pair
    best = run = 1 if nums else 0
    for a, b in zip(nums, nums[1:]):
        run = run + 1 if a % 2 != b % 2 else 1
        best = max(best, run)
    return best

# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    assert longest_alternating([1, 3, 2, 4]) == 2
    assert longest_alternating([1, 2, 3, 4, 5]) == 5
    assert longest_alternating([]) == 0
    assert longest_alternating([7]) == 1
    for _ in range(200):
        nums = [random.randint(0, 9)
                for _ in range(random.randint(1, 10))]
        fixed = ParityQueries(nums)
        live = ParityWithUpdates(nums)
        for _ in range(20):
            if random.random() < 0.4:
                i = random.randrange(len(nums))
                nums[i] = random.randint(0, 9)
                live.update(i, nums[i])
            l = random.randrange(len(nums))
            r = random.randrange(l, len(nums))
        fixed = ParityQueries(nums)
        l = random.randrange(len(nums))
        r = random.randrange(l, len(nums))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 2 more

Answer parity-alternation range queries | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-parity-alternation-range-queries

Answer range queries for alternating parity subarrays | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-range-queries-for-alternating-parity-subarrays

STUDY PATH: question 122 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 2 of 8

Template fit: the same balance test on the cake heights | asked 389 days ago | frequency Single report

EASY | IN APP LC 724 | ColorStack | technical interview | 122 of 288

Cake Balance Line

ASKED 2025-09-26 | 389 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 135/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: your app's interview approach (LeetCode #724)

LeetCode match: LC 724 Find Pivot Index (cake version)

Frequency: Single report | Last asked: 2025-09-26 | Roles: SWE Intern

QUESTION

A cake is described by a list heights , where heights[i] is the amount of cake sitting at position i (positions are left to right). You want to place a single vertical cut exactly at some position i so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position i itself is consumed by the cut and counts for neither side). Return the smallest index i where sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no such position exists.

Function signature

```
def cake_balance_index(heights: list[int]) -> int:
    ...
```

Follow-up (as reported): Do it in one pass and O(1) extra space by tracking a running left sum against total - left - heights[i] .

EXAMPLES

Example 1

Input	`heights = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.`

Example 2

Input	`heights = [1, 2, 3]`
Output	`-1`
Why	`No cut position balances the two sides.`

Example 3

Input	`heights = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is empty (0); right is 1+(-1)=0.`

CONSTRAINTS

QUESTION

- 1 <= len(heights) <= 10^5 .
- -10^4 <= heights[i] <= 10^4 (heights may be negative in this abstracted version).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_cake(heights):
    n = len(heights)
    prefix, total = [0] * n, 0
    for i, h in enumerate(heights):
        prefix[i] = total
        total += h
    suffix, total = [0] * n, 0
    for i in range(n_-_1,_-1,_-1):
        suffix[i] = total
        total += heights[i]
    for i in range(n):
        if prefix[i] == suffix[i]:
            return i
    return -1
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It stores both running sums although one number is enough.

OPTIMAL SOLUTION · Running left sum

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which cut makes the two sides as equal as possible?

Same scan, but track the smallest absolute difference |left - (total - left - h)| and the first index that reaches it. If a perfectly balanced cut exists it wins with difference 0.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Cake Balance Line
#
# A cake is described by a list heights , where heights[i]
# is the amount of cake sitting at position i (positions
# are left to right). You want to place a single vertical
# cut exactly at some position i so that the total amount
# of cake strictly to the left of the cut equals the total
```

```
# amount strictly to the right of the cut (the cake at
# position i itself is consumed by the cut and counts for
# neither side). Return the smallest index i where
# sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no
# such position exists. Function signature: def
# cake_balance_index(heights: list[int]) -> int: ...
# Follow-up (as reported): Do it in one pass and O(1)
# extra space by tracking a running left sum against total
# - left - heights[i] .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which cut makes the two sides as equal as possible?
#
# INTERVIEW NOTES
# Same as the pivot index: the cut at i balances when the
# left sum equals total - left - heights[i]. Keep the left
# sum as a running value; no prefix array is needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Running left sum
# Idea: The right side is total - left - heights[i].
def cake_balance_index(heights):
    total, left = sum(heights), 0
    for i, h in enumerate(heights):
        if left == total - left - h:
            return i
        left += h
    return -1

# ---- main question: tests ----
def test_main_question():
    random.seed(32)
    assert cake_balance_index([1, 7, 3, 6, 5, 6]) == 3
    assert cake_balance_index([1, 2, 3]) == -1
    assert cake_balance_index([2, 1, -1]) == 0


# ============================================================
# FOLLOW-UP 1: Which cut makes the two sides as equal as
# ============================================================

# ---- follow-up 1: function ----
# Answer: Same scan, but track the smallest absolute
# difference |left - (total - left - h)| and the first
# index that reaches it. If a perfectly balanced cut
# exists it wins with difference 0.
def closest_cut(heights):
    total, left = sum(heights), 0
    best, best_i = float("inf"), -1
    for i, h in enumerate(heights):
        gap = abs(left - (total - left - h))
        if gap < best:
            best, best_i = gap, i
        left += h
    return best_i
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(400):
        h = [
            random.randint(-3, 4)
            for _ in range(random.randint(0, 8))
        ]
        if h:
            gaps = [
                abs(sum(h[:i]) - sum(h[i + 1 :]))
                for i in range(len(h))
            ]
            assert closest_cut(h) == gaps.index(
                min(gaps)
            )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 123 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 3 of 8

Template fit: the template as written: left total equals total - left - x | asked 394 days ago | frequency Single report

EASY | IN APP LC 724 | ColorStack | technical interview | 123 of 288

Find the Pivot Index (Balanced Split) in O(n)

ASKED 2025-09-21 | 394 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 134/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: your app's interview approach (LeetCode #724)

LeetCode match: LC 724 Find Pivot Index

Frequency: Single report | Last asked: 2025-09-21 | Roles: SWE Intern

QUESTION

Given an integer array nums , return the leftmost index i such that the sum of all elements strictly to the left of i equals the sum of all elements strictly to the right of i . The element at i itself is not counted on either side. For the first index, the left sum is 0 ; for the last index, the right sum is 0 . If no such index exists, return -1 . Your solution must run in O(n) time and O(1) extra space.

Function signature

```
def pivot_index(nums: list[int]) -> int:
    ...
```

Follow-up (as reported): If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

EXAMPLES

Example 1	
Input	`nums = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.`

Example 2	
Input	`nums = [1, 2, 3]`
Output	`-1`
Why	`No index balances the two sides.`

Example 3	
Input	`nums = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is 0; right is 1 + (-1) = 0.`

CONSTRAINTS

QUESTION

- 1 <= len(nums) <= 10^4
- -1000 <= nums[i] <= 1000

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_pivot(nums):
    for i in range(len(nums)):
        if sum(nums[:i]) == sum(nums[i + 1 :]):
            return i
    return -1
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each index re-adds the whole array.

OPTIMAL SOLUTION · Running left sum

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Several pivots can exist: return all of them.

Do not return at the first match; collect every index where the left sum equals the right sum while the scan continues. Still O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Pivot Index (Balanced Split) in
# O(n)
#
# Given an integer array nums , return the leftmost index
# i such that the sum of all elements strictly to the left
# of i equals the sum of all elements strictly to the
# right of i . The element at i itself is not counted on
# either side. For the first index, the left sum is 0 ;
# for the last index, the right sum is 0 . If no such
# index exists, return -1 . Your solution must run in O(n)
# time and O(1) extra space. Function signature: def
# pivot_index(nums: list[int]) -> int: ... Follow-up (as
# reported): If several pivots exist, return them all.
# Hint: keep scanning after the first match, maintaining
# the running left sum.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Several pivots can exist: return all of them.
```

```
#
# INTERVIEW NOTES
# At the pivot, left sum equals total minus left sum minus
# the element. So sweep once keeping the running left sum
# and compare it with total - left - nums[i]; no second
# array is needed, which gives O(1) extra space.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Running left sum
# Idea: The right sum is total - left - nums[i], so one
# pass is enough.
def pivot_index(nums):
    total, left = sum(nums), 0
```

```
    for i, x in enumerate(nums):
        if left == total - left - x:
            return i
        left += x
    return -1

# ---- main question: tests ----
def test_main_question():
    random.seed(19)
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1
    assert pivot_index([2, 1, -1]) == 0


# ============================================================
# FOLLOW-UP 1: Several pivots can exist: return all of
# ============================================================

# ---- follow-up 1: function ----
# Answer: Do not return at the first match; collect every
# index where the left sum equals the right sum while the
# scan continues. Still O(n).
def all_pivots(nums):
    total, left, found = sum(nums), 0, []
    for i, x in enumerate(nums):
        if left == total - left - x:
            found.append(i)
        left += x
    return found

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(400):
        nums = [
            random.randint(-3, 3)
            for _ in range(random.randint(0, 8))
        ]
        slow = [
            i
            for i in range(len(nums))
            if sum(nums[:i]) == sum(nums[i + 1 :])
        ]
```

```
        assert all_pivots(nums) == slow
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 124 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 4 of 8

Template fit: next[i] = next occurrence of the same value; reach[i] = smallest next over i..end; the range i..r is all distinct when reach[i] > r | asked 21 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 124 of 288

Range Duplicate Check with Constant-Time Queries

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 271/288**

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

QUESTION

Preprocess an array once so that each query unique(l, r) answers in O(1) whether all elements from index l to index r (inclusive) are different.

Function signature

class RangeUnique:
  def __init__(self, a: list)
  def unique(self, l: int, r: int) -> bool

EXAMPLES

Example 1

Input	`ru = RangeUnique([1, 2, 3, 1, 4])`
	`ru.unique(0, 2)`
	`ru.unique(0, 3)`
	`ru.unique(1, 4)`

Output	`ru.unique(0, 2) -> true`
	`ru.unique(0, 3) -> false`
	`ru.unique(1, 4) -> true`

Why	`Indexes 0..3 hold 1, 2, 3, 1: the value 1 repeats. Ranges 0..2 and 1..4 have no repeats.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

BRUTE FORCE

TIME	SPACE
O(r - l) per query	**O(r - l)**

WHY IT IS SLOW

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

TIME	SPACE
O(n) preprocessing, O(1) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Range Duplicate Check with Constant-Time
# Queries
#
# Preprocess an array once so that each query unique(l, r)
# answers in O(1) whether all elements from index l to
# index r (inclusive) are different.
#
# Signature:
#   class RangeUnique:
#       def __init__(self, a: list)
#       def unique(self, l: int, r: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array changes between queries.
#
# INTERVIEW NOTES
# A range [l, r] has a duplicate exactly when some index i
# in it has its next equal element at an index <= r. Let
# next[i] be the next index holding the same value and
# reach[l] the smallest next[i] over all i >= l. Then the
# range is all-unique iff reach[l] > r, a single
# comparison per query after one linear pass.

# ====================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")

# ---- main question: main function ----
# EASY VERSION (say it first)
def unique_with_set(a, l, r):
    # easy version: put the range in a set
    return len(set(a[l:r + 1])) == r - l + 1
```

```
# STAGE 2: sliding window
def longest_unique_window(a):
    # stage 2: sliding window with a counter
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
        seen[x] += 1
        # shrink until x occurs only once
        while seen[x] > 1:
            seen[a[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best


# BASE QUESTION (optimal): Next-occurrence array with suffix minimum
# Idea: Scan right to left with a map from value to its latest index to fill next[i] (infinity if none). Then reach[i] = min(next[i], reach[i + 1]). Query(l, r) is reach[l] > r.
# FOLLOW-UP 1: The array changes between queries.
# Answer: Rebuild next and reach in O(n) after a batch of changes when reads dominate.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
```

```
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r
```

```
    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()

# ---- main question: tests ----
def test_main_question():
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            plain = unique_with_set(a, l, r)
            assert q.unique(l, r) == plain
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4
    # an update can create or remove a duplicate
    q.update(3, 9)
    assert q.unique(0, 4)
    q.update(0, 9)
    assert not q.unique(0, 3)


# ============================================================
# FOLLOW-UP 1: The array changes between queries.
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

STUDY PATH: question 125 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 5 of 8

Template fit: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip | asked 138 days ago

MEDIUM | PracHub | Technical Screen | 125 of 288

Find a Horizontal Cut That Bisects Rectangle Area

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 193/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #850 Rectangle Area II

LeetCode match: No direct match; related LC 850 Rectangle Area II (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-08-07)

Original: https://prachub.com/coding-questions/find-a-horizontal-cut-that-bisects-rectangle-area

QUESTION

Given many disjoint axis-aligned rectangles (x1, y1, x2, y2), return the y of the horizontal line that splits their combined area in half, so the area below the line equals the area above it. Return an exact fraction (no floating-point error).

Function signature

def bisecting_cut(rects: list) -> Fraction

EXAMPLES

Example 1	
Input	`[(0, 0, 2, 2)]`
Output	`1`
Why	`One 2x2 square: the middle line is y = 1.`

Example 2	
Input	`[(0, 0, 1, 1), (0, 2, 1, 4)]`
Output	`5/2`
Why	`Areas 1 and 2, total 3, so half is 1.5: all of the first rectangle (area 1) plus 0.5 of the second, which starts at y = 2: y = 5/2.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(rects):
    total = sum(area(r) for r in rects)
    lo = lowest_edge(rects)
    hi = highest_edge(rects)
    for _ in range(100):
        mid = (lo + hi) / 2
        if area_below(rects, mid) >= total / 2:
            hi = mid
        else:
            lo = mid
    return hi
```

TIME	SPACE
O(R * iterations)	O(1)

WHY IT IS SLOW

Each step sums over all rectangles and the answer is only as precise as floating point allows. A sweep over sorted edges with exact fractions finds the slab and the cut directly in O(R log R).

OPTIMAL SOLUTION · Sweep the y events and interpolate inside one slab

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

FOLLOW-UP 2 Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

FOLLOW-UP 3 A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

FOLLOW-UP 4 A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by p * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import defaultdict
from fractions import Fraction


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def total_area(rects):
    # rects are disjoint (x1, y1, x2, y2)
    return sum((x2 - x1) * (y2 - y1)
               for x1, y1, x2, y2 in rects)

# ---- main question: main function ----
# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths.
# FOLLOW-UP 4: A cut that leaves a given fraction of the
# area below it, not exactly half.
# Answer: Replace 'half of the total' by p * total in the
# slab test; the interpolation formula inside the slab is
# the same. The code returns an exact Fraction and is
# checked by computing the area below the returned cut.
def cut_height(rects, share=Fraction(1, 2)):
    # FOLLOW-UP 4: share= leaves any fraction below the
    # cut
    # the height with `share` of the total area below it
    # (1/2 = bisect). Exact Fractions, never floats.
    want = share * total_area(rects)
    # width gained when the sweep passes each height
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
```

```
    ys = sorted(events)
    width, so_far = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        # inside this slab the covered width is constant
        width += events[y0]
        gain = width * (y1 - y0)
        if width and so_far + gain >= want:
            # the cut is inside this slab: interpolate
            return y0 + (want - so_far) / width
        so_far += gain
    return Fraction(ys[-1])

# ---- main question: tests ----
def test_main_question():
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    assert cut_height(stacked) == 1
    # one tall rectangle: halfway up
    assert cut_height([(0, 0, 1, 4)]) == 2
    # a gap between rectangles: the cut sits at the first
    # top
    gap = [(0, 0, 1, 1), (0, 5, 1, 6)]
    assert cut_height(gap) == 1
    # the cut can fall inside a slab with a fractional
    # height
    odd = [(0, 0, 3, 1), (0, 1, 1, 2)]
    y = cut_height(odd)
    assert area_below(odd, y) * 2 == total_area(odd)
    # a different share: 1/4 of the area below
    q = cut_height(stacked, Fraction(1, 4))
    below = area_below(stacked, q)
    assert below * 4 == total_area(stacked)
    # large coordinates stay exact
    big = [(0, 0, 10 ** 9, 10 ** 9)]
    assert cut_height(big) == Fraction(10 ** 9, 2)


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 2.
```

```
# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 3.

# ============================================================
# FOLLOW-UP 4: A cut that leaves a given fraction of the
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 4.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything above is defined)
# ============================================================
def _area_below(rects, y):
    return sum(
        (x2 - x1) * max(0, min(y, y2) - y1)
        for x1, y1, x2, y2 in rects
    )


def bisect_cut_search(rects, iterations=100):
    total = sum(
        (x2 - x1) * (y2 - y1)
        for x1, y1, x2, y2 in rects
    )
    lo = min(r[1] for r in rects)
    hi = max(r[3] for r in rects)
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if 2 * _area_below(rects, mid) >= total:
            hi = mid
```

```
        else:
            lo = mid
    return hi


# HELPERS for the checks below
def area_below(rects, y):
    # exact area under the line at height y (used to
    # check)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
```

```
            total += (x2 - x1) * (top - y1)
    return total


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def horizontal_bisector(rectangles: list[list[float]]) ->
float:
```

Sample call

```
horizontal_bisector([[0, 0, 4, 2]])
```

Test cases

Input	Expected
`([[0, 0, 4, 2]],)`	`1.0`
`([[0, 0, 2, 4], [10, 1, 2, 2]],)`	`2.0`

ALSO REPORTED · 1 more

Find where to cut cake into equal areas | PracHub | Technical Screen

https://prachub.com/coding-questions/find-where-to-cut-cake-into-equal-areas

STUDY PATH: question 126 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 6 of 8

Template fit: prefix sums plus first/last position maps: look up the value whose removal would balance both sides | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 126 of 288

Check pivot after removing one element

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 133/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #724 Find Pivot Index

LeetCode match: LC 724 Find Pivot Index (variant: one removal) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/check-pivot-after-removing-one-element

QUESTION

A pivot index of an array is an index whose left-side sum equals its right-side sum (the pivot element itself is excluded). Remove exactly one element from the array, then ask whether the remaining array has a pivot index. Return one pair (removed index, pivot index), both as indexes in the original array, or None if no removal creates a pivot. Negative numbers and duplicates are allowed.

Function signature

def pivot_after_removal(a: list) -> tuple | None

EXAMPLES

Example 1

Input	`[2, 1, -1, 3]`
Output	`(0, 3)`
Why	`Removing the 2 at index 0 leaves [1, -1, 3]; the 3 (original index 3) has left sum 1 + (-1) = 0 and nothing on its right.`

Example 2

Input	`[1, 2, 3]`
Output	`null`
Why	`Whichever element is removed, no pivot appears.`

Example 3

Input	`[0, 0]`
Output	`(0, 1)`
Why	`Remove either 0: the other one is a pivot of an array with one element.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    for r in range(len(a)):
        b = a[:r] + a[r + 1:]
        for i in range(len(b)):
            if sum(b[:i]) == sum(b[i + 1:]):
                return True
    return False
```

TIME	SPACE
O(n^3)	O(n)
O(n^2) with prefix sums	

WHY IT IS SLOW

n removals times n pivots times a sum: O(n^3) (O(n^2) with prefix sums). For each pivot the value to remove is determined by prefix sums, so a dictionary lookup makes it O(n).

OPTIMAL SOLUTION · Closed-form value to remove, found in a dictionary of positions

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

FOLLOW-UP 2 At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

FOLLOW-UP 3 Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check pivot after removing one element
#
```

```
# A pivot index of an array is an index whose left-side
# sum equals its right-side sum (the pivot element itself
# is excluded). Remove exactly one element from the array,
# then ask whether the remaining array has a pivot index.
# Return one pair (removed index, pivot index), both as
# indexes in the original array, or None if no removal
# creates a pivot. Negative numbers and duplicates are
# allowed.
#
# Signature:
#   def pivot_after_removal(a: list) -> tuple | None
#
# Examples:
# [2, 1, -1, 3]  ->  (0, 3)
# [1, 2, 3]  ->  null
#
# FOLLOW-UPS (answered below, in this order):
# 1. Without any removal (LeetCode 724).
# 2. At most one removal (removal optional).
# 3. Return which element to remove and the pivot.
#
# INTERVIEW NOTES
# Use prefix sums. For a pivot at original index i and a
# removed element on one side, the balance equation fixes
# the value that has to be removed: it is a closed form in
# the prefix sum, the total and a[i]. So for every
# candidate pivot you only have to ask 'does that value
# occur on the correct side?', which a dictionary of first
# and last positions answers in O(1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import itertools


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
def pivot_after_removal(a):
    # remove ONE element, then is there a pivot: an
    # element
    # whose left sum equals its right sum? Returns
    # (removed
    # index, pivot index) or None. O(n) with two scans.
    n, total = len(a), sum(a)
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    # key[j]: left + left + a[j] when nothing is removed
    key = [2 * prefix[j] + a[j] for j in range(n)]
    # pivot j AFTER the removed index i:
    #   2 * prefix[j] - a[i] = total - a[j]  -> a[i] = key
    # - total
    where, ok = {}, None
    for j in range(n):
        i = where.get(key[j] - total)
        if i is not None:
            return i, j
        where[a[j]] = j
    # pivot j BEFORE the removed index i:
    #   2 * prefix[j] + a[j] = total - a[i]  -> a[i] =
    # total - key
    where = {}
    for j in range(n - 1, -1, -1):
        i = where.get(total - key[j])
        if i is not None:
            return i, j
        where[a[j]] = j
    return ok

# ---- main question: tests ----
def test_main_question():
    # [1,7,3,6,5,6] has a pivot at index 3 without
    # removal,
    # but removing one element never gives another
    assert pivot_after_removal([1, 7, 3, 6, 5, 6]) is None
```

```
# ============================================================
# FOLLOW-UP 1: Without any removal (LeetCode 724).
# ============================================================

# ---- follow-up 1: function ----
# Answer: One pass: the pivot is the index where left sum
# equals total minus left minus the element. O(n), O(1).
def pivot_index(a):
    # LeetCode 724: no removal, O(1) extra space
    total, left = sum(a), 0
    for i, x in enumerate(a):
        if left == total - left - x:
            return i
        left += x
    return -1

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # [1, 1, 1, 1] has no pivot, but removing the element
    # at
    # index 1 leaves [1, 1, 1] whose middle element is one
    assert pivot_index([1, 1, 1, 1]) == -1
    assert pivot_after_removal([1, 1, 1, 1]) == (1, 2)
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1


# ============================================================
# FOLLOW-UP 2: At most one removal (removal optional).
# ============================================================

# ---- follow-up 2: function ----
# Answer: Answer true if pivot_index finds a pivot,
# otherwise run the removal test above.
def has_pivot(a, allow_removal=True):
    # follow-up: removal is optional
    if pivot_index(a) != -1:
        return True
    if not allow_removal:
        return False
    return pivot_after_removal(a) is not None

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert not has_pivot([1, 2, 3], allow_removal=False)
```

```
    assert has_pivot([1, 1, 1, 1])
    ones = [1, 1, 1, 1]
    assert not has_pivot(ones, allow_removal=False)


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> bool:
```

Sample call

```
solution([1, 2])
```

Test cases

Input	Expected
([1, 2],)	True
([2, 1, 1, 2],)	True

STUDY PATH: question 127 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 7 of 8

Template fit: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link | asked 182 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 127 of 288

Gold Chain Split into Two Equal Halves

ASKED 2026-04-21 | 182 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 261/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

A gold chain is an array of segment weights. Remove exactly one segment, reconnect the rest, then cut the chain once so that the two parts have equal total weight. Return whether this is possible for some choice of the removed segment and the cut.

Function signature

def gold_chain(a: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4]`
Output	`true`
Why	`Remove the 4: [1, 2, 3] splits as [1, 2] and [3], both 3.`

Example 2	
Input	`[1, 1, 5]`
Output	`true`
Why	`Remove the 5: [1, 1] splits into 1 and 1.`

Example 3	
Input	`[1, 2, 4]`
Output	`false`
Why	`Removing 1 leaves 6 (halves of 3 need [2] | [4]: no), removing 2 leaves 5, removing 4 leaves 3: nothing works.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Gold Chain Split into Two Equal Halves
#
# A gold chain is an array of segment weights. Remove
# exactly one segment, reconnect the rest, then cut the
# chain once so that the two parts have equal total
# weight. Return whether this is possible for some choice
# of the removed segment and the cut.
#
# Signature:
#   def gold_chain(a: list) -> bool
#
# Examples:
# [1, 2, 3, 4]  ->  true
# [1, 1, 5]  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. List every valid (removed segment, cut position)
# pair.
# 2. The chain is a circle (cut anywhere).
# 3. Remove k segments instead of one.
#
# INTERVIEW NOTES
# After removing segment r the target half is (S - a[r]) /
# 2, and a cut means some prefix of the remaining chain
# has exactly that weight. A prefix of the remaining chain
# is either an ordinary prefix of the original array
# (stops before r) or an original prefix minus a[r]
# (passes over r). Each is one value to look up in a hash
# of prefix sums, so every r costs O(1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import itertools


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P

# ---- main question: main function ----
# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[j] be the sum of the first j segments.
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False

# ---- main question: tests ----
def test_main_question():
    # remove the middle 1, then cut 5 | 5
    assert gold_chain([5, 1, 5])
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
```

```
    assert not gold_chain([1, 2, 4])


# ============================================================
# FOLLOW-UP 1: List every valid (removed segment, cut
# ============================================================

# ---- follow-up 1: function ----
# Answer: The answer itself can be quadratic, so a simple
# loop is acceptable.
def valid_pairs(a):
    # follow-up: every (removed index, cut position) pair;
    # the answer can be quadratic, so a loop is fine
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total, run = sum(rest), 0
        # cut after `cut` segments of the reconnected
        # chain
        for cut in range(1, len(rest)):
            run += rest[cut - 1]
            if run * 2 == total:
                pairs.append((r, cut))
    return pairs

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert (1, 1) in valid_pairs([1, 2, 1])
    for chain in itertools.product(range(1, 5), repeat=5):
        assert gold_chain(list(chain)) == bool(
            valid_pairs(list(chain)))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill The equal-subset-sum and
partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

STUDY PATH: question 128 of 288 | topic 15 of 43: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) | 8 of 8

Template fit: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points) | no date

MEDIUM | Exponent | 128 of 288

Count the number of ways to split a string.

ASKED (ANY CO.) ~3 YEARS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 208/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1573 Number of Ways to Split a String (likely) (inferred from short statement)

Last asked (any company): 3 years ago, about 2023-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/3256/count-ways-to-split-string

QUESTION

A binary string s is split into three non-empty parts by two cuts. Count the ways in which the three parts contain the same number of 1s, and return the count modulo 1,000,000,007.

Function signature

def numWays(s: str) -> int

EXAMPLES

Example 1	
Input	`'10101'`
Output	`4`
Why	`Four ways to split '10101' so each part has one 1 (for example 1|010|1 and 10|10|1).`

Example 2	
Input	`'1001'`
Output	`0`
Why	`Two 1s cannot be divided into three equal groups.`

Example 3	
Input	`'0000'`
Output	`3`
Why	`No 1s: any two cuts work: choose 2 of the 3 gaps = 3 ways.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n, count = len(s), 0
    for a in range(1, n - 1):
        for b in range(a + 1, n):
            parts = (s[:a], s[a:b], s[b:])
            ones = {p.count("1") for p in parts}
            if len(ones) == 1:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

O(n^2) pairs. The cuts only depend on where the ones are, so the answer is a product of two gap sizes, computed in O(n).

OPTIMAL SOLUTION · Count the ones, multiply the gap sizes

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Split into m parts with equal ones.

The same argument: each of the m - 1 cut points sits in the gap after a multiple of k ones, so multiply the gap sizes; with no ones at all it is C(n - 1, m - 1).

FOLLOW-UP 2 Count ways to split into palindromes.

That is a different problem: dp[i] counts splits of the prefix, adding dp[j] for every palindromic s[j:i]; precompute the palindromic substrings with expand-around-center or a DP table in O(n^2).

FOLLOW-UP 3 The numbers get huge.

Return the answer modulo 10^9 + 7 and apply the modulus after every multiplication; in languages with fixed-width integers use 64-bit intermediates. **Cost:** O(n) with modular arithmetic; no big integers are needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count the number of ways to split a
# string.
#
# A binary string s is split into three non-empty parts by
# two cuts. Count the ways in which the three parts
# contain the same number of 1s, and return the count
# modulo 1,000,000,007.
#
# Signature:
#   def numWays(s: str) -> int
#
# Examples:
# '10101'  ->  4
# '1001'  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Split into m parts with equal ones.
# 2. Count ways to split into palindromes.
# 3. The numbers get huge.
#
# INTERVIEW NOTES
# Ask which split question it is before solving. For
# 'equal ones' the answer depends only on where the ones
# are: the cuts have to fall inside the stretches of zeros
# that separate the groups of ones, so the count is a
# product of (zeros in the gap + 1). All-zero strings are
# a special case (choose any two cut points).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from math import comb
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
MOD = 10 ** 9 + 7
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Count the ones, multiply the
# gap sizes
# Idea: Let ones be the total number of ones. If it is not
# divisible by 3 the answer is 0; if it is 0, any two of
# the n - 1 cut positions work, C(n - 1, 2).
# FOLLOW-UP 1: Split into m parts with equal ones.
# Answer: The same argument: each of the m - 1 cut points
# sits in the gap after a multiple of k ones, so multiply
# the gap sizes; with no ones at all it is C(n - 1, m -
# 1).
# FOLLOW-UP 3: The numbers get huge.
# Answer: Return the answer modulo 10^9 + 7 and apply the
# modulus after every multiplication; in languages with
# fixed-width integers use 64-bit intermediates. Cost:
# O(n) with modular arithmetic; no big integers are
# needed.
def ways_to_split(s, parts=3, mod=MOD):
    # a binary string cut into `parts` non-empty pieces
    # that
    # hold the same number of 1s (LeetCode 1573 for
    # parts=3)
    n = len(s)
    if n < parts:
        return 0
    # FOLLOW-UP 1: parts=m: m pieces with equal ones
    ones = [i for i, ch in enumerate(s) if ch == "1"]
    if len(ones) % parts:
        return 0
    if not ones:
        # no ones: any choice of parts - 1 cut points
        # works
        total = comb(n - 1, parts - 1)
        return total % mod if mod else total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        # the cut lies in the run of zeros between the
        # last
        # 1 of piece `cut` and the first 1 of the next
        # piece
        gap = ones[cut * each] - ones[cut * each - 1]
        answer *= gap
        # FOLLOW-UP 3: huge counts: answer modulo 10^9 + 7
```

```
        if mod:
            answer %= mod
    return answer

# ---- main question: tests ----
def test_main_question():
    assert ways_to_split("10101") == 4
    assert ways_to_split("1001") == 0
    assert ways_to_split("0000") == 3
    assert ways_to_split("100100010100110") == 12
    assert ways_to_split("1100", parts=2) == 1
    assert ways_to_split("0000", parts=2) == 3
    assert ways_to_split("", parts=3) == 0
    # the modulus keeps the answer small for long strings
    big = "0" * 5000
    assert ways_to_split(big) == comb(4999, 2) % MOD
    assert ways_to_split(big, mod=None) == comb(4999, 2)


# ============================================================
# FOLLOW-UP 1: Split into m parts with equal ones.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Count ways to split into palindromes.
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is a different problem: dp[i] counts splits
# of the prefix, adding dp[j] for every palindromic
# s[j:i]; precompute the palindromic substrings with
# expand-around-center or a DP table in O(n^2).
def count_palindrome_splits(s):
    # follow-up: split into palindromes. dp[i] = ways to
    # cut
    # the first i letters; add dp[j] for every palindrome
    # s[j:i]. The table of palindromes is built first.
    n = len(s)
    pal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            inner = j - i < 2 or pal[i + 1][j - 1]
            pal[i][j] = s[i] == s[j] and inner
    dp = [1] + [0] * n
```

```
    for i in range(1, n + 1):
        for j in range(i):
            if pal[j][i - 1]:
                dp[i] += dp[j]
```

```
    return dp[n]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_palindrome_splits("aab") == 2
    assert count_palindrome_splits("aaa") == 4
    assert count_palindrome_splits("") == 1


# ============================================================
# FOLLOW-UP 3: The numbers get huge.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 16 OF 43

Sorted intervals: merge while they touch, then read the gaps or search

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	5	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sorted intervals: merge while they touch, then read the gaps or search | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	129. Check an Interval Against Existing Intervals for Conflicts	PracHub	2026-08-08
M	130. Find Containing Range	PracHub	2026-06-04
M	131. Union of Sorted Interval Lists	1Point3Acres	2026-08-16
M	132. Range Module (Insert and Point Query)	ColorStack	2026-01-20
M	133. Meetings Minus a Do-Not-Schedule Block	ColorStack	2025-10-03
M	134. Merge Intervals	Exponent	no date
H	135. Find safe travel intervals between planet influences	PracHub	2025-11-22
H	136. Implement a function that processes a stream of interval data.	Exponent	no date

CODE TEMPLATE | GROUP 16 OF 43 | 8 QUESTIONS, ONE PATTERN

Sorted intervals: merge while they touch, then read the gaps or search

The one idea these questions share

Sort by start. Walk once: if the next interval starts before the last one ends, extend the last; otherwise start a new one. The merged list is sorted and disjoint, so gaps are the spaces between neighbours and any point can be found with bisect.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:            # CHANGE 1: overlap or touch? (< to keep touching ones apart)
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

2. Helper functions that usually get written next to it

```
from bisect import bisect_left, bisect_right


def gaps(merged, lo=float("-inf"), hi=float("inf")):
    edges = [lo] + [x for iv in merged for x in iv] + [hi]
    return [(edges[i], edges[i + 1]) for i in range(0, len(edges), 2)]


def containing(merged, x):
    starts = [s for s, _ in merged]
    i = bisect_right(starts, x) - 1
    return tuple(merged[i]) if i >= 0 and merged[i][1] >= x else None
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
129	Check an Interval Against Existing Intervals for Conflicts	bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end
130	Find Containing Range	merge once, keep the starts in a list, and bisect to find the one interval that could contain x
131	Union of Sorted Interval Lists	walk both sorted lists with two indices, always taking the smaller start next, then the same merge test
132	Range Module (Insert and Point Query)	keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval
133	Meetings Minus a Do-Not-Schedule Block	merge first, then for each merged meeting keep the part before the blocked window and the part after it
134	Merge Intervals	the template as written
135	Find safe travel intervals between planet influences	turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)
136	Implement a function that processes a stream of interval data.	the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 129 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 1 of 8

Template fit: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end | asked 73 days ago

EASY | IN APP LC 729 | PracHub | Technical Screen | 129 of 288

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 26.5 | 30 DAYS 0 | PRIORITY 153/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Design a calendar of half-open intervals [start, end). book(start, end) books the interval and returns True when it does not overlap any interval booked so far, and returns False (booking nothing) when it does. Intervals that only touch, one ending exactly where the next starts, do not conflict.

Function signature

class Calendar:

def book(self, start: int, end: int) -> bool

EXAMPLES

Example 1

Input	`c = Calendar()`
	`c.book(10, 20)`
	`c.book(15, 25)`
	`c.book(20, 30)`
	`c.book(5, 10)`
	`c.book(12, 14)`

Output	`c.book(10, 20) -> true`
	`c.book(15, 25) -> false`
	`c.book(20, 30) -> true`
	`c.book(5, 10) -> true`
	`c.book(12, 14) -> false`

Why	`[15, 25) overlaps [10, 20) and [12, 14) lies inside it, so both fail. [20, 30) and [5, 10) only touch the booked interval at an endpoint, so they succeed.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check an Interval Against Existing
# Intervals for Conflicts
#
# Design a calendar of half-open intervals [start, end).
# book(start, end) books the interval and returns True
# when it does not overlap any interval booked so far, and
# returns False (booking nothing) when it does. Intervals
# that only touch, one ending exactly where the next
# starts, do not conflict.
#
# Signature:
#   class Calendar:
#       def book(self, start: int, end: int) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Allow double booking but never triple booking
# (LeetCode 731).
# 2. Report the maximum number of overlapping bookings
# after each booking (LeetCode 732).
# 3. Closed intervals instead of half-open.
#
# INTERVIEW NOTES
# Two half-open intervals [s1, e1) and [s2, e2) overlap
# exactly when s1 < e2 and s2 < e1. If the booked
# intervals are kept sorted and disjoint, only the
# neighbours at the insertion point can conflict, so a
# binary search replaces the scan.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_right
from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sorted starts with bisect:
```

```
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True

# ---- main question: tests ----
def test_main_question():
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    # sorted calendar
    c = Calendar()
    assert c.book(10, 20) and c.book(20, 30)
    assert not c.book(15, 25) and c.book(5, 10)


# ============================================================
# FOLLOW-UP 1: Allow double booking but never triple
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a difference map (+1 at start, -1 at end)
# and sweep it in sorted order; if the running count ever
# exceeds 2, undo the change and reject. O(n log n) per
# booking, or use a segment tree for faster.
class DoubleBooking:
    """LeetCode 731: two overlaps are fine, three are not.
    LeetCode 732: also report the largest overlap."""

    def __init__(self, limit=2):
        self.delta = defaultdict(int)
        self.limit = limit

    def peak(self):
        # sweep the +1 / -1 changes in time order
        run = best = 0
        for t in sorted(self.delta):
            run += self.delta[t]
            best = max(best, run)
        return best

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        if self.peak() > self.limit:
            # undo the change and reject the booking
            self.delta[start] -= 1
            self.delta[end] += 1
            return False
        return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # double booking is allowed, triple booking is not
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)
    assert not d.book(17, 18)
    assert d.peak() == 2
    # limit 1 is the plain no-overlap calendar
    one = DoubleBooking(limit=1)
    assert one.book(1, 5) and not one.book(4, 6)


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def conflicts(candidate, intervals):
    # half-open [start, end): touching ends do NOT
    # conflict
```

```
    s, e = candidate
    return any(a < e and s < b for a, b in intervals)
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: l
ist[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
([4,7],[[1,3],[7,9]])	False
([4,7],[[1,5],[8,9]])	True

STUDY PATH: question 130 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 2 of 8

Template fit: merge once, keep the starts in a list, and bisect to find the one interval that could contain x | asked 138 days ago

MEDIUM | PracHub | Technical Screen | 130 of 288

★ TOP 75: rank 16 in the Medium and Hard priority list

★ Find Containing Range

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 65.2** | **30 DAYS 41.1** | **PRIORITY 15/288**

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: No direct match; related LC 56 Merge Intervals + binary search (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/find-containing-range

QUESTION

Given a collection of inclusive ranges [start, end] (overlapping or touching ranges are merged), find(x) returns the merged range that contains the number x, or None when no range contains it.

Function signature

class RangeIndex:
  def __init__(self, ranges: list)
  def find(self, x: int) -> tuple | None

EXAMPLES

Example 1

Input	`r = RangeIndex([(1, 3), (2, 6), (8, 10)])`
	`r.find(5)`
	`r.find(7)`
	`r.find(10)`

Output	`r.find(5) -> (1, 6)`
	`r.find(7) -> null`
	`r.find(10) -> (8, 10)`

Why	`[1, 3] and [2, 6] overlap and merge into [1, 6]; 7 falls in the gap between the merged ranges.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def find_containing(ranges, x):
    for start, end in ranges:
        if start <= x <= end:
            return (start, end)
    return None
```

BRUTE FORCE

TIME	SPACE
O(n) per query	**O(n)**

WHY IT IS SLOW

Every query walks all n ranges. Merging once (O(n log n)) and binary searching the sorted starts makes each query O(log n).

OPTIMAL SOLUTION · Merge once, then binary search the starts

TIME	SPACE
O(n log n) to build, O(log n) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

FOLLOW-UP 2 Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

FOLLOW-UP 3 Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find Containing Range
#
# Given a collection of inclusive ranges [start, end]
# (overlapping or touching ranges are merged), find(x)
# returns the merged range that contains the number x, or
# None when no range contains it.
#
# Signature:
#   class RangeIndex:
#       def __init__(self, ranges: list)
#       def find(self, x: int) -> tuple | None
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return every original range that contains x, not the
# merged one.
# 2. Ranges are inserted and deleted over time.
# 3. Queries are given in bulk.
#
# INTERVIEW NOTES
# Merge the ranges once so they are disjoint and sorted;
# then 'which range contains x' is 'the last range whose
# start is at most x, and does its end reach x'. That is
# one binary search per query. Be clear whether the answer
# is the original range or the merged one, since they
# differ when ranges overlap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, x) - 1 is the only
```

```
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
# FOLLOW-UP 1: Return every original range that contains
# x, not the merged one.
# Answer: Sort by start and consider only ranges starting
# at or before x (one bisect); among those, keep the ones
# whose end is at least x. With many overlaps use an
# interval tree for O(log n + k) per query.
class RangeIndex:
    """Merged, sorted, disjoint closed ranges."""

    def __init__(self, ranges=()):
        self.starts, self.ends = [], []
        self.original = sorted(ranges)
        for s, e in self.original:
            self.insert(s, e)

    # FOLLOW-UP 2: ranges can be inserted over time
    def insert(self, s, e):
        # first range ending at or after s ...
        i = bisect_left(self.ends, s)
        # ... up to the last range starting at or before e
        j = bisect_right(self.starts, e)
        if i < j:
            s = min(s, self.starts[i])
            e = max(e, self.ends[j - 1])
        # replace everything the new range touches
        self.starts[i:j] = [s]
        self.ends[i:j] = [e]

    def find(self, x):
        # the merged range containing x, or None
        i = bisect_right(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return self.starts[i], self.ends[i]
        return None

    # FOLLOW-UP 1: every ORIGINAL range containing x
    def find_original(self, x):
        # every ORIGINAL range that contains x
        k = bisect_right([s for s, _ in self.original], x)
        return [r for r in self.original[:k] if r[1] >= x]

    # FOLLOW-UP 3: queries given in bulk
    def find_many(self, queries):
        # bulk: sort the queries and sweep once
        order = sorted(range(len(queries)),
                       key=lambda i: queries[i])
        out, j = [None] * len(queries), 0
        for i in order:
            x = queries[i]
            # skip ranges that end before x
            while j < len(self.ends) and self.ends[j] < x:
                j += 1
            inside = j < len(self.starts)
            if inside and self.starts[j] <= x:
                out[i] = (self.starts[j], self.ends[j])
        return out

# ---- main question: tests ----
def test_main_question():
    idx = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert idx.find(4) == (1, 6)
    assert idx.find(7) is None
    assert idx.find(8) == (8, 10)
    assert idx.find(0) is None and idx.find(99) is None
    # the original ranges that cover 2
    assert idx.find_original(2) == [(1, 3), (2, 6)]
    # touching ranges merge: (6, 8) joins the first two
    # blocks
    idx.insert(6, 8)
    assert idx.find(7) == (1, 10)
    assert idx.find(12) is None
    qs = [4, 20, 16, 9]
    assert idx.find_many(qs) == [idx.find(q) for q in qs]


# ============================================================
# FOLLOW-UP 1: Return every original range that contains
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(ranges: list[list[int]], queries: list[int])
-> list[list[int]]:
```

Sample call

```
solution([[1, 3], [50, 100]], [2, 50, 10])
```

Test cases

Input	Expected
`([[1, 3], [50, 100]], [2, 50, 10])`	`[[1, 3], [50, 100], None]`
`([[5, 8], [-10, -5], [0, 0]], [-7, 0, 8, 9, -11])`	`[[-10, -5], [0, 0], [5, 8], None, None]`

STUDY PATH: question 131 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 3 of 8

Template fit: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test | asked 65 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 131 of 288

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 160/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Two lists of intervals are each sorted and non-overlapping on their own. Return the union of both lists as sorted, non-overlapping intervals. Intervals that touch, such as (1, 3) and (3, 5), are merged unless touching=False.

Function signature

def union_two(a: list, b: list, touching=True) -> list

EXAMPLES

Example 1	
Input	`[(1, 3), (6, 9)], [(2, 5), (10, 12)]`
Output	`[(1, 5), (6, 9), (10, 12)]`
Why	`(1, 3) and (2, 5) overlap and become (1, 5); (6, 9) and (10, 12) stay.`

Example 2	
Input	`[(1, 3)], [(3, 5)]`
Output	`[(1, 5)]`
Why	`(1, 3) and (3, 5) touch at 3, so they merge.`

Example 3	
Input	`[(1, 3)], [(3, 5)], touching = false`
Output	`[(1, 3), (3, 5)]`
Why	`With touching=False they only merge when they truly overlap.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Union of Sorted Interval Lists
#
# Two lists of intervals are each sorted and non-
# overlapping on their own. Return the union of both lists
# as sorted, non-overlapping intervals. Intervals that
# touch, such as (1, 3) and (3, 5), are merged unless
# touching=False.
#
# Signature:
#   def union_two(a: list, b: list, touching=True) -> list
#
# Examples:
# [(1, 3), (6, 9)], [(2, 5), (10, 12)]  ->  [(1, 5), (6,
#     9), (10, 12)]
# [(1, 3)], [(3, 5)]  ->  [(1, 5)]
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the lists are too big for memory?
# 2. What if one list may overlap itself?
# 3. Do touching intervals merge?
# 4. Each list may overlap itself or be unsorted.
#
# INTERVIEW NOTES
# Each list is already sorted, so you can walk both with
# two pointers like the merge step of merge sort: always
# take the interval with the smaller start next. Once the
# intervals come out in start order, merging is the usual
# one-pass fold: extend the last output interval if the
# next one overlaps or touches it, otherwise start a new
# one.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# FOLLOW-UP 1: What if the lists are too big for memory?
# Answer: Read each list through an iterator (a file or a
# paged API). union_k only ever holds one interval per
# list plus the current merged interval, so memory is O(K)
# and it can emit results as it goes.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)

# ---- main question: main function ----
# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))

# ---- main question: tests ----
```

```
def test_main_question():
    a = [(1, 3), (6, 8)]
    b = [(2, 4), (9, 10)]
    assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
    # closed intervals that touch merge
    assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
    # half-open intervals that only touch stay apart
    assert union_two([(1, 3)], [(3, 5)], False) == [
        (1, 3), (3, 5)]
    # three lists, one of them empty
    three = union_k([a, b, []])
    assert three == [(1, 4), (6, 8), (9, 10)]


# ============================================================
# FOLLOW-UP 1: What if the lists are too big for memory?
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: What if one list may overlap itself?
# ============================================================

# ---- follow-up 2: function ----
# Answer: The two-pointer merge already works, because it
# folds every incoming interval into the last output
# interval.
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))
```

```
# ========================================================
# FOLLOW-UP 3
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ========================================================
# FOLLOW-UP 4: Each list may overlap itself or be
# ========================================================

# ---- follow-up 4: function ----
# Answer: Normalise each list first (sort and merge it by
# itself, O(n log n)), after which the two-pointer union
# applies. Or sort everything together if memory allows.
```

```
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))

# ---- follow-up 4: tests ----
def test_follow_up_4():
    # a list that overlaps itself, and an unsorted one
    merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
    assert merged == [(1, 12)]
    assert union_unsorted([(9, 10), (1, 2)], []) == [
        (1, 2), (9, 10)]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

STUDY PATH: question 132 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 4 of 8

Template fit: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval | asked 273 days ago | frequency Single report

MEDIUM | IN APP LC 715, 56 | ColorStack | technical interview | 132 of 288

Range Module (Insert and Point Query)

ASKED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 14/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: your app's interview approach (LeetCode #715)

LeetCode match: LC 715 Range Module / LC 56 Merge Intervals

Frequency: Single report | Last asked: 2026-01-20 | Roles: SWE Intern

QUESTION

Design a data structure that tracks a set of half-open ranges [left, right) over the number line. add_range(left, right) adds the interval; overlapping or touching intervals are merged so that only disjoint, non-adjacent intervals remain. query_point(x) returns True when x lies inside a tracked interval; because the intervals are half-open, a point equal to a right endpoint is not inside.

Function signature

```
class RangeModule:
    def add_range(self, left: float, right: float) -> None
    def query_point(self, x: float) -> bool
```

EXAMPLES

Example 1

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.query_point(15)`
	`rm.query_point(20)`
	`rm.query_point(10)`

Output	`rm.query_point(15) -> true`
	`rm.query_point(20) -> false`
	`rm.query_point(10) -> true`

Why	`[10, 20) contains 10 and 15 but not 20.`

QUESTION

Example 2

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.add_range(20, 30)`
	`rm.query_point(20)`
	`rm.query_point(25)`
	`rm.query_point(30)`

Output	`rm.query_point(20) -> true`
	`rm.query_point(25) -> true`
	`rm.query_point(30) -> false`

Why	`The touching ranges merge into [10, 30), so 20 is now covered; 30 still is not.`

Example 3

Input	`rm = RangeModule()` `rm.query_point(5)`
Output	`rm.query_point(5) -> false`
Why	`Nothing has been added yet.`

CONSTRAINTS

- Endpoints and query points are integers or floats with -10^9 <= value <= 10^9 .
- For every add_range call, left < right .
- Up to 10^5 total operations.
- Touching intervals merge: add_range(1, 5) followed by add_range(5, 10) yields one interval [1,
- 10) .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class ListRanges:
    def __init__(self):
        self.items = []
    def add_range(self, left, right):
        self.items.append((left, right))
    def query_point(self, x):
        return any(
            a <= x < b for a, b in self.items
        )
```

TIME	SPACE
O(n) per query	O(n)

WHY IT IS SLOW

Every query walks every added interval.

OPTIMAL SOLUTION · Merged intervals with bisect

TIME	SPACE
O(log n) query	O(n)

OPTIMAL SOLUTION · Merged intervals with bisect

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add remove_range(left, right).

Find the block of intervals that overlap [left, right) with bisect, keep the part of the first one left of the removal and the part of the last one right of it, and splice the remainders in.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Range Module (Insert and Point Query)
#
# Design a data structure that tracks a set of half-open
# ranges [left, right) over the number line.
# add_range(left, right) adds the interval; overlapping or
# touching intervals are merged so that only disjoint,
# non-adjacent intervals remain. query_point(x) returns
# True when x lies inside a tracked interval; because the
# intervals are half-open, a point equal to a right
# endpoint is not inside.
#
# Signature:
#   class RangeModule:
#       def add_range(self, left: float, right: float) ->
None
#       def query_point(self, x: float) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add remove_range(left, right).
#
# INTERVIEW NOTES
# Keep the tracked intervals merged and sorted in two
# parallel lists, lefts and rights. For add_range, bisect
# finds the first interval that touches or overlaps the
# new one and the first one that starts after it;
# everything between them collapses into one interval. For
# query_point, bisect on the lefts finds the only interval
# that could contain x.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Merged intervals with bisect
# Idea: Disjoint sorted intervals mean one bisect locates
# the only candidate for a point.
class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []

    def add_range(self, left, right):
        i = bisect_left(self.rights, left)
        j = bisect_right(self.lefts, right)
        if i < j:
            left = min(left, self.lefts[i])
            right = max(right, self.rights[j - 1])
        self.lefts[i:j] = [left]
        self.rights[i:j] = [right]

    def query_point(self, x):
        i = bisect_right(self.lefts, x) - 1
        return i >= 0 and x < self.rights[i]

# ---- main question: tests ----
def test_main_question():
    random.seed(83)
    rm = RangeModule()
    rm.add_range(10, 20)
    assert (
        rm.query_point(15)
        and rm.query_point(10)
        and not rm.query_point(20)
    )
    rm.add_range(20, 30)
    assert (
        rm.query_point(20)
        and rm.query_point(25)
        and not rm.query_point(30)
    )
    assert not RangeModule().query_point(5)


# ============================================================
# FOLLOW-UP 1: Add remove_range(left, right).
# ============================================================

# ---- follow-up 1: function ----
# Answer: Find the block of intervals that overlap [left,
# right) with bisect, keep the part of the first one left
# of the removal and the part of the last one right of it,
# and splice the remainders in.
class RemovableRangeModule(RangeModule):
    def remove_range(self, left, right):
        i = bisect_right(self.rights, left)
        j = bisect_left(self.lefts, right)
        if i >= j:
            return
        keep = []
        if self.lefts[i] < left:
            keep.append((self.lefts[i], left))
        if self.rights[j - 1] > right:
            keep.append((right, self.rights[j - 1]))
        self.lefts[i:j] = [a for a, _ in keep]
        self.rights[i:j] = [b for _, b in keep]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 133 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 5 of 8

Template fit: merge first, then for each merged meeting keep the part before the blocked window and the part after it | asked 382 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 133 of 288

Meetings Minus a Do-Not-Schedule Block

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 13/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals + interval subtraction (related)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

You are given a list of meetings , each a (start, end) interval with start < end , and one "do-not- schedule" (DNS) block (dns_start, dns_end) . Consider the set of all time points covered by at least one meeting. Return the smallest list of non-overlapping, sorted intervals that covers exactly those points except any point lying inside the DNS block (treat the DNS block as removing the open coverage on (dns_start, dns_end) ; equivalently split coverage at those bounds). Meetings may overlap and are given in any order. Return the result sorted by start time.

Function signature

```
def subtract_dns(meetings: list[tuple[int, int]],
                 dns: tuple[int, int]) -> list[tuple[int, int]]:
    ...
```

Follow-up (as reported): Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

EXAMPLES

Example 1	
Input	`meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)`
Output	`[(1,10),(12,20),(35,40),(50,65)]`
Why	`Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40] into [12,20] and [35,40].`

Example 2	
Input	`meetings = [(0,10)], dns = (0, 10)`
Output	`[]`
Why	`The DNS block removes the entire covered range.`

QUESTION

Example 3	
Input	`meetings = [(0,5),(6,9)], dns = (100, 200)`
Output	`[(0,5),(6,9)]`
Why	`The DNS block covers nothing that was scheduled.`

CONSTRAINTS

- 1 <= len(meetings) <= 10^5 .
- Coordinates are integers; 0 <= start < end <= 10^9 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_subtract(meetings, dns):
    units = set()
    for s, e in meetings:
        units.update(range(s, e))
    units -= set(range(dns[0], dns[1]))
    out = []
    for t in sorted(units):
        if out and out[-1][1] == t:
            out[-1] = (out[-1][0], t + 1)
        else:
            out.append((t, t + 1))
    return out
```

TIME	SPACE
O(total length)	O(total length)

WHY IT IS SLOW

It touches every unit of time covered, which is huge when times are large.

OPTIMAL SOLUTION · Merge, then cut

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are several do-not-schedule blocks. How do you subtract all of them?

Merge both the meetings and the blocks. Walk the merged meetings with a pointer into the blocks: keep the part of the meeting before the next block, skip past the block, and continue with what is left. O((n + b) log (n + b)).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Meetings Minus a Do-Not-Schedule Block
#
# You are given a list of meetings , each a (start, end)
# interval with start < end , and one "do-not- schedule"
# (DNS) block (dns_start, dns_end) . Consider the set of
# all time points covered by at least one meeting. Return
# the smallest list of non-overlapping, sorted intervals
# that covers exactly those points except any point lying
# inside the DNS block (treat the DNS block as removing
# the open coverage on (dns_start, dns_end) ; equivalently
# split coverage at those bounds). Meetings may overlap
# and are given in any order. Return the result sorted by
# start time. Function signature: def
# subtract_dns(meetings: list[tuple[int, int]], dns:
# tuple[int, int]) -> list[tuple[int, int]]: ... Follow-up
# (as reported): Generalize to a list of non-overlapping
# DNS blocks: merge the meetings first, then walk both
# sorted lists together subtracting each block.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are several do-not-schedule blocks. How do you
# subtract all of them?
#
# INTERVIEW NOTES
# Merge the meetings into disjoint intervals first, then
# cut out the do-not-schedule block: a merged interval
# keeps a left piece [start, min(end, dns_start)] if it
# starts before the block, and a right piece [max(start,
# dns_end), end] if it ends after it. Several blocks
# become a walk over two sorted lists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Merge, then cut
```

```
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def merge_intervals(meetings):
    merged = []
    for s, e in sorted(meetings):
        if merged and s <= merged[-1][1]:
            merged[-1] = (
                merged[-1][0],
                max(merged[-1][1], e),
            )
        else:
            merged.append((s, e))
    return merged
```

```
# BASE QUESTION (optimal): Merge, then cut
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def subtract_dns(meetings, dns):
    lo, hi = dns
    out = []
    for s, e in merge_intervals(meetings):
        if s < lo:
            out.append((s, min(e, lo)))
        if e > hi:
            out.append((max(s, hi), e))
    return out

# ---- main question: tests ----
def test_main_question():
    assert merge_intervals([(1, 3), (2, 6), (8, 10)]) == [
(1, 6), (8, 10)]
    assert subtract_dns([(1, 5), (7, 12)], (4, 9)) == [(1,
 4), (9, 12)]


# ============================================================
# FOLLOW-UP 1: There are several do-not-schedule blocks.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge both the meetings and the blocks. Walk the
# merged meetings with a pointer into the blocks: keep the
# part of the meeting before the next block, skip past the
# block, and continue with what is left. O((n + b) log (n
# + b)).
def subtract_many(meetings, blocks):
```

```
    out, blocks = [], merge_intervals(blocks)
    for s, e in merge_intervals(meetings):
        cur = s
        for lo, hi in blocks:
            if hi <= cur or lo >= e:
                continue
            if lo > cur:
                out.append((cur, lo))
            cur = max(cur, hi)
        if cur < e:
            out.append((cur, e))
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 134 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 6 of 8

Template fit: the template as written | no date

MEDIUM | IN APP LC 56 | Exponent | 134 of 288

★ TOP 75: rank 11 in the Medium and Hard priority list

★ Merge Intervals

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 12/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: your app's interview approach (LeetCode #56)

LeetCode match: LC 56 Merge Intervals

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +1219 interviews

Original: https://www.aced.io/questions/1602/merge-intervals

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each

interval, compare it with the last interval in th

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 6], [8, 10], [15, 18]]`
Output	`[[1, 6], [8, 10], [15, 18]]`
Why	`[1, 3] and [2, 6] overlap and merge into [1, 6].`

QUESTION

Example 2	
Input	`[[1, 4], [4, 5]]`
Output	`[[1, 5]]`
Why	`Intervals that touch at an endpoint are merged as well.`

Example 3	
Input	`[[8, 10], [1, 3], [2, 4]]`
Output	`[[1, 4], [8, 10]]`
Why	`Unsorted input is fine.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

TIME	SPACE
O(n^2) or worse	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Sort by start and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Insert one interval into an already merged list (LeetCode 57).

Copy the intervals that end before it, merge the ones that overlap, copy the rest: O(n) with no sort.

FOLLOW-UP 2 Intervals of touching integers ([1, 2] and [3, 4]) should merge.

Change the overlap test to start <= last_end + 1. For real-valued ranges that would be wrong, so state which you assume.

FOLLOW-UP 3 Too many intervals for memory.

Sort externally (external merge sort) and stream the merge: the one-pass sweep needs only the current interval in memory. **Cost:** O(n log n) external sort with bounded memory runs, then an O(n) streaming merge with O(1) memory.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Merge Intervals
#
# Merge Intervals. An interval is a representation of a
# range of values along a number line. For example, [1,3]
# represents all numbers between 1 and 3, inclusive. Given
# a collection of intervals, the task is to merge all
# overlapping intervals. For example, consider the
# intervals [1,3],[2,6],[8,10],[15,18]. The function
# should return merged intervals where overlapping
# intervals are combined. In this case, intervals [1,3]
# and [2,6] overlap, so they should be merged into [1,6].
# Examples intervals = [[1,3], [2,6], [8,10], [15,18]]
# output: [[1,6], [8,10], [15,18]] intervals = [[1,4],
# [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]]
# output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]]
# output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals
# function takes an array of intervals and merges all
# overlapping intervals into a single interval.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Insert one interval into an already merged list
# (LeetCode 57).
# 2. Intervals of touching integers ([1, 2] and [3, 4])
# should merge.
# 3. Too many intervals for memory.
#
# INTERVIEW NOTES
# After sorting by start, an interval can overlap only the
# interval just before it in the output, because
# everything earlier ends before that one starts. So one
# pass with a 'current merged interval' is enough: extend
# its end, or start a new one.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort by start and sweep
# Idea: Sort by start. If an interval starts at or before
# the end of the last output interval, extend that end
# with max; otherwise append it. The sort guarantees
# nothing earlier can overlap the new interval.
def merge_intervals(intervals, touching_ints=False):
    out = []
    # sort by start, then extend the last interval while
    # they overlap
    for s, e in sorted(intervals):
        gap = 1 if touching_ints else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out

# ---- main question: tests ----
def test_main_question():
    spans = [[1, 3], [2, 6], [8, 10], [15, 18]]
    got = merge_intervals(spans)
    assert got == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    got = merge_intervals([[1, 2], [3, 4]])
    assert got == [[1, 2], [3, 4]]
    got = merge_intervals([[1, 2], [3, 4]], True)
    assert got == [[1, 4]]


# ============================================================
# FOLLOW-UP 1: Insert one interval into an already merged
# ============================================================

# ---- follow-up 1: function ----
# Answer: Copy the intervals that end before it, merge the
# ones that overlap, copy the rest: O(n) with no sort.
def insert_interval(merged, new):
    # LeetCode 57: already merged and sorted, so O(n) and
    # no sort
    out, i, n = [], 0, len(merged)
    # 1) everything that ends before the new one starts
```

```
    while i < n and merged[i][1] < new[0]:
        out.append(merged[i])
        i += 1
    # 2) everything that overlaps it is absorbed
    s, e = new
    while i < n and merged[i][0] <= e:
        s, e = min(s, merged[i][0]), max(e, merged[i][1])
        i += 1
    out.append([s, e])
    # 3) the rest
    return out + merged[i:]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    got = insert_interval([[1, 3], [6, 9]], [2, 5])
    assert got == [[1, 5], [6, 9]]
    assert insert_interval([], [5, 7]) == [[5, 7]]


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
```

```
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# =============================================================
# FOLLOW-UP 3: Too many intervals for memory.
# =============================================================

# ---- follow-up 3: function ----
# Answer: Sort externally (external merge sort) and stream
# the merge: the one-pass sweep needs only the current
# interval in memory. Cost: O(n log n) external sort with
# bounded memory runs, then an O(n) streaming merge with
# O(1) memory.
def merge_stream(sorted_intervals):
    # intervals arrive sorted by start (for example after
    # an
    # external sort): only the current one is kept in
    # memory
    cur = None
    for s, e in sorted_intervals:
        if cur and s <= cur[1]:
            cur[1] = max(cur[1], e)
        else:
            if cur:
```

```
                yield tuple(cur)
            cur = [s, e]
    if cur:
        yield tuple(cur)


# Answer: Sort externally (external merge sort) and stream
# the merge: the one-pass sweep needs only the current
# interval in memory. Cost: O(n log n) external sort with
# bounded memory runs, then an O(n) streaming merge with
# O(1) memory.
def merge_many_sorted(sources):
    # several already-sorted files: a heap merge, then the
    # sweep
    return list(merge_stream(heapq.merge(*sources)))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    stream = list(merge_stream([(1, 3), (2, 6), (8, 10)]))
    assert stream == [(1, 6), (8, 10)]
    parts = [[(1, 3), (8, 9)], [(2, 4), (9, 12)]]
    assert merge_many_sorted(parts) == [(1, 4), (8, 12)]


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
output: [[1,6], [8,10], [15,18]]

intervals = [[1,4], [4,5]]
output: [[1,5]]

intervals = [[1,4], [2,3]]
output: [[1,4]]

intervals = [[1,2], [3,4], [5,6], [7,8]]
output: [[1,2], [3,4], [5,6], [7,8]]
```

The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in the result list. If the current interval overlaps with the last interval in the result (i.e., the start of the current interval is less than or equal to the end of the last interval), merge them by updating the end of the last interval in the result to the maximum end time of both intervals. If there is no overlap, add the current interval to the result list as a separate interval. Return the Result: After processing all intervals, the result list contains the merged intervals. Time Complexity: The dominant factor in the time complexity is the sorting step, which takes O(n log n), where n is the number of intervals. The merging process involves a single pass through the sorted intervals, which takes O(n) time. Therefore, the overall time complexity is O(n log n). Space Complexity: The space complexity is O(n), which accounts for the space required to store the result list. This is because, in the worst case, none of the intervals overlap, and the result list will contain all the original intervals.In this mock interview video, Lewin (Dropbox SWE), answers the interview question, "Given an array of intervals, merge the overlapping intervals and return an array of the non-overlapping intervals that cover all intervals in the input."No answers contributed by the community yet.

STUDY PATH: question 135 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 7 of 8

Template fit: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends) | asked 332 days ago

HARD | PracHub | Technical Screen | 135 of 288

★ TOP 75: rank 10 in the Medium and Hard priority list

★ Find safe travel intervals between planet influences

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 10/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals (then take gaps)

Posted on PracHub: 2025-11-22 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/find-safe-travel-intervals-between-planet-influences

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

QUESTION

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8,
    12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(planets):
    pts = set()
    for c, r in planets:
        pts |= {c - r, c + r}
    bounds = [-INF] + sorted(pts) + [INF]
    safe = []
    for a, b in zip(bounds, bounds[1:]):
        mid = midpoint(a, b)
        if not any(abs(mid - c) < r
                   for c, r in planets):
            safe.append((a, b))
    return safe
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Each midpoint is tested against every planet, O(n^2). Merging the unsafe intervals after sorting and reading the gaps is O(n log n).

OPTIMAL SOLUTION · Merge the unsafe intervals, then read off the gaps

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

FOLLOW-UP 2 Are the endpoints safe?

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

FOLLOW-UP 3 Planets arrive over time.

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find safe travel intervals between planet
# influences
#
# You are planning a space route along a one-dimensional
# line (the x-axis). You are given a list of planets. Each
# planet is represented by an integer pair \((c, r)\): - c
# is the coordinate of the planet's center on the x-axis.
# - r is a non-negative integer radius of its
# gravitational influence. A point x on the x-axis is
# **unsafe** for a space traveler if it lies within the
# gravitational influence of at least one planet: \[ |x -
# c_i| \le r_i \quad \text{for some planet } i \]
# Equivalently, each planet makes the interval \([c_i -
# r_i,\; c_i + r_i]\) unsafe. Task: - Consider the entire
# real line as the possible travel path. - Compute all
# **maximal continuous intervals** on this line where the
# traveler is **safe**, i.e., points not covered by any
# planet's unsafe interval. - Return these safe intervals
# as a sorted list of disjoint intervals.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the part between two given positions matters (a
# route from A to B).
# 2. Are the endpoints safe?
# 3. Planets arrive over time.
#
# INTERVIEW NOTES
# The safe points are the complement of the union of the
# unsafe intervals. Merge the unsafe intervals first (sort
# by start, extend while they overlap or touch), and the
# safe intervals are exactly the gaps between consecutive
# merged intervals plus the two unbounded ends. Endpoints
# themselves are unsafe because the influence is closed
# (<=).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
INF = float("inf")


def unsafe_blocks(planets):
    # planet (c, r) makes [c - r, c + r] unsafe; merge
    # overlaps
    merged = []
    spans = sorted((c - r, c + r) for c, r in planets)
    for a, b in spans:
        # sharing even one point joins two blocks
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    return merged

# ---- main question: main function ----
# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe).
def safe_intervals(planets):
    # the gaps between unsafe blocks (open intervals)
    merged = unsafe_blocks(planets)
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out

# ---- main question: tests ----
def test_main_question():
    planets = [(0, 2), (10, 1), (3, 1)]
```

```
    # unsafe: [-2, 4] and [9, 11]
    assert safe_intervals(planets) == [(-INF, -2), (4, 9),
                                       (11, INF)]
    assert safe_intervals([]) == [(-INF, INF)]
    # touching blocks merge: [0, 2] and [2, 4]
```

```
    assert safe_intervals([(1, 1), (3, 1)]) == [
        (-INF, 0), (4, INF)]


# ============================================================
# FOLLOW-UP 1: Only the part between two given positions
# ============================================================

# ---- follow-up 1: function ----
# Answer: Intersect each safe interval with [A, B] and
# keep the non-empty ones. Or clip the planets first; the
# merge-and-gap logic is the same.
def safe_between(planets, a, b):
    # a route from a to b: clip every safe interval to [a,
    # b]
    out = []
    for lo, hi in safe_intervals(planets):
        lo, hi = max(lo, a), min(hi, b)
        if lo < hi:
            out.append((lo, hi))
    return out

# ---- follow-up 1: tests ----
def test_follow_up_1():
    planets = [(0, 2), (10, 1), (3, 1)]
    # only the part of the route between 5 and 20 matters
    route = safe_between(planets, 5, 20)
    assert route == [(5, 9), (11, 20)]


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Planets arrive over time.
# ============================================================

# ---- follow-up 3: helper functions ----
def safe_intervals_from(blocks):
    if not blocks:
        return [(-INF, INF)]
    out = [(-INF, blocks[0][0])]
    for prev, nxt in zip(blocks, blocks[1:]):
```

```
        out.append((prev[1], nxt[0]))
    out.append((blocks[-1][1], INF))
    return out

# ---- follow-up 3: function ----
# Answer: Keep the merged unsafe intervals in a sorted
# structure; an insert merges neighbours (like the stream-
# of-intervals problem) and the safe intervals are always
# the gaps.
class LiveRoute:
    """Planets arrive over time: keep blocks merged."""

    def __init__(self):
        self.blocks = []

    def add(self, c, r):
        a, b = c - r, c + r
        keep = []
        for s, e in self.blocks:
```

```
            if e < a or s > b:
                keep.append([s, e])
            else:
                # overlaps the new block: absorb it
                a, b = min(a, s), max(b, e)
        keep.append([a, b])
        self.blocks = sorted(keep)

    def safe(self):
        return safe_intervals_from(self.blocks)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    planets = [(0, 2), (10, 1), (3, 1)]
    live = LiveRoute()
    for c, r in planets:
        live.add(c, r)
    assert live.safe() == safe_intervals(planets)
    # a big planet at 6 swallows both blocks
    live.add(6, 3)
    assert live.safe() == [(-INF, -2), (11, INF)]


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(planets: list[list[int]]) -> list[list[int]]:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`[(None, None)]`
`[(3, 1), (10, 2)]`	`[(None, 2), (4, 8), (12, None)]`

STUDY PATH: question 136 of 288 | topic 16 of 43: Sorted intervals: merge while they touch, then read the gaps or search | 8 of 8

Template fit: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs | no date

HARD | DESIGN-STYLE (skippable) | Exponent | 136 of 288

Implement a function that processes a stream of interval data.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 190/288

ABOUT THIS QUESTION

Topic: Sorted intervals: merge while they touch, then read the gaps or search

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 352 Data Stream as Disjoint Intervals (likely) (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6245/implement-function-process-interval-data

QUESTION

Intervals arrive one at a time. add(start, end) receives a new interval, and intervals() returns the current list of disjoint intervals, sorted by start; intervals that overlap or touch are merged.

Function signature

class IntervalStream:
  def add(self, start: int, end: int) -> None
  def intervals(self) -> list

EXAMPLES

Example 1	

Input	`s = IntervalStream()`
	`s.add(1, 3)`
	`s.add(6, 9)`
	`s.intervals()`
	`s.add(2, 5)`
	`s.intervals()`
	`s.add(5, 6)`
	`s.intervals()`

Output	`s.intervals() -> [(1, 3), (6, 9)]`
	`s.intervals() -> [(1, 5), (6, 9)]`
	`s.intervals() -> [(1, 9)]`

Why	`(2, 5) overlaps (1, 3) and extends it to (1, 5); (5, 6) touches (1, 5) at 5 and (6, 9) at 6, so everything merges into (1, 9).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
intervals = []
def add(start, end):
    intervals.append((start, end))
def query():
    merged = []
    for s, e in sorted(intervals):
        if merged and s <= merged[-1][1]:
            last = merged[-1]
            last[1] = max(last[1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O(n log n) per query	O(n)

WHY IT IS SLOW

Every query re-sorts and re-merges the whole history, O(n log n) per query. Keeping the merged intervals sorted and only splicing the affected block makes an add cost O(log n) to find plus the shifted block.

OPTIMAL SOLUTION · Sorted disjoint intervals with binary search

TIME	SPACE
O(log n) to find the spot, O(n) worst-case list shift	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you keep adds fast for a very large number of intervals?

Use an ordered map (TreeMap, or a balanced tree / skip list in Python via the sortedcontainers package) keyed by start so a merge removes the swallowed intervals in amortized O(log n) instead of shifting an array. Each interval is inserted once and removed at most once, so n adds cost O(n log n) overall.

FOLLOW-UP 2 Should touching intervals such as [1, 3] and [4, 5] merge?

Agree on the semantics first. For integer points, [1, 3] and [4, 5] cover every point from 1 to 5, so they merge; use bisect_left(self.ends, start - 1). For real-valued closed intervals only overlap or a shared endpoint merges, which is what the code above does. For half-open intervals [a, b), a shared endpoint merges but nothing else does.

FOLLOW-UP 3 Answer 'is this point covered?' and 'how much is covered in total?'

The sorted disjoint intervals already support both: a point is covered if the interval with the largest start at or before it ends at or after it (one bisect), and the total length is the sum of the interval lengths (or keep it updated on each add).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a function that processes a
# stream of interval data.
#
# Intervals arrive one at a time. add(start, end) receives
# a new interval, and intervals() returns the current list
# of disjoint intervals, sorted by start; intervals that
# overlap or touch are merged.
#
# Signature:
#   class IntervalStream:
#       def add(self, start: int, end: int) -> None
#       def intervals(self) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you keep adds fast for a very large number of
# intervals?
# 2. Should touching intervals such as [1, 3] and [4, 5]
# merge?
# 3. Answer 'is this point covered?' and 'how much is
# covered in total?'
#
# INTERVIEW NOTES
# Keep the data in its simplest canonical form: merged
# intervals sorted by start with gaps between them. A new
# interval can only touch a contiguous block of existing
# ones, found with two binary searches, and that block
# collapses into one interval. State what 'touching' means
# before you code (shared endpoint, or only real overlap).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Sorted disjoint intervals with
# binary search
# Idea: Keep the merged intervals sorted by start. A new
# interval only interacts with the intervals found by two
# binary searches; replace that slice by the single merged
# interval. Touching closed intervals merge.
# FOLLOW-UP 3: Answer 'is this point covered?' and 'how
# much is covered in total?'
# Answer: The sorted disjoint intervals already support
# both: a point is covered if the interval with the
# largest start at or before it ends at or after it (one
# bisect), and the total length is the sum of the interval
# lengths (or keep it updated on each add).
class IntervalSet:
    """Keeps closed intervals merged and sorted.
    integer=True: [1, 3] and [4, 5] cover 1..5 and merge.
    integer=False: only overlap or a shared end merges."""

    def __init__(self, integer=True):
        self.starts, self.ends = [], []
        self.gap = 1 if integer else 0
        # length of [a, b] is b - a + extra
        self.extra = 1 if integer else 0
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        # first interval that touches the new one, and the
        # first one that is completely to its right
        i = bisect_left(self.ends, start - self.gap)
        j = bisect_right(self.starts, end + self.gap)
        for k in range(i, j):
            # the swallowed intervals leave the total ...
            self.covered -= self.length(self.starts[k],
                                        self.ends[k])
        if i < j:
            start = min(start, self.starts[i])
            end = max(end, self.ends[j - 1])
        # ... and the merged one joins it
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]
```

```
        self.covered += self.length(start, end)

    # FOLLOW-UP 3: is this point covered?
    def contains(self, point):
        # the interval with the largest start <= point
        i = bisect_right(self.starts, point) - 1
        return i >= 0 and self.ends[i] >= point

    # FOLLOW-UP 3: how much is covered in total?
    def total(self):
        return self.covered

    def intervals(self):
        return list(zip(self.starts, self.ends))


# FEED a stream and print each step
def process(stream, integer=True):
    # feed the stream; yield the merged list after each
    # add
    cover = IntervalSet(integer)
    for start, end in stream:
        cover.add(start, end)
        yield cover.intervals()

# ---- main question: tests ----
def test_main_question():
    feed = [(1, 3), (8, 10), (4, 5), (6, 7)]
    steps = list(process(feed))
    assert steps[0] == [(1, 3)]
    assert steps[1] == [(1, 3), (8, 10)]
```

```
    # (4, 5) touches (1, 3) for integers; (6, 7) then
    # touches both sides, so everything becomes one
    assert steps[2] == [(1, 5), (8, 10)]
    assert steps[3] == [(1, 10)]
    real = list(process([(1, 3), (4, 5), (3, 4)], False))
    assert real[1] == [(1, 3), (4, 5)]
    assert real[2] == [(1, 5)]
    box = IntervalSet()
    for pair in [(5, 8), (1, 2), (3, 4), (20, 21)]:
        box.add(*pair)
    assert box.intervals() == [(1, 8), (20, 21)]
    assert box.total() == 10
    assert box.contains(7) and box.contains(20)
    assert not box.contains(9) and not box.contains(0)
```

```
    for _ in range(200):
        box, points = IntervalSet(), set()
        for _ in range(random.randint(1, 8)):
            a = random.randint(0, 30)
            b = a + random.randint(0, 6)
            box.add(a, b)
            points |= set(range(a, b + 1))
            assert box.total() == len(points)
        for p in range(-1, 40):
            assert box.contains(p) == (p in points)


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Answer 'is this point covered?' and 'how
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.



# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 17 OF 43

Trie: nested dicts, one character per level

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Trie: nested dicts, one character per level | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

M	137. URL Request Handler Router	1Point3Acres	2026-10-16
M	138. Trie Prefix Search / Autocomplete	1Point3Acres	2026-07-02
M	139. Build Prefix Lookup with a Trie	PracHub	2026-07-03
M	140. Greedy Longest-Match Dictionary Tokenizer	1Point3Acres	2026-06-15
M	141. Check if all substrings are dictionary words	PracHub	2026-03-11
M	142. Return dictionary words matching a prefix	PracHub	2026-03-09
M	143. Implement Trie Insert and Exact-Word Search	PracHub	2026-02-20
H	144. Implement Longest-Match Text Replacement	PracHub	2026-01-20

CODE TEMPLATE | GROUP 17 OF 43 | 8 QUESTIONS, ONE PATTERN

Trie: nested dicts, one character per level

The one idea these questions share

Store words as nested dicts, one character per level, with an end marker that holds whatever you want to remember. Insert walks and creates; search walks and fails when a character is missing; prefix queries stop in the middle; longest-match keeps the last end marker you passed.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
END = "$"


def build_trie(words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node[END] = w                           # CHANGE 1: what you store at the end of a word
    return root


def walk(root, text):
    node = root
    for ch in text:
        node = node.get(ch)
        if node is None:
            return None
    return node
```

2. Helper functions that usually get written next to it

```
def words_under(node):
    out = []
    for key, child in node.items():
        if key == END:
            out.append(child)
        else:
            out.extend(words_under(child))
    return sorted(out)


def longest_match(root, text, start):
    node, best = root, None
    for j in range(start, len(text)):
        node = node.get(text[j])
        if node is None:
            break
        if END in node:
            best = node[END]
    return best
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
137	URL Request Handler Router	one trie level per URL segment; a node has literal children and one wildcard child; try the literal first
138	Trie Prefix Search / Autocomplete	walk to the prefix node; return the words under it (autocomplete)

#	Question	What you change in the template
139	Build Prefix Lookup with a Trie	build_trie, then walk to the prefix node and collect every word below it
140	Greedy Longest-Match Dictionary Tokenizer	the same longest-match walk, with a priority rule to choose between equal-length matches
141	Check if all substrings are dictionary words	for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check
142	Return dictionary words matching a prefix	no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)
143	Implement Trie Insert and Exact-Word Search	insert is build_trie; search is walk and then END in node
144	Implement Longest-Match Text Replacement	at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 137 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 1 of 8

Template fit: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first | asked 4 days ago | frequency Low

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen, onsite-coding | 137 of 288

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 282/288**

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

Design a request router. add(pattern) registers a URL pattern whose segments are either literal text or a placeholder written {name}; match(url) returns the registered pattern that handles the URL, or None. When a URL matches both a literal segment and a placeholder, the literal wins.

Function signature

class Router:
  def add(self, pattern: str) -> None
  def match(self, url: str) -> str | None

EXAMPLES

Example 1

Input	`r = Router()`
	`r.add('/users/{userId}/photo/{photoId}')`
	`r.add('/users/me/photo/{photoId}')`
	`r.match('/users/123/photo/456')`
	`r.match('/users/me/photo/9')`
	`r.match('/orders/1')`

Output	`r.match('/users/123/photo/456') -> '/users/{userId}/photo/{photoId}'`
	`r.match('/users/me/photo/9') -> '/users/me/photo/{photoId}'`
	`r.match('/orders/1') -> null`

Why	`The placeholder pattern handles any user id, but for /users/me/... the literal 'me' pattern wins; an unknown path matches nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

TIME	SPACE
O(H * L)	**O(1)**
per lookup	extra

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching)	**O(total pattern length)**
per lookup, O(total pattern length) to build	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: URL Request Handler Router
#
# Design a request router. add(pattern) registers a URL
# pattern whose segments are either literal text or a
# placeholder written {name}; match(url) returns the
# registered pattern that handles the URL, or None. When a
# URL matches both a literal segment and a placeholder,
# the literal wins.
#
# Signature:
#   class Router:
#       def add(self, pattern: str) -> None
#       def match(self, url: str) -> str | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Different HTTP methods per route.
# 2. A wildcard at the end that matches the rest of the
# path (/static/*).
# 3. Return the extracted parameters (id = 7).
#
# INTERVIEW NOTES
# Matching a path against many patterns is a prefix
# problem, so share the prefixes in a trie of segments.
# The catch is the wildcard: a literal branch can fail
# later, so you cannot pick it greedily. Try the literal
# child first and fall back to the wildcard child. That
# also encodes the precedence rule for free.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None

# ---- main question: main function ----
# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child.
# FOLLOW-UP 1: Different HTTP methods per route.
# Answer: Make the method the first segment of the path (a
# tiny trick) or keep a dict method to trie. Lookup uses
# the method as the first step.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)

# ---- main question: tests ----
def test_main_question():
    r = Router()
    r.add("/users/{userId}/photo/{photoId}", "photo")
    r.add("/users/me", "me")
    r.add("/static/*", "static")
```

```
    r.add("/orders/{id}", "create", method="POST")
    hit = r.match("/users/123/photo/456")
    ids = {"userId": "123", "photoId": "456"}
    assert hit == ("photo", ids)
```

```
    # a literal beats a placeholder
    assert r.match("/users/me")[0] == "me"
    assert r.match("/users/me/photo/9")[0] == "photo"
    # trailing wildcard takes the rest
    assert r.match("/static/css/app.css")[0] == "static"
    # different method, different route
    post = r.match("/orders/5", "POST")
    assert post == ("create", {"id": "5"})
    assert r.match("/orders/5")[0] is None
    words = ["a", "b", "{}", "*"]


# ========================================================
# FOLLOW-UP 1: Different HTTP methods per route.
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

STUDY PATH: question 138 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 2 of 8

Template fit: walk to the prefix node; return the words under it (autocomplete) | asked 110 days ago | frequency Medium

MEDIUM | IN APP LC 208 | 1Point3Acres | tech-screen, onsite-coding | 138 of 288

Trie Prefix Search / Autocomplete

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 124/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: your app's interview approach (LeetCode #208)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Frequency: Medium | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

QUESTION

Build a trie from a list of words. search_prefix(prefix) returns every word that starts with the prefix, sorted alphabetically (empty list when none).

Function signature

```
class Trie:
    def insert(self, word: str) -> None
    def search_prefix(self, prefix: str) -> list
```

EXAMPLES

Example 1

Input	`t = Trie(['car', 'card', 'care', 'cat', 'dog'])`
	`t.search_prefix('car')`
	`t.search_prefix('ca')`
	`t.search_prefix('x')`

Output	`t.search_prefix('car') -> ['car', 'card', 'care']`
	`t.search_prefix('ca') -> ['car', 'card', 'care', 'cat']`
	`t.search_prefix('x') -> []`

Why	`'car' is itself a word that starts with 'car'; the prefix 'x' matches nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def search_prefix(words, prefix):
    found = []
    for w in words:
        if w.startswith(prefix):
            found.append(w)
    return sorted(found)
```

BRUTE FORCE

TIME	SPACE
O(n * L + n log n) per query	**O(output)**

WHY IT IS SLOW

Every query reads the whole dictionary. A trie reaches the prefix in O(length of the prefix) and reads only the completions.

OPTIMAL SOLUTION · Trie with a DFS over sorted children

TIME	SPACE
O(L + output) per query after O(total characters) to build	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best K completions ranked by frequency.

Store at every node the K best (frequency, word) pairs and update them while inserting; a query then reads one node in O(L + K) and never walks the subtree. The price is more memory and slower inserts, which is the right trade when reads dominate (autocomplete).

FOLLOW-UP 2 Allow one wrong character (fuzzy match).

Walk the trie with a budget of one edit: at each level follow the matching child with the full budget and every other child with the budget reduced to 0. The work grows with the alphabet size times the prefix length, not with the dictionary size.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Trie Prefix Search / Autocomplete
#
# Build a trie from a list of words. search_prefix(prefix)
# returns every word that starts with the prefix, sorted
# alphabetically (empty list when none).
#
# Signature:
#   class Trie:
#       def insert(self, word: str) -> None
#       def search_prefix(self, prefix: str) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best K completions ranked by
# frequency.
# 2. Allow one wrong character (fuzzy match).
```

```
#
# INTERVIEW NOTES
# A trie shares prefixes, so walking down the query prefix
# costs O(length of the prefix) no matter how many words
# exist. Everything under that node is a completion. For
# autocomplete you either collect the words below the
# prefix node with a DFS (visiting children in sorted
# order gives sorted output) or store the best few words
# at every node while inserting.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}
        # how often the word ending here was inserted
        self.freq = 0
        # best (negative freq, word) pairs under this node
        self.top = []
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list.
class Trie:
    def __init__(self, k=3):
        self.root, self.k = Node(), k

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            node = node.children.setdefault(ch, Node())
            path.append(node)
        node.freq += freq
        # refresh the top-k list on every node of the path
        for n in path:
            n.top = [x for x in n.top if x[1] != word]
            n.top.append((-node.freq, word))
            n.top.sort()
            del n.top[self.k:]

    def find(self, prefix):
        # walk down the prefix; None if it is missing
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search_prefix(self, prefix):
        # every word with this prefix, in alphabetical
        # order
        node = self.find(prefix)
        out = []
```

```
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack:
            n, text = stack.pop()
            if n.freq:
```

```
                out.append(text)
            for ch in n.children:
                stack.append((n.children[ch], text + ch))
        return sorted(out)

    def suggest(self, prefix):
        # best k completions, read from one node: O(L + k)
        node = self.find(prefix)
        return [w for _, w in node.top] if node else []

    def fuzzy(self, prefix):
        # words whose first letters differ in at most one
        # place
        out, stack = [], [(self.root, "", 0)]
        while stack:
            n, text, wrong = stack.pop()
            if len(text) == len(prefix):
                out += self.search_below(n, text)
                continue
            for ch, child in n.children.items():
                miss = wrong + (ch != prefix[len(text)])
                if miss <= 1:
                    stack.append((child, text + ch, miss))
        return sorted(out)

    def search_below(self, node, text):
        out, stack = [], [(node, text)]
        while stack:
            n, t = stack.pop()
            if n.freq:
                out.append(t)
            for ch, child in n.children.items():
                stack.append((child, t + ch))
        return out

# ---- main question: tests ----
def test_main_question():
    t = Trie(k=2)
    for w in ["apple", "app", "apricot", "banana"]:
        t.insert(w)
    ap = t.search_prefix("ap")
    assert ap == ["app", "apple", "apricot"]
    assert t.search_prefix("x") == []
    assert t.search_prefix("") == [
        "app", "apple", "apricot", "banana"]
```

```
    # frequency ranking: app was typed three more times
    t.insert("app", 3)
    assert t.suggest("ap") == ["app", "apple"]
    assert t.suggest("zzz") == []
    # one wrong letter in the prefix is allowed
    assert "banana" in t.fuzzy("vana")
    assert t.fuzzy("zzzz") == []


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY PATH: question 139 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 3 of 8

Template fit: build_trie, then walk to the prefix node and collect every word below it | asked 109 days ago

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 139 of 288

Build Prefix Lookup with a Trie

POSTED 2026-07-03 | 109 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 121/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree) (inferred from overview)

Posted on PracHub: 2026-07-03 (updated 2026-07-04)

Original:
https://prachub.com/coding-questions/build-prefix-lookup-with-a-trie

QUESTION

Build a trie from a list of words. completions(prefix) returns every inserted word that starts with the prefix (including the prefix itself when it is a word), sorted alphabetically, or an empty list when none does.

Function signature

class PrefixTrie:
  def insert(self, word: str) -> None
  def completions(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = PrefixTrie(['apple', 'app', 'apply', 'apt', 'bat'])`
	`t.completions('app')`
	`t.completions('ap')`
	`t.completions('z')`

Output	`t.completions('app') -> ['app', 'apple', 'apply']`
	`t.completions('ap') -> ['app', 'apple', 'apply', 'apt']`
	`t.completions('z') -> []`

Why	`'app' is itself a word and a prefix of two more; the prefix 'z' matches nothing.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(words, prefix):
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found
```

BRUTE FORCE

TIME	SPACE
O(W * L) per query	O(1) extra

WHY IT IS SLOW

Every query touches every word, O(W * L). A trie walks only the prefix and then the subtree below it.

OPTIMAL SOLUTION · Trie with a DFS below the prefix node

TIME	SPACE
O(L) to walk the prefix + O(size of the answer)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

FOLLOW-UP 2 Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

FOLLOW-UP 3 Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. **Cost:** a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Build Prefix Lookup with a Trie
#
# Build a trie from a list of words. completions(prefix)
# returns every inserted word that starts with the prefix
# (including the prefix itself when it is a word), sorted
```

```
# alphabetically, or an empty list when none does.
#
# Signature:
#   class PrefixTrie:
#       def insert(self, word: str) -> None
#       def completions(self, prefix: str) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many words start with this prefix?
# 2. Return only the top 3 completions (by frequency or
# alphabetically).
# 3. Memory is large (many words with long common
# prefixes).
#
# INTERVIEW NOTES
# Prefix questions repeat work when each query scans the
# whole word list. A trie stores each prefix once: walk
# down by the query, then everything below that node is a
# completion. Keep counters or a word flag in the nodes to
# answer follow-ups such as 'how many words start with
# this' or 'top K completions' without another walk.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}
        # does a word end here?
        self.is_word = False
```

```
        # how many words pass through this node
        self.count = 0

# ---- main question: main function ----
# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
# FOLLOW-UP 1: How many words start with this prefix?
# Answer: Store a counter in each node that is incremented
# when a word passes through it; the answer is the counter
# at the end of the prefix walk, O(L) with no DFS.
class PrefixTrie:
    def __init__(self, words=()):
        self.root = Node()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.count += 1
        # a repeated word is only counted as one word
        if not node.is_word:
            node.is_word = True
        else:
```

```
            self.undo_count(word)

    def undo_count(self, word):
        node = self.root
        for ch in word:
            node = node.children[ch]
            node.count -= 1

    def walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node
```

```
    # FOLLOW-UP 1: how many words start with the prefix
    def count_prefix(self, prefix):
        # how many words start with the prefix: O(len)
        node = self.walk(prefix)
        if prefix == "":
            return self.root_count()
        return node.count if node else 0

    def root_count(self):
        kids = self.root.children.values()
        return sum(c.count for c in kids)

    # FOLLOW-UP 2: only the top k completions
    def suggest(self, prefix, k=3):
        # first k completions in alphabetical order; the
        # DFS
        # visits children in order and stops after k words
        node = self.walk(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack and len(out) < k:
            n, text = stack.pop()
            if n.is_word:
                out.append(text)
            # reversed so the smallest letter is popped
            # first
            for ch in sorted(n.children, reverse=True):
                stack.append((n.children[ch], text + ch))
        return out

    def completions(self, prefix):
        # every word with the prefix
        return self.suggest(prefix, k=10 ** 9)

# ---- main question: tests ----
def test_main_question():
    t = PrefixTrie(["apple", "app", "apricot", "banana",
                    "apple"])
    ap = t.completions("ap")
    assert ap == ["app", "apple", "apricot"]
    everything = t.completions("")
    assert everything[-1] == "banana"
    assert t.completions("x") == []
```

```
    assert t.count_prefix("ap") == 3
    assert t.count_prefix("b") == 1
    assert t.count_prefix("zz") == 0
    # the top 2 in alphabetical order
    assert t.suggest("ap", 2) == ["app", "apple"]
    assert t.count_prefix("") == 4


# ============================================================
# FOLLOW-UP 1: How many words start with this prefix?
# ============================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], queries: list[str]) -> list
[list[str]]:
SAMPLE INPUT
{"words":["car","cat","dog"],"queries":["ca","d"]}
```

Test cases

Input	Expected
`{"words":["car","cat","dog"],"queries":["ca","d"]}`	`[["car","cat"],["dog"]]`
`{"words":[],"queries":["a"]}`	`[[]]`

ALSO REPORTED · 1 more

Trie Prefix Search / Autocomplete | 1Point3Acres | tech-screen, onsite-coding | asked 2026-07-02 | Medium

https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

Preparation: Write the trie class from memory in under 12 min. Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY PATH: question 140 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 4 of 8

Template fit: the same longest-match walk, with a priority rule to choose between equal-length matches | asked 127 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen | 140 of 288

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 275/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.

Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.

Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

EXAMPLES

QUESTION

Example 1

Input	`t = Tokenizer({'ab': 1, 'abc': 2, 'c': 3})`
	`t.tokenize('abcab')`
	`t.tokenize('abx')`

Output	`t.tokenize('abcab') -> [2, 1]` `t.tokenize('abx') -> [1, 'x']`
Why	At each position the longest dictionary key wins: 'abc' (not 'ab' followed by 'c'), then 'ab'. A character that starts no key is emitted as it is.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L)	**O(dictionary)**
lookups, each O(L)	

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	**O(total key length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Greedy Longest-Match Dictionary Tokenizer
#
# Phone-screen coding round (R1). Tokenize a text against
# a dictionary of key:id entries using greedy longest-
# match; emit the matched id when a key matches, otherwise
# output the unmatched character as a literal. One follow-
# up on changing the tie-breaking / priority rule for
# which token wins. A second follow-up appears when the
# dictionary is very large: discuss how to keep the
# matcher memory- and lookup-efficient at scale. Input: a
# text string and a dictionary mapping key (string) -> id.
# Sweep the text left-to-right. At each position, find the
# longest dictionary key that matches a prefix of the
# remaining text; emit its id. If no key matches at the
# current position, emit the single character as a literal
# and advance by one. Output the concatenation (or list)
# of ids and literal characters in order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The dictionary is huge.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary key
# that is a prefix of the remaining text. Putting the keys
# in a trie lets you walk the text character by character
# from that position and remember the deepest end-of-key
# node you passed, instead of trying every length. If the
# tie-break rule changes (priority instead of length),
# store the priority at the terminal node and keep the
# best terminal seen.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Trie walk keeping the deepest
```

```
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
# FOLLOW-UP 1: The dictionary is huge.
# Answer: A flat map of every key wastes memory because
# keys share prefixes. A compressed (radix) trie stores
# each shared stretch once, or shard the keys by first
# characters across machines.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best
```

```
    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out


# SIMPLER VERSION: one dictionary lookup per length
def tokenize_map(text, table):
    # simpler version: a dictionary lookup per length
    out, i = [], 0
    longest = max(map(len, table), default=0)
    while i < len(text):
        top = min(longest, len(text) - i)
        for size in range(top, 0, -1):
            if text[i:i + size] in table:
                out.append(table[text[i:i + size]])
                i += size
                break
        else:
            out.append(text[i])
            i += 1
    return out

# ---- main question: tests ----
def test_main_question():
    table = {"ab": 1, "abc": 2, "c": 3}
    t = Tokenizer(table)
    # abc is longer than ab, so it wins at position 0
    assert t.tokenize("abcab") == [2, 1]
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
```

```
    assert tokenize_map("abcab", table) == [2, 1]
    # a single-letter key still applies the longest rule
    assert t.tokenize("cab") == [3, 1]
    # follow-up: priority by dictionary order instead
```

```
    order = list(table)
    pri = Tokenizer(table, priority=order.index)
    assert pri.tokenize("abc") == [1, 3]


# ============================================================
# FOLLOW-UP 1: The dictionary is huge.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

STUDY PATH: question 141 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 5 of 8

Template fit: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check | asked 223 days ago

MEDIUM | PracHub | Technical Screen | 141 of 288

Check if all substrings are dictionary words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 95/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: No direct match; related LC 139 Word Break

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-dictionary-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

EXAMPLES

Example 1	
Input	`'abcd', {'abc', 'abcd', 'bcd'}`
Output	`true`
Why	`Every substring of length 3 or more (abc, bcd, abcd) is in the dictionary.`

Example 2	
Input	`'abcd', {'abc', 'abcd'}`
Output	`false`
Why	`'bcd' is missing from the dictionary.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, dictionary):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            if s[i:j] not in dictionary:
                return False
    return True
```

TIME	SPACE
O(n^2)	O(1)
lookups, O(n^3) characters hashed	extra

WHY IT IS SLOW

About n^2 / 2 lookups and each hashes a whole substring, O(n^3) characters. Walking a trie from each start, one character at a time, is O(n^2) with no hashing of long strings.

OPTIMAL SOLUTION · Walk a trie from each start position

TIME	SPACE
O(n^2)	O(dictionary size)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

FOLLOW-UP 2 The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

FOLLOW-UP 3 The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. **Cost:** still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check if all substrings are dictionary
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** is a valid dictionary word (i.e., for all i, j
# with 0 <= i <= j < n and j - i + 1 >= 3, s[i..j] in
# dict). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - You may assume
# dictionary lookups are available (e.g.,
# dict.contains(word)), or you can preprocess the
# dictionary. Example: - If s = "cats", you must check
# "cat", "ats", and "cats" (and any other substrings of
# length >= 3).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first failing substring.
# 2. The string is very long.
# 3. The dictionary is huge and sharded.
#
# INTERVIEW NOTES
# Every substring is a separate requirement, so no clever
# skipping exists: you must test about n^2 / 2 substrings.
# What you can improve is the cost per test and the early
# exit. A trie lets you extend a start position one
# character at a time without rehashing the whole
# substring, and the first failure ends everything.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 1: Return the first failing substring.
# Answer: Same loops, but return the substring instead of
# False, which is what a user wants to see. Check the
```

```
# shorter ones first so the report is the smallest
# counterexample.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def first_failing(s, words, min_len=3):
    # the SHORTEST substring (length >= min_len) that is
    # not a
    # word, or None. Short lengths first gives the
    # smallest
    # counterexample, and cheap checks run before
    # expensive ones.
    words = set(words)
    longest = max(map(len, words), default=0)
    for length in range(min_len, len(s) + 1):
        # longer than every word: the first window already
        # fails
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along s[i:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end.
def all_substrings_words(s, words, min_len=3):
    return first_failing(s, words, min_len) is None


# SAME CHECK with a trie
class Trie:
    """The same check with a trie."""

    def __init__(self, words):
        self.root = {}
        for w in words:
            node = self.root
            for ch in w:
                node = node.setdefault(ch, {})
            node["$"] = True
```

```
    def all_substrings(self, s, min_len=3):
        for i in range(len(s)):
            node = self.root
            for depth, ch in enumerate(s[i:], 1):
                node = node.get(ch)
                if node is None:
                    # no word has this prefix: fails if
                    # the
                    # substring is long enough to count
                    return len(s) - i < min_len
                if depth >= min_len and "$" not in node:
                    return False
        return True

# ---- main question: tests ----
def test_main_question():
    words = {"aaa", "aaaa", "aaaaa"}
    assert all_substrings_words("aaaaa", words)
    assert Trie(words).all_substrings("aaaaa")
    # "aab" is a window that is not a word
    assert first_failing("aaab", words) == "aab"
    assert first_failing("aab", words) == "aab"
    assert not all_substrings_words("aab", words)
    assert not Trie(words).all_substrings("aab")
    # shorter than 3 letters: nothing to check
    assert all_substrings_words("ab", set())
    assert Trie([]).all_substrings("ab")
    # longer than every word: fails immediately
    assert first_failing("aaaaaa", words) == "aaaaaa"


# ============================================================
# FOLLOW-UP 1: Return the first failing substring.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, dictionary: list[str]) -> bool:
```

Sample call

```
solution('cat', ['cat'])
```

Test cases

Input	Expected
`('cat', ['cat'])`	`True`
`('cats', ['cat', 'cas', 'cts', 'ats', 'cats'])`	`True`

STUDY PATH: question 142 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 6 of 8

Template fit: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix) | asked 225 days ago

MEDIUM | IN APP LC 208 | PracHub | Technical Screen | 142 of 288

Return dictionary words matching a prefix

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 122/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/return-dictionary-words-matching-a-prefix

QUESTION

You are given a dictionary of words. Return all words that start with a given prefix. Matching is case-sensitive, duplicates in the dictionary are returned as often as they occur, and the result is an empty list when nothing matches. The index returns the matches in alphabetical order; limit (optional) stops after that many words.

Function signature

class PrefixIndex:
  def __init__(self, words: list)
  def matches(self, prefix: str, limit=None) -> list

EXAMPLES

Example 1

Input	`idx = PrefixIndex(['apple', 'app', 'Apple', 'apply', 'apple', 'bat'])`
	`idx.matches('app')`
	`idx.matches('app', limit=2)`
	`idx.matches('cat')`

Output	`idx.matches('app') -> ['app', 'apple', 'apple', 'apply']`
	`idx.matches('app', limit=2) -> ['app', 'apple']`
	`idx.matches('cat') -> []`

Why	`'Apple' starts with a capital letter so it does not match 'app'; 'apple' is listed twice because it is in the dictionary twice.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, prefix):
    found = [w for w in words
             if w.startswith(prefix)]
    return found
```

TIME	SPACE
O(W * L) per query	O(m)

WHY IT IS SLOW

Each query scans the whole dictionary, O(W * L). With the words sorted, binary search finds the block of matches directly.

OPTIMAL SOLUTION · Sort once, binary search the range

TIME	SPACE
O(log W + m) per query after O(W log W) sorting	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

FOLLOW-UP 2 Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

FOLLOW-UP 3 Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Return dictionary words matching a prefix
#
# You are given a dictionary of words. Return all words
# that start with a given prefix. Matching is case-
# sensitive, duplicates in the dictionary are returned as
# often as they occur, and the result is an empty list
```

```
# when nothing matches. The index returns the matches in
# alphabetical order; limit (optional) stops after that
# many words.
#
# Signature:
#   class PrefixIndex:
#       def __init__(self, words: list)
#       def matches(self, prefix: str, limit=None) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the first K matches (autocomplete).
# 2. Words are added while queries run.
# 3. Case-insensitive matching.
#
# INTERVIEW NOTES
# If the dictionary is sorted, all words with a prefix
# form one contiguous block, and its edges are found with
# binary search: the first word >= prefix, and the first
# word that is >= prefix with its last character
# incremented. For many queries on a static list this
# beats a trie in memory.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, bisect_right, insort


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort once, binary search the
# range
```

```
# Idea: With the words sorted, the matches are the words
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix.
# FOLLOW-UP 1: Only the first K matches (autocomplete).
# Answer: Stop the forward walk after K words, as the
# limit parameter does; with the sorted list the first K
# in alphabetical order are already the first K in the
# block.
# FOLLOW-UP 2: Words are added while queries run.
# Answer: Keep the list sorted with bisect.insort (O(n)
# per insert, fast in practice) or use a balanced tree; a
# trie handles inserts in O(L).
class PrefixIndex:
    def __init__(self, words):
        # a sorted copy: all words with a prefix sit
        # together
        self.words = sorted(words)

    # FOLLOW-UP 1: limit= returns only the first K
    def matches(self, prefix, limit=None):
        # first possible position, then walk while it
        # matches
        lo = bisect_left(self.words, prefix)
        # one past the last word that starts with the
        # prefix
        hi = bisect_right(self.words, prefix + "")
        found = self.words[lo:hi]
        return found if limit is None else found[:limit]

    # FOLLOW-UP 2: words added while queries run
    def add(self, word):
        # words can arrive while queries run: keep it
        # sorted
        insort(self.words, word)


# SIMPLE SCAN (say it first)
def matches_scan(words, prefix):
    # the simple O(n * L) answer, in the original order
    return [w for w in words if w.startswith(prefix)]

# ---- main question: tests ----
def test_main_question():
    words = ["apple", "app", "apricot", "Apple", "banana",
             "app"]
    idx = PrefixIndex(words)
    # duplicates are kept; matching is case sensitive
    assert idx.matches("ap") == ["app", "app", "apple",
                                 "apricot"]
    assert idx.matches("x") == []
    assert idx.matches("") == sorted(words)
    assert idx.matches("ap", limit=2) == ["app", "app"]
    scan = sorted(matches_scan(words, "ap"))
    assert scan == idx.matches("ap")
    idx.add("apex")
    assert "apex" in idx.matches("ap")


# ============================================================
# FOLLOW-UP 1: Only the first K matches (autocomplete).
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Words are added while queries run.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Case-insensitive matching.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Store a casefolded sort key next to each word (a
# tuple (key, word)) and search on the key; do not
# lowercase the displayed word.
class CaseInsensitiveIndex:
    """Ignore case, but show the original words."""

    def __init__(self, words):
        keyed = ((w.casefold(), w) for w in words)
        self.items = sorted(keyed)

    def matches(self, prefix):
        key = prefix.casefold()
        lo = bisect_left(self.items, (key,))
        out = []
        while lo < len(self.items) and \
                self.items[lo][0].startswith(key):
            out.append(self.items[lo][1])
            lo += 1
        return out

# ---- follow-up 3: tests ----
def test_follow_up_3():
    words = ["apple", "app", "apricot", "Apple", "banana",
             "app"]
```

```
    ci = CaseInsensitiveIndex(words)
    got = ci.matches("APP")
    assert got == ["app", "app", "Apple", "apple"]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], prefix: str) -> list[str]:
```

Sample call

```
solution(['apple', 'app', 'banana', 'apply'], 'app')
```

Test cases

Input	Expected
`(['apple', 'app', 'banana', 'apply'], 'app')`	`['apple', 'app', 'apply']`
`(['Cat', 'car', 'Cart', 'dog'], 'Ca')`	`['Cat', 'Cart']`

STUDY PATH: question 143 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 7 of 8

Template fit: insert is build_trie; search is walk and then END in node | asked 242 days ago

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 143 of 288

Implement Trie Insert and Exact-Word Search

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 123/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-02-20 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-trie-insert-and-exact-word-search

QUESTION

Implement a trie with insert(word), search(word) and startsWith(prefix). search is true only when the complete word was inserted earlier (a longer stored word that merely starts with it does not count); startsWith is true when some inserted word begins with the prefix.

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search(self, word: str) -> bool
  def startsWith(self, prefix: str) -> bool

EXAMPLES

Example 1

Input	`t = Trie()`
	`t.insert('apple')`
	`t.search('apple')`
	`t.search('app')`
	`t.startsWith('app')`
	`t.insert('app')`
	`t.search('app')`

Output	`t.search('apple') -> true`
	`t.search('app') -> false`
	`t.startsWith('app') -> true`
	`t.search('app') -> true`

Why	`'app' is only a prefix of 'apple' until it is inserted on its own.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
words = []
def insert(w):
    words.append(w)
def search(w):
    return w in words
def starts_with(p):
    for word in words:
        if word.startswith(p):
            return True
    return False
```

TIME	SPACE
O(W * L) per operation	O(total characters)

WHY IT IS SLOW

Each call scans every stored word, O(W * L). A trie with an end-of-word flag makes every call O(L).

OPTIMAL SOLUTION · Trie of nodes with an end-of-word flag

TIME	SPACE
O(L) per operation	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

FOLLOW-UP 2 Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. **Cost:** O(L) time and O(L) space for the path stack.

FOLLOW-UP 3 Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. **Cost:** O(L) per operation with dictionary children; memory O(total characters).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Trie Insert and Exact-Word
# Search
#
# Implement a trie with insert(word), search(word) and
# startsWith(prefix). search is true only when the
# complete word was inserted earlier (a longer stored word
# that merely starts with it does not count); startsWith
# is true when some inserted word begins with the prefix.
#
# Signature:
#   class Trie:
#       def insert(self, word: str) -> None
#       def search(self, word: str) -> bool
#       def startsWith(self, prefix: str) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support '.' as a wildcard in search (LeetCode 211).
# 2. Delete a word.
# 3. Unicode or large alphabets.
#
# INTERVIEW NOTES
# A trie needs a flag that marks 'a word ends here';
# without it search('app') would be true whenever 'apple'
# was inserted. Walking a prefix and checking the flag are
# the two operations: search checks the flag, startsWith
# does not.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}
        self.end = False
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
# FOLLOW-UP 1: Support '.' as a wildcard in search
# (LeetCode 211).
# Answer: On '.', try every child at that level and
# recurse; other characters follow the single matching
# child. Worst case O(26^L) per query on adversarial
# patterns, so mention the bound.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters; None if the path breaks
        node = self.root
        for ch in text:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
        node.end = True

    def search(self, word):
        # the whole word must have been inserted
        node = self.walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self.walk(prefix) is not None

    # FOLLOW-UP 1: '.' matches any one letter
    def search_wild(self, pattern):
        # '.' matches any one letter (LeetCode 211)
        def go(node, i):
            if i == len(pattern):
```

```
                return node.end
            ch = pattern[i]
            if ch == ".":
                # try every child at this level
                kids = node.children.values()
                return any(go(c, i + 1) for c in kids)
            child = node.children.get(ch)
            return child is not None and go(child, i + 1)

        return go(self.root, 0)

    # FOLLOW-UP 2: delete a word
    def delete(self, word):
        # unmark the word, then prune nodes nothing needs
        path = [self.root]
        for ch in word:
            nxt = path[-1].children.get(ch)
            if nxt is None:
                return False
            path.append(nxt)
        if not path[-1].end:
            return False
        path[-1].end = False
        # walk back up removing empty, non-word nodes
        for i in range(len(word), 0, -1):
            node = path[i]
            if node.end or node.children:
                break
            del path[i - 1].children[word[i - 1]]
        return True

# ---- main question: tests ----
def test_main_question():
    ops = [("insert", "apple"), ("search", "apple"),
           ("search", "app"), ("insert", "app"),
           ("search", "app")]
    assert run(ops) == [True, False, True]
    t = Trie()
    t.insert("apple")
    # a shared prefix is not a match for search
    assert not t.search("app") and t.starts_with("app")
    assert not t.starts_with("b")
```

```
    t.insert("bad")
    t.insert("dad")
    assert t.search_wild("b.d") and t.search_wild(".ad")
```

```
    assert not t.search_wild("b..d")
    assert t.search_wild("...")
    # delete keeps shorter words that share the path
    t.insert("app")
    assert t.delete("apple")
    assert t.search("app") and not t.search("apple")
    assert not t.delete("apple")
    assert t.delete("app") and not t.starts_with("a")


# ============================================================
# FOLLOW-UP 1: Support '.' as a wildcard in search
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def run(operations):
    # ["insert", w] or ["search", w] -> results of the
    # searches
    trie, out = Trie(), []
    for op, word in operations:
        if op == "insert":
            trie.insert(word)
        else:
            out.append(trie.search(word))
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def process_trie_operations(operations: list[list[str]]) -
> list[bool]:
```

Sample call

```
process_trie_operations([['insert', 'apple'], ['search', '
apple'], ['search', 'app'], ['insert', 'app'], ['search',
'app']])
```

Test cases

Input	Expected
`([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['search', 'app']],)`	`[True, False, True]`
`([],)`	`[]`

STUDY PATH: question 144 of 288 | topic 17 of 43: Trie: nested dicts, one character per level | 8 of 8

Template fit: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it | asked 273 days ago

HARD | IN APP LC 648 | PracHub | Technical Screen | 144 of 288

Implement Longest-Match Text Replacement

POSTED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TALK ALL TIME 31.9 30 DAYS 0 PRIORITY 140/288

ABOUT THIS QUESTION

Topic: Trie: nested dicts, one character per level

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #648 Replace Words

LeetCode match: LC 648 Replace Words / LC 833 Find And Replace in String

Posted on PracHub: 2026-01-20 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/implement-longest-match-text-replacement

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed.

Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2",
 "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

QUESTION

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy":
    "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

QUESTION

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints
0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def replace(text, entries):
    out, i = [], 0
    while i < len(text):
        best = ""
        for token in entries:
            if text.startswith(token, i):
                if len(token) > len(best):
                    best = token
        if best:
            out.append(entries[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out
```

TIME	SPACE
O(len(text) * tokens * token length)	O(1) extra

WHY IT IS SLOW

Each position tests every token. A trie of the tokens is walked from i, remembering the last token end, so a position costs at most the longest token.

OPTIMAL SOLUTION · Trie walk that remembers the last token end

TIME	SPACE
O(len(text) * longest token)	O(total token length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

FOLLOW-UP 2 The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

FOLLOW-UP 3 Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. **Cost:** still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Longest-Match Text Replacement
#
# You are given a text string and a dictionary of
# replacement entries. Each dictionary entry has the form
# token:id, where token is a string pattern and id is the
# value that should be emitted when that token is matched.
# Implement a function that scans the text from left to
# right and produces an output sequence using greedy
# longest-prefix matching: 1. At the current text
# position, find all dictionary tokens that match the
# substring starting at that position. 2. If at least one
# token matches, choose the longest matching token, append
# its corresponding id to the output, and advance by the
# length of that token. 3. If no token matches, append the
# original character at the current position to the output
# and advance by one character. 4. Continue until the
# entire text has been processed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority is not longest match: insertion order, a
# score, or a custom comparator.
# 2. The text is huge or the dictionary changes often.
# 3. Overlapping matches or word boundaries.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary token
# starting there. Checking every token costs the size of
# the dictionary per character, so share prefixes in a
# trie: walk the text from the current position down the
# trie, remember the last node that ends a token, and stop
# when the trie has no further edge. The remembered node
# is the longest match.
```

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length.
# FOLLOW-UP 1: The priority is not longest match:
# insertion order, a score, or a custom comparator.
# Answer: Collect every token that matches at the position
# (walk the trie and gather all token ends along the path,
# not just the last) and choose with the comparator. The
# trie gives the candidates; the policy is separate.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id. A trie shares the
        # prefixes.
        self.root = {}
        for token, ident in mapping.items():
            node = self.root
            for ch in token:
                node = node.setdefault(ch, {})
            node["$"] = (token, ident)

    def candidates(self, text, i):
        # every token that matches at position i, shortest
        # first (all ends along the trie path, not just
        # the last)
        out, node, j = [], self.root, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                out.append(node["$"])
        return out

    # FOLLOW-UP 1: choose= picks by priority, not length
    def replace(self, text, choose=None,
                word_boundary=False):
        # choose(candidates) picks the winner; the default
        # is the
        # longest token
        choose = choose or (lambda cs: cs[-1])
        out, i = [], 0
        while i < len(text):
```

```
            found = self.candidates(text, i)
            # FOLLOW-UP 3: only whole-word matches
            if word_boundary:
                found = [c for c in found if
                         self.edge(text, i, len(c[0]))]
            if found:
                token, ident = choose(found)
                out.append(ident)
                i += len(token)
            else:
                # nothing matches here: keep the character
                out.append(text[i])
                i += 1
        return out

    @staticmethod
    def edge(text, i, size):
        # letters must not touch the match on either side
        before = i == 0 or not text[i - 1].isalnum()
        end = i + size
        after = end == len(text)
        after = after or not text[end].isalnum()
        return before and after

# ---- main question: tests ----
def test_main_question():
    table = {"ab": 1, "abc": 2, "c": 3}
    r = Replacer(table)
    # abc beats ab at position 0
    assert r.replace("abcab") == [2, 1]
    assert r.replace("xyz") == ["x", "y", "z"]
    assert r.replace("") == []
    # priority by insertion order instead of length
    order = list(table)
    first = r.replace("abc", lambda cs: min(
        cs, key=lambda c: order.index(c[0])))
    assert first == [1, 3]
    # whole words only
    words = Replacer({"cat": "C"})
    got = words.replace("cat concat cat", None, True)
    assert got[0] == "C" and got[-1] == "C"
    assert "C" not in got[1:-1]


# ============================================================
```

```
# FOLLOW-UP 1: The priority is not longest match
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(text: str, dictionary: dict[str, str]) -> lis
t[str]:
```

Sample call

```
solution("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
```

Test cases

Input	Expected
("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})	["2", "d", "3", "z"]

Input	Expected
`("aaaaa", {"a": "X", "aa": "Y"})`	`["Y", "Y", "X"]`

ALSO REPORTED · 1 more

Greedy Longest-Match Dictionary Tokenizer | 1Point3Acres | phone-screen | asked 2026-06-15 | Low

https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.

Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.

Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).

If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

Preparation: Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 18 OF 43

A dict for lookup plus an ordered container for who is next (lazy deletion)

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	6	1
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

A dict for lookup plus an ordered container for who is next (lazy deletion) | 8 questions | tap a title to jump to it

READ FIRST: the code template for these 8 questions (tap here) >

E	145. Implement Batched Undo/Redo Layer	PracHub	2026-02-04
M	146. Design a restaurant waitlist system	PracHub	2026-03-01
M	147. Implement Python's itertools.tee	1Point3Acres	2026-08-20
M	148. Store With Cheapest-Item Removal	ColorStack	2025-09-29
M	149. Round-Robin Job Scheduler	ColorStack	2025-08-19
M	150. Aggregate FizzBuzz Offer Matches	ColorStack	2025-10-13
M	151. Restaurant Waitlist: join / delete / find_first_match	1Point3Acres	2026-02-01
H	152. Implement LRU Cache.	Exponent	no date

CODE TEMPLATE | GROUP 18 OF 43 | 8 QUESTIONS, ONE PATTERN

A dict for lookup plus an ordered container for who is next (lazy deletion)

The one idea these questions share

Keep a dict for O(1) lookup and a deque, list or heap for the order. When something is removed, delete it from the dict only and skip stale entries when you read from the order. LRU caches, waitlists, undo stacks and schedulers all fit.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


class OrderedStore:
    def __init__(self):
        self.order = deque()                  # CHANGE 1: the container that says who is next
        self.live = {}                        # key -> ticket, for O(1) lookup and lazy deletion
        self.ticket = 0

    def add(self, key):
        self.ticket += 1
        self.live[key] = self.ticket
        self.order.append((self.ticket, key))

    def remove(self, key):
        self.live.pop(key, None)              # lazy: the entry stays in the deque

    def next(self):
        while self.order and self.live.get(self.order[0][1]) != self.order[0][0]:
            self.order.popleft()              # CHANGE 2: skip entries that were removed
        return self.order[0][1] if self.order else None
```

2. Helper functions that usually get written next to it

```
from collections import OrderedDict


class LRU:                                    # the short version of the same idea
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()

    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)
        return self.d[k]

    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
145	Implement Batched Undo/Redo Layer	two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

#	Question	What you change in the template
146	Design a restaurant waitlist system	a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots
147	Implement Python's itertools.tee	one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue
148	Store With Cheapest-Item Removal	the order is a heap keyed by (price, timestamp); remove_cheapest pops it
149	Round-Robin Job Scheduler	a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time
150	Aggregate FizzBuzz Offer Matches	a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front
151	Restaurant Waitlist: join / delete / find_first_match	a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets
152	Implement LRU Cache.	dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 145 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 1 of 8

Template fit: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch | asked 258 days ago

EASY | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 145 of 288

Implement Batched Undo/Redo Layer

POSTED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 179/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: LC 1472 Design Browser History (related: undo/redo stacks)

Posted on PracHub: 2026-02-04 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/implement-batched-undo-redo-layer

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value afte
r the batch string old_value; // value before the batc
h bool has_old_value; // whether the key existed befor
e the batch }
```

QUESTION

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.
Constraints
0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'),
    ('end',), ('snapshot',), ('undo',),
    ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color':
    'red'}]
```

Explanation: A single batch is committed, undone, and redone.

```
Input: [('begin',), ('set', 'a', '1'), ('end',),
    ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',),
    ('undo',), ('snapshot',), ('redo',),
    ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'},
    {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
undo, redo = [], []
def commit_batch():
    undo.append(copy.deepcopy(props))
    redo.clear()
def do_undo():
    redo.append(copy.deepcopy(props))
    props.clear()
    props.update(undo.pop())
def do_redo():
    undo.append(copy.deepcopy(props))
    props.clear()
    props.update(redo.pop())
```

TIME	SPACE
O(size of document) per batch	**O(size * batches)**

WHY IT IS SLOW

A one-key change copies the whole document, O(size) per batch and O(size * batches) memory. Storing only the old and new value of each changed key makes each batch O(keys changed).

OPTIMAL SOLUTION · Store only the changed keys per batch

TIME	SPACE
O(keys changed in the batch) per operation	**O(total changed keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

FOLLOW-UP 2 Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

FOLLOW-UP 3 Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

FOLLOW-UP 4 Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Batched Undo/Redo Layer
#
# You are implementing a simplified document layer for a
# design tool. The layer stores properties as a
# map<string, string>. Implement a class that supports the
# following operations: - beginBatch(): start a batch of
# edits. - setProperty(key, value): set or overwrite a
# property inside the current batch. - endBatch(): commit
# the batch as one undoable action. - undo(): revert the
# most recently committed batch. - redo(): reapply the
# most recently undone batch. Requirements: 1. A batch may
# update the same key multiple times. 2. undo() must
# restore the exact document state from before that batch.
# 3. redo() must restore the exact document state after
# that batch. 4. If a property did not exist before the
# batch, then undo() should remove that property. 5. If
# undo() has been called and then a new batch is
# committed, the redo history must be cleared. 6. Follow-
# up: make the history memory-efficient.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Limit the history to the last N batches.
# 2. Nested batches.
# 3. Undo of individual setProperty calls, not whole
# batches.
# 4. Limit the history to the last N batches.
#
# INTERVIEW NOTES
# Undo and redo are two stacks of actions. The question is
# what an action is: a batch is one action, so record per
# batch only what is needed to reverse it (the old value
# of each key touched, and the new one), not a whole copy
# of the document. A new edit after an undo kills the redo
# history.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class Change:
    """One key in one batch: value before and after."""
```

```
    def __init__(self, key, old, has_old, new):
        self.key, self.old, self.has_old, self.new = (
            key, old, has_old, new)

# ---- main question: main function ----
# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack.
# FOLLOW-UP 1: Limit the history to the last N batches.
# Answer: Keep the undo stack in a deque with maxlen N so
# the oldest batch is dropped automatically. Redo is
# unaffected.
# FOLLOW-UP 2: Nested batches.
# Answer: Keep a stack of open batches and merge a child
# into its parent on endBatch (keep the parent's old value
# if the key was already touched). Only the outermost
# endBatch commits an undoable action.
class Layer:
    def __init__(self, history=None):
        self.props = {}
        # FOLLOW-UP 1, 4: history=N keeps the last N
        # batches
        # committed batches; a deque with a maximum length
        # forgets the oldest ones when the history is
        # capped
        self.undo_stack = deque(maxlen=history)
        self.redo_stack = []
        # open batches (a stack, so batches can nest):
        # each is
        # a dict key -> Change, so a key appears only once
        self.open = []

    def begin_batch(self):
        self.open.append({})
```

```
    def set_property(self, key, value):
        if not self.open:
            # outside a batch: a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open[-1]
        if key in batch:
            # touched again: keep the ORIGINAL old value
            # and
            # only update the final value
            batch[key].new = value
        else:
            batch[key] = Change(key, self.props.get(key),
                                key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open.pop()
        if self.open:
            # FOLLOW-UP 2: nested batches fold into the
            # parent
            # a nested batch folds into its parent
            self.merge(self.open[-1], batch)
        elif batch:
            # only the outermost batch is one undoable
            # action;
            # a new action clears the redo history
            self.undo_stack.append(list(batch.values()))
            self.redo_stack.clear()

    @staticmethod
```

```
    def merge(parent, child):
        for key, change in child.items():
            if key in parent:
                parent[key].new = change.new
            else:
                parent[key] = change

    def undo(self):
        if not self.undo_stack:
            return False
        changes = self.undo_stack.pop()
```

```
        for c in changes:
            if c.has_old:
                self.props[c.key] = c.old
            else:
                # the key did not exist before: remove it
                self.props.pop(c.key, None)
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if not self.redo_stack:
            return False
        changes = self.redo_stack.pop()
        for c in changes:
            self.props[c.key] = c.new
        self.undo_stack.append(changes)
        return True

# ---- main question: tests ----
def test_main_question():
    layer = Layer()
    layer.begin_batch()
    layer.set_property("color", "red")
    layer.set_property("color", "blue")
    layer.set_property("size", "4")
    layer.end_batch()
    assert layer.props == {"color": "blue", "size": "4"}
    # each key is stored once, whatever the number of
    # edits
    assert len(layer.undo_stack[0]) == 2
    layer.undo()
    assert layer.props == {}
    layer.redo()
    assert layer.props == {"color": "blue", "size": "4"}
    layer.begin_batch()
    layer.set_property("color", "green")
    layer.end_batch()
    layer.undo()
    # the old value comes back, not the first value of the
    # batch
    assert layer.props["color"] == "blue"
    # a new batch clears the redo history
    layer.set_property("x", "1")
    assert not layer.redo()
    # nested batches commit as one action
```

```
    layer.begin_batch()
    layer.begin_batch()
    layer.set_property("y", "2")
    layer.end_batch()
    layer.set_property("z", "3")
    layer.end_batch()
    layer.undo()
    assert "y" not in layer.props
    assert "z" not in layer.props
    # history limited to the last 2 batches
    small = Layer(history=2)
    for i in range(4):
        small.set_property("k", str(i))
    assert len(small.undo_stack) == 2
    small.undo()
```

```
    small.undo()
    assert not small.undo() and small.props["k"] == "1"


# ============================================================
# FOLLOW-UP 1: Limit the history to the last N batches.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Nested batches.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[list[str]]) -> list[dict[str
, str]]:
SAMPLE INPUT
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot
',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
```

Test cases

Input	Expected
`[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'color': 'red'}, {}, {'color': 'red'}]`
`[('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]`

STUDY PATH: question 146 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 2 of 8

Template fit: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots | asked 233 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 146 of 288

★ TOP 75: rank 73 in the Medium and Hard priority list

★ Design a restaurant waitlist system

POSTED 2026-03-01 | 233 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.3 | 30 DAYS 0 | PRIORITY 56/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #146 LRU Cache

LeetCode match: No direct match; related LC 146 LRU Cache (ordered map)

Posted on PracHub: 2026-03-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/design-a-restaurant-waitlist-system

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **c** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
- Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
- Returns how many parties are ahead of this party in the current waitlist order (0-based).

QUESTION

- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.add(1, 'ann', 4)`
	`w.add(2, 'bob', 2)`
	`w.add(3, 'cy', 3)`
	`w.seat(3)`
	`w.cancel(3)`
	`w.seat(3)`
	`w.seat(4)`

Output	`w.seat(3) -> 2`
	`w.cancel(3) -> true`
	`w.seat(3) -> null`
	`w.seat(4) -> 1`

Why	`A table for 3 takes the earliest party that fits: bob (2), not ann (4). After cy cancels, nobody left fits a table for 3; a table for 4 seats ann.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parties = []  # in arrival order
def join(party):
    parties.append(party)
def seat(capacity):
    for party in parties:
        if party.size <= capacity:
            parties.remove(party)
            return party
    return None
def cancel(party_id):
    for party in parties:
        if party.id == party_id:
            parties.remove(party)
            return
```

TIME	SPACE
O(n)	**O(n)**
per seat and per cancel	

WHY IT IS SLOW

Both operations walk the list, O(n). A segment tree of the smallest size per range finds the leftmost party that fits in O(log n).

OPTIMAL SOLUTION · Segment tree of minimum size over arrival order

TIME	SPACE
O(log n) per operation	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

FOLLOW-UP 2 Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

FOLLOW-UP 3 Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

FOLLOW-UP 4 Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design a restaurant waitlist system
#
# You are implementing the waitlist system for a
# restaurant. Parties arrive over time, can cancel, and
# get seated when a table becomes available. ## Rules -
# Each party has a unique partyId, a name, and a size
# (number of people). - Parties are seated in **arrival
# order**, but only if they **fit** the available table. -
# When a table of capacity c becomes available, you must
# seat the **earliest-arrived** party whose size <= c. -
# If no party fits, the table remains unused for that
# event. ## Operations to support Design a data structure
# / class that supports the following operations
# efficiently: 1. addParty(name, size) -> partyId - Adds a
# new party to the end of the waitlist and returns its id.
# 2. cancelParty(partyId) -> bool - Removes the party from
# the waitlist if present. - Returns true if removed,
# otherwise false. 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Parties that wait too long should be seated first
# (starvation).
# 2. Seat the best-fitting party, not the earliest.
# 3. Estimate the waiting time of a party.
# 4. Seat the best-fitting party (the largest that fits),
# not the earliest.
#
# INTERVIEW NOTES
# 'Earliest arrival whose size fits' is a query on an
# ordered sequence: find the leftmost position whose value
# is at most c. A scan does it in O(n); a segment tree
# that stores the minimum size over each range answers it
# in O(log n) by walking down to the leftmost child whose
# range minimum fits. Cancelling or seating simply sets
# that position to infinity.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_right, insort
from collections import defaultdict, deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots).
class Waitlist:
    """Earliest party that fits: a segment tree of the
    smallest size over arrival slots."""

    def __init__(self, capacity=1024):
        self.cap = capacity
        self.tree = [INF] * (2 * capacity)
        self.count = 0
        self.slot, self.who = {}, {}

    def update(self, i, value):
        i += self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = min(self.tree[2 * i],
                               self.tree[2 * i + 1])

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
```

```
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        i = self.slot.get(party_id)
        if i is None or self.tree[i + self.cap] == INF:
            return False
        self.update(i, INF)
        return True
```

```
    def seat(self, capacity):
        # no party small enough: nobody is seated
        if self.tree[1] > capacity:
            return None
        i = 1
        # go left whenever the left half holds a party
        # that fits
        while i < self.cap:
            left = self.tree[2 * i] <= capacity
            i = 2 * i if left else 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]

# ---- main question: tests ----
def test_main_question():
    w = Waitlist(8)
    w.add("p1", "Ann", 4)
    w.add("p2", "Bob", 2)
    w.add("p3", "Cy", 6)
    # a table for 3: the earliest party that fits is Bob
    assert w.seat(3) == "p2"
    # cancel Ann; a table for 6 now seats Cy
    assert w.cancel("p1") and not w.cancel("p1")
    assert w.seat(6) == "p3"
    assert w.seat(10) is None


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Seat the best-fitting party, not the
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is a different policy: keep sizes in a
# sorted structure of (size, arrival) and take the largest
# size at most the capacity (ties by arrival). A sorted
# container with bisect gives O(log n) lookups.
# FOLLOW-UP 4: Seat the best-fitting party (the largest
# that fits), not the earliest.
# Answer: Group waiting parties by size in queues (arrival
```

```
# order inside a size) and keep the sizes in a sorted
# list: the best fit for a table of capacity c is the
# largest size at most c, found with bisect, then the
# oldest party of that size.
class BestFit:
    """Policy B: the largest party that fits the table."""

    def __init__(self):
        # size -> parties of that size, oldest first
        self.queues = defaultdict(deque)
        self.sizes = []

    def add(self, party_id, size):
        if not self.queues[size]:
            insort(self.sizes, size)
        self.queues[size].append(party_id)

    def seat(self, capacity):
        k = bisect_right(self.sizes, capacity)
        if k == 0:
            return None
        size = self.sizes[k - 1]
        party = self.queues[size].popleft()
        if not self.queues[size]:
            self.sizes.pop(k - 1)
        return party

# ---- follow-up 2: tests ----
def test_follow_up_2():
    b = BestFit()
    parties = (("a", 2), ("b", 4), ("c", 4), ("d", 6))
    for pid, size in parties:
        b.add(pid, size)
    # a table for 5: the biggest fit is size 4, oldest
    # first
    assert b.seat(5) == "b"
    assert b.seat(5) == "c"
    assert b.seat(5) == "a"
    assert b.seat(1) is None


# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.
```

```
# ============================================================
# FOLLOW-UP 4: Seat the best-fitting party (the largest
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Test cases

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('seatTable', 2), ('seatTable', 4)]`	`[1, 2, 2, 1]`
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('addParty', 'C', 3), ('getPosition', 3), ('seatTable', 2), ('getPosition', 3), ('cancelParty', 1), ('getPosition', 3), ('cancelParty', 1), ('seatTable', 2), ('seatTable', 3), ('getPosition', 3)]`	`[1, 2, 3, 2, 2, 1, True, 0, False, None, 3, -1]`

ALSO REPORTED · 1 more

Restaurant Waitlist: join / delete / find_first_match | 1Point3Acres | onsite-coding | asked 2026-02-01 | Single report

https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

Preparation: Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY PATH: question 147 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 3 of 8

Template fit: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue | asked 61 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 147 of 288

Implement Python's itertools.tee

ASKED 2026-08-20 | 61 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 285/288**

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement itertools.tee: given an iterable and n, return n independent iterators that each produce the same values from the one source. The source must be read only once, and a value must be kept only until every iterator has taken it.

Function signature

def tee_queues(iterable, n=2) -> tuple

EXAMPLES

Example 1	

Input	`a, b = tee_queues([1, 2, 3])`
	`next(a)`
	`next(a)`
	`next(b)`
	`list(b)`
	`list(a)`

Output	`next(a) -> 1`
	`next(a) -> 2`
	`next(b) -> 1`
	`list(b) -> [2, 3]`
	`list(a) -> [3]`

Why	`Iterator a runs ahead by two values; b still sees every value from the start, and neither disturbs the other.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

BRUTE FORCE

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement Python's itertools.tee
#
# Implement itertools.tee: given an iterable and n, return
# n independent iterators that each produce the same
```

```
# values from the one source. The source must be read only
# once, and a value must be kept only until every iterator
# has taken it.
#
# Signature:
#   def tee_queues(iterable, n=2) -> tuple
#
# FOLLOW-UPS (answered below, in this order):
# 1. Memory grows when one consumer stalls. Is there a way
# to free items automatically?
# 2. Make it safe for several threads.
# 3. Why not just copy the iterator?
#
# INTERVIEW NOTES
# The source can only be consumed once, so someone must
# remember items until every consumer has read them. The
# state to keep is exactly 'items the slowest consumer has
# not read yet', so either give each consumer a queue that
# the leader fills, or keep one shared linked list with a
# pointer per consumer so memory is freed automatically.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import threading
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
```

```
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)

# ---- main question: tests ----
def test_main_question():
    a, b = tee_queues([1, 2, 3])
    assert next(a) == 1
    assert next(a) == 2
    assert next(b) == 1
    assert list(b) == [2, 3]
    assert list(a) == [3]
```

```
# ============================================================
# FOLLOW-UP 1: Memory grows when one consumer stalls. Is
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use one shared singly linked list with a pointer
# per consumer. The only references to old nodes are the
# consumers' pointers, so Python's garbage collector frees
# nodes behind the slowest consumer without any
# bookkeeping.
class Cell:
    # one link of the shared list
    def __init__(self):
        self.filled = False
        self.value = None
```

```
        self.next = None

# ============================================================
# FOLLOW-UP 2: Make it safe for several threads.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Pull from the source under one lock and append
# to the queues inside it. Without the lock two consumers
# can pull two items and append them in different orders
# to different queues.
def tee(iterable, n=2):
    source = iter(iterable)
    # one lock keeps the pulls in a single order
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    # the first reader pulls the value
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value, cell = cell.value, cell.next
            yield value

    # every reader starts at the same first cell
    head = Cell()
    return tuple(consumer(head) for _ in range(n))

# ---- follow-up 2: tests ----
def test_follow_up_2():
    a, b = tee(range(5))
    # readers are independent, at any speed
    assert next(a) == 0 and next(a) == 1
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")
    # works on an endless source, which tee_naive cannot
    def count():
```

```
        i = 0
        while True:
            yield i
            i += 1
    e1, e2 = tee(count())
    assert [next(e1) for _ in range(3)] == [0, 1, 2]
    assert next(e2) == 0
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

STUDY PATH: question 148 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 4 of 8

Template fit: the order is a heap keyed by (price, timestamp); remove_cheapest pops it | asked 386 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 148 of 288

Store With Cheapest-Item Removal

ASKED 2025-09-29 | 386 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 223/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1048 Longest String Chain

LeetCode match: LC 1048 / design: priority store (related: heap with ties)

Frequency: Single report | Last asked: 2025-09-29 | Roles: SWE Intern

QUESTION

Design a Store that holds items. Each item has a string value , an integer price in cents, and an integer timestamp . Support: add(value, price, timestamp) : insert an item. remove_cheapest() : remove and return the value of the item with the smallest price. If several items share the smallest price, remove the one with the smallest timestamp . Return None if the store is empty. A heap keyed by (price, timestamp) gives O(log n) per operation.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...
    def add(self, value: str, price: int, timestamp: int) -> None:
        ...
    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item
(ties broken by
earliest timestamp)."""
        ...
```

Follow-up (as reported): Improve remove_cheapest from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

EXAMPLES

Example 1

Input	`add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()`
Output	`"b"`
Why	`"b" has the smallest price (2 cents).`

QUESTION

Example 2

Input	`add("x",7,10); add("y",7,4); remove_cheapest()`
Output	`"y"`
Why	`Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).`

Example 3

Input	`remove_cheapest() # on an empty store`
Output	`None`

CONSTRAINTS

- Prices are integers with 1 <= price <= 10000 (i.e. 1 cent up to $100.00).
- Up to 10^6 operations.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class BruteStore:
    def __init__(self):
        self.items = []
    def add(self, value, price, timestamp):
        self.items.append(
            (price, timestamp, value)
        )
    def remove_cheapest(self):
        if not self.items:
            return None
        item = min(
            self.items,
            key=lambda x: (x[0], x[1]),
        )
        self.items.remove(item)
        return item[2]
```

TIME	SPACE
O(n) per removal	O(n)

WHY IT IS SLOW

remove_cheapest looks at every item.

OPTIMAL SOLUTION · A heap keyed by price and timestamp

TIME	SPACE
O(log n) per operation	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Prices are integers from 0 to 10000. Can removal be O(1) amortized?

Use an array of 10001 buckets, one per price, each a small heap by timestamp, and a pointer to the lowest price that might be non-empty. add lowers the pointer if needed; remove_cheapest moves it up past empty buckets, so the pointer only moves up between adds.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Store With Cheapest-Item Removal
#
# Design a Store that holds items. Each item has a string
# value , an integer price in cents, and an integer
# timestamp . Support: add(value, price, timestamp) :
# insert an item. remove_cheapest() : remove and return
# the value of the item with the smallest price. If
# several items share the smallest price, remove the one
# with the smallest timestamp . Return None if the store
# is empty. A heap keyed by (price, timestamp) gives O(log
# n) per operation. Function signature: class Store: def
# __init__(self) -> None: ... def add(self, value: str,
# price: int, timestamp: int) -> None: ... def
# remove_cheapest(self) -> str | None: """Remove and
# return the value of the lowest-priced item (ties broken
# by earliest timestamp)""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Prices are integers from 0 to 10000. Can removal be
# O(1) amortized?
#
# INTERVIEW NOTES
# Removing the cheapest item with ties broken by the
# earliest timestamp is a priority queue keyed by (price,
# timestamp). A heap gives O(log n) per operation. When
# prices are bounded, an array of buckets indexed by price
# plus a pointer to the lowest non-empty price gives O(1)
# amortized removal.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): A heap keyed by price and
# timestamp
```

```
# Idea: The smallest (price, timestamp) is always at the
# top of the heap.
class Store:
    def __init__(self):
        self.heap = []

    def add(self, value, price, timestamp):
        heapq.heappush(
            self.heap, (price, timestamp, value)
        )

    def remove_cheapest(self):
        if not self.heap:
            return None
        return heapq.heappop(self.heap)[2]

# ---- main question: tests ----
def test_main_question():
    random.seed(34)
    s = Store()
    s.add("a", 5, 1)
    s.add("b", 2, 2)
    s.add("c", 8, 3)
    assert s.remove_cheapest() == "b"
    s = Store()
    s.add("x", 7, 10)
    s.add("y", 7, 4)
    assert s.remove_cheapest() == "y"
    assert Store().remove_cheapest() is None


# ============================================================
# FOLLOW-UP 1: Prices are integers from 0 to 10000. Can
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use an array of 10001 buckets, one per price,
# each a small heap by timestamp, and a pointer to the
# lowest price that might be non-empty.
class BucketStore:
    MAX_PRICE = 10000

    def __init__(self):
```

```
        self.buckets = [
            [] for _ in range(self.MAX_PRICE + 1)
        ]
        self.low = self.MAX_PRICE + 1

    def add(self, value, price, timestamp):
        heapq.heappush(
            self.buckets[price], (timestamp, value)
        )
        self.low = min(self.low, price)

    def remove_cheapest(self):
        while (
            self.low <= self.MAX_PRICE
            and not self.buckets[self.low]
        ):
            self.low += 1
        if self.low > self.MAX_PRICE:
            return None
        return heapq.heappop(self.buckets[self.low])[1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 149 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 5 of 8

Template fit: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time | asked 427 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 149 of 288

Round-Robin Job Scheduler

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 214/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1700 Number of Students Unable to Eat Lunch (related: queue simulation)

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round- robin approach. Reconstruct it as follows. You are given a list of jobs, each with a unique string id and a positive integer burst (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time quantum : it takes the job at the front of the queue, runs it for min(quantum, remaining) time units, and then — if the job still has work left — puts it at the back of the queue; otherwise the job completes at that moment. Return the list of (id, completion_time) pairs in the order the jobs complete.

Function signature

```
def round_robin(jobs: list[tuple[str, int]], quantum: int) ->
list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id,
completion_time) pairs in completion
order."""
    ...
```

Follow-up (as reported): Report note ("a constraint was changed"): add per-job arrival times so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

EXAMPLES

Example 1

Input	`jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2`
Output	`[("C", 6), ("A", 8), ("B", 9)]`
Why	`t=0-2 A (rem 2), t=2-4 B (rem 1), t=4-6 C completes at 6, t=6-8 A completes at 8, t=8-9 B completes at 9.`

QUESTION

Example 2

Input	`jobs = [("X", 5)], quantum = 2`
Output	`[("X", 5)]`
Why	`One job, run in slices 2+2+1, finishing at time 5.`

Example 3

Input	`jobs = [("P", 1), ("Q", 1)], quantum = 3`
Output	`[("P", 1), ("Q", 2)]`
Why	`Each finishes within a single quantum: P at 1, Q at 2.`

CONSTRAINTS

- 1 <= len(jobs) <= 10^4
- 1 <= burst <= 10^6 , 1 <= quantum <= 10^6
- Job ids are unique, non-empty strings.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_round_robin(jobs, quantum):
    remaining = [b for _, b in jobs]
    clock, done = 0, []
    while len(done) < len(jobs):
        for i, (jid, _) in enumerate(jobs):
            if remaining[i] > 0:
                run = min(
                    quantum, remaining[i]
                )
                clock += run
                remaining[i] -= run
                if remaining[i] == 0:
                    done.append((jid, clock))
    return done
```

TIME	SPACE
O(n * rounds)	O(n)

WHY IT IS SLOW

It rescans the whole list every round, including jobs that already finished.

OPTIMAL SOLUTION · A queue of jobs

TIME	SPACE
O(total burst / quantum + n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each job has an arrival time. How do you change it?

Sort the jobs by arrival. Before every pick admit all jobs whose arrival is at most the clock; a job preempted after its slice goes to the back after the jobs that arrived during the slice. If the queue is empty, jump the clock to the next arrival.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Round-Robin Job Scheduler
#
# The third question was "not really leetcode style" but
# was a job-scheduling problem solved with a round- robin
# approach. Reconstruct it as follows. You are given a
# list of jobs, each with a unique string id and a
# positive integer burst (the number of time units of CPU
# it needs). All jobs are available at time 0 and are
# placed in a ready queue in the given order. The CPU runs
# jobs round-robin with a fixed time quantum : it takes
# the job at the front of the queue, runs it for
# min(quantum, remaining) time units, and then — if the
# job still has work left — puts it at the back of the
# queue; otherwise the job completes at that moment.
# Return the list of (id, completion_time) pairs in the
# order the jobs complete.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each job has an arrival time. How do you change it?
#
# INTERVIEW NOTES
# Round-robin is a queue simulation: take the front job,
# run it for min(quantum, remaining), and push it to the
# back if it still has work. The clock only moves forward
# by the time actually run, so completion times fall out
# directly. An arrival time just adds a second queue of
# jobs not yet released.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A queue of jobs

# Idea: Run the front job for one slice and send it to the
# back if it is not finished.
def round_robin(jobs, quantum):
    queue = deque([jid, burst] for jid, burst in jobs)
    clock, done = 0, []
    while queue:
        job = queue.popleft()
        run = min(quantum, job[1])
        clock += run
        job[1] -= run
        if job[1] > 0:
            # not finished: back of the line
            queue.append(job)
        else:
            done.append((job[0], clock))
    return done

# ---- main question: tests ----
def test_main_question():
    random.seed(3)
    assert round_robin(
        [("A", 4), ("B", 3), ("C", 2)], 2
    ) == [("C", 6), ("A", 8), ("B", 9)]
    assert round_robin([("X", 5)], 2) == [("X", 5)]
    assert round_robin([("P", 1), ("Q", 1)], 3) == [
        ("P", 1),
        ("Q", 2),
    ]


# ============================================================
# FOLLOW-UP 1: Each job has an arrival time. How do you
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort the jobs by arrival. Before every pick
# admit all jobs whose arrival is at most the clock; a job
# preempted after its slice goes to the back after the
# jobs that arrived during the slice.
def round_robin_arrivals(jobs, quantum):
    # jobs are (id, burst, arrival)
    pending = deque(sorted(jobs, key=lambda j: j[2]))
    queue, clock, done = deque(), 0, []
    while pending or queue:
        if not queue:
            clock = max(clock, pending[0][2])
        while pending and pending[0][2] <= clock:
            jid, burst, _ = pending.popleft()
            queue.append([jid, burst])
        job = queue.popleft()
        run = min(quantum, job[1])
        clock += run
        job[1] -= run
        # jobs that arrived while this one ran line up
        # before it
        while pending and pending[0][2] <= clock:
            jid, burst, _ = pending.popleft()
            queue.append([jid, burst])
        if job[1] > 0:
            queue.append(job)
        else:
            done.append((job[0], clock))
    return done

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        jobs = [
            (str(i), random.randint(1, 7))
            for i in range(random.randint(1, 6))
        ]
        q = random.randint(1, 4)
        # with every arrival at time 0 the arrival version
        # is the same
        assert round_robin_arrivals(
            [(a, b, 0) for a, b in jobs], q
```

```
        ) == round_robin(jobs, q)
    assert round_robin_arrivals(
        [("A", 3, 0), ("B", 1, 5)], 2
    ) == [("A", 3), ("B", 6)]


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 150 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 6 of 8

Template fit: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front | asked 372 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 150 of 288

Aggregate FizzBuzz Offer Matches

ASKED 2025-10-13 | 372 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 226/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

LeetCode match: Order-book matching with queues (related: LC 1801 Number of Orders in the Backlog)

Frequency: Single report | Last asked: 2025-10-13 | Roles: SWE Intern

QUESTION

You are given a list of offers in priority order (earliest = highest priority). Each offer is a dict with keys type (either "Fizz" or "Buzz"), name (a string), and quantity (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be partially fulfilled. Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming min(remaining quantities) and recording a match (fizz_name, buzz_name, matched_quantity) ; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue. Aggregate matches so the same (fizz_name, buzz_name) pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: "FizzBuzz! <quantity> <FizzName>:<BuzzName>" .

Function signature

def fizzbuzz_matches(offers: list[dict]) -> list[str]:

...

EXAMPLES

Example 1

Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 5}, {"type": "Buzz", "name": "X", "quantity": 3}, {"type": "Buzz", "name": "Y", "quantity": 4}, ]`
Output	`["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]`
Why	`A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining 2 -> (A,Y,2); Y's leftover 2 waits with no Fizz to match.`

QUESTION

Example 2

Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 3}, {"type": "Buzz", "name": "X", "quantity": 1}, {"type": "Buzz", "name": "X", "quantity": 2}, ]`
Output	`["FizzBuzz! 3 A:X"]`
Why	`A matches the first X for 1 and the second X for 2. Both are the pair (A, X), so they aggregate to a single "FizzBuzz! 3 A:X".`

CONSTRAINTS

- 0 <= len(offers) <= 10**4 ; quantities are positive integers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_matches(offers):
    fizz, buzz, totals = [], [], {}
    for offer in offers:
        mine = (
            fizz
            if offer["type"] == "Fizz"
            else:
                buzz
        )
        other = (
            buzz
            if offer["type"] == "Fizz"
            else:
                fizz
        )
        left = offer["quantity"]
        while left and other:
            m = min(left, other[0][1])
            pair = (
                (offer["name"], other[0][0])
                if offer["type"] == "Fizz"
                else:
                    (
                    other[0][0],
                    offer["name"],
                )
            )
            totals[pair] = (
                totals.get(pair, 0) + m
            )
            left -= m
            other[0][1] -= m
            if other[0][1] == 0:
                other.pop(0)
        if left:
            mine.append([offer["name"], left])
    return [
        f"FizzBuzz! {q} {f}:{b}"
        for (f, b), q in totals.items()
    ]
```

TIME	SPACE
O(n * matches)	O(n)

WHY IT IS SLOW

Removing from the front of a list shifts every other element.

OPTIMAL SOLUTION · Two FIFO queues and an ordered total

TIME	SPACE
O(offers + matches)	O(offers)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 After processing, report which offers were not fully matched.

Keep the two deques as the state: whatever remains in them at the end is the unmatched quantity per offer, in priority order. Return both queues (or merge them).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Aggregate FizzBuzz Offer Matches
#
# You are given a list of offers in priority order
# (earliest = highest priority). Each offer is a dict with
# keys type (either "Fizz" or "Buzz" ), name (a string),
# and quantity (a positive integer). Fizz offers match
# against Buzz offers on a first-come, first-served basis,
# and offers may be partially fulfilled. Process the
# offers in order, maintaining a FIFO queue of unmatched
# Fizz offers and a FIFO queue of unmatched Buzz offers.
# When a new offer arrives, match it against the front of
# the opposite queue repeatedly, each time consuming
# min(remaining quantities) and recording a match
# (fizz_name, buzz_name, matched_quantity) ; drop an offer
# from the queue once its quantity hits 0. If the arriving
# offer still has quantity left when the opposite queue
# empties, enqueue it in its own queue. Aggregate matches
# so the same (fizz_name, buzz_name) pair is reported once
# with its summed quantity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. After processing, report which offers were not fully
# matched.
#
# INTERVIEW NOTES
# Fizz and Buzz offers behave like the two sides of an
# order book matched first come, first served. Keep one
# FIFO queue per side. An arriving offer consumes the
# front of the opposite queue while both have quantity
# left, recording each (fizz, buzz, quantity) match;
# leftovers wait in their own queue. A dictionary keyed by
# the pair, ordered by first appearance, aggregates the
# matches.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
```

```
# ============================================================

from collections import deque
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two FIFO queues and an ordered
# total
# Idea: Match against the opposite queue's front until the
# offer or the queue is empty.
def fizzbuzz_matches(offers):
    queues = {"Fizz": deque(), "Buzz": deque()}
    totals = {}
    for offer in offers:
        mine = queues[offer["type"]]
        other = queues[
            (
                "Buzz"
                if offer["type"] == "Fizz"
                else "Fizz"
            )
        ]
        left = offer["quantity"]
        while left and other:
            front = other[0]
            m = min(left, front[1])
            fizz_name = (
                offer["name"]
                if offer["type"] == "Fizz"
                else front[0]
            )
            buzz_name = (
                front[0]
                if offer["type"] == "Fizz"
                else offer["name"]
            )
            totals[(fizz_name, buzz_name)] = (
                totals.get((fizz_name, buzz_name), 0)
                + m
            )
            left -= m
            front[1] -= m
            if front[1] == 0:
                other.popleft()
        if left:
            mine.append([offer["name"], left])
```

```
    return [
        f"FizzBuzz! {q} {f}:{b}"
        for (f, b), q in totals.items()
    ]

# ---- main question: tests ----
def test_main_question():
    random.seed(63)
    offers = [
        {"type": "Fizz", "name": "A", "quantity": 5},
        {"type": "Buzz", "name": "X", "quantity": 3},
        {"type": "Buzz", "name": "Y", "quantity": 4},
    ]
    assert fizzbuzz_matches(offers) == [
        "FizzBuzz! 3 A:X",
        "FizzBuzz! 2 A:Y",
    ]
    offers2 = [
        {"type": "Fizz", "name": "A", "quantity": 3},
        {"type": "Buzz", "name": "X", "quantity": 1},
        {"type": "Buzz", "name": "X", "quantity": 2},
```

```
    ]
    assert fizzbuzz_matches(offers2) == [
        "FizzBuzz! 3 A:X"
    ]


# ========================================================
# FOLLOW-UP 1: After processing, report which offers were
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the two deques as the state: whatever
# remains in them at the end is the unmatched quantity per
# offer, in priority order. Return both queues (or merge
# them).
def unmatched_after(offers):
    queues = {"Fizz": deque(), "Buzz": deque()}
    for offer in offers:
        mine = queues[offer["type"]]
        other = queues[
            (
                "Buzz"
                if offer["type"] == "Fizz"
                else "Fizz"
            )
        ]
        left = offer["quantity"]
        while left and other:
            m = min(left, other[0][1])
            left -= m
            other[0][1] -= m
            if other[0][1] == 0:
                other.popleft()
        if left:
            mine.append([offer["name"], left])
    return {
        side: [(name, q) for name, q in dq]
        for side, dq in queues.items()
    }

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(400):
        offers = [
            {
                "type": random.choice(["Fizz", "Buzz"]),
                "name": random.choice("ABCDE"),
                "quantity": random.randint(1, 5),
            }
            for _ in range(random.randint(0, 8))
        ]
        left = unmatched_after(offers)
        bought = sum(
            int(line.split()[1])
            for line in fizzbuzz_matches(offers)
        )
        total_f = sum(
            o["quantity"]
            for o in offers
            if o["type"] == "Fizz"
        )
        total_b = sum(
            o["quantity"]
            for o in offers
            if o["type"] == "Buzz"
        )
        assert (
            sum(q for _, q in left["Fizz"])
            == total_f - bought
        )
        assert (
            sum(q for _, q in left["Buzz"])
            == total_b - bought
        )


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 151 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 7 of 8

Template fit: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets | asked 261 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 151 of 288

Restaurant Waitlist: join / delete / find_first_match

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 276/288

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

QUESTION

Design a restaurant waitlist. join(name, size) adds a party with its size at the back. delete(name) removes a party (it may be anywhere in the queue). find_first_match(table) returns the name of the party that joined earliest among those whose size fits the table (size at most table), or None.

Function signature

class Waitlist:
  def join(self, name: str, size: int) -> None
  def delete(self, name: str) -> None
  def find_first_match(self, table: int) -> str | None

EXAMPLES

Example 1	

Input	`w = Waitlist()`
	`w.join('ann', 4)`
	`w.join('bob', 2)`
	`w.join('cy', 3)`
	`w.find_first_match(3)`
	`w.delete('bob')`
	`w.find_first_match(3)`
	`w.find_first_match(1)`

Output	`w.find_first_match(3) -> 'bob'`
	`w.find_first_match(3) -> 'cy'`
	`w.find_first_match(1) -> null`

Why	`A table for 3 skips ann (4 is too big) and takes bob, who joined before cy. After bob leaves it is cy. No party fits a table for 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
queue = []  # in arrival order
def join(user):
    queue.append(user)
def delete(user):
    queue.remove(user)
def find_first_match(t):
    for user in queue:
        if user.party_size <= t:
            return user
    return None
```

TIME	SPACE
O(n) per find and per delete	O(n)

WHY IT IS SLOW

Delete and find are linear in the list. One queue per party size with lazy deletion makes join and delete O(1) and find touch only the queue fronts.

OPTIMAL SOLUTION · Queue per party size with lazy deletion

TIME	SPACE
O(1) join and delete, O(S) find (S distinct sizes up to the table size)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Restaurant Waitlist: join / delete /
# find_first_match
#
# Design a restaurant waitlist. join(name, size) adds a
# party with its size at the back. delete(name) removes a
# party (it may be anywhere in the queue).
# find_first_match(table) returns the name of the party
# that joined earliest among those whose size fits the
```

```
# table (size at most table), or None.
#
# Signature:
#   class Waitlist:
#       def join(self, name: str, size: int) -> None
#       def delete(self, name: str) -> None
#       def find_first_match(self, table: int) -> str | No
ne
#
# FOLLOW-UPS (answered below, in this order):
# 1. What would you cache to speed up repeated finds?
#
# INTERVIEW NOTES
# Join needs the order of arrival, delete can hit anyone,
# and find must return the earliest person whose party
# fits the table. Keep one queue per party size and a
# sequence number per person: find looks at the front of
# every queue for sizes up to the table size and takes the
# smallest sequence number. Deletions are marked and
# skipped lazily when a queue front is read.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
```

```
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
        return queue[0] if queue else None
```

```
    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
        return best[1] if best else None

# ---- main question: tests ----
def test_main_question():
    w = Waitlist()
```

```
    for name, size in [("Alice", 2), ("Bob", 4)]:
        w.join(name, size)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"
    # find does not remove anyone
    assert w.find_first_match(5) == "Alice"
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    # the same size: arrival order decides
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"
    # deleting someone who is not there is harmless
    w.delete("Nobody")


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY PATH: question 152 of 288 | topic 18 of 43: A dict for lookup plus an ordered container for who is next (lazy deletion) | 8 of 8

Template fit: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end) | no date

HARD | IN APP LC 146 | DESIGN-STYLE (skippable) | Exponent | 152 of 288

★ TOP 75: rank 64 in the Medium and Hard priority list

★ Implement LRU Cache.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 57.3** | **30 DAYS 0** | **PRIORITY 55/288**

ABOUT THIS QUESTION

Topic: A dict for lookup plus an ordered container for who is next (lazy deletion)

Optimal answer from: your app's interview approach (LeetCode #146)

LeetCode match: LC 146 LRU Cache

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +2562 interviews

Original: https://www.aced.io/questions/3625/lru-cache

QUESTION

Your program keeps re-reading the same data from disk, so you want an in-memory cache that holds only the n most recently used items (a least-recently-used cache). Write a class LRUCache(n) with get(key), which returns the stored value or -1, and put(key, value), which stores a value and, when the cache is full, first removes the item that was used least recently. Both get and put count as a use. Both must run in O(1).

Function signature

class LRUCache:
  def __init__(self, n: int)
  def get(self, key) -> int
  def put(self, key, value) -> None

EXAMPLES

Example 1

Input	`c = LRUCache(2)`
	`c.put(1, 1)`
	`c.put(2, 2)`
	`c.get(1)`
	`c.put(3, 3)`
	`c.get(2)`
	`c.put(4, 4)`
	`c.get(1)`
	`c.get(3)`
	`c.get(4)`

Output	`c.get(1) -> 1`
	`c.get(2) -> -1`
	`c.get(1) -> -1`
	`c.get(3) -> 3`
	`c.get(4) -> 4`

Why	`get(1) makes key 1 the most recent, so put(3, 3) evicts key 2; then put(4, 4) evicts key 1 (older than 3).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []  # (key, value), newest last
def get(k):
    for i, (key, value) in enumerate(items):
        if key == k:
            items.append(items.pop(i))  # newest
            return value
    return -1
def put(k, v):
    for i, (key, _) in enumerate(items):
        if key == k:
            items.pop(i)
            break
    else:
        if len(items) == capacity:
            items.pop(0)  # oldest
    items.append((k, v))
```

TIME	SPACE
O(n) per operation	**O(capacity)**

WHY IT IS SLOW

Every operation scans the list, O(capacity). A hash map to nodes of a doubly linked list gives O(1) lookup and O(1) move-to-front and eviction.

OPTIMAL SOLUTION · Hash map plus doubly linked list (built by hand)

TIME	SPACE
O(1) per operation	**O(capacity)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many threads share the cache.

Wrap get and put in one lock (both reorder the list, so even get is a write). For more throughput shard the cache by key hash, each shard with its own lock and capacity.

FOLLOW-UP 2 Entries expire after a time to live (TTL).

Store an expiry time with each entry, check it on get (a stale entry is treated as missing and removed), and optionally sweep from the old end of the list on put. The recency order and the O(1) operations are unchanged.

FOLLOW-UP 3 Least frequently used (LFU) instead of least recently used.

Keep a frequency per key and buckets of keys per frequency (each bucket an ordered dict) with a pointer to the minimum frequency; get and put move a key to the next bucket in O(1) (LeetCode 460).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement LRU Cache.
#
# Your program keeps re-reading the same data from disk,
# so you want an in-memory cache that holds only the n
# most recently used items (a least-recently-used cache).
# Write a class LRUCache(n) with get(key), which returns
# the stored value or -1, and put(key, value), which
# stores a value and, when the cache is full, first
# removes the item that was used least recently. Both get
# and put count as a use. Both must run in O(1).
#
# Signature:
#   class LRUCache:
#       def __init__(self, n: int)
#       def get(self, key) -> int
#       def put(self, key, value) -> None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many threads share the cache.
# 2. Entries expire after a time to live (TTL).
# 3. Least frequently used (LFU) instead of least recently
# used.
#
# INTERVIEW NOTES
# You need two things at once: find a key quickly (a hash
# map) and know which key is the oldest use (an ordering
# you can update in O(1)). A doubly linked list gives O(1)
# move-to-front and remove-from-back, and each hash map
# entry points at its list node. Python's OrderedDict is
# exactly that pair; in an interview be ready to build the
# linked list by hand.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import threading
import time
from collections import OrderedDict, defaultdict


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = None

# ---- main question: main function ----
# BASE QUESTION (optimal): Hash map plus doubly linked
# list (built by hand)
```

```
# Idea: Each key maps to a node of a doubly linked list
# ordered by recency, with sentinel head and tail nodes so
# no operation has to special-case an empty list.
class LRUCache:
    """get and put in O(1): a hash map plus a doubly
    linked list, most recently used item at the front."""

    def __init__(self, capacity):
        self.cap, self.map = capacity, {}
        # sentinel nodes mean no special cases for the
        # ends
        self.head, self.tail = Node(), Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.map.get(key)
        if node is None:
            return -1
        # reading makes it the most recently used
        self.unlink(node)
        self.push_front(node)
```

```
        return node.value

    def put(self, key, value):
        node = self.map.get(key)
        if node:
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.map) == self.cap:
            # full: evict the least recently used (at the
            # back)
            oldest = self.tail.prev
            self.unlink(oldest)
            del self.map[oldest.key]
        node = Node(key, value)
        self.map[key] = node
        self.push_front(node)

# ---- main question: tests ----
def test_main_question():
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == 1
    # 2 is now the least recently used, so 3 evicts it
    c.put(3, 3)
    assert c.get(2) == -1 and c.get(3) == 3
    c.put(1, 10)
    assert c.get(1) == 10


# ============================================================
# FOLLOW-UP 1: Many threads share the cache.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Wrap get and put in one lock (both reorder the
# list, so even get is a write). For more throughput shard
# the cache by key hash, each shard with its own lock and
# capacity.
```

```
class SafeLRU(LRUCache):
    """Many threads: even get edits the list: one lock."""

    def __init__(self, capacity):
```

```
        super().__init__(capacity)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return super().get(key)

    def put(self, key, value):
        with self.lock:
            super().put(key, value)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = SafeLRU(1)
    s.put("a", 1)
    s.put("b", 2)
    assert s.get("a") == -1 and s.get("b") == 2
    now = [0]


# ============================================================
# FOLLOW-UP 2: Entries expire after a time to live (TTL).
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store an expiry time with each entry, check it
# on get (a stale entry is treated as missing and
# removed), and optionally sweep from the old end of the
# list on put. The recency order and the O(1) operations
# are unchanged.
class TTLCache(LRUCache):
    """Entries also expire after ttl seconds."""

    def __init__(self, capacity, ttl,
                 clock=time.monotonic):
        super().__init__(capacity)
        self.ttl, self.clock, self.expiry = ttl, clock, {}

    def get(self, key):
        # a stale entry counts as missing and is removed
        stale = key in self.map and (
            self.expiry[key] <= self.clock())
        if stale:
            self.unlink(self.map.pop(key))
            del self.expiry[key]
```

```
        return super().get(key)

    def put(self, key, value):
        super().put(key, value)
        self.expiry[key] = self.clock() + self.ttl
        # drop expiry records of evicted keys
        for k in list(self.expiry):
            if k not in self.map:
                del self.expiry[k]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    now = [0]
    t = TTLCache(2, ttl=10, clock=lambda: now[0])
    t.put("x", 1)
    assert t.get("x") == 1
    now[0] = 11
    assert t.get("x") == -1


# ============================================================
# FOLLOW-UP 3: Least frequently used (LFU) instead of
```

```
# ============================================================

# ---- follow-up 3: function ----
# Answer: Keep a frequency per key and buckets of keys per
# frequency (each bucket an ordered dict) with a pointer
# to the minimum frequency; get and put move a key to the
# next bucket in O(1) (LeetCode 460).
class LFUCache:
    """Least FREQUENTLY used (LeetCode 460), O(1)."""

    def __init__(self, capacity):
        self.cap = capacity
        self.value, self.freq = {}, {}
        # frequency -> keys in that bucket, oldest first
        self.bucket = defaultdict(OrderedDict)
        self.low = 0

    def touch(self, key):
        f = self.freq[key]
        del self.bucket[f][key]
        # if this was the lowest bucket and it is empty,
        # move up
        if not self.bucket[f] and self.low == f:
```

```
            self.low += 1
        self.freq[key] = f + 1
        self.bucket[f + 1][key] = None

    def get(self, key):
        if key not in self.value:
            return -1
        self.touch(key)
        return self.value[key]

    def put(self, key, value):
        if self.cap == 0:
            return
        if key in self.value:
            self.value[key] = value
            self.touch(key)
            return
        if len(self.value) == self.cap:
            # evict the oldest key of the lowest frequency
            bucket = self.bucket[self.low]
            old, _ = bucket.popitem(last=False)
            del self.value[old], self.freq[old]
        self.value[key], self.freq[key] = value, 1
        self.bucket[1][key] = None
        self.low = 1

# ---- follow-up 3: tests ----
def test_follow_up_3():
    f = LFUCache(2)
    f.put(1, 1)
    f.put(2, 2)
    assert f.get(1) == 1
    # 2 was used least often, so it goes
    f.put(3, 3)
    assert f.get(2) == -1 and f.get(3) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time?

```
Pythonclass LRUCache(n):
  set(key, value)
  get(key)
```

Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated.

```
Pythoncache = LRUCache(2) # Limit of 2 items

cache.set('user1', 'Alex')
cache.set('user2', 'Brian')
cache.set('user3', 'Chris')
```

Here user1 is empty because it was the least recently used and thus removed to make room for user3:

```
Pythoncache.get('user1') # => None
cache.get('user2') # => 'Brian'
cache.get('user3') # => 'Chris'
```

OFFICIAL PAGE TEXT

Hint 1If you're having trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array won't work because moving items around takes O(n) time. Could you combine two different data structures to maintain this order and also get fast lookups?Hint 3You can use a doubly linked list to store the items in the correct order. When an item is used, just move its node to the front of the list. The least recently used node will always end up at the end of the list.Hint 4To look up items without traversing the entire list, you can also add a hash table that maps from the key to the item's node.Challenge 1Whats the time complexity of your solution?This problem is challenging because of the combination of constraints such as key-value lookup, size limits, and time complexity. Ultimately, there is no single data structure that fulfills all of these needs, so we have to get creative! Our solution is to implement the cache class with a linked list and a hash table. The linked list is doubly-linked, which allows us to insert and remove items in constant time without traversing the entire list. Every time an item is set, updated, or retrieved, we just move its Node to the front of the list. When the cache reaches its size limit, we can easily make room by removing the item at the end of the list. The linked list gives us O(1) insertion time, but by itself it has O(n) lookup time because we have to traverse the entire list to find a particular item. That's not ideal since we want to be able to look up an item quickly using its key. That's where the hash table comes in: we can use it add a mapping from an item's key to its Node. Together, these data structures give us a time complexity of O(1) for insertion, deletion, and retrieval in the worst case, and a space footprint of O(n). No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 19 OF 43

Rolling state: the next state comes only from the previous one

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	4	1
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Rolling state: the next state comes only from the previous one | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

E	153. Climbing Stairs	Exponent	no date
E	154. Fibonacci Numbers	Exponent	no date
M	155. Maximize sum without choosing adjacent elements	PracHub	2026-01-06
M	156. Given n houses in a line with money in each, find the maximum amount a robber can steal, witho...	Exponent	no date
M	157. Paint House II	ColorStack	2025-10-17
M	158. Write SQL code to publish the Fibonacci series.	Exponent	no date
H	159. Constrained Dice Roll Sequences	ColorStack	2025-10-06

CODE TEMPLATE | GROUP 19 OF 43 | 7 QUESTIONS, ONE PATTERN

Rolling state: the next state comes only from the previous one

The one idea these questions share

You never need the whole table, only the previous state: one number, a pair, or one row. Write the state, then one line that builds the new state from the old one, and loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def roll(items):
    take, skip = 0, 0                           # CHANGE 1: the state
    for x in items:
        take, skip = skip + x, max(take, skip)     # CHANGE 2: new state from the old state
    return max(take, skip)
```

2. Helper functions that usually get written next to it

```
def two_smallest(row):                    # the trick behind "best of the others"
    order = sorted(range(len(row)), key=lambda j: row[j])
    return order[0], order[1] if len(order) > 1 else order[0]


def fib_pair(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
153	Climbing Stairs	state = the last two counts: a, b = b, a + b, repeated n - 1 times
154	Fibonacci Numbers	the same pair update starting from (0, 1)
155	Maximize sum without choosing adjacent elements	exactly the house robber update on the array
156	Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.	state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)
157	Paint House II	state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)
158	Write SQL code to publish the Fibonacci series.	WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query
159	Constrained Dice Roll Sequences	state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 153 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 1 of 7

Template fit: state = the last two counts: a, b = b, a + b, repeated n - 1 times | no date

EASY | IN APP LC 70 | Exponent | 153 of 288

★ TOP 10 EASY: rank 6 among Easy problems

★ Climbing Stairs

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 62.9 | 30 DAYS 51.4 | PRIORITY 9/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: your app's interview approach (LeetCode #70)

LeetCode match: LC 70 Climbing Stairs

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3255/climbing-stairs

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.
Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:
1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

EXAMPLES

Example 1	
Input	2
Output	2
Why	`Two ways to climb 2 steps: 1+1 or 2.`

QUESTION

Example 2	
Input	3
Output	3
Why	`Three ways to climb 3 steps: 1+1+1, 1+2, 2+1.`

Example 3	
Input	5
Output	8
Why	`Eight ways for 5 steps.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def ways(n):
    if n <= 1:
        return 1
    return ways(n - 1) + ways(n - 2)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

The same values are computed over and over: O(2^n). Keeping the last two values in a loop is O(n) time and O(1) space (and a matrix power gives O(log n)).

OPTIMAL SOLUTION · Two running values

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 n is huge (10^18).

Use matrix exponentiation of [[1, 1], [1, 0]] (or fast doubling) for O(log n) multiplications; take the result modulo a prime if asked.

FOLLOW-UP 2 You can take 1 to k steps (or a given set of step sizes).

ways[i] = sum of ways[i - s] over allowed step sizes s; for 1..k keep a sliding-window sum for O(n).

FOLLOW-UP 3 Some steps are broken (cannot be stood on).

Set ways[i] = 0 for broken steps while filling the table; everything else is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Climbing Stairs
#
# Given an integer n, representing the number of steps in
# a staircase, write a function climbStairs that returns
# the number of distinct ways to climb to the top of the
# staircase. Each time you can either climb 1 step or 2
# steps. You may assume that n is a non-negative integer.
# Examples Input: n = 1 Output: 1 Explanation: There is
# only 1 way to climb a staircase with 1 step. Input: n =
# 2 Output: 2 Explanation: There are 2 ways to climb a
# staircase with 2 steps: 1. Climb 1 step + 1 step 2.
# Climb 2 steps directly Input: n = 3 Output: 3
# Explanation: There are 3 ways to climb a staircase with
# 3 steps: 1. Climb 1 step + 1 step + 1 step 2. Climb 1
# step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4
# Output: 5 Explanation: There are 5 ways to climb a
# staircase with 4 steps: 1. Climb 1 step + 1 step + 1
# step + 1 step 2. Climb 1 step + 1 step + 2 steps 3.
# Climb 1 step + 2 steps + 1 step 4.
#
# FOLLOW-UPS (answered below, in this order):
# 1. n is huge (10^18).
# 2. You can take 1 to k steps (or a given set of step
# sizes).
# 3. Some steps are broken (cannot be stood on).
#
# INTERVIEW NOTES
# The last move was either a 1-step from n - 1 or a 2-step
# from n - 2, so ways(n) = ways(n - 1) + ways(n - 2): the
# Fibonacci recurrence. Only the previous two values
# matter, so two variables replace the whole table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Two running values
# Idea: Keep ways for the previous two steps and advance:
# (a, b) = (b, a + b). Start with ways(0) = ways(1) = 1.
# No table and no recursion.
def climb(n):
    # ways(n) = ways(n - 1) + ways(n - 2): O(n), O(1)
    # space
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n >= 1 else 1

# ---- main question: tests ----
def test_main_question():
    got = [climb(n) for n in range(1, 6)]
    assert got == [1, 2, 3, 5, 8]
    assert climb(0) == 1


# ============================================================
# FOLLOW-UP 1: n is huge (10^18).
# ============================================================

# ---- follow-up 1: helper functions ----
def mat_mult(x, y, mod):
    out = [[0, 0], [0, 0]]
    for i in range(2):
        for j in range(2):
            cells = (x[i][k] * y[k][j] for k in range(2))
            total = sum(cells)
            out[i][j] = total % mod
    return out

# ---- follow-up 1: function ----
# Answer: Use matrix exponentiation of [[1, 1], [1, 0]]
# (or fast doubling) for O(log n) multiplications; take
# the result modulo a prime if asked.
def climb_huge(n, mod=10 ** 9 + 7):
    # n up to 10^18: matrix power of [[1, 1], [1, 0]]
    result, base = [[1, 0], [0, 1]], [[1, 1], [1, 0]]
    e = n
    while e:
        if e & 1:
            result = mat_mult(result, base, mod)
        base = mat_mult(base, base, mod)
        e >>= 1
    # result = [[F(n+1), F(n)], [F(n), F(n-1)]]
    return result[0][0] % mod

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert climb_huge(5) == 8 and climb_huge(1) == 1
    assert 0 <= climb_huge(10 ** 18) < 10 ** 9 + 7
    assert climb_huge(50) == climb(50) % (10 ** 9 + 7)


# ============================================================
# FOLLOW-UP 2: You can take 1 to k steps (or a given set
# ============================================================

# ---- follow-up 2: function ----
# Answer: ways[i] = sum of ways[i - s] over allowed step
# sizes s; for 1..k keep a sliding-window sum for O(n).
def climb_steps(n, steps):
    # any allowed step sizes: sum the ways of the earlier
    # stairs
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        ways[i] = sum(ways[i - s] for s in steps
                      if i >= s)
    return ways[n]


# Answer: ways[i] = sum of ways[i - s] over allowed step
# sizes s; for 1..k keep a sliding-window sum for O(n).
def climb_up_to_k(n, k):
    # steps 1..k in O(n): keep a sliding-window sum
    ways, window = [1] + [0] * n, 1
    for i in range(1, n + 1):
```

```
        ways[i] = window
        window += ways[i]
        if i - k >= 0:
            window -= ways[i - k]
    return ways[n]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert climb_steps(5, (1, 2)) == 8
    assert climb_steps(4, (1, 3)) == 3
    assert climb_up_to_k(5, 2) == 8
```

```
    assert climb_up_to_k(4, 3) == 7


# ========================================================
# FOLLOW-UP 3: Some steps are broken (cannot be stood on).
# ========================================================

# ---- follow-up 3: function ----
# Answer: Set ways[i] = 0 for broken steps while filling
# the table; everything else is unchanged.
def climb_broken(n, broken):
    # broken steps cannot be stood on: zero ways there
    bad = set(broken)
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        if i not in bad:
            two_back = ways[i - 2] if i > 1 else 0
            ways[i] = ways[i - 1] + two_back
    return ways[n]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert climb_broken(5, [3]) == 2
    assert climb_broken(3, [1, 2]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples

OFFICIAL PAGE TEXT

```
Input: n = 1
Output: 1
Explanation: There is only 1 way to climb a staircase
with 1 step.

Input: n = 2
Output: 2
Explanation: There are 2 ways to climb a staircase wit
h 2 steps:
1. Climb 1 step + 1 step
2. Climb 2 steps directly

Input: n = 3
Output: 3
Explanation: There are 3 ways to climb a staircase wit
h 3 steps:
1. Climb 1 step + 1 step + 1 step
2. Climb 1 step + 2 steps
3. Climb 2 steps + 1 step

Input: n = 4
Output: 5
Explanation: There are 5 ways to climb a staircase wit
h 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step
2. Climb 1 step + 1 step + 2 steps
3. Climb 1 step + 2 steps + 1 step
4. Climb 2 steps + 1 step + 1 step
5. Climb 2 steps + 2 steps
```

Challenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] = 1 (1 way to reach the first step, which is a single 1-step) For each subsequent step i, the number of ways to reach that step is the sum of the ways to reach the previous step (ways[i - 1]) and the step before that (ways[i - 2]). This is because you can get to step i either from step i - 1 with a 1-step or from step i - 2 with a 2-step. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we iterate through the steps from 2 to n, updating the number of ways for each step. Space Complexity: The space complexity is O(n) due to the additional array ways used to store the number of ways for each step. If space optimization is needed, this can be reduced to O(1) by keeping only the last two computed values. Solution 2: Combinatorial Approach Our solution leverages combinatorial mathematics to calculate

OFFICIAL PAGE TEXT

the number of distinct ways to climb n stairs. The approach is based on counting the number of valid sequences of steps (1-step and 2-steps) that sum up to n. We use the binomial coefficient to count the number of ways to arrange a certain number of 1-steps and 2-steps. For each possible number of 2-steps (k), we compute the number of ways to choose k positions from n - k steps (the remaining steps will be 1-steps). This is done using the formula for combinations: C(n,k)C(n, k)C(n,k), where C(n,k)=n!k!*(n-k)!C(n, k) = \frac{n!}{k! \cdot (n-k)!}C(n,k)=k!*(n-k)!n!. The total number of ways is the sum of these combinations for all valid values of k. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we compute the combination for each k from 0 to n // 2, and each combination computation is efficient. Space Complexity: The space complexity is O(1) as we only use a few additional variables for storing intermediate results, regardless of the input size.No answers contributed by the community yet.

STUDY PATH: question 154 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 2 of 7

Template fit: the same pair update starting from (0, 1) | no date

EASY | IN APP LC 509 | Exponent | 154 of 288

Fibonacci Numbers

ASKED (ANY CO.) ~9 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 46.3 | 30 DAYS 0 | PRIORITY 89/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: your app's interview approach (LeetCode #509)

LeetCode match: LC 509 Fibonacci Number

Last asked (any company): 9 months ago, about 2026-01 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/3631/fibonacci

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

EXAMPLES

QUESTION

Example 1

Input	`0`
Output	`1`
Why	`F(0) = 1 and F(1) = 1, as defined in this question.`

Example 2

Input	`5`
Output	`8`
Why	`1, 1, 2, 3, 5, 8: F(5) = 8.`

Example 3

Input	`10`
Output	`89`
Why	`F(10) = 89.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fib(n):
    if n < 2:
        return 1  # fib(0) = fib(1) = 1
    return fib(n - 1) + fib(n - 2)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

The call tree has about 1.6^n nodes because the same values are recomputed. A loop with two variables is O(n) time and O(1) space; fast doubling is O(log n).

OPTIMAL SOLUTION · Iteration with two variables

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 n is up to 10^18 and the answer is needed modulo a prime.

Fast doubling (or matrix power) with every multiplication reduced modulo the prime: O(log n) steps with small numbers.

FOLLOW-UP 2 Why is the recursive version slow, and how do you fix it without changing the structure?

It recomputes the same values exponentially often (the call tree has about phi^n nodes). Memoisation (lru_cache) makes each value computed once, so O(n) time with O(n) memory; bottom-up with two variables drops the memory to O(1).

FOLLOW-UP 3 Closed form (Binet's formula).

F(n) = round(phi^n / sqrt(5)) is exact only while floating point has enough precision (up to about n = 70); beyond that it gives wrong digits, so use the integer methods.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Fibonacci Numbers
#
# Write a function fib(n) that returns the nth Fibonacci
# number. The Fibonacci sequence is defined as F(n) =
# F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which
# produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ...
# Examples Pythonfib(0) # => 1 fib(1) # => 1 fib(2) # => 2
# fib(3) # => 3 ... fib(10) # => 89 fib(20) # => 10946Hint
# 1The Fibonacci sequence is defined recursively since it
# refers to itself.Hint 2We can start with a recursive
# implementation and optimize it.Hint 3Each call to F(n)
# has two subproblems: F(n-1) and F(n-2). Can we memoize
# these calculations?Challenge 1Does your solution work in
# O(n) time?This question is a classic example of how
# dynamic programming, or memoization, can help us solve
# problems faster by reducing repetitive computations.
#
# FOLLOW-UPS (answered below, in this order):
# 1. n is up to 10^18 and the answer is needed modulo a
# prime.
# 2. Why is the recursive version slow, and how do you fix
# it without changing the structure?
# 3. Closed form (Binet's formula).
#
# INTERVIEW NOTES
# Each term needs only the previous two, so a loop with
# two variables is O(n) time and O(1) space; the naive
# recursion does exponential repeated work. Since the
# recurrence is linear it can also be turned into a matrix
# power, or into the fast doubling identities, for O(log
# n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from functools import lru_cache
from math import sqrt
```

```
# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# THE DEFINITION (slow, say it first)
def fib_slow(n):
    # the definition: correct but about 1.6 ** n calls
    if n < 2:
        return 1
    return fib_slow(n - 1) + fib_slow(n - 2)


# BASE QUESTION (optimal): Iteration with two variables
# Idea: Keep the previous two values and advance n times.
# Memoising the recursion gives the same time with O(n)
# memory; the loop needs no memory and no recursion.
def fib(n):
    # bottom-up with two variables: O(n) time, O(1) memory
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a

# ---- main question: tests ----
def test_main_question():
    first = [fib(n) for n in range(7)]
    assert first == [1, 1, 2, 3, 5, 8, 13]
    assert fib(10) == 89 and fib(20) == 10946
    assert fib_slow(10) == 89


# ============================================================
# FOLLOW-UP 1: n is up to 10^18 and the answer is needed
# ============================================================

# ---- follow-up 1: helper functions ----
# this problem starts at fib(0) = 1, fib(1) = 1
MOD = 10 ** 9 + 7


def pair(n, mod):
    # follow-up: fast doubling. Returns (F(n), F(n + 1))
    # of
    # the standard sequence 0, 1, 1, 2 using
    #   F(2k)     = F(k) * (2 * F(k + 1) - F(k))
    #   F(2k + 1) = F(k) ** 2 + F(k + 1) ** 2
```

```
    if n == 0:
        return 0, 1
    a, b = pair(n // 2, mod)
    c = a * (2 * b - a) % mod
    d = (a * a + b * b) % mod
    return (d, c + d) if n % 2 else (c, d)

# ---- follow-up 1: function ----
# Answer: Fast doubling (or matrix power) with every
# multiplication reduced modulo the prime: O(log n) steps
# with small numbers.
def fib_mod(n, mod=MOD):
    # fib(n) here is the standard F(n + 1): O(log n) steps
    # with small numbers, so n = 10 ** 18 is easy
    return pair(n + 1, mod)[0] % mod

# ============================================================
# FOLLOW-UP 2: Why is the recursive version slow, and how
# ============================================================

# ---- follow-up 2: function ----
# Answer: It recomputes the same values exponentially
# often (the call tree has about phi^n nodes). Memoisation
```

```
# (lru_cache) makes each value computed once, so O(n) time
# with O(n) memory; bottom-up with two variables drops the
# memory to O(1).
@lru_cache(maxsize=None)
def fib_memo(n):
    # follow-up: the same structure with a cache, so every
    # value is computed once: O(n) time and memory
    if n < 2:
        return 1
    return fib_memo(n - 1) + fib_memo(n - 2)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert fib_memo(20) == 10946
    assert fib(0) == 1 and fib(1) == 1
    for n in range(0, 200):
        assert fib_mod(n, MOD) == fib(n) % MOD
        big = 10 ** 18 + 9
        assert fib_mod(n, big) == fib(n) % big
    # a huge n finishes at once
    assert 0 <= fib_mod(10 ** 18) < MOD
```

```
# ============================================================
# FOLLOW-UP 3: Closed form (Binet's formula).
# ============================================================

# ---- follow-up 3: function ----
# Answer: F(n) = round(phi^n / sqrt(5)) is exact only
# while floating point has enough precision (up to about n
# = 70); beyond that it gives wrong digits, so use the
# integer methods.
def fib_binet(n):
    # follow-up: closed form. Floats run out of precision.
    phi = (1 + sqrt(5)) / 2
    return round(phi ** (n + 1) / sqrt(5))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # Binet is exact for a while, then floating point
    # fails
    assert all(fib_binet(n) == fib(n) for n in range(60))
    late = range(60, 200)
    bad = [n for n in late if fib_binet(n) != fib(n)]
    assert bad and bad[0] < 100


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern:

```
1, 1, 2, 3, 5, 8, 13 ...
```

Examples

OFFICIAL PAGE TEXT

```
Pythonfib(0)  # => 1
fib(1)  # => 1
fib(2)  # => 2
fib(3)  # => 3
...
fib(10) # => 89
fib(20) # => 10946
```

Hint 1The Fibonacci sequence is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant calculations. Space Complexity: The memo dictionary stores each Fibonacci number from 0 to n, which requires O(n) space.No answers contributed by the community yet.

STUDY PATH: question 155 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 3 of 7

Template fit: exactly the house robber update on the array | asked 287 days ago

MEDIUM | IN APP LC 198 | PracHub | Technical Screen | 155 of 288

★ TOP 75: rank 61 in the Medium and Hard priority list

★ Maximize sum without choosing adjacent elements

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.2 | 30 DAYS 0 | PRIORITY 57/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #198 House Robber

LeetCode match: LC 198 House Robber (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/maximize-sum-without-choosing-adjacent-elements

QUESTION

Given a list of non-negative numbers, choose some of them so that no two chosen numbers are neighbours in the list, and return the largest possible sum of the chosen numbers.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	`[2, 7, 9, 3, 1]`
Output	`12`
Why	`Choose 2, 9 and 1 (positions 0, 2, 4): 12.`

Example 2	
Input	`[5]`
Output	`5`
Why	`One number: take it.`

Example 3	
Input	`[]`
Output	`0`
Why	`Nothing to choose: 0.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximize sum without choosing adjacent
# elements
#
# Given a list of non-negative numbers, choose some of
# them so that no two chosen numbers are neighbours in the
# list, and return the largest possible sum of the chosen
# numbers.
```

```
#
# Signature:
#   def rob_line(nums: list) -> int
#
# Examples:
# [2, 7, 9, 3, 1]  ->  12
# [5]  ->  5
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
```

```
def rob_line(nums):
    # take: best when the current element is taken,
    # skip: best when it is not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)

# ---- main question: tests ----
def test_main_question():
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_line([5]) == 5



# ============================================================
# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# ============================================================

# ---- follow-up 1: function ----
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last are neighbours: ignore one of
    # them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([4]) == 4



# ============================================================
# FOLLOW-UP 2: Return which houses to rob.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
```

```
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i onward, filled from the
    # right
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    chosen, i = [], 0
    while i < n:
        if nums[i] + dp[i + 2] == dp[i] and nums[i] > 0:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return dp[0], chosen

# ---- follow-up 2: tests ----
def test_follow_up_2():
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
    values = [2, 7, 9, 3, 1]
    assert sum(values[i] for i in houses) == total


# ============================================================
# FOLLOW-UP 3: Robbing a house also alerts the next k
# ============================================================

# ---- follow-up 3: function ----
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_gap(nums, k):
    # taking a house blocks the next k houses (k = 1 is
    # the
    # classic problem): dp[i] = max(dp[i + 1], nums[i] +
    # dp[i + k + 1])
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
```

```
    return dp[0]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rob_gap([2, 7, 9, 3, 1], 1) == 12
    # blocking two houses after each pick
    assert rob_gap([5, 1, 1, 5, 1, 1, 5], 2) == 15



# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> int:
```

Sample call

```
solution([2, 7, 9, 3, 1])
```

Test cases

Input	Expected
([2, 7, 9, 3, 1],)	12
([1, 2, 3, 1],)	4

STUDY PATH: question 156 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 4 of 7

Template fit: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip) | no date

MEDIUM | IN APP LC 198 | Exponent | 156 of 288

★ TOP 75: rank 74 in the Medium and Hard priority list

★ Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.2 | 30 DAYS 0 | PRIORITY 58/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: your app's interview approach (LeetCode #198)

LeetCode match: LC 198 House Robber (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4646/house-robber

QUESTION

Houses stand in a line and house i holds nums[i] money. A robber cannot rob two houses that are next to each other. Return the largest total amount he can steal.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	[1, 2, 3, 1]
Output	4
Why	Rob houses 0 and 2: 1 + 3 = 4.

Example 2	
Input	[2, 7, 9, 3, 1]
Output	12
Why	Rob houses 0, 2 and 4: 2 + 9 + 1 = 12.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

BRUTE FORCE

WHY IT IS SLOW

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given n houses in a line with money in
# each, find the maximum amount a robber can steal,
# without stealing from two adjacent houses.
#
# Houses stand in a line and house i holds nums[i] money.
# A robber cannot rob two houses that are next to each
# other. Return the largest total amount he can steal.
#
# Signature:
#   def rob_line(nums: list) -> int
#
# Examples:
# [1, 2, 3, 1]  ->  4
# [2, 7, 9, 3, 1]  ->  12
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
```

```
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
```

```
def rob_line(nums):
    # take = best sum if we take this house; skip = if we
    # do not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)

# ---- main question: tests ----
def test_main_question():
    assert rob_line([1, 2, 3, 1]) == 4
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0


# ============================================================
# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# ============================================================

# ---- follow-up 1: function ----
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last houses touch: ignore one of them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([5]) == 5


# ============================================================
# FOLLOW-UP 2: Return which houses to rob.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i to the end (filled from
    # the right)
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    houses, i = [], 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            houses.append(i)
            i += 2
        else:
            i += 1
    return dp[0], houses

# ---- follow-up 2: tests ----
def test_follow_up_2():
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]


# ============================================================
# FOLLOW-UP 3: Robbing a house also alerts the next k
# ============================================================

# ---- follow-up 3: function ----
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_alarm(nums, k):
    # robbing a house alerts the next k houses
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rob_alarm([2, 7, 9, 3, 1], 1) == 12
    assert rob_alarm([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given an array representing the value in the houses e.g. [100, 20, 40, 70, 80]. Return 100 + 40 + 80 = 220

STUDY PATH: question 157 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 5 of 7

Template fit: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest) | asked 368 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 157 of 288

Paint House II

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 227/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 265 Paint House II

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

There are n houses in a row, each to be painted one of k colors. costs[i][j] is the cost to paint house i with color j . No two adjacent houses may share the same color. Return the minimum total cost to paint all houses.

Function signature

def min_cost_paint_houses(costs: list[list[int]]) -> int:
    ...

Follow-up (as reported): Reduce the per-row transition from O(k) per cell to O(1) amortized by tracking the two smallest values of the previous row.

EXAMPLES

Example 1	
Input	`costs = [[1, 5, 3], [2, 9, 4]]`
Output	`5`
Why	`Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2 (3) and house 1 color 0 (2) -> 5.`

Example 2	
Input	`costs = [[1, 3], [2, 4]]`
Output	`5`
Why	`1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.`

Example 3	
Input	`costs = [[7]]`
Output	`7`
Why	`A single house, one color.`

CONSTRAINTS

- costs is an n x k grid, 0 <= n , k >= 1 (and k >= 2 whenever n >= 2 , so a valid painting exists).
- Costs are non-negative integers. Return 0 for n == 0 .
- Aim for O(n*k) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import product
def brute_paint(costs):
    n, k = len(costs), len(costs[0])
    best = float("inf")
    for colors in product(range(k),_repeatn):
        if all(
            a != b
            for a, b in zip(colors, colors[1:])
        ):
            best = min(
                best,
                sum(
                    costs[i][c]
                    for i, c in enumerate(
                        colors
                    )
                ),
            )
    return best
```

TIME	SPACE
O(k^n * n)	O(n)

WHY IT IS SLOW

It enumerates all k^n ways to colour the row.

OPTIMAL SOLUTION · Two smallest values of the previous row

TIME	SPACE
O(n k)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return which colour each house gets, not just the cost.

Store the DP row for every house (O(n k) memory), then walk back from the cheapest last cell: at house i pick the cheapest entry of the row of house i - 1 whose colour differs from the colour of house i + 1... in practice keep the argmin of the previous row and the second argmin when you store each row.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Paint House II
#
# There are n houses in a row, each to be painted one of k
```

```
# colors. costs[i][j] is the cost to paint house i with
# color j . No two adjacent houses may share the same
# color. Return the minimum total cost to paint all
# houses. Function signature: def
# min_cost_paint_houses(costs: list[list[int]]) -> int:
# ... Follow-up (as reported): Reduce the per-row
# transition from O(k) per cell to O(1) amortized by
# tracking the two smallest values of the previous row.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return which colour each house gets, not just the
# cost.
#
# INTERVIEW NOTES
# dp[i][j] is the cheapest cost to paint houses 0..i with
# house i in colour j: costs[i][j] plus the cheapest
# previous row entry that is not colour j. Taking the
# minimum over k - 1 colours for every cell is O(n k^2);
# keeping only the smallest and second smallest values of
# the previous row gives each cell its answer in O(1), for
# O(n k) total.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two smallest values of the
# previous row
# Idea: A cell only needs the cheapest previous colour
# that differs from its own.
def min_cost_paint_houses(costs):
    prev = [0] * len(costs[0])
    for row in costs:
        best = sorted(
            range(len(prev)), key=lambda j: prev[j]
        )[:2]
        first = best[0]
        second = best[1] if len(best) > 1 else best[0]
        prev = [
            row[j]
            + (
                prev[second]
                if j == first
                else prev[first]
            )
            for j in range(len(row))
        ]
    return min(prev)

# ---- main question: tests ----
def test_main_question():
    random.seed(68)
    assert (
        min_cost_paint_houses([[1, 5, 3], [2, 9, 4]])
        == 5
    )
    assert min_cost_paint_houses([[1, 3], [2, 4]]) == 5
    assert min_cost_paint_houses([[7]]) == 7
```

```
# ============================================================
# FOLLOW-UP 1: Return which colour each house gets, not
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store the DP row for every house (O(n k)
# memory), then walk back from the cheapest last cell: at
# house i pick the cheapest entry of the row of house i -
# 1 whose colour differs from the colour of house i + 1...
def paint_plan(costs):
    n, k = len(costs), len(costs[0])
    rows = []
    prev = [0] * k
    for row in costs:
        cur = [
            row[j]
            + min(
                (prev[x] for x in range(k) if x != j),
                default=0,
            )
            for j in range(k)
        ]
        rows.append(cur)
        prev = cur
    colour = min(range(k), key=lambda j: rows[-1][j])
    total, plan = rows[-1][colour], [colour]
    for i in range(n - 1, 0, -1):
        need = rows[i][colour] - costs[i][colour]
        colour = next(
            x
            for x in range(k)
            if x != colour and rows[i - 1][x] == need
        )
        plan.append(colour)
    return total, plan[::-1]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        n, k = random.randint(1, 5), random.randint(
            2, 4
        )
        costs = [
            [random.randint(1, 9) for _ in range(k)]
            for _ in range(n)
        ]
        total, plan = paint_plan(costs)
        assert all(
            a != b for a, b in zip(plan, plan[1:])
        )
        assert (
            sum(costs[i][c] for i, c in enumerate(plan))
            == total
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 158 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 6 of 7

Template fit: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query | no date

MEDIUM | Exponent | 158 of 288

Write SQL code to publish the Fibonacci series.

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 256/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: written and tested here for this exact question

LeetCode match: No LC match (SQL) (inferred from short statement)

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2779/sql-code-fibonacci-series

QUESTION

Write a SQL query that publishes the first N Fibonacci numbers (0, 1, 1, 2, 3, 5, ...), one per row and in order, where N is given as the parameter :limit. Use a recursive common table expression.

Function signature

-- result: one column named fibonacci, N rows

EXAMPLES

Example 1	
Input	`:limit = 8`
Output	`0, 1, 1, 2, 3, 5, 8, 13`
Why	`Eight rows: each number is the sum of the two before it.`

Example 2	
Input	`:limit = 1`
Output	`0`
Why	`A single row.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
a, b = 0, 1
for _ in range(n):
    execute("INSERT INTO fib VALUES (?)", a)
    a, b = b, a + b
```

TIME	SPACE
O(n)	O(n)
statements	

WHY IT IS SLOW

BRUTE FORCE

One statement and one round trip per term, and a loop outside the query language. A recursive CTE produces all rows in one set-based statement.

OPTIMAL SOLUTION · Recursive CTE carrying (n, a, b)

TIME	SPACE
O(n)	O(1)
	working set (O(n) result)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the even terms, or the sum of the first n terms.

Keep the CTE and filter or aggregate in the outer query: WHERE a % 2 = 0 for the even terms, SUM(a) for the total. The recursion stays the same.

FOLLOW-UP 2 Which database?

PostgreSQL and MySQL 8 use WITH RECURSIVE; SQL Server drops the word RECURSIVE and may need a CAST so the column type is wide enough (bigint) in the anchor row; Oracle accepts WITH without RECURSIVE too. State the dialect assumption first.

FOLLOW-UP 3 Large n overflows.

Use a wider type (BIGINT, NUMERIC, or DECIMAL) in the anchor row so the column type is wide enough from the start; arbitrary-size values need a language with big integers. **Cost:** O(n) rows regardless of the type; a wider type only costs more bytes per row.

FOLLOW-UP 4 Use it from application code with a parameter.

Bind the limit as a query parameter (never build the SQL by string concatenation) and fetch the single column. The recursion lives inside the database, so there is one round trip regardless of n.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Write SQL code to publish the Fibonacci
# series.
#
# Write a SQL query that publishes the first N Fibonacci
```

```
# numbers (0, 1, 1, 2, 3, 5, ...), one per row and in
# order, where N is given as the parameter :limit. Use a
# recursive common table expression.
#
# Signature:
#   -- result: one column named fibonacci, N rows
#
# Examples:
# :limit = 8  ->  0, 1, 1, 2, 3, 5, 8, 13
# :limit = 1  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the even terms, or the sum of the first n terms.
# 2. Which database?
# 3. Large n overflows.
# 4. Use it from application code with a parameter.
#
# INTERVIEW NOTES
# A sequence that depends on the previous rows is a
# recursive CTE: one anchor row, one recursive step, and a
# stopping condition in the step. Carry the two previous
# values in each row (a, b) so the next row is (b, a + b);
# a row-per-term table is built in one statement with no
# procedural loop.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import sqlite3


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# a recursive query: each row is computed from the
```

```
# previous one
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""

# ---- main question: main function ----
# BASE QUESTION (optimal): Recursive CTE carrying (n, a,
# b)
# Idea: The anchor row is (1, 0, 1): the first term is 0
# and the next pair is (0, 1). Each recursive step
# produces (n + 1, b, a + b) while n is below the limit,
# so the term in column a is the Fibonacci number.
# FOLLOW-UP 4: Use it from application code with a
# parameter.
# Answer: Bind the limit as a query parameter (never build
# the SQL by string concatenation) and fetch the single
# column. The recursion lives inside the database, so
# there is one round trip regardless of n.
def fibonacci(count):
    # FOLLOW-UP 4: from app code: count is a parameter
    # one round trip: the recursion runs inside the
    # database;
    # the limit is a bound parameter, never glued into the
    # SQL
    with sqlite3.connect(":memory:") as db:
        rows = db.execute(FIB_SQL, (count,)).fetchall()
    return [r[0] for r in rows]


# CHECK in Python
def fibonacci_python(count):
    # the same sequence in Python, used to check the SQL
    a, b, out = 0, 1, []
    for _ in range(count):
        out.append(a)
        a, b = b, a + b
    return out

# ---- main question: tests ----
def test_main_question():
```

```
    got = fibonacci(10)
    assert got == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    assert fibonacci(10) == fibonacci_python(10)
    assert fibonacci(1) == [0]


# ============================================================
# FOLLOW-UP 1: Only the even terms, or the sum of the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the CTE and filter or aggregate in the
# outer query: WHERE a % 2 = 0 for the even terms, SUM(a)
# for the total. The recursion stays the same.
def even_terms(count):
    # follow-up: filter in an outer query
    sql = f"SELECT a FROM ({FIB_SQL}) WHERE a % 2 = 0"
    with sqlite3.connect(":memory:") as db:
        return [r[0] for r in db.execute(sql, (count,))]


# Answer: Keep the CTE and filter or aggregate in the
# outer query: WHERE a % 2 = 0 for the even terms, SUM(a)
# for the total. The recursion stays the same.
def total(count):
    # follow-up: the sum of the first n terms
    sql = "SELECT SUM(a) FROM (" + FIB_SQL + ")"
    with sqlite3.connect(":memory:") as db:
        return db.execute(sql, (count,)).fetchone()[0]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert even_terms(10) == [0, 2, 8, 34]
    assert total(10) == sum(fibonacci_python(10))
    # 80 terms stay exact in a 64-bit column
    assert fibonacci(80) == fibonacci_python(80)


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Use it from application code with a
```

```
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 159 of 288 | topic 19 of 43: Rolling state: the next state comes only from the previous one | 7 of 7

Template fit: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit) | asked 379 days ago | frequency Single report

HARD | ColorStack | technical interview | 159 of 288

Constrained Dice Roll Sequences

ASKED 2025-10-06 | 379 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 205/288

ABOUT THIS QUESTION

Topic: Rolling state: the next state comes only from the previous one

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1223 Dice Roll Simulation

Frequency: Single report | Last asked: 2025-10-06 | Roles: SWE Intern

QUESTION

You roll a single 6-faced die exactly n times. You are given an array roll_max of length 6, where roll_max[i] is the maximum number of times the value i + 1 may appear consecutively. That is, you may never roll the same value more than roll_max[i] times in a row. Return the number of distinct sequences of n rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo 10**9 + 7 .

Function signature

def count_dice_sequences(n: int, roll_max: list[int]) -> int:
    ...

Follow-up (as reported): The interviewer's actual twist used two dice per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

EXAMPLES

Example 1

Input	`n = 2, roll_max = [1, 1, 2, 2, 2, 3]`
Output	`34`
Why	`There are 6*6 = 36 sequences of length 2. Two of them are forbidden: rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.`

Example 2

Input	`n = 2, roll_max = [1, 1, 1, 1, 1, 1]`
Output	`30`
Why	`No value may repeat consecutively, so the 6 sequences "value v then v" are removed: 36 - 6 = 30.`

QUESTION

Example 3

Input	`n = 3, roll_max = [1, 1, 1, 2, 2, 3]`
Output	`181`
Why	`Hand-verified against the memoized DP over (rolls_used, last_value, run_length).`

CONSTRAINTS

- 1 <= n <= 5000 roll_max has exactly 6 entries, each 1 <= roll_max[i] <= 15 .
- A naive brute force over all 6**n sequences is far too slow; you must memoize over (rolls_used, last_value, consecutive_count_of_last_value) .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import groupby, product
def brute_dice(n, roll_max):
    count = 0
    for seq in product(range(6),_repeatn):
        if all(
            len(list(g)) <= roll_max[v]
            for v, g in groupby(seq)
        ):
            count += 1
    return count
```

TIME	SPACE
O(6^n * n)	O(n)

WHY IT IS SLOW

It generates all 6^n sequences and checks the runs of each.

OPTIMAL SOLUTION · DP over the last value and its run length

TIME	SPACE
O(n * 6 * R)	O(6 * R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each turn rolls a pair of dice together, and the limit applies to each die independently. How many joint sequences are there?

The two dice never influence each other, so the joint sequences are the pairs of one valid sequence per die: the answer is the single-die count squared (mod 10^9 + 7). Tracking both run lengths in one DP gives the same number with a much bigger state.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Constrained Dice Roll Sequences
#
# You roll a single 6-faced die exactly n times. You are
# given an array roll_max of length 6, where roll_max[i]
# is the maximum number of times the value i + 1 may
# appear consecutively. That is, you may never roll the
# same value more than roll_max[i] times in a row. Return
# the number of distinct sequences of n rolls that respect
# every consecutive-count limit. Because the answer can be
# very large, return it modulo 10**9 + 7 . Function
# signature: def count_dice_sequences(n: int, roll_max:
# list[int]) -> int: ... Follow-up (as reported): The
# interviewer's actual twist used two dice per turn (a
# pair rolled together, with the consecutive limit applied
# to each die independently). Extend the state to track
# the run lengths of both dice and count the joint
# sequences.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each turn rolls a pair of dice together, and the
# limit applies to each die independently. How many joint
# sequences are there?
#
# INTERVIEW NOTES
# Count sequences by what has to be remembered about the
# end of the prefix: the last value and how many times it
# was repeated in a row. dp[last][run] is the number of
# valid prefixes of the current length; rolling a
# different value resets the run to 1, rolling the same
# value extends it if the limit allows.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): DP over the last value and its
# run length
# Idea: dp[v][r] counts prefixes ending in value v
# repeated r times.
def count_dice_sequences(n, roll_max):
    mod = 10**9 + 7
    dp = [[0] * (roll_max[v] + 1) for v in range(6)]
    for v in range(6):
        dp[v][1] = 1
    for _ in range(n - 1):
        total = sum(sum(row) for row in dp) % mod
        new = [
            [0] * (roll_max[v] + 1) for v in range(6)
        ]
        for v in range(6):
            # a different previous value starts a run of
            # length one
            new[v][1] = (total - sum(dp[v])) % mod
            for r in range(1, roll_max[v]):
                new[v][r + 1] = dp[v][r]
        dp = new
    return sum(sum(row) for row in dp) % mod

# ---- main question: tests ----
def test_main_question():
    random.seed(47)
    assert (
        count_dice_sequences(2, [1, 1, 2, 2, 2, 3])
        == 34
    )
    assert (
        count_dice_sequences(2, [1, 1, 1, 1, 1, 1])
        == 30
    )
    assert (
        count_dice_sequences(3, [1, 1, 1, 2, 2, 3])
        == 181
    )
    assert (
        count_dice_sequences(1, [1, 1, 1, 1, 1, 1]) == 6
    )


# ============================================================
```

```
# FOLLOW-UP 1: Each turn rolls a pair of dice together,
# ============================================================

# ---- follow-up 1: function ----
# Answer: The two dice never influence each other, so the
# joint sequences are the pairs of one valid sequence per
# die: the answer is the single-die count squared (mod
# 10^9 + 7).
def count_two_dice(n, roll_max):
    one = count_dice_sequences(n, roll_max)
    return one * one % (10**9 + 7)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert (
        count_two_dice(2, [1, 1, 2, 2, 2, 3]) == 34 * 34
    )
    assert (
        0
        <= count_dice_sequences(500, [15] * 6)
        < 10**9 + 7
    )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 20 OF 43

Two pointers: move i and j toward each other (or along two lists)

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	3
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Two pointers: move i and j toward each other (or along two lists) | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

E	160. Reverse a linked list.	Exponent	no date
E	161. Sort a binary array in linear time.	Exponent	no date
E	162. Is this a valid palindrome?	Exponent	no date
E	163. Reverse a Sentence	Exponent	no date
M	164. Find median of merged RLE arrays	PracHub	2026-01-06
M	165. Run-Length Vector Storage + Dot Product	1Point3Acres	2026-01-21
M	166. Count Unique Triplets That Sum to Zero	ColorStack	2025-09-20

CODE TEMPLATE | GROUP 20 OF 43 | 7 QUESTIONS, ONE PATTERN

Two pointers: move i and j toward each other (or along two lists)

The one idea these questions share

Put one pointer at each end (or one at the start of each of two sorted lists). Look at the two items, decide which pointer moves, and stop when they meet. Most of these are O(n) because each pointer only moves one way.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def has_pair(nums, target):                    # nums sorted
    i, j = 0, len(nums) - 1
    while i < j:
        s = nums[i] + nums[j]                  # CHANGE 1: what you compute from the two items
        if s == target:
            return True
        if s < target:                         # CHANGE 2: which pointer moves
            i += 1
        else:
            j -= 1
    return False
```

2. Helper functions that usually get written next to it

```
def reverse_range(a, i, j):
    while i < j:
        a[i], a[j] = a[j], a[i]
        i, j = i + 1, j - 1


def walk_two(a, b):                            # merge order of two sorted lists
    i = j = 0
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            yield a[i]; i += 1
        else:
            yield b[j]; j += 1
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
160	Reverse a linked list.	three pointers prev, current, next: point current.next back at prev, then move all three one step
161	Sort a binary array in linear time.	left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them
162	Is this a valid palindrome?	compare the two ends; skip characters that are not letters or digits; stop when the pointers cross
163	Reverse a Sentence	swap from both ends to reverse the whole array, then reverse every word the same way
164	Find median of merged RLE arrays	walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position
165	Run-Length Vector Storage + Dot Product	one pointer per run list; multiply the values and take min(remaining counts) of the two current runs
166	Count Unique Triplets That Sum to Zero	sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 160 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 1 of 7

Template fit: three pointers prev, current, next: point current.next back at prev, then move all three one step | no date

EASY | IN APP LC 206 | Exponent | 160 of 288

Reverse a linked list.

ASKED (ANY CO.) ~4 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 59.8 | 30 DAYS 0 | PRIORITY 43/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #206)

LeetCode match: LC 206 Reverse Linked List

Last asked (any company): 4 months ago, about 2026-06 (Exponent shows only a rough age) | +921 interviews

Original: https://www.aced.io/questions/3247/reverse-linked-list

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

EXAMPLES

Example 1	
Input	`head = 1 -> 2 -> 3`
Output	`vals(reverse_list(head)) -> [3, 2, 1]`
Why	`1 -> 2 -> 3 becomes 3 -> 2 -> 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reverse_list(head):
    values = []
    node = head
    while node:
        values.append(node.val)
        node = node.next
    new_head = None
    for v in values:
        node = ListNode(v)
        node.next = new_head
        new_head = node  # push to front
    return new_head
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Time is O(n), but it allocates a whole new list, O(n) extra space. Flipping each node's next pointer in place with three variables needs O(1) space.

OPTIMAL SOLUTION · Flip the pointers in one pass

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it recursively.

Reverse the rest first, then make the next node point back at this one and cut this node's own pointer. An inner helper that carries prev (as coded) keeps it tail-recursive in spirit; the call stack is O(n), so mention the depth limit.

FOLLOW-UP 2 Reverse only positions left to right (LeetCode 92).

Walk to the node before the segment, reverse right - left + 1 nodes with the same loop, then reconnect: the old first node of the segment must point to what followed the segment, and the node before it points to the new first.

FOLLOW-UP 3 Reverse in groups of k and leave a short last group alone (LeetCode 25).

Check that k nodes exist before reversing a group, reverse them with the same loop, reconnect both ends, and move the group pointer to the old first node of the group.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reverse a linked list.
#
# Given the head of a singly linked list, reverse the list
# and return its head. Implement a function reverseList
# that takes the head of the linked list as input and
# returns the new head of the reversed list. Refer to the
# ListNode class/struct and do not comment it out. Example
# Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 ->
# 1Challenge 1Can you come up with a solution with a time
# complexity of O(n)?Challenge 2Can you come up with a
# solution with a space complexity of O(1)?
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it recursively.
# 2. Reverse only positions left to right (LeetCode 92).
# 3. Reverse in groups of k and leave a short last group
```

```
# alone (LeetCode 25).
#
# INTERVIEW NOTES
# Reversing means every next pointer has to flip, so hold
# three things: the node you are on, the one before it
# (what it will point to) and the one after it (so you do
# not lose the rest). Always save next before overwriting
# it. For 'reverse part of the list' remember the node
# before the part and the node that will end up last in
# it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Flip the pointers in one pass
# Idea: Walk with prev and cur. Save cur.next, point cur
# at prev, then move both forward. When cur is None, prev
# is the new head. Saving next before the overwrite is the
# step people forget.
```

```
def reverse_list(head):
    # point every node back at the one before it
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev

# ---- main question: tests ----
def test_main_question():
    assert to_list(reverse_list(make([1, 2, 3, 4]))) == [
        4, 3, 2, 1]
    assert reverse_list(None) is None


# ============================================================
# FOLLOW-UP 1: Do it recursively.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Reverse the rest first, then make the next node
# point back at this one and cut this node's own pointer.
def reverse_recursive(head, prev=None):
    # the call stack is O(n): mention the depth limit
    if head is None:
        return prev
    nxt, head.next = head.next, prev
    return reverse_recursive(nxt, head)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    got = to_list(reverse_recursive(make([1, 2, 3])))
    assert got == [3, 2, 1]


# ============================================================
# FOLLOW-UP 2: Reverse only positions left to right
# ============================================================

# ---- follow-up 2: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

# ---- follow-up 2: function ----
# Answer: Walk to the node before the segment, reverse
# right - left + 1 nodes with the same loop, then
# reconnect: the old first node of the segment must point
# to what followed the segment, and the node before it
# points to the new first.
def reverse_between(head, left, right):
    # LeetCode 92: reverse positions left..right (1-based)
    dummy = ListNode(0, head)
    before = dummy
    for _ in range(left - 1):
        before = before.next
    # reverse right - left + 1 nodes with the same loop
    prev, cur = None, before.next
    for _ in range(right - left + 1):
        cur.next, prev, cur = prev, cur, cur.next
    # reconnect: the old first node points past the
    # segment
    before.next.next = cur
    before.next = prev
    return dummy.next

# ---- follow-up 2: tests ----
def test_follow_up_2():
    part = reverse_between(make([1, 2, 3, 4, 5]), 2, 4)
    assert to_list(part) == [1, 4, 3, 2, 5]
    got = to_list(reverse_between(make([1, 2]), 1, 2))
    assert got == [2, 1]


# ============================================================
# FOLLOW-UP 3: Reverse in groups of k and leave a short
# ============================================================

# ---- follow-up 3: function ----
# Answer: Check that k nodes exist before reversing a
# group, reverse them with the same loop, reconnect both
# ends, and move the group pointer to the old first node
# of the group.
def reverse_k_group(head, k):
    # LeetCode 25: reverse each full group of k; a short
    # last
    # group is left alone
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        # is there a full group of k nodes?
```

```
        end = group_prev
        for _ in range(k):
            end = end.next
            if end is None:
                return dummy.next
        first, after = group_prev.next, end.next
        prev, cur = after, first
        while cur is not after:
            cur.next, prev, cur = prev, cur, cur.next
        group_prev.next = prev
        group_prev = first

# ---- follow-up 3: tests ----
def test_follow_up_3():
    grouped = reverse_k_group(make([1, 2, 3, 4, 5]), 2)
    assert to_list(grouped) == [2, 1, 4, 3, 5]
    got = to_list(reverse_k_group(make([1, 2, 3]), 3))
    assert got == [3, 2, 1]
    assert to_list(reverse_k_group(make([1]), 2)) == [1]
```

```
# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def make(values):
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example

```
Input: 1 -> 2 -> 3 -> 4
Output: 4 -> 3 -> 2 -> 1
```

OFFICIAL PAGE TEXT

Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?Our solution reverses the linked list by iterating through it while maintaining pointers to keep track of the previous node and the current node. For each node, it saves the next node, then reverses the link by pointing the current node's next to the previous node. It then moves the prev pointer to the current node and advances the current pointer to the saved next node. This process continues until the end of the list is reached. At the end, the prev pointer points to the new head of the reversed list, which is returned. The solution uses two pointers, prev and current, to achieve this reversal. This approach ensures that each node is processed exactly once, making it both efficient and straightforward. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the linked list. This is because we iterate through the list once, performing constant-time operations (pointer adjustments) for each node. Space Complexity: The solution uses O(1) space, as it only uses a fixed amount of extra space (three pointers: prev, current, and nextNode). No additional data structures are used, making it very space-efficient. NoteCheck out our lesson on the two pointer approach to learn more about what it is and how to use it!No answers

contributed by the community yet.

STUDY PATH: question 161 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 2 of 7

Template fit: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them | no date

EASY | Exponent | 161 of 288

Sort a binary array in linear time.

ASKED (ANY CO.) ~5 MONTHS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 53.6** | **30 DAYS 0** | **PRIORITY 74/288**

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #75 Sort Colors

LeetCode match: LC 75 Sort Colors (binary version) (inferred from short statement)

Last asked (any company): 5 months ago, about 2026-05 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/971/sort-binary-array-linear-time

QUESTION

Sort an array that contains only 0s and 1s in place, in one pass over the array (O(n) time) and with O(1) extra space.

Function signature

def sort_binary(arr: list) -> list

EXAMPLES

Example 1	
Input	`[1, 0, 1, 0, 0, 1]`
Output	`[0, 0, 0, 1, 1, 1]`
Why	`All the 0s move to the front.`

Example 2	
Input	`[0, 0, 1, 1]`
Output	`[0, 0, 1, 1]`
Why	`Already sorted arrays stay as they are.`

Example 3	
Input	`[1]`
Output	`[1]`
Why	`A single value.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr):
    return sorted(arr)  # O(n log n)
```

TIME	SPACE
O(n log n)	O(n)

BRUTE FORCE

WHY IT IS SLOW

A comparison sort is O(n log n) and needs a copy; with only two values two pointers (swap a 1 on the left with a 0 on the right) finish in one O(n) pass in place.

OPTIMAL SOLUTION · Two pointers swapping from both ends

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not just count zeros and fill?

Counting is also O(n) time and O(1) space and is fine for bare numbers. It needs two passes, and it cannot sort records that merely have a binary key, because it loses the objects. The swap version sorts the records themselves.

FOLLOW-UP 2 Count the zeros and rebuild (also O(n)).

Fine for bare numbers: one pass to count, one to fill. It cannot sort records that merely have a 0 or 1 key, because it loses the objects; the swapping version can.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sort a binary array in linear time.
#
# Sort an array that contains only 0s and 1s in place, in
# one pass over the array (O(n) time) and with O(1) extra
# space.
#
# Signature:
#   def sort_binary(arr: list) -> list
#
# Examples:
# [1, 0, 1, 0, 0, 1]  ->  [0, 0, 0, 1, 1, 1]
# [0, 0, 1, 1]  ->  [0, 0, 1, 1]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not just count zeros and fill?
# 2. Count the zeros and rebuild (also O(n)).
#
# INTERVIEW NOTES
# With only two values the sorted array is 'all zeros then
# all ones', so you only have to move zeros left and ones
# right. Two pointers from the two ends do it in one pass:
# swap whenever the left pointer sees a 1 and the right
# pointer a 0.

# ================================================
```

```
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two pointers swapping from both ends
# Idea: Keep lo at the next position for a 0 and hi at the next position for a 1. Advance lo past zeros, retreat hi past ones, and when both are stuck swap them. One pass, in place.
def sort_binary(arr):
    # two pointers: zeros collect on the left, ones on the right; every step swaps or moves one pointer: O(n)
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            # a 1 on the left and a 0 on the right: swap them
            arr[lo], arr[hi] = arr[hi], arr[lo]
            lo += 1
            hi -= 1
    return arr


# RECORDS with a 0/1 key
def sort_records(items, key):
    # records that merely have a 0/1 key: swapping keeps the
    # objects (counting would lose them). Not stable.
    lo, hi = 0, len(items) - 1
    while lo < hi:
        if key(items[lo]) == 0:
            lo += 1
        elif key(items[hi]) == 1:
            hi -= 1
        else:
            items[lo], items[hi] = items[hi], items[lo]
            lo += 1
            hi -= 1
    return items


# VARIANT: 0, 1 and 2 (Dutch flag)
def sort_three(arr):
    # the same idea for 0, 1, 2 (Dutch national flag):
    # [0, low) are 0, [high, n) are 2, the middle is 1
    low, mid, high = 0, 0, len(arr)
    while mid < high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 2:
            high -= 1
            arr[mid], arr[high] = arr[high], arr[mid]
        else:
            mid += 1
    return arr

# ---- main question: tests ----
def test_main_question():
    mixed = [1, 0, 1, 0, 1, 0]
    assert sort_binary(mixed) == [0, 0, 0, 1, 1, 1]
    assert sort_binary([]) == []
    assert sort_binary([1]) == [1]
    assert sort_binary([1, 1, 0]) == [0, 1, 1]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# ============================================================
# FOLLOW-UP 2: Count the zeros and rebuild (also O(n)).
# ============================================================

# ---- follow-up 2: function ----
# Answer: Fine for bare numbers: one pass to count, one to fill. It cannot sort records that merely have a 0 or 1 key, because it loses the objects; the swapping version can.
def sort_by_counting(arr):
    # follow-up: count the zeros, then rewrite the array.
    # Fine for bare numbers, useless for records.
    zeros = arr.count(0)
    return [0] * zeros + [1] * (len(arr) - zeros)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    for arr in ([0, 0], [1, 1], [1, 0], [0, 1, 0, 0, 1]):
        assert sort_binary(arr[:]) == sorted(arr)
        assert sort_by_counting(arr) == sorted(arr)
    people = [("ann", 1), ("bob", 0),
              ("cy", 1), ("di", 0)]
    done = sort_records(people[:], key=lambda p: p[1])
    keys = [p[1] for p in done]
    assert keys == [0, 0, 1, 1]
    # no record was lost: the same objects are still there
    assert sorted(done) == sorted(people)
    flag = sort_three([2, 0, 1, 2, 0, 1])
    assert flag == [0, 0, 1, 1, 2, 2]
    assert sort_three([2, 2, 0]) == [0, 2, 2]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 162 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 3 of 7

Template fit: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross | no date

EASY | IN APP LC 125 | Exponent | 162 of 288

Is this a valid palindrome?

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 51.4 | PRIORITY 26/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #125)

LeetCode match: LC 125 Valid Palindrome

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age) | +612 interviews

Original: https://www.aced.io/questions/4056/validate-palindrome-check

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input

once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

EXAMPLES

Example 1

Input	`'A man, a plan, a canal: Panama'`
Output	`true`
Why	`Ignoring punctuation, spaces and case it reads 'amanaplanacanalpanama'.`

QUESTION

Example 2

Input	`'race a car'`
Output	`false`
Why	`'raceacar' is not a palindrome.`

Example 3

Input	`' '`
Output	`true`
Why	`Nothing but a space: nothing to compare, so it is a palindrome.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(s):
    t = [c.lower() for c in s if c.isalnum()]
    return t == t[::-1]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but two extra copies of the string, O(n) space. Two pointers moving inward, skipping punctuation, need O(1) space and stop at the first mismatch.

OPTIMAL SOLUTION · Two pointers that skip ignored characters

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You may delete at most one character (LeetCode 680).

Move inward until the first mismatch, then the answer is whether skipping the left character or skipping the right character leaves a palindrome. That is two more linear checks, so O(n) overall.

FOLLOW-UP 2 Unicode, accents, or locale-specific case.

Normalise first (unicodedata.normalize and casefold) and decide what counts as a letter. str.isalnum handles many scripts, but combining marks need normalisation.

FOLLOW-UP 3 Longest palindromic substring.

Expand around each of the 2n - 1 centers in O(n^2), or use Manacher's algorithm in O(n). The two-pointer comparison is the building block.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Is this a valid palindrome?
#
# Given a string s, return true if it is a palindrome, and
# false otherwise. A string is considered a palindrome if,
# after converting all uppercase letters into lowercase
# letters and removing all non-alphanumeric characters, it
# reads the same forward and backward. Write a function
# isPalindrome that implements this logic. Examples Input:
# s = "H123!@321h" Output: true Input: s = "race a car"
# Output: false Constraints 1 <= s.length <= 2 * 10^5 The
# input string s can include any printable ASCII
# characters. Challenge 1Can you come up with a solution
# that achieves O(n) time complexity?Challenge 2Can you
# come up with a solution that achieves O(1) space
# complexity?Solution 1: Filtering and Reversing Approach
# This solution processes the input string by first
# filtering out non-alphanumeric characters and converting
# all remaining characters to lowercase.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You may delete at most one character (LeetCode 680).
# 2. Unicode, accents, or locale-specific case.
# 3. Longest palindromic substring.
#
# INTERVIEW NOTES
# A palindrome is symmetric, so compare the two ends and
# move inward; there is no need to build the reversed
# string. Decide what to ignore (non-alphanumerics, case)
# and skip those characters with the pointers instead of
# copying a cleaned string.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import unicodedata


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# FOLLOW-UP 2: Unicode, accents, or locale-specific case.
# Answer: Normalise first (unicodedata.normalize and
# casefold) and decide what counts as a letter.
# str.isalnum handles many scripts, but combining marks
# need normalisation.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def clean(s):
    # follow-up: unicode. Decompose accents, drop the
    # combining marks, fold case, keep letters and digits.
    text = unicodedata.normalize("NFKD", s)
    text = "".join(ch for ch in text
                   if not unicodedata.combining(ch))
    return [ch for ch in text.casefold() if ch.isalnum()]

# ---- main question: main function ----
# BASE QUESTION (optimal): Two pointers that skip ignored
# characters
# Idea: Move left and right toward each other, skipping
# characters that are not letters or digits, and compare
# the lowercase forms. The first mismatch ends it. Nothing
# is copied.
def is_palindrome(s):
    # compare the cleaned characters from both ends inward
    chars = clean(s)
    lo, hi = 0, len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            return False
        lo += 1
        hi -= 1
    return True


# SAME check without a cleaned copy
def is_palindrome_in_place(s):
    # the same without building a cleaned copy: two
    # pointers
    # on the original string skip non-alphanumeric
    # characters
    lo, hi = 0, len(s) - 1
    while lo < hi:
        if not s[lo].isalnum():
            lo += 1
        elif not s[hi].isalnum():
            hi -= 1
        elif s[lo].lower() != s[hi].lower():
            return False
        else:
            lo += 1
            hi -= 1
    return True

# ---- main question: tests ----
def test_main_question():
    assert is_palindrome('A man, a plan, a canal: Panama')
    assert not is_palindrome('race a car')
    assert is_palindrome(' ')


# ============================================================
# FOLLOW-UP 1: You may delete at most one character
# ============================================================

# ---- follow-up 1: function ----
# Answer: Move inward until the first mismatch, then the
# answer is whether skipping the left character or
# skipping the right character leaves a palindrome. That
# is two more linear checks, so O(n) overall.
def almost_palindrome(s):
    # follow-up (LeetCode 680): at most one deletion. At
    # the
    # first mismatch try skipping the left or the right
    # character; each is one more linear check.
    chars = clean(s)

    def straight(lo, hi):
        while lo < hi:
```

```
            if chars[lo] != chars[hi]:
                return False
            lo += 1
            hi -= 1
        return True

    lo, hi = 0, len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            skip_left = straight(lo + 1, hi)
            return skip_left or straight(lo, hi - 1)
```

```
        lo += 1
        hi -= 1
    return True

# ============================================================
# FOLLOW-UP 2: Unicode, accents, or locale-specific case.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Longest palindromic substring.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Expand around each of the 2n - 1 centers in
# O(n^2), or use Manacher's algorithm in O(n). The two-
# pointer comparison is the building block.
def longest_palindromic_substring(s):
    # follow-up: expand around each of the 2n - 1 centres
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples

```
Input: s = "H123!@321h"
Output: true

Input: s = "race a car"
Output: false
```

OFFICIAL PAGE TEXT

Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their reversed copy. This space is needed to hold the filtered string and its reversed version. Solution 2: Two-Pointer Approach This solution uses a two-pointer technique to efficiently check if the string is a palindrome. The idea is to use one pointer starting from the beginning of the string (left) and another pointer from the end (right).

The pointers move towards each other, skipping non-alphanumeric characters and comparing characters that are alphanumeric. Pointer Initialization: Set two pointers: left at the beginning and right at the end of the string. Skipping Non-Alphanumeric Characters: Move the left pointer to the right while skipping non-alphanumeric characters, and similarly move the right pointer to the left. Comparison: Compare the characters at left and right in a case-insensitive manner. If characters do not match, the string is not a palindrome. Update Pointers: Move the pointers closer to each other and repeat until the pointers meet or cross. Time Complexity: The time complexity is O(n), where n is the length of the string. Each character is processed at most twice (once by each pointer). Space Complexity: The space complexity is O(1) as it only uses a constant amount of extra space for the two pointers, regardless of the size of the input string. NoteCheck out our lesson on the two-pointer approach to learn more about it and its applications!No answers contributed by the community yet.

STUDY PATH: question 163 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 4 of 7

Template fit: swap from both ends to reverse the whole array, then reverse every word the same way | no date

EASY | IN APP LC 186 | Exponent | 163 of 288

Reverse a Sentence

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 231/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #186)

LeetCode match: LC 186 Reverse Words in a String II

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3062/reverse-a-sentence

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

EXAMPLES

Example 1

Input	`arr = ['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']`
Output	`''.join(reverse_words(list('the sky is blue'))) -> 'blue is sky the'`
Why	`The words come out in reverse order while each word keeps its letters in order (the array is reversed in place).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reverse_words(arr):
    sentence = "".join(arr)
    words = sentence.split()
    return list(" ".join(reversed(words)))
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but extra space for the word list, and it collapses repeated spaces. Reversing the whole character array and then each word in place needs O(1) extra space.

OPTIMAL SOLUTION · Reverse the whole array, then reverse each word in place

TIME	SPACE
O(n)	O(1) extra on the character array

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated spaces.

The split-based version already normalises them; the in-place version keeps them, so say which you want. To normalise in place add a two-pointer pass that writes single spaces.

FOLLOW-UP 2 Reverse the letters of each word but keep the word order (LeetCode 557).

Split on single spaces and reverse each word, or reverse each run in place without the first full reversal.

FOLLOW-UP 3 The sentence is a linked list of characters.

Reverse each word segment, then reverse the whole list, using the pointer-flip routine for linked lists.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reverse a Sentence
#
# You are given an array of characters arrthat consists of
# sequences of characters separated by space characters.
# Each space-delimited sequence of characters defines a
```

```
# word. Implement a function reverseWords that reverses
# the order of the words in the array in the most
# efficient manner. Explain your solution and analyze its
# time and space complexities. Example input: arr = [ 'p',
# 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e',
# 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e' ]
# output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ',
# 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e',
# 'c', 't' ]Hint 1Even if your language of choice enables
# this, don't be tempted to use standard functions.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# spaces.
# 2. Reverse the letters of each word but keep the word
# order (LeetCode 557).
# 3. The sentence is a linked list of characters.
#
# INTERVIEW NOTES
# Reversing the order of words but not the letters is the
# same as reversing everything and then reversing each
# word back. That is two passes of the same helper and
# needs no extra memory when the sentence is a mutable
# character array. First decide how to treat repeated
# spaces.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1


def each_word(arr):
    # yield the (start, end) of every word: a run of
    # characters without a space
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        yield i, j
        i = j + 1

# ---- main question: main function ----
# BASE QUESTION (optimal): Reverse the whole array, then
# reverse each word in place
# Idea: Reverse every character; each word is now
# backwards but in the right place. Walk the array and
# reverse each maximal run of non-space characters.
# Everything happens inside the same array.
```

```
def reverse_words(arr):
    # reverse the whole array, so the words are in the
    # right
    # order but spelled backwards; then reverse each word
    # back. O(n) time and O(1) extra space.
    reverse_range(arr, 0, len(arr) - 1)
    for start, end in list(each_word(arr)):
        reverse_range(arr, start, end)
    return arr

# ---- main question: tests ----
def test_main_question():
    text = list("perfect makes practice")
    flipped = "".join(reverse_words(text))
    assert flipped == "practice makes perfect"
```

```
    assert reverse_words([]) == []
    assert reverse_words(list("one")) == list("one")
    # spaces stay where they are in the in-place version
    spaced = reverse_words(list(" a  bc "))
    assert "".join(spaced) == " bc  a "


# ============================================================
# FOLLOW-UP 1: The input may have leading, trailing or
# ============================================================

# ---- follow-up 1: function ----
# Answer: The split-based version already normalises them;
# the in-place version keeps them, so say which you want.
# To normalise in place add a two-pointer pass that writes
# single spaces.
def normalise_spaces(arr):
    # follow-up: leading, trailing and repeated spaces. A
    # second pointer writes the words with single spaces;
    # the array is cut to the new length.
    write = 0
    for read in range(len(arr)):
        if arr[read] != " ":
            arr[write] = arr[read]
            write += 1
        elif write and arr[write - 1] != " ":
            arr[write] = " "
            write += 1
    if write and arr[write - 1] == " ":
        write -= 1
    del arr[write:]
    return arr

# ============================================================
# FOLLOW-UP 2: Reverse the letters of each word but keep
# ============================================================

# ---- follow-up 2: function ----
# Answer: Split on single spaces and reverse each word, or
# reverse each run in place without the first full
# reversal.
def reverse_each_word(arr):
    # follow-up (LeetCode 557): keep the word order,
    # reverse
    # the letters of every word
```

```
    for start, end in list(each_word(arr)):
        reverse_range(arr, start, end)
    return arr

# ---- follow-up 2: tests ----
def test_follow_up_2():
    letters = reverse_each_word(list("let's code"))
    assert "".join(letters) == "s'tel edoc"
    neat = normalise_spaces(list("  a   bc  "))
    assert "".join(neat) == "a bc"
    assert normalise_spaces(list("   ")) == []
```

```
    assert normalise_spaces(list("x")) == ["x"]
    messy = list("  one   two three ")
    tidy = reverse_words(normalise_spaces(messy))
    assert "".join(tidy) == "three two one"
    for _ in range(300):
        s = "".join(random.choice("ab c")
                    for _ in range(random.randint(0, 12)))
        parts = s.split(" ")
        got = "".join(reverse_words(list(s)))
        assert got == " ".join(reversed(parts))


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example

```
input:  arr = [ 'p', 'e', 'r', 'f', 'e', 'c', 't', '
',
               'm', 'a', 'k', 'e', 's', '  ',
               'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'
 ]

output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', '  '
,
          'm', 'a', 'k', 'e', 's', '  ',
          'p', 'e', 'r', 'f', 'e', 'c', 't' ]
```

OFFICIAL PAGE TEXT

Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible solution is doing a linear iteration on arr while pushing a copy of every word to a stack and then pulling them in reverse order while copying the words back into arr. Another approach is initializing a new array at the same length, iterating over arr from right to left, and copying any sequence of characters to the new array from left to right. Both approaches take O(N) time and at least O(N) space. A more elegant and efficient approach is to reverse all the characters in arr and then reverse the

characters in each word separately. While the first reverse gives us the words in the reverse order as we wanted, it also reverses the characters of each word. To fix that, we do the second reverse, which reverses each word separately. Reversing items in an array is done by a 'mirror' function, that swaps the items of every 2 indices with the same distance from the middle. Pseudocode:

```
function reverseWords(arr):
    # reverse all characters:
    n = arr.length
    mirrorReverse(arr, 0, n-1)

    # reverse each word:
    wordStart = null
    for i from 0 to n-1:
        if (arr[i] == ' '):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i-1)
                wordStart = null
        else if (i == n-1):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i)
        else:
            if (wordStart == null):
                wordStart = i

    return arr

# helper function - reverses the order of items in arr
# please note that this is language dependent:
# if are arrays sent by value, reversing should be don
e in place

function mirrorReverse(arr, start, end):
    tmp = null
    while (start < end):
        tmp = arr[start]
```

OFFICIAL PAGE TEXT

```
        arr[start] = arr[end]
        arr[end] = tmp
        start++
        end--
```

Time Complexity: traversing the array twice with a constant number of actions for each item is linear O(N). Space Complexity: using iteration indices and one temp variable takes constant O(1) memory. Answer to bonus question: The mirrorReverse function could be implemented with a single index with left to right linear iteration and swapping arr[i] and arr[n-1-i] as long as i < n-1-i. No answers contributed by the community yet.Related questionsReverse a linked list.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScriptC#JavaC++CGoRubySwiftPythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY PATH: question 164 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 5 of 7

Template fit: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position | asked 287 days ago

MEDIUM | PracHub | Technical Screen | 164 of 288

★ TOP 75: rank 2 in the Medium and Hard priority list

★ Find median of merged RLE arrays

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 5/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #4 Median of Two Sorted Arrays

LeetCode match: LC 4 Median of Two Sorted Arrays (RLE variant) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-median-of-merged-rle-arrays

QUESTION

Two sorted arrays are stored run-length encoded as (value, count) pairs in increasing order of value. Return the median of all their elements together, without expanding the runs (a count can be huge).

Function signature

def median_rle(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	`[(1, 2), (5, 1)], [(3, 3)]`
Output	`3`
Why	`Combined sequence 1, 1, 3, 3, 3, 5: the two middle elements are 3 and 3.`

Example 2	
Input	`[(1, 1)], [(2, 1), (9, 1)]`
Output	`2`
Why	`Combined sequence 1, 2, 9: the middle one is 2.`

Example 3	
Input	`[(1, 1000000)], [(5, 1000000)]`
Output	`3`
Why	`Counts may be enormous: a million 1s and a million 5s have median 3.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(runs_a, runs_b):
    dense = []
    for value, count in runs_a + runs_b:
        dense += [value] * count
    dense.sort()
    n = len(dense)
    if n % 2:
        return dense[n // 2]
    mid = n // 2
    return (dense[mid - 1] + dense[mid]) / 2
```

TIME	SPACE
O(total count)	O(total count)

WHY IT IS SLOW

Needs time and memory proportional to the total count, which can be 10^9 or more. Walking the two run lists in value order and counting up to the middle rank costs only the number of runs.

OPTIMAL SOLUTION · Merge the runs and walk to the two middle ranks

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 K arrays instead of two.

Merge all runs by value (heap of K heads, or sort the combined run list if memory allows) and walk to the two middle ranks. O(R log K) with a heap, R the total number of runs.

FOLLOW-UP 2 The arrays are huge and sorted, and you want O(log) time.

Binary search on the value: for a candidate value count how many elements are at most it in each array (binary search on prefix sums of counts) and find the smallest value with rank above the middle. O(log(range) * log(runs)).

FOLLOW-UP 3 Runs are not sorted by value.

Sort the runs by value first, O(R log R), then use the same walk.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find median of merged RLE arrays
#
# Two sorted arrays are stored run-length encoded as
# (value, count) pairs in increasing order of value.
# Return the median of all their elements together,
# without expanding the runs (a count can be huge).
#
# Signature:
#   def median_rle(a: list, b: list) -> float
#
# Examples:
# [(1, 2), (5, 1)], [(3, 3)]  ->  3
# [(1, 1)], [(2, 1), (9, 1)]  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. K arrays instead of two.
# 2. The arrays are huge and sorted, and you want O(log)
# time.
# 3. Runs are not sorted by value.
#
# INTERVIEW NOTES
# Counts can be huge, so never expand a run. You only need
# the elements at ranks (N - 1) // 2 and N // 2 in the
# merged order. Merge the two run lists by value, counting
# how many elements you have passed, and read the value
# when the count reaches each rank.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from bisect import bisect_right
from itertools import accumulate


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def kth_smallest(runs_a, runs_b, k):
```

```
    # zero-based rank in the merged order, walking the
    # runs
    # of two sorted run-length lists; nothing is expanded
    i = j = seen = 0
    while True:
        take_a = j >= len(runs_b)
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        value, count = runs_a[i] if take_a else runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1

# ---- main question: main function ----
# BASE QUESTION (optimal): Merge the runs and walk to the
# two middle ranks
# Idea: Let N be the sum of all counts.
def median_rle(a, b):
    # a and b: [(value, count)] sorted by value
    total = sum(c for _, c in a) + sum(c for _, c in b)
    if total % 2:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2

# ---- main question: tests ----
def test_main_question():
    a = [(1, 3), (4, 1)]
    b = [(2, 2), (5, 2)]
    # merged: 1 1 1 2 2 4 5 5 -> middle two are 2 and 2
    assert median_rle(a, b) == 2


# ============================================================
# FOLLOW-UP 1: K arrays instead of two.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Merge all runs by value (heap of K heads, or
# sort the combined run list if memory allows) and walk to
```

```
# the two middle ranks. O(R log K) with a heap, R the
# total number of runs.
def median_k_arrays(arrays):
    # K arrays: merge the run lists by value, then walk to
    # the
    # middle ranks
    runs = list(heapq.merge(*arrays))
    total = sum(c for _, c in runs)

    def kth(k):
        seen = 0
        for value, count in runs:
            seen += count
            if seen > k:
                return value

    if total % 2:
        return kth(total // 2)
    return (kth(total // 2 - 1) + kth(total // 2)) / 2

# ---- follow-up 1: tests ----
def test_follow_up_1():
    a = [(1, 3), (4, 1)]
    b = [(2, 2), (5, 2)]
    assert median_k_arrays([a, b]) == 2


# ============================================================
# FOLLOW-UP 2: The arrays are huge and sorted, and you
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search on the value: for a candidate
# value count how many elements are at most it in each
# array (binary search on prefix sums of counts) and find
# the smallest value with rank above the middle.
# O(log(range) * log(runs)).
def median_by_value(arrays):
    # sorted arrays, whole-number values: binary search
    # the
    # value; count_at_most uses prefix sums of the counts
```

```
    prefixes = []
    for runs in arrays:
        values = [v for v, _ in runs]
        sums = list(accumulate(c for _, c in runs))
        prefixes.append((values, sums))

    def count_at_most(x):
        total = 0
        for values, sums in prefixes:
            i = bisect_right(values, x)
            total += sums[i - 1] if i else 0
        return total

    n = count_at_most(10 ** 18)

    def kth(k):
        lo, hi = -(10 ** 18), 10 ** 18
        while lo < hi:
            mid = (lo + hi) // 2
            # smallest value with more than k elements <=
            # it
            if count_at_most(mid) > k:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if n % 2:
        return kth(n // 2)
    return (kth(n // 2 - 1) + kth(n // 2)) / 2

# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [(1, 3), (4, 1)]
    b = [(2, 2), (5, 2)]
    assert median_by_value([a, b]) == 2
    # an odd total: 1, 7, 7, 7, 7, 7 plus one more 7
    assert median_rle([(7, 6)], [(1, 1)]) == 7
    # huge counts are never expanded
    big = [(1, 10 ** 12)]
    assert median_rle(big, [(3, 10 ** 12)]) == 2
    c = [(0, 1), (10, 1)]
    three = [a, b, c]
    by_merge = median_k_arrays(three)
    assert by_merge == median_by_value(three)


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution([(1, 2), (5, 1)], [(2, 3)])
```

Test cases

Input	Expected
`([(1, 2), (5, 1)], [(2, 3)])`	`2.0`
`([(1, 1)], [(2, 1)])`	`1.5`

STUDY PATH: question 165 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 6 of 7

Template fit: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs | asked 272 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 165 of 288

Run-Length Vector Storage + Dot Product

ASKED 2026-01-21 | 272 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 197/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

QUESTION

Two vectors of the same length are stored run-length encoded as (value, count) pairs. Compute their dot product without expanding them to full arrays.

Function signature

def rle_dot(a: list, b: list) -> int

EXAMPLES

Example 1	
Input	`[(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]`
Output	`25`
Why	`A = [3,3,3,0,0,0,4,4], B = [1,1,1,1,2,2,2,2]: 3*1*3 + 0 + 4*2*2 = 9 + 16 = 25.`

Example 2	
Input	`[(2, 1)], [(5, 1)]`
Output	`10`
Why	`One element each: 2 * 5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def decode(runs):
    dense = []
    for value, count in runs:
        dense += [value] * count
    return dense
def brute(runs_a, runs_b):
    a, b = decode(runs_a), decode(runs_b)
    return sum(x * y for x, y in zip(a, b))
```

BRUTE FORCE

TIME	SPACE
O(logical length)	O(logical length)

WHY IT IS SLOW

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

OPTIMAL SOLUTION · Two-pointer over runs

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

FOLLOW-UP 2 How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

FOLLOW-UP 3 One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Run-Length Vector Storage + Dot Product
#
# Two vectors of the same length are stored run-length
# encoded as (value, count) pairs. Compute their dot
# product without expanding them to full arrays.
#
# Signature:
#   def rle_dot(a: list, b: list) -> int
#
```

```
# Examples:
# [(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]  ->  25
# [(2, 1)], [(5, 1)]  ->  10
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if one vector is mostly zeros and stored
# sparsely, and the other as runs?
# 2. How should the data be updated if you set element i?
# 3. One vector is a sparse map (index to value) and the
# other is run-length encoded.
#
# INTERVIEW NOTES
# A run is a stretch where both numbers are constant, so
# the dot product of two runs is value times value times
# the length of their overlap. Walk the two run lists like
# the merge step of merge sort: consume the smaller
# remaining length from both, and advance whichever run
# you finished.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_right
from itertools import accumulate


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Two-pointer over runs
```

```
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both).
def rle_dot(a, b):
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total

# ---- main question: tests ----
def test_main_question():
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    # 3*1*3 + 0 + 4*2*2 = 25
```

```
    assert rle_dot(ra, rb) == 25
    assert rle_dot([], rb) == 0
    dense = sum(x * y for x, y in zip(A, B))
    assert dense == 25
    # sparse vector with non-zeros at 0 and 7
    sparse = {0: 2, 7: 5}


# ============================================================
# FOLLOW-UP 1: What if one vector is mostly zeros and
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Iterate the sparse side's nonzero indices and
# binary-search the run list for each index to read the
# other value: O(s log r). Choose the smaller
# representation to drive the loop.
# FOLLOW-UP 3: One vector is a sparse map (index to value)
# and the other is run-length encoded.
# Answer: Iterate the smaller structure (the sparse non-
# zeros) and binary search the run list for each index
# using prefix run lengths: O(s log r) instead of
# expanding anything.
def sparse_dot_rle(sparse, runs):
    # sparse: {index: value}; runs: RLE of the other
    # vector
    ends = list(accumulate(count for _, count in runs))
    total = 0
    # iterate the small side; binary search the runs
    for index, value in sparse.items():
        k = bisect_right(ends, index)
        if k < len(runs):
            total += value * runs[k][0]
    return total

# ---- follow-up 1: tests ----
def test_follow_up_1():
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    # sparse vector with non-zeros at 0 and 7
    sparse = {0: 2, 7: 5}
    assert sparse_dot_rle(sparse, rb) == 2 * 1 + 5 * 2


# ============================================================
# FOLLOW-UP 2: How should the data be updated if you set
# ============================================================

# ---- follow-up 2: function ----
# Answer: Locate the run containing i (binary search over
# prefix run lengths), then split it into at most three
# runs and merge equal neighbours.
def set_at(runs, index, value):
    # update one position: split a run, then merge
    # neighbours
    flat, pos = [], 0
```

```
    for v, count in runs:
        if pos <= index < pos + count:
            flat += [(v, index - pos), (value, 1),
                     (v, pos + count - index - 1)]
        else:
            flat.append((v, count))
        pos += count
    merged = []
    for v, count in flat:
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
            merged.append((v, count))
```

```
    return merged

# ---- follow-up 2: tests ----
def test_follow_up_2():
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    # setting a value splits and merges runs
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1),
                                (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]


# ============================================================
# FOLLOW-UP 3: One vector is a sparse map (index to value)
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def rle_encode(values):
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1][1] += 1
        else:
            runs.append([v, 1])
    return [tuple(r) for r in runs]
```

```
def rle_decode(runs):
    return [v for v, count in runs for _ in range(count)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

STUDY PATH: question 166 of 288 | topic 20 of 43: Two pointers: move i and j toward each other (or along two lists) | 7 of 7

Template fit: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once | asked 395 days ago | frequency Single report

MEDIUM | IN APP LC 15 | ColorStack | technical interview | 166 of 288

Count Unique Triplets That Sum to Zero

ASKED 2025-09-20 | 395 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 218/288

ABOUT THIS QUESTION

Topic: Two pointers: move i and j toward each other (or along two lists)

Optimal answer from: your app's interview approach (LeetCode #15)

LeetCode match: LC 15 3Sum (count of unique triplets)

Frequency: Single report | Last asked: 2025-09-20 | Roles: SWE Intern

QUESTION

Given an integer array nums , count how many unique triplets (nums[i], nums[j], nums[k]) with three distinct indices i < j < k satisfy nums[i] + nums[j] + nums[k] == 0 . Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Function signature

```
def count_zero_sum_triplets(nums: list[int]) -> int:
    ...
```

Follow-up (as reported): Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

EXAMPLES

Example 1

Input	`nums = [-1, 0, 1, 2, -1, -4]`
Output	`2`
Why	`The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1]. Even though [-1, 0, 1] can be formed with different index sets, it is counted once.`

Example 2

Input	`nums = [0, 0, 0, 0]`
Output	`1`
Why	`The only triplet is [0, 0, 0].`

Example 3

Input	`nums = [1, 2, 3]`
Output	`0`
Why	`No three elements sum to 0.`

CONSTRAINTS

QUESTION

- 0 <= len(nums) <= 3000
- -10^5 <= nums[i] <= 10^5
- The array may contain duplicates.
- If len(nums) < 3 , the answer is 0 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_triplets(nums):
    found = set()
    for i in range(len(nums)):
        for j in range(i_+_1,_len(nums)):
            for k in range(j_+_1,_len(nums)):
                if (
                    nums[i] + nums[j] + nums[k]
                    == 0
                ):
                    found.add(
                        tuple(
                            sorted(
                                (
                                    nums[i],
                                    nums[j],
                                    nums[k],
                                )
                            )
                        )
                    )
    return len(found)
```

TIME	SPACE
O(n^3)	O(triplets)

WHY IT IS SLOW

Tries all index triples and stores the sorted value triple to remove repeats.

OPTIMAL SOLUTION · Sort and two pointers

TIME	SPACE
O(n^2)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the unique triplets themselves.

The same sweep, appending [nums[i], nums[left], nums[right]] on every hit instead of counting. The output size can be O(n^2), which dominates the cost.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Unique Triplets That Sum to Zero
#
# Given an integer array nums , count how many unique
# triplets (nums[i], nums[j], nums[k]) with three distinct
# indices i < j < k satisfy nums[i] + nums[j] + nums[k] ==
# 0 . Two triplets are considered the same if they consist
# of the same multiset of values, regardless of the
# indices they came from. Return the number of distinct
# value-triplets (not the number of index combinations).
# Function signature: def count_zero_sum_triplets(nums:
# list[int]) -> int: ... Follow-up (as reported): Return
# the actual list of unique triplets instead of the count.
# Hint: sort the array, fix one element, and use two
# pointers while skipping duplicate values.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the unique triplets themselves.
#
# INTERVIEW NOTES
# Unique value triplets means duplicates must be skipped.
# Sort, fix the first element, and run two pointers on the
# rest: the sum is too small, move left up; too large,
# move right down. Skipping equal values for all three
# positions keeps each triplet counted once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Sort and two pointers
# Idea: Fix the smallest value and sweep the other two
# inward, skipping duplicates.
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
```

```
    count = 0
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
```

```
                right -= 1
                while (
                    left < right
                    and nums[left] == nums[left - 1]
                ):
                    left += 1
    return count

# ---- main question: tests ----
def test_main_question():
    random.seed(15)
    assert (
        count_zero_sum_triplets([-1, 0, 1, 2, -1, -4])
        == 2
    )
    assert count_zero_sum_triplets([0, 0, 0, 0]) == 1
    assert count_zero_sum_triplets([1, 2, 3]) == 0


# ============================================================
# FOLLOW-UP 1: Return the unique triplets themselves.
# ============================================================

# ---- follow-up 1: function ----
# Answer: The same sweep, appending [nums[i], nums[left],
# nums[right]] on every hit instead of counting. The
# output size can be O(n^2), which dominates the cost.
def zero_sum_triplets(nums):
    nums = sorted(nums)
    out = []
    for i in range(len(nums) - 2):
```

```
        if i and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                out.append(
                    [nums[i], nums[left], nums[right]]
                )
                left += 1
                right -= 1
                while (
                    left < right
                    and nums[left] == nums[left - 1]
                ):
                    left += 1
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 21 OF 43

Walk two strings at once with two indices

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	3	1
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Walk two strings at once with two indices | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

E	167. Find the First Occurrence of a Substring	ColorStack	2025-09-16
E	168. Determine if two words are sorted in lexicographic order.	Exponent	no date
E	169. Return all unique substrings of a given string.	Exponent	no date
M	170. Can board states be transformed?	PracHub	2026-04-01
M	171. Decide string transform with directional tokens and walls	PracHub	2025-07-15
M	172. Determine if two sentences are similar.	Exponent	no date
H	173. Find Dictionary Matches for a Jammed String	PracHub	2026-04-09

CODE TEMPLATE | GROUP 21 OF 43 | 7 QUESTIONS, ONE PATTERN

Walk two strings at once with two indices

The one idea these questions share

Keep i on one string and j on the other. Skip characters that do not matter, compare the rest, and decide what to do at the first difference. Substring search, ordering, similarity and move-based transforms are all this loop.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def first_difference(a, b, skip="."):
    i = j = 0
    while i < len(a) and j < len(b):
        while i < len(a) and a[i] == skip:      # CHANGE 1: skip characters that do not matter
            i += 1
        while j < len(b) and b[j] == skip:
            j += 1
        if i == len(a) or j == len(b):
            break
        if a[i] != b[j]:                        # CHANGE 2: the rule when they differ
            return i, j
        i, j = i + 1, j + 1
    return None
```

2. Helper functions that usually get written next to it

```
def runs(s):                                    # run-length encoding: "aaab" -> [("a", 3), ("b", 1)]
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return [(c, n) for c, n in out]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
167	Find the First Occurrence of a Substring	try every start i; inside, j walks the needle; a full match returns i
168	Determine if two words are sorted in lexicographic order.	walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter
169	Return all unique substrings of a given string.	sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes
170	Can board states be transformed?	skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right
171	Decide string transform with directional tokens and walls	the same walk with dots as blanks and the movement rule from the statement (walls block)
172	Determine if two sentences are similar.	zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)
173	Find Dictionary Matches for a Jammed String	compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 167 of 288 | topic 21 of 43: Walk two strings at once with two indices | 1 of 7

Template fit: try every start i; inside, j walks the needle; a full match returns i | asked 399 days ago | frequency Single report

EASY | IN APP LC 28 | ColorStack | technical interview | 167 of 288

Find the First Occurrence of a Substring

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 233/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: your app's interview approach (LeetCode #28)

LeetCode match: LC 28 Find the Index of the First Occurrence in a String

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Reconstructing the "find substring in array" item as substring search: given two strings haystack and needle , return the index of the first occurrence of needle in haystack , or -1 if needle is not part of haystack .

Function signature

def str_str(haystack: str, needle: str) -> int:
"""Return the index of the first occurrence of needle in haystack, or -1."""
...

Follow-up (as reported): Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

EXAMPLES

Example 1

Input	`haystack = "sadbutsad", needle = "sad"`
Output	`0`
Why	`"sad" occurs at indices 0 and 6; the first is index 0.`

Example 2

Input	`haystack = "leetcode", needle = "leeto"`
Output	`-1`
Why	`"leeto" does not occur in "leetcode".`

Example 3

Input	`haystack = "abc", needle = "c"`
Output	`2`

CONSTRAINTS

- 1 <= len(haystack), len(needle) <= 10^4 haystack and needle consist of only lowercase English letters.

QUESTION

- If needle is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_str_str(haystack, needle):
    n, m = len(haystack), len(needle)
    for i in range(n_-_m_+_1):
        if haystack[i:
            i + m] == needle:
            return i
    return -1
```

TIME	SPACE
O(n * m)	O(1)

WHY IT IS SLOW

Each of the n - m + 1 starts may compare up to m characters.

OPTIMAL SOLUTION · Naive scan with early exit

TIME	SPACE
O(n * m) worst case	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it O(n + m) in the worst case.

Knuth-Morris-Pratt: table[i] is the length of the longest proper prefix of needle[:i + 1] that is also a suffix. While scanning the haystack, on a mismatch fall back to table[j - 1] instead of restarting, so the haystack pointer only moves forward.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the First Occurrence of a Substring
#
# Reconstructing the "find substring in array" item as
# substring search: given two strings haystack and needle
# , return the index of the first occurrence of needle in
# haystack , or -1 if needle is not part of haystack .
# Function signature: def str_str(haystack: str, needle:
# str) -> int: """Return the index of the first occurrence
# of needle in haystack, or -1.""" ... Follow-up (as
# reported): Achieve O(n + m) worst-case time with the
# Knuth-Morris-Pratt (KMP) failure-function approach
# instead of the naive O(n * m) scan.
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. Make it O(n + m) in the worst case.
#
# INTERVIEW NOTES
# Substring search compares the needle at every start. The
# naive scan restarts after a mismatch; KMP precomputes,
# for each prefix of the needle, the longest proper prefix
# that is also a suffix, so after a mismatch the haystack
# pointer never moves back.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Naive scan with early exit
# Idea: Compare character by character and stop at the
# first mismatch.
def str_str(haystack, needle):
    n, m = len(haystack), len(needle)
    for i in range(n - m + 1):
        j = 0
```

```
        while j < m and haystack[i + j] == needle[j]:
            j += 1
        if j == m:
            return i
    return -1

# ---- main question: tests ----
def test_main_question():
    random.seed(10)
    assert str_str("sadbutsad", "sad") == 0
    assert str_str("leetcode", "leeto") == -1
    assert str_str("abc", "c") == 2
    assert str_str("abc", "") == 0


# ============================================================
# FOLLOW-UP 1: Make it O(n + m) in the worst case.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Knuth-Morris-Pratt: table[i] is the length of
# the longest proper prefix of needle[:i + 1] that is also
# a suffix.
def str_str_kmp(haystack, needle):
    if not needle:
        return 0
    table, j = [0] * len(needle), 0
    for i in range(1, len(needle)):
        while j and needle[i] != needle[j]:
            j = table[j - 1]
        if needle[i] == needle[j]:
            j += 1
        table[i] = j
    j = 0
    for i, ch in enumerate(haystack):
        while j and ch != needle[j]:
            j = table[j - 1]
        if ch == needle[j]:
```

```
            j += 1
        if j == len(needle):
            return i - j + 1
    return -1


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 168 of 288 | topic 21 of 43: Walk two strings at once with two indices | 2 of 7

Template fit: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter | no date

EASY | IN APP LC 953 | Exponent | 168 of 288

Determine if two words are sorted in lexicographic order.

ASKED (ANY CO.) ~3 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 192/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: your app's interview approach (LeetCode #953)

LeetCode match: LC 953 Verifying an Alien Dictionary (inferred from short statement)

Last asked (any company): 3 years ago, about 2023-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/3431/check-lexicographic-order-two-words

QUESTION

Words are ordered using a custom alphabet: rank maps each letter to its position in that alphabet. Return whether word a comes before word b (or is equal to it). The first letter where the words differ decides; if one word is a prefix of the other, the shorter one comes first.

Function signature

def in_order(a: str, b: str, rank: dict) -> bool

EXAMPLES

Example 1

Input	`alphabet = 'hlabcdefgijkmnopqrstuvwxyz' # 'h' comes first, then 'l', then a, b, c ...`
	`rank = {c: i for i, c in enumerate(alphabet)}`
	`in_order('hello', 'leetcode', rank)`
	`in_order('word', 'world', rank)`
	`in_order('app', 'apple', rank)`

Output	`in_order('hello', 'leetcode', rank) -> true`
	`in_order('word', 'world', rank) -> false`
	`in_order('app', 'apple', rank) -> true`

Why	`'h' ranks before 'l', so hello comes first. word vs world differ at d versus l, and d ranks after l, so the order is wrong. A prefix always comes first.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sorted_pair(a, b, order):
    rank = {c: i for i, c in enumerate(order)}
    ranks_a = [rank[c] for c in a]
    ranks_b = [rank[c] for c in b]
    return ranks_a <= ranks_b
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It builds both full rank lists even when the first letters already decide the answer. Comparing character by character and stopping at the first difference is O(min(len a, len b)) with O(1) extra space.

OPTIMAL SOLUTION · Compare character by character, optionally with a custom alphabet

TIME	SPACE
O(min(len a, len b))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What about uppercase, accents, or case-insensitive order?

Normalize first (casefold, unicode normalization) or give the rank map entries for the extra characters. Python's own string comparison orders by code point, which is not locale order, so use locale.strxfrm or a collation library when real language order matters.

FOLLOW-UP 2 A whole list of words in an alien alphabet (LeetCode 953).

Check each adjacent pair; sortedness of a list is the AND of its adjacent pairs. Build the rank table once.

FOLLOW-UP 3 Case-insensitive or accented comparison.

Normalise (casefold, unicode normalisation) before building ranks, or use locale-aware collation keys; plain code-point order is not language order.

FOLLOW-UP 4 Prefix rule.

'app' comes before 'apple' but 'apple' does not come before 'app': compare lengths only after all shared positions are equal.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if two words are sorted in
# lexicographic order.
#
# Words are ordered using a custom alphabet: rank maps
# each letter to its position in that alphabet. Return
# whether word a comes before word b (or is equal to it).
# The first letter where the words differ decides; if one
# word is a prefix of the other, the shorter one comes
# first.
#
# Signature:
#   def in_order(a: str, b: str, rank: dict) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. What about uppercase, accents, or case-insensitive
# order?
# 2. A whole list of words in an alien alphabet (LeetCode
# 953).
# 3. Case-insensitive or accented comparison.
# 4. Prefix rule.
#
# INTERVIEW NOTES
# Compare character by character until the first
# difference; whichever character comes first in the
# alphabet decides, and if one word is a prefix of the
# other the shorter word comes first. A custom alphabet is
# just a lookup table from letter to rank.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import unicodedata
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
ALPHABET = "abcdefghijklmnopqrstuvwxyz"
```

```
# ---- main question: main function ----
# RANKS of the alphabet
def make_rank(order=ALPHABET):
    # the position of every letter in the alphabet
    return {ch: i for i, ch in enumerate(order)}


# BASE QUESTION (optimal): Compare character by character,
# optionally with a custom alphabet
# Idea: Find the first position where the words differ and
# compare the two characters' ranks. If one word is a
# prefix of the other, the shorter one comes first, so
# 'app' is before 'apple' but not the reverse.
def in_order(a, b, rank):
    # compare letter by letter by rank; the first
    # difference
    # decides. If one word is a prefix of the other, the
    # shorter one comes first. A character outside the
    # alphabet is ranked after all the known ones.
    def key(ch):
        return rank.get(ch, len(rank) + ord(ch))

    for x, y in zip(a, b):
        if x != y:
            return key(x) < key(y)
    return len(a) <= len(b)

# ---- main question: tests ----
def test_main_question():
    rank = make_rank()
    assert in_order("apple", "banana", rank)
    assert not in_order("banana", "apple", rank)
    # the prefix rule
    assert in_order("app", "apple", rank)
    assert not in_order("apple", "app", rank)
    assert in_order("same", "same", rank)
    assert in_order("", "a", rank)
    alien = "hlabcdefgijkmnopqrstuvwxyz"
    assert words_sorted(["hello", "leetcode"], alien)
    alien = "worldabcefghijkmnpqstuvxyz"
    mixed = ["word", "world", "row"]
    assert not words_sorted(mixed, alien)
    assert not words_sorted(["apple", "app"], ALPHABET)
    assert words_sorted([], ALPHABET)
```

```
    assert words_sorted(["x"], ALPHABET)
    # case and accents only matter when asked for
    assert not words_sorted(["A", "b"], ALPHABET)
    assert words_sorted(["A", "b"], fold_case=True)
    accents = ["éclair", "ecole"]
    assert words_sorted(accents, fold_case=True)
    for _ in range(500):
        a = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 4)))
        b = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 4)))
        assert in_order(a, b, rank) == (a <= b)


# ============================================================
# FOLLOW-UP 1: What about uppercase, accents, or case-
# ============================================================

# ---- follow-up 1: function ----
# Answer: Normalize first (casefold, unicode
# normalization) or give the rank map entries for the
# extra characters.
def fold(word):
    # follow-up: ignore case and accents before comparing
    text = unicodedata.normalize("NFKD", word.casefold())
    return "".join(ch for ch in text
                   if not unicodedata.combining(ch))

# ============================================================
# FOLLOW-UP 2: A whole list of words in an alien alphabet
# ============================================================

# ---- follow-up 2: function ----
# Answer: Check each adjacent pair; sortedness of a list
# is the AND of its adjacent pairs. Build the rank table
# once.
def words_sorted(words, order=ALPHABET, fold_case=False):
    # follow-up (LeetCode 953): a whole list in an alien
    # alphabet is sorted when every adjacent pair is
    rank = make_rank(order)
    if fold_case:
```

```
        words = [fold(w) for w in words]
    return all(in_order(a, b, rank)
               for a, b in zip(words, words[1:]))
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 169 of 288 | topic 21 of 43: Walk two strings at once with two indices | 3 of 7

Template fit: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes | no date

EASY | Exponent | 169 of 288

Return all unique substrings of a given string.

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 279/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (substring enumeration) (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/1223/unique-substrings-given-string

QUESTION

Return every distinct substring of a string, sorted alphabetically (a substring is a contiguous piece, the whole string and single letters included).

Function signature

def unique_substrings(s: str) -> list

EXAMPLES

Example 1	
Input	`'abc'`
Output	`['a', 'ab', 'abc', 'b', 'bc', 'c']`
Why	`Six different substrings of 'abc'.`

Example 2	
Input	`'aaa'`
Output	`['a', 'aa', 'aaa']`
Why	`'a' appears three times but is listed once.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    result = set()
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
            result.add(s[i:j])
    return result
```

TIME	SPACE
O(n^3)	O(n^3)

WHY IT IS SLOW

BRUTE FORCE

There are about n^2 / 2 substrings and each slice copies up to n characters: O(n^3) time and space. If only the count is needed, a suffix array with LCP counts them in O(n log n) without storing any.

OPTIMAL SOLUTION · Count with a suffix array and LCP

TIME	SPACE
O(n^2 log n) with naive sorting (O(n log n) with a real suffix array)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List them without building duplicates first.

Insert every suffix into a trie; a substring is new exactly when its trie node is created. O(n^2) time and nodes, and the substrings appear without duplicates.

FOLLOW-UP 2 Return the k-th lexicographic distinct substring, or the longest repeated substring.

Both use the suffix array with LCP: the longest repeated substring is the maximum LCP value, and the k-th substring is found by walking the sorted suffixes and subtracting the new prefixes each one contributes.

FOLLOW-UP 3 The string has a million characters.

Quadratic is out. Build a suffix automaton (O(n)) and count distinct substrings as the sum of len(v) - len(link(v)) over states, or use a suffix array with O(n log n) construction.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Return all unique substrings of a given
# string.
#
# Return every distinct substring of a string, sorted
# alphabetically (a substring is a contiguous piece, the
# whole string and single letters included).
#
# Signature:
#   def unique_substrings(s: str) -> list
#
# Examples:
# 'abc'  ->  ['a', 'ab', 'abc', 'b', 'bc', 'c']
# 'aaa'  ->  ['a', 'aa', 'aaa']
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. List them without building duplicates first.
# 2. Return the k-th lexicographic distinct substring, or
# the longest repeated substring.
# 3. The string has a million characters.
#
# INTERVIEW NOTES
# If the output must list the substrings, its size already
# is quadratic, so a set over all slices is the honest
# answer. If only the count is needed, nothing needs to be
# stored: distinct substrings equal total suffix lengths
# minus the sum of longest-common-prefixes between
# adjacent sorted suffixes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Count with a suffix array and
# LCP
```

```
# Idea: Sort the suffixes; the number of distinct
# substrings is sum over suffixes of (length) minus sum of
# the LCP of neighbours in sorted order, because each new
# suffix adds only the prefixes that the previous one did
# not already contribute.
def unique_substrings(s):
    # simplest: every slice into a set, then sort. O(n^3).
    return sorted({s[i:j] for i in range(len(s))
                   for j in range(i + 1, len(s) + 1)})

# ---- main question: tests ----
def test_main_question():
    assert unique_substrings("abc") == [
        "a", "ab", "abc", "b", "bc", "c"]
    assert unique_substrings("aaa") == ["a", "aa", "aaa"]
    assert unique_substrings("") == []


# ============================================================
# FOLLOW-UP 1: List them without building duplicates
# ============================================================

# ---- follow-up 1: function ----
# Answer: Insert every suffix into a trie; a substring is
# new exactly when its trie node is created. O(n^2) time
# and nodes, and the substrings appear without duplicates.
def trie_substrings(s):
    # follow-up: no duplicates built first. Insert every
    # suffix into a trie; a substring is new exactly when
    # its
    # node is created while inserting.
    root = {}
    for start in range(len(s)):
        node = root
        for end in range(start, len(s)):
            ch = s[end]
            if ch not in node:
                node[ch] = {}
                yield s[start:end + 1]
            node = node[ch]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    three = sorted(trie_substrings("aaa"))
    assert three == ["a", "aa", "aaa"]
```

```
# ============================================================
# FOLLOW-UP 2: Return the k-th lexicographic distinct
# ============================================================

# ---- follow-up 2: helper functions ----
def suffix_array(s):
    # start positions of all suffixes in sorted order
    return sorted(range(len(s)), key=lambda i: s[i:])


def lcp_array(s, sa):
    # Kasai: lcp[i] = common prefix of suffix sa[i] and
    # the
    # one before it, found in O(n) in total
    n = len(s)
    rank = [0] * n
    for i, start in enumerate(sa):
        rank[start] = i
    lcp, h = [0] * n, 0
    for start in range(n):
        if rank[start] > 0:
            prev = sa[rank[start] - 1]
            while (start + h < n and prev + h < n
                   and s[start + h] == s[prev + h]):
                h += 1
            lcp[rank[start]] = h
            h = max(h - 1, 0)
        else:
            h = 0
    return lcp

# ---- follow-up 2: function ----
# Answer: Both use the suffix array with LCP: the longest
# repeated substring is the maximum LCP value, and the
# k-th substring is found by walking the sorted suffixes
# and subtracting the new prefixes each one contributes.
def kth_substring(s, k):
    # follow-up: the k-th distinct substring in dictionary
    # order. Walk the sorted suffixes: each one brings
    # n - start - lcp new prefixes.
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
    for i, start in enumerate(sa):
        fresh = len(s) - start - lcp[i]
        if k <= fresh:
            return s[start:start + lcp[i] + k]
        k -= fresh
    return None


# Answer: Both use the suffix array with LCP: the longest
# repeated substring is the maximum LCP value, and the
# k-th substring is found by walking the sorted suffixes
# and subtracting the new prefixes each one contributes.
def longest_repeated(s):
    # follow-up: the biggest lcp value is the longest
    # substring that occurs at least twice
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
    if not s or max(lcp) == 0:
        return ""
    best = max(range(len(s)), key=lcp.__getitem__)
    return s[sa[best]:sa[best] + lcp[best]]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert kth_substring("abc", 4) == "b"
    assert kth_substring("abc", 7) is None
    assert longest_repeated("banana") == "ana"
    assert longest_repeated("abcd") == ""


# ============================================================
# FOLLOW-UP 3: The string has a million characters.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Quadratic is out. Build a suffix automaton
# (O(n)) and count distinct substrings as the sum of
# len(v) - len(link(v)) over states, or use a suffix array
# with O(n log n) construction.
def count_distinct(s):
    # follow-up: a million characters. A suffix automaton
    # is
    # built in O(n); the distinct substrings number
    # sum(len[v] - len[link[v]]) over its states.
    length, link, nxt = [0], [-1], [{}]
    last = 0
```

```
    for ch in s:
        cur = len(length)
        length.append(length[last] + 1)
        link.append(0)
        nxt.append({})
        p = last
        while p != -1 and ch not in nxt[p]:
            nxt[p][ch] = cur
            p = link[p]
        if p != -1:
            q = nxt[p][ch]
            if length[p] + 1 == length[q]:
                link[cur] = q
            else:
                # split q: a clone with the shorter length
                clone = len(length)
                length.append(length[p] + 1)
                link.append(link[q])
                nxt.append(dict(nxt[q]))
                while p != -1 and nxt[p].get(ch) == q:
                    nxt[p][ch] = clone
                    p = link[p]
                link[q] = link[cur] = clone
        last = cur
    return sum(length[v] - length[link[v]]
               for v in range(1, len(length)))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert count_distinct("aaa") == 3
    assert count_distinct("") == 0
    for _ in range(300):
        s = "".join(random.choice("abc")
                    for _ in range(random.randint(0, 9)))
        want = unique_substrings(s)
        assert sorted(trie_substrings(s)) == want
        assert len(list(trie_substrings(s))) == len(want)
        assert count_distinct(s) == len(want)
        for k, sub in enumerate(want, 1):
            assert kth_substring(s, k) == sub


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 170 of 288 | topic 21 of 43: Walk two strings at once with two indices | 4 of 7

Template fit: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right | asked 202 days ago

MEDIUM | PracHub | Technical Screen | 170 of 288

Can board states be transformed?

POSTED 2026-04-01 | 202 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 107/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (inferred from overview)

Posted on PracHub: 2026-04-01 (updated 2026-04-19)

Original:
https://prachub.com/coding-questions/can-board-states-be-transformed

QUESTION

A board is a string of 'L', 'R', '#' and 'X'. 'X' is an empty square. An 'L' piece may move left into an empty square, an 'R' piece may move right into an empty square, pieces never jump over each other, and '#' (a wall piece) never moves. Return whether start can be turned into end by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	'RXXLRXRXL', 'XRLXXRRLX'
Output	true
Why	L moves left and R moves right while keeping their order: this classic board is reachable.

Example 2	
Input	'XL', 'LX'
Output	true
Why	L can move left.

Example 3	
Input	'LX', 'XL'
Output	false
Why	L cannot move to the right.

Example 4	
Input	'#X', 'X#'
Output	false
Why	A wall never moves.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in legal_moves(board):
            if nxt not in seen:  # XL->LX, RX->XR
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential in the length	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially. Pieces never cross, so compare the sequence of L and R pieces and check that no L moved right and no R moved left, in O(n).

OPTIMAL SOLUTION · Two pointers over the pieces

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

FOLLOW-UP 2 Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

FOLLOW-UP 3 Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Can board states be transformed?
#
# A board is a string of 'L', 'R', '#' and 'X'. 'X' is an
# empty square. An 'L' piece may move left into an empty
# square, an 'R' piece may move right into an empty
# square, pieces never jump over each other, and '#' (a
# wall piece) never moves. Return whether start can be
# turned into end by any number of moves.
#
# Signature:
#   def can_transform(start: str, end: str) -> bool
#
# Examples:
# 'RXXLRXRXL', 'XRLXXRRLX'  ->  true
# 'XL', 'LX'  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minimum number of moves.
# 2. Add walls that cannot be crossed (next question).
# 3. Pieces can move one step per round and several move
# at once.
#
# INTERVIEW NOTES
# Think invariants, not search. Pieces cannot jump each
# other, so the sequence of L and R letters (ignoring
# blanks) cannot change. Within that, an L can only go
# left and an R only right, so compare the position of
# each piece in the start and in the target. That reduces
# exponential BFS to one pass with two pointers.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def pieces(board):
    # the non-blank symbols with their positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "X"]

# ---- main question: main function ----
# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter.
def can_transform(start, end):
    # L moves left, R moves right, X is blank. Pieces
    # never pass
    # each other, so the order of the non-X symbols is
    # fixed;
    # a '#' is a wall: it never moves and no piece crosses
    # it.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        # an L may only end up further left, an R further
        # right
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
        # a wall must stay exactly where it is
        if c1 == "#" and i != j:
            return False
    return True

# ---- main question: tests ----
def test_main_question():
    assert can_transform("RXXLRXRXL", "XRLXXRRLX")
    assert not can_transform("X", "L")
    assert can_transform("XXXL", "XXLX")
    # R can not move left, L can not move right
    assert not can_transform("XR", "RX")
    assert not can_transform("LX", "XL")
    # pieces keep their order: RL can not become LR
    assert not can_transform("RL", "LR")
```

```
# ============================================================
# FOLLOW-UP 1: Return the minimum number of moves.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Because pieces never interact beyond keeping
# their order, each piece moves independently from its
# start index to its target index, so the minimum is the
# sum of the absolute index differences when the
# transformation is possible.
def min_moves(start, end):
    # every piece moves on its own, one cell per move
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_moves("RXXLRXRXL", "XRLXXRRLX") == 4
    assert min_moves("RXX", "XXR") == 2
    assert min_moves("RX", "XR") == 1
    assert min_moves("LL", "RR") is None
    # a wall splits the board: R can not jump over it
    assert can_transform("R#X", "R#X")
    assert not can_transform("R#X", "X#R")


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Pieces can move one step per round and
# ============================================================

# ---- follow-up 3: function ----
# Answer: The count of rounds is the maximum over pieces
# of their distance, if pieces never block each other;
```

```
# with blocking you simulate rounds or use the per-piece
# arrival times with the ordering constraint.
def rounds_needed(start, end):
    # if all pieces move at once and never block each
    # other,
```

```
    # the number of rounds is the longest single trip
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    trips = [abs(i - j) for (_, i), (_, j) in zip(a, b)]
    return max(trips, default=0)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rounds_needed("RXX", "XXR") == 2
    assert rounds_needed("LL", "RR") is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(start: str, target: str) -> bool:
```

Sample call

```
solution("R_L|__L", "_RL|L__")
```

Test cases

Input	Expected
("R_L|__L", "_RL|L__")	True
("L_|R", "L|_R")	False

STUDY PATH: question 171 of 288 | topic 21 of 43: Walk two strings at once with two indices | 5 of 7

Template fit: the same walk with dots as blanks and the movement rule from the statement (walls block) | asked 462 days ago

MEDIUM | PracHub | Technical Screen | 171 of 288

Decide string transform with directional tokens and walls

POSTED 2025-07-15 | 462 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 108/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (with X walls)

Posted on PracHub: 2025-07-15 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/decide-string-transform-with-directional-tokens-and-walls

QUESTION

Two equal-length strings start and target use the characters 'L', 'R', '.' (an empty slot) and 'X' (an immovable wall). An 'L' token can move left into empty slots, an 'R' token can move right into empty slots, tokens can never pass through each other or through a wall, and 'X' never moves. Return whether start can be turned into target by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

Example 1	
Input	`'R..L', '.RL.'`
Output	`true`
Why	`R moves one step right and L one step left: both stay in order.`

Example 2	
Input	`'L.', '.L'`
Output	`false`
Why	`L can only move left, never right.`

Example 3	
Input	`'X.', '.X'`
Output	`false`
Why	`A wall cannot move.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in one_step(board):
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially with the length. Comparing the non-dot sequence (walls and pieces in order) with position rules (L never moves right, R never moves left, walls stay) is O(n).

OPTIMAL SOLUTION · Compare the non-dot sequences with position rules

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

FOLLOW-UP 2 Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

FOLLOW-UP 3 The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

FOLLOW-UP 4 Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Decide string transform with directional
# tokens and walls
#
# Two equal-length strings start and target use the
# characters 'L', 'R', '.' (an empty slot) and 'X' (an
# immovable wall). An 'L' token can move left into empty
# slots, an 'R' token can move right into empty slots,
# tokens can never pass through each other or through a
# wall, and 'X' never moves. Return whether start can be
# turned into target by any number of moves.
#
# Signature:
#   def can_transform(start: str, end: str) -> bool
#
# Examples:
# 'R..L', '.RL.'  ->  true
# 'L.', '.L'  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Segments between walls are independent.
# 2. Return the moves or the minimum number of steps.
# 3. The board is 2D.
# 4. Return the minimum number of moves.
#
# INTERVIEW NOTES
# Walls are pieces that never move. Ignore the dots and
# compare the sequences of L, R and X: they must match in
# order and in count, and each X must be at the same index
# in both strings. For the L and R pieces keep the same
# rule: L never moves right, R never moves left. Order is
# preserved, so no piece can get across a wall.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import itertools


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def pieces(board):
    # everything except the empty '.' slots, with
    # positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "."]

# ---- main question: main function ----
# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index.
def can_transform(start, end):
    # 'L' moves left, 'R' moves right, 'X' is an immovable
    # wall. Pieces never pass each other, so their order
    # is
    # fixed; each one only has a direction it may move.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        if c1 == "X" and i != j:
            return False
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
    return True

# ---- main question: tests ----
def test_main_question():
    assert can_transform("R.L", ".RL") is True
    assert not can_transform("RL.", "LR.")
    # a wall cannot move and splits the board
    assert can_transform("R.X.L", ".RX.L") is True
    assert not can_transform("R.X", "X.R")
```

```
# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Return the minimum number of moves.
# ============================================================

# ---- follow-up 4: function ----
# MIN MOVES (pieces move independently)
# FOLLOW-UP 4: Return the minimum number of moves.
# Answer: When the transformation is possible each piece
# moves independently from its start index to its target
# index, so the minimum is the sum of the absolute index
# differences (checked against BFS on small strings).
def min_moves(start, end):
```

```
    # pieces move independently: the sum of index
    # differences
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))

# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert min_moves("R..L", ".RL.") == 2
    assert min_moves("L.", ".L") is None
    boards = []
    for n in range(1, 5):
        for p in itertools.product("LR.X", repeat=n):
            boards.append("".join(p))


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def canTransform(start: str, target: str) -> bool:
```

Sample call

```
canTransform('R...L', 'RL...')
```

Test cases

Input	Expected
`('R...L', 'RL...')`	`True`
`('R.L', 'R..L')`	`False`

STUDY PATH: question 172 of 288 | topic 21 of 43: Walk two strings at once with two indices | 6 of 7

Template fit: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions) | no date

MEDIUM | Exponent | 172 of 288

Determine if two sentences are similar.

ASKED (ANY CO.) ~4 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 21 | 30 DAYS 0 | PRIORITY 165/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1813 Sentence Similarity III

Last asked (any company): 4 years ago, about 2022-10 (Exponent shows only a rough age) | 1 interview

Original: https://www.aced.io/questions/2280/determine-similar-sentences

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

EXAMPLES

Example 1

Input	`['great', 'acting', 'skills'], ['fine', 'drama', 'talent'], [['great', 'fine'], ['acting', 'drama'], ['skills', 'talent']]`
Output	`true`
Why	`Each word is paired with a similar word.`

QUESTION

Example 2

Input	`['a'], ['a', 'b'], []`
Output	`false`
Why	`Sentences of different lengths are never similar.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    for a, b in zip(s1, s2):
        if a == b:
            continue
        if [a, b] in pairs or [b, a] in pairs:
            continue
        return False
    return True
```

TIME	SPACE
O(n * P)	O(1)

WHY IT IS SLOW

Each word pair scans the whole pair list, O(n * P). Putting the pairs into a set makes each check O(1).

OPTIMAL SOLUTION · Set of pairs (not transitive)

TIME	SPACE
O(n + P)	O(P)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Similarity is transitive (a ~ b and b ~ c means a ~ c).

Union-find over the words: union each pair, then two words are similar if they are equal or have the same root. Path compression keeps it near O(n + P).

FOLLOW-UP 2 One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813).

Match the longest common prefix and the longest common suffix of the word lists; the shorter list must be covered by prefix plus suffix.

FOLLOW-UP 3 Case should not matter, or words can have synonyms with scores.

Normalise with casefold before hashing. For scores, a graph with weighted edges and a threshold query turns it into reachability along strong edges.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if two sentences are similar.
#
# Determine if Two Sentences Are Similar. Two sentences
# are similar if they have the same length and each pair
# of corresponding words in the two sentences is similar.
# The similarity between words is defined by the provided
# list of similar word pairs. A word is always similar to
# itself. For example, if we have the list of similar word
# pairs as [("great", "good"), ("acting","drama"),
# ("skills","talent")], then the sentences "You have great
# acting skills" and "You have good drama talent" are
# similar.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Similarity is transitive (a ~ b and b ~ c means a ~
# c).
# 2. One sentence is the other with a sentence of words
# inserted somewhere (LeetCode 1813).
# 3. Case should not matter, or words can have synonyms
# with scores.
#
# INTERVIEW NOTES
# Ask which similarity you have: from a pair list,
# transitive or not. Without transitivity it is a set
# lookup per word pair. With transitivity, similar words
# form groups, which is union-find (or BFS on a word
# graph). The variant where a block is inserted is a
# different problem: match the common prefix and suffix of
# the word lists.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Set of pairs (not transitive)
# Idea: Put each pair into a set in both directions, so a
# word pair is similar if the words are equal or the pair
# is in the set. Sentences of different length are never
# similar.
```

```
def similar_direct(s1, s2, pairs):
    # same length, and every pair of words is equal or a
    # listed pair (in either direction)
    if len(s1) != len(s2):
        return False
    allowed = {frozenset(p) for p in pairs}
    return all(a == b or frozenset((a, b)) in allowed
               for a, b in zip(s1, s2))

# ---- main question: tests ----
def test_main_question():
    pairs = [("great", "good"), ("acting", "drama"),
             ("skills", "talent")]
    a = "You have great acting skills".split()
    b = "You have good drama talent".split()
    assert similar_direct(a, b, pairs)
    assert not similar_direct(a, b[:-1], pairs)
    assert not similar_direct(["a"], ["b"], [])
    assert similar_direct(["a"], ["a"], [])
    # chain: good ~ fine and fine ~ okay, so good ~ okay
    chain = [("good", "fine"), ("fine", "okay")]
    assert not similar_direct(["good"], ["okay"], chain)


# ============================================================
# FOLLOW-UP 1: Similarity is transitive (a ~ b and b ~ c
# ============================================================

# ---- follow-up 1: helper functions ----
class DisjointSet:
    def __init__(self):
        self.parent = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        # path compression: point everything on the way up
        # straight at the root
```

```
        while self.parent[x] != root:
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)

# ---- follow-up 1: function ----
# Answer: Union-find over the words: union each pair, then
# two words are similar if they are equal or have the same
# root. Path compression keeps it near O(n + P).
def similar_transitive(s1, s2, pairs):
    # follow-up: a ~ b and b ~ c means a ~ c, so words are
    # grouped with union-find
    if len(s1) != len(s2):
        return False
    groups = DisjointSet()
    for a, b in pairs:
        groups.union(a, b)
    return all(a == b or groups.find(a) == groups.find(b)
               for a, b in zip(s1, s2))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # chain: good ~ fine and fine ~ okay, so good ~ okay
    chain = [("good", "fine"), ("fine", "okay")]
    assert similar_transitive(["good"], ["okay"], chain)


# ============================================================
# FOLLOW-UP 2: One sentence is the other with a sentence
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Match the longest common prefix and the longest
# common suffix of the word lists; the shorter list must
# be covered by prefix plus suffix.
def similar_insertion(s1, s2):
    # follow-up (LeetCode 1813): one sentence is the other
    # with a block of words inserted somewhere. The
    # shorter
    # one must be covered by a common prefix plus a common
    # suffix of the two.
    if len(s1) > len(s2):
        s1, s2 = s2, s1
```

```
    head = 0
    while head < len(s1) and s1[head] == s2[head]:
        head += 1
    tail = 0
    room = len(s1) - head
    while tail < room and s1[-1 - tail] == s2[-1 - tail]:
        tail += 1
    return head + tail == len(s1)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert similar_insertion("My name is Haley".split(),
                             "My Haley".split())
    assert similar_insertion(["of"], ["A", "lot", "of"])
    eating = "Eating right now".split()
    assert similar_insertion(eating, ["Eating"])
    assert not similar_insertion(["a", "b"], ["c", "d"])


# ============================================================
# FOLLOW-UP 3: Case should not matter, or words can have
# ============================================================

# ---- follow-up 3: function ----
# Answer: Normalise with casefold before hashing. For
# scores, a graph with weighted edges and a threshold
# query turns it into reachability along strong edges.
def normalise(words):
    # follow-up: case does not matter
    return [w.casefold() for w in words]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert normalise(["Great"]) == ["great"]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples

```
sentence1 = ["Let's", "code", "in", "Python"]
sentence2 = ["Let's", "program", "in", "Python"]
similarPairs = [
    ("code", "program"),
]
output: true

sentence1 = ["I", "love", "to", "play", "football"]
sentence2 = ["I", "love", "playing", "soccer"]
similarPairs = [("play", "playing"), ("football", "soc
cer")]
output: false, different sentence lengths

sentence1 = ["Do", "you", "like", "coffee"]
sentence2 = ["Do", "you", "love", "coffee"]
similarPairs = [
    ("like", "enjoy"),
    ("coffee", "tea"),
]
output: false  # "like" is not similar to "love" based
 on the given pairs.

sentence1 = ["I", "really", "love", "leetcode", "and",
 "apples"]
sentence2 = ["I", "so", "like", "codesignal", "and", "
oranges"]
similarPairs = [
    ("very", "so"),
    ("love", "adore"),
    ("really", "very"),
    ("leetcode", "codesignal"),
```

```
    ("apples", "oranges"),
    ("like", "adore"),
]
output: true
```

OFFICIAL PAGE TEXT

The provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are connected if they are similar according to the provided pairs. Then, it performs a depth-first search (DFS) to check if each pair of corresponding words from the two sentences are connected in the graph. The Depth-First Search (DFS) algorithm is well-suited for this question as it efficiently explores the graph structure created from the similar word pairs. DFS is ideal for checking connectivity between words, which aligns with the task of determining if two words are similar based on the provided pairs. Its memory efficiency is beneficial for handling potentially large sets of sentences and word pairs. Additionally, DFS offers a simple implementation, either through recursion or using a stack, which aids in writing clean and understandable code. Its flexibility allows for easy modification to incorporate additional constraints or optimizations if needed. Therefore, DFS provides an elegant and effective solution for traversing the graph and determining sentence similarity. Remember to handle potential off-by-one errors when implementing this solution due to index-based iteration over the sentences. Time Complexity: Let n be the maximum number of words in the sentences, and m be the number of similar word pairs. Constructing the

graph takes O(m) time. Then, for each word pair, the DFS takes O(n) time in the worst case, resulting in a total time complexity of O(n + m). Space Complexity: In the worst case, the graph can contain O(m) edges. Additionally, in the DFS, the space complexity is O(n) due to the recursion stack or auxiliary data structures to track visited nodes. Therefore, the overall space complexity is O(n + m).Watch Simon, software engineer at Google, answer the question "determine if two sentences are similar."No answers contributed by the community yet.Related questionsDetermine if a number is a power of two.Determine if two words are sorted in lexicographic order.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start lea

rningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY PATH: question 173 of 288 | topic 21 of 43: Walk two strings at once with two indices | 7 of 7

Template fit: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3 | asked 194 days ago

HARD | PracHub | Technical Screen | 173 of 288

Find Dictionary Matches for a Jammed String

POSTED 2026-04-09 | 194 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 155/288

ABOUT THIS QUESTION

Topic: Walk two strings at once with two indices

Optimal answer from: the interview site's own solution

LeetCode match: LC 809 Expressive Words (inferred from overview)

Posted on PracHub: 2026-04-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/find-dictionary-matches-for-a-jammed-string

QUESTION

A word was typed on a keyboard where some keys got stuck, so a letter may be repeated. Given a dictionary of words, matches(query) returns the dictionary words the query could be a jammed version of: the runs of equal letters line up with the word's runs, and each run of the query is either exactly as long as in the word or longer and at least min_stretch (default 3) letters long.

Function signature

class JammedIndex:
  def __init__(self, words: list)
  def matches(self, query: str, min_stretch: int = 3) -> list

EXAMPLES

Example 1

Input	`idx = JammedIndex(['hello', 'help', 'hel'])`
	`idx.matches('heeello')`
	`idx.matches('hello')`
	`idx.matches('heello')`
	`idx.matches('helllo')`

Output	`idx.matches('heeello') -> ['hello']`
	`idx.matches('hello') -> ['hello']`
	`idx.matches('heello') -> []`
	`idx.matches('helllo') -> ['hello']`

Why	`'heeello' stretches the e to 3 letters, which is allowed. 'heello' only stretches it to 2 (shorter than min_stretch), and 'helllo' stretches the l run from 2 to 3 letters, which is allowed.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def matches(query, words):
    runs_q = runs(query)  # [(letter, count)]
    result = []
    for word in words:
        runs_w = runs(word)
        if compatible(runs_w, runs_q):
            result.append(word)
    return result
```

TIME	SPACE
O(W * L) per query	O(L)

WHY IT IS SLOW

Every query touches every word, O(W * L), even words whose letters cannot match. Grouping words by their run letters in a hash map checks only the right bucket.

OPTIMAL SOLUTION · Index the dictionary by its run letters

TIME	SPACE
O(L) per query plus the size of the matching bucket	O(total dictionary length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

FOLLOW-UP 2 Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. **Cost:** O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

FOLLOW-UP 3 The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find Dictionary Matches for a Jammed
# String
#
# A word was typed on a keyboard where some keys got
# stuck, so a letter may be repeated. Given a dictionary
# of words, matches(query) returns the dictionary words
# the query could be a jammed version of: the runs of
# equal letters line up with the word's runs, and each run
# of the query is either exactly as long as in the word or
# longer and at least min_stretch (default 3) letters
# long.
#
# Signature:
#   class JammedIndex:
#       def __init__(self, words: list)
#       def matches(self, query: str, min_stretch: int = 3
) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. A different stretch rule: any run may be longer, no
# minimum.
# 2. Millions of dictionary words and many queries.
# 3. The jammed string may also be missing letters.
#
# INTERVIEW NOTES
# 'Same letters, stretched' means compare run-length
# encodings, not characters. The letter order of the runs
# has to match exactly, so the sequence of distinct run
# letters is a hash key that cuts the dictionary down
# before any counts are compared.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from itertools import groupby


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]
    return [(ch, len(list(g))) for ch, g in groupby(word)]

# ---- main question: main function ----
# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
# FOLLOW-UP 1: A different stretch rule: any run may be
# longer, no minimum.
# Answer: Only the comparison changes. The index (grouping
# by run letters) is the same, so make the minimum stretch
# a parameter, as the code does with min_stretch.
class JammedIndex:
    """A query run may be longer than the word's run
    when it is at least min_stretch long."""

    def __init__(self, words):
        # words grouped by their run letters ("hello" ->
        # "helo")
        self.groups = {}
        for w in words:
            r = runs(w)
            letters = "".join(ch for ch, _ in r)
            self.groups.setdefault(letters, []).append(
                (w, [n for _, n in r]))

    # FOLLOW-UP 1: min_stretch=1: any run may be longer
    def matches(self, query, min_stretch=3):
        r = runs(query)
        letters = "".join(ch for ch, _ in r)
        counts = [n for _, n in r]
        found = []
        # only words with the same run letters can match
        group = self.groups.get(letters, [])
        for word, word_counts in group:
            if all(self.fits(q, w, min_stretch)
                   for q, w in zip(counts, word_counts)):
                found.append(word)
        return sorted(found)
```

```
    @staticmethod
    def fits(q, w, min_stretch):
        # the same length, or a stretch of at least
        # min_stretch
        return q == w or (q > w and q >= min_stretch)

# ---- main question: tests ----
def test_main_question():
    index = JammedIndex(["hello", "help", "hi", "hip"])
    # e stretched to 4 and o stretched to 3
    assert index.matches("heeeellooooo") == ["hello"]
    assert index.matches("hello") == ["hello"]
    # a stretch of only 2 is not enough
    assert index.matches("heello") == []
    assert index.matches("hiiii") == ["hi"]
    assert index.matches("zzz") == []
    # a smaller minimum accepts the short stretch
    short = index.matches("heello", min_stretch=2)
    assert short == ["hello"]
    # no minimum: any longer run is fine
    assert index.matches("hiip", min_stretch=1) == ["hip"]
    want = [("a", 2), ("b", 1), ("c", 3)]
    assert runs("aabccc") == want


# ============================================================
# FOLLOW-UP 1: A different stretch rule: any run may be
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(dictionary_words: list[str], jammed_string: s
tr) -> list[str]:
```

Sample call

```
solution(['help', 'hello', 'shell', 'hellp'], 'hellp')
```

Test cases

Input	Expected
(['help', 'hello', 'shell', 'hellp'], 'hellp')	['help', 'hellp']
(['hello', 'halo', 'heeello'], 'hheeellllo')	['hello', 'heeello']

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 22 OF 43

Simulate a machine: a class with state and one branch per kind of event

7 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	5	1
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Simulate a machine: a class with state and one branch per kind of event | 7 questions | tap a title to jump to it

READ FIRST: the code template for these 7 questions (tap here) >

E	174. Mario-Style Score Tracker (OOP)	ColorStack	2025-10-11
M	175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win	1Point3Acres	2026-05-18
M	176. Design Tic-Tac-Toe With K Players	PracHub	2026-02-12
M	177. Infinite Deck Card Duel	ColorStack	2025-09-26
M	178. Implement iterator merging lists with filtering	PracHub	2026-01-06
M	179. Design an elevator control system	PracHub	2025-09-06
H	180. Simulate In-Place Cellular State Updates	PracHub	2026-05-04

CODE TEMPLATE | GROUP 22 OF 43 | 7 QUESTIONS, ONE PATTERN

Simulate a machine: a class with state and one branch per kind of event

The one idea these questions share

Put the state in __init__, give it one method that takes an event, and write one branch per event kind. Board games keep a grid or counters and check for a win after each move; game-of-life style updates compute every cell from the old state before changing it.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
class Machine:
    def __init__(self):
        self.score, self.lives = 0, 3             # CHANGE 1: the state
        self.over = False

    def process(self, event):
        if self.over:
            return
        if event == "coin":                       # CHANGE 2: one branch per kind of event
            self.score += 100
        elif event == "damage":
            self.lives -= 1
            self.over = self.lives == 0
```

2. Helper functions that usually get written next to it

```
def neighbors8(r, c):
    return [(r + dr, c + dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if dr or dc]


def run_through(board, r, c, dr, dc, player):    # length of the line of one player through a cell
    n, run = len(board), 1
    for sign in (1, -1):
        a, b = r + sign * dr, c + sign * dc
        while 0 <= a < n and 0 <= b < n and board[a][b] == player:
            run += 1
            a, b = a + sign * dr, b + sign * dc
    return run
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
174	Mario-Style Score Tracker (OOP)	the template as written: an event name decides how score, lives and size change
175	Tic-Tac-Toe System with K Players and 3-in-a-Row Win	store the board; after each move count the run through the new cell in four directions and compare with the target length
176	Design Tic-Tac-Toe With K Players	per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n
177	Infinite Deck Card Duel	an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins
178	Implement iterator merging lists with filtering	keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()
179	Design an elevator control system	each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

Contents Start the questions >

Master contents (all topics)

#	Question	What you change in the template
180	Simulate In-Place Cellular State Updates	encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 174 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 1 of 7

Template fit: the template as written: an event name decides how score, lives and size change | asked 374 days ago | frequency Single report

EASY | ColorStack | technical interview | 174 of 288

Mario-Style Score Tracker (OOP)

ASKED 2025-10-11 | 374 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 182/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: State machine design (related: LC 1472 / simulation)

Frequency: Single report | Last asked: 2025-10-11 | Roles: SWE Intern

QUESTION

Design a class that tracks the state of a platformer character across a sequence of events. The character has a score (starts at 0), a lives count (starts at 3), and a big flag (starts False). Process events with these rules: "coin" : add 100 to the score. "mushroom" : add 1000 to the score and set big = True . "enemy" : add 200 to the score (the character stomps an enemy). "damage" : if big is True , set big = False (no life lost); otherwise decrease lives by 1. Score is unchanged. The character is alive while lives > 0 . Once lives reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...
    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...
    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...
    def get_score(self) -> int:
        ...
    def is_alive(self) -> bool:
        ...
```

EXAMPLES

QUESTION

Example 1

Input	`g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]); (g.get_score(), g.is_alive())`
Output	`(1300, True)`
Why	`coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False (no life lost); damage -> small, lives 3->2. Score 1300, still alive.`

Example 2

Input	`g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(), g.is_alive())`
Output	`(0, False)`
Why	`Three small-state hits drop lives 3->0 (game over). The trailing "coin" is ignored.`

CONSTRAINTS

- Event strings are always one of the four listed above.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_mario(events):
    score, lives, big = 0, 3, False
    for e in events:
        if lives == 0:
            break
        if e == "coin":
            score += 100
        elif e == "mushroom":
            score, big = score + 1000, True
        elif e == "enemy":
            score += 200
        elif e == "damage":
            if big:
                big = False
            else:
                lives -= 1
    return score, lives > 0
```

TIME	SPACE
O(events)	O(1)

WHY IT IS SLOW

It is the same rule set written as a pure function; it is the oracle for the class.

OPTIMAL SOLUTION · A small state class

TIME	SPACE
O(events)	O(1)

OPTIMAL SOLUTION · A small state class

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add undo() to take back the last event.

Before applying an event push a snapshot (score, lives, big) on a stack; undo pops it and restores the three fields. Events ignored after game over should not push a snapshot.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Mario-Style Score Tracker (OOP)
#
# Design a class that tracks the state of a platformer
# character across a sequence of events. The character has
# a score (starts at 0), a lives count (starts at 3), and
# a big flag (starts False ). Process events with these
# rules: "coin" : add 100 to the score. "mushroom" : add
# 1000 to the score and set big = True . "enemy" : add 200
# to the score (the character stomps an enemy). "damage" :
# if big is True , set big = False (no life lost);
# otherwise decrease lives by 1. Score is unchanged. The
# character is alive while lives > 0 . Once lives reaches
# 0, the game is over and any further events are ignored.
# Implement the class and a method to run a batch of
# events. Function signature: class MarioGame: def
# __init__(self) -> None: """score=0, lives=3,
# big=False.""" ... def process(self, event: str) -> None:
# """Apply one event, ignoring it if the game is already
# over.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add undo() to take back the last event.
#
# INTERVIEW NOTES
# A small state machine: score, lives and a big flag
# change according to fixed rules, and once lives reach
# zero every later event is ignored. Keep the rules in one
# method and let run() call it for each event; never reach
# into the fields from outside.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A small state class
# Idea: One method owns the rules; the game ends when
# lives hit zero.
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return
        if event == "coin":
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score

    def is_alive(self):
        return self.lives > 0

# ---- main question: tests ----
def test_main_question():
    random.seed(60)
    g = MarioGame()
    g.run(
        [
            "coin",
            "mushroom",
            "enemy",
            "damage",
            "damage",
        ]
    )
    assert (g.get_score(), g.is_alive()) == (1300, True)
    g = MarioGame()
    g.run(["damage", "damage", "damage", "coin"])
    assert (g.get_score(), g.is_alive()) == (0, False)


# ============================================================
# FOLLOW-UP 1: Add undo() to take back the last event.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Before applying an event push a snapshot (score,
# lives, big) on a stack; undo pops it and restores the
# three fields. Events ignored after game over should not
# push a snapshot.
class UndoMarioGame(MarioGame):
    def __init__(self):
        super().__init__()
        self.history = []

    def process(self, event):
        if self.lives == 0:
            return
        self.history.append(
            (self.score, self.lives, self.big)
```

```
        )
        super().process(event)

    def undo(self):
        if self.history:
            self.score, self.lives, self.big = (
                self.history.pop()
            )

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        events = [
            random.choice(
                ["coin", "mushroom", "enemy", "damage"]
            )
            for _ in range(random.randint(0, 12))
        ]
        g = MarioGame()
        g.run(events)
        u = UndoMarioGame()
        u.run(events)
        for _ in range(len(events)):
            u.undo()
        assert (u.get_score(), u.lives, u.big) == (
            0,
            3,
            False,
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 175 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 2 of 7

Template fit: store the board; after each move count the run through the new cell in four directions and compare with the target length | asked 155 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, tech-screen | 175 of 288

Tic-Tac-Toe System with K Players and 3-in-a-Row Win

ASKED 2026-05-18 | 155 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 175/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 348 Design Tic-Tac-Toe (related)

Frequency: Single report | Last asked: 2026-05-18 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

QUESTION

Design Tic-Tac-Toe for k players on an n x n board. Players move in turn (1, 2, ..., k, then 1 again). move(player, r, c) places a mark and returns 'player p wins' as soon as that player has 3 marks in a straight line (row, column or diagonal), 'draw' when the board is full, and 'continue' otherwise. A move out of turn, off the board or onto a taken cell raises an error.

Function signature

class TicTacToe:
  def __init__(self, n: int, k: int, target: int = 3)
  def move(self, player: int, r: int, c: int) -> str

EXAMPLES

Example 1

Input	`g = TicTacToe(3, 2)`
	`g.move(1, 0, 0)`
	`g.move(2, 1, 0)`
	`g.move(1, 0, 1)`
	`g.move(2, 1, 1)`
	`g.move(1, 0, 2)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 1, 0) -> 'continue'`
	`g.move(1, 0, 1) -> 'continue'`
	`g.move(2, 1, 1) -> 'continue'`
	`g.move(1, 0, 2) -> 'player 1 wins'`

Why	`Player 1 takes the whole top row while player 2 has two in the middle row: player 1 wins.`

QUESTION

Example 2

Input	`g = TicTacToe(3, 3)`
	`g.move(1, 0, 0)`
	`g.move(2, 0, 1)`
	`g.move(3, 0, 2)`
	`g.move(1, 1, 1)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 0, 1) -> 'continue'`
	`g.move(3, 0, 2) -> 'continue'`
	`g.move(1, 1, 1) -> 'continue'`

Why	`With three players the turns go 1, 2, 3, 1; nobody has three in a line yet.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def status(player):
    for r, c in all_cells(board):
        for dr, dc in DIRECTIONS:
            cells = line_of_3(r, c, dr, dc)
            if all(board[x][y] == player
                   for x, y in cells):
                return "WIN"
    if is_full(board):
        return "DRAW"
    return "PLAYING"
```

TIME	SPACE
O(n^2) per move	O(n^2)

WHY IT IS SLOW

Scans O(n^2) cells per move. Only lines through the cell just played can have changed, so checking four directions around it is O(1).

OPTIMAL SOLUTION · Check the four lines through the new mark

TIME	SPACE
O(1) per move (O(target) when generalised)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes when the winning length is a parameter w?

Replace the constant 3 by w in the run check: the cost per move becomes O(w) because each of the four directions walks at most w - 1 cells each way. The board size does not matter any more, which is the point of checking locally.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Tic-Tac-Toe System with K Players and
# 3-in-a-Row Win
#
# Design Tic-Tac-Toe for k players on an n x n board.
# Players move in turn (1, 2, ..., k, then 1 again).
# move(player, r, c) places a mark and returns 'player p
# wins' as soon as that player has 3 marks in a straight
# line (row, column or diagonal), 'draw' when the board is
# full, and 'continue' otherwise. A move out of turn, off
# the board or onto a taken cell raises an error.
#
# Signature:
#   class TicTacToe:
#       def __init__(self, n: int, k: int, target: int = 3
)
#       def move(self, player: int, r: int, c: int) -> str
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes when the winning length is a parameter
# w?
#
# INTERVIEW NOTES
# A move can only create a win through the cell just
# played, so after each move look only along the four
# lines through that cell (row, column, both diagonals)
# and count how far the same player's marks extend in both
# directions. With a target of 3 that is a handful of
# cells per move, instead of rescanning the whole board.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Check the four lines through
# the new mark
# Idea: Place the mark, then for each of the four
# directions walk outward both ways while the cells hold
# the same player and add the two counts plus one. A total
# of at least 3 wins immediately.
# FOLLOW-UP 1: What changes when the winning length is a
# parameter w?
# Answer: Replace the constant 3 by w in the run check:
# the cost per move becomes O(w) because each of the four
# directions walks at most w - 1 cells each way. The board
# size does not matter any more, which is the point of
# checking locally.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        # marks of the same player in a line through (r,
        # c)
        player, run = self.board[r][c], 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.inside(a, b) and \
                    self.board[a][b] == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        # whose turn is it? players go 1, 2, ..., k, 1,
        # ...
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c]:
            raise ValueError("illegal cell")
```

```
        self.board[r][c] = player
        self.moves += 1
        # only lines through the new mark can have changed
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.over = True
                return f"player {player} wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"

# ---- main question: tests ----
def test_main_question():
    g = TicTacToe(3, 2)
    plan = [(1, 0, 0), (2, 1, 0), (1, 0, 1),
            (2, 1, 1), (1, 0, 2)]
    results = [g.move(p, r, c) for p, r, c in plan]
    assert results[:-1] == ["continue"] * 4
    assert results[-1] == "player 1 wins"
    # three players on a 4 x 4 board: player 2 wins down
    h = TicTacToe(4, 3)
    seq = [(1, 0, 0), (2, 0, 3), (3, 3, 3), (1, 1, 0),
           (2, 1, 3), (3, 3, 0), (1, 2, 1), (2, 2, 3)]
    out = [h.move(p, r, c) for p, r, c in seq]
    assert out[-1] == "player 2 wins"
    # wrong turn and used cell are rejected
    for bad in ((2, 0, 0), (1, 5, 5)):
        try:
            TicTacToe(3, 2).move(*bad)
            raise AssertionError(bad)
        except ValueError:
            pass
    # the win length is a parameter: 4 in a row
    w = TicTacToe(5, 2, target=4)
    for p, r, c in [(1, 0, 0), (2, 1, 0), (1, 0, 1),
                    (2, 1, 1), (1, 0, 2), (2, 1, 2)]:
        assert w.move(p, r, c) == "continue"
```

```
    for _ in range(100):
        cells = [(r, c) for r in range(4)
                 for c in range(4)]
        random.shuffle(cells)
```

```
# ============================================================
# FOLLOW-UP 1: What changes when the winning length is a
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY PATH: question 176 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 3 of 7

Template fit: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n | asked 250 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 176 of 288

Design Tic-Tac-Toe With K Players

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 174/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: the interview site's own solution

LeetCode match: LC 348 Design Tic-Tac-Toe

Posted on PracHub: 2026-02-12 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/design-tic-tac-toe-with-k-players

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.

QUESTION

- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2,
    1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue',
    'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def move(player, r, c):
    board[r][c] = player
    lines = [row(r), column(c)]
    if r == c:
        lines.append(main_diagonal())
    if r + c == n - 1:
        lines.append(anti_diagonal())
    for line in lines:
        if line.count(player) == n:
            return "WIN"
    return "CONTINUE"
```

TIME	SPACE
O(n) per move	O(n^2)

WHY IT IS SLOW

BRUTE FORCE

Each move rescans up to four lines, O(n). Keeping a counter per player per row, column and diagonal makes each move O(1).

OPTIMAL SOLUTION · Counters per player and per line

TIME	SPACE
O(1) per move	**O(k * n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(m) per move.

FOLLOW-UP 2 Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

FOLLOW-UP 3 The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. **Cost:** O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design Tic-Tac-Toe With K Players
#
# Design and implement a Tic-Tac-Toe game engine. You are
# given: - k players, each with a unique player ID or
# mark. - An n x n board, initially empty. - Players take
# turns placing their own mark on an empty cell. Unlike
# classic Tic-Tac-Toe, a player wins as soon as they form
# **3 consecutive marks** in a straight line, regardless
# of how large n is. A straight line may be horizontal,
# vertical, diagonal down-right, or diagonal down-left.
# Implement a class or set of functions that supports
# making moves and returns the game status after each
# move, such as: - "Game Over" if a move is attempted
# after the game has ended. - "<player> won" when the
# latest move causes that player to have 3 consecutive
# marks. - A non-terminal status such as "Continue" if the
# game should keep going. Your implementation should
# handle invalid moves, such as placing a mark outside the
# board or on an occupied cell.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Win with m in a row, not the full line.
# 2. Undo the last move.
# 3. The board is huge or unbounded.
#
# INTERVIEW NOTES
# Do not look at the board after each move. A win needs n
# marks of the same player on one line, so keep a counter
# per player and per line (each row, each column, the two
# diagonals) and bump the at-most-four lines that the move
# touches. The move wins if any of those counters reaches
# n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))

# ---- main question: main function ----
# BASE QUESTION (optimal): Counters per player and per line
# Idea: rows[p][r], cols[p][c], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n.
# FOLLOW-UP 1: Win with m in a row, not the full line.
# Answer: Counters per line no longer work, because a line
# can hold a run anywhere. After each move look along the
# four directions from that cell, counting same-player
# neighbours up to m in both directions. O(m) per move.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        # FOLLOW-UP 3: a huge or unbounded board
        # only occupied cells are stored (works for huge
        # boards)
        self.cells = {}
        self.history = []
        self.winner = None

    def turn(self):
        # players go 1, 2, ..., k, then 1 again
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
```

```
        if self.winner is not None:
            return "Game Over"
        # an invalid move changes nothing
        if not 1 <= player <= self.k:
            return "Invalid move"
```

```
        if player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n):
            return "Invalid move"
        if (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        # only the four lines through the new mark can win
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc, player)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.winner = player
                return f"{player} won"
        return "Continue"

    # FOLLOW-UP 2: undo the last move
    def undo(self):
        # take the last move back (also reopens a finished
        # game)
        if not self.history:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True

# ---- main question: tests ----
def test_main_question():
    moves = [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1],
             [1, 0, 2], [2, 2, 2]]
    assert process(3, 2, moves) == [
        "Continue", "Continue", "Continue", "Continue",
        "1 won", "Game Over"]
    # invalid moves do not use up the turn
    got = process(3, 2, [[2, 0, 0], [1, 5, 5], [1, 0, 0],
                         [2, 0, 0], [3, 1, 1], [2, 1, 1]])
    bad = "Invalid move"
    ok = "Continue"
    assert got == [bad, bad, ok, bad, bad, ok]
    g = TicTacToe(3, 2)
    for m in moves[:5]:
        g.move(*m)
    assert g.winner == 1 and g.undo() and g.winner is None
    assert g.move(1, 0, 2) == "1 won"
```

```
    # a huge board costs nothing: only taken cells are
    # stored
    big = TicTacToe(10 ** 9, 2)
    assert big.move(1, 10 ** 9 - 1, 5) == "Continue"


# ============================================================
# FOLLOW-UP 1: Win with m in a row, not the full line.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def process(n, k, moves):
    game = TicTacToe(n, k)
    return [game.move(*m) for m in moves]
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, k: int, moves: list[list[int]]) -> li
st[str]:
```

Sample call

```
solution(3, 2, [])
```

Test cases

Input	Expected
`(3, 2, [])`	`[]`
`(5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])`	`['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']`

ALSO REPORTED · 1 more

Tic-Tac-Toe System with K Players and 3-in-a-Row Win | 1Point3Acres | phone-screen, tech-screen | asked 2026-05-18 | Single report

https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.

Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.

Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

Preparation: Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.

Practice local directional win checks with boundary guards. Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY PATH: question 177 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 4 of 7

Template fit: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins | asked 389 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 177 of 288

Infinite Deck Card Duel

ASKED 2025-09-26 | 389 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 186/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 384 Shuffle an Array (related: seeded random simulation)

Frequency: Single report | Last asked: 2025-09-26 | Roles: SWE Intern

QUESTION

Design an InfiniteDeck that can be drawn from forever: each call to draw() returns a uniformly random integer card value in [1, 13] (cards are never exhausted). Make the deck seedable so games are reproducible. Then simulate a duel between two players. In each round, each player draws hand_size cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a draw (nobody scores). Play num_rounds rounds and tally round wins per player. Finally, declare a champion: the player is champion only if they lead the other in round wins by at least 2; otherwise there is no champion. Implement: InfiniteDeck.draw() returning the next card. round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning draw), based on the highest card in each hand. decide_champion(wins1, wins2) -> 1 , 2 , or 0 (0 meaning no champion), applying the "lead by at least 2" rule. play_game(...) tying it together (uses the deck for randomness).

QUESTION

Function signature

```
import random
class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...
    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...
def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...
def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...
def play_game(deck: InfiniteDeck, hand_size: int = 5,
num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Follow-up (as reported): Extend to n players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round. Introduce multiple independent InfiniteDeck instances (or one shared deck with per-card metadata) to support probabilistic house rules.

EXAMPLES

Example 1

Input	`round_winner([3, 7, 2], [7, 1, 9])`
Output	`2`
Why	`Player 2's highest card (9) beats player 1's highest (7).`

Example 2

Input	`round_winner([5, 5], [5, 4])`
Output	`0`
Why	`Both players' highest card is 5, so the round is a draw.`

CONSTRAINTS

- 1 <= hand_size <= 20 , 1 <= num_rounds <= 100000 , card values in [1, 13] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
import random
def brute_game(seed, hand_size, rounds):
    rng = random.Random(seed)
    w1 = w2 = 0
    for _ in range(rounds):
        h1 = [
            rng.randint(1, 13)
            for _ in range(hand_size)
        ]
        h2 = [
            rng.randint(1, 13)
            for _ in range(hand_size)
        ]
        if max(h1) > max(h2):
            w1 += 1
        elif max(h2) > max(h1):
            w2 += 1
    if w1 - w2 >= 2:
        return 1
    return 2 if w2 - w1 >= 2 else 0
```

TIME	SPACE
O(rounds * hand)	O(1)

WHY IT IS SLOW

It repeats the same logic without the classes; it is only here to check that the class version gives the same game.

OPTIMAL SOLUTION · A seedable deck and pure game rules

TIME	SPACE
O(rounds * hand)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Extend it to n players.

Replace the two counters with a dictionary keyed by player id. Each round every player draws a hand; the round goes to the player whose best card is strictly highest, and ties for the top give nobody a point. The champion needs a lead of at least 2 over the second best.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Infinite Deck Card Duel
#
# Design an InfiniteDeck that can be drawn from forever:
# each call to draw() returns a uniformly random integer
# card value in [1, 13] (cards are never exhausted). Make
# the deck seedable so games are reproducible. Then
# simulate a duel between two players. In each round, each
# player draws hand_size cards. The round winner is the
# player holding the single highest card that round; if
# both players' highest cards are equal, the round is a
# draw (nobody scores). Play num_rounds rounds and tally
# round wins per player. Finally, declare a champion: the
# player is champion only if they lead the other in round
# wins by at least 2; otherwise there is no champion.
# Implement: InfiniteDeck.draw() returning the next card.
# round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning
# draw), based on the highest card in each hand.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Extend it to n players.
#
# INTERVIEW NOTES
# Reproducible games need an injectable random source:
# wrap random.Random(seed) in a class so the same seed
# replays the same game. Keep the game logic pure
# (round_winner and decide_champion take plain values),
# which makes it testable without randomness.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random
import random as _r


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
class InfiniteDeck:
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def round_winner(hand1, hand2):
    if max(hand1) == max(hand2):
        return 0
    return 1 if max(hand1) > max(hand2) else 2


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    return 2 if wins2 - wins1 >= 2 else 0


# BASE QUESTION (optimal): A seedable deck and pure game
```

```
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for _ in range(num_rounds):
        h1 = [deck.draw() for _ in range(hand_size)]
        h2 = [deck.draw() for _ in range(hand_size)]
        winner = round_winner(h1, h2)
        if winner:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])
```

```
# ---- main question: tests ----
def test_main_question():
    random.seed(28)
    assert round_winner([3, 7, 2], [7, 1, 9]) == 2
    assert round_winner([5, 5], [5, 4]) == 0
    assert (
        decide_champion(51, 49) == 1
        and decide_champion(50, 49) == 0
        and decide_champion(10, 13) == 2
    )
    for seed in range(40):
        hand, rounds = _r.randint(1, 4), _r.randint(
            1, 12
        )
        assert play_game(
            InfiniteDeck(seed), hand, rounds
        ) == play_game(InfiniteDeck(seed), hand, rounds)


# ========================================================
# FOLLOW-UP 1: Extend it to n players.
# ========================================================

# ---- follow-up 1: function ----
# Answer: Replace the two counters with a dictionary keyed
# by player id. Each round every player draws a hand; the
# round goes to the player whose best card is strictly
# highest, and ties for the top give nobody a point.
def play_game_n(
    deck, players, hand_size=5, num_rounds=100
):
    wins = {p: 0 for p in range(1, players + 1)}
    for _ in range(num_rounds):
        best = {
            p: max(
                deck.draw() for _ in range(hand_size)
            )
            for p in wins
        }
        top = max(best.values())
        leaders = [
            p for p, v in best.items() if v == top
        ]
        if len(leaders) == 1:
            wins[leaders[0]] += 1
```

```
    order = sorted(wins.items(), key=lambda kv: -kv[1])
    if (
        len(order) > 1
        and order[0][1] - order[1][1] >= 2
    ):
        return order[0][0]
    return 0

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert play_game_n(InfiniteDeck(3), 2, 3, 30) in (
        0,
        1,
        2,
    )
    assert play_game_n(InfiniteDeck(3), 4, 3, 30) in (
        0,
        1,
        2,
        3,
        4,
    )


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 178 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 5 of 7

Template fit: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next() | asked 287 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 178 of 288

Implement iterator merging lists with filtering

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 176/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #281 Zigzag Iterator

LeetCode match: LC 281 Zigzag Iterator (+ filter/dedupe) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/implement-iterator-merging-lists-with-filtering

QUESTION

Design an iterator over several lists that takes one element from each list in turn (a zigzag), skipping any value in the blocked set and any value that was already returned. hasNext() says whether another value is available and next() returns it.

Function signature

```
class FilteredZigzag:
    def __init__(self, lists: list, blocked=())
    def hasNext(self) -> bool
    def next(self) -> int
```

EXAMPLES

Example 1

Input	`it = FilteredZigzag([[1, 2, 3], [4, 5, 6, 7], [8, 9]], blocked=[2, 5])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 4, 8, 9, 3, 6, 7]`
Why	`Order of visit: 1, 4, 8, then 2 (blocked), 5 (blocked), 9, then 3, 6, then 7. The blocked values never appear.`

QUESTION

Example 2

Input	`it = FilteredZigzag([[1, 1], [1, 2]])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 2]`
Why	`Repeated values are returned only once.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merged_iterator(lists, blocked):
    seen = set()
    for x in round_robin(lists):
        if x in blocked or x in seen:
            continue
        seen.add(x)
        yield x
```

TIME	SPACE
O(total)	O(total)

WHY IT IS SLOW

It reads everything up front and keeps it all in memory, so it is not lazy and never works on a long or unbounded source. A lookahead value per iterator computes only the next valid item.

OPTIMAL SOLUTION · Lazy iterator with one pointer per list and a lookahead

TIME	SPACE
O(1) amortized per value	O(number of lists + distinct ids yielded)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

FOLLOW-UP 2 Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

FOLLOW-UP 3 The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. **Cost:** O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false suppressions.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement iterator merging lists with
# filtering
#
# Design an iterator over several lists that takes one
# element from each list in turn (a zigzag), skipping any
# value in the blocked set and any value that was already
# returned. hasNext() says whether another value is
# available and next() returns it.
#
# Signature:
#   class FilteredZigzag:
#       def __init__(self, lists: list, blocked=())
#       def hasNext(self) -> bool
#       def next(self) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are sorted and the output must stay sorted.
# 2. Read the lists one after another, not interleaved.
# 3. The seen set grows without bound.
#
# INTERVIEW NOTES
# An iterator does its work lazily: hasNext() must be able
# to answer without consuming. So keep a lookahead, the
# next valid value already computed, and advance to the
# one after it when next() is called. Filtering and de-
# duplication live in the 'find the next valid value' step
# so both hasNext and next stay trivial.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from itertools import chain


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Lazy iterator with one pointer
```

```
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
class FilteredZigzag:
    """Take one item from each list in turn; skip blocked
    ids and ids already returned. Has hasNext / next."""

    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.pos = [0] * len(lists)
        self.turn = 0
        self.seen = set()
        # one item of lookahead makes hasNext easy
        self.ahead = None
        self.advance()

    def advance(self):
        self.ahead = None
        misses = 0
        # stop when every list had nothing left in a full
        # round
        while misses < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.pos[i] >= len(self.lists[i]):
                misses += 1
                continue
            value = self.lists[i][self.pos[i]]
            self.pos[i] += 1
            misses = 0
            blocked = value in self.blocked
            skip = blocked or value in self.seen
            if not skip:
                self.ahead = value
                return

    def has_next(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value

# ---- main question: tests ----
def test_main_question():
    lists = [[1, 2, 3], [2, 4], [5]]
    it = FilteredZigzag(lists, blocked=[4])
    out = []
    while it.has_next():
        out.append(it.next())
    # 1, 2, 5, then 2 (seen), 3, 4 (blocked)
    assert out == [1, 2, 5, 3]
    empty = FilteredZigzag([[], []])
    assert not empty.has_next()


# ============================================================
# FOLLOW-UP 1: The lists are sorted and the output must
# ============================================================

# ---- follow-up 1: function ----
# Answer: Replace the round robin with a k-way merge on a
# heap (heapq.merge), and drop a value equal to the
# previous output instead of keeping a seen set: memory
# O(k).
def merge_sorted(lists, blocked=()):
    # sorted lists, sorted output: a k-way merge; equal
    # values
    # come out together, so remember only the previous one
    blocked, last = set(blocked), object()
```

```
    for value in heapq.merge(*lists):
        if value in blocked or value == last:
            continue
        last = value
        yield value

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert list(merge_sorted([[1, 3, 5], [1, 2, 5]],
                             blocked=[3])) == [1, 2, 5]


# ============================================================
# FOLLOW-UP 2: Read the lists one after another, not
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Chain the lists with a generator; the same
# blocked/seen filtering applies. Generators give hasNext-
# style behaviour via a one-item lookahead.
def read_in_turn(lists, blocked=()):
    # one list after another, with the same filtering
    blocked, seen = set(blocked), set()
    for value in chain.from_iterable(lists):
        if value not in blocked and value not in seen:
            seen.add(value)
            yield value

# ---- follow-up 2: tests ----
def test_follow_up_2():
    chained = read_in_turn([[1, 2], [2, 3]], [1])
    assert list(chained) == [2, 3]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
solution([3, 1, 2, 3], [2, 4, 1, 5], {4})
```

Test cases

Input	Expected
([3, 1, 2, 3], [2, 4, 1, 5], {4})	[3, 1, 2, 5]
([], [1, 1], set())	[1]

STUDY PATH: question 179 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 6 of 7

Template fit: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor | asked 409 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 179 of 288

Design an elevator control system

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 281/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (OOP / low-level design)

Posted on PracHub: 2025-09-06 (updated 2026-10-07)

Original: https://prachub.com/interview-questions/design-an-elevator-control-system-2

QUESTION

Design the control logic for a multi-elevator building. A Request(floor, direction) is a call from a floor (direction +1 for up, -1 for down). Each Elevator keeps its current floor, direction and a set of stops. cost(request) estimates how expensive it is for this car to take the request (the distance, plus a detour penalty when the car is heading away); the dispatcher gives the request to the cheapest car. step() moves the car one floor towards its stops and returns the floor it served, if any.

Function signature

class Elevator:
  def __init__(self, car_id, floor=0)
  def cost(self, req: Request) -> float
  def step(self) -> int | None

EXAMPLES

Example 1

Input	`e = Elevator(1, floor=0)`
	`e.stops.update({3, 5})`
	`[e.step() for _ in range(6)]`

Output	`[e.step() for _ in range(6)] ->` `[null, null, null, 3, null, 5]`
Why	`The car climbs one floor per step and stops at floors 3 and 5 (served when it is on the floor at the start of a step).`

QUESTION

Example 2

Input	`idle = Elevator(2, floor=7)`
	`busy = Elevator(3, floor=2)`
	`busy.direction = 1`
	`busy.stops.update({9})`
	`idle.cost(Request(4, -1))`
	`busy.cost(Request(4, 1))`
	`busy.cost(Request(4, -1))`

Output	`idle.cost(Request(4, -1)) -> 3`
	`busy.cost(Request(4, 1)) -> 2`
	`busy.cost(Request(4, -1)) -> 14`

Why	`An idle car costs its distance (3). A car already going up that will pass floor 4 going the same way costs its distance (2); going the other way it must finish its run first, so it costs much more.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
requests = deque()  # arrival order
def run():
    while requests:
        req = requests.popleft()
        car = nearest_idle_car(req.floor)
        car.go_to(req.floor)
        car.go_to(req.destination)
```

TIME	SPACE
O(requests * cars)	O(requests)

WHY IT IS SLOW

Cars zig-zag across the building and requests wait behind far-away ones, so total travel and waiting are high. Serving stops in sweep order (SCAN) and assigning by cost cuts both.

OPTIMAL SOLUTION · Elevator (SCAN) per car and cost-based assignment of calls

TIME	SPACE
O(cars) per call, O(1) per car per step	O(stops)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

FOLLOW-UP 2 Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

FOLLOW-UP 3 Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

FOLLOW-UP 4 Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design an elevator control system
#
# Design the control logic for a multi-elevator building.
# A Request(floor, direction) is a call from a floor
# (direction +1 for up, -1 for down). Each Elevator keeps
# its current floor, direction and a set of stops.
# cost(request) estimates how expensive it is for this car
# to take the request (the distance, plus a detour penalty
# when the car is heading away); the dispatcher gives the
# request to the cheapest car. step() moves the car one
# floor towards its stops and returns the floor it served,
# if any.
#
# Signature:
#   class Elevator:
#       def __init__(self, car_id, floor=0)
#       def cost(self, req: Request) -> float
#       def step(self) -> int | None
#
# FOLLOW-UPS (answered below, in this order):
# 1. Peak traffic (morning lobby rush, evening exodus).
# 2. Fairness and starvation.
# 3. Safety and fault tolerance.
# 4. Morning rush: idle cars should wait at the lobby.
#
# INTERVIEW NOTES
# Separate three things: the data (Request), one car's
# behaviour (Elevator: floor, direction, stops,
# door/state) and the policy that decides which car takes
# a call (Controller). Say the scheduling rule out loud: a
# car keeps going in its direction serving stops, then
# reverses (SCAN/LOOK), which avoids the back-and-forth of
# first-come-first-served. Add a step() so the whole thing
# can be simulated and tested.

# ============================================================
```

```
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")


class Request:
    def __init__(self, floor, direction):
        # direction: +1 up, -1 down
        self.floor, self.direction = floor, direction

# ---- main question: main function ----
# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor = car_id, floor
        # 0 = idle, +1 = going up, -1 = going down
        self.direction = 0
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        # how bad is it to give this car the request?
        if not self.in_service:
            return INF
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        # on the way: the car is moving toward the floor
        # and
        # the call goes the same way
        gap = req.floor - self.floor
        ahead = gap * self.direction >= 0
        if ahead and req.direction == self.direction:
            return dist
        # otherwise it finishes its sweep first
        span = max(self.stops) - min(self.stops) + 1
        return dist + 2 * span + 10

    def step(self):
```

```
        # one tick: serve a stop here, then move one floor
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        # keep the direction while there are stops ahead
        # (SCAN)
        if (self.direction >= 0 and up) or (
                self.direction < 0 and not down):
```

```
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening
# exodus).
# Answer: Add modes: during the morning rush park idle
# cars in the lobby and let them go up with a full load;
# in the evening do the opposite.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars, self.lobby = cars, lobby

    def assign(self, req):
        # the cheapest in-service car takes the call
        car = min(self.cars, key=lambda c: c.cost(req))
        if car.cost(req) == INF:
            return None
        car.stops.add(req.floor)
        return car.id

    def tick(self):
```

```
        return {c.id: c.step() for c in self.cars}

    # FOLLOW-UP 3: a failed car hands its stops over
    def fail(self, car_id):
        # a broken car: take it out and hand its stops to
        # others
        car = next(c for c in self.cars if c.id == car_id)
        car.in_service = False
        orphans, car.stops = car.stops, set()
        for floor in orphans:
            self.assign(Request(floor, 0))

    # FOLLOW-UP 1, 4: morning rush: idle cars wait in the
    # lobby
    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and not car.stops:
                car.stops.add(self.lobby)

# ---- main question: tests ----
def test_main_question():
    car = Elevator(1, floor=0)
    car.stops.update({3, 5})
    assert [car.step() for _ in range(6)] == [None, None,
None, 3, None, 5]


# ============================================================
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 180 of 288 | topic 22 of 43: Simulate a machine: a class with state and one branch per kind of event | 7 of 7

Template fit: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end | asked 169 days ago

HARD | IN APP LC 289 | PracHub | Technical Screen | 180 of 288

Simulate In-Place Cellular State Updates

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 23.4 | 30 DAYS 0 | PRIORITY 164/288

ABOUT THIS QUESTION

Topic: Simulate a machine: a class with state and one branch per kind of event

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #289 Game of Life

LeetCode match: LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-07)

Original: https://prachub.com/coding-questions/simulate-in-place-cellular-state-updates

QUESTION

Conway's Game of Life on a grid of 0 (dead) and 1 (alive) cells. A live cell with 2 or 3 live neighbours (of its 8 neighbours) stays alive, a dead cell with exactly 3 live neighbours becomes alive, and every other cell is dead in the next generation. All cells change at the same time. Do it in place with O(1) extra space; with wrap=True the grid wraps around its edges.

Function signature

def step_in_place(board: list, wrap=False) -> list

EXAMPLES

Example 1	
Input	`[[0, 1, 0], [0, 1, 0], [0, 1, 0]]`
Output	`[[0, 0, 0], [1, 1, 1], [0, 0, 0]]`
Why	The vertical blinker turns into a horizontal one.

Example 2	
Input	`[[1, 1], [1, 1]]`
Output	`[[1, 1], [1, 1]]`
Why	A 2x2 block is stable: every cell has 3 live neighbours.

Example 3	
Input	`[[0, 0, 0], [0, 1, 0], [0, 0, 0]]`
Output	`[[0, 0, 0], [0, 0, 0], [0, 0, 0]]`
Why	A lonely cell dies of isolation.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def step(board):
    old = [row[:] for row in board]
    for r, c in all_cells(old):
        live = live_neighbours(old, r, c)
        alive = old[r][c] == 1
        if alive and live in (2, 3):
            board[r][c] = 1
        elif not alive and live == 3:
            board[r][c] = 1
        else:
            board[r][c] = 0
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Time O(m * n) is already optimal; the cost is a second full board, O(m * n) extra space. Storing the next state in a spare bit of each cell removes the copy.

OPTIMAL SOLUTION · Two-bit encoding in the same board

TIME	SPACE
O(m * n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The board is infinite, or huge and mostly empty.

Store only the live cells in a set. A cell's next state depends on its live neighbor count, so count how many live neighbors every candidate cell has with a Counter and keep cells with 3, or with 2 if already live. Work is proportional to the number of live cells.

FOLLOW-UP 2 Why is bit 0 used when counting neighbors?

Neighbors that were already processed in this pass carry their next state in bit 1. Masking with & 1 always reads the original generation, which is the whole point of the encoding.

FOLLOW-UP 3 Cells wrap around the edges (a torus).

Replace the bounds check with modular indexing: (r + dr) % m and (c + dc) % n. Be careful on very small boards where the same neighbor can be reached through two offsets.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Simulate In-Place Cellular State Updates
#
# Conway's Game of Life on a grid of 0 (dead) and 1
# (alive) cells. A live cell with 2 or 3 live neighbours
# (of its 8 neighbours) stays alive, a dead cell with
# exactly 3 live neighbours becomes alive, and every other
# cell is dead in the next generation. All cells change at
# the same time. Do it in place with O(1) extra space;
# with wrap=True the grid wraps around its edges.
#
# Signature:
#   def step_in_place(board: list, wrap=False) -> list
#
# Examples:
# [[0, 1, 0], [0, 1, 0], [0, 1, 0]]  ->  [[0, 0, 0], [1,
#     1, 1], [0, 0, 0]]
# [[1, 1], [1, 1]]  ->  [[1, 1], [1, 1]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The board is infinite, or huge and mostly empty.
# 2. Why is bit 0 used when counting neighbors?
# 3. Cells wrap around the edges (a torus).
#
# INTERVIEW NOTES
# 'Update every cell at once, in place' means a cell's new
# value must not disturb its neighbors' reads. The trick
# is to store both the old and the new state in one cell
# (two bits), then decode in a second pass. Whenever you
# need old and new values together with no extra grid,
# think 'pack two states into one number'.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
AROUND = [(dr, dc) for dr in (-1, 0, 1)
          for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]


def live_neighbors(board, r, c, wrap=False):
    m, n = len(board), len(board[0])
    count = 0
    for dr, dc in AROUND:
        a, b = r + dr, c + dc
        if wrap:
            # a torus: the edges are glued together
            a, b = a % m, b % n
        elif not (0 <= a < m and 0 <= b < n):
            continue
        # bit 0 is the OLD state, even for updated cells
        count += board[a][b] & 1
    return count

# ---- main question: main function ----
# BASE QUESTION (optimal): Two-bit encoding in the same
# board
# Idea: Bit 0 holds the current state and bit 1 holds the
# next state. First pass: count neighbors using only bit 0
# (value & 1) and set bit 1 by the rules.
def step_in_place(board, wrap=False):
    m, n = len(board), len(board[0])
    for r in range(m):
        for c in range(n):
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            # bit 1 stores the NEW state
            if live == 3 or (alive and live == 2):
                board[r][c] |= 2
    for r in range(m):
        for c in range(n):
            # shift the new state into place
            board[r][c] >>= 1
    return board

# ---- main question: tests ----
def test_main_question():
    assert step_in_place([[0, 1, 0], [0, 1, 0], [0, 1, 0]]
) == [[0, 0, 0], [1, 1, 1], [0, 0, 0]]
    assert step_in_place([[1, 1], [1, 1]]) == [[1, 1], [1,
 1]]
    assert step_in_place([[0, 0, 0], [0, 1, 0], [0, 0, 0]]
) == [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
# ============================================================
# FOLLOW-UP 1: The board is infinite, or huge and mostly
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store only the live cells in a set. A cell's
# next state depends on its live neighbor count, so count
# how many live neighbors every candidate cell has with a
# Counter and keep cells with 3, or with 2 if already
# live.
def step_sparse(live):
    # infinite or huge board: only the live cells are
    # stored
    counts = Counter()
    for r, c in live:
        for dr, dc in AROUND:
            counts[(r + dr, c + dc)] += 1
    return {cell for cell, k in counts.items()
            if k == 3 or (k == 2 and cell in live)}


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]])
```

Test cases

Input	Expected
([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]],)	[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
([[1, 1], [1, 1]],)	[[1, 1], [1, 1]]

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 23 OF 43

Top-K by frequency: Counter plus nsmallest

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

6
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Top-K by frequency: Counter plus nsmallest | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M	181. Top-K Frequent in Large Logs	1Point3Acres	2026-07-19
M	182. Rearrange Tasks With Cooldown	PracHub	2026-06-03
M	183. Implement frequency + distance top-K queries	PracHub	2026-02-12
M	184. Solve chat, grid paths, and car rentals	PracHub	2025-12-02
M	185. Top-K Chatters by Word Count	ColorStack	2025-09-21
M	186. Find top/bottom-k words in list or stream	PracHub	2026-01-11

CODE TEMPLATE | GROUP 23 OF 43 | 6 QUESTIONS, ONE PATTERN

Top-K by frequency: Counter plus nsmallest

The one idea these questions share

Count with a Counter, then ask heapq.nsmallest for the k best using a key of (negative count, tie-break). That is the whole solution for almost every top-K question; only what you count and the tie-break change.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq
from collections import Counter


def top_k(items, k, key=lambda x: x):
    counts = Counter(key(x) for x in items)          # CHANGE 1: what you count
    return heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))   # CHANGE 2: the tie-break
```

2. Helper functions that usually get written next to it

```
def bottom_k(items, k):
    counts = Counter(items)
    return heapq.nsmallest(k, counts, key=lambda w: (counts[w], w))


def counts_of(items):
    return dict(Counter(items))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
181	Top-K Frequent in Large Logs	count the first word of every line (the speaker); return the names of the k most frequent
182	Rearrange Tasks With Cooldown	only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)
183	Implement frequency + distance top-K queries	count the point tuples; the tie-break is distance from the origin, then x, then y
184	Solve chat, grid paths, and car rentals	the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment
185	Top-K Chatters by Word Count	count len(body.split()) per speaker instead of lines; ties by name
186	Find top/bottom-k words in list or stream	the same Counter; run nsmallest once with -count and once with +count for the bottom-k

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 181 of 288 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 1 of 6

Template fit: count the first word of every line (the speaker); return the names of the k most frequent | asked 93 days ago | frequency Medium

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 181 of 288

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 25/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

A chat log is a list of lines, each starting with the speaker's name ('A says hi'). Return the k speakers who appear most often; ties are broken alphabetically by name.

Function signature

def top_k_speakers(chat_log: list, k: int) -> list

EXAMPLES

Example 1	
Input	`['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 2`
Output	`['A', 'B']`
Why	`A spoke 3 times, B and C once each; B comes before C alphabetically.`

Example 2	
Input	`['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 3`
Output	`['A', 'B', 'C']`
Why	`Asking for 3 returns everyone, most frequent first.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d)	O(d)

BRUTE FORCE

WHY IT IS SLOW

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Top-K Frequent in Large Logs
#
# A chat log is a list of lines, each starting with the
# speaker's name ('A says hi'). Return the k speakers who
# appear most often; ties are broken alphabetically by
# name.
#
# Signature:
#   def top_k_speakers(chat_log: list, k: int) -> list
#
# Examples:
# ['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A
#     says yo'], 2  ->  ['A', 'B']
# ['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A
#     says yo'], 3  ->  ['A', 'B', 'C']
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you do it across many machines when the log
# does not fit on one?
# 2. What if the log arrives as an unbounded stream?
# 3. Which approach would you choose and why?
# 4. The log is a stream and memory is limited.
#
# INTERVIEW NOTES
# Count the speakers first; a log line is just a way to
# read the user id. Then you only need the K largest
# counts: a heap of size K does that without sorting all
# users. For logs spread over many machines the local top
# K are not enough on their own, so say what extra
# information you send.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])

# ---- main question: main function ----
# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]

# ---- main question: tests ----
def test_main_question():
    log = ["A says hi", "B says hi", "A says bye",
           "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]


# ============================================================
# FOLLOW-UP 1: How do you do it across many machines when
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count per shard and send each shard's local
# top-K to a coordinator.
def merge_shards(shard_logs, k):
    # exact answer: add the full counts of every shard
```

```
    total = Counter()
    for log in shard_logs:
        total.update(speaker(line) for line in log)
    best = heapq.nsmallest(k, total.items(), key=rank_key)
    return [user for user, _ in best]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]
    assert merge_shards([s1, s2], 1) == ["B"]


# ============================================================
# FOLLOW-UP 2: What if the log arrives as an unbounded
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use the Misra-Gries or Space-Saving summary with
# m counters. Any user with more than n / (m + 1) messages
# is guaranteed to be in the summary, and a second pass
# (or a count-min sketch for the estimate) can confirm the
# real counts.
# FOLLOW-UP 4: The log is a stream and memory is limited.
# Answer: Misra-Gries keeps m counters; any user with more
# than n / (m + 1) messages is guaranteed to remain in the
# summary (checked above). A second pass, or a count-min
# sketch, then gives the real counts of the survivors.
class MisraGries:
    """m counters: a user with over n / (m + 1) of the
    n messages always stays in the summary."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
```

```
        else:
            # no room: every counter pays one
            for u in list(self.counts):
                self.counts[u] -= 1
```

```
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    mg = MisraGries(2)
    for u in "AAABBC":
        mg.add(u)
    # A has 3 of 6 messages (> 6 / 3), so it must survive
    assert "A" in mg.candidates()


# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: The log is a stream and memory is limited.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

STUDY PATH: question 182 of 288 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 2 of 6

Template fit: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count) | asked 139 days ago

MEDIUM | IN APP LC 621 | PracHub | Technical Screen | 182 of 288

Rearrange Tasks With Cooldown

POSTED 2026-06-03 | 139 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 106/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #621 Task Scheduler

LeetCode match: LC 621 Task Scheduler (inferred from overview)

Posted on PracHub: 2026-06-03 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/rearrange-tasks-with-cooldown

QUESTION

You are given a list of tasks (letters; the same letter means the same kind of task). Every task takes one unit of time, and two tasks of the same kind must have at least n other tasks or idle units between them (the cooldown). The CPU may stay idle. Return the least number of time units needed to finish all the tasks.

Function signature

def least_interval(tasks: list, n: int) -> int

EXAMPLES

Example 1	
Input	`['A', 'A', 'A', 'B', 'B', 'B'], 2`
Output	`8`
Why	`A B idle A B idle A B: three A's and three B's with a cooldown of 2 take 8 units.`

Example 2	
Input	`['A', 'A', 'A', 'B', 'B', 'B'], 0`
Output	`6`
Why	`No cooldown: just run the 6 tasks back to back.`

Example 3	
Input	`['A', 'B', 'C'], 5`
Output	`3`
Why	`All tasks differ, so no cooldown is ever needed.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
    while any(left.values()):
        ready = [x for x in left if left[x]
                 and free_at.get(x, 0) <= t]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t
```

TIME	SPACE
O(T log k) where T is the answer	O(k)

WHY IT IS SLOW

It simulates every slot, including long idle stretches, O(T log k). The answer follows from the most frequent task: max(len(tasks), (maxCount - 1) * (n + 1) + number of tasks with that count).

OPTIMAL SOLUTION · Closed form from the most frequent task

TIME	SPACE
O(len(tasks))	O(1) for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

FOLLOW-UP 2 Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

FOLLOW-UP 3 Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Rearrange Tasks With Cooldown
#
# You are given a list of tasks (letters; the same letter
# means the same kind of task). Every task takes one unit
# of time, and two tasks of the same kind must have at
# least n other tasks or idle units between them (the
# cooldown). The CPU may stay idle. Return the least
# number of time units needed to finish all the tasks.
#
# Signature:
#   def least_interval(tasks: list, n: int) -> int
#
# Examples:
# ['A', 'A', 'A', 'B', 'B', 'B'], 2  ->  8
# ['A', 'A', 'A', 'B', 'B', 'B'], 0  ->  6
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print an actual schedule, not just the time.
# 2. Different cooldowns per task.
# 3. Tasks have priorities or deadlines.
#
# INTERVIEW NOTES
# The most frequent task is the bottleneck. Lay it out
# with gaps of n between its copies; every other task can
# fill gaps. Total length is the larger of 'that skeleton'
# and 'just the number of tasks' (when there are so many
# tasks the gaps are all filled and no idle time is
# needed). The skeleton has (maxcount - 1) full blocks of
# size n + 1 plus one last block holding every task tied
# for the maximum.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter, deque


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let m be the highest frequency and k the number of
# tasks that have that frequency. The skeleton needs (m -
# 1) * (n + 1) + k slots.
def least_interval(tasks, n):
    # closed form: the most frequent task sets the frame
    counts = Counter(tasks).values()
    most = max(counts, default=0)
    # how many tasks are tied for the most
    tied = sum(1 for c in counts if c == most)
    if not tasks:
        return 0
    # (most - 1) full frames of n + 1 slots, plus the last
    # row
    return max(len(tasks), (most - 1) * (n + 1) + tied)

# ---- main question: tests ----
def test_main_question():
    tasks = list("AAABBB")
    assert least_interval(tasks, 2) == 8
    assert least_interval(tasks, 0) == 6
    assert least_interval([], 3) == 0
    # many different tasks: no idle time needed
    assert least_interval(list("ABCDEFG"), 2) == 7


# ============================================================
# FOLLOW-UP 1: Print an actual schedule, not just the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use the heap-with-cooldown-queue construction:
# run the available task with the most copies left, park
# it for n slots, and emit an idle marker when nothing is
# available. Or rearrange a string so equal letters are at
# least k apart.
def schedule(tasks, n):
    # an actual schedule, with "idle" where nothing may
    # run
    heap = [(-c, t) for t, c in Counter(tasks).items()]
    heapq.heapify(heap)
    # tasks cooling down: (time ready, -left, task)
    cooling, out, now = deque(), [], 0
    while heap or cooling:
        # bring back tasks whose cooldown is over
        while cooling and cooling[0][0] <= now:
            _, left, task = cooling.popleft()
            heapq.heappush(heap, (left, task))
        if heap:
            # run the task with the most copies left
            left, task = heapq.heappop(heap)
            out.append(task)
            if left + 1 < 0:
                ready = now + n + 1
                cooling.append((ready, left + 1, task))
        else:
            out.append("idle")
        now += 1
    return out

# ---- follow-up 1: tests ----
def test_follow_up_1():
    tasks = list("AAABBB")
    plan = schedule(tasks, 2)
    assert len(plan) == 8 and valid(plan, 2)
    assert plan.count("idle") == 2
    for t, n in ((list("AAAABBC"), 3), (list("AB"), 5)):
        plan = schedule(t, n)
        assert valid(plan, n)
        assert len(plan) == least_interval(t, n)


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def valid(plan, n):
    # equal tasks must be more than n slots apart
    last = {}
    for i, task in enumerate(plan):
        if task == "idle":
            continue
```

```
        if task in last and i - last[task] <= n:
            return False
        last[task] = i
    return True
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(tasks: list[str], cooldown: int) -> str:
```

Sample call

```
solution(['A', 'A', 'A', 'B', 'B', 'C'], 2)
```

Test cases

Input	Expected
`(['A', 'A', 'A', 'B', 'B', 'C'], 2)`	`'ABCAB_A'`
`(['A', 'A', 'A', 'B'], 2)`	`'AB_A__A'`

ALSO REPORTED · 1 more

Design a task scheduler in Python. | Exponent

https://www.aced.io/questions/3504/design-task-scheduler-python

STUDY PATH: question 183 of 288 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 3 of 6

Template fit: count the point tuples; the tie-break is distance from the origin, then x, then y | asked 250 days ago

MEDIUM | PracHub | Technical Screen | 183 of 288

★ TOP 75: rank 28 in the Medium and Hard priority list

★ Implement frequency + distance top-K queries

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 24/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #347 Top K Frequent Elements, LeetCode #973 K Closest Points to Origin

LeetCode match: LC 347 Top K Frequent Elements + LC 973 K Closest Points to Origin

Posted on PracHub: 2026-02-12 (updated 2026-04-28)

Original: https://prachub.com/coding-questions/implement-frequency-plus-distance-top-k-queries

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **x** ascending, then **y** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **k**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

QUESTION

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1]
    appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]],
    3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each
    appear twice. Their squared distances are 4, 5,
    and 5, so [0,2] comes first. Between [1,2] and
    [2,1], x is smaller for [1,2].
```

QUESTION

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like `(-frequency, distance, x, y)`.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either `['set', key, value, timestamp]` or `['getClosest', key, timestamp]`. For every `getClosest` operation, return the stored value whose timestamp is closest to the query tim

BRUTE FORCE · Q1: sort all distinct points by the full key

BRUTE FORCE · PSEUDO-CODE

```
def top_k_points(points, k):
    counts = Counter(map(tuple, points))
    def rank(p):
        x, y = p
        return (-counts[p], x * x + y * y, x, y)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct points, O(d log d); a size-K heap with the same key is O(d log k).

BRUTE FORCE · Q2: scan every timestamp of the key

BRUTE FORCE · PSEUDO-CODE

```
def get_closest(key, t):
    best = None
    for ts, value in store[key]:
        gap = abs(ts - t)
        if best is None or (gap, ts) < best[:2]:
            best = (gap, ts, value)
    return best[2]
```

TIME	SPACE
O(m) per query	O(total entries)

WHY IT IS SLOW

Linear in the number of stored timestamps per query. Timestamps arrive in order, so binary search on the sorted list finds the neighbours in O(log m).

OPTIMAL SOLUTION · Q1: heap with the full key

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: parallel sorted lists and bisect

TIME	SPACE
O(1) per set, O(log m) per getClosest	O(total entries)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log k) per query), or maintain a sorted structure updated on every add if queries dominate.

FOLLOW-UP 2 Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

FOLLOW-UP 3 Q2 returns the closest value at or before the timestamp (LeetCode 981).

Use bisect_right(timestamps, ts) - 1 and return that value or an empty string if the index is negative; there is no tie question.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement frequency + distance top-K
# queries
#
# You will implement solutions for two coding interview
# questions. ## Question 1: Return the top-K points by
# frequency (with distance tie-break) You are given an
# array points of 2D integer points, where each point is
# represented as [x, y]. The array may contain duplicates
# (the same [x, y] can appear multiple times). Define: -
# freq(p) = number of times point p appears in the array.
# - dist(p) = squared Euclidean distance to the origin,
# x^2 + y^2. Return **k distinct points** sorted by the
# following priority: 1. Higher frequency first (freq
# descending) 2. If frequencies tie, smaller distance
# first (dist ascending) 3. If still tied, sort by x
```

```
# ascending, then y ascending ### Input - points: list of
# length n, each element [x, y] - k: integer, 1 <= k <= (#
# of distinct points) ### Output - A list of k distinct
# points following the ranking above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1 with points arriving over time.
# 2. Q2 with timestamps arriving out of order.
# 3. Q2 returns the closest value at or before the
# timestamp (LeetCode 981).
#
# INTERVIEW NOTES
# Q1 is a ranking: say the full sort key out loud, count
# with a hash map, and then you only need the K best
# (heap), not a full sort. Q2 keeps each key's timestamps
# in a sorted list, so the nearest one is next to the
# insertion point: binary search, then compare the two
# neighbours.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from bisect import bisect_left, bisect_right, insort
from collections import Counter, defaultdict
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Q1: heap with the full key
# Idea: Count with a Counter, then heapq.nsmallest(k, ...)
# with the key (-frequency, squared distance, x, y).
# Squared distances avoid square roots and are exact for
# integers.
def top_points(points, k):
    # more frequent first, then closer to the origin, then
    # x, y
    counts = Counter(map(tuple, points))

    def rank(p):
        dist = p[0] ** 2 + p[1] ** 2
        return (-counts[p], dist, p[0], p[1])

    best = heapq.nsmallest(k, counts, key=rank)
    return [list(p) for p in best]


# PART 2: closest timestamp value
class TimeStore:
    """set(key, value, ts) and closest-time reads."""

    def __init__(self):
        # key -> sorted [(timestamp, value)]
        self.data = defaultdict(list)

    def set(self, key, value, ts):
        # insort also handles timestamps arriving out of
        # order
        insort(self.data[key], (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
        if not items:
            return ""
        # the first stored time at or after ts, and the
        # one before
        i = bisect_left(items, (ts,))
        options = items[max(0, i - 1):i + 1]
        # closest wins; on a tie the smaller timestamp
        # wins
        def gap(item):
            return (abs(item[0] - ts), item[0])
        best = min(options, key=gap)
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value not after ts
        items = self.data.get(key, [])
        i = bisect_right(items, (ts, chr(0x10FFFF))) - 1
        return items[i][1] if i >= 0 else ""

# ---- main question: tests ----
def test_main_question():
    pts = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]
    # (1,1) and (2,0) appear twice; (2,0) is farther? 4 vs
    # 2
    assert top_points(pts, 2) == [[1, 1], [2, 0]]
    four = top_points(pts, 4)
    assert four == [[1, 1], [2, 0], [0, 2], [3, 3]]
    assert top_points([[5, 5]], 1) == [[5, 5]]


# ============================================================
# FOLLOW-UP 1: Q1 with points arriving over time.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep a running Counter and compute the top K on
# demand with a heap (O(d log k) per query), or maintain a
# sorted structure updated on every add if queries
# dominate.
class TopPointsStream:
    # follow-up: points arrive one at a time. The counts
    # are
    # kept as points arrive, so top_k never recounts
    # history.
    def __init__(self):
        self.counts = Counter()

    def add(self, x, y):
        self.counts[(x, y)] += 1

    def top_k(self, k):
        def rank(p):
            dist = p[0] ** 2 + p[1] ** 2
            return (-self.counts[p], dist, p[0], p[1])

        best = heapq.nsmallest(k, self.counts, key=rank)
        return [list(p) for p in best]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    pts = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]
    live = TopPointsStream()
    for x, y in pts:
        live.add(x, y)
    assert live.top_k(2) == [[1, 1], [2, 0]]
    live.add(3, 3)
    live.add(3, 3)
    live.add(3, 3)
    # (3, 3) is now the most frequent point
    assert live.top_k(1) == [[3, 3]]
    s = TimeStore()
    for ts, v in ((2, "x"), (10, "y"), (15, "z")):
```

```
        s.set("a", v, ts)
    assert s.get_closest("a", 11) == "y"
    assert s.get_closest("a", 14) == "z"
    # a tie: 10 and 15 are both 2.5 away from 12.5, 12 ->
    # 10
    assert s.get_closest("a", 12) == "y"
    assert s.get_closest("b", 1) == ""
    assert s.get_closest("a", 100) == "z"
    assert s.get_closest("a", -5) == "x"
    assert s.get_at_or_before("a", 14) == "y"
    assert s.get_at_or_before("a", 1) == ""
    s.set("a", "late", 5)
    assert s.get_closest("a", 6) == "late"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(points: list[list[int]], k: int) -> list[list
[int]]:
```

Sample call

```
solution([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
```

Test cases

Input	Expected
([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)	[[2, 2], [1, 1]]
([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)	[[0, 2], [1, 2], [2, 1]]

STUDY PATH: question 184 of 288 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 4 of 6

Template fit: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment | asked 322 days ago

MEDIUM | PracHub | Technical Screen | 184 of 288

★ TOP 75: rank 70 in the Medium and Hard priority list

★ Solve chat, grid paths, and car rentals

POSTED 2025-12-02 | 322 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 50/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words, LeetCode #62 Unique Paths, LeetCode #253 Meeting Rooms II

LeetCode match: LC 692 Top K Frequent Words + LC 62/63 Unique Paths + LC 253

Posted on PracHub: 2025-12-02 (updated 2026-05-02)

Original: https://prachub.com/interview-questions/solve-chat-grid-paths-and-car-rentals

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **k** (1 <= k <= number of distinct users).

Output:

- Return a list of **k** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **k** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **k** is much smaller than the number of distinct users. Design and implement a function, for example:

QUESTION

- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:
- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

QUESTION

Tasks: 1.
Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.
Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.
Output:

- An integer `C` = minimal number of cars needed.
- A mapping from each `rental_id` to a `car_id` in the range `1..C`, such that no two rentals assigned to the same `car_id` overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals `[s1, e1)` and `[s2, e2)` overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if `e1 == s2` they do **not** over

EXAMPLES

Example 1

Input	`['ann', 'bob', 'ann', 'cy', 'bob', 'ann'], 2`
Output	`['ann', 'bob']`
Why	`Task 1: ann sent 3 messages, bob 2, cy 1; the top 2 are ann and bob.`

Example 2

Input	`3, 3`
Output	`2`
Why	`Task 2: paths over 3 columns of a 3-row grid that start and end in the bottom row, moving up-right, right or down-right.`

Example 3

Input	`[(1, 0, 5), (2, 1, 4), (3, 5, 8)]`
Output	`(2, {1: 0, 2: 1, 3: 1})`
Why	`Task 3: rentals are (id, pickup, return). Rental 3 starts at 5, after rental 2 returned its car at 4, so it reuses that car: 2 cars in total.`

BRUTE FORCE · Q1: sort all users by (count, user id)

BRUTE FORCE · PSEUDO-CODE

```
def top_k_users(log, k):
    counts = Counter(msg.user for msg in log)
    def rank(u):
        return (-counts[u], u)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

BRUTE FORCE · Q1: sort all users by (count, user id)

Sorting all d distinct users costs O(d log d) even though only K are needed; a heap of size K costs O(d log k).

BRUTE FORCE · Q2: plain recursion over the moves

BRUTE FORCE · PSEUDO-CODE

```
def ways(r, c):
    if c == n - 1:
        return 1 if r == n - 1 else 0
    total = 0
    for dr in (-1, 0, 1):
        if 0 <= r + dr < n:
            total += ways(r + dr, c + 1)
    return total
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three calls per cell and the same cell is reached by many paths, so the call tree has about 3^n nodes. One column at a time with a rolling array is O(m * n).

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

BRUTE FORCE · PSEUDO-CODE

```
def cars_needed(rentals):
    free_at = []  # per car
    for start, end in sorted(rentals):
        for i, f in enumerate(free_at):
            if f <= start:
                free_at[i] = end  # reuse
                break
        else:
            free_at.append(end)  # new car
    return len(free_at)
```

TIME	SPACE
O(R^2)	O(R)

WHY IT IS SLOW

Each rental scans every car, so the cost is O(R * cars), up to O(R^2). A heap of free times finds the earliest-free car in O(log R).

OPTIMAL SOLUTION · Q1: top-K active users with a size-K heap

TIME	SPACE
O(n + d log k) for d distinct users	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: count constrained paths with a rolling column

TIME	SPACE
O(m * n)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q3: minimum cars and one valid assignment

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

FOLLOW-UP 2 Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

FOLLOW-UP 3 Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

FOLLOW-UP 4 Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(m * n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve chat, grid paths, and car rentals
#
# You are given three independent programming tasks. For
# each, describe and implement an efficient algorithm, and
# be prepared to analyze its time and space complexity.
# --- ### Q1. Find top-K most active chat users You are
# given a log of chat messages from a messaging
# application. **Input**: - An array messages of length n,
# where each element messages[i] is the user_id (string or
# integer) of the sender of the i-th message. - An integer
# k (1 <= k <= number of distinct users). **Output**: -
# Return a list of k user IDs corresponding to the users
# who have sent the most messages. **Details &
# requirements**: - The "activity" of a user is defined as
# the number of messages they have sent (i.e., their
# frequency in messages). - If there are fewer than k
# distinct users, return all distinct users.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1: what if the log is a stream and memory is
# limited?
# 2. Q2: what changes if there are obstacles or the path
# may also go straight up and down?
# 3. Q3: how do you answer 'is one more rental possible'
# online?
# 4. Q2 with obstacles.
#
# INTERVIEW NOTES
# Three unrelated tasks, so say so and give each a one-
# line plan. Chat: count, then take the top K with a heap.
# Grid: each column depends only on the previous one, so
# keep one column. Cars: the minimum number of cars is the
# largest number of rentals that overlap at once, which a
# heap of return times tracks while also giving an
# assignment.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def top_k_users(messages, k):
```

```
    # Q1, heap: O(n + d log k); sorted order breaks ties
    counts = Counter(messages)
    best = heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return [user for user, _ in best]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def count_paths(m, n, blocked=(), mod=None):
    # Q2: start (m-1, 0), end (m-1, n-1), moves right-up,
    # right, right-down. One column of ways is enough:
    # O(m).
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
            # FOLLOW-UP 4: blocked cells (obstacles) give
            # no paths
            if (r, c) in wall:
                # a blocked cell contributes no paths
                continue
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
```

```
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total % mod if mod else total
        ways = nxt
    return ways[m - 1]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def min_cars(rentals):
    # Q3: rentals are (pickup, return); how many cars are
    # needed?
    # A heap holds the return times of the cars on the
    # road.
    cars = []
    for pickup, back in sorted(rentals):
        if cars and cars[0] <= pickup:
            # a car came back in time: reuse it
            heapq.heapreplace(cars, back)
        else:
            heapq.heappush(cars, back)
    return len(cars)

# ---- main question: tests ----
def test_main_question():
    log = ["a", "b", "a", "c", "a", "b"]
    assert top_k_users(log, 2) == ["a", "b"]
    assert top_k_users_sorted(log, 5) == ["a", "b", "c"]
    # grid paths: a 3 x 3 grid has 2 paths along the
    # bottom
    assert count_paths(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths(4, 5) == 9
    # an obstacle in the middle of the bottom row
    assert count_paths(3, 3, blocked=[(2, 1)]) == 1
    modulus = 10 ** 9 + 7
```

```
    assert count_paths(5, 30, mod=modulus) < modulus
    cars = [(1, 4), (2, 5), (4, 6), (5, 8)]
    assert min_cars(cars) == 2
    assert min_cars([]) == 0
```

```
# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Q3: how do you answer 'is one more rental
# ============================================================

# ---- follow-up 3: function ----
# Answer: Keep the busy heap and the free-car count
# between calls. A new rental fits without a new car if
# any car's return time is <= its pickup, which is a heap
# peek after popping everything that returned before the
# pickup.
class RentalDesk:
    """Online: can one more rental be served right now?"""

    def __init__(self, fleet):
        self.fleet, self.out = fleet, []

    def rent(self, pickup, back):
        # cars returned by `pickup` are free again
        while self.out and self.out[0] <= pickup:
            heapq.heappop(self.out)
        if len(self.out) >= self.fleet:
            return False
        heapq.heappush(self.out, back)
        return True

# ---- follow-up 3: tests ----
def test_follow_up_3():
    desk = RentalDesk(1)
    assert desk.rent(1, 4) and not desk.rent(2, 3)
    assert desk.rent(4, 6)


# ============================================================
```

```
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# Q1 second approach: sort every user
def top_k_users_sorted(messages, k):
    # Q1, second approach: sort every user, O(d log d)
    counts = Counter(messages)
    ranked = sorted(counts, key=lambda u: (-counts[u], u))
    return ranked[:k]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity.

Q1. Find top-K most active chat users

You are given a log of chat messages from a messaging application.

Input:

An array messages of length n , where each element messages[i] is the user_id (string or integer) of the sender of the i-th message.
An integer k (1 <= k <= number of distinct users).

Output:

Return a list of k user IDs corresponding to the users who have sent the most messages.

Details & requirements:

The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in messages).
If there are fewer than k distinct users, return all distinct users.
If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by user_id in ascending order.
Aim for an algorithm that is more efficient than sorting all users by count when k is much smaller than the number of distinct users.

Design and implement a function, for example:

List<UserId> topKActiveUsers(List<UserId> messages, int k)

Also explain:

At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
The time and space complexity of each approach.
Q2. Count paths in a 2D grid with constrained moves

You have a 2D grid with m rows and n columns. Index rows from top to bottom as 0..m-1 and columns from left to right as 0..n-1.

The start cell is the bottom-left corner: (m-1, 0) .
The end cell is the bottom-right corner: (m-1, n-1) .

From any cell (r, c), you may move only to the next column to the right, in one of up to three ways (if within bounds):

Right-up: (r-1, c+1)
Right: (r, c+1)
Right-down: (r+1, c+1)

You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

FULL QUESTION TEXT FROM THE SITE

Given integers m and n , compute the total number of distinct paths from the start cell (m-1, 0) to the end cell (m-1, n-1) using only the allowed moves.

Requirements:

Implement a function such as long countPaths(int m, int n) .
The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct.
Q3. Minimum cars needed for rentals and assignment

You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

An array of R rental records. Each record has:
rental_id : a unique identifier (e.g., integer or string).
pickup_time : an integer start time.
return_time : an integer end time, with pickup_time < return_time .

Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time t and another starts at time t, the same car can serve both.

Tasks:

Capacity : Compute the minimum number of cars required so that all rentals can be fulfilled without time conflicts on any single car.
Assignment : Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

An integer C = minimal number of cars needed.
A mapping from each rental_id to a car_id in the range 1..C , such that no two rentals assigned to the same car_id overlap in time.

Requirements & clarifications:

Overlap definition: two rentals with intervals [s1, e1) and [s2, e2) overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if e1 == s2 they do not overlap.
Aim for an algorithm that is efficient for up to, say, 10^5 rental records.
Discuss a naive approach and then an optimized approach (for example, using a min-heap/priority queue keyed by the earliest return time to track currently in-use cars).
Analyze the time and space complexity of your final solution.

STUDY PATH: question 185 of 288 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 5 of 6

Template fit: count len(body.split()) per speaker instead of lines; ties by name | asked 394 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 185 of 288

Top-K Chatters by Word Count

ASKED 2025-09-21 | 394 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 138/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (related: count words per author)

Frequency: Single report | Last asked: 2025-09-21 | Roles: SWE Intern

QUESTION

You are given a chat transcript as a list of message lines. Each line has the form "Name: message text" , where everything before the first ": " is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non- whitespace characters in the body). A person may appear on many lines; sum their words. Return the k people who typed the most words, as a list of (name, word_count) tuples sorted by word_count descending. Break ties by name in ascending (lexicographic) order.

Function signature

```
def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:
    ...
```

Follow-up (as reported): The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a ChatAnalyzer class that ingests lines incrementally via add_line(line) and answers top_k(k) on demand.

EXAMPLES

Example 1

Input	`lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1`
Output	`[("Alice", 6)]`
Why	`Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.`

QUESTION

Example 2

Input	`lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2`
Output	`[("Ann", 2), ("Bob", 2)]`
Why	`Ann and Bob both have 2 words; tie broken alphabetically, so Ann precedes Bob. Cara (1 word) is excluded.`

Example 3

Input	`lines = [], k = 3`
Output	`[]`

CONSTRAINTS

- 0 <= len(lines) <= 10^5
- Every line contains exactly one ": " separating name from body; the name is non-empty and contains no ": " .
- The body may be empty (0 words).
- 1 <= k ; if k exceeds the number of distinct people, return all of them.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_chatters(lines, k):
    counts = {}
    for line in lines:
        name, body = line.split(": ", 1)
        counts[name] = counts.get(
            name, 0
        ) + len(body.split())
    ranked = sorted(
        counts.items(),
        key=lambda kv: (-kv[1], kv[0]),
    )
    return ranked[:k]
```

TIME	SPACE
O(p log p)	O(p)

WHY IT IS SLOW

It sorts all p authors although only the top k are needed.

OPTIMAL SOLUTION · Counter and a bounded selection

TIME	SPACE
O(n + p log k)	O(p)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Wrap it in a ChatAnalyzer class that takes lines one at a time and answers top_k on demand.

Keep the Counter as state; add_line parses and updates it in O(words), top_k runs the selection. If top_k is called far more often than add_line, keep a sorted structure or a heap with lazy updates instead of recomputing.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Top-K Chatters by Word Count
#
# You are given a chat transcript as a list of message
# lines. Each line has the form "Name: message text" ,
# where everything before the first ": " is the author's
# name and everything after it is the message body. Count
# the number of words each person typed across the entire
# transcript (a word is a maximal run of non- whitespace
# characters in the body). A person may appear on many
# lines; sum their words. Return the k people who typed
# the most words, as a list of (name, word_count) tuples
# sorted by word_count descending. Break ties by name in
# ascending (lexicographic) order. Function signature: def
# top_k_chatters(lines: list[str], k: int) ->
# list[tuple[str, int]]: ... Follow-up (as reported): The
# interviewer's real ask was to implement the parsing
# inside a class alongside stubbed API methods.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Wrap it in a ChatAnalyzer class that takes lines one
# at a time and answers top_k on demand.
#
# INTERVIEW NOTES
# Two steps: parse each line at the first ': ' and add the
# number of words in the body to the author's total, then
# take the k largest totals with ties broken by name. A
# heap of size k (or sorting by the key (-count, name))
# handles the ordering.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Counter and a bounded selection
# Idea: Sum the words per author, then pick the k best
# with a heap keyed by (-count, name).
def top_k_chatters(lines, k):
    counts = Counter()
    for line in lines:
        name, body = line.split(": ", 1)
        counts[name] += len(body.split())
    return heapq.nsmallest(
        k,
        counts.items(),
        key=lambda kv: (-kv[1], kv[0]),
    )

# ---- main question: tests ----
def test_main_question():
    random.seed(18)
    assert top_k_chatters(
        [
            "Alice: hello there friend",
            "Bob: hi",
            "Alice: how are you",
        ],
        1,
    ) == [("Alice", 6)]
    assert top_k_chatters(
        ["Ann: a b", "Bob: c d", "Cara: e"], 2
    ) == [("Ann", 2), ("Bob", 2)]
    assert top_k_chatters([], 3) == []
    names = ["Zed", "amy", "Bo", "Cy"]


# ============================================================
# FOLLOW-UP 1: Wrap it in a ChatAnalyzer class that takes
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the Counter as state; add_line parses and
# updates it in O(words), top_k runs the selection. If
# top_k is called far more often than add_line, keep a
# sorted structure or a heap with lazy updates instead of
# recomputing.
class ChatAnalyzer:
    def __init__(self):
        self.counts = Counter()

    def add_line(self, line):
        name, body = line.split(": ", 1)
        self.counts[name] += len(body.split())

    def top_k(self, k):
        return heapq.nsmallest(
            k,
            self.counts.items(),
            key=lambda kv: (-kv[1], kv[0]),
        )

# ---- follow-up 1: tests ----
def test_follow_up_1():
    names = ["Zed", "amy", "Bo", "Cy"]
    for _ in range(300):
        lines = [
            random.choice(names)
            + ": "
            + " ".join(
                "w" * random.randint(1, 3)
                for _ in range(random.randint(1, 4))
            )
            for _ in range(random.randint(0, 8))
        ]
        k = random.randint(1, 5)
        chat = ChatAnalyzer()
        for line in lines:
            chat.add_line(line)
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 186 of 288 | topic 23 of 43: Top-K by frequency: Counter plus nsmallest | 6 of 6

Template fit: the same Counter; run nsmallest once with -count and once with +count for the bottom-k | asked 282 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 186 of 288

Find top/bottom-k words in list or stream

POSTED 2026-01-11 | 282 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 136/288

ABOUT THIS QUESTION

Topic: Top-K by frequency: Counter plus nsmallest

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (+ stream variant)

Posted on PracHub: 2026-01-11 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/find-top-bottom-k-words-in-list-or-stream

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

QUESTION

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love',
    'coding'], 2)
Expected Output: [['i', 'love'], ['coding',
    'leetcode']]
Explanation: `i` and `love` both appear twice, so
    lexicographical order gives `['i', 'love']`.
    `coding` and `leetcode` both appear once, so
    lexicographical order gives `['coding',
    'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry',
    'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'],
    ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

Hints
Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).
Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.
Constraints
1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested
Examples
Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]
Expected Output: [['apple', 'banana'], ['banana', 'apple']]
Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]
Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]
Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.
Hints
Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, k):
    counts = Counter(words)
    def rank(w):
        return (-counts[w], w)
    ranked = sorted(counts, key=rank)
    most = ranked[:k]
    least = ranked[::-1][:k]
    return most, least
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct words, O(d log d). Two heaps of size K are O(d log k); a stream only needs the counts, and a Space-Saving sketch bounds memory for the top K.

OPTIMAL SOLUTION · Counter plus two heaps of size K

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / m times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

FOLLOW-UP 2 topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

FOLLOW-UP 3 Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find top/bottom-k words in list or stream
#
# You are given words (strings) either as a finite list or
# as an unbounded stream. 1) **List version**: Given an
# array words and an integer k, return: - the k most
# frequent distinct words, and - the k least frequent
# distinct words. Specify how you break ties (e.g.,
# lexicographically, or any order). 2) **Stream version
# (online)**: Design a data structure that supports: -
# ingest(word): process the next word in the stream -
# topK(k): return the k most frequent distinct words seen
# so far - bottomK(k): return the k least frequent
# distinct words seen so far Discuss time/space complexity
# and how you would implement these operations
# efficiently. 3) **Variant (discuss/design)**: Extend the
# stream design to support returning the **first k**
# and/or **last k** words that are currently **non-
# repeating** (i.e., words with frequency exactly 1)
# according to stream arrival order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream is too big to count every distinct word.
# 2. topK must be O(k), not O(d log k), because it is
# called all the time.
# 3. Case and punctuation.
#
# INTERVIEW NOTES
# Count first; then you only need the K best and K worst,
# which a heap does without sorting everything. For a
# stream, ingesting is just a counter increment, and the
# answer is computed from the counts on demand. If memory
# is bounded, only heavy hitters (top-K) can be
# approximated; the rarest words cannot.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import Counter, defaultdict
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# k) without a full sort.
def top_bottom(words, k):
    # ties are broken alphabetically
    c = Counter(words)
    top = heapq.nsmallest(k, c, key=lambda w: (-c[w], w))
```

```
    low = heapq.nsmallest(k, c, key=lambda w: (c[w], w))
    bottom = low
    return top, bottom

# ---- main question: tests ----
def test_main_question():
    words = "a b a c b a d d".split()
    top, bottom = top_bottom(words, 2)
    assert top == ["a", "b"] and bottom == ["c", "b"]


# ======================================================
# FOLLOW-UP 1: The stream is too big to count every
# ======================================================

# ---- follow-up 1: function ----
# Answer: Use the Space-Saving (or Misra-Gries) summary
# with m counters: a word that appears more than n / m
# times is guaranteed to be tracked, and its stored count
# is an upper bound.
class SpaceSaving:
    """Too many distinct words: m counters. A word seen
    over n / m times is always tracked."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, word):
        if word in self.counts:
            self.counts[word] += 1
```

```
        elif len(self.counts) < self.m:
            self.counts[word] = 1
        else:
            # replace the smallest counter; the new word
            # inherits its count + 1 (an upper bound)
            low = min(self.counts, key=self.counts.get)
            self.counts[word] = self.counts.pop(low) + 1

    def top_k(self, k):
        return sorted(self.counts, key=lambda w: (
            -self.counts[w], w))[:k]

# ======================================================
# FOLLOW-UP 2: topK must be O(k), not O(d log k), because
# ======================================================

# ---- follow-up 2: function ----
# Answer: Keep words in buckets by count (a linked list of
# count buckets, like the LFU cache structure); the most
# frequent words are in the last bucket. An increment
# moves a word one bucket up in O(1).
class FreqBuckets:
    """Stream: add(word) is O(1); top_k reads only the
    highest count buckets."""

    def __init__(self):
        self.count = defaultdict(int)
        # count -> words with exactly that count
        self.bucket = defaultdict(dict)
        self.high = 0

    def add(self, word):
        c = self.count[word]
        if c:
            # move the word up one bucket
            del self.bucket[c][word]
            if not self.bucket[c]:
                del self.bucket[c]
        self.count[word] = c + 1
        self.bucket[c + 1][word] = True
        self.high = max(self.high, c + 1)
```

```
    def top_k(self, k):
        out = []
        level = self.high
```

```
        while level > 0 and len(out) < k:
            # words inside one bucket: alphabetical
            out += sorted(self.bucket.get(level, ()))
            level -= 1
        return out[:k]

    def bottom_k(self, k):
        out = []
        for level in sorted(self.bucket):
            out += sorted(self.bucket[level])
            if len(out) >= k:
                break
        return out[:k]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    words = "a b a c b a d d".split()
    f = FreqBuckets()
    for w in words:
        f.add(w)
    assert f.top_k(2) == ["a", "b"]
    assert f.bottom_k(2) == ["c", "b"]
    assert f.top_k(10) == ["a", "b", "d", "c"]
    s = SpaceSaving(2)
    for w in "x x x y z x x":
        s.add(w) if w.strip() else None
    assert s.top_k(1) == ["x"]


# ======================================================
# FOLLOW-UP 3: Case and punctuation.
# ======================================================

# ---- follow-up 3: function ----
# Answer: Normalise before counting (lowercase, strip
# punctuation) and say so; the counting and ranking code
# is unchanged.
def normalise(text):
    # case and punctuation: clean before counting
    letters = (ch for ch in text.lower() if ch.isalnum())
    return "".join(letters)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert normalise("Hello, World!") == "helloworld"
```

```


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], k: int) -> list[list[str]]:
```

Sample call

```
solution(['i', 'love', 'leetcode', 'i', 'love', 'coding'],
 2)
```

Test cases

Input	Expected
(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)	[['i', 'love'], ['coding', 'leetcode']]
(['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)	[['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 24 OF 43

Digits and characters: peel one at a time and carry

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	3	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Digits and characters: peel one at a time and carry | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

E	187. Caesar Cipher: Recover Key, Encrypt, and Decrypt	ColorStack	2026-02-08
M	188. Add two big integers from digit lists	PracHub	2025-11-24
M	189. Solve several coding interview problems	PracHub	2026-03-04
M	190. Check Whether Digits Are Monotone Increasing	PracHub	2026-07-04
H	191. Check Digits and Combine Ranges	PracHub	2026-03-29
H	192. Permutation Sequence	ColorStack	2025-10-17

CODE TEMPLATE | GROUP 24 OF 43 | 6 QUESTIONS, ONE PATTERN

Digits and characters: peel one at a time and carry

The one idea these questions share

Work one digit or character at a time: n % 10 peels the last digit, n // 10 removes it, a carry moves overflow to the next place, and (ord(c) - 97 + k) % 26 shifts a letter. Build the answer in a list and reverse or join at the end.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def add_digits(a, b):                          # two digit lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10)                 # CHANGE 1: what you build per place
        carry = total // 10                    # CHANGE 2: what you carry
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

2. Helper functions that usually get written next to it

```
def digits_of(n):
    out = []
    while n:
        n, d = divmod(n, 10)
        out.append(d)
    return out[::-1]


def shift_letter(ch, k):
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
187	Caesar Cipher: Recover Key, Encrypt, and Decrypt	per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k
188	Add two big integers from digit lists	the template as written
189	Solve several coding interview problems	the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps
190	Check Whether Digits Are Monotone Increasing	peel digits from the right; each digit must not exceed the previous one you saw
191	Check Digits and Combine Ranges	palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half
192	Permutation Sequence	factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 187 of 288 | topic 24 of 43: Digits and characters: peel one at a time and carry | 1 of 6

Template fit: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k | asked 254 days ago | frequency Single report

EASY | ColorStack | technical interview | 187 of 288

Caesar Cipher: Recover Key, Encrypt, and Decrypt

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 243/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1160 / Caesar cipher (related: character shifts)

Frequency: Single report | Last asked: 2026-02-08 | Roles: SWE Intern

QUESTION

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount k (0 <= k < 26) with wraparound (z +1 -> a); all non-lowercase characters were left unchanged. You are given one example pair — a known plaintext and its corresponding ciphertext — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement: find_shift(plaintext, ciphertext) -> int : return the shift k (0..25) that maps plaintext to ciphertext . The two strings have equal length and contain at least one lowercase letter that pins down k . encrypt(text, k) -> str : shift each lowercase letter forward by k with wraparound; leave other characters unchanged. decrypt(text, k) -> str : the inverse of encrypt — shift each lowercase letter backward by k with wraparound.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...
def encrypt(text: str, k: int) -> str:
    ...
def decrypt(text: str, k: int) -> str:
    ...
```

Follow-up (as reported): Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.

EXAMPLES

Example 1

Input	`find_shift("abc", "def")`
Output	`3`
Why	`'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.`

QUESTION

Example 2

Input	`encrypt("hello world", 3)`
Output	`"khoor zruog"`
Why	`Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space is unchanged.`

Example 3

Input	`decrypt("khoor zruog", 3)`
Output	`"hello world"`
Why	`decrypt inverts encrypt, so the original message is recovered.`

Example 4

Input	`encrypt("xyz", 3)`
Output	`"abc"`
Why	`Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.`

CONSTRAINTS

- 1 <= len(text) <= 10^5 . text may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters a – z are shifted.
- For find_shift , len(plaintext) == len(ciphertext) and they align character by character.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_find_shift(plaintext, ciphertext):
    for k in range(26):
        if encrypt(plaintext, k) == ciphertext:
            return k
    return -1
```

TIME	SPACE
O(26 n)	O(n)

WHY IT IS SLOW

It encrypts the whole text for each guess although one letter pair fixes the key.

OPTIMAL SOLUTION · Key from one letter pair, shared shift helper

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You only have a ciphertext. How do you break it?

Try all 26 shifts and score each candidate by how English it looks: add a weight for every common letter (e, t, a, o, i, n, s, h, r, d, l, u) and keep the shift with the best score. Longer texts make the frequency signal reliable.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Caesar Cipher: Recover Key, Encrypt, and
# Decrypt
#
# A message was scrambled with a Caesar cipher: every
# lowercase letter was shifted forward by the same secret
# amount k (0 <= k < 26) with wraparound ( z +1 -> a );
# all non-lowercase characters were left unchanged. You
# are given one example pair — a known plaintext and its
# corresponding ciphertext — from which you must recover
# the secret shift, and then implement the cipher in both
# directions. Specifically implement:
# find_shift(plaintext, ciphertext) -> int : return the
# shift k (0..25) that maps plaintext to ciphertext . The
# two strings have equal length and contain at least one
# lowercase letter that pins down k . encrypt(text, k) ->
# str : shift each lowercase letter forward by k with
# wraparound; leave other characters unchanged.
# decrypt(text, k) -> str : the inverse of encrypt — shift
# each lowercase letter backward by k with wraparound.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You only have a ciphertext. How do you break it?
#
# INTERVIEW NOTES
# A Caesar shift moves each lowercase letter forward by k
# modulo 26. The key is recovered from any one lowercase
# letter pair: (ord(cipher) - ord(plain)) mod 26. Encrypt
# adds k, decrypt subtracts k, both with the same helper
# and the wrap-around handled by the modulo.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
```

```
def find_shift(plaintext, ciphertext):
    for p, c in zip(plaintext, ciphertext):
        if "a" <= p <= "z":
            return (ord(c) - ord(p)) % 26
    return 0


# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def shift_text(text, k):
    out = []
    for ch in text:
        if "a" <= ch <= "z":
            ch = chr((ord(ch) - 97 + k) % 26 + 97)
        out.append(ch)
    return "".join(out)


# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def encrypt(text, k):
    return shift_text(text, k)


# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def decrypt(text, k):
    return shift_text(text, -k)

# ---- main question: tests ----
def test_main_question():
    random.seed(88)
    assert find_shift("abc", "def") == 3
    assert encrypt("hello world", 3) == "khoor zruog"
    assert decrypt("khoor zruog", 3) == "hello world"
```

```
    assert encrypt("xyz", 3) == "abc"
    sample = (
        "the quick brown fox jumps over the lazy dog "
    )
    sample += "while the sun sets over the quiet hills "
    sample += "and the rivers run to the sea"


# ============================================================
# FOLLOW-UP 1: You only have a ciphertext. How do you
# ============================================================

# ---- follow-up 1: function ----
# Answer: Try all 26 shifts and score each candidate by
# how English it looks: add a weight for every common
# letter (e, t, a, o, i, n, s, h, r, d, l, u) and keep the
# shift with the best score. Longer texts make the
# frequency signal reliable.
def break_caesar(ciphertext):
    common = "etaoinshrdlu"
    weight = {ch: 12 - i for i, ch in enumerate(common)}

    def score(k):
        text = decrypt(ciphertext, k)
        return sum(weight.get(ch, 0) for ch in text)

    return max(range(26), key=score)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    sample = (
```

```
        "the quick brown fox jumps over the lazy dog "
    )
    sample += "while the sun sets over the quiet hills "
    sample += "and the rivers run to the sea"
    for k in range(26):
        assert decrypt(encrypt(sample, k), k) == sample
        assert break_caesar(encrypt(sample, k)) == k
    assert encrypt("a1 b!", 2) == "c1 d!"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 188 of 288 | topic 24 of 43: Digits and characters: peel one at a time and carry | 2 of 6

Template fit: the template as written | asked 330 days ago

MEDIUM | IN APP LC 2 | PracHub | Technical Screen | 188 of 288

★ TOP 75: rank 1 in the Medium and Hard priority list

★ Add two big integers from digit lists

POSTED 2025-11-24 | 330 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 78.7 | 30 DAYS 69.3 | PRIORITY 1/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2 Add Two Numbers

LeetCode match: LC 2 Add Two Numbers

Posted on PracHub: 2025-11-24 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/add-two-big-integers-from-digit-lists

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

QUESTION

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7,
    0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_lists(a, b):
    x = digits_to_int(a)  # reverse-order digits
    y = digits_to_int(b)
    return int_to_digits(x + y)
```

BRUTE FORCE

TIME	SPACE
O(n^2) for the string-to-integer conversions	O(n)

WHY IT IS SLOW

It relies on arbitrary-size integers and the conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

OPTIMAL SOLUTION · Digit by digit with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

FOLLOW-UP 2 Multiply instead of add.

Schoolbook multiplication: the product of digits a[i] and b[j] lands in position i + j + 1, with the carry spilling to i + j. O(m * n); Karatsuba or FFT for very long numbers.

FOLLOW-UP 3 Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Add two big integers from digit lists
#
# You are given two non-empty singly linked lists (or
# arrays) representing two non-negative integers. The
# digits are stored in **reverse order**, and each node
# contains a single digit from 0 to 9. Your task is to add
# the two numbers and return the sum as a new linked list,
# also in reverse order. You may assume: - Each list has
# at least one node. - The integers do not contain leading
# zeros, except the number 0 itself. ### Example Input: -
# List 1: 2 -> 4 -> 3 (represents the integer 342) - List
2:
# 5 -> 6 -> 4 (represents the integer 465) Output: - 7 ->
0 ->
# 8 (represents the integer 807, since 342 + 465 = 807)
# ### Another example Input: - List 1: 9 -> 9 -> 9
# (represents 999) - List 2: 1 (represents 1) Output: - 0
# -> 0 -> 0 -> 1 (represents 1000) ### Constraints - Let n
 be
# the length of the first list and m be the length of the
# second list. - 1 <= n, m <= 10^5.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The digits are stored least significant first in a
# linked list.
# 2. Multiply instead of add.
# 3. Subtract, or numbers in another base.
#
# INTERVIEW NOTES
# Addition by hand: start at the least significant end,
# add the two digits and the carry, write digit % 10,
# carry digit // 10. The only edge cases are different
# lengths and a final carry. Never convert to a machine
# integer in an interview about big numbers, since the
# point is that the numbers do not fit.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices i and
# j; at each step add a[i] (or 0 once exhausted), b[j] (or
# 0) and the carry, append sum % 10, and carry = sum //
# 10.
def add_digits(a, b):
    # arrays with the most significant digit FIRST
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        total += a[i] if i >= 0 else 0
        total += b[j] if j >= 0 else 0
        out.append(total % 10)
        carry = total // 10
        i, j = i - 1, j - 1
    return out[::-1] or [0]

# ---- main question: tests ----
def test_main_question():
    assert add_digits([9, 9], [1]) == [1, 0, 0]
    assert add_digits([1, 2, 3], [4, 5]) == [1, 6, 8]


# ============================================================
# FOLLOW-UP 1: The digits are stored least significant
# ============================================================

# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

# ---- follow-up 1: function ----
```

```
# Answer: Then the head is already the low end, so walk
# both lists together with the same carry loop and build
# the result list as you go. No reversal needed (LeetCode
# 2).
def add_lists(a, b):
    # digits least significant first: add with a carry,
    # one pass
    dummy = tail = ListNode()
    carry = 0
    while a or b or carry:
        total = carry
```

```
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail, carry = tail.next, total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # 342 + 465 = 807, digits reversed
    got = add_lists(make([2, 4, 3]), make([5, 6, 4]))
    assert to_digits(got) == [7, 0, 8]
    # a final carry adds a digit: 99 + 1 = 100
    carry = add_lists(make([9, 9]), make([1]))
    assert to_digits(carry) == [0, 0, 1]
    assert add_digits([9, 9, 9], [1]) == [1, 0, 0, 0]
    assert add_digits([0], [0]) == [0]


# ============================================================
# FOLLOW-UP 2: Multiply instead of add.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Schoolbook multiplication: the product of digits
# a[i] and b[j] lands in position i + j + 1, with the
# carry spilling to i + j. O(m * n); Karatsuba or FFT for
# very long numbers.
def multiply_digits(a, b):
    # schoolbook: digit a[i] * b[j] lands at position i +
    # j + 1
    res = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            total = a[i] * b[j] + res[i + j + 1]
            res[i + j + 1] = total % 10
            res[i + j] += total // 10
    while len(res) > 1 and res[0] == 0:
        res.pop(0)
    return res

# ---- follow-up 2: tests ----
def test_follow_up_2():
    product = multiply_digits([1, 2, 3], [4, 5])
```

```
    assert product == [5, 5, 3, 5]
    assert multiply_digits([0], [9, 9]) == [0]


# ============================================================
# FOLLOW-UP 3: Subtract, or numbers in another base.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Subtraction borrows instead of carrying and
# needs the larger number first (compare lengths, then
# digits). Another base replaces 10 with the base in % and
# //.
def subtract_digits(a, b):
    # a - b where a >= b (most significant first): borrow
```

```
    i, j, borrow, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0:
        d = a[i] - borrow - (b[j] if j >= 0 else 0)
        borrow = 1 if d < 0 else 0
        out.append(d + 10 * borrow)
        i, j = i - 1, j - 1
    while len(out) > 1 and out[-1] == 0:
        out.pop()
    return out[::-1]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert subtract_digits([1, 0, 0], [1]) == [9, 9]
    assert subtract_digits([5, 0], [5, 0]) == [0]


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
# bove is defined)
# ============================================================
# HELPERS for the checks below
def make(digits):
    # digits least significant first -> linked list
    head = tail = ListNode(digits[0])
    for d in digits[1:]:
        tail.next = ListNode(d)
        tail = tail.next
    return head


def to_digits(node):
```

```
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Sample call

```
add_two_numbers([2, 4, 3], [5, 6, 4])
```

Test cases

Input	Expected
([2, 4, 3], [5, 6, 4])	[7, 0, 8]
([9, 9, 9], [1])	[0, 0, 0, 1]

STUDY PATH: question 189 of 288 | topic 24 of 43: Digits and characters: peel one at a time and carry | 3 of 6

Template fit: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps | asked 230 days ago

MEDIUM | PracHub | Technical Screen | 189 of 288

Solve several coding interview problems

POSTED 2026-03-04 | 230 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 38.3 | 30 DAYS 0 | PRIORITY 112/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #160 Intersection of Two Linked Lists

LeetCode match: No direct match (multi-part); related LC 415 Add Strings, LC 160 Intersection of Two Linked Lists (inferred from overview)

Posted on PracHub: 2026-03-04 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-several-coding-interview-problems

QUESTION

Three small tasks. (1) add_strings(a, b): add two non-negative integers given as digit strings and return the sum as a string, without converting the whole numbers. (2) intersection(head_a, head_b): two singly linked lists may share a common tail; return the first shared node, or None. (3) overlap_depth(rects): rectangles (x1, y1, x2, y2) include their left and bottom edges but not their right and top edges; return the largest number of rectangles that cover a single point.

Function signature

def add_strings(a: str, b: str) -> str
def intersection(head_a, head_b)
def overlap_depth(rects: list) -> int

EXAMPLES

Example 1

Input	`'999', '1'`
Output	`'1000'`
Why	`A carry runs through every digit: 999 + 1 = 1000.`

Example 2

Input	`'456', '77'`
Output	`'533'`
Why	`456 + 77 = 533.`

QUESTION

Example 3

Input	`class N:`
	`def __init__(self, val, next=None):`
	`self.val, self.next = val, next`
	`shared = N(8, N(9))`
	`a = N(1, N(2, shared))`
	`b = N(5, shared)`
	`intersection(a, b).val`

Output	`intersection(a, b).val -> 8`
Why	`List a is 1, 2, 8, 9 and list b is 5, 8, 9: they join at the node holding 8.`

Example 4

Input	`[(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]`
Output	`3`
Why	`The point (3, 3) lies inside all three rectangles.`

Example 5

Input	`[(0, 0, 2, 2), (2, 0, 4, 2)]`
Output	`1`
Why	`Rectangles that only share an edge do not overlap.`

BRUTE FORCE · Big numbers: convert to integers and add

BRUTE FORCE · PSEUDO-CODE

```
def add_strings(a, b):
    return str(int(a) + int(b))
```

TIME	SPACE
O(n^2)	O(n)
for the conversions	

WHY IT IS SLOW

Relies on arbitrary-precision integers (not in every language) and the string-to-integer conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

BRUTE FORCE · Chains that meet: store one chain in a set

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Chains that meet: store one chain in a set

```
def intersection(a, b):
    seen = set()
    node = a
    while node:
        seen.add(node)
        node = node.parent
    node = b
    while node:
        if node in seen:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(m + n)	O(m)

WHY IT IS SLOW

O(m + n) time but O(m) extra memory. Two pointers that switch to the other chain's head at the end meet at the shared node with O(1) memory.

BRUTE FORCE · Overlap: test every time point

BRUTE FORCE · PSEUDO-CODE

```
def max_overlap(intervals):
    lo = min(s for s, e in intervals)
    hi = max(e for s, e in intervals)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in intervals:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Big numbers: digit by digit from the right with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Chains that meet: two pointers that switch heads at the end

TIME	SPACE
O(m + n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Overlap: sweep over sorted start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ancestors in a tree with parent pointers, not a linked list.

The same two-pointer trick works on parent chains (LeetCode 1650); with depths known, lift the deeper node first and then move both up together.

FOLLOW-UP 2 Add two big numbers stored as linked lists.

If the lists store the least significant digit first, walk both together with the carry (LeetCode 2); if most significant first, reverse them or use stacks.

FOLLOW-UP 3 Count overlapping rectangles instead of intervals.

Sweep over x with a data structure over y (a segment tree with range add and max) to find the largest point depth in O(n log n); the pairwise count is O(n^2) unless you use a sweep with an interval tree.

FOLLOW-UP 4 Add two big numbers stored as linked lists (least significant digit first).

Walk both lists together with a carry and build the result list as you go; no reversal needed. O(max(m, n)) time. If the most significant digit comes first, reverse the lists or use stacks.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve several coding interview problems
#
# Three small tasks. (1) add_strings(a, b): add two non-
```

```
# negative integers given as digit strings and return the
# sum as a string, without converting the whole numbers.
# (2) intersection(head_a, head_b): two singly linked
# lists may share a common tail; return the first shared
# node, or None. (3) overlap_depth(rects): rectangles (x1,
# y1, x2, y2) include their left and bottom edges but not
# their right and top edges; return the largest number of
# rectangles that cover a single point.
#
# Signature:
#   def add_strings(a: str, b: str) -> str
#   def intersection(head_a, head_b)
#   def overlap_depth(rects: list) -> int
#
# Examples:
# '999', '1'  ->  '1000'
# '456', '77'  ->  '533'
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ancestors in a tree with parent pointers, not a
# linked list.
# 2. Add two big numbers stored as linked lists.
# 3. Count overlapping rectangles instead of intervals.
# 4. Add two big numbers stored as linked lists (least
# significant digit first).
#
# INTERVIEW NOTES
# A set of unrelated small tasks: split them, restate each
# contract, and solve the smallest one first. Big numbers:
# add digit by digit with a carry. Two chains that meet:
# walk both and switch to the other head at the end so the
# lengths cancel. Overlap counting: sort start and end
# events and keep a running count.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def add_strings(a, b):
    # big numbers kept as digit strings, added from the
    # right
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        out.append(str(total % 10))
        carry = total // 10
        i, j = i - 1, j - 1
    return "".join(out[::-1])


# BASE QUESTION (optimal): Big numbers: digit by digit
```

```
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def intersection(head_a, head_b):
    # two chains meeting: each pointer walks its chain and
    # then
    # the other one, so both walk the same total length
    a, b = head_a, head_b
    while a is not b:
        a = a.next if a else head_b
        b = b.next if b else head_a
    return a


# BASE QUESTION (optimal): Big numbers: digit by digit
```

```
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def overlap_depth(rects):
    # rects (x1, y1, x2, y2), half-open: the largest
    # number of
    # rectangles sharing one point. The best point can be
    # a
    # rectangle's lower-left corner, so test those.
    best = 0
    for x, y, _, _ in rects:
        inside = [r for r in rects if r[0] <= x < r[2]
                  and r[1] <= y < r[3]]
        best = max(best, len(inside))
    return best

# ---- main question: tests ----
def test_main_question():
    assert add_strings("999", "1") == "1000"
    assert add_strings("0", "0") == "0"
    big = add_strings("123456789123456789", "987654321")
    assert big == str(123456789123456789 + 987654321)


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Add two big numbers stored as linked lists.
# ============================================================

# ---- follow-up 2: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

# ---- follow-up 2: function ----
# Answer: If the lists store the least significant digit
# first, walk both together with the carry (LeetCode 2);
# if most significant first, reverse them or use stacks.
# FOLLOW-UP 4: Add two big numbers stored as linked lists
```

```
# (least significant digit first).
# Answer: Walk both lists together with a carry and build
# the result list as you go; no reversal needed. O(max(m,
# n)) time. If the most significant digit comes first,
# reverse the lists or use stacks.
def add_lists(a, b):
    # digits stored least significant first: add with a
```

```
    # carry
    dummy = tail = ListNode()
    carry = 0
    while a or b or carry:
        total = carry
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next

# ---- follow-up 2: tests ----
def test_follow_up_2():
    shared = ListNode(8, ListNode(9))
    a = ListNode(1, ListNode(2, shared))
    b = ListNode(5, shared)
    assert intersection(a, b) is shared
    # no meeting: both end at None
    assert intersection(ListNode(1), ListNode(2)) is None
    # 342 + 465 = 807 with digits reversed
    x = ListNode(2, ListNode(4, ListNode(3)))
    y = ListNode(5, ListNode(6, ListNode(4)))
    assert to_list(add_lists(x, y)) == [7, 0, 8]
    rects = [(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]
    assert overlap_depth(rects) == 3


# ============================================================
# FOLLOW-UP 3: Count overlapping rectangles instead of
# ============================================================

# ---- follow-up 3: function ----
# Answer: Sweep over x with a data structure over y (a
# segment tree with range add and max) to find the largest
# point depth in O(n log n); the pairwise count is O(n^2)
```

```
# unless you use a sweep with an interval tree.
def overlapping_pairs(rects):
    # how many pairs of rectangles overlap?
    count = 0
    for i in range(len(rects)):
        for j in range(i + 1, len(rects)):
            a, b = rects[i], rects[j]
            if a[0] < b[2] and b[0] < a[2] \
                    and a[1] < b[3] and b[1] < a[3]:
                count += 1
    return count

# ---- follow-up 3: tests ----
def test_follow_up_3():
    rects = [(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]
    assert overlapping_pairs(rects) == 3
    assert overlap_depth([]) == 0


# ============================================================
# FOLLOW-UP 4: Add two big numbers stored as linked lists
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def to_list(node):
    out = []
    while node:
```

```
        out.append(node.val)
        node = node.next
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> list[str]:
```

Sample call

```
solution('.-')
```

Test cases

Input	Expected
`('.-',)`	`['A', 'ET']`
`('...',)`	`['EEE', 'EI', 'IE', 'S']`

STUDY PATH: question 190 of 288 | topic 24 of 43: Digits and characters: peel one at a time and carry | 4 of 6

Template fit: peel digits from the right; each digit must not exceed the previous one you saw | asked 108 days ago

MEDIUM | PracHub | Technical Screen | 190 of 288

Check Whether Digits Are Monotone Increasing

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 253/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy digit scan) (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/check-whether-digits-are-monotone-increasing

QUESTION

Return true when the decimal digits of a non-negative integer never decrease from left to right (repeated digits are allowed). Zero and single-digit numbers qualify.

Function signature

def is_monotone_digits(n: int) -> bool

EXAMPLES

Example 1	
Input	1234
Output	true
Why	1, 2, 3, 4 only goes up.

Example 2	
Input	1223
Output	true
Why	Equal neighbours are allowed.

Example 3	
Input	332
Output	false
Why	3, 3, 2 goes down at the end.

Example 4	
Input	0
Output	true
Why	Zero qualifies.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_monotone(x):
    s = str(x)
    return s == "".join(sorted(s))
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting costs O(d log d) and copies the digits. Comparing each digit with its neighbour while peeling digits with % 10 is O(d) with O(1) space.

OPTIMAL SOLUTION · Peel digits from the right and compare with the previous one

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

FOLLOW-UP 2 Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

FOLLOW-UP 3 Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check Whether Digits Are Monotone
# Increasing
#
# Return true when the decimal digits of a non-negative
# integer never decrease from left to right (repeated
# digits are allowed). Zero and single-digit numbers
# qualify.
#
# Signature:
#   def is_monotone_digits(n: int) -> bool
#
```

```
# Examples:
# 1234  ->  true
# 1223  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. Strictly increasing digits.
# 2. Negative numbers or numbers in another base.
# 3. Find the next monotone number after n.
#
# INTERVIEW NOTES
# Compare neighbouring digits. You can read the digits
# from the right with % 10 and // 10, remembering the
# previous (right-hand) digit: the number is monotone if
# no digit is larger than the one to its right. Say first
# whether equal neighbours are allowed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with n % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
```

```
# single digit and zero are monotone.
def is_monotone_digits(n):
    # non-decreasing digits, read from the right with n %
    # 10
    prev = 9
    while n:
        d = n % 10
        # a digit bigger than the one to its right: a
        # descent
        if d > prev:
            return False
        prev = d
        n //= 10
    return True


# SAME TEST on the text of the number
def is_monotone_string(n):
    # the same test on the text of the number
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))

# ---- main question: tests ----
def test_main_question():
    assert is_monotone_digits(0) and is_monotone_digits(7)
    assert is_monotone_digits(1223)
    assert is_monotone_digits(999)
    assert not is_monotone_digits(332)
    assert not is_monotone_digits(10)


# ============================================================
# FOLLOW-UP 1: Strictly increasing digits.
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Use >= instead of > when comparing with the
# previous digit, and start prev at 10 so a single digit
# passes.
def is_strictly_monotone(n):
    # strictly increasing: start prev at 10, compare with
    # >=
    prev = 10
    while n:
        d = n % 10
```

```
        if d >= prev:
            return False
        prev = d
        n //= 10
    return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert is_strictly_monotone(1234)
    assert not is_strictly_monotone(1224)
    assert is_strictly_monotone(5)


# ============================================================
# FOLLOW-UP 2: Negative numbers or numbers in another
# ============================================================

# ---- follow-up 2: function ----
# Answer: Decide what monotone means for the sign (usually
# apply it to the absolute value). For base b replace 10
# with b in % and //.
def is_monotone_base(n, base=10):
    # any base: use the base for % and //
    prev = base - 1
    while n:
        d = n % base
        if d > prev:
            return False
        prev = d
        n //= base
    return True

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # binary 11 -> 3 is monotone; binary 10 -> 2 is not
    assert is_monotone_base(3, 2)
    assert not is_monotone_base(2, 2)


# ============================================================
# FOLLOW-UP 3: Find the next monotone number after n.
# ============================================================

# ---- follow-up 3: function ----
# Answer: That is the left-to-right fix from the previous
```

```
# question: find the first descent and copy that digit
# across the rest.
def next_monotone(n):
    # smallest monotone number AFTER n: fix the first
    # descent
    d = list(str(n + 1))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert next_monotone(332) == 333
    assert next_monotone(9) == 11
```

```
    for n in range(2000):
        by_text = is_monotone_string(n)
        assert is_monotone_digits(n) == by_text


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def has_monotone_increasing_digits(n: int) -> bool:
SAMPLE INPUT
0
```

Test cases

Input	Expected
0	True
7	True

STUDY PATH: question 191 of 288 | topic 24 of 43: Digits and characters: peel one at a time and carry | 5 of 6

Template fit: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half | asked 205 days ago

HARD | PracHub | Technical Screen | 191 of 288

★ TOP 75: rank 4 in the Medium and Hard priority list

★ Check Digits and Combine Ranges

POSTED 2026-03-29 | 205 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 73.6 | 30 DAYS 63.4 | PRIORITY 2/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #9 Palindrome Number, LeetCode #56 Merge Intervals, LeetCode #228 Summary Ranges

LeetCode match: LC 9 Palindrome Number + LC 56 Merge Intervals / LC 228 Summary Ranges

Posted on PracHub: 2026-03-29 (updated 2026-05-19)

Original: https://prachub.com/coding-questions/check-digits-and-combine-ranges

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer `x`, return `true` if its decimal representation reads the same from left to right and from right to left; otherwise return `false`. Requirements and edge cases:

- Negative integers should return `false` because of the leading minus sign.
- You may assume `x` fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals `intervals`, where each interval is represented as `[start, end]`, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.

QUESTION

- Two intervals overlap if they share at least one point. For example, `[1, 4]` and `[4, 5]` should be combined into `[1, 5]`.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.
Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
Input: [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
Input: [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints

QUESTION

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

BRUTE FORCE · Task 1: convert to a string and compare with its reverse

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(x):
    s = str(x)
    return x >= 0 and s == s[::-1]
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

O(d) time but it builds strings, which the question forbids. Reversing only the second half of the digits arithmetically (x % 10, x // 10) uses O(1) space.

BRUTE FORCE · Task 2: merge any two overlapping ranges until none overlap

BRUTE FORCE · PSEUDO-CODE

```
def combine(ranges):
    ranges = list(ranges)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(ranges, 2):
            if overlaps(a, b):
                ranges.remove(a)
                ranges.remove(b)
                ranges.append(union(a, b))
                merged = True
                break
    return sorted(ranges)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Task 1: reverse only the second half arithmetically

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: sort by start, then one sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

FOLLOW-UP 2 Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

FOLLOW-UP 3 Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check Digits and Combine Ranges
#
# You are asked to solve two independent coding tasks. ### Task 1: Check Whether an Integer Reads the Same Backward
# Given an integer x, return true if its decimal
# representation reads the same from left to right and
# from right to left; otherwise return false. Requirements
# and edge cases: - Negative integers should return false
# because of the leading minus sign. - You may assume x
# fits in a signed 32-bit integer. - Try to solve it
# without converting the integer to a string. Example:
# text Input: x = 121 Output: true text Input: x = -121
# Output: false text Input: x = 10 Output: false ### Task
# 2: Combine Overlapping Ranges Given an array of
# intervals intervals, where each interval is represented
# as [start, end], combine all overlapping intervals and
# return an array of non-overlapping intervals that covers
# exactly the same ranges.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Adjacent integer ranges ([1, 2] and [3, 4]) should
# merge too.
# 2. Summarise a sorted list of integers as ranges
# (LeetCode 228).
# 3. Palindrome in another base, or as a linked list.
#
# INTERVIEW NOTES
```

```
# Task 1: a number is a palindrome if its second half
# reversed equals its first half, so reverse only half the
# digits (stop when the reversed part catches up) and
# avoid overflow. Negatives and numbers ending in 0 are
# never palindromes. Task 2: after sorting by start, an
# overlap can only be with the last merged range, so one
# pass with a 'current' range is enough.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): reverse only the second half
# arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
def is_palindrome(x, base=10):
    # no string: build the reversed second half
    if x < 0 or (x % base == 0 and x != 0):
        return False
    rev = 0
    # stop when the reversed half catches up with the rest
    while x > rev:
        rev = rev * base + x % base
        x //= base
    # an odd number of digits has a middle digit to drop
    return x == rev or x == rev // base


# TASK 2: combine ranges
def merge_ranges(intervals, adjacent=False):
    out = []
    # sort by start so overlaps are neighbours
    for s, e in sorted(intervals):
        # sharing one point counts as overlap: [1,4] +
        # [4,5]
        gap = 1 if adjacent else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out

# ---- main question: tests ----
def test_main_question():
    assert is_palindrome(121) and is_palindrome(0)
    assert not is_palindrome(-121)
    assert not is_palindrome(10)
    assert is_palindrome(1221) and not is_palindrome(123)
    # binary 101 is 5; binary 110 is 6
    assert is_palindrome(5, 2) and not is_palindrome(6, 2)
    ranges = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_ranges(ranges)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_ranges([[1, 4], [4, 5]]) == [[1, 5]]
```

```
    # any input order, and an empty list
    shuffled = merge_ranges([[8, 10], [1, 3]])
    assert shuffled == [[1, 3], [8, 10]]
    assert merge_ranges([]) == []
    # adjacent integer ranges merge only when asked
    touching = [[1, 2], [3, 4]]
    assert merge_ranges(touching) == touching
    assert merge_ranges(touching, True) == [[1, 4]]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Summarise a sorted list of integers as
# ============================================================

# ---- follow-up 2: function ----
# Answer: One pass keeping the current range and extending
# it while the next number is exactly one more; the same
# 'extend or start new' structure as the merge.
def summarize_ranges(nums):
    # LeetCode 228: a sorted list of ints as ranges
    out = []
    for x in nums:
        if out and x == out[-1][1] + 1:
            # exactly one more: extend the current range
            out[-1][1] = x
        else:
            out.append([x, x])
    return out

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert summarize_ranges([0, 1, 2, 4, 5, 7]) == [
        [0, 2], [4, 5], [7, 7]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(x: int) -> bool:
SAMPLE INPUT
121
```

Test cases

Input	Expected
121	True
-121	False

STUDY PATH: question 192 of 288 | topic 24 of 43: Digits and characters: peel one at a time and carry | 6 of 6

Template fit: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits | asked 368 days ago | frequency Single report

HARD | ColorStack | technical interview | 192 of 288

Permutation Sequence

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 206/288

ABOUT THIS QUESTION

Topic: Digits and characters: peel one at a time and carry

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 60 Permutation Sequence

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

The set [1, 2, ..., n] has n! distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from 1 , return the k -th permutation as a string. Aim for an O(n) -ish construction using the factorial number system rather than generating all permutations.

Function signature

```
def get_permutation(n: int, k: int) -> str:
    ...
```

Follow-up (as reported): A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids O(n!) .

EXAMPLES

Example 1	
Input	`n = 3, k = 3`
Output	`"213"`
Why	`Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".`

Example 2	
Input	`n = 4, k = 9`
Output	`"2314"`
Why	`The 9th permutation of 1234 in lexicographic order is "2314".`

Example 3	
Input	`n = 3, k = 1`
Output	`"123"`
Why	`The first permutation is the sorted sequence.`

CONSTRAINTS

- 1 <= n <= 9
- 1 <= k <= n!

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import permutations
def brute_permutation(n, k):
    for i, p in enumerate(
        permutations(range(1, n + 1)), 1
    ):
        if i == k:
            return "".join(map(str, p))
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

It builds permutations one by one until it reaches the k-th.

OPTIMAL SOLUTION · Factorial number system

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Given a permutation, what is its position k in the list?

Run the idea backwards: for every position count how many unused digits are smaller than the current one and multiply by the factorial of the digits that remain; add one at the end.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Permutation Sequence
#
# The set [1, 2, ..., n] has n! distinct permutations.
# Listing them in strictly increasing (lexicographic)
# order and numbering them from 1 , return the k -th
# permutation as a string. Aim for an O(n) -ish
# construction using the factorial number system rather
# than generating all permutations. Function signature:
# def get_permutation(n: int, k: int) -> str: ... Follow-
# up (as reported): A classmate asked whether a brute-
# force "generate all and index" is acceptable; discuss
# why the factorial-base method avoids O(n!) .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Given a permutation, what is its position k in the
# list?
#
# INTERVIEW NOTES
# The permutations in order are grouped: the first (n -
# 1)! start with the smallest digit, the next (n - 1)!
# with the second smallest, and so on. So the index k - 1
# divided by (n - 1)! picks the first digit from the
```

```
# sorted remaining digits, the remainder picks the rest:
# the factorial number system. No permutation needs to be
# generated.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from math import factorial
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Factorial number system
# Idea: Each digit is chosen by dividing the remaining
# index by the size of a block of permutations.
```

```
def get_permutation(n, k):
    digits = list(range(1, n + 1))
    k -= 1
    out = []
    for i in range(n, 0, -1):
        block = factorial(i - 1)
        out.append(digits.pop(k // block))
        k %= block
    return "".join(map(str, out))

# ---- main question: tests ----
def test_main_question():
    random.seed(67)
    assert get_permutation(3, 3) == "213"
    assert get_permutation(4, 9) == "2314"
    assert get_permutation(3, 1) == "123"


# ============================================================
# FOLLOW-UP 1: Given a permutation, what is its position k
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the idea backwards: for every position count
# how many unused digits are smaller than the current one
# and multiply by the factorial of the digits that remain;
# add one at the end.
def permutation_rank(perm):
    digits = sorted(int(c) for c in perm)
    rank = 0
    for i, c in enumerate(perm):
        smaller = digits.index(int(c))
        rank += smaller * factorial(len(perm) - i - 1)
        digits.pop(smaller)
    return rank + 1

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for n in range(1, 7):
        for k in range(
            1,
            factorial(n) + 1,
            max(1, factorial(n) // 40),
        ):
            assert (
                permutation_rank(get_permutation(n, k))
                == k
            )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 25 OF 43

Look at the input once and keep only a small summary (and count your calls)

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	2
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Look at the input once and keep only a small summary (and count your calls) | 6 questions | tap a title to jump to it

READ FIRST: the code template for these 6 questions (tap here) >

M	193. Two-Day Log Intersection under Memory Limit	1Point3Acres	2025-11-27
M	194. Find shared objects across two log files	PracHub	2025-11-05
M	195. Streaming Points: Emit K Within Distance	1Point3Acres	2026-06-23
M	196. Minimize calls to find all bad test pairs	PracHub	2025-11-05
H	197. Find All Bad Pairs Using runTest Oracle	1Point3Acres	2025-11-27
H	198. Find maximum value in huge text file	PracHub	2025-12-08

CODE TEMPLATE | GROUP 25 OF 43 | 6 QUESTIONS, ONE PATTERN

Look at the input once and keep only a small summary (and count your calls)

The one idea these questions share

When the input is too big to hold, read it one item at a time and keep a tiny summary: a maximum, a set from the smaller file, a sorted window. When asking a question is expensive (an oracle), use binary search on how many items you include so the number of calls stays small.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def stream_max(lines):
    best = None                                  # CHANGE 1: the small summary you keep
    for line in lines:                           # a file object gives one line at a time
        line = line.strip()
        if not line:
            continue
        value = int(line)                        # CHANGE 2: parse what matters from the line
        if best is None or value > best:
            best = value                         # CHANGE 3: update the summary
    return best
```

2. Helper functions that usually get written next to it

```
def first_failing_prefix(items, passes):         # binary search with an oracle: fewest calls
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if passes(items[:mid + 1]):
            lo = mid + 1
        else:
            hi = mid
    return lo if lo < len(items) else -1
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
193	Two-Day Log Intersection under Memory Limit	dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes
194	Find shared objects across two log files	load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set
195	Streaming Points: Emit K Within Distance	keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked
196	Minimize calls to find all bad test pairs	the same group testing: put a whole group in one call and split only the groups that fail
197	Find All Bad Pairs Using runTest Oracle	for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left
198	Find maximum value in huge text file	the template as written; the file is never loaded as a whole

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 193 of 288 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 1 of 6

Template fit: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes | asked 327 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 193 of 288

Two-Day Log Intersection under Memory Limit

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 28/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 349 Intersection of Two Arrays (related)

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients
    {c1, c2}.
Output: [X]
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(first, second):
    result = []
    for obj in {r.obj for r in first}:
        if obj not in {r.obj for r in second}:
            continue
        rows = [r for r in first + second
                if r.obj == obj]
        if len({r.client for r in rows}) >= 2:
            result.append(obj)
    return result
```

TIME	SPACE
O(n * m)	O(n + m)

WHY IT IS SLOW

O(n * m) time and both files must fit in memory. Sorting both files by object and merging them with two pointers is O(n log n) and holds one object's clients at a time.

OPTIMAL SOLUTION · Hash map of client sets (everything fits in memory)

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Two-Day Log Intersection under Memory
# Limit
#
# PhD intern coding round 1: two log files with
# (timestamp, obj_id, client_id) entries. Find obj_ids
# that (a) appear in both files and (b) have at least two
# distinct client_ids. Follow-up imposes a memory limit,
# expecting external sort + two-pointer. Input: two log
# files; each line is (timestamp, obj_id, client_id). Find
# every obj_id such that: (a) appears in both files, and
# (b) is associated with at least 2 distinct client_ids
# (aggregated across both files). Follow-up 1: complexity
```

```
# of set-intersection on the file contents. Follow-up 2:
# memory cannot hold either file fully — design a
# streaming solution. Examples: File A: (t1, X, c1), (t2,
# X, c2), (t3, Y, c1) File B: (t4, X, c1), (t5, Z, c2) ->
# Only X is in both files; X has 2 distinct clients {c1,
# c2}. Output: [X]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why prefer sort and two pointers to a streaming hash
# map?
#
# INTERVIEW NOTES
# In memory it is a hash map from object to the set of
# clients, filtered to objects that appear in both files
# and have at least two clients. When the files do not fit
# in memory, sort both by object id (external sort) and
# walk them together: all rows of one object are adjacent,
# so you only ever hold the clients of one object at a
# time.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
# Idea: Build object -> set of clients for each file, keep
# the objects that appear in both, and count the union of
# their client sets.
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)

# ---- main question: tests ----
def test_main_question():
    day1 = [(1, "X", "c1"), (2, "X", "c2"),
            (3, "Y", "c1")]
    day2 = [(4, "X", "c1"), (5, "Z", "c2")]
    assert log_objects(day1, day2) == ["X"]
    def by_object(rows):
        return sorted(rows, key=lambda r: r[1])


# ============================================================
# FOLLOW-UP 1: Why prefer sort and two pointers to a
# ============================================================

# ---- follow-up 1: helper functions ----
def groups(rows):
    # rows sorted by object: yield (object, its clients)
    current, clients = None, set()
    for _, obj, client in rows:
        if obj != current:
            if current is not None:
                yield current, clients
            current, clients = obj, set()
        clients.add(client)
    if current is not None:
        yield current, clients
```

```
# ---- follow-up 1: function ----
# Answer: A hash map needs memory proportional to the
# number of distinct objects (and clients), which is
# exactly what the limit forbids.
def log_objects_sorted(rows1, rows2):
    # memory limit: both files arrive sorted by object (an
    # external sort), so only one object's clients are
    # held
    out = []
    g1, g2 = groups(rows1), groups(rows2)
    a, b = next(g1, None), next(g2, None)
    while a and b:
        if a[0] == b[0]:
            # same object in both files
            if len(a[1] | b[1]) >= 2:
                out.append(a[0])
            a, b = next(g1, None), next(g2, None)
        elif a[0] < b[0]:
            a = next(g1, None)
        else:
            b = next(g2, None)
    return out

# ---- follow-up 1: tests ----
def test_follow_up_1():
    day1 = [(1, "X", "c1"), (2, "X", "c2"),
            (3, "Y", "c1")]
    day2 = [(4, "X", "c1"), (5, "Z", "c2")]
    def by_object(rows):
        return sorted(rows, key=lambda r: r[1])
    assert log_objects_sorted(by_object(day1),
                              by_object(day2)) == ["X"]
    # one client in total is not enough
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []
    assert log_objects_sorted(one, one) == []
    # clients may be split across the two days
    d1, d2 = [(1, "A", "c1")], [(2, "A", "c2")]
    assert log_objects(d1, d2) == ["A"]
    assert log_objects_sorted(d1, d2) == ["A"]
    assert log_objects([], day2) == []


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY PATH: question 194 of 288 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 2 of 6

Template fit: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set | asked 349 days ago

MEDIUM | PracHub | Technical Screen | 194 of 288

★ TOP 75: rank 35 in the Medium and Hard priority list

★ Find shared objects across two log files

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 27/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: the interview site's own solution

LeetCode match: LC 349 Intersection of Two Arrays

Posted on PracHub: 2025-11-05 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-shared-objects-across-two-log-files

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.
   - You may still sequentially scan each file and you may perform external sorting.
   - Describe how you would modify your approach to work under this memory constraint.
   - Analyze the time and space complexity of this memory-efficient solution.

QUESTION

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']],
    [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(file_a, file_b):
    shared = set()
    for a in file_a:
        for b in file_b:
            if a.obj_id == b.obj_id:
                shared.add(a.obj_id)
    return shared
```

TIME	SPACE
O(n * m)	O(1) extra

WHY IT IS SLOW

O(n * m) comparisons. Loading the smaller file's ids into a hash set and streaming the other file is O(n + m); if neither fits, partition both by a hash of the id.

OPTIMAL SOLUTION · Hash set of the smaller file's ids

TIME	SPACE
O(n + m)	O(distinct ids of the smaller file)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

FOLLOW-UP 2 Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

FOLLOW-UP 3 A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. **Cost:** O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find shared objects across two log files
#
# You are given two large log files representing activity
# on two different days. Each line of each log has three
# fields: - timestamp: a time value (you may treat it as
# an opaque string or integer) - obj_id: identifier of an
# object (string or integer) - client_id: identifier of a
# client (string or integer) The logs are not necessarily
# sorted by any field. Define an object as **interesting**
# if: 1. Its obj_id appears at least once in day 1's log
# **and** at least once in day 2's log; and 2. Across all
# of its appearances in both days combined, it is
# associated with at least **two distinct** client_id
# values. ### Tasks 1. Design and implement a function
# that, given file handles (or streaming iterators) for
# the two log files, returns the set of all such
# interesting obj_ids. 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both files are too big for memory.
# 2. Want objects seen on both days within an hour of each
# other.
# 3. A tiny memory budget and an approximate answer is
# fine.
#
# INTERVIEW NOTES
# A common key in two files is a join. If one file fits in
# memory, load its keys into a set and stream the other.
# If neither fits, split both files by hash of the key
# into buckets small enough for memory; equal keys always
# land in the same bucket number, so you only compare
# bucket i of one file with bucket i of the other (Grace
# hash join).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
import zlib
from collections import defaultdict
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(day1, day2):
    # rows are [timestamp, obj_id, client_id]. Remember,
    # per
    # object, which days it appeared on and its clients.
    days, clients = defaultdict(set), defaultdict(set)
    for number, rows in ((1, day1), (2, day2)):
        for _, obj, client in rows:
            days[obj].add(number)
            clients[obj].add(client)
    return sorted(o for o in days
                  if len(days[o]) == 2
                  and len(clients[o]) >= 2)


# OTHER WAY: external sort of both days
def shared_objects_sorted(day1, day2):
    # the other way: external sort both days by object,
    # then a
    # merge that holds only one object's clients at a time
    a = sorted(day1, key=lambda r: r[1])
    b = sorted(day2, key=lambda r: r[1])
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
        else:
            obj, clients = a[i][1], set()
            while i < len(a) and a[i][1] == obj:
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
```

```
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                out.append(obj)
    return out

# ---- main question: tests ----
def test_main_question():
    d1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
    d2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]
    assert shared_objects(d1, d2) == ["o1"]
    assert shared_objects([], [["t", "o", "c"]]) == []


# ============================================================
# FOLLOW-UP 1: Both files are too big for memory.
# ============================================================

# ---- follow-up 1: helper functions ----
def bucket_of(obj, buckets):
    # a stable hash (Python's own hash changes between
    # runs)
    return zlib.crc32(str(obj).encode()) % buckets
```

```
# ---- follow-up 1: function ----
# Answer: Hash-partition both files into B buckets on disk
# by hash of the object id, then process bucket pairs one
# at a time with the in-memory set join. Choose B so that
# one bucket of the smaller file fits in memory.
def shared_objects_partitioned(day1, day2, buckets=4):
    # memory limit: write each row to a bucket "file"
    # chosen by
    # the hash of its object, then solve one bucket at a
    # time.
    # All rows of an object land in the same bucket.
    parts1 = [[] for _ in range(buckets)]
    parts2 = [[] for _ in range(buckets)]
    for rows, parts in ((day1, parts1), (day2, parts2)):
        for row in rows:
            parts[bucket_of(row[1], buckets)].append(row)
    out = []
    for b in range(buckets):
        out += shared_objects(parts1[b], parts2[b])
    return sorted(out)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    d1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
    d2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]
    assert shared_objects_partitioned(d1, d2, 3) == ["o1"]
    assert shared_objects_sorted(d1, d2) == ["o1"]
    many1 = [[i, f"o{i % 7}", f"c{i % 3}"]
             for i in range(40)]
    many2 = [[i, f"o{i % 5}", f"c{i % 2}"]
             for i in range(40)]
    want = shared_objects(many1, many2)
    parts = shared_objects_partitioned(many1, many2, 5)
    assert parts == want
    assert shared_objects_sorted(many1, many2) == want


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: A tiny memory budget and an approximate
# ============================================================

# ---- follow-up 3: function ----
# Answer: Build a Bloom filter from one file and test the
# other against it, accepting a small false-positive rate;
# verify the few hits exactly in a second pass if needed.
class Bloom:
    """Tiny memory, approximate: may say 'yes' for a few
    objects that were never added."""

    def __init__(self, bits=1 << 16, hashes=3):
        self.bits, self.hashes = bytearray(bits), hashes

    def spots(self, key):
        for seed in range(self.hashes):
            text = f"{seed}:{key}".encode()
            yield zlib.crc32(text) % len(self.bits)

    def add(self, key):
        for s in self.spots(key):
            self.bits[s] = 1
```

```
    def maybe(self, key):
        return all(self.bits[s] for s in self.spots(key))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    d1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
    bloom = Bloom()
    for row in d1:
        bloom.add(row[1])
    assert bloom.maybe("o1") and not bloom.maybe("o999")


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Two-Day Log Intersection under Memory Limit | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.
Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).
The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

Preparation: Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY PATH: question 195 of 288 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 3 of 6

Template fit: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked | asked 119 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 195 of 288

Streaming Points: Emit K Within Distance

ASKED 2026-06-23 | 119 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 259/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Positions (floats) arrive one at a time. Whenever k stored points fit inside a span of d (largest minus smallest is at most d), output those k points and remove them. add(x) returns the groups emitted because of this arrival (a list of lists).

Function signature

class PointStream:
  def __init__(self, k: int, d: float)
  def add(self, x: float) -> list

EXAMPLES

Example 1

Input	`s = PointStream(3, 2.0)`
	`s.add(1.0)`
	`s.add(5.0)`
	`s.add(2.0)`
	`s.add(1.5)`

Output	`s.add(1.0) -> []`
	`s.add(5.0) -> []`
	`s.add(2.0) -> []`
	`s.add(1.5) -> [[1, 1.5, 2]]`

Why	After 1, 5, 2 no three points fit in a span of 2 (1 and 5 are too far apart); adding 1.5 gives 1, 1.5, 2 with span 1, which is emitted and removed.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
points = []
def on_point(x):
    points.append(x)
    points.sort()
    emitted = []
    while True:
        group = find_group()
        if group is None:
            return emitted
        emitted.append(group)
        remove(points, group)
def find_group():
    for i in range(len(points) - k + 1):
        if points[i + k - 1] - points[i] <= d:
            return points[i:i + k]
    return None
```

TIME	SPACE
O(n log n) per point	O(n)

WHY IT IS SLOW

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

TIME	SPACE
O(log n + K) per check, plus list shifting	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

FOLLOW-UP 2 Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

FOLLOW-UP 3 Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

FOLLOW-UP 4 The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Streaming Points: Emit K Within Distance
#
# Positions (floats) arrive one at a time. Whenever k
# stored points fit inside a span of d (largest minus
# smallest is at most d), output those k points and remove
# them. add(x) returns the groups emitted because of this
# arrival (a list of lists).
#
# Signature:
#   class PointStream:
#       def __init__(self, k: int, d: float)
#       def add(self, x: float) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if 'within d' means every pair, and points are
# in 2D?
# 2. Floating-point coordinates.
# 3. Several groups qualify at once.
# 4. The condition is 'every pair within d', not 'span at
# most d'.
#
# INTERVIEW NOTES
# On a line, K points are all within distance d exactly
# when the largest minus the smallest is at most d, so
# only K consecutive points in sorted order can form a
# group. Keep the points sorted and look only where
# something changed: a window that contains the new point,
# or the seam left by removed points.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Sorted list, check only windows

# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i

# ---- main question: tests ----
def test_main_question():
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
    assert s.add(1.0) == []
    # 0.5, 1.0, 1.4 span 0.9 <= 1.0
    out = s.add(1.4)
    assert out == [[0.5, 1.0, 1.4]]
```

```
    # the far point is still waiting
    assert s.points == [5.0]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
```

```
-UP 3.

# ============================================================
# FOLLOW-UP 4: The condition is 'every pair within d', not
# ============================================================

# ---- follow-up 4: function ----
# Answer: On a line they are the same: the largest
# pairwise distance is max - min, so checking the span is
# enough (verified against checking all pairs). In 2D they
# differ and become a clique problem.
def all_pairs_within(group, d):
    # "every pair within d": on a line this equals the
    # span
    pairs = [(a, b) for a in group for b in group]
    return all(abs(a - b) <= d for a, b in pairs)

# ---- follow-up 4: tests ----
def test_follow_up_4():
    s = PointStream(3, 1.0)
    # 0.5, 1.0, 1.4 span 0.9 <= 1.0
    out = s.add(1.4)
    # on a line, span and pairwise checks agree
    for group in out:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0
    # k = 2 groups closest neighbours as they appear
    t = PointStream(2, 0.5)
```

```
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

STUDY PATH: question 196 of 288 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 4 of 6

Template fit: the same group testing: put a whole group in one call and split only the groups that fail | asked 349 days ago

MEDIUM | PracHub | Technical Screen | 196 of 288

Minimize calls to find all bad test pairs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 254/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (adaptive group testing)

Posted on PracHub: 2025-11-05 (updated 2026-06-18)

Original: https://prachub.com/coding-questions/minimize-calls-to-find-all-bad-test-pairs

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

QUESTION

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints

0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for a, b in combinations(tests, 2):
    if not runTest({a, b}):
        bad.append((a, b))
```

TIME	SPACE
n(n - 1) / 2 calls	O(1)

WHY IT IS SLOW

n(n - 1) / 2 calls even when nothing is bad. Keeping groups of mutually compatible tests and binary searching only when a group fails needs about n + p log n calls for p bad pairs.

OPTIMAL SOLUTION · Independent groups plus binary search for partners

TIME	SPACE
O(n * g + p log n) calls (g = number of groups, p = number of bad pairs)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls does it need, and can you prove a lower bound?

With p bad pairs among n tests about p log n calls are needed in the worst case (each pair carries log n bits of information about where it is), and the group method uses O(n * g + p log n). With no bad pairs it uses n - 1 calls, one per test after the first.

FOLLOW-UP 2 Only one bad pair exists.

Binary search for the smallest failing prefix (its last element is in the pair), then binary search among the earlier elements for the partner: about 2 log n calls.

FOLLOW-UP 3 runTest is expensive but can run in parallel.

Test all groups for the new element in the same round, or process several elements at once with their own groups, trading a few extra calls for fewer sequential rounds.

FOLLOW-UP 4 Only one bad pair exists.

Binary search the shortest failing prefix (its last element is in the pair), then binary search the earlier tests for the partner: about 2 log2(n) calls instead of n(n - 1) / 2 (verified by counting calls).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimize calls to find all bad test pairs
#
# You have \(n\) atomic tests \(t_1, t_2, \dots, t_n\).
# You are given access to a black-box function: python
# bool runTest(set<Test> S) which behaves as follows: -
# runTest(S) == True if the subset \(S\) contains **no**
# incompatible (bad) pair of tests. - runTest(S) == False
# if the subset \(S\) contains **at least one**
# incompatible (bad) pair of tests. There is some unknown
# set of unordered **bad pairs** \((t_i, t_j)\), where
# \(i\ne j\). You do **not** know in advance which pairs
# are bad or how many bad pairs there are. Your goal is to
# discover **all** bad pairs while making as few calls to
# runTest as possible. You may adaptively choose each
# subset \(S\) based on the results of previous calls. ###
# Tasks 1. Design an algorithm that, using only calls to
# runTest, finds **all** bad pairs \((t_i, t_j)\). 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls does it need, and can you prove a
# lower bound?
# 2. Only one bad pair exists.
# 3. runTest is expensive but can run in parallel.
# 4. Only one bad pair exists.
#
# INTERVIEW NOTES
# Each call is a yes/no question about a whole set, so
# binary search is possible: a failing set can be narrowed
# by halving. Keep the tests you have processed in groups
# that each pass runTest (independent sets). A new test
# only needs to be checked against each group, and if a
# group plus the new test fails, binary search finds its
# partners in that group in about log(size) calls each.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import itertools



# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def shortest_failing_prefix(items, extra, run_test):
    # binary search: the smallest prefix of `items` that
    # fails
    # together with `extra`; its last item is a partner
    lo, hi = 1, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(items[:mid]) | extra):
            lo = mid + 1
        else:
```

```
            hi = mid
    return items[lo - 1]


def partners(group, t, run_test):
    # every member of the group that is bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)
    return found

# ---- main question: main function ----
# BASE QUESTION (optimal): Independent groups plus binary
# search for partners
# Idea: Process tests one by one. Keep groups of tests
# that pass runTest.
def find_bad_pairs(tests, run_test):
    # keep groups of tests that pass together; a new test
    # is
    # checked against each group and joins one with no
    # partner
    groups, pairs = [], set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            if not found and not placed:
```

```
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs

# ---- main question: tests ----
def test_main_question():
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls
    bad = {(3, 17), (5, 29)}
    run, calls = make_oracle(bad)
    assert find_bad_pairs(list(range(30)), run) == bad
    assert calls[0] < 435 // 2
    run, calls = make_oracle(set())
    assert find_bad_pairs(list(range(30)), run) == set()
    assert calls[0] <= 30
    run, calls = make_oracle({(10, 40)})


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Only one bad pair exists.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Binary search for the smallest failing prefix
# (its last element is in the pair), then binary search
```

```
# among the earlier elements for the partner: about 2 log
# n calls.
# FOLLOW-UP 4: Only one bad pair exists.
# Answer: Binary search the shortest failing prefix (its
# last element is in the pair), then binary search the
# earlier tests for the partner: about 2 log2(n) calls
```

```
# instead of n(n - 1) / 2 (verified by counting calls).
def one_bad_pair(tests, run_test):
    # exactly one bad pair: about 2 * log2(n) calls
    if run_test(set(tests)):
        return None
    # the shortest failing prefix ends with one of the
    # pair
    b = shortest_failing_prefix(tests, set(), run_test)
    earlier = tests[:tests.index(b)]
    # then search the earlier tests for the partner
    a = shortest_failing_prefix(earlier, {b}, run_test)
    return (a, b)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls
    bad = {(3, 17), (5, 29)}
    run, calls = make_oracle(bad)
    run, calls = make_oracle(set())
    run, calls = make_oracle({(10, 40)})
    assert one_bad_pair(list(range(64)), run) == (10, 40)
    # about 2 * log2(64) = 12 calls, far below 2016 pairs
    assert calls[0] <= 16
    run, _ = make_oracle({(1, 2)})


# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Only one bad pair exists.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, bad_pairs: list[list[int]]) -> tuple[
list[list[int]], int]:
```

Sample call

```
solution(0, [])
```

Test cases

Input	Expected
`(0, [])`	`([], 0)`
`(4, [])`	`([], 3)`

ALSO REPORTED · 1 more

Find All Bad Pairs Using runTest Oracle | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition.
Algorithm sketch:
Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.
Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive $O(N^2)$ pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

Preparation: Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

STUDY PATH: question 197 of 288 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 5 of 6

Template fit: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left | asked 327 days ago | frequency Single report

HARD | 1Point3Acres | onsite-coding | 197 of 288

Find All Bad Pairs Using runTest Oracle

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 252/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, RS

Original: https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **S** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair
    somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair
    containing items from this set.
... binary search to isolate.
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for i in range(n):
    for j in range(i + 1, n):
        if not runTest({i, j}):
            bad.append((i, j))
```

TIME	SPACE
O(N^2) calls	O(1)

WHY IT IS SLOW

Always N(N - 1) / 2 calls. Testing whole groups and binary searching a failing group finds each bad pair in about log N calls.

OPTIMAL SOLUTION · Group the tests and binary search for partners

TIME	SPACE
O(K log N + N * G) about calls (K bad pairs, G groups)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls are needed in the best case and the worst case?

With no bad pairs it takes N - 1 calls, one per test after the first. With K bad pairs each one costs about log N calls to locate, so roughly K log N plus the cost of building clean groups. The brute force is always N(N-1)/2, which is far larger as soon as bad pairs are rare.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find All Bad Pairs Using runTest Oracle
#
# PhD intern coding round 2: given a black-box oracle
# runTest(S) that returns true iff a subset S contains no
# bad pair, identify all bad pairs across N test items
# with as few oracle calls as possible. Items t_1, ...,
# t_N are given. Oracle: runTest(S: Set[Item]) -> bool
# returns true iff S contains zero bad pairs; otherwise
# false. Output: every bad pair (t_i, t_j). Minimize total
# oracle calls. Examples: N=4, bad pairs = {(1,2)}.
# runTest({1,2,3,4}) -> false -> at least one bad pair
# somewhere. runTest({1,3,4}) -> true -> 2 is in every bad
# pair containing items from this set. ... binary search
# to isolate.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls are needed in the best case and the
# worst case?
#
# INTERVIEW NOTES
# Testing every pair costs N(N-1)/2 calls. A single call
# on a whole set answers 'is there any bad pair inside',
# so you can discard good sets fast and binary-search for
# the culprit inside a bad one. Add the tests one at a
# time to groups known to be clean: if a group plus the
# new test fails, binary search the group for a partner;
# otherwise the new test joins it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import itertools
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def first_partner(group, t, run_test):
    # binary search: the shortest prefix of the group that
```

```
    # fails together with t; its last member is a partner
    lo, hi = 1, len(group)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(group[:mid]) | {t}):
            lo = mid + 1
        else:
            hi = mid
    return group[lo - 1]


def partners(group, t, run_test):
    # all members of the group that are bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        # take it out and test the others again
        rest.remove(x)
    return found

# ---- main question: main function ----
# BASE QUESTION (optimal): Group the tests and binary
# search for partners
# Idea: Keep groups of tests that pass together.
def find_bad_pairs(items, run_test):
    # groups of items that pass together (no bad pair
    # inside)
    groups, pairs = [], set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            # t joins the first group with no partner for
            # it
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs

# ---- main question: tests ----
```

```
def test_main_question():
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls
    bad = {(3, 17), (5, 29)}
    run, calls = make_oracle(bad)
    assert find_bad_pairs(list(range(30)), run) == bad
    assert calls[0] < 435 // 2
    # no bad pairs: about one call per item
    run, calls = make_oracle(set())
    assert find_bad_pairs(list(range(30)), run) == set()
    assert calls[0] <= 30
    run, _ = make_oracle({(1, 2)})


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

STUDY PATH: question 198 of 288 | topic 25 of 43: Look at the input once and keep only a small summary (and count your calls) | 6 of 6

Template fit: the template as written; the file is never loaded as a whole | asked 316 days ago

HARD | PracHub | Technical Screen | 198 of 288

Find maximum value in huge text file

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 245/288

ABOUT THIS QUESTION

Topic: Look at the input once and keep only a small summary (and count your calls)

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: No LC match (systems / scalability question; the Hard tag looks inflated)

Posted on PracHub: 2025-12-08 (updated 2026-06-27)

Original: https://prachub.com/interview-questions/find-maximum-value-in-huge-text-file

QUESTION

A text file is too large to load into memory (tens or hundreds of gigabytes). Each line holds one signed 64-bit integer. Return the largest value by reading the file sequentially, holding only a small amount in memory.

Function signature

def max_in_file(path: str) -> int

EXAMPLES

Example 1

Input	`nums.txt contains the lines:`
	`5`
	`-3`
	`12`
	`7`

Output	`max_in_file(path) -> 12`
Why	`Only the current line and the best value so far are in memory, so the file size does not matter.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(path):
    numbers = []
    for line in open(path):
        numbers.append(int(line))  # in memory
    return max(numbers)
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Needs memory for the whole file, which is exactly what is not available at tens of gigabytes. Streaming with one running maximum uses O(1) memory.

OPTIMAL SOLUTION · Single machine: stream the file, keep one running maximum

TIME	SPACE
O(n)	O(1) plus the read buffer

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: split byte ranges on line boundaries (map), max of the maxima (reduce)

TIME	SPACE
O(n / workers) plus I/O limits	O(workers)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Edge cases to mention.

Empty file (return None or raise, by contract); a final line without a newline; blank lines; malformed lines; values at the 64-bit extremes; an all-negative file (do not initialize the max to 0); and Windows line endings. Decide whether a bad line fails the job or is skipped and counted.

FOLLOW-UP 2 How do you scale to many machines and handle failures?

Assign byte ranges (map tasks) to workers through a coordinator, and make each task idempotent (it only returns its maximum), so a failed or slow worker is simply retried or speculatively re-executed elsewhere. The reduce step is a max over a tiny list. On a distributed filesystem, align ranges to its block size for locality. **Cost:** O(n / workers) wall-clock time plus O(workers) to combine; total work stays O(n).

FOLLOW-UP 3 Is it I/O-bound or CPU-bound, and why does that matter?

Parsing integers is cheap compared with disk throughput, so one sequential reader often saturates the disk and extra threads help only if the data sits on several disks or on fast SSD or network storage. Measure first; the parallel version pays off mainly across machines or storage devices. **Cost:** reading takes about n / disk throughput and parsing is O(n) as well, so extra threads help only beyond the disk's speed.

FOLLOW-UP 4 The file has billions of lines and one machine is too slow.

Split by byte offsets across machines (each skips to the next newline), compute local maxima, and take the maximum of those few numbers. Tasks are idempotent, so a failed worker can simply be retried. **Cost:** O(n / machines) per machine and O(machines) to combine; a retry re-runs only the failed byte range.

FOLLOW-UP 5 Now return the K largest values instead of only the maximum.

Keep a min-heap of size K while streaming: push a value if the heap is not full, or replace the root when the value is larger. Memory O(K), time O(n log K); merge the per-worker heaps for a parallel version.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find maximum value in huge text file
#
# A text file is too large to load into memory (tens or
# hundreds of gigabytes). Each line holds one signed
# 64-bit integer. Return the largest value by reading the
# file sequentially, holding only a small amount in
# memory.
#
# Signature:
#   def max_in_file(path: str) -> int
#
# FOLLOW-UPS (answered below, in this order):
# 1. Edge cases to mention.
# 2. How do you scale to many machines and handle
# failures?
# 3. Is it I/O-bound or CPU-bound, and why does that
# matter?
# 4. The file has billions of lines and one machine is too
# slow.
# 5. Now return the K largest values instead of only the
# maximum.
#
# INTERVIEW NOTES
# The answer is a running maximum, which needs one
# variable, so memory should not depend on the file size.
# Read the file as a stream. To go faster across cores,
# give each worker a byte range that starts and ends on
# line boundaries; the global maximum is the maximum of
# the partial maxima.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
import heapq
import os
import tempfile


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Single machine: stream the
# file, keep one running maximum
# Idea: Never load the file. Read it through a buffered
# iterator and keep one variable, the maximum so far,
# started at None so that an all-negative file or a one-
# line file is handled correctly (do not start at 0).
def max_in_file(path):
    # one sequential pass, one number in memory
    best = None
    with open(path, "r", buffering=1 << 20) as f:
        for line in f:
            line = line.strip()
            # blank lines are ignored (state this
            # contract)
            if not line:
                continue
            value = int(line)
            # start from None, not 0: all numbers may be
            # negative
            if best is None or value > best:
                best = value
    return best


# ONE WORKER: its byte range
def max_in_range(path, start, end):
    # one worker: lines that START inside [start, end)
    best = None
    with open(path, "rb") as f:
        f.seek(start)
        if start:
            # we may be in the middle of a line: the
            # previous
            # worker owns it, so skip to the next line
            # start
            f.seek(start - 1)
            f.readline()
        while f.tell() < end:
            line = f.readline()
            if not line:
                break
            if line.strip():
                v = int(line)
```

```
                best = v if best is None else max(best, v)
    return best

# ---- main question: tests ----
def test_main_question():
    def write(lines):
        f = tempfile.NamedTemporaryFile("w", delete=False)
        f.write("".join(lines))
        f.close()
        return f.name
    p = write(["5\n", "-3\n", "\n", "99\n", "12"])
    assert max_in_file(p) == 99


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
```

```
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: The file has billions of lines and one
# ============================================================

# ---- follow-up 4: function ----
# Answer: Split by byte offsets across machines (each
# skips to the next newline), compute local maxima, and
# take the maximum of those few numbers. Tasks are
# idempotent, so a failed worker can simply be retried.
def parallel_max(path, workers=4):
    # split by byte offsets; combine the few local maxima
    size = os.path.getsize(path)
    cuts = [size * i // workers
            for i in range(workers + 1)]
    parts = [max_in_range(path, a, b)
             for a, b in zip(cuts, cuts[1:])]
    parts = [p for p in parts if p is not None]
    return max(parts) if parts else None

# ============================================================
# FOLLOW-UP 5: Now return the K largest values instead of
# ============================================================

# ---- follow-up 5: function ----
# Answer: Keep a min-heap of size K while streaming: push
# a value if the heap is not full, or replace the root
# when the value is larger. Memory O(K), time O(n log K);
# merge the per-worker heaps for a parallel version.
def top_k_in_file(path, k):
    # the k largest values: a min-heap of size k, O(k)
    # memory
    heap = []
    with open(path) as f:
        for line in f:
            if not line.strip():
                continue
            value = int(line)
            if len(heap) < k:
                heapq.heappush(heap, value)
            elif value > heap[0]:
                heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)

# ---- follow-up 5: tests ----
def test_follow_up_5():
    def write(lines):
        f = tempfile.NamedTemporaryFile("w", delete=False)
        f.write("".join(lines))
        f.close()
        return f.name
    p = write(["5\n", "-3\n", "\n", "99\n", "12"])
    assert top_k_in_file(p, 2) == [99, 12]
    assert parallel_max(p, 3) == 99
    # all negative numbers, extreme 64-bit values
    q = write(["-7\n", "-2\n", "-9\n"])
    assert max_in_file(q) == -2
    big = write([f"{2 ** 63 - 1}\n", f"{-2 ** 63}\n"])
    assert max_in_file(big) == 2 ** 63 - 1
    # an empty file has no maximum
    e = write([])
    assert max_in_file(e) is None
    assert parallel_max(e) is None
    # every worker count agrees on a bigger file
    nums = [(i * 7919) % 10007 - 5000
            for i in range(2000)]
    r = write([f"{x}\n" for x in nums])
    for w in (1, 2, 5, 8):
        assert parallel_max(r, w) == max(nums)
    for name in (p, q, big, e, r):
        os.remove(name)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

You are given the path to a very large text file on disk. The file has the following properties:

Each line contains exactly one signed 64-bit integer (in decimal).
The file size can be tens or hundreds of gigabytes, so it cannot be fully loaded into memory.
You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks).
Tasks
Single-machine solution
Design an algorithm to compute the maximum integer value in this file.
Assume you have limited RAM (much smaller than the file size).
Discuss how you would read the file and track the result.
Give high-level pseudocode.
Analyze time and space complexity.
Performance improvement / parallelization
Suppose you now have a multi-core machine or multiple machines available and want to speed up the computation.
How would you modify your approach to take advantage of parallelism (e.g., processing the file in chunks)?
Explain how you would combine partial results to get the global maximum.
Mention any edge cases or failure scenarios you would consider (e.g., very large values, empty file).

Assume you do not have a database; you are working directly with the file system and your programming language's standard I/O libraries.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 26 OF 43

Grid DP: each cell from its neighbours above and to the left

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	2
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Grid DP: each cell from its neighbours above and to the left | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	199. Grid Path Count with Diagonal Moves and Checkpoints	1Point3Acres	2026-09-29
M	200. Longest Same-Color Grid Sequence	1Point3Acres	2026-09-08
M	201. Count Same-Color Squares in Matrix	1Point3Acres	no date
H	202. Find the Longest Straight Run of One Color	PracHub	2026-08-23
H	203. Count same-color squares in a character grid	PracHub	2025-11-22

CODE TEMPLATE | GROUP 26 OF 43 | 5 QUESTIONS, ONE PATTERN

Grid DP: each cell from its neighbours above and to the left

The one idea these questions share

Walk the grid row by row and keep only the previous row. A cell's value depends on the cell above, the cell to the left, and the diagonal. Change the condition for a cell to count and the formula that combines the three neighbours.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def grid_dp(g):
    m, n = len(g), len(g[0])
    prev, total = [0] * n, 0
    for i in range(m):
        cur = [1] * n
        for j in range(n):
            if i and j and g[i][j] == g[i - 1][j] == g[i][j - 1] == g[i - 1][j - 1]:   # CHANGE 1
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])                      # CHANGE 2
        total += sum(cur)                                                               # CHANGE 3
        prev = cur
    return total
```

2. Helper functions that usually get written next to it

```
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))      # the four line directions on a grid


def inside(r, c, rows, cols):
    return 0 <= r < rows and 0 <= c < cols
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
199	Grid Path Count with Diagonal Moves and Checkpoints	advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint
200	Longest Same-Color Grid Sequence	keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest
201	Count Same-Color Squares in Matrix	the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes
202	Find the Longest Straight Run of One Color	the same four run lengths; extend a run when the neighbour in that direction has the same colour
203	Count same-color squares in a character grid	exactly the same recurrence on characters

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 199 of 288 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 1 of 5

Template fit: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint | asked 21 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 199 of 288

Grid Path Count with Diagonal Moves and Checkpoints

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 48.9 | 30 DAYS 33 | PRIORITY 42/288

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #62 Unique Paths

LeetCode match: LC 62 Unique Paths (related)

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/grid-path-count-three-moves-checkpoints

QUESTION

Count the paths in an n x m grid from the bottom-left cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1). From (r, c) you may move up-right (r - 1, c + 1), right (r, c + 1) or down-right (r + 1, c + 1) and must stay inside the grid. Optionally you are given checkpoint cells that every path has to pass through; with ordered=True they must be visited in the order given.

Function signature

def count_paths(n: int, m: int, checkpoints=(), ordered=False) -> int

EXAMPLES

Example 1	
Input	3, 3
Output	2
Why	Two paths: through (1, 1) and then down-right, or through (2, 1) and then right.

Example 2	
Input	3, 3, checkpoints = [(1, 1)]
Output	1
Why	The path must also pass through (1, 1), the middle cell.

Example 3	
Input	3, 4, checkpoints = [(1, 1), (2, 2)], ordered = true
Output	1
Why	Two checkpoints in different columns, visited in column order.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def count(r, c, seen):
    if c == m - 1:
        done = r == n - 1 and checkpoints <= seen
        return 1 if done else 0
    total = 0
    for dr in (-1, 0, 1):
        nr = r + dr
        if 0 <= nr < n:
            nxt = seen | {(nr, c + 1)}
            total += count(nr, c + 1, nxt)
    return total
answer = count(n - 1, 0, {(n - 1, 0)})
```

TIME	SPACE
O(3^m)	O(m)

WHY IT IS SLOW

Up to 3^m paths. Every move advances one column, so a dp over (column, row) with a rolling array counts them in O(n * m).

OPTIMAL SOLUTION · Column DP with a rolling array and checkpoint filters

TIME	SPACE
O(n * m)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Grid Path Count with Diagonal Moves and
# Checkpoints
#
# Count the paths in an n x m grid from the bottom-left
# cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1).
# From (r, c) you may move up-right (r - 1, c + 1), right
# (r, c + 1) or down-right (r + 1, c + 1) and must stay
# inside the grid. Optionally you are given checkpoint
# cells that every path has to pass through; with
# ordered=True they must be visited in the order given.
#
# Signature:
#   def count_paths(n: int, m: int, checkpoints=(), ordere
```

```
d=False) -> int
#
# Examples:
# 3, 3  ->  2
# 3, 3, checkpoints = [(1, 1)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two checkpoints share a column.
#
# INTERVIEW NOTES
# Every move goes exactly one column to the right, so a
# path visits each column exactly once and the state is
# just 'which row am I in at this column'. That is a DP
# over columns with a single rolling array. A checkpoint
# (r, c) simply says the path must be in row r at column
# c, so you zero every other row of that column.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def step(ways):
    # next column from the current column only
    n = len(ways)
```

```
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]


def keep_row(ways, row):
    # a checkpoint forces the path through one row
    return [x if r == row else 0
            for r, x in enumerate(ways)]


# FOLLOW-UP 1: Two checkpoints share a column.
# Answer: No path can visit both, because a path is in
# exactly one row per column, so the answer is 0 (the code
# returns it before the DP). Checkpoints at the start or
# end cell are fine: they only filter rows of the first or
# last column.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows

# ---- main question: main function ----
# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
```

```

# Idea: ways[r] is the number of paths that are in row r
# at the current column. The next column is ways'[r] =
# ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint
# column keep only the checkpoint's row.
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
```

```
    # one rolling column is enough: O(n) space
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]

# ---- main question: tests ----
def test_main_question():
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    # force the middle column through row 1
    assert count_paths(3, 3, [(1, 1)]) == 1
    # two checkpoints in one column: impossible
    assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0
    # ordered checkpoints must go left to right
    assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1
    assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0


# ============================================================
# FOLLOW-UP 1: Two checkpoints share a column.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def count_paths_table(n, m):
    # full table: ways[c][r] paths reach row r, column c
    ways = [[0] * n for _ in range(m)]
    ways[0][n - 1] = 1
    for c in range(1, m):
        for r in range(n):
            # arrive from up-right, right or down-right
            for pr in (r - 1, r, r + 1):
                if 0 <= pr < n:
```

```
                    ways[c][r] += ways[c - 1][pr]
    return ways[m - 1][n - 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

STUDY PATH: question 200 of 288 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 2 of 5

Template fit: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest | asked 42 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 200 of 288

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 169/288

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colours (letters such as R, G, B), return the length of the longest run of one colour along a straight line: horizontal, vertical, or diagonal (in either diagonal direction).

Function signature

def longest_same_color_line(g: list) -> int

EXAMPLES

Example 1	
Input	`['RRRG', 'RRBB', 'GRRR', 'BGBR']`
Output	`4`
Why	`The main diagonal is R, R, R, R from corner to corner: 4. No other line is longer.`

Example 2	
Input	`['RGB', 'BGR', 'RGB']`
Output	`3`
Why	`The longest line is the vertical run of three G in the middle column.`

Example 3	
Input	`['RR', 'RR']`
Output	`2`
Why	`A single colour fills the grid: the longest line is a full row or column.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Longest Same-Color Grid Sequence
#
# Given a grid of colours (letters such as R, G, B),
# return the length of the longest run of one colour along
# a straight line: horizontal, vertical, or diagonal (in
# either diagonal direction).
#
# Signature:
#   def longest_same_color_line(g: list) -> int
#
# Examples:
# ['RRRG', 'RRBB', 'GRRR', 'BGBR']  ->  4
# ['RGB', 'BGR', 'RGB']  ->  3
#
# FOLLOW-UPS (answered below, in this order):
# 1. The line may also bend (a connected region).
#
# INTERVIEW NOTES
```

```
# A straight line can run in four directions (right, down,
# down-right, down-left). For each cell and each direction
# keep the length of the same-colour run that ends there:
# it is the predecessor's run plus one if the colours
# match, otherwise 1. Because every run only looks one
# cell back in a fixed direction, one pass in reading
# order is enough.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
```

```
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best

# ---- main question: tests ----
def test_main_question():
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3


# ============================================================
# FOLLOW-UP 1: The line may also bend (a connected
```

```
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Then it is a different problem: the largest
# connected same-colour region, found with BFS/DFS or
# union-find in O(m * n). The run-length DP only handles
# straight lines because each run depends on exactly one
# predecessor.
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_region(["RR", "GR"]) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY PATH: question 201 of 288 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 3 of 5

Template fit: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes | no date | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 201 of 288

Count Same-Color Squares in Matrix

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 78/288

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #221 Maximal Square

LeetCode match: LC 221 Maximal Square (related)

Frequency: Low | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.
Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).
DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.
Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

Re-checks the same cells again and again, O(m * n * min(m, n)^3). A dp on the bottom-right corner reuses the neighbours' results in O(m * n).

OPTIMAL SOLUTION · dp on the bottom-right corner

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the largest side instead of the count.

Keep the same dp and take the maximum instead of adding. The common bug is checking only the diagonal neighbour: the up and left neighbours must also match, otherwise a different colour can sit inside the square.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Same-Color Squares in Matrix
#
# LeetCode 221 variant — count maximal same-color square
# sub-grids. DP on each cell as the bottom-right corner.
# Input: m × n grid of colored cells. Output: count of
# square sub-grids whose cells are all the same color (or
# maximum side length, depending on prompt). DP: dp[i][j]
# = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if
# grid[i][j] == grid[i-1][j] == grid[i][j-1] ==
# grid[i-1][j-1] else 1. Sum dp[i][j] across the grid for
# the count variant; max for the side variant. Examples:
# The plain "count all-1 squares" DP generalizes to same-
# color directly.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the largest side instead of the count.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That square
# exists only if the three neighbours above, left and
# above-left have the same colour as the cell and can
# themselves carry a square, so dp is 1 plus the smallest
# of their dp values. Summing dp over all cells counts
# every square once (each cell contributes one square per
# side length up to dp).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def square_sides(g):
    # side[i][j]: the biggest same-colour square whose
    # bottom-right corner is (i, j); one row at a time
    if not g:
        return []
    prev, out = [0] * len(g[0]), []
    for i, row in enumerate(g):
        cur = [1] * len(row)
        for j, c in enumerate(row):
            # all three neighbours must match this cell
            if i and j and g[i - 1][j] == c \
                    and g[i][j - 1] == c \
                    and g[i - 1][j - 1] == c:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        out.append(cur)
        prev = cur
    return out

# ---- main question: main function ----
# BASE QUESTION (optimal): dp on the bottom-right corner
# Idea: dp[j] holds the best square side ending in the
# current row at column j.
def count_same_color_squares(g):
    # every cell contributes one square per side up to its
    # side
    return sum(sum(row) for row in square_sides(g))

# ---- main question: tests ----
def test_main_question():
    # a 3 x 3 block of one colour: 9 + 4 + 1 squares
    block = ["RRR", "RRR", "RRR"]
    assert count_same_color_squares(block) == 14


# ========================================================
# FOLLOW-UP 1: Return only the largest side instead of the
# ========================================================

# ---- follow-up 1: function ----
# Answer: Keep the same dp and take the maximum instead of
# adding. The common bug is checking only the diagonal
# neighbour: the up and left neighbours must also match,
# otherwise a different colour can sit inside the square.
def largest_side(g):
    # the variant that asks for the biggest square only
    return max((max(row) for row in square_sides(g)),
               default=0)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # a 3 x 3 block of one colour: 9 + 4 + 1 squares
    block = ["RRR", "RRR", "RRR"]
    assert largest_side(block) == 3
    # a different colour inside blocks the bigger squares
    holes = ["RRR", "RGR", "RRR"]
    assert largest_side(holes) == 1
    assert count_same_color_squares(holes) == 9
    assert count_same_color_squares([]) == 0
    assert largest_side([]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the all-1-squares DP from memory in under 8 min. Generalize to multi-color in <2 min — only the equality check changes.

STUDY PATH: question 202 of 288 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 4 of 5

Template fit: the same four run lengths; extend a run when the neighbour in that direction has the same colour | asked 58 days ago

HARD | PracHub | Technical Screen | 202 of 288

Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 166/288

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: the interview site's own solution

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **R** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

QUESTION

The down-left diagonal from the top-right cell contains three **G** cells. No horizontal or vertical sequence has that length.

EXAMPLES

Example 1	
Input	`['RRRG', 'RRBB', 'GRRR', 'BGBR']`
Output	`4`
Why	`Four R cells lie on the main diagonal.`

Example 2	
Input	`['RGB', 'BGR', 'RGB']`
Output	`3`
Why	`Three G cells in the middle column.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION · DP per direction with a rolling row

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the Longest Straight Run of One
# Color
#
# Given a rectangular grid of colors, return the length of
# the longest consecutive sequence of cells with the same
# color along a horizontal, vertical, or diagonal straight
# line. A sequence cannot change direction or skip a cell.
# Both diagonal orientations count. The length is the
# number of cells in the sequence. ### Input - grid: an
# array of strings of equal length. Each character
# represents one cell's color. ### Output Return the
# maximum length as an integer. ### Constraints and Edge
# Cases - For this practice version, the grid contains at
# least one row and one column, with at most 1000000 cells
# in total. - Colors are uppercase English letters; equal
# letters mean equal colors. - A one-cell grid has answer
# 1. - Diagonal lines may run down-right or down-left.
# Reading the same line in reverse does not change its
# length. - A connected region that bends is not
# necessarily one valid sequence.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the run is, not only its length.
# 2. Require the run to be at least K and count how many
# such runs exist.
# 3. The grid is huge and arrives row by row.
#
# INTERVIEW NOTES
# A run in a fixed direction depends only on the cell
# before it in that direction, so each of the four
# directions is a one-line recurrence (like longest common
# suffix). Only four directions are needed because a line
# read backwards is the same line. Rolling only the
# previous row keeps memory at O(n).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)

# ---- main question: main function ----
# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))

# ---- main question: tests ----
def test_main_question():
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run(g1) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0


# ============================================================
# FOLLOW-UP 1: Return where the run is, not only its
# ============================================================

# ---- follow-up 1: function ----
# Answer: Only start a walk at cells that begin a run (the
# previous cell in that direction has a different color);
# that keeps the total work linear, and you record the
# start cell and direction of the best run.
def longest_run_where(grid):
    # the length, the start cell and the direction
    if not grid or not grid[0]:
        return 0, None, None
    return max(runs(grid), key=lambda x: x[0])

# ---- follow-up 1: tests ----
def test_follow_up_1():
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    length, start, direction = longest_run_where(g1)
    assert length == 4


# ============================================================
# FOLLOW-UP 2: Require the run to be at least K and count
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Count a run once, at its start, when its length
# is at least K (use the begins-a-run test above).
# Counting at every cell would count a long run many
# times.
def count_runs_at_least(grid, k):
    # each run is counted once, at its start
    if not grid or not grid[0]:
        return 0
    return sum(1 for n, _, _ in runs(grid) if n >= k)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    # runs of length at least 3 are counted once each
    assert count_runs_at_least(g1, 3) == 4
```

```
# ========================================================
# FOLLOW-UP 3: The grid is huge and arrives row by row.
# ========================================================

# ---- follow-up 3: function ----
# Answer: The rolling-row version already needs only the
# previous row, so it works on a stream of rows with O(n)
# memory.
def longest_run_rows(row_stream):
    # rows arrive one by one: keep only the previous row's
    # runs
    best, prev = 0, None
    h = v = d = a = None
    for row in row_stream:
        n = len(row)
        nh, nv = [1] * n, [1] * n
        nd, na = [1] * n, [1] * n
        for c in range(n):
            if c and row[c] == row[c - 1]:
                nh[c] = nh[c - 1] + 1
            if prev is not None:
                if row[c] == prev[c]:
                    nv[c] = v[c] + 1
                if c and row[c] == prev[c - 1]:
                    nd[c] = d[c - 1] + 1
                if c + 1 < n and row[c] == prev[c + 1]:
                    na[c] = a[c + 1] + 1
            best = max(best, nh[c], nv[c], nd[c], na[c])
        prev, h, v, d, a = row, nh, nv, nd, na
    return best

# ---- follow-up 3: tests ----
def test_follow_up_3():
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run_rows(iter(g1)) == 4


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
`(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)`	4
`(['RBG', 'BGR', 'GRB'],)`	3

ALSO REPORTED · 1 more

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.

Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY PATH: question 203 of 288 | topic 26 of 43: Grid DP: each cell from its neighbours above and to the left | 5 of 5

Template fit: exactly the same recurrence on characters | asked 332 days ago

HARD | PracHub | Technical Screen | 203 of 288

Count same-color squares in a character grid

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 76/288

ABOUT THIS QUESTION

Topic: Grid DP: each cell from its neighbours above and to the left

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1277 Count Square Submatrices with All Ones, LeetCode #221 Maximal Square

LeetCode match: LC 1277 Count Square Submatrices with All Ones / LC 221 Maximal Square

Posted on PracHub: 2025-11-22 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/count-same-color-squares-in-a-character-grid

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has `m` rows and **n** columns.
- `grid[i][j]` is an alphabetic character (e.g., `'a'`–`'z'` or `'A'`–`'Z'`). A **square** in this grid is defined as a contiguous `k × k` submatrix aligned with the grid axes, for some integer `k >= 1`. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by `'a'`. Your function should return the total count of such monochromatic squares for a given input grid.

QUESTION

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.

Constraints

0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

The same cells are rechecked for every size: O(m * n * min(m, n)^3). A table of the largest same-colour square ending at each cell, summed over all cells, is O(m * n).

OPTIMAL SOLUTION · DP: side of the largest square ending at each cell, summed

TIME	SPACE
O(m * n)	O(n) with a rolling row

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the largest square (LeetCode 221).

Keep the maximum of the same dp table instead of the sum.

FOLLOW-UP 2 Allow at most one different cell inside a square.

The simple min-of-three recurrence no longer works. Use 2D prefix counts per colour and test each (corner, size) with binary search on the size, or DP with a second layer 'one defect used'.

FOLLOW-UP 3 Count only squares of colour c, or only side >= 2.

Filter while summing: add dp only for cells of colour c, or add max(0, dp - 1) for sizes at least 2.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count same-color squares in a character
# grid
#
# You are given a 2D grid (matrix) of characters. Each
# character represents a color: cells with the same
# character are considered the same color. Formally: - The
# grid has m rows and n columns. - grid[i][j] is an
# alphabetic character (e.g., 'a'–'z' or 'A'–'Z'). A
# **square** in this grid is defined as a contiguous k × k
# submatrix aligned with the grid axes, for some integer k
# >= 1. A square is **monochromatic** if **all** of its
# cells contain the **same** character (i.e., the same
# color). Task: - Count and return the total number of
# monochromatic squares in the grid. - Count all sizes of
# squares (1×1, 2×2, ..., up to the largest possible that
# fits in the grid). Example (just for clarity, not
# necessarily exhaustive): If the grid is: a a b a a b b b
# b Then some monochromatic squares include: - All 1×1
# cells (each single cell is a monochromatic square).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the largest square (LeetCode 221).
# 2. Allow at most one different cell inside a square.
# 3. Count only squares of colour c, or only side >= 2.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That is also
# the number of squares ending at that corner (one per
# size from 1 up to it). A square of side k at (i, j)
# needs squares of side k - 1 at the three cells above,
# left and above-left, all of the same colour, hence the
# min of those three plus one.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================


# ---- main question: helper functions ----
def square_table(grid):
    # side[i][j]: the biggest same-colour square with its
    # bottom-right corner at (i, j)
    prev, rows = [0] * len(grid[0]), []
    for i, row in enumerate(grid):
        cur = [1] * len(row)
        for j, ch in enumerate(row):
            # the up, left and up-left cells must match
            # too
            if i and j and ch == grid[i - 1][j] \
                    and ch == grid[i][j - 1] \
                    and ch == grid[i - 1][j - 1]:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        rows.append(cur)
        prev = cur
    return rows

# ---- main question: main function ----
# BASE QUESTION (optimal): DP: side of the largest square
# ending at each cell, summed
# Idea: If the cell matches its top, left and top-left
# neighbours then dp = 1 + min(dp top, dp left, dp top-
# left); otherwise 1. Summing dp over all cells counts
# every square exactly once, at its bottom-right corner.
def count_same_color_squares(grid):
    # each cell adds one square for every size up to its
    # side
    return sum(sum(row) for row in square_table(grid))

# ---- main question: tests ----
def test_main_question():
    g = ["aab", "aab", "bbb"]
    # singles 9, plus the 2 x 2 block of a's
    assert count_same_color_squares(g) == 10


# ============================================================
# FOLLOW-UP 1: Only the largest square (LeetCode 221).
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep the maximum of the same dp table instead of
# the sum.
def largest_square(grid):
    # LeetCode 221 style: only the biggest side
    return max(max(row) for row in square_table(grid))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    g = ["aab", "aab", "bbb"]
    assert largest_square(g) == 2


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.
```

```
# ========================================================
# FOLLOW-UP 3: Count only squares of colour c, or only
# ========================================================

# ---- follow-up 3: function ----
# Answer: Filter while summing: add dp only for cells of
# colour c, or add max(0, dp - 1) for sizes at least 2.
def count_of_color(grid, color, min_side=1):
    # squares of one colour, and/or of side at least
    # min_side
    total = 0
    for i, row in enumerate(square_table(grid)):
        for j, side in enumerate(row):
            if grid[i][j] == color:
                # sides min_side .. side are counted
                total += max(0, side - min_side + 1)
    return total

# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = ["aab", "aab", "bbb"]
    assert count_of_color(g, "a") == 5
    assert count_of_color(g, "b") == 5
    full = ["zzz", "zzz", "zzz"]
    # 9 + 4 + 1 squares
    assert count_same_color_squares(full) == 14
    assert count_of_color(full, "z", 2) == 5
    assert count_of_color(full, "q") == 0
    assert count_same_color_squares(["a"]) == 1
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> int:
SAMPLE INPUT
([])
```

Test cases

Input	Expected
`([])`	`0`
`(["aab", "aab", "bbb"])`	`10`

ALSO REPORTED · 1 more

Count Same-Color Squares in Matrix | 1Point3Acres | onsite-coding | Low

https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

ALSO REPORTED · 1 more

Variant reported in both intern VO and an additional NG round. Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

Preparation: Write the all-1-squares DP from memory in under 8 min.
Generalize to multi-color in <2 min — only the equality check changes.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 27 OF 43

Nested structures: a stack of contexts, or recursion with a cache

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Nested structures: a stack of contexts, or recursion with a cache | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	204. Decode String	1Point3Acres	2026-07-29
M	205. Template Variable Expansion with Follow-Ups	1Point3Acres	2026-10-17
M	206. Expand Recursive Key-Value Placeholders in a Template	PracHub	2026-08-04
M	207. Nested List Weight Sum (LC 339) with Follow-Ups	1Point3Acres	2026-08-16
M	208. Nested Function Expression Evaluator	1Point3Acres	2026-06-10

CODE TEMPLATE | GROUP 27 OF 43 | 5 QUESTIONS, ONE PATTERN

Nested structures: a stack of contexts, or recursion with a cache

The one idea these questions share

When something opens (a bracket, a parenthesis, a placeholder), save the current context on a stack and start a fresh one. When it closes, pop the saved context and fold the result back in. Recursive expansion does the same with a function call and a cache.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def decode(s):
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":                                  # CHANGE 1: what opens a context
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":                                  # CHANGE 2: what closes it and how it folds back
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)
```

2. Helper functions that usually get written next to it

```
def expand(text, mapping, seen=()):          # recursive placeholders {name} with cycle detection
    out, i = [], 0
    while i < len(text):
        if text[i] == "{":
            j = text.index("}", i)
            name = text[i + 1:j]
            if name in seen:
                raise ValueError("cycle: " + name)
            out.append(expand(mapping[name], mapping, seen + (name,)))
            i = j + 1
        else:
            out.append(text[i])
            i += 1
    return "".join(out)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
204	Decode String	the template as written
205	Template Variable Expansion with Follow-Ups	resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle
206	Expand Recursive Key-Value Placeholders in a Template	regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles
207	Nested List Weight Sum (LC 339) with Follow-Ups	depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)
208	Nested Function Expression Evaluator	tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 204 of 288 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 1 of 5

Template fit: the template as written | asked 83 days ago | frequency Low

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 204 of 288

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 52/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

An encoded string uses k[text] to mean that text is repeated k times; the brackets can be nested. Return the decoded string. (k is a positive integer, possibly with several digits; the input is always valid.)

Function signature

def decode(s: str) -> str

EXAMPLES

Example 1	
Input	`'3[a]2[bc]'`
Output	`'aaabcbc'`
Why	`3 copies of a, then 2 copies of bc.`

Example 2	
Input	`'2[abc]3[cd]ef'`
Output	`'abcabccdcdcdef'`
Why	`Plain letters after the brackets are kept as they are.`

Example 3	
Input	`'3[a2[c]]'`
Output	`'accaccacc'`
Why	`Nested: the inner 2[c] first (cc), then 3 copies of acc.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Decode String
#
# An encoded string uses k[text] to mean that text is
# repeated k times; the brackets can be nested. Return the
# decoded string. (k is a positive integer, possibly with
# several digits; the input is always valid.)
#
# Signature:
#   def decode(s: str) -> str
#
# Examples:
# '3[a]2[bc]'  ->  'aaabcbc'
# '2[abc]3[cd]ef'  ->  'abcabccdcdcdef'
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you do it recursively, and what is the
# risk?
# 2. What if the decoded string is enormous?
# 3. The decoded string would be enormous.
# 4. Return the character at position k only.
# 5. Invalid input (missing bracket, count with no
# bracket).
#
# INTERVIEW NOTES
# Nested repeats like 3[a2[c]] are a bracket structure,
# which means a stack: when you see '[' save what you have
# built and the repeat count; when you see ']' repeat the
# inner string and glue it to the saved prefix. Read
# counts as full numbers (more than one digit).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----

# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames

# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
    return cur

# ---- main question: tests ----
def test_main_question():
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.
```

```
# ============================================================
# FOLLOW-UP 3: The decoded string would be enormous.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Compute its length (or a given character) from
# the stack without building it: keep (length so far,
# repeat count) frames and combine as prev + inner *
# times. The length can be computed for strings that could
# never fit in memory.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total

# ---- follow-up 3: tests ----
def test_follow_up_3():
    items = parse("3[a2[c]]")
    assert length(items) == 9
```

```
# ============================================================
# FOLLOW-UP 4: Return the character at position k only.
# ============================================================

# ---- follow-up 4: function ----
# Answer: Walk the structure with lengths: at each repeat
# reduce k modulo the inner length to find where in the
# inner string it falls (LeetCode 880 style).
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None

# ---- follow-up 4: tests ----
def test_follow_up_4():
    items = parse("3[a2[c]]")
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

STUDY PATH: question 205 of 288 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 2 of 5

Template fit: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle | asked 3 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 205 of 288

Template Variable Expansion with Follow-Ups

ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 274/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.

Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.

Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.

Follow-up: detect cyclic dependencies among map values. Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	O(depth)

WHY IT IS SLOW

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	O(keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Template Variable Expansion with Follow-
# Ups
#
# Replace %KEY% placeholders using a key-value map whose
# values may contain other placeholders. Clarify malformed
# delimiters and literal-percent escaping, flatten nested
# values, and detect cyclic dependencies before expansion.
```

```
# Input: a key-value map and a string containing
# placeholders delimited by %, such as %USER%. Replace
# every placeholder with the corresponding value. Map
# values can themselves contain placeholders, so resolve
# or flatten dependencies before expanding the final
# string. Define behavior for malformed delimiters such as
# unmatched percent signs, for a literal % in output, and
# for nested-looking forms before coding; the prompt
# intentionally leaves these contracts open. Follow-up:
# detect cyclic dependencies among map values. Model keys
# as a dependency graph and use indegrees plus a queue to
# determine whether a complete topological resolution is
# possible.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Detect cycles before expanding.
#
# INTERVIEW NOTES
# Agree the grammar first: %NAME% is a placeholder, %% is
# a literal percent sign, an unmatched % is an error and
# an unknown name is an error. Values may contain
# placeholders themselves, so expand a value by expanding
# its own placeholders first. A cycle (A uses B, B uses A)
# would recurse forever, so detect it with a visiting set
# or, equivalently, with indegrees and a queue.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1

# ---- main question: main function ----
# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME.
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)

# ---- main question: tests ----
def test_main_question():
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"


# ============================================================
# FOLLOW-UP 1: Detect cycles before expanding.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Model each key as a vertex with an edge to every
# key its value mentions, count indegrees and run Kahn's
# algorithm: if fewer vertices than keys come out, some
# keys sit on a cycle and cannot be resolved.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
        indeg[k] = len(known)
        for x in known:
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    assert not has_cycle(m)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
```

```
        except error:
            pass
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY PATH: question 206 of 288 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 3 of 5

Template fit: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles | asked 77 days ago

MEDIUM | PracHub | Technical Screen | 206 of 288

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 53/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Expand %KEY% placeholders in a template. A placeholder is replaced by its value from the mapping, and placeholders inside the values are expanded too, recursively. An unknown key is left exactly as written. A cycle (A needs B needs A) raises an error.

Function signature

def expand_template(template: str, mapping: dict) -> str

EXAMPLES

Example 1	
Input	`'%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING': 'Hello %NAME%'}`
Output	`'Hello Ada, welcome!'`
Why	`%GREETING% expands to 'Hello %NAME%', and %NAME% is expanded in turn.`

Example 2	
Input	`'Hi %WHO%', {}`
Output	`'Hi %WHO%'`
Why	`An unknown key is kept as written.`

Example 3	
Input	`'%A%', {'A': '%B%', 'B': '%A%'}`
Output	`raises ValueError: cycle at A`
Why	`A needs B and B needs A: a cycle is reported instead of looping forever.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Expand Recursive Key-Value Placeholders
# in a Template
#
# Expand %KEY% placeholders in a template. A placeholder
# is replaced by its value from the mapping, and
# placeholders inside the values are expanded too,
# recursively. An unknown key is left exactly as written.
# A cycle (A needs B needs A) raises an error.
#
# Signature:
#   def expand_template(template: str, mapping: dict) -> s
tr
#
# Examples:
# '%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING':
#     'Hello %NAME%'}  ->  'Hello Ada, welcome!'
# 'Hi %WHO%', {}  ->  'Hi %WHO%'
#
# FOLLOW-UPS (answered below, in this order):
# 1. A literal percent sign must be possible.
# 2. An unknown key should be an error, or replaced by an
# empty string.
# 3. Values are huge and expansion explodes.
#
# INTERVIEW NOTES
# Expanding one key can require expanding others, so
# define expand(key) recursively and remember results
# (memoization) so a key used many times is expanded once.
# A 'currently expanding' set detects cycles: meeting a
# key that is already on the stack means a cycle. Decide
# the policy for unknown keys and for literal percent
# signs up front.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import re


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
PATTERN = re.compile(r"%%|%(\w+)%")

# ---- main question: main function ----
# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
# FOLLOW-UP 1: A literal percent sign must be possible.
# Answer: Choose an escape such as %% and handle it in the
# same scan: when the text has %%, output one % and skip
# both characters; otherwise try to match a placeholder.
# Test percent signs next to placeholders.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)

# ---- main question: tests ----
def test_main_question():
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    want = "I am admin My home is /admin/home"
    assert expand(text, m) == want
    # unknown keys: kept, emptied, or an error
    assert expand("a %X% b", m) == "a %X% b"
    assert expand("a %X% b", m, unknown="empty") == "a  b"
    try:
        expand("%X%", m, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass
    # a literal percent, even next to a placeholder
    assert expand("100%% of %USER%", m) == "100% of admin"
    # a cycle is reported
    try:
        expand("%A%", {"A": "%B%", "B": "%A%"})
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
    # a key that doubles the previous one explodes
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom[f"K{i}"] = f"%K{i - 1}%%K{i - 1}%"
    try:
        expand("%K29%", boom, limit=1000)
```

```
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass
```

```
# ============================================================
# FOLLOW-UP 1: A literal percent sign must be possible.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY PATH: question 207 of 288 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 4 of 5

Template fit: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go) | asked 65 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 207 of 288

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 177/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

A nested list holds integers and further lists. The weight of an integer is its depth: 1 at the top level, 2 inside one inner list, 3 inside two, and so on. Return the sum of every integer times its depth. A second version reads the nested list from a string like '[1,[4,[6]]]'.

Function signature

def weight_sum(nested: list) -> int

def weight_sum_string(s: str) -> int

EXAMPLES

Example 1	
Input	`[3, 8, [2, 14], [2, [91]]]`
Output	`320`
Why	`3 + 8 at depth 1, 2 + 14 and 2 at depth 2, and 91 at depth 3: 11 + 32 + 4 + 273 = 320.`

Example 2	
Input	`'[1,[4,[6]]]'`
Output	`27`
Why	`The string version of [1, [4, [6]]]: 1*1 + 4*2 + 6*3 = 27.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# A nested list holds integers and further lists. The
# weight of an integer is its depth: 1 at the top level, 2
# inside one inner list, 3 inside two, and so on. Return
# the sum of every integer times its depth. A second
# version reads the nested list from a string like
# '[1,[4,[6]]]'.
#
# Signature:
#   def weight_sum(nested: list) -> int
#   def weight_sum_string(s: str) -> int
#
# Examples:
# [3, 8, [2, 14], [2, [91]]]  ->  320
# '[1,[4,[6]]]'  ->  27
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1

# ---- main question: main function ----
# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total

# ---- main question: tests ----
def test_main_question():
    assert weight_sum([3, 8, [2, 14], [2, [91]]]) == 320
    assert weight_sum_string('[1,[4,[6]]]') == 27


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: What is the inverse version (deeper
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is LeetCode 364. Do a level-by-level pass
# that keeps a running sum of everything seen so far and
# add it to the total once per level: total += running_sum
# on each level.
```

```
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# less, LeetCode 364).
# Answer: Process level by level and keep a running sum of
# every integer seen so far; add the running sum to the
# total once per level that holds an integer or lies above
# one (trim trailing levels that contain only empty
# lists).
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)

# =====================================================
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# =====================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

STUDY PATH: question 208 of 288 | topic 27 of 43: Nested structures: a stack of contexts, or recursion with a cache | 5 of 5

Template fit: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function | asked 132 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 208 of 288

Nested Function Expression Evaluator

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 199/288

ABOUT THIS QUESTION

Topic: Nested structures: a stack of contexts, or recursion with a cache

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 770 Basic Calculator IV (related)

Frequency: Low | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

Evaluate an expression made of integers and nested function calls: add and mul (any number of arguments), sub and pow (two), div (integer division truncating toward zero), min and max (any number). Return the integer result.

Function signature

def evaluate(expr: str) -> int

EXAMPLES

Example 1	
Input	`'add(1, sub(1, 0))'`
Output	`2`
Why	`sub(1, 0) is 1, so add(1, 1) = 2.`

Example 2	
Input	`'add(2, mul(3, pow(4, 5)))'`
Output	`3074`
Why	`pow(4, 5) = 1024, times 3 = 3072, plus 2 = 3074.`

Example 3	
Input	`'max(3, div(-7, 2), min(5, 9))'`
Output	`5`
Why	`div(-7, 2) truncates toward zero to -3; max of 3, -3 and 5 is 5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def evaluate(text):
    while has_call(text):
        call = find_innermost_call(text)
        value = apply(call)  # e.g. add(2, 3)
        text = text.replace(call, str(value))
    return int(text)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION · Tokenizer + recursive descent over name(arg, arg, ...)

TIME	SPACE
O(n)	O(depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

FOLLOW-UP 2 How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

FOLLOW-UP 3 What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

FOLLOW-UP 4 Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import math
import re


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")


# FOLLOW-UP 3: What about division?
# Answer: State the contract: integer or float, truncate
# toward zero or floor (Python's // floors, which differs
# for negatives), and divide by zero handling. The
# implementation above truncates toward zero.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}

# ---- main question: main function ----
# BASE QUESTION (optimal): Tokenizer + recursive descent
```

```
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses.
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        # FOLLOW-UP 4: variables come from env
        # a name not followed by "(" is a variable
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value

# ---- main question: tests ----
def test_main_question():
    assert evaluate("add(1, sub(1, 0))") == 2
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    # variadic functions
    assert evaluate("add(1, 2, 3, 4)") == 10
    assert evaluate("max(3, min(9, 4), 1)") == 4
    # variables come from an environment
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15
    # division truncates toward zero
    assert evaluate("div(-7, 2)") == -3
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass
    # the iterative version agrees and has no depth limit
    big = "add(2, mul(3, pow(4, 5)))"


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# ============================================================
# FOLLOW-UP 2: How do you make it iterative?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use two stacks: one of pending function names
# and one of lists of evaluated arguments.
def evaluate_iterative(expr):
    # two stacks: function names and their argument lists
    names, args, tokens = [], [[]], TOKEN.findall(expr)
    for tok in tokens:
        if re.fullmatch(r"-?\d+", tok):
            args[-1].append(int(tok))
        elif tok == "(":
            # the name just pushed now owns a new list
            args.append([])
        elif tok == ")":
            result = OPS[names.pop()](*args.pop())
            args[-1].append(result)
        elif tok != ",":
            names.append(tok)
    return args[0][0]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    # the iterative version agrees and has no depth limit
    big = "add(2, mul(3, pow(4, 5)))"
    assert evaluate_iterative(big) == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000
```

```
    assert evaluate_iterative(deep) == 3000


# ============================================================
# FOLLOW-UP 3: What about division?
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
_OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a**b,
    "div": lambda a, b: int(a / b),
    "min": min,
    "max": max,
}


def evaluate_by_replacement(expr):
    pattern = re.compile(r"([A-Za-z_]\w*)\(([^()]*)\)")
    while True:
        m = pattern.search(expr)
        if not m:
            return int(expr.strip())
        args = [int(x) for x in m.group(2).split(",")]
        expr = (
            expr[: m.start()]
            + str(_OPS[m.group(1)](*args))
            + expr[m.end() :]
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 28 OF 43

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	209. Shortest Path Nodes and Escaping a Pursuer	1Point3Acres	2026-09-29
M	210. Radio Broadcast Reachability	ColorStack	2026-01-30
M	211. Minimum Direction Violations in a Directed Graph	1Point3Acres	2026-07-19
M	212. Compute shortest paths with blocked nodes	PracHub	2025-09-06
M	213. Shortest Directed Cycle Through a Node	1Point3Acres	no date

CODE TEMPLATE | GROUP 28 OF 43 | 5 QUESTIONS, ONE PATTERN

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

The one idea these questions share

The same frontier loop as Tier 1, with three small upgrades: a step can cost 0 or 1 (free steps go to the front of the queue), you can return the moment you reach the goal, and the neighbours of a node can be computed on the fly instead of stored.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque


def cheapest_steps(start, neighbors, goal=None):
    INF = float("inf")
    dist = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        if u == goal:                       # CHANGE 1: stop early at the goal
            return dist[u]
        for v, w in neighbors(u):           # CHANGE 2: w is 1, or 0 for a free step
            if dist.get(v, INF) > dist[u] + w:
                dist[v] = dist[u] + w
                if w == 0:
                    queue.appendleft(v)     # free step: goes to the front
                else:
                    queue.append(v)
    return dist
```

2. Helper functions that usually get written next to it

```
def adjacency(n, edges, directed=True):
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append((v, 1))
        if not directed:
            adj[v].append((u, 1))
    return adj


def within(points, radius):                # neighbours worked out on the fly
    limit = radius * radius
    def neighbors(i):
        x, y = points[i]
        return [(j, 1) for j, (a, b) in enumerate(points)
                if j != i and (x - a) ** 2 + (y - b) ** 2 <= limit]
    return neighbors
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
209	Shortest Path Nodes and Escaping a Pursuer	run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db
210	Radio Broadcast Reachability	neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached
211	Minimum Direction Violations in a Directed Graph	an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

Contents Start the questions >

Master contents (all topics)

#	Question	What you change in the template
212	Compute shortest paths with blocked nodes	neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1
213	Shortest Directed Cycle Through a Node	start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 209 of 288 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 1 of 5

Template fit: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db | asked 21 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 209 of 288

Shortest Path Nodes and Escaping a Pursuer

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 272/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

QUESTION

In an unweighted, undirected graph (an adjacency dict) Alice is at node a and wants to reach the target t. Return the length (number of edges) of the shortest path from a to t found with BFS. The follow-ups ask for every node that lies on some shortest path, and whether Alice can reach t before a pursuer Bob.

Function signature

def shortest_length(adj: dict, a, t) -> int

EXAMPLES

Example 1	
Input	`{0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3`
Output	`2`
Why	`0 -> 1 -> 3 and 0 -> 2 -> 3 both take 2 edges.`

Example 2	
Input	`{0: []}, 0, 0`
Output	`0`
Why	`A start that is the target needs 0 edges.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

TIME	SPACE
exponential in the worst ...	O(paths)
exponential in the worst case	

WHY IT IS SLOW

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Path Nodes and Escaping a
# Pursuer
#
# In an unweighted, undirected graph (an adjacency dict)
# Alice is at node a and wants to reach the target t.
# Return the length (number of edges) of the shortest path
# from a to t found with BFS. The follow-ups ask for every
# node that lies on some shortest path, and whether Alice
# can reach t before a pursuer Bob.
#
# Signature:
#   def shortest_length(adj: dict, a, t) -> int
```

```
#
# Examples:
# {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3  ->
#     2
# {0: []}, 0, 0  ->  0
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if Bob can also meet Alice in the middle of an
# edge?
#
# INTERVIEW NOTES
# BFS from Alice gives dA and BFS from the target gives
# dT. A node is on some shortest path exactly when dA[v] +
# dT[v] equals the shortest length. For the pursuer, BFS
# from Bob gives dB. State the rule you assume: Alice is
# caught at a node v if Bob can be there no later than she
# is (dB[v] <= dA[v]). Alice is safe when a shortest path
# exists whose every node has dB[v] > dA[v].

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# HELPERS used by the code below
def bfs(adj, s):
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


FAR = 10 ** 9

# ---- main question: main function ----
# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)


# QUESTION FOLLOW-UP 1: all nodes on a shortest path
def nodes_on_shortest_paths(adj, a, t):
    da, dt = bfs(adj, a), bfs(adj, t)
    if t not in da:
        return set()
    # a node is on some shortest path if the two
    # distances add up to the full length
    return {v for v in da if v in dt
            and da[v] + dt[v] == da[t]}
```

```
# QUESTION FOLLOW-UP 2: can Alice escape Bob?
def can_escape(adj, a, t, b, meet_on_edge=False):
    da, db = bfs(adj, a), bfs(adj, b)
    if t not in da:
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)
```

```
    def safe(u, v):
        # Bob must reach v strictly after Alice does
        if db.get(v, FAR) <= da[v]:
            return False
        # FOLLOW-UP 1: meet_on_edge=True: they meet inside
        # an edge
        # optional: they swap places on the edge u-v
        swap = db.get(v, FAR) == da[u]
        swap = swap and db.get(u, FAR) == da[v]
        return not (meet_on_edge and swap)

    # Bob standing on Alice's start catches her at once
    if db.get(a, FAR) <= 0:
        return False
    # walk the shortest-path layers using safe nodes only
    frontier, seen = [a], {a}
    while frontier:
        nxt = []
        for u in frontier:
            for v in adj[u]:
                if v in on_path and da[v] == da[u] + 1 \
                        and v not in seen and safe(u, v):
                    seen.add(v)
                    nxt.append(v)
        frontier = nxt
    return t in seen

# ---- main question: tests ----
def test_main_question():
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3],
           3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    on_path = nodes_on_shortest_paths(adj, 0, 4)
    assert on_path == {0, 1, 2, 3, 4}
    # Bob waiting at the target catches Alice there
    assert not can_escape(adj, 0, 4, 4)
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    # Bob starts at the target and reaches it first
    assert not can_escape(chain, 0, 3, 3)
    # Bob starts behind Alice: she stays ahead
    assert can_escape(chain, 1, 3, 0)


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

STUDY PATH: question 210 of 288 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 2 of 5

Template fit: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached | asked 263 days ago | frequency Single report

MEDIUM | IN APP LC 547 | ColorStack | technical interview | 210 of 288

Radio Broadcast Reachability

ASKED 2026-01-30 | 263 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40.7 | 30 DAYS 0 | PRIORITY 105/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: your app's interview approach (LeetCode #547)

LeetCode match: LC 547 Number of Provinces / LC 1971 Find if Path Exists (implicit graph)

Frequency: Single report | Last asked: 2026-01-30 | Roles: SWE Intern

QUESTION

You are given a list of radios placed on a 2D plane, each described by its (x, y) coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance radius (inclusive). A message can travel from one radio to another either directly (within radius) or through a chain of intermediate radios that each relay it to a next radio within radius . Given the radios, the broadcast radius , a source index, and a target index, determine whether a message originating at source can reach target through zero or more hops. Return True or False .

Function signature

```
from typing import List, Tuple
def can_reach(radios: List[Tuple[float, float]],
              radius: float,
              source: int,
              target: int) -> bool:
    ...
```

Follow-up (as reported): Model the radios as nodes and connect any two within radius , then run BFS (or DFS / union-find) from source . To avoid floating-point issues, compare squared distances against radius * radius .

EXAMPLES

Example 1

Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 2`
Output	`True`
Why	`0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.`

QUESTION

Example 2

Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 3`
Output	`False`
Why	`Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest; no chain of hops reaches it.`

Example 3

Input	`radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1`
Output	`True`
Why	`The Euclidean distance is exactly 5.0, which is within radius.`

CONSTRAINTS

- 1 <= len(radios) <= 2000
- Coordinates are integers or floats with -10^4 <= x, y <= 10^4 . radius >= 0 .
- 0 <= source, target < len(radios) .
- Two radios a and b are directly connected when sqrt((ax-bx)^2 + (ay-by)^2) <= radius .
- If source == target , the answer is True .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_can_reach(
    radios, radius, source, target
):
    reached = {source}
    changed = True
    while changed:
        changed = False
        for i in list(reached):
            for j, (x, y) in enumerate(radios):
                dx, dy = (
                    radios[i][0] - x,
                    radios[i][1] - y,
                )
                if (
                    j not in reached
                    and dx * dx + dy * dy
                    <= radius * radius
                ):
                    reached.add(j)
                    changed = True
    return target in reached
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

It re-scans all pairs after every round although a queue visits each radio once.

OPTIMAL SOLUTION · BFS over the implicit graph

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Thousands of source/target queries on the same radios.

Build the connected components once with union-find (O(n^2) pair checks), then every query is two finds. Radios in one component can reach each other.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Radio Broadcast Reachability
#
# You are given a list of radios placed on a 2D plane,
# each described by its (x, y) coordinates. Every radio
# can broadcast to any other radio within a fixed
# Euclidean distance radius (inclusive). A message can
# travel from one radio to another either directly (within
# radius ) or through a chain of intermediate radios that
# each relay it to a next radio within radius . Given the
# radios, the broadcast radius , a source index, and a
# target index, determine whether a message originating at
# source can reach target through zero or more hops.
# Return True or False . Function signature: from typing
# import List, Tuple def can_reach(radios:
# List[Tuple[float, float]], radius: float, source: int,
# target: int) -> bool: ... Follow-up (as reported): Model
# the radios as nodes and connect any two within radius ,
# then run BFS (or DFS / union-find) from source .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Thousands of source/target queries on the same
# radios.
#
# INTERVIEW NOTES
# Radios are nodes, and two radios are joined when their
# squared distance is at most radius squared (squared
# distances avoid sqrt and float error). Run a BFS from
# the source over this implicit graph and see whether it
# reaches the target. Checking all pairs at each step is
# O(n^2) in total.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): BFS over the implicit graph
# Idea: Compare squared distances; every radio enters the
# queue once.
def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
    queue = deque([source])
    while queue:
        u = queue.popleft()
        if u == target:
            return True
        for v, (x, y) in enumerate(radios):
            dx, dy = radios[u][0] - x, radios[u][1] - y
            if (
                not seen[v]
                and dx * dx + dy * dy <= limit
            ):
                seen[v] = True
                queue.append(v)
    return False

# ---- main question: tests ----
def test_main_question():
    random.seed(85)
    line = [(0, 0), (1, 0), (2, 0), (5, 0)]
    assert can_reach(line, 1.5, 0, 2) is True
    assert can_reach(line, 1.5, 0, 3) is False
    assert (
        can_reach([(0, 0), (3, 4)], 5.0, 0, 1) is True
    )
    assert can_reach(line, 1, 3, 3) is True


# ============================================================
# FOLLOW-UP 1: Thousands of source/target queries on the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Build the connected components once with union-
# find (O(n^2) pair checks), then every query is two
# finds. Radios in one component can reach each other.
class RadioNetwork:
```

```
    def __init__(self, radios, radius):
        n = len(radios)
        self.parent = list(range(n))
        for i in range(n):
            for j in range(i + 1, n):
                dx = radios[i][0] - radios[j][0]
                dy = radios[i][1] - radios[j][1]
                if dx * dx + dy * dy <= radius * radius:
                    self.parent[self.find(i)] = (
                        self.find(j)
                    )

    def find(self, a):
        while self.parent[a] != a:
            self.parent[a] = self.parent[self.parent[a]]
            a = self.parent[a]
        return a

    def can_reach(self, source, target):
        return self.find(source) == self.find(target)
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 211 of 288 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 3 of 5

Template fit: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft | asked 93 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 211 of 288

Minimum Direction Violations in a Directed Graph

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 10.6 | 30 DAYS 0 | PRIORITY 189/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

A directed graph has n nodes. Walking along an edge in its own direction is free; walking against it costs 1 (a direction violation). Return the minimum total cost of a walk from start to end, or -1 if end cannot be reached at all.

Function signature

def min_violations(n: int, edges: list, start: int, end: int) -> int

EXAMPLES

Example 1

Input	`5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4`
Output	`2`
Why	`Path 0 -> 1 (free), 1 -> 2 against edge 2 -> 1 (cost 1), 2 -> 3 (free), 3 -> 4 against edge 4 -> 3 (cost 1): total 2.`

Example 2

Input	`3, [[0, 1]], 0, 2`
Output	`-1`
Why	`Node 2 has no edge at all, so it cannot be reached.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimum Direction Violations in a
# Directed Graph
#
# A directed graph has n nodes. Walking along an edge in
# its own direction is free; walking against it costs 1 (a
# direction violation). Return the minimum total cost of a
# walk from start to end, or -1 if end cannot be reached
# at all.
#
# Signature:
#   def min_violations(n: int, edges: list, start: int, en
d: int) -> int
#
# Examples:
# 5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4  ->  2
# 3, [[0, 1]], 0, 2  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is 0-1 BFS correct, and how does it compare with
# Dijkstra?
# 2. What if edge costs can be arbitrary non-negative
# numbers?
# 3. Edge costs are arbitrary non-negative numbers
# (different for each direction).
#
# INTERVIEW NOTES
# Going against an edge costs 1 and going with it costs 0,
# so it is a shortest-path problem with costs of only 0
# and 1. Plain BFS counts edges, not violations, so it is
# the wrong tool. A deque lets 0-cost moves jump the queue
# (push front) and 1-cost moves wait their turn (push
# back), which is Dijkstra without the heap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import deque


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
INF = float("inf")

# ---- main question: main function ----
# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back.
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]

# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # path 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 4) == 2
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
```

```
# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Edge costs are arbitrary non-negative
# ============================================================

# ---- follow-up 3: function ----
# Answer: Use Dijkstra with a heap on the same doubled
# graph, giving each arc its own cost. 0-1 BFS only works
# because the costs are 0 or 1.
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = [[] for _ in range(n)]
    for u, v, fwd, back in edges:
        graph[u].append((v, fwd))
        graph[v].append((u, back))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        # skip outdated heap entries
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
```

```
    return -1 if dist[end] == INF else dist[end]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
    assert cheapest_route(5, costed, 0, 4) == 2
```

```
    # different cost per direction
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

STUDY PATH: question 212 of 288 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 4 of 5

Template fit: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1 | asked 409 days ago

MEDIUM | PracHub | Technical Screen | 212 of 288

★ TOP 75: rank 65 in the Medium and Hard priority list

★ Compute shortest paths with blocked nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 67/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #743 Network Delay Time

LeetCode match: LC 743 Network Delay Time (Dijkstra with blocked nodes skipped)

Posted on PracHub: 2025-09-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/compute-shortest-paths-with-blocked-nodes

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3],
    False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, blocked, s):
    keep = []
    for u, v, w in edges:
        if u not in blocked and v not in blocked:
            keep.append((u, v, w))
    d = floyd_warshall(n, keep)  # all pairs
    return d[s]
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) and O(V^2) memory when one source is needed. A BFS from the source that never enters a blocked node is O(V + E).

OPTIMAL SOLUTION · BFS that never enters a blocked node

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

FOLLOW-UP 2 Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

FOLLOW-UP 3 Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute shortest paths with blocked nodes
#
# Given a graph with nodes and edges and a designated
```

```
# source node s, compute the shortest distance from s to
# every other node. Some nodes are inaccessible (blocked)
# and cannot be traversed or reached; treat them as
# removed from the graph. Return a distance for each node
# where unreachable nodes (including blocked ones) are -1.
# Describe your algorithm, data structures, and time/space
# complexity, and note any differences for directed vs.
# undirected graphs. Overview: This question evaluates
# proficiency with graph algorithms, shortest-path and
# reachability concepts when nodes are removed or blocked,
# and the ability to analyze time and space complexity
# across directed and undirected graphs.Given n nodes
# labeled 0..n-1, edges, a source, and blocked nodes,
# return shortest unweighted distances from the source.
# Blocked and unreachable nodes must be -1. Set
# directed=True to treat edges as directed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges have weights.
# 2. Directed versus undirected.
# 3. Nodes get blocked over time and you must keep
# distances up to date.
#
# INTERVIEW NOTES
# Blocked nodes are simply not in the graph, so skip them
# when you queue neighbours and never start from one.
# After that it is plain BFS from the source: first
# discovery gives the shortest distance. State the edge
# cases: a blocked source makes everything unreachable,
# and directed edges must only be added in their given
# direction.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import defaultdict, deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1.
def blocked_distances(n, edges, source, blocked,
                      directed=False):
    # BFS that treats blocked nodes as removed;
    # unreachable
    # and blocked nodes get -1
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in bad and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist

# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1), (1, 2), (0, 3), (3, 2), (2, 4)]
    free = blocked_distances(5, edges, 0, [])
    assert free == [0, 1, 2, 1, 3]
    # blocking 1 forces the route through 3
    assert blocked_distances(5, edges, 0, [1])[2] == 2
    # blocking 2 cuts node 4 off; blocked nodes are -1
    cut = blocked_distances(5, edges, 0, [2])
    assert cut == [0, 1, -1, 1, -1]
    start = blocked_distances(3, edges[:2], 0, [0])
    assert start == [-1, -1, -1]
    # directed edges only go one way
    path = [(0, 1), (1, 2)]
    one_way = blocked_distances(3, path, 2, [], True)
    assert one_way == [-1, -1, 0]
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]


# ============================================================
# FOLLOW-UP 1: The edges have weights.
# ============================================================

# ---- follow-up 1: helper functions ----
# HELPERS used by the code below
INF = float("inf")

# ---- follow-up 1: function ----
# Answer: BFS is no longer correct: use Dijkstra with a
# heap and skip blocked nodes the same way. O((V + E) log
# V). Directed and undirected differ only in whether you
# add the reverse edge.
def blocked_distances_weighted(n, edges, source, blocked,
                               directed=False):
    # edges (u, v, w): BFS is wrong now, use Dijkstra
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        if not directed:
            adj[v].append((u, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in bad and d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return [-1 if d == INF else d for d in dist]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]
    wd = blocked_distances_weighted
    assert wd(3, weighted, 0, []) == [0, 5, 6]
    assert wd(3, weighted, 0, [1])[2] == 10


# ============================================================
# RUN EVERYTHING
```

```
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 213 of 288 | topic 28 of 43: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly | 5 of 5

Template fit: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v | no date | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 213 of 288

Shortest Directed Cycle Through a Node

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 198/288

ABOUT THIS QUESTION

Topic: Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

QUESTION

Given a directed graph with n nodes and a list of edges (a, b), return the length (number of edges) of the shortest cycle that passes through node v, or -1 if v is on no cycle.

Function signature

def shortest_cycle_through(n: int, edges: list, v: int) -> int

EXAMPLES

Example 1	
Input	`3, [(0, 1), (1, 2), (2, 0)], 0`
Output	`3`
Why	`0 -> 1 -> 2 -> 0 is a cycle of 3 edges.`

Example 2	
Input	`3, [(0, 1), (1, 2)], 0`
Output	`-1`
Why	`No edge leads back to 0.`

Example 3	
Input	`3, [(0, 1), (1, 0), (1, 2), (2, 0)], 0`
Output	`2`
Why	`A shorter cycle 0 -> 1 -> 0 exists next to the longer one.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
best = INF
def dfs(u, length, on_path):
    global best
    for w in graph[u]:
        if w == v:
            best = min(best, length + 1)
        elif w not in on_path:
            dfs(w, length + 1, on_path | {w})
dfs(v, 0, {v})
answer = -1 if best == INF else best
```

TIME	SPACE
exponential	O(V)

WHY IT IS SLOW

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

OPTIMAL SOLUTION · BFS from v, close the cycle on the first edge back to v

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

FOLLOW-UP 2 What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

FOLLOW-UP 3 The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Directed Cycle Through a Node
#
# Given a directed graph with n nodes and a list of edges
# (a, b), return the length (number of edges) of the
# shortest cycle that passes through node v, or -1 if v is
# on no cycle.
#
# Signature:
#   def shortest_cycle_through(n: int, edges: list, v: int
) -> int
#
# Examples:
# 3, [(0, 1), (1, 2), (2, 0)], 0  ->  3
# 3, [(0, 1), (1, 2)], 0  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you find the shortest cycle in the whole
# graph?
# 2. What if edges have positive weights?
# 3. The shortest cycle anywhere in the graph (the girth).
#
# INTERVIEW NOTES
# A cycle through v is a path from v back to v, so find
# shortest distances from v and watch for an edge that
# points back at v. Breadth-first search discovers nodes
# in order of distance, so the first edge back to v closes
# the shortest cycle. Do not mark v as 'seen' in a way
# that hides the edge that returns to it.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj

# ---- main question: main function ----
# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is dist[u] + 1 and is the shortest.
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1

# ---- main question: tests ----
def test_main_question():
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    path = [(0, 1), (1, 2)]
    assert shortest_cycle_through(3, path, 0) == -1
    # a self loop is a cycle of length 1
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1
    # a shorter cycle exists elsewhere: 1 <-> 2
    both = ring + [(1, 2), (2, 1)]


# ============================================================
# FOLLOW-UP 1: How do you find the shortest cycle in the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the same BFS from every vertex and take the
# minimum: O(V * (V + E)). For undirected graphs, BFS from
# each node and treat any non-tree edge as closing a cycle
# of length dist[u] + dist[w] + 1 (the LeetCode 2608
# version).
# FOLLOW-UP 3: The shortest cycle anywhere in the graph
# (the girth).
# Answer: Run the same BFS from every node and take the
# smallest result: O(V * (V + E)). That is the standard
# way and fine for graphs up to a few thousand nodes.
def girth(n, edges):
    # the shortest cycle anywhere: BFS from every node
    sizes = [shortest_cycle_through(n, edges, v)
             for v in range(n)]
    found = [s for s in sizes if s > 0]
    return min(found) if found else -1

# ---- follow-up 1: tests ----
def test_follow_up_1():
    ring = [(0, 1), (1, 2), (2, 0)]
    # a shorter cycle exists elsewhere: 1 <-> 2
    both = ring + [(1, 2), (2, 1)]
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1
    # weighted: the cheap way back is 0 -> 1 -> 0 (cost 3)
    weighted = [(0, 1, 1), (1, 0, 2),
                (0, 2, 1), (2, 0, 9)]


# ============================================================
# FOLLOW-UP 2: What if edges have positive weights?
# ============================================================

# ---- follow-up 2: helper functions ----
INF = float("inf")

# ---- follow-up 2: function ----
# Answer: Replace BFS with Dijkstra from v and close the
# cycle at the cheapest dist[u] + weight(u, v). That is
# O((V + E) log V) with a heap. With negative weights use
```

```
# Bellman-Ford and handle negative cycles separately.
def weighted_cycle_through(n, edges, v):
    # positive weights: Dijkstra, then close the cycle at
    # v
```

```
    adj = make_graph(n, edges)
    dist, heap, best = {v: 0}, [(0, v)], INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
        for w, cost in adj[u]:
            if w == v:
                best = min(best, d + cost)
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    return -1 if best == INF else best

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # weighted: the cheap way back is 0 -> 1 -> 0 (cost 3)
    weighted = [(0, 1, 1), (1, 0, 2),
                (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3


# ============================================================
# FOLLOW-UP 3: The shortest cycle anywhere in the graph
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 29 OF 43

Topological sort (Kahn) and cycle detection

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Topological sort (Kahn) and cycle detection | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	214. Course Scheduling and Dependency Variants	1Point3Acres	2026-10-04
M	215. Validate Race Results (Cycle Detection)	ColorStack	2025-10-01
M	216. Solve tree, graph, sliding-window problems	PracHub	2025-08-04
M	217. Validate course catalog dependencies	PracHub	2025-08-01
M	218. Course Schedule	Exponent	no date

CODE TEMPLATE | GROUP 29 OF 43 | 5 QUESTIONS, ONE PATTERN

Topological sort (Kahn) and cycle detection

The one idea these questions share

Count the incoming edges of every node, start from the nodes with none, and peel them off one by one. If every node got peeled there is a valid order; if some never did, there is a cycle. The questions only differ in what the nodes are called and whether you return the order or a yes/no.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import deque, defaultdict


def topo_order(nodes, edges):
    adj, indeg = defaultdict(list), {v: 0 for v in nodes}
    for u, v in edges:                          # CHANGE 1: what an edge means
        adj[u].append(v)
        indeg[v] += 1
    queue = deque(v for v in nodes if indeg[v] == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    return order if len(order) == len(indeg) else None   # CHANGE 2: None means a cycle
```

2. Helper functions that usually get written next to it

```
def nodes_of(edges):                        # when the statement does not list the nodes
    return sorted({x for e in edges for x in e}, key=str)


def has_unknown(nodes, edges):              # prerequisites that name something that does not exist
    known = set(nodes)
    return sorted({x for e in edges for x in e if x not in known}, key=str)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
214	Course Scheduling and Dependency Variants	build the vertex set from the edges too; return the order or None
215	Validate Race Results (Cycle Detection)	edge = winner -> loser; the results are consistent exactly when there is no cycle
216	Solve tree, graph, sliding-window problems	the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops
217	Validate course catalog dependencies	first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle
218	Course Schedule	edge = prerequisite -> course; return the order, or an empty list when there is a cycle

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 214 of 288 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 1 of 5

Template fit: build the vertex set from the edges too; return the order or None | asked 16 days ago | frequency Medium

MEDIUM | IN APP LC 210 | 1Point3Acres | onsite-coding | 214 of 288

Course Scheduling and Dependency Variants

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 209/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: your app's interview approach (LeetCode #210)

LeetCode match: LC 210 Course Schedule II

Frequency: Medium | Last asked: 2026-10-04

Original: https://www.1point3acres.com/interview/problems/company/google/course-schedule-dependency-variants

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

QUESTION

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1:
    contradiction)
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, rules):
    for order in permutations(range(n)):
        pos = {v: i for i, v in enumerate(order)}
        ok = all(pos[a] < pos[b]
                 for a, b in rules)
        if ok:
            return True
    return False
```

TIME	SPACE
O(V! * E)	O(V)

WHY IT IS SLOW

V! orderings. Kahn's algorithm repeatedly takes a vertex with no unmet prerequisites and finishes in O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (course order, or can two sequences merge)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (u, v) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Course Scheduling and Dependency Variants
#
```

```
# Topological-ordering family. Variants: course ordering
# with validity checks and a nested-list extension;
# merging two arrays whose internal order defines
# precedence (return false on a conflict cycle); and a
# damaged-node dependency-path variant that minimizes
# broken nodes on a route. Course-ordering variant Work
# through the course-ordering problem, including validity
# checks, complexity analysis, and test cases. Extend the
# result to a nested-list format. The grouping rule is
# unspecified; clarify what each inner list represents
# before implementing the extension. Damaged-node and path
# variant Remove damaged nodes and their downstream
# effects, then find a valid path. A follow-up minimizes
# broken nodes on a required route. Clarify the graph
# direction, path endpoints, and meaning of broken nodes
# before choosing an algorithm. Merge-two-ordered-
# sequences variant Input: two integer arrays a and b.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you check that an order is valid?
#
# INTERVIEW NOTES
# Every 'A must come before B' rule is an edge. A valid
# ordering exists exactly when the graph has no cycle, and
# Kahn's algorithm finds one: repeatedly take a vertex
# with no unmet prerequisites. The two-array merge is the
# same problem: each adjacent pair inside an array is an
# edge and equal values are the same vertex; the merge is
# possible only if the combined graph has no cycle.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]


def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)

# ---- main question: main function ----
# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None
```

```
# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None

# ---- main question: tests ----
def test_main_question():
    # course order with an isolated course 3
    courses, rules = range(4), [(0, 1), (1, 2)]
    order = topological_order(courses, rules)


# ============================================================
# FOLLOW-UP 1: How do you check that an order is valid?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Record each vertex's position in the order,
# check that every vertex appears exactly once, then check
# that for every edge (u, v) the position of u is smaller
# than the position of v.
def is_valid_order(order, vertices, edges):
    # every vertex must appear exactly once
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    pos = {v: i for i, v in enumerate(order)}
    # every prerequisite must come before its dependent
    return all(pos[u] < pos[v] for u, v in edges)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # course order with an isolated course 3
    courses, rules = range(4), [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
    assert is_valid_order(order, courses, rules)
    # a swapped order breaks a rule
    swapped = [1, 0, 2, 3]
    assert not is_valid_order(swapped, courses, rules)
```

```
    # a cycle has no order
    loop = [(0, 1), (1, 0)]
    assert topological_order(range(2), loop) is None
    # merge examples from the question
    assert can_merge([1, 2, 3], [2, 4, 5])
    merged = merge_two_orders([1, 2, 3], [2, 4, 5])
    assert merged is not None
    assert not can_merge([1, 6, 4], [4, 1])
```

```
    # a repeated value in one array is a self conflict
    assert not can_merge([1, 2, 1], [])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

STUDY PATH: question 215 of 288 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 2 of 5

Template fit: edge = winner -> loser; the results are consistent exactly when there is no cycle | asked 384 days ago | frequency Single report

MEDIUM | IN APP LC 207 | ColorStack | technical interview | 215 of 288

Validate Race Results (Cycle Detection)

ASKED 2025-10-01 | 384 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 38/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule (race results version)

Frequency: Single report | Last asked: 2025-10-01 | Roles: SWE Intern

QUESTION

You are given a list of pairwise race results as (winner, loser) string pairs, meaning winner beat loser . The results are consistent if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge winner -> loser) contains no cycle. Return True if the results are valid (acyclic) and False otherwise.

Function signature

def results_are_valid(results: list[tuple[str, str]]) -> bool:

...

Follow-up (as reported): Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

EXAMPLES

Example 1

Input	`results = [("A","B"), ("B","C")]`
Output	`True`
Why	`A beat B beat C — a consistent ranking A > B > C, no cycle.`

Example 2

Input	`results = [("A","B"), ("B","C"), ("C","A")]`
Output	`False`
Why	`A > B > C > A is a contradiction (a cycle).`

Example 3

Input	`results = []`
Output	`True`
Why	`No results impose no contradiction.`

CONSTRAINTS

- 0 <= len(results) <= 10^5 .

QUESTION

- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (True).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_valid(results):
    graph = {}
    for w, l in results:
        graph.setdefault(w, []).append(l)
        graph.setdefault(l, [])
    state = {}
    def dfs(u):
        state[u] = 1
        for v in graph[u]:
            if state.get(v) == 1 or (
                v not in state and not dfs(v)
            ):
                return False
        state[u] = 2
        return True
    return all(
        u in state or dfs(u) for u in graph
    )
```

TIME	SPACE
O(V + E)	O(V) recursion

WHY IT IS SLOW

Correct, but a chain of 100,000 results exceeds Python's recursion limit.

OPTIMAL SOLUTION · Kahn's algorithm

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Recursive DFS hits Python's recursion limit on long chains. What do you do?

Either use Kahn's algorithm (already iterative) or run the three-colour DFS with an explicit stack of (node, next neighbour index) so the depth is limited by memory, not by the interpreter.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Validate Race Results (Cycle Detection)
#
# You are given a list of pairwise race results as
# (winner, loser) string pairs, meaning winner beat loser
# . The results are consistent if they can all be true
# simultaneously under a strict ranking — equivalently, if
# the directed graph (edge winner -> loser ) contains no
# cycle. Return True if the results are valid (acyclic)
# and False otherwise. Function signature: def
# results_are_valid(results: list[tuple[str, str]]) ->
# bool: ... Follow-up (as reported): Recursive DFS can hit
# Python's recursion limit on long chains; convert to an
# iterative DFS with an explicit stack, or use Kahn's
# topological-sort algorithm.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Recursive DFS hits Python's recursion limit on long
# chains. What do you do?
#
# INTERVIEW NOTES
# Pairwise results are consistent exactly when the
# directed graph winner -> loser has no cycle, so this is
# cycle detection. Kahn's algorithm removes nodes with no
# incoming edges layer by layer; if every node gets
# removed there is no cycle. It is iterative, so long
# chains cannot blow the recursion limit.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Kahn's algorithm
# Idea: Peel off nodes with no incoming edge; everything
```

```
# peels off exactly when there is no cycle.
def results_are_valid(results):
    graph, indeg = {}, {}
    for w, l in set(results):
        graph.setdefault(w, []).append(l)
        graph.setdefault(l, [])
        indeg[l] = indeg.get(l, 0) + 1
        indeg.setdefault(w, 0)
    queue = deque(u for u in graph if indeg[u] == 0)
    removed = 0
    while queue:
        u = queue.popleft()
        removed += 1
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    return removed == len(graph)

# ---- main question: tests ----
def test_main_question():
    random.seed(41)
    assert (
        results_are_valid([("A", "B"), ("B", "C")])
        is True
    )
    assert (
        results_are_valid(
            [("A", "B"), ("B", "C"), ("C", "A")]
        )
        is False
    )
    assert results_are_valid([]) is True
    assert results_are_valid([("A", "A")]) is False
    long_chain = [
        (str(i), str(i + 1)) for i in range(5000)
    ]


# ============================================================
# FOLLOW-UP 1: Recursive DFS hits Python's recursion limit
# ============================================================

# ---- follow-up 1: function ----
# Answer: Either use Kahn's algorithm (already iterative)
```

```
# or run the three-colour DFS with an explicit stack of
# (node, next neighbour index) so the depth is limited by
# memory, not by the interpreter.
def results_are_valid_dfs(results):
    graph = {}
    for w, l in results:
        graph.setdefault(w, []).append(l)
        graph.setdefault(l, [])
    state = {}
    for start in graph:
        if start in state:
            continue
        state[start] = 1
        stack = [(start, iter(graph[start]))]
        while stack:
            u, it = stack[-1]
            for v in it:
                if state.get(v) == 1:
                    return False
                if v not in state:
                    state[v] = 1
                    stack.append((v, iter(graph[v])))
                    break
            else:
                state[u] = 2
                stack.pop()
    return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    long_chain = [
        (str(i), str(i + 1)) for i in range(5000)
    ]
    assert results_are_valid(
        long_chain
    ) and results_are_valid_dfs(long_chain)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 216 of 288 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 3 of 5

Template fit: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops | asked 442 days ago

MEDIUM | PracHub | Technical Screen | 216 of 288

★ TOP 75: rank 46 in the Medium and Hard priority list

★ Solve tree, graph, sliding-window problems

POSTED 2025-08-04 | 442 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 35/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #103 Binary Tree Zigzag Level Order Traversal, LeetCode #207 Course Schedule

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal + LC 207 Course Schedule

Posted on PracHub: 2025-08-04 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/solve-tree-graph-sliding-window-problems

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters. https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/ https://leetcode.com/problems/course-schedule/description/ https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

BRUTE FORCE · Course schedule: DFS from every course looking for a cycle

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Each search can walk the whole graph again, O(V * (V + E)). Kahn's algorithm (remove courses with no unmet prerequisites) is O(V + E).

BRUTE FORCE · Longest window: check every substring

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, K):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) > K:
                break
            best = max(best, j - i + 1)
    return best
```

TIME	SPACE
O(n^2)	O(k)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map that shrinks from the left when it exceeds K distinct characters is O(n).

BRUTE FORCE · Zigzag: level order, then reverse the odd levels

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time with one extra copy of every second level; filling each level from alternating ends of a deque avoids it.

OPTIMAL SOLUTION · Course schedule: Kahn's topological sort

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Longest window: sliding window with a count map

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Zigzag: fill each level from the correct end

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

FOLLOW-UP 2 Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

FOLLOW-UP 3 Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve tree, graph, sliding-window
# problems
```

```
#
# ##### Question LeetCode 103. Binary Tree Zigzag Level
# Order Traversal – given a binary tree root, return its
# zigzag level-order traversal. LeetCode 207. Course
# Schedule – given course prerequisites, determine if all
# courses can be finished by detecting cycles with
# topological sort. LeetCode 340. Longest Substring with
# At Most K Distinct Characters – given string s and
# integer k, return the length of the longest substring
# containing at most k distinct characters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Course Schedule II: return an order in which to take
# the courses.
# 2. Longest substring with exactly K distinct characters.
# 3. Zigzag on an n-ary tree, or the right-to-left
# variant.
#
# INTERVIEW NOTES
# Three standard patterns. Level order needs a queue and a
# direction flag per level. 'Can everything be finished'
# is cycle detection in a directed graph: Kahn's algorithm
# removes nodes with no remaining prerequisites, and a
# cycle leaves some behind. 'Longest window with at most K
# of something' is a sliding window with a count map that
# shrinks from the left when the limit is exceeded.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 1: Course Schedule II: return an order in
```

```
# which to take the courses.
# Answer: The same Kahn loop records the order in which
# courses are removed; if fewer than n courses are
# removed, return an empty list.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def course_order(n, prereqs):
    # (a, b): to take a you must take b first. Kahn's
    # algorithm;
    # if some course never frees up there is a cycle
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def zigzag(root):
    # level order, alternating direction
    out, left_to_right = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            # append for left-to-right, appendleft
            # otherwise
            if left_to_right:
                level.append(node.val)
            else:
```

```
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        left_to_right = not left_to_right
    return out


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def can_finish(n, prereqs):
    return len(course_order(n, prereqs)) == n


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def longest_at_most_k(s, k):
    # sliding window with a count of each letter inside it
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # too many distinct letters: shrink from the left
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best

# ---- main question: tests ----
def test_main_question():
    tree = build([3, 9, 20, None, None, 15, 7])
    assert zigzag(tree) == [[3], [20, 9], [15, 7]]
    assert zigzag(None) == []
    assert can_finish(2, [(1, 0)])
```

```
    assert not can_finish(2, [(1, 0), (0, 1)])
    order = course_order(4, [(1, 0), (2, 1), (3, 2)])
    assert order == [0, 1, 2, 3]
    assert course_order(2, [(0, 1), (1, 0)]) == []
    assert longest_at_most_k("eceba", 2) == 3
    assert longest_at_most_k("aa", 1) == 2
    assert longest_at_most_k("abc", 0) == 0
```

```
# ============================================================
# FOLLOW-UP 1: Course Schedule II: return an order in
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Longest substring with exactly K distinct
# ============================================================

# ---- follow-up 2: helper functions ----
def count_at_most_k(s, k):
    # number of substrings with at most k distinct letters
    if k < 1:
        return 0
    count, left, total = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += right - left + 1
    return total

# ---- follow-up 2: function ----
# Answer: Take atMost(K) minus atMost(K - 1) for counting
# substrings, or keep the window and only record lengths
# when the distinct count is exactly K (careful with the
# shrink).
def count_exactly_k(s, k):
    # exactly k = at most k minus at most k - 1
    most = count_at_most_k(s, k)
    return most - count_at_most_k(s, k - 1)

# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert count_exactly_k("abab", 2) == 6
    assert count_at_most_k("abab", 1) == 4


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(values):
    # level-order list, None for a missing child
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(root: list[str]) -> list[list[int]]:
```

Sample call

```
solution([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

STUDY PATH: question 217 of 288 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 4 of 5

Template fit: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle | asked 445 days ago

MEDIUM | IN APP LC 207 | PracHub | Technical Screen | 217 of 288

★ TOP 75: rank 33 in the Medium and Hard priority list

★ Validate course catalog dependencies

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 36/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #207 Course Schedule

LeetCode match: LC 207 Course Schedule

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/validate-course-catalog-dependencies

QUESTION

You are given a set of course ids and prerequisite pairs (u, v), meaning course v requires course u. The catalog is valid when no prerequisite mentions a course that does not exist and there is no cycle. Return (True, [], order) with a possible study order when valid; (False, missing, None) listing the unknown course ids; or (False, [], cycle) with one cycle that was found.

Function signature

def validate_catalog(courses: set, prereqs: list) -> tuple

EXAMPLES

Example 1	
Input	`{1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]`
Output	`(true, [], [1, 4, 2, 3])`
Why	`Course 1 comes first, 4 is unrelated, then 2 and 3.`

Example 2	
Input	`{1, 2}, [(1, 9)]`
Output	`(false, [9], null)`
Why	`Course 9 is not in the catalog.`

Example 3	
Input	`{1, 2}, [(1, 2), (2, 1)]`
Output	`(false, [], [1, 2, 1])`
Why	`1 requires 2 and 2 requires 1: a cycle.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def validate(courses, prereqs):
    for u, v in prereqs:
        if u not in courses or v not in courses:
            return False  # unknown id
    graph = build_graph(prereqs)
    for c in courses:
        if reaches(graph, c, c):
            return False  # cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Every course can trigger a walk of the whole graph, O(V * (V + E)). Kahn's algorithm is O(V + E) and its leftovers also reveal a cycle to report.

OPTIMAL SOLUTION · Missing references, then Kahn's algorithm, then a cycle from the leftovers

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

FOLLOW-UP 2 Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

FOLLOW-UP 3 Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Validate course catalog dependencies
#
# You are given a set of course ids and prerequisite pairs
# (u, v), meaning course v requires course u. The catalog
# is valid when no prerequisite mentions a course that
# does not exist and there is no cycle. Return (True, [],
# order) with a possible study order when valid; (False,
# missing, None) listing the unknown course ids; or
# (False, [], cycle) with one cycle that was found.
#
# Signature:
#   def validate_catalog(courses: set, prereqs: list) -> t
uple
#
# Examples:
# {1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]  ->  (true, [],
#     [1, 4, 2, 3])
# {1, 2}, [(1, 9)]  ->  (false, [9], null)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return all missing course references and one cycle.
# 2. Group courses into semesters (the earliest round each
# can be taken).
# 3. Find all cycles, not just one.
#
# INTERVIEW NOTES
# Two separate validity rules: every course mentioned must
# exist, and the dependency graph must have no cycle. The
# first is a set lookup per pair. For the second, Kahn's
# algorithm removes courses with no unmet prerequisites;
# anything left over is on a cycle or depends on one, and
# a DFS restricted to the leftovers reveals an actual
# cycle to report.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 3: Find all cycles, not just one.
# Answer: Compute strongly connected components (Tarjan or
# Kosaraju); every component with more than one course (or
# a self-loop) is a cycle group to report.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def find_cycle(known, prereqs, indeg):
    # every leftover course still has a leftover
    # prerequisite,
    # so walking backwards must eventually repeat a course
    left = {c for c in known if indeg[c] > 0}
    before = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            before[v].append(u)
    x = min(left, key=str)
    walk, pos = [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[pos[x]:] + [x]
    # walked against the edges: reverse to list
    # prerequisites first
    return cycle[::-1]

# ---- main question: main function ----
# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
# remove courses with no remaining prerequisites, building
# an order.
# FOLLOW-UP 1: Return all missing course references and
# one cycle.
# Answer: The function above returns the sorted unknown
# ids when any exist and otherwise a cycle (a list of
# courses starting and ending at the same course), found
# by walking backwards along prerequisites inside the set
# of courses Kahn's algorithm could not remove (every such
# course has a leftover prerequisite, so the walk cannot
# dead-end).
```

```
def validate_catalog(courses, prereqs):
    # prereqs (u, v): course v requires course u.
    # Returns (ok, missing ids, order or one cycle)
    known = set(courses)
    missing = sorted({x for pair in prereqs for x in pair
                      if x not in known}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in known}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    ready = [c for c in known if indeg[c] == 0]
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, indeg)

# ---- main question: tests ----
def test_main_question():
    courses = ["a", "b", "c", "d"]
    pre = [("a", "b"), ("b", "c"), ("a", "d")]
    ok, missing, order = validate_catalog(courses, pre)
    assert ok and order[0] == "a"
    assert order.index("b") < order.index("c")
    # an unknown course id
    bad = validate_catalog(courses, pre + [("zzz", "a")])
    assert bad == (False, ["zzz"], None)
    # a cycle b -> c -> b
    loop = pre + [("c", "b")]
    ok, _, cycle = validate_catalog(courses, loop)
    assert not ok and cycle[0] == cycle[-1]
```

```
    assert set(cycle) == {"b", "c"}


# ============================================================
# FOLLOW-UP 1: Return all missing course references and
```

```
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Group courses into semesters (the earliest
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run Kahn's algorithm level by level: all courses
# with no remaining prerequisites form round 1, remove
# them, the next set forms round 2, and so on. The number
# of rounds is the longest dependency chain.
def semesters(courses, prereqs):
    # earliest round each course can be taken: Kahn by
    # levels
    adj = defaultdict(list)
    indeg = {c: 0 for c in courses}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    level = sorted(c for c in courses if indeg[c] == 0)
    rounds = []
    while level:
        rounds.append(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = sorted(nxt)
    return rounds

# ---- follow-up 2: tests ----
def test_follow_up_2():
    courses = ["a", "b", "c", "d"]
    pre = [("a", "b"), ("b", "c"), ("a", "d")]
    rounds = semesters(courses, pre)
    assert rounds == [["a"], ["b", "d"], ["c"]]
    assert validate_catalog([], []) == (True, [], [])


# ============================================================
# FOLLOW-UP 3: Find all cycles, not just one.
# ============================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(course_ids: list[str], prereqs: list[tuple[st
r, str]]) -> tuple[bool, list[str], list[str], list[str]]:
```

Sample call

```
solution(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])
```

Test cases

Input	Expected
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])`	`(True, ['A', 'B', 'C'], [], [])`
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C'), ('C', 'A')])`	`(False, [], [], ['A', 'B', 'C', 'A'])`

STUDY PATH: question 218 of 288 | topic 29 of 43: Topological sort (Kahn) and cycle detection | 5 of 5

Template fit: edge = prerequisite -> course; return the order, or an empty list when there is a cycle | no date

MEDIUM | IN APP LC 207 | Exponent | 218 of 288

★ TOP 75: rank 40 in the Medium and Hard priority list

★ Course Schedule

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 37/288

ABOUT THIS QUESTION

Topic: Topological sort (Kahn) and cycle detection

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1017 interviews

Original: https://www.aced.io/questions/3313/course-schedule

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

EXAMPLES

Example 1

Input	`2, [[1, 0]]`
Output	`true`
Why	`Course 1 requires course 0: finishing both is possible.`

Example 2

Input	`2, [[1, 0], [0, 1]]`
Output	`false`
Why	`Course 1 requires 0 and course 0 requires 1: a cycle.`

QUESTION

Example 3

Input	`2, [[1, 0]]`
Output	`[0, 1]`
Why	`A valid order: take 0 before 1.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

The same parts of the graph are walked again for many starting courses, O(V * (V + E)). Kahn's algorithm (take courses with no unmet prerequisites) is O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (topological sort)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the order of courses (LeetCode 210).

The same Kahn loop records the order; if fewer than n courses are taken return an empty list. O(V + E).

FOLLOW-UP 2 The minimum number of semesters if you can take any number of courses per semester.

Run Kahn's algorithm level by level: each level is one semester. The number of levels is the longest chain of prerequisites (return -1 if there is a cycle).

FOLLOW-UP 3 Report an actual cycle.

After Kahn's algorithm, the courses left over all still have an unmet prerequisite among themselves; walk backwards along prerequisites inside that set until a course repeats to extract a cycle.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Course Schedule
#
# You are given numCourses and a list of prerequisites,
# where each element in prerequisites[i] = [a_i, b_i]
# means you must complete course b_i before taking course
# a_i. Write a function canFinish that returns True if
# it's possible to finish all courses, and False if it's
# not. You need to complete all the courses for the result
# to be True. Assume there are no duplicate prerequisites,
# and numCourses is a non-negative number. Examples Input:
# numCourses = 3, prerequisites = [[2, 1], [1, 0]] Output:
# True Explanation: You can complete course 0 first, then
# course 1, and finally course 2. Input: numCourses = 4,
# prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]] Output:
# False Explanation: The prerequisites form a cycle,
# making it impossible to finish all courses.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the order of courses (LeetCode 210).
# 2. The minimum number of semesters if you can take any
# number of courses per semester.
# 3. Report an actual cycle.
#
# INTERVIEW NOTES
# Finishing every course is possible exactly when the
# prerequisite graph has no cycle. Kahn's algorithm tests
# that by repeatedly taking courses with no unmet
# prerequisites; if you can take all of them there is no
# cycle, and the order you took them is a valid schedule.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def build(n, prereqs):
    # (a, b): course b must be taken before course a
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    return adj, indeg


# FOLLOW-UP 1: Return the order of courses (LeetCode 210).
# Answer: The same Kahn loop records the order; if fewer
# than n courses are taken return an empty list. O(V + E).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
```

```
def find_order(n, prereqs):
    # Kahn's algorithm: take courses with no unmet
    # prerequisite
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    # fewer than n taken: a cycle blocks the rest
    return order if len(order) == n else []

# ---- main question: main function ----
# BASE QUESTION (optimal): Kahn's algorithm (topological
# sort)
# Idea: Count each course's unmet prerequisites and queue
# those with none. Take a course, reduce the counts of the
# courses that need it, and queue any that reach zero.
def can_finish(n, prereqs):
    return len(find_order(n, prereqs)) == n

# ---- main question: tests ----
def test_main_question():
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    diamond = [(1, 0), (2, 0), (3, 1), (3, 2)]
    assert find_order(4, diamond)[0] == 0
```

```
    assert find_order(2, [(1, 0), (0, 1)]) == []
    assert can_finish(3, []) and can_finish(0, [])


# ============================================================
# FOLLOW-UP 1: Return the order of courses (LeetCode 210).
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# ============================================================
# FOLLOW-UP 2: The minimum number of semesters if you can
# ============================================================

# ---- follow-up 2: function ----
# Answer: Run Kahn's algorithm level by level: each level
# is one semester. The number of levels is the longest
# chain of prerequisites (return -1 if there is a cycle).
def min_semesters(n, prereqs):
    # any number of courses per semester: one BFS level
    # each
    adj, indeg = build(n, prereqs)
    level = [c for c in range(n) if indeg[c] == 0]
    semesters = taken = 0
    while level:
        semesters += 1
        taken += len(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = nxt
    return semesters if taken == n else -1

# ---- follow-up 2: tests ----
def test_follow_up_2():
    diamond = [(1, 0), (2, 0), (3, 1), (3, 2)]
    assert min_semesters(4, diamond) == 3
```

```
    assert min_semesters(2, [(1, 0), (0, 1)]) == -1
    assert min_semesters(3, []) == 1
    pairs = [(1, 0), (2, 1), (1, 2), (3, 2)]


# =============================================================
```

```
# FOLLOW-UP 3: Report an actual cycle.
# =============================================================

# ---- follow-up 3: function ----
# Answer: After Kahn's algorithm, the courses left over
# all still have an unmet prerequisite among themselves;
# walk backwards along prerequisites inside that set until
# a course repeats to extract a cycle.
def find_cycle(n, prereqs):
    # the leftover courses all have a leftover
    # prerequisite, so
    # walking backwards along prerequisites repeats a
    # course
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    while queue:
        c = queue.popleft()
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    left = {c for c in range(n) if indeg[c] > 0}
    if not left:
        return []
    before = defaultdict(list)
    for a, b in prereqs:
        if a in left and b in left:
            before[a].append(b)
    x, walk, pos = min(left), [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    return walk[pos[x]:]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    pairs = [(1, 0), (2, 1), (1, 2), (3, 2)]
    cycle = find_cycle(4, pairs)
    assert set(cycle) == {1, 2}
    assert find_cycle(2, [(1, 0)]) == []


# =============================================================
# RUN EVERYTHING
```

```
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples

```
Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]
]
Output: True
Explanation: You can complete course 0 first, then cou
rse 1, and finally course 2.

Input: numCourses = 4, prerequisites = [[3, 2], [2, 1]
, [1, 0], [0, 3]]
Output: False
Explanation: The prerequisites form a cycle, making it
 impossible to finish all courses.

Input: numCourses = 5, prerequisites = [[4, 2], [3, 1]
, [2, 0]]
Output: True
Explanation: You can complete the courses in order fro
m course 0 to course 4.
```

Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.Our solution processes the input data to determine if it's possible to finish all courses given the prerequisite constraints. It constructs a directed graph where nodes represent courses and edges represent the prerequisite relationships. The solution employs a queue to manage nodes with zero in-degrees (courses with no remaining prerequisites) and uses a list to keep track of the number of prerequisites (in-degrees) for each course. For each course with zero in-degrees, the solution dequeues it and processes its neighbors, reducing their in-degrees. If a neighbor's in-degree becomes zero, it is added to the queue. This approach ensures that all courses with no remaining prerequisites are processed and their dependencies are updated accordingly. By maintaining a count of the number of processed courses, the solution can determine if all courses can be finished. If the count of processed courses matches the total number of courses, it implies that there are no cyclic dependencies, and thus all courses can be completed. The solution uses a queue to efficiently manage courses with zero in-degrees and a vector to keep track of in-degrees. This method ensures that each course is processed only once, making the

OFFICIAL PAGE TEXT

approach both clear and efficient. NoteWe are using a BFS (implemented by a queue) for this solution because it helps process nodes level by level (course by course), ensuring that we first handle courses with no dependencies before moving to those that rely on other courses. This approach ensures that we can detect cycles in the course prerequisites, which would make it impossible to complete all courses. Time Complexity: The solution has a time complexity of O(numCourses + numPrerequisites). This is because we process each course (vertex) and each prerequisite pair (edge) exactly once. The time complexity reflects the total number of courses (numCourses) and the number of prerequisite relationships (numPrerequisites). Space Complexity: The solution uses O(numCourses + numPrerequisites) space for the adjacency list and the in-degree vector. The space complexity accounts for the storage needed for both the graph representation and the queue, which in the worst case, contains all courses.No answers contributed by the community yet.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 30 OF 43

Counting pairs by looking up each partner in a Counter or a bucket

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Counting pairs by looking up each partner in a Counter or a bucket | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	219. Progressive Array Partitioning	1Point3Acres	2026-08-11
M	220. K-diff Pairs in an Array	ColorStack	2025-09-16
M	221. Count Axis-Aligned Squares Among Grid Points	ColorStack	2025-11-03
M	222. Sum of Two Squares in Two Ways	HackTheRounds	no date
H	223. Maximum-Area Rectangle from Planar Points	1Point3Acres	2026-09-29

CODE TEMPLATE | GROUP 30 OF 43 | 5 QUESTIONS, ONE PATTERN

Counting pairs by looking up each partner in a Counter or a bucket

The one idea these questions share

For each value, compute the partner it would need (v + k, 2v, the other corner of a square) and look it up in a Counter or a set. When the partner depends on two things, enumerate the pairs once and bucket them by a shared key.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import Counter


def count_pairs(nums, k):
    counts = Counter(nums)                        # CHANGE 1: what you index
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)     # CHANGE 2: the partner you look up
```

2. Helper functions that usually get written next to it

```
from collections import defaultdict


def bucket_pairs(points):                    # group point pairs by a shared key (here: the midpoint and length)
    buckets = defaultdict(list)
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            p, q = points[i], points[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            buckets[key].append((p, q))
    return buckets
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
219	Progressive Array Partitioning	Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)
220	K-diff Pairs in an Array	the template as written: partner = v + k; for k == 0 count values that appear twice
221	Count Axis-Aligned Squares Among Grid Points	a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2
222	Sum of Two Squares in Two Ways	enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice
223	Maximum-Area Rectangle from Planar Points	bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 219 of 288 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 1 of 5

Template fit: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts) | asked 70 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 219 of 288

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 258/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

The problem is given in stages; stage 1 is: group the equal values of an array together. Return true when every group has a different size (no two values occur the same number of times). The function takes the occurrence count of each value as a dict, so counting stays separate from the rule and later stages can change only the rule.

Function signature

def stage1(counts: dict) -> bool

counts maps value -> how

many times it occurs

EXAMPLES

Example 1	

Input	`from collections import Counter`
	`nums = [1, 2, 3, 4, 2, 3, 3, 4, 4, 4]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> true`
Why	`The groups have sizes 1, 2, 3, 4: all different, so true.`

Example 2	

Input	`from collections import Counter`
	`nums = [1, 2, 3]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> false`
Why	`Every value occurs once, so all three groups have size 1: false.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	O(d)

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# The problem is given in stages; stage 1 is: group the
# equal values of an array together. Return true when
# every group has a different size (no two values occur
# the same number of times). The function takes the
# occurrence count of each value as a dict, so counting
# stays separate from the rule and later stages can change
```

```
# only the rule.
#
# Signature:
#   def stage1(counts: dict) -> bool        # counts maps
value -> how many times it occurs
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
# example, which is exactly the situation where you ask.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
# Idea: All counts are different exactly when the set of
```

```
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)

# ---- main question: tests ----
def test_main_question():
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True


# ============================================================
# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Do not guess. Write down what the second example
# pair implies, and check any candidate rule against BOTH.
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
```

```
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
```

```
            return False
    return True

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts. Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY PATH: question 220 of 288 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 2 of 5

Template fit: the template as written: partner = v + k; for k == 0 count values that appear twice | asked 399 days ago | frequency Single report

MEDIUM | IN APP LC 532 | ColorStack | technical interview | 220 of 288

K-diff Pairs in an Array

ASKED 2025-09-16 | 399 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 216/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: your app's interview approach (LeetCode #532)

LeetCode match: LC 532 K-diff Pairs in an Array

Frequency: Single report | Last asked: 2025-09-16 | Roles: SWE Intern

QUESTION

Given an integer array nums and an integer k , return the number of unique k-diff pairs in the array. A k-diff pair is a pair of integers (nums[i], nums[j]) where: i != j , and abs(nums[i] - nums[j]) == k . Two k-diff pairs (a, b) and (c, d) are considered the same (counted once) if they consist of the same pair of values. Count each distinct value pair only once.

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [3, 1, 4, 1, 5], k = 2`
Output	`2`
Why	`The unique 2-diff value pairs are (1, 3) and (3, 5).`

Example 2	
Input	`nums = [1, 2, 3, 4, 5], k = 1`
Output	`4`
Why	`(1,2), (2,3), (3,4), (4,5).`

Example 3	
Input	`nums = [1, 3, 1, 5, 4], k = 0`
Output	`1`
Why	`With k = 0 count values that appear at least twice; only 1 does.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -10^7 <= nums[i] <= 10^7
- 0 <= k <= 10^7

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_k_diff(nums, k):
    seen = set()
    for i in range(len(nums)):
        for j in range(i_+_1,_len(nums)):
            if abs(nums[i] - nums[j]) == k:
                seen.add(
                    (
                        min(nums[i], nums[j]),
                        max(nums[i], nums[j]),
                    )
                )
    return len(seen)
```

TIME	SPACE
O(n^2)	O(pairs)

WHY IT IS SLOW

Compares all index pairs and stores the value pairs to remove repeats.

OPTIMAL SOLUTION · Count the values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without a hash map.

Sort, then use two pointers: advance the right pointer when the gap is too small and the left when too large; when the gap equals k count it and skip duplicate values on both sides. O(n log n) time, O(1) extra space.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: K-diff Pairs in an Array
#
# Given an integer array nums and an integer k , return
# the number of unique k-diff pairs in the array. A k-diff
# pair is a pair of integers (nums[i], nums[j]) where: i
# != j , and abs(nums[i] - nums[j]) == k . Two k-diff
# pairs (a, b) and (c, d) are considered the same (counted
# once) if they consist of the same pair of values. Count
# each distinct value pair only once. Function signature:
# def count_k_diff_pairs(nums: list[int], k: int) -> int:
# """Return the number of unique value-pairs with absolute
# difference k.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without a hash map.
```

```
#
# INTERVIEW NOTES
# Counting unique value pairs, not index pairs, means
# working with distinct values. With a counter: for k > 0
# count values v whose v + k also exists; for k = 0 count
# values that appear at least twice. Sorting with two
# pointers gives the same answer in O(1) extra space.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ========================================================

from collections import Counter
import random


# ========================================================
# MAIN QUESTION
# ========================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Count the values
# Idea: A value v starts a pair when v + k is present; for
# k = 0 it must appear twice.
def count_k_diff_pairs(nums, k):
    if k < 0:
        return 0
```

```
    counts = Counter(nums)
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)

# ---- main question: tests ----
def test_main_question():
    random.seed(8)
    assert count_k_diff_pairs([3, 1, 4, 1, 5], 2) == 2
    assert count_k_diff_pairs([1, 2, 3, 4, 5], 1) == 4
    assert count_k_diff_pairs([1, 3, 1, 5, 4], 0) == 1


# ========================================================
# FOLLOW-UP 1: Do it without a hash map.
# ========================================================

# ---- follow-up 1: function ----
# Answer: Sort, then use two pointers: advance the right
# pointer when the gap is too small and the left when too
# large; when the gap equals k count it and skip duplicate
# values on both sides. O(n log n) time, O(1) extra space.
def count_k_diff_sorted(nums, k):
    nums = sorted(nums)
    left, right, pairs = 0, 1, 0
    while right < len(nums) and left < len(nums):
        if (
            left == right
            or nums[right] - nums[left] < k
        ):
            right += 1
        elif nums[right] - nums[left] > k:
            left += 1
        else:
            pairs += 1
            left += 1
            while (
                left < len(nums)
                and nums[left] == nums[left - 1]
```

```
            ):
                left += 1
    return pairs


# ========================================================
```

```
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 221 of 288 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 3 of 5

Template fit: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2 | asked 351 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 221 of 288

Count Axis-Aligned Squares Among Grid Points

ASKED 2025-11-03 | 351 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 228/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #2013 Detect Squares

LeetCode match: LC 2013 Detect Squares (related: LC 223 / counting squares)

Frequency: Single report | Last asked: 2025-11-03 | Roles: SWE Intern

QUESTION

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice points in the plane. Count the number of axis-aligned squares whose four corners all appear in points . A square is axis-aligned when its sides are parallel to the x- and y-axes. Return the count of such squares.

Function signature

```
from typing import List, Tuple
def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...
```

Follow-up (as reported): A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

EXAMPLES

Example 1	
Input	`points = [(0, 0), (0, 1), (1, 0), (1, 1)]`
Output	`1`
Why	`The four points form exactly one unit square.`

QUESTION

Example 2	
Input	`points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]`
Output	`5`
Why	`A 3x3 grid of points has four 1x1 squares and one 2x2 square.`

Example 3	
Input	`points = [(0, 0), (2, 0), (0, 1), (2, 1)]`
Output	`0`
Why	`These form a 2x1 rectangle, which is not a square.`

CONSTRAINTS

- 0 <= len(points) <= 2000 ; all points are distinct.
- Each point is (x, y) with -10^9 <= x, y <= 10^9 .
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from itertools import combinations
def brute_squares(points):
    count = 0
    for quad in combinations(points,_4):
        xs = sorted({p[0] for p in quad})
        ys = sorted({p[1] for p in quad})
        if (
            len(xs) == len(ys) == 2
            and xs[1] - xs[0] == ys[1] - ys[0]
        ):
            corners = {
                (x, y) for x in xs for y in ys
            }
            if corners == set(quad):
                count += 1
    return count
```

TIME	SPACE
O(n^4)	O(1)

WHY IT IS SLOW

It looks at every group of four points although two opposite corners already fix the other two.

OPTIMAL SOLUTION · Opposite corners and a set

TIME	SPACE
O(n^2)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Points may repeat. Count every distinct choice of four points.

Keep a Counter. For each unordered pair of distinct diagonal points multiply the four counts; halve the total because each square has two diagonals.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Count Axis-Aligned Squares Among Grid
# Points
#
# The interviewer described a diagram of squares and
# rectangles and asked which data structure to use and how
# to count the squares. Framed concretely: you are given a
# list of distinct integer lattice points in the plane.
# Count the number of axis-aligned squares whose four
# corners all appear in points . A square is axis-aligned
# when its sides are parallel to the x- and y-axes. Return
# the count of such squares. Function signature: from
# typing import List, Tuple def
# count_axis_aligned_squares(points: List[Tuple[int,
# int]]) -> int: """Return the number of axis-aligned
# squares whose 4 corners are all in points.""" ...
# Follow-up (as reported): A cleaner O(n^2) approach: put
# all points in a set, then for every pair that could be
# two opposite corners of a square (equal absolute x- and
# y-difference), check the other two corners; each square
# is found via both its diagonals, so divide the total by
# 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Points may repeat. Count every distinct choice of
# four points.
#
# INTERVIEW NOTES
# Put every point in a set. Two points that sit on
# opposite corners of an axis-aligned square have |dx| ==
# |dy| > 0, and then the other two corners are fixed: (x1,
# y2) and (x2, y1). Check both in the set. Every square
# has two diagonals, so each is found twice over unordered
# pairs: divide by 2.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Opposite corners and a set
# Idea: Two opposite corners decide the other two, so one
# set lookup per pair is enough.
def count_axis_aligned_squares(points):
    seen = set(points)
    total = 0
    for i, (x1, y1) in enumerate(points):
        for x2, y2 in points[i + 1 :]:
            if abs(x1 - x2) == abs(y1 - y2) != 0:
                if (x1, y2) in seen and (
                    x2,
                    y1,
                ) in seen:
                    total += 1
    return total // 2

# ---- main question: tests ----
def test_main_question():
    random.seed(78)
    assert (
        count_axis_aligned_squares(
            [(0, 0), (0, 1), (1, 0), (1, 1)]
        )
        == 1
    )
    grid = [(x, y) for x in range(3) for y in range(3)]
    assert count_axis_aligned_squares(grid) == 5
    assert (
        count_axis_aligned_squares(
            [(0, 0), (2, 0), (0, 1), (2, 1)]
        )
        == 0
    )
    assert count_axis_aligned_squares([]) == 0


# ============================================================
# FOLLOW-UP 1: Points may repeat. Count every distinct
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Keep a Counter. For each unordered pair of
# distinct diagonal points multiply the four counts; halve
# the total because each square has two diagonals.
def count_squares_with_duplicates(points):
    cnt = Counter(points)
    keys = list(cnt)
    total = 0
    for i, (x1, y1) in enumerate(keys):
        for x2, y2 in keys[i + 1 :]:
            if abs(x1 - x2) == abs(y1 - y2) != 0:
                total += (
                    cnt[(x1, y1)]
                    * cnt[(x2, y2)]
                    * cnt.get((x1, y2), 0)
                    * cnt.get((x2, y1), 0)
                )
    return total // 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 222 of 288 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 4 of 5

Template fit: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice | no date

MEDIUM | HackTheRounds | Onsite | 222 of 288

Sum of Two Squares in Two Ways

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 266/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: the interview site's own solution

QUESTION

Given an integer n, find all integers s < n that can be written as a sum of two positive squares in exactly two different ways (the order of the two squares does not matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return them in increasing order.

Function signature

def two_squares_two_ways(n: int) -> list

EXAMPLES

Example 1	
Input	`100`
Output	`[50, 65, 85]`
Why	`50 = 1 + 49 = 25 + 25 and 65 = 1 + 64 = 16 + 49 both have exactly two ways.`

Example 2	
Input	`50`
Output	`[]`
Why	`Nothing below 50 qualifies.`

CONSTRAINTS

- 1 <= n <= 2^31 - 1

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for s in range(1, n):
        ways = 0
        for a in range(1, isqrt(s) + 1):
            rest = s - a * a
            b = isqrt(rest)
            if b >= a and b * b == rest:
                ways += 1
        if ways == 2:
            result.append(s)
    return result
```

TIME	SPACE
O(n * sqrt(n))	O(1)

WHY IT IS SLOW

BRUTE FORCE

Every s searches all pairs again, O(n * sqrt(n)). Generating each pair once and counting sums in a hash map is O(n).

OPTIMAL SOLUTION · Count every pair once in a hash map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Exactly k ways instead of two.

Change the comparison from == 2 to == k; the counting loop is the same. For k = 3 the smallest is 325.

FOLLOW-UP 2 Allow zero (0^2 + b^2).

Start a at 0; then perfect squares like 25 gain a representation, so some answers appear or disappear. Check against the examples to see which convention the question uses (here, positive only).

FOLLOW-UP 3 n is huge (10^12) and you only need to test one number.

Factorise the number: the count of representations as a sum of two squares depends only on the primes congruent to 1 mod 4 (Jacobi's formula), so a factorisation answers it in sublinear time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sum of Two Squares in Two Ways
#
# Given an integer n, find all integers s < n that can be
# written as a sum of two positive squares in exactly two
# different ways (the order of the two squares does not
# matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return
# them in increasing order.
#
# Signature:
#   def two_squares_two_ways(n: int) -> list
#
# Examples:
# 100  ->  [50, 65, 85]
# 50  ->  []
#
# FOLLOW-UPS (answered below, in this order):
# 1. Exactly k ways instead of two.
# 2. Allow zero (0^2 + b^2).
# 3. n is huge (10^12) and you only need to test one
# number.
```

```
#
# INTERVIEW NOTES
# Do not test each s separately. Generate every pair (a,
# b) once, add its sum to a counter, and read the answer
# from the counter. The loop is bounded by a^2 + b^2 < n,
# so a and b are at most sqrt(n); a <= b removes order
# duplicates.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter
from math import isqrt


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Count every pair once in a hash
```

```
# map
# Idea: Loop a from 1 while 2a^2 < n and b from a while
# a^2 + b^2 < n, adding each sum to a Counter. There are
# about pi * n / 8 pairs. The answer is the sorted list of
# sums whose count is exactly 2.
# FOLLOW-UP 1: Exactly k ways instead of two.
# Answer: Change the comparison from == 2 to == k; the
# counting loop is the same. For k = 3 the smallest is
# 325.
# FOLLOW-UP 2: Allow zero (0^2 + b^2).
# Answer: Start a at 0; then perfect squares like 25 gain
# a representation, so some answers appear or disappear.
# Check against the examples to see which convention the
# question uses (here, positive only).
def sums_in_k_ways(n, k=2, zero=False):
    # every s < n is counted once per pair a <= b with
    # a*a + b*b = s. Keep the sums seen exactly k times.
    ways = Counter()
    # FOLLOW-UP 2: zero=True allows 0^2 + b^2
    a = 0 if zero else 1
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            ways[a * a + b * b] += 1
            b += 1
        a += 1
    # FOLLOW-UP 1: k ways instead of exactly two
    return sorted(s for s, c in ways.items()
                  if c == k and s > 0)

# ---- main question: tests ----
def test_main_question():
    assert sums_in_k_ways(100) == [50, 65, 85]
    assert sums_in_k_ways(10) == []
    assert sums_in_k_ways(1000, 3)[0] == 325
    # with zero allowed, perfect squares gain a way
    assert 25 not in sums_in_k_ways(100)
    assert 25 in sums_in_k_ways(100, 2, zero=True)


# ============================================================
# FOLLOW-UP 1: Exactly k ways instead of two.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.
```

```
# ============================================================
# FOLLOW-UP 2: Allow zero (0^2 + b^2).
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: n is huge (10^12) and you only need to test
# ============================================================

# ---- follow-up 3: helper functions ----
def factorise(m):
    # trial division: fine up to about 10 ** 12
    out, p = {}, 2
    while p * p <= m:
        while m % p == 0:
            out[p] = out.get(p, 0) + 1
            m //= p
        p += 1
    if m > 1:
        out[m] = out.get(m, 0) + 1
    return out

# ---- follow-up 3: function ----
# Answer: Factorise the number: the count of
# representations as a sum of two squares depends only on
# the primes congruent to 1 mod 4 (Jacobi's formula), so a
# factorisation answers it in sublinear time.
def representations(m, zero=False):
    # follow-up: ONE huge number. The number of ways
    # depends
    # only on the primes that are 1 mod 4 (Jacobi): with
    # B = product of (exponent + 1) there are 4B ordered
    # signed pairs. Remove the special shapes a == 0 and
    # a == b, then every other unordered pair is counted 8
    # times.
    if m <= 0:
        return 0
    boost = 1
    for p, e in factorise(m).items():
        if p % 4 == 3 and e % 2:
            return 0
        if p % 4 == 1:
            boost *= e + 1
    is_square = isqrt(m) ** 2 == m
    half = m % 2 == 0 and isqrt(m // 2) ** 2 == m // 2
```

```
    plain = (4 * boost - 4 * is_square - 4 * half) // 8
    return plain + half + (is_square if zero else 0)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert representations(50) == 2
    assert representations(325) == 3
    assert representations(3) == 0
    assert representations(25) == 1
    assert representations(25, zero=True) == 2
    # a 12 digit number in a moment (13 * 5 * 5 * ...)
    assert representations(5 ** 5 * 13 * 17 * 29) > 10
    for m in range(1, 600):
        both = representations(m, True)
        if m < 100:
            two = representations(m) == 2
            assert (m in sums_in_k_ways(100)) == two


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 1 more

Find numbers with exactly two sum-of-squares forms | PracHub | Technical Screen

https://prachub.com/coding-questions/find-numbers-with-exactly-two-sum-of-squares-forms

STUDY PATH: question 223 of 288 | topic 30 of 43: Counting pairs by looking up each partner in a Counter or a bucket | 5 of 5

Template fit: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle | asked 21 days ago | frequency Low

HARD | 1Point3Acres | onsite-coding | 223 of 288

Maximum-Area Rectangle from Planar Points

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 201/288

ABOUT THIS QUESTION

Topic: Counting pairs by looking up each partner in a Counter or a bucket

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #963 Minimum Area Rectangle II

LeetCode match: LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

EXAMPLES

Example 1	
Input	`[(1, 1), (1, 3), (3, 1), (3, 3)]`
Output	`4`
Why	`The four points form a 2 x 2 axis-aligned square.`

Example 2	
Input	`[(0, 1), (1, 0), (2, 1), (1, 2)]`
Output	`2`
Why	`A tilted rectangle: a square with diagonal 2 has area 2.`

Example 3	
Input	`[(0, 0), (1, 0), (0, 1)]`
Output	`0`
Why	`Three points never make a rectangle.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

TIME	SPACE
O(n^2) pairs (worst case O(n^3) if every pair shares a bucket)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum-Area Rectangle from Planar Points
#
# Given 2D points, find the maximum area of a rectangle
# whose four corners are input points; sides need not be
# parallel to the axes. Candidates compare it to LC 963
# (Minimum Area Rectangle II) but maximizing. Interviewers
# ask to improve beyond an O(N^3) approach. Input: a list
# of 2D coordinate points. Return the maximum area of any
# rectangle whose four vertices are all input points.
```

```
# Rectangle sides do not need to be parallel to the
# coordinate axes. Follow-up pressure: start from an
# O(N^3) idea, then optimize toward a better complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can you do better than O(n^2) pairs?
#
# INTERVIEW NOTES
# Four points form a rectangle exactly when two of its
# diagonals share the same midpoint and have the same
# length. So you never need to pick four points: bucket
# every pair of points by (sum of the two points, squared
# distance). Two different pairs in one bucket are the two
# diagonals of a rectangle, and the area is the cross
# product of two adjacent sides.

# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ========================================================

from collections import defaultdict
from itertools import combinations


# ========================================================
# MAIN QUESTION
# ========================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx


# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# Answer: Not in general: the answer can depend on all
# pairs, and even deciding whether any rectangle exists is
# believed to need about n^2 work.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))

# ---- main question: main function ----
# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)


# RELATED VARIANT: smallest area (LeetCode 963)
def min_rect_area(points):
    # LeetCode 963 asks for the smallest instead
    return min(all_rectangle_areas(points), default=0)

# ---- main question: tests ----
def test_main_question():
    # a tilted square of area 2
    sq = [(0, 0), (1, 1), (0, 2), (-1, 1)]
    assert max_rect_area(sq) == 2
    # axis-aligned 2 x 1 rectangle plus a stray point
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]
    assert max_rect_area(rect) == 2
    # collinear points make no rectangle
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0
    # the minimum version picks the small one
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2
    assert max_rect_area(two) == 6


# ========================================================
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.

Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.

Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 31 OF 43

Tree DP: build a parent order once, then add up from the leaves

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	2
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree DP: build a parent order once, then add up from the leaves | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	224. Compute File System Total Size	1Point3Acres	2026-09-02
M	225. Minimum-Cost Tree Disconnection	1Point3Acres	2026-09-09
M	226. Given a large n-ary tree where the root node can pass information to one child per iteration, and i...	Exponent	no date
H	227. Compute distance-sum from every tree node	PracHub	2026-01-22
H	228. Tree Distance Sum (Re-root DP)	1Point3Acres	2026-01-19

CODE TEMPLATE | GROUP 31 OF 43 | 5 QUESTIONS, ONE PATTERN

Tree DP: build a parent order once, then add up from the leaves

The one idea these questions share

Get an order where every parent comes before its children (a BFS order from the root), then sweep it backwards so every child is finished before its parent. Subtree sizes and sums fall out of that one backwards sweep; the re-rooting questions add a second forward sweep.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def subtree_sums(n, adj, root=0):
    parent, order = tree_order(n, adj, root)
    size, down = [1] * n, [0] * n
    for u in reversed(order):             # children before parents
        p = parent[u]
        if p != -1:
            size[p] += size[u]
            down[p] += down[u] + size[u]  # CHANGE: what you add up
    return parent, order, size, down
```

2. Helper functions that usually get written next to it

```
def tree_order(n, adj, root=0):
    parent, order = {root: -1}, [root]
    for u in order:                       # BFS order: parents before children
        for v in adj[u]:
            if v not in parent:
                parent[v] = u
                order.append(v)
    return parent, order


def adjacency_from_edges(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
224	Compute File System Total Size	add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)
225	Minimum-Cost Tree Disconnection	backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free
226	Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.	t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root
227	Compute distance-sum from every tree node	down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child])
228	Tree Distance Sum (Re-root DP)	exactly the same two sweeps (this is the re-rooting version of the question)

Contents Start the questions >

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 224 of 288 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 1 of 5

Template fit: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set) | asked 48 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 224 of 288

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 264/288

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

A file system is a tree: File(size) is a file and Dir(*children) is a directory holding files and directories. Return the total size of all files. A directory object can be shared (it appears under more than one parent); its files are counted once.

Function signature

def total_size(root) -> int

EXAMPLES

Example 1	
Input	`root = Dir(Dir(File(10), File(20)), File(5), Dir(File(30)))` `total_size(root)`
Output	`total_size(root) -> 65`
Why	`10 + 20 + 5 + 30 = 65.`

Example 2	

Input	`shared = Dir(File(7))`
	`root = Dir(shared, shared, File(1))`
	`total_size(root)`

Output	`total_size(root) -> 8`
Why	`The same directory appears twice but its 7 is counted once: 7 + 1 = 8.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

BRUTE FORCE

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute File System Total Size
#
# A file system is a tree: File(size) is a file and
# Dir(*children) is a directory holding files and
# directories. Return the total size of all files. A
# directory object can be shared (it appears under more
# than one parent); its files are counted once.
#
# Signature:
#   def total_size(root) -> int
#
```

```
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
class File:
    def __init__(self, size):
        self.size, self.parent = size, None
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total

# ---- main question: tests ----
def test_main_question():
    root = Dir(Dir(File(10), File(20)), File(5), Dir(File(
30)))
    assert total_size(root) == 65
```

```
# ============================================================
# FOLLOW-UP 1: The tree is queried often and files change.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Cache each directory's total and, when a file
# changes, clear the cache only for its ancestors (walk
# parent links). A query recomputes only cleared nodes.
# Updates cost O(depth).
```

```
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent

# ============================================================
# FOLLOW-UP 2: Hard links: the same file appears under two
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Find the largest directory.
# ============================================================

# ---- follow-up 3: helper functions ----
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
```

```
            node.cached = sum(size_of(c) for c in kids)
    return root.cached

# ---- follow-up 3: function ----
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def directory_totals(root):
    # the size of every directory, in one pass
    cached_total(root)
    out, stack = [], [root]
    while stack:
```

```
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
            stack.extend(node.children)
    return out


# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
```

```
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

STUDY PATH: question 225 of 288 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 2 of 5

Template fit: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free | asked 41 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 225 of 288

Minimum-Cost Tree Disconnection

ASKED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 257/288

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

QUESTION

A rooted tree is given as directed edges (parent, child, cost) with node 0 as the root. Cut edges so that no path from the root to any original leaf remains. The price of a cut set is the sum of the costs of the edges cut (costs may be negative). Return the minimum total price.

Function signature

def disconnect_cost(n: int, edges: list) -> int

EXAMPLES

Example 1	
Input	`3, [(0, 1, 3), (0, 2, 2)]`
Output	`5`
Why	`Two leaves hang off the root: both edges must be cut, 3 + 2 = 5.`

Example 2	
Input	`3, [(0, 1, 4), (1, 2, 1)]`
Output	`1`
Why	`A chain 0 - 1 - 2: cutting the cheaper edge 1 -> 2 (cost 1) already separates the leaf.`

Example 3	
Input	`3, [(0, 1, -1), (1, 2, -1)]`
Output	`-2`
Why	`Negative costs: cutting both edges costs -2, which beats cutting only one.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, edges):
    best = INF
    for k in range(len(edges) + 1):
        for cut in combinations(edges, k):
            tree = remove_edges(edges, cut)
            if not reaches_leaf(tree, root):
                cost = sum(w for _, _, w in cut)
                best = min(best, cost)
    return best
```

TIME	SPACE
O(2^E * n)	O(n)

WHY IT IS SLOW

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

OPTIMAL SOLUTION · Tree DP: best(v) = cheapest cuts inside v's subtree that disconnect v from its leaves

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

FOLLOW-UP 2 The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

FOLLOW-UP 3 Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Minimum-Cost Tree Disconnection
#
# A rooted tree is given as directed edges (parent, child,
# cost) with node 0 as the root. Cut edges so that no path
# from the root to any original leaf remains. The price of
# a cut set is the sum of the costs of the edges cut
# (costs may be negative). Return the minimum total price.
#
# Signature:
#   def disconnect_cost(n: int, edges: list) -> int
#
# Examples:
# 3, [(0, 1, 3), (0, 2, 2)]  ->  5
# 3, [(0, 1, 4), (1, 2, 1)]  ->  1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Weights are non-negative.
# 2. The tree is a long chain and recursion overflows.
# 3. Return which edges to cut.
#
# INTERVIEW NOTES
# On a tree, every decision about a subtree depends only
# on the edge above it, so define a function of the node
# that returns the best cost inside its subtree. For each
# child you either cut the edge above it (pay its weight)
# or keep it and make the child itself disconnected from
# the leaves below. With negative weights, a cut edge
# frees the whole subtree, so you may cut every negative
# edge in it for free gain.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
INF = float("inf")


def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    # FOLLOW-UP 2: a loop, not recursion: safe on a long
    # chain
    # breadth-first order: parents come before children
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free

# ---- main question: main function ----
# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free(c)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(c), which is impossible when c is
# a leaf.
# FOLLOW-UP 1: Weights are non-negative.
# Answer: Then free(c) is always 0 and the recurrence
# collapses to best(v) = sum over children of (w if the
# child is a leaf, else min(w, best(child))).
# FOLLOW-UP 2: The tree is a long chain and recursion
# overflows.
# Answer: Compute nodes in reverse BFS order (children
```

```
# before parents) and store best and free in arrays, no
# recursion needed. State the space as O(n).
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]

# ---- main question: tests ----
def test_main_question():
    # the question's chain A -(-1)-> B -(-1)-> C
    neg = [(0, 1, -1), (1, 2, -1)]
    assert disconnect_cost(3, neg) == -2


# ============================================================
# FOLLOW-UP 1: Weights are non-negative.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: The tree is a long chain and recursion
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Return which edges to cut.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Remember at each child which option was cheaper,
# then walk down from the root collecting the cut edges.
# Cut edges also include every negative edge below any cut
# edge.
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts, stack = [], [0]
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            keep = best[c] if kids[c] else INF
            if w + free[c] <= keep:
                # cut here, plus negative edges below c
                cuts.append((v, c))
                below = [c]
                while below:
                    x = below.pop()
```

```
                    for y, wy in kids[x]:
                        if wy < 0:
```

```
                        cuts.append((x, y))
                    below.append(y)
            else:
                stack.append(c)
    return cuts

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # the question's chain A -(-1)-> B -(-1)-> C
    neg = [(0, 1, -1), (1, 2, -1)]
    cuts = sorted(edges_to_cut(3, neg))
    assert cuts == [(0, 1), (1, 2)]
    # non-negative costs: cut the cheaper edge
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3
    assert edges_to_cut(3, chain) == [(1, 2)]
    # a single node needs no cut
    assert disconnect_cost(1, []) == 0
    # a long chain does not overflow the stack
    big = [(i, i + 1, 1) for i in range(5000)]
    assert disconnect_cost(5001, big) == 1


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

STUDY PATH: question 226 of 288 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 3 of 5

Template fit: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root | no date

MEDIUM | IN APP LC 1376 | Exponent | 226 of 288

Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

ASKED (ANY CO.) ~4 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 159/288

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: your app's interview approach (LeetCode #1376)

LeetCode match: LC 1376 Time Needed to Inform All Employees

Last asked (any company): 4 years ago, about 2022-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/2411/minimum-iterations-information-distribution-n-ary-tree

QUESTION

Information starts at the root of a tree. In every iteration a node that already has the information can pass it to one of its children (the tree maps each node to its children as (child, cost) pairs; in the basic problem every cost is 1 iteration). Return the minimum number of iterations until every node has the information.

Function signature

def min_rounds(tree: dict, root) -> int

EXAMPLES

Example 1	
Input	`{'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]}, 'r'`
Output	`3`
Why	`The root informs a, b and c in iterations 1, 2, 3, while a informs d in iteration 2: everyone knows after 3 iterations.`

Example 2	
Input	`{'r': [('a', 1)], 'a': [('b', 1)]}, 'r'`
Output	`2`
Why	`A path r - a - b needs one iteration per edge.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def T(v):
    kids = children[v]
    if not kids:
        return 0
    best = INF
    for order in permutations(kids):
        finish = 0
        for i, c in enumerate(order, 1):
            finish = max(finish, i + T(c))
        best = min(best, finish)
    return best
```

TIME	SPACE
O(product of d! over nodes)	O(n)

WHY IT IS SLOW

A node with d children has d! orderings, so the cost is the product of d! over all nodes. Sorting the children by decreasing T gives the same answer in O(d log d).

OPTIMAL SOLUTION · Post-order DP: serve the slowest subtree first

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is it optimal to serve the child with the largest subtree time first?

Exchange argument: if a child with time a is served in round i and one with time b >= a in round i + 1, swapping them changes the finish times from max(i + a, i + 1 + b) to max(i + b, i + 1 + a), and the second is never larger because b >= a. So sorting by decreasing T minimizes the maximum. **Cost:** sorting the children at each node makes the whole computation O(n log n); the exchange argument adds nothing.

FOLLOW-UP 2 What if each edge has a different transmission cost?

Replace the unit step by the cost: the child served in order position i can only start after the sum of the costs of the edges before it, so T(v) = max over i of (prefix cost up to child_i + T(child_i)). The best order is still decreasing (cost + T) only when costs are equal; in general it becomes a scheduling problem solved by sorting on T(child) - cost, i.e. Smith's rule.

FOLLOW-UP 3 Each call takes a different time per child.

The order still matters, and the right rule is Jackson's: serve children in decreasing order of their own finishing time T, whatever each call costs, because the finish of the child served at prefix cost P is P + T and the largest tail should start earliest. Checked against trying every order on random trees.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given a large n-ary tree where the root
# node can pass information to one child per iteration,
# and in subsequent iterations, nodes can pass information
# to one child while the root continues with others, find
# the minimum number of iterations to reach all nodes.
#
# Information starts at the root of a tree. In every
# iteration a node that already has the information can
# pass it to one of its children (the tree maps each node
# to its children as (child, cost) pairs; in the basic
# problem every cost is 1 iteration). Return the minimum
# number of iterations until every node has the
# information.
#
# Signature:
#   def min_rounds(tree: dict, root) -> int
#
# Examples:
# {'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]},
#     'r'  ->  3
# {'r': [('a', 1)], 'a': [('b', 1)]}, 'r'  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is it optimal to serve the child with the largest
# subtree time first?
# 2. What if each edge has a different transmission cost?
# 3. Each call takes a different time per child.
#
# INTERVIEW NOTES
# Each node can call one child per round, so a node with
# several children must choose an order. The subtree that
# needs the most rounds should be called first, because it
# starts earliest and the others can wait. That makes the
# time at a node the maximum over children of (position in
# the calling order + the child's own time), a bottom-up
# computation.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random
```

```
# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
def order_children(tree, node):
    # children of `node` as (child, cost) pairs
    return tree.get(node, [])


# FOLLOW-UP 2: What if each edge has a different
# transmission cost?
# Answer: Replace the unit step by the cost: the child
# served in order position i can only start after the sum
# of the costs of the edges before it, so T(v) = max over
# i of (prefix cost up to child_i + T(child_i)).
# FOLLOW-UP 3: Each call takes a different time per child.
# Answer: The order still matters, and the right rule is
# Jackson's: serve children in decreasing order of their
# own finishing time T, whatever each call costs, because
# the finish of the child served at prefix cost P is P + T
# and the largest tail should start earliest.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def finish_times(tree, root):
    # time[v] = rounds until v and everything below has
    # the
    # news, once v has it. v calls its children one after
    # another; the call to the i-th child costs the edges
    # so
    # far, and that child then needs time[child] more.
    # Best order (Jackson's rule): the child with the
    # LARGEST time[child] first.
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        kids = order_children(tree, node)
        stack.extend(c for c, _ in kids)
    time, plan = {}, {}
    # process children before parents
    for node in reversed(order):
```

```
        kids = sorted(order_children(tree, node),
                      key=lambda kc: -time[kc[0]])
        spent, worst = 0, 0
        for child, cost in kids:
            spent += cost
            worst = max(worst, spent + time[child])
        time[node] = worst
        plan[node] = [c for c, _ in kids]
    return time, plan

# ---- main question: main function ----
# BASE QUESTION (optimal): Post-order DP: serve the
# slowest subtree first
# Idea: Let T(v) be the rounds needed to inform the
# subtree of v once v knows. v informs its children one
# per round.
def min_rounds(tree, root):
    return finish_times(tree, root)[0][root]


# UNIT cost helper (every call takes 1 round)
def unit_tree(children):
    # every call takes one round
    return {v: [(c, 1) for c in kids]
            for v, kids in children.items()}

# ---- main question: tests ----
def test_main_question():
    # a root with three leaves needs 3 rounds
    star = unit_tree({"r": ["a", "b", "c"]})
```

```
    assert min_rounds(star, "r") == 3
    assert min_rounds({}, "r") == 0
    # serve the child with the deeper subtree first
    deep = unit_tree({"r": ["a", "b"], "b": ["c", "d"]})
    assert min_rounds(deep, "r") == 3
    # the plan: b (the slow subtree) is called before a
    plan = finish_times(deep, "r")[1]
    assert plan["r"] == ["b", "a"]
    # calls with different costs: (child, cost)
    costly = {"r": [("a", 5), ("b", 1)], "b": [("c", 1)]}
    assert min_rounds(costly, "r") == 6
    for _ in range(200):
        tree, nodes = {}, [0]
        for v in range(1, random.randint(1, 7)):
            parent = random.choice(nodes)
```

```
            cost = random.randint(1, 4)
            tree.setdefault(parent, []).append((v, cost))
            nodes.append(v)
    # a very deep tree does not need recursion
    chain = {i: [(i + 1, 1)] for i in range(20000)}
    assert min_rounds(chain, 0) == 20000


# =========================================================
# FOLLOW-UP 1
# =========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# =========================================================
# FOLLOW-UP 2: What if each edge has a different
# =========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# =========================================================
# FOLLOW-UP 3: Each call takes a different time per child.
# =========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY PATH: question 227 of 288 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 4 of 5

Template fit: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child]) | asked 271 days ago

HARD | IN APP LC 834 | PracHub | Technical Screen | 227 of 288

Compute distance-sum from every tree node

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 143/288

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #834 Sum of Distances in Tree

LeetCode match: LC 834 Sum of Distances in Tree

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-distance-sum-from-every-tree-node

QUESTION

A tree has n nodes numbered 0 to n - 1 and n - 1 edges (u, v). For every node, return the sum of the distances from that node to all the other nodes.

Function signature

def distance_sums(n: int, edges: list) -> list

EXAMPLES

Example 1	
Input	`4, [(0, 1), (0, 2), (2, 3)]`
Output	`[4, 6, 4, 6]`
Why	`Node 0: 1 + 1 + 2 = 4. Node 1 is a leaf: 1 + 2 + 3 = 6. Node 2: 1 + 2 + 1 = 4. Node 3: 1 + 2 + 3 = 6.`

Example 2	
Input	`3, [(0, 1), (1, 2)]`
Output	`[3, 2, 3]`
Why	`A path of three nodes: the middle one is closest to the others.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = []
    for node in range(n):
        dist = bfs(adj, node)
        ans.append(sum(dist))
    return ans
```

TIME	SPACE
O(n^2)	O(n)

BRUTE FORCE

WHY IT IS SLOW

n searches of O(n) each, O(n^2). Compute the answer for the root, then move the root edge by edge using subtree sizes (re-rooting), O(n).

OPTIMAL SOLUTION · Re-rooting with subtree sizes

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

FOLLOW-UP 2 The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

FOLLOW-UP 3 Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Compute distance-sum from every tree node
#
# A tree has n nodes numbered 0 to n - 1 and n - 1 edges
# (u, v). For every node, return the sum of the distances
# from that node to all the other nodes.
#
# Signature:
#   def distance_sums(n: int, edges: list) -> list
#
# Examples:
# 4, [(0, 1), (0, 2), (2, 3)]  ->  [4, 6, 4, 6]
# 3, [(0, 1), (1, 2)]  ->  [3, 2, 3]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Find the node with the smallest distance sum (the
# tree's centroid).
# 2. The tree has weighted edges.
# 3. Maximum distance (eccentricity) from each node
```

```
# instead of the sum.
#
# INTERVIEW NOTES
# Computing the sum of distances from each node separately
# costs O(n) per node. Instead compute it once for the
# root, then derive the neighbour's answer in O(1): moving
# the root from a parent to its child makes every node in
# the child's subtree one step closer and every other node
# one step farther. That needs only the subtree sizes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def build(n, edges):
    adj = [[] for _ in range(n)]
    for e in edges:
        u, v = e[0], e[1]
```

```
        w = e[2] if len(e) > 2 else 1
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (a tree: one pass)
    dist, stack = {s: 0}, [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist

# ---- main question: main function ----
# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[v] (subtree
# sizes) and ans[0] = the sum of depths.
def distance_sums(n, edges):
    # sum of distances from every node; edges may be
    # weighted
    adj = build(n, edges)
    parent, order, up = [-1] * n, [0], [0] * n
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v], up[v] = u, w
                order.append(v)
    size = [1] * n
    for u in reversed(order):
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    # the root's sum is a plain sum of distances
    ans = [0] * n
    ans[0] = sum(distances_from(adj, 0).values())
    # root moves across an edge of weight w: size[u] nodes
    # get
    # w closer, the other n - size[u] get w farther
    for u in order[1:]:
```

```
        shift = up[u] * (n - 2 * size[u])
        ans[u] = ans[parent[u]] + shift
    return ans
```

```
# ---- main question: tests ----
def test_main_question():
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
    assert distance_sums(6, edges) == want


# ============================================================
# FOLLOW-UP 1: Find the node with the smallest distance
# ============================================================

# ---- follow-up 1: function ----
# Answer: The answer array above gives it directly: argmin
# of ans. Equivalent: the centroid, found by walking
# toward the heavier subtree.
def best_center(n, edges):
    # the node with the smallest distance sum
    sums = distance_sums(n, edges)
    return sums.index(min(sums))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    assert best_center(6, edges) == 2
    assert distance_sums(1, []) == [0]
    # weighted: the path 0 -(2)- 1 -(3)- 2
    w = [[0, 1, 2], [1, 2, 3]]
    assert distance_sums(3, w) == [7, 5, 8]


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Maximum distance (eccentricity) from each
# ============================================================

# ---- follow-up 3: function ----
# Answer: The farthest node from any node is one end of a
# diameter; compute distances from both diameter ends and
# take the maximum per node. O(n).
def eccentricities(n, edges):
```

```
    # the farthest node from anywhere is an end of a
    # diameter
    adj = build(n, edges)
    d0 = distances_from(adj, 0)
    a = max(d0, key=d0.get)
    da = distances_from(adj, a)
    b = max(da, key=da.get)
    db = distances_from(adj, b)
    return [max(da[v], db[v]) for v in range(n)]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    # eccentricity: the farthest node from each node
    assert eccentricities(6, edges) == [2, 3, 2, 3, 3, 3]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]]) -> list[int]:
```

Sample call

```
solution(1, [])
```

Test cases

Input	Expected
`(1, [])`	`[0]`
`(4, [[0, 1], [1, 2], [1, 3]])`	`[5, 3, 5, 5]`

ALSO REPORTED · 1 more

Tree Distance Sum (Re-root DP) | 1Point3Acres | onsite-coding | asked 2026-01-19 | Single report

https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

The interviewer explicitly liked the "baseline first, optimize later" structure — name the O(n²) and move on within the first ~10 minutes.
Two-pass technique to memorize:
Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].
Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].
Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

Preparation: Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.
Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

STUDY PATH: question 228 of 288 | topic 31 of 43: Tree DP: build a parent order once, then add up from the leaves | 5 of 5

Template fit: exactly the same two sweeps (this is the re-rooting version of the question) | asked 274 days ago | frequency Single report

HARD | IN APP LC 834 | 1Point3Acres | onsite-coding | 228 of 288

Tree Distance Sum (Re-root DP)

ASKED 2026-01-19 | 274 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 144/288

ABOUT THIS QUESTION

Topic: Tree DP: build a parent order once, then add up from the leaves

Optimal answer from: your app's interview approach (LeetCode #834)

LeetCode match: LC 834 Sum of Distances in Tree

Frequency: Single report | Last asked: 2026-01-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = [0] * n
    for s in range(n):
        dist = bfs(adj, s)
        ans[s] = sum(dist)
    return ans
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n BFS runs of O(n) each, O(n^2). Moving the root across one edge changes every distance by exactly 1, so two passes over the tree give all answers in O(n).

OPTIMAL SOLUTION · Two passes: subtree sizes, then re-root

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the re-root formula correct?

Compare the distance from every node w to p and to its child v. If w is inside v's subtree the path to p goes through v, so dist(w, p) = dist(w, v) + 1. If w is outside it, dist(w, v) = dist(w, p) + 1. There are size[v] nodes of the first kind and n - size[v] of the second, so ans[v] = ans[p] - size[v] + (n - size[v]).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Tree Distance Sum (Re-root DP)
#
# Onsite coding round 1: given an undirected tree on N
# nodes, return for every node the sum of distances to all
# other nodes. Equivalent to LC 834. Interviewer expects
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the re-root formula correct?
#
# INTERVIEW NOTES
# Running a BFS from every node costs O(n^2). Notice that
# moving the root from a parent p to its child v changes
# every distance by exactly one: the nodes inside v's
# subtree get closer by 1, all the others get farther by
# 1. So compute the answer for one root, then derive each
# child's answer from its parent's in O(1) using the
# subtree sizes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def build(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
# order compute size[u] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer.
```

```
def tree_distance_sums(n, edges):
    if n == 0:
        return []
    adj = build(n, edges)
    # order: parents before children (no recursion)
    parent, order = [-1] * n, [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    # size[u]: nodes in u's subtree
    # inside[u]: distances from u to its own subtree
    size, inside = [1] * n, [0] * n
    for u in reversed(order):
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]
    # the root's answer is already complete
    ans = [0] * n
    ans[0] = inside[0]
    # moving the root from p to child u: the size[u] nodes
    # below get 1 closer, the other n - size[u] get 1
    # farther
    for u in order[1:]:
        ans[u] = ans[parent[u]] + n - 2 * size[u]
    return ans

# ---- main question: tests ----
def test_main_question():
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
    assert tree_distance_sums(6, edges) == want
    assert tree_distance_sums(1, []) == [0]
    assert tree_distance_sums(0, []) == []
    # a chain of 100000 nodes: no recursion trouble
    n = 100000
    chain = [(i, i - 1) for i in range(1, n)]
    first = tree_distance_sums(n, chain)[0]
    assert first == n * (n - 1) // 2


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.
Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 32 OF 43

Counting or reaching an amount: the knapsack loop

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

5
MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Counting or reaching an amount: the knapsack loop | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	229. Maximize Shelf Space (Unbounded Knapsack)	ColorStack	2025-10-17
M	230. Coin Change	Exponent	no date
M	231. Partition an array into two sub-arrays with equal sum.	Exponent	no date
M	232. Explain how to find a target sum in an array.	Exponent	no date
M	233. Reverse Coin Change	HackTheRounds	no date

CODE TEMPLATE | GROUP 32 OF 43 | 5 QUESTIONS, ONE PATTERN

Counting or reaching an amount: the knapsack loop

The one idea these questions share

Make an array indexed by amount. For each item, sweep the amounts and let ways[t] depend on ways[t - item]. Sweep upwards when an item can be reused, downwards when each item can be used once. Swap += for min(...) + 1 (fewest items) or or (reachable).

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def count_ways(coins, target):
    ways = [1] + [0] * target              # CHANGE 1: meaning of ways[0] and the start values
    for c in coins:                        # items outside = combinations; amounts outside = ordered
        for t in range(c, target + 1):     # CHANGE 2: upwards = reuse, downwards = once each
            ways[t] += ways[t - c]         # CHANGE 3: += count, min(...)+1 fewest, or |= reachable
    return ways[target]
```

2. Helper functions that usually get written next to it

```
def min_coins(coins, amount):
    INF = amount + 1
    best = [0] + [INF] * amount
    for t in range(1, amount + 1):
        for c in coins:
            if c <= t:
                best[t] = min(best[t], best[t - c] + 1)
    return best[amount] if best[amount] != INF else -1


def can_split_equal(nums):
    total = sum(nums)
    if total % 2:
        return False
    reach = 1                              # bit k set = sum k reachable
    for x in nums:
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
229	Maximize Shelf Space (Unbounded Knapsack)	reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity
230	Coin Change	fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity
231	Partition an array into two sub-arrays with equal sum.	subset sum to total / 2, written with a bitset: reach |= reach << x
232	Explain how to find a target sum in an array.	turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)
233	Reverse Coin Change	given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 229 of 288 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 1 of 5

Template fit: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity | asked 368 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 229 of 288

Maximize Shelf Space (Unbounded Knapsack)

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 48.5 | 30 DAYS 0 | PRIORITY 83/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #322 Coin Change

LeetCode match: LC 322 Coin Change / unbounded subset sum (related)

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

A shelf has a fixed integer capacity capacity . You have boxes of various integer sizes given in sizes , and you may use any number of each box, including duplicates (each size is available infinitely). Choose boxes so their total size does not exceed capacity , maximizing the total size used. Return that maximum total.

Function signature

```
def max_shelf_space(capacity: int, sizes: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	capacity = 10, sizes = [3, 5, 7]
Output	10
Why	3 + 7 = 10 exactly fills the shelf.

Example 2	
Input	capacity = 11, sizes = [5, 3]
Output	11
Why	5 + 3 + 3 = 11.

Example 3	
Input	capacity = 7, sizes = [4, 5]
Output	5
Why	Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.

CONSTRAINTS

- 0 <= capacity <= 10**4 .
- 1 <= len(sizes) <= 100 , each size is a positive integer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_shelf(capacity, sizes):
    best = 0
    for s in sizes:
        if s <= capacity:
            best = max(
                best,
                s
                + brute_shelf(
                    capacity - s, sizes
                ),
            )
    return best
```

TIME	SPACE
exponential	O(capacity)

WHY IT IS SLOW

The same remaining capacity is solved again and again.

OPTIMAL SOLUTION · Reachable totals DP

TIME	SPACE
O(capacity * boxes)	O(capacity)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Each box size is only available once.

That is a 0/1 knapsack (subset sum). Process the boxes one at a time and iterate the capacity downward so a box is not used twice: reachable[c] |= reachable[c - s].

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximize Shelf Space (Unbounded Knapsack)
#
# A shelf has a fixed integer capacity capacity . You have
# boxes of various integer sizes given in sizes , and you
# may use any number of each box, including duplicates
# (each size is available infinitely). Choose boxes so
# their total size does not exceed capacity , maximizing
# the total size used. Return that maximum total. Function
# signature: def max_shelf_space(capacity: int, sizes:
# list[int]) -> int: ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Each box size is only available once.
#
# INTERVIEW NOTES
# Which totals up to the capacity can be built from
# unlimited boxes of the given sizes? reachable[c] is true
# if some size s fits and reachable[c - s] is true. The
```

```
# answer is the largest reachable total, a one-dimensional
# unbounded knapsack where the value of a box equals its
# size.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Reachable totals DP
# Idea: A total is reachable if removing one box leaves a
# reachable total.
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
    for c in range(1, capacity + 1):
        reachable[c] = any(
            s <= c and reachable[c - s] for s in sizes
        )
    return max(
        c for c in range(capacity + 1) if reachable[c]
    )

# ---- main question: tests ----
def test_main_question():
    assert max_shelf_space(10, [3, 5, 7]) == 10
    assert max_shelf_space(11, [5, 3]) == 11
    assert max_shelf_space(7, [4, 5]) == 5
    assert max_shelf_space(3, [5]) == 0


# ============================================================
# FOLLOW-UP 1: Each box size is only available once.
# ============================================================

# ---- follow-up 1: function ----
# Answer: That is a 0/1 knapsack (subset sum). Process the
# boxes one at a time and iterate the capacity downward so
# a box is not used twice: reachable[c] |= reachable[c -
# s].
def max_shelf_space_once(capacity, sizes):
    reachable = [True] + [False] * capacity
    for s in sizes:
        for c in range(capacity, s - 1, -1):
            reachable[c] = (
                reachable[c] or reachable[c - s]
            )
    return max(
        c for c in range(capacity + 1) if reachable[c]
    )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 230 of 288 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 2 of 5

Template fit: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity | no date

MEDIUM | IN APP LC 322 | Exponent | 230 of 288

Coin Change

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 48.5 | 30 DAYS 0 | PRIORITY 82/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: your app's interview approach (LeetCode #322)

LeetCode match: LC 322 Coin Change

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +27 interviews

Original: https://www.aced.io/questions/1142/coin-change

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each

recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

EXAMPLES

Example 1	
Input	`[1, 2, 5], 11`
Output	`3`
Why	`5 + 5 + 1 = 11: 3 coins.`

QUESTION

Example 2	
Input	`[2], 3`
Output	`-1`
Why	`A coin of 2 can never make 3.`

Example 3	
Input	`[1], 0`
Output	`0`
Why	`Zero needs no coins.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fewest(a):
    if a == 0:
        return 0
    best = INF
    for c in coins:
        if c <= a:
            best = min(best, 1 + fewest(a - c))
    return best
```

TIME	SPACE
O(coins^amount)	O(amount)

WHY IT IS SLOW

The same amount is recomputed through many coin orders, exponential. Filling fewest[a] for a from 0 upward is O(amount * coins).

OPTIMAL SOLUTION · Bottom-up DP over amounts

TIME	SPACE
O(amount * coins)	O(amount)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the number of combinations that make the amount (LeetCode 518).

Loop over coins in the outer loop and amounts in the inner loop, adding ways[a - c]; the order of the loops makes combinations (not permutations) count once.

FOLLOW-UP 2 Return which coins are used.

Remember the last coin chosen for each amount (a parent array) and walk back from the target.

FOLLOW-UP 3 Does greedy work?

Only for special coin systems (like US coins). For general coins it can be wrong (coins 1, 3, 4, amount 6). Mention the counterexample, then give the DP.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Coin Change
#
# You are given an integer array coins representing
# different coin denominations and an integer amount
# representing a total amount of money. Write a function
# coinChange that returns the fewest number of coins
# needed to make up that amount. If that amount cannot be
# made up by any combination of the coins, return -1. You
# may assume that you have an infinite number of each kind
# of coin. Examples PythonInput: coins = [1, 2, 5], amount
# = 11 Output: 3 Explanation: The minimum number of coins
# needed is 3 (11 = 5 + 5 + 1). Input: coins = [4, 5],
# amount = 7 Output: -1 Explanation: It is not possible to
# make up the amount 7 with coins of denominations 4 and
# 5.Solution 1: Brute-Force approach The naive approach to
# solving the coin change problem involves a recursive
# strategy to explore all possible combinations of coins
# to find the minimum number needed to make up the target
# amount.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the number of combinations that make the amount
# (LeetCode 518).
# 2. Return which coins are used.
# 3. Does greedy work?
#
# INTERVIEW NOTES
# The fewest coins for amount a is one coin plus the
# fewest for a - c, for the best coin c; that is a table
# over amounts built from 0 upward. Greedy (always take
# the biggest coin) is wrong for general coin systems,
# which is the classic trap: with coins 1, 3, 4 the amount
# 6 needs two coins (3 + 3), not three (4 + 1 + 1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
INF = float("inf")


# ---- main question: main function ----
# BASE QUESTION (optimal): Bottom-up DP over amounts
# Idea: best[0] = 0, and for a from 1 to amount, best[a] =
# 1 + min(best[a - c]) over coins c <= a (infinity if none
# works). The answer is best[amount] or -1. Each amount is
# solved once.
def coin_change(coins, amount):
    # best[a] = fewest coins that make exactly a
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    return -1 if best[amount] == INF else best[amount]

# ---- main question: tests ----
def test_main_question():
    assert coin_change([1, 2, 5], 11) == 3
    assert coin_change([2], 3) == -1
    assert coin_change([1], 0) == 0


# ============================================================
# FOLLOW-UP 1: Count the number of combinations that make
# ============================================================

# ---- follow-up 1: function ----
# Answer: Loop over coins in the outer loop and amounts in
# the inner loop, adding ways[a - c]; the order of the
# loops makes combinations (not permutations) count once.
def count_ways(coins, amount):
    # follow-up (LeetCode 518): how many COMBINATIONS.
    # Coins
    # in the outer loop means each combination is counted
    # once, not once per order.
    ways = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
    return ways[amount]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_ways([1, 2, 5], 5) == 4
    assert count_ways([2], 3) == 0
    assert count_ways([10], 10) == 1
    assert count_ways([1, 2], 0) == 1


# ============================================================
# FOLLOW-UP 2: Return which coins are used.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Remember the last coin chosen for each amount (a
# parent array) and walk back from the target.
def coins_used(coins, amount):
    # follow-up: which coins. Remember the last coin
    # chosen
    # for every amount, then walk back from the target.
    best = [0] + [INF] * amount
    last = [0] * (amount + 1)
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
                last[a] = c
    if best[amount] == INF:
        return None
    used, a = [], amount
    while a:
        used.append(last[a])
```

```
        a -= last[a]
    return sorted(used, reverse=True)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert coins_used([1, 2, 5], 11) == [5, 5, 1]
    assert coins_used([2], 3) is None


# =============================================================
# FOLLOW-UP 3: Does greedy work?
# =============================================================
```

```
# ---- follow-up 3: function ----
# Answer: Only for special coin systems (like US coins).
# For general coins it can be wrong (coins 1, 3, 4, amount
# 6). Mention the counterexample, then give the DP.
def greedy(coins, amount):
    # follow-up: does greedy work? Only for special coin
    # systems. Take the biggest coin that still fits.
    used = []
    for c in sorted(coins, reverse=True):
        while amount >= c:
            amount -= c
            used.append(c)
    return used if amount == 0 else None

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # greedy takes 4 + 1 + 1 (3 coins), the best is 3 + 3
    assert greedy([1, 3, 4], 6) == [4, 1, 1]
    assert coin_change([1, 3, 4], 6) == 2
    assert sorted(coins_used([1, 3, 4], 6)) == [3, 3]
    for _ in range(300):
        size = random.randint(1, 3)
        coins = random.sample(range(1, 9), size)
        amount = random.randint(0, 25)
        used = coins_used(coins, amount)
        if used is not None:
            assert sum(used) == amount


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples

OFFICIAL PAGE TEXT

```
PythonInput: coins = [1, 2, 5], amount = 11
Output: 3
Explanation: The minimum number of coins needed is 3 (
11 = 5 + 5 + 1).

Input: coins = [4, 5], amount = 7
Output: -1
Explanation: It is not possible to make up the amount
7 with coins of denominations 4 and 5.
```

Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the minimum number of coins found or -1 if it is not possible to make the target amount with the given coins.

```
PseudocodeFUNCTION coin_change_naive(coins, amount):
    IF amount == 0:
        RETURN 0  // Base case: No coins are needed to
 make amount 0

    IF amount < 0:
        RETURN infinity  // Amount is negative, path i
s not feasible

    min_coins = infinity

    FOR EACH coin IN coins:
        result = coin_change_naive(coins, amount - coi
n)  // Recursively solve for the remaining amount
        IF result != infinity:
            min_coins = MIN(min_coins, result + 1)  //
 Update min_coins with the minimum value

    RETURN min_coins IF min_coins != infinity ELSE -1
 // Return the minimum number of coins or -1 if no sol
ution exists
```

OFFICIAL PAGE TEXT

Time Complexity: The solution has a time complexity of O(m^n), where n is the amount and m is the number of coin denominations. The exponential time complexity arises from the recursive exploration of all possible combinations of coins. Each recursive call generates multiple further calls, leading to an exponential growth in the number of computations required. The total number of recursive calls can be as large as mnm^nmn due to the need to try each coin denomination for each sub-amount. Space Complexity: The space complexity of the naive approach is O(n), where n is the target amount. The space complexity is determined by the maximum depth of the recursion stack. In the worst case, the recursion depth can be as deep as the target amount n (when using coins of value 1). Therefore, the space required for the call stack is proportional to the target amount. As you can see, this solution does not perform very well. Let's take a look at a better solution. Solution 2: Dynamic programming approach Our solution employs a dynamic programming approach to determine the fewest number of coins needed to make up a given amount. We use a DP array, where each index represents a specific amount, and its value represents the minimum number of coins required to achieve that amount. The array is initialized with a value greater than the maximum possible number of coins

needed (infinity), except for the base case where dp[0] is set to 0, indicating that zero coins are needed to make amount 0. For each amount from 1 to the target amount, we iterate through each coin denomination and update the DP array. If the current amount minus the coin value is non-negative and the previous amount can be formed, we update the current amount’s value to be the minimum of its current value or one more than the value of the previous amount. This ensures that we are considering the fewest coins needed by checking each possible coin combination. After processing all amounts, if the value for the target amount is still set to infinity, it means it's not possible to form the target amount with the given coins, and we return -1. Otherwise, we return the value at dp[amount], which represents the minimum number of coins needed. Time Complexity: The solution has a time complexity of O(n * m), where n is the amount and m is the number of coin denominations. This is because we iterate over each amount from 1 to the target amount, and for each amount, we iterate over all coin denominations to update the DP array. Space Complexity: The solution uses O(n) space for the DP array, where n is the target amount. The array stores the minimum number of coins required to make each amount up to the target amount. This space complexity is necessary to hold the results for

all subproblems up to the target amount.No answers contributed by the community yet.

STUDY PATH: question 231 of 288 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 3 of 5

Template fit: subset sum to total / 2, written with a bitset: reach |= reach << x | no date

MEDIUM | IN APP LC 416 | Exponent | 231 of 288

Partition an array into two sub-arrays with equal sum.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 86/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: your app's interview approach (LeetCode #416)

LeetCode match: LC 416 Partition Equal Subset Sum

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/969/partition-array-equal-sum

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5] Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum. Input: [2, 7, 3, 1] Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

EXAMPLES

Example 1	
Input	`[1, 5, 11, 5]`
Output	`true`
Why	`[1, 5, 5] and [11] both add up to 11.`

Example 2	
Input	`[1, 2, 3, 5]`
Output	`false`
Why	`The total is 11, which is odd.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    total = sum(nums)
    if total % 2:
        return False
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            if sum(subset) == total // 2:
                return True
    return False
```

BRUTE FORCE

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n subsets. Subset-sum as a reachability table (or a bitset shifted by each number) is O(n * sum).

OPTIMAL SOLUTION · Subset-sum as a bitset

TIME	SPACE
O(n * sum / 64) with big-integer shifts (O(n * sum) as a boolean table)	O(sum)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the two subsets.

Keep, for each reachable sum, the list of indices (or a parent pointer) that first reached it; read off the subset for total / 2 and give the rest to the other side.

FOLLOW-UP 2 Minimise the difference between the two parts (LeetCode 1049).

Compute all reachable subset sums with the same bitset and choose the reachable s closest to total / 2: the answer is total - 2s.

FOLLOW-UP 3 The parts must be contiguous sub-arrays (a split point).

Then only one split can work: compute prefix sums and check whether some prefix equals total / 2. O(n), O(1) space; no DP is needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Partition an array into two sub-arrays
# with equal sum.
#
# Given an array of integers, determine if it can be
# partitioned into two subsets such that the sum of the
# elements in each subset is equal. A partition is
# considered valid if it divides the array into two
# subsets where both subsets have the same sum. Examples
# Input: [2, 4, 11, 5] Output: True Explanation: The array
# can be partitioned into two subsets [2, 4, 5] and [11]
# with equal sum. Input: [2, 7, 3, 1] Output: False
```

```
# Explanation: The array cannot be partitioned into two
# subsets with equal sums.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the two subsets.
# 2. Minimise the difference between the two parts
# (LeetCode 1049).
# 3. The parts must be contiguous sub-arrays (a split
# point).
#
# INTERVIEW NOTES
# Two equal parts mean each sums to total / 2, so if the
# total is odd the answer is no. Otherwise the question is
# whether some subset reaches total / 2, which is subset-
# sum. A bitset where bit s says 'sum s is reachable'
# updates with one shift and one OR per number.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Subset-sum as a bitset
# Idea: reach is an integer whose bit s is set when some
# subset sums to s; start with bit 0. For each number x do
# reach |= reach << x. The answer is whether bit total / 2
```

```
# is set.
def can_partition(nums):
    # reachable subset sums as the bits of one integer
    total = sum(nums)
    if total % 2:
        return False
    reach = 1
    for x in nums:
        # every old sum s also gives s + x
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)

# ---- main question: tests ----
def test_main_question():
    # 2 + 4 + 5 = 11 = the other part
    assert can_partition([2, 4, 11, 5]) is True
    assert can_partition([1, 2, 5]) is False
    assert can_partition([]) is True
    assert can_partition([1]) is False


# ============================================================
# FOLLOW-UP 1: Return the two subsets.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Keep, for each reachable sum, the list of
# indices (or a parent pointer) that first reached it;
# read off the subset for total / 2 and give the rest to
# the other side.
def partition_subsets(nums):
    # also return the two subsets: remember which item
    # first
    # reached each sum
    total = sum(nums)
```

```
    if total % 2:
        return None
    half = total // 2
    first = {0: -1}
    for i, x in enumerate(nums):
        for s in list(first):
            if s + x <= half and s + x not in first:
                first[s + x] = i
    if half not in first:
        return None
```

```
    chosen, s = [], half
    while s:
        i = first[s]
        chosen.append(i)
        s -= nums[i]
    left = sorted(chosen)
    rest = [i for i in range(len(nums))
            if i not in chosen]
    return ([nums[i] for i in left],
            [nums[i] for i in rest])

# ---- follow-up 1: tests ----
def test_follow_up_1():
    left, right = partition_subsets([1, 5, 11, 5])
    assert sum(left) == sum(right) == 11
    assert sorted(left + right) == [1, 5, 5, 11]
    assert partition_subsets([1, 2, 5]) is None


# ============================================================
# FOLLOW-UP 2: Minimise the difference between the two
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute all reachable subset sums with the same
# bitset and choose the reachable s closest to total / 2:
# the answer is total - 2s.
def min_difference(nums):
    # LeetCode 1049: make the two sums as close as
    # possible
    total = sum(nums)
    reach = 1
    for x in nums:
        reach |= reach << x
    sums = range(total // 2 + 1)
    best = max(s for s in sums if reach >> s & 1)
    return total - 2 * best

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_difference([1, 6, 11, 5]) == 1
    assert min_difference([2, 4, 11, 5]) == 0


# ============================================================
```

```
# FOLLOW-UP 3: The parts must be contiguous sub-arrays (a
# ============================================================

# ---- follow-up 3: function ----
# Answer: Then only one split can work: compute prefix
# sums and check whether some prefix equals total / 2.
# O(n), O(1) space; no DP is needed.
def can_split_contiguous(nums):
    # the parts must be a prefix and a suffix: no DP
    # needed
    total, prefix = sum(nums), 0
    if total % 2:
        return False
    for x in nums:
        prefix += x
        if prefix * 2 == total:
```

```
            return True
    return False

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert can_split_contiguous([1, 2, 3, 6])
    assert not can_split_contiguous([1, 5, 11, 5])


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples

```
Input: [2, 4, 11, 5]
Output: True
Explanation: The array can be partitioned into two sub
sets [2, 4, 5] and [11] with equal sum.

Input: [2, 7, 3, 1]
Output: False
Explanation: The array cannot be partitioned into two
subsets with equal sums.
```

OFFICIAL PAGE TEXT

Our solution processes the input array by first calculating the total sum of the array elements and then determining if it can be partitioned into two subsets with equal sums. The core idea is based on dynamic programming. Calculate Total Sum: We start by computing the total sum of the array elements. If the total sum is odd, it is immediately impossible to partition it into two subsets with equal sums, so we return false. Define Target Sum: If the total sum is even, we calculate the target sum for each subset, which is half of the total sum. Dynamic Programming Array: We use a boolean array dp where dp[i] indicates whether a subset sum of i is achievable. We initialize dp[0] to true because a subset sum of 0 is always achievable (by taking no elements). Iterate Through the Array: For each number in the array, we update the dp array in reverse order (from target down to the current number). This reverse iteration ensures that each number is only used once in the current update. If a subset sum i - num was achievable, then adding the current number num makes dp[i] achievable. We update dp[i] to true accordingly. Check Final Result: After processing all numbers, if dp[target] is true, it means there is a subset with the required sum equal to target, indicating that the array can be partitioned into two subsets with equal sums. Otherwise, we return false. The use of a boolean array ensures that we efficiently keep

track of achievable subset sums. Time Complexity: The solution has a time complexity of O(n * target), where n is the number of elements in the array and target is half of the total sum. This is because we iterate through the array and update the dp array for each element. Space Complexity: The solution uses O(target) space for the dp array, where target is half of the total sum. This space is needed to store the achievable subset sums.No answers contributed by the community yet.

STUDY PATH: question 232 of 288 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 4 of 5

Template fit: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards) | no date

MEDIUM | IN APP LC 494 | Exponent | 232 of 288

Explain how to find a target sum in an array.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE | ALL TIME 41.1 | 30 DAYS 0 | PRIORITY 102/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: your app's interview approach (LeetCode #494)

LeetCode match: LC 494 Target Sum

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | 3 interviews

Original: https://www.aced.io/questions/1439/find-target-sum-array

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of

symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

EXAMPLES

Example 1

Input	`[1, 1, 1, 1, 1], 3`
Output	`5`
Why	`Put + or - in front of every number; five ways to reach 3 with five 1s.`

QUESTION

Example 2

Input	`[1], 1`
Output	`1`
Why	`+1 is the only way.`

Example 3

Input	`[1, 2], 5`
Output	`-1`
Why	`No sign choice gives 5.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def go(i, total):
    if i == len(nums):
        return 1 if total == S else 0
    plus = go(i + 1, total + nums[i])
    minus = go(i + 1, total - nums[i])
    return plus + minus
answer = go(0, 0)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two branches per number: O(2^n). The numbers with a + sign form a subset with sum (total + S) / 2, so a subset-sum counting table is O(n * sum).

OPTIMAL SOLUTION · Subset-sum counting: choose P with sum (total + S) / 2

TIME	SPACE
O(n * sum)	O(sum)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The question was really 'find two numbers that add up to the target' (Two Sum).

One pass with a dictionary from value to index: for each x look for target - x among earlier values. O(n) time, O(n) space; with a sorted array use two pointers in O(1) space.

FOLLOW-UP 2 Only the existence of a subset with the target sum.

The same subset-sum table with booleans (or a bitset: reach |= reach << x) answers existence in O(n * sum / word size).

FOLLOW-UP 3 Negative numbers or a huge sum.

With negatives shift all values to be non-negative, or use a dictionary from reachable sum to count (memoised DFS on (index, sum)); with a huge sum the dictionary is only as large as the number of distinct reachable sums. **Cost:** O(n * R) with a dictionary over R reachable sums; shifting for negatives adds O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Explain how to find a target sum in an
# array.
#
# Determine the Number of Ways to Assign Symbols to Reach
# a Target Sum. You are given a list of non-negative
# integers, a1, a2, ..., an, and a target sum S. Your task
# is to assign each integer in the list a symbol + or -
# and calculate how many different ways there are to
# arrange these symbols such that the sum of the integers
# equals S after applying the assigned symbols. For
# example, if you have the list [1, 1, 1, 1, 1] and the
# target sum 3, one way to reach the target sum is to
# assign + to three 1s and - to two 1s, like so: +1 +1 +1
# +1 -1. Another way could be +1 -1 +1 +1 +1. You need to
# return the total number of unique arrangements that meet
# the target sum S.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The question was really 'find two numbers that add up
# to the target' (Two Sum).
# 2. Only the existence of a subset with the target sum.
# 3. Negative numbers or a huge sum.
#
# INTERVIEW NOTES
# Put a + in front of the numbers in a set P and a - in
# front of the rest N. Then sum(P) - sum(N) = S and sum(P)
# + sum(N) = total, so sum(P) = (total + S) / 2. The
# problem becomes 'count the subsets with this sum', a
# one-dimensional knapsack count. If (total + S) is odd or
# negative the answer is 0.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Subset-sum counting: choose P
# with sum (total + S) / 2
# Idea: Let need = (total + S) / 2 (return 0 if it is not
# a non-negative integer). ways[s] counts the subsets with
# sum s; for each number x update ways from high to low:
# ways[s] += ways[s - x]. The answer is ways[need].
def count_signs(nums, target):
    # +/- signs reaching `target` equal a subset with sum
    # (total + target) / 2 that gets the plus signs. So
    # the
    # question becomes: how many subsets have that sum?
    total = sum(nums)
    if abs(target) > total or (total + target) % 2:
        return -1
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
        # go downward so every number is used at most once
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]

# ---- main question: tests ----
def test_main_question():
    assert count_signs([1, 1, 1, 1, 1], 3) == 5
    assert count_signs([1, 2, 1], 2) == 2
    assert count_signs([1, 2, 7, 1, 5, 3], 8) == -1
    assert count_signs([1], 5) == -1
    assert count_signs([0, 0, 1], 1) == 4
    assert count_signs([], 0) == 1


# ============================================================
# FOLLOW-UP 1: The question was really 'find two numbers
# ============================================================

# ---- follow-up 1: function ----
# Answer: One pass with a dictionary from value to index:
# for each x look for target - x among earlier values.
# O(n) time, O(n) space; with a sorted array use two
# pointers in O(1) space.
def two_sum(nums, target):
    # the question may have meant Two Sum: one pass, a
    # dictionary from value to index of earlier numbers
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return seen[target - x], i
        seen[x] = i
    return None


# Answer: One pass with a dictionary from value to index:
# for each x look for target - x among earlier values.
# O(n) time, O(n) space; with a sorted array use two
# pointers in O(1) space.
def two_sum_sorted(nums, target):
    # sorted input: two pointers, O(1) extra space
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        total = nums[lo] + nums[hi]
        if total == target:
            return lo, hi
        if total < target:
            lo += 1
        else:
            hi -= 1
    return None

# ============================================================
# FOLLOW-UP 2: Only the existence of a subset with the
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: The same subset-sum table with booleans (or a
# bitset: reach |= reach << x) answers existence in O(n *
# sum / word size).
def subset_exists(nums, goal):
    # follow-up: only whether a subset reaches the sum.
    # The
    # set of reachable sums is one integer used as a
    # bitset.
    reach = 1
    for x in nums:
        reach |= reach << x
    return bool(reach >> goal & 1)

# ============================================================
```

```
# FOLLOW-UP 3: Negative numbers or a huge sum.
# ============================================================

# ---- follow-up 3: function ----
# Answer: With negatives shift all values to be non-
# negative, or use a dictionary from reachable sum to
# count (memoised DFS on (index, sum)); with a huge sum
# the dictionary is only as large as the number of
# distinct reachable sums.
def count_signs_dict(nums, target):
    # follow-up: negative numbers or a huge sum. A
    # dictionary
    # from "reachable sum" to "number of ways" holds only
    # the
    # sums that really occur.
    seen = Counter({0: 1})
    for x in nums:
        nxt = Counter()
        for s, ways in seen.items():
            nxt[s + x] += ways
            nxt[s - x] += ways
        seen = nxt
    return seen[target]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert count_signs_dict([1, 1, 1, 1, 1], 3) == 5
    # negative numbers work with the dictionary version
    assert count_signs_dict([-1, 2, 1], 2) == 2
    assert subset_exists([3, 34, 4, 12, 5, 2], 9)
    assert not subset_exists([3, 34, 4], 8)
    assert two_sum([2, 7, 11, 15], 9) == (0, 1)
    assert two_sum([1, 2], 7) is None
    assert two_sum_sorted([1, 3, 4, 8], 11) == (1, 3)
    for _ in range(300):
        nums = [random.randint(0, 4)
                for _ in range(random.randint(0, 7))]
        target = random.randint(-8, 12)
        want = count_signs_dict(nums, target)
        got = count_signs(nums, target)
        assert got == want or (got == -1 and want == 0)


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples

```
nums = [1, 1, 1, 1, 1]
targetSum = 3
output: 5

nums = [1, 2, 1]
targetSum = 2
output: 2

nums = [1, 2, 7, 1, 5, 3]
targetSum = 8
output: -1 (Explanation: Since the total sum of the nu
mbers plus the target is odd, it's not possible to rea
ch the target sum.)
```

The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\text{nums} = [a_1, a_2, ..., a_n]nums=[a1,a2,...,an], we need to assign either a +++ or a --- to each number so that the total sum equals a target SSS.Let's denote the sum of the entire list as sum(nums)=totalSum\text{sum(nums)} = \text{totalSum}sum(nums)=totalSum.The problem can be rephrased as finding subsets of nums\text{nums}nums such that the difference between the sum of elements in the subset and the sum of elements not in the subset equals SSS.Formulating the Equation: Let PPP be the sum of elements with a +++ sign and NNN be the sum of elements with a --- sign. We need P-N=SP - N = SP-N=S. We also know that P+N=totalSumP + N = \text{totalSum}P+N=totalSum, where totalSum\text{totalSum}totalSum is the sum of all

OFFICIAL PAGE TEXT

elements in nums\text{nums}nums. Combining the Equations: Adding these two equations:P-N+P+N=S+totalSum2P=S+totalSumP - N + P + N = S + \text{totalSum}\\ 2P = S + \text{totalSum} P-N+P+N=S+totalSum2P=S+totalSumTherefore, P=S+totalSum2P = \frac{S + \text{totalSum}}{2}P=2S+totalSum.The problem now reduces to finding subsets of nums\text{nums}nums whose sum equals S+totalSum2\frac{S + \text{totalSum}}{2}2S+totalSum. Approach We define a recursive function find_subset that takes an index, the list of numbers, the target subset sum, and a memoization table (dp). The function recursively explores two possibilities at each step: Include the current number in the subset. Exclude the current number from the subset. The base cases for the recursion are: If the target subset sum (s) is zero, it means we found a valid subset, so we return 1. If we reach the end of the list without finding the subset sum, we return 0. If we have already computed the value for the current state (index and subset sum), we return the stored value from the memoization table to avoid redundant calculations. The changeSigns function calculates the total sum of the numbers and checks if it is possible to partition them to meet the required target sum S. If it is possible, it initializes the memoization table and

calls the find_subset function to compute the number of ways to achieve the target sum. Solution Time Complexity: The time complexity of this solution is O(n x (totalSum + S)) / 2, where n is the number of elements in the input list. This is because each state in the memoization table can be computed in constant time, and there are n x (totalSum + S) / 2 states to fill. Space Complexity: The space complexity of this solution is O(n x (totalSum + S) / 2) due to the memoization table used to store intermediate results. Additionally, the recursive call stack can go up to O(n) in the worst case, but this is dominated by the space required for the memoization table.Watch our mock Flipkart SWE (software engineering) interview. Neamah asks Rohan (Flipkart SWE) the Target Sum leetcode question.No answers contributed by the community yet.Related questionsFind a triplet in an array with a given sum.Given an array, find the two sum.Find quadruplets in an array with a given sum.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start

OFFICIAL PAGE TEXT

learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

STUDY PATH: question 233 of 288 | topic 32 of 43: Counting or reaching an amount: the knapsack loop | 5 of 5

Template fit: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild | no date

MEDIUM | HackTheRounds | Onsite | 233 of 288

Reverse Coin Change

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 114/288

ABOUT THIS QUESTION

Topic: Counting or reaching an amount: the knapsack loop

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #518 Coin Change II

LeetCode match: Inverse of LC 518

QUESTION

In the coin change counting problem, dp[i] is the number of ways to make amount i from a set of coins (dp[0] = 1). In the reverse problem you are given the dp array and must find a set of coins that produces it, or None if no coin set does.

Function signature

def coins_from_dp(dp: list) -> list | None

EXAMPLES

Example 1	
Input	`[1, 1, 2, 2, 3, 3]`
Output	`[1, 2]`
Why	`Coins 1 and 2 make 1, 1, 2, 2, 3, 3 ways for the amounts 0 to 5.`

Example 2	
Input	`[1, 0, 1, 0, 1]`
Output	`[2]`
Why	`Only the coin 2 can make 2 and 4 but not 1 and 3.`

Example 3	
Input	`[1, 2, 3]`
Output	`null`
Why	`No coin set can produce these counts.`

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(dp):
    n = len(dp) - 1
    for k in range(n + 1):
        values = range(1, n + 1)
        for coins in combinations(values, k):
            if ways_table(coins, n) == dp:
                return list(coins)
    return None
```

TIME	SPACE
O(2^n * n * coins)	O(n)

WHY IT IS SLOW

2^n candidate sets, each with an O(n * coins) table. Scanning amounts upward and adding a coin whenever the target has one more way than the table is O(n^2).

OPTIMAL SOLUTION · Scan upward and add a coin whenever the table falls short

TIME	SPACE
O(n * coins) = O(n^2) worst case	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array might not come from any set of coins.

Return None (or raise) when dp[0] != 1, when an amount needs more than one extra way, or when the rebuilt table differs from the input at the end; the code does all three checks.

FOLLOW-UP 2 Order of the coins matters (the dp counts permutations, not combinations).

That is a different recurrence: dp[a] = sum of dp[a - c] over coins. Inverting it is a linear-algebra style problem: at each amount the new coins explain the excess; ask which counting is used.

FOLLOW-UP 3 Each coin may be used at most once.

The forward recurrence iterates amounts downward. The inverse still scans upward adding a coin when the target exceeds the table, but the table update must use the 0/1 form.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Reverse Coin Change
#
# In the coin change counting problem, dp[i] is the number
# of ways to make amount i from a set of coins (dp[0] =
# 1). In the reverse problem you are given the dp array
# and must find a set of coins that produces it, or None
# if no coin set does.
#
# Signature:
#   def coins_from_dp(dp: list) -> list | None
#
# Examples:
# [1, 1, 2, 2, 3, 3]  ->  [1, 2]
# [1, 0, 1, 0, 1]  ->  [2]
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array might not come from any set of coins.
# 2. Order of the coins matters (the dp counts
# permutations, not combinations).
# 3. Each coin may be used at most once.
#
# INTERVIEW NOTES
# Think of building the ways table forward: adding a coin
# c changes dp[c] by exactly one (the single coin c). So
# scanning amounts upward, the first amount where the
# target has more ways than your table can only be
# explained by a new coin of exactly that value. Add it,
# rebuild, and continue; at the end compare with the
# target to detect an impossible input.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Scan upward and add a coin
# whenever the table falls short
# Idea: Keep the ways table for the coins found so far.
# FOLLOW-UP 1: The array might not come from any set of
# coins.
# Answer: Return None (or raise) when dp[0] != 1, when an
# amount needs more than one extra way, or when the
# rebuilt table differs from the input at the end; the
# code does all three checks.
def coins_from_dp(dp):
    # backward: rebuild the table with the coins found so
    # far. The first amount where the real dp is exactly
    # one
    # bigger must be a new coin (it adds the single way
    # "just that coin"). Anything else is impossible.
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(a, n + 1):
            cur[x] += cur[x - a]
    # the table rebuilt from the coins must equal the
    # input
    return coins if cur == list(dp) else None

# ---- main question: tests ----
def test_main_question():
    assert coins_from_dp([1, 1, 2, 2, 3]) == [1, 2]
    assert coins_from_dp([1, 1, 1, 2, 2, 2, 3]) == [1, 3]
    assert coins_from_dp([1]) == []
    assert coins_from_dp([]) is None
    # not a table: dp[0] must be 1
    assert coins_from_dp([2, 1]) is None
    # an amount that jumps by more than one new way
    assert coins_from_dp([1, 3]) is None
    # looks fine early, but the whole table does not match
    assert coins_from_dp([1, 1, 2, 2, 5]) is None
```

```
# ============================================================
# FOLLOW-UP 1: The array might not come from any set of
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Order of the coins matters (the dp counts
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is a different recurrence: dp[a] = sum of
# dp[a - c] over coins. Inverting it is a linear-algebra
# style problem: at each amount the new coins explain the
# excess; ask which counting is used.
def ordered_table(coins, n):
    # follow-up: the order of coins matters
    # (permutations):
    # dp[a] = sum of dp[a - c] over the coins
    dp = [1] + [0] * n
    for a in range(1, n + 1):
        dp[a] = sum(dp[a - c] for c in coins if c <= a)
    return dp


# Answer: That is a different recurrence: dp[a] = sum of
# dp[a - c] over coins. Inverting it is a linear-algebra
# style problem: at each amount the new coins explain the
# excess; ask which counting is used.
def coins_from_ordered(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    coins = []
    for a in range(1, n + 1):
        # what the known coins already explain at amount a
        known = sum(dp[a - c] for c in coins)
        if dp[a] == known + 1:
            coins.append(a)
        elif dp[a] != known:
            return None
```

```
    again = ordered_table(coins, n)
    return coins if again == list(dp) else None

# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    table = ordered_table([1, 2], 6)
    assert coins_from_ordered(table) == [1, 2]
    assert coins_from_ordered([1, 1, 2, 3, 5]) == [1, 2]


# ========================================================
# FOLLOW-UP 3: Each coin may be used at most once.
# ========================================================

# ---- follow-up 3: function ----
# Answer: The forward recurrence iterates amounts
# downward. The inverse still scans upward adding a coin
# when the target exceeds the table, but the table update
# must use the 0/1 form.
def once_table(coins, n):
    # follow-up: each coin used at most once; amounts are
    # visited downward in the forward step
    dp = [1] + [0] * n
    for c in coins:
        for a in range(n, c - 1, -1):
            dp[a] += dp[a - c]
    return dp


# Answer: The forward recurrence iterates amounts
# downward. The inverse still scans upward adding a coin
# when the target exceeds the table, but the table update
# must use the 0/1 form.
def coins_from_once(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(n, a - 1, -1):
            cur[x] += cur[x - a]
    return coins if cur == list(dp) else None

# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    table = once_table([1, 2, 3], 6)
    assert coins_from_once(table) == [1, 2, 3]
    for _ in range(300):
        size = random.randint(0, 4)
        coins = sorted(random.sample(range(1, 10), size))
        n = random.randint(0, 14)
        small = [c for c in coins if c <= n]
        table = ways_table(coins, n)
        assert coins_from_dp(table) == small
        table = ordered_table(coins, n)
        assert coins_from_ordered(table) == small
        table = once_table(coins, n)
        assert coins_from_once(table) == small
        junk = [1]
        junk += [random.randint(0, 3) for _ in range(n)]
        found = coins_from_dp(junk)
        if found is not None:
            assert ways_table(found, n) == junk


# ========================================================
```

```
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ========================================================
# HELPERS used by the code below
def ways_table(coins, n):
    # forward: dp[a] = number of combinations making a
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 33 OF 43

Prefix sums, Tier 2: running total plus a dict of earlier totals

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Prefix sums, Tier 2: running total plus a dict of earlier totals | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	234. Maximum-Sum Subarray With Equal Endpoints	ColorStack	2025-09-29
M	235. Maximum Subarray Sum With a Minimum Length Constraint	ColorStack	2025-08-19
M	236. Solve array-sum and city-community problems	PracHub	2026-02-20
M	237. Find the maximum subarray sum.	Exponent	no date
H	238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two e...	Exponent	no date

CODE TEMPLATE | GROUP 33 OF 43 | 5 QUESTIONS, ONE PATTERN

Prefix sums, Tier 2: running total plus a dict of earlier totals

The one idea these questions share

Keep a running total. At each step ask the dict about an earlier total: total - k (subarray sums to k), total % k (sum divisible by k), or the smallest earlier total (best subarray ending here). Store the first index, a count, or the minimum.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def count_sum_k(nums, k):
    seen, total, count = {0: 1}, 0, 0       # CHANGE 1: what seen stores (count, first index, minimum)
    for x in nums:
        total += x
        count += seen.get(total - k, 0)      # CHANGE 2: the earlier total you look for
        seen[total] = seen.get(total, 0) + 1 # CHANGE 3: what you remember about this total
    return count
```

2. Helper functions that usually get written next to it

```
def max_subarray(nums):
    best, smallest, total = float("-inf"), 0, 0
    for x in nums:
        total += x
        best = max(best, total - smallest)   # best subarray that ends here
        smallest = min(smallest, total)      # update after, so the subarray is never empty
    return best
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
234	Maximum-Sum Subarray With Equal Endpoints	per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total
235	Maximum Subarray Sum With a Minimum Length Constraint	the smallest earlier total is only allowed from min_len positions back, so update it one step late
236	Solve array-sum and city-community problems	count subarrays summing to k with the template as written; the second task is union-find over building cells
237	Find the maximum subarray sum.	the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)
238	Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.	key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 234 of 288 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 1 of 5

Template fit: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total | asked 386 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 234 of 288

Maximum-Sum Subarray With Equal Endpoints

ASKED 2025-09-29 | 386 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 66 | 30 DAYS 46.9 | PRIORITY 8/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K

LeetCode match: LC 560 Subarray Sum Equals K (related: equal endpoints, maximum sum)

Frequency: Single report | Last asked: 2025-09-29 | Roles: SWE Intern

QUESTION

Given an integer array nums , choose a (contiguous) subarray whose first and last elements are equal and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Function signature

def max_sum_equal_endpoints(nums: list[int]) -> list[int]:
    ...

Follow-up (as reported): Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

EXAMPLES

Example 1

Input	`nums = [1, 3, -2, 4, 1]`
Output	`[1, 3, -2, 4, 1]`
Why	The value 1 appears at indices 0 and 4; that whole window sums to 7, beating any single element.

Example 2

Input	`nums = [5, -10, 5]`
Output	`[5]`
Why	The window from the first 5 to the last 5 sums to 0, which is worse than taking a single 5 (sum 5).

QUESTION

Example 3

Input	`nums = [-3, -1, -2]`
Output	`[-1]`
Why	All values are distinct, so only single elements qualify; -1 is the largest.

CONSTRAINTS

- 1 <= len(nums) <= 10^5 .
- -10^4 <= nums[i] <= 10^4 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_equal_ends(nums):
    best = None
    for j in range(len(nums)):
        for i in range(j_+_1):
            if nums[i] == nums[j]:
                key = (
                    -sum(nums[i : j + 1]),
                    i,
                    j - i,
                )
                if best is None or key < best:
                    best = key
    return (
        nums[best[1] : best[1] + best[2] + 1]
        if best
        else:
            []
    )
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Each end index looks back at every earlier index with the same value.

OPTIMAL SOLUTION · Best earlier prefix per value

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best sum.

Drop the tie-breaking bookkeeping: the same sweep with max(prefix[j + 1] - smallest prefix of this value so far).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum-Sum Subarray With Equal Endpoints
#
# Given an integer array nums , choose a (contiguous)
# subarray whose first and last elements are equal and
# whose sum is as large as possible. A single element is a
# valid subarray (its first and last elements coincide).
# Return the chosen subarray itself (as a list). If
# several subarrays tie for the maximum sum, return the
# one with the smallest starting index; if they still tie,
# return the shortest. Function signature: def
# max_sum_equal_endpoints(nums: list[int]) -> list[int]:
# ... Follow-up (as reported): Achieve O(n): for each
# value, you only ever care about pairing the earliest
# occurrence (with the best prefix sum) against later
# occurrences.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best sum.
#
# INTERVIEW NOTES
# A subarray from i to j has equal endpoints when nums[i]
# == nums[j], and its sum is prefix[j + 1] - prefix[i].
# For a fixed j the best partner i is the occurrence of
# the same value with the smallest prefix[i]; keep that
# per value in a dictionary while sweeping j.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Best earlier prefix per value
# Idea: For each end the best start is the same value with
# the smallest prefix sum so far.
def max_sum_equal_endpoints(nums):
```

```
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    best_start, best = {}, None
    for j, x in enumerate(nums):
        if (
            x not in best_start
            or prefix[j] < prefix[best_start[x]]
        ):
            best_start[x] = j
        i = best_start[x]
        key = (-(prefix[j + 1] - prefix[i]), i, j - i)
        if best is None or key < best:
            best = key
    return (
        nums[best[1] : best[1] + best[2] + 1]
        if best
        else []
    )

# ---- main question: tests ----
def test_main_question():
    random.seed(33)
    assert max_sum_equal_endpoints(
        [1, 3, -2, 4, 1]
    ) == [1, 3, -2, 4, 1]
    assert max_sum_equal_endpoints([5, -10, 5]) == [5]
    assert max_sum_equal_endpoints([-3, -1, -2]) == [-1]
    assert max_sum_equal_endpoints([]) == []


# ============================================================
# FOLLOW-UP 1: Return only the best sum.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Drop the tie-breaking bookkeeping: the same
# sweep with max(prefix[j + 1] - smallest prefix of this
# value so far).
def max_equal_ends_sum(nums):
    smallest, best, total = {}, None, 0
    for x in nums:
        smallest[x] = min(smallest.get(x, total), total)
        total += x
        cand = total - smallest[x]
        best = cand if best is None else max(best, cand)
    return best


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 235 of 288 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 2 of 5

Template fit: the smallest earlier total is only allowed from min_len positions back, so update it one step late | asked 427 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 235 of 288

Maximum Subarray Sum With a Minimum Length Constraint

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 64.4 | 30 DAYS 41.1 | PRIORITY 18/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #53 Maximum Subarray

LeetCode match: LC 53 Maximum Subarray (related: minimum length)

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint"). Given an integer array nums and an integer min_len , find the contiguous subarray (containing at least min_len elements) that has the largest sum, and return that sum.

Function signature

```
def max_subarray_min_len(nums: list[int], min_len: int) -> int:
    """Return the maximum sum over all contiguous subarrays of length >= min_len."""
    ...
```

Follow-up (as reported): The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range [min_len, max_len] . Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

EXAMPLES

Example 1

Input	`nums = [1, -2, 3, 4, -1], min_len = 2`
Output	`7`
Why	`The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays of length >= 2.`

Example 2

Input	`nums = [-1, -2, -3], min_len = 2`
Output	`-3`
Why	`Every element is negative. The best length->=2 subarray is [-1, -2] with sum -3.`

QUESTION

Example 3

Input	`nums = [5, 4, -1, 7, 8], min_len = 1`
Output	`23`
Why	`With min_len = 1 this reduces to ordinary Kadane; the whole array sums to 23.`

CONSTRAINTS

- 1 <= min_len <= len(nums) <= 10^5
- -10^4 <= nums[i] <= 10^4
- The array is guaranteed to be non-empty and min_len never exceeds len(nums) , so a valid subarray always exists.
- Values may be negative; the answer can therefore be negative.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_min_len(nums, min_len):
    best = None
    for start in range(len(nums)):
        total = 0
        for end in range(start,_len(nums)):
            total += nums[end]
            if end - start + 1 >= min_len:
                best = (
                    total
                    if best is None
                    else:
                        max(best, total)
                )
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every start index extends to every end index at least min_len long, O(n^2) sums.

OPTIMAL SOLUTION · Prefix sums and a running minimum

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now the length must also be at most max_len. How does it change?

The allowed start prefixes now form a sliding window of indexes [end - max_len, end - min_len]. Keep the prefix indexes in a deque ordered by increasing prefix value; pop from the back while the new value is smaller, pop from the front when an index leaves the window. The minimum is the front. O(n).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Maximum Subarray Sum With a Minimum
# Length Constraint
#
# This is the classic "maximum subarray" problem with the
# extra constraint the report mentioned ("Max sub array
# question with a weird constraint"). Given an integer
# array nums and an integer min_len , find the contiguous
# subarray (containing at least min_len elements) that has
# the largest sum, and return that sum. Function
# signature: def max_subarray_min_len(nums: list[int],
# min_len: int) -> int: """Return the maximum sum over all
# contiguous subarrays of length >= min_len.""" ...
# Follow-up (as reported): The report said each question
# had a follow-up where a constraint was changed. Now
# bound the subarray length to the range [min_len,
# max_len] . Hint: maintain a sliding window of prefix-sum
# minima with a monotonic deque.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now the length must also be at most max_len. How does
# it change?
#
# INTERVIEW NOTES
# Kadane's algorithm cannot enforce a minimum length, so
# switch to prefix sums: the best subarray ending at i
# with length at least min_len is prefix[i + 1] minus the
# smallest prefix[j] with j <= i + 1 - min_len. Keep that
# minimum as a running value while sweeping.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Prefix sums and a running
# minimum
# Idea: Sweep the end of the subarray; the best start is
# the smallest prefix sum that leaves at least min_len
# elements.
def max_subarray_min_len(nums, min_len):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    best, smallest = None, float("inf")
    for end in range(min_len, len(nums) + 1):
        # the prefix that is allowed to start a valid
        # subarray
        smallest = min(smallest, prefix[end - min_len])
        cand = prefix[end] - smallest
        best = cand if best is None else max(best, cand)
    return best

# ---- main question: tests ----
def test_main_question():
    random.seed(1)
    assert (
        max_subarray_min_len([1, -2, 3, 4, -1], 2) == 7
    )
    assert max_subarray_min_len([-1, -2, -3], 2) == -3
    assert (
        max_subarray_min_len([5, 4, -1, 7, 8], 1) == 23
    )


# ============================================================
# FOLLOW-UP 1: Now the length must also be at most
# ============================================================

# ---- follow-up 1: function ----
# Answer: The allowed start prefixes now form a sliding
# window of indexes [end - max_len, end - min_len].
def max_subarray_len_range(nums, min_len, max_len):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    window, best = deque(), None
    for end in range(min_len, len(nums) + 1):
        # the start prefix that just became allowed
        new = end - min_len
        while (
            window and prefix[window[-1]] >= prefix[new]
        ):
            window.pop()
        window.append(new)
        # drop starts that would make the subarray too
        # long
        while window[0] < end - max_len:
            window.popleft()
        cand = prefix[end] - prefix[window[0]]
        best = cand if best is None else max(best, cand)
    return best

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        nums = [
            random.randint(-6, 6)
            for _ in range(random.randint(1, 9))
        ]
        m = random.randint(1, len(nums))
        hi = random.randint(m, len(nums))
        slow = max(
            sum(nums[i:j])
            for i in range(len(nums))
            for j in range(
                i + m, min(len(nums), i + hi) + 1
            )
```

```
        )
        assert (
            max_subarray_len_range(nums, m, hi) == slow
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 236 of 288 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 3 of 5

Template fit: count subarrays summing to k with the template as written; the second task is union-find over building cells | asked 242 days ago

MEDIUM | PracHub | Technical Screen | 236 of 288

★ TOP 75: rank 13 in the Medium and Hard priority list

★ Solve array-sum and city-community problems

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 66 | 30 DAYS 46.9 | PRIORITY 7/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K, LeetCode #200 Number of Islands

LeetCode match: LC 560 Subarray Sum Equals K + LC 200 Number of Islands (inferred from overview)

Posted on PracHub: 2026-02-20 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-array-sum-and-city-community-problems

QUESTION

Two tasks. (1) count_subarrays(nums, k): count the contiguous subarrays whose elements add up to k. (2) communities(grid): a grid has 1 for a building and 0 for empty land; buildings that touch up, down, left or right belong to the same community. Return the number of communities.

Function signature

def count_subarrays(nums: list, k: int) -> int

def communities(grid: list) -> int

EXAMPLES

Example 1	
Input	`[1, 1, 1], 2`
Output	`2`
Why	`[1, 1] appears twice (positions 0-1 and 1-2).`

Example 2	
Input	`[1, 2, 3], 3`
Output	`2`
Why	`[1, 2] and [3] both add up to 3.`

QUESTION

Example 3	
Input	`[[1, 1, 0], [0, 0, 1], [1, 0, 1]]`
Output	`3`
Why	`Top-left pair, the single building at the right of the middle row joined to the one below it, and the bottom-left: 3 communities.`

BRUTE FORCE · Task 1: try every subarray

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, K):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == K:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2). A dictionary of prefix sums (how many earlier prefixes equal current - K) is O(n) and also works on a stream and with negative numbers.

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

BRUTE FORCE · PSEUDO-CODE

```
label = {}
for i, cell in enumerate(buildings):
    label[cell] = i
changed = True
while changed:
    changed = False
    for cell in buildings:
        group = [cell] + neighbours(cell)
        best = min(label[x] for x in group)
        if best < label[cell]:
            label[cell] = best
            changed = True
answer = len(set(label.values()))
```

TIME	SPACE
O((m * n)^2) worst case	O(m * n)

WHY IT IS SLOW

Labels spread one step per round, so a long snake needs about m * n rounds of O(m * n): O((m * n)^2). Union-Find or one flood fill is near-linear.

OPTIMAL SOLUTION · Task 1: prefix sums in a dictionary

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: Union-Find over building cells

TIME	SPACE
O(m * n * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

FOLLOW-UP 2 Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

FOLLOW-UP 3 Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve array-sum and city-community
# problems
#
# Two tasks. (1) count_subarrays(nums, k): count the
# contiguous subarrays whose elements add up to k. (2)
# communities(grid): a grid has 1 for a building and 0 for
# empty land; buildings that touch up, down, left or right
# belong to the same community. Return the number of
# communities.
#
# Signature:
#   def count_subarrays(nums: list, k: int) -> int
#   def communities(grid: list) -> int
#
# Examples:
# [1, 1, 1], 2  ->  2
# [1, 2, 3], 3  ->  2
#
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers are all positive.
# 2. Longest subarray with sum K.
# 3. Buildings are added over time and you report the
# number of communities after each addition.
#
# INTERVIEW NOTES
# Subarray sums with negatives allowed cannot use a
# sliding window; use prefix sums: a subarray ending here
# sums to K exactly when an earlier prefix equals (current
# prefix - K), so a dictionary of prefix counts answers it
# in one pass and works for streams. Communities are
# connected components, so flood fill or union-find over
# the building cells.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# ---- main question: main function ----
# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def count_subarrays(nums, k):
    # prefix sums: a subarray ending here has sum k if
    # some
    # earlier prefix equals (prefix - k)
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def communities(grid):
    # connected groups of buildings (1) in a grid
    seen, count = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < len(grid)):
                        continue
                    if not (0 <= b < len(grid[0])):
                        continue
                    land = grid[a][b] == 1
                    if land and (a, b) not in seen:
                        seen.add((a, b))
```

```
                        stack.append((a, b))
    return count

# ---- main question: tests ----
```

```
def test_main_question():
    assert count_subarrays([1, 1, 1], 2) == 2
    assert count_subarrays([1, -1, 0], 0) == 3
    assert count_subarrays([], 0) == 0


# ============================================================
# FOLLOW-UP 1: The numbers are all positive.
# ============================================================

# ---- follow-up 1: function ----
# Answer: A sliding window works in O(1) memory: grow the
# right end, shrink the left while the sum exceeds K. With
# negatives the window is not monotone, which is why
# prefix sums are needed.
def count_positive(nums, k):
    # all numbers positive: a sliding window, O(1) memory
    count = left = total = 0
    for right, x in enumerate(nums):
        total += x
        while total > k and left <= right:
            total -= nums[left]
            left += 1
        if total == k:
            count += 1
    return count

# ============================================================
# FOLLOW-UP 2: Longest subarray with sum K.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Store the first index of each prefix sum instead
# of a count, and take the longest distance to prefix - K.
def longest_subarray(nums, k):
    # store the FIRST index of each prefix sum
    first, prefix, best = {0: -1}, 0, 0
    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, i - first[prefix - k])
        first.setdefault(prefix, i)
    return best

# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert longest_subarray([1, -1, 5, -2, 3], 3) == 4
    assert longest_subarray([1, 2], 9) == 0
    assert count_positive([1, 2, 3, 2, 1], 3) == 3
    grid = [[1, 1, 0], [0, 0, 1], [1, 0, 1]]
    assert communities(grid) == 3
    adds = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]


# ============================================================
# FOLLOW-UP 3: Buildings are added over time and you
# ============================================================

# ---- follow-up 3: function ----
# Answer: Union-Find with a counter: +1 for the new
# building, -1 for each union with a different
# neighbouring community (LeetCode 305).
def communities_online(rows, cols, additions):
    # buildings added one by one: count after each
    # (LeetCode 305)
    parent, count, out = {}, 0, []
```

```
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for r, c in additions:
        if (r, c) not in parent:
            parent[(r, c)] = (r, c)
            count += 1
            for dr, dc in STEPS:
                nb = (r + dr, c + dc)
                if nb in parent:
                    a, b = find((r, c)), find(nb)
                    if a != b:
                        parent[a] = b
                        # two communities became one
                        count -= 1
        out.append(count)
    return out

# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    adds = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]
    counts = communities_online(3, 3, adds)
    assert counts == [1, 1, 2, 2, 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], k: int) -> int:
```

Sample call

```
solution([1, 1, 1], 2)
```

Test cases

Input	Expected
([1, 1, 1], 2)	2
([1, 2, 3], 3)	2

STUDY PATH: question 237 of 288 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 4 of 5

Template fit: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea) | no date

MEDIUM | IN APP LC 53 | Exponent | 237 of 288

★ TOP 75: rank 20 in the Medium and Hard priority list

★ Find the maximum subarray sum.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 64.4 | 30 DAYS 41.1 | PRIORITY 17/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: your app's interview approach (LeetCode #53)

LeetCode match: LC 53 Maximum Subarray

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +1522 interviews

Original: https://www.aced.io/questions/973/maximum-subarray-sum

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

EXAMPLES

Example 1	
Input	`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`
Output	`6`
Why	`[4, -1, 2, 1] has the largest sum, 6.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single element.`

Example 3	
Input	`[5, 4, -1, 7, 8]`
Output	`23`
Why	`All numbers are positive: take them all.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    best = -INF
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2) subarrays. Kadane's algorithm keeps the best sum ending at each position (extend or restart) in O(n) time and O(1) space.

OPTIMAL SOLUTION · Kadane's algorithm

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subarray itself.

Remember where the current run started: when starting fresh at i set start = i, and when cur beats best record (start, i). Same single pass.

FOLLOW-UP 2 The array is circular (LeetCode 918).

The best circular subarray is either an ordinary one or the whole array minus the worst ordinary one (the minimum subarray): max(kadane max, total - kadane min), taking care of the all-negative case.

FOLLOW-UP 3 Divide and conquer instead of Kadane.

Split the array; the best subarray is entirely left, entirely right, or crosses the middle (best suffix of the left plus best prefix of the right). T(n) = 2T(n/2) + O(n) = O(n log n); useful when the data is split across machines.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the maximum subarray sum.
#
# Given an array of integers nums, write a function
# maxSubarraySum to find the maximum sum of a contiguous
# subarray within the array and return that maximum sum.
# The subarray must be contiguous, meaning that the
# elements must appear consecutively in the original
# array. Examples Input: nums = [2, 3, -2, 4] Output: 7
# Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7. Input:
# nums = [1, -1, -5, -4] Output: 1 Explanation: The
# maximum sum is 1, which is the single element with the
# highest value.Challenge 1Can you come up with a solution
# with a O(n) time complexity?Challenge 2Can you come up
# with a solution with a O(1) space complexity?
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subarray itself.
# 2. The array is circular (LeetCode 918).
# 3. Divide and conquer instead of Kadane.
#
# INTERVIEW NOTES
# The best subarray ending at position i either extends
# the best one ending at i - 1 or starts fresh at i; it
# extends only if the previous best is positive. So keep
# one number, the best sum ending here, and one answer:
# cur = max(x, cur + x), best = max(best, cur).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Kadane's algorithm
# Idea: cur is the best sum of a subarray ending at the
# current element: either the element alone or the element
# added to the previous cur. best is the maximum cur seen.
def max_subarray(nums):
```

```
    # Kadane: the best sum of a run ending here either
    # extends
    # the previous run or starts again at this element
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

# ---- main question: tests ----
def test_main_question():
    assert max_subarray([2, 3, -2, 4]) == 7
    assert max_subarray([-3, -1, -2]) == -1
    assert max_subarray([5]) == 5
    mixed = [-2, 1, -3, 4, -1, 2, 1, -5, 4]


# ============================================================
# FOLLOW-UP 1: Return the subarray itself.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Remember where the current run started: when
# starting fresh at i set start = i, and when cur beats
# best record (start, i). Same single pass.
```

```
def max_subarray_range(nums):
    # also say where it is: remember where the run started
    cur, start = nums[0], 0
    best, span = nums[0], (0, 0)
    for i in range(1, len(nums)):
        if nums[i] > cur + nums[i]:
            cur, start = nums[i], i
        else:
            cur += nums[i]
        if cur > best:
            best, span = cur, (start, i)
    return best, span

# ---- follow-up 1: tests ----
def test_follow_up_1():
    mixed = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
    assert max_subarray_range(mixed) == (6, (3, 6))


# ============================================================
```

```
# FOLLOW-UP 2: The array is circular (LeetCode 918).
# ============================================================

# ---- follow-up 2: function ----
# Answer: The best circular subarray is either an ordinary
# one or the whole array minus the worst ordinary one (the
# minimum subarray): max(kadane max, total - kadane min),
# taking care of the all-negative case.
def max_circular(nums):
    # circular array (LeetCode 918): the best run either
    # does not
    # wrap, or wraps and equals total minus the WORST run
    best = max_subarray(nums)
    total = sum(nums)
    worst = -max_subarray([-x for x in nums])
    # all negative: the wrap would be empty, so use best
    if total == worst:
        return best
    return max(best, total - worst)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert max_circular([5, -3, 5]) == 10
    assert max_circular([-3, -2, -3]) == -2
    assert max_circular([1, -2, 3, -2]) == 3
    samples = ([2, 3, -2, 4], [-1, -2], [1, -1, 1, -1, 5])


# ============================================================
# FOLLOW-UP 3: Divide and conquer instead of Kadane.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Split the array; the best subarray is entirely
# left, entirely right, or crosses the middle (best suffix
# of the left plus best prefix of the right). T(n) =
# 2T(n/2) + O(n) = O(n log n); useful when the data is
# split across machines.
def max_subarray_dc(nums, lo=0, hi=None):
    # divide and conquer: left, right, or crossing the
    # middle
    hi = len(nums) if hi is None else hi
    if hi - lo == 1:
        return nums[lo]
    mid = (lo + hi) // 2
```

```
    left_best = right_best = float("-inf")
    run = 0
    for i in range(mid - 1, lo - 1, -1):
        run += nums[i]
        left_best = max(left_best, run)
    run = 0
```

```
    for i in range(mid, hi):
        run += nums[i]
        right_best = max(right_best, run)
    return max(max_subarray_dc(nums, lo, mid),
               max_subarray_dc(nums, mid, hi),
               left_best + right_best)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    samples = ([2, 3, -2, 4], [-1, -2], [1, -1, 1, -1, 5])
    for arr in samples:
        assert max_subarray_dc(arr) == max_subarray(arr)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples

```
Input: nums = [2, 3, -2, 4]
Output: 7
Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.

Input: nums = [1, -1, -5, -4]
Output: 1
Explanation: The maximum sum is 1, which is the single
 element with the highest value.
```

OFFICIAL PAGE TEXT

Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?Our solution for finding the maximum subarray sum processes the input array by iterating through it while maintaining two variables: max_current and max_global. The max_current variable keeps track of the maximum sum of the subarray ending at the current position, while max_global stores the maximum sum encountered so far. For each element in the array, we update max_current by taking the maximum of the current element alone or the sum of the current element and max_current. This decision ensures that max_current always holds the maximum sum of the subarray that ends at the current element. We then update max_global to be the maximum of max_global and max_current. If the input array is empty, the function returns 0 since there are no subarrays to consider. This approach uses a simple iteration and a couple of variables, making it both straightforward and efficient. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (comparison and addition) for each element. Space Complexity: The solution uses O(1) space since it only requires a fixed amount of extra space

for the max_current and max_global variables, regardless of the input size. This makes the solution very efficient in terms of space usage. NoteThis algorithm is known as Kadane's Algorithm.No answers contributed by the community yet.

STUDY PATH: question 238 of 288 | topic 33 of 43: Prefix sums, Tier 2: running total plus a dict of earlier totals | 5 of 5

Template fit: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k | no date

HARD | IN APP LC 523 | Exponent | 238 of 288

Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 156/288

ABOUT THIS QUESTION

Topic: Prefix sums, Tier 2: running total plus a dict of earlier totals

Optimal answer from: your app's interview approach (LeetCode #523)

LeetCode match: LC 523 Continuous Subarray Sum

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +610 interviews

Original: https://www.aced.io/questions/2765/subarray-sum-multiple-of-k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

EXAMPLES

Example 1

Input	`[23, 2, 4, 6, 7], 6`
Output	`true`
Why	`[2, 4] adds up to 6, a multiple of 6.`

Example 2

Input	`[23, 2, 6, 4, 7], 13`
Output	`false`
Why	`No subarray of at least two elements adds up to a multiple of 13.`

QUESTION

Example 3

Input	`[23, 2, 6, 4, 7], 6`
Output	`true`
Why	`[23, 2, 6, 4, 7] adds up to 42, a multiple of 6.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    for i in range(len(nums)):
        total = nums[i]
        for j in range(i + 1, len(nums)):
            total += nums[j]
            if total % k == 0:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2) pairs. Two prefix sums with the same remainder mod k bound a subarray whose sum is a multiple of k, so a dictionary of first-seen remainders is O(n).

OPTIMAL SOLUTION · Prefix sums mod k with the first index of each remainder

TIME	SPACE
O(n)	O(min(n, k))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the subarrays whose sum is a multiple of k (any length, LeetCode 974).

Count how many times each remainder has occurred and add that count each time the remainder repeats (Python's % is already non-negative for negative sums). O(n).

FOLLOW-UP 2 Why store only the first index of each remainder?

A later occurrence of the same remainder would make the subarray shorter, never longer, so the first index is the best partner for the length-at-least-two condition.

FOLLOW-UP 3 The values can be negative or k huge.

Python's modulo gives a non-negative remainder; in languages where % can be negative normalise with ((s % k) + k) % k. A huge k just means the dictionary holds up to n entries. **Cost:** O(n) time with up to min(n, k) dictionary entries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given an integer array nums and an
# integer k, return true if nums has a subarray of at
# least two elements whose sum is a multiple of k.
#
# Determine if a given array contains a subarray of at
# least two elements whose sum is a multiple of a
# specified number k. An array is considered to have a
# "good subarray" if there exists at least one subarray
# (consisting of two or more elements) such that the sum
# of the elements in this subarray is a multiple of k. For
# example, the array [23, 2, 4, 7] with k = 6 has a "good
# subarray" ([2, 4]), as the sum 6 is a multiple of k = 6.
# The array [5, 0, 0, 0] with k = 3 also has a "good
# subarray" ([0, 0]), since the sum is 0, and 0 is a
# multiple of 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays whose sum is a multiple of k (any
# length, LeetCode 974).
# 2. Why store only the first index of each remainder?
# 3. The values can be negative or k huge.
#
# INTERVIEW NOTES
# A subarray sum is a difference of two prefix sums, and a
# multiple of k means the two prefix sums have the same
# remainder mod k. So remember the first index at which
# each remainder appeared; seeing it again at least two
# positions later gives a subarray of length at least two.
# Seed the table with remainder 0 at index -1 for
# subarrays that start at the beginning.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Prefix sums mod k with the
# first index of each remainder
# Idea: Walk the array keeping the running sum modulo k.
# If this remainder was seen before at index j and the
# current index i satisfies i - j >= 2, the elements
# between form a good subarray.
def has_good_subarray(nums, k):
    # prefix sums modulo k: two equal remainders at
    # indexes
    # at least 2 apart mean a block of 2 or more items
    # whose
    # sum is a multiple of k. Keep only the FIRST index of
    # each remainder: a later one only gives a shorter
    # block.
    first = {0: -1}
    total = 0
    for i, x in enumerate(nums):
        total += x
        # k == 0 means "equal to zero": compare the sums
        key = total % k if k else total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return True
    return False


# RETURN the subarray itself
def find_good_subarray(nums, k):
    # the same scan, but return the (start, end) indexes
    first = {0: -1}
    total = 0
    for i, x in enumerate(nums):
        total += x
        key = total % k if k else total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return first[key] + 1, i
    return None

# ---- main question: tests ----
def test_main_question():
    assert has_good_subarray([23, 2, 4, 7], 6)
```

```
    assert has_good_subarray([23, 2, 6, 4, 7], 6)
    assert not has_good_subarray([23, 2, 6, 4, 7], 13)
    assert find_good_subarray([23, 2, 4, 7], 6) == (1, 2)
    # a single item is not enough, even for 0 and k
    assert not has_good_subarray([6], 6)
    assert has_good_subarray([0, 0], 5)
    assert not has_good_subarray([0, 1, 0], 0)
    assert has_good_subarray([0, 0], 0)
    assert has_good_subarray([5, -5], 7)
    assert has_good_subarray([1, 2], 10 ** 9) is False


# ============================================================
# FOLLOW-UP 1: Count the subarrays whose sum is a multiple
# ============================================================

# ---- follow-up 1: function ----
# Answer: Count how many times each remainder has occurred
# and add that count each time the remainder repeats
# (Python's % is already non-negative for negative sums).
# O(n).
def count_multiples(nums, k):
    # follow-up (LeetCode 974): count the subarrays of any
    # length whose sum is a multiple of k. Python's % is
    # never
    # negative; elsewhere use ((s % k) + k) % k.
    seen = {0: 1}
    total = answer = 0
    for x in nums:
        total += x
        key = total % k
        # every earlier equal remainder is one more
        # subarray
```

```
        answer += seen.get(key, 0)
        seen[key] = seen.get(key, 0) + 1
    return answer

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_multiples([4, 5, 0, -2, -3, 1], 5) == 7


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples

```
nums = [23, 2, 4, 7], k = 6
output: true

nums = [5, 0, 0, 0], k = 3
output: true

nums = null, k = 1
output: false
```

OFFICIAL PAGE TEXT

In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?Our solution processes the input array by iterating through it while maintaining a hash map (remainder_index_map) of remainders and their corresponding indices. For each element in the array, we accumulate the sum (sum_so_far) and compute the remainder when divided by k. We check if this remainder has been encountered before in the hash map. If the remainder is not in the hash map, we store the current index associated with this remainder. If the remainder has been seen before, we calculate the distance between the current index and the index of the previously seen remainder. If this distance is greater than or equal to the minimum subarray length (MIN_GOOD_SUBARRAY_LIMIT), we return True, indicating that a subarray with the required properties exists. If the loop completes without finding such a subarray, we return False. The use of a hash map ensures that each element is processed efficiently, as it allows constant-time lookups and insertions. Time Complexity: The solution has a time complexity of O(n), where n is the

number of elements in the array. This is because we iterate through the array once, performing constant-time operations (hash map lookups and insertions) for each element. Space Complexity: The solution uses O(n) space for the hash map that stores the remainders and their indices. In the worst case, all remainders are unique, resulting in the hash map containing n entries.Let’s say you are given an integer array nums and an integer k. Return true if nums has a subarray where its length is at least two, and the sum of the elements in the subarray is a multiple of k. Watch Neeraj, a Senior Engineering Manager, give his answer to this software engineering interview question.No answers contributed by the community yet.Related questionsGiven an array of n integers and a number k, find the maximum number of elements whose sum is less than k, and return their sum.Given an array of integers and an integer k, find the number of continuous subarrays whose sum equals k.Given an array of integers and a number N, find the length of the longest contiguous subarray such that the difference between any two elements in the subarray is less than N.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from

OFFICIAL PAGE TEXT

mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScript JavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 34 OF 43

Group by a canonical key

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	3	1
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Group by a canonical key | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

E	239. Given an array of strings, return the words that can be typed using letters of the alphabet on only...	Exponent	no date
M	240. Check if all substrings are anagrams of words	PracHub	2026-03-11
M	241. Group anagrams	Exponent	no date
M	242. Given an array of strings, return groups of "buddy strings," where the strings have the same lengt...	Exponent	no date
H	243. Given two strings and two allowed operations, swapping positions of letters and converting all oc...	Exponent	no date

CODE TEMPLATE | GROUP 34 OF 43 | 5 QUESTIONS, ONE PATTERN

Group by a canonical key

The one idea these questions share

Find a key that is the same for everything that should be grouped (sorted letters, letter counts, shifts between letters, the keyboard row) and put the items into a dict under that key. Equal key means same group.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from collections import defaultdict


def group_by_key(words, key):
    groups = defaultdict(list)
    for w in words:
        groups[key(w)].append(w)              # CHANGE: the key function decides everything
    return list(groups.values())
```

2. Helper functions that usually get written next to it

```
from collections import Counter


def letter_counts(word):                     # a hashable signature that ignores order
    return tuple(sorted(Counter(word).items()))


def shift_signature(word):
    return tuple((ord(word[i]) - ord(word[i - 1])) % 26 for i in range(1, len(word)))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
239	Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.	key = the keyboard row of each letter; a word qualifies when its letters map to one row
240	Check if all substrings are anagrams of words	key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it
241	Group anagrams	key = the sorted letters (or a tuple of 26 letter counts)
242	Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.	key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)
243	Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?	same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 239 of 288 | topic 34 of 43: Group by a canonical key | 1 of 5

Template fit: key = the keyboard row of each letter; a word qualifies when its letters map to one row | no date

EASY | Exponent | 239 of 288

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 181/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 500 Keyboard Row (inferred from short statement)

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age)

Original:
https://www.aced.io/questions/7293/return-words-one-keyboard-row

QUESTION

Given an array of words, return those that can be typed using letters from a single row of an American keyboard (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore upper and lower case and keep the input order.

Function signature

def findWords(words: list) -> list

EXAMPLES

Example 1	
Input	`['Hello', 'Alaska', 'Dad', 'Peace']`
Output	`['Alaska', 'Dad']`
Why	`'Alaska' and 'Dad' use only the middle row; 'Hello' and 'Peace' mix rows.`

Example 2	
Input	`['omk']`
Output	`[]`
Why	`'omk' mixes the top and bottom rows.`

Example 3	
Input	`['adsdf', 'sfd']`
Output	`['adsdf', 'sfd']`
Why	`Both words stay on one row.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
ROWS = [set("qwertyuiop"), set("asdfghjkl"),
        set("zxcvbnm")]
def brute(words):
    found = []
    for word in words:
        letters = set(word.lower())
        for row in ROWS:
            if letters <= row:
                found.append(word)
                break
    return found
```

TIME	SPACE
O(W * L * 3)	O(1)

WHY IT IS SLOW

Up to three scans per word; building one letter-to-row table makes each word a single scan, O(total characters).

OPTIMAL SOLUTION · Map each letter to its row and compare sets

TIME	SPACE
O(total characters)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?

Nothing changes except the row table, which is built from the layout argument. Build the letter-to-row map once and reuse it across queries.

FOLLOW-UP 2 Other layouts (Dvorak, AZERTY) or a phone keypad.

Pass the rows as a parameter and build the letter-to-row dict from it. Nothing else changes.

FOLLOW-UP 3 A different keyboard layout.

Pass the rows as a parameter and build the letter-to-row map from it (a phone keypad or Dvorak works the same way). Letters that are on no row make the word invalid, which the code checks.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given an array of strings, return the
# words that can be typed using letters of the alphabet on
# only one row of American keyboard.
#
# Given an array of words, return those that can be typed
# using letters from a single row of an American keyboard
# (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore
# upper and lower case and keep the input order.
#
# Signature:
#   def findWords(words: list) -> list
#
# Examples:
# ['Hello', 'Alaska', 'Dad', 'Peace']  ->  ['Alaska',
#     'Dad']
# ['omk']  ->  []
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the keyboard layout is a parameter (Dvorak,
# AZERTY, a phone keypad)?
# 2. Other layouts (Dvorak, AZERTY) or a phone keypad.
# 3. A different keyboard layout.
#
# INTERVIEW NOTES
# Each letter belongs to exactly one row, so the question
# for a word is whether all its letters map to the same
# row. Build the letter-to-row lookup once and check each
# word with a set of rows.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]

# ---- main question: main function ----
```

```
# LAYOUT table (change it for other keyboards)
def row_map(rows):
    # letter -> index of its row; built once and reused
    return {ch: i for i, row in enumerate(rows)
            for ch in row}


# BASE QUESTION (optimal): Map each letter to its row and
# compare sets
# Idea: Build a dict from letter to row index (26
# entries). A word qualifies if all of its lowercased
# letters map to the same row, i.e. the set of rows has
# size one. Lowercase first so 'Alaska' works.
# FOLLOW-UP 3: A different keyboard layout.
```

```
# Answer: Pass the rows as a parameter and build the
# letter-to-row map from it (a phone keypad or Dvorak
# works the same way). Letters that are on no row make the
# word invalid, which the code checks.
def one_row_words(words, rows=QWERTY):
    # FOLLOW-UP 3: pass rows= for another keyboard layout
    row_of = row_map(rows)
    out = []
    for w in words:
        letters = w.lower()
        # a letter that is on no row makes the word
        # invalid
        if any(c not in row_of for c in letters):
            continue
        # all letters must come from the same row
        if len({row_of[c] for c in letters}) <= 1:
            out.append(w)
    return out

# ---- main question: tests ----
def test_main_question():
    words = ["Hello", "Alaska", "Dad", "Peace"]
    assert one_row_words(words) == ["Alaska", "Dad"]
    assert one_row_words([]) == []
    assert one_row_words(["", "a1"]) == [""]
    # another layout: a phone keypad
    keypad = ["abc", "def", "ghi"]
    got = one_row_words(["bad", "fed", "cab"], keypad)
    assert got == ["fed", "cab"]
    # Dvorak top row
    dvorak = ["pyfgcrl", "aoeuidhtns", "qjkxbmwvz"]
```

```
    assert one_row_words(["tent"], dvorak) == ["tent"]


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: A different keyboard layout.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 240 of 288 | topic 34 of 43: Group by a canonical key | 2 of 5

Template fit: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it | asked 223 days ago

MEDIUM | PracHub | Technical Screen | 240 of 288

★ TOP 75: rank 22 in the Medium and Hard priority list

★ Check if all substrings are anagrams of words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 51.4 | PRIORITY 20/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #49 Group Anagrams

LeetCode match: No direct match; related LC 49 Group Anagrams

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-anagrams-of-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - "Can be rearranged into a word" means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

EXAMPLES

Example 1

Input	`'abca', ['abc', 'cab', 'bca', 'abca']`
Output	`true`
Why	`Every substring of length 3 or more of 'abca' is an anagram of a dictionary word.`

Example 2

Input	`'abca', ['abc']`
Output	`false`
Why	`'bca' is an anagram of 'abc' but 'abca' has no match.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, words):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            key = sorted(s[i:j])
            match = [w for w in words
                     if key == sorted(w)]
            if not match:
                return False
    return True
```

TIME	SPACE
O(n^2 * W * L log L)	O(1) extra

WHY IT IS SLOW

Each substring is compared with every word and sorted again: O(n^2 * W * L log L). A set of word signatures, with the substring's count vector updated incrementally, makes each check O(alphabet).

OPTIMAL SOLUTION · Set of signatures, incremental count vectors

TIME	SPACE
O(n^2 * A) for alphabet size A	O(dictionary size)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. **Cost:** O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

FOLLOW-UP 2 Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

FOLLOW-UP 3 Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

FOLLOW-UP 4 Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Check if all substrings are anagrams of
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** can be **rearranged (anagrammed)** into a valid
# dictionary word. Formally, for every substring t =
# s[i..j] with |t| >= 3, there exists some word w in dict
# such that w is a permutation of t (same multiset of
# characters). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - “Can be rearranged
# into a word” means the substring and the dictionary word
# have exactly the same character counts. Example: - If t
# = "act", it passes if the dictionary contains any of {
# "act", "cat", "tac" }.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A bigger alphabet or Unicode.
# 2. Only substrings of lengths that exist in the
# dictionary need checking.
# 3. Subsets of the letters (any sub-multiset of a word)
# instead of a full permutation.
# 4. Report the first failing substring instead of True or
# False.
#
# INTERVIEW NOTES
# Anagrams share a signature: the multiset of letters (a
# sorted string or a 26-count tuple). So turn the
# dictionary into a set of signatures once, and test each
# substring by its signature. Extending a substring by one
# letter changes its count vector in O(1), so you do not
# rebuild the signature from scratch.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
ALPHABET = "abcdefghijklmnopqrstuvwxyz"
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3).
def all_anagram_words(s, words, min_len=3):
    # every substring needs a dictionary word with the
    # same
    # letters. Count vectors are updated as the window
    # grows.
    index = {c: i for i, c in enumerate(ALPHABET)}
    sigs = set()
    for w in words:
        v = [0] * len(ALPHABET)
        for ch in w:
            v[index[ch]] += 1
        sigs.add(tuple(v))
    for i in range(len(s)):
        v = [0] * len(ALPHABET)
        for j in range(i, len(s)):
            v[index[s[j]]] += 1
            long_enough = j - i + 1 >= min_len
            if long_enough and tuple(v) not in sigs:
                return False
    return True

# ---- main question: tests ----
def test_main_question():
    words = {"cat", "act", "tac", "tact", "cats"}
    assert all_anagram_words("cat", words)
    assert all_anagram_words("tca", words)
    assert not all_anagram_words("tcaz", words)


# ============================================================
# FOLLOW-UP 1: A bigger alphabet or Unicode.
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Replace the fixed array by a Counter and use a
# frozen tuple of sorted (letter, count) pairs, or a hash
# of the counts, as the signature.
def all_anagram_words_any_alphabet(s, words, min_len=3):
    # Unicode or a big alphabet: a frozen Counter as
    # signature
    sigs = {frozenset(Counter(w).items()) for w in words}
    for i in range(len(s)):
        count = Counter()
        for j in range(i, len(s)):
            count[s[j]] += 1
            key = frozenset(count.items())
            if j - i + 1 >= min_len and key not in sigs:
                return False
    return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = {"cat", "act", "tac", "tact", "cats"}
    assert all_anagram_words_any_alphabet("cat", words)


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
```

```
# FOLLOW-UP 3
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ========================================================
# FOLLOW-UP 4: Report the first failing substring instead
# ========================================================

# ---- follow-up 4: helper functions ----
def signature(text):
    # the letters in sorted order: equal for anagrams
    return "".join(sorted(text))

# ---- follow-up 4: function ----
# Answer: Test substrings by length (shortest first) so
```

```
# the report is a smallest counterexample; the signature
# set makes each test one lookup after sorting the
# substring's letters.
def first_failing(s, words, min_len=3):
    # smallest counterexample: lengths from short to long
    sigs = {signature(w) for w in words}
    lengths = {len(w) for w in words}
    for length in range(min_len, len(s) + 1):
        # no word has this length: the first window fails
        if length not in lengths:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if signature(piece) not in sigs:
                return piece
    return None

# ---- follow-up 4: tests ----
def test_follow_up_4():
    words = {"cat", "act", "tac", "tact", "cats"}
    assert first_failing("cat", words) is None
    # "catz" fails at length 3? "atz" is not an anagram
    assert first_failing("catz", words) == "atz"
    assert all_anagram_words("ab", set())
    assert first_failing("aaaa", {"aaa"}) == "aaaa"


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, words: list[str]) -> bool:
```

Sample call

```
solution("act", ["cat", "dog"])
```

Test cases

Input	Expected
("act", ["cat", "dog"])	True
("abca", ["cab", "abca", "zzz"])	True

STUDY PATH: question 241 of 288 | topic 34 of 43: Group by a canonical key | 3 of 5

Template fit: key = the sorted letters (or a tuple of 26 letter counts) | no date

MEDIUM | IN APP LC 49 | Exponent | 241 of 288

★ TOP 75: rank 23 in the Medium and Hard priority list

★ Group anagrams

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 57.7** | **30 DAYS 51.4** | **PRIORITY 19/288**

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: your app's interview approach (LeetCode #49)

LeetCode match: LC 49 Group Anagrams (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4676/group-anagrams

QUESTION

Given a list of words, group together the words that are anagrams of each other (the same letters in a different order). Return the groups in any order.

Function signature

def group_by_sorting(words: list) -> list

EXAMPLES

Example 1	
Input	`['eat', 'tea', 'tan', 'ate', 'nat', 'bat']`
Output	`[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]`
Why	`eat, tea and ate use the same letters; tan and nat; bat is alone.`

Example 2	
Input	`['']`
Output	`[['']]`
Why	`An empty string is its own group.`

Example 3	
Input	`['a']`
Output	`[['a']]`
Why	`One word.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(words):
    groups = []
    for word in words:
        key = sorted(word)
        for group in groups:
            if sorted(group[0]) == key:
                group.append(word)
                break
        else:
            groups.append([word])
    return groups
```

TIME	SPACE
O(n * g * L log L)	O(n)

WHY IT IS SLOW

Each word is compared with many groups, O(n * groups * L log L). Using the sorted letters (or a letter-count tuple) as a dictionary key groups all words in one pass.

OPTIMAL SOLUTION · Dictionary keyed by the sorted letters

TIME	SPACE
O(n * L log L)	O(n * L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the key O(L) instead of O(L log L).

Use a tuple of 26 letter counts as the key (no sort). Total O(n * L); for large alphabets use a Counter turned into a frozenset of items or a sorted tuple of (char, count).

FOLLOW-UP 2 Case-insensitive, Unicode, or ignore spaces and punctuation.

Normalise first (casefold, strip non-letters, Unicode normalisation) and then compute the key. Say what counts as a letter.

FOLLOW-UP 3 The words are in many files.

Compute the key on each machine and hash-partition by key so every anagram group lands on one worker, which then groups locally (a MapReduce shuffle).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Group anagrams
#
# Given a list of words, group together the words that are
# anagrams of each other (the same letters in a different
# order). Return the groups in any order.
#
# Signature:
#   def group_by_sorting(words: list) -> list
#
# Examples:
# ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']  ->  [['eat',
#     'tea', 'ate'], ['tan', 'nat'], ['bat']]
# ['']  ->  [['']]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the key O(L) instead of O(L log L).
# 2. Case-insensitive, Unicode, or ignore spaces and
# punctuation.
# 3. The words are in many files.
#
# INTERVIEW NOTES
# Anagrams are strings with the same letters in any order,
# so they have the same canonical form: the sorted
# letters, or the count of each letter. Use that as a
# dictionary key and group in one pass; you never compare
# two strings directly.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import unicodedata
from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Dictionary keyed by the sorted
# letters
```

```
# Idea: Sort the letters of each word and use the
# resulting string as the key; words with the same key are
# anagrams. One pass, one dictionary.
def group_by_sorting(words):
    # anagrams share the same letters in sorted order
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())

# ---- main question: tests ----
def test_main_question():
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    want = [["eat", "tea", "ate"], ["tan", "nat"],
            ["bat"]]
    assert group_by_sorting(words) == want


# ============================================================
# FOLLOW-UP 1: Make the key O(L) instead of O(L log L).
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use a tuple of 26 letter counts as the key (no
# sort). Total O(n * L); for large alphabets use a Counter
# turned into a frozenset of items or a sorted tuple of
# (char, count).
def group_by_counts(words):
    # key = 26 letter counts: O(L) per word instead of O(L
    # log L)
    groups = defaultdict(list)
    for w in words:
        counts = [0] * 26
        for ch in w:
            counts[ord(ch) - ord("a")] += 1
        groups[tuple(counts)].append(w)
    return list(groups.values())

# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    want = [["eat", "tea", "ate"], ["tan", "nat"],
            ["bat"]]
    assert group_by_counts(words) == want
    assert group_by_sorting([]) == []
```

```
    assert group_by_sorting([""]) == [[""]]
    mixed = ["Listen", "Silent", "enlist", "a gentleman"]


# ============================================================
# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces
# ============================================================

# ---- follow-up 2: function ----
# Answer: Normalise first (casefold, strip non-letters,
# Unicode normalisation) and then compute the key. Say
# what counts as a letter.
def normalise(word):
    # case, accents, spaces and punctuation: decide what
    # counts
    text = unicodedata.normalize("NFKD", word).casefold()
    return "".join(ch for ch in text if ch.isalpha())


# Answer: Normalise first (casefold, strip non-letters,
# Unicode normalisation) and then compute the key. Say
# what counts as a letter.
def group_normalised(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(normalise(w)))].append(w)
    return list(groups.values())

# ---- follow-up 2: tests ----
def test_follow_up_2():
    mixed = ["Listen", "Silent", "enlist", "a gentleman"]
    got = group_normalised(mixed)[0]
    assert got == ["Listen", "Silent",
                                          "enlist"]
    assert normalise("Café!") == "cafe"


# ============================================================
# FOLLOW-UP 3: The words are in many files.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Compute the key on each machine and hash-
# partition by key so every anagram group lands on one
# worker, which then groups locally (a MapReduce shuffle).
```

```
def shard_of(word, workers):
    # many files / machines: hash the key so one anagram
    # group
    # always lands on the same worker
    key = "".join(sorted(word))
    return sum(map(ord, key)) % workers
```

```python
# ---- follow-up 3: tests ----
def test_follow_up_3():
    # anagrams always go to the same worker
    assert shard_of("eat", 8) == shard_of("tea", 8)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given an array of strings, group together the words that are anagrams (e.g. gas, sag) of each other. The order of the groups or the words within the groups does not matter in the final result.

STUDY PATH: question 242 of 288 | topic 34 of 43: Group by a canonical key | 4 of 5

Template fit: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences) | no date

MEDIUM | IN APP LC 249 | Exponent | 242 of 288

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

ASKED (ANY CO.) ~8 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 35.4 | 30 DAYS 0 | PRIORITY 126/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: your app's interview approach (LeetCode #249)

LeetCode match: LC 249 Group Shifted Strings (inferred from short statement)

Last asked (any company): 8 months ago, about 2026-02 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6356/group-buddy-strings-same-distance

QUESTION

Two strings are buddy strings when they have the same length and the distance between each pair of neighbouring letters is the same (the alphabet wraps around after z), so one is the other shifted by a constant. Group the strings into buddy groups and return the groups in any order.

Function signature

def group_buddies(words: list) -> list

EXAMPLES

Example 1	
Input	`['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']`
Output	`[['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a', 'z']]`
Why	`'abc', 'bcd' and 'xyz' all step +1, +1; 'az' and 'ba' both step +25 (a to z wraps); single letters 'a' and 'z' form a group.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(words):
    groups = []
    for s in words:
        for group in groups:
            if is_shift(group[0], s):
                group.append(s)
                break
        else:
            groups.append([s])
    return groups
def is_shift(a, b):
    if len(a) != len(b):
        return False
    shifts = {(ord(y) - ord(x)) % 26
              for x, y in zip(a, b)}
    return len(shifts) <= 1
```

TIME	SPACE
O(n * groups * L)	O(n)

WHY IT IS SLOW

Each string is compared with the representative of every group, O(n * groups * L). Using the sequence of gaps between neighbouring letters as a dictionary key groups everything in one pass.

OPTIMAL SOLUTION · Group by the tuple of successive differences

TIME	SPACE
O(total characters)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if shifting is not cyclic (z does not wrap to a)?

Drop the % 26 and use the raw differences as the key. Two strings with the same raw gaps differ by one fixed integer shift, and both are already inside 'a'..'z', so that shift is automatically legal: the gap tuple alone decides buddies, with no extra range check. What changes is that 'az' and 'ba' are no longer buddies (gaps 25 and -1 instead of 25 and 25).

FOLLOW-UP 2 How would you do this with strings that are too many for one machine?

Compute the gap key on each machine, hash-partition the strings by key (so every buddy lands on the same worker), and group locally. The key is the only thing exchanged, so shuffle cost is one pass over the data. **Cost:** O(total characters / machines) per machine to compute the keys, plus one shuffle of the keys; each group is then built locally in O(group size).

FOLLOW-UP 3 Shifting does not wrap around (z stays z).

Use the raw gaps (no % 26) as the key. Equal raw gaps mean one fixed integer shift, which is automatically legal because both strings are already letters, so no extra check is needed.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given an array of strings, return groups
# of "buddy strings," where the strings have the same
# length and the distance between corresponding characters
# is the same.
#
# Two strings are buddy strings when they have the same
# length and the distance between each pair of
# neighbouring letters is the same (the alphabet wraps
# around after z), so one is the other shifted by a
# constant. Group the strings into buddy groups and return
# the groups in any order.
#
# Signature:
#   def group_buddies(words: list) -> list
#
# Examples:
# ['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']  ->
#     [['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a',
#     'z']]
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if shifting is not cyclic (z does not wrap to
# a)?
# 2. How would you do this with strings that are too many
# for one machine?
# 3. Shifting does not wrap around (z stays z).
#
# INTERVIEW NOTES
# Buddies are shifted copies of each other, so what they
# share is the shape: the gaps between neighbouring
# letters. Turn each string into its shape (a tuple of
# gaps mod 26) and group by it with a dictionary; you
# never compare two strings directly. Mention that single
# letters all have the same (empty) shape.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict
import random
```

```


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to
    # a.
    # Words that are shifts of each other share this key.
    return tuple((ord(b) - ord(a)) % 26
                 for a, b in zip(word, word[1:]))

# ---- main question: main function ----
# BASE QUESTION (optimal): Group by the tuple of
# successive differences
# Idea: Two strings are buddies when every letter is
# shifted by the same amount, which means the gaps between
# neighbouring letters (mod 26) are identical. Use those
# gaps as the hash key.
def group_buddies(words, key=shift_key):
    # words of different length have keys of different
    # length, so they never land in the same group
    groups = defaultdict(list)
    for w in words:
        groups[(len(w), key(w))].append(w)
    return list(groups.values())

# ---- main question: tests ----
def test_main_question():
    words = ["abc", "bcd", "acef", "xyz", "az", "ba",
             "a", "z"]
    got = canonical(group_buddies(words))
    assert got == [["a", "z"], ["abc", "bcd", "xyz"],
                   ["acef"], ["az", "ba"]]


# ============================================================
# FOLLOW-UP 1: What if shifting is not cyclic (z does not
# ============================================================

# ---- follow-up 1: function ----
# Answer: Drop the % 26 and use the raw differences as the
# key.
# FOLLOW-UP 3: Shifting does not wrap around (z stays z).
```

```
# Answer: Use the raw gaps (no % 26) as the key. Equal raw
# gaps mean one fixed integer shift, which is
# automatically legal because both strings are already
# letters, so no extra check is needed.
def raw_key(word):
    # follow-up: no wrap around (z stays z). The raw gaps
    # already decide it: one fixed integer shift is legal
    # because both words are already made of letters.
    return tuple(ord(b) - ord(a)
                 for a, b in zip(word, word[1:]))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = ["abc", "bcd", "acef", "xyz", "az", "ba",
             "a", "z"]
    # without wrap around "az" and "ba" are different
    plain = canonical(group_buddies(words, raw_key))
    assert ["az", "ba"] not in plain
    assert ["abc", "bcd", "xyz"] in plain
    assert group_buddies([]) == []


# ============================================================
# FOLLOW-UP 2: How would you do this with strings that are
# ============================================================

# ---- follow-up 2: function ----
# Answer: Compute the gap key on each machine, hash-
# partition the strings by key (so every buddy lands on
# the same worker), and group locally. The key is the only
# thing exchanged, so shuffle cost is one pass over the
# data.
def group_distributed(words, machines, key=shift_key):
    # follow-up: too many words for one machine. Each
```

```
    # machine
    # computes keys for its slice, every pair (key, word)
    # is
    # sent to machine hash(key) % machines, and each
    # groups what it receives. Only the key travels.
    slices = [words[i::machines] for i in range(machines)]
    inbox = [[] for _ in range(machines)]
    for part in slices:
        for w in part:
            k = (len(w), key(w))
            inbox[hash(k) % machines].append((k, w))
```

```
    result = []
    for mail in inbox:
        local = defaultdict(list)
        for k, w in mail:
            local[k].append(w)
        result.extend(local.values())
    return result

# ---- follow-up 2: tests ----
def test_follow_up_2():
    for _ in range(200):
        ws = []
        for _ in range(random.randint(0, 10)):
            size = random.randint(1, 3)
            ws.append("".join(random.choice("abyz")
                              for _ in range(size)))
        groups = group_buddies(ws)
        assert sum(len(g) for g in groups) == len(ws)
        # two different groups are never buddies
        parts = random.randint(1, 4)
        mine = group_distributed(ws, parts)
        assert canonical(mine) == canonical(groups)


# ============================================================
# FOLLOW-UP 3: Shifting does not wrap around (z stays z).
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def canonical(groups):
    return sorted(sorted(g) for g in groups)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 243 of 288 | topic 34 of 43: Group by a canonical key | 5 of 5

Template fit: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values | no date

HARD | Exponent | 243 of 288

Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

ASKED (ANY CO.) ~6 MONTHS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 183/288

ABOUT THIS QUESTION

Topic: Group by a canonical key

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1153 String Transforms Into Another String

LeetCode match: No direct match; related LC 1153 String Transforms Into Another String (inferred from short statement)

Last asked (any company): 6 months ago, about 2026-04 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/6330/determine-string-transformability-allowed-operations

QUESTION

Two strings are 'close' if you can turn one into the other with these operations, used any number of times: (1) swap the positions of any two letters; (2) take two letters that occur in the string and swap all of their occurrences (every a becomes b and every b becomes a). Return whether the first string can be transformed into the second.

Function signature

def close_strings(s: str, t: str) -> bool

EXAMPLES

Example 1	
Input	`'abc', 'bca'`
Output	`true`
Why	`Swapping positions turns 'abc' into 'bca'.`

Example 2	
Input	`'a', 'aa'`
Output	`false`
Why	`The lengths differ.`

QUESTION

Example 3	
Input	`'cabbba', 'abbccc'`
Output	`true`
Why	`Use operation 1 to reorder, then operation 2 to swap the letters a and b, and c and ... until the counts match.`

Example 4	
Input	`'uau', 'ssx'`
Output	`false`
Why	`'u' appears in the first string but not the second, so they cannot be close.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(s, t):
    seen, queue = {s}, deque([s])
    while queue:
        cur = queue.popleft()
        if cur == t:
            return True
        for nxt in swap_positions(cur):
            add_if_new(nxt, seen, queue)
        for nxt in convert_letters(cur):
            add_if_new(nxt, seen, queue)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable strings grows exponentially with the length. Because swaps make order irrelevant, comparing the sets of letters and their sorted counts decides it in O(n).

OPTIMAL SOLUTION · Reading 1 (LeetCode 1657): same letter set and same multiset of frequencies

TIME	SPACE
O(n)	O(1) for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Reading 2 (LeetCode 1153): one-directional conversion, no swaps

TIME	SPACE
O(n)	O(1)

OPTIMAL SOLUTION · Reading 2 (LeetCode 1153): one-directional conversion, no swaps

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the two operations are combined (one-directional conversion plus swaps)?

Swaps make positions irrelevant, so only letter counts matter. Conversions can only merge letter classes, never split one. So t is reachable only if the counts of t can be formed by grouping the counts of s into the t letters (each s letter's count goes wholly to one t letter) and, unless the relabeling is an identity or chain, there is a spare letter. With at most 26 classes the grouping is a small bin-packing search.

FOLLOW-UP 2 Why does a full 26-letter target make the one-directional version impossible?

Every conversion rewrites a letter into another letter, and if a mapping contains a cycle (a to b and b to a) you need a temporary letter not currently in use to break it. If t uses all 26 letters then s must use all 26 as well (the mapping is a function onto 26 distinct letters), so the mapping is a permutation and any non-identity permutation has a cycle. The only possible case is therefore s equal to t.

FOLLOW-UP 3 Only position swaps are allowed.

Then s can become t exactly when they are anagrams: compare the letter counts. O(n).

FOLLOW-UP 4 Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode 1153).

Each source letter must map to one target letter, and if the strings differ and the target already uses all 26 letters there is no spare letter to break mapping cycles, so it is impossible. Verified against brute force on a small alphabet.

FOLLOW-UP 5 Return the sequence of operations, or the minimum number.

Minimum operations for the swap-only case is n minus the number of cycles in the permutation that sorts s into t; for letter-identity swaps it is the number of cycles in the letter mapping minus one per cycle of length greater than one.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Given two strings and two allowed
# operations, swapping positions of letters and converting
# all occurrences of one character to another, can the
# first string be transformed into the second?
#
# Two strings are 'close' if you can turn one into the
# other with these operations, used any number of times:
# (1) swap the positions of any two letters; (2) take two
# letters that occur in the string and swap all of their
# occurrences (every a becomes b and every b becomes a).
# Return whether the first string can be transformed into
# the second.
#
# Signature:
#   def close_strings(s: str, t: str) -> bool
#
# Examples:
# 'abc', 'bca'  ->  true
# 'a', 'aa'  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the two operations are combined (one-
# directional conversion plus swaps)?
# 2. Why does a full 26-letter target make the one-
# directional version impossible?
# 3. Only position swaps are allowed.
# 4. Conversion is one-directional (every a becomes b) and
# there are no position swaps (LeetCode 1153).
# 5. Return the sequence of operations, or the minimum
# number.
#
# INTERVIEW NOTES
# Decide which operations you really have. If you can swap
# any two letters (positions) and swap the identities of
# two letters, the answer depends only on which letters
# appear and how often (the multiset of counts). If
# conversion is one-way (every a becomes b), letters can
# merge but never split, and cycles in the mapping need a
# spare letter. Name the reading before you code.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
```

```
# ============================================================

from collections import Counter


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
ALPHABET = 26


def can_group(have, want):
    # can the counts in `have` be grouped (each used once)
    # so that the group sums are exactly the counts in
    # `want`? Letters merge, they never split.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
    left = list(want)

    def place(i):
        if i == len(have):
            return all(x == 0 for x in left)
```

```
        tried = set()
        for j in range(len(left)):
            # equal bins behave the same: try each size
            # once
            fits = left[j] >= have[i]
            if fits and left[j] not in tried:
                tried.add(left[j])
                left[j] -= have[i]
                if place(i + 1):
                    return True
                left[j] += have[i]
        return False

    return place(0)

# ---- main question: main function ----
# BASE QUESTION (optimal): (LeetCode 1657): same letter
# set and same multiset of frequencies
# Idea: Operation 1 reorders positions freely, so only the
# multiset of letters matters. Operation 2 exchanges the
```

```
# identities of two letters that exist, which permutes the
# frequencies among existing letters.
def both_operations(s, t, alphabet=ALPHABET):
    # swaps make positions irrelevant: only the letter
    # counts matter, and a conversion can only merge or
    # rename a letter
    if len(s) != len(t):
        return False
    cs, ct = Counter(s), Counter(t)
    if cs == ct:
        return True
    mine, goal = list(cs.values()), list(ct.values())
    if not can_group(mine, goal):
        return False
    # only identical sizes mean no merge happens. With
    # every
    # letter used there is no spare letter to rename
    # safely.
    if len(cs) == len(ct) == alphabet:
        return False
    return True

# ---- main question: tests ----
def test_main_question():
    assert both_operations('abc', 'bca')
    assert not both_operations('a', 'aa')
    assert both_operations('cabbba', 'abbccc')


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Only position swaps are allowed.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Then s can become t exactly when they are
```

```
# anagrams: compare the letter counts. O(n).
def swaps_only(s, t):
    # swaps only rearrange: reachable exactly when the
    # strings are anagrams (same letter counts)
    return Counter(s) == Counter(t)

# ============================================================
# FOLLOW-UP 4: Conversion is one-directional (every a
# ============================================================

# ---- follow-up 4: function ----
# Answer: Each source letter must map to one target
# letter, and if the strings differ and the target already
# uses all 26 letters there is no spare letter to break
# mapping cycles, so it is impossible.
def convert_only(s, t, alphabet=ALPHABET):
    # no swaps, conversion turns EVERY x into one letter y
    # (LeetCode 1153). Each source letter needs one
    # target.
    if len(s) != len(t):
        return False
    if s == t:
        return True
    mapping = {}
    for a, b in zip(s, t):
        if mapping.setdefault(a, b) != b:
            return False
    # a mapping with a cycle (a to b, b to a) needs a
    # spare
    # letter, so a target that uses every letter has none
    return len(set(t)) < alphabet

# ============================================================
# FOLLOW-UP 5: Return the sequence of operations, or the
# ============================================================

# ---- follow-up 5: function ----
# Answer: Minimum operations for the swap-only case is n
# minus the number of cycles in the permutation that sorts
# s into t; for letter-identity swaps it is the number of
# cycles in the letter mapping minus one per cycle of
# length greater than one.
def min_swaps(s, t):
    # follow-up: fewest swaps when the letters are all
    # different: n minus the number of cycles of the
```

```
    # permutation that sorts s into t
    where = {c: i for i, c in enumerate(t)}
    seen, cycles = set(), 0
    for i in range(len(s)):
        if i not in seen:
            cycles += 1
            while i not in seen:
                seen.add(i)
                i = where[s[i]]
    return len(s) - cycles

# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def swap_sequence(s, t):
    # the swaps themselves (distinct letters): put the
    # right
    # letter into each position, one swap per misplaced
    # one
    cur, steps = list(s), []
    where = {c: i for i, c in enumerate(cur)}
    for i, want in enumerate(t):
        if cur[i] != want:
            j = where[want]
            where[cur[i]] = j
            where[want] = i
            cur[i], cur[j] = cur[j], cur[i]
```

```
            steps.append((i, j))
    return steps


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 35 OF 43

K-way merge: a heap of the next item from each source

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	2	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

K-way merge: a heap of the next item from each source | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

E	244. Find k pairs with smallest sums	PracHub	2025-11-01
M	245. K Largest Elements Across Sorted Lists	ColorStack	2025-10-31
M	246. Sort a nearly sorted (or K sorted) array.	Exponent	no date
H	247. Merge k sorted linked lists.	Exponent	no date
H	248. Return top-k subset sums in descending order	PracHub	2026-01-07

CODE TEMPLATE | GROUP 35 OF 43 | 5 QUESTIONS, ONE PATTERN

K-way merge: a heap of the next item from each source

The one idea these questions share

Put the first item of every list (with its list number and position) on a heap. Pop the smallest, record it, and push the next item from the same list. After k pops you have the k best without sorting everything.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import heapq


def k_way(lists, k):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]   # CHANGE 1: what you seed with
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        val, i, j = heapq.heappop(heap)
        out.append(val)
        if j + 1 < len(lists[i]):                                    # CHANGE 2: the successor
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out
```

2. Helper functions that usually get written next to it

```
def merge_two(a, b):                      # the simple two-list case of the same idea
    i = j = 0
    out = []
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i] <= b[j]):
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
244	Find k pairs with smallest sums	seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)
245	K Largest Elements Across Sorted Lists	seed with the LAST item of each list (negated); the successor is the item before it; reverse the result
246	Sort a nearly sorted (or K sorted) array.	keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)
247	Merge k sorted linked lists.	seed with each list head (value, list number, node); the successor of a node is node.next
248	Return top-k subset sums in descending order	heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 244 of 288 | topic 35 of 43: K-way merge: a heap of the next item from each source | 1 of 5

Template fit: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1) | asked 353 days ago

EASY | IN APP LC 373 | PracHub | Technical Screen | 244 of 288

Find k pairs with smallest sums

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 139/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #373 Find K Pairs with Smallest Sums

LeetCode match: LC 373 Find K Pairs with Smallest Sums

Posted on PracHub: 2025-11-01 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-k-pairs-with-smallest-sums

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **`k pairs (arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17 The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

EXAMPLES

QUESTION

Example 1	
Input	`[1, 7, 11], [2, 4, 6], 3`
Output	`[(1, 2), (1, 4), (1, 6)]`
Why	`Pairs (one from each array) in order of their sum.`

Example 2	
Input	`[1, 1, 2], [1, 2, 3], 2`
Output	`[(1, 1), (1, 1)]`
Why	`Equal numbers give equal pairs.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, b, k):
    pairs = [(x, y) for x in a for y in b]
    pairs.sort(key=lambda p: p[0] + p[1])
    return pairs[:k]
```

TIME	SPACE
O(m * n log(m * n))	O(m * n)

WHY IT IS SLOW

m * n pairs sorted: with 10^5 elements in each array that is 10^10 pairs. Because both arrays are sorted, a min-heap that starts from each x's first pair and expands only the next pair in that row is O(k log k).

OPTIMAL SOLUTION · Min-heap over the sorted rows

TIME	SPACE
O(k log min(k, m))	O(min(k, m))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

FOLLOW-UP 2 Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

FOLLOW-UP 3 Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. **Cost:** O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find k pairs with smallest sums
#
# You are given two **sorted** integer arrays arr1 and
# arr2 (each sorted in non-decreasing order), and an
# integer k. Consider all possible pairs formed by taking
# one element from arr1 and one element from arr2. For
# indices i and j, the pair is (arr1[i], arr2[j]) with sum
# arr1[i] + arr2[j]. Your task is to return the **k
# pairs** (arr1[i], arr2[j]) with the smallest sums. If
# there are fewer than k total possible pairs, return all
# possible pairs. You may return the pairs in any order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The arrays are not sorted.
# 2. Only the k-th smallest sum, with huge arrays.
# 3. Three arrays, or k pairs with the largest sums.
#
# INTERVIEW NOTES
# Because both arrays are sorted, the pair (i, j + 1) is
# never smaller than (i, j), so each row i of the 'matrix
# of sums' is sorted. The k smallest pairs are the k
# smallest items of a collection of sorted rows, which is
# a heap problem: start with the first element of each row
# (or the first few) and each time you pop a pair, push
# its successor in the row.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
from bisect import bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (a[i] + b[0], i, 0) for the first min(k, m)
# values of a. Pop the smallest, record the pair, and push
```

```
# the next pair in the same row, (a[i] + b[j + 1], i, j +
# 1). After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    # a and b sorted: a heap holds the best unused pair of
    # each row i; popping (i, j) offers (i, j + 1)
    if not a or not b or k <= 0:
        return []
    rows = range(min(k, len(a)))
    heap = [(a[i] + b[0], i, 0) for i in rows]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            nxt = a[i] + b[j + 1]
            heapq.heappush(heap, (nxt, i, j + 1))
    return out

# ---- main question: tests ----
def test_main_question():
    a, b = [1, 7, 11], [2, 4, 6]
    three = k_smallest_pairs(a, b, 3)
    assert three == [(1, 2), (1, 4), (1, 6)]
    assert k_smallest_pairs(a, b, 4)[3] == (7, 2)
    assert k_smallest_pairs([1, 1, 2], [1, 2, 3], 2) == [
        (1, 1), (1, 1)]
    assert k_smallest_pairs([], [1], 3) == []
    # k bigger than the number of pairs: all of them
    assert len(k_smallest_pairs([1], [1, 2], 10)) == 2


# ============================================================
# FOLLOW-UP 1: The arrays are not sorted.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort both first (O(m log m + n log n)); the heap
# method needs sorted rows. For one huge array you may
# instead use a heap of size k with a selection of
# candidates.
def k_smallest_pairs_unsorted(a, b, k):
    # the heap method needs sorted rows: sort first
    return k_smallest_pairs(sorted(a), sorted(b), k)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    fixed = k_smallest_pairs_unsorted([7, 1], [4, 2], 2)
    assert fixed == [(1, 2), (1, 4)]


# ============================================================
# FOLLOW-UP 2: Only the k-th smallest sum, with huge
# ============================================================

# ---- follow-up 2: helper functions ----
def count_at_most(a, b, limit):
    # pairs with sum <= limit: for every x binary search
    # in b
    return sum(bisect_right(b, limit - x) for x in a)

# ---- follow-up 2: function ----
# Answer: Binary search the sum value and count how many
# pairs have sum at most it with two pointers (O(m + n)
# per check): O((m + n) log(range)) with O(1) space, no
# heap.
def kth_smallest_sum(a, b, k):
    # huge arrays: binary search the SUM, no heap needed
    lo, hi = a[0] + b[0], a[-1] + b[-1]
    while lo < hi:
        mid = (lo + hi) // 2
        if count_at_most(a, b, mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

```
# ============================================================
# FOLLOW-UP 3: Three arrays, or k pairs with the largest
# ============================================================

# ---- follow-up 3: function ----
# Answer: For largest sums run the same algorithm on the
# reversed arrays (or negate the values). For three arrays
# use a heap of triples (i, j, l), pushing the three
# neighbours with a visited set.
def k_largest_pairs(a, b, k):
    # largest sums: negate the values and run the same
    # code
    neg_a = [-x for x in sorted(a, reverse=True)]
    neg_b = [-y for y in sorted(b, reverse=True)]
    pairs = k_smallest_pairs(neg_a, neg_b, k)
    return [(-x, -y) for x, y in pairs]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    a, b = [1, 7, 11], [2, 4, 6]
    assert k_largest_pairs(a, b, 2) == [(11, 6), (11, 4)]
    # the 4th smallest sum is 9 (7 + 2); check by sorting
    # all
    sums = sorted(x + y for x in a for y in b)
    for k in range(1, 10):
        assert kth_smallest_sum(a, b, k) == sums[k - 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 245 of 288 | topic 35 of 43: K-way merge: a heap of the next item from each source | 2 of 5

Template fit: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result | asked 354 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 245 of 288

K Largest Elements Across Sorted Lists

ASKED 2025-10-31 | 354 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 55.2 | 30 DAYS 33 | PRIORITY 30/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #23 Merge k Sorted Lists, LeetCode #215 Kth Largest Element in an Array

LeetCode match: LC 23 Merge k Sorted Lists / LC 215 (related: k largest across sorted lists)

Frequency: Single report | Last asked: 2025-10-31 | Roles: SWE Intern

QUESTION

You are given a list of inner lists, where each inner list is sorted in ascending order. Return the k largest elements taken from the union of all the inner lists, and return them sorted in ascending order. Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Function signature

```
from typing import List
def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Follow-up (as reported): Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop k times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

EXAMPLES

Example 1

Input	`lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4`
Output	`[6, 7, 8, 9]`
Why	`The four largest values overall are 9, 8, 7, 6, returned ascending.`

Example 2

Input	`lists = [[10, 20, 30]], k = 2`
Output	`[20, 30]`

QUESTION

Example 3

Input	`lists = [[5], [1, 3]], k = 5`
Output	`[1, 3, 5]`
Why	`k exceeds the total count, so all elements are returned sorted.`

Example 4

Input	`lists = [[1, 2], [3, 4]], k = 0`
Output	`[]`

CONSTRAINTS

- Let m be the number of inner lists and T the total number of elements across all of them.
- 0 <= m <= 10^4 , 0 <= T <= 10^6 .
- Each element is an integer in [-10^9, 10^9] .
- 0 <= k . If k == 0 return [] . If k >= T , return every element sorted ascending.
- Some inner lists may be empty.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_k_largest(lists, k):
    flat = sorted(
        x for lst in lists for x in lst
    )
    return (
        flat[max(0, len(flat) - k) :]
        if k
        else:
            []
    )
```

TIME	SPACE
O(N log N)	O(N)

WHY IT IS SLOW

It sorts all N elements although only k are needed.

OPTIMAL SOLUTION · A max-heap over the ends of the lists

TIME	SPACE
O(m + k log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are not sorted, or arrive as streams. What changes?

Keep a min-heap of the k largest values seen so far: push each value and pop the smallest when the heap has more than k items. O(N log k) time and O(k) memory, in one pass.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: K Largest Elements Across Sorted Lists
#
# You are given a list of inner lists, where each inner
# list is sorted in ascending order. Return the k largest
# elements taken from the union of all the inner lists,
# and return them sorted in ascending order. Duplicate
# values are kept (an element counts once per position it
# occupies). The interviewer first accepted a simple
# solution, then asked for an optimized one. Function
# signature: from typing import List def
# k_largest_sorted(lists: List[List[int]], k: int) ->
# List[int]: """Return the k largest elements across all
# (individually ascending) inner lists, in ascending
# sorted order.""" ... Follow-up (as reported): Optimize
# beyond concatenate-and-sort. Hint: because each list is
# already sorted, seed a max-heap with the last element of
# every non-empty list and pop k times, pushing the
# predecessor within a list after each pop; collect the
# popped values and reverse them.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are not sorted, or arrive as streams. What
# changes?
#
# INTERVIEW NOTES
# Sorting everything costs O(N log N). Because every list
# is already sorted, the largest remaining elements are at
# the ends of the lists: seed a max-heap with the last
# element of each list, pop the largest k times, and push
# the predecessor from the same list after each pop. That
# is O(k log m) for m lists after the seeding.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): A max-heap over the ends of the
# lists
# Idea: The next largest element is always at the end of
# one of the lists.
def k_largest_sorted(lists, k):
    heap = [
        (-lst[-1], i, len(lst) - 1)
        for i, lst in enumerate(lists)
        if lst
    ]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        neg, i, j = heapq.heappop(heap)
        out.append(-neg)
        if j:
            heapq.heappush(
                heap, (-lists[i][j - 1], i, j - 1)
            )
    return out[::-1]

# ---- main question: tests ----
def test_main_question():
    random.seed(77)
    assert k_largest_sorted(
        [[1, 4, 7], [2, 5, 8], [3, 6, 9]], 4
    ) == [6, 7, 8, 9]
    assert k_largest_sorted([[10, 20, 30]], 2) == [
        20,
        30,
    ]
    assert k_largest_sorted([[5], [1, 3]], 5) == [
        1,
        3,
        5,
    ]
    assert k_largest_sorted([[1, 2], [3, 4]], 0) == []


# ============================================================
# FOLLOW-UP 1: The lists are not sorted, or arrive as
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Keep a min-heap of the k largest values seen so
# far: push each value and pop the smallest when the heap
# has more than k items. O(N log k) time and O(k) memory,
# in one pass.
def k_largest_stream(lists, k):
    heap = []
    for lst in lists:
        for x in lst:
            if len(heap) < k:
                heapq.heappush(heap, x)
            elif k and x > heap[0]:
                heapq.heapreplace(heap, x)
    return sorted(heap)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 246 of 288 | topic 35 of 43: K-way merge: a heap of the next item from each source | 3 of 5

Template fit: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace) | no date

MEDIUM | Exponent | 246 of 288

★ TOP 75: rank 41 in the Medium and Hard priority list

★ Sort a nearly sorted (or K sorted) array.

ASKED (ANY CO.) ~A YEAR AGO | OLDER

STUDY: TYPE | ALL TIME 55.2 | 30 DAYS 33 | PRIORITY 31/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #215 Kth Largest Element in an Array, LeetCode #23 Merge k Sorted Lists

LeetCode match: No direct match; related LC 215 Kth Largest Element, LC 23 (inferred from short statement)

Last asked (any company): a year ago, about 2025-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/5392/k-sorted-array

QUESTION

In a k-sorted array every element is at most k positions away from where it would be in the sorted array. Sort such an array in O(n log k) time (better than a full sort when k is small).

Function signature

def sort_k_sorted(arr: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[6, 5, 3, 2, 8, 10, 9], 3`
Output	`[2, 3, 5, 6, 8, 9, 10]`
Why	`Every number is within 3 positions of its place in the sorted order.`

Example 2	
Input	`[2, 1, 4, 3], 1`
Output	`[1, 2, 3, 4]`
Why	`k = 1: neighbours may be swapped.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arr):
    return sorted(arr)  # ignores the K hint
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

O(n log n) regardless of K. Each element is at most K places from its sorted position, so a min-heap holding a window of K + 1 elements emits the sorted order in O(n log K).

OPTIMAL SOLUTION · Min-heap of size K + 1

TIME	SPACE
O(n log K)	O(K)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 K is not given. How do you find it?

Sort a copy with the original index as a tie-break and take the largest |original index - sorted index|; that is the smallest valid K. O(n log n), so only worth it if you will sort many similar arrays.

FOLLOW-UP 2 The data is a stream you cannot hold in memory.

The heap of K + 1 elements is already a streaming algorithm: it emits the next sorted element after each arrival once it holds K + 1 elements. Memory O(K).

FOLLOW-UP 3 K is close to n.

Then O(n log K) approaches O(n log n) and the heap has no advantage; use the library sort. State the crossover.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Sort a nearly sorted (or K sorted) array.
#
# In a k-sorted array every element is at most k positions
# away from where it would be in the sorted array. Sort
# such an array in O(n log k) time (better than a full
# sort when k is small).
#
# Signature:
#   def sort_k_sorted(arr: list, k: int) -> list
#
# Examples:
# [6, 5, 3, 2, 8, 10, 9], 3  ->  [2, 3, 5, 6, 8, 9, 10]
# [2, 1, 4, 3], 1  ->  [1, 2, 3, 4]
#
# FOLLOW-UPS (answered below, in this order):
# 1. K is not given. How do you find it?
# 2. The data is a stream you cannot hold in memory.
# 3. K is close to n.
#
# INTERVIEW NOTES
```

```
# If each element is at most K places from where it
# belongs, then the smallest element of the whole array
# must be among the first K + 1 elements. A min-heap
# holding a sliding window of K + 1 elements always has
# the next output on top, so you never need to sort the
# whole array.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Min-heap of size K + 1
# Idea: Push the first K + 1 elements. Then for each
# remaining element, pop the smallest into the output and
# push the new element. At the end pop the heap empty.
def sort_k_sorted(arr, k):
    # every item is at most k places from its final spot,
    # so
    # the smallest of the next k + 1 items is the next
    # answer. Keep k items in a heap, and for each new one
    # push it and pop the smallest: O(n log k).
    heap = list(arr[:k])
    heapq.heapify(heap)
    out = []
    for x in arr[k:]:
        # push the new item and pop the smallest in one go
        out.append(heapq.heappushpop(heap, x))
    while heap:
        out.append(heapq.heappop(heap))
    return out

# ---- main question: tests ----
def test_main_question():
    nearly = [6, 5, 3, 2, 8, 10, 9]
    assert sort_k_sorted(nearly, 3) == sorted(nearly)
    assert sort_k_sorted([], 2) == []
    assert sort_k_sorted([3, 1, 2], 5) == [1, 2, 3]
    assert sort_k_sorted([1, 2, 3], 0) == [1, 2, 3]


# ============================================================
# FOLLOW-UP 1: K is not given. How do you find it?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Sort a copy with the original index as a tie-
# break and take the largest |original index - sorted
# index|; that is the smallest valid K. O(n log n), so
# only worth it if you will sort many similar arrays.
def find_k(arr):
    # follow-up: the smallest valid k. Sort the indexes by
    # value (ties keep their order) and take the largest
    # distance between a position and its sorted position.
    order = sorted(range(len(arr)), key=arr.__getitem__)
    shifts = (abs(pos - i) for pos, i in enumerate(order))
    return max(shifts, default=0)
```

```
# ============================================================
# FOLLOW-UP 2: The data is a stream you cannot hold in
# ============================================================

# ---- follow-up 2: function ----
# Answer: The heap of K + 1 elements is already a
# streaming algorithm: it emits the next sorted element
# after each arrival once it holds K + 1 elements. Memory
# O(K).
def stream_sort(items, k):
    # follow-up: a stream that does not fit in memory. The
    # same heap, as a generator: memory is O(k).
    heap = []
    for x in items:
        heapq.heappush(heap, x)
        if len(heap) > k:
            yield heapq.heappop(heap)
    while heap:
        yield heapq.heappop(heap)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    nearly = [6, 5, 3, 2, 8, 10, 9]
    out = list(stream_sort(iter(nearly), 3))
    assert out == sorted(nearly)
    assert find_k([1, 2, 3]) == 0
    assert find_k([2, 1, 3]) == 1
    assert find_k([6, 5, 3, 2, 8, 10, 9]) == 3
    assert find_k([]) == 0


# ============================================================
# FOLLOW-UP 3: K is close to n.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Then O(n log K) approaches O(n log n) and the
# heap has no advantage; use the library sort. State the
# crossover.
def sort_any(arr, k):
    # follow-up: when k is close to n the heap is no
    # better
    # than the library sort, so use that
    if k >= len(arr) // 2:
        return sorted(arr)
    return sort_k_sorted(arr, k)
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    for _ in range(300):
        arr = [random.randint(0, 9)
               for _ in range(random.randint(0, 9))]
        k = find_k(arr)
        # k is valid, and one less is not (unless k is 0)
        assert sort_k_sorted(arr, k) == sorted(arr)
        assert sort_any(arr, k) == sorted(arr)
    # distinct values: compare with the plain definition
    for _ in range(300):
        size = random.randint(2, 8)
        arr = random.sample(range(30), size)
        k = find_k(arr)
        where = {v: i for i, v in enumerate(sorted(arr))}
        moves = enumerate(arr)
        far = max(abs(i - where[v]) for i, v in moves)
        assert k == far
        assert sort_k_sorted(arr, k) == sorted(arr)


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY PATH: question 247 of 288 | topic 35 of 43: K-way merge: a heap of the next item from each source | 4 of 5

Template fit: seed with each list head (value, list number, node); the successor of a node is node.next | no date

HARD | IN APP LC 23 | Exponent | 247 of 288

Merge k sorted linked lists.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 49.4 | 30 DAYS 0 | PRIORITY 81/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: your app's interview approach (LeetCode #23)

LeetCode match: LC 23 Merge k Sorted Lists

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +816 interviews

Original: https://www.aced.io/questions/1167/merge-k-sorted-linked-lists

QUESTION

You are given k linked lists, each sorted in ascending order. Merge them into one sorted linked list and return its head. (In the examples the linked lists are written as arrays.)

Function signature

def merge_k_heap(lists: list) -> ListNode

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 5], [1, 3, 4], [2, 6]]`
Output	`_to_list(merge_k_heap([mk([1, 4, 5]), mk([1, 3, 4]), mk([2, 6])])) -> [1, 1, 2, 3, 4, 4, 5, 6]`
Why	`The three sorted lists merge into one sorted list.`

Example 2	
Input	`lists = []`
Output	`_to_list(merge_k_heap([])) -> []`
Why	`No lists: an empty result.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(lists):
    values = []
    for head in lists:
        while head:
            values.append(head.val)
            head = head.next
    values.sort()
    dummy = tail = ListNode(0)
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next
```

BRUTE FORCE

TIME	SPACE
O(N log N)	O(N)

WHY IT IS SLOW

O(N log N) time and O(N) extra memory, ignoring that each list is already sorted. A min-heap of the k current heads is O(N log k) with O(k) memory.

OPTIMAL SOLUTION · Min-heap of the current heads

TIME	SPACE
O(N log k)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Merge the lists in pairs instead of using a heap.

Repeatedly merge neighbouring pairs with the two-list merge; there are log k rounds, each touching all N nodes: O(N log k) time and O(1) extra space (no heap).

FOLLOW-UP 2 The lists are too large for memory (files or streams).

The heap approach already works on streams: it holds one item per list. Read each input through an iterator and write the output as you go; this is the merge phase of an external sort (heapq.merge does it). **Cost:** O(N log k) time and O(k) memory; one sequential read per list.

FOLLOW-UP 3 Why not merge the lists one by one into a running result?

Merging list i into a result of size about i * (N / k) costs O(i * N / k), so the total is O(k * N), far worse than O(N log k).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def _to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

# ---- main question: main function ----
# BASE QUESTION (optimal): Min-heap of the current heads
# Idea: Push the head of each non-empty list as (value,
# list index, node). Pop the smallest, append it to the
# output, and push its successor. The list index is a tie-
# breaker so nodes are never compared.
def merge_k_heap(lists):
    # a heap holds the current head of every list; the
    # node
    # index breaks ties so nodes are never compared
    heap = [(n.val, i, n)
            for i, n in enumerate(lists) if n]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
```

```
        tail = node
        if node.next:
            nxt = node.next
            heapq.heappush(heap, (nxt.val, i, nxt))
    return dummy.next


# HELPER: merge two lists
def merge_two(a, b):
    dummy = tail = ListNode()
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next

# ---- main question: tests ----
def test_main_question():
    data = [[1, 4, 5], [1, 3, 4], [2, 6]]
    want = [1, 1, 2, 3, 4, 4, 5, 6]
    got = to_list(merge_k_heap([make(v) for v in data]))
    assert got == want


# ============================================================
# FOLLOW-UP 1: Merge the lists in pairs instead of using a
# ============================================================

# ---- follow-up 1: function ----
# Answer: Repeatedly merge neighbouring pairs with the
# two-list merge; there are log k rounds, each touching
# all N nodes: O(N log k) time and O(1) extra space (no
# heap).
```

```
def merge_k_pairs(lists):
    # no heap: merge neighbours in pairs, log k rounds,
    # O(1) space
    lists = [n for n in lists if n]
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
```

```
            if i + 1 < len(lists):
                pair = (lists[i], lists[i + 1])
                merged.append(merge_two(*pair))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    data = [[1, 4, 5], [1, 3, 4], [2, 6]]
    want = [1, 1, 2, 3, 4, 4, 5, 6]
    got = to_list(merge_k_pairs([make(v) for v in data]))
    assert got == want
    assert merge_k_heap([]) is None
    assert merge_k_pairs([None]) is None
    assert to_list(merge_k_heap([None, make([1])])) == [1]


# ============================================================
# FOLLOW-UP 2: The lists are too large for memory (files
# ============================================================

# ---- follow-up 2: function ----
# Answer: The heap approach already works on streams: it
# holds one item per list. Read each input through an
# iterator and write the output as you go; this is the
# merge phase of an external sort (heapq.merge does it).
def merge_streams(iterators):
    # lists too big for memory: stream them (heapq.merge
    # keeps
    # one item per input), the merge step of an external
    # sort
    return heapq.merge(*iterators)

# ---- follow-up 2: tests ----
def test_follow_up_2():
    data = [[1, 4, 5], [1, 3, 4], [2, 6]]
    want = [1, 1, 2, 3, 4, 4, 5, 6]
    got = list(merge_streams(iter(v) for v in data))
    assert got == want
    got = to_list(merge_two(make([1, 3]), make([2])))
    assert got == [1, 2, 3]
```

```
# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
def _from_list(vals):
    head = None
    for v in reversed(vals):
        head = ListNode(v, head)
    return head


def merge_by_sorting(lists):
    vals = sorted(
        v for head in lists for v in _to_list(head)
    )
    return _from_list(vals)
```

```
# HELPERS for the checks below
def make(values):
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

OFFICIAL PAGE TEXT

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.

STUDY PATH: question 248 of 288 | topic 35 of 43: K-way merge: a heap of the next item from each source | 5 of 5

Template fit: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next | asked 286 days ago

HARD | PracHub | Technical Screen | 248 of 288

Return top-k subset sums in descending order

POSTED 2026-01-07 | 286 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 194/288

ABOUT THIS QUESTION

Topic: K-way merge: a heap of the next item from each source

Optimal answer from: the interview site's own solution

LeetCode match: LC 2386 Find the K-Sum of an Array

Posted on PracHub: 2026-01-07 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/return-top-k-subset-sums-in-descending-order

QUESTION

Given a list of integers (possibly negative), return the k largest subset sums in descending order. The empty subset counts and has sum 0; different subsets with the same sum are listed separately.

Function signature

def top_k_sums(nums: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3], 4`
Output	`[6, 5, 4, 3]`
Why	`The eight subset sums are 6, 5, 4, 3, 3, 2, 1, 0; the top four are 6, 5, 4, 3.`

Example 2	
Input	`[3, -2, 1], 3`
Output	`[4, 3, 2]`
Why	`With a negative number: 3 + 1 = 4, then 3, then 3 + (-2) + 1 = 2.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    sums = []
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            sums.append(sum(subset))
    sums.sort(reverse=True)
    return sums[:k]
```

BRUTE FORCE

TIME	SPACE
O(2^n * n)	O(2^n)

WHY IT IS SLOW

2^n subsets and a sort of them: O(2^n * n) time and O(2^n) space, hopeless past about n = 20. A heap that grows the k best sums step by step is O(n log n + k log k).

OPTIMAL SOLUTION · Heap of the k smallest sums of absolute values

TIME	SPACE
O(n log n + k log k)	O(n + k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

FOLLOW-UP 2 k can be as large as 2^n.

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

FOLLOW-UP 3 Limit subsets to at most m elements.

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Return top-k subset sums in descending
# order
#
# Given a list of integers (possibly negative), return the
# k largest subset sums in descending order. The empty
# subset counts and has sum 0; different subsets with the
# same sum are listed separately.
#
# Signature:
#   def top_k_sums(nums: list, k: int) -> list
#
# Examples:
```

```
# [1, 2, 3], 4  ->  [6, 5, 4, 3]
# [3, -2, 1], 3  ->  [4, 3, 2]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsets themselves, not only the sums.
# 2. k can be as large as 2^n.
# 3. Limit subsets to at most m elements.
#
# INTERVIEW NOTES
# k is small compared with 2^n, so never enumerate all
# subsets. Turn the problem into 'k smallest sums of non-
# negative numbers': subtract everything from the sum of
# the positive values. Then grow answers in increasing
# order with a heap where each state has exactly two
# successors (extend with the next element, or swap the
# last element for the next one).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Heap of the k smallest sums of
```

```
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values.
def top_k_sums(nums, k):
    # Idea: start from the best subset (all positives).
    # Every
    # other subset is that one with some elements
    # "flipped",
    # which costs |x|. So the k best subsets are the k
    # cheapest sets of flips: a heap over sorted |x|.
    base = sum(x for x in nums if x > 0)
    cost = sorted(abs(x) for x in nums)
    out = [0]
    heap = [(cost[0], 0)] if cost else []
    while heap and len(out) < k:
        s, i = heapq.heappop(heap)
        out.append(s)
        if i + 1 < len(cost):
            # two successors, neither is ever cheaper:
            # add the next element ...
            heapq.heappush(heap, (s + cost[i + 1], i + 1))
            # ... or move the last flipped one up by one
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1))
    return [base - c for c in out[:k]]

# ---- main question: tests ----
def test_main_question():
    nums = [3, -1, 2, -4]
    assert top_k_sums(nums, 4) == [5, 4, 3, 2]
    assert top_k_sums([], 3) == [0]
    cases = ([5, -2, 7, 1], [-1, -2, -3], [0, 4, -4])
```

```
# ============================================================
# FOLLOW-UP 1: Return the subsets themselves, not only the
# ============================================================

# ---- follow-up 1: function ----
# Answer: Store each heap state as a node with (last
# index, parent node) instead of copying lists, and
# rebuild a subset by walking parents only for the k
# answers. That keeps pushes O(1).
```

```
def top_k_subsets(nums, k):
    # also return the subsets: states keep a link to their
    # parent instead of copying lists
    order = sorted(range(len(nums)),
                   key=lambda i: abs(nums[i]))
    cost = [abs(nums[i]) for i in order]
    best = {i for i in range(len(nums)) if nums[i] > 0}

    def subset(chain):
        flipped = set()
        while chain is not None:
            flipped.add(order[chain[0]])
            chain = chain[1]
        return sorted(nums[i] for i in best ^ flipped)

    base = sum(nums[i] for i in best)
    out = [(base, sorted(nums[i] for i in best))]
    heap = [(cost[0], 0, (0, None))] if cost else []
    while heap and len(out) < k:
        s, i, chain = heapq.heappop(heap)
        out.append((base - s, subset(chain)))
        if i + 1 < len(cost):
            heapq.heappush(heap, (s + cost[i + 1], i + 1,
                                  (i + 1, chain)))
            # replace the last flip i by i + 1
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1,
                                  (i + 1, chain[1])))
    return out[:k]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    nums = [3, -1, 2, -4]
    # the subsets add up to the sums
    for total, sub in top_k_subsets(nums, 4):
        assert sum(sub) == total
    totals = [t for t, _ in top_k_subsets(nums, 4)]
    assert totals == [5, 4, 3, 2]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(w: list[int], k: int) -> list[int]:
```

Sample call

```
solution([3, 2, 1], 4)
```

Test cases

Input	Expected
([3, 2, 1], 4)	[6, 5, 4, 3]
([5], 2)	[5, 0]

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 36 OF 43

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	4
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

E	249. Next Birthday (BST In-Order Successor with Wraparound)	ColorStack	2025-10-17
M	250. Lowest Common Ancestor	HackTheRounds	no date
M	251. Determine if a given binary tree is a binary search tree (BST).	Exponent	no date
M	252. Find mode in a trinary search tree	PracHub	2025-11-16
M	253. Next Upcoming Birthday in a Family Tree	ColorStack	2025-10-02

CODE TEMPLATE | GROUP 36 OF 43 | 5 QUESTIONS, ONE PATTERN

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

The one idea these questions share

Same recursive walk as Tier 1, but now the information also travels down: lower and upper bounds for a binary search tree, or the target nodes you are looking for. The BST rule (left < node < right) lets you go left or right instead of visiting everything.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def check(node, lo=float("-inf"), hi=float("inf")):
    if node is None:
        return True                          # CHANGE 1: base case
    if not lo < node.val < hi:               # CHANGE 2: the rule this node must obey
        return False
    return check(node.left, lo, node.val) and check(node.right, node.val, hi)


def descend(root, x):                        # BST search: one path, O(height)
    best, node = None, root
    while node:
        if node.val > x:                     # CHANGE 3: go left or right by the rule
            best, node = node.val, node.left
        else:
            node = node.right
    return best
```

2. Helper functions that usually get written next to it

```
def inorder(root):                           # left, node, right: sorted order for a BST
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right


def leftmost(root):                          # the smallest key (the wrap-around answer)
    while root.left:
        root = root.left
    return root.val
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
249	Next Birthday (BST In-Order Successor with Wraparound)	descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node
250	Lowest Common Ancestor	dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor
251	Determine if a given binary tree is a binary search tree (BST).	check(): pass lo and hi bounds down; fail when a value leaves its allowed window
252	Find mode in a trinary search tree	in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)
253	Next Upcoming Birthday in a Family Tree	walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today

Contents Start the questions >

Master contents (all topics)

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 249 of 288 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 1 of 5

Template fit: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node | asked 368 days ago | frequency Single report

EASY | IN APP LC 285 | ColorStack | technical interview | 249 of 288

Next Birthday (BST In-Order Successor with Wraparound)

ASKED 2025-10-17 | 368 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 240/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: your app's interview approach (LeetCode #285)

LeetCode match: LC 285 Inorder Successor in BST

Frequency: Single report | Last asked: 2025-10-17 | Roles: SWE Intern

QUESTION

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur. Each birthday is stored as an integer day-of-year in [1, 365] , and the tree is a valid BST keyed on that value. Given the root of the BST and an integer today , return the next upcoming birthday: the smallest key that is strictly greater than today . If no key is strictly greater than today , the calendar wraps around, so return the smallest key in the whole tree. If the tree is empty, return None .

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def next_birthday(root: Optional[TreeNode], today: int) ->
Optional[int]:
    """Return the smallest key strictly greater than today,
wrapping around
    to the smallest key if none is greater. Return None if the tree
is empty."""
    ...
```

Follow-up (as reported): If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: bisect_right .

EXAMPLES

QUESTION

Example 1

Input	`root = above, today = 25`
Output	`30`
Why	`Smallest key strictly greater than 25 is 30.`

Example 2

Input	`root = above, today = 40`
Output	`10`
Why	`No key > 40, so wrap around to the smallest key, 10.`

Example 3

Input	`root = above, today = 5`
Output	`10`
Why	`Smallest key greater than 5 is 10.`

Example 4

Input	`root = None, today = 100`
Output	`None`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 ; all node values are distinct and in [1, 365] .
- 1 <= today <= 365 .
- Aim for O(h) time where h is the tree height (do not sort all nodes if you can avoid it).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_next_birthday(root, today):
    keys = []
    def inorder(node):
        if node:
            inorder(node.left)
            keys.append(node.val)
            inorder(node.right)
    inorder(root)
    if not keys:
        return None
    return next(
        (k for k in keys if k > today), keys[0]
    )
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

It visits every node although the BST order points to the answer.

OPTIMAL SOLUTION · Descend the BST once

TIME	SPACE
O(h)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is not a BST. What now?

Collect every value, sort once, then binary-search with bisect_right for the first value above today and wrap to index 0. O(n log n) once, O(log n) per query.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Next Birthday (BST In-Order Successor
# with Wraparound)
#
# This is the "alarm / next-birthday" problem: from a set
# of birthdays stored in a binary search tree, find the
# next one that will occur. Each birthday is stored as an
# integer day-of-year in [1, 365] , and the tree is a
# valid BST keyed on that value. Given the root of the BST
# and an integer today , return the next upcoming
# birthday: the smallest key that is strictly greater than
# today . If no key is strictly greater than today , the
# calendar wraps around, so return the smallest key in the
# whole tree. If the tree is empty, return None .
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is not a BST. What now?
#
# INTERVIEW NOTES
# Walk down the BST once. At a node whose value is above
# today it is a candidate, so remember it and go left to
# look for a smaller candidate; otherwise go right. The
# last candidate is the successor. If there is none, the
# answer wraps to the leftmost node.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from bisect import bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Descend the BST once
# Idea: A node above today is a candidate; its left
# subtree may hold a closer one.
def next_birthday(root, today):
    if root is None:
        return None
    best, node = None, root
    while node:
        if node.val > today:
            best, node = node.val, node.left
        else:
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val

# ---- main question: tests ----
def test_main_question():
    root = TreeNode(20, TreeNode(10), TreeNode(30, None, T
reeNode(40)))
    assert next_birthday(root, 25) == 30
    assert next_birthday(root, 40) == 10
    assert next_birthday(root, 5) == 10
    assert next_birthday(None, 100) is None


# ============================================================
# FOLLOW-UP 1: The tree is not a BST. What now?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Collect every value, sort once, then binary-
# search with bisect_right for the first value above today
# and wrap to index 0. O(n log n) once, O(log n) per
# query.
def next_birthday_unsorted(root, today):
    keys = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        keys.append(node.val)
        stack += [
            c for c in (node.left, node.right) if c
        ]
    if not keys:
        return None
    keys.sort()
    i = bisect_right(keys, today)
    return keys[i % len(keys)]


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
```

```
run_all_tests()
```

STUDY PATH: question 250 of 288 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 2 of 5

Template fit: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor | no date

MEDIUM | IN APP LC 236 | HackTheRounds | Onsite | 250 of 288

Lowest Common Ancestor

STUDY: TYPE | ALL TIME 41.5 | 30 DAYS 0 | PRIORITY 101/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #236 Lowest Common Ancestor of a Binary Tree

LeetCode match: LC 236

QUESTION

Given the root of a binary tree and two of its nodes p and q, find their lowest common ancestor: the lowest node that has both p and q as descendants (a node can be a descendant of itself).

Function signature

def lowestCommonAncestor_tree(root: BNode, p: BNode, q: BNode) -> BNode

EXAMPLES

Example 1

Input	`root = [3, [5, [6], [2]], [1]]`
	`(value, left, right)`
	`first query: p = 5, q = 1`
	`second query: p = 5, q = 6`

Output	`lowestCommonAncestor_tree(root, p, q).val -> 3` `lowestCommonAncestor_tree(root, p, p.left).val -> 5`
Why	`5 and 1 are in different subtrees of 3, so 3 is their lowest common ancestor. 5 is an ancestor of 6, and a node may be its own ancestor, so the answer is 5.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def path_to(root, target):
    if root is None:
        return None
    if root is target:
        return [root]
    for child in root.children:
        sub = path_to(child, target)
        if sub:
            return [root] + sub
    return None
def lca(root, p, q):
    path_p = path_to(root, p)
    path_q = path_to(root, q)
    answer = None
    for a, b in zip(path_p, path_q):
        if a is not b:
            break
        answer = a
    return answer
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Two full paths stored, O(n) space, and two traversals. One recursive pass that reports whether each subtree contains p or q finds the answer in O(n) time and O(h) space.

OPTIMAL SOLUTION · One recursive pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is n-ary.

The same recursion: collect the non-empty results from all children; if two or more children reported a hit this node is the answer, otherwise pass the single result (or None) up.

FOLLOW-UP 2 A very deep tree, or no recursion allowed.

Do one traversal that records each node's parent (a dict), then use the ancestor-set method from the parent-pointer version. O(n) time and space, no recursion.

FOLLOW-UP 3 p or q might not be in the tree.

The recursive answer is wrong in that case (it returns the one that exists). Count how many of p and q were seen during the DFS, or first check both exist, and return None if not.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = (
            val,
            left,
            right,
        )

# ---- main question: main function ----
# BASE QUESTION (optimal): One recursive pass
# Idea: dfs(node) returns node if node is p or q,
# otherwise combines its children: if both children return
# something, node is the LCA; if only one does, pass that
# up.
# FOLLOW-UP 1: The tree is n-ary.
# Answer: The same recursion: collect the non-empty
# results from all children; if two or more children
# reported a hit this node is the answer, otherwise pass
# the single result (or None) up.
def lca(root, p, q):
    # each call reports where p or q was found below. A
    # node
    # with hits from two different children (or that is p
    # or
    # q itself) is the answer; a single hit is passed up.
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        # FOLLOW-UP 1: n-ary: collect hits from every
        # child
        hits = [r for r in map(find, node.children) if r]
        if len(hits) >= 2:
            return node
        return hits[0] if hits else None

    return find(root)

# ---- main question: tests ----
def test_main_question():
    six, four = Node(6), Node(4)
    two = Node(2, [Node(7), four])
    five = Node(5, [six, two])
    one = Node(1, [Node(0), Node(8)])
    root = Node(3, [five, one])
    assert lca(root, six, four) is five
    assert lca(root, five, four) is five
    assert lca(root, six, one) is root
    # an n-ary tree: three children under one node
    kids = [Node(i) for i in range(3)]
    top = Node("top", [Node("mid", kids), Node("side")])
    assert lca(top, kids[0], kids[2]).val == "mid"


# ============================================================
# FOLLOW-UP 1: The tree is n-ary.
# ============================================================
# Its code sits in the main question section above, because the main code calls it: look for the lines marked FOLLOW-UP 1.

# ============================================================
# FOLLOW-UP 2: A very deep tree, or no recursion allowed.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Do one traversal that records each node's parent
# (a dict), then use the ancestor-set method from the
# parent-pointer version. O(n) time and space, no
# recursion.
def parents_of(root):
    # follow-up: one traversal with a stack (no recursion)
    # that records every node's parent
    parent, stack = {root: None}, [root]
    while stack:
        node = stack.pop()
        for child in node.children:
            parent[child] = node
            stack.append(child)
    return parent


# Answer: Do one traversal that records each node's parent
# (a dict), then use the ancestor-set method from the
# parent-pointer version. O(n) time and space, no
# recursion.
def lca_iterative(root, p, q):
    # climb from p collecting its ancestors, then climb
    # from
    # q until we meet one of them: O(n) time and space
    parent = parents_of(root)
    above = set()
    node = p
    while node:
        above.add(node)
        node = parent[node]
    node = q
    while node not in above:
        node = parent[node]
    return node

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # an n-ary tree: three children under one node
    kids = [Node(i) for i in range(3)]
    top = Node("top", [Node("mid", kids), Node("side")])
    found = lca_iterative(top, kids[0], kids[2])
    assert found.val == "mid"
    stranger = Node(404)


# ============================================================
# FOLLOW-UP 3: p or q might not be in the tree.
# ============================================================
```

```
# ---- follow-up 3: function ----
# Answer: The recursive answer is wrong in that case (it
# returns the one that exists). Count how many of p and q
# were seen during the DFS, or first check both exist, and
# return None if not.
def lca_checked(root, p, q):
```

```
    # follow-up: p or q may be missing from the tree. The
    # plain recursion would return the one that exists, so
    # first check that both are really in the tree.
    parent = parents_of(root)
    if p not in parent or q not in parent:
        return None
    return lca_iterative(root, p, q)

# ---- follow-up 3: tests ----
def test_follow_up_3():
    six, four = Node(6), Node(4)
    two = Node(2, [Node(7), four])
    five = Node(5, [six, two])
    one = Node(1, [Node(0), Node(8)])
    root = Node(3, [five, one])
    stranger = Node(404)
    assert lca_checked(root, six, stranger) is None
    assert lca_checked(root, six, four) is five
    # a very deep tree: the iterative version has no limit
    chain = Node(0)
    cur = chain
    for i in range(1, 5000):
        child = Node(i)
        cur.children.append(child)
        cur = child
    assert lca_iterative(chain, cur, chain) is chain
    for _ in range(100):
        # a random tree: every new node gets a random
        # parent
        nodes = [Node(0)]
        for i in range(1, random.randint(1, 10)):
            child = Node(i)
            random.choice(nodes).children.append(child)
            nodes.append(child)
        p, q = random.choice(nodes), random.choice(nodes)


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
def _path(root, target):
    stack = [(root, [root])]
    while stack:
        n, path = stack.pop()
        if n is target:
```

```
            return path
        for c in (n.left, n.right):
            if c:
                stack.append((c, path + [c]))
    return []


def lca_paths(root, p, q):
    a, b = _path(root, p), _path(root, q)
    lca = None
    for x, y in zip(a, b):
        if x is y:
            lca = x
        else:
            break
    return lca


# HELPERS for the checks below
```

```
class Node:
    def __init__(self, val, children=None):
        self.val = val
        # two children for a binary tree, any number for
        # n-ary
        self.children = children or []


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 251 of 288 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 3 of 5

Template fit: check(): pass lo and hi bounds down; fail when a value leaves its allowed window | no date

MEDIUM | IN APP LC 98 | Exponent | 251 of 288

Determine if a given binary tree is a binary search tree (BST).

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE | ALL TIME 36 | 30 DAYS 41.1 | PRIORITY 59/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: your app's interview approach (LeetCode #98)

LeetCode match: LC 98 Validate Binary Search Tree (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | +28 interviews

Original: https://www.aced.io/questions/1158/determine-binary-tree-bst

QUESTION

Given the root of a binary tree, determine whether it is a valid binary search tree: for every node, all values in its left subtree are smaller and all values in its right subtree are larger (strictly).

Function signature

def isValidBST(root: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[2, 1, 3]`
Output	`true`
Why	`2 with 1 on the left and 3 on the right.`

Example 2	
Input	`[5, 1, 4, null, null, 3, 6]`
Output	`false`
Why	`The 3 sits in the right subtree of 5 but is smaller than 5.`

Example 3	
Input	`[2, 2, 2]`
Output	`false`
Why	`A node equal to its parent breaks the strict rule.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_bst(root):
    values = inorder(root)
    for a, b in zip(values, values[1:]):
        if a >= b:
            return False
    return True
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but stores every value, O(n) space. Passing the allowed (low, high) range down the tree checks each node once with O(height) space and stops at the first violation.

OPTIMAL SOLUTION · Pass the allowed range (low, high) down

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Duplicates are allowed (all on the left, or all on the right).

Make one of the two bounds inclusive: for 'duplicates go left' every node must satisfy lo < val <= hi (the bound inherited from a left ancestor is inclusive, the one from a right ancestor is exclusive). Tested on trees built by inserting duplicates to the left.

FOLLOW-UP 2 Do it with an in-order walk that keeps only the previous value.

Walk in order with an explicit stack and compare each value with the previous one (strictly greater). O(h) space and early exit.

FOLLOW-UP 3 Recover a BST where exactly two nodes were swapped (LeetCode 99).

An in-order walk shows one or two descents; the first and last nodes involved in the descents are the swapped pair. O(n) time and O(h) space (O(1) with Morris traversal).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine if a given binary tree is a
# binary search tree (BST).
#
# Given the root of a binary tree, determine whether it is
# a valid binary search tree: for every node, all values
# in its left subtree are smaller and all values in its
# right subtree are larger (strictly).
#
# Signature:
#   def isValidBST(root: TreeNode) -> bool
#
# Examples:
# [2, 1, 3]  ->  true
# [5, 1, 4, null, null, 3, 6]  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. Duplicates are allowed (all on the left, or all on
# the right).
# 2. Do it with an in-order walk that keeps only the
# previous value.
# 3. Recover a BST where exactly two nodes were swapped
# (LeetCode 99).
#
# INTERVIEW NOTES
# Checking only a node against its two children is not
# enough; every node in the left subtree must be smaller
# than the node, not just the left child. Pass down the
# allowed open interval (low, high) for each subtree, or
# use the fact that an in-order walk of a BST is strictly
# increasing.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Pass the allowed range (low,
```

```
# high) down
# Idea: Each node must lie strictly between low and high.
# The left child inherits (low, node.val) and the right
# child (node.val, high). The root starts with (-inf,
# +inf).
# FOLLOW-UP 1: Duplicates are allowed (all on the left, or
# all on the right).
# Answer: Make one of the two bounds inclusive: for
# 'duplicates go left' every node must satisfy lo < val <=
# hi (the bound inherited from a left ancestor is
# inclusive, the one from a right ancestor is exclusive).
def is_bst(root, dup="none"):
    # FOLLOW-UP 1: dup="left" or "right" allows duplicates
    # every node must lie between the bounds from its
    # ancestors. dup says where an equal value may go:
    # "none" (no duplicates), "left" or "right".
    def fits(val, lo, hi):
        if lo is not None:
            if val < lo or (val == lo and dup != "right"):
                return False
        if hi is not None:
            if val > hi or (val == hi and dup != "left"):
                return False
        return True

    stack = [(root, None, None)]
    while stack:
        node, lo, hi = stack.pop()
        if node is None:
            continue
        if not fits(node.val, lo, hi):
            return False
        stack.append((node.left, lo, node.val))
        stack.append((node.right, node.val, hi))
    return True

# ---- main question: tests ----
def test_main_question():
    good = TreeNode(2, TreeNode(1), TreeNode(3))
    bad = TreeNode(5, TreeNode(1), TreeNode(4, TreeNode(3)
, TreeNode(6)))
    assert is_bst(good)
    assert not is_bst(bad)


# ============================================================
```

```
# FOLLOW-UP 1: Duplicates are allowed (all on the left, or
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: Do it with an in-order walk that keeps only
# ============================================================

# ---- follow-up 2: function ----
# Answer: Walk in order with an explicit stack and compare
# each value with the previous one (strictly greater).
# O(h) space and early exit.
def is_bst_inorder(root):
    # follow-up: an in-order walk must be strictly
    # increasing,
    # so only the previous value is kept (O(height) space)
    stack, node, prev = [], root, None
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if prev is not None and node.val <= prev:
            return False
        prev, node = node.val, node.right
    return True

# ============================================================
# FOLLOW-UP 3: Recover a BST where exactly two nodes were
# ============================================================

# ---- follow-up 3: function ----
# Answer: An in-order walk shows one or two descents; the
# first and last nodes involved in the descents are the
# swapped pair. O(n) time and O(h) space (O(1) with Morris
# traversal).
def recover(root):
    # follow-up (LeetCode 99): two nodes swapped their
```

```
    # values. In order, one or two "descents" show up; the
    # first node of the first and the second node of the
    # last
    # descent are the swapped pair.
    stack, node, prev = [], root, None
    first = second = None
    while stack or node:
```

```
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if prev is not None and node.val < prev.val:
            second = node
            if first is None:
                first = prev
        prev, node = node, node.right
    if first:
        first.val, second.val = second.val, first.val
    return root

# ======================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ======================================================
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def insert(root, val, dup="left"):
    # plain BST insertion (used to build test trees)
    if root is None:
        return TreeNode(val)
    cur = root
    while True:
        go_left = val < cur.val or (
            val == cur.val and dup == "left")
        side = "left" if go_left else "right"
        if getattr(cur, side) is None:
            setattr(cur, side, TreeNode(val))
            return root
        cur = getattr(cur, side)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Determine if a given binary tree is a valid Binary Search Tree (BST). HintA binary tree is considered a "valid BST" if for every node: All the values in its left subtree are strictly less than the node's value. All the values in its right subtree are strictly greater than the node's value. Both the left and right subtrees must also be valid BSTs. Examples

OFFICIAL PAGE TEXT

```
All trees defined in level-order
root = [5, 2, 6]
output: true

root = [5, 2, 4, null, null, 3, 6]
output: false

root = []
output: true
```

In these examples, the output is true if the given binary tree root is a valid BST, and false otherwise.Our solution to validate whether a binary tree is a Binary Search Tree (BST) uses a recursive approach. We traverse the tree while maintaining a range for valid node values (min and max). At each node, we check whether its value lies within the given range: If the node's value is not within the allowed range, we return False, indicating the tree is not a valid BST. If the node's value is valid, we recursively check its left and right subtrees: The left subtree must have values strictly less than the current node's value. The right subtree must have values strictly greater than the current node's value. For the left subtree, the range becomes (min, current_node_value). For the right subtree, the range becomes (current_node_value, max). We use nullptr to indicate unbounded ranges when there is no minimum or maximum restriction (e.g., for the root node or outermost nodes). If all nodes satisfy their respective conditions, we return True, confirming that the tree is a valid BST. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the binary tree. This is because we traverse each node exactly once, performing constant-time checks at each node. Space Complexity: The space complexity is O(h), where h is the height of the tree. This space is used by the

recursion stack. In the worst case (for an unbalanced tree), the height could be n, making the space complexity O(n). For a balanced tree, the height would be O(log n).No answers contributed by the community yet.

STUDY PATH: question 252 of 288 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 4 of 5

Template fit: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s) | asked 338 days ago

MEDIUM | IN APP LC 501 | PracHub | Technical Screen | 252 of 288

Find mode in a trinary search tree

POSTED 2025-11-16 | 338 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 158/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #501 Find Mode in Binary Search Tree

LeetCode match: LC 501 Find Mode in Binary Search Tree

Posted on PracHub: 2025-11-16 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/find-mode-in-a-trinary-search-tree

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val N ode middle; // all values = val Node right; // all val ues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

QUESTION

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

Hints
In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.
Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []

QUESTION

Explanation: The tree is empty, so there are no modes.
Input: ([-2147483648], [-1], [-1], [-1], 0)
Expected Output: [-2147483648]
Explanation: A single-node tree has that node's value as the only mode.
Hints
Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def modes(root):
    counts = Counter()
    stack = [root]
    while stack:
        node = stack.pop()
        if node:
            counts[node.val] += 1
            stack += [node.left, node.middle,
                      node.right]
    top = max(counts.values())
    return [v for v in counts
            if counts[v] == top]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but memory for every distinct value. In a valid tree an in-order walk (left, middle, right) gives equal values consecutively, so a running counter needs only O(height) space.

OPTIMAL SOLUTION · In-order traversal counting runs (O(height) extra space)

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

FOLLOW-UP 2 Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

FOLLOW-UP 3 The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. **Cost:** O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find mode in a trinary search tree
#
# You are given the root of a **trinary search tree**
# where each node has up to three children: text class
# Node { int val; Node left; // all values < val Node
# middle; // all values = val Node right; // all values >
# val } Assume the tree is intended to satisfy the
# ordering rules above. ## Task Return the **mode(s)** of
# the tree: the value(s) that occur most frequently among
# all nodes. - If there is one mode, return a single-
# element list. - If multiple values tie for highest
# frequency, return **all** such values (in any order). ##
# Constraints (typical) - Number of nodes n can be large
# (e.g., up to 10^5). - Values can be any 32-bit signed
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree may be broken (it does not respect the
# ordering).
# 2. Only O(1) extra space (no stack, no dictionary).
# 3. The tree is huge and values arrive in a stream.
#
# INTERVIEW NOTES
# In a valid search tree an in-order walk (left, middle,
# right) visits values in sorted order, so equal values
# are adjacent and you can count runs with a single
# running counter and O(height) memory instead of a
# dictionary. If the tree may be broken, the order
# guarantee is gone and you need a real frequency table.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter


# ============================================================
```

```
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): In-order traversal counting
# runs (O(height) extra space)
# Idea: Walk left, middle, right with an explicit stack.
def find_modes(root):
    # in-order: equal values are next to each other, so a
    # running count needs no dictionary
    best, modes, cur, run = 0, [], None, 0

    def visit(v):
        nonlocal best, modes, cur, run
        run = run + 1 if v == cur else 1
        cur = v
        if run > best:
            best, modes = run, [v]
        elif run == best and v not in modes:
            modes.append(v)

    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        visit(node.val)
        # the middle chain repeats node.val once per child
        mid = node.middle
        while mid:
            visit(mid.val)
            mid = mid.middle
        node = node.right
    return sorted(modes)

# ---- main question: tests ----
def test_main_question():
    #        5
    #     /  |  \
    #    3   5   8
    #   /|       |
    #  3 3       8
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    tree = TNode(5, threes, TNode(5),
                 TNode(8, None, TNode(8)))
    assert find_modes(tree) == [3]
```

```
# ============================================================
# FOLLOW-UP 1: The tree may be broken (it does not respect
# ============================================================

# ---- follow-up 1: function ----
# Answer: In-order no longer gives sorted values, so equal
# values need not be adjacent. Fall back to a frequency
# table over a full traversal (the Counter version): O(n)
# time, O(distinct values) space.
def find_modes_counter(root):
    # a broken tree (ordering not respected): count
    # everything
    counts, stack = Counter(), [root] if root else []
    while stack:
        node = stack.pop()
        counts[node.val] += 1
        for child in (node.left, node.middle, node.right):
            if child:
                stack.append(child)
    if not counts:
        return []
    top = max(counts.values())
    return sorted(v for v in counts if counts[v] == top)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    #        5
    #     /  |  \
    #    3   5   8
    #   /|       |
    #  3 3       8
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    tree = TNode(5, threes, TNode(5),
                 TNode(8, None, TNode(8)))
    assert find_modes_counter(tree) == [3]
    # a tie between 1 and 2
    tie = TNode(2, TNode(1), TNode(2), None)
    assert find_modes(tie) == [2]
    assert find_modes_counter(tie) == [2]
    assert find_modes(None) == []
    assert find_modes_counter(None) == []
    tie2 = TNode(2, TNode(1), None, TNode(3))
    assert find_modes(tie2) == [1, 2, 3]
```

```
# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class TNode:
    def __init__(self, val, left=None, middle=None,
                 right=None):
        # left < val, middle == val, right > val
        self.val, self.left = val, left
        self.middle, self.right = middle, right


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(values: list[int], left: list[int], middle: l
ist[int], right: list[int], root: int) -> list[int]:
```

Sample call

```
solution([], [], [], [], -1)
```

Test cases

Input	Expected
([], [], [], [], -1)	[]
([42], [-1], [-1], [-1], 0)	[42]

STUDY PATH: question 253 of 288 | topic 36 of 43: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up | 5 of 5

Template fit: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today | asked 383 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 253 of 288

Next Upcoming Birthday in a Family Tree

ASKED 2025-10-02 | 383 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 224/288

ABOUT THIS QUESTION

Topic: Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

Optimal answer from: written and tested here for this exact question

LeetCode match: Tree traversal with a cyclic date key (related: LC 1507 / next greater)

Frequency: Single report | Last asked: 2025-10-02 | Roles: SWE Intern

QUESTION

You are given a family tree rooted at "me". Each Person node has a name , a birthday given as (month, day) , and pointers to parent1 and parent2 (either may be None). Following parent pointers reaches the whole ancestry; treat the reachable set of Person nodes as the tree to search (each distinct person counted once, even if reachable by two paths). Given today's date (today_month, today_day) , implement alarm_next_birthday(me, today_month, today_day) that scans every person and returns the (month, day) of the soonest upcoming birthday, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (me is None), return the empty string "" . Use this ordering key to compare dates: key(m, d) = (m - 1) * 31 + d , and cyclic distance (key(person) - key(today)) mod 372 . Return the person with the smallest such distance; if several people share that soonest date, return that (month, day) .

QUESTION

Function signature

```
from typing import Optional, Union
class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2
def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

Follow-up (as reported): Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return "" .

EXAMPLES

Example 1

Input	`me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)`
Output	`(9, 25)`
Why	`keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10- 260) mod 372 = 122). Alice is soonest.`

Example 2

Input	`me = None; today = (5, 5)`
Output	`""`
Why	`No people to check.`

Example 3

Input	`me = Me(3,3) alone; today = (3, 3)`
Output	`(3, 3)`
Why	`The only birthday is today, distance 0.`

CONSTRAINTS

- 1 <= month <= 12 , 1 <= day <= 31 .
- Up to 10^5 people; parent pointers form a DAG (no cycles).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def key(month, day):
    return (month - 1) * 31 + day
def brute_alarm(me, tm, td):
    people, stack, seen = (
        [],
        [me] if me else [],
        set(),
    )
    while stack:
        p = stack.pop()
        if id(p) in seen:
            continue
        seen.add(id(p))
        people.append(p)
        stack += [
            q
            for q in (p.parent1, p.parent2)
            if q
        ]
    if not people:
        return ""
    people.sort(
        key=lambda p: (
            key(p.month, p.day) - key(tm, td)
        )
        % 372
    )
    return (people[0].month, people[0].day)
```

TIME	SPACE
O(p log p)	O(p)

WHY IT IS SLOW

It sorts all p people although only the nearest is needed.

OPTIMAL SOLUTION · One traversal tracking the minimum distance

TIME	SPACE
O(p)	O(p)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support repeated alarms: each call returns the next soonest birthday, then an empty string when all are used.

Collect the distinct (distance, month, day) values once, sort them, and keep an index: each call returns the next entry. If people can be added later, use a heap keyed by distance instead.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def key(month, day):
    return (month - 1) * 31 + day

# ---- main question: main function ----
# BASE QUESTION (optimal): One traversal tracking the
# minimum distance
# Idea: Walk every distinct ancestor once and keep the
# smallest cyclic distance.
class Person:
    def __init__(
        self,
        name,
        month,
        day,
        parent1=None,
        parent2=None,
    ):
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2


# BASE QUESTION (optimal): One traversal tracking the
# minimum distance
# Idea: Walk every distinct ancestor once and keep the
# smallest cyclic distance.
def alarm_next_birthday(me, today_month, today_day):
    if me is None:
        return ""
    today = key(today_month, today_day)
    best, answer = None, ""
    stack, seen = [me], set()
    while stack:
        p = stack.pop()
        if id(p) in seen:
            continue
        seen.add(id(p))
        dist = (key(p.month, p.day) - today) % 372
        if best is None or dist < best:
            best, answer = dist, (p.month, p.day)
        stack += [
            q for q in (p.parent1, p.parent2) if q
        ]
    return answer

# ---- main question: tests ----
def test_main_question():
    random.seed(36)
    alice, bob = Person("Alice", 9, 25), Person(
        "Bob", 1, 10
    )
    me = Person("Me", 12, 1, alice, bob)
    assert alarm_next_birthday(me, 9, 12) == (9, 25)
```

```
    assert alarm_next_birthday(None, 5, 5) == ""
    assert alarm_next_birthday(
        Person("Me", 3, 3), 3, 3
    ) == (3, 3)


# =====================================================
# FOLLOW-UP 1: Support repeated alarms: each call returns
# =====================================================

# ---- follow-up 1: function ----
# Answer: Collect the distinct (distance, month, day)
# values once, sort them, and keep an index: each call
# returns the next entry. If people can be added later,
# use a heap keyed by distance instead.
class BirthdayAlarms:
    def __init__(self, me, today_month, today_day):
        today = key(today_month, today_day)
        dates, stack, seen = (
            set(),
            [me] if me else [],
            set(),
        )
        while stack:
            p = stack.pop()
            if id(p) in seen:
                continue
            seen.add(id(p))
            dates.add(
                (
                    (key(p.month, p.day) - today) % 372,
                    p.month,
                    p.day,
                )
            )
            stack += [
                q for q in (p.parent1, p.parent2) if q
            ]
        self.order = sorted(dates)
        self.at = 0

    def next(self):
        if self.at >= len(self.order):
            return ""
        _, month, day = self.order[self.at]
        self.at += 1
        return (month, day)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    alice, bob = Person("Alice", 9, 25), Person(
        "Bob", 1, 10
    )
    me = Person("Me", 12, 1, alice, bob)
    alarms = BirthdayAlarms(me, 9, 12)
    assert [alarms.next() for _ in range(4)] == [
        (9, 25),
        (12, 1),
        (1, 10),
        "",
    ]


# =====================================================
# RUN EVERYTHING

# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 37 OF 43

Random pick without repeats: swap the chosen item with the last one and pop

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Random pick without repeats: swap the chosen item with the last one and pop | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	254. Random Generator Without Repeats	ColorStack	2025-10-10
M	255. Random Song Shuffler With a No-Repeat Window	ColorStack	2025-10-06
M	256. Design set with O(1) random access	PracHub	2026-01-01
M	257. Unique Random Number Generator	ColorStack	2025-10-03
H	258. Shuffle array using random integer API	PracHub	2025-12-08

CODE TEMPLATE | GROUP 37 OF 43 | 5 QUESTIONS, ONE PATTERN

Random pick without repeats: swap the chosen item with the last one and pop

The one idea these questions share

To pick a random item and never pick it again in O(1), keep the items in an array. Choose a random index, swap that item with the last one, pop the last, and fix its position in the index dict. The same trick gives O(1) delete and O(1) random access.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
import random


class PickPool:
    def __init__(self, items, seed=None):
        self.a = list(items)
        self.pos = {x: i for i, x in enumerate(self.a)}   # CHANGE 1: lookup by value (for delete)
        self.rng = random.Random(seed)

    def take_random(self):
        i = self.rng.randrange(len(self.a))
        x, last = self.a[i], self.a[-1]
        self.a[i] = last                                  # the last item fills the hole
        self.pos[last] = i
        self.a.pop()
        del self.pos[x]
        return x                                          # CHANGE 2: what you hand back
```

2. Helper functions that usually get written next to it

```
def fisher_yates(nums, rand_int):             # rand_int(lo, hi) is inclusive on both ends
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
254	Random Generator Without Repeats	the template as written: values 0..n-1, every call returns a new one; raise when empty
255	Random Song Shuffler With a No-Repeat Window	the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque
256	Design set with O(1) random access	add appends and records the position; delete swaps with the last item and pops; get_random picks a random index
257	Unique Random Number Generator	the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)
258	Shuffle array using random integer API	Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 254 of 288 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 1 of 5

Template fit: the template as written: values 0..n-1, every call returns a new one; raise when empty | asked 375 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 254 of 288

Random Generator Without Repeats

ASKED 2025-10-10 | 375 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 88/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 384 Shuffle an Array / LC 380 (related: no-repeat generator)

Frequency: Single report | Last asked: 2025-10-10 | Roles: SWE Intern

QUESTION

Design a class RandomGenerator initialized with a positive integer n . Each call to generate() returns one integer chosen uniformly at random from the values in [0, n-1] that have not yet been returned. After n calls every value in [0, n-1] has been returned exactly once; a further call must raise IndexError (the pool is empty). Each generate() call must run in O(1) time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Function signature

```
import random
class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...
    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Follow-up (as reported): Add a reset() that makes all n values available again in O(1) amortized.

EXAMPLES

QUESTION

Example 1

Input	`g = RandomGenerator(3); [g.generate() for _ in range(3)]`
Output	`e.g. [2, 0, 1]`
Why	`The three calls return a permutation of {0, 1, 2} with no repeats; any of the 6 permutations is a valid output. A 4th call would raise IndexError.`

Example 2

Input	`g = RandomGenerator(1); g.generate()`
Output	`0`
Why	`Only one value exists.`

CONSTRAINTS

- 1 <= n <= 10**6 .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
import random
class ShuffleOnce:
    def __init__(self, n):
        self.items = list(range(n))
        random.shuffle(self.items)
    def generate(self):
        return self.items.pop()
```

TIME	SPACE
O(n) setup, O(1) per call	O(n)

WHY IT IS SLOW

A reset needs a new O(n) shuffle, and the whole permutation is fixed in advance.

OPTIMAL SOLUTION · Partial Fisher-Yates with an active region

TIME	SPACE
O(1) per call	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add reset() so all n values are available again in O(1).

Set size back to n. The array still contains every value 0..n-1 (only their order changed), and the next draws are uniform over the active region, so nothing needs to be rebuilt.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Random Generator Without Repeats
#
# Design a class RandomGenerator initialized with a
# positive integer n . Each call to generate() returns one
# integer chosen uniformly at random from the values in
# [0, n-1] that have not yet been returned. After n calls
# every value in [0, n-1] has been returned exactly once;
# a further call must raise IndexError (the pool is
# empty). Each generate() call must run in O(1) time. The
# intended approach maintains an array of the remaining
# values and, on each call, swaps a randomly chosen
# remaining value to the end of the active region and
# shrinks that region (partial Fisher-Yates). Function
# signature: import random class RandomGenerator: def
# __init__(self, n: int) -> None: """Prepare to yield a
# random permutation of [0, n-1], one value per
# generate()."""  ... def generate(self) -> int: """Return
# a uniformly random not-yet-returned value in O(1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add reset() so all n values are available again in
# O(1).
#
# INTERVIEW NOTES
# Keep the values that are still available in the first
# `size` slots of an array. To draw, pick a random slot
# among them, hand out its value, swap it with the last
# active slot and shrink the active region: a partial
# Fisher-Yates shuffle, O(1) per call. Resetting is just
# setting size back to n, because the array still holds
# every value (in some order, which is as good as any).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Partial Fisher-Yates with an
# active region
# Idea: Swap the drawn value to the end of the active
# region and shrink it.
class RandomGenerator:
    def __init__(self, n, seed=None):
        self.n = n
        self.values = list(range(n))
        self.size = n
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError(
                "all values have been generated"
            )
        i = self.rng.randrange(self.size)
        self.size -= 1
        self.values[i], self.values[self.size] = (
            self.values[self.size],
            self.values[i],
        )
        return self.values[self.size]

# ---- main question: tests ----
def test_main_question():
    gen = RandomGenerator(5, seed=1)
    assert sorted(gen.generate() for _ in range(5)) == [0,
 1, 2, 3, 4]


# ============================================================
# FOLLOW-UP 1: Add reset() so all n values are available
# ============================================================

# ---- follow-up 1: function ----
# Answer: Set size back to n. The array still contains
# every value 0..n-1 (only their order changed), and the
# next draws are uniform over the active region, so
# nothing needs to be rebuilt.
class ResettableGenerator(RandomGenerator):
    def reset(self):
        self.size = self.n
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 255 of 288 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 2 of 5

Template fit: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque | asked 379 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 255 of 288

Random Song Shuffler With a No-Repeat Window

ASKED 2025-10-06 | 379 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 87/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1) (related: no-repeat window)

Frequency: Single report | Last asked: 2025-10-06 | Roles: SWE Intern

QUESTION

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the last k songs played may not be replayed. In other words, a song becomes eligible again only after k other (distinct) plays have occurred since it was last played. Implement the class so that each call to play() runs in O(1) time (no scanning of the playlist). The intended technique is a valid_songs list of currently eligible songs plus a song_to_index dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Function signature

```
import random
class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...
    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Follow-up (as reported): The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per play() .

EXAMPLES

QUESTION

Example 1

Input	`SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()`
Output	`e.g. "B", "A", "C", "B"`
Why	`k=1 forbids repeating the single most recent song. "B" then "A" is fine (A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is valid.`

Example 2

Input	`SongShuffler(["X", "Y"], k=0); play(), play()`
Output	`e.g. "Y", "Y"`
Why	`With k=0 there is no restriction, so repeats are allowed.`

CONSTRAINTS

- 1 <= len(songs) <= 10**5 , song titles are distinct strings.
- 0 <= k < len(songs) (so at least one song is always eligible). play() returns any eligible song chosen uniformly at random; every returned song must not be among the previous k returned songs.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
from collections import deque
class ScanShuffler:
    def __init__(self, songs, k, seed=None):
        self.songs, self.k = list(songs), k
        self.recent = deque()
        self.rng = random.Random(seed)
    def play(self):
        choices = [
            s
            for s in self.songs
            if s not in self.recent
        ]
        song = self.rng.choice(choices)
        self.recent.append(song)
        if len(self.recent) > self.k:
            self.recent.popleft()
        return song
```

TIME	SPACE
O(n) per play	O(n)

WHY IT IS SLOW

Every play scans the whole playlist to find the songs not in the window.

OPTIMAL SOLUTION · Eligible array, index map and swap-and-pop

TIME	SPACE
O(1) per play	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is a plain deque or list not enough for O(1)?

A deque of the last k songs is fine for the window, but choosing a random eligible song needs a structure you can sample from and delete from in O(1). A list gives O(1) sampling; deleting from the middle is O(n) unless you swap the element with the last one and pop, which needs the index map to know where a song is. That pair is what makes play O(1).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Random Song Shuffler With a No-Repeat
# Window
#
# Design a shuffler that plays songs at random from a
# fixed playlist, subject to one rule: the last k songs
# played may not be replayed. In other words, a song
# becomes eligible again only after k other (distinct)
# plays have occurred since it was last played. Implement
# the class so that each call to play() runs in O(1) time
# (no scanning of the playlist). The intended technique is
# a valid_songs list of currently eligible songs plus a
# song_to_index dictionary, using swap-and-pop to move a
# just-played song out of the eligible region and to re-
# admit the song that is leaving the window. Function
# signature: import random class SongShuffler: def
# __init__(self, songs: list[str], k: int) -> None:
# """Initialize with the playlist and the no-repeat window
# size k.""" ... def play(self) -> str: """Return a
# uniformly random eligible song in O(1); it must not be
# one of the last k songs returned.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is a plain deque or list not enough for O(1)?
#
# INTERVIEW NOTES
# The songs allowed right now form a set that must support
# a uniform random pick, remove, and re-insert in O(1): an
# array of eligible songs plus a dictionary song -> index,
# using swap-and-pop for removal. The last k played songs
# wait in a queue; when a new song is played, the oldest
# one in the queue becomes eligible again.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION

# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Eligible array, index map and
# swap-and-pop
# Idea: Remove the played song from the eligible array by
# swapping it with the last element; the song leaving the
# window is appended back.
class SongShuffler:
    def __init__(self, songs, k, seed=None):
        self.valid = list(songs)
        self.index = {
            s: i for i, s in enumerate(self.valid)
        }
        self.recent = deque()
        self.k = k
        self.rng = random.Random(seed)

    def play(self):
        i = self.rng.randrange(len(self.valid))
        song = self.valid[i]
        last = self.valid[-1]
        # swap-and-pop: the last song takes the hole
        self.valid[i] = last
        self.index[last] = i
        self.valid.pop()
        del self.index[song]
        self.recent.append(song)
        if len(self.recent) > self.k:
            back = self.recent.popleft()
            self.index[back] = len(self.valid)
            self.valid.append(back)
        return song

# ---- main question: tests ----
def test_main_question():
    random.seed(49)
    tiny = SongShuffler(["A", "B", "C"], 1, seed=3)
    seq = [tiny.play() for _ in range(40)]
    assert all(a != b for a, b in zip(seq, seq[1:]))
    assert (
        len(SongShuffler(["X", "Y"], 0, seed=1).valid)
        == 2
    )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 256 of 288 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 3 of 5

Template fit: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index | asked 292 days ago

MEDIUM | IN APP LC 380 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 256 of 288

Design set with O(1) random access

POSTED 2026-01-01 | 292 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 85/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1)

Posted on PracHub: 2026-01-01 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/design-set-with-o-1-random-access

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.

QUESTION

- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom'
    ,'add','getRandom'], [[1],[2],[],[1],[],[2],[]],
    7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','g
    etRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True,
    None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
items = []
def add(x):
    if x in items:
        return False
    items.append(x)
    return True
def delete(x):
    if x not in items:
        return False
    items.remove(x)
    return True
def get_random():
    return random.choice(items)
```

TIME	SPACE
O(n) for add and delete, O(1) getRandom	**O(n)**

WHY IT IS SLOW

Membership checks and deletion are O(n). An array plus a dictionary from value to index, deleting by swapping with the last element, makes all three operations O(1).

OPTIMAL SOLUTION · Array of values plus a dict from value to index

TIME	SPACE
O(1) average per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

FOLLOW-UP 2 What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

FOLLOW-UP 3 Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Design set with O(1) random access
#
# Design a data structure ("FancySet") that stores unique
# integers and supports the following operations, each in
# average O(1) time: - add(x): Insert value x if it is not
# already present. Return whether the insert happened. -
# delete(x): Remove value x if it exists. Return whether a
# deletion happened. - getRandom(): Return a uniformly
# random element currently in the set. Requirements /
# follow-ups: - All operations should be average O(1). -
# Handle corner cases such as calling getRandom() on an
# empty set (define and justify behavior). - Discuss how
# you would make this data structure thread-safe. Compare
# using a single lock for the whole set vs finer-grained
# locking (e.g., element-level locks), and explain trade-
# offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make it thread-safe: one lock or finer-grained
# locking?
# 2. What should getRandom do on an empty set?
# 3. Duplicates allowed (a multiset, LeetCode 381).
#
# INTERVIEW NOTES
# A hash set gives O(1) add and delete but cannot pick a
# random element in O(1); an array can pick a random
# element but deleting from the middle is O(n). Combine
# them: keep the values in an array and a dict from value
# to array index. To delete, swap the element with the
# last one, pop the end, and update the moved element's
# index.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random
import threading



# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key.
class FancySet:
    """add, delete and getRandom, each O(1) on average."""

    def __init__(self, rng=None):
        # the values in a list (for random access) and a
```

```
        # map
        # from value to its index in that list
        self.items, self.pos = [], {}
        self.rng = rng or random

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True

    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        # move the last item into the hole, then drop the
        # tail
        self.items[i] = last
        self.pos[last] = i
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        if not self.items:
            # say what happens on an empty set
            raise IndexError("empty set")
        i = self.rng.randrange(len(self.items))
        return self.items[i]
```

```
# ---- main question: tests ----
def test_main_question():
    s = FancySet(random.Random(1))
    assert s.add(1) and s.add(2) and not s.add(1)
    assert s.delete(1) and not s.delete(1)
    assert s.get_random() == 2
    s.delete(2)
    try:
        s.get_random()
        raise AssertionError("empty set should raise")
    except IndexError:
        pass
    # every value is picked about equally often
    big = FancySet(random.Random(5))
    for v in range(4):
        big.add(v)
    picks = [big.get_random() for _ in range(4000)]
    for v in range(4):
        assert 850 < picks.count(v) < 1150
    # deleting from the middle keeps the map consistent
    for v in (0, 2):
        big.delete(v)
    assert sorted(big.items) == [1, 3]
    for v, i in big.pos.items():
        assert big.items[i] == v


# ============================================================
# FOLLOW-UP 1: Make it thread-safe: one lock or finer-
# ============================================================

# ---- follow-up 1: function ----
# Answer: One lock around every operation is simplest and
# correct (delete touches the array, the dict and a second
# element's index together, so they must change as one
# unit).
class SafeSet(FancySet):
    """One lock: delete changes three things at once."""

    def __init__(self, rng=None):
        super().__init__(rng)
        self.lock = threading.Lock()
```

```
    def add(self, x):
        with self.lock:
```

```
            return super().add(x)

    def delete(self, x):
        with self.lock:
            return super().delete(x)

    def get_random(self):
        with self.lock:
            return super().get_random()

# ---- follow-up 1: tests ----
def test_follow_up_1():
    safe = SafeSet()
    assert safe.add(7) and safe.get_random() == 7


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Duplicates allowed (a multiset, LeetCode
# ============================================================

# ---- follow-up 3: function ----
# Answer: Map each value to a set of indices in the array;
# delete swaps with the last element and fixes both index
# sets. getRandom stays a uniform pick over array
# positions, so values are chosen in proportion to their
# counts.
class RandomizedCollection:
    """Duplicates allowed (LeetCode 381)."""

    def __init__(self):
        self.items = []
        # value -> set of indices where it sits in the
        # list
        self.where = {}

    def insert(self, x):
        first = x not in self.where or not self.where[x]
        spots = self.where.setdefault(x, set())
        spots.add(len(self.items))
        self.items.append(x)
```

```
        return first

    def remove(self, x):
        if not self.where.get(x):
            return False
        i = self.where[x].pop()
        end = len(self.items) - 1
        if i != end:
            # move the last item into the hole and fix its
            # index set
            last = self.items[end]
            self.items[i] = last
            self.where[last].discard(end)
            self.where[last].add(i)
        self.items.pop()
        return True

    def get_random(self):
        return random.choice(self.items)

# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    rc = RandomizedCollection()
    assert rc.insert(5)
    assert not rc.insert(5)
    assert rc.insert(6)
    assert rc.remove(5) and rc.remove(5)
    assert not rc.remove(5)
    assert rc.items == [6]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[str], values: list[list[int]], seed: int) -> str:
```

Sample call

```
solution(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
```

Test cases

Input	Expected
`(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)`	`[True, True, 1, True, 2, False, 2]`
`(['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)`	`[None, True, False, False, True, None]`

STUDY PATH: question 257 of 288 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 4 of 5

Template fit: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls) | asked 382 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 257 of 288

Unique Random Number Generator

ASKED 2025-10-03 | 382 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 187/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 384 Shuffle an Array / lazy Fisher-Yates (related)

Frequency: Single report | Last asked: 2025-10-03 | Roles: SWE Intern

QUESTION

Design a class UniqueRandom initialized with an integer n . Each call to generate() returns a number drawn uniformly at random from [0, n-1] that has not been returned before, so across successive calls no value ever repeats. After all n distinct values have been produced, further calls return None . Every unused value must remain equally likely on each draw (no bias), and generate() must run in O(1) time per call.

Function signature

```
import random
class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...
    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...
```

Follow-up (as reported): Use a partial Fisher-Yates shuffle: keep a virtual array 0..n-1 , and on the i -th call swap a random unused slot into position i using a sparse dict of moved entries, giving O(1) time and O(number-of- draws) space.

EXAMPLES

Example 1	
Input	`g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]`
Output	`a permutation of [0, 1, 2] (e.g. [2, 0, 1])`
Why	`The three calls return all of 0,1,2 in some random order with no repeats.`

QUESTION

Example 2	
Input	`(continuing above) g.generate()`
Output	`None`
Why	`All 3 values already returned, so the pool is exhausted.`

Example 3	
Input	`g = UniqueRandom(1); [g.generate(), g.generate()]`
Output	`[0, None]`
Why	`The only value 0 is returned once, then the generator is exhausted.`

CONSTRAINTS

- 1 <= n <= 10^7 .
- Support an optional seed for reproducible tests.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
import random
class ShuffledPool:
    def __init__(self, n, seed=None):
        self.items = list(range(n))
        random.Random(seed).shuffle(self.items)
    def generate(self):
        return (
            self.items.pop()
            if self.items
            else:
                None
        )
```

TIME	SPACE
O(n)	O(n)
setup, O(1) per call	

WHY IT IS SLOW

It stores all n values even if only a few are ever drawn.

OPTIMAL SOLUTION · Lazy Fisher-Yates with a dictionary

TIME	SPACE
O(1)	O(draws)
per call	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add a reset() so the same generator can run again.

Clear the dictionary and set the draw counter back to zero: O(draws) once, and all n values are available again.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Unique Random Number Generator
#
# Design a class UniqueRandom initialized with an integer
# n . Each call to generate() returns a number drawn
# uniformly at random from [0, n-1] that has not been
# returned before, so across successive calls no value
# ever repeats. After all n distinct values have been
# produced, further calls return None . Every unused value
# must remain equally likely on each draw (no bias), and
# generate() must run in O(1) time per call. Function
# signature: import random class UniqueRandom: def
# __init__(self, n: int, seed: int | None = None) -> None:
# ... def generate(self) -> int | None: """Return a not-
# yet-returned number in [0, n-1], or None once all n are
# exhausted.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a reset() so the same generator can run again.
#
# INTERVIEW NOTES
# Drawing without repeats is the Fisher-Yates shuffle done
# lazily: on draw i pick a random slot j in [i, n - 1],
# return the value that lives at j, and move the value at
# i into slot j. Only the slots that have been touched
# need storing, so a dictionary of moved values gives O(1)
# per draw and O(draws) memory even for a huge n.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Lazy Fisher-Yates with a
# dictionary
# Idea: Swap a random unused slot into position i,
```

```
# remembering only the moved slots.
class UniqueRandom:
    def __init__(self, n, seed=None):
        self.n = n
        self.drawn = 0
        self.moved = {}
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.n:
            return None
        i = self.drawn
        j = self.rng.randrange(i, self.n)
        value = self.moved.get(j, j)
        # the value that sat at slot i takes the place of
        # the one just drawn
        self.moved[j] = self.moved.get(i, i)
        self.drawn += 1
        return value

# ---- main question: tests ----
def test_main_question():
    gen = UniqueRandom(5, seed=1)
    assert sorted(gen.generate() for _ in range(5)) == [0,
 1, 2, 3, 4]
    assert gen.generate() is None


# ============================================================
# FOLLOW-UP 1: Add a reset() so the same generator can run
# ============================================================

# ---- follow-up 1: function ----
# Answer: Clear the dictionary and set the draw counter
# back to zero: O(draws) once, and all n values are
# available again.
class ResettableUniqueRandom(UniqueRandom):
    def reset(self):
        self.moved.clear()
        self.drawn = 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 258 of 288 | topic 37 of 43: Random pick without repeats: swap the chosen item with the last one and pop | 5 of 5

Template fit: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j] | asked 316 days ago

HARD | PracHub | Technical Screen | 258 of 288

Shuffle array using random integer API

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 184/288

ABOUT THIS QUESTION

Topic: Random pick without repeats: swap the chosen item with the last one and pop

Optimal answer from: the interview site's own solution

LeetCode match: LC 384 Shuffle an Array

Posted on PracHub: 2025-12-08 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/shuffle-array-using-random-integer-api

QUESTION

You have an array nums and no built-in shuffle, only rand_int(l, r), which returns a uniformly random integer in [l, r] (both ends included). Shuffle nums in place so that every one of the n! orders is equally likely.

Function signature

def shuffle(nums: list, rand_int) -> list

EXAMPLES

Example 1	

Input	`rand_int = lambda l, r: l # a fake 'random' source that always returns the lowest allowed index`
	`nums = [1, 2, 3, 4]`
	`shuffle(nums, rand_int)`

Output	`shuffle(nums, rand_int) -> [2, 3, 4, 1]`
Why	`Fisher-Yates walks i from the end and swaps nums[i] with nums[rand_int(0, i)]. With the fake source the swaps are (3,0), (2,0), (1,0), giving a predictable result; with a real source each of the 24 orders has probability 1/24.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def shuffle(nums):
    keyed = []
    for x in nums:
        keyed.append((rand_int(0, 10 ** 9), x))
    keyed.sort()
    return [x for _, x in keyed]
```

BRUTE FORCE

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

O(n log n) time and O(n) extra space, and equal random keys make the result very slightly non-uniform. Fisher-Yates (swap position i with a random position in 0..i) is O(n), in place and exactly uniform.

OPTIMAL SOLUTION · Fisher-Yates

TIME	SPACE
O(n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only k random elements.

Run Fisher-Yates for k steps from the front (swap position i with a random index in i..n-1) and return the first k. O(k) after copying, still uniform.

FOLLOW-UP 2 Only a coin-flip API exists, no rand_int.

Build a uniform integer in [0, m) by rejection sampling: draw ceil(log2 m) random bits and retry if the number is at least m. That keeps every value equally likely, unlike taking a remainder.

FOLLOW-UP 3 Why is the biased version biased?

It makes n choices of n options, n^n equally likely paths, and n! does not divide n^n for n >= 3, so some permutations must appear more often than others.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shuffle array using random integer API
#
# You have an array nums and no built-in shuffle, only
# rand_int(l, r), which returns a uniformly random integer
# in [l, r] (both ends included). Shuffle nums in place so
# that every one of the n! orders is equally likely.
#
# Signature:
#   def shuffle(nums: list, rand_int) -> list
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only k random elements.
```

```
# 2. Only a coin-flip API exists, no rand_int.
# 3. Why is the biased version biased?
#
# INTERVIEW NOTES
# 'Shuffle' means every one of the n! orders must be
# equally likely. Count outcomes: a procedure with n^n
# equally likely random choices cannot give n! equal
# buckets, so it is biased. Fisher-Yates makes n choices
# with sizes n, n-1, ..., 1, which multiply to exactly n!.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def shuffle_by_keys(nums):
    keyed = sorted(
        (
            (rand_int(0, 10**9), i)
            for i in range(len(nums))
        )
    )
    return [nums[i] for _, i in keyed]

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Fisher-Yates
# Idea: Walk i from n - 1 down to 1, pick j uniformly from
# 0..i (inclusive) and swap nums[i] with nums[j].
def shuffle(nums, rand_int):
    # Fisher-Yates: from the end, swap each position with
    # a
    # random one at or before it. All n! orders are
    # equally
    # likely because each step picks uniformly among i +
    # 1.
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums

# ---- main question: tests ----
def test_main_question():
    lowest = lambda low, high: low
    assert shuffle([1, 2, 3, 4], lowest) == [2, 3, 4, 1]


# ============================================================
# FOLLOW-UP 1: Return only k random elements.
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run Fisher-Yates for k steps from the front
# (swap position i with a random index in i..n-1) and
# return the first k. O(k) after copying, still uniform.
def sample_k(nums, k, rand_int):
    # k random elements: only k Fisher-Yates steps, from
    # the front
    a = list(nums)
    for i in range(min(k, len(a))):
        j = rand_int(i, len(a) - 1)
        a[i], a[j] = a[j], a[i]
    return a[:k]

# ============================================================
# FOLLOW-UP 2: Only a coin-flip API exists, no rand_int.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Build a uniform integer in [0, m) by rejection
# sampling: draw ceil(log2 m) random bits and retry if the
# number is at least m. That keeps every value equally
# likely, unlike taking a remainder.
def rand_below(m, flip):
    # only a coin flip exists: rejection sampling gives a
    # uniform number in [0, m) (a remainder would be
    # biased)
    bits = max(1, (m - 1).bit_length())
    while True:
        x = 0
        for _ in range(bits):
            x = x * 2 + flip()
        if x < m:
            return x

# ============================================================
# FOLLOW-UP 3: Why is the biased version biased?
# ============================================================

# ---- follow-up 3: function ----
# Answer: It makes n choices of n options, n^n equally
# likely paths, and n! does not divide n^n for n >= 3, so
# some permutations must appear more often than others.
def biased_shuffle(nums, rand_int):
    # the common WRONG version: n^n paths cannot be spread
    # over
    # n! orders evenly (shown only to explain the bias)
    for i in range(len(nums)):
        j = rand_int(0, len(nums) - 1)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], random_indices: list[int]) -
> list[int]:
```

Sample call

```
solution([1, 2, 3, 4], [1, 1, 0])
```

Test cases

Input	Expected
([1, 2, 3, 4], [1, 1, 0])	[3, 1, 4, 2]
([], [])	[]

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 38 OF 43

Build and serialize: write a structure out, read it back with the same walk

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3	2
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Build and serialize: write a structure out, read it back with the same walk | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	259. Construct a binary tree given its preorder and inorder traversals.	Exponent	no date
M	260. Write functions to serialize and deserialize a list of strings.	Exponent	no date
M	261. Print a Directory Tree	ColorStack	2025-09-21
H	262. Implement a decode and encode function of a general N-ary tree.	Exponent	no date
H	263. Serialize and Deserialize a Graph	ColorStack	2025-09-24

CODE TEMPLATE | GROUP 38 OF 43 | 5 QUESTIONS, ONE PATTERN

Build and serialize: write a structure out, read it back with the same walk

The one idea these questions share

Encoding is one walk that writes tokens; decoding is the same walk reading tokens back (an iterator or an index). Put a length or a child count before the data so the reader always knows how much to take.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def encode(node, out):                      # CHANGE 1: what you write per node
    if node is None:
        out.append("#")
        return
    out.append(str(node.val))
    encode(node.left, out)
    encode(node.right, out)


def decode(tokens):
    it = iter(tokens)

    def build():
        tok = next(it)
        if tok == "#":
            return None
        node = Node(int(tok))               # CHANGE 2: build the node from what you read
        node.left = build()
        node.right = build()
        return node

    return build()
```

2. Helper functions that usually get written next to it

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def read_tokens(text):                      # the reader side of a length-prefixed format
    i, out = 0, []
    while i < len(text):
        j = text.index("#", i)
        n = int(text[i:j])
        out.append(text[j + 1:j + 1 + n])
        i = j + 1 + n
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
259	Construct a binary tree given its preorder and inorder traversals.	the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right
260	Write functions to serialize and deserialize a list of strings.	write len + '#' + text for each string; read the length, then take exactly that many characters
261	Print a Directory Tree	build nested dicts from the path parts, then print them with a recursive walk that indents by depth

Contents Start the questions >

Master contents (all topics)

#	Question	What you change in the template
262	Implement a decode and encode function of a general N-ary tree.	write value and child count for each node (preorder); read the count to know how many children to build
263	Serialize and Deserialize a Graph	BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 259 of 288 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 1 of 5

Template fit: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right | no date

MEDIUM | IN APP LC 105 | Exponent | 259 of 288

Construct a binary tree given its preorder and inorder traversals.

ASKED (ANY CO.) ~A MONTH AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 38.3 | 30 DAYS 0 | PRIORITY 111/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: your app's interview approach (LeetCode #105)

LeetCode match: LC 105 Construct Binary Tree from Preorder and Inorder Traversal

Last asked (any company): a month ago, about 2026-09 (Exponent shows only a rough age) | +26 interviews

Original: https://www.aced.io/questions/3417/construct-binary-tree-preorder-inorder-traversal

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7]

Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]

Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

EXAMPLES

Example 1

Input	`preorder = [3, 9, 20, 15, 7]` `inorder = [9, 3, 15, 20, 7]`
Output	`values(build_pre_in([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])) -> [3, 9, 20, null, null, 15, 7]`
Why	`The first preorder value 3 is the root; in the inorder list 9 is left of it and 15, 20, 7 are right. The tree is shown level by level.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def build(pre, inord):
    if not pre:
        return None
    root = Node(pre[0])
    i = inord.index(pre[0])  # linear search
    root.left = build(pre[1:i + 1], inord[:i])
    root.right = build(pre[i + 1:],
                       inord[i + 1:])
    return root
```

TIME	SPACE
O(n^2)	O(n^2) for the slices

WHY IT IS SLOW

The linear search and the list copies make a skewed tree O(n^2) in time and space. A dictionary of inorder positions with index bounds is O(n).

OPTIMAL SOLUTION · Hash map of inorder positions and index bounds

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You are given postorder and inorder instead.

The root is the last postorder element, and because postorder reads left, right, root, reading it backwards yields root, right, left, so build the right subtree before the left.

FOLLOW-UP 2 Values can repeat.

Then the tree is not unique and the inorder index of the root is ambiguous. Ask for a rule (or for a preorder with null markers). Preorder plus postorder alone also cannot determine a general binary tree.

FOLLOW-UP 3 The tree is a 100,000-node chain.

The recursion is as deep as the tree. Use an explicit stack: push nodes while walking preorder, and pop back to the parent when the next node's inorder index shows you have finished the left side.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Construct a binary tree given its
# preorder and inorder traversals.
#
# Given two integer arrays, preorder and inorder, which
# represent the pre-order and in-order traversal of a
# binary tree, construct and return the binary tree. Pre-
# order traversal visits nodes in this order: root -> left
# subtree -> right subtree In-order traversal visits nodes
# in this order: left subtree -> root -> right subtree You
# need to construct the binary tree and return its root
# node. Examples preorder = [3, 9, 20, 15, 7], inorder =
# [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7]
# Explanation: Given the two arrays, the binary tree
# constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2],
# inorder = [2, 1] output: [1, 2] Explanation: Given the
# two arrays, the binary tree constructed is: 1 / 2
# Constraints: The number of nodes in the tree is in the
# range [1, 3000]. preorder and inorder consist of unique
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You are given postorder and inorder instead.
# 2. Values can repeat.
# 3. The tree is a 100,000-node chain.
#
# INTERVIEW NOTES
# Preorder tells you the root (its first element), and
# inorder tells you how many nodes go left (everything
# before the root in inorder). That cut gives two
# independent smaller problems. The only expensive step is
# locating the root in inorder, which a hash map of value
# to index makes O(1).

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
# HELPERS used by the code below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

# ---- main question: main function ----
# BASE QUESTION (optimal): Hash map of inorder positions
# and index bounds
# Idea: Map each value to its inorder index.
def build_pre_in(preorder, inorder):
    # preorder: root, left, right. The root's position in
    # the
    # inorder list splits it into the left and right
    # parts.
    where = {v: i for i, v in enumerate(inorder)}
    nxt = 0

    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(preorder[nxt])
        nxt += 1
        mid = where[root.val]
        root.left = build(lo, mid)
        root.right = build(mid + 1, hi)
        return root

    return build(0, len(inorder))

# ---- main question: tests ----
def test_main_question():
    pre, ino = [3, 9, 20, 15, 7], [9, 3, 15, 20, 7]
    tree = build_pre_in(pre, ino)
    assert preorder_of(tree) == pre
    assert inorder_of(tree) == ino
    post = [9, 15, 7, 20, 3]


# ============================================================
# FOLLOW-UP 1: You are given postorder and inorder
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: The root is the last postorder element, and
# because postorder reads left, right, root, reading it
# backwards yields root, right, left, so build the right
# subtree before the left.
def build_post_in(inorder, postorder):
    # postorder: left, right, root. Read it BACKWARDS:
    # root,
    # right, left, so build the right subtree first.
    where = {v: i for i, v in enumerate(inorder)}
    nxt = len(postorder) - 1

    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(postorder[nxt])
        nxt -= 1
        mid = where[root.val]
        root.right = build(mid + 1, hi)
        root.left = build(lo, mid)
        return root

    return build(0, len(inorder))

# ---- follow-up 1: tests ----
def test_follow_up_1():
    pre, ino = [3, 9, 20, 15, 7], [9, 3, 15, 20, 7]
    post = [9, 15, 7, 20, 3]
    tree2 = build_post_in(ino, post)
    assert preorder_of(tree2) == pre


# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: The tree is a 100,000-node chain.
# ============================================================
```

```
# ---- follow-up 3: function ----
# Answer: The recursion is as deep as the tree. Use an
# explicit stack: push nodes while walking preorder, and
# pop back to the parent when the next node's inorder
```

```
# index shows you have finished the left side.
def build_pre_in_iterative(preorder, inorder):
    # a 100,000-node chain: no recursion, an explicit
    # stack
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    stack, in_i = [root], 0
    for val in preorder[1:]:
        node = TreeNode(val)
        if stack[-1].val != inorder[in_i]:
            # the left side of the top node is not
            # finished
            stack[-1].left = node
        else:
            # pop back to the node whose right child this
            # is
            parent = None
            while stack and stack[-1].val == (
                    inorder[in_i]):
                parent = stack.pop()
                in_i += 1
            parent.right = node
        stack.append(node)
    return root

# ---- follow-up 3: tests ----
def test_follow_up_3():
    pre, ino = [3, 9, 20, 15, 7], [9, 3, 15, 20, 7]
    tree3 = build_pre_in_iterative(pre, ino)
    assert preorder_of(tree3) == pre
    assert inorder_of(tree3) == ino
    assert build_pre_in([], []) is None
    # a very deep chain works without recursion
    n = 50000
    chain = list(range(n))
    deep = build_pre_in_iterative(chain, chain[::-1])
    assert preorder_of(deep) == chain


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
def inorder_of(root):
```

```
    out, stack, node = [], [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.append(node.val)
        node = node.right
    return out


def preorder_of(root):
    out, stack = [], [root] if root else []
    while stack:
        node = stack.pop()
        out.append(node.val)
        for child in (node.right, node.left):
            if child:
                stack.append(child)
    return out
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples

```
preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20,
 7]
output: [3, 9, 20, null, null, 15, 7]
Explanation:
Given the two arrays, the binary tree constructed is:
        3
       / \
      9   20
         /  \
        15   7
```

```
preorder = [1, 2], inorder = [2, 1]
output: [1, 2]
Explanation:
Given the two arrays, the binary tree constructed is:
    1
   /
  2
```

OFFICIAL PAGE TEXT

Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order. Our solution to construct a binary tree from pre-order and in-order traversal arrays uses a recursive approach. We first identify the root node from the pre-order array, then split the in-order array into left and right subtrees based on the root's position in the in-order array. We continue this process recursively to build the entire binary tree. We start by picking the first element of the preorder array as the root of the tree. The index of this root element in the inorder array helps us divide the tree into left and right subtrees. The elements to the left of the root in the inorder array form the left subtree, and the elements to the right form the right subtree. We then recursively build the left and right subtrees using the next available elements in the preorder array. To ensure efficient lookups of the root's index in the inorder array, we create a hash map (inorder_index_map) that maps values to their indices in the inorder array. This allows us to retrieve the position of the root in constant time. NoteRecursion: Base Case: If there are no elements to construct the tree (i.e., the preorder or inorder array is empty), we return nullptr. Recursive Step: We extract the root from the preorder

array and find its position in the inorder array using the hash map. This splits the inorder array into left and right subtrees. We then recursively build the left and right subtrees and attach them to the root. By maintaining the correct indices for both the pre-order and in-order arrays, we can recursively construct the entire tree. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the tree. This is because we traverse both the preorder and inorder arrays once, and we use a hash map for constant-time lookups of root indices in the inorder array. Space Complexity: The solution uses O(n) space for the hash map that stores the indices of elements in the inorder array, and the recursion stack requires O(h) space, where h is the height of the tree. In the worst case (a skewed tree), the height of the tree can be O(n), so the total space complexity is O(n).No answers contributed by the community yet.

STUDY PATH: question 260 of 288 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 2 of 5

Template fit: write len + '#' + text for each string; read the length, then take exactly that many characters | no date

MEDIUM | IN APP LC 271 | Exponent | 260 of 288

Write functions to serialize and deserialize a list of strings.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 28.2 | 30 DAYS 0 | PRIORITY 149/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: your app's interview approach (LeetCode #271)

LeetCode match: LC 271 Encode and Decode Strings

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age) | 2 interviews

Original: https://www.aced.io/questions/1307/serialize-deserialize-list-strings

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

EXAMPLES

QUESTION

Example 1

Input	`items = ['hello', 'wor#ld', ''] serialize(items) deserialize(serialize(items)) == items`
Output	`serialize(items) -> '5#hello6#wor#ld0#' deserialize(serialize(items)) == items -> true`
Why	`Each string is written as its length, a #, and its text, so a # inside a string cannot confuse the reader.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def serialize(items):
    return ",".join(items)
def deserialize(text):
    return text.split(",")  # breaks on commas
```

TIME	SPACE
O(total length)	O(total length)

WHY IT IS SLOW

It is fast (O(total length)) but wrong: a string that contains the delimiter breaks it, and an empty list cannot be told from a list of one empty string. Writing each string's length before it fixes both.

OPTIMAL SOLUTION · Length-prefix every string

TIME	SPACE
O(total length)	O(total length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use escaping instead of a length prefix.

Escape the separator and the escape character inside each string, then terminate each item with the unescaped separator. The decoder tracks whether the previous character was an escape. It is streamable, while the length prefix can be read in O(1) jumps.

FOLLOW-UP 2 The strings are long or binary.

Use bytes and a fixed-width length (for example a 4-byte big-endian integer from struct.pack). Fixed-width lengths avoid the digit-scanning loop and work for any byte value.

FOLLOW-UP 3 Just use JSON?

Yes if the format is free to choose: json.dumps handles quoting and Unicode correctly. Mention it as the practical answer, but code the length-prefix version when the interviewer wants to see you handle the delimiter problem yourself.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Write functions to serialize and
# deserialize a list of strings.
#
# Serialize and Deserialize a List of Strings. You are
# tasked with writing two functions: serialize and
# deserialize. The serialize function will take a list of
# strings as input and return a single string. The
# deserialize function will take this single string and
# reconstruct the original list of strings. Details The
# serialize function should concatenate all strings in the
# list into a single string, with each original string's
# length encoded as a single character prefix. The
# deserialize function should be able to take the single
# string and accurately split it back into the original
# list of strings, using the encoded length characters as
# guides. For example, consider a list ["hello", "world"].
# The serialization of this list might be
# "\x05hello\x05world", where \x05 indicates that the
# following string has a length of 5 characters.
# Constraints The maximum length of any string is 65535
# (0xFFFF).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use escaping instead of a length prefix.
# 2. The strings are long or binary.
# 3. Just use JSON?
#
# INTERVIEW NOTES
# Any text can appear inside a string, so a delimiter is
# never safe by itself. The fix is to say how long each
# string is before you write it (length prefix), so the
# decoder never has to look for a separator inside the
# data. Always check two corner cases: the empty list and
# a list containing the empty string.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import json
import struct
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def serialize(items):
    # length prefix: "<length>#<text>" for every string.
    # The
    # reader knows exactly how much to read, so the text
    # may
    # hold any character, even "#" and digits.
    return "".join(f"{len(s)}#{s}" for s in items)


# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def deserialize(text):
    items, i = [], 0
    while i < len(text):
        # read the digits up to the "#"
        sep = text.index("#", i)
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items

# ---- main question: tests ----
def test_main_question():
    tricky = ["", "a", "12#x", "hello, world", "\\", ",,",
              "é??", "3#abc", "\n"]


# ============================================================
# FOLLOW-UP 1: Use escaping instead of a length prefix.
# ============================================================

# ---- follow-up 1: function ----
```

```
# Answer: Escape the separator and the escape character
# inside each string, then terminate each item with the
# unescaped separator. The decoder tracks whether the
# previous character was an escape.
def serialize_escaped(items):
    # follow-up: escaping. Inside an item "\" becomes "\\"
    # and "," becomes "\,"; every item ends with a plain
    # ",".
    out = []
    for s in items:
        s = s.replace("\\", "\\\\").replace(",", "\\,")
        out.append(s + ",")
    return "".join(out)


# Answer: Escape the separator and the escape character
# inside each string, then terminate each item with the
# unescaped separator. The decoder tracks whether the
# previous character was an escape.
def deserialize_escaped(text):
    items, cur, escaped = [], [], False
    for ch in text:
        if escaped:
            # the character after "\" is taken literally
            cur.append(ch)
            escaped = False
        elif ch == "\\":
            escaped = True
        elif ch == ",":
            items.append("".join(cur))
            cur = []
```

```
        else:
            cur.append(ch)
    if cur or escaped:
        raise ValueError("truncated input")
    return items

# ============================================================
# FOLLOW-UP 2: The strings are long or binary.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Use bytes and a fixed-width length (for example
# a 4-byte big-endian integer from struct.pack). Fixed-
# width lengths avoid the digit-scanning loop and work for
```

```
# any byte value.
def serialize_bytes(items):
    # follow-up: long or binary data. A fixed 4 byte
    # length
    # (big-endian) before each item: no digits to scan.
    out = bytearray()
    for s in items:
        raw = s
        if isinstance(s, str):
            raw = s.encode("utf-8")
        out += struct.pack(">I", len(raw)) + raw
    return bytes(out)


# Answer: Use bytes and a fixed-width length (for example
# a 4-byte big-endian integer from struct.pack). Fixed-
# width lengths avoid the digit-scanning loop and work for
# any byte value.
def deserialize_bytes(data):
    items, i = [], 0
    while i < len(data):
        (size,) = struct.unpack(">I", data[i:i + 4])
        items.append(data[i + 4:i + 4 + size])
        i += 4 + size
    return items

# ============================================================
# FOLLOW-UP 3: Just use JSON?
# ============================================================

# ---- follow-up 3: function ----
# Answer: Yes if the format is free to choose: json.dumps
# handles quoting and Unicode correctly.
def serialize_json(items):
    # follow-up: the practical answer when the format is
    # free
    return json.dumps(items)


# Answer: Yes if the format is free to choose: json.dumps
# handles quoting and Unicode correctly.
def deserialize_json(text):
    return json.loads(text)

# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    tricky = ["", "a", "12#x", "hello, world", "\\", ",,",
              "é??", "3#abc", "\n"]
    for data in ([], [""], ["a"], tricky):
        assert deserialize(serialize(data)) == data
        assert deserialize_escaped(
            serialize_escaped(data)) == data
        back = deserialize_json(serialize_json(data))
        assert back == data
        raw = deserialize_bytes(serialize_bytes(data))
        assert [r.decode("utf-8") for r in raw] == data
    assert serialize(["ab", "c"]) == "2#ab1#c"
    assert serialize_escaped(["a,b"]) == "a\\,b,"
    assert serialize_escaped(["",]) == ","
    assert serialize_escaped([]) == ""
    assert serialize_bytes(["a"]) == b"\x00\x00\x00\x01a"
    # binary data that is not text works too
    blob = [bytes([0, 255, 10])]
    packed = serialize_bytes(blob)
    assert deserialize_bytes(packed) == blob
    try:
        deserialize_escaped("abc")
        raise AssertionError("should fail")
    except ValueError:
        pass


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples

```
lst = ['hello', 'world']
serialized = serialize(lst)
# serialized output: "\x05hello\x05world"
deserialized = deserialize(serialized)
# deserialized output should be: ['hello', 'world']

lst = ['']
serialized = serialize(lst)
# serialized output: "\x00"
deserialized = deserialize(serialized)
# deserialized output should be: ['']

lst = ['a', 'bc', 'def']
serialized = serialize(lst)
# serialized output: "\x01a\x02bc\x03def"
deserialized = deserialize(serialized)
# deserialized output should be: ['a', 'bc', 'def']
```

OFFICIAL PAGE TEXT

Our solution serializes and deserializes a list of strings by encoding the length of each string as a single character and concatenating it with the original string. The serialize function processes the input list of strings by iterating through each string. For every string, it first encodes the length of the string as a single character using the encodeNumber function. The encoded length is then appended to a result list, followed by the string itself. The result list is ultimately joined into a single string, which represents the serialized form of the list of strings. The deserialize function takes the serialized string and reconstructs the original list of strings. It starts by iterating through the serialized string, decoding the length of each string using the decodeNumber function, then extracting the string of that length from the serialized data. These strings are collected in a list and returned as the final deserialized result. Time Complexity: The time complexity of both serialize and deserialize is O(n), where n is the total number of characters across all strings in the input list. In serialize, we iterate through every string, performing constant-time operations (encoding the length and appending the string) for each string. Similarly, in deserialize, we iterate through the serialized string, decoding lengths and extracting substrings, which are also constant-time operations for each step. Space Complexity:

The space complexity is O(n) for both functions since we store the serialized form of the input list (which has a size proportional to the total number of characters across all strings) and reconstruct the list in deserialize.Daniel answers "Write a pair of functions to serialize and deserialize a list of strings."No answers contributed by the community yet.Related questionsWrite a function to do Excel-style operations.Write a function to check whether a string contains a substring.Given a list of bounding boxes and annotations, write a function to compute Intersection over Union.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to

run code

STUDY PATH: question 261 of 288 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 3 of 5

Template fit: build nested dicts from the path parts, then print them with a recursive walk that indents by depth | asked 394 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 261 of 288

Print a Directory Tree

ASKED 2025-09-21 | 394 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 219/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #588 Design In-Memory File System

LeetCode match: LC 588 Design In-Memory File System (related: tree from paths)

Frequency: Single report | Last asked: 2025-09-21 | Roles: SWE Intern

QUESTION

You are given a list of file/directory paths , each a '/' -separated string (for example "dir/subdir/file.txt"). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string. Rules: Each path component is a node. A component that appears under the same parent in several paths must be printed only once. Indent each node with 2 * depth spaces, where the top-level components have depth 0 . Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort). The output must not have a trailing newline.

Function signature

def print_directory_tree(paths: list[str]) -> str:

...

Follow-up (as reported): Add box-drawing connectors (, ,) like the real tree command. Hint: track, per level, whether the current node is the last child of its parent.

EXAMPLES

Example 1

Input	`paths = ["a/b/c", "a/b/d", "a/e"]`
Output	`"a\n b\n c\n d\n e"`
Why	`Rendered: a b c d e`

Example 2

Input	`paths = ["src/main.py", "src/util.py", "README.md"]`
Output	`"src\n main.py\n util.py\nREADME.md"`
Why	`"src" and its two files come first (first-seen order), then the top-level "README.md".`

QUESTION

Example 3

Input	`paths = []`
Output	`""`

CONSTRAINTS

- 0 <= len(paths) <= 10^4
- Each path is non-empty, has no leading/trailing '/' , and no empty components.
- If paths is empty, return the empty string "" .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_tree(paths):
    root = []
    for path in paths:
        level = root
        for part in path.split(/):
            for node in level:
                if node[0] == part:
                    break
            else:
                node = [part, []]
                level.append(node)
            level = node[1]
    lines = []
    def walk(level, depth):
        for name, kids in level:
            lines.append("  " * depth + name)
            walk(kids, depth + 1)
    walk(root, 0)
    return "\n".join(lines)
```

TIME	SPACE
O(total length * width)	O(total length)

WHY IT IS SLOW

Every component searches the list of children of its parent one by one.

OPTIMAL SOLUTION · Nested dictionaries and a DFS

TIME	SPACE
O(total length)	O(total length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print it with box-drawing connectors like the real tree command.

Carry a prefix down the recursion. For each child choose the connector: the last child gets the corner and its subtree continues with blank space, the others get the tee and their subtree continues with a vertical bar.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Print a Directory Tree
#
# You are given a list of file/directory paths , each a
# '/' -separated string (for example "dir/subdir/file.txt"
# ). Reproduce what a Unix directory listing tool prints:
# build the directory hierarchy implied by the paths and
# return it as a single newline-separated string. Rules:
# Each path component is a node. A component that appears
# under the same parent in several paths must be printed
# only once. Indent each node with 2 * depth spaces, where
# the top-level components have depth 0 . Children are
# printed in the order they are first encountered across
# the input list (preserve first-seen order; do not sort).
# The output must not have a trailing newline. Function
# signature: def print_directory_tree(paths: list[str]) ->
# str: ... Follow-up (as reported): Add box-drawing
# connectors ( , , ) like the real tree command.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print it with box-drawing connectors like the real
# tree command.
#
# INTERVIEW NOTES
# The paths describe a tree where shared prefixes merge.
# Insert every path into a nested dictionary (a trie of
# path components; Python dicts keep insertion order,
# which gives the first-seen order for free), then print
# it with a depth-first walk that indents two spaces per
# level.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Nested dictionaries and a DFS
# Idea: Insert every path into a dict of dicts, then print
# depth first with two spaces per level.
def print_directory_tree(paths):
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            node = node.setdefault(part, {})
    lines = []

    def walk(node, depth):
        for name, kids in node.items():
            lines.append("  " * depth + name)
            walk(kids, depth + 1)

    walk(root, 0)
    return "\n".join(lines)

# ---- main question: tests ----
def test_main_question():
    random.seed(17)
    want = "a\n  b\n    c\n    d\n  e"
    assert (
        print_directory_tree(["a/b/c", "a/b/d", "a/e"])
        == want
    )
    want = "src\n  main.py\n  util.py\nREADME.md"
    assert (
        print_directory_tree(
            ["src/main.py", "src/util.py", "README.md"]
        )
        == want
    )
    assert print_directory_tree([]) == ""
    corner, tee = (
        "\u2514\u2500\u2500 ",
        "\u251c\u2500\u2500 ",
    )


# ============================================================
# FOLLOW-UP 1: Print it with box-drawing connectors like
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: Carry a prefix down the recursion. For each
# child choose the connector: the last child gets the
# corner and its subtree continues with blank space, the
# others get the tee and their subtree continues with a
# vertical bar.
def tree_with_connectors(paths):
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            node = node.setdefault(part, {})
    lines = []
    tee = "\u251c\u2500\u2500 "
    elbow = "\u2514\u2500\u2500 "
    bar = "\u2502   "

    def walk(node, prefix):
        items = list(node.items())
        for i, (name, kids) in enumerate(items):
            last = i == len(items) - 1
            glyph = elbow if last else tee
            lines.append(prefix + glyph + name)
            walk(
                kids, prefix + ("    " if last else bar)
            )

    walk(root, "")
    return "\n".join(lines)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    corner, tee = (
        "\u2514\u2500\u2500 ",
        "\u251c\u2500\u2500 ",
    )
    assert (
        tree_with_connectors(["a/b", "a/c"])
        == corner
        + "a\n    "
        + tee
        + "b\n    "
```

```
        + corner
        + "c"
    )
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 262 of 288 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 4 of 5

Template fit: write value and child count for each node (preorder); read the count to know how many children to build | no date

HARD | Exponent | 262 of 288

Implement a decode and encode function of a general N-ary tree.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 173/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 428 Serialize and Deserialize N-ary Tree (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4516/decode-encode-function-general-tree

QUESTION

A general tree has a value in every node and any number of children. Write encode(root), which turns the tree into a string, and decode(text), which rebuilds the identical tree from that string.

Function signature

def encode(root: Node) -> str

def decode(text: str) -> Node

EXAMPLES

Example 1	

Input	`root = Node(1, [Node(3, [Node(5), Node(6)]), Node(2), Node(4)])`
	`text = encode(root)`
	`text`
	`encode(decode(text)) == text`

Output	`text -> '[1, 3, 3, 2, 5, 0, 6, 0, 2, 0, 4, 0]'` `encode(decode(text)) == text -> true`
Why	`Each node is written as its value followed by its number of children, in pre-order: 1 has 3 children, 3 has 2, and so on. Decoding reads exactly that many children for each node, so encoding the decoded tree gives the same text.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def encode(node):
    parts = [encode(c) for c in node.children]
    kids = ",".join(parts)
    return f"{node.val}[{kids}]"
def decode(text):
    pos = 0
    def parse():
        nonlocal pos
        start = pos
        while text[pos] != "[":
            pos += 1
        node = Node(text[start:pos])
        pos += 1  # skip "["
        while text[pos] != "]":
            node.children.append(parse())
            if text[pos] == ",":
                pos += 1
        pos += 1  # skip "]"
        return node
    return parse()
```

TIME	SPACE
O(n * h) for building strings (O(n) with a list buffer)	O(n)

WHY IT IS SLOW

Joining child strings copies text at every level (O(n * h) in the worst case) and the recursive parser can overflow on a deep tree; values containing brackets or commas break it. Pre-order with child counts is O(n) and needs no markers.

OPTIMAL SOLUTION · Pre-order with child counts, iterative decode

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The values are arbitrary strings that may contain spaces, commas or brackets.

Do not invent delimiters. Put the values in a real container: a JSON list of alternating value and child count, which handles quoting for you, or length-prefix every value (5#hello). The structure part (the counts) stays the same.

FOLLOW-UP 2 The tree is huge and you want a compact binary encoding.

Counts and node values can be packed with variable-length integers (varint), which makes small values one byte. Keep the same pre-order count layout so decoding is still one pass. Add a version byte at the front so the format can change later. **Cost:** still O(n) time and space; variable-length integers cost about one byte per small value.

FOLLOW-UP 3 Encode in level order instead.

Write each node's value and child count in BFS order and decode with a queue: pop a parent, read its count, attach that many new nodes, and push them. BFS order is handy when the consumer wants to stream levels.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a decode and encode function of
# a general N-ary tree.
#
# A general tree has a value in every node and any number
# of children. Write encode(root), which turns the tree
# into a string, and decode(text), which rebuilds the
# identical tree from that string.
#
# Signature:
#   def encode(root: Node) -> str
#   def decode(text: str) -> Node
#
# FOLLOW-UPS (answered below, in this order):
# 1. The values are arbitrary strings that may contain
# spaces, commas or brackets.
# 2. The tree is huge and you want a compact binary
# encoding.
# 3. Encode in level order instead.
#
# INTERVIEW NOTES
# Serialization questions are about a format that can be
# read back unambiguously. Ask what a node holds and
# whether values can contain your delimiters. A node can
# have any number of children, so the decoder must be told
# where the children end: either with a closing marker or
# by writing the child count next to each value. The count
# form needs no closing markers and can be decoded with an
# explicit stack, so deep trees are safe.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import json
from collections import deque
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
class Node:
    def __init__(self, val, children=None):
        self.val = val
        self.children = children or []


def preorder_pairs(root):
    # (value, number of children) for every node in pre-
    # order,
    # iterative so a very deep tree cannot overflow the
    # stack
    out = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        out.append((node.val, len(node.children)))
        # reversed: the first child must be popped first
        stack.extend(reversed(node.children))
    return out


def build_preorder(pairs):
    # the reverse: rebuild the tree from the same pairs
    pairs = iter(pairs)
    first = next(pairs, None)
    if first is None:
        return None
    root = Node(first[0])
    # each stack entry is a parent and the children it
    # still has to receive
    stack = [[root, first[1]]]
    while stack:
        top = stack[-1]
        if top[1] == 0:
            stack.pop()
            continue
        top[1] -= 1
        val, count = next(pairs)
        child = Node(val)
        top[0].children.append(child)
        stack.append([child, count])
    return root

# ---- main question: main function ----
# BASE QUESTION (optimal): Pre-order with child counts,
```

```
# iterative decode
# Idea: Write (value, number of children) in pre-order.
# FOLLOW-UP 1: The values are arbitrary strings that may
# contain spaces, commas or brackets.
# Answer: Do not invent delimiters. Put the values in a
# real container: a JSON list of alternating value and
# child count, which handles quoting for you, or length-
# prefix every value (5#hello). The structure part (the
# counts) stays the same.
def encode(root):
    # FOLLOW-UP 1: arbitrary string values
    # values may hold spaces, commas or brackets: JSON
    # does
    # the quoting, so no invented delimiter can clash
    flat = []
    for val, count in preorder_pairs(root):
        flat += [val, count]
    return json.dumps(flat)
```

```
# BASE QUESTION (optimal): Pre-order with child counts,
# iterative decode
# Idea: Write (value, number of children) in pre-order.
def decode(text):
    flat = json.loads(text)
    return build_preorder(zip(flat[::2], flat[1::2]))

# ---- main question: tests ----
def test_main_question():
    tree = Node("a, b", [
        Node("[x]", [Node("")]), Node("é ??"),
        Node('q"uote', [Node("1"), Node("2")])])
    assert same(decode(encode(tree)), tree)


# ============================================================
# FOLLOW-UP 1: The values are arbitrary strings that may
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: The tree is huge and you want a compact
# ============================================================

# ---- follow-up 2: helper functions ----
```

```
def put_varint(n, out):
    # follow-up: a compact binary form. 7 bits per byte,
    # the
    # top bit says that another byte follows
    while n >= 128:
        out.append(n % 128 + 128)
        n //= 128
    out.append(n)


def get_varint(data, pos):
    n, shift = 0, 0
    while data[pos] >= 128:
        n += (data[pos] - 128) << shift
        shift += 7
        pos += 1
    return n + (data[pos] << shift), pos + 1

# ---- follow-up 2: function ----
# Answer: Counts and node values can be packed with
# variable-length integers (varint), which makes small
# values one byte. Keep the same pre-order count layout so
# decoding is still one pass.
def encode_binary(root):
    # a version byte first, so the format can change later
    out = bytearray([1])
    for val, count in preorder_pairs(root):
        raw = val.encode("utf-8")
        put_varint(len(raw), out)
        out += raw
        put_varint(count, out)
    return bytes(out)


# Answer: Counts and node values can be packed with
# variable-length integers (varint), which makes small
# values one byte. Keep the same pre-order count layout so
# decoding is still one pass.
def decode_binary(data):
    if data[0] != 1:
        raise ValueError("unknown version")
    pairs, pos = [], 1
    while pos < len(data):
        size, pos = get_varint(data, pos)
```

```
        val = data[pos:pos + size].decode("utf-8")
```

```
        count, pos = get_varint(data, pos + size)
        pairs.append((val, count))
    return build_preorder(pairs)

# ============================================================
# FOLLOW-UP 3: Encode in level order instead.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Write each node's value and child count in BFS
# order and decode with a queue: pop a parent, read its
# count, attach that many new nodes, and push them. BFS
# order is handy when the consumer wants to stream levels.
def level_pairs(root):
    # follow-up: the same pairs in level order (BFS)
    out, queue = [], deque([root] if root else [])
    while queue:
        node = queue.popleft()
        out.append((node.val, len(node.children)))
        queue.extend(node.children)
    return out


# Answer: Write each node's value and child count in BFS
# order and decode with a queue: pop a parent, read its
# count, attach that many new nodes, and push them. BFS
# order is handy when the consumer wants to stream levels.
def build_levels(pairs):
    # decode with a queue: pop a parent, attach its
    # children
    pairs = iter(pairs)
    first = next(pairs, None)
    if first is None:
        return None
    root = Node(first[0])
    queue = deque([(root, first[1])])
    while queue:
        parent, count = queue.popleft()
        for _ in range(count):
            val, kids = next(pairs)
            child = Node(val)
            parent.children.append(child)
            queue.append((child, kids))
    return root
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = Node("a, b", [
        Node("[x]", [Node("")]), Node("é ??"),
        Node('q"uote', [Node("1"), Node("2")])])
    assert same(build_levels(level_pairs(tree)), tree)
    assert same(decode_binary(encode_binary(tree)), tree)
    assert encode(None) == "[]" and decode("[]") is None
    assert decode_binary(encode_binary(None)) is None
    assert build_levels(level_pairs(None)) is None
    # a very deep chain works because nothing is recursive
    chain = Node("0")
    cur = chain
    for i in range(1, 5000):
        cur.children.append(Node(str(i)))
        cur = cur.children[0]
    assert same(decode(encode(chain)), chain)
    # a node with 300 children needs a two byte count
    wide = Node("w", [Node(str(i)) for i in range(300)])
    assert same(decode_binary(encode_binary(wide)), wide)
    assert len(preorder_pairs(wide)) == 301


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
```

```
# ============================================================
# HELPERS for the checks below
def same(a, b):
    return preorder_pairs(a) == preorder_pairs(b)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

STUDY PATH: question 263 of 288 | topic 38 of 43: Build and serialize: write a structure out, read it back with the same walk | 5 of 5

Template fit: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking | asked 391 days ago | frequency Single report

HARD | ColorStack | technical interview | 263 of 288

Serialize and Deserialize a Graph

ASKED 2025-09-24 | 391 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 203/288

ABOUT THIS QUESTION

Topic: Build and serialize: write a structure out, read it back with the same walk

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #133 Clone Graph, LeetCode #297 Serialize and Deserialize Binary Tree

LeetCode match: LC 133 Clone Graph / LC 297 Serialize and Deserialize Binary Tree (related)

Frequency: Single report | Last asked: 2025-09-24 | Roles: SWE Intern

QUESTION

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer val and a list of neighbors . The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original. You are given the reference node class below. Implement serialize (node -> string) and deserialize (string -> node). serialize(None) must round-trip to None via deserialize .

Function signature

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from node into a string."""
        ...
    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Follow-up (as reported): Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

EXAMPLES

QUESTION

Example 1

Input	`adjList = [[]] # a single node with no neighbors`
Output	`A single node with val 1 and an empty neighbor list.`

Example 2

Input	`node = None # empty graph`
Output	`codec.deserialize(codec.serialize(None)) is None`

CONSTRAINTS

- 0 <= number of nodes <= 100
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_edges(node):
    # the edge list found by a plain traversal
        with a visited set
    seen, edges, stack = {node.val}, [], [node]
    while stack:
        cur = stack.pop()
        for nb in cur.neighbors:
            edges.append((cur.val, nb.val))
            if nb.val not in seen:
                seen.add(nb.val)
                stack.append(nb)
    return sorted(edges)
```

TIME	SPACE
does not finish on a cycle	-

WHY IT IS SLOW

Without remembering visited nodes a cycle is expanded forever; a visited set is the minimum fix.

OPTIMAL SOLUTION · BFS to a text, two passes back

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support directed graphs, where the adjacency is not symmetric.

The format already stores each node's own neighbour list, so a directed edge u -> v is kept as given and nothing assumes the reverse edge exists. Only nodes reachable from the starting node are encoded, which is the same as for the undirected case.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Serialize and Deserialize a Graph
#
# Design a codec that can serialize a connected,
# undirected graph to a string and deserialize that string
# back into an identical graph. Each node has a unique
# integer val and a list of neighbors . The graph may
# contain cycles. After a round trip, the reconstructed
# graph must have the same node values and the same
# adjacency relationships as the original. You are given
# the reference node class below. Implement serialize
# (node -> string) and deserialize (string -> node).
# serialize(None) must round-trip to None via deserialize
# . Function signature: class Node: def __init__(self,
# val: int = 0, neighbors: list["Node"] | None = None):
# self.val = val self.neighbors = neighbors if neighbors
# is not None else [] class Codec: def serialize(self,
# node: "Node | None") -> str: """Encode the whole graph
# reachable from node into a string.""" ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support directed graphs, where the adjacency is not
# symmetric.
#
# INTERVIEW NOTES
# A graph can contain cycles, so the encoding must name
# nodes by value and list their neighbours instead of
# nesting them like a tree. Walk the graph with BFS from
# the given node once, writing one entry per node
# ('val:n1,n2'); deserialize creates all nodes first, then
# links the neighbours, which also handles cycles and
# directed edges.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = (
            neighbors if neighbors is not None else []
        )
```

```
# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
class Codec:
    def serialize(self, node):
        if node is None:
            return ""
        seen, queue, parts = (
            {node.val},
            deque([node]),
            [],
        )
        while queue:
            cur = queue.popleft()
            parts.append(
                f"{cur.val}:"
                + ",".join(
                    str(n.val) for n in cur.neighbors
                )
            )
            for nb in cur.neighbors:
                if nb.val not in seen:
                    seen.add(nb.val)
                    queue.append(nb)
        return ";".join(parts)

    def deserialize(self, data):
        if not data:
            return None
```

```
        rows = [
            part.split(":") for part in data.split(";")
        ]
        nodes = {int(v): Node(int(v)) for v, _ in rows}
        for v, nbs in rows:
            nodes[int(v)].neighbors = [
                nodes[int(x)]
                for x in nbs.split(",")
                if x
            ]
        return nodes[int(rows[0][0])]

# ---- main question: tests ----
def test_main_question():
    random.seed(25)
    codec = Codec()
    assert (
        codec.serialize(None) == ""
        and codec.deserialize("") is None
    )


# ============================================================
# FOLLOW-UP 1: Support directed graphs, where the
# ============================================================

# ---- follow-up 1: function ----
# Answer: The format already stores each node's own
# neighbour list, so a directed edge u -> v is kept as
# given and nothing assumes the reverse edge exists.
def build_graph(adj):
    nodes = [Node(i + 1) for i in range(len(adj))]
    for node, out in zip(nodes, adj):
        node.neighbors = [nodes[j - 1] for j in out]
    return nodes[0] if nodes else None

# ---- follow-up 1: tests ----
def test_follow_up_1():
    codec = Codec()
    cycle = build_graph(
        [[2, 4], [1, 3], [2, 4], [1, 3]]
```

```
    )
    back = codec.deserialize(codec.serialize(cycle))
    single = codec.deserialize(
        codec.serialize(build_graph([[]]))
```

```
    )
    assert single.val == 1 and single.neighbors == []
    for _ in range(200):
        n = random.randint(1, 6)
        adj = [
            sorted(
                random.sample(
                    range(1, n + 1),
                    random.randint(0, min(3, n)),
                )
            )
            for _ in range(n)
        ]
        start = build_graph(adj)
        again = codec.deserialize(
            codec.serialize(start)
        )
        assert codec.serialize(
            again
        ) == codec.serialize(start)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 39 OF 43

Games: win or lose with a cache, and guessing with feedback

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	4
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Games: win or lose with a cache, and guessing with feedback | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	264. Choose optimal guesses for green-only Wordle	PracHub	2025-10-14
H	265. Wordle-Style 5-Letter Minimum-Guess Strategy	1Point3Acres	2025-12-15
H	266. Determine winner in stack-merging game	PracHub	2026-01-06
H	267. Find secret word with match-count feedback	PracHub	2026-02-11
H	268. Babylon Tile-Stack Merge Game	1Point3Acres	2025-12-29

CODE TEMPLATE | GROUP 39 OF 43 | 5 QUESTIONS, ONE PATTERN

Games: win or lose with a cache, and guessing with feedback

The one idea these questions share

For two-player games, a state is winning if some move leads to a state where the opponent loses; cache every state. For guessing games, pick the guess that shrinks the worst remaining group of candidates the most, then keep only the candidates that agree with the feedback.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
from functools import lru_cache


def first_player_wins(state, moves):
    @lru_cache(maxsize=None)
    def wins(s):
        # CHANGE: moves(s) lists the states you can move to; you win if some move leaves the opponent losing
        return any(not wins(t) for t in moves(s))

    return wins(state)
```

2. Helper functions that usually get written next to it

```
from collections import Counter


def match_count(a, b):                          # feedback: how many positions agree
    return sum(x == y for x, y in zip(a, b))


def best_guess(candidates):                     # minimise the largest group left after the feedback
    return min(candidates, key=lambda g: (max(Counter(match_count(g, w) for w in candidates).values()), g))
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
264	Choose optimal guesses for green-only Wordle	bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups
265	Wordle-Style 5-Letter Minimum-Guess Strategy	ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest)
266	Determine winner in stack-merging game	state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap
267	Find secret word with match-count feedback	guess the word that minimises the size of the biggest feedback group; keep the words with the same match count
268	Babylon Tile-Stack Merge Game	state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 264 of 288 | topic 39 of 43: Games: win or lose with a cache, and guessing with feedback | 1 of 5

Template fit: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups | asked 371 days ago

MEDIUM | PracHub | Technical Screen | 264 of 288

Choose optimal guesses for green-only Wordle

POSTED 2025-10-14 | 371 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 99/288

ABOUT THIS QUESTION

Topic: Games: win or lose with a cache, and guessing with feedback

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: No direct match; related LC 843 Guess the Word (inferred from overview)

Posted on PracHub: 2025-10-14 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/choose-optimal-guesses-for-green-only-wordle

QUESTION

You are guessing a hidden word from a candidate list. After each guess you only learn which positions are green, that is, where your letter equals the answer's letter. best_guess(words) picks the candidate whose answer pattern splits the remaining words most evenly: the one with the smallest expected size of the group left (sum of squared group sizes divided by the number of words); ties go to the alphabetically smaller word.

Function signature

def green(guess: str, answer: str) -> tuple
def best_guess(words: list) -> str

EXAMPLES

Example 1

Input	`'cat', 'car'`
Output	`(true, true, false)`
Why	`c and a match, the last letters differ.`

Example 2

Input	`['cat', 'car', 'cap', 'dog']`
Output	`'cap'`
Why	`'cap', 'car' and 'cat' split the other words equally well and 'cap' wins the alphabetical tie-break; 'dog' shares no letter positions with the others, so it tells less.`

QUESTION

Example 3

Input	`['tea']`
Output	`'tea'`
Why	`With a single candidate the answer is that candidate.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(guesses, answers):
    best, best_score = None, INF
    for g in guesses:
        groups = Counter()
        for a in answers:
            groups[green_pattern(g, a)] += 1
        sizes = groups.values()
        score = sum(c * c for c in sizes)
        score /= len(answers)
        if score < best_score:
            best, best_score = g, score
    return best
```

TIME	SPACE
O(W^2 * L)	O(W)

WHY IT IS SLOW

Compares every pair of words, O(W^2 * L). Bitsets per (position, letter) split the candidates into pattern groups with a few AND operations.

OPTIMAL SOLUTION · Bitsets per (position, letter), splitting groups with AND

TIME	SPACE
O(W * L * groups) with bit-parallel work per group	O(W * L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Play a whole game, using the feedback after each guess.

Keep the candidates that would have produced exactly the observed green pattern (filter_candidates), then choose the best guess among the remaining ones. Each round at least removes the guess itself, so the game ends.

FOLLOW-UP 2 Answers are not equally likely (weights).

Replace group sizes by group weights: score = sum(weight(group)^2) / total weight. The grouping step is unchanged; use sums of weights instead of popcounts.

FOLLOW-UP 3 Real Wordle also gives yellow feedback.

The pattern becomes three-valued per position (green, yellow, grey) with the usual duplicate-letter rules, so the pattern space is 3^L; bitsets still work if you precompute patterns, but the computation is O(W^2) pattern evaluations for a full table.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Choose optimal guesses for green-only
# Wordle
#
# You are guessing a hidden word from a candidate list.
# After each guess you only learn which positions are
# green, that is, where your letter equals the answer's
# letter. best_guess(words) picks the candidate whose
# answer pattern splits the remaining words most evenly:
# the one with the smallest expected size of the group
# left (sum of squared group sizes divided by the number
# of words); ties go to the alphabetically smaller word.
#
# Signature:
#   def green(guess: str, answer: str) -> tuple
#   def best_guess(words: list) -> str
#
# Examples:
# 'cat', 'car'  ->  (true, true, false)
# ['cat', 'car', 'cap', 'dog']  ->  'cap'
#
# FOLLOW-UPS (answered below, in this order):
# 1. Play a whole game, using the feedback after each
# guess.
# 2. Answers are not equally likely (weights).
# 3. Real Wordle also gives yellow feedback.
#
# INTERVIEW NOTES
# A guess splits the candidates into groups by which
# positions turned green. If the groups have sizes s_1 ...
# s_k out of W candidates, the expected size of what
# remains is the sum of s_i squared divided by W (a
# candidate falls in a group of size s with probability s
# / W, and then s remain). Pick the guess with the
# smallest value. To make it fast, store for each
# (position, letter) the set of words with that letter
# there as a bitset, so the groups come from a few AND
# operations.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
from collections import defaultdict
from fractions import Fraction


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
# HELPERS used by the code below
def green(guess, answer):
    # green-only feedback: which positions match exactly
    return tuple(g == a for g, a in zip(guess, answer))


def partition(guess, words):
    # group the possible answers by the feedback they
    # would give
    groups = defaultdict(list)
    for w in words:
        groups[green(guess, w)].append(w)
    return list(groups.values())


def score(guess, words, weight=None):
    # expected number of answers left after this guess:
    # sum(group^2) / total, with optional per-word weights
    weight = weight or (lambda w: 1)
    total = sum(weight(w) for w in words)
    spread = sum(sum(weight(w) for w in g) ** 2
                 for g in partition(guess, words))
    return Fraction(spread, total)

# ---- main question: main function ----
# BASE QUESTION (optimal): Bitsets per (position, letter),
# splitting groups with AND
# Idea: Index the words: bits[pos][letter] is an integer
# whose bit i is set when word i has that letter at pos.
def best_guess(words, weight=None):
    # the guess that leaves the fewest answers on average
    # (ties: alphabetical, so the result is deterministic)
    def rank(g):
        return (score(g, words, weight), g)
    return min(words, key=rank)
```

```

# FASTER: bitsets
def best_guess_bitsets(words):
    # faster: one bitmask per (position, letter); each
    # position
    # splits every group in two with two AND operations
    W = len(words)
    bits = [defaultdict(int)
            for _ in range(len(words[0]))]
    for i, w in enumerate(words):
        for pos, ch in enumerate(w):
            bits[pos][ch] |= 1 << i
    best = None
    for g in words:
        groups = [(1 << W) - 1]
        for pos, ch in enumerate(g):
            mask, nxt = bits[pos][ch], []
            for grp in groups:
                for part in (grp & mask, grp & ~mask):
                    if part:
                        nxt.append(part)
            groups = nxt
        sizes = [bin(g).count("1") for g in groups]
        spread = sum(s * s for s in sizes)
        cand = (Fraction(spread, W), g)
        if best is None or cand < best:
            best = cand
    return best[1]

# ---- main question: tests ----
def test_main_question():
    words = ["crane", "crate", "grape", "grade", "trade",
             "slate", "plate"]
    assert best_guess(words) == best_guess_bitsets(words)
```

```
    # a guess can never leave more than all the words
    assert score("crane", words) <= len(words)
    for secret in words:
        path = play(words, secret)
        assert path[-1] == secret
        # each round removes at least the guess itself
        assert len(path) <= len(words)
    # weights: a very likely word makes its guess more
    # attractive
    heavy = lambda w: 100 if w == "slate" else 1
    slate = score("slate", words, heavy)
```

```
    crane = score("crane", words, heavy)
    assert slate < crane
    assert partition("crane", ["crane"]) == [["crane"]]


# ======================================================
# FOLLOW-UP 1: Play a whole game, using the feedback after
# ======================================================

# ---- follow-up 1: function ----
# Answer: Keep the candidates that would have produced
# exactly the observed green pattern (filter_candidates),
# then choose the best guess among the remaining ones.
# Each round at least removes the guess itself, so the
# game ends.
def filter_candidates(words, guess, pattern):
    # keep only answers that would give the observed
    # feedback
    return [w for w in words
            if green(guess, w) == pattern]

# ======================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ======================================================
# HELPERS for the checks below
def play(words, secret):
    # a whole game: guess, read the feedback, narrow down
    candidates, guesses = list(words), []
    while True:
        guess = best_guess(candidates)
        guesses.append(guess)
        if guess == secret:
            return guesses
        pattern = green(guess, secret)
        candidates = filter_candidates(
            candidates, guess, pattern)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are playing a simplified 5-letter word guessing game.

FULL QUESTION TEXT FROM THE SITE

There is a hidden secret word of length 5 .
You have a dictionary of valid 5-letter words. Each word may also have an associated frequency/weight representing how likely it is to be the secret.
When you guess a 5-letter word, the game returns only “green” information (correct letter in the correct position). There is no information about letters that exist in the word but are in the wrong position (“yellow”), and no explicit info about letters that are absent.
Feedback definition

For a guess g and secret s, define the feedback as a 5-bit mask m where:

m[i] = 1 iff g[i] == s[i]
otherwise m[i] = 0

Example: g="crane", s="cared" -> mask is 1 0 0 0 0 (only position 0 matches).

Task

Given:

words : a list of N valid 5-letter words
weight[w] (optional): a positive number for each word (if omitted, assume uniform)
candidates : the current set of possible secret words consistent with feedback so far (initially all words)

Write an algorithm/function to choose the next guess that is optimal under the following objective:

Choose a guess word g (typically from words) that minimizes the expected size of the remaining candidate set after receiving the green-only mask, where the expectation is taken over the secret word drawn from candidates according to weight .

Formally, if C is the current candidate set and C_m(g) is the subset of candidates that would produce mask m for guess g, choose g that minimizes:

FULL QUESTION TEXT FROM THE SITE

E
[
|
C
m
a
s
k
(
g
)
|
]
=
sum
m
in
{
0
,
1
}
5
P
(

m
a
s
k
=
m
)
*
|
C
m
(
g
)
|
E[|C
mask

(g)|]=sum
min{0,1}
5

P(mask=m)*|C
m

(g)|

FULL QUESTION TEXT FROM THE SITE

where
P
(
m
a
s
k
=
m
)
P(mask=m) is induced by the weighted distribution over secrets in C.

Input/Output expectations
Input sizes: 1 <= N <= 5,000 (you may propose optimizations if needed), word length is always 5.
Output: the selected next guess word.
Follow-ups (optional)
How would you incorporate non-uniform word frequencies?
How would you scale if N is very large (e.g., 100k)?
How would you change the objective to minimize expected number of guesses instead of expected remaining candidates?

ALSO REPORTED · 1 more

Wordle-Style 5-Letter Minimum-Guess Strategy | 1Point3Acres | phone-screen, onsite-coding | asked 2025-12-15 | Single report

https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

Preparation: Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

STUDY PATH: question 265 of 288 | topic 39 of 43: Games: win or lose with a cache, and guessing with feedback | 2 of 5

Template fit: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest) | asked 309 days ago | frequency Single report

HARD | 1Point3Acres | phone-screen, onsite-coding | 265 of 288

Wordle-Style 5-Letter Minimum-Guess Strategy

ASKED 2025-12-15 | 309 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 98/288

ABOUT THIS QUESTION

Topic: Games: win or lose with a cache, and guessing with feedback

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word (related)

Frequency: Single report | Last asked: 2025-12-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

QUESTION

A hidden word has 5 lowercase letters. The only thing you can ask is the oracle query(pos, letter), which says whether the letter is at that position (positions 0 to 4). Find the word with as few oracle calls as possible. by_frequency tries letters in order of English frequency; with_dictionary also knows a list of candidate words and always asks the question that splits the candidates most evenly. Both return (word, number of calls).

Function signature

def by_frequency(query) -> (str, int)
def with_dictionary(query, dictionary: list) -> (str, int)

EXAMPLES

Example 1

Input	`secret = 'crane'`
	`query = lambda pos, letter: secret[pos] == letter`
	`by_frequency(query)`
	`with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape'])`

Output	`by_frequency(query) -> ('crane', 31)` `with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape']) -> ('crane', 2)`
Why	`Without a dictionary every position is searched letter by letter; with five candidate words a handful of well-chosen questions are enough.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
word = ""
for i in range(5):
    for c in "abcdefghijklmnopqrstuvwxyz":
        if query(i, c):
            word += c
            break
```

TIME	SPACE
130 calls	O(1)

WHY IT IS SLOW

Always 130 calls. Asking likely letters first, skipping the implied last letter, or splitting a dictionary of candidates in half needs far fewer.

OPTIMAL SOLUTION · Frequency order, or entropy split when a dictionary is known

TIME	SPACE
at most 5 x 25 calls with... at most 5 x 25 calls without a dictionary; about log2(words) with one	O(words)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What is the lower bound and when is it reached?

Every call returns at most one bit, so at least ceil(log2(candidates)) calls are needed. The entropy split reaches it only when a question can cut the candidates exactly in half; real dictionaries usually need a few more. A greedy scoring rule (closest to half) is enough to state; the interviewer mostly wants the entropy idea.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Wordle-Style 5-Letter Minimum-Guess
# Strategy
#
# A hidden word has 5 lowercase letters. The only thing
# you can ask is the oracle query(pos, letter), which says
# whether the letter is at that position (positions 0 to
# 4). Find the word with as few oracle calls as possible.
# by_frequency tries letters in order of English
# frequency; with_dictionary also knows a list of
# candidate words and always asks the question that splits
# the candidates most evenly. Both return (word, number of
```

```
# calls).
#
# Signature:
#   def by_frequency(query) -> (str, int)
#   def with_dictionary(query, dictionary: list) -> (str,
int)
#
# FOLLOW-UPS (answered below, in this order):
# 1. What is the lower bound and when is it reached?
#
# INTERVIEW NOTES
# Each call returns one bit, so you can never beat
# ceil(log2(number of candidates)) calls. Without a
# dictionary the best you can do is order the questions by
# likelihood and stop at the first yes for each position
# (and skip the 26th letter, because it is implied). With
# a dictionary, keep the list of words still possible and
# ask the (position, letter) question whose yes/no answer
# splits that list most evenly.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# letters from most to least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"
```

```
# FOLLOW-UP 1: What is the lower bound and when is it
# reached?
# Answer: Every call returns at most one bit, so at least
# ceil(log2(candidates)) calls are needed. The entropy
# split reaches it only when a question can cut the
# candidates exactly in half; real dictionaries usually
# need a few more.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_score(words, i, c):
    # how evenly does asking (position i, letter c) split?
    yes = sum(w[i] == c for w in words)
    return min(yes, len(words) - yes)

# ---- main question: main function ----
# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def by_frequency(query):
    # no dictionary: common letters first, stop at the
    # first
    # yes, and skip the 26th letter (it must be the one
    # left)
    calls, word = 0, []
    for i in range(5):
        for k, c in enumerate(BY_FREQUENCY):
            if k == 25:
                word.append(c)
                break
            calls += 1
            if query(i, c):
                word.append(c)
                break
    return "".join(word), calls


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
```

```
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def with_dictionary(query, dictionary):
    # keep the words that still fit every answer so far
    words, calls = list(dictionary), 0
    while len(words) > 1:
        options = {(i, w[i]) for w in words
                   for i in range(5)}
        # ask the question that cuts the list closest to
        # half
        i, c = max(options, key=lambda o: split_score(
            words, o[0], o[1]))
        calls += 1
        hit = query(i, c)
        words = [w for w in words if (w[i] == c) == hit]
    return words[0], calls

# ---- main question: tests ----
def test_main_question():
    def oracle(word):
        return lambda i, c: word[i] == c
    secret = "grape"
    word, calls = by_frequency(oracle(secret))
    assert word == secret and calls <= 125
    words = ["grape", "grade", "trade", "crane", "plane",
             "slate", "stare", "share"]
    for target in words:
        ask = oracle(target)
        found, used = with_dictionary(ask, words)
        assert found == target
        # about log2(8) = 3 questions suffice; allow a few
        # more
        assert used <= 6
    # lower bound: one bit per call, so ceil(log2(8)) = 3
    # calls
    assert min(with_dictionary(oracle(w), words)[1]
               for w in words) >= 3


# ============================================================
# FOLLOW-UP 1: What is the lower bound and when is it
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

STUDY PATH: question 266 of 288 | topic 39 of 43: Games: win or lose with a cache, and guessing with feedback | 3 of 5

Template fit: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap | asked 287 days ago

HARD | PracHub | Technical Screen | 266 of 288

Determine winner in stack-merging game

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 31.2 | 30 DAYS 0 | PRIORITY 141/288

ABOUT THIS QUESTION

Topic: Games: win or lose with a cache, and guessing with feedback

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #877 Stone Game

LeetCode match: Related: LC 292 Nim Game, LC 464 Can I Win, LC 877 Stone Game (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-14)

Original: https://prachub.com/coding-questions/determine-winner-in-stack-merging-game

QUESTION

A game starts with stacks of the given sizes. A move merges two stacks into one if their combined size is at most cap. The player who cannot move loses (in the misere variant, the player who cannot move wins). Return whether the first player wins with best play.

Function signature

def first_player_wins(stacks: list, cap: int, misere=False) -> bool

EXAMPLES

Example 1	
Input	`[1, 1], 2`
Output	`true`
Why	`Merge the two stacks of 1 into 2; the opponent has a single stack and cannot move.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`true`
Why	`Any merge makes a stack of 2 next to a stack of 1, whose sum 3 exceeds the cap: the opponent is stuck.`

Example 3	
Input	`[3], 5`
Output	`false`
Why	`A single stack: the first player cannot move and loses.`

QUESTION

Example 4	
Input	`[3], 5, misere = true`
Output	`true`
Why	`The same single stack in the misere variant: being unable to move wins.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(state):
    for nxt in all_merges(state):
        if not wins(nxt):
            return True  # opponent loses
    return False  # no move wins
```

TIME
exponential in the number...
exponential in the number of stacks

SPACE
O(depth)

WHY IT IS SLOW

The same position is reached through many different move orders, so the work is exponential. Remembering the result per position (sorted state as key) solves each position once.

OPTIMAL SOLUTION · Memoize on the sorted state

TIME
O(number of distinct sorted states * k^2)

SPACE
O(number of states)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The game is a sum of independent games (several separate boards or piles).

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

FOLLOW-UP 2 Misere play (the player who cannot move wins).

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

FOLLOW-UP 3 Count the number of winning first moves.

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Determine winner in stack-merging game
#
# A game starts with stacks of the given sizes. A move
# merges two stacks into one if their combined size is at
# most cap. The player who cannot move loses (in the
# misere variant, the player who cannot move wins). Return
# whether the first player wins with best play.
#
# Signature:
#   def first_player_wins(stacks: list, cap: int, misere=F
alse) -> bool
#
# Examples:
# [1, 1], 2  ->  true
# [1, 1, 1], 2  ->  true
#
# FOLLOW-UPS (answered below, in this order):
# 1. The game is a sum of independent games (several
# separate boards or piles).
# 2. Misere play (the player who cannot move wins).
# 3. Count the number of winning first moves.
#
# INTERVIEW NOTES
# Two-player game with perfect play and no draws: define
# win(state) = some move leads to a state where the
# opponent loses. A state with no moves is a loss for
# whoever must move. If a move does not care about stack
# order, sort the state so equal positions share one cache
# entry. If the game splits into independent parts, use
# Grundy numbers and XOR them.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from functools import lru_cache


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
```

```
def moves(state, cap):
    # merge two stacks if the new stack fits under the cap
    seen = set()
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = [x for k, x in enumerate(state)
                        if k not in (i, j)]
                merged = state[i] + state[j]
                new = tuple(sorted(rest + [merged]))
                # equal results are the same move
                if new not in seen:
                    seen.add(new)
                    yield new

# ---- main question: main function ----
# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def first_player_wins(stacks, cap, misere=False):
    # the player who cannot move loses (misere: wins)
    @lru_cache(None)
    def wins(state):
        options = list(moves(state, cap))
        if not options:
            return misere
        # a winning move leaves the opponent in a losing
        # state
        return any(not wins(nxt) for nxt in options)

    return wins(tuple(sorted(stacks)))

# ---- main question: tests ----
def test_main_question():
    # one stack: no move, so the first player loses
    assert not first_player_wins([3], 5)
    # two stacks that can merge: one move ends it, so we
    # win
    assert first_player_wins([1, 1], 5)
    # three stacks: after one merge the other two may
    # merge too
    assert first_player_wins([1, 1, 1], 2) is True
    # in misere play the result flips for a single stack
    assert first_player_wins([3], 5, misere=True)


# ============================================================
# FOLLOW-UP 1: The game is a sum of independent games
# ============================================================

# ---- follow-up 1: function ----
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def grundy(stacks, cap):
    # Sprague-Grundy number of one game (mex of its moves)
    @lru_cache(None)
    def g(state):
        reach = {g(nxt) for nxt in moves(state, cap)}
        number = 0
        while number in reach:
            number += 1
        return number

    return g(tuple(sorted(stacks)))


# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
```

```
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def sum_of_games_wins(boards, cap):
    # independent boards: XOR their Grundy numbers
    total = 0
    for board in boards:
        total ^= grundy(board, cap)
    return total != 0
```

```
# ============================================================
# FOLLOW-UP 2: Misere play (the player who cannot move
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Count the number of winning first moves.
# ============================================================

# ---- follow-up 3: function ----
# Answer: Keep the same memoized win/lose function and
# count the moves whose resulting state is a loss for the
# opponent.
def winning_first_moves(stacks, cap):
    # how many different first moves win?
    state = tuple(sorted(stacks))
    return sum(1 for nxt in moves(state, cap)
               if not first_player_wins(nxt, cap))

# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert winning_first_moves([1, 1, 1], 2) >= 1
    # normal play: a position wins exactly when Grundy !=
    # 0
    for state in ([1, 2, 3], [2, 2, 2, 2], [1, 1, 4]):
        win = first_player_wins(state, 6)
        assert win == (grundy(state, 6) != 0)
    # two separate boards: XOR of their numbers
    both = sum_of_games_wins([[1, 1], [1, 1]], 5)
    assert both is False


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(colors: list[str]) -> bool:
SAMPLE INPUT
['Y']
```

Test cases

Input	Expected
`['Y']`	`False`
`['Y', 'W']`	`True`

ALSO REPORTED · 1 more

Babylon Tile-Stack Merge Game | 1Point3Acres | onsite-coding | asked 2025-12-29 | Single report

https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API. Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".

Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).

Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

Preparation: Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.

Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.

Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

STUDY PATH: question 267 of 288 | topic 39 of 43: Games: win or lose with a cache, and guessing with feedback | 4 of 5

Template fit: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count | asked 251 days ago

HARD | IN APP LC 843 | PracHub | Technical Screen | 267 of 288

Find secret word with match-count feedback

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 97/288

ABOUT THIS QUESTION

Topic: Games: win or lose with a cache, and guessing with feedback

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word

Posted on PracHub: 2026-02-11 (updated 2026-04-30)

Original: https://prachub.com/coding-questions/find-secret-word-with-match-count-feedback

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- `matchCount(guess) -> int`: when you submit a word `guess` from the list, the API returns the number of positions `i` such that `guess[i] == secret[i]` (i.e., exact character match at the same index). You are allowed to call `matchCount`

at most 10 times.

Task

Design/implement a function (e.g., `findSecretWord(words)`) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- `words` contains between 1 and 100 words.
- All words have equal length `L` (commonly `L = 6`).
- Every guess must be a word from `words`.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once `matchCount(guess) == L` (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length `L`. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- `matchCount(guess) -> int`

QUESTION

- It returns how many positions `i` satisfy `guess[i] == secret[i]` You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement `solution(words, secret)` and simulate the API inside your function. The parameter `secret` is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string `""`.

Constraints`1 <= len(words) <= 1001 <= len(words[i]) <= 10`All words are unique, lowercase, and have the same length `Lsecret` is guaranteed to be one of the words in `words`Your algorithm may use at most 10 simulated `matchCount` callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"],
    "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until `acckzz` is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns `k`, any valid remaining candidate must also have exactly `k` matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    candidates = list(words)
    while True:
        guess = candidates[0]
        m = matchCount(guess)
        if m == len(guess):
            return guess
        candidates = [w for w in candidates
                      if matches(w, guess) == m]
```

TIME	SPACE
O(guesses * W * L)	O(W)

WHY IT IS SLOW

BRUTE FORCE

A poor guess may remove very few candidates, so there is no guarantee of finishing in 10 guesses. Guessing the word whose largest match-count group is smallest (minimax) keeps the list shrinking fast.

OPTIMAL SOLUTION · Guess the word that minimises the size of its largest match-count group

TIME	SPACE
O(guesses * W^2 * L)	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Show how the candidate list shrinks.

After a guess with answer m, only words with the same match count against the guess survive (the secret and the candidate agree on every comparison). Printing the sizes round by round shows why a balanced guess finishes in a few rounds.

FOLLOW-UP 2 Guess at random instead of the minimax choice.

Expected to finish in far fewer than W guesses, but not guaranteed within 10 on a hard list. The minimax choice is the safe answer; random with a 'zero-match' preference (guess words that share no letters with earlier guesses) is a cheap improvement.

FOLLOW-UP 3 The word list is much larger.

Computing all pairwise match counts is O(W^2 L). Sample candidates when choosing a guess, or precompute buckets by letter at each position to count matches faster.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import Counter
import itertools
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def match(a, b):
    # number of positions holding the same letter
    return sum(x == y for x, y in zip(a, b))

# ---- main question: main function ----
# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def best_guess(candidates):
    # minimax: the guess whose worst answer leaves the
    # fewest
    # candidates (ties: alphabetical)
    def worst(g):
        sizes = Counter(match(g, w) for w in candidates)
        return max(sizes.values()), g
    return min(candidates, key=worst)


# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API
```

```
    candidates, guesses = list(words), 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        # only words with the same score against this
        # guess
        # can still be the secret
        candidates = [w for w in candidates
                      if match(guess, w) == score]
    return None, guesses

# ---- main question: tests ----
def test_main_question():
    words = ["".join(p) for p in
             itertools.product("abc", repeat=4)]
    worst = 0
    for secret in words:
        calls = []

        def api(guess, secret=secret):
            calls.append(guess)
            return match(guess, secret)

        found, used = find_secret(words, api)
        assert found == secret and used == len(calls)
        worst = max(worst, used)
    # 81 words are always found inside the limit of 10
    assert worst <= 10
    single = find_secret(["only"], lambda g: 4)
    assert single == ("only", 1)


# ============================================================
# FOLLOW-UP 1: Show how the candidate list shrinks.
# ============================================================
```

```
# ---- follow-up 1: function ----
# Answer: After a guess with answer m, only words with the
# same match count against the guess survive (the secret
# and the candidate agree on every comparison).
def shrink_report(words, secret):
    # candidate counts after each guess (shows why it
```

```
    # works)
    sizes, candidates = [len(words)], list(words)
    while len(candidates) > 1:
        guess = best_guess(candidates)
        score = match(guess, secret)
        if score == len(secret):
            break
        candidates = [w for w in candidates
                      if match(guess, w) == score]
        sizes.append(len(candidates))
    return sizes

# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = ["".join(p) for p in
             itertools.product("abc", repeat=4)]
    sizes = shrink_report(words, "abca")
    assert sizes[0] == 81
    assert sizes == sorted(sizes, reverse=True)


# =============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# =============================================================
def _match(a, b):
    return sum(x == y for x, y in zip(a, b))


def find_secret_first(words, secret):
    cand, guesses = list(words), 0
    while True:
        g = cand[0]
        guesses += 1
        m = _match(g, secret)
        if m == len(secret):
            return guesses
        cand = [w for w in cand if _match(g, w) == m]


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], secret: str) -> str:
```

Sample call

```
solution(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz
")
```

Test cases

Input	Expected
(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")	'acckzz'
(["secret"], "secret")	'secret'

STUDY PATH: question 268 of 288 | topic 39 of 43: Games: win or lose with a cache, and guessing with feedback | 5 of 5

Template fit: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply | asked 295 days ago | frequency Single report

HARD | 1Point3Acres | onsite-coding | 268 of 288

Babylon Tile-Stack Merge Game

ASKED 2025-12-29 | 295 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 251/288

ABOUT THIS QUESTION

Topic: Games: win or lose with a cache, and guessing with feedback

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2025-12-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

QUESTION

A two-player game on stacks of tiles. Each stack has a height and a colour (the colour of its top tile). A move places one stack on top of another when the two stacks have the same height or the same colour; the new stack has the combined height and the colour of the stack that was placed on top. The player who cannot move loses. Given the stacks as (height, colour) pairs, return whether the player to move wins with best play.

Function signature

def babylon_wins(stacks: list) -> bool

EXAMPLES

Example 1	
Input	`[(1, 'Y'), (1, 'W')]`
Output	`true`
Why	`Two stacks of height 1: the mover merges them, and the opponent has nothing left to move.`

Example 2	
Input	`[(1, 'Y')]`
Output	`false`
Why	`A single stack: the mover has no move and loses.`

Example 3	
Input	`[(1, 'Y'), (2, 'W')]`
Output	`false`
Why	`Heights differ and colours differ: no move is possible, so the mover loses.`

QUESTION

Example 4	
Input	`[(1, 'Y'), (1, 'Y'), (1, 'W'), (1, 'W')]`
Output	`true`
Why	`Four height-1 stacks of two colours: the mover has a winning plan.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(position):
    for i, j in legal_moves(position):
        nxt = merge(position, i, j)
        if not wins(nxt):
            return True  # opponent loses
    return False
```

TIME	SPACE
exponential	O(depth)

WHY IT IS SLOW

Explores the same positions many times, exponential. Sorting the stacks and caching each position's result visits every distinct position once.

OPTIMAL SOLUTION · Negamax with memoisation on the sorted stack tuple

TIME	SPACE
O(states * moves^2)	O(states)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How big is the real state space?

With 12 starting stacks the first position has at most 66 unordered pairs and the number of reachable multisets of (height, colour) stacks shrinks quickly because every merge removes a stack, so memoised search is feasible. In the interview build the clean state class, legal moves and play(move) first and add the solver only if there is time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Babylon Tile-Stack Merge Game
#
# A two-player game on stacks of tiles. Each stack has a
```

```
# height and a colour (the colour of its top tile). A move
# places one stack on top of another when the two stacks
# have the same height or the same colour; the new stack
# has the combined height and the colour of the stack that
# was placed on top. The player who cannot move loses.
# Given the stacks as (height, colour) pairs, return
# whether the player to move wins with best play.
#
# Signature:
#   def babylon_wins(stacks: list) -> bool
#
# Examples:
# [(1, 'Y'), (1, 'W')]  ->  true
# [(1, 'Y')]  ->  false
#
# FOLLOW-UPS (answered below, in this order):
# 1. How big is the real state space?
#
# INTERVIEW NOTES
# Model a state as the list of stacks, each as (height,
# top colour). A legal move puts stack i on stack j when
# the heights are equal or the top colours are equal; the
# new stack has the summed height and the colour of the
# stack that was placed on top. The player with no legal
# move loses, so a state is winning when some move leads
# to a losing state. The order of stacks does not matter,
# so sort the list to share states in a cache.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import itertools


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
class Babylon:
    # a stack is (height, colour of its top tile)
    def __init__(self, stacks=None):
        if stacks is None:
            # 12 tiles: Y, W, G, B three times each
            stacks = [(1, c) for c in "YWGB" * 3]
        self.stacks = list(stacks)

    def legal_moves(self):
        # put stack i on top of stack j
        moves = []
        for i, j in itertools.permutations(
                range(len(self.stacks)), 2):
            h1, c1 = self.stacks[i]
            h2, c2 = self.stacks[j]
            # same height, or the same colour on top
            if h1 == h2 or c1 == c2:
                moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move (the original is kept)
        (h1, c1), (h2, _) = self.stacks[i], self.stacks[j]
        rest = [s for k, s in enumerate(self.stacks)
                if k not in (i, j)]
        # heights add; the top colour is the moved stack's
        return rest + [(h1 + h2, c1)]
```

```
    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
        # the player with no move loses
        return not self.legal_moves()

# ---- main question: main function ----
# BASE QUESTION (optimal): Negamax with memoisation on the
# sorted stack tuple
# Idea: legal_moves lists the ordered pairs that may
# merge, apply builds the next state, and wins(state) is
# true when some move reaches a state where the opponent
# loses.
```

```
def wins(stacks):
    # does the player to move win with best play?
    cache = {}

    def solve(state):
        if state in cache:
            return cache[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            nxt = tuple(sorted(game.after(i, j)))
            # a move to a position the opponent loses wins
            if not solve(nxt):
                result = True
                break
        cache[state] = result
        return result

    # sorted stacks: equal positions share one cache entry
    return solve(tuple(sorted(stacks)))


# PICK the winning move itself
def best_move(game):
    # pick a move that leaves the opponent lost, if any
    for i, j in game.legal_moves():
        if not wins(game.after(i, j)):
            return i, j
    moves = game.legal_moves()
    return moves[0] if moves else None

# ---- main question: tests ----
def test_main_question():
    # one stack: no moves, so the player to move loses
    assert Babylon([(1, "Y")]).is_over()
    assert not wins([(1, "Y")])
    # two equal stacks: one move ends the game, so we win
    assert wins([(1, "Y"), (1, "W")])
    g = Babylon([(1, "Y"), (1, "Y"), (1, "W")])
    assert (0, 1) in g.legal_moves()
    # play a whole game with the solver; the loser is
    # stuck
    while not g.is_over():
        g.play(*best_move(g))
    assert len(g.stacks) >= 1
```

```
    try:
        Babylon([(1, "Y"), (2, "W")]).play(0, 1)
        raise AssertionError("move should be illegal")
    except ValueError:
        pass
    # the real start has 12 stacks and up to 132 ordered
    # moves
    assert len(Babylon().legal_moves()) > 0
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 40 OF 43

Sliding window: grow on the right, shrink on the left while it is too bad

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	1
MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Sliding window: grow on the right, shrink on the left while it is too bad | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

M	269. Temperature Monitoring Data Structure	1Point3Acres	2026-09-10
M	270. Shortest Subarray with K Distinct Integers	HackTheRounds	no date
M	271. Find the minimum window substring.	Exponent	no date
M	272. Footpath Beauty After Repainting	ColorStack	2025-09-22
H	273. Necklace Cut into Two Halves with Equal D/R Counts	1Point3Acres	2026-05-03

CODE TEMPLATE | GROUP 40 OF 43 | 5 QUESTIONS, ONE PATTERN

Sliding window: grow on the right, shrink on the left while it is too bad

The one idea these questions share

Move the right end one step at a time and add that item to your state. While the window breaks the rule, remove items from the left end. After each step the window is valid, so record its length (or the shortest valid one).

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def longest_window(items, bad, allowed):
    left = used = best = 0
    for right, x in enumerate(items):
        used += bad(x)                          # CHANGE 1: add the right item to the state
        while used > allowed:                   # CHANGE 2: when the window is invalid
            used -= bad(items[left])            # CHANGE 3: remove the left item from the state
            left += 1
        best = max(best, right - left + 1)      # record the window
    return best
```

2. Helper functions that usually get written next to it

```
from collections import Counter, deque


def fixed_window_sums(nums, size):
    total = sum(nums[:size])
    out = [total]
    for i in range(size, len(nums)):
        total += nums[i] - nums[i - size]       # add the new item, drop the old one
        out.append(total)
    return out


def window_max(nums, size):                     # monotonic deque: the front is always the maximum
    q, out = deque(), []
    for i, x in enumerate(nums):
        while q and nums[q[-1]] <= x:
            q.pop()
        q.append(i)
        if q[0] <= i - size:
            q.popleft()
        if i >= size - 1:
            out.append(nums[q[0]])
    return out
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
269	Temperature Monitoring Data Structure	a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum
270	Shortest Subarray with K Distinct Integers	state = a count dict; while it holds at least k distinct values, record the window and shrink from the left
271	Find the minimum window substring.	state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink
272	Footpath Beauty After Repainting	bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer
273	Necklace Cut into Two Halves with Equal D/R Counts	a fixed-size window of half the length slides around the doubled string, keeping a count of D

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 269 of 288 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 1 of 5

Template fit: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum | asked 40 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 269 of 288

Temperature Monitoring Data Structure

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 283/288**

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

QUESTION

Design a monitor over the last n temperature readings. insert(x) adds a reading (the oldest one drops out once more than n are stored), average() returns the mean of the stored readings, and maximum() returns the largest of them in O(1).

Function signature

class Monitor:
  def __init__(self, n: int)
  def insert(self, x: float) -> None
  def average(self) -> float
  def maximum(self) -> float

EXAMPLES

Example 1

Input	`m = Monitor(3)`
	`m.insert(20)`
	`m.insert(22)`
	`m.insert(19)`
	`m.average()`
	`m.insert(25)`
	`m.average()`
	`m.maximum()`
	`m.insert(10)`
	`m.maximum()`

Output	`m.average() -> 20.333333`
	`m.average() -> 22`
	`m.maximum() -> 25`
	`m.maximum() -> 25`

Why	`With n = 3 the window slides: after 25 arrives the window is 22, 19, 25 (average 22, maximum 25); after 10 it is 19, 25, 10, so the maximum is still 25.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
readings = []
def insert(x):
    readings.append(x)
def average():
    window = readings[-N:]
    return sum(window) / len(window)
def maximum():
    return max(readings[-N:])
```

TIME	SPACE
O(N) per average or max	**O(N)**

WHY IT IS SLOW

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

OPTIMAL SOLUTION · Running sum plus a monotonic deque

TIME	SPACE
O(1) amortized per operation	**O(N)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

FOLLOW-UP 2 Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

FOLLOW-UP 3 Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Temperature Monitoring Data Structure
#
# Design a monitor over the last n temperature readings.
# insert(x) adds a reading (the oldest one drops out once
# more than n are stored), average() returns the mean of
# the stored readings, and maximum() returns the largest
# of them in O(1).
#
# Signature:
#   class Monitor:
#       def __init__(self, n: int)
#       def insert(self, x: float) -> None
#       def average(self) -> float
#       def maximum(self) -> float
#
# FOLLOW-UPS (answered below, in this order):
# 1. The window is the last D days (elapsed time), not the
# last N readings.
# 2. Also support minimum, or the median.
# 3. Floating-point drift in the running sum.
#
# INTERVIEW NOTES
# Three operations on the last N readings. The average
# only needs a running sum, and the maximum needs a
# monotonic deque that drops smaller values from the back
# as larger ones arrive, so the front is always the max of
# the window. Decide first whether the window is a count
# of readings or a span of time.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# FOLLOW-UP 2: Also support minimum, or the median.
# Answer: Minimum uses a second monotonic deque with the
# opposite comparison. A median needs two heaps with lazy
# deletion or a sorted container, O(log N) per update,
# because it is not decomposable like sum and max.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
        self.maxq.drop_while(too_old)
        self.minq.drop_while(too_old)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()

# ---- main question: main function ----
# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
# is full.
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)

# ---- main question: tests ----
def test_main_question():
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    # window is [1, 4, 2]
```

```
    assert m.maximum() == 4 and m.minimum() == 1
    assert m.average() == 7 / 3
    m.insert(0)
    # window is [4, 2, 0]
```

```
    assert m.maximum() == 4 and m.minimum() == 0


# ========================================================
# FOLLOW-UP 1: The window is the last D days (elapsed
# ========================================================

# ---- follow-up 1: function ----
# Answer: Store (timestamp, value) pairs and evict from
# the front while the timestamp is at or before now minus
# the span, in both the window and the max deque.
# Everything else is unchanged.
class TimedMonitor(SlidingStats):
    """Window = readings from the last `span` days."""

    def __init__(self, span):
        super().__init__()
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        # drop readings at or before day - span
        cutoff = day - self.span
        self.expire(lambda key: key <= cutoff)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        t.insert(day, x)
    # day 1 is two days old, so it left the window
    assert t.maximum() == 5 and t.average() == 4


# ========================================================
# FOLLOW-UP 2: Also support minimum, or the median.
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

STUDY PATH: question 270 of 288 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 2 of 5

Template fit: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left | no date

MEDIUM | HackTheRounds | Onsite | 270 of 288

★ TOP 75: rank 66 in the Medium and Hard priority list

★ Shortest Subarray with K Distinct Integers

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 33 | PRIORITY 45/288

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #992 Subarrays with K Different Integers, LeetCode #76 Minimum Window Substring

LeetCode match: LC 992 / LC 76 flavor

QUESTION

Given an array nums and an integer k, return the length of the shortest contiguous subarray that contains at least k different integers, or -1 if there is no such subarray.

Function signature

def shortest_k_distinct(nums: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 1, 3, 2], 3`
Output	`3`
Why	`[2, 1, 3] is the shortest window holding three different numbers.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`-1`
Why	`Only one distinct number exists.`

Example 3	
Input	`[1, 2], 2`
Output	`2`
Why	`The whole array is needed.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    best = -1
    for i in range(len(nums)):
        seen = set()
        for j in range(i, len(nums)):
            seen.add(nums[j])
            if len(seen) >= k:
                size = j - i + 1
                if best == -1 or size < best:
                    best = size
                break
    return best
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map expands on the right and shrinks on the left, so each element is handled twice: O(n).

OPTIMAL SOLUTION · Sliding window with a frequency map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the subarrays with exactly k distinct values (LeetCode 992).

exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding window that shrinks while the distinct count exceeds the limit. Both are O(n).

FOLLOW-UP 2 Longest subarray with at most k distinct values.

The same window, but keep the longest window whose distinct count never exceeds k (shrink while it does). O(n).

FOLLOW-UP 3 The values are strings or large objects.

The frequency map works on any hashable value; for huge objects hash them first or map them to integer ids so the window comparisons stay cheap. **Cost:** O(n) plus hashing each element; mapping values to integer ids costs O(n) once.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Shortest Subarray with K Distinct
# Integers
#
# Given an array nums and an integer k, return the length
# of the shortest contiguous subarray that contains at
# least k different integers, or -1 if there is no such
# subarray.
#
# Signature:
#   def shortest_k_distinct(nums: list, k: int) -> int
#
# Examples:
# [1, 2, 1, 3, 2], 3  ->  3
# [1, 1, 1], 2  ->  -1
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays with exactly k distinct values
# (LeetCode 992).
# 2. Longest subarray with at most k distinct values.
# 3. The values are strings or large objects.
#
# INTERVIEW NOTES
# Once a window has at least k distinct values, shrinking
# it from the left can only keep or lose distinct values,
# so the shortest valid window ending at each right end is
# found by shrinking as far as possible. A frequency map
# tells you when a value leaves the window, so each index
# enters and leaves once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
```

```
# BASE QUESTION (optimal): Sliding window with a frequency
# map
# Idea: Expand the right end, counting each value. While
# the window has at least k distinct values, record its
# length and shrink from the left (deleting a value from
# the map when its count reaches 0).
def shortest_k_distinct(nums, k):
    # a window grows on the right; whenever it holds at
    # least k distinct values, record it and shrink from
    # the
    # left as far as it stays valid
    seen, left, best = Counter(), 0, None
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) >= k:
            size = right - left + 1
            if best is None or size < best:
                best = size
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
    return -1 if best is None else best

# ---- main question: tests ----
def test_main_question():
    assert shortest_k_distinct([1, 2, 1, 3, 4], 3) == 3
    assert shortest_k_distinct([1, 1, 1], 2) == -1
    assert shortest_k_distinct([5], 1) == 1
    assert shortest_k_distinct([], 1) == -1


# ============================================================
# FOLLOW-UP 1: Count the subarrays with exactly k distinct
# ============================================================

# ---- follow-up 1: helper functions ----
def at_most(nums, k):
    # the number of subarrays with at most k distinct
    # values:
    # for every right end, all windows starting in
    # [left, right] qualify
    seen, left, total = Counter(), 0, 0
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) > k:
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
        total += right - left + 1
    return total

# ---- follow-up 1: function ----
# Answer: exactly(k) = atMost(k) - atMost(k - 1), where
# atMost is a sliding window that shrinks while the
# distinct count exceeds the limit. Both are O(n).
def exactly(nums, k):
    # follow-up (LeetCode 992): exactly k distinct values
    return at_most(nums, k) - at_most(nums, k - 1)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert exactly([1, 2, 1, 2, 3], 2) == 7
    assert exactly([1, 2, 1, 3, 4], 3) == 3


# ============================================================
# FOLLOW-UP 2: Longest subarray with at most k distinct
# ============================================================

# ---- follow-up 2: function ----
# Answer: The same window, but keep the longest window
# whose distinct count never exceeds k (shrink while it
# does). O(n).
def longest_at_most(nums, k):
    # follow-up: the longest window with at most k
    # distinct
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) > k:
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
```

```
        best = max(best, right - left + 1)
    return best

# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert longest_at_most([1, 2, 1, 2, 3], 2) == 4
    words = ["a", "b", "a", "c"]


# ============================================================
# FOLLOW-UP 3: The values are strings or large objects.
# ============================================================

# ---- follow-up 3: function ----
# Answer: The frequency map works on any hashable value;
# for huge objects hash them first or map them to integer
# ids so the window comparisons stay cheap. Cost: O(n)
# plus hashing each element; mapping values to integer ids
# costs O(n) once.
def to_ids(items):
    # follow-up: strings or big objects. Map each distinct
    # value to a small integer once, then reuse the same
    # code.
    ids = {}
    return [ids.setdefault(x, len(ids)) for x in items]

# ---- follow-up 3: tests ----
def test_follow_up_3():
    words = ["a", "b", "a", "c"]
    assert shortest_k_distinct(to_ids(words), 3) == 3
    assert to_ids(["x", "y", "x"]) == [0, 1, 0]
    for _ in range(400):
        nums = [random.randint(1, 4)
                for _ in range(random.randint(0, 10))]
        k = random.randint(1, 5)
        count = sum(len(set(nums[i:j + 1])) == k
                    for i in range(len(nums))
                    for j in range(i, len(nums)))
        assert exactly(nums, k) == count


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

ALSO REPORTED · 2 more

Find shortest subarray with at least k distinct | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-at-least-k-distinct

Find shortest subarray with >=k distinct integers | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-k-distinct-integers

STUDY PATH: question 271 of 288 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 3 of 5

Template fit: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink | no date

MEDIUM | IN APP LC 76 | Exponent | 271 of 288

★ TOP 75: rank 55 in the Medium and Hard priority list

★ Find the minimum window substring.

ASKED (ANY CO.) ~7 MONTHS AGO | OLDER

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 33 | PRIORITY 44/288

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: your app's interview approach (LeetCode #76)

LeetCode match: LC 76 Minimum Window Substring (inferred from short statement)

Last asked (any company): 7 months ago, about 2026-03 (Exponent shows only a rough age) | +36 interviews

Original: https://www.aced.io/questions/1687/minimum-window-substring

QUESTION

Given strings s and t, return the shortest substring of s that contains every character of t (a character that appears twice in t must appear at least twice in the substring). Return the empty string if there is no such substring.

Function signature

def min_window(s: str, t: str) -> str

EXAMPLES

Example 1	
Input	'ADOBECODEBANC', 'ABC'
Output	'BANC'
Why	BANC is the shortest window that holds an A, a B and a C.

Example 2	
Input	'a', 'a'
Output	'a'
Why	The whole string is the only window.

Example 3	
Input	'a', 'aa'
Output	''
Why	t needs two a's but s has only one.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(s, t):
    found = []
    for i in range(len(s)):
        for j in range(i + len(t), len(s) + 1):
            part = s[i:j]
            if contains_all(part, t):
                found.append(part)
    return min(found, key=len, default="")
```

TIME	SPACE
O(n^2 * m)	O(m)

WHY IT IS SLOW

O(n^2) substrings, each checked in O(m) or more. A sliding window that grows until valid and then shrinks from the left is O(n + m).

OPTIMAL SOLUTION · Sliding window with a 'still needed' counter

TIME	SPACE
O(n + m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The smallest window containing every distinct character of s itself.

Use t = the set of distinct characters of s and run the same window. A related problem: the longest substring with at most k distinct characters uses a shrink-when-too-many window.

FOLLOW-UP 2 Return all minimum windows, or the number of valid substrings.

For all minimum windows record every window equal to the best length. For counting, when the window [left, right] is valid, every extension to the right is valid too, so add len(s) - right for each left.

FOLLOW-UP 3 Order matters: t must appear as a subsequence.

That is a different problem (LeetCode 727): DP over positions, or a two-pass scan (forward to match, backward to shrink) for each possible end.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Find the minimum window substring.
#
# Given strings s and t, return the shortest substring of
# s that contains every character of t (a character that
# appears twice in t must appear at least twice in the
# substring). Return the empty string if there is no such
# substring.
#
# Signature:
#   def min_window(s: str, t: str) -> str
#
# Examples:
# 'ADOBECODEBANC', 'ABC'  ->  'BANC'
# 'a', 'a'  ->  'a'
#
# FOLLOW-UPS (answered below, in this order):
# 1. The smallest window containing every distinct
# character of s itself.
# 2. Return all minimum windows, or the number of valid
# substrings.
# 3. Order matters: t must appear as a subsequence.
#
# INTERVIEW NOTES
# Expand the right end until the window covers everything
# in t, then shrink from the left as far as it stays
# valid, record the length, and continue. A count of
# 'characters still missing' makes the validity check
# O(1): it goes down when a needed character is added and
# up when one is removed. Each index enters and leaves the
# window once.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import Counter
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: helper functions ----
def valid_windows(s, t):
    # sliding window: grow `right` until the window holds
    # all
    # of t, then shrink `left` while it still does. Yield
    # every (left, right) that is valid when it is seen.
    need = Counter(t)
    missing = len(t)
    left = 0
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:
            yield left, right
            # drop the leftmost character and see if it
            # hurts
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1

# ---- main question: main function ----
# BASE QUESTION (optimal): Sliding window with a 'still
```

```
# needed' counter
# Idea: need[c] is how many more of c the window requires,
# and missing counts the distinct requirements still
# unmet.
def min_window(s, t):
    # the shortest window; the first one if there is a tie
    best, width = None, 0
    for left, right in valid_windows(s, t):
        if best is None or right - left < width:
            best, width = (left, right), right - left
    return s[best[0]:best[1] + 1] if best else ""

# ---- main question: tests ----
def test_main_question():
    assert min_window("ADOBECODEBANC", "ABC") == "BANC"
    assert min_window("a", "a") == "a"
    assert min_window("a", "aa") == ""
    assert min_window("ab", "b") == "b"
    assert min_window("aab", "aab") == "aab"
```

```
# ============================================================
# FOLLOW-UP 1: The smallest window containing every
# ============================================================

# ---- follow-up 1: function ----
# Answer: Use t = the set of distinct characters of s and
# run the same window. A related problem: the longest
# substring with at most k distinct characters uses a
# shrink-when-too-many window.
def window_all_distinct(s):
    # follow-up: contain every distinct character of s
    # itself
    return min_window(s, "".join(set(s)))

# ============================================================
# FOLLOW-UP 2: Return all minimum windows, or the number
# ============================================================

# ---- follow-up 2: function ----
# Answer: For all minimum windows record every window
# equal to the best length. For counting, when the window
# [left, right] is valid, every extension to the right is
# valid too, so add len(s) - right for each left.
def all_min_windows(s, t):
    # follow-up: every window of the minimum length
    found = list(valid_windows(s, t))
    if not found:
        return []
    size = min(r - l for l, r in found)
    return [s[l:r + 1] for l, r in found if r - l == size]


# Answer: For all minimum windows record every window
# equal to the best length. For counting, when the window
# [left, right] is valid, every extension to the right is
# valid too, so add len(s) - right for each left.
def count_valid_substrings(s, t):
    # follow-up: how many substrings contain all of t. For
    # each left the first valid right is seen once; every
    # longer end is valid too.
    found = valid_windows(s, t)
    return sum(len(s) - right for _, right in found)

# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert all_min_windows("abcabc", "abc") == [
        "abc", "bca", "cab", "abc"]
    assert all_min_windows("xyz", "q") == []
    assert count_valid_substrings("abcabc", "abc") == 10
    assert window_all_distinct("aabcbcdbca") == "dbca"
    assert longest_k_distinct("eceba", 2) == 3
```

```
    assert longest_k_distinct("aa", 1) == 2
    text = "abcdebdde"


# ========================================================
# FOLLOW-UP 3: Order matters: t must appear as a
# ========================================================

# ---- follow-up 3: function ----
# Answer: That is a different problem (LeetCode 727): DP
# over positions, or a two-pass scan (forward to match,
# backward to shrink) for each possible end.
def min_window_subsequence(s, t):
    # follow-up (LeetCode 727): t must appear IN ORDER.
    # Match
    # forward from a start to find the end, then walk back
    # from that end to find the tightest start.
    best, width, i = None, 0, 0
    while i < len(s):
        j, k = 0, i
        while k < len(s) and j < len(t):
            if s[k] == t[j]:
                j += 1
            k += 1
        if j < len(t):
            break
        end, j, k = k - 1, len(t) - 1, k - 1
        while j >= 0:
            if s[k] == t[j]:
                j -= 1
            k -= 1
        start = k + 1
        if best is None or end - start < width:
            best, width = (start, end), end - start
        i = start + 1
    return s[best[0]:best[1] + 1] if best else ""

# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    text = "abcdebdde"
    assert min_window_subsequence(text, "bde") == "bcde"
    assert min_window_subsequence("abc", "ca") == ""


# ========================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ========================================================
# HELPERS for the checks below
def longest_k_distinct(s, k):
    # related: the longest substring with at most k
    # distinct
    # letters shrinks when it has too many
    seen, left, best = Counter(), 0, 0
    for right, ch in enumerate(s):
        seen[ch] += 1
        while len(seen) > k:
            seen[s[left]] -= 1
            if seen[s[left]] == 0:
                del seen[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade nowLewin (Dropbox SWE) answers the minimum window substring interview question.

STUDY PATH: question 272 of 288 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 4 of 5

Template fit: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer | asked 393 days ago | frequency Single report

MEDIUM | IN APP LC 1004 | ColorStack | technical interview | 272 of 288

Footpath Beauty After Repainting

ASKED 2025-09-22 | 393 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 220/288

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: your app's interview approach (LeetCode #1004)

LeetCode match: LC 1004 Max Consecutive Ones III

Frequency: Single report | Last asked: 2025-09-22 | Roles: SWE Intern

QUESTION

A footpath is built from n blocks laid in a line, painted with one of k colors. block_colors[i] is the color of block i (colors are integers in 1..k). The beauty value for a given favorite color c is the length of the longest run of consecutive blocks that are all color c . You are allowed to repaint at most m blocks (each to any color you choose). Return the maximum possible beauty value for favorite color c after repainting at most m blocks.

Function signature

def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:

...

Follow-up (as reported): (Reported) Answer q queries, each with a different favorite color, and return the beauty value (with m fixed) for each. Hint: a sliding window per query, or precompute runs. (Reported) The base version with m = 0 was asked first; the repaint version is the twist.

EXAMPLES

Example 1

Input	`block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1`
Output	`4`
Why	`Repaint one of the 2's. The best window has at most one non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.`

Example 2

Input	`block_colors = [3, 3, 3], c = 3, m = 0`
Output	`3`
Why	`All blocks are already color 3; no repaint needed.`

QUESTION

Example 3

Input	`block_colors = [2, 2, 2], c = 1, m = 2`
Output	`2`
Why	`Only 2 repaints allowed, so at most 2 consecutive c blocks.`

CONSTRAINTS

- 1 <= n <= 10^5 , block_colors has length n
- 1 <= k , 1 <= c <= k , 1 <= block_colors[i] <= k
- 0 <= m <= n
- With m = 0 , the answer is simply the longest existing run of color c .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_beauty(colors, c, m):
    best = 0
    for start in range(len(colors)):
        other = 0
        for end in range(start,_len(colors)):
            other += colors[end] != c
            if other <= m:
                best = max(
                    best, end - start + 1
                )
    return best
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every start and end pair recounts its blocks.

OPTIMAL SOLUTION · Sliding window with at most m repaints

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Answer many queries, each with a different favourite colour (m fixed).

Group the positions of every colour once. For a colour run a window over its positions: the blocks between the first and last position of a window that are not that colour are the repaints needed; keep it at most m. The run length is the window size plus the unused repaints, capped by n. Total work is O(n + queries * positions of that colour).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Footpath Beauty After Repainting
#
# A footpath is built from n blocks laid in a line,
# painted with one of k colors. block_colors[i] is the
# color of block i (colors are integers in 1..k ). The
# beauty value for a given favorite color c is the length
# of the longest run of consecutive blocks that are all
# color c . You are allowed to repaint at most m blocks
# (each to any color you choose). Return the maximum
# possible beauty value for favorite color c after
# repainting at most m blocks. Function signature: def
# max_beauty_after_repaint(block_colors: list[int], c:
# int, m: int) -> int: ... Follow-up (as reported):
# (Reported) Answer q queries, each with a different
# favorite color, and return the beauty value (with m
# fixed) for each. Hint: a sliding window per query, or
# precompute runs. (Reported) The base version with m = 0
# was asked first; the repaint version is the twist.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Answer many queries, each with a different favourite
# colour (m fixed).
#
# INTERVIEW NOTES
# Repainting at most m blocks to the favourite colour
# means: find the longest window that contains at most m
# blocks of another colour, because those are the ones to
# repaint. A sliding window keeps the count of other-
# colour blocks and shrinks from the left when it goes
# over m.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import defaultdict
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Sliding window with at most m
# repaints
# Idea: The longest window holding at most m other-colour
# blocks is the answer.
def max_beauty_after_repaint(block_colors, c, m):
    left, other, best = 0, 0, 0
    for right, color in enumerate(block_colors):
        other += color != c
        while other > m:
            other -= block_colors[left] != c
            left += 1
        best = max(best, right - left + 1)
    return best

# ---- main question: tests ----
def test_main_question():
    random.seed(20)
    assert (
        max_beauty_after_repaint(
            [1, 2, 1, 1, 2, 1], 1, 1
        )
        == 4
    )
    assert (
        max_beauty_after_repaint([3, 3, 3], 3, 0) == 3
    )
    assert (
        max_beauty_after_repaint([2, 2, 2], 1, 2) == 2
    )


# ============================================================
# FOLLOW-UP 1: Answer many queries, each with a different
# ============================================================

# ---- follow-up 1: function ----
# Answer: Group the positions of every colour once. For a
# colour run a window over its positions: the blocks
# between the first and last position of a window that are
# not that colour are the repaints needed; keep it at most
# m.
def beauty_queries(block_colors, queries, m):
    n = len(block_colors)
```

```
    where = defaultdict(list)
    for i, color in enumerate(block_colors):
        where[color].append(i)
    out = []
    for c in queries:
        pos, left, best = where.get(c, []), 0, 0
        for right in range(len(pos)):
            while (
                pos[right] - pos[left] - (right - left)
                > m
            ):
                left += 1
            best = max(
                best, min(n, right - left + 1 + m)
            )
        out.append(best if pos else min(n, m))
    return out


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 273 of 288 | topic 40 of 43: Sliding window: grow on the right, shrink on the left while it is too bad | 5 of 5

Template fit: a fixed-size window of half the length slides around the doubled string, keeping a count of D | asked 170 days ago | frequency Single report

HARD | 1Point3Acres | onsite-coding | 273 of 288

Necklace Cut into Two Halves with Equal D/R Counts

ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 248/288

ABOUT THIS QUESTION

Topic: Sliding window: grow on the right, shrink on the left while it is too bad

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

QUESTION

A necklace is a circular string of beads D and R. Cut it in two places into two arcs of equal length that contain the same number of D beads (and so the same number of R beads). Return the two cut positions and the two arcs, or None if no such cut exists.

Function signature

def necklace_two_cuts(s: str) -> tuple | None

(cut1, cut2, arc1, arc2)

EXAMPLES

Example 1	
Input	`'DRDR'`
Output	`(0, 2, 'DR', 'DR')`
Why	`Cut at 0 and 2: both arcs are DR.`

Example 2	
Input	`'DDRR'`
Output	`(1, 3, 'DR', 'RD')`
Why	`Linear halves DD and RR are unbalanced, but rotating the cuts by one gives DR and RD.`

Example 3	
Input	`'DDR'`
Output	`null`
Why	`An odd number of beads cannot be split into equal arcs.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Necklace Cut into Two Halves with Equal
# D/R Counts
#
# A necklace is a circular string of beads D and R. Cut it
# in two places into two arcs of equal length that contain
# the same number of D beads (and so the same number of R
# beads). Return the two cut positions and the two arcs,
# or None if no such cut exists.
#
# Signature:
#   def necklace_two_cuts(s: str) -> tuple | None      # (
cut1, cut2, arc1, arc2)
#
# Examples:
# 'DRDR'  ->  (0, 2, 'DR', 'DR')
# 'DDRR'  ->  (1, 3, 'DR', 'RD')
#
# FOLLOW-UPS (answered below, in this order):
# 1. When is a single cut enough?
# 2. What if the two pieces must be equal strings, not
# just equal counts?
# 3. The two pieces must be equal strings, not just equal
# counts.
# 4. The two pieces must be equal strings, not just have
# equal counts.
#
# INTERVIEW NOTES
# Two arcs of a circle with the same letter counts must
# each have exactly half the D's and half the R's, so the
# total counts must be even and each arc has length n / 2.
# Slide a window of length n / 2 around the circle: the
# window and its complement have D counts adding to the
# total, and the count changes by at most one per step, so
# it must hit half somewhere.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length n/2, and the total D and R
# counts must both be even. Slide a window of length n/2
# around the circle (use s + s), tracking its D count.
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None


# SINGLE CUT case
def single_cut(s):
    # one cut on the line: a prefix with half of
    # everything
    half = len(s) // 2
    if len(s) % 2:
        return None
    d_prefix = s[:half].count("D")
    return half if d_prefix * 2 == s.count("D") else None

# ---- main question: tests ----
def test_main_question():
    # DRDR: one cut between the halves works
    assert single_cut("DRDR") == 2
```

```
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and cuts[2].count("D") == 1
    # DDRR needs two cuts on the circle: DR + DR
    assert single_cut("DDRR") is None
    i, j, a, b = necklace_two_cuts("DDRR")
    assert sorted(a) == sorted(b) == ["D", "R"]
    # odd totals have no answer
    assert necklace_two_cuts("DDR") is None
    assert necklace_two_cuts("DRDRD" + "R") is None


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: What if the two pieces must be equal
# ============================================================

# ---- follow-up 2: function ----
# Answer: That is a different question: the necklace must
# be a square rotation, i.e. some rotation of s is t + t.
# FOLLOW-UP 3: The two pieces must be equal strings, not
# just equal counts.
# Answer: Then the necklace must be a square in some
# rotation: some rotation of s equals t + t. Test with s
# in (s + s)[1:-1] restricted to an even-length split, or
```

```
# compare s[i:i + n/2] with s[i + n/2:i + n] using a
# rolling hash or KMP.
# FOLLOW-UP 4: The two pieces must be equal strings, not
# just have equal counts.
# Answer: A rotation of the necklace must be of the form t
# + t. Try every rotation and compare its two halves
# (O(n^2), or O(n) by searching s in (s + s)[1:-1] and
# checking the match at an offset of n / 2).
def equal_string_halves(s):
    # follow-up: the two pieces must be the same string
    n = len(s)
    if n % 2:
        return None
    half, doubled = n // 2, s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
```

```
    return None

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # equal strings: DRDR is DR + DR, DDRR is not
    assert equal_string_halves("DRDR") is not None
    assert equal_string_halves("DDRR") is None


# ============================================================
# FOLLOW-UP 3: The two pieces must be equal strings, not
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: The two pieces must be equal strings, not
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 41 OF 43

Parent pointers: store who is above each node and climb

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	4
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Parent pointers: store who is above each node and climb | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

E	274. Tree Node.	Exponent	no date
M	275. Employee Hierarchy Add / Delete	1Point3Acres	2026-06-10
M	276. Validate parent array forms a tree	PracHub	2026-02-11
M	277. Lowest Common Ancestor of a Binary Tree III	HackTheRounds	no date
M	278. Root a Tree So It Becomes a Binary Tree	ColorStack	2025-11-07

CODE TEMPLATE | GROUP 41 OF 43 | 5 QUESTIONS, ONE PATTERN

Parent pointers: store who is above each node and climb

The one idea these questions share

Keep parent[x] (or manager[x]). Climbing is a while loop that follows parent until it runs out. Questions about common ancestors, cycles and who reports to whom are all small variations of that climb.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def climb(x, parent):
    while x is not None:
        yield x
        x = parent.get(x)              # CHANGE 1: stop rule (no parent, or a repeat = cycle)


def lowest_common(a, b, parent):
    above = set(climb(a, parent))
    for x in climb(b, parent):         # CHANGE 2: what you look for while climbing
        if x in above:
            return x
    return None
```

2. Helper functions that usually get written next to it

```
def depth(x, parent):
    return sum(1 for _ in climb(x, parent)) - 1


def has_cycle(parent):
    for start in parent:
        seen, x = set(), start
        while x is not None:
            if x in seen:
                return True
            seen.add(x)
            x = parent.get(x)
    return False
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
274	Tree Node.	the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf
275	Employee Hierarchy Add / Delete	manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager
276	Validate parent array forms a tree	climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root
277	Lowest Common Ancestor of a Binary Tree III	two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor
278	Root a Tree So It Becomes a Binary Tree	the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 274 of 288 | topic 41 of 43: Parent pointers: store who is above each node and climb | 1 of 5

Template fit: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf | no date

EASY | Exponent | 274 of 288

Tree Node.

ASKED (ANY CO.) ~5 MONTHS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 278/288**

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (tree node classification)

Last asked (any company): 5 months ago, about 2026-05 (Exponent shows only a rough age) | +38 interviews

Original: https://www.aced.io/questions/3957/tree-node

QUESTION

A binary tree is stored in a table tree_node_table(id, p_id), where p_id is the id of the parent node (NULL for the root). Write a query that labels every node: 'Root' if it has no parent, 'Inner' if it has a parent and at least one child, and 'Leaf' if it has a parent and no children. The output has the columns id and node_types, ordered by id.

Function signature

-- columns: id, node_types

EXAMPLES

Example 1

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL), (2, 1), (3, 1), (4, 2), (5, 2);`

Output	`1, Root`
	`2, Inner`
	`3, Leaf`
	`4, Leaf`
	`5, Leaf`

Why	`Node 1 has no parent, node 2 has a parent and children 4 and 5, and nodes 3, 4 and 5 have no children.`

Example 2

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL);`
Output	`1, Root`
Why	`A tree with one node: it has no parent, so it is the Root (not a Leaf).`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
SELECT id, 'Root' AS node_types
FROM tree_node_table WHERE p_id IS NULL
UNION
SELECT id, 'Inner'
FROM tree_node_table t
WHERE p_id IS NOT NULL
  AND EXISTS (SELECT 1 FROM tree_node_table c
              WHERE c.p_id = t.id)
UNION
SELECT id, 'Leaf'
FROM tree_node_table t
WHERE p_id IS NOT NULL
  AND NOT EXISTS (SELECT 1 FROM tree_node_table c
                  WHERE c.p_id = t.id);
```

TIME	SPACE
O(n^2)	**O(n)**
without an index on p_id	

WHY IT IS SLOW

The correlated subquery runs once per row and the table is scanned three times, O(n^2) without an index. One CASE expression with a single set of parent ids scans once.

OPTIMAL SOLUTION · One CASE expression

TIME	SPACE
O(n)	**O(n)**
with the set of parent ids built once	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not use NOT IN for the leaves?

If the subquery returns a NULL (the root's p_id), id NOT IN (...) is never true, so you get no rows. Filter NULLs out of the subquery or use NOT EXISTS.

FOLLOW-UP 2 The table has no row for the root's parent and p_id can point to missing ids.

Validate with a LEFT JOIN to find orphans. A node whose p_id is not NULL but missing from the table is a data error, not a root; decide whether to report it.

FOLLOW-UP 3 Add the depth of each node.

A recursive CTE starting from the root rows and joining children on p_id, carrying depth + 1, gives every node's depth in one query.

FOLLOW-UP 4 Add the depth of each node.

A recursive CTE starts from the root rows (depth 0) and joins the table to itself on p_id, adding 1 at every step; each node appears once because every node has one parent. Runs in one query, O(n) with an index on p_id.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Tree Node.
#
# A binary tree is stored in a table tree_node_table(id,
# p_id), where p_id is the id of the parent node (NULL for
# the root). Write a query that labels every node: 'Root'
# if it has no parent, 'Inner' if it has a parent and at
# least one child, and 'Leaf' if it has a parent and no
# children. The output has the columns id and node_types,
# ordered by id.
#
# Signature:
#   -- columns: id, node_types
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not use NOT IN for the leaves?
# 2. The table has no row for the root's parent and p_id
# can point to missing ids.
# 3. Add the depth of each node.
# 4. Add the depth of each node.
#
# INTERVIEW NOTES
# Each row is a node with its parent id. A node is the
# root if it has no parent, a leaf if nobody lists it as a
# parent, and inner otherwise. Order the tests so the root
# is decided first (a single-node tree is a root, not a
# leaf). Beware NOT IN with NULLs: it returns nothing if
# the subquery contains a NULL.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import sqlite3


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): One CASE expression
# Idea: CASE tests in order: no parent means Root, an id
```

```
# that appears as someone's parent means Inner, anything
# else is a Leaf.
# Root: no parent. Inner: has a parent and a child.
# Leaf: has a parent and no child. The Root test comes
# first, so a one-node tree is a Root, not a Leaf.
```

```
TYPES_SQL = """
SELECT t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (SELECT 1 FROM tree_node_table c
                 WHERE c.p_id = t.id) THEN 'Inner'
    ELSE 'Leaf'
  END AS node_types
FROM tree_node_table t
ORDER BY t.id
"""

# ---- main question: tests ----
def test_main_question():
    #       1
    #      / \
    #     2   3
    #    / \
    #   4   5
    rows = [(1, None), (2, 1), (3, 1), (4, 2), (5, 2)]
    got = run(rows, TYPES_SQL)
    want = [(1, "Root"), (2, "Inner"), (3, "Leaf"),
            (4, "Leaf"), (5, "Leaf")]
    assert got == want
    # a single node is the Root
    assert run([(7, None)], TYPES_SQL) == [(7, "Root")]


# ============================================================
# FOLLOW-UP 1: Why not use NOT IN for the leaves?
# ============================================================

# ---- follow-up 1: function ----
# Answer: If the subquery returns a NULL (the root's
# p_id), id NOT IN (...) is never true, so you get no
# rows. Filter NULLs out of the subquery or use NOT
# EXISTS.
# the NOT IN trap: the subquery contains the root's NULL
# p_id, so "id NOT IN (...)" is never true: no rows
NOT_IN_SQL = """
SELECT id FROM tree_node_table
WHERE id NOT IN (SELECT p_id FROM tree_node_table)
"""


# Answer: If the subquery returns a NULL (the root's
# p_id), id NOT IN (...) is never true, so you get no
# rows. Filter NULLs out of the subquery or use NOT
# EXISTS.
# the same idea done safely with NOT EXISTS
LEAVES_SQL = """
SELECT t.id FROM tree_node_table t
WHERE NOT EXISTS (SELECT 1 FROM tree_node_table c
                  WHERE c.p_id = t.id)
ORDER BY t.id
"""

# ---- follow-up 1: tests ----
def test_follow_up_1():
    #       1
    #      / \
    #     2   3
    #    / \
    #   4   5
    rows = [(1, None), (2, 1), (3, 1), (4, 2), (5, 2)]
    # NOT IN finds nothing because of the NULL
    assert run(rows, NOT_IN_SQL) == []
    assert run(rows, LEAVES_SQL) == [(3,), (4,), (5,)]


# ============================================================
# FOLLOW-UP 2: The table has no row for the root's parent
```

```
# ======================================================

# ---- follow-up 2: function ----
# Answer: Validate with a LEFT JOIN to find orphans. A
# node whose p_id is not NULL but missing from the table
# is a data error, not a root; decide whether to report
# it.
ORPHANS_SQL = """
SELECT t.id FROM tree_node_table t
LEFT JOIN tree_node_table p ON p.id = t.p_id
WHERE t.p_id IS NOT NULL AND p.id IS NULL
ORDER BY t.id
"""
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    #       1
    #      / \
    #     2   3
    #    / \
    #   4   5
    rows = [(1, None), (2, 1), (3, 1), (4, 2), (5, 2)]
    # node 9 points at a parent that does not exist
    assert run(rows + [(9, 42)], ORPHANS_SQL) == [(9,)]
    assert run(rows, ORPHANS_SQL) == []


# ======================================================
# FOLLOW-UP 3: Add the depth of each node.
# ======================================================

# ---- follow-up 3: function ----
# Answer: A recursive CTE starting from the root rows and
# joining children on p_id, carrying depth + 1, gives
# every node's depth in one query.
# FOLLOW-UP 4: Add the depth of each node.
# Answer: A recursive CTE starts from the root rows (depth
# 0) and joins the table to itself on p_id, adding 1 at
# every step; each node appears once because every node
# has one parent. Runs in one query, O(n) with an index on
# p_id.
# follow-up: depth of every node. Start from the roots and
# walk down one level at a time (each node has one parent)
DEPTH_SQL = """
WITH RECURSIVE walk(id, depth) AS (
  SELECT id, 0 FROM tree_node_table WHERE p_id IS NULL
  UNION ALL
  SELECT t.id, w.depth + 1
  FROM tree_node_table t JOIN walk w ON t.p_id = w.id
)
SELECT id, depth FROM walk ORDER BY id
"""

# ---- follow-up 3: tests ----
def test_follow_up_3():
    #       1
    #      / \
    #     2   3
```

```
    #    / \
    #   4   5
    rows = [(1, None), (2, 1), (3, 1), (4, 2), (5, 2)]
    got = run(rows, DEPTH_SQL)
    assert got == [(1, 0), (2, 1), (3, 1), (4, 2), (5, 2)]


# ======================================================
# FOLLOW-UP 4: Add the depth of each node.
# ======================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.
```

```
# ======================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ======================================================
# HELPERS for the checks below
# the table has one row per node; the root has p_id NULL
SCHEMA = "CREATE TABLE tree_node_table (id INT, p_id INT)"


INSERT = "INSERT INTO tree_node_table VALUES (?, ?)"


def make_db(rows):
    # build an in-memory database holding the given rows
    db = sqlite3.connect(":memory:")
    db.execute(SCHEMA)
    db.executemany(INSERT, rows)
    return db


def run(rows, sql):
    # run one query on a fresh database
    with make_db(rows) as db:
        return db.execute(sql).fetchall()


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

OFFICIAL PAGE TEXT

Given a binary tree, identify which nodes are leaf nodes, root nodes or inner nodes. Leaf nodes: A node with no children Root nodes: A node with no parent Inner nodes: A node with both children and parent nodes You are given the following table tree_node_table: idp_idintegerinteger where p_id is the id of the parent node. Your output should contain the following columns: id, node_types (’Root’, ‘Inner’, ‘Leaf’).No answers contributed by the community yet.

STUDY PATH: question 275 of 288 | topic 41 of 43: Parent pointers: store who is above each node and climb | 2 of 5

Template fit: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager | asked 132 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 275 of 288

Employee Hierarchy Add / Delete

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 287/288**

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Maintain a company hierarchy. Org(ceo) starts with the CEO. add(emp, mgr) adds a new employee under an existing manager (False if emp already exists or mgr does not). delete(emp) removes an employee and their direct reports move up to the deleted person's manager. The CEO can only be deleted when exactly one person reports to them, who then becomes the CEO.

Function signature

class Org:
  def __init__(self, ceo)
  def add(self, emp, mgr) -> bool
  def delete(self, emp) -> bool

EXAMPLES

Example 1

Input	`o = Org('ceo')`
	`o.add('vp', 'ceo')`
	`o.add('dev', 'vp')`
	`o.add('qa', 'vp')`
	`o.add('dev', 'ceo')`
	`o.delete('vp')`
	`o.manager['dev']`
	`o.delete('ceo')`

Output	`o.add('vp', 'ceo') -> true`
	`o.add('dev', 'vp') -> true`
	`o.add('qa', 'vp') -> true`
	`o.add('dev', 'ceo') -> false`
	`o.delete('vp') -> true`
	`o.manager['dev'] -> 'ceo'`
	`o.delete('ceo') -> false`

Why	`'dev' already exists so the second add fails. Deleting 'vp' moves dev and qa up to the ceo. The ceo now has two reports, so the ceo cannot be deleted.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
manager = {}  # employee -> manager
def delete(e):
    reports = [x for x in manager
               if manager[x] == e]
    for x in reports:
        manager[x] = manager[e]
    del manager[e]
```

TIME	SPACE
O(n) per delete	**O(n)**

WHY IT IS SLOW

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION · Parent map plus children sets

TIME	SPACE
O(r) per delete where r is the number of reports	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

FOLLOW-UP 2 When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

FOLLOW-UP 3 The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Employee Hierarchy Add / Delete
#
# Maintain a company hierarchy. Org(ceo) starts with the
# CEO. add(emp, mgr) adds a new employee under an existing
# manager (False if emp already exists or mgr does not).
# delete(emp) removes an employee and their direct reports
# move up to the deleted person's manager. The CEO can
# only be deleted when exactly one person reports to them,
# who then becomes the CEO.
#
# Signature:
#   class Org:
#       def __init__(self, ceo)
#       def add(self, emp, mgr) -> bool
#       def delete(self, emp) -> bool
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support moving an employee under another manager and
# reject cycles.
# 2. When a manager leaves, promote the most senior report
# instead of the grandparent.
# 3. The CEO leaves.
#
# INTERVIEW NOTES
# A hierarchy is a tree you update, so store links in both
# directions (parent per node, children per node). Then
# every operation touches only the neighbours of one node
# instead of searching the tree. For each delete case
# (leaf, middle, root) say the policy first, because the
# data structure is easy once the rule is fixed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Parent map plus children sets
```

```
# Idea: Keep manager[emp] and reports[emp] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
# FOLLOW-UP 1: Support moving an employee under another
# manager and reject cycles.
# Answer: Walk up from the proposed new manager through
# manager pointers; if you meet the employee being moved,
# the move would put a node under its own descendant, so
# reject it. O(depth).
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        # FOLLOW-UP 3: the CEO leaves
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        # FOLLOW-UP 2: promote the most senior report
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    # FOLLOW-UP 1: move an employee, reject cycles
    def move(self, emp, new_mgr):
        # walk up from the new manager: meeting emp is a
        # cycle
        node = new_mgr
        while node is not None:
            if node == emp:
                return False
            node = self.manager[node]
        self.reassign(emp, new_mgr)
        return True

# ---- main question: tests ----
def test_main_question():
    o = Org("ceo")
    for e, m in [("a", "ceo"), ("b", "a"), ("c", "a")]:
```

```
        assert o.add(e, m)
    assert not o.add("a", "ceo")
    assert not o.add("x", "nobody")
    # deleting a manager: reports go to the grandparent
```

```
    assert o.delete("a")
    assert o.manager["b"] == "ceo"
    assert o.manager["c"] == "ceo"
    # a move that would create a cycle is rejected
    o.add("d", "b")
    assert not o.move("b", "d") and o.move("d", "c")
    # promote the most senior report instead
    p = Org("ceo")
    for e, m in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        p.add(e, m)
    assert p.delete("m", "senior")
    assert p.manager["x"] == "ceo"
    assert p.manager["y"] == "x"
    # the CEO leaves: the most senior report takes over
    q = Org("ceo")
    q.add("a", "ceo")
    q.add("b", "ceo")
    assert q.delete("ceo")
    assert q.root == "a" and q.manager["b"] == "a"
    assert not Org("solo").delete("solo")


# ============================================================
# FOLLOW-UP 1: Support moving an employee under another
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

STUDY PATH: question 276 of 288 | topic 41 of 43: Parent pointers: store who is above each node and climb | 3 of 5

Template fit: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root | asked 251 days ago

MEDIUM | IN APP LC 261 | PracHub | Technical Screen | 276 of 288

Validate parent array forms a tree

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK ALL TIME 29 30 DAYS 0 PRIORITY 148/288

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #261 Graph Valid Tree

LeetCode match: LC 261 Graph Valid Tree

Posted on PracHub: 2026-02-11 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/validate-parent-array-forms-a-tree

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

QUESTION

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints
1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are
    0->2->1, 2->1, and 3->2->1, so every node reaches
    the same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_tree(parent):
    n = len(parent)
    roots = [i for i in range(n)
             if parent[i] == -1]
    if len(roots) != 1:
        return False
    if any(p >= n for p in parent):
        return False
    for node in range(n):
        cur, steps = node, 0
        while cur != -1 and steps <= n:
            cur = parent[cur]
            steps += 1
        if cur != -1:
            return False  # a cycle
    return True
```

TIME	SPACE
O(n^2)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Every node walks its whole chain, O(n^2). Marking nodes as 'on this walk' or 'verified' lets every node be processed once, O(n).

OPTIMAL SOLUTION · One root, then colour each node once while walking up

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is given as undirected edges (LeetCode 261).

A tree has exactly n - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(n alpha(n)).

FOLLOW-UP 2 Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

FOLLOW-UP 3 A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Validate parent array forms a tree
#
# ## Problem You are given an integer array parent of
# length n describing a directed parent pointer for each
# node i (nodes are labeled 0..n-1). - parent[i] is the
# parent of node i. - Exactly one node should be the root,
# indicated by parent[root] = -1. Determine whether these
# n nodes form a **valid rooted tree**. A valid rooted
# tree must satisfy all of the following: 1. **Exactly one
# root** (exactly one index i with parent[i] = -1). 2.
# **No cycles** (following parent pointers from any node
# must eventually reach the root). 3. **Connectivity**
# (every node is reachable from the root; equivalently,
# every node has exactly one simple path to the root). ##
# Input - parent: integer array of length n where each
# value is either -1 or in [0, n-1]. ## Output - Return
# true if parent represents a valid rooted tree; otherwise
# return false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is given as undirected edges (LeetCode
# 261).
# 2. Report the reason it is invalid.
# 3. A binary tree given as child arrays (LeetCode 1361).
#
# INTERVIEW NOTES
# Exactly one -1 gives the root; then the rest is whether
# every node reaches it. Following parents from each node
# can loop (a cycle) or run out of range. Do it with
# memoised walks so every node is processed once: mark
# nodes on the current walk, and if the walk meets one of
# them again it is a cycle; if it meets a node already
# known to reach the root, all nodes on the walk reach it
# too.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================


# ============================================================
# MAIN QUESTION
```

```
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 2: Report the reason it is invalid.
# Answer: Return which rule failed: number of roots, an
# out-of-range parent, or the node where the cycle was
# detected (the node that was met twice on one walk).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def reason(parent):
    # None if valid, else the first rule that fails
    n = len(parent)
    roots = parent.count(-1)
    if roots != 1:
        return f"{roots} roots (need exactly 1)"
    for i, p in enumerate(parent):
        if p != -1 and not 0 <= p < n:
            return f"node {i}: out-of-range parent {p}"
        if p == i:
            return f"node {i} is its own parent"
    # 0 = unseen, 1 = on the current walk, 2 = checked
    state = [0] * n
    for start in range(n):
        path, x = [], start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        # meeting a node twice on one walk is a cycle
        if x != -1 and state[x] == 1:
            return f"cycle through node {x}"
        for v in path:
            state[v] = 2
    return None

# ---- main question: main function ----
# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'.
```

```
def validate_parent_array(parent):
    return reason(parent) is None

# ---- main question: tests ----
```

```
def test_main_question():
    assert validate_parent_array([-1, 0, 0, 1])
    assert validate_parent_array([-1])
    # two roots, none, and a cycle 1 -> 2 -> 1
    assert not validate_parent_array([-1, -1])
    assert not validate_parent_array([1, 0])
    assert "cycle" in reason([-1, 2, 1])
    assert not validate_parent_array([-1, 5])
    assert "out-of-range" in reason([-1, 5])
    assert "own parent" in reason([-1, 1])
    star = [(0, 1), (0, 2), (0, 3), (1, 4)]


# ========================================================
# FOLLOW-UP 1: The graph is given as undirected edges
# ========================================================

# ---- follow-up 1: function ----
# Answer: A tree has exactly n - 1 edges and no cycle:
# check the edge count, then union endpoints and fail on
# an edge that joins two nodes already connected. O(n
# alpha(n)).
def valid_tree_edges(n, edges):
    # LeetCode 261: undirected edges
    if len(edges) != n - 1:
        return False
    root = list(range(n))

    def find(x):
        while root[x] != x:
            root[x] = root[root[x]]
            x = root[x]
        return x

    for u, v in edges:
        a, b = find(u), find(v)
        # an edge inside a group closes a cycle
        if a == b:
            return False
        root[a] = b
    return True

# ---- follow-up 1: tests ----
def test_follow_up_1():
    star = [(0, 1), (0, 2), (0, 3), (1, 4)]
```

```
    assert valid_tree_edges(5, star)
    loop = [(0, 1), (1, 2), (2, 0)]
    assert not valid_tree_edges(4, loop)
    left, right = [1, -1, 3, -1], [2, -1, -1, -1]


# ========================================================
# FOLLOW-UP 2: Report the reason it is invalid.
# ========================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ========================================================
# FOLLOW-UP 3: A binary tree given as child arrays
# ========================================================

# ---- follow-up 3: function ----
# Answer: Check that exactly one node has in-degree 0,
# every other node has in-degree 1, and that a traversal
# from the root visits all n nodes.
def valid_binary_tree(n, left, right):
```

```
    # LeetCode 1361: child arrays, -1 means no child
    indeg = [0] * n
    for kids in (left, right):
        for c in kids:
            if c != -1:
                indeg[c] += 1
    roots = [i for i in range(n) if indeg[i] == 0]
    if len(roots) != 1 or max(indeg) > 1:
        return False
    seen, stack = {roots[0]}, [roots[0]]
    while stack:
        u = stack.pop()
        for c in (left[u], right[u]):
            if c != -1:
                if c in seen:
                    return False
                seen.add(c)
                stack.append(c)
    # every node must be reachable from the root
    return len(seen) == n

# ---- follow-up 3: tests ----
def test_follow_up_3():
    left, right = [1, -1, 3, -1], [2, -1, -1, -1]
    assert valid_binary_tree(4, left, right)
```

```
    assert not valid_binary_tree(2, [1, 0], [-1, -1])


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(parent: list[int]) -> bool:
SAMPLE INPUT
[-1, 0, 0, 1, 1]
```

Test cases

Input	Expected
[-1, 0, 0, 1, 1]	True
[2, -1, 1, 2]	True

STUDY PATH: question 277 of 288 | topic 41 of 43: Parent pointers: store who is above each node and climb | 4 of 5

Template fit: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor | no date

MEDIUM | IN APP LC 1650 | HackTheRounds | Phone Screen | 277 of 288

Lowest Common Ancestor of a Binary Tree III

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 185/288

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1650 Lowest Common Ancestor of a Binary Tree III

LeetCode match: LC 1650

QUESTION

Given two nodes p and q of a binary tree, return their lowest common ancestor: the deepest node that has both of them as descendants (a node counts as a descendant of itself). Every node has a parent pointer, but you are not given the root.

Function signature

def lca_swap(p: PNode, q: PNode) -> PNode

EXAMPLES

Example 1

Input

```
class PNode:
def __init__(self, val):
self.val, self.parent = val, None
root, a, b, c = PNode(3), PNode(5),
PNode(1), PNode(6)
a.parent = b.parent = root
c.parent = a
lca_swap(c, b).val
lca_swap(c, a).val
lca_swap(a, b).val
```

Output

```
lca_swap(c, b).val -> 3
lca_swap(c, a).val -> 5
lca_swap(a, b).val -> 3
```

Why

Node 6 hangs under 5, which hangs under the root 3 next to node 1. The ancestor of 6 and 1 is the root 3; the ancestor of 6 and its own parent 5 is 5 itself.

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lca(p, q):
    ancestors = set()
    node = p
    while node:
        ancestors.add(node)
        node = node.parent
    node = q
    while node:
        if node in ancestors:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(h)	O(h)

WHY IT IS SLOW

Uses O(h) extra space for the set. Two pointers that switch to the other node's start when they fall off the root meet at the answer with O(1) space.

OPTIMAL SOLUTION · Two pointers that switch to the other node at the end

TIME	SPACE
O(h)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without the swap trick (it feels like magic).

Compute the depth of each node by walking up, lift the deeper one by the difference, then step both up together until they meet. Same O(h) time and O(1) space and easier to justify.

FOLLOW-UP 2 Many LCA queries on the same tree.

Preprocess with binary lifting (up[k][v] is the 2^k-th ancestor): O(n log n) to build, O(log n) per query. Or an Euler tour with a range-minimum structure for O(1) queries.

FOLLOW-UP 3 The nodes might not be in the same tree.

Then the two pointers both end as None after two passes, and None is the correct answer. The loop terminates because both pointers reach None at the same step.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Lowest Common Ancestor of a Binary Tree
# III
#
# Given two nodes p and q of a binary tree, return their
# lowest common ancestor: the deepest node that has both
# of them as descendants (a node counts as a descendant of
# itself). Every node has a parent pointer, but you are
# not given the root.
#
# Signature:
#   def lca_swap(p: PNode, q: PNode) -> PNode
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without the swap trick (it feels like magic).
# 2. Many LCA queries on the same tree.
# 3. The nodes might not be in the same tree.
#
# INTERVIEW NOTES
# With parent pointers each node is the head of a linked
# list that ends at the root, so the question is where two
# linked lists meet. If the lists have different lengths,
# walk each pointer to the end and then restart it on the
# other node: both pointers then travel the same total
# distance and arrive at the meeting node together.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def lca_set(p, q):
    seen = set()
    while p:
        seen.add(id(p))
        p = p.parent
    while q:
        if id(q) in seen:
            return q
        q = q.parent
    return None

# ---- main question: main function ----
# BASE QUESTION (optimal): Two pointers that switch to the
# other node at the end
# Idea: Move a from p and b from q one step up per round.
# When a pointer falls off the root, restart it at the
# other node.
def lca_swap(p, q):
    # two pointers climb to the root; at the top each one
    # jumps to the other node's start. Both walk the same
    # total distance, so they meet at the LCA (or both
    # reach
    # None together when the nodes are in different
    # trees).
    a, b = p, q
    while a is not b:
        a = a.parent if a else q
        b = b.parent if b else p
    return a

# ---- main question: tests ----
def test_main_question():
    root = Node(3)
    five = attach(root, 5, "left")
    one = attach(root, 1, "right")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    assert lca_swap(six, four) is five
    assert lca_swap(five, four) is five
    assert lca_swap(six, one) is root
    assert lca_swap(root, root) is root
    other = Node(99)
    assert lca_swap(six, other) is None


# ============================================================
# FOLLOW-UP 1: Do it without the swap trick (it feels like
# ============================================================

# ---- follow-up 1: helper functions ----
def depth(node):
    d = 0
    while node.parent:
        node = node.parent
        d += 1
    return d

# ---- follow-up 1: function ----
# Answer: Compute the depth of each node by walking up,
# lift the deeper one by the difference, then step both up
# together until they meet. Same O(h) time and O(1) space
# and easier to justify.
def lca_by_depth(p, q):
    # follow-up: no trick. Lift the deeper node by the
    # difference in depth, then climb both until they
    # meet.
    dp, dq = depth(p), depth(q)
    while dp > dq:
        p, dp = p.parent, dp - 1
    while dq > dp:
        q, dq = q.parent, dq - 1
    while p is not q:
        p, q = p.parent, q.parent
        if p is None or q is None:
            return None
    return p

# ---- follow-up 1: tests ----
def test_follow_up_1():
    root = Node(3)
    five = attach(root, 5, "left")
    one = attach(root, 1, "right")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    other = Node(99)
    assert lca_by_depth(six, other) is None
    assert lca_by_depth(four, one) is root


# ============================================================
```

```
# FOLLOW-UP 2: Many LCA queries on the same tree.
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: Preprocess with binary lifting (up[k][v] is the
# 2^k-th ancestor): O(n log n) to build, O(log n) per
# query. Or an Euler tour with a range-minimum structure
# for O(1) queries.
class LiftingLCA:
    """Follow-up: many queries on one tree.
    up[k][i] is the 2**k-th ancestor of node i: O(n log n)
    to build and O(log n) per query."""

    def __init__(self, root):
        self.ids = {root: 0}
        self.nodes = [root]
        parent, self.depth = [0], [0]
        # the list grows while we walk it: a BFS order
        for node in self.nodes:
            at = self.ids[node]
            for child in (node.left, node.right):
                if child:
                    self.ids[child] = len(self.nodes)
                    self.nodes.append(child)
                    parent.append(at)
                    self.depth.append(self.depth[at] + 1)
        self.up = [parent]
        count = len(self.nodes)
        while (1 << len(self.up)) < count:
            prev = self.up[-1]
            row = [prev[prev[i]] for i in range(count)]
            self.up.append(row)

    def query(self, p, q):
        a, b = self.ids[p], self.ids[q]
        if self.depth[a] < self.depth[b]:
            a, b = b, a
        # lift the deeper node to the same depth
        diff = self.depth[a] - self.depth[b]
        for k, row in enumerate(self.up):
            if diff >> k & 1:
                a = row[a]
        if a == b:
            return self.nodes[a]
        # lift both as high as possible while they differ
        for row in reversed(self.up):
            if row[a] != row[b]:
                a, b = row[a], row[b]
```

```
        return self.nodes[self.up[0][a]]

# ---- follow-up 2: tests ----
def test_follow_up_2():
    root = Node(3)
    five = attach(root, 5, "left")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    index = LiftingLCA(root)
    assert index.query(six, four) is five
    assert index.query(root, four) is root


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# ============================================================
# HELPERS for the checks below
class Node:
    def __init__(self, val, parent=None):
        self.val, self.parent = val, parent
        self.left = self.right = None
```

```
def attach(parent, val, side):
    # create a child and link it both ways
    child = Node(val, parent)
    setattr(parent, side, child)
    return child


def random_tree(size):
    import random
    root = Node(0)
    nodes = [root]
    while len(nodes) < size:
        parent = random.choice(nodes)
        side = random.choice(["left", "right"])
        if getattr(parent, side) is None:
            nodes.append(attach(parent, len(nodes), side))
    return nodes


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 278 of 288 | topic 41 of 43: Parent pointers: store who is above each node and climb | 5 of 5

Template fit: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2 | asked 347 days ago | frequency Single report

MEDIUM | ColorStack | technical interview | 278 of 288

Root a Tree So It Becomes a Binary Tree

ASKED 2025-11-07 | 347 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 230/288

ABOUT THIS QUESTION

Topic: Parent pointers: store who is above each node and climb

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1245 Tree Diameter (related: rooting a tree, degrees)

Frequency: Single report | Last asked: 2025-11-07 | Roles: SWE Intern

QUESTION

You are given a connected, undirected, acyclic graph (i.e. a tree) with n nodes labeled 0 .. n-1 , described by an edge list. It is guaranteed that every node has at most 3 neighbors. Choose a node to be the root. When rooted at node r , one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a binary tree if every node has at most 2 children. Return the smallest label of a node that can serve as such a root. If no node works, return -1 . Key observation (the trick): a non-root node with degree d has d - 1 children, so it needs d <= 3 ; the root has degree children, so the root needs degree <= 2 . Since all nodes already have degree <= 3 , any node with degree <= 2 is a valid root — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for n >= 1 .

Function signature

```
from typing import List, Tuple
def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) ->
int:
    """Return the smallest node label that can root the tree as a
binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Follow-up (as reported): Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by depth % 2 from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

EXAMPLES

QUESTION

Example 1

Input	`n = 4, edges = [(0, 1), (0, 2), (0, 3)]`
Output	`1`
Why	`Node 0 has degree 3, so as root it would have 3 children (invalid). Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.`

Example 2

Input	`n = 3, edges = [(0, 1), (1, 2)]`
Output	`0`
Why	`A path; every node has degree <= 2, so the smallest label 0 works.`

Example 3

Input	`n = 1, edges = []`
Output	`0`
Why	`A single node is trivially a binary tree.`

CONSTRAINTS

- 1 <= n <= 10^5 ; the input is a valid tree with n - 1 edges (for n == 1 , no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_root(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    for root in range(n):
        parent = {root: -1}
        stack, ok = [root], True
        while stack:
            u = stack.pop()
            kids = [
                v
                for v in adj[u]
                if v != parent[u]
            ]
            ok = ok and len(kids) <= 2
            for v in kids:
                parent[v] = u
                stack.append(v)
        if ok:
            return root
    return -1
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

It re-roots the whole tree for each candidate even though only the root's degree matters.

OPTIMAL SOLUTION · First node of degree at most 2

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Also 2-colour the nodes so a parent and its children differ and equal depths share a colour.

Colour each node by depth % 2 from the chosen root; in a tree this satisfies both rules, so a valid colouring exists whenever a valid root does.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Root a Tree So It Becomes a Binary Tree
#
# You are given a connected, undirected, acyclic graph
# (i.e. a tree) with n nodes labeled 0 .. n-1 , described
# by an edge list. It is guaranteed that every node has at
# most 3 neighbors. Choose a node to be the root. When
# rooted at node r , one neighbor of each non-root node
# becomes its parent and the rest become its children; the
# root has no parent, so all of its neighbors become
# children. The rooted tree is a binary tree if every node
# has at most 2 children. Return the smallest label of a
# node that can serve as such a root. If no node works,
# return -1 . Key observation (the trick): a non-root node
# with degree d has d - 1 children, so it needs d <= 3 ;
# the root has degree children, so the root needs degree
# <= 2 .
#
# FOLLOW-UPS (answered below, in this order):
# 1. Also 2-colour the nodes so a parent and its children
# differ and equal depths share a colour.
#
# INTERVIEW NOTES
# Root the tree at r. Every non-root node has one parent
# edge, so it has degree - 1 children; the root has degree
# children. A node is fine as long as its children number
# at most 2, and degrees are at most 3 already. So only
# the root can break the rule, and any node of degree 2 or
# less is a valid root. The smallest label with degree at
# most 2 is the answer.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```

# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): First node of degree at most 2
# Idea: Only the root has as many children as its degree;
# every other node has one fewer.
def find_binary_tree_root(n, edges):
    degree = [0] * n
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    for node in range(n):
        if degree[node] <= 2:
            return node
    return -1

# ---- main question: tests ----
def test_main_question():
    random.seed(81)
    assert (
        find_binary_tree_root(
            4, [(0, 1), (0, 2), (0, 3)]
        )
        == 1
    )
    assert (
        find_binary_tree_root(3, [(0, 1), (1, 2)]) == 0
    )
    assert find_binary_tree_root(1, []) == 0


# ============================================================
# FOLLOW-UP 1: Also 2-colour the nodes so a parent and its
# ============================================================

# ---- follow-up 1: function ----
# Answer: Colour each node by depth % 2 from the chosen
# root; in a tree this satisfies both rules, so a valid
# colouring exists whenever a valid root does.
def root_and_coloring(n, edges):
    root = find_binary_tree_root(n, edges)
    if root == -1:
        return None
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
```

```
    color = {root: 0}
    queue = deque([root])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in color:
                color[v] = 1 - color[u]
                queue.append(v)
    return root, [color[i] for i in range(n)]

# ---- follow-up 1: tests ----
def test_follow_up_1():
    for _ in range(300):
        n = random.randint(1, 10)
        deg = [0] * n
        edges = []
        for v in range(1, n):
```

```
            options = [
                u for u in range(v) if deg[u] < 3
            ]
            u = random.choice(options)
            edges.append(
                (u, v)
                if random.random() < 0.5
                else (v, u)
            )
            deg[u] += 1
            deg[v] += 1
        root, color = root_and_coloring(n, edges)
        assert color[root] == 0
        assert all(
            color[a] != color[b] for a, b in edges
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 42 OF 43

Build or transform a grid with index arithmetic

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	4
EASY	MEDIUM

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Build or transform a grid with index arithmetic | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

E	279. Filled ASCII Diamond	ColorStack	2025-09-26
M	280. Connected Crop Arrangement on a Grid	1Point3Acres	2026-10-04
M	281. Rotate Any m×n 2D Matrix	1Point3Acres	no date
M	282. Solve matrix groups and recipe inventory	PracHub	2026-01-22
M	283. Range Sum Query 2D (Immutable Matrix)	ColorStack	2025-08-19

CODE TEMPLATE | GROUP 42 OF 43 | 5 QUESTIONS, ONE PATTERN

Build or transform a grid with index arithmetic

The one idea these questions share

Decide where each output cell comes from with a formula in (r, c), then build the whole grid with one comprehension. Rotation, diamonds, block summaries and prefix tables are all 'cell(r, c) = something computed from r and c'.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def build_grid(rows, cols, cell):
    return [[cell(r, c) for c in range(cols)] for r in range(rows)]    # CHANGE: the formula for one cell
```

2. Helper functions that usually get written next to it

```
def prefix_2d(matrix):
    rows, cols = len(matrix), len(matrix[0])
    p = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            p[r + 1][c + 1] = matrix[r][c] + p[r][c + 1] + p[r + 1][c] - p[r][c]
    return p


def snake_cells(rows, cols):
    return [(r, c if r % 2 == 0 else cols - 1 - c) for r in range(rows) for c in range(cols)]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
279	Filled ASCII Diamond	row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle
280	Connected Crop Arrangement on a Grid	fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip
281	Rotate Any m×n 2D Matrix	clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))
282	Solve matrix groups and recipe inventory	block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort
283	Range Sum Query 2D (Immutable Matrix)	prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

STUDY PATH: question 279 of 288 | topic 42 of 43: Build or transform a grid with index arithmetic | 1 of 5

Template fit: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle | asked 389 days ago | frequency Single report

EASY | ColorStack | technical interview | 279 of 288

Filled ASCII Diamond

ASKED 2025-09-26 | 389 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 235/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

LeetCode match: Simulation (print a diamond)

Frequency: Single report | Last asked: 2025-09-26 | Roles: SWE Intern

QUESTION

Given a non-negative integer radius R , render a filled diamond on a square grid of side 2*R + 1 . The grid is indexed by (i, j) with 0 <= i, j <= 2*R , and its center is (R, R) . A cell belongs to the diamond (draw 'x') if its Manhattan distance from the center is at most R , i.e. abs(i - R) + abs(j - R) <= R . Every other cell is background (draw '_'). Return the picture as a list of 2*R + 1 strings, one per row (top to bottom), each of length 2*R + 1 .

Function signature

def render_diamond(R: int) -> list[str]:

...

Follow-up (as reported): The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1 times, padded with '_' .)

EXAMPLES

Example 1

Input	`R = 1`
Output	`["_x_", "xxx", "_x_"]`
Why	`Center (1,1). Row 0 has distance 1 from center vertically, so only column 1 (distance 0) is inside.`

Example 2

Input	`R = 2`
Output	`["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]`
Why	`Each row i allows columns within (R - abs(i-R)) of the center.`

Example 3

Input	`R = 0`
Output	`["x"]`
Why	`A 1x1 grid whose only cell is the center.`

CONSTRAINTS

QUESTION

- 0 <= R <= 500 .
- For R = 0 the result is the single cell ["x"] .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute_diamond(R):
    return [
        "".join(
            (
                "x"
                if abs(i - R) + abs(j - R) <= R
                else:
                    "_"
            )
            for j in range(2 * R + 1)
        )
        for i in range(2 * R + 1)
    ]
```

TIME	SPACE
O(R^2)	O(R^2)

WHY IT IS SLOW

Each of the (2R + 1)^2 cells runs the distance test.

OPTIMAL SOLUTION · Build each row from its width

TIME	SPACE
O(R^2) characters	O(R^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Draw only the outline of the diamond.

Keep the same loop but put 'x' only where the distance is exactly R; inside cells become background. In the row form: the two end cells of each row are 'x' (one cell at the tips).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Filled ASCII Diamond
#
# Given a non-negative integer radius R , render a filled
# diamond on a square grid of side 2*R + 1 . The grid is
# indexed by (i, j) with 0 <= i, j <= 2*R , and its center
# is (R, R) . A cell belongs to the diamond (draw 'x' ) if
# its Manhattan distance from the center is at most R ,
# i.e. abs(i - R) + abs(j - R) <= R . Every other cell is
# background (draw '_' ). Return the picture as a list of
```

```
# 2*R + 1 strings, one per row (top to bottom), each of
# length 2*R + 1 . Function signature: def
# render_diamond(R: int) -> list[str]: ... Follow-up (as
# reported): The naive fill is O(R^2). Can you produce
# each row directly from its width instead of testing
# every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1
# times, padded with '_' .)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Draw only the outline of the diamond.
#
# INTERVIEW NOTES
# A cell is inside the diamond when its Manhattan distance
# from the centre is at most R. Testing every cell is
# O(R^2) work for an O(R^2) picture, which is already
# optimal, but each row can be written directly: row i has
# width 2 * (R - |i - R|) + 1 of 'x' centred in a row of
# underscores.

# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ======================================================

import random


# ======================================================
# MAIN QUESTION
# ======================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Build each row from its width
# Idea: Row i has 2 * (R - |i - R|) + 1 x's with |i - R|
```

```
# underscores on each side.
def render_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = abs(i - R)
        rows.append(
            "_" * pad
            + "x" * (2 * (R - pad) + 1)
            + "_" * pad
        )
    return rows

# ---- main question: tests ----
def test_main_question():
    random.seed(27)
    assert render_diamond(1) == ["_x_", "xxx", "_x_"]
    assert render_diamond(2) == [
        "__x__",
        "_xxx_",
        "xxxxx",
        "_xxx_",
        "__x__",
    ]
    assert render_diamond(0) == ["x"]


# ======================================================
# FOLLOW-UP 1: Draw only the outline of the diamond.
# ======================================================

# ---- follow-up 1: function ----
# Answer: Keep the same loop but put 'x' only where the
# distance is exactly R; inside cells become background.
```

```
# In the row form: the two end cells of each row are 'x'
# (one cell at the tips).
def render_hollow_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = abs(i - R)
        row = ["_"] * (2 * R + 1)
        row[pad] = row[2 * R - pad] = "x"
        rows.append("".join(row))
    return rows

# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    assert render_hollow_diamond(2) == [
        "__x__",
        "_x_x_",
        "x___x",
        "_x_x_",
        "__x__",
    ]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 280 of 288 | topic 42 of 43: Build or transform a grid with index arithmetic | 2 of 5

Template fit: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip | asked 16 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen | 280 of 288

Connected Crop Arrangement on a Grid

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 267/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

QUESTION

A garden is an n x m grid. There are k crop types numbered 1..k, and crop i must fill exactly counts[i - 1] cells; the counts add up to n * m so every cell is used. Fill the grid so that the cells of each crop form one connected region (cells joined up, down, left or right). Return any valid grid.

Function signature

def arrange_crops(n: int, m: int, counts: list) -> list

EXAMPLES

Example 1	
Input	`2, 3, [2, 3, 1]`
Output	`[[1, 1, 2], [3, 2, 2]]`
Why	`One valid answer: crop 3 is the single cell at the bottom left; crops 1 and 2 are each connected.`

Example 2	
Input	`1, 4, [1, 3]`
Output	`[[1, 2, 2, 2]]`
Why	`A single row: the crops simply sit side by side.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

BRUTE FORCE

TIME	SPACE
exponential	O(n * m)

WHY IT IS SLOW

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

OPTIMAL SOLUTION · Fill crops along a snake path

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Connected Crop Arrangement on a Grid
#
# A garden is an n x m grid. There are k crop types
# numbered 1..k, and crop i must fill exactly counts[i -
# 1] cells; the counts add up to n * m so every cell is
# used. Fill the grid so that the cells of each crop form
# one connected region (cells joined up, down, left or
# right). Return any valid grid.
#
# Signature:
#   def arrange_crops(n: int, m: int, counts: list) -> lis
t
#
# Examples:
# 2, 3, [2, 3, 1]  ->  [[1, 1, 2], [3, 2, 2]]
# 1, 4, [1, 3]  ->  [[1, 2, 2, 2]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two adjacent square gardens (N x N next to M x M).
#
# INTERVIEW NOTES
# Walk the grid in a snake (left to right on one row,
# right to left on the next) so that every cell is next to
# the one before it. If you give crop 1 the first count[1]
# cells of that walk, crop 2 the next count[2] cells and
```

```
# so on, each crop is a contiguous stretch of the walk, so
# its cells are connected. No search or backtracking is
# needed.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import defaultdict


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid

# ---- main question: main function ----
# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)

# ---- main question: tests ----
def test_main_question():
    # a 2 x 3 field with two crops
    g = arrange_crops(2, 3, [2, 4])
    assert crops_valid(g, [2, 4])
    # many tiny crops still connect
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]


# ============================================================
# FOLLOW-UP 1: Two adjacent square gardens (N x N next to
# ============================================================

# ---- follow-up 1: helper functions ----
def two_gardens_path(n, m):
    # garden A is n x n; garden B is m x m on its right
    path = []
    # snake down and up the columns of garden A
    for c in range(n):
        # the last column must end in row 0
        down = (c % 2 == 0) == (n % 2 == 0)
        rows = range(n)
        if not down:
            rows = range(n - 1, -1, -1)
        path += [(r, c) for r in rows]
    # step into garden B at row 0 and snake through it
    for r in range(m):
        cols = range(n, n + m) if r % 2 == 0 \
            else range(n + m - 1, n - 1, -1)
        path += [(r, c) for c in cols]
    return path

# ---- follow-up 1: function ----
# Answer: Use a different Hamiltonian path of the combined
# L-shaped area: snake down and up the columns of the
# first garden so that the last column ends in the top
# row, step into the second garden there and snake through
# it.
def arrange_two_gardens(n, m, counts):
    assert sum(counts) == n * n + m * m
    # the grid is wide enough for both gardens
    grid = [[0] * (n + m) for _ in range(max(n, m))]
    path = two_gardens_path(n, m)
    return fill_along(grid, path, counts)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]
    g2 = arrange_two_gardens(2, 3, counts)
    assert crops_valid(g2, counts)
    # the path visits every cell once, step by step
    p = two_gardens_path(2, 3)
    assert len(set(p)) == 13
    assert all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1
               for a, b in zip(p, p[1:]))


# ============================================================
# SMALL HELPERS THE TESTS USE (they run after everything above is defined)
# ============================================================

# ============================================================
# HELPERS for the checks below
def crops_valid(grid, counts):
    cells = defaultdict(set)
    for i, row in enumerate(grid):
        for j, v in enumerate(row):
            if v:
                cells[v].add((i, j))
    for crop, cnt in enumerate(counts, 1):
        group = cells[crop]
        # the right number of cells
        if len(group) != cnt:
            return False
        # flood fill from one cell must reach them all
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for x in ((r + 1, c), (r - 1, c),
                      (r, c + 1), (r, c - 1)):
                if x in group:
                    stack.append(x)
        if seen != group:
            return False
    return True
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

STUDY PATH: question 281 of 288 | topic 42 of 43: Build or transform a grid with index arithmetic | 3 of 5

Template fit: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1])) | no date | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 281 of 288

Rotate Any m×n 2D Matrix

STUDY: TYPE | **ALL TIME 56.3** | **30 DAYS 0** | **PRIORITY 64/288**

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

Rotate an m x n matrix 90 degrees clockwise and return the resulting n x m matrix.

Function signature

def rotate_cw(matrix: list) -> list

EXAMPLES

Example 1	
Input	`[[1, 2, 3], [4, 5, 6]]`
Output	`[[4, 1], [5, 2], [6, 3]]`
Why	`The last row becomes the first column: [4, 1] is the old first column read from the bottom.`

Example 2	
Input	`[[1, 2], [3, 4]]`
Output	`[[3, 1], [4, 2]]`
Why	`A square matrix.`

Example 3	
Input	`[[1, 2, 3]]`
Output	`[[1], [2], [3]]`
Why	`A single row becomes a single column.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

BRUTE FORCE

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Rotate Any m×n 2D Matrix
#
# Rotate an m x n matrix 90 degrees clockwise and return
# the resulting n x m matrix.
#
# Signature:
#   def rotate_cw(matrix: list) -> list
#
# Examples:
```

```
# [[1, 2, 3], [4, 5, 6]]  ->  [[4, 1], [5, 2], [6, 3]]
# [[1, 2], [3, 4]]  ->  [[3, 1], [4, 2]]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can a non-square matrix be rotated in place?
# 2. How would you rotate by k * 90 degrees?
# 3. Rotate by k quarter turns.
#
# INTERVIEW NOTES
# A 90 degree clockwise rotation sends the cell at row i,
# column j to row j, column (m - 1 - i) of the new n x m
# matrix. That formula is the whole problem; 'reverse the
# rows then transpose' is just a short way to write it. A
# rectangle cannot be rotated in place because its shape
# changes.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
# is why it cannot be done in place the way LeetCode 48
# is.
```

```
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]


# COUNTER-CLOCKWISE
def rotate_ccw(matrix):
    # swap rows and columns, then reverse the rows
    return [list(row) for row in zip(*matrix)][::-1]


# INDEX RULE version
def rotate_by_index(matrix):
    # the same result from the index rule, built once:
    # new[j][m - 1 - i] = old[i][j]
    m, n = len(matrix), len(matrix[0]) if matrix else 0
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new

# ---- main question: tests ----
def test_main_question():
    a = [[1, 2, 3], [4, 5, 6]]
    # the 2 x 3 example becomes 3 x 2
    assert rotate_cw(a) == [[4, 1], [5, 2], [6, 3]]
    assert rotate_by_index(a) == rotate_cw(a)
    assert rotate_ccw(a) == [[3, 6], [2, 5], [1, 4]]
    # a clockwise then an anticlockwise turn cancel out
    assert rotate_ccw(rotate_cw(a)) == a
```

```
# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: How would you rotate by k * 90 degrees?
# ============================================================

# ---- follow-up 2: function ----
# Answer: Reduce k modulo 4 first. Then apply the single-
# turn function that many times, or write the four direct
```

```
# index maps so you only build the output once.
# FOLLOW-UP 3: Rotate by k quarter turns.
# Answer: Reduce k modulo 4 first; four turns are the
# identity. Apply the single-turn function that many times
# (at most three), or write the three index formulas
# directly to build the answer once.
def rotate_k(matrix, k):
    # four quarter turns are the identity: use k mod 4
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix

# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [[1, 2, 3], [4, 5, 6]]
    assert rotate_k(a, 4) == a
    assert rotate_k(a, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(a, 5) == rotate_cw(a)
    assert rotate_cw([[7]]) == [[7]]
    assert rotate_cw([]) == []


# ============================================================
# FOLLOW-UP 3: Rotate by k quarter turns.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

STUDY PATH: question 282 of 288 | topic 42 of 43: Build or transform a grid with index arithmetic | 4 of 5

Template fit: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort | asked 271 days ago

MEDIUM | PracHub | Technical Screen | 282 of 288

Solve matrix groups and recipe inventory

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.8 | 30 DAYS 0 | PRIORITY 130/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #2115 Find All Possible Recipes from Given Supplies

LeetCode match: No direct match; related LC 2115 Find All Possible Recipes from Given Supplies (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-12)

Original: https://prachub.com/coding-questions/solve-matrix-groups-and-recipe-inventory

QUESTION

Two tasks. (1) block_mins(matrix, k): cut the matrix into k x k blocks (blocks at the right and bottom edge may be smaller) and return the minimum of every block as a smaller matrix. (2) possible_recipes(recipes, supplies): recipes maps a recipe name to the ingredients it needs, and an ingredient may itself be another recipe; return the recipes that can be made from the supplies.

Function signature

def block_mins(matrix: list, k: int) -> list

def possible_recipes(recipes: dict, supplies: list) -> list

EXAMPLES

Example 1

Input	`[[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2`
Output	`[[1, 1], [3, 5]]`
Why	`Top-left block (3, 1, 5, 9) has minimum 1, top-right (4, 1, 2, 6) has 1; the bottom row forms smaller blocks with minimum 3 and 5.`

QUESTION

Example 2

Input	`{'bread': ['flour', 'yeast'], 'sandwich': ['bread', 'ham'], 'cake': ['flour', 'egg', 'sugar']}, ['flour', 'yeast', 'ham', 'egg']`
Output	`['bread', 'sandwich']`
Why	`bread needs flour and yeast (available); sandwich needs bread and ham; cake needs sugar, which is missing.`

BRUTE FORCE · Blocks: collect each block's values and call min

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, h, w):
    answers = []
    for block in blocks(grid, h, w):
        cells = []
        for r, c in block:
            cells.append(grid[r][c])
        answers.append(min(cells))
    return answers
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Copies every block into a list. One pass that updates the minimum of cell (r // k, c // k) needs no copies.

BRUTE FORCE · Recipes: keep re-scanning until no recipe changes

BRUTE FORCE · PSEUDO-CODE

```
available = set(supplies)
while True:
    added = False
    for recipe in recipes:
        if recipe.name in available:
            continue
        needs = recipe.needs
        if all(i in available for i in needs):
            available.add(recipe.name)
            added = True
    if not added:
        break
```

TIME	SPACE
O(R^2 * I)	O(R)

WHY IT IS SLOW

Many passes over all recipes, O(R^2 * I). Counting each recipe's missing ingredients and processing items as they become available (Kahn's algorithm) is O(R + I).

OPTIMAL SOLUTION · Blocks: one pass updating a minimum per block

TIME	SPACE
O(m * n)	O(blocks)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Recipes: Kahn's algorithm on the ingredient graph

TIME	SPACE
O(R + I)	O(R + I)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

FOLLOW-UP 2 Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

FOLLOW-UP 3 Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours over just those recipes tells the two cases apart.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Solve matrix groups and recipe inventory
#
# Two tasks. (1) block_mins(matrix, k): cut the matrix
# into k x k blocks (blocks at the right and bottom edge
# may be smaller) and return the minimum of every block as
# a smaller matrix. (2) possible_recipes(recipes,
# supplies): recipes maps a recipe name to the ingredients
# it needs, and an ingredient may itself be another
# recipe; return the recipes that can be made from the
# supplies.
#
# Signature:
#   def block_mins(matrix: list, k: int) -> list
#   def possible_recipes(recipes: dict, supplies: list) ->
 list
#
# Examples:
# [[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2  ->  [[1,
#     1], [3, 5]]
# {'bread': ['flour', 'yeast'], 'sandwich': ['bread',
#     'ham'], 'cake': ['flour', 'egg', 'sugar']},
#     ['flour', 'yeast', 'ham', 'egg']  ->  ['bread',
#     'sandwich']
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ingredients have quantities and a shared ingredient
# can only be used once.
# 2. Blocks that do not tile the matrix exactly.
# 3. Detect which recipes are impossible because of a
# cycle.
#
# INTERVIEW NOTES
# Blocks: visit each cell once and update the minimum of
# its block, indexed by (row // k, col // k). Recipes:
# this is a dependency graph; a recipe is makeable when
# all its ingredients are supplies or makeable recipes.
# Count unmet ingredients per recipe and process recipes
# as they become available (Kahn's algorithm); recipes on
# a cycle never become available.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================
```

```
from collections import defaultdict, deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Blocks: one pass updating a
# minimum per block
# Idea: Each cell updates the minimum of block (r // k, c
# // k). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, k):
    # the minimum of every k x k block; edge blocks may be
    # smaller, which r // k and c // k handle by
    # themselves
    m, n = len(matrix), len(matrix[0])
    rows, cols = (m + k - 1) // k, (n + k - 1) // k
    out = [[float("inf")] * cols for _ in range(rows)]
    for r in range(m):
        for c in range(n):
            cell = out[r // k][c // k]
            out[r // k][c // k] = min(cell, matrix[r][c])
    return out


# PART 2: which recipes can be made
def possible_recipes(recipes, supplies):
    # recipes: name -> list of ingredients (which may be
    # other
    # recipes). Kahn's algorithm over missing-ingredient
    # counts.
    have = set(supplies)
```

```
    missing, users = {}, defaultdict(list)
    for name, items in recipes.items():
        missing[name] = 0
        for item in set(items):
            if item not in have:
                missing[name] += 1
                users[item].append(name)
    queue = deque(r for r, m in missing.items() if m == 0)
    made = []
    while queue:
```

```
        name = queue.popleft()
        made.append(name)
        # a finished recipe is a new ingredient for its
        # users
        for other in users[name]:
            missing[other] -= 1
            if missing[other] == 0:
                queue.append(other)
    return made

# ---- main question: tests ----
def test_main_question():
    matrix = [[5, 3, 8, 1], [2, 9, 4, 7], [6, 0, 3, 2]]
    # 2 x 2 blocks, the last row of blocks is only 1 row
    # tall
    assert block_mins(matrix, 2) == [[2, 1], [0, 2]]
    assert block_mins([[4]], 3) == [[4]]
    recipes = {"bread": ["flour", "yeast"],
               "sandwich": ["bread", "meat"],
               "burger": ["bread", "meat", "bun"]}
    stock = ["flour", "yeast", "meat"]
    got = possible_recipes(recipes, stock)
    assert got == ["bread", "sandwich"]
    # two recipes that need each other, and a missing item
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert possible_recipes(cyc, []) == []


# ========================================================
# FOLLOW-UP 1: Ingredients have quantities and a shared
# ========================================================

# ---- follow-up 1: function ----
# Answer: Then availability is not enough: you also have
# to decide the order in which to spend scarce items.
def recipes_with_quantities(recipes, needs, stock):
    # ingredients have amounts and stock is used up. Make
    # each recipe at most once, in priority order, while
    # the
    # stock allows (a made recipe is new stock). This is a
    # heuristic: the exact best choice is a flow problem.
    stock, made = dict(stock), []
    progress = True
    while progress:
        progress = False
```

```
        for name, need in zip(recipes, needs):
            if name in made:
                continue
            enough = all(stock.get(x, 0) >= q
                         for x, q in need.items())
            if enough:
                for x, q in need.items():
                    stock[x] -= q
                stock[name] = stock.get(name, 0) + 1
                made.append(name)
                progress = True
    return made

# ========================================================
# FOLLOW-UP 2
# ========================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ========================================================
# FOLLOW-UP 3: Detect which recipes are impossible because
# ========================================================

# ---- follow-up 3: function ----
# Answer: After Kahn's algorithm, recipes whose counts
# never reached zero are either missing an ingredient or
# on a cycle; a DFS with colours over just those recipes
# tells the two cases apart.
def why_impossible(recipes, supplies):
    # for each recipe that cannot be made: a missing raw
    # ingredient, or a cycle
    made = set(possible_recipes(recipes, supplies))
    known = set(supplies) | set(recipes)
    reasons = {}
    for name, items in recipes.items():
        if name in made:
            continue
        gone = [i for i in items if i not in known]
        reasons[name] = "cycle"
        if gone:
            reasons[name] = "missing " + gone[0]
    return reasons

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # two recipes that need each other, and a missing item
```

```
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert why_impossible(cyc, []) == {"a": "cycle",
                                       "b": "cycle",
                                       "c": "missing x"}
    # amounts: one bread uses the only 2 flour, so the
    # cake
    # (which needs 2 flour too) cannot be made
    names = ["bread", "cake"]
    needs = [{"flour": 2}, {"flour": 2}]
    two = {"flour": 2}
    made = recipes_with_quantities(names, needs, two)
    assert made == ["bread"]
    four = {"flour": 4}
    both = recipes_with_quantities(names, needs, four)
    assert both == ["bread", "cake"]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[int]:
```

Sample call

```
solution([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11
,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]])
```

Test cases

Input	Expected
`([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]],)`	`(0, 3, 3)`
`([[-1,5,6,-1,7,8],[9,10,11,12,13,14],[15,16,17,18,19,20],[21,22,23,24,25,26],[27,28,29,30,31,32],[33,34,35,36,37,38]],)`	`(-1, 0, 0)`

STUDY PATH: question 283 of 288 | topic 42 of 43: Build or transform a grid with index arithmetic | 5 of 5

Template fit: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups | asked 427 days ago | frequency Single report

MEDIUM | IN APP LC 304 | ColorStack | technical interview | 283 of 288

Range Sum Query 2D (Immutable Matrix)

ASKED 2025-08-19 | 427 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 213/288

ABOUT THIS QUESTION

Topic: Build or transform a grid with index arithmetic

Optimal answer from: your app's interview approach (LeetCode #304)

LeetCode match: LC 304 Range Sum Query 2D - Immutable

Frequency: Single report | Last asked: 2025-08-19 | Roles: SWE Intern

QUESTION

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner (row1, col1) and lower-right corner (row2, col2) , inclusive. Implement a class NumMatrix that is initialized once with the matrix and can then answer many sum_region queries efficiently (each query in O(1) after preprocessing).

Function signature

class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...
    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...

Follow-up (as reported): Follow-up constraint change from the report: what if the matrix is mutable (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

EXAMPLES

Example 1	
Input	`matrix = [[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]] sum_region(2, 1, 4, 3)`
Output	`8`
Why	`The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums to 8.`

QUESTION

Example 2	
Input	`same matrix; sum_region(1, 1, 2, 2)`
Output	`11`
Why	`Block [[6,3],[2,0]] sums to 11.`

Example 3	
Input	`same matrix; sum_region(1, 2, 2, 4)`
Output	`12`
Why	`Block [[3,2,1],[0,1,5]] sums to 12.`

CONSTRAINTS

- 1 <= rows, cols <= 200
- -10^5 <= matrix[i][j] <= 10^5
- 0 <= row1 <= row2 < rows , 0 <= col1 <= col2 < cols
- There may be up to 10^4 calls to sum_region .

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
class BruteMatrix:
    def __init__(self, matrix):
        self.m = matrix
    def sum_region(self, r1, c1, r2, c2):
        return sum(
            self.m[r][c]
            for r in range(r1, r2 + 1)
            for c in range(c1, c2 + 1)
        )
```

TIME	SPACE
O(R * C) per query	O(1)

WHY IT IS SLOW

Each query walks every cell of its rectangle.

OPTIMAL SOLUTION · 2D prefix sums and inclusion-exclusion

TIME	SPACE
O(R * C) build, O(1) per query	O(R * C)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The matrix can change between queries (update a cell). What do you use?

A 2D Binary Indexed Tree (Fenwick tree): update(r, c, delta) walks both indexes up by their lowest set bit, and a prefix query walks them down. Each operation is O(log R * log C), and a rectangle is four prefix queries.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Range Sum Query 2D (Immutable Matrix)
#
# Given a 2D integer matrix, handle multiple queries of
# the following type: return the sum of the elements
# inside the rectangle defined by its upper-left corner
# (row1, col1) and lower-right corner (row2, col2) ,
# inclusive. Implement a class NumMatrix that is
# initialized once with the matrix and can then answer
# many sum_region queries efficiently (each query in O(1)
# after preprocessing). Function signature: class
# NumMatrix: def __init__(self, matrix: list[list[int]])
# -> None: """Preprocess the matrix (e.g., build a 2D
# prefix-sum table)""" ... def sum_region(self, row1:
# int, col1: int, row2: int, col2: int) -> int: """Return
# the sum of the submatrix with corners (row1, col1) and
# (row2, col2), inclusive.""" ... Follow-up (as reported):
# Follow-up constraint change from the report: what if the
# matrix is mutable (values can be updated between
# queries)?
#
# FOLLOW-UPS (answered below, in this order):
# 1. The matrix can change between queries (update a
# cell). What do you use?
#
# INTERVIEW NOTES
# Many queries on a fixed matrix means paying once for
# preprocessing. A 2D prefix table P[r][c] = sum of the
# rectangle from the corner to (r - 1, c - 1) turns any
# rectangle sum into four lookups by inclusion-exclusion:
# big - top - left + overlap.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
```

```
# ---- main question: main function ----
# BASE QUESTION (optimal): 2D prefix sums and inclusion-
# exclusion
# Idea: P holds the sum of the rectangle from the top-left
# corner; a query combines four entries.
class NumMatrix:
    def __init__(self, matrix):
        rows, cols = len(matrix), (
            len(matrix[0]) if matrix else 0
        )
        self.p = [
            [0] * (cols + 1) for _ in range(rows + 1)
        ]
        for r in range(rows):
            for c in range(cols):
                # the cell plus the block above and the
                # block to the left,
                # minus the corner block that was counted
                # twice
                self.p[r + 1][c + 1] = (
                    matrix[r][c]
                    + self.p[r][c + 1]
                    + self.p[r + 1][c]
                    - self.p[r][c]
                )

    def sum_region(self, r1, c1, r2, c2):
        p = self.p
        return (
            p[r2 + 1][c2 + 1]
            - p[r1][c2 + 1]
            - p[r2 + 1][c1]
            + p[r1][c1]
        )

# ---- main question: tests ----
def test_main_question():
    random.seed(2)
    grid = [
        [3, 0, 1, 4, 2],
        [5, 6, 3, 2, 1],
        [1, 2, 0, 1, 5],
        [4, 1, 0, 1, 7],
        [1, 0, 3, 0, 5],
    ]
```

```
# ============================================================
# FOLLOW-UP 1: The matrix can change between queries
# ============================================================

# ---- follow-up 1: function ----
# Answer: A 2D Binary Indexed Tree (Fenwick tree):
# update(r, c, delta) walks both indexes up by their
# lowest set bit, and a prefix query walks them down. Each
# operation is O(log R * log C), and a rectangle is four
# prefix queries.
class MutableMatrix:
    def __init__(self, matrix):
        self.rows, self.cols = len(matrix), len(
            matrix[0]
        )
        self.vals = [
            [0] * self.cols for _ in range(self.rows)
        ]
        self.tree = [
            [0] * (self.cols + 1)
            for _ in range(self.rows + 1)
        ]
        for r in range(self.rows):
            for c in range(self.cols):
                self.update(r, c, matrix[r][c])

    def update(self, r, c, value):
        delta = value - self.vals[r][c]
        self.vals[r][c] = value
        i = r + 1
        while i <= self.rows:
            j = c + 1
            while j <= self.cols:
                self.tree[i][j] += delta
                j += j & -j
            i += i & -i

    def prefix(self, r, c):
        # sum of rows 0..r - 1 and columns 0..c - 1
        total, i = 0, r
```

```
        while i > 0:
            j = c
            while j > 0:
```

```
                total += self.tree[i][j]
                j -= j & -j
            i -= i & -i
        return total

    def sum_region(self, r1, c1, r2, c2):
        return (
            self.prefix(r2 + 1, c2 + 1)
            - self.prefix(r1, c2 + 1)
            - self.prefix(r2 + 1, c1)
            + self.prefix(r1, c1)
        )


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GOOGLE | 1POINT3ACRES, EXPONENT, COLORSTACK, HACKTHEROUNDS, PRACHUB | TOPIC 43 OF 43

Split into pieces, fix each piece, join (and pack words into lines)

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	2	2
EASY	MEDIUM	HARD

How to use

- The page right after the contents is the CODE TEMPLATE of this group: learn it first.
- Every question here is that template with a few marked lines changed; the table on the template page says which.
- Questions run from Easy to Medium to Hard.
- Every page has < Prev | Contents | Next > at the top. Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Split into pieces, fix each piece, join (and pack words into lines) | 5 questions | tap a title to jump to it

READ FIRST: the code template for these 5 questions (tap here) >

E	284. Implement string formatting and request tracking tasks	PracHub	2026-02-11
M	285. Snake Case to Camel Case with Repository-Wide Refactor	1Point3Acres	2026-07-14
M	286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	1Point3Acres	2026-02-01
H	287. Implement a function to justify a list of words within a given width.	Exponent	no date
H	288. Convert Valid Snake Case to Lower Camel Case	PracHub	2026-04-16

CODE TEMPLATE | GROUP 43 OF 43 | 5 QUESTIONS, ONE PATTERN

Split into pieces, fix each piece, join (and pack words into lines)

The one idea these questions share

Parsing questions split the text on a separator, transform every piece with one small rule, and join again. Layout questions go word by word and decide whether the next word still fits on the current line.

1. Learn this code (the pattern). Lines marked CHANGE are the only ones that differ between questions.

```
def reformat(text, sep="_"):
    head, *rest = text.split(sep)
    return head + "".join(p[:1].upper() + p[1:] for p in rest)   # CHANGE: the rule applied to each piece


def pack_lines(words, width):
    lines, cur = [], []
    for w in words:
        if cur and len(" ".join(cur + [w])) > width:             # does the next word still fit?
            lines.append(cur)
            cur = []
        cur.append(w)
    return lines + ([cur] if cur else [])
```

2. Helper functions that usually get written next to it

```
def spread_spaces(line_words, width):           # distribute spaces between words, extra ones on the left
    gaps = len(line_words) - 1
    if gaps == 0:
        return line_words[0].ljust(width)
    spaces, extra = divmod(width - sum(map(len, line_words)), gaps)
    out = ""
    for k, w in enumerate(line_words[:-1]):
        out += w + " " * (spaces + (1 if k < extra else 0))
    return out + line_words[-1]
```

3. What you change for each question in this group (the numbers match the contents)

#	Question	What you change in the template
284	Implement string formatting and request tracking tasks	split, fix each piece, join; the request-tracking part is a small dict
285	Snake Case to Camel Case with Repository-Wide Refactor	the same function applied to every identifier found in the files
286	Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line
287	Implement a function to justify a list of words within a given width.	pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned
288	Convert Valid Snake Case to Lower Camel Case	reformat(): split on '_', keep the first piece, capitalise the others

On interview day: say the pattern out loud first (what is in the frontier, the heap, the dict), write the template from memory, then change only the CHANGE lines. Each question below shows this fit again at the top of its first page.

Contents Start the questions >

STUDY PATH: question 284 of 288 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 1 of 5

Template fit: split, fix each piece, join; the request-tracking part is a small dict | asked 251 days ago

EASY | PracHub | Technical Screen | 284 of 288

Implement string formatting and request tracking tasks

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 180/288

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: LC 246 Strobogrammatic Number (+ formatting and tracking subtasks) (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-05-21)

Original: https://prachub.com/interview-questions/implement-string-formatting-and-request-tracking-tasks

QUESTION

Four small tasks. (A) snake_to_camel(s): convert snake_case to camelCase, ignoring leading, trailing and repeated underscores. (B) format_thousands(n): write an integer with a comma between every group of three digits (keep the minus sign). (C) ocr_valid(t, rot): an OCR scanner may read a label upside down; given the rotation table rot (character -> character seen after turning 180 degrees), return true if t, or t turned 180 degrees (reversed and mapped through rot), is an integer from 1 to 200 without leading zeros. (D) Server: begin(rid, ts) and end(rid, ts) arrive in any order for each request id; drain(watermark) returns the completed requests as 'id begin=b end=e' lines in order of end time, only those with end at most the watermark (all of them when no watermark is given).

Function signature

```
def snake_to_camel(s: str) -> str
def format_thousands(n: int) -> str
def ocr_valid(t: str, rot: dict) -> bool
class Server:
    def begin(self, rid, ts) -> None
    def end(self, rid, ts) -> None
    def drain(self, watermark=None) -> list
```

EXAMPLES

Example 1

Input	`'make__it_camel_'`
Output	`'makeItCamel'`
Why	Repeated and trailing underscores are ignored.

QUESTION

Example 2

Input	`1234567`
Output	`'1,234,567'`
Why	A comma before every third digit from the right.

Example 3

Input	`-1000`
Output	`'-1,000'`
Why	The sign stays at the front.

Example 4

Input	`rot = {'0': '0', '1': '1', '2': '2', '5': '5', '6': '9', '8': '8', '9': '6'}`
	`ocr_valid('186', rot)`
	`ocr_valid('981', rot)`
	`ocr_valid('500', rot)`

Output	`ocr_valid('186', rot) -> true`
	`ocr_valid('981', rot) -> true`
	`ocr_valid('500', rot) -> false`

Why	981 is above 200, but turned upside down it reads 186, which is valid. 500 is too large and its rotation 005 has leading zeros.

Example 5

Input	`sv = Server()`
	`sv.begin('r2', 5)`
	`sv.end('r1', 4)`
	`sv.begin('r1', 1)`
	`sv.end('r2', 9)`
	`sv.drain(8)`
	`sv.drain()`

Output	`sv.drain(8) -> ['r1 begin=1 end=4']` `sv.drain() -> ['r2 begin=5 end=9']`
Why	r1 finished at 4 and r2 at 9. Draining with watermark 8 prints only r1; draining everything then prints r2.

BRUTE FORCE · Task A by walking characters

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    result, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            result += ch.upper() if up else ch
            up = False
    return result
```

TIME	SPACE
O(n)	O(n)

BRUTE FORCE · Task A by walking characters

WHY IT IS SLOW

Linear and fine; the split-based version is shorter. The only care is leading and repeated underscores.

BRUTE FORCE · Task B with the language's format specifier

BRUTE FORCE · PSEUDO-CODE

```
def format_number(n):
    return format(n, ",")  # built-in separator
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

Correct and O(d), but it hides the algorithm; interviews usually ask for the grouping by hand: peel three digits at a time from the right.

BRUTE FORCE · Task C by checking all three readings with a straightforward helper

BRUTE FORCE · PSEUDO-CODE

```
def is_valid_rotation(text):
    if in_range(text):  # a number 1..200
        return True
    if not all(ch in ROTATE for ch in text):
        return False
    turned = [ROTATE[c] for c in reversed(text)]
    return in_range("".join(turned))
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It is already linear; there is little to speed up. The risk is correctness: unmapped characters make the rotation invalid, and leading zeros are not valid numbers.

BRUTE FORCE · Task D by storing all events and sorting when asked

BRUTE FORCE · PSEUDO-CODE

```
begin, end = {}, {}
def on_begin(rid, t):
    begin[rid] = t
def on_end(rid, t):
    end[rid] = t
def print_finished():
    done = [r for r in end if r in begin]
    done.sort(key=lambda r: (end[r], r))
    for r in done:
        print(r, begin[r], end[r])
        del begin[r], end[r]
```

BRUTE FORCE · Task D by storing all events and sorting when asked

TIME	SPACE
O(n log n) per print	O(n)

WHY IT IS SLOW

Each print sorts everything pending, O(n log n); a min-heap of completed requests prints in order with O(log n) per event, releasing only entries below a safe watermark.

OPTIMAL SOLUTION · Task A: snake_case to camelCase

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task B: thousands separators

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task C: valid as-is or after a 180 degree rotation

TIME	SPACE
O(L)	O(L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task D: request tracker that prints completed requests in end-time order

TIME	SPACE
O(log n) per event	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

FOLLOW-UP 2 How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

FOLLOW-UP 3 Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

FOLLOW-UP 4 The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. **Cost:** O(log n) per event for the eviction heap; memory O(requests inside the retention window).

FOLLOW-UP 5 Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement string formatting and request
# tracking tasks
#
# Four small tasks. (A) snake_to_camel(s): convert
# snake_case to camelCase, ignoring leading, trailing and
# repeated underscores. (B) format_thousands(n): write an
# integer with a comma between every group of three digits
# (keep the minus sign). (C) ocr_valid(t, rot): an OCR
# scanner may read a label upside down; given the rotation
# table rot (character -> character seen after turning 180
# degrees), return true if t, or t turned 180 degrees
# (reversed and mapped through rot), is an integer from 1
# to 200 without leading zeros. (D) Server: begin(rid, ts)
# and end(rid, ts) arrive in any order for each request
# id; drain(watermark) returns the completed requests as
# 'id begin=b end=e' lines in order of end time, only
# those with end at most the watermark (all of them when
# no watermark is given).
#
# Signature:
#   def snake_to_camel(s: str) -> str
#   def format_thousands(n: int) -> str
#   def ocr_valid(t: str, rot: dict) -> bool
#   class Server:
#       def begin(self, rid, ts) -> None
#       def end(self, rid, ts) -> None
#       def drain(self, watermark=None) -> list
#
# Examples:
# 'make__it_camel_'  ->  'makeItCamel'
# 1234567  ->  '1,234,567'
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not print as soon as a request completes in the
# follow-up?
# 2. How do you handle bad input (duplicate begin, end
# before begin, never-completed requests)?
# 3. Rotation map provided at runtime with extra pairs.
# 4. The tracker must bound its memory.
# 5. Another locale: a different separator or grouping
# (Indian style 12,34,567).
#
# INTERVIEW NOTES
# Four independent tasks, so solve and test each
# separately and say the contract for each before coding:
# what an empty or negative input returns, what 'valid'
# means for the rotated number, and for the tracker when
# it is safe to print in order. For the tracker,
# completion is the second event of a pair, and order
# printing is a heap that is drained only below a
# watermark.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
ROTATE = {"0": "0", "1": "1", "6": "9",
          "8": "8", "9": "6"}

# ---- main question: main function ----
# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def snake_to_camel(s):
    # foo_bar -> fooBar (empty parts are ignored)
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    rest = [w.capitalize() for w in words[1:]]
    return words[0] + "".join(rest)


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
```

```
# FOLLOW-UP 5: Another locale: a different separator or
# grouping (Indian style 12,34,567).
# Answer: Parameterise the separator and the group size:
# the lowest group may differ from the others (3 digits,
# then groups of 2 in the Indian system). The loop peels a
# group from the right each time.
def format_number(n, sep=",", sizes=(3,)):
    # FOLLOW-UP 5: sep= and sizes=(3, 2) for other locales
    # group the digits from the right: 1234567 ->
    # 1,234,567.
    # sizes = (3, 2) is the Indian style 12,34,567: the
    # first
    # group has 3 digits, every later one has 2.
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts, i = [], 0
    while len(digits) > sizes[min(i, len(sizes) - 1)]:
        size = sizes[min(i, len(sizes) - 1)]
        # peel one group off the right end
        parts.append(digits[-size:])
        digits = digits[:-size]
        i += 1
    parts.append(digits)
    return sign + sep.join(reversed(parts))


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def is_rotation_valid(s, mapping=ROTATE):
    # does the number read the same upside down (180
    # degrees)?
    # a character not in the map makes it invalid
    if any(ch not in mapping for ch in s):
        return False
    # rotate every digit and read the result backwards
    turned = "".join(mapping[ch] for ch in reversed(s))
    return turned == s


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
```

```
# The first word stays lowercase and every later word is
# capitalized.
class RequestTracker:
    """begin(id, t) and end(id, t). Finished requests are
    printed in the order they BEGAN: one is printed only
    when everything that began before it is done."""

    def __init__(self):
        self.order = deque()
        self.begin_at, self.end_at = {}, {}

    def begin(self, rid, t):
        if rid in self.begin_at:
            raise ValueError("duplicate begin")
        self.begin_at[rid] = t
        self.order.append(rid)

    def end(self, rid, t):
        if rid not in self.begin_at:
            raise ValueError("end without begin")
        if t < self.begin_at[rid]:
            raise ValueError("ends before it begins")
        self.end_at[rid] = t
        # print from the front while the oldest are
        # finished
        out = []
        while self.order and self.order[0] in self.end_at:
            first = self.order.popleft()
```

```
            out.append((first, self.begin_at[first],
                        self.end_at[first]))
        return out

# ---- main question: tests ----
def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("__a__b_") == "aB"
    assert snake_to_camel("") == ""
    assert format_number(1234567) == "1,234,567"
    assert format_number(-1000) == "-1,000"
    assert format_number(999) == "999"
    assert format_number(0) == "0"
    indian = format_number(1234567, sizes=(3, 2))
    assert indian == "12,34,567"
    assert format_number(1234, ".") == "1.234"
    assert is_rotation_valid("69")
```

```
    assert is_rotation_valid("818")
    assert not is_rotation_valid("962")
    assert not is_rotation_valid("12")
    assert is_rotation_valid("") is True
    t = RequestTracker()
    t.begin("a", 1)
    t.begin("b", 2)
    # b finished first, but a has not: nothing can be
    # printed
    assert t.end("b", 5) == []
    # a finishes: both are printed, in begin order
    assert t.end("a", 7) == [("a", 1, 7), ("b", 2, 5)]
    again = lambda: t.begin("a", 9)
    orphan = lambda: t.end("zz", 1)
    for call in (again, orphan):
        try:
            call()
            raise AssertionError("should fail")
        except ValueError:
            pass


# ============================================================
# FOLLOW-UP 1
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# FOLLOW-UP 5: Another locale: a different separator or
# ============================================================
```

```
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 5.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FULL QUESTION TEXT FROM THE SITE

You are given several small coding tasks. Implement each as described.

Task A — Convert snake_case to camelCase

Write a function that converts an identifier from snake_case to camelCase.

Input: a string s containing lowercase letters a-z and underscores _ .
Output: the camelCase version.
Rules/edge cases:
Consecutive underscores should be treated as a single word separator (ignore empty parts).
If s starts/ends with underscores, ignore those underscores.
The first word remains lowercase; each subsequent word is capitalized.

Example:

"hello_world" -> "helloWorld"
"__my__var__name_" -> "myVarName"
Task B — Format an integer with thousands separators

Write a function that formats a (possibly negative) integer with commas as thousands separators.

Input: an integer n (may be large; assume it fits in 64-bit signed integer).
Output: a string representation using commas every 3 digits from the right.

Examples:

123 -> "123"
1234 -> "1,234"
-12345678 -> "-12,345,678"
Task C — Validate an OCR string under 180° rotation

An OCR system outputs a string t that may contain digits and a few letters. You need to decide whether it can represent an integer between 1 and 200 inclusive under either normal reading or after a 180-degree rotation.

Define a rotation mapping rot(c) for characters; rotation is valid only if every character in the string has a mapping. Rotating a whole string means:

reverse the string, and
replace each character c with rot(c) .

FULL QUESTION TEXT FROM THE SITE

Return true if either:

t as-is is a valid base-10 integer in [1, 200] , OR
rotate180(t) is a valid base-10 integer in [1, 200] .

Assume the rotation map is provided as a dictionary (examples from the interview note included):

'6' <-> '9'
plus any additional pairs given at runtime (e.g., 'h' -> '4' , 'L' -> '7'). Characters not in the map are invalid for rotation.

Examples (assuming map includes 6<->9, 0->0, 1->1, 8->8):

"69" rotates to "69" and is valid (69 in range) -> true
"201" is out of range, rotate may or may not help -> depends on mapping -> compute
Task D — Track request begin/end times and print in a required order

You are implementing a Server class that receives events for requests. Each request has a unique requestId.

You must implement two methods:

begin(requestId, timestamp)
end(requestId, timestamp)
Part 1

Whenever both timestamps for a request are known, print/log a line:

requestId begin=<beginTs> end=<endTs>

Assumptions:

Events may arrive out of order (an end can arrive before begin).
There may be multiple in-flight requests.
Follow-up

Instead of printing immediately when a pair is complete, the server must always print completed requests in increasing order of end time (earliest end printed first). If multiple have the same end time, break ties by requestId.

Design the data structures and logic to support this ordering efficiently.

STUDY PATH: question 285 of 288 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 2 of 5

Template fit: the same function applied to every identifier found in the files | asked 98 days ago | frequency Single report

MEDIUM | 1Point3Acres | tech-screen | 285 of 288

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 273/288**

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Snake Case to Camel Case with Repository-
# Wide Refactor
#
# Convert a valid snake_case identifier to camelCase, then
# design a safe repository-wide migration for variables
# and function names without changing third-party symbols,
# public interfaces, or the tests' ability to detect
# regressions. Implement a conversion from a valid
# snake_case identifier to camelCase. The base input is
# guaranteed valid: no leading underscore, trailing
# underscore, or doubled separator. Example: foo_bar ->
# fooBar. Extend the discussion to a repository too large
# for manual review: replace snake-case variables and
# functions across the codebase. Explain how to verify
# that the repository still works after the automated
# rewrite, including when test-package identifiers are
# rewritten too. Preserve third-party functions and public
# interfaces; explain how the migration distinguishes
# symbols that may change from contracts that must remain
# stable. Examples: foo_bar -> fooBar
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you prove the migration is safe?
#
# INTERVIEW NOTES
# Converting foo_bar to fooBar is easy. The risk is in the
# migration: a plain text replace also changes strings,
# comments, third-party names and public interfaces. Work
```

```
# on tokens (or the syntax tree) instead, rename only
# symbols defined in your own code, keep an explicit
# allow-list of public names, and prove nothing broke by
# running the tests before and after.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import ast
import io
import tokenize
```

```
# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)

# ---- main question: tests ----
def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")


# ============================================================
# FOLLOW-UP 1: How do you prove the migration is safe?
# ============================================================

# ---- follow-up 1: function ----
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def owned_names(source, keep=()):
    # names this code defines itself: safe to rename
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
```

```
            args = node.args.args + node.args.kwonlyargs
            names.update(a.arg for a in args)
        elif isinstance(node, ast.Name):
            if isinstance(node.ctx, ast.Store):
                names.add(node.id)
    # only real snake_case, never dunder or private names
    return {n for n in names if "_" in n.strip("_")
            and not n.startswith("_") and n not in keep}


# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, prev = [], None
    # work on tokens: strings and comments are untouched
    for tok in tokenize.generate_tokens(
            io.StringIO(source).readline):
        # skip attribute names (after a dot): they may be
        # third-party, so they stay as written
        dot = prev is not None and prev.string == "."
        is_name = tok.type == tokenize.NAME
        if is_name and tok.string in owned and not dot:
            new = snake_to_camel(tok.string)
            tok = tok._replace(string=new)
        out.append(tok)
        prev = tok
    return tokenize.untokenize(out)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")
    new = rename_identifiers(src, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    # strings, comments and third-party calls are intact
    assert "'x_y'" in new and "# load_data is fine" in new
    assert "resultValue = loadData('a')" in new
```

```
    # the migrated code still runs
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY PATH: question 286 of 288 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 3 of 5

Template fit: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line | asked 261 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 286 of 288

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 262/288

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space
    would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo
    bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wrap(text, width):
    count = 0
    for paragraph in text.split("\n"):
        line = ""
        for word in paragraph.split():
            joined = len(line) + 1 + len(word)
            if line and joined > width:
                count += 1  # finish the line
                line = word
            else:
                line = line + " " + word
                line = line.strip()
        count += 1  # last line
    return count
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION · Greedy with a running line length

TIME	SPACE
O(n)	O(1) extra beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

FOLLOW-UP 2 A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

FOLLOW-UP 3 Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Google Doc Line Wrap (Word Wrap with
# Whitespace and Newlines)
#
# Examples: text = "Hello world\nFoo bar baz qux" width =
# 10 Line 1: "Hello" (5 chars; "world" + space would be 11
# -> break) Line 2: "world" Line 3: "Foo bar" (7 chars;
# "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux"
# Output: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the lines, not only the count.
# 2. A word longer than the width must be broken.
# 3. Minimise raggedness instead of lines.
#
# INTERVIEW NOTES
# Greedy is optimal for the question 'fewest lines': put
# as many words on a line as fit. The decision per word is
# only 'fits with a space', 'fits as the first word on the
# line', or 'start a new line', so track the current line
# length, not the line text. Treat newline as a hard break
# and decide up front what empty lines and long words do.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================



# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
# FOLLOW-UP 1: Return the lines, not only the count.
# Answer: Keep the same loop but append the finished line
# to a list. Build
# each line from a list of words and join once, so each
# word is copied once.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
```

```
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines


# FOLLOW-UP 2: A word longer than the width must be
# broken.
# Answer: Before placing a word, while it is longer than
# the width flush the current line and emit width-sized
# chunks of the word, then continue with the remainder as
# a normal word.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]

# ---- main question: main function ----
# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
# len(word) with the width. Only the length is tracked,
# never the joined line.
def wrap_count(text, width):
    return len(wrap_lines(text, width))
```

```
# ---- main question: tests ----
def test_main_question():
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == [
        "Hello", "world", "Foo bar", "baz qux"]
    assert wrap_count(text, 10) == 4
    # blank lines count as lines
    assert wrap_count("a\n\nb", 5) == 3
    # a long word is split when asked
    cut = wrap_lines("abcdefgh", 3, True)
    assert cut == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]


# ============================================================
# FOLLOW-UP 1: Return the lines, not only the count.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: A word longer than the width must be
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Minimise raggedness instead of lines.
# ============================================================

# ---- follow-up 3: function ----
# Answer: That is Knuth's word-wrap DP: dp[i] is the best
# cost of laying out words i.. with cost (spaces left)^2
# on each line except the last; O(n * words per line).
# Greedy is not optimal for this objective.
def min_raggedness(words, width):
    # Knuth: minimise the squared spare space of every
    # line
    # except the last; greedy is NOT optimal for this goal
    n = len(words)
    INF = float("inf")
    # cost[i]: best total cost for words[i:]
    cost, nxt = [INF] * (n + 1), [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):
```

```
            length += len(words[j]) + 1
            if length > width:
                break
            spare = width - length
            # the last line costs nothing
            extra = 0 if j == n - 1 else spare ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                nxt[i] = j + 1
    lines, i = [], 0
    while i < n:
        lines.append(" ".join(words[i:nxt[i]]))
        i = nxt[i]
    return cost[0], lines

# ---- follow-up 3: tests ----
def test_follow_up_3():
    # the best layout can beat the greedy one
    cost, lines = min_raggedness(
        "aaa bb cc ddddd".split(), 6)
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

STUDY PATH: question 287 of 288 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 4 of 5

Template fit: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned | no date

HARD | IN APP LC 68 | Exponent | 287 of 288

★ TOP 75: rank 56 in the Medium and Hard priority list

★ Implement a function to justify a list of words within a given width.

ASKED (ANY CO.) ~2 YEARS AGO | OLDER

STUDY: TYPE | ALL TIME 44.7 | 30 DAYS 0 | PRIORITY 92/288

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: your app's interview approach (LeetCode #68)

LeetCode match: LC 68 Text Justification (inferred from short statement)

Last asked (any company): 2 years ago, about 2024-10 (Exponent shows only a rough age)

Original:
https://www.aced.io/questions/4903/function-justify-words-within-width

QUESTION

Given a list of words and a line width, pack the words into lines of exactly that width (full justification): put as many words on a line as fit (with at least one space between them), then spread the extra spaces between the words as evenly as possible, giving the leftmost gaps the extra ones. The last line, and any line with a single word, is left-justified and padded with spaces on the right. Return the list of lines.

Function signature

def fullJustify(words: list, max_width: int) -> list

EXAMPLES

Example 1	
Input	`['This', 'is', 'an', 'example', 'of', 'text', 'justification.'], 16`
Output	`['This is an', 'example of text', 'justification. ']`
Why	`Three lines of width 16: the first two are spread out, the last is left-aligned and padded.`

Example 2	
Input	`['a', 'b'], 5`
Output	`['a b ']`
Why	`Both words fit on one line, which is the last line: left-aligned with single spaces and padded on the right.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def justify(words, width):
    lines, i = [], 0
    while i < len(words):
        j = i + 1
        while j < len(words):
            if not fits(words[i:j + 1], width):
                break
            j += 1
        last = j == len(words)
        row = format_line(words[i:j],
                          width, last)
        lines.append(row)
        i = j
    return lines
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Every attempt to add a word re-joins the whole line to measure it, so each line costs about (words per line) times its length. Tracking only a running length makes it linear in the total characters.

OPTIMAL SOLUTION · Greedy line packing

TIME	SPACE
O(total characters)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you change it to fully justify the last line too, or to center the text?

Only the final-line branch changes. For full justification drop the special case for the last line (it would be spread like every other line). For centering, split the leftover spaces between the two sides, giving the extra space to the right. The greedy packing step stays identical.

FOLLOW-UP 2 What if the words contain wide characters or different pixel widths?

Replace len(word) by a measure(word) function (for example font metrics or East Asian width) and compare against the pixel width. Packing and distribution logic stay the same; spaces are measured in the same unit, so the spare width may not divide evenly and the remainder must be assigned to gaps in whole space units.

FOLLOW-UP 3 Fully justify the last line too.

Drop the special case for the last line (a line with more than one word is spread like the others). Single-word lines stay left-aligned.

FOLLOW-UP 4 Words have pixel widths (a proportional font).

Replace len(word) with a measure function and compare against the pixel width; the extra space is distributed in whole space units, so the remainder rule stays the same.

FOLLOW-UP 5 Minimise raggedness instead of packing greedily.

That is the Knuth-Plass style DP: cost of a line = (spare space)^2, minimise the total over all ways to break the text. O(n * words per line).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Implement a function to justify a list of
# words within a given width.
#
# Given a list of words and a line width, pack the words
# into lines of exactly that width (full justification):
# put as many words on a line as fit (with at least one
# space between them), then spread the extra spaces
# between the words as evenly as possible, giving the
# leftmost gaps the extra ones. The last line, and any
# line with a single word, is left-justified and padded
# with spaces on the right. Return the list of lines.
#
# Signature:
#   def fullJustify(words: list, max_width: int) -> list
#
# Examples:
# ['This', 'is', 'an', 'example', 'of', 'text',
#     'justification.'], 16  ->  ['This    is    an',
#     'example  of text', 'justification.  ']
# ['a', 'b'], 5  ->  ['a b  ']
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you change it to fully justify the last
# line too, or to center the text?
# 2. What if the words contain wide characters or
# different pixel widths?
# 3. Fully justify the last line too.
# 4. Words have pixel widths (a proportional font).
# 5. Minimise raggedness instead of packing greedily.
#
# INTERVIEW NOTES
# Two decisions, kept separate: which words go on each
# line (pack greedily while the words plus one space
# between them fit), and how to spread the leftover spaces
# (an equal share in each gap, the extra spaces to the
# leftmost gaps). The last line and one-word lines are the
# special cases to name before coding.
```

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```
import random


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: helper functions ----
def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))


def center(line, width, measure=len):
    # leftover spaces split on both sides, the odd one
    # right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)


# FOLLOW-UP 2: What if the words contain wide characters
# or different pixel widths?
# Answer: Replace len(word) by a measure(word) function
# (for example font metrics or East Asian width) and
# compare against the pixel width.
# FOLLOW-UP 4: Words have pixel widths (a proportional
# font).
# Answer: Replace len(word) with a measure function and
# compare against the pixel width; the extra space is
# distributed in whole space units, so the remainder rule
# stays the same.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line
    lines, line, used = [], [], 0
    for w in words:
        size = measure(w)
        # `used` counts the words and one space between
        # each
        if line and used + 1 + size > width:
            lines.append(line)
            line, used = [], 0
        used += size + (1 if line else 0)
        line.append(w)
    if line:
        lines.append(line)
    return lines


# FOLLOW-UP 2: What if the words contain wide characters
# or different pixel widths?
# Answer: Replace len(word) by a measure(word) function
# (for example font metrics or East Asian width) and
# compare against the pixel width.
# FOLLOW-UP 4: Words have pixel widths (a proportional
# font).
# Answer: Replace len(word) with a measure function and
# compare against the pixel width; the extra space is
# distributed in whole space units, so the remainder rule
```

```
# stays the same.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def spread(line, width, measure=len):
    # fully justified: spaces between the words, extra
    # ones
    # go to the LEFT gaps first. One word stays left-
    # aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    spaces = width - sum(measure(w) for w in line)
    gaps = len(line) - 1
    each, extra = divmod(spaces, gaps)
    out = ""
    for i, w in enumerate(line[:-1]):
        out += w + " " * (each + (1 if i < extra else 0))
    return out + line[-1]

# ---- main question: main function ----
# BASE QUESTION (optimal): Greedy line packing
# Idea: Pack as many words as fit on a line (counting one
# space between words). Spread the spare spaces over the
# gaps, giving the extra ones to the leftmost gaps.
# FOLLOW-UP 1: How would you change it to fully justify
# the last line too, or to center the text?
# Answer: Only the final-line branch changes. For full
# justification drop the special case for the last line
# (it would be spread like every other line).
```

```
# FOLLOW-UP 3: Fully justify the last line too.
# Answer: Drop the special case for the last line (a line
# with more than one word is spread like the others).
# Single-word lines stay left-aligned.
def justify(words, width, last="left", measure=len,
            lines=None):
    # last: "left" (the classic rule), "full" (spread it
    # too)
    # or "center". `lines` lets the caller bring its own
    # line breaks (see pack_optimal).
    lines = lines or pack_greedy(words, width, measure)
    out = []
    for i, line in enumerate(lines):
        if i < len(lines) - 1:
            out.append(spread(line, width, measure))
        # FOLLOW-UP 1, 3: justify the last line too
        elif last == "full":
            out.append(spread(line, width, measure))
        # FOLLOW-UP 1: centre the last line
        elif last == "center":
            out.append(center(line, width, measure))
        else:
            out.append(left_align(line, width, measure))
    return out

# ---- main question: tests ----
def test_main_question():
    text = ("This is an example of text justification."
            ).split()
    got = justify(text, 16)
    assert got == ["This    is    an", "example  of text",
                   "justification.  "]
    words = "What must be acknowledgment shall be".split()
    got = justify(words, 16)
    assert got == ["What   must   be", "acknowledgment  ",
                   "shall be        "]
    assert all(len(line) == 16 for line in got)
    full = justify(words, 16, last="full")
    assert full[2] == "shall         be"
    mid = justify(words, 16, last="center")
    assert mid[2] == "    shall be    "
    assert justify(["a"], 3) == ["a  "]
    assert justify([], 5) == []
    # pixel widths: every character is 2 units wide
    wide = justify(["ab", "cd", "ef"], 12,
                   measure=lambda s: 2 * len(s))
    assert wide == ["ab    cd", "ef" + " " * 8]


# ============================================================
# FOLLOW-UP 1: How would you change it to fully justify
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 1.

# ============================================================
# FOLLOW-UP 2: What if the words contain wide characters
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 2.

# ============================================================
# FOLLOW-UP 3: Fully justify the last line too.
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 3.

# ============================================================
# FOLLOW-UP 4: Words have pixel widths (a proportional
# ============================================================
# Its code sits in the main question section above, becaus
e the main code calls it: look for the lines marked FOLLOW
-UP 4.

# ============================================================
# FOLLOW-UP 5: Minimise raggedness instead of packing
# ============================================================

# ---- follow-up 5: function ----
# Answer: That is the Knuth-Plass style DP: cost of a line
# = (spare space)^2, minimise the total over all ways to
# break the text. O(n * words per line).
def pack_optimal(words, width):
    # follow-up: choose line breaks that minimise the
    # total
    # raggedness (Knuth-Plass idea). best[i] is the
    # cheapest
    # way to set the first i words.
    n = len(words)
    best = [0] + [float("inf")] * n
    prev = [0] * (n + 1)
    for i in range(1, n + 1):
        size = -1
        # a line holds words j..i-1, try the longest first
        for j in range(i - 1, -1, -1):
            size += len(words[j]) + 1
            if size > width:
                break
            cost = 0 if i == n else (width - size) ** 2
            if best[j] + cost < best[i]:
                best[i], prev[i] = best[j] + cost, j
    lines, i = [], n
    while i > 0:
        lines.append(words[prev[i]:i])
        i = prev[i]
    return lines[::-1]

# ---- follow-up 5: tests ----
def test_follow_up_5():
    for _ in range(200):
        ws = ["x" * random.randint(1, 6)
              for _ in range(random.randint(1, 9))]
```

```
        width = 8
        greedy = pack_greedy(ws, width)
        best = pack_optimal(ws, width)
        assert sum(best, []) == ws
        fits = [len(" ".join(ln)) for ln in best]
        assert max(fits) <= width
        assert raggedness(best, width) <= raggedness(
            greedy, width)


# =============================================================
# SMALL HELPERS THE TESTS USE (they run after everything a
bove is defined)
# =============================================================
# HELPERS for the checks below
def raggedness(lines, width):
    # sum of the squares of the unused space (last line
    # free)
    cost = 0
    for line in lines[:-1]:
        cost += (width - len(" ".join(line))) ** 2
    return cost


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

STUDY PATH: question 288 of 288 | topic 43 of 43: Split into pieces, fix each piece, join (and pack words into lines) | 5 of 5

Template fit: reformat(): split on '_', keep the first piece, capitalise the others | asked 187 days ago

HARD | PracHub | Technical Screen | 288 of 288

Convert Valid Snake Case to Lower Camel Case

POSTED 2026-04-16 | 187 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 244/288

ABOUT THIS QUESTION

Topic: Split into pieces, fix each piece, join (and pack words into lines)

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy string manipulation)

Posted on PracHub: 2026-04-16 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/convert-valid-snake-case-to-lower-camel-case

QUESTION

Given a valid snake_case identifier (one or more lowercase words separated by single underscores, with no leading, trailing or doubled underscores), convert it to lowerCamelCase: keep the first word as it is, remove the underscores and capitalise the first letter of every later word.

Function signature

def snake_to_camel(s: str) -> str

EXAMPLES

Example 1	
Input	`'get_http_response_code'`
Output	`'getHttpResponseCode'`
Why	`Words are joined and every word after the first is capitalised.`

Example 2	
Input	`'user_id'`
Output	`'userId'`
Why	`Two words.`

Example 3	
Input	`'name'`
Output	`'name'`
Why	`A single word has nothing to change.`

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def to_camel(s):
    out, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            out += ch.upper() if up else ch
            up = False
    return out
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Already O(n); it is just longer than needed. Splitting on '_' and capitalising each word after the first is the short form.

OPTIMAL SOLUTION · Split on underscores and capitalize

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

FOLLOW-UP 2 Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

FULL INTERVIEW CODE · clean layout: main question, then each follow-up

One clean program for the whole question: the main question first (its helper functions, its main function and its tests), then every follow-up in its own labelled section with its own helpers, function and tests. Every definition comes before it is used; the test functions are called from run_all_tests() at the bottom, so the file runs top to bottom. The brute force is in its own section above.

```
# MAIN QUESTION: Convert Valid Snake Case to Lower Camel
# Case
#
# Given a valid snake_case identifier (one or more
# lowercase words separated by single underscores, with no
# leading, trailing or doubled underscores), convert it to
# lowerCamelCase: keep the first word as it is, remove the
# underscores and capitalise the first letter of every
# later word.
#
# Signature:
```

```
#   def snake_to_camel(s: str) -> str
#
# Examples:
# 'get_http_response_code'  ->  'getHttpResponseCode'
# 'user_id'  ->  'userId'
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# underscores, or mixed case.
# 2. Convert back from camel case to snake case.
#
# INTERVIEW NOTES
# The statement says the input is valid (single
# underscores, no edge underscores), so rely on that and
# say what you would add if it were not. The pattern is
# 'first word unchanged, capitalize the first letter of
# each later word'.

# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tes
ts.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its func
tion(s), its tests.
#   At the bottom, run_all_tests() calls every test functi
on in this order.
# ============================================================

import re


# ============================================================
# MAIN QUESTION
# ============================================================

# ---- main question: main function ----
# BASE QUESTION (optimal): Split on underscores and
```

```
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(s):
    # foo_bar_baz -> fooBarBaz
    first, *rest = s.split("_")
    caps = [w[:1].upper() + w[1:] for w in rest]
    return first + "".join(caps)

# ---- main question: tests ----
def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    got = snake_to_camel("get_http_response")
    assert got == "getHttpResponse"


# ============================================================
# FOLLOW-UP 1: The input may have leading, trailing or
# ============================================================

# ---- follow-up 1: function ----
# Answer: Drop empty parts after the split, lowercase the
# first word, capitalize the rest. Say what you do with
# digits and non-letters, and keep the rule in one place.
def snake_to_camel_tolerant(s):
    # leading, trailing or repeated underscores and mixed
    # case: drop empty parts, lowercase the first word
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    first = words[0].lower()
    rest = [w.capitalize() for w in words[1:]]
    return first + "".join(rest)

# ---- follow-up 1: tests ----
def test_follow_up_1():
    # messy input
    messy = snake_to_camel_tolerant("_Foo__BAR_")
    assert messy == "fooBar"
    assert snake_to_camel_tolerant("___") == ""
```

```
# ============================================================
# FOLLOW-UP 2: Convert back from camel case to snake case.
# ============================================================

# ---- follow-up 2: function ----
# Answer: Insert an underscore before each uppercase
# letter and lowercase it. Acronyms such as HTTPServer
# need a smarter rule (split between a lowercase-to-
# uppercase edge and between an acronym and the next
# word).
def camel_to_snake(s):
    # fooBarBaz -> foo_bar_baz, and HTTPServer ->
    # http_server
    # 1) split an acronym from the next word: HTTPServer
    # -> HTTP_Server
    s = re.sub(r"([A-Z]+)([A-Z][a-z])", r"\1_\2", s)
    # 2) split a lower-case letter or digit from an upper-
    # case one
    s = re.sub(r"([a-z0-9])([A-Z])", r"\1_\2", s)
    return s.lower()

# ---- follow-up 2: tests ----
def test_follow_up_2():
    # and back again
    assert camel_to_snake("fooBarBaz") == "foo_bar_baz"
    assert camel_to_snake("HTTPServer") == "http_server"
    assert camel_to_snake("x") == "x"
    camel = snake_to_camel("my_var_2")
    assert camel_to_snake(camel) == "my_var2"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

FUNCTION AND TEST CASES

Signature

```
def solve(name) -> str:
```

Sample call

```
solve('a')
```

Test cases

Input	Expected
`('a',)`	`'a'`
`('already',)`	`'already'`

ALSO REPORTED · 2 more

Define the scope and requirements for a snake_case to camelCase converter function. | Exponent

https://www.aced.io/questions/6328/define-scope-snake-case-camel-case-converter

Snake Case to Camel Case with Repository-Wide Refactor | 1Point3Acres | tech-screen | asked 2026-07-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

Preparation: Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.